达梦数据

SQL MANUAL

SQL语言使用手册



目录

第 1 章 结构化查询语言 DM_SQL 简介 1
1.1 DM_SQL 语言的特点 2
1.2 保留字与标识符 3
1.3 DM_SQL 语言的功能及语句 3
1.4 DM_SQL 所支持的数据类型 4
1.4.1 常规数据类型 4
1.4.2 日期时间数据类型 11
1.4.3 多媒体数据类型 19
1.5 DM_SQL 语言支持的表达式 20
1.5.1 数值表达式 20
1.5.2 字符串表达式 25
1.5.3 时间值表达式 25
1.5.4 时间间隔值表达式 29
1.5.5 运算符的优先级 31
1.6 DM_SQL 语言支持的数据库模式 32
第 2 章 手册中的示例说明 33
2.1 示例库说明 33
2.2 示例库脚本 47
2.2.1 创建表空间 47
2.2.2 创建模式及表 48
2.2.3 插入数据 61

第 3 章 数据定义语句 90
3.1 数据库修改语句 90
3.2 管理用户 98
3.2.1 用户定义语句 98
3.2.2 修改用户语句 109
3.2.3 用户删除语句 115
3.3 管理模式 116
3.3.1 模式定义语句 116
3.3.2 设置当前模式语句 120
3.3.3 模式删除语句 120
3.4 管理表空间 121
3.4.1 表空间定义语句 121
3.4.2 修改表空间语句 127
3.4.3 表空间删除语句 131
3.4.4 表空间失效文件检查 132
3.4.5 表空间失效文件恢复准备 133
3.4.6 表空间失效文件恢复 133
3.5 管理表 134
3.5.1 表定义语句 134
3.5.2 表修改语句 212
3.5.3 基表删除语句 254
3.5.4 基表数据删除语句 256
3.5.5 事务型 HUGE 表数据重整 257
3.6 管理索引 258

3.6.1 索引定义语句.....258
3.6.2 索引修改语句.....270
3.6.3 索引删除语句.....278
3.7 管理位图连接索引.....279
3.7.1 位图连接索引定义语句.....279
3.7.2 位图连接索引删除语句.....284
3.8 管理全文索引.....285
3.8.1 全文索引定义语句.....285
3.8.2 全文索引更新语句.....287
3.8.3 全文索引删除语句.....289
3.8.4 用户自定义分词词库.....290
3.9 管理数组索引.....292
3.9.1 数组索引定义语句.....292
3.9.2 数组索引修改语句.....293
3.9.3 数组索引使用.....293
3.9.4 数组索引删除语句.....296
3.10 管理序列.....296
3.10.1 序列定义语句.....296
3.10.2 序列修改语句.....301
3.10.3 序列删除语句.....305
3.11 管理 SQL 域.....306
3.11.1 创建 DOMAIN.....306
3.11.2 使用 DOMAIN.....308
3.11.3 删除 DOMAIN.....308

3.12 管理上下文.....309
3.12.1 创建上下文.....310
3.12.2 删除上下文.....313
3.13 管理目录.....315
3.13.1 创建目录.....315
3.13.2 删除目录.....316
3.14 设置当前会话.....317
3.14.1 时区信息.....317
3.14.2 日期串语言.....318
3.14.3 日期串格式.....319
3.14.4 自然语言排序方式.....320
3.14.5 大小写敏感.....321
3.14.6 本地并行开关.....323
3.15 注释语句.....324
3.16 设置 INI 参数.....326
3.16.1 设置对全局有效.....326
3.16.2 设置仅对当前会话有效.....327
3.17 修改系统语句.....328
3.18 设置列、索引生成统计信息.....329
3.19 设置表生成统计信息.....331
3.20 管理 PROFILE.....333
3.20.1 创建 PROFILE.....333
3.20.2 修改 PROFILE.....334
3.20.3 删除 PROFILE.....334

第 4 章 数据查询语句 336
4.1 单表查询 355
4.1.1 简单查询 355
4.1.2 带条件查询 356
4.1.3 集函数 361
4.1.4 分析函数 380
4.1.5 情况表达式 413
4.2 连接查询 419
4.2.1 交叉连接 419
4.2.2 自然连接(NATURAL JOIN) 421
4.2.3 JOIN … USING 422
4.2.4 JOIN…ON 422
4.2.5 自连接 423
4.2.6 内连接(INNER JOIN) 424
4.2.7 外连接(OUTER JOIN) 426
4.2.8 JOIN APPLY 435
4.3 子查询 436
4.3.1 标量子查询 437
4.3.2 表子查询 439
4.3.3 派生表子查询 444
4.3.4 定量比较 445
4.3.5 带 EXISTS 谓词的子查询 447
4.3.6 多列表子查询 448
4.4 WITH 子句 450

4.4.1 WITH FUNCTION 子句.....451
4.4.2 WITH CTE 子句.....452
4.5 集合查询.....461
4.6 GROUP BY 和 HAVING 子句.....465
4.6.1 GROUP BY 子句的使用.....466
4.6.2 ROLLUP 的使用.....468
4.6.3 CUBE 的使用.....471
4.6.4 GROUPING 的使用.....474
4.6.5 GROUPING SETS 的使用.....475
4.6.6 GROUPING_ID 的使用.....477
4.6.7 GROUP_ID 的使用.....479
4.6.8 HAVING 子句的使用.....481
4.7 ORDER BY 子句.....482
4.8 FOR UPDATE 子句.....485
4.9 TOP 子句.....487
4.10 LIMIT 限定条件.....489
4.10.1 LIMIT 子句.....489
4.10.2 ROW_LIMIT 子句.....491
4.11 PIVOT 和 UNPIVOT 子句.....493
4.11.1 PIVOT 子句.....493
4.11.2 UNPIVOT 子句.....504
4.12 全文检索.....510
4.13 层次查询子句.....514
4.13.1 层次查询子句.....514

4.13.2 层次查询相关伪列.....515
4.13.3 层次查询相关操作符.....516
4.13.4 层次查询相关函数.....516
4.13.5 层次查询层内排序.....517
4.13.6 层次查询的限制.....517
4.14 并行查询.....525
4.15 ROWNUM.....527
4.16 BINARY 前缀.....528
4.17 数组查询.....532
4.18 TABLE 函数查询.....533
4.19 查看执行计划.....536
4.19.1 EXPLAIN.....536
4.19.2 EXPLAIN FOR.....538
4.20 SAMPLE 子句.....545
4.21 水平分区表查询.....547
4.22 指定索引查询.....553
第 5 章 数据的插入、删除和修改.....555
5.1 数据插入语句.....555
5.2 数据修改语句.....565
5.3 数据删除语句.....571
5.4 MERGE INTO 语句.....577
5.5 伪列的使用.....582
5.5.1 ROWID.....582
5.5.2 USER.....583

5.5.3 TRXID 583
5.5.4 PHYROWID 583
5.6 DM 自增列的使用 583
5.6.1 DM 自增列定义 584
5.6.2 SET IDENTITY_INSERT 属性 586
第 6 章 视图 591
6.1 视图的作用 591
6.2 视图的定义 593
6.3 视图的删除 598
6.4 视图的查询 600
6.5 视图的编译 602
6.6 视图数据的更新 603
第 7 章 物化视图 606
7.1 物化视图的定义 606
7.2 物化视图的修改 613
7.3 物化视图的删除 614
7.4 物化视图的刷新 615
7.5 物化视图允许的操作 616
7.6 物化视图日志的定义 616
7.7 物化视图日志的删除 619
7.8 物化视图的限制 620
7.8.1 物化视图的一般限制 620
7.8.2 物化视图的分类 621
7.8.3 快速刷新通用约束 621

7.8.4 物化视图信息查看.....623

7.9 物化视图的编译.....624

第 8 章 函数.....626

8.1 数值函数.....640

8.2 字符串函数.....673

8.3 日期时间函数.....724

8.4 空值判断函数.....779

8.5 类型转换函数.....783

8.6 杂类函数.....795

第 9 章 一致性和并发性.....805

9.1 DM 事务相关语句.....805

9.1.1 事务的开始.....805

9.1.2 事务的结束.....806

9.1.3 保存点相关语句.....808

9.1.4 设置事务隔离级及读写特性.....812

9.2 DM 手动上锁语句.....815

第 10 章 外部函数.....819

10.1 C 外部函数.....819

10.1.1 生成动态库.....820

10.1.2 C 外部函数创建.....824

10.1.3 举例说明.....826

10.2 JAVA 外部函数.....831

10.2.1 生成 jar 包.....831

10.2.2 JAVA 外部函数创建.....831

10.2.3 举例说明.....834
10.3 DMAP 使用说明.....835
10.3.1 启动 DMAP.....836
10.3.2 使用 DMAP 执行 C 外部函数.....837
10.3.3 DMAP 日志.....837
第 11 章 包.....838
11.1 创建包.....838
11.1.1 创建包规范.....838
11.1.2 创建包主体.....840
11.2 重编译包.....843
11.3 删除包.....844
11.3.1 删除包规范.....844
11.3.2 删除包主体.....845
11.4 应用实例.....845
第 12 章 类类型.....851
12.1 普通 CLASS 类型.....851
12.1.1 声明类.....852
12.1.2 实现类.....855
12.1.3 重编译类.....859
12.1.4 删除类.....860
12.1.5 应用实例.....861
12.2 JAVA CLASS 类型.....863
12.2.1 定义 JAVA 类.....864
12.2.2 重编译 JAVA 类.....867

12.2.3 删除 JAVA 类.....867
12.2.4 应用实例.....868
12.3 使用规则.....869
第 13 章 自定义类型.....872
13.1 创建类型.....872
13.2 创建类型体.....875
13.3 重编译类型.....876
13.4 删除类型.....877
13.4.1 删除类型.....877
13.4.2 删除类型体.....878
13.5 自定义类型的使用.....879
13.5.1 使用规则.....879
13.5.2 应用实例.....879
13.5.3 IS OF TYPE 的使用.....882
第 14 章 触发器.....885
14.1 触发器的定义.....886
14.1.1 表触发器.....886
14.1.2 事件触发器.....903
14.1.3 时间触发器.....921
14.2 触发器替换.....924
14.3 设计触发器的原则.....924
14.4 触发器的删除.....925
14.5 禁止和允许触发器.....926
14.6 触发器的重编.....927

14.7 触发器应用举例.....928
14.7.1 使用触发器实现审计功能.....928
14.7.2 使用触发器维护数据完整性.....929
14.7.3 使用触发器保障数据安全性.....931
14.7.4 使用触发器生成字段默认值.....933
第 15 章 同义词.....935
15.1 创建同义词.....935
15.2 删除同义词.....938
第 16 章 外部链接.....940
16.1 创建外部链接.....940
16.2 删除外部链接.....951
16.3 使用外部链接.....952
第 17 章 闪回.....955
17.1 闪回表.....955
17.1.1 闪回表定义.....955
17.1.2 使用说明.....956
17.1.3 使用示例.....957
17.2 闪回查询.....964
17.2.1 闪回查询子句.....965
17.2.2 闪回版本查询.....970
17.2.3 闪回事务查询.....973
第 18 章 JSON.....975
18.1 数据类型.....975
18.1.1 string.....976

18.1.2 number 978
18.1.3 true、false 979
18.1.4 null 980
18.1.5 object 982
18.1.6 array 984
18.2 函数 985
18.2.1 JSON 函数 985
18.2.2 JSONB 函数 1033
18.2.3 CAST 1053
18.3 函数参数详解 1056
18.3.1 路径表达式 1056
18.3.2 PRETTY 和 ASCII 1057
18.3.3 WRAPPER 项 1059
18.3.4 ERROR 项 1060
18.4 运算符 1062
18.4.1 <json_exp> -> <exp> 1062
18.4.2 <json_exp> ->> <exp> 1065
18.4.3 <json_exp> - <exp> 1069
18.4.4 <jsonb_exp1> @> <jsonb_exp2> 1071
18.4.5 <jsonb_exp1> || <jsonb_exp2> 1072
18.5 CHECK 约束 1073
18.6 比较规则 1076
18.6.1 JSON_MODE=1 时 1076
18.6.2 JSON_MODE=2 时 1079

18.7 视图.....1081
18.7.1 视图使用说明.....1081
18.7.2 DBA_JSON_COLUMNS.....1082
18.7.3 USER_JSON_COLUMNS.....1082
18.7.4 ALL_JSON_COLUMNS.....1082
18.8 应用实例.....1083
第 19 章 高级日志.....1088
19.1 简介.....1088
19.2 使用须知.....1088
19.3 语法.....1088
19.3.1 管理日志辅助表.....1088
19.3.2 使用日志辅助表的规则与约束.....1089
19.3.3 日志辅助表结构.....1090
19.3.4 系统过程.....1092
19.4 使用高级日志同步数据的原则.....1092
19.5 应用实例.....1095
19.5.1 创建不带主键的源表.....1095
19.5.2 创建带主键的源表.....1101
第 20 章 自定义运算符.....1106
20.1 创建自定义运算符.....1106
20.2 删除自定义运算符.....1108
20.3 使用自定义运算符.....1109
20.4 应用实例.....1110
第 21 章 自定义集函数.....1114

21.1 创建自定义集函数 1114
21.1.1 创建对象类型 1114
21.1.2 创建集函数 1117
21.2 使用场景 1118
21.3 删除自定义集函数 1119
21.4 应用实例 1119
21.4.1 创建 1120
21.4.2 使用 1122
21.4.3 删除 1127
第 22 章 XML 数据解析 1128
22.1 数据构造 1128
22.1.1 XMLPARSE 1128
22.1.2 XMLELEMENT 1129
22.1.3 XMLFOREST 1131
22.1.4 XMLAGG 1132
22.1.5 FOR XML PATH 1133
22.2 数据查询 1136
22.2.1 SF_XMLQUERY 1136
22.2.2 XMLQUERY 1137
22.2.3 XMLTABLE 1137
22.2.4 EXISTSNODE 1140
22.2.5 EXTRACTVALUE 1142
22.2.6 XMLSEQUENCE 1143
22.2.7 EXTRACT 1144

22.3 数据修改 1147
22.3.1 DELETEXML 1147
22.3.2 APPENDCHILDXML 1148
22.3.3 INSERTCHILDXML 1150
22.3.4 INSERTCHILDXMLAFTER 1152
22.3.5 INSERTCHILDXMLBEFORE 1154
22.3.6 INSERTXMLBEFORE 1156
22.3.7 INSERTXMLAFTER 1158
22.3.8 UPDATEXML 1160
22.4 数据转换 1162
22.4.1 XMLTRANSFORM 1162
22.4.2 XMLSERIALIZE 1164
22.4.3 XMLCAST 1167
附录 1 关键字和保留字 1168
附录 2 SQL 语法书写规则 1175
附录 3 系统存储过程和函数 1180
1. INI 参数管理 1180
2. 系统信息管理 1200
3. 备份恢复管理 1247
4. 定时器管理 1259
5. 模式对象相关信息管理 1266
6. 数据守护管理 1295
7. DMMPP 管理 1314
8. 日志与检查点管理 1318

9. 统计信息 1322
10. 资源监测 1343
11. 类型别名 1361
12. 杂类函数/过程 1364
13. 编目函数调用的系统函数 1401
14. BFILE 1422
15. 定制会话级 INI 参数 1422
16. 为 SQL 注入 HINT 规则 1425
17. 时区设置 1438
18. IP 1441
19. ROWID 1450
20. 系统包 1457
21. 用户自定义分词词库 1464
附录 4 DM 技术支持 1470
手册版本 1473

第 1 章 结构化查询语言 DM_SQL 简介

结构化查询语言 SQL(Structured Query Language)是在 1974 年提出的一种关系数据库语言。由于 SQL 语言接近英语的语句结构，方便简洁、使用灵活、功能强大，倍受用户及计算机工业界的欢迎，被众多计算机公司和数据库厂商所采用，经各公司的不断修改、扩充和完善，SQL 语言最终发展成为关系数据库的标准语言。

SQL 的第一个标准是 1986 年 10 月由美国国家标准化组织(ANSI)公布的 ANSI X3.135-1986 数据库语言 SQL，简称 SQL-86，1987 年国际标准化组织(ISO)也通过了这一标准。以后通过对 SQL-86 的不断修改和完善，于 1989 年第二次公布了 SQL 标准 ISO/IEC 9075-1989(E)，即 SQL-89。1992 年又公布了 SQL 标准 ISO/IEC 9075:1992，即 SQL-92。1999 年公布了 ISO/IEC 9075:1999，即 SQL-3(也称 SQL-99)。之后在 2003 年公布了 ISO/IEC 9075:2003，即 SQL:2003；在 2008 年公布了 ISO/IEC 9075:2008，即 SQL:2008；在 2011 年公布了 ISO/IEC 9075:2011，即 SQL:2011。由于 SQL 标准的内容越来越庞杂，绝大多数情况下，说起 SQL 符合程度，其实是指 SQL-92 中最核心的部分，从 SQL-99 后不再对标准符合程度进行分级，而是改成了核心兼容性和特性兼容性。

SQL 成为国际标准以后，其影响远远超出了数据库领域。例如在 CAD、软件工程、人工智能、分布式等领域，人们不仅把 SQL 作为检索数据的语言规范，而且也把 SQL 作为检索图形、图象、声音、文字等信息类型的语言规范。目前，世界上大型的著名数据库管理系统均支持 SQL 语言，如 Oracle、Sybase、SQL Server、DB2 等。在未来相当长的时间里，SQL 仍将是数据库领域以至信息领域中数据处理的主流语言之一。

由于不同的 DBMS 产品，大都按自己产品的特点对 SQL 语言进行了扩充，很难完全符合 SQL 标准。目前在 DBMS 市场上已将 SQL 的符合率作为衡量产品质量的重要指标，并研制成专门的测试软件，如 NIST。目前，DM 数据库管理系统 SQL-92 入门级符合率达到 100%，

过渡级符合率达到 95%，并且部分支持 SQL-99、SQL:2003、SQL:2008 和 SQL:2011 的特性。同时 DM 还兼容 Oracle 11g 和 SQL Server 2008 的部分语言特性。本章主要介绍 DM 系统所支持的 SQL 语言——DM_SQL 语言。

1.1 DM_SQL 语言的特点

DM_SQL 语言符合结构化查询语言 SQL 标准，是标准 SQL 的扩充。它集数据定义、数据查询、数据操纵和数据控制于一体，是一种统一的、综合的关系数据库语言。它功能强大，使用简单方便、容易为用户掌握。DM_SQL 语言具有如下特点：

1. 功能一体化

DM_SQL 的功能一体化表现在以下两个方面：

1） DM_SQL 支持多媒体数据类型，用户在建表时可直接使用。DM 系统在处理常规数据与多媒体数据时达到了四个一体化：一体化定义、一体化存储、一体化检索、一体化处理，最大限度地提高了数据库管理系统处理多媒体的能力和速度；

2） DM_SQL 语言集数据库的定义、查询、更新、控制、维护、恢复、安全等一系列操作于一体，每一项操作都只需一种操作符表示，格式规范，风格一致，简单方便，很容易为用户所掌握。

2. 两种用户接口使用统一语法结构的语言

DM_SQL 语言既是自含式语言，又是嵌入式语言。作为自含式语言，它能独立运行于联机交互方式。作为嵌入式语言，DM_SQL 语句能够嵌入到 C 和 C++语言程序中，将高级语言(也称主语言)灵活的表达能力、强大的计算功能与 DM_SQL 语言的数据处理功能相结合，完成各种复杂的事务处理。而在这两种不同的使用方式中，DM_SQL 语言的语法结构是一致的，从而为用户使用提供了极大的方便性和灵活性。

3. 高度非过程化

DM_SQL 语言是一种非过程化语言。用户只需指出“做什么”，而不需指出“怎么做”，对

数据存取路径的选择以及 DM_SQL 语句功能的实现均由系统自动完成，与用户编制的应用程序与具体的机器及关系 DBMS 的实现细节无关，从而方便了用户，提高了应用程序的开发效率，也增强了数据独立性和应用系统的可移植性。

4. 面向集合的操作方式

DM_SQL 语言采用了集合操作方式。不仅查询结果可以是元组的集合，而且一次插入、删除、修改操作的对象也可以是元组的集合，相对于面向记录的数据库语言(一次只能操作一条记录)来说，DM_SQL 语言的使用简化了用户的处理，提高了应用程序的运行效率。

5. 语言简洁，方便易学

DM_SQL 语言功能强大，格式规范，表达简洁，接近英语的语法结构，容易为用户所掌握。

1.2 保留字与标识符

标识符的语法规则兼容标准 GJB 1382A-9X，标识符分为正规标识符和定界标识符两大类。

正规标识符以字母、_、$、#或汉字开头，后面可以跟随字母、数字、_、$、#或者汉字，正规标识符的最大长度是 128 个英文字符或 64 个汉字。正规标识符不能是保留字。

正规标识符的例子：A，test1，_TABLE_B，表 1。

定界标识符的标识符体用双引号括起来时，标识符体可以包含任意字符，特别地，其中使用连续两个双引号转义为一个双引号。

定界标识符的例子："table"，"A"，"!@#$"。

保留字的清单参见附录 1 关键字和保留字。

1.3 DM_SQL 语言的功能及语句

DM_SQL 语言是一种介于关系代数与关系演算之间的语言，其功能主要包括数据定义、

查询、操纵和控制四个方面，通过各种不同的 SQL 语句来实现。按照所实现的功能，DM_SQL 语句分为以下几种：

1. 用户、模式、基表、视图、索引、序列、全文索引、存储过程、触发器等数据库对象的定义和删除语句，数据库、用户、基表、视图、索引、全文索引等数据库对象的修改语句；
2. 查询(含全文检索)、插入、删除、修改语句；
3. 数据库安全语句。包括创建角色语句、删除角色语句，授权语句、回收权限语句，修改登录口令语句，审计设置语句、取消审计设置语句等。

在嵌入方式中，为了协调 DM_SQL 语言与主语言不同的数据处理方式，DM_SQL 语言引入了游标的概念。因此在嵌入方式下，除了数据查询语句(一次查询一条记录)外，还有几种与游标有关的语句：

1. 游标的定义、打开、关闭、拨动语句；
2. 游标定位方式的数据修改与删除语句。

为了有效维护数据库的完整性和一致性，支持 DBMS 的并发控制机制，DM_SQL 语言提供了事务的回滚(ROLLBACK)与提交(COMMIT)语句。同时 DM 允许选择实施事务级读一致性，它保证同一事务内的可重复读，为此 DM 提供用户多种手动上锁语句，和设置事务隔离级别语句。

1.4 DM_SQL 所支持的数据类型

数据类型是可表示值的集。值的逻辑表示是<字值>。值的物理表示依赖于实现。DM 系统具有 SQL-92 的绝大部分数据类型，以及部分 SQL-99 和 SQL Server 2000 的数据类型。

1.4.1 常规数据类型

1. **字符数据类型**

CHAR 类型

语法：CHAR[(长度)] 或 CHAR(字符长度 CHAR)

功能：CHAR 数据类型指定定长字符串。

在基表中，定义 CHAR 类型的列时，可指定字节长度（1~32767）。例如：CHAR(100)。如果未指定长度，缺省为 1 字节。也可以指定字符长度(1~32767)。例如：CHAR(100 CHAR)。

基表中 CHAR 类型列的最大存储长度由数据库页面大小决定，CHAR 数据类型最大存储长度和页面大小的对应关系请见下表 1.4.1。但是，在表达式计算中，该类型的长度上限不受页面大小限制，为 32767。

表 1.4.1 最大存储长度和页面大小的对应关系

数据库页面大小	**实际最大存储长度**
4K	约 1900 字节
8K	约 3900 字节
16K	约 8000 字节
32K	约 16000 字节

这个限制长度只针对基表中的列，在定义变量的时候，可以不受这个限制长度的限制。另外，实际插入表中的列长度要受到记录长度的约束，每条记录总长度不能大于页面大小的一半。

CHARACTER 类型

语法：CHARACTER[(长度)] 或 CHARACTER (字符长度 CHAR)

功能：与 CHAR 相同。

VARCHAR 类型

语法：VARCHAR[(长度 [CHAR])]

功能：VARCHAR 数据类型指定变长字符串，用法类似 CHAR 数据类型，可以指定一个不超过 32767 的正整数作为字节或字符长度，例如：VARCHAR（100)指定 100 字节长度；VARCHAR(100 CHAR)指定 100 字符长度。如果未指定长度，缺省为 8188 字节。

在基表中，当没有指定 USING LONG ROW 存储选项时，插入 VARCHAR 数据类型的

实际最大存储长度由数据库页面大小决定，具体最大长度算法如表 1.4.1；如果指定了 USING LONG ROW 存储选项，则插入 VARCHAR 数据类型的长度不受数据库页面大小限制。VARCHAR 类型在表达式计算中的长度上限不受页面大小限制，为 32767。

CHAR 同 VARCHAR 的区别在于前者长度不足时，系统自动填充空格，而后者只占用实际的字节空间。另外，实际插入表中的列长度要受到记录长度的约束，每条记录总长度不能大于页面大小的一半。

VARCHAR2 类型

```
语法：VARCHAR2[(长度 [CHAR])]
```

功能：VARCHAR2 类型和 VARCHAR 类型用法相同。

ROWID 类型

```
语法：ROWID
```

功能：ROWID 类型数据由 18 位字符组成，用来表示 ROWID 数据。18 位字符由“4 位站点号+6 位分区号+8 位物理行号”组成。ROWID 类型数据可通过 SF_BUILD_ROWID() 构造而来。

表中的 ROWID 类型列，可以用于排序或创建索引。但是 ROWID 类型不支持作为分区列和自定义类型的属性数据类型。ROWID 列与字符类型一样，支持 MAX，MIN 等集函数，不支持 SUM，AVG 等集函数。

例 创建一个含有 ROWID 类型的数据库表。

先构造 ROWID 数据。假定站点号为 1，分区号为 2，物理行号为 50。使用 SF_BUILD_ROWID 函数构造出一个 ROWID 类型数据。

```
SELECT SF_BUILD_ROWID(1,2,50);
```

查询结果如下：

```
AAABAAAAACAAAAAAAy
```

其中，AAAB 为站点号、AAAAAC 为分区号、AAAAAAAy 为 ROWID 值。

创建含有 ROWID 类型的表，并插入数据。

```
CREATE TABLE T(C1 INT,C2 ROWID);
INSERT INTO T VALUES(8,'AAABAAAAACAAAAAAAy');
SELECT C1,C2,ROWID FROM T;
```

c2 列为插入的值，ROWID 为当前数据行的伪列 ROWID。查询结果如下:

```
行号          C1              C2                      ROWID
---------- ----------- ----------------- ------------------
1             8              AAABAAAAACAAAAAAAy AAAAAAAAAAAAAAAAAB
```

2. 数值数据类型

1) 精确数值数据类型

NUMERIC 类型

```
语法：NUMERIC[(精度 [, 标度])]
```

功能：NUMERIC数据类型用于存储零、正负定点数。其中：精度是一个无符号整数，定义了总的数字数，精度范围是1至38。标度定义了小数点右边的数字位数。一个数的标度不应大于其精度，如果实际标度大于指定标度，那么超出标度的位数将会四舍五入省去。例如：NUMERIC(4,1)定义了小数点前面3位和小数点后面1位，共4位的数字，范围在-999.9到999.9。所有NUMERIC数据类型，如果其值超过精度，DM会返回一个出错信息，如果超过标度，则多余的位会被截断。

当指定精度而不指定标度时，缺省标度为0，相当于整数。如果不指定精度和标度，标度根据缺省精度和输入值的小数位动态调整。缺省精度由INI参数DECIMAL_ENHANCED决定。当DECIMAL_ENHANCED=1，缺省精度为40；当DECIMAL_ENHANCED=0，缺省精度为38。

NUMERIC数据类型计算结果精度和NUMERIC数据类型的精度不同。NUMERIC数据类型计算结果可以为零、正负定点数。其中，正数的精度范围$1*10^{-128}$~$9.99\ldots\ldots9*10^{125}$，负数

的精度范围为 -1*10 $^{-128}$ ~-9.99 … … 9*10 ^{123} 。计算结果的有效精度由 INI 参数 DECIMAL_ENHANCED决定，超过有效精度位数的数字全用0补齐。

例 当DECIMAL_ENHANCED=0时，变量a，b最多只能存储38位。但是计算的结果c为76位（处于计算结果的精度范围之内）。打印结果的有效精度为38位，38位是四舍五入后的结果，超过38位的全部用0补齐。

```
DECLARE
  a  NUMERIC :=12345678901234567890123456789012345678;      //38 位
  b  NUMERIC :=10000000000000000000000000000000000001;     //38 位
  c  NUMERIC;
BEGIN
  c := a*b;            //c 用来存储计算结果
print c;
END;
/
打印结果为:
1234567890123456789012345678901234567900000000000000000000000000000000000000
```

DECIMAL类型

```
语法：DECIMAL[(精度 [, 标度])]
```

功能：与 NUMERIC 相同。

DEC 类型

```
语法：DEC[(精度[, 标度])]
```

功能：与 DECIMAL 相同。

NUMBER 类型

```
语法：NUMBER[(精度[, 标度])]
```

功能：与 NUMERIC 相同。

INTEGER 类型

语法：INTEGER

功能：用于存储有符号整数，精度为 10，标度为 0。取值范围为：-2147483648 (-2^{31})~+2147483647($2^{31}-1$)。

INT 类型

语法：INT

功能：与 INTEGER 相同。

PLS_INTEGER 类型

语法：PLS_INTEGER

功能：与 INTEGER 相同。

BIGINT 类型

语法：BIGINT

功能：用于存储有符号整数，精度为 19，标度为 0。取值范围为：-9223372036854775808(-2^{63})~9223372036854775807($2^{63}-1$)。

TINYINT 类型

语法：TINYINT

功能：用于存储有符号整数，精度为 3，标度为 0。取值范围为：-128~+127。

BYTE 类型

语法：BYTE

功能：与 TINYINT 相似，精度为 3，标度为 0。

SMALLINT 类型

语法：SMALLINT

功能：用于存储有符号整数，精度为 5，标度为 0。取值范围为：

-32768(-2^{15})~+32767(2^{15}-1)。

BINARY 类型

语法：BINARY[(长度)]

功能：BINARY 数据类型用来存储定长二进制数据。在基表中，定义 BINARY 类型的列时，其最大存储长度由数据库页面大小决定，可以指定一个不超过其最大存储长度的正整数作为列长度，缺省长度为 1 个字节。最大存储长度见表 1.4.1。BINARY 类型在表达式计算中的长度上限为 32767。BINARY 常量以 0x 开始，后面跟着数据的十六进制表示，例如：0x2A3B4058。

VARBINARY 类型

语法：VARBINARY[(长度)]

功能：VARBINARY 数据类型用来存储变长二进制数据，用法类似 BINARY 数据类型，可以指定一个不超过 32767 的正整数作为数据长度。缺省长度为 8188 个字节。VARBINARY 数据类型的实际最大存储长度由数据库页面大小决定，具体最大长度算法与 VARCHAR 类型的相同，其在表达式计算中的长度上限也与 VARCHAR 类型相同，为 32767。

RAW 类型

语法：RAW[(长度)]

功能：与 VARBINARY 相同。

2） 近似数值数据类型

FLOAT 类型

语法：FLOAT[(精度)]

功能：FLOAT 是带二进制精度的浮点数，精度范围（1~126）。当精度小于等于 24 时，DM 将其转换为标准 C 语言中的 REAL 类型；当精度大于 24 时，转换为标准 C 语言中的 DOUBLE 类型。

FLOAT 取值范围-1.7976931348623158*10^{308}~1.7976931348623158*10^{308}。

DOUBLE 类型

语法：DOUBLE[(精度)]

功能：DOUBLE 是带二进制精度的浮点数。DOUBLE 类型的设置是为了移植的兼容性。该类型直接使用标准 C 语言中 DOUBLE。精度与取值范围与 FLOAT 一样。

REAL 类型

语法：REAL

功能：REAL 是带二进制精度的浮点数，但它不能由用户指定使用的精度，系统指定其二进制精度为 24，十进制精度为 7。取值范围-3.4*10^{38}~3.4*10^{38}。

DOUBLE PRECISION 类型

语法：DOUBLE PRECISION[(精度)]

功能: 该类型指明双精度浮点数。DOUBLE PRECISION 类型的设置是为了移植的兼容性。该类型直接使用标准 C 语言中 DOUBLE。精度与取值范围与 FLOAT 一样。

3） 位串数据类型

BIT 类型

语法：BIT

功能：BIT 类型用于存储整数数据 1、0 或 NULL，只有 0 才转换为假，其他非空、非 0 值都会自动转换为真，可以用来支持 ODBC 和 JDBC 的布尔数据类型。DM 的 BIT 类型与 SQL SERVER2000 的 BIT 数据类型相似。

功能与 ODBC 和 JDBC 的 BOOL 相同。

1.4.2 日期时间数据类型

日期时间数据类型分为一般日期时间数据类型、时间间隔数据类型和时区数据类型三类，用于存储日期、时间和它们之间的间隔信息。

1. 一般日期时间数据类型

1） DATE 类型

```
语法：DATE
```

功能：DATE 类型包括年、月、日信息，定义了'-4712-01-01'和'9999-12-31'之间任何一个有效的格里高利日期。DM 支持儒略历，并考虑了历史上从儒略历转换至格里高利日期时的异常，将'1582-10-05'到'1582-10-14'之间的日期统一当做'1582-10-15'处理。

DATE 值的书写方式有两种：一是 DATE'年月日'；二是'年月日'。其中，年月日之间可以使用分隔符或者没有分隔符。分隔符是指除大小写字母、数字以及双引号之外的所有单字节字符且是可打印的。例如：空格、回车键、tab 键、- / , . : *等标点符号。年月日中第一个非 0 数值前的 0 亦可省略，例如：'0001-01-01'等价于'1-1-1'。

例如：

```
CREATE TABLE T2(C1 DATE,C2 DATE,C3 DATE );
INSERT INTO T2 VALUES(DATE '1999-10-01','1999/10/01','1999.10.01');
```

2） TIME 类型

```
语法：TIME[(小数秒精度)]
```

功能：TIME 类型包括时、分、秒信息，定义了一个在'00:00:00.000000'和'23:59:59.999999'之间的有效时间。TIME 类型的小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~6，如果未定义，缺省精度为 0。

TIME 值的书写方式有两种：一是 TIME'时:分:秒'；二是'时:分:秒'。

例如：

```
CREATE TABLE T2(C1 TIME(2),C2 TIME,C3 TIME);
INSERT INTO T2 VALUES(TIME '09:10:21.20','09:10:21','9:10:21.49');
```

3） TIMESTAMP/DATETIME 类型

```
语法：TIMESTAMP[(小数秒精度)]
      DATETIME[(小数秒精度)]
```

功能：TIMESTAMP/DATETIME 类型包括年、月、日、时、分、秒信息，定义了一个在'-4712-01-01 00:00:00.000000000'和'9999-12-31 23:59:59.999999999'之间的有效格里高利日期时间。

TIMESTAMP 类型的小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~9，默认秒精度为 6。可以通过 IFUN_DATETIME_MODE 指定返回值或参数为 TIMESTAMP/TIMESTAMP WITH TIME ZONE 类型的系统函数的最大精度。当 IFUN_DATETIME_MODE 取值为 0 时，默认取值为 0，此时毫秒精度最大为 6，IFUN_DATETIME_MODE 为 1 时，毫秒精度最大为 9。

与 DATE 类型相同，DM 不计算'1582-10-05'到'1582-10-14'之间的 10 天。

TIMESTAMP 值的书写方式有两种：一是 TIMESTAMP'<DATE 值> <TIME 值>'；二是'<DATE 值> <TIME 值>'。语法中，TIMESTAMP 也可以写为 DATETIME。

例如：

```
CREATE TABLE T2(C1 TIMESTAMP,C2 DATETIME,C3 TIMESTAMP,C4 DATETIME,C5 TIMESTAMP);
INSERT INTO T2 VALUES(TIMESTAMP '2002-12-12 09:10:21',TIMESTAMP '2002-12-12
09:10:21','2002/12/12 09:10:21','2002.12.12 09:10:21',DATETIME'2002-12-12 09:10:21' );
```

2. 时间间隔数据类型

DM 支持两类十三种时间间隔类型：两类是年-月间隔类和日-时间隔类，它们通过时间间隔限定符区分，前者结合了日期字段年和月，后者结合了时间字段日、时、分、秒。由时间间隔数据类型所描述的值总是有符号的。

需要说明的是，使用时间间隔数据类型时，如果使用了其引导精度的默认精度，要注意保持精度匹配，否则会出现错误。如果不指定精度，那么将使用默认精度。

(一) 年-月间隔类

1) INTERVAL YEAR TO MONTH 类型

```
语法：INTERVAL YEAR［(引导精度)］TO MONTH
```

功能：描述一个若干年若干月的间隔，引导精度规定了年的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。月的取值范围在 0 到 11 之间。例如：INTERVAL YEAR(4) TO MONTH，其中 YEAR(4)表示年的精度为 4，表示范围为负 9999 年零 11 月到正 9999 年零 11 月。一个合适的字值例子是：INTERVAL '15-8' YEAR(2) TO MONTH。

2）INTERVAL YEAR 类型

语法：INTERVAL YEAR［(引导精度)］

功能：描述一个若干年的间隔，引导精度规定了年的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。例如：INTERVAL YEAR(4)，其中 YEAR(4)表示年的精度为 4，表示范围为负 9999 年到正 9999 年。一个合适的字值例子是：INTERVAL '15' YEAR(2)。

3）INTERVAL MONTH 类型

语法：INTERVAL MONTH［(引导精度)］

功能：描述一个若干月的间隔，引导精度规定了月的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。例如：INTERVAL MONTH(4)，其中 MONTH(4)表示月的精度为 4，表示范围为负 9999 月到正 9999 月。一个合适的字值例子是：INTERVAL '15' MONTH(2)。

（二） 日-时间隔类

1） INTERVAL DAY 类型

语法：INTERVAL DAY［(引导精度)］

功能：描述一个若干日的间隔，引导精度规定了日的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。例如：INTERVAL DAY(3)，其中 DAY (3)表示日的精度为 3，表示范围为负 999 日到正 999 日。一个合适的字值例子是：INTERVAL '150' DAY(3)。

2）INTERVAL DAY TO HOUR 类型

语法：INTERVAL DAY［(引导精度)］ TO HOUR

功能：描述一个若干日若干小时的间隔，引导精度规定了日的取值范围。引导精度取值

范围为 1~9，如果未定义，缺省精度为 2。而时的取值范围在 0 到 23 之间。例如：INTERVAL DAY(1) TO HOUR，其中 DAY(1)表示日的精度为 1，表示范围为负 9 日零 23 小时到正 9 日零 23 小时。一个合适的字值例子是：INTERVAL '9 23' DAY(2) TO HOUR。

3）INTERVAL DAY TO MINUTE 类型

语法：INTERVAL DAY［(引导精度)］ TO MINUTE

功能：描述一个若干日若干小时若干分钟的间隔，引导精度规定了日的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。而小时的取值范围在 0 到 23 之间，分钟的取值范围在 0 到 59 之间。例如：INTERVAL DAY(2) TO MINUTE，其中 DAY(2)表示日的精度为 2，表示范围为负 99 日零 23 小时零 59 分到正 99 日零 23 小时零 59 分。一个合适的字值例子是：INTERVAL '9 23:12' DAY(2) TO MINUTE。

4）INTERVAL DAY TO SECOND 类型

语法：INTERVAL DAY［(引导精度)］ TO SECOND [(小数秒精度)]

功能：描述一个若干日若干小时若干分钟若干秒的间隔，引导精度规定了日的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~6，如果不定义小数秒精度默认精度为 6。小时的取值范围在 0 到 23 之间，分钟的取值范围在 0 到 59 之间。例如：INTERVAL DAY(2) TO SECOND(1)，其中 DAY(2)表示日的精度为 2，SECOND(1)表示秒的小数点后面取 1 位，表示范围为负 99 日零 23 小时零 59 分零 59.9 秒到正 99 日零 23 小时零 59 分零 59.9 秒。一个合适的字值例子是：INTERVAL '9 23:12:01.1' DAY(2) TO SECOND(1)。

5）INTERVAL HOUR 类型

语法：INTERVAL HOUR［(引导精度)］

功能：描述一个若干小时的间隔，引导精度规定了小时的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。例如：INTERVAL HOUR(3)，其中 HOUR (3)表示时的精度为 3，表示范围为负 999 小时到正 999 小时。例如：INTERVAL '150' HOUR(3)。

6）INTERVAL HOUR TO MINUTE 类型

语法：INTERVAL HOUR［(引导精度)］ TO MINUTE

功能：描述一个若干小时若干分钟的间隔，引导精度规定了小时的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。而分钟的取值范围在 0 到 59 之间。例如：INTERVAL HOUR(2) TO MINUTE，其中 HOUR(2)表示小时的精度为 2，表示范围为负 99 小时零 59 分到正 99 小时零 59 分。一个合适的字值例子是：INTERVAL '23:12' HOUR(2) TO MINUTE。

7）INTERVAL HOUR TO SECOND 类型

语法：INTERVAL HOUR［(引导精度)］ TO SECOND [(小数秒精度)]

功能：描述一个若干小时若干分钟若干秒的间隔，引导精度规定了小时的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~6，如果未定义，缺省精度为 6。分钟的取值范围在 0 到 59 之间。例如：INTERVAL HOUR(2) TO SECOND(1)，其中 HOUR(2)表示小时的精度为 2，SECOND(1)表示秒的小数点后面取 1 位，表示范围为负 99 小时零 59 分零 59.9 秒到正 99 小时零 59 分零 59.9 秒。一个合适的字值例子是：INTERVAL '23:12:01.1' HOUR(2) TO SECOND(1)。

8）INTERVAL MINUTE 类型

语法：INTERVAL MINUTE［(引导精度)］

功能：描述一个若干分钟的间隔，引导精度规定了分钟的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。例如：INTERVAL MINUTE(3)，其中 MINUTE(3)表示分钟的精度为 3，表示范围为负 999 分钟到正 999 分钟。一个合适的字值例子是：INTERVAL '150' MINUTE(3)。

9）INTERVAL MINUTE TO SECOND 类型

语法：INTERVAL MINUTE［(引导精度)］ TO SECOND [(小数秒精度)]

功能：描述一个若干分钟若干秒的间隔，引导精度规定了分钟的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~6，如果未定义，缺省精度为 6。例如：INTERVAL MINUTE(2) TO SECOND(1)，其中 MINUTE(2)表示分钟的精度为 2，SECOND(1)表示秒的小数点后面取 1 位，表示范围为负 99 分零 59.9 秒到正 99 分零 59.9 秒。一个合适的字值例子是：INTERVAL '12:01.1' MINUTE(2) TO SECOND(1)。

10） INTERVAL SECOND 类型

```
语法：INTERVAL SECOND［(引导精度［,小数秒精度］)］
```

功能：描述一个若干秒的间隔，引导精度规定了秒整数部分的取值范围。引导精度取值范围为 1~9，如果未定义，缺省精度为 2。小数秒精度规定了秒字段中小数点后面的位数，取值范围为 0~6，如果未定义，缺省精度为 6。例如：INTERVAL SECOND(2,1)，表示范围为负 99.9 秒到正 99.9 秒。一个合适的字值例子是：INTERVAL '51.1' SECOND(2,1)。

3. 时区数据类型

DM 支持两种时区类型：标准时区类型和本地时区类型。

(一) 标准时区类型：带时区的 TIME 类型和带时区的 TIMESTAMP 类型。

1） TIME WITH TIME ZONE 类型

```
语法：TIME［(小数秒精度)］WITH TIME ZONE
```

功能：描述一个带时区的 TIME 值，其定义是在 TIME 类型的后面加上时区信息。TIME 值部分与 1.4.2 节 1. 一般日期时间数据类型中的描述一致；<时区>部分的实质是 INTERVAL HOUR TO MINUTE 类型，取值范围：-12:59 与+14:00 之间。

TIME WITH TIME ZONE 值的书写方式有两种：一是 TIME'时分秒 <时区>'；二是'时分秒<时区>'。

例如：

```
CREATE TABLE T2(C1 TIME (2) WITH TIME ZONE,C2 TIME (2) WITH TIME ZONE);
```

```
INSERT INTO T2 VALUES(TIME '09:10:21 +8:00','09:10:21 -11:00');
```

2） TIMESTAMP WITH TIME ZONE 类型

```
语法：TIMESTAMP［(小数秒精度)］WITH TIME ZONE
```

功能：描述一个带时区的 TIMESTAMP 值，其定义是在 TIMESTAMP 类型的后面加上时区信息。TIMESTAMP 值部分与 1.4.2 节 1.一般日期时间数据类型中的描述一致，<时区>部分的实质是 INTERVAL HOUR TO MINUTE 类型，取值范围：-12:59 与+14:00 之间。

TIMESTAMP WITH TIME ZONE 值的书写方式有两种：一是 TIMESTAMP'<DATE 值> <TIME 值> <时区>'；二是'<DATE 值> <TIME 值> <时区>'。语法中，TIMESTAMP 也可以写为 DATETIME。

例如：

```
CREATE TABLE T2(C1 TIMESTAMP(2) WITH TIME ZONE,C2 TIMESTAMP(2) WITH TIME ZONE,C3
TIMESTAMP(2) WITH TIME ZONE,C4 TIMESTAMP(2) WITH TIME ZONE);
INSERT INTO T2 VALUES(TIMESTAMP '2002-12-12 09:10:21 +8:00','2002-12-12 9:10:21
+9:00','2002/12/12 09:10:21 -10:00','2002.12.12 09:10:21 -5:00');
```

（二） 本地时区类型：本地时区的 TIMESTAMP 类型。

1） TIMESTAMP WITH LOCAL TIME ZONE 类型

```
语法：TIMESTAMP［(小数秒精度)］WITH LOCAL TIME ZONE
```

功能：描述一个本地时区的 TIMESTAMP 值，能够将标准时区类型 TIMESTAMP WITH TIME ZONE 类型转化为本地时区类型，如果插入的值没有指定时区，则默认为本地时区。

TIMESTAMP WITH LOCAL TIME ZONE 值的书写方式有两种：一是 TIMESTAMP'<DATE 值> <TIME 值> <时区>'；二是'<DATE 值> <TIME 值> <时区>'。语法中，TIMESTAMP 也可以写为 DATETIME。

例如：

```
CREATE TABLE T2(C1 TIMESTAMP(3) WITH LOCAL TIME ZONE,C2 TIMESTAMP(3) WITH LOCAL TIME
```

```
ZONE );
INSERT INTO T2 VALUES(TIMESTAMP '2002-12-12 09:10:21 +8:00', TIMESTAMP '2002-12-12 09:10:21');
SELECT * FROM T2;
```

查询结果如下:

```
2002-12-12 09:10:21.000       2002-12-12 09:10:21.000
```

1.4.3 多媒体数据类型

多媒体数据类型的字值有两种格式：一是字符串，例如：'ABCD'，二是 BINARY，例如：0x61626364。

TEXT、LONG、LONGVARCHAR、CLOB、JSON、JSONB 只支持字符串。

BFILE 不适用上面两种格式。BFILE 指明的文件只能只读访问。

BINARY 格式的多媒体数据类型以及 BFILE 数据类型不支持比较。

不支持为多媒体数据类型的字段指定精度。

1. TEXT 类型

```
语法：TEXT
```

功能：TEXT 为变长字符串类型。其字符串的长度最大为 100G-1 字节。DM 利用它存储长的文本串。

2. LONG、LONGVARCHAR(又名 TEXT)类型

```
语法：LONG/LONGVARCHAR
```

功能：与 TEXT 相同。

3. IMAGE 类型

```
语法：IMAGE
```

功能：IMAGE 用于指明多媒体信息中的图像类型。图像由不定长的像素点阵组成，长度最大为 100G-1 字节。该类型除了存储图像数据之外，还可用于存储任何其它二进制数据。

4. LONGVARBINARY(又名 IMAGE)类型

语法：LONGVARBINARY

功能：与 IMAGE 相同。

5. BLOB 类型

语法：BLOB

功能：BLOB 类型用于指明变长的二进制大对象，长度最大为 100G-1 字节。

6. CLOB 类型

语法：CLOB

功能：与 TEXT 相同。

7. BFILE 类型

语法：BFILE

功能：BFILE 用于指明存储在操作系统中的二进制文件，文件存储在操作系统而非数据库中，仅能进行只读访问。

8. JSON、JSONB 类型

语法：JSON/JSONB

功能：JSON 和 JSONB 是轻量级的数据格式，易于阅读和编写，对于二者之间的区别和具体使用方法请参考第 18 章 JSON。

1.5 DM_SQL 语言支持的表达式

DM 支持多种类型的表达式，包括数值表达式、字符串表达式、时间值表达式、时间间隔值表达式等。本节中引用的数据库实例请参见第 2 章 手册中的示例说明。

1.5.1 数值表达式

1. 一元算符 + 和 -

```
语法：+exp 、-exp
```

（exp 代表表达式，下同）

功能：当单独使用时，+ 和 – 代表表达式的正负号。

例如：

```
SELECT -(-5), +NOWPRICE FROM PRODUCTION.PRODUCT WHERE NOWPRICE<10;
```

查询结果为：5　　6.1000

注：在 SQL 中由于两短横即“--”，表示“注释开始”，则双负号必须是-(-5)，而不是--5。

2. 一元算符 ~

```
语法：~exp
```

功能：按位非算符，要求参与运算的操作数都为整数数据类型。

例如：

```
SELECT ~10 FROM DUAL;
```

查询结果为：-11

3. 二元算符 +、-、*、/、%

```
语法：exp1+exp2 、exp1- exp2 、exp1*exp2 、exp1/exp2 、exp1%exp2
```

（exp1 代表表达式 1，exp2 代表表达式 2，下同）

功能：当在表达式之间使用+、-、*、/、%时，分别表示加、减、乘、除、取余数运算。对于结果的精度规定如下：

1） 只有精确数值数据类型的运算

两个相同类型的整数运算的结果类型不变，两个不同类型的整数运算的结果类型转换为范围较大的那个整数类型。

整数与 NUMERIC，DEC 等类型运算时，SMALLINT 类型的精度固定为 5，标度为 0；

INTEGER 类型的精度固定为 10，标度为 0；BIGINT 类型的精度固定为 19，标度为 0。

exp1+exp2 结果的精度为二者整数部分长度(即精度一标度)的最大值与二者标度的最大值之和，标度是二者标度的最大值；

exp1-exp2 结果的精度为二者整数部分长度的最大值与二者标度的最大值之和，标度是二者标度的最大值；

exp1*exp2 结果的精度为二者的精度之和，标度也是二者的标度之和；

exp1/exp2 结果为 DEC 类型（缺省的精度为 38，标度未限定)；

exp1%exp2 结果为 DEC 类型（缺省的精度为 38，标度未限定)。

例 1 使用+、-运算

```
SELECT NOWPRICE+1, NOWPRICE-1 FROM PRODUCTION.PRODUCT WHERE NOWPRICE<10;
```

查询结果为：7.1　5.1

例 2 使用*、/运算

```
SELECT NOWPRICE*10, NOWPRICE/10 FROM PRODUCTION.PRODUCT WHERE NOWPRICE<10;
```

查询结果为：61　　0.61

例 3 使用%运算

```
SELECT NOWPRICE%4 FROM PRODUCTION.PRODUCT WHERE NOWPRICE<10;
```

查询结果为：2.1

2） 有近似数值数据类型的运算

对于 exp1+exp2 、exp1- exp2 、exp1*exp2 、exp1/exp2 、exp1%exp2 中 exp1 和 exp2 只要有一个为近似数值数据类型，则结果为近似数值数据类型。

例如：

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT, C2 DOUBLE);

INSERT INTO TEST VALUES(5, 3.3);
```

```
SELECT C1-C2, C1/C2 FROM TEST;
```

查询结果为：1.700000000000000E+00　　1.5151515151515150E+00

4. 二元算符 &

```
语法：exp1 & exp2
```

功能：按位与算符，要求参与运算的操作数都为整数数据类型。

例如：

```
SELECT  20 & 10 FROM DUAL;
```

查询结果为：0

注：在 DISQL 中不支持该运算符，建议使用 BITAND 函数代替。

5. 二元算符 |

```
语法：exp1 | exp2
```

功能：按位或算符，要求参与运算的操作数都为整数数据类型。

例如：

```
SELECT  20 | 10 FROM DUAL;
```

查询结果为：30

6. 二元算符 ^

```
语法：exp1 ^ exp2
```

功能：按位异或算符，要求参与运算的操作数都为整数数据类型。

例如：

```
SELECT  20 ^ 10 FROM DUAL;
```

查询结果为：30

7. 二元算符<<、>>

```
语法：exp1 << exp2
      exp1 >> exp2
```

功能：左移、右移运算符，要求参与运算的操作数只能为整数数据类型、精确数据类型。

例 1 左移运算符的使用

```
SELECT  1 << 3 FROM DUAL;
```

查询结果为：8

例 2 右移运算符的使用

```
SELECT  8 >> 3 FROM DUAL;
```

查询结果为：1

当在表达式之间使用<<、>> 时，对于表达式 exp1、exp2 的类型规则如下：

1） 只有整数数据类型的运算

左右操作数都为 smallint、tinyint 时操作结果为 int 类型；

左右操作数都为 int 类型时操作结果为 int 类型；

左右操作数都为 bigint 类型时操作结果为 bigint 类型。

左操作数是 int、smallint、tinyint 类型，右操作数为 int、smallint、tinyint 类型时，操作结果为范围较大的类型。

当左操作数或右操作数有一个是 bigint 类型时，操作结果为 bigint 类型。

2） 有精确数据类型的运算：

当左操作数、右操作数都是精确数据类型时，分别四舍五入转化成 bigint 类型后运算，结果为 bigint 类型；

当整数与精确数据类型运算时，将精确数据类型四舍五入转化成 bigint 类型后运算，结果为 bigint 类型。

3） 有字符串数据类型的运算

字符串指数字字符串，不支持字符串与字符串运算；

当整数与数字字符串数据类型运算时，结果为整数数据类型；

当精确数据类型与字符串数据类型运算时，结果为 bigint 类型。

1.5.2 字符串表达式

连接 ||

```
语法：STR1 || STR2
```

（STR1 代表字符串 1，STR2 代表字符串 2）

功能：连接操作符对两个运算数进行运算，其中每一个都是对属于同一字符集的字符串的求值。它以给定的顺序将字符串连接在一起，并返回一个字符串。其长度等于两个运算数长度之和。如果两个运算数中有一个是 NULL，则 NULL 等价为空串。

兼容以下规则：

1） MySQL 规则：两个运算数中有一个是 NULL 时，返回 NULL。

2） PG 规则：两个运算数中有一个是 NULL 时，返回 NULL。

例如：

```
select '武汉' || ADDRESS1 from PERSON.ADDRESS WHERE ADDRESSID=3;
```

查询结果为：武汉青山区青翠苑 1 号

1.5.3 时间值表达式

时间值表达式的结果为时间值类型，包括日期(DATE)类型，时间(TIME)类型和时间戳(TIMESTAMP)间隔类型。DM SQL 不是对于任何的日期时间和间隔运算数的组合都可以计算。如果任何一个运算数是 NULL，运算结果也是 NULL。下面列出了有效的可能性和结果的数据类型。

1. 日期+间隔，日期-间隔和间隔+日期，得到日期

日期表达式的计算是根据有效格里高利历日期的规则。如果结果是一个无效的日期，表达式将出错。参与运算的间隔类型只能是 INTERVAL YEAR、INTERVAL MONTH、INTERVAL YEAR TO MONTH、INTERVAL DAY。

如果间隔运算数是年-月间隔，则没有从运算数的 DAY 字段的进位。

例 对日期值进行+、-年间隔运算

```
select PUBLISHTIME + INTERVAL '1' YEAR, PUBLISHTIME - INTERVAL '1' YEAR from
PRODUCTION.PRODUCT where PRODUCTID=1;
```

查询结果为：2006-04-01 2004-04-01

2. 时间+间隔，时间-间隔和间隔+时间，得到时间

时间表达式的计算是根据有效格里高利历日期的规则。如果结果是一个无效的时间，表达式将出错。参与运算的间隔类型只能是 INTERVAL DAY、INTERVAL HOUR、INTERVAL MINUTE、INTERVAL SECOND、INTERVAL DAY TO HOUR、INTERVAL DAY TO MINUTE、INTERVAL DAY TO SECOND、INTERVAL HOUR TO MINUTE、INTERVAL HOUR TO SECOND、INTERVAL MINUTE TO SECOND。

当结果的小时值大于等于 24 时，时间表达式是对 24 模的计算。

例 对时间值进行+、-小时间隔运算

```
DROP TABLE IF EXISTS test;
CREATE TABLE test(c1 TIME);
INSERT INTO test VALUES(TIME '19:00:00');
SELECT c1 + INTERVAL '9' HOUR, c1 - INTERVAL '9' HOUR FROM test;
```

查询结果为：04:00:00 10:00:00

3. 时间戳记+间隔，时间戳记-间隔和间隔+时间戳记，得到时间戳记

时间戳记表达式的计算是根据有效格里高利历日期的规则。如果结果是一个无效的时间

戳记，表达式将出错。参与运算的间隔类型只能是 INTERVAL YEAR、INTERVAL MONTH、INTERVAL YEAR TO MONTH、INTERVAL DAY、INTERVAL HOUR、INTERVAL MINUTE、INTERVAL SECOND、INTERVAL DAY TO HOUR、INTERVAL DAY TO MINUTE、INTERVAL DAY TO SECOND、INTERVAL HOUR TO MINUTE、INTERVAL HOUR TO SECOND、INTERVAL MINUTE TO SECOND。

与时间的计算不同，当结果的小时值大于等于 24 时，结果进位到天。

例 对时间戳值进行+、-小时间隔运算

```
DROP TABLE IF EXISTS test;
CREATE TABLE test(c1 TIMESTAMP);
INSERT INTO test VALUES(TIMESTAMP'2007-07-15 19:00:00');
SELECT c1 + INTERVAL '9' HOUR, c1 - INTERVAL '9' HOUR FROM test;
```

查询结果为：2007-07-16 04:00:00.000000　2007-07-15 10:00:00.000000

注：在含有 SECOND 值的运算数之间的一个运算的结果具有等于运算数的小数秒精度的小数秒精度。

4. 日期+数值，日期-数值和数值+日期，得到日期

日期与数值的运算，等价于日期与一个 INTERVAL '数值' DAY 的时间间隔的运算。

例 1　未设置+或-的数值时返回操作发生的日期

```
SELECT CURDATE(); //假设该查询操作发生在 2011 年 9 月 29 日
```

查询结果为：2011-09-29

例 2　返回当前日期+数值后的结果

```
SELECT CURDATE() + 2;
```

查询结果为：2011-10-01

例 3 返回当前日期-数值的结果

```
SELECT CURDATE() - 100;
```

查询结果为：2011-06-21

5. 时间戳记+数值，时间戳记-数值和数值+时间戳记，得到时间戳记

时间戳记与数值的运算，将数值看作以天为单位，转化为一个 INTERVAL DAY TO SECOND(6)的时间间隔，然后进行运算。

例 时间戳加上 2.358 天。

```
SELECT TIMESTAMP '2003-09-29 08:59:59.123' + 2.358;
```

查询结果为：2003-10-01 17:35:30.323000

6. 时间戳支持使用 AT TIME ZONE 语法进行时区转换，得到带时区的时间戳记

对于时间戳数据或返回值为时间戳的表达式，可以使用 AT TIME ZONE <转换表达式>进行时区转换。

<转换表达式>可以设置为：

- 时区偏移量：格式为'(+|-)HH:MM'，指定一个时区作为标准时区（UTC）的偏移量。例如'+8：00'表示比 UTC 早 8 个小时的时区。
- DBTIMEZONE：DM 在创建时设置（指定或默认）的数据库所用时区，DM 默认使用的是‘+8:00’时区。
- SESSIONTIMEZONE：当前会话设置的时区。默认为数据库时区，设置了则为在最近的 ALTER SESSION 语句中建立的会话时区。
- 时区地区名：指定一个时区地区名称，数据库会返回对应时区地区名的时区的值。例如，可以指定为'Asia/Hong_Kong'。
- 表达式：若为一个返回具有有效时区格式的字符串，则数据库返回对应时区的值；若不为有效时区格式，则数据库报错。

例 1 指定时区偏移量为'-3:00'。

```
SELECT TIMESTAMP '2023-06-01 18:05:23.234 +08:00' AT TIME ZONE '-3:00' FROM DUAL;
```

查询结果为：2023-06-01 07:05:23.234000 -03:00

例 2 指定时区地区名为'America/Los_Angeles'。

```
SELECT FROM_TZ(CAST('2024-06-20 05:32:29' AS TIMESTAMP), TZ_OFFSET('Asia/Shanghai'))
AT TIME ZONE 'America/Los_Angeles' AS LA FROM DUAL;
```

查询结果为：2024-06-19 14:32:29.000000 -07:00

1.5.4 时间间隔值表达式

1. 日期-日期，得到间隔

由于得到的结果可能会是“年-月-日”间隔，而这是不支持的间隔类型，故要对结果强制使用语法：

(日期表达式-日期表达式)<时间间隔限定符>

结果由<时间间隔限定符>中最不重要的日期字段决定。

例 日期值-日期值，得到间隔

```
SELECT (PUBLISHTIME-DATE'1990-01-01')YEAR TO MONTH FROM PRODUCTION.PRODUCT WHERE
PRODUCTID=1;
```

查询结果为：INTERVAL '15-3' YEAR(9) TO MONTH

2. 时间-时间，得到间隔

要对结果强制使用语法：

(时间表达式-时间表达式)<时间间隔限定符>

结果由<时间间隔限定符>中最不重要的时间字段决定。

例 时间值-时间值，得到间隔

```
SELECT (TIME'19:00:00'-TIME'10:00:00') HOUR;
```

查询结果为：INTERVAL '9' HOUR(9)

3. 时间戳记-时间戳记，得到间隔

要对结果强制使用语法：

(时间戳记表达式-时间戳记表达式)<时间间隔限定符>

结果由<时间间隔限定符>中最不重要的日期时间字段决定。

例 时间戳值-时间戳值，得到间隔

```
SELECT (TIMESTAMP '2007-07-15 19:00:00' - TIMESTAMP '2007-01-15 19:00:00') HOUR;
```

查询结果为：INTERVAL '4344' HOUR(9)

4. 年月间隔+年月间隔和年月间隔-年月间隔，得到年月间隔

参加运算的两个间隔必须有相同的数据类型，若得出无效的间隔的表达式将出错。结果的子类型包括运算数子类型所有的域，关于结果的引导精度规定如下：

如果二者的子类型相同，则为二者引导精度的最大值；如果二者的子类型不同，则为与结果类型首字段相同的那个运算数的引导精度。

例 对年月间隔进行+、-运算，得到年月间隔

```
SELECT INTERVAL'2007-07'YEAR(4) TO MONTH + INTERVAL'7'MONTH, INTERVAL'2007-07'YEAR(4)
TO MONTH - INTERVAL'7'MONTH;
```

查询结果为：INTERVAL '2008-2' YEAR(9) TO MONTH

INTERVAL '2007-0' YEAR(9) TO MONTH

5. 日时间隔+日时间隔和日时间隔-日时间隔，得到日时间隔

参加运算的两个间隔必须有相同的数据类型，若得出无效的间隔表达式将出错。结果的子类型包含运算数子类型所有的域，结果的小数秒精度为两运算数的小数秒精度的最大值，关于结果的引导精度规定如下：

如果二者的子类型相同，则为二者引导精度的最大值；如果二者的子类型不同，则为与结果类型首字段相同的那个运算数的引导精度。

例 对日时间隔进行+、-运算，得到日时间隔

```
SELECT INTERVAL'7 15'DAY TO HOUR + INTERVAL'10:10'MINUTE TO SECOND, INTERVAL'7 15'DAY TO
HOUR - INTERVAL'10:10'MINUTE TO SECOND;
```

查询结果为：INTERVAL '7 15:10:10.000000' DAY(9) TO SECOND(6)

INTERVAL '7 14:49:50.000000' DAY(9) TO SECOND(6)

6. 间隔*数字，间隔/数字和数字*间隔，得到间隔

若得出无效的间隔表达式将出错。结果的子类型、小数秒精度、引导精度和原间隔相同。

例 对间隔进行*、/运算

```
SELECT INTERVAL'10:10'MINUTE TO SECOND * 3,INTERVAL'10:10'MINUTE TO SECOND / 3;
```

查询结果为：INTERVAL '30:30.000000' MINUTE(9) TO SECOND(6)

INTERVAL '3:23.333333' MINUTE(9) TO SECOND(6)

1.5.5 运算符的优先级

当一个复杂的表达式有多个运算符时，运算符优先性决定执行运算的先后次序。运算符有下面这些优先等级(从高到低排列)。在较低等级的运算符之前先对较高等级的运算符进行求值。

```
( )
+(一元正)、-(一元负)、~(一元按位非)
*(乘)、/(除)、%(取余数)
+(加)、-(减)
|| (串联)
<<(左移)、>>(右移)
&(按位与)
^(按位异)
|(按位或)
```

1.6 DM_SQL 语言支持的数据库模式

DM_SQL 语言支持关系数据库的三级模式，外模式对应于视图和部分基表，模式对应于基表，基表是独立存在的表。一个或若干个基表存放于一个存贮文件中，存贮文件中的逻辑结构组成了关系数据库的内模式。DM_SQL 语言本身不提供对内模式的操纵语句。

视图是从基表或其它视图上导出的表，DM 只将视图的定义保存在数据字典中。该定义实际为一查询语句，再为该查询语句取一名字即为视图名。每次调用该视图时，实际上是执行其对应的查询语句，导出的查询结果即为该视图的数据。所以视图并无自己的数据，它是一个虚表，其数据仍存放在导出该视图的基表之中。当基表中的数据改变时，视图中查询的数据也随之改变，因此，视图象一个窗口，用户透过它可看到自己权限内的数据。视图一旦定义也可以为多个用户所共享，对视图作类似于基表的一些操作就像对基表一样方便。

综上所述，SQL 语言对关系数据库三级模式的支持如图 1.6.1 所示。

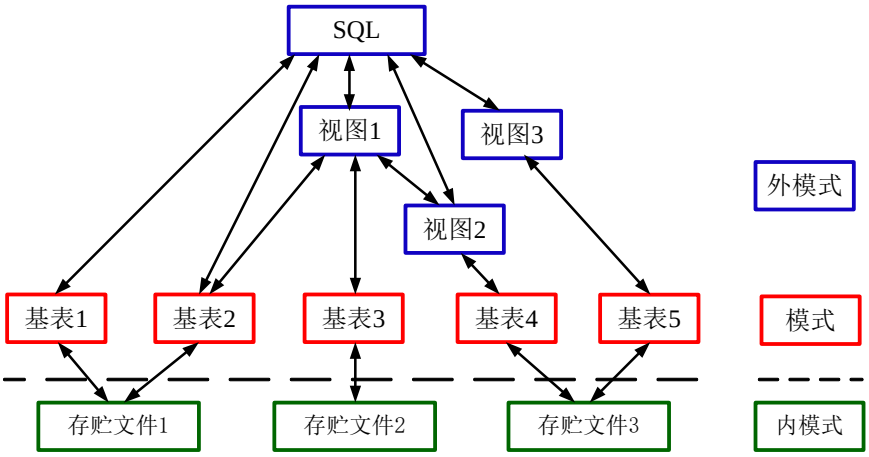


图 1.6.1 关系数据库的三级模式

第 2 章 手册中的示例说明

为方便读者阅读并尽快学会使用 DM 系统，本手册在介绍利用 DM 建立、维护数据库以及对数据库进行的各种操作中，使用了示例库 BOOKSHOP。本章将对该示例库进行说明。

2.1 示例库说明

示例库 BOOKSHOP 模拟武汉代理图书的某销售公司，该公司欲建立在线购物平台来拓展其代理产品的销售渠道，该在线购物平台支持网上产品信息浏览、订购等服务(仅限同城内销售及送货)。该销售公司的雇员、部门信息、业务方案等是所有实例的基础。

安装该示例后，将在数据库中创建 BOOKSHOP 表空间，同时创建 RESOURCES、PERSON、PRODUCTION、PURCHASING、SALES、OTHER 这 6 个模式和相关的表。

示例库 BOOKSHOP 所包含的模式、表如下:

表 2.1.1 BOOKSHOP

模式	包含相关对象	包含的表
RESOURCES	公司基本信息表	EMPLOYEE EMPLOYEE_ADDRESS DEPARTMENT EMPLOYEE_DEPARTMENT
PERSON	各个客户、雇员、供应商的名称和地址	ADDRESS ADDRESS_TYPE PERSON PERSON_TYPE
PRODUCTION	产品销售的信息	PRODUCT

		PRODUCT_CATEGORY PRODUCT_SUBCATEGORY PRODUCT_INVENTORY LOCATION PRODUCT_REVIEW PRODUCT_VENDOR
PURCHASING	供应商列表(采购订单等)	PURCHASEORDER_HEADER PURCHASEORDER_DETIAL VENDOR VENDOR_ADDRESS VENDOR_PERSON
SALES	与客户销售相关的数据(销售定单等)	CUSTMER CUSTMER_ADDRESS SALESORDER_HEADER SALESORDER_DETAIL SALESPERSON
OTHER	用到的其他表	DEPARTMENT EMPSALARY ACCOUNT ACTIONS READER READERAUDIT DEPTTAB

		EMPTAB SALGRADE COMPANYHOLIDAYS

EMPLOYEE

该公司的雇员信息表，包含在 RESOURCES 模式中。

表 2.1.2 EMPLOYEE

列	数据类型	是否为空	说明
EMPLOYEEID	INT	非空	主键，自增列
NATIONALNO	VARCHAR(18)	非空	身份证号码
PERSONID	INT	非空	指向 PERSON.PERSONID 的外键
LOGINID	VARCHAR(256)	非空	用户登录 ID
TITLE	VARCHAR(50)	非空	职位
MANAGERID	INT	空	直接上司 ID，指 EMPLOYEE.EMPLOYEEID 的外键
BIRTHDATE	DATE	非空	出生日期
MARITALSTATUS	CHAR(1)	非空	S=未婚 M=已婚
PHOTO	IMAGE	空	照片
HAIRDATE	DATE	非空	入职时间
SALARY	DEC(19,4)	非空	薪资（元）

EMPLOYEE_ADDRESS

将雇员映射到 ADDRESS 表中的地址信息，包含在 RESOURCES 模式中。

表 2.1.3 EMPLOYEE_ADDRESS

列	数据类型	是否为空	说明
EMPLOYEEID	INT	非空	指向 EMPLOYEE.EMPLOYEEID 的外键
ADDRESSID	INT	非空	指向 ADDRESS.ADDRESSID 的外键

EMPLOYEE _DEPARTMENT

将雇员映射到 DEPARTMENT 表中的部门信息，包含在 RESOURCES 模式中。

表 2.1.4 EMPLOYEE _DEPARTMENT

列	数据类型	是否为空	说明
EMPLOYEEID	INT	非空	指向 EMPLOYEE.EMPLOYEEID 的外键
DEPARTMENTID	INT	非空	员工所在部门，指向 DEPARTMENT. DEPARTMENTID 的外键
STARTDATE	DATE	非空	在该部门开始工作的日期
ENDDATE	DATE	空	离开该部门的日期 空=雇员在当前部门

DEPARTMENT

该公司的部门信息，包含在 RESOURCES 模式中。

表 2.1.5 DEPARTMENT

列	数据类型	是否为空	说明
DEPARTENTID	INT	非空	主键， 自增列
NAME	VARCHAR(50)	非空	部门名称

PERSON

该公司所有雇员、供应商、客户的姓名信息表，包含在 PERSON 模式中。

表 2.1.6 PERSON

列	数据类型	是否为空	说明

PERSONID	INT	非空	主键， 自增列，聚集索引（也叫聚簇索引）
NAME	VARCHAR(50)	非空	姓名
SEX	CHAR(1)	非空	M=男 F=女
EMAIL	VARCHAR(50)	空	电子邮件地址
PHONE	VARCHAR(25)	空	电话

PERSON_TYPE

存储 PERSON 表中的联系人类型，客户中的联系人类型有采购经理、采购代表，供应商的联系人类型有销售经理、销售代表。包含在 PERSON 模式中。

表 2.1.7 PERSON_TYPE

列	数据类型	是否为空	说明
PERSON_TYPEID	INT	非空	主键， 自增列
NAME	VARCHAR(50)	非空	联系人类型说明

ADDRESS

雇员、客户、供应商的地址信息，包含在 PERSON 模式中。

表 2.1.8 ADDRESS

列	数据类型	是否为空	说明
ADDRESSID	INT	非空	主键， 自增列
ADDRESS1	VARCHAR(60)	非空	第一通讯地址
ADDRESS2	VARCHAR(60)	空	第二通讯地址
CITY	VARCHAR(30)	非空	市/县名称
POSTALCODE	VARCHAR(15)	非空	邮政编码

ADDRESS_TYPE

为雇员、客户、供应商定义地址类型的表，包含在 PERSON 模式中。

表 2.1.9 ADDRESS_TYPE

列	数据类型	是否为空	说明
ADDRESS_TYPEID	INT	非空	主键，自增列
NAME	VARCHAR(50)	非空	地址类型的说明如开票地址、家庭地址、送货地址、公司地址等

CUSTOMER

当前客户信息，包含在 SALES 模式中。

表 2.1.10 CUSTOMER

列	数据类型	是否为空	说明
CUSTOMERID	INT	非空	主键， 自增列
PERSONID	INT	非空	指向 PERSON.PERSONID 的外键

CUSTOMER_ADDRESS

将客户映射到某个地址或多个地址，包含在 SALES 模式中。

表 2.1.11 CUSTOMER_ADDRESS

列	数据类型	是否为空	说明
CUSTOMERID	INT	非空	主键，指向 CUSTOMER.CUSTOMERID 的外键
ADDRESSID	INT	非空	主键，指向 ADDRESS.ADDRESSID 的外键
ADDRESS_TYPEID	INT	非空	地址类型，指向 ADDRESS_TYPE.ADDRESS_TYPEID 的外键

SALESPERSON

销售代表的销售统计信息，包含在 SALES 模式中。

表 2.1.12 SALESPERSON

列	数据类型	是否为空	说明
SALESPERSONID	INT	非空	主键， 自增列
EMPLOYEEID	INT	非空	该销售代表对应的雇员号，指向 EMPLOYEE.EMPLOYEEID 的外键
SALESTHISYEAR	DEC(19,4)	非空	今年到目前为止销售总额(万)
SALESLASTYEAR	DEC(19,4)	非空	去年销售总额(万)

VENDOR

所有供应商公司信息，包含在 PURCHASING 模式中。

表 2.1.13 VENDOR

列	数据类型	是否为空	说明
VENDORID	INT	非空	主键， 自增列
ACCOUNTNO	VARCHAR(15)	非空	供应商账户号
NAME	VARCHAR(50)	非空	供应商名称
ACTIVEFLAG	BIT	非空	0=不再使用供应商提供的产品 1=正在使用供应商提供的产品 CHECK 约束
WEBURL	VARCHAR(1024)	空	供应商服务 URL
CREDIT	INT	非空	1=高级 2=很好 3=较好 4=一般 5=较差

			CHECK 约束

VENDOR_ADDRESS

所有供应商地址信息，包含在 PURCHASING 模式中。

表 2.1.14 VENDOR_ADDRESS

列	数据类型	是否为空	说明
VENDORID	INT	非空	主键， 指向 VENDOR.VENDORID 的外键
ADDRESSID	INT	非空	主键，指向 ADDRESS.ADDRESSID 外键
ADDRESS_TYPEID	INT	非空	地址类型，指向 ADDRESS_TYPE.ADDRESS_TYPEID 外键

VENDOR_PERSON

供应商联系人信息表。

表 2.1.15 VENDOR_PERSON

列	数据类型	是否为空	说明
VENDORID	INT	非空	主键，指向 VENDOR.VENDORID 外键
PERSONID	INT	非空	主键，指向 PERSON.PERSONID 外键
PERSON_TYPEID	INT	非空	联系人的类型，指向 PERSON_TYPE.PERSON_TYPEID 的外键

PRODUCT

该公司售出的所有产品(图书)，包含在 PRODUCTION 模式中。

表 2.1.16 PRODUCT

列	数据类型	是否为空	说明
PRODUCTID	INT	非空	主键，自增列

NAME	VARCHAR(50)	非空	产品名称
AUTHOR	VARCHAR(25)	非空	作者
PUBLISHER	VARCHAR(50)	非空	出版社
PUBLISHTIME	DATE	非空	出版时间
PRODUCT_SUBCATEGORYID	INT	非空	产品所属子类别，PRODUCT_SUBCATEGORY .PRODUCT_SUBCATEGORYID 的外键
PRODUCTNO	VARCHAR(25)	非空	唯一产品标识号， unique 约束
DESCRIPTION	TEXT	空	产品的简介
PHOTO	IMAGE	空	产品的照片
SATETYSTOCKLEVEL	SMALLINT	非空	最小库存量
ORIGINALPRICE	DEC(19,4)	非空	原价(初始价格)
NOWPRICE	DEC(19,4)	非空	当前销售价格
DISCOUNT	DECIMAL(2,1)	非空	折扣
TYPE	VARCHAR(5)	空	产品开本规格，如 16 开
PAPERTOTAL	INT	空	总页数
WORDTOTAL	INT	空	总字数
SELLSTARTTIME	DATE	非空	开始销售日期
SELLENDTIME	DATE	空	停止销售的日期

PRODUCT_CATEGORY

产品分类表，包含在 PRODUCTION 模式中。

表 2.1.17 PRODUCT_CATEGORY

列	数据类型	是否为空	说明

PRODUCT_CATEGORYID	INT	非空	产品类别 ID，主键，自增列
NAME	VARCHAR(50)	非空	产品类别名称

PRODUCT_SUBCATEGORY

产品子分类表，包含在 PRODUCTION 模式中。

表 2.1.18 PRODUCT_SUBCATEGORY

列	数据类型	是否为空	说明
PRODUCT_SUBCATEGORYID	INT	非空	产品子类别 ID，主键,自增列
PRODUCT_CATEGORYID	INT	非空	产品类别 ID
NAME	VARCHAR(50)	非空	产品类别名称

PRODUCT_INVENTORY

产品的库存信息，包含在 PRODUCTION 模式中。

表 2.1.19 PRODUCT_INVENTORY

列	数据类型	是否为空	说明
PRODUCTID	INT	非空	产品标识，指向 PRODUCT.PRODUCTID 的外键
LOCATIONID	INT	非空	库存位置标识，指向 LOCATION.LOCATIONID 的外键
QUANTITY	INT	非空	库存位置的产品数量

PRODUCT_VENDOR

产品分类表，包含在 PRODUCTION 模式中。

表 2.1.20 PRODUCT_VENDOR

列	数据类型	是否为空	说明
PRODUCTID	INT	非空	主键，指向 PRODUCT.PRODUCTID 外键
VENDORID	INT	非空	主键，指向 VENDOR.VENDORID 外键
STANDARDPRICE	DEC(19,4)	非空	通常价格

LASTPRICE	DEC(19,4)	空	上次采购价格
LASTDATE	DATE	空	上次收到供应商产品的日期
MINQTY	INT	非空	应订购的最小定购数量
MAXQTY	INT	非空	应订购的最大定购数量
ONORDERQTY	INT	空	当前定购的数量

PRODUCT_REVIEW

已售产品的评论表，包含在 PRODCUTION 模式中。

表 2.1.21 PRODUCT_REVIEW

列	数据类型	是否为空	说明
PRODUCT_REVIEWID	INT	非空	主键，自增列
PRODUCTID	INT	非空	指向 PRODUCT.PRODUCTID 外键
NAME	VARCHAR(50)	非空	评论人姓名
REVIEWDATE	DATE	非空	提交评论的日期
EMAIL	VARCHAR(50)	非空	评论人的 EMAIL 地址
RATING	INT	非空	评论人给出的产品等级 范围 1-5，5 为最高级，CHECK 约束
COMMENTS	TEXT	空	评论人的注释

LOCATION

产品的库存地址信息，包含在 PRODUCTION 模式中。

表 2.1.22 LOCATION

列	数据类型	是否为空	说明
LOCATIONID	INT	非空	主键，自增列

NAME	VARCHAR(50)	非空	地点说明
PRODUCT_SUBCATEGORYID	INT	非空	指向 PRODUCT_SUBCATEGORY(PRODUCT_SUBCATEGORYID)的外键

SALESORDER_HEADER

销售订单常规信息表，包含在 SALES 模式中。

表 2.1.23 SALESORDER_HEADER

列	数据类型	是否为空	说明
SALESORDERID	INT	非空	订单 ID，主键，自增列
ORDERDATE	DATE	非空	创建订单的日期
DUEDATE	DATE	非空	客户订单到期的日期
STATUS	TINYINT	非空	订单状态， CHECK 约束 1=待用户确认 2=待发货 3=已发货 4=已完成 5=意外终结
ONLINEORDERFLAG	BIT	非空	0=销售人员下的订单(包括电话订单) 1=在线订单
CUSTOMERID	INT	非空	客户标识号，指向 CUSTOMER.CUSTOMERID 的外键
SALESPERSONID	INT	非空	销售代表 ID，指向 SALESPERSON.SALESPERSONID 的外键
ADDRESSID	INT	非空	客户收货地址，指向 ADDRESS.ADDRESSID 外键

SHIPMETHOD	BIT	非空	发货方法 0=免费送货(仅限中心城区) 1=有偿送货(>=400 元，免费送货；<400，运费 10 元)
SUBTOTAL	DEC(19,4)	非空	销售小计 计算方式:SUM(SALESORDER_DETAIL.LINETOTAL)
FREIGHT	DEC(19,4)	非空	运费(非中城区，subtotal>=400，freight=0；<400， freight=10；中心城区， freight=0)
TOTAL	DEC(19,4)	非空	应付款总计 SUBTOTAL+FREIGHT
COMMENTS	TEXT	空	销售代表备注说明

SALESORDER_DETAIL

与特定销售订单关联的各个产品信息表，包含在 SALES 模式中。

表 2.1.24 SALESORDER_DETAIL

列	数据类型	是否为空	说明
SALESORDERID	INT	非空	主键, 指向 SALESORDER_HEADER.SALESORDERID 的外键
SALESORDER_DETAILID	INT	非空	主键，确保数据唯一性的连续编号
CARRIERNO	VARCHAR(25)	非空	发货跟踪号
PRODUCTID	INT	非空	定购产品的 ID，指向 PRODUCT.PRODUCTID 的外键
ORDERQTY	INT	非空	每件产品定购数量
LINETOTAL	DEC(19,4)	非空	产品的销售小计

PURCHASEORDER_HEADER

采购订单常规信息，包含在 PURCHASING 模式中。

表 2.1.25 PURCHASEORDER_HEADER

列	数据类型	是否为空	说明
PURCHASEORDERID	INT	非空	订单 ID，主键，自增列
ORDERDATE	DATE	非空	创建订单的日期
STATUS	TINYINT	非空	订单状态， CHECK 约束 0=等待批准 1=已批准 2=已拒绝 3=已完成
EMPLOYEEID	INT	非空	创建采购订单的雇员 ID，指向 EMPLOYEE.EMPLOYEEID 的外键
VENDORID	INT	非空	所采购产品的供应商 ID，指向 VENDOR.VENDORID 的外键
SHIPMETHOD	VARCHAR(50)	非空	发货方法
SUBTOTAL	DEC(19,4)	非空	产品价格小计
TAX	DEC(19,4)	非空	税额
FREIGHT	DEC(19,4)	非空	运费
TOTAL	DEC(19,4)	非空	应向供应商付款总计(SUBTOTAL+ TAX+ FREIGHT)

PURCHASEORDER_DETAIL

与特定采购订单相关联的每个产品的采购信息，包含在 PURCHASING 模式中。

表 2.1.26 PURCHASEORDER_DETAIL

列	数据类型	是否	说明

		为空	
PURCHASEORDERID	INT	非空	主键，指向 PURCHASEORDER_HEADER .PURCHASEORDERID 的外键
PURCHASEORDER_DETAILID	INT	非空	主键，确保数据唯一性的连续编号
DUEDATE	DATE	非空	希望从供应商收到产品的日期
PRODUCTID	INT	非空	定购产品的 ID，指向 PRODUCT.PRODUCTID 的外键
ORDERQTY	INT	非空	定购数量
PRICE	DEC(19,4)	非空	单件产品的价格
SUBTOTAL	DEC(19,4)	非空	产品价格小计(RPICE*ORDERQTY)
RECEIVEDQTY	DECIMAL(8,2)	非空	实际从供应商收到的数量
REJECTEDQTY	DECIMAL(8,2)	非空	检查时拒收的数量
STOCKEDQTY	DECIMAL(8,2)	非空	纳入库存的数量

2.2 示例库脚本

安装 DM 时，如果选择安装 BOOKSHOP 示例库，系统会自动安装一个名为 BOOKSHOP 的示例库。该示例库中，数据库名称为 DAMENG，实例名为 DMSERVER。

如果安装时没有选择安装示例库，可以通过如下 SQL 语句自行创建：

2.2.1 创建表空间

```
//创建示例库表空间
CREATE TABLESPACE BOOKSHOP DATAFILE 'BOOKSHOP.DBF' size 150;
```

2.2.2 创建模式及表

```
//创建模式

CREATE SCHEMA RESOURCES AUTHORIZATION SYSDBA;

/

CREATE SCHEMA PERSON AUTHORIZATION SYSDBA;

/

CREATE SCHEMA SALES AUTHORIZATION SYSDBA;

/

CREATE SCHEMA PRODUCTION AUTHORIZATION SYSDBA;

/

CREATE SCHEMA PURCHASING AUTHORIZATION SYSDBA;

/

CREATE SCHEMA OTHER AUTHORIZATION SYSDBA;

/



//创建表

//CREATE ADDRESS

CREATE TABLE PERSON.ADDRESS

(ADDRESSID INT IDENTITY(1,1) PRIMARY KEY,

ADDRESS1 VARCHAR(60) NOT NULL,

ADDRESS2 VARCHAR(60),

CITY VARCHAR(30) NOT NULL,

POSTALCODE VARCHAR(15) NOT NULL) STORAGE (on BOOKSHOP);
```

```
//CREATE ADDRESS_TYPE
CREATE TABLE PERSON.ADDRESS_TYPE
(ADDRESS_TYPEID INT IDENTITY(1,1) PRIMARY KEY,
NAME VARCHAR(50) NOT NULL) STORAGE (ON BOOKSHOP);

//CREATE PERSON
CREATE TABLE PERSON.PERSON
(
PERSONID   INT IDENTITY(1,1) CLUSTER PRIMARY KEY,
SEX CHAR(1) NOT NULL,
NAME VARCHAR(50) NOT NULL,
EMAIL VARCHAR(50),
PHONE VARCHAR(25)) STORAGE (ON BOOKSHOP);

//CREATE PERSON_TYPE
CREATE TABLE PERSON.PERSON_TYPE
(PERSON_TYPEID INT IDENTITY(1,1) PRIMARY KEY,
NAME VARCHAR(256) NOT NULL) STORAGE (ON BOOKSHOP);

//CREATE DEPARTMENT
CREATE TABLE RESOURCES.DEPARTMENT(DEPARTMENTID INT IDENTITY(1,1) PRIMARY KEY,NAME
VARCHAR(50) NOT NULL) STORAGE (ON BOOKSHOP);
```

```
//CREATE EMPLOYEE
CREATE TABLE RESOURCES.EMPLOYEE(
        EMPLOYEEID INT IDENTITY(1,1) PRIMARY KEY ,
        NATIONALNO VARCHAR(18) NOT NULL,
        PERSONID INT NOT NULL REFERENCES PERSON.PERSON(PERSONID),
        LOGINID VARCHAR(256) NOT NULL,
        TITLE VARCHAR(50) NOT NULL,
        MANAGERID INT,
        BIRTHDATE DATE NOT NULL,
        MARITALSTATUS CHAR(1) NOT NULL,
        PHOTO IMAGE,
        HAIRDATE DATE NOT NULL,
        SALARY DEC(19,4) NOT NULL
) STORAGE (ON BOOKSHOP);

//CREATE EMPLOYEE_ADDRESS
CREATE TABLE RESOURCES.EMPLOYEE_ADDRESS
(ADDRESSID INT NOT NULL REFERENCES PERSON.ADDRESS(ADDRESSID),
EMPLOYEEID INT NOT NULL REFERENCES RESOURCES.EMPLOYEE(EMPLOYEEID)) STORAGE (ON
BOOKSHOP);

//CREATE EMPLOYEE_DEPARTMENT
CREATE TABLE RESOURCES.EMPLOYEE_DEPARTMENT
(EMPLOYEEID INT NOT NULL REFERENCES RESOURCES.EMPLOYEE(EMPLOYEEID),
```

```
                DEPARTMENTID INT NOT NULL REFERENCES RESOURCES.DEPARTMENT(DEPARTMENTID),
                STARTDATE DATE NOT NULL,
                ENDDATE DATE) STORAGE (ON BOOKSHOP);


//CREATE CUSTOMER
CREATE TABLE SALES.CUSTOMER(CUSTOMERID INT IDENTITY(1,1) PRIMARY KEY ,PERSONID INT NOT
NULL REFERENCES PERSON.PERSON(PERSONID)) STORAGE (ON BOOKSHOP);


//CREATE CUSTOMER_ADDRESS
CREATE TABLE SALES.CUSTOMER_ADDRESS
(CUSTOMERID INT REFERENCES SALES.CUSTOMER(CUSTOMERID),
ADDRESSID INT REFERENCES PERSON.ADDRESS(ADDRESSID),
ADDRESS_TYPEID INT NOT NULL REFERENCES PERSON.ADDRESS_TYPE(ADDRESS_TYPEID),
PRIMARY KEY (CUSTOMERID,ADDRESSID)) STORAGE (ON BOOKSHOP);


//CREATE PRODUCT_CATEGORY
CREATE TABLE PRODUCTION.PRODUCT_CATEGORY
(PRODUCT_CATEGORYID INT IDENTITY(1,1) PRIMARY KEY,
 NAME VARCHAR(50) NOT NULL) STORAGE (ON BOOKSHOP);


//CREATE   PRODUCT_SUBCATEGORY
CREATE TABLE PRODUCTION.PRODUCT_SUBCATEGORY
(PRODUCT_SUBCATEGORYID INT IDENTITY(1,1) PRIMARY KEY,
PRODUCT_CATEGORYID INT NOT NULL ,
```

```
NAME VARCHAR(50) NOT NULL) STORAGE (ON BOOKSHOP);



//CREATE PRODUCT

CREATE TABLE PRODUCTION.PRODUCT

(PRODUCTID INT IDENTITY(1,1) PRIMARY KEY,

 NAME VARCHAR(50) NOT NULL,

 AUTHOR VARCHAR(25) NOT NULL,

 PUBLISHER VARCHAR(50) NOT NULL,

 PUBLISHTIME DATE NOT NULL,

 PRODUCT_SUBCATEGORYID INT NOT NULL REFERENCES

PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_SUBCATEGORYID),

 PRODUCTNO VARCHAR(25) NOT NULL,

 SATETYSTOCKLEVEL SMALLINT NOT NULL,

 ORIGINALPRICE DEC(19,4) NOT NULL,

 NOWPRICE DEC(19,4) NOT NULL,

 DISCOUNT DECIMAL(2,1) NOT NULL,

 DESCRIPTION TEXT,

 PHOTO IMAGE,

 TYPE VARCHAR(5),

 PAPERTOTAL INT,

 WORDTOTAL INT,

 SELLSTARTTIME DATE NOT NULL,

 SELLENDTIME DATE,

 UNIQUE(PRODUCTNO)) STORAGE (ON BOOKSHOP);
```

```
//CREATE LOCATION
CREATE TABLE PRODUCTION.LOCATION
(LOCATIONID INT IDENTITY(1,1) PRIMARY KEY,
PRODUCT_SUBCATEGORYID INT NOT NULL REFERENCES
      PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_SUBCATEGORYID),
NAME VARCHAR(50) NOT NULL) STORAGE (ON BOOKSHOP);

//CREATE PRODUCT_INVENTORY
CREATE TABLE PRODUCTION.PRODUCT_INVENTORY
(PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
LOCATIONID INT NOT NULL REFERENCES PRODUCTION.LOCATION(LOCATIONID),
QUANTITY INT NOT NULL) STORAGE (ON BOOKSHOP);

//CREATE PRODUCT_REVIEW
CREATE TABLE PRODUCTION.PRODUCT_REVIEW
(PRODUCT_REVIEWID INT IDENTITY(1,1) PRIMARY KEY,
PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
NAME VARCHAR(50) NOT NULL,
REVIEWDATE DATE NOT NULL,
EMAIL VARCHAR(50) NOT NULL,
RATING INT NOT NULL CHECK(RATING IN(1,2,3,4,5)),
COMMENTS TEXT) STORAGE (ON BOOKSHOP);
```

```
//CREATE VENDOR

CREATE TABLE PURCHASING.VENDOR

(VENDORID INT IDENTITY(1,1) PRIMARY KEY,

ACCOUNTNO VARCHAR(15) NOT NULL,

NAME VARCHAR(50) NOT NULL,

ACTIVEFLAG BIT NOT NULL,

WEBURL VARCHAR(1024),

CREDIT INT NOT NULL CHECK(CREDIT IN(1,2,3,4,5))) STORAGE (ON BOOKSHOP);



//CREATE VENDOR_ADDRESS

CREATE TABLE PURCHASING.VENDOR_ADDRESS

(VENDORID INT NOT NULL REFERENCES PURCHASING.VENDOR(VENDORID),

ADDRESSID INT NOT NULL REFERENCES PERSON.ADDRESS(ADDRESSID),

ADDRESS_TYPEID INT NOT NULL REFERENCES PERSON.ADDRESS_TYPE(ADDRESS_TYPEID),

PRIMARY KEY (VENDORID,ADDRESSID)) STORAGE (ON BOOKSHOP);



//CREATE VENDOR_PERSON

CREATE TABLE PURCHASING.VENDOR_PERSON

(VENDORID INT NOT NULL REFERENCES PURCHASING.VENDOR(VENDORID),

PERSONID INT NOT NULL REFERENCES PERSON.PERSON(PERSONID),

PERSON_TYPEID INT NOT NULL REFERENCES PERSON.PERSON_TYPE(PERSON_TYPEID),

PRIMARY KEY (VENDORID,PERSONID)) STORAGE (ON BOOKSHOP);



//CREATE PRODUCT_VENDOR
```

```
CREATE TABLE PRODUCTION.PRODUCT_VENDOR
(PRODUCTID INT REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
 VENDORID INT REFERENCES PURCHASING.VENDOR(VENDORID),
 STANDARDPRICE DEC(19,4) NOT NULL,
 LASTPRICE DEC(19,4),
 LASTDATE DATE,
 MINQTY INT NOT NULL,
 MAXQTY INT NOT NULL,
 ONORDERQTY INT,
 PRIMARY KEY(PRODUCTID,VENDORID)) STORAGE (ON BOOKSHOP);


//CREATE SALESPERSON
CREATE TABLE SALES.SALESPERSON
(SALESPERSONID INT IDENTITY(1,1) PRIMARY KEY,
EMPLOYEEID INT NOT NULL REFERENCES RESOURCES.EMPLOYEE(EMPLOYEEID),
SALESTHISYEAR DEC(19,4) NOT NULL,
SALESLASTYEAR DEC(19,4) NOT NULL) STORAGE (ON BOOKSHOP);


//CREATE PURCHASEORDER_HEADER
CREATE TABLE PURCHASING.PURCHASEORDER_HEADER
(PURCHASEORDERID INT IDENTITY(1,1) PRIMARY KEY,
ORDERDATE DATE NOT NULL,
STATUS TINYINT NOT NULL CHECK(STATUS IN(0,1,2,3)),
EMPLOYEEID INT NOT NULL REFERENCES RESOURCES.EMPLOYEE(EMPLOYEEID),
```

```
VENDORID INT NOT NULL REFERENCES PURCHASING.VENDOR(VENDORID),
SHIPMETHOD VARCHAR(50) NOT NULL,
SUBTOTAL DEC(19,4) NOT NULL,
TAX DEC(19,4) NOT NULL,
FREIGHT DEC(19,4) NOT NULL,
TOTAL DEC(19,4) NOT NULL) STORAGE (ON BOOKSHOP);

//CREATE PURCHASEORDER_DETAIL
CREATE TABLE PURCHASING.PURCHASEORDER_DETAIL
(PURCHASEORDERID INT NOT NULL REFERENCES
      PURCHASING.PURCHASEORDER_HEADER(PURCHASEORDERID),
PURCHASEORDER_DETAILID INT NOT NULL,
DUEDATE DATE NOT NULL,
PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
ORDERQTY INT NOT NULL,
PRICE DEC(19,4) NOT NULL,
SUBTOTAL DEC(19,4) NOT NULL,
RECEIVEDQTY INT NOT NULL,
REJECTEDQTY INT NOT NULL,
STOCKEDQTY INT NOT NULL,
PRIMARY KEY(PURCHASEORDERID,PURCHASEORDER_DETAILID)) STORAGE (ON BOOKSHOP);

//CREATE SALESORDER_HEADER
CREATE TABLE SALES.SALESORDER_HEADER
```

```
(SALESORDERID INT IDENTITY(1,1) PRIMARY KEY,
ORDERDATE DATE NOT NULL,
DUEDATE DATE NOT NULL,
STATUS TINYINT NOT NULL CHECK(STATUS IN(0,1,2,3,4,5)),
ONLINEORDERFLAG BIT NOT NULL,
CUSTOMERID INT NOT NULL REFERENCES SALES.CUSTOMER(CUSTOMERID),
SALESPERSONID INT NOT NULL REFERENCES SALES.SALESPERSON(SALESPERSONID),
ADDRESSID INT NOT NULL REFERENCES PERSON.ADDRESS(ADDRESSID),
SHIPMETHOD BIT NOT NULL,
SUBTOTAL DEC(19,4) NOT NULL,
FREIGHT DEC(19,4) NOT NULL,
TOTAL DEC(19,4) NOT NULL,
COMMENTS TEXT) STORAGE (ON BOOKSHOP);

//CREATE SALESORDER_DETAIL
CREATE TABLE SALES.SALESORDER_DETAIL
(SALESORDERID INT NOT NULL REFERENCES    SALES.SALESORDER_HEADER(SALESORDERID),
SALESORDER_DETAILID INT NOT NULL,
CARRIERNO VARCHAR(25) NOT NULL,
PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
ORDERQTY INT NOT NULL,
LINETOTAL DEC(19,4) NOT NULL,
PRIMARY KEY(SALESORDERID,SALESORDER_DETAILID)) STORAGE (ON BOOKSHOP);
```

```
//CREATE OTHER.DEPARTMENT

CREATE TABLE OTHER.DEPARTMENT

(

HIGH_DEP VARCHAR(50),

DEP_NAME VARCHAR(50)) STORAGE (ON BOOKSHOP);



//CREATE OTHER.EMPSALARY

CREATE TABLE OTHER.EMPSALARY

(

ENAME CHAR(10),

EMPNO NUMERIC(4),

SAL NUMERIC(4)) STORAGE (ON BOOKSHOP);



//CREATE OTHER.ACCOUNT

CREATE TABLE OTHER.ACCOUNT

(

"ACCOUNT_ID" INTEGER NOT NULL,

"BAL" DEC(10,2),

PRIMARY KEY("ACCOUNT_ID"))STORAGE (ON BOOKSHOP);



//CREATE OTHER.ACTIONS

CREATE TABLE OTHER.ACTIONS

(

"ACCOUNT_ID" INTEGER NOT NULL,
```

```
"OPER_TYPE" CHAR(1),

"NEW_VALUE" DEC(10,2),

"STATUS" VARCHAR(50),

PRIMARY KEY("ACCOUNT_ID"))STORAGE (ON BOOKSHOP);


//CREATE OTHER.READER

CREATE TABLE OTHER.READER

(

READER_ID     INT PRIMARY KEY,

NAME              VARCHAR(30),

AGE               SMALLINT,

GENDER            CHAR,

MAJOR             VARCHAR(30)) STORAGE (ON BOOKSHOP);


//CREATE OTHER.READERAUDIT

CREATE TABLE OTHER.READERAUDIT

(

CHANGE_TYPE CHAR NOT NULL,

CHANGED_BY    VARCHAR(8)    NOT NULL,

OP_TIMESTAMP       DATE NOT NULL,

OLD_READER_ID     INT,

OLD_NAME        VARCHAR(30),

OLD_AGE          SMALLINT,

OLD_GENDER     CHAR,
```

```
OLD_MAJOR       VARCHAR(30),
NEW_READER_ID     INT,
NEW_NAME        VARCHAR(30),
NEW_AGE         SMALLINT,
NEW_GENDER      CHAR,
NEW_MAJOR       VARCHAR(30)) STORAGE (ON BOOKSHOP);

//CREATE OTHER.DEPTTAB
CREATE TABLE OTHER.DEPTTAB
(
DEPTNO    INT PRIMARY KEY,
DNAME    VARCHAR(15),
LOC    VARCHAR(25)) STORAGE (ON BOOKSHOP);

//CREATE OTHER.EMPTAB
CREATE TABLE OTHER.EMPTAB
(
EMPNO    INT PRIMARY KEY,
ENAME    VARCHAR(15) NOT NULL,
JOB    VARCHAR(10),
SAL    FLOAT,
DEPTNO    INT) STORAGE (on BOOKSHOP);

//CREATE OTHER.SALGRADE
```

```
CREATE TABLE OTHER.SALGRADE

(

LOSAL                FLOAT,

HISAL                FLOAT,

JOB_CLASSIFICATION VARCHAR(10)) STORAGE (ON BOOKSHOP);



//CREATE OTHER.COMPANYHOLIDAYS

CREATE TABLE OTHER.COMPANYHOLIDAYS

(

HOLIDAY   DATE) STORAGE (ON BOOKSHOP);
```

2.2.3 插入数据

```
//插入数据

//INSERT ADDRESS

INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区 369 号金地

太阳城 56-1-202','','武汉市洪山区','430073');

INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区 369 号金地

太阳城 57-2-302','','武汉市洪山区','430073');

INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('青山区青翠苑 1 号

','','武汉市青山区','430080');

INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('武昌区武船新村

115 号','','武汉市武昌区','430063');

INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('汉阳大道熊家湾 15

号','','武汉市汉阳区','430050');
```

```
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区保利花园
50-1-304','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区保利花园
51-1-702','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区关山春晓
51-1-702','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('江汉区发展大道
561 号','','武汉市江汉区','430023');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('江汉区发展大道
555 号','','武汉市江汉区','430023');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('武昌区武船新村 1
号','','武汉市武昌区','430063');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('江汉区发展大道
423 号','','武汉市江汉区','430023');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区关山春晓
55-1-202','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区关山春晓
10-1-202','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区关山春晓
11-1-202','','武汉市洪山区','430073');
INSERT INTO PERSON.ADDRESS(ADDRESS1,ADDRESS2,CITY,POSTALCODE) VALUES('洪山区光谷软件园
C1_501','','武汉市洪山区','430073');


//INSERT ADDRESS_TYPE
```

```
INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('发货地址');

INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('送货地址');

INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('家庭地址');

INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('公司地址');


//INSERT DEPARTMENT

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('采购部门');

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('销售部门');

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('人力资源');

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('行政部门');

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('广告部');


//INSERT PERSON

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','李丽

','lily@sina.com','02788548562');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('M','王刚','','02787584562');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('M','李勇','','02782585462');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','郭艳','','02787785462');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','孙丽','','13055173012');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('M','黄非','','13355173012');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','王菲','','13255173012');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('M','张平','','13455173012');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('M','张红','','13555173012');

INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','刘佳','','13955173012');
```

```
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','王南','','15955173012');
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','李飞','','15954173012');
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','张大海','','15955673012');
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','王宇轩','','15955175012');
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','桑泽恩','','15955173024');
INSERT INTO PERSON.PERSON(SEX,NAME,EMAIL,PHONE) VALUES('F','刘青','','15955173055');
INSERT INTO PERSON.PERSON(SEX,NAME,PHONE) VALUES('F','杨凤兰','02785584662');

//INSERT PERSON_TYPE
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('采购经理');
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('采购代表');
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('销售经理');
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('销售代表');

//INSERT CUSTOMER
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
WHERE NAME='刘青'));
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
WHERE NAME='桑泽恩'));
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
WHERE NAME='王宇轩'));
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
WHERE NAME='张大海'));
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
```

```
WHERE NAME='李飞'));
INSERT INTO SALES.CUSTOMER(PERSONID) VALUES((SELECT PERSONID FROM PERSON.PERSON
WHERE NAME='王南'));

//INSERT   CUSTOMER_ADDRESS
INSERT INTO SALES.CUSTOMER_ADDRESS(CUSTOMERID,ADDRESSID,ADDRESS_TYPEID)
SELECT CUSTOMERID,11,2 FROM SALES.CUSTOMER;
INSERT INTO SALES.CUSTOMER_ADDRESS(CUSTOMERID,ADDRESSID,ADDRESS_TYPEID)
SELECT CUSTOMERID,12,3 FROM SALES.CUSTOMER;

//INSERT EMPLOYEE
INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921197908051523',1,'L1','总经理','','1979-08-05','S','','2002-05-02',40000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198008051523',2,'L2','销售经理',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='总经理'),'1980-08-05','S','','2002-05-02', 26000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
```

```
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198408051523',3,'L3','采购经理',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='总经理'),'1981-08-05','S','','2002-05-02',23000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198208051523',4,'L4','销售代表',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='销售经理'),'1982-08-05','S','','2002-05-02',15000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198308051523',5,'L5','销售代表',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='销售经理'),'1983-08-05','S','','2002-05-02',16000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198408051523',6,'L6','采购代表',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='采购经理'),'1984-08-05','S','','2005-05-02',12000);

INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
```

```
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921197708051523',7,'L7','人力资源部经理',(SELECT EMPLOYEEID FROM
RESOURCES.EMPLOYEE WHERE TITLE='总经理'),'1977-08-05','M','','2002-05-02',25000);


INSERT INTO
RESOURCES.EMPLOYEE(NATIONALNO,PERSONID,LOGINID,TITLE,MANAGERID,BIRTHDATE,MARITALSTAT
US,PHOTO,HAIRDATE,SALARY)
VALUES('420921198008071523',8,'L8','系统管理员',(SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE TITLE='人力资源部经理'),'1980-08-07','S','','2004-05-02',20000);
//INSERT EMPLOYEE_DEPARTMENT
INSERT INTO
RESOURCES.EMPLOYEE_DEPARTMENT(EMPLOYEEID,DEPARTMENTID,STARTDATE,ENDDATE)
SELECT EMPLOYEEID,'2','2005-02-01',null FROM RESOURCES.EMPLOYEE WHERE
RESOURCES.EMPLOYEE.TITLE='销售代表' OR RESOURCES.EMPLOYEE.TITLE='销售经理';
INSERT INTO
RESOURCES.EMPLOYEE_DEPARTMENT(EMPLOYEEID,DEPARTMENTID,STARTDATE,ENDDATE)
SELECT EMPLOYEEID,'1','2005-02-01',null FROM RESOURCES.EMPLOYEE WHERE
RESOURCES.EMPLOYEE.TITLE='采购代表' OR RESOURCES.EMPLOYEE.TITLE='采购经理';
INSERT INTO
RESOURCES.EMPLOYEE_DEPARTMENT(EMPLOYEEID,DEPARTMENTID,STARTDATE,ENDDATE)
SELECT EMPLOYEEID,'3','2005-02-01',null FROM RESOURCES.EMPLOYEE WHERE
RESOURCES.EMPLOYEE.TITLE='系统管理员';
INSERT INTO
RESOURCES.EMPLOYEE_DEPARTMENT(EMPLOYEEID,DEPARTMENTID,STARTDATE,ENDDATE)
```

```
SELECT EMPLOYEEID,'4','2001-02-01',null FROM RESOURCES.EMPLOYEE WHERE

RESOURCES.EMPLOYEE.TITLE='总经理';


//INSERT EMPLOYEE_ADDRESS

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(1,1);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(2,2);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(3,3);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(4,4);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(5,5);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(6,6);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(7,7);

INSERT INTO RESOURCES.EMPLOYEE_ADDRESS(EMPLOYEEID,ADDRESSID) VALUES(8,8);


//INSERT SALES.SALESPERSON

INSERT INTO SALES.SALESPERSON(EMPLOYEEID,SALESTHISYEAR,SALESLASTYEAR)

SELECT EMPLOYEEID,8,10 FROM RESOURCES.EMPLOYEE WHERE TITLE='销售代表';


//INSERT VENDOR

INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)

VALUES('00','上海画报出版社','1','','2');

INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)

VALUES('00','长江文艺出版社','1','','2');

INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)

VALUES('00','北京十月文艺出版社','1','','1');
```

```
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','人民邮电出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','清华大学出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','中华书局','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','广州出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','上海出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','21 世纪出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','外语教学与研究出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','机械工业出版社','1','','1');
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT)
VALUES('00','文学出版社','1','','1');

//INSERT VENDOR_ADDRESS
INSERT INTO PURCHASING.VENDOR_ADDRESS(VENDORID,ADDRESSID,ADDRESS_TYPEID)
SELECT VENDORID,9,1 FROM PURCHASING.VENDOR;
INSERT INTO PURCHASING.VENDOR_ADDRESS(VENDORID,ADDRESSID,ADDRESS_TYPEID)
SELECT VENDORID,10,4 FROM PURCHASING.VENDOR;
```

```
//INSERT VENDOR_PERSON
INSERT INTO PURCHASING.VENDOR_PERSON(VENDORID,PERSONID,PERSON_TYPEID)
SELECT VENDORID,9,4 FROM PURCHASING.VENDOR;

//INSERT PRODUCT_CATEGORRY
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('小说');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('文学');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('计算机');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('英语');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('管理');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('少儿');
INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES('金融');

//INSERT PRODUCT_SUBCATEGORY
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'世界名著');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'武侠');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'科幻');
```

```
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'四大名著');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'军事');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='小说'),'社会');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME) VALUES(10,'
历史');

INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'文集');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'纪实文学');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'文学理论');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'中国古诗词');
```

```
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'中国现当代诗');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'戏剧');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='文学'),'民间文学');


INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'计算机理论');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'计算机体系结构');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'操作系统');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'程序设计');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
```

```
NAME='计算机'),'数据库');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'软件工程');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'信息安全');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='计算机'),'多媒体');


INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语词汇');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语语法');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语听力');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语口语');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语阅读');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='英语'),'英语写作');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'行政管理');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'项目管理');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'质量管理与控制');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'商业道德');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'经营管理');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='管理'),'财务管理');
```

```
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'幼儿启蒙');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'益智游戏');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'童话');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'卡通');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'励志');
INSERT INTO PRODUCTION.PRODUCT_SUBCATEGORY(PRODUCT_CATEGORYID,NAME)
VALUES((SELECT PRODUCT_CATEGORYID FROM PRODUCTION.PRODUCT_CATEGORY WHERE
NAME='少儿'),'少儿英语');


//INSERT PRODUCT
INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
```

```
VALUES('红楼梦','曹雪芹,高鹗','中华书局','2005-4-1','9787101046120',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='四大名
著'),'10','19','15.2','8.0','曹雪芹，是中国文学史上最伟大也是最复杂的作家，《红楼梦》也是中国文学史上最伟
大而又最复杂的作品。《红楼梦》写的是封建贵族的青年贾宝玉、林黛玉、薛宝钗之间的恋爱和婚姻悲剧，
而且以此为中心，写出了当时具有代表性的贾、王、史、薛四大家族的兴衰，其中又以贾府为中心，揭露
了封建社会后期的种种黑暗和罪恶，及其不可克服的内在矛盾，对腐朽的封建统治阶级和行将崩溃的封建
制度作了有力的批判，使读者预感到它必然要走向覆灭的命运。本书是一部具有高度思想性和高度艺术性
的伟大作品，从本书反映的思想倾向来看，作者具有初步的民主主义思想，他对现实社会包括宫廷及官场
的黑暗，封建贵族阶级及其家庭的腐朽，封建的科举制度、婚姻制度、奴婢制度、等级制度，以及与此相
适应的社会统治思想即孔孟之道和程朱理学、社会道德观念等等，都进行了深刻的批判并且提出了朦胧的
带有初步民主主义性质的理想和主张。这些理想和主张正是当时正在滋长的资本主义经济萌芽因素的曲折
反映。','','16','943','933000','2006-03-20','');


INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('水浒传','施耐庵，罗贯中','中华书局','2005-4-1','9787101046137',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='四大名
著'),'10','19','14.3','7.5','《水浒传》是宋江起义故事在民间长期流传基础上产生出来的，吸收了民间文学的营
养。《水浒传》是我国人民最喜爱的古典长篇白话小说之一。它产生于明代，是在宋、元以来有关水浒的故
事、话本、戏曲的基础上，由作者加工整理、创作而成的。全书以宋江领导的农民起义为主要题材，艺术
地再现了中国古代人民反抗压迫、英勇斗争的悲壮画卷。作品充分暴露了封建统治阶级的腐朽和残暴，揭
露了当时尖锐对立的社会矛盾和“官逼民反”的残酷现实，成功地塑造了鲁智深、李逵、武松、林冲、阮小七
```

等一批英雄人物。小说故事情节曲折，语言生动，人物性格鲜明，具有高度的艺术成就。但作品歌颂、美化宋江，鼓吹“忠义”和“替天行道”，表现出严重的思想局限。','','16','922','912000','2006-03-20','');

INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATEGORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERTOTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('老人与海','海明威','上海出版社','2006-8-1','9787532740093',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='世界名著'),'10','10','6.1','6.1','海明威(1899 — 1961)，美国著名作家、诺贝尔文学奖获得者。《老人与海》是他最具代表性的作品之一。','','16','98','67000','2006-03-20','');

INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATEGORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERTOTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('射雕英雄传(全四册)','金庸','广州出版社','2005-12-1','9787807310822',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='武侠'),'10','32','21.7','6.8','自幼家破人亡的郭靖，随母流落蒙古大漠，这傻头傻脑但有情有义的小伙子倒也是有福气，他不但习得了江南六怪的绝艺、全真教马钰的内功、洪七公的隆龙十八掌、双手互博之术、九阴真经等盖世武功，还让古灵精怪的小美女黄蓉这辈子跟定了他。这部原名『大漠英雄传』的小说是金庸小说中最广为普罗大众接受、传颂的一部，其中出了许多有名又奇特的人物，东邪西毒南帝北丐中神通，还有武功灵光、脑袋不灵光的老顽童周伯通，他们有特立独行的性格、作为和人生观，让人叹为观止。书中对历史多有着墨，中原武林及蒙古大漠的生活情形随着人物的生长环境变迁而有不同的叙述，异族统治之下的

小老百姓心情写来丝丝入扣，本书对情的感觉是很含蓄的，尤其是郭靖与拖雷、华筝无猜的童年之谊，他与江南六怪的师生之谊等等，还有全真七子中长春子丘处机的侠义行为及其与郭杨二人风雪中的一段情谊，也有很豪气的叙述。神算子瑛姑及一灯大师和周伯通的一场孽恋，是最出乎人意料的一段，成人世界的恋情可比小儿女的青涩恋燕还复杂多了。郭靖以扭胜巧的人生经历和「为国为民，侠之大者」的儒侠风范，也是书中最大要旨。距离这本书完成的时间已有四十年了，书中的单纯诚朴的人物性格还深深的留在读者心中，本书故事也多改编成电影、电视剧等，受欢迎程度可见一斑。','','16','','1153000','2006-03-20','');

INSERT INTO

PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE

GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT

OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)

VALUES('鲁迅文集(小说、散文、杂文)全两册','鲁迅','','2006-9-1','9787509000724',(SELECT

PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='文集

'),'10','39.8','20','5.0','','','16','684','680000','2006-03-20','');

INSERT INTO

PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE

GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT

OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)

VALUES('长征','王树增','人民文学出版社','2006-9-1','9787020057986',(SELECT

PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='纪实文

学'),'10','53','37.7','6.4','','','16','683','670000','2006-03-20','');

INSERT INTO

```
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('数据结构(C 语言版)(附光盘)','严蔚敏，吴伟民','清华大学出版社
','2007-3-1','9787302147510',(SELECT PRODUCT_SUBCATEGORYID FROM
PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='计算机理论'),'10','30','25.5','8.5','《数据结构》
(C 语言版)是为“数据结构”课程编写的教材，也可作为学习数据结构及其算法的 C 程序设计的参数教材。本
书的前半部分从抽象数据类型的角度讨论各种基本类型的数据结构及其应用
','','8','334','493000','2006-03-20','');

INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('工作中无小事','陈满麒','机械工业出版社','2006-1-1','9787111182252',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='行政管
理'),'10','16.8','11.4','6.8','本书立足于当今企业中常见的轻视小事，做事浮躁等现象，从人性的弱点这一独特角
度，挖掘出员工轻视小事的根本原因，具有深厚的人文关怀，极易引起员工的共鸣。它有助于员工端正心
态，摒弃做事贪大的浮躁心理，把小事做好做到位，从而提高整个企业的工作质量。当重视小事成为员工
的一种习惯，当责任感成为一种生活态度，他们将会与胜任、优秀、成功同行，责任、忠诚、敬业也将不
再是一句空洞的企业宣传口号。本书是一本提升企业竞争力、建设企业文化的指导手册，一本员工素质培
训的完美读本，一本所有公务员、公司职员的必读书。','','8','152','70000','2006-03-20','');

INSERT INTO
```

```
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('突破英文基础词汇','刘毅','外语教学与研究出版社','2003-8-1','9787560035024',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='英语词
汇'),'10','15.9','11.1','7.0','本书所列单词共计 1300 个，加上各词的衍生词、同义词及反义词，则实际收录约 3000
词，均为平时最常用、最容易接触到的单词。详细列出各词的国际音标、词性说明及中文解释，省却查字
典的麻烦。每一课分为五个部分，以便于分段记忆。在课前有预备测验，每一部分之后有习题，课后有效
果检测，可借助于重复测验来加深对单词的印象，并学习如何活用单词。','','8','350','','2006-03-20','');

INSERT INTO
PRODUCTION.PRODUCT(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATE
GORYID,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,DESCRIPTION,PHOTO,TYPE,PAPERT
OTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
VALUES('噼里啪啦丛书(全 7 册)','(日)佐佐木洋子','21 世纪出版社','1901-01-01','9787539125992',(SELECT
PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY WHERE NAME='幼儿启
蒙'),'10','58','42','6.1','噼里啪啦系列丛书包括：《我要拉巴巴》《我去刷牙》《我要洗澡》《你好》《草莓点心》《车
来了》《我喜欢游泳》共 7 册。 这是日本画家佐佐木洋子编绘的，分别描绘孩子在刷牙、洗澡、游玩、吃
点心等各种时候所碰到的问题，以风趣的方式教会他们人生的最初的知识。书中的图形不仅夸张诱人，而
且采用了一些局部折叠的方式，在书页中可以不时翻开一些折叠面，让人看到图画内部的东西，这是很符
合低幼儿童的阅读心理的。','','8','','','2006-03-20','');

//INSERT LOCATION
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='世界名著'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='武侠'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='科幻'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='军事'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='社会'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='文集'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='纪实文学'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='文学理论'),'库存 1-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='中国古诗词'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='中国现当代诗'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='戏剧'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='民间文学'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='计算机理论'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='计算机体系结构'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='操作系统'),'库存 1-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='程序设计'),'库存 1-货架 3');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='数据库'),'库存 1-货架 3');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='软件工程'),'库存 1-货架 3');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='信息安全'),'库存 1-货架 3');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='多媒体'),'库存 1-货架 3');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语词汇'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语语法'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语听力'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语口语'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语阅读'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='英语写作'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='行政管理'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='项目管理'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='质量管理与控制'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='商业道德'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='经营管理'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='财务管理'),'库存 1-货架 4');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
```

```
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='幼儿启蒙'),'库存 2-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='益智游戏'),'库存 2-货架 1');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='童话'),'库存 2-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='卡通'),'库存 2-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='励志'),'库存 2-货架 2');
INSERT INTO PRODUCTION.LOCATION(PRODUCT_SUBCATEGORYID,NAME)
VALUES((SELECT PRODUCT_SUBCATEGORYID FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE NAME='少儿英语'),'库存 2-货架 2');


//INSERT PRODUCT_INVENTORY
INSERT INTO PRODUCTION.PRODUCT_INVENTORY(PRODUCTID,LOCATIONID,QUANTITY)
SELECT T1.PRODUCTID,T2.LOCATIONID,100 FROM PRODUCTION.PRODUCT T1,PRODUCTION.LOCATION
T2
WHERE T1.PRODUCT_SUBCATEGORYID=T2.PRODUCT_SUBCATEGORYID;
```

```
//INSERT PRODUCT_REVIEW
INSERT INTO
PRODUCTION.PRODUCT_REVIEW(PRODUCTID,NAME,REVIEWDATE,EMAIL,RATING,COMMENTS)
SELECT PRODUCTID,'刘青','2007-05-06','zhangping@sina.com','1','送货快' from
PRODUCTION.PRODUCT;
INSERT INTO
PRODUCTION.PRODUCT_REVIEW(PRODUCTID,NAME,REVIEWDATE,EMAIL,RATING,COMMENTS)
SELECT PRODUCTID,'桑泽恩','2007-05-06','zhangping@sina.com','1','服务态度好' from
PRODUCTION.PRODUCT;

//INSERT PRODUCT_VENDOR
INSERT INTO
PRODUCTION.PRODUCT_VENDOR(PRODUCTID,VENDORID,STANDARDPRICE,LASTPRICE,LASTDATE,MI
NQTY,MAXQTY,ONORDERQTY)
SELECT PRODUCTID,VENDORID,25,'','10','100'," FROM PRODUCTION.PRODUCT,PURCHASING.VENDOR
WHERE PRODUCTION.PRODUCT.PUBLISHER=PURCHASING.VENDOR.NAME;

//INSER SALESORDER_HEADER
INSERT INTO
SALES.SALESORDER_HEADER(ORDERDATE,DUEDATE,STATUS,ONLINEORDERFLAG,CUSTOMERID,SALES
PERSONID,ADDRESSID,SHIPMETHOD,SUBTOTAL,FREIGHT,TOTAL,COMMENTS)
VALUES ('2007-05-06','2007-5-07',2,1,1,2,3,0,36.9,0,36.9,'上午送到');
INSERT INTO
SALES.SALESORDER_HEADER(ORDERDATE,DUEDATE,STATUS,ONLINEORDERFLAG,CUSTOMERID,SALES
```

```
PERSONID,ADDRESSID,SHIPMETHOD,SUBTOTAL,FREIGHT,TOTAL,COMMENTS)

VALUES ('2007-05-07','2007-5-07',1,1,1,1,1,0,36.9,0,36.9,'上午送到');



//INSERT SALESORDER_DETAIL

INSERT INTO

SALES.SALESORDER_DETAIL(SALESORDERID,SALESORDER_DETAILID,CARRIERNO,PRODUCTID,ORDER

QTY,LINETOTAL)

SELECT SALESORDERID,'1','2007052',1,1,15.2 FROM SALES.SALESORDER_HEADER;

INSERT INTO

SALES.SALESORDER_DETAIL(SALESORDERID,SALESORDER_DETAILID,CARRIERNO,PRODUCTID,ORDER

QTY,LINETOTAL)

SELECT SALESORDERID,'2','2007053',3,1,21.7 FROM SALES.SALESORDER_HEADER;



UPDATE SALES.SALESPERSON SET SALESLASTYEAR = 20.0000 WHERE SALESPERSONID = 2;



INSERT INTO

PURCHASING.PURCHASEORDER_HEADER(ORDERDATE,STATUS,EMPLOYEEID,VENDORID,SHIPMETHOD,

SUBTOTAL,TAX,FREIGHT,TOTAL)

VALUES('2006-7-21',1,6,5,'快递',5000.00,600.00,800.00,6400.00);



//INSERT DEPARTMENT

INSERT INTO OTHER.DEPARTMENT VALUES(NULL, '总公司');

INSERT INTO OTHER.DEPARTMENT VALUES('总公司', '服务部');

INSERT INTO OTHER.DEPARTMENT VALUES('总公司', '采购部');
```

```
INSERT INTO OTHER.DEPARTMENT VALUES('总公司', '财务部');

INSERT INTO OTHER.DEPARTMENT VALUES('服务部', '网络服务部');

INSERT INTO OTHER.DEPARTMENT VALUES('服务部', '读者服务部');

INSERT INTO OTHER.DEPARTMENT VALUES('服务部', '企业服务部');

INSERT INTO OTHER.DEPARTMENT VALUES('读者服务部', '书籍借阅服务部');

INSERT INTO OTHER.DEPARTMENT VALUES('读者服务部', '书籍阅览服务部');


//INSERT EMPSALARY

INSERT INTO OTHER.EMPSALARY VALUES ('KING',7839,5000);

INSERT INTO OTHER.EMPSALARY VALUES ('SCOTT',7788,3000);

INSERT INTO OTHER.EMPSALARY VALUES ('FORD',7902,3000);

INSERT INTO OTHER.EMPSALARY VALUES ('JONES',7566,2975);

INSERT INTO OTHER.EMPSALARY VALUES ('BLAKE',7698,2850);

INSERT INTO OTHER.EMPSALARY VALUES ('CLARK',7782,2450);

INSERT INTO OTHER.EMPSALARY VALUES ('ALLEN',7499,1600);

INSERT INTO OTHER.EMPSALARY VALUES ('TURNER',7844,1500);

INSERT INTO OTHER.EMPSALARY VALUES ('MILLER',7934,1300);

INSERT INTO OTHER.EMPSALARY VALUES ('WARD',7521,1250);

INSERT INTO OTHER.EMPSALARY VALUES ('MARTIN',7654,1250);

INSERT INTO OTHER.EMPSALARY VALUES ('ADAMS',7876,1100);

INSERT INTO OTHER.EMPSALARY VALUES ('JAMES',7900,950);

INSERT INTO OTHER.EMPSALARY VALUES ('SMITH',7369,800);


// INSERT ACCOUNT
```

```
INSERT INTO OTHER.ACCOUNT VALUES(1,1000);

INSERT INTO OTHER.ACCOUNT VALUES(2,2000);

INSERT INTO OTHER.ACCOUNT VALUES(3,1500);

INSERT INTO OTHER.ACCOUNT VALUES(4,6500);

INSERT INTO OTHER.ACCOUNT VALUES(5,500);


// INSERT ACTIONS

INSERT INTO OTHER.ACTIONS VALUES(3,'U',599,NULL);

INSERT INTO OTHER.ACTIONS VALUES(6,'I',20099,NULL);

INSERT INTO OTHER.ACTIONS VALUES(5,'D',NULL,NULL);

INSERT INTO OTHER.ACTIONS VALUES(7,'U',1599,NULL);

INSERT INTO OTHER.ACTIONS VALUES(1,'I',399,NULL);

INSERT INTO OTHER.ACTIONS VALUES(9,'D',NULL,NULL);

INSERT INTO OTHER.ACTIONS VALUES(10,'X',NULL,NULL);


//INSERT READER

INSERT INTO OTHER.READER VALUES(10, 'Bill', 19, 'M', 'Computer');

INSERT INTO OTHER.READER VALUES(11, 'Susan', 18, 'F', 'History');

INSERT INTO OTHER.READER VALUES(12, 'John', 19, 'M', 'Computer');
```

第 3 章 数据定义语句

本章介绍 DM 的数据定义语句，包括数据库修改语句、用户管理语句、模式管理语句、表空间管理语句、表管理语句等等。

需要注意的是，在数据定义语句中有时需要指定一些文件的路径，无论用户指定的是绝对路径还是相对路径，DM 在处理时最终都会将其统一处理为绝对路径，DM 规定这个绝对路径的长度不能超过 256 字节。

3.1 数据库修改语句

一个数据库创建成功后，可以修改日志文件大小、增加和重命名日志文件、可以移动数据文件；可以修改数据库的状态和模式；还可以进行归档配置。

语法格式

```
ALTER DATABASE <修改数据库子句>;

<修改数据库子句>::=

    RESIZE LOGFILE <文件路径> TO <文件大小>|

    ADD LOGFILE <文件说明项>{,<文件说明项>}|

    DROP LOGFILE <文件路径>|

    ADD NODE LOGFILE <文件说明项>,<文件说明项>{,<文件说明项>}|

    RENAME LOGFILE <文件路径>{,<文件路径>} TO <文件路径>{,<文件路径>}|

    OPEN [FORCE] |

    MOUNT |

    SUSPEND |

    NORMAL [FORCE] |
```

```
    PRIMARY [FORCE] |

    STANDBY [FORCE] |

    ARCHIVELOG |

    NOARCHIVELOG |

    <ADD|MODIFY|DELETE> ARCHIVELOG <归档配置语句> |

    ARCHIVELOG CURRENT

<文件说明项> ::= <文件路径>SIZE <文件大小>

<归档配置语句>::= 'TYPE = <归档类型>,DEST = <归档目标>'

<归档类型>::=

    LOCAL [<文件和空间限制设置>][,ARCH_FLUSH_BUF_SIZE = <归档合并刷盘缓存大小>][,
HANG_FLAG=<0|1>] |

    REALTIME [,WAIT_APPLY=<0|1>] |

    ASYNC , ARCH_TIMER_NAME = <定时器名称>[,ARCH_SEND_DELAY = <归档延时发送时间>] |

    SYNC [,ARCH_RECOVER_TIME = <同步备库的异步恢复时间间隔>] |

    REMOTE ,INCOMING_PATH = <远程归档路径> |

    TIMELY [,WAIT_APPLY=<0|1>]

<文件和空间限制设置>::=[,FILE_SIZE = <文件大小>][,SPACE_LIMIT = <空间大小限制>]
```

参数

1. RESIZE LOGFILE 将日志文件修改到指定大小。只能增加文件的大小，否则失败；

2. DROP LOGFILE 要求指定的重做日志文件满足以下几个条件，否则失败：

1)指定文件必须是当前节点自身的重做日志文件。

2)指定文件不是当前正在写的重做日志文件，并且该文件上不存在有效的重做日志。

3)指定文件必须是 RLOG 表空间中 FILE_ID 最大的那个文件。

4)指定文件不能是 0 号或 1 号重做日志文件。

5)指定文件删除前后的日志空间都应该是安全的。即当前已使用的日志空间不应该超过 RLOG_RESERVE_THRESHOLD，并且删除该文件不会导致剩余日志空间过小。

3. <文件路径> 指明被操作的数据文件在操作系统下的绝对路径：'路径 + 数据文件名'。例如：'C:\DMDBMS\data\dmlog_0.log';

4．<文件大小> 重做日志文件的大小，单位 MB。在 32 位版本下，取值范围为 256 和 2048 之间的整数，缺省值为 2048；在 64 位版本下，取值范围为 256 和 8192 之间的整数，缺省值为 4096;

5．<归档目标> 指归档日志所在位置，若本地归档，则本地归档目录；若远程归档，则为远程服务实例名；删除操作，只需指定归档目标;

6．<归档类型> 指归档操作类型，包括 LOCAL/REALTIME/ASYNC/SYNC/REMOTE/TIMELY，分别表示本地归档/实时归档/异步归档/同步归档/远程归档/主备即时归档;

7 HANG_FLAG 本地归档写入失败时系统是否挂起标志。取值 0 或 1，0 不挂起；1 挂起。缺省为 1（第一份本地归档系统内固定设为 1，设 0 实际也不起作用）;

8. WAIT_APPLY 指定实时归档/即时归档是否为高性能模式或事务一致模式。取值 0 或 1，0 表示高性能模式；1 表示事务一致模式。若未指定 WAIT_APPLY，则以 dmarch.ini 中的 ARCH_WAIT_APPLY 配置项为准;

9．<空间大小限制> 整数值，取值范围 1024~2147483647，若设为 0，表示不限制，仅本地归档有效;

10．<定时器名称> 异步归档中指定的定时器名称，仅异步归档有效;

11. <归档延时发送时间> 指源库到异步备库的归档延时发送时间，单位为分钟，取值范围 0~1440，缺省为 0，表示不启用归档延时发送功能。仅异步归档有效。如果源库是 DSC

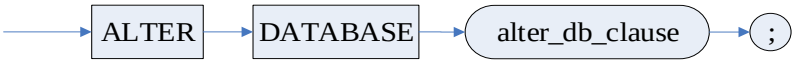
集群，建议用户配置时保证各节点上配置的值是一致的，并保证各节点所在机器的时钟一致，避免控制节点发生切换后计算出的归档延时发送时间不一致；

12. <同步备库的异步恢复时间间隔> 指主库向同步备库发起异步恢复的时间间隔。单位秒，取值范围 1~86400，缺省为 1，表示主库每隔 1 秒检查一次同步备库的归档状态，若为 Invalid 且和同步备库通信正常，则发起异步恢复；

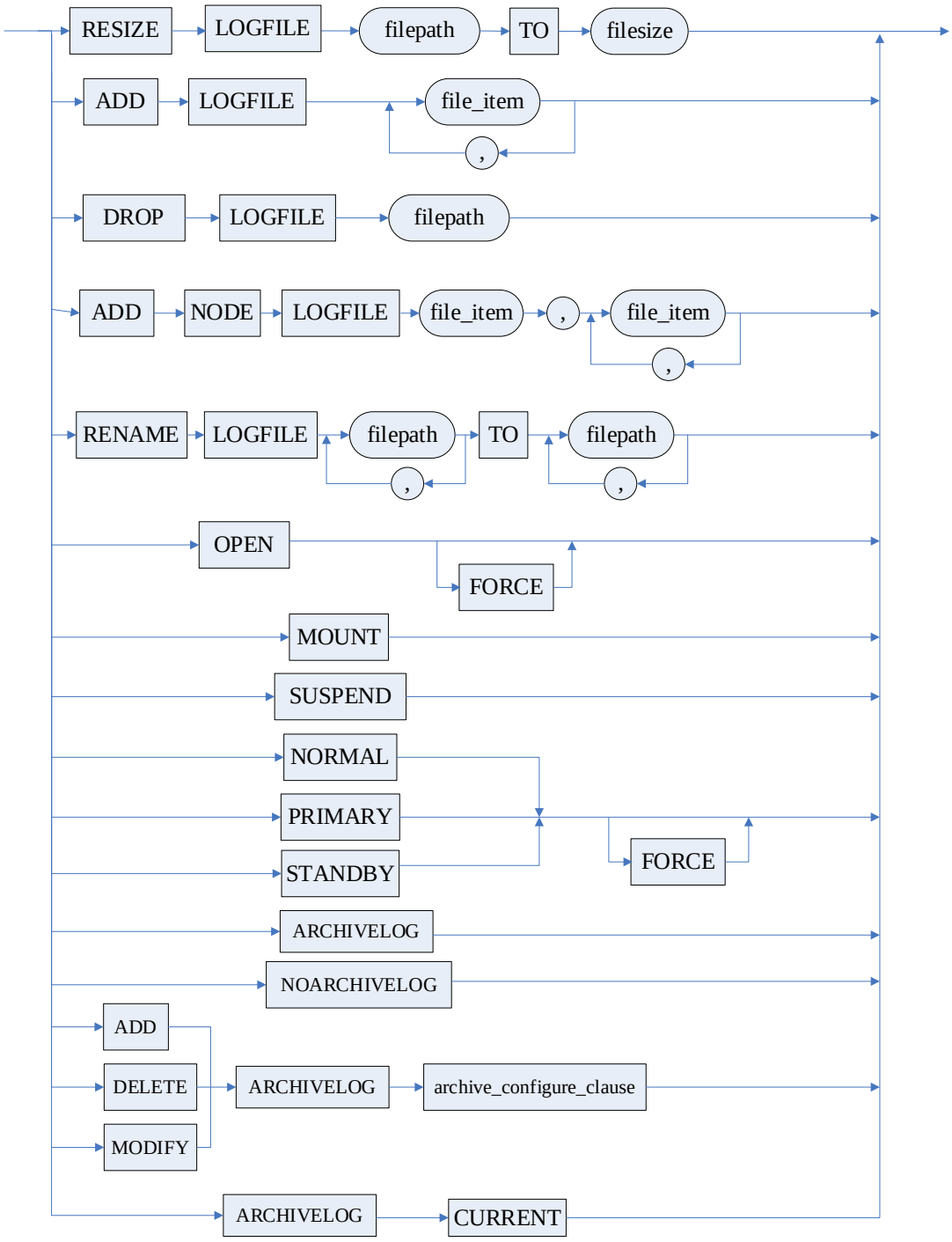
13. <归档合并刷盘缓存大小> 整数值，单位为 MB，取值范围 0~128，若设为 0，表示不使用归档合并刷盘。

图例

数据库修改语句



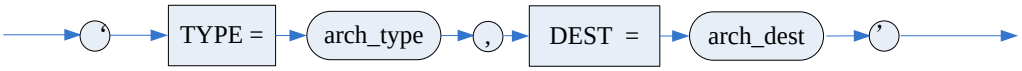
修改数据库子句（alter_db_clause）



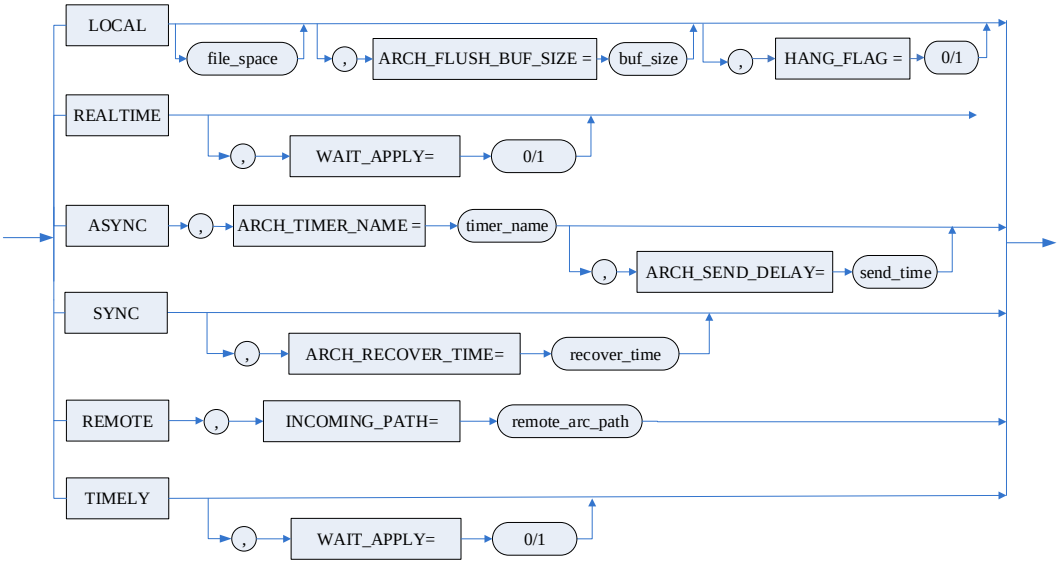
文件说明项（file_item）



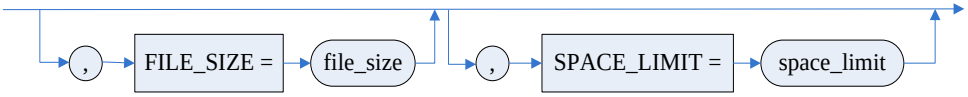
归档配置语句（archive_configure_clause）



归档类型（arch_type）



文件和空间限制设置（file_space）



语句功能

供具有 DBA 和具有 ALTER DATABASE 权限的用户修改数据库。

使用说明

1. 归档的配置也可以通过 DM.INI 参数 ARCH_INI 和归档配置文件 DMARCH.INI 进行，可参看《DM8 系统管理员手册》，SQL 语句提供了在 DM 服务器运行时对归档配置进行动态修改的手段，通过 SQL 语句修改成功后会将相关配置写入 DMARCH.INI 中；

2. 修改日志文件大小时，只能增加文件的大小，否则失败；

3. ADD NODE LOGFILE 用于 DMDSC 集群扩展节点时使用；

4. 只有 MOUNT 状态 NORMAL 模式下才能启用/关闭归档或者重命名归档日志文件；

5. 针对 REMOTE 归档，仅允许在 MOUNT 状态 NORMAL 模式下进行添加、修改或删除；

6. 针对 LOCAL 归档，仅允许在 MOUNT 状态 NORMAL 模式进行添加和修改，不允许删除本地归档。其中，<文件和空间限制设置>中 SPACE_LIMIT 和 FILE_SIZE 两项，在 MOUNT

状态 NORMAL 模式或 OPEN 状态下均可被修改；

7. 针对 REALTIME/TIMELY 归档，在 MOUNT 状态 NORMAL 模式或 OPEN 状态下均可进行添加或删除，但仅允许在 MOUNT 状态 NORMAL 模式下修改 WAIT_APPLY 配置项；

8. ARCHIVELOG CURRENT 把新生成的，还未归档的联机日志都进行归档；

9. 本地归档仅支持修改 space_limit/file_size 配置项值；

10. 非归档模式下，不允许添加、修改或删除任何归档配置；

11. 目前支持的归档配置中，INCOMING_PATH、ARCH_FLUSH_BUF_SIZE、HANG_FLAG、ARCH_TIMER_NAME、WAIT_APPLY 仅允许在 MOUNT 状态下修改；

12. DMDSC 数据守护环境中，删除 REALTIME/TIMELY 归档前，需要先依次连接 DMDSC 主库每个节点执行 SP_SET_ARCH_STATUS 使待删除的归档失效，之后数据守护环境中的主库和备库才能执行 ALTER DATABASE DELETE 语句删除 REALTIME/TIMELY 归档。

13. NORMAL、PRIMARY、STANDBY 三种数据库模式切换在 MOUNT 状态下可以直接执行，在 OPEN 状态下需要加 FORCE 关键字强制执行（OPEN 状态下的模式切换仅在单机和 DSC 环境下有效）。

举例说明

假设数据库 BOOKSHOP 页面大小为 8K，数据文件存放路径为 C:\DMDBMS\data。

例 1 给数据库增加一个日志文件 C:\DMDBMS\data\dmlog_0.log，其大小为 200M。

```
ALTER DATABASE ADD LOGFILE 'C:\DMDBMS\data\dmlog_0.log'  SIZE 200;
```

例 2 扩展数据库中的日志文件 C:\DMDBMS\data\dmlog_0.log，使其大小增大为 300M。

```
ALTER DATABASE RESIZE LOGFILE 'C:\DMDBMS\data\dmlog_0.log'  TO 300;
```

例 3 重命名日志文件 C:\DMDBMS\data\dmlog_0.log 为 d:\dmlog_1.log。

```
ALTER DATABASE MOUNT;
```

```
ALTER DATABASE RENAME LOGFILE 'C:\DMDBMS\data\dmlog_0.log' TO 'd:\dmlog_1.log';
ALTER DATABASE OPEN;
```

例 4 设置数据库为 MOUNT 状态 NORMAL 模式，并开启归档模式。

```
ALTER DATABASE MOUNT;
ALTER DATABASE NORMAL;
ALTER DATABASE ARCHIVELOG;
```

例 5 增加本地归档配置，归档目录为 c:\arch_local，文件大小为 128MB，空间限制为 1024MB。需要保证当前数据库处于 MOUNT 状态、NORMAL 模式以及归档模式下。

```
ALTER DATABASE ADD ARCHIVELOG 'TYPE = local, DEST = c:\arch_local, FILE_SIZE = 128,
SPACE_LIMIT = 1024';
```

例 6 增加一个实时归档配置，远程服务实例名为 realtime，需事先配置 mal。

```
ALTER DATABASE ADD ARCHIVELOG ' TYPE = REALTIME, DEST = realtime';
```

例 7 增加一个异步归档配置，远程服务实例名为 asyn1，定时器名为 timer1，需事先配置好 mal 和 timer。timer 可以通过 SP_ADD_TIMER 等定时器管理相关的函数进行动态配置。

```
ALTER DATABASE ADD ARCHIVELOG 'TYPE = ASYNC, DEST = asyn1, ARCH_TIMER_NAME = timer1';
```

例 8 增加一个异步归档配置，远程服务实例名为 asyn2，定时器名为 timer2，源库到异步备库的归档延时发送时间为 10 分钟，需事先配置好 mal 和 timer。timer 可以通过 SP_ADD_TIMER 等定时器管理相关的函数进行动态配置。

```
ALTER DATABASE ADD ARCHIVELOG 'TYPE=ASYNC, DEST=asyn2, ARCH_TIMER_NAME=timer2,
ARCH_SEND_DELAY=10';
```

例 9 设置数据库状态为 OPEN。

```
ALTER DATABASE OPEN;
```

例 10 MOUNT 状态下设置数据库模式为 PRIMARY。

```
ALTER DATABASE MOUNT;

ALTER DATABASE PRIMARY;

ALTER DATABASE OPEN FORCE;
```

例 11 OPEN 状态下设置数据库模式为 STANDBY。

```
ALTER DATABASE OPEN;

ALTER DATABASE STANDBY FORCE;

ALTER DATABASE OPEN FORCE;
```

3.2 管理用户

3.2.1 用户定义语句

在数据库中创建新的用户，DM 中直接用 USER 与数据库服务器建立连接。

语法格式

```
CREATE USER [IF NOT EXISTS] <用户名> IDENTIFIED <身份验证模式> [PASSWORD_POLICY <口令策略>][<锁定子句>][<存储加密密钥>][<只读标志>][<资源限制子句>][<密码过期子句>][<允许 IP 子句>][<禁止 IP 子句>][<允许时间子句>][<禁止时间子句>][<TABLESPACE 子句>][<INDEX_TABLESPACE 子句>][<表空间配额子句>];
<身份验证模式> ::=
    <数据库身份验证模式>|
    <外部身份验证模式>
<数据库身份验证模式> ::= BY <口令> [<散列选项>]
<散列选项> ::= HASH WITH [<密码引擎名>.]<散列算法> [<加盐选项>]
<加盐选项> ::= [NO] SALT
<外部身份验证模式> ::=
```

```
    EXTERNALLY |

    EXTERNALLY AS <用户 DN>

<锁定子句> ::=

    ACCOUNT LOCK |

    ACCOUNT UNLOCK

<存储加密密钥> ::= ENCRYPT BY <口令>

<只读标志> ::= [NOT] READ ONLY

<资源限制子句> ::=

    DROP PROFILE |

    PROFILE <profile 名> |

    LIMIT <资源设置>

<资源设置> ::=

    <资源设置项>{,<资源设置项>} |

    <资源设置项>{ <资源设置项>}

<资源设置项> ::=

    GOLBAL_SESSION_PER_USER<参数设置> |

    SESSION_PER_USER <参数设置> |

    CONNECT_IDLE_TIME <参数设置> |

    CONNECT_TIME <参数设置> |

    CPU_PER_CALL <参数设置> |

    CPU_PER_SESSION <参数设置> |

    MEM_SPACE <参数设置> |

    READ_PER_CALL <参数设置> |

    READ_PER_SESSION <参数设置> |
```

```
    FAILED_LOGIN_ATTEMPTS <参数设置> |
    PASSWORD_LIFE_TIME <参数设置> |
    PASSWORD_REUSE_TIME <参数设置> |
    PASSWORD_REUSE_MAX <参数设置> |
    PASSWORD_LOCK_TIME <参数设置> |
    PASSWORD_GRACE_TIME <参数设置>|
    INACTIVE_ACCOUNT_TIME<参数设置>
<参数设置> ::=
    <参数值> |
    UNLIMITED |
    DEFAULT
<密码过期子句> ::= PASSWORD EXPIRE
<允许 IP 子句> ::=
    ALLOW_IP NULL |
    ALLOW_IP <IP 项>{,<IP 项>}
<禁止 IP 子句> ::=
    NOT_ALLOW_IP NULL |
    NOT_ALLOW_IP <IP 项>{,<IP 项>}
<IP 项> ::=
    <具体 IP> |
    <网段>
<允许时间子句> ::= ALLOW_DATETIME <时间项>{,<时间项>}
<禁止时间子句> ::= NOT_ALLOW_DATETIME <时间项>{,<时间项>}
<时间项> ::=
```

```
    <具体时间段> |
    <规则时间段>
<具体时间段> ::= <具体日期> <具体时间> TO <具体日期> <具体时间>
<规则时间段> ::= <规则时间标志> <具体时间> TO <规则时间标志> <具体时间>
<规则时间标志> ::=
    MON |
    TUE |
    WED |
    THURS |
    FRI |
    SAT |
    SUN
<TABLESPACE 子句> ::= DEFAULT TABLESPACE <表空间名>
<INDEX_TABLESPACE 子句> ::= DEFAULT INDEX TABLESPACE <表空间名>
<表空间配额子句> ::=
    QUOTA UNLIMITED |
    QUOTA 0 |
    QUOTA <配额大小> ON <表空间名>{ QUOTA <配额大小> ON <表空间名>}
<配额大小> ::=
    UNLIMITED |
    <空间大小> [K|M|G|T]
```

参数

1. <用户名> 指明要创建的用户名称，用户名称最大长度 128 字节；

2. <参数设置> 用于限制用户对 DM 数据库服务器系统资源的使用；

3. 系统在创建用户时，必须指定一种身份验证模式：<数据库身份验证模式>或者<外部身份验证模式>。

<数据库身份验证模式>中如果缺省了<散列选项>，则采用 HASH WITH SHA512 SALT。系统用户(SYSDBA、SYSAUDITOR、SYSSSO、SYSDBO)均采用 HASH WITH SHA512 SALT 方式。如果 MANAGER 登录时勾选了“保存口令（S）”，这个口令保存在客户端则采用 HASH WITH AES256 SALT 方式。<密码引擎名>缺省为使用内部算法名，内部算法无引擎名。<加盐选项>缺省为 SALT。

<外部身份验证模式>支持 LDAP 身份验证、SSL 身份验证和 KERBEROS 身份验证。外部身份验证功能仅在安全版本中提供支持，具体请参考《DM8 安全管理》2.3 节。

4. <口令策略>用于设置口令策略，取值 15 或 31。15：禁止与用户名相同、口令长度需大于等于 INI 参数 PWD_MIN_LEN 设置的值、须同时包含大写字母(A~Z)和小写字母(a-z)、至少包含一个数字(0-9)；31：除了取值 15 的策略外，还要求至少包含一个标点符号(英文输入法状态下，除“ 和空格外的所有符号)。

5. <存储加密密钥> 用于与半透明加密配合使用，缺省情况下系统自动生成一个密钥，半透明加密时用户仅能查看到自己插入的数据；

6. <只读标志> 表示该登录是否只能对数据库进行只读操作，缺省为可读写。用户改为只读/非只读后，用户需要重新登录，会话才生效；

7. <资源设置项> 的各参数设置说明见下表：

表 3.2.1 资源设置项说明

资源设置项	说明	最大值	最小值	缺省值
GLOBAL_SESSION_PER_USER	在所有 DSC 实例中，一个用户可以同时拥有的会话数量	32768	1	系统所能提供的最大值
SESSION_PER_USER	在一个实例中，一个用户可以同时拥有的会话数量	32768	1	安全版本中默认值为

				4096; 其他版本中默认值为系统所能提供的最大值
CONNECT_TIME	一个会话连接、访问和操作数据库服务器的时间上限。 单位由 INI 参数 RESOURCE_FLAG 决定。 RESOURCE_FLAG 取值 1、0 时，CONNECT_TIME 单位分别为秒、分钟。 RESOURCE_FLAG 缺省值为 0	当 RESOURCE_FLAG 取值 1、0 时，缺省值分别为 86400 秒、1440 分钟	1	无限制
CONNECT_IDLE_TIME	会话最大空闲时间。单位由 INI 参数 RESOURCE_FLAG 决定。 RESOURCE_FLAG 取值 1、0 时，CONNECT_IDLE_TIME 单位分别为秒、分钟。 RESOURCE_FLAG 缺省值为 0	当 RESOURCE_FLAG 取值 1、0 时，缺省值分别为 86400 秒、1440 分钟	1	无限制
FAILED_LOGIN_ATTEMPTS	将引起一个账户被锁定的连续注册失败的次数	100	1	3, 系统预设管理员用户默认无限制
CPU_PER_SESSION	一个会话允许使用的 CPU 时间上限（单位：秒）	31536000(365 天)	1	无限制
CPU_PER_CALL	用户的一个请求能够使用的 CPU 时间上限（单位：秒）	86400（1 天）	1	无限制
READ_PER_SESSION	会话能够读取的总数据页数上限	2147483646	1	无限制
READ_PER_CALL	每个请求能够读取的数据页数	2147483646	1	无限制
MEM_SPACE	会话占有的私有内存空间上限（单位：MB）	2147483647	1	无限制
PASSWORD_LIFE_TIME	一个口令在其终止前可以使用的天数	365	1	无限制
PASSWORD_REUSE_TIME	一个口令在可以重新使用前必须经过的天数	365	1	无限制
PASSWORD_REUSE_MAX	一个口令在可以重新使用前必须改变的次数	32768	1	无限制
PASSWORD_LOCK_TIME	如果超过 FAILED_LOGIN_ATTEMPTS 设置值, 一个账户将被锁定的分钟	1440（1 天）	1	1，系统预设管理员用户默认

	数			无限制
PASSWORD_GRACE_TIME	以天为单位的口令过期宽限时间，过期口令超过该期限后，禁止执行除修改口令以外的其他操作	30	1	10，系统预设管理员用户默认无限制
INACTIVE_ACCOUNT_TIME	指定用户的非活跃时间，用户超过指定时间未登录将被自动锁定	24855	15	无限制

8. <密码过期子句> 用于设置密码过期。密码过期可使用 ALTER USER 语句进行重设；

9. 允许 IP 和禁止 IP 用于控制此登录是否可以从某个 IP 访问数据库，其中禁止 IP 优先。在设置 IP 时，可以利用*来设置网段，如 192.168.0.*。设置的允许和禁止 IP 需要用双引号括起来；

10. 允许时间段和禁止时间段用于控制此登录是否可以在某个时间段访问数据库，其中禁止时间段优先。设置的时间段中的日期和时间要分别用双引号括起来。在设置时间段时，有两种方式：

1） 具体时间段，如 2016 年 1 月 1 日 8：30 至 2006 年 2 月 1 日 17：00；

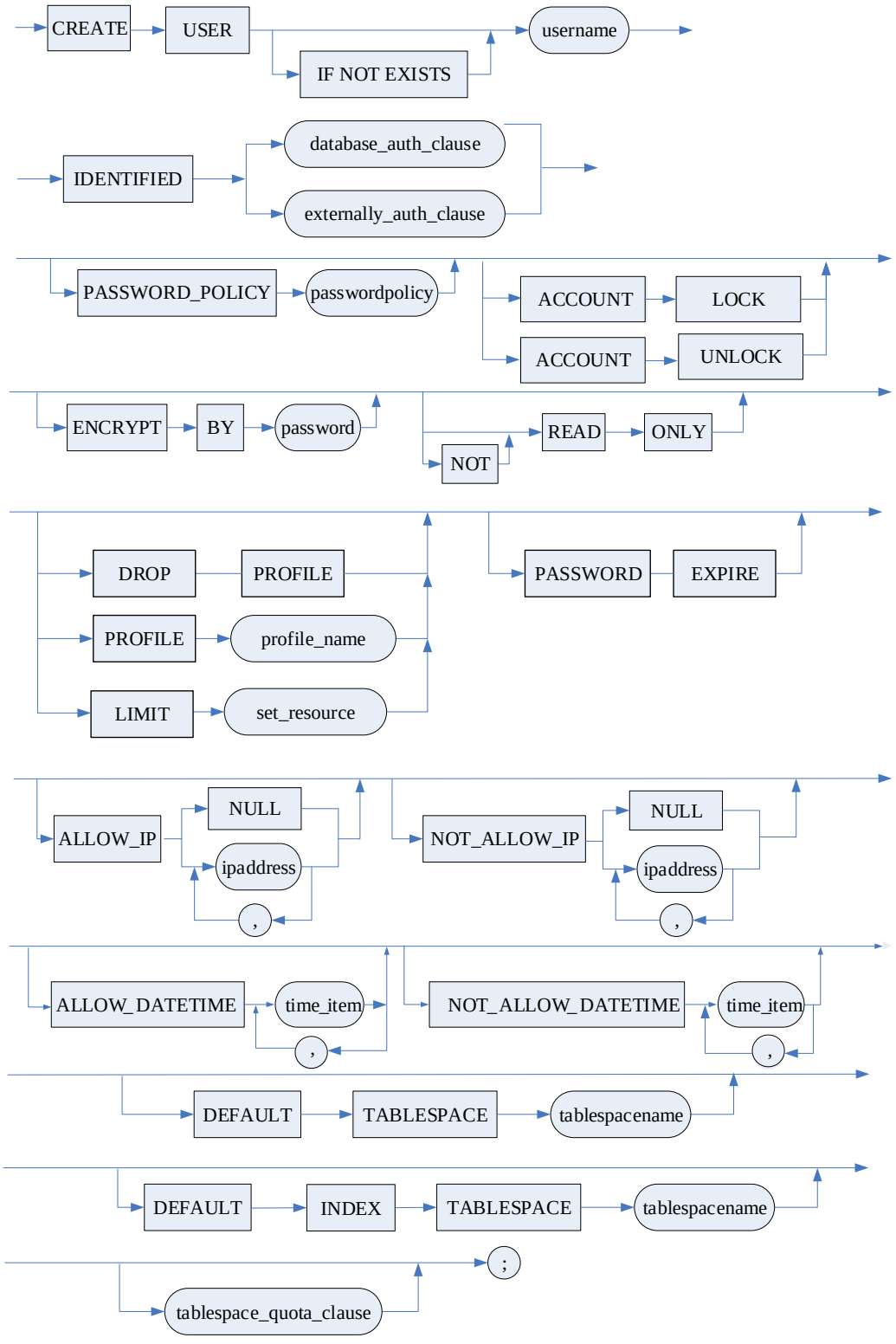
2） 规则时间段，如 每周一 8：30 至 每周五 17：00。

11. 默认用户表空间<TABLESPACE 子句>和默认索引表空间<INDEX_TABLESPACE 子句>不能使用 SYSTEM、RLOG、ROLL、TEMP 表空间；

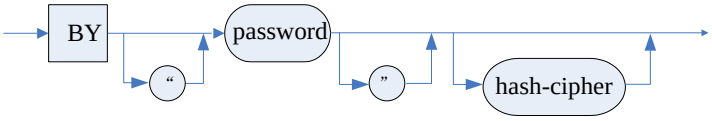
12. <表空间配额子句> 用于设置用户对表空间的使用空间大小限制。QUOTA UNLIMITED 表示用户对所有表空间均无配额限制，缺省为 QUOTA UNLIMITED。QUOTA 0 表示用户对所有表空间的配额均为 0。允许指定<表空间名>来设置用户对特定表空间的配额大小，配额上限为 100T，<空间大小>的默认单位为字节。

图例

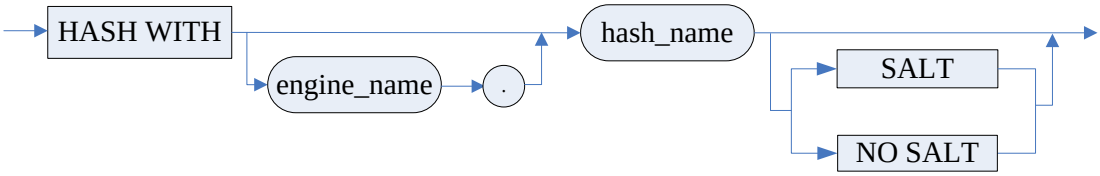
用户定义语句



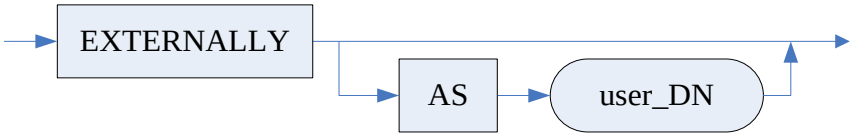
数据库身份验证模式（database_auth_clause）



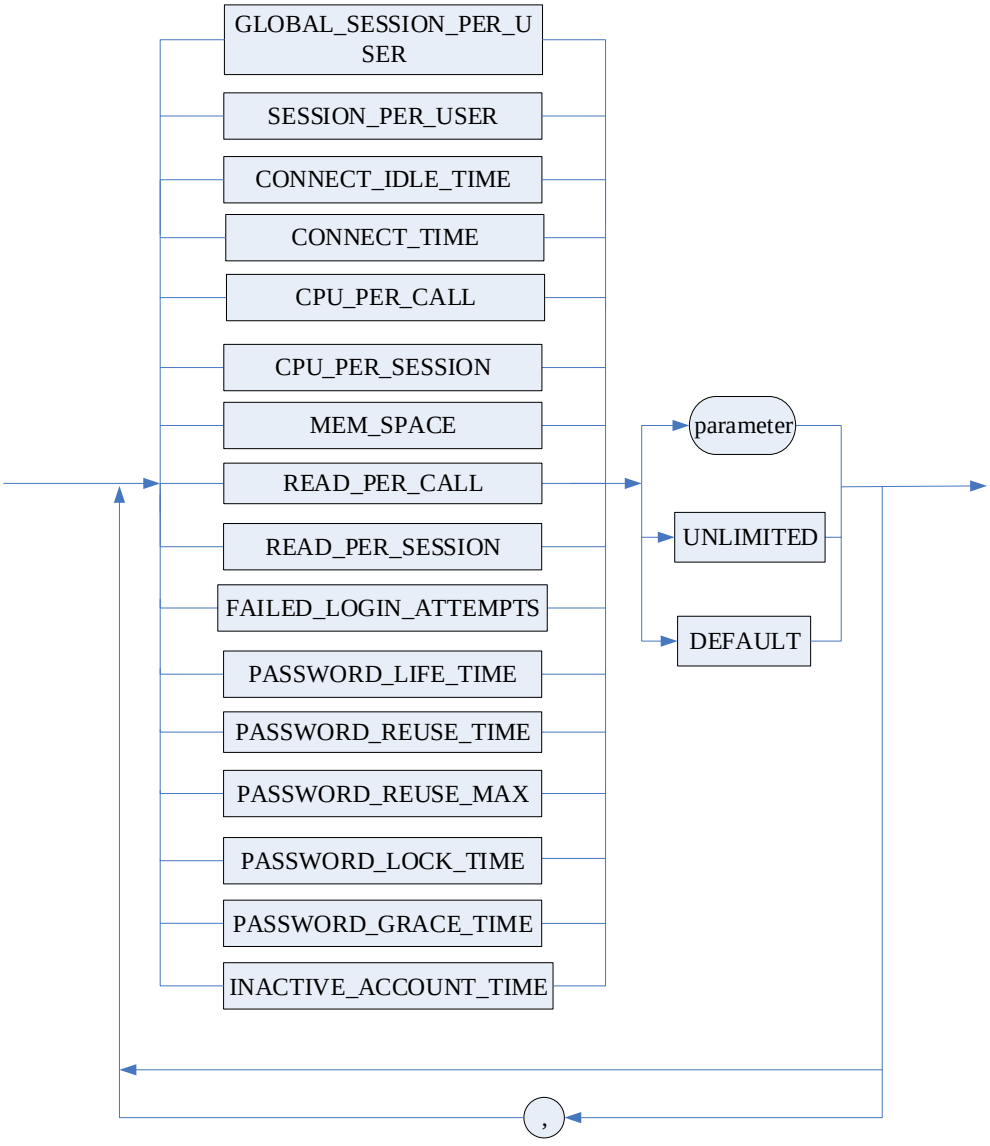
散列选项（hash_cipher）



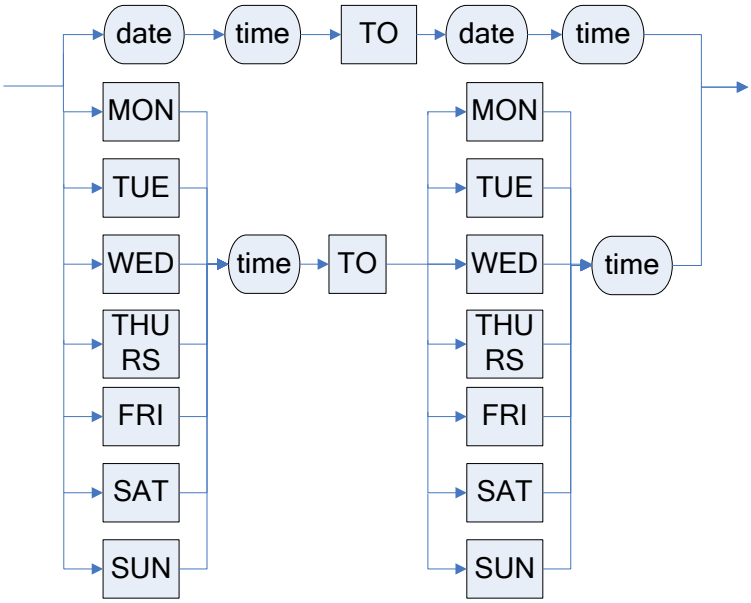
外部身份验证模式（externally_auth_clause）



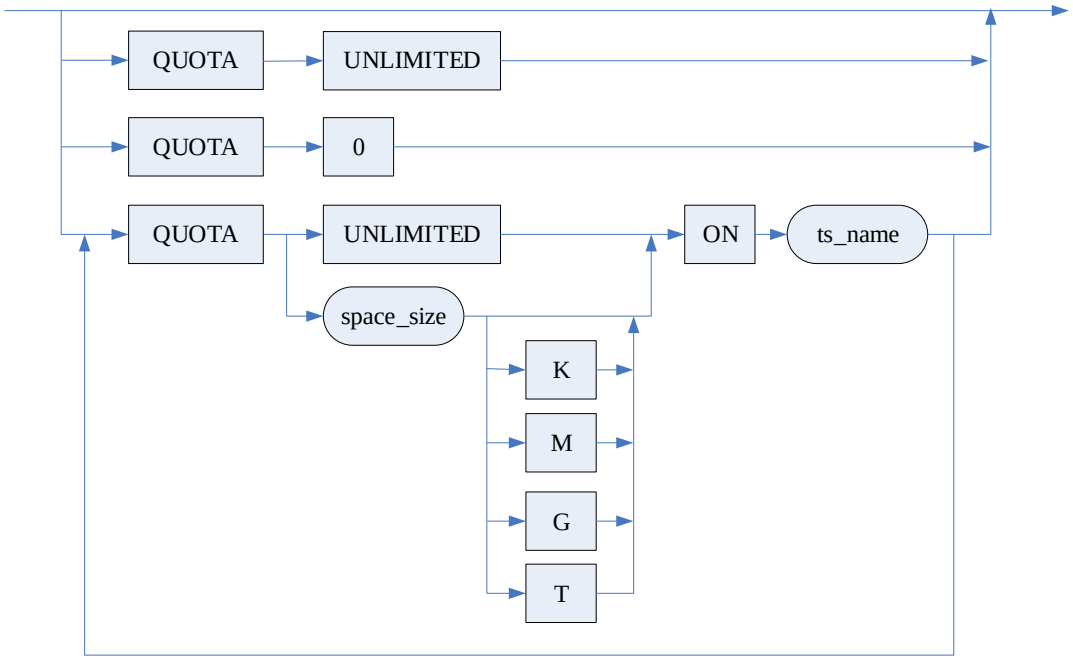
资源设置（set_resource）



时间项（time_item）



表空间配额子句（tablespace_quota_clause）



语句功能

创建新的用户。创建用户的操作一般只能由系统预设用户 SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 完成，如果普通用户需要创建用户，必须具有 CREATE USER 的数据库权限。

使用说明

1. 用户名在服务器中必须唯一。如果用户名已存在则报错，若指定 IF NOT EXISTS 关键字，用户名已存在不报错，忽略本次用户创建操作；

2. 系统为一个用户存储的信息主要有：用户名、口令、资源限制；

3. 用户口令以密文形式存储；

4. 如果没有指定用户默认表空间，则系统指定 MAIN 表空间为用户的默认表空间；

5. 如果没有指定用户默认索引表空间，则 HUGE 表的索引缺省存储在用户的默认表空间中，普通表的索引缺省存储在表的聚集索引所在的表空间中。临时表的索引永远在 TEMP 表空间；

6. 系统预先设置了四个用户，分别为 SYSDBA、SYSAUDITOR、SYSSSO 和 SYSDBO，其中 SYSDBO 用户仅四权分立下支持。各预设用户拥有的具体权限请参考《DM8 安全管理》；

7. <资源限制子句> 使用 DROP PROFILE 删除关联的 profile 文件；使用 PROFILE <profile 名>指定关联的 profile 文件；使用 LIMIT <资源设置>直接设置资源设置项。关联 profile 文件后的用户，资源设置项由关联的 profile 文件统一管理配置，无法再通过使用 LIMIT <资源设置>直接设置资源设置项。直到通过修改用户指定 DROP PROFILE 解除关联或关联的 PROFILE 被级联删除之后，才可以使用 LIMIT <资源设置>设置；

8. <表空间配额子句> 不支持对系统表空间、临时表空间、回滚表空间设置配额；不支持对安全用户、审计用户设置配额。

举例说明

例 1 创建用户名为 BOOKSHOP_USER、口令为 DMsys_123、会话超时为 30 分钟的用户。

```
CREATE USER BOOKSHOP_USER IDENTIFIED BY DMsys_123 LIMIT CONNECT_TIME 30;
```

例 2 设置创建 user1，设置密码为过期。需进行重设才能使用。

```
//使用 SYSDBA 登录
CREATE USER user1 IDENTIFIED BY DMsys_123456 PASSWORD EXPIRE;
```

```
//使用 user1/DMsys_123456 登录。显示密码过期。无法进行下一步操作
SQL> conn user1/DMsys_123456@localhost:5236
服务器[localhost:5236]:处于普通打开状态
登录使用时间 : 7.617(ms)
口令剩余有效时间(天) : 0
口令是否过期 : 过期

//使用 SYSDBA 登录。重设密码
ALTER USER user1 IDENTIFIED BY DMsys_123456;

//使用 user1/DMsys_123456 登录。密码正常。可继续操作数据库
SQL> conn user1/DMsys_123456@localhost:5236
服务器[localhost:5236]:处于普通打开状态
登录使用时间 : 6.332(ms)
```

3.2.2 修改用户语句

修改数据库中的用户。

语法格式

```
ALTER USER <用户名> [<修改用户子句>] | [<用户代理功能子句>];
<修改用户子句> ::= [IDENTIFIED <身份验证模式>] [REPLACE <口令>] [PASSWORD_POLICY <口令策略>] [<锁定子句>] [<存储加密密钥>] [<只读标志>][<资源限制子句>][<密码过期子句>][DISCARD OLD PASSWORD] [<允许 IP 子句>][<禁止 IP 子句>][<允许时间子句>][<禁止时间子句>][<TABLESPACE 子句>][<INDEX_TABLESPACE 子句>][<SCHEMA 子句>][<表空间配额子句>]
```

<身份验证模式> ::= <数据库身份验证模式>|<外部身份验证模式>

<数据库身份验证模式> ::= BY <口令> [RETAIN CURRENT PASSWORD][<散列选项>]

<散列选项> ::= 参考 3.2.1 用户定义语句中的<散列选项>

<外部身份验证模式> ::= 参考 3.2.1 用户定义语句中的<外部身份验证模式>

<锁定子句> ::= ACCOUNT <LOCK | UNLOCK>

<存储加密密钥> ::= ENCRYPT BY <口令>

<只读标志> ::= [NOT] READ ONLY

<资源限制子句> ::= 参考 3.2.1 用户定义语句中的<资源限制子句>

<密码过期子句> ::= PASSWORD EXPIRE

<允许 IP 子句> ::=

ALLOW_IP NULL |

ALLOW_IP <IP 项>{,<IP 项>}

<禁止 IP 子句> ::=

NOT_ALLOW_IP NULL |

NOT_ALLOW_IP <IP 项>{,<IP 项>}

<IP 项> ::= <具体 IP> | <网段>

<允许时间子句> ::= ALLOW_DATETIME <时间项>{,<时间项>}

<禁止时间子句> ::= NOT_ALLOW_DATETIME <时间项>{,<时间项>}

<时间项> ::= 参考 3.2.1 用户定义语句中的<时间项>

<TABLESPACE 子句> ::=DEFAULT TABLESPACE <表空间名>

<INDEX_TABLESPACE 子句> ::= DEFAULT INDEX TABLESPACE <表空间名>

<SCHEMA 子句> ::= ON SCHEMA <模式名>

<表空间配额子句> ::= 参考 3.2.1 用户定义语句中的<表空间配额子句>

<用户代理功能子句> ::= <GRANT | REVOKE> CONNECT THROUGH <代理用户名>

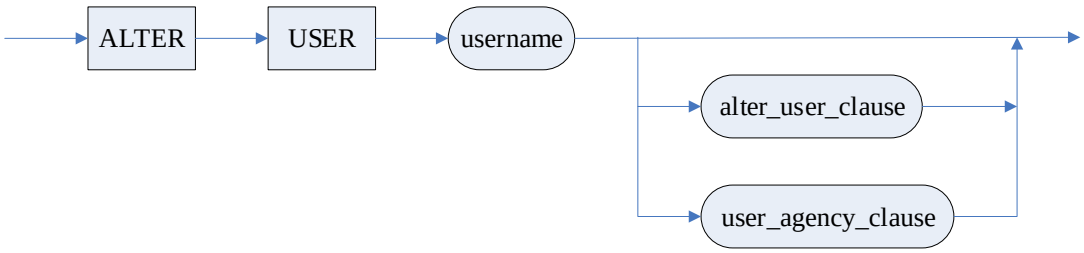
参数

用户代理功能子句用于赋予用户 B 能够以代理的身份认证登录用户 A 的权限，即 CONNECT THROUGH 权限。其中，用户 B 表示参数<代理用户名>指定的用户，用户 A 表示参数<用户名>指定的用户。关键字 GRANT 表示赋予权限，关键字 REVOKE 表示收回权限。

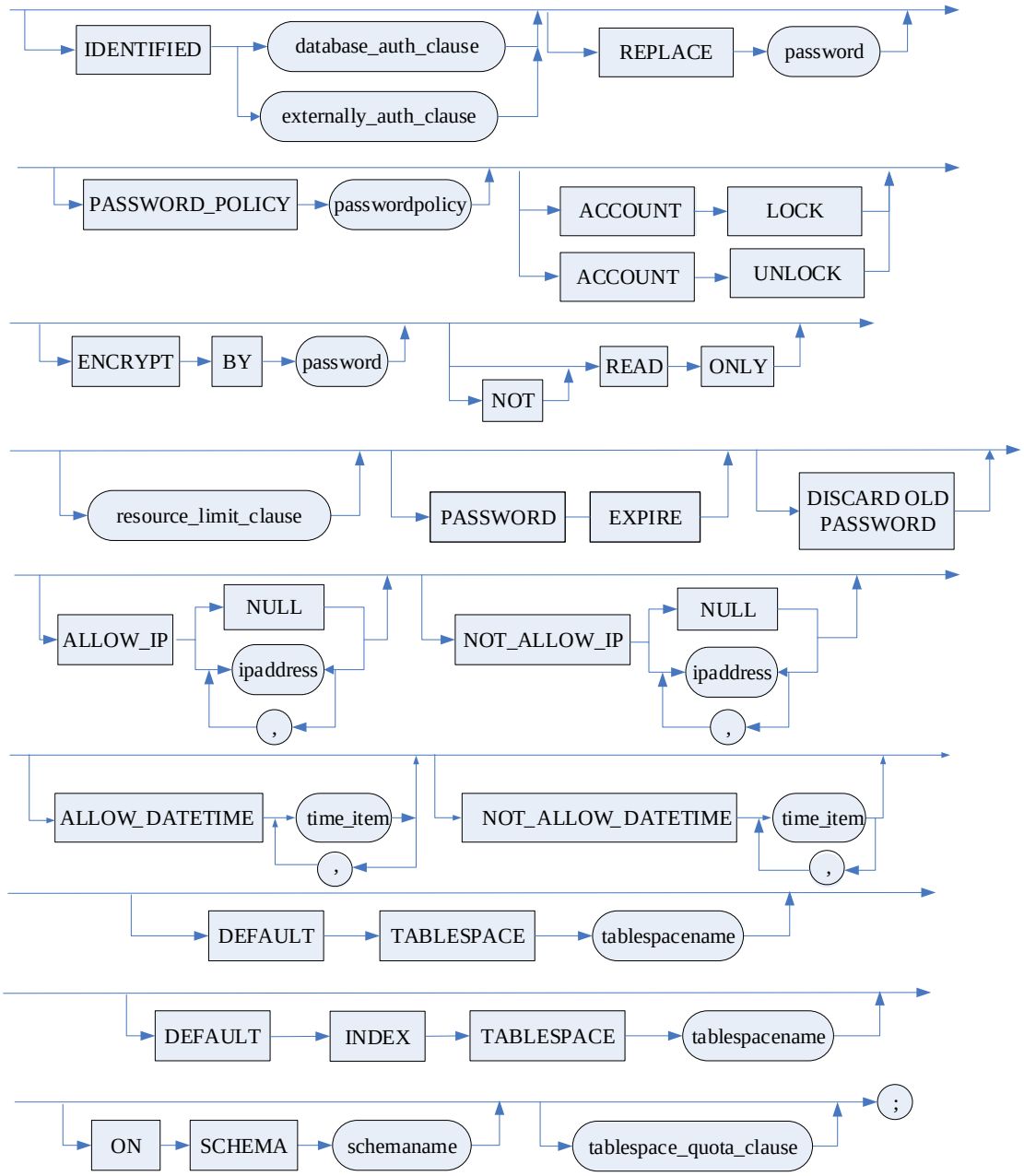
其余参数同用户定义语句的参数规定一样。

图例

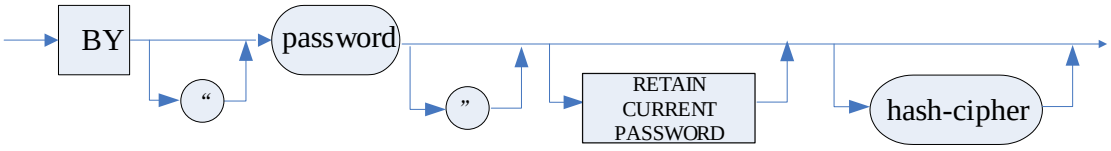
用户修改语句



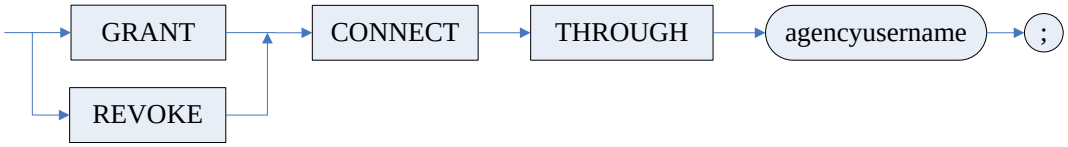
修改用户子句（alter_user_clause）



数据库身份验证模式（database_auth_clause）



用户代理功能子句（user_agency_clause）



语句功能

修改用户。SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 可以无条件修改同类型的用户的信息。普通用户如果想修改自己和其他用户信息，必须具有 ALTER USER 数据库权限。

使用说明

1．每个用户均可修改自身的口令，SYSDBA 用户可强制修改所有其他用户的口令(在数据库验证方式下)；

2．只有具备 ALTER USER 权限的用户才能修改其身份验证模式、系统角色及资源限制项；

3. 不论 DM.INI 的 DDL_AUTO_COMMIT 设置为自动提交还是非自动提交，ALTER USER 操作都会被自动提交；

4. 修改用户口令时，口令策略应符合创建该用户时指定的口令策略；

5. 修改用户口令语句中，RETAIN CURRENT PASSWORD 表示保留原始口令，同时增加新的辅助口令；

6. DISCARD OLD PASSWORD 表示有两种口令的情况下，废弃原始口令；

7. 原始口令为空的用户无法 RETAIN CURRENT PASSWORD；

8. 用户存在原始口令时，未指定 RETAIN CURRENT PASSWORD 的情况下更改口令，原始口令保持不变；

9. 修改自己用户的口令，需要有 APPLICATION_PASSWORD_ADMIN 权限才能使用保留原始口令和失效原始口令的操作；

10. 修改其他用户的口令，需要有 ALTER USER 权限；

11．不能修改系统固定用户的系统角色；

12. 不能修改系统固定用户为只读；

13. REPLACE <口令> 用于在修改数据库身份验证模式下的用户口令时，指定用户原始口令，与 INI 参数 PASSWORD_VERIFICATION 有关，该参数缺省为 0。当 INI 参数 PASSWORD_VERIFICATION 为 1 时，用户修改自己的口令时必须指定 REPLACE 语句，并

且在指定后，会校验原口令正确性，如果原始口令校验不通过则无法修改用户口令；拥有修改当前用户权限的其他用户在修改当前用户口令时可以不指定 REPLACE 子句，指定时不会校验原始口令。当 INI 参数 PASSWORD_VERIFICATION 为 0 时，可选择是否指定 REPLACE 子句；若用户修改自己的口令时指定了 REPLACE 子句，会校验原始口令的正确性；

14. <SCHEMA 子句>用于设置用户的缺省模式；

15. 针对用户 B 以代理的身份认证登录用户 A 的权限，即 CONNECT THROUGH 权限，需注意：

1）只有拥有 DBA 角色权限的用户（DBA/DB_POLICY_ADMIN/DB_AUDIT_ADMIN/DB_OBJECT_ADMIN）才能赋予用户 B 该权限；

2）只有赋予用户 B 以 CONNECT THROUGH 权限后，用户 B 才能以代理的身份认证登录用户 A；

3）用户 A、B 可以为 SYSDBA 用户，且用户 A、B 可以为同一用户；

4）如果用户 A 或用户 B 不存在，则报错；

5）用户 A、B 类型（普通/安全/审计/OBJECT）必须相同，否则报错；

6）系统表 SYSGRANTS 中查不到 CONNECT THROUGH 权限；

7）无法对三权分立和四权分立的角色设置 CONNECT THROUGH 权限；

8）CONNECT THROUGH 权限无法和用户其他属性（如口令策略）同时设置。

16．<资源限制子句> 使用 DROP PROFILE 删除关联的 profile 文件；使用 PROFILE <profile 名>指定关联的 profile 文件；使用 LIMIT <资源设置>直接设置资源设置项，此处只需要设置要修改的设置项即可，其他设置项值保持不变；

17. <密码过期子句> 用于设置密码过期；

18. <表空间配额子句> 新修改的配额限制不会影响用户已经使用的表空间；若用户拥有 UNLIMITED TABLESPACE 权限，则对该用户指定<表空间配额子句>无效，该用户始终对

所有表空间均无配额限制；

19. 其他参数的取值、意义与 CREATE USER 中的要求一样。

举例说明

例 1 修改用户 BOOKSHOP_USER，会话空闲期为无限制，最大连接数为 10。

```
ALTER USER BOOKSHOP_USER LIMIT SESSION_PER_USER 10, CONNECT_IDLE_TIME UNLIMITED;
```

例 2 赋予用户 USER2 代理权限，使用户 USER2 可以认证登录用户 USER1。

```
ALTER USER USER1 GRANT CONNECT THROUGH USER2;
```

3.2.3 用户删除语句

删除用户。

语法格式

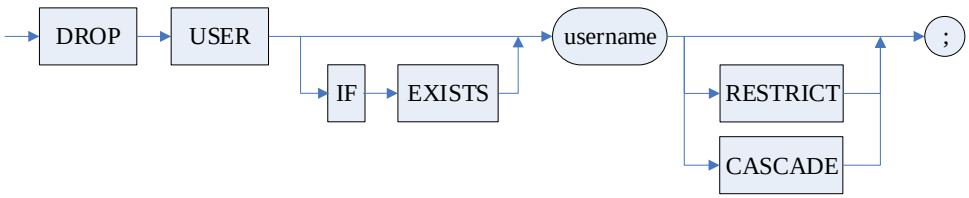
```
DROP USER [IF EXISTS] <用户名> [RESTRICT | CASCADE];
```

参数

<用户名> 指明被删除的用户。

图例

用户删除语句



语句功能

删除指定用户。删除用户的操作一般由 SYSDBA、SYSSSO、SYSAUDITOR 和 SYSDBO 完成，他们可以删除同类型的其他用户。普通用户要删除其他用户，必须具有 DROP USER 权限。

使用说明

1．系统自动创建的四个系统用户 SYSDBA、SYSAUDITOR、SYSSSO 和 SYSDBO 不能被删除；

2．具有 DROP USER 权限的用户即可进行删除用户操作；

3．执行此语句将导致 DM 删除数据库中该用户建立的所有对象，且不可恢复。如果要保存这些实体，请参考 REVOKE 语句；

4. 删除不存在的用户会报错。若指定 IF EXISTS 关键字，删除不存在的用户，不会报错；

5．如果未使用 CASCADE 选项，若该用户建立了数据库对象（如表、视图、过程或函数），或其他用户对象引用了该用户的对象，或在该用户的表上存在其它用户建立的视图，DM 将返回错误信息，而不删除此用户；

6．如果使用了 CASCADE 选项，除数据库中该用户及其创建的所有对象被删除外，如果其他用户创建的表引用了该用户表上的主关键字或唯一关键字，或者在该表上创建了视图，DM 还将自动删除相应的引用完整性约束及视图依赖关系；

7．正在使用中的用户可以被删除，删除后重登录或者做操作会报错。

举例说明

例 删除用户 BOOKSHOP_USER。

```
DROP USER BOOKSHOP_USER;
```

3.3管理模式

3.3.1 模式定义语句

模式定义语句创建一个架构，并且可以在概念上将其看作是包含表、视图和权限定义的对象。在 DM 中，一个用户可以创建多个模式，一个模式中的对象(表、视图)可以被多个用户使用。

系统为每一个用户自动建立了一个与用户名同名的模式作为默认模式，用户还可以用模

式定义语句建立其它模式。

语法格式

```
<模式定义子句 1> | <模式定义子句 2>

<模式定义子句 1> ::=

    CREATE SCHEMA <模式名> [AUTHORIZATION <用户名>][<DDL_GRANT 子句> {< DDL_GRANT 子句>}];

<模式定义子句 2> ::=

    CREATE SCHEMA AUTHORIZATION <用户名> [<DDL_GRANT 子句> {< DDL_GRANT 子句>}];

<DDL_GRANT 子句> ::=

    <表定义> | <域定义>| <基表修改> | <索引定义> | <视图定义> | <序列定义> | <存储过程定义> | <存储函数定义> | <外部函数定义> |<触发器定义> | <特权定义> | <全文索引定义> | <同义词定义> | <包定义> | <包体定义> | <类定义> | <类体定义> | <外部链接定义>] | <物化视图定义> | <物化视图日志定义> | <注释定义> | <自定义运算符定义>
```

参数

1. <模式名> 指明要创建的模式的名字，最大长度 128 字节；
2. <表定义> 建表语句；
3. <域定义> 域定义语句；
4. <基表修改> 基表修改语句；
5. <索引定义> 索引定义语句；
6. <视图定义> 建视图语句；
7. <序列定义> 建序列语句；
8. <存储过程定义> 存储过程定义语句；
9. <存储函数定义> 存储函数定义语句；
10. <外部函数定义> 外部函数定义语句；

11. <触发器定义> 建触发器语句;

12. <特权定义> 授权语句;

13. <全文索引定义> 全文索引定义语句;

14. <同义词定义> 同义词定义语句;

15. <包定义> 包定义语句;

16. <包体定义> 包体定义语句;

17. <类定义> 类定义语句;

18. <类体定义> 类体定义语句;

19. <外部链接定义> 外部链接定义语句;

20. <物化视图定义> 物化视图定义语句;

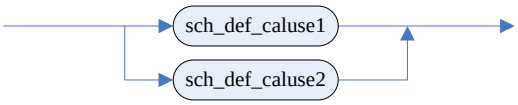
21. <物化视图日志定义> 物化视图日志定义语句;

22. <注释定义> 注释定义语句;

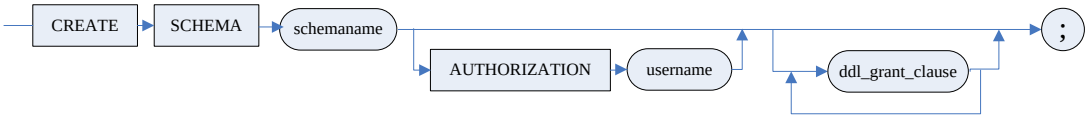
23. <自定义运算符定义> 自定义运算符定义语句。

图例

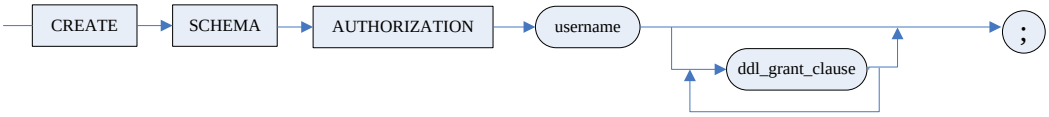
模式定义语句



sch_def_clause1



sch_def_clause2



ddl_grant_clause: 略

语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE SCHEMA 或 CREATE ANY SCHEMA 权限的用户在指定数据库中定义模式。

使用说明

1. 在创建新的模式时，如果此模式所属用户下已存在同名的模式，那么创建模式的操作会被跳过，而如果后续还有 DDL 子句，根据权限判断是否可在已存在模式上执行这些 DDL 操作；

2. AUTHORIZATION <用户名> 指定拥有该模式的用户。缺省为 SYSDBA。创建者为 SYSDBA 用户才可以使用此选项。非 DBA 用户（普通用户）不能使用该选项创建其他用户拥有的模式；

3. 使用 sch_def_clause2 创建模式时，模式名与用户名相同；

4. 使用该语句的用户必须具有 DBA 或 CREATE SCHEMA 权限；

5. DM 使用 DMSQL 程序模式执行创建模式语句，因此创建模式语句中的标识符不能使用系统的保留字；

6. 定义模式时，用户可以用单条语句同时建多个表、视图，同时进行多项授权；

7. 模式一旦定义，该用户所建表、视图等均属该模式，其它用户访问该用户所建立的表、视图等均需在表名、视图名前冠以模式名；而建表者访问自己当前模式所建表、视图时模式名可省；若没有指定当前模式，系统自动以当前用户名作为模式名；

8. 模式定义语句中的基表修改子句只允许添加表约束；

9. 模式定义语句中的索引定义子句不能定义聚集索引；

10. 模式未定义之前，其它用户访问该用户所建的表、视图等均需在表名前冠以建表者名；

11. 模式定义语句不允许与其它 SQL 语句一起执行；

12. 在 DIsql 中使用该语句必须以“/”结束。

举例说明

例 用户 SYSDBA 创建模式 SCHEMA1，建立的模式属于 SYSDBA。

```
CREATE SCHEMA SCHEMA1 AUTHORIZATION SYSDBA;
```

3.3.2 设置当前模式语句

设置当前模式。

语法格式

```
SET SCHEMA <模式名>;
```

图例

设置当前模式语句



语句功能

供具有 DBA 权限的用户设置当前模式语句。

举例说明

例 SYSDBA 用户将当前的模式从 SYSDBA 换到 SALES 模式。

```
SET SCHEMA SALES;
```

3.3.3 模式删除语句

在 DM 系统中，允许用户删除整个模式。

语法格式

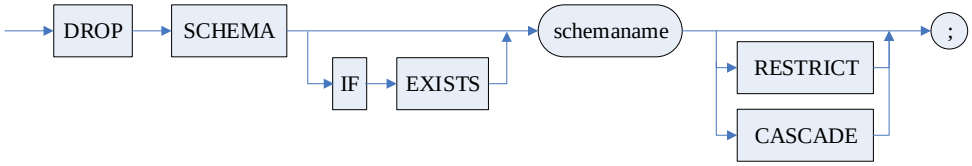
```
DROP SCHEMA [IF EXISTS] <模式名> [RESTRICT | CASCADE];
```

参数

<模式名> 指要删除的模式名。

图例

模式删除语句



语句功能

供具有 DBA 角色（三权分立）或 DROP ANY SCHEMA 权限的用户修改表空间。

使用说明

1. 删除不存在的模式会报错。若指定 IF EXISTS 关键字，删除不存在的模式，不会报错；

2. 用该语句的用户必须具有 DBA 权限或是该模式的所有者；

3. 如果使用 RESTRICT 选项，只有当模式为空时删除才能成功，否则，当模式中存在数据库对象时则删除失败。默认选项为 RESTRICT 选项；

4. 如果使用 CASCADE 选项，则将整个模式、模式中的对象，以及与该模式相关的依赖关系都删除。

举例说明

例 以 SYSDBA 身份登录数据库后，删除 BOOKSHOP 库中模式 SCHEMA1。

```
DROP SCHEMA SCHEMA1 CASCADE;
```

3.4 管理表空间

3.4.1 表空间定义语句

创建表空间。

DM 数据库中的表空间可以分为普通表空间和混合表空间。使用<HUGE 路径子句>创建的表空间为混合表空间，未使用<HUGE 路径子句>创建的表空间即为普通表空间。普通表空间只能存储普通表（非 HUGE 表）；而混合表空间既可以存储普通表又可以存储 HUGE 表。

语法格式

```
CREATE TABLESPACE [IF NOT EXISTS] <表空间名> <数据文件子句>[<数据页缓冲池子句>][<存储加密子句>][<指定 ASM 存储子句>] [<HUGE 路径子句>][<STORAGE 子句>]

<STORAGE 子句> ::=
    STORAGE (ON <RAFT 组名>) |
    STORAGE (ON <BP 组名>)
<数据文件子句> ::= DATAFILE <文件说明项>{,<文件说明项>}
<文件说明项> ::= <文件路径> [ MIRROR <文件路径>] SIZE <文件大小>[<自动扩展子句>]
<自动扩展子句> ::= AUTOEXTEND <ON [<每次扩展大小子句>][<最大大小子句>] |OFF>
<每次扩展大小子句> ::= NEXT <扩展大小>
<最大大小子句> ::=
    MAXSIZE <文件最大大小>  |
    UNLIMITED
<数据页缓冲池子句> ::= CACHE = <缓冲池名>
<存储加密子句> ::= ENCRYPT WITH <加密算法> <密码子句>
<密码子句>::=BY <加密密码> |
            BY WRAPPED <加密密码密文>
<指定 ASM 存储子句>: :  = [<条带化类型子句>][<镜像类型子句>]
<条带化类型子句> ::= STRIPING<0 | 32 | 64 | 128 | 256>
<镜像类型子句> ::= HIGH | NORMAL | EXTERNAL
<HUGE 路径子句> ::= WITH HUGE PATH <HUGE 数据文件路径>
```

参数

1. <表空间名> 表空间的名称，最大长度 128 字节；

2. <文件路径> 指明新生成的数据文件在操作系统下的路径 + 新数据文件名。数据文件的存放路径符合 DM 安装路径的规则，若指定目录不存在则自动创建相应目录。若路径是相对路径，仅支持路径起始位置的当前目录的相对路径“./”。例如：支持“./data/TS.DBF”，不支持“../data/TS.DBF”、“/./TS.DBF”等；DSC 镜像环境下若创建表空间时指定多个 ASM

文件路径，要求数据文件必须属于同一磁盘组；

3. MIRROR 数据文件镜像，用于在数据文件出现损坏时替代数据文件进行服务；MIRROR 数据文件的<文件路径>必须是绝对路径。要使用数据文件镜像，必须在建库时开启页校验的参数 PAGE_CHECK；

4. <文件大小> 整数值，指明新增数据文件的大小，单位 MB，取值范围 4096*页大小~2147483647*页大小；存在特殊情况：若控制系统中为表空间提前预留的簇个数的 INI 参数 TS_RESERVED_EXTENTS 取缺省值 64，且数据库初始化时指定簇大小 EXTENT_SIZE 为 64，此时表空间预留簇个数占用空间大小正好等于表空间大小的理论最小值（4096*页大小）；而新创建的表空间至少要占用 1 个簇，由于当前剩余的空间不满足 TS_RESERVED_EXTENTS 参数指定的预留簇个数要求，系统会按照 INI 参数 TS_AUTO_EXTEND_SIZE 的值，自动扩展表空间大小直至满足预留簇个数所需的空间要求；

5. <每次扩展大小子句>指明数据库文件每次扩展的大小，单位 MB，取值范围为 1~2048。如果不指定此子句，数据库文件也会自动扩展，每次扩展的大小由 INI 参数 TS_AUTO_EXTEND_SIZE 控制；

6. <最大大小子句>指明数据库文件的最大大小，单位 MB，如果不指定此子句，则为无限制；

7. <缓冲池名> 系统数据页缓冲池名 NORMAL 或 KEEP。缓冲池名 KEEP 是 DM 的保留关键字，使用时必须加双引号；

8. <加密算法> 可以是系统内置的加密算法也可以是第三方加密算法，详情请参考手册《DM8 安全管理》；

9. <加密密码> 必须满足长度大于等于 INI 参数 PWD_MIN_LEN 设置的值，同时不超过 32，且包含大写、小写、数字。若未指定，由 DM 随机生成。

<加密密码密文>提供一种创建和现有的加密表/表空间一样密钥的表/表空间的语法，

和 WRAPPED 关键词搭配使用。加密密码密文可通过查看现有的加密表/表空间定义语句获取。例如：select dbms_metadata.get_ddl('TABLESPACE','TS03');查看到的 CREATE TABLESPACE "TS03" DATAFILE '/data/sdb/DAMENG/ts03.dbf' SIZE 32 AUTOEXTEND ON NEXT 0 MAXSIZE 16777215 CACHE = "NORMAL" ENCRYPT WITH OPENSSL_SM4_OFB_V1 BY WRAPPED '0xDEFBACCB9F0D15FB420717215C138B3D55E83D2A72B528751EF40D0B4719B540160A90456BACB7A225BE040AB4FCF84E36BD58C30C708607' COPY 0 MICRO WITH HUGE PATH '/data/sdb/DAMENG/ts03' MICRO; 。则'0xDEFBACCB9F0D15FB420717215C138B3D55E83D2A72B528751EF40D0B4719B540160A90456BACB7A225BE040AB4FCF84E36BD58C30C708607'即为加密密码密文。

10. <指定 ASM 存储子句>专门用于 DMASM 镜像环境，用于指定表空间数据文件的条带化粒度和镜像类型。

<条带化类型子句>用于指定数据文件条带化粒度。取值 0、32、64、128 和 256，缺省值 32，单位 KB。取值 0 表示粗粒度，粗粒度条带是将文件按 AU 大小分割为一个个数据块。取值 32~256 表示细粒度，细粒度条带是将文件按照条带化粒度大小分割为一个个数据块。

<镜像类型子句>用于指定数据文件镜像类型，即文件的副本数。取值 EXTERNAL、NORMAL、HIGH，分别表示单副本、两副本和三副本，缺省值为 ASM 数据文件所属磁盘组的副本数。

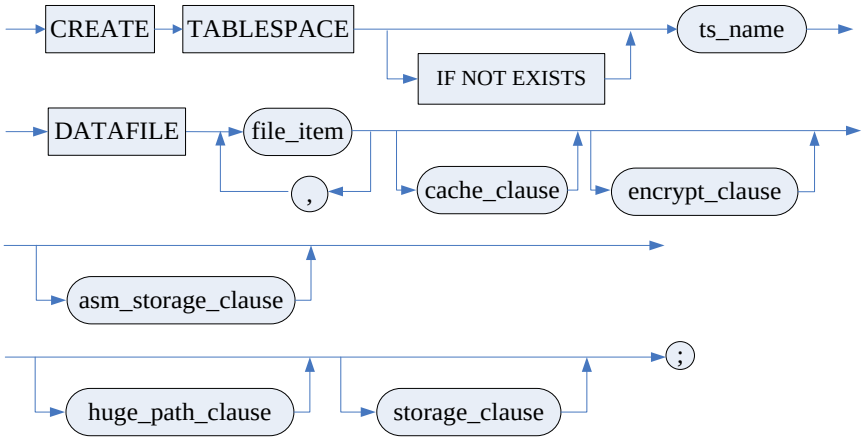
11. <HUGE 路径子句> 用于创建一个混合表空间。HUGE 数据文件存储在<HUGE 路径子句>指定的路径中，普通（非 HUGE）数据文件存储在<数据文件子句>指定的路径中。

12. <STORAGE 子句> DMDPC 专用，对单节点如果使用了<STORAGE 子句>会直接忽略。通过<RAFT 组名>或<BP 组名>指定一个 RAFT 组作为表空间的存储位置。RAFT 组名或 BP 组名必须是已存在的组名。<表空间名>、<RAFT 组名> 、 <BP 组名>三者不能同名。

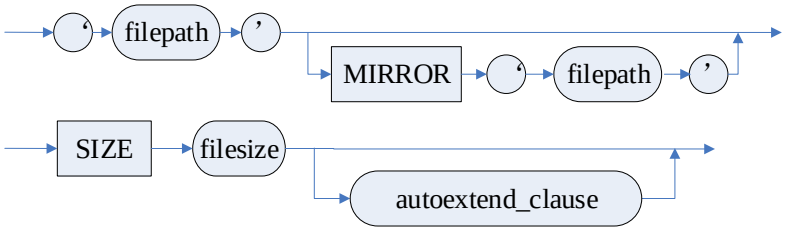
若指定的是<RAFT 组名>，则为明确指定 RAFT 组名。若指定的是<BP 组名>，则系统会从指定的 BP 组中随机挑选一个 RAFT 组。如果<STORAGE 子句>缺省，即<RAFT 组名>和<BP 组名>均未指定，则系统会从现有的 BP RAFT 组中随机挑选一个作为表空间的存储位置。

图例

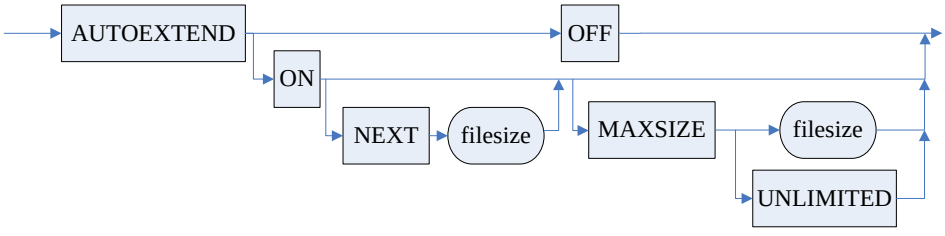
表空间定义语句



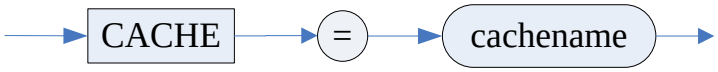
文件说明项（file_item）



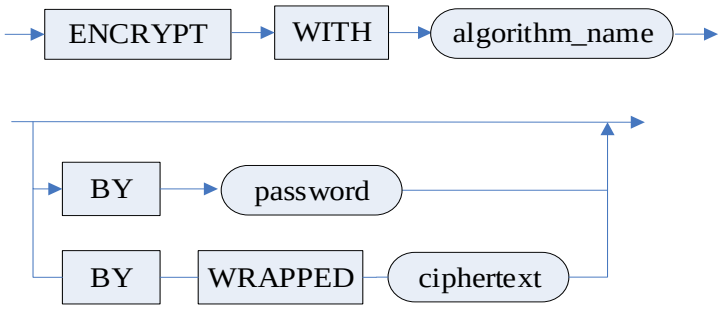
自动扩展子句（autoextend_clause）



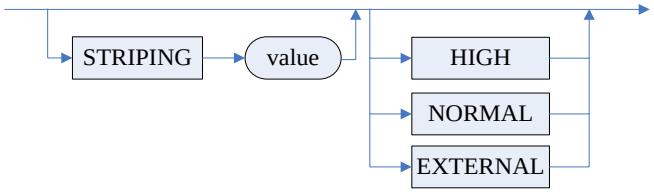
数据页缓冲池子句（cache_clause）



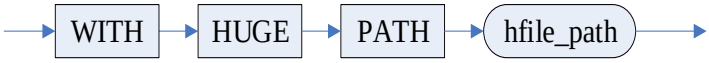
存储加密子句（encrypt_clause）



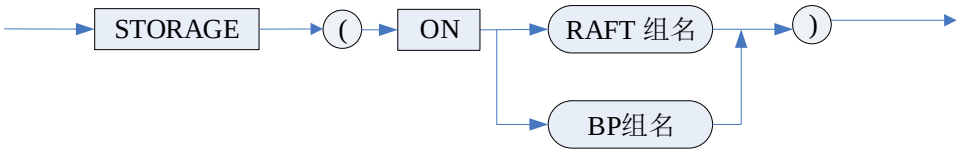
指定 ASM 存储子句（asm_storage_clause）



HUGE 路径子句（huge_path_clause）



STORAGE 子句（storage_clause）



语句功能

供具有 DBA 角色 或具有 CREATE TABLESPACE 权限的用户定义表空间。

使用说明

1. 表空间名在数据库中必须唯一。如果表空间名已存在则报错。若指定 IF NOT EXISTS 关键字，表空间名已存在不报错，忽略本次表空间创建操作；

2. 一个表空间中，数据文件和镜像文件的个数一起不能超过 INI 参数 TS_FIL_MAX_ID 设置的值；

3. 如果全库已经加密，就不再支持表空间加密；

4. SYSTEM 表空间、TEMP 表空间不允许关闭自动扩展；

5. TEMP 表空间相关信息记录在 dmtemp.ctl 控制文件中，DM 服务器启动时根据 dmtemp.ctl 中的内容构建 TEMP 表空间数据文件。

6. DSC 镜像环境下若创建表空间时指定多个 ASM 文件路径，要求数据文件必须属于同一磁盘组。

举例说明

例 1 以 SYSDBA 身份登录数据库后，创建表空间 TS1，指定数据文件 TS1.dbf，大小 128M。

```
CREATE TABLESPACE TS1 DATAFILE 'd:\TS1.dbf' SIZE 128;
```

例 2 在基于 DMASM 镜像的 DMDSC 中，创建一个数据文件条带化粒度为 128KB、镜像类型为 HIGH(三副本)的表空间。

```
CREATE TABLESPACE TS1 DATAFILE '+DATA/dsc_mirror/test.dbf' SIZE 128 STRIPING 128 HIGH;
```

3.4.2 修改表空间语句

修改表空间。

语法格式

```
ALTER TABLESPACE <表空间名> [ONLINE | OFFLINE | CORRUPT|<表空间重命名子句> | <数据
文件重命名子句>|<增加数据文件子句>|<修改文件大小子句>|<修改文件自动扩展子句>|<数据页缓冲池
子句>|<DSC 集群表空间负载均衡子句>|<增加 HUGE 路径子句>|<删除表空间文件>]
<表空间重命名子句> ::= RENAME TO <表空间名>
<数据文件重命名子句>::= RENAME DATAFILE <文件路径>{,<文件路径>} TO <文件路径>{,<文件路径>}
<增加数据文件子句> ::= ADD <数据文件子句>
<数据文件子句> ::= 参考 3.4.1 表空间定义语句中的<数据文件子句>
<修改文件大小子句> ::= RESIZE DATAFILE <文件路径> TO <文件大小> [ON RAFT_NAME]
<修改文件自动扩展子句> ::= DATAFILE <文件路径>{,<文件路径>}[<自动扩展子句>]
<自动扩展子句> ::= 参考 3.4.1 表空间定义语句中的<自动扩展子句>
```

```
<数据页缓冲池子句> ::= CACHE = <缓冲池名>

<DSC 集群表空间负载均衡子句> ::= OPTIMIZE <DSC 集群节点号>

<增加 HUGE 路径子句> ::= ADD HUGE PATH <HUGE 数据文件路径>

<删除表空间文件>::=DROP DATAFILE <文件路径>
```

参数

1. <表空间名> 表空间的名称;

2. ONLINE | OFFLINE | CORRUPT 表示表空间的状态。ONLINE 为联机状态，ONLINE 时才允许用户访问该表空间中的数据；OFFLINE 为脱机状态， OFFLINE 时不允许访问该表空间中的数据；CORRUPT 为损坏状态，当表空间处于 CORRUPT 状态时，只有被还原恢复后才能提供服务，否则不能使用只能删除。三种状态的相互转换情况：ONLINE←→OFFLINE→CORRUPT。

3. <文件路径> 指明数据文件在操作系统下的路径 + 新数据文件名。数据文件的存放路径符合 DM 安装路径的规则，且该路径必须是已经存在的；

4. <文件大小> 整数值，指明新增数据文件的大小，单位 MB;

5. ON RAFT_NAME 仅在 DPC 环境下有效，非 DPC 环境下直接忽略指定的 RAFT_NAME；且 DPC 环境下，若修改非临时表空间时指定了 RAFT_NAME，也直接忽略。DPC 环境下连接 SP/BS 时，可指定修改单个节点所在的 RAFT（包括 SP/MP/BP/BS）临时表空间文件的大小，不指定时则广播修改除 MP 以外的 RAFT 临时表空间文件的大小；直连 MP 时无论是否指定 RAFT，都只修改 MP 本地临时文件大小;

6. <缓冲池名> 系统数据页缓冲池名 NORMAL 或 KEEP;

7. <增加 HUGE 路径子句> 增加一个 HUGE 数据文件路径。最多可添加 127 个 HUGE 数据文件路径;

8. <删除表空间文件> 删除表空间中某一路径对应的数据文件。删除的前提条件为：表空间必须处于 ONLINE 状态;数据库必须是 PRIMARY 或者 NORMAL 模式,并且处于 OPEN

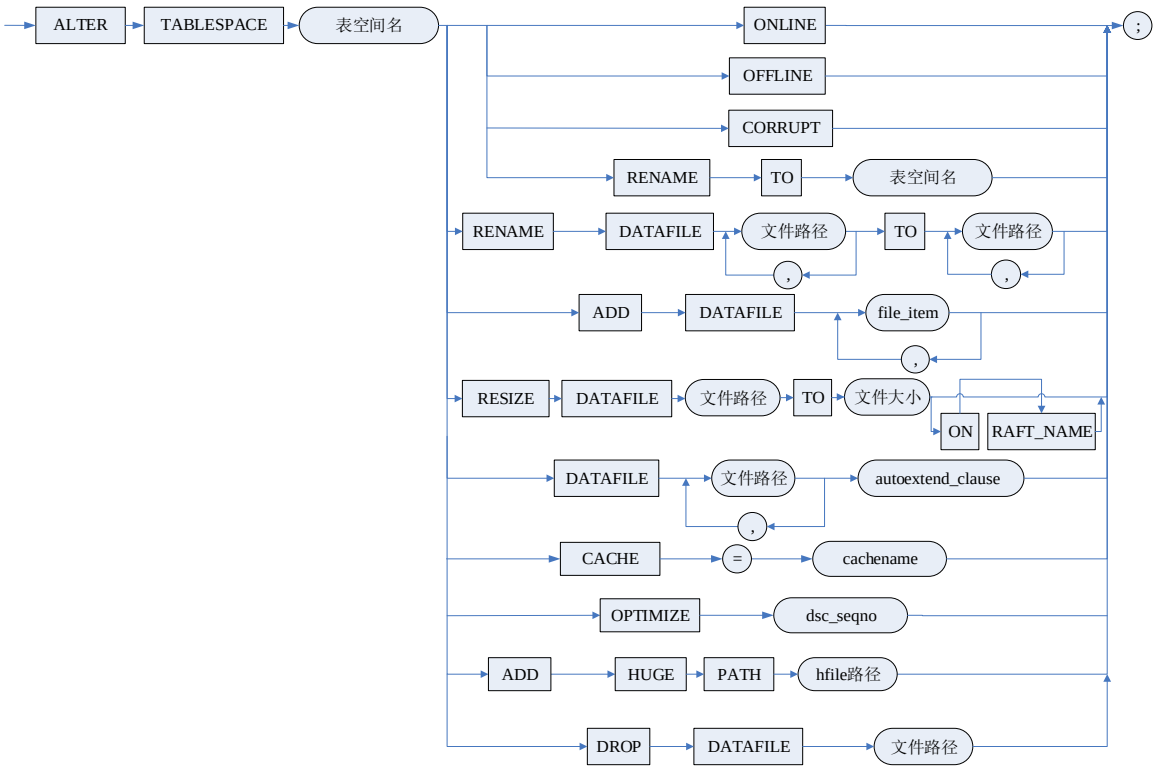
状态；不支持 SYSTEM、回滚表空间；联机日志不能删除 0 号和 1 号文件；其它表空间则不能删除 0 号文件或表空间中唯一文件；必须先删除最大文件 ID 的文件；执行文件删除前必须删除从该文件分配了簇的数据库对象；

9. <修改文件大小子句> 将表空间中某一路径对应的数据文件进行体积更改。对文件大小进行缩减的前提条件为：表空间必须处于 ONLINE 状态；数据库必须处于 OPEN 状态。另外，不支持缩减联机日志；只有当指定偏移之后的簇被释放后截断操作才能执行，因而操作并不总是能够截断到指定偏移；

10. <数据文件重命名子句>：源文件路径无要求，对于 TO 后的目标文件路径，若路径是相对路径，仅支持路径起始位置的当前目录的相对路径“./”。例如：支持“./data/TS.DBF”，不支持“../data/TS.DBF”、“/./TS.DBF”等。

图例

修改表空间语句



语句功能

供具有 DBA 角色或 ALTER TABLESPACE 权限的用户修改表空间。

使用说明

1. 不论 DM.INI 的 DDL_AUTO_COMMIT 设置为自动提交还是非自动提交，ALTER TABLESPACE 操作都会被自动提交；

2. SYSTEM 表空间、TEMP 表空间不允许关闭自动扩展；

3. 如果表空间有未提交事务时，表空间不能修改为 OFFLINE 状态；

4. 重命名表空间数据文件时，表空间必须处于 OFFLINE 状态，修改成功后再将表空间修改为 ONLINE 状态；

5. 表空间如果发生损坏（表空间还原失败，或者数据文件丢失或损坏）的情况下，允许将表空间切换为 CORRUPT 状态，并删除损坏的表空间，如果表空间上定义有对象，需要先将所有对象删除，再删除表空间；

6. DSC 集群表空间负载均衡子句用于在 DSC 集群环境中进行基于表空间的负载均衡设置，可指定优化节点号，当 INI 参数 DSC_TABLESPACE_BALANCE 为 1 时，符合条件的查询语句会被自动重连至<DSC 集群节点号>指定的节点执行，从而实现负载均衡。当指定的<DSC 集群节点号>为非法节点号时，此表空间的优化节点失效；

7. 对普通表空间使用<增加 HUGE 路径子句>，可将普通表空间升级为混合表空间；对混合表空间使用<增加 HUGE 路径子句>，可为混合表空间添加新的 HUGE 数据文件路径。

举例说明

例 1 将表空间 TS1 名字修改为 TS2。

```
ALTER TABLESPACE TS1 RENAME TO TS2;
```

例 2 增加一个路径为 d:\TS1_1.dbf，大小为 128M 的数据文件到表空间 TS1。

```
ALTER TABLESPACE TS1 ADD DATAFILE  'd:\TS1_1.dbf' SIZE 128;
```

例 3 修改表空间 TS1 中数据文件 d:\TS1.dbf 的大小为 200M。

```
ALTER TABLESPACE TS1 RESIZE DATAFILE  'd:\TS1.dbf' TO 200;
```

例 4 重命名表空间 TS1 的数据文件 d:\TS1.dbf 为 e:\TS1_0.dbf。

```
ALTER TABLESPACE TS1 OFFLINE;

ALTER TABLESPACE TS1 RENAME DATAFILE 'd:\TS1.dbf' TO 'e:\TS1_0.dbf';

ALTER TABLESPACE TS1 ONLINE;
```

例 5 修改表空间 TS1 的数据文件 d:\TS1.dbf 自动扩展属性为每次扩展 10M，最大文件大小为 1G。

```
ALTER TABLESPACE TS1 DATAFILE  'd:\TS1.dbf' AUTOEXTEND ON NEXT 10 MAXSIZE 1000;
```

例 6 修改表空间 TS1 缓冲池名字为 KEEP。

```
ALTER TABLESPACE TS1 CACHE="KEEP";
```

例 7 修改表空间为 CORRUPT 状态，注意只有在表空间处于 OFFLINE 状态或表空间损坏的情况下才允许使用。

```
ALTER TABLESPACE TS1 CORRUPT;
```

例 8 为表空间 TS1 添加 HUGE 数据文件路径

```
ALTER TABLESPACE TS1 ADD HUGE PATH 'D:\dmdbms\data\DAMENG\TS1\HUGE2';
```

3.4.3 表空间删除语句

删除表空间。

语法格式

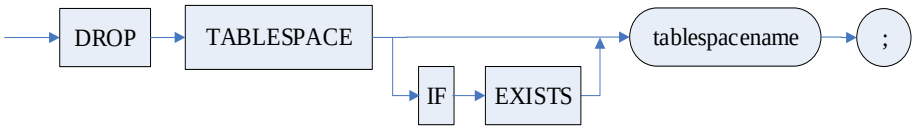
```
DROP TABLESPACE [IF EXISTS] <表空间名>
```

参数

<表空间名> 所要删除的表空间的名称。

图例

表空间删除语句



语句功能

供具有 DBA 角色或 DROP TABLESPACE 权限的用户删除表空间。

使用说明

1. 删除不存在的表空间会报错。若指定 IF EXISTS 关键字，删除不存在的表空间，不会报错;

2. SYSTEM、RLOG、ROLL 和 TEMP 表空间不允许删除;

3. 系统处于 SUSPEND 或 MOUNT 状态时不允许删除表空间，系统只有处于 OPEN 状态下才允许删除表空间。

举例说明

例 以 SYSDBA 身份登录数据库后，删除表空间 TS1。

```
DROP TABLESPACE TS1;
```

3.4.4 表空间失效文件检查

表空间恢复失效文件的检查。

语法格式

```
SP_FILE_SYS_CHECK ();
```

语句功能

在 LINUX 操作系统下，检查是否有数据文件被删除。

使用说明

该过程只在 LINUX 下有效。

举例说明

```
SP_FILE_SYS_CHECK ();
```

3.4.5 表空间失效文件恢复准备

表空间恢复失效文件的准备。

语法格式

```
SP_TABLESPACE_PREPARE_RECOVER(<表空间名>);
```

语句功能

在 LINUX 操作系统下，如果出现了正在使用数据文件被删除的情况，该过程完成失效文件恢复的准备工作。

使用说明

该过程只在 LINUX 下有效。

举例说明

```
SP_TABLESPACE_PREPARE_RECOVER('MAIN');
```

3.4.6 表空间失效文件恢复

表空间失效文件的恢复。

语法格式

```
SP_TABLESPACE_RECOVER(<表空间名>);
```

语句功能

在 LINUX 操作系统下，如果出现了正在使用数据文件被删除的情况，在调用了恢复准备的 SP_TABLESPACE_PREPARE_RECOVER 及在 OS 系统内完成了数据文件的复制后，调用该过程完成文件的恢复工作。表空间失效文件恢复的详细步骤可参考《DM8 系统管理员手册》。

使用说明

1. 该过程只在 LINUX 下有效;

2. 在 SP_TABLESPACE_PREPARE_RECOVER 及在 OS 系统内完成了数据文件的复制后调用。

举例说明

```
SP_TABLESPACE_RECOVER('MAIN');
```

3.5 管理表

3.5.1 表定义语句

用户数据库建立后，就可以定义基表来保存用户数据的结构。DM 数据库的表可以分为两类，分别为数据库内部表和外部表，数据库内部表由数据库管理系统自行组织管理，而外部表在数据库的外部组织，是操作系统文件。其中内部表包括：数据库基表、HUGE 表和水平分区表。手册中如无明确说明均指数据库基表。下面分别对这各种表的创建与使用进行详细描述。

3.5.1.1 定义数据库基表

用户数据库建立后，就可以定义基表来保存用户数据的结构。需指定如下信息：

1. 表名、表所属的模式名;

2. 列定义;

3. 完整性约束。

语法格式

```
CREATE [[GLOBAL] TEMPORARY] TABLE [IF NOT EXISTS] <表名定义> <表结构定义>;

<表名定义> ::= [<模式名>.] <表名>

<表结构定义>::=<表结构定义 1> | <表结构定义 2>

<表结构定义 1>::= (<列定义> {,<列定义>} [,<表级约束定义>{,<表级约束定义>}])[<属性子句>] [<延
```

迟段分配子句>] [<压缩子句>][表并行度子句][<高级日志子句>] [<add_log 子句>] [<DISTRIBUTE 子句>][<AUTO_INCREMENT 子句>]

<表结构定义 2>::= [<属性子句>] [<延迟段分配子句>] [<压缩子句>][表并行度子句]AS <不带 INTO 的 SELECT 语句>[<add_log 子句>] [<DISTRIBUTE 子句>];

<列定义> ::= <不同类型列定义> [<列定义子句>] [<STORAGE 子句>][<存储加密子句>][COMMENT '<列注释>']

<不同类型列定义> ::=<普通列列定义>|<虚拟列列定义>

<普通列列定义>::= <列名> <数据类型>

<虚拟列列定义> ::= <列名>[<数据类型>] [GENERATED ALWAYS]AS (<虚拟列定义>) [VIRTUAL] [VISIBLE]

<列定义子句> ::=

DEFAULT [ON NULL] <列缺省值表达式> |

<自增列子句> |

<列级约束定义> |

DEFAULT [ON NULL] <列缺省值表达式> <列级约束定义> |

<自增列子句> <列级约束定义> |

<列级约束定义> DEFAULT [ON NULL] <列缺省值表达式> |

<列级约束定义> <自增列子句>

<自增列子句> ::=

IDENTITY [(<种子>,<增量>)]|

AUTO_INCREMENT

<列级约束定义> ::= <列级完整性约束>{ <列级完整性约束>}

<列级完整性约束> ::= [CONSTRAINT <约束名>] <column_constraint_action> [<失效生效选项>]

<column_constraint_action>::=

```
    [NOT] NULL |
    <唯一性约束选项> [USING INDEX TABLESPACE {<表空间名> | DEFAULT}] |
    <引用约束> |
    CHECK (<检验条件>)|
    NOT VISIBLE
<唯一性约束选项> ::=
    PRIMARY KEY |
    [NOT] CLUSTER PRIMARY KEY |
    CLUSTER [UNIQUE] KEY |
    UNIQUE
<引用约束> ::= [FOREIGN KEY] REFERENCES [PENDANT] [<模式名>.]<表名>[(<列名>{[,<列名>]})]
[MATCH  <FULL|PARTIAL|SIMPLE>][<引用触发动作>] [WITH INDEX]
<引用触发动作> ::=
    <UPDATE 规则> [<DELETE 规则>] |
    <DELETE 规则> [<UPDATE 规则>]
<UPDATE 规则> ::= ON UPDATE <引用动作>
<DELETE 规则> ::= ON DELETE <引用动作>
<引用动作> ::= CASCADE | SET NULL | SET DEFAULT | NO ACTION
<失效生效选项>::=ENABLE | DISABLE
<STORAGE 子句> ::= STORAGE(<STORAGE 项> {,<STORAGE 项>})
<STORAGE 项> ::=
    INITIAL <初始簇数目> |
    NEXT <下次分配簇数目> |
    MINEXTENTS <最小保留簇数目> |
```

ON <表空间名> |

FILLFACTOR <填充比例> |

BRANCH <BRANCH 数> |

BRANCH (<BRANCH 数>, <NOBRANCH 数>) |

NOBRANCH |

CLUSTERBTR |

WITH COUNTER |

WITHOUT COUNTER |

USING LONG ROW|

DISABLE USING LONG ROW

<存储加密子句>::= <透明存储加密子句>|<半透明存储加密子句>

<透明存储加密子句>::= <透明存储加密子句 1>|<透明存储加密子句 2>

<透明存储加密子句 1>::= ENCRYPT [<透明加密用法>]

<透明存储加密子句 2>::= ENCRYPT <透明加密用法><散列选项>

<透明加密用法> ::=

WITH <加密算法> [<透明加密选项>] |

<透明加密选项>

<透明加密选项> ::= <透明加密选项 1> |<透明加密选项 2> |<透明加密选项 3>

<透明加密选项 1> ::= AUTO

<透明加密选项 2> ::= AUTO BY <列存储密钥>

<透明加密选项 3> ::= AUTO BY WRAPPED <列存储密钥的密文>

<半透明存储加密子句> ::= ENCRYPT [WITH <加密算法>] MANUAL [<半透明加密选项>][<散列选项>]

<半透明加密选项> ::= <半透明加密选项 1> | <半透明加密选项 2> | <半透明加密选项 3>

<半透明加密选项 1> ::= <可见用户列表>

<半透明加密选项 2> ::= BY <列存储密钥> [<可见用户列表>]

<半透明加密选项 3> ::= BY WRAPPED <列存储密钥的密文> [<可见用户列表>]

<可见用户列表> ::= USER ([<用户名> {,<用户名>}])

<散列选项> ::= HASH WITH <散列算法> [<加盐选项>]

<加盐选项> ::= [NO] SALT

<表级约束定义>::=[CONSTRAINT <约束名>] <表级约束子句>[<失效生效选项>][<VALIDATE 选项>]

<表级约束子句>::=<表级完整性约束>

<表级完整性约束> ::=

<唯一性约束选项> (<列名> {,<列名>}) [USING INDEX TABLESPACE{ <表空间名> | DEFAULT}] [GLOBAL] |

FOREIGN KEY (<列名>{,<列名>}) <引用约束> |

CHECK (<检验条件>)

<属性子句>::= **<表空间子句>**|

ON COMMIT <DELETE | PRESERVE> ROWS|

<空间限制子句>|

<STORAGE 子句>

<表空间子句>::=TABLESPACE <表空间名>

<空间限制子句> ::=

DISKSPACE LIMIT <空间大小>|

DISKSPACE UNLIMITED

<延迟段分配子句> ::=

SEGMENT CREATION IMMEDIATE |

```
    SEGMENT CREATION DEFERRED

<压缩子句> ::=

    COMPRESS |

    COMPRESS (<列名> {,<列名>}) |

    COMPRESS EXCEPT (<列名> {,<列名>})

<表并行度子句> ::=

    PARALLEL|

    PARALLEL<并行度>|

    NOPARALLEL

<高级日志子句> ::= WITH ADVANCED LOG

<add_log 子句>::= ADD LOGIC LOG

<DISTRIBUTE 子句> ::=

    DISTRIBUTED [RANDOMLY | FULLY]|

    DISTRIBUTED BY [<HASH>](<列名> {,<列名>})|

    DISTRIBUTED BY RANGE (<列名> {,<列名>})(<范围分布项> {,<范围分布项>})|

    DISTRIBUTED BY LIST (<列名> {,<列名>})(<列表分布项> {,<列表分布项>})

<范围分布项> ::=

    VALUES LESS THAN (<范围表达式>{,<范围表达式>}) ON <实例名>|

    VALUES EQU OR LESS THAN (<范围表达式>{,<范围表达式>}) ON <实例名>

<范围表达式> ::= MAXVALUE | <表达式>

<列表分布项> ::= VALUES (DEFAULT | <表达式>{,<表达式>}) ON <实例名>

<AUTO_INCREMENT 子句>::  = AUTO_INCREMENT [=] <起始边界值>

<不带 INTO 的 SELECT 语句> ::= <查询表达式>|<带参数查询语句>

<带参数查询语句>::=<子查询> | (<带参数查询语句>)
```

```
<VALIDATE 选项>::=NOVALIDATE | VALIDATE
```

参数

1. <模式名> 指明该表属于哪个模式，缺省为当前模式；

2. <表名> 指明被创建的基表名，基表名最大长度 128 字节；

3. <列名> 指明基表中的列名，列名最大长度 128 字节；

4. <数据类型> 指明列的数据类型；支持设置定长串的字符长度，如 char(n char)，n 为在 1~32767 中的整数；当 ini 参数 NVARCHAR_LENGTH_IN_CHAR 设置为 1 时，支持 nchar 设置定长串的字符长度，此时 nchar(n)等价于 char(n char)的处理；

5. <列缺省值表达式> 如果之后的 INSERT 语句省略了插入的列值，那么此项为列值指定一个缺省值，可以通过 DEFAULT 指定一个值。DEFAULT 表达式串的长度不能超过 2048 字节；对指定了 ON NULL 的列，插入数据时若数据为 NULL，则该位置数据被替换为列缺省值进行插入；

6. <列级完整性约束定义> 支持同时指定多个列级完整性约束，各约束之间使用空格分隔。列级完整性约束中各个参数含义如下：

   1) NULL 指明指定列可以包含空值，为缺省选项；

   2) NOT NULL 非空约束，指明指定列不可以包含空值；

   3) UNIQUE 唯一性约束，指明指定列作为唯一关键字；

   4) PRIMARY KEY 主键约束，指明指定列作为基表的主关键字，INI 参数 PK_WITH_CLUSTER 控制该约束默认为 CLUSTER 或 NOT CLUSTER；

   5) CLUSTER PRIMARY KEY 主键约束，指明指定列作为基表的聚集索引主关键字；

   6) NOT CLUSTER PRIMARY KEY 主键约束，指明指定列作为基表的非聚集索引主关键字；

   7) CLUSTER KEY指定列为聚集索引键，但是是非唯一的；

8） CLUSTER UNIQUE KEY 指定列为聚集索引键，并且是唯一的；

9） USING INDEX TABLESPACE <表空间名> 指定索引存储的表空间，DMDPC 环境下忽略该选项。

缺省该子句或使用 USING INDEX TABLESPACE DEFAULT 均为缺省情况。缺省情况下，当约束使用了 CLUSTER 关键字（CLUSTER PRIMARY KEY 、CLUSTER KEY、CLUSTER UNIQUE KEY 或当 INI 参数 PK_WITH_CLUSTER=1 时的 PRIMARY KEY）时，约束将使用表聚集索引的存储表空间作为约束的缺省存储表空间。

当约束未使用 CLUSTER 关键字时，约束将使用<INDEX_TABLESPACE 子句>指定的默认索引表空间作为约束的缺省存储表空间，若未指定用户默认索引表空间，将使用表聚集索引的存储表空间；

10）REFERENCES 指明指定列的引用约束。引用约束要求引用对应列类型必须基本一致。所谓基本，是因为 CHAR 与 VARCHAR，BINARY 与 VARBINARY，TINYINT、SMALLINT 与 INT 在此被认为是一致的。如果有 WITH INDEX 选项，则为引用约束建立索引，否则不建立索引，通过其他内部机制保证约束正确性；

11） CHECK 检查约束，指明指定列必须满足的条件；

12）NOT VISIBLE 列不可见，当指定某列不可见时，使用 SELECT *进行查询时将不添加该列作为选择列。使用 INSERT 无显式指定列列表进行插入时，值列表不能包含隐藏列的值。

7. <失效生效选项> 用于指定约束的状态：ENABLE 启用；DISABLE 失效。缺省为启用。

8. <表级完整性约束>中的参数：

1） UNIQUE 唯一性约束，指明指定列或列的组合作为唯一关键字；

2） PRIMARY KEY 主键约束，指明指定列或列的组合作为基表的主关键字。指明

CLUSTER，表明是主关键字上的聚集索引；指明 NOT CLUSTER，表明是主关键字上的非聚集索引；也可通过 INI 参数 PK_WITH_CLUSTER 控制该约束默认为 CLUSTER 或 NOT CLUSTER；

3） USING INDEX TABLESPACE <表空间名> 指定索引存储的表空间，DMDPC 环境下忽略该选项；

4） FOREIGN KEY 指明表级的引用约束，如果使用 WITH INDEX 选项，则为引用约束建立索引，否则不建立索引，通过其他内部机制保证约束正确性；

5） CHECK 检查约束，指明基表中的每一行必须满足的条件；

6） 与列级约束之间不应该存在冲突。

当指定<唯一性约束选项>为 UNIQUE 或 PRIMARY KEY 约束时，支持通过指定 GLOBAL 关键字，通过约束创建全局索引；由于全局索引只能是二级索引，因此当指定 CLUSTER 关键字创建聚集索引时，会创建为本地索引，但表定义、约束定义中仍然有 GLOBAL 语法项。GLOBAL 关键词仅在分区表中生效，在普通表上指定既不生效也不报错；

9. ON COMMIT<DELETE | PRESERVE>ROWS 用来指定临时表（TEMPORARY）中的数据是事务级或会话级的，缺省情况下是事务级。ON COMMIT DELETE ROWS：指定临时表是事务级的，每次事务提交或回滚之后，表中所有数据都被删除；ON COMMIT PRESERVE ROWS：指定临时表是会话级的，会话结束时才清空表；

10. CHECK <检验条件> 指明表中一列或多列能否接受的数据值或格式；

11. <查询表达式>和<子查询> 定义请查看数据查询章节；

12. STORAGE 项中：

BRANCH、NOBRANCH 是堆表创建关键字，堆表为“扁平 B 树表”。这两个参数用来指定堆表并发分支 BRANCH 和非并发分支 NOBRANCH 的数目。<BRANCH 数>取值范围为 1~64，<NOBRANCH 数>取值范围为 1~64。

分支数目主要用于插入场景中。适当提高分支数量可有效地提升并发数据处理效率，但是分支过多又会增加扫描的代价，因此，合适的<BRANCH 数>和<NOBRANCH 数>值需用户根据实际情况来决定。<BRANCH 数>主要在 FLDR 导入和查询插入等并发、批量插入场景中使用；<NOBRANCH 数>在非并发、单行插入场景中使用。

1) NOBRANCH：指定创建的表为堆表，并发分支个数为 0，非并发分支个数为 1；

2) BRANCH（<BRANCH 数>, <NOBRANCH 数>）：指定创建的表为堆表，并发分支个数为<BRANCH 数>，非并发个数为<NOBRANCH 数>；

3) BRANCH <BRANCH 数>：指定创建的表为堆表，并发分支个数为<BRANCH 数>，非并发分支个数为 0。

13. CLUSTERBTR 指定创建的表为非堆表，即普通 B 树表；

14. <虚拟列定义> 指明定义虚拟列的表达式；

15. <并行度> 创建数据表时，为 TABLE 指定并行度，并行度的合法范围为 0~128；

16. <高级日志子句> 指定创建日志辅助表，具体可参考 19.3.1.1 创建日志辅助表；

17. <add_log 子句> 开启表的物理逻辑日志记录功能，缺省为不开启。<add_log 子句>开启时和 INI 参数 RLOG_IGNORE_TABLE_SET=1 时功能一样；

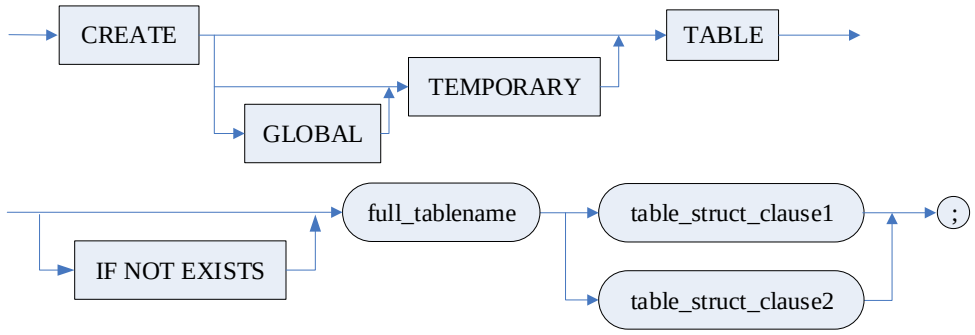
18. <AUTO_INCREMENT 子句> 用于指定隐式插入值的起始边界值，即当隐式插入时，系统自动增长的自增列值必须大于等于起始边界值。不指定默认为 1。要求大于等于 0 小于数值的最大值；

19. <表空间子句>不能和<STORAGE 子句>中的 ON <表空间名>同时使用；

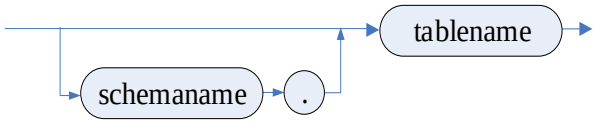
20. <VALIDATE 选项> 用于约束是否检查原有数据，VALIDATE 表示启用检查，NOVALIDATE 表示不启用检查，若在 ENABLE 下缺省本选项则默认值为 VALIDATE，若在 DISABLE 下缺省本选项则默认值为 NOVALIDATE。不支持 DISABLE VALIDATE 选项。

图例

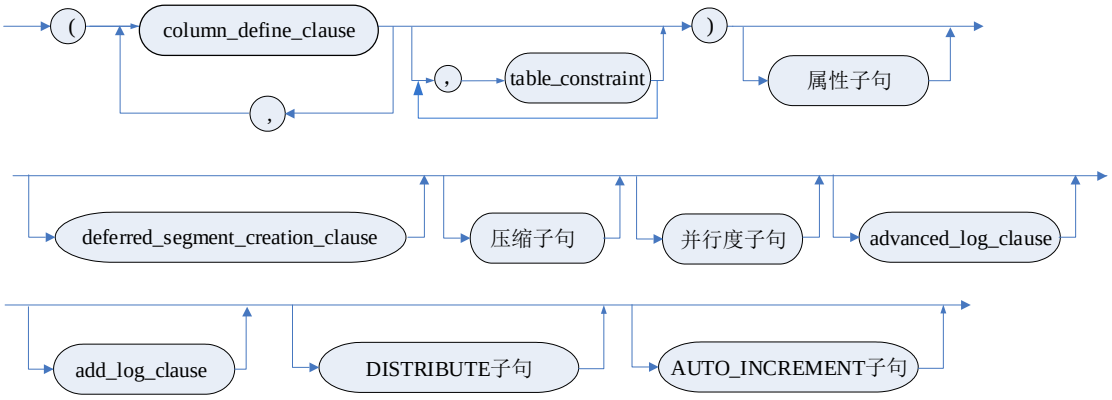
表定义语句



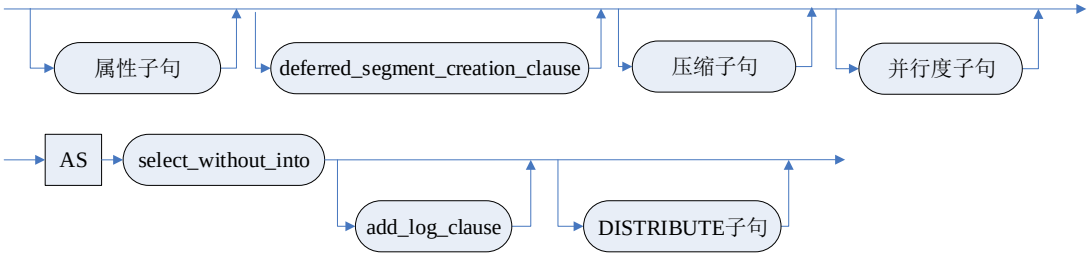
full_tablename



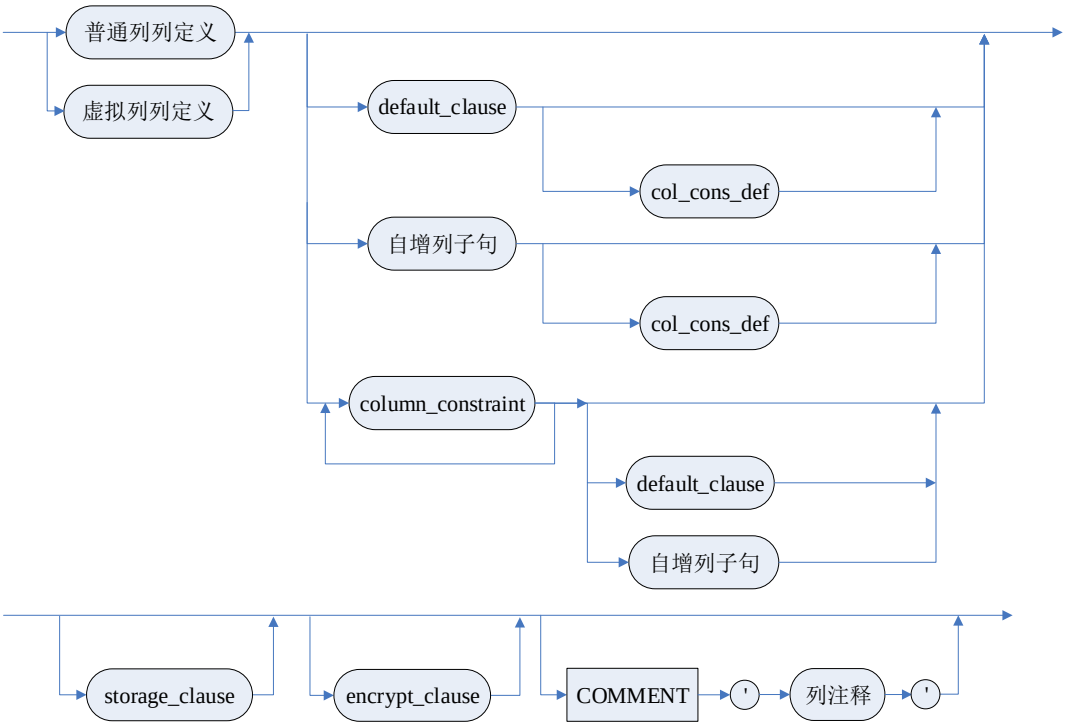
table_struct_clause1



table_struct_clause2



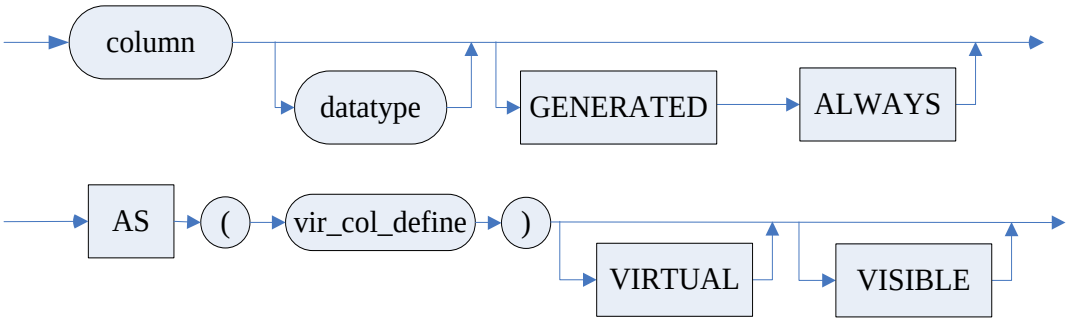
column_define_clause



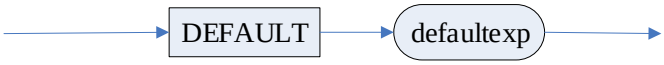
<普通列列定义>



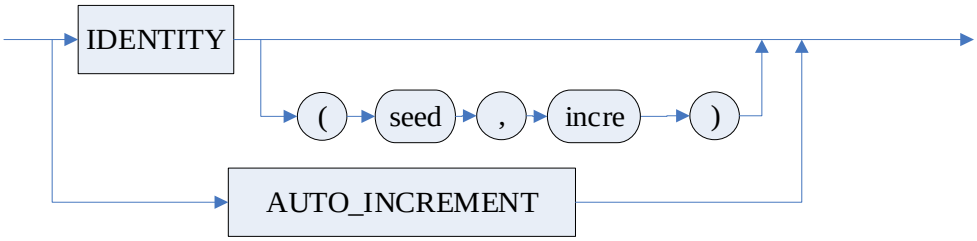
<虚拟列列定义>



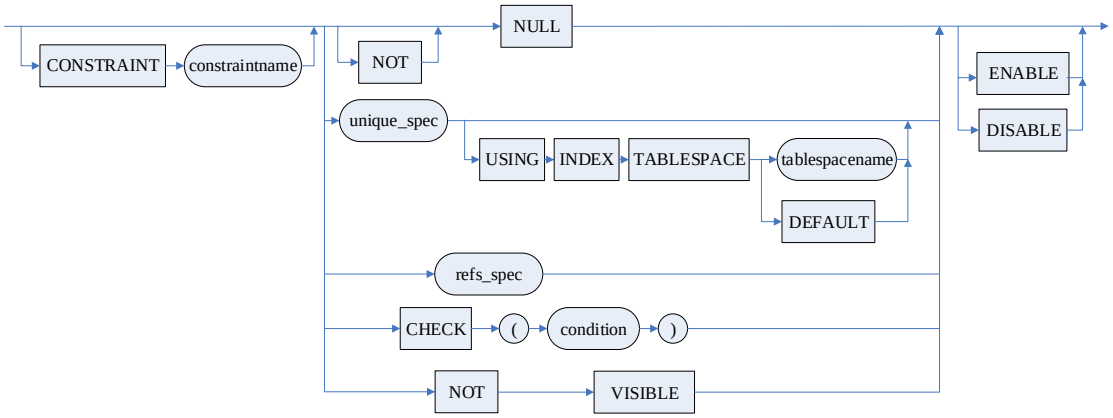
default_clause



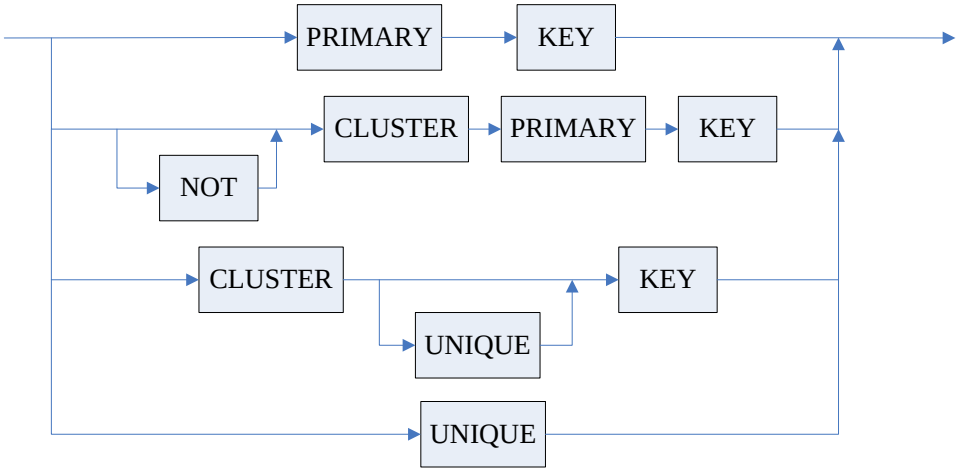
<自增列子句>



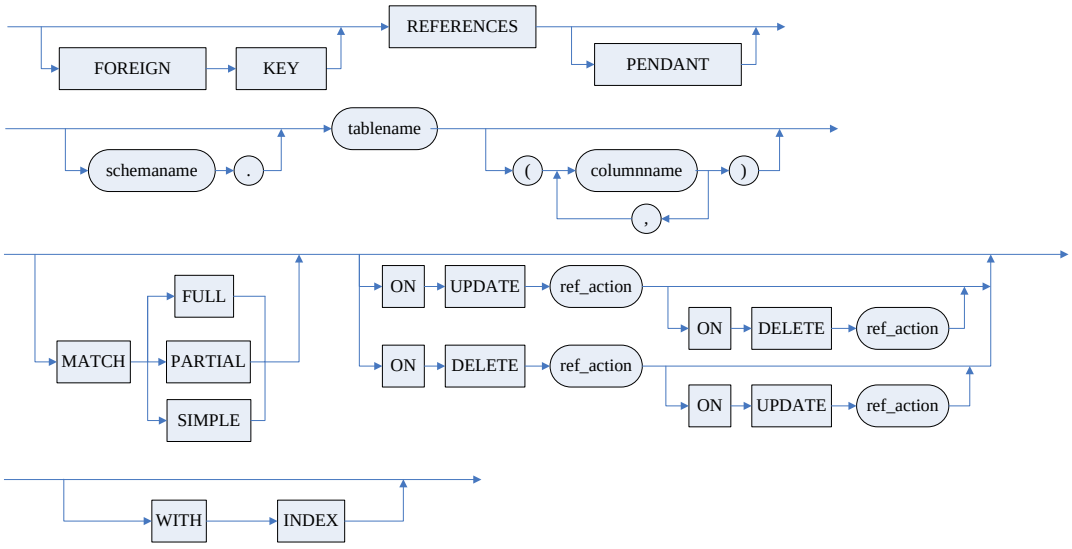
column_constraint



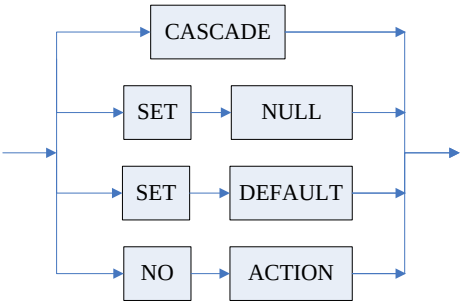
唯一性约束选项（unique_spec）



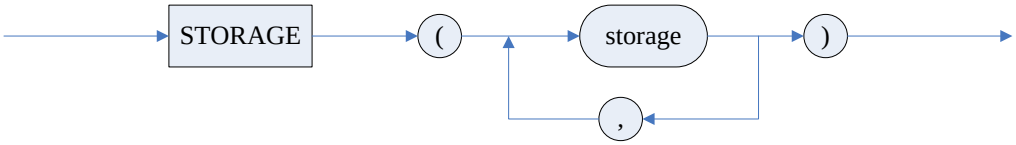
refs_spec



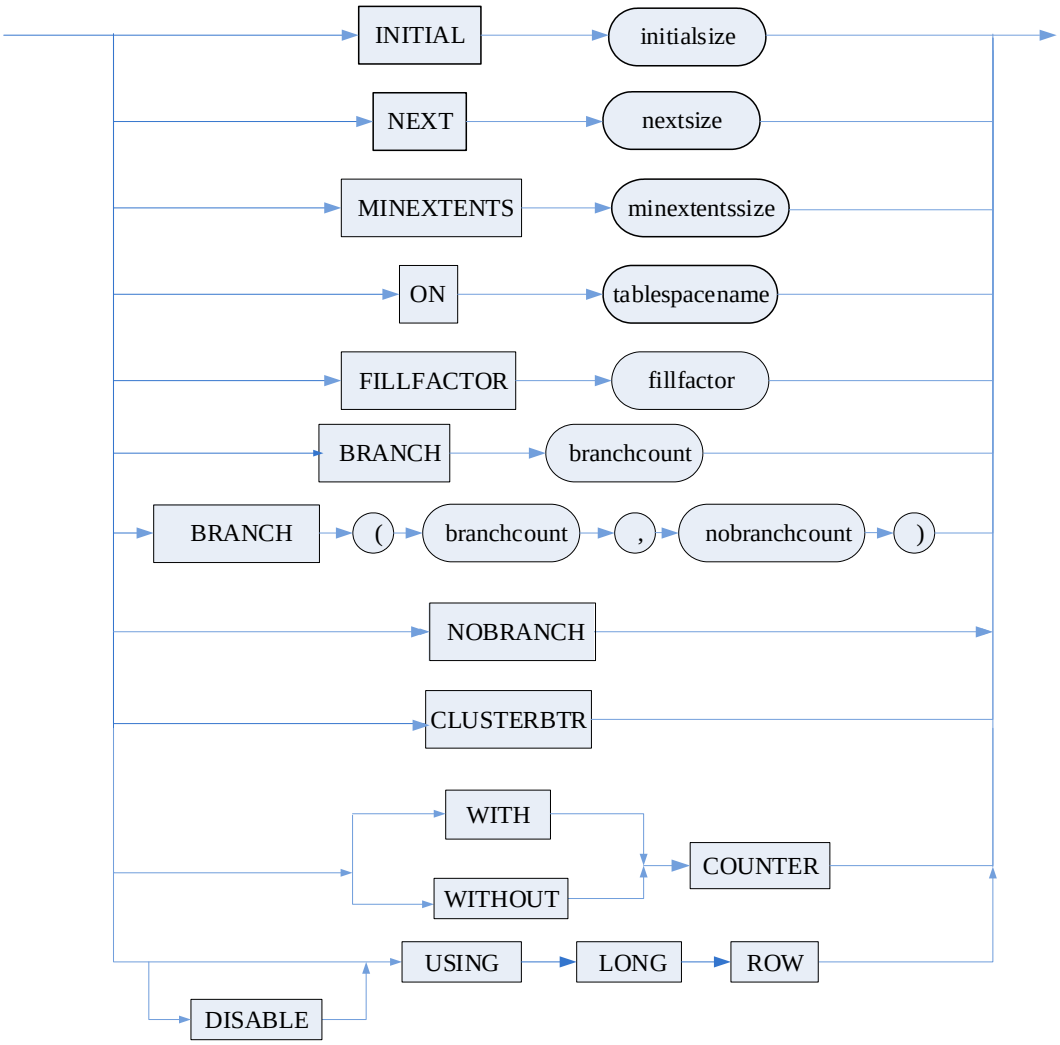
ref_action



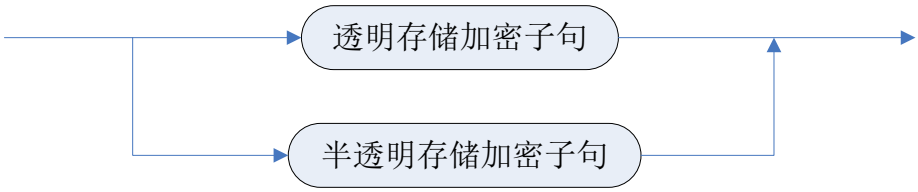
storage 子句



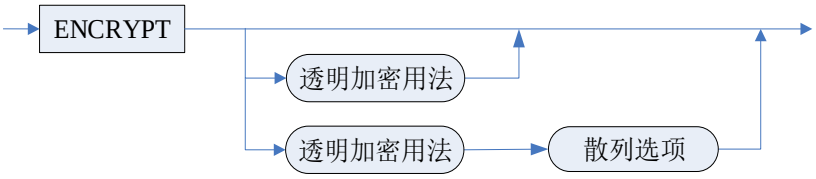
storage



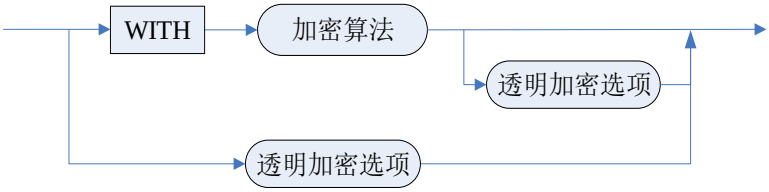
存储加密子句



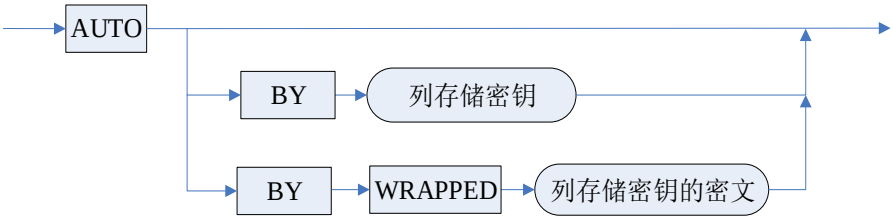
透明存储加密子句



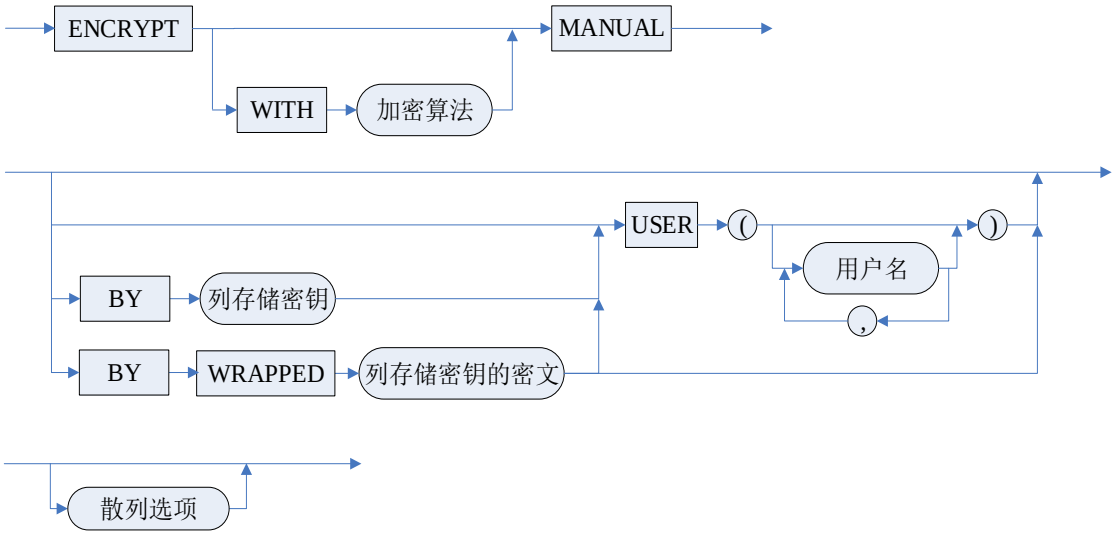
透明加密用法



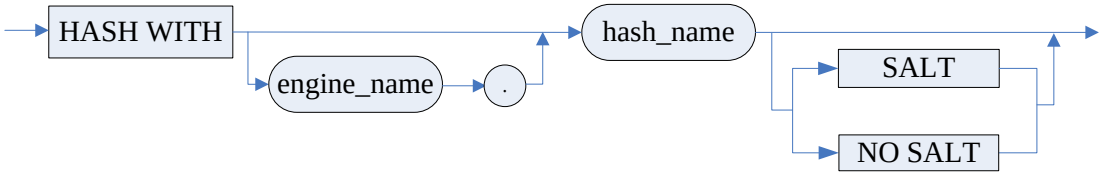
透明加密选项



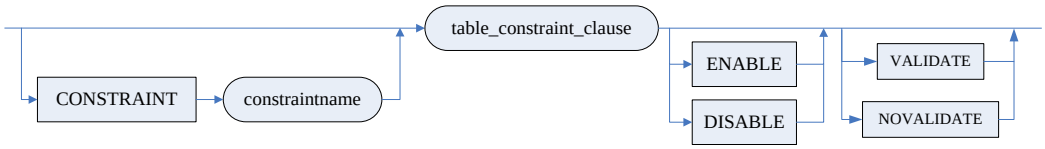
半透明存储加密子句



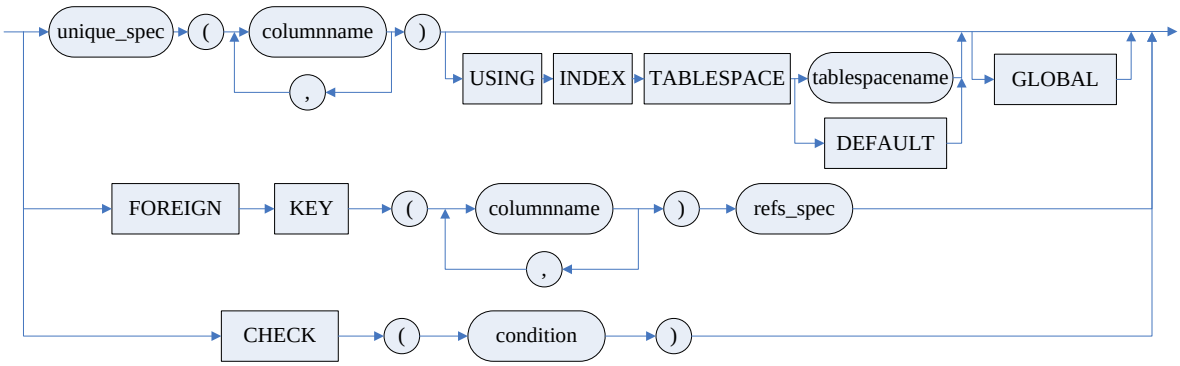
散列选项（hash_cipher）



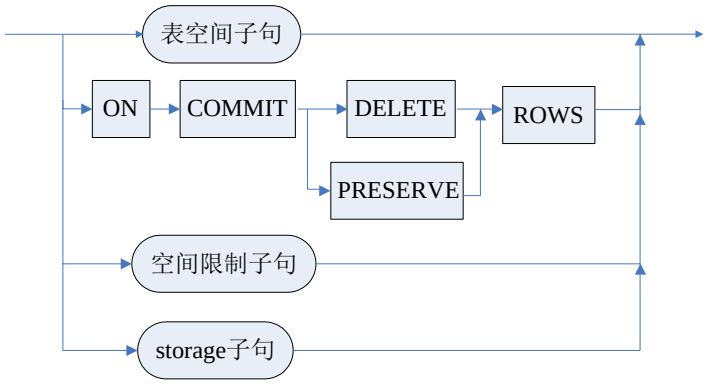
table_constraint



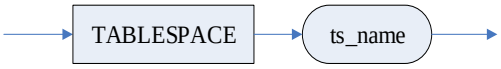
table_constraint_clause



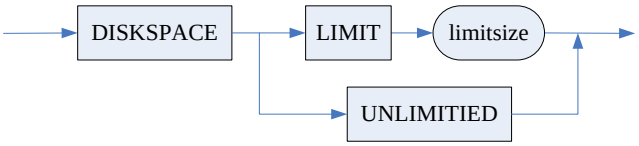
属性子句



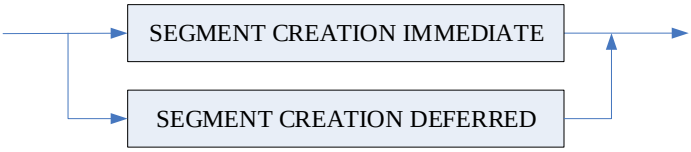
表空间子句



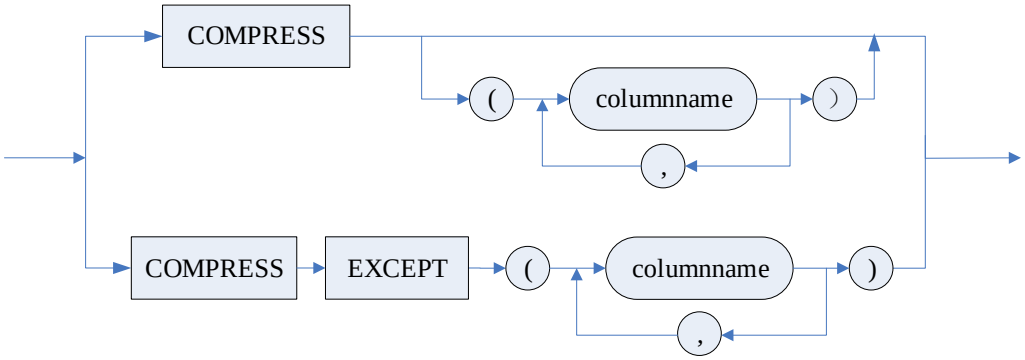
空间限制子句



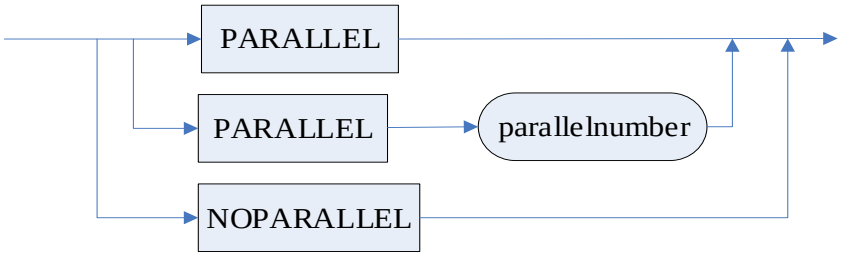
延迟段分配子句（deferred_segment_creation_clause）



压缩子句



表并行度子句



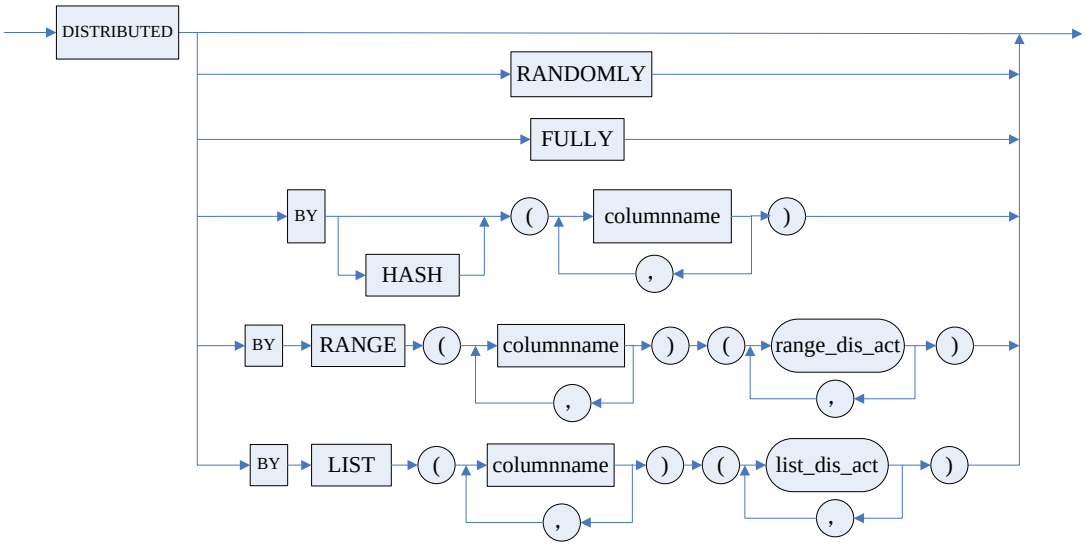
高级日志子句（advanced_log_clause）



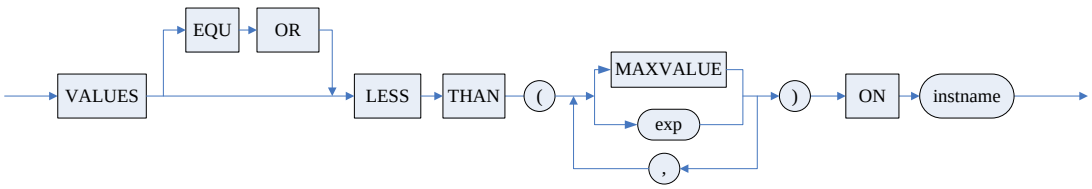
add_log 子句（add_log_clause）



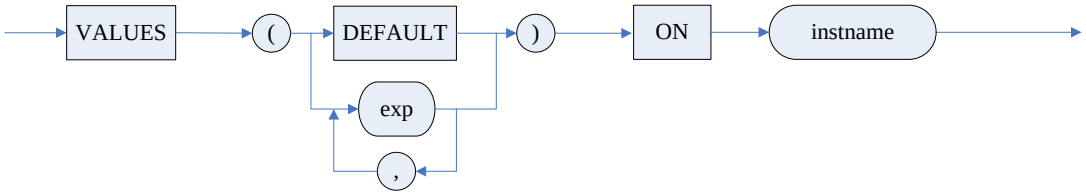
DISTRIBUTE 子句



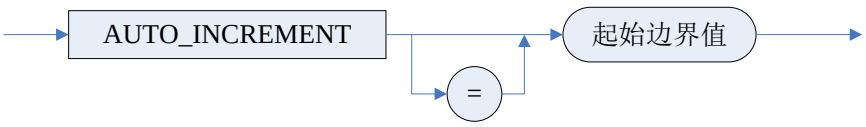
range_dis_act



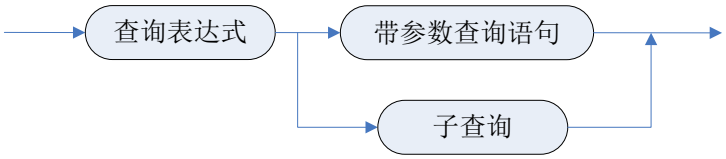
list_dis_act



AUTO_INCREMENT 子句



不带 INTO 的 SELECT 语句



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE TABLE 或 CREATE ANY TABLE 权限的用户定义基表。

使用说明

1. <表名> 指定了所要建立的基表名。表名需要是合法的标识符，且满足 SQL 语法要求。在一个<模式>中，<基表名>、<视图名>均不相同。如果<模式名>缺省，则缺省为当前模式。若指定 TEMPORARY，则表示该表为一个临时表，只在一个会话中有效，当一个会话结束，该临时表被自动清空。局部临时表创建时可以不指定模式名或指定当前函数、过程等语句块所在的模式名，局部临时表的表名只能以'#'作为首字符，且第二个字符不能为“#”，当表名以“##”开头时，该表为全局临时表；

2. 表名不允许使用以下字符串作为前缀：BM$_、BMJ$_、MDRT$_、MLOG$_、MTAB$_、MVIEW$_、MTRG$_、STAT$_、BIN$_。表名不允许使用以下字符串作为后缀：$ALOG、$AUX、$DAUX、$RAUX、$ROT、$UAUX；

3. 当 ENABLE_TMP_TAB_ROLLBACK 为 0 时，不允许对临时表创建主键约束以及唯一约束；

4. GLOBAL 目前仅支持 GLOBAL 临时表，因此建临时表时是否指定 GLOBAL 效果是一样的；

5. 表名最大长度为 128 个字节；

6. 所建基表至少要包含一个<列名>指定的列，在一个基表中，各<列名>不得相同。一张基表中至多可以包含 2048 列；

7. 虚拟列上存在约束时，不支持使用 dmfldr 导入；

8. <DEFAULT 子句> 指定列的缺省值，如：DEFAULT DATE '2015-12-26'；

9. 如果未指明 NOT NULL，也未指明<DEFAULT 子句>，则隐含为 DEFAULT NULL；

10. DM 提供两种自增列方式：IDENTITY 自增列和 AUTO_INCREMENT 自增列。两者不能同时指定。

    1) <IDENTITY 子句> 自增列不能使用<DEFAULT 子句>。<IDENTITY 子句>的种子和增量缺省值均为 1。

    2) AUTO_INCREMENT 自增列。AUTO_INCREMENT 列必须为唯一性约束的部分，只支持整数类型（支持 TINYINT/SMALLINT/INT/BIGINT，不支持 dec(N,0)等），不能违反主键的唯一性约束。

AUTO_INCREMENT 关键字需要和<AUTO_INCREMENT 子句>、三个 AUTO_INCREMENT 相关 INI 参数（AUTO_INCREMENT_INCREMENT，AUTO_INCREMENT_OFFSET，NO_AUTO_VALUE_ON_ZERO）一起配合使用。当表中没有 AUTO_INCREMENT 关键字时，仍然可以指定<AUTO_INCREMENT 子句>，但不会生效。

<AUTO_INCREMENT 子句> 用于指定隐式插入值的起始边界值，即当隐式插入时，系统自动增长的自增列值 X 必须大于等于起始边界值。

INI 参数 AUTO_INCREMENT_INCREMENT，动态会话级，表示 AUTO_INCREMENT 的步长。取值范围 1~65535。缺省值为 1。

INI 参数 AUTO_INCREMENT_OFFSET，动态会话级，表示 AUTO_INCREMENT 的基准偏移。取值范围 1~65535。缺省值为 1。

INI 参数 NO_AUTO_VALUE_ON_ZERO，动态会话级，表示 AUTO_INCREMENT 列插

入 0 时，是否自动插入自增的下一个值。取值范围 0、1。0 否，插入 0；1 是，插入自增值。缺省值为 1。

隐式生成的自增列值 X 由系统根据 AUTO_INCREMENT_OFFSET 、AUTO_INCREMENT_INCREMENT 等因子自动计算得出，计算公式 X=AUTO_INCREMENT_OFFSET+n*AUTO_INCREMENT_INCREMENT，n 为从 1 开始依次递增的正整数，即第一个 X 值理论上为 AUTO_INCREMENT_OFFSET+1*AUTO_INCREMENT_INCREMENT，第二个 X 值理论上为 AUTO_INCREMENT_OFFSET+2*AUTO_INCREMENT_INCREMENT。同时，X 值必须满足以下两个条件：1) 大于等于起始边界值；2) 大于当前自增列值中最大值（包括显式和隐式）。例如，用户指定的起始边界值为 20，AUTO_INCREMENT_OFFSET=6，AUTO_INCREMENT_INCREMENT=10，则第一个 X 值理论上为 16，第二个 X 值理论上为 26，但是由于 X 值必须大于等于 20，因此第一个 X 值实际上为 26，第二个 X 值为 36，之后假设用户在该自增列中显式插入值 50，即该自增列中的第三个值为 50，由于 X 必须大于当前自增列值中最大值，因此下一个生成的 X 值为 56，即该自增列中的第四个值为 56。

例如，现有三个 AUTO_INCREMENT 自增列，进行四次插入操作，前两次为隐式插入，第三次为显式插入，第四次为隐式插入，则四次操作插入的值如下表所示。

表 3.5.1 AUTO_INCREMENT 示例

定义	1 号自增列	2 号自增列	3 号自增列
起始边界值	20	12	40
AUTO_INCREMENT_OFFSET	6	10	27
AUTO_INCREMENT_INCREMENT	10	7	6
理论上的隐式插入值	16、26、36、46、56、66……	17、24、31、38、45、52……	33、39、45、51、57、63……

第一次插入值（隐式）	26	17	45
第二次插入值（隐式）	36	24	51
第三次插入值（显式）	50	35	60
第四次插入值（隐式）	56	38	63

11. 若设置 DEFAULT ON NULL，相应列上会隐式添加 NOT NULL 约束，手动添加的约束不能与之冲突；不能在对象类型的列上指定 DEFAULT ON NULL；

12. <列缺省值表达式>的数据类型必须与本列的<数据类型>一致。缺省值表达式存在以下几点约束：

1） 仅支持只读系统函数或指定 FOR CALCULATE 创建的存储函数；

2） 不支持表列；

3） 不支持包变量或语句参数；

4） 不支持查询表达式；

5） 不支持 LIKE；

6） 不支持 CONTAINS 表达式。

13. 如果列定义为 NOT NULL，则当该列插入空值时会报错；

14. 约束被 DM 用来对数据实施业务规则，完成对数据完整性的控制。DM_SQL 中主要定义了以下几种类型的约束：非空约束、唯一性约束、主键约束、引用约束和检查约束。如果完整性约束只涉及当前正在定义的列，则既可定义成列级完整性约束，也可以定义成表级完整性约束；如果完整性约束涉及到该基表的多个列，则只能在语句的后面定义成表级完整性约束。

定义与该表有关的列级或表级完整性约束时，可以用 CONSTRAINT<约束名>子句对约束命名，系统中相同模式下的约束名不得重复，且不能指定为系统格式的索引名（格式为前缀关键字 INDEX 与数值组合：'INDEX<系统索引数值>'，系统索引数值范围为 33554432~50331647）或系统格式的约束名（格式为前缀关键字 CONS 与数值组合：

'CONS<系统约束数值>'，系统约束数值范围为 134217728~150994943）。如果不指定约束名，系统将为此约束自动命名。经定义后的完整性约束被存入系统的数据字典中，用户操作数据库时，由 DBMS 自动检查该操作是否违背这些完整性约束条件。

1） 非空约束主要用于防止向一列添加空值，这就确保了该列在表中的每一行都存在一个有意义的值。

a） 该约束仅用于列级；

b） 如果定义了列约束为 NOT NULL，则其<列缺省值表达式>不能将该列指定为 NULL；

c） 空值即为未知的值，没有大小，不可比较。除关键字列外，其列可以取空值。不可取空值的列要用 NOT NULL 进行说明。

2） 唯一性约束主要用于防止一个值或一组值在表中的特定列里出现不止一次，确保数据的完整性。

a） 唯一性约束是通过唯一索引来实现的。创建了一个唯一索引也就创建了一个唯一性约束；同样的，创建了一个唯一性约束，也就同时创建了一个唯一索引，这种情况下唯一索引是由系统自动创建的；

b） NULL 值是不参加唯一性约束的检查的。DM 系统允许插入多个 NULL 值。对于组合的唯一性约束，只要插入的数据中涉及到唯一性约束的列有一个或多个 NULL 值，系统则认为这笔数据不违反唯一性约束。

3） 主键约束确保了表中构成主键的一列或一组列的所有值是唯一的。主键主要用于识别表中的特定行。主键约束是唯一性约束的特例。

a） 可以指定多个列共同组成主键，最多支持 63 个列；

b） 主键约束涉及的列必须为非空。通常情况下，DM 系统会自动在主键约束涉及的列上自动创建非空约束；

c） 每个表中只能有一个主键；

d） 主键约束是通过创建唯一索引来实现的。DM 系统允许用户自己定义创建主键时，通过 CLUSTER 或 NOT CLUSTER 关键字来指明创建索引的类型。CLUSTER 指明该主键是创建在聚集索引上的，NOT CLUSTER 指明该主键是创建在非聚集索引上的。在 DM.INI 配置文件中，可以指定配置项使表中的主键自动转化为聚集主键，该配置项为 PK_WITH_CLUSTER。默认情况下，PK_WITH_CLUSTER 为 0，即建表时指定的主键不会自动转化为聚集主键；若为 1，则主键自动变为聚集主键。堆表和列存储表不允许建立聚集主键。

4） 引用约束用于保证相关数据的完整性。引用约束中构成外键的一列或一组列，其值必须至少匹配其参照的表中的一行的一个主键或唯一键值。我们把这种数据的相关性称为引用关系，外键所在的表称为引用表，外键参照的表称为被引用表。

a） 引用约束指明的被引用表上必须已经建立了相关主键或唯一索引。也就是说，必须保持引用约束所引用的数据必须是唯一的；

b） 引用约束的检查规则：

i. 插入规则：外键的插入值必须匹配其被引用表的某个键值。

ii. 更新规则：外键的更新值必须匹配被引用表的某个键值。当修改被引用表中的主键值时，如果定义约束时的选项是 NO ACTION，且更新结果会违反引用约束则不允许更新；如果定义的是 SET NULL 则将引用表上的相关外键值置为 NULL；如果定义的是 CASCADE，那么引用表上的相关外键值将被修改为同样的值；如里定义的是 SET DEFAULT，则把引用列置为该列的缺省值。

iii. 删除规则：当从被引用表中删除一行数据时，如果定义约束时的选项是 NO ACTION，就不删除引用表上的相关外键值；如果定义的是 SET

NULL 则将引用表上的相关外键值置为 NULL；如果定义的是 CASCADE，那么引用表上的相关外键值将被删除；如里定义的是 SET DEFAULT，则把每个引用列置为“<列缺省值表达式>”规则中所指定的缺省值。

c） NULL 值不参加引用约束的检查。受引用约束的表，如果要插入的涉及到引用约束的列值有一个或多个 NULL，则认为插入值不违反引用约束；

d） MPP 环境下，引用列和被引用列都必需包含分布列，且分布情况完全相同；

e） MPP 环境下，不支持创建 SET NULL 或 SET DEFAULT 约束检查规则的引用约束。

5） 检查约束用于对将要插入的数据实施指定的检查，从而保证表中的数据都符合指定的限制。<检验条件>必须是一个有意义的布尔表达式，其中的每个列名必须是本表中定义的列，但列的类型不得为多媒体数据类型，并且不应包含子查询、集函数。

15. 可以使用空间限制子句 DISKSPACE LIMIT 来限制表的最大存储空间，以 MB 为单位，取值范围为 1~1048576，关键字 UNLIMITED 表示无限制。系统不支持查询建表情况下指定空间限制；

16. 可以使用 STORAGE 子句指定表的存储信息：

2） 初始簇数目：指建立表时分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；

3） 下次分配簇数目：指当表空间不够时，从数据文件中分配的簇个数，必须为整数，最小值为 1，最大值为 256，缺省为 1；

4） 最小保留簇数目：当删除表中的记录后，如果表使用的簇数目小于这个值，就不再释放表的空间，必须为整数，最小值为 1，最大值为 256，缺省为 1；

5）表空间名：在指定的表空间上建表，表空间必须已存在，缺省为该用户的默认表空间；

6）填充比例：指定存储数据时每个数据页和索引页的充满程度，取值范围为 0 到 100。可以通过设置 INI 参数 DEFAULT_FILLFACTOR 来设置填充比例的默认值。DEFAULT_FILLFACTOR 的取值范围为 0~100，取缺省值 0 时，等价于 100，表示全满填充。插入数据时填充比例的值越低，可由新数据使用的空间就越多；更新数据时填充比例的值越大，更新导致出现的页分裂的几率越大。同样，创建索引时，填充比例的值越低，可由新索引项使用的空间也就越多；

7）BRANCH 和 NOBRANCH：指定 BRANCH 和 NOBRANCH 的个数；

8）CLUSTERBTR：当 INI 参数 LIST_TABLE = 1 时，指定 CLUSTERBTR，则建立的表为普通 B 树表而非堆表；

9）WITH COUNTER：在表上维护当前表内的行数；WITHOUT COUNTER：表上只维护一个非实时的大概的行数；

对用户的影响：例如 SELECT COUNT(*) FROM test；如果表 test 是 WITH COUNTER 属性，服务器直接取行数返回即可，可以快速响应；如果表 test 是 WITHOUT COUNTER 属性，服务器需要先扫描 B 树获取行数返回后才能响应。不同的场景，根据需要灵活选择 COUNTER 属性。WITH COUNTER 属性可以通过 ALTER TABLE 语句修改。若省略该选项，默认是 WITH COUNTER 属性。

10）USING LONG ROW：显式开启超长记录存储功能。除此之外，还可以通过设置 INI 参数 CTAB_WITH_LONG_ROW=1 来隐式开启该功能，隐式开启需满足一定的附加条件，具体请参考《DM8 系统管理员手册》

CTAB_WITH_LONG_ROW 介绍。超长记录存储功能是指当 DM 行存储的记录长度超过页大小一半时，先尝试将过长的变长字符串转换为行外 BLOB 存

储，如果转换后仍超长则报错。建议变长字段定义长度不超过页大小，否则在处理排序等操作时报错。临时表、HUGE 表、外部表不支持 USING LONG ROW 选项。水平分区子表的 USING LONG ROW 选项自动采用与主表保持一致的方式，两者不同的情况下，直接忽略水平分区子表的 USING LONG ROW 选项。

11) DISABLE USING LONG ROW 可以关闭 INI 参数 CTAB_WITH_LONG_ROW=1 时隐式开启的超长记录存储功能。

17. <延迟段分配子句>用于指定是否启用延迟段分配功能。若不启用，则在建表时为表分配数据段；若启用，则向表中插入第一条数据时为表分配数据段。SEGMENT CREATION IMMEDIATE 表示不启用，SEGMENT CREATION DEFERRED 表示启用。若不指定<延迟段分配子句>，则根据 INI 参数 DEFERRED_SEGMENT_CREATION 决定是否启用延迟段分配功能。HUGE 表不支持延迟段分配功能，忽略该子句；

18. <压缩子句> 只是语法支持，功能已经取消；

19. 记录的列长度总和不超过页大小的一半，变长数据类型实际是否越界还需要判断实际记录的长度，如 VARCHAR、VARBINARY 等数据类型，而定长数据类型的长度是实际数据长度，如 INT、CHAR（注意建库参数 CHAR_FIX_STORAGE 为 N/0 时，CHAR 类型转换成 VARCHAR2 类型存储）、BINARY 等数据类型。例如，若建库参数页大小为 16K、CHAR_FIX_STORAGE 为 1，可以定义 CREATE TABLE TEST(C1 VARCHAR(8000),C2 INT)，但是不能定义 CREATE TABLE TEST(C1 CHAR(8000)，C2 INT)；

20. DM 具备自动断句功能；

21. 在对列指定存储加密属性时，对用户的数据在保存到物理介质之前使用指定的加密算法加密，防止数据泄露；

22. <加密算法>可以是系统内置的加密算法也可以是第三方加密算法，详情请参考手册《DM8 安全管理》。当使用第三方加密算法时，用户需要将已实现的第三方加密动态库放到 bin 目录下的文件夹 external_crypto_libs 中，DM 支持加载多个第三方加密动态库，

然后重启 DM 服务器即可引用其中的算法;

23. <散列算法>用于保证用户数据的完整性，若用户数据被非法修改，则能判断该数据不合法。散列算法可以是系统内置的散列算法也可以是第三方散列算法，详情请参考手册《DM8 安全管理》。加盐选项可以与散列算法配合使用;

24. <透明存储加密子句>是指用透明加密的方式加密列上的数据，在数据库中保存加密该列的密钥，执行 DML 语句的过程中自动获取密钥。关于表列透明加密的具体介绍请参考手册《DM8 安全管理》;

25. <半透明存储加密子句>是指用半透明加密的方式加密列上的数据, DM 使用当前操作用户的存储密钥对数据进行加密，半透明加密列的密文后追加了 4 个字节的 UID，用户查询数据时, 若当前会话用户 ID 与列密文数据中存储的 UID 相同, 则返回明文, 否则返回 NULL。关于表列半透明加密的具体介绍请参考手册《DM8 安全管理》;

26. 可以使用 DISTRIBUTE 子句指定表的分布类型，创建为分布表:

1) 单机模式下建的分布表和普通表一样，但是不能创建指定实例名的分布表（如范围分布表和 LIST 分布表）。

2) 在 MPP 模式下建分布表，如果未指定列则默认为 RANDOMLY 分布表（随机分布表）。

3) 分布列类型不支持 BLOB、CLOB、IMAGE、TEXT、LONGVARCHAR、BIT、BINARY、VARBINARY、LONGVARBINARY、时间间隔类型和用户自定义类型。

4) HASH 分布、RANGE 分布、LIST 分布允许更新分布列，并支持包含大字段列的表的分布列更新, 但包含 INSTEAD OF 触发器的表、堆表不允许更新分布列。

5) 对于 FULLY 分布表（复制分布表），只支持单表查询的更新和删除操作，并且查询项或者条件表达式中都不能包含 ROWID 伪列表达式。

6) RANGE 分布表（范围分布表）和 LIST 分布表（列表分布表），分布列与分布列值列表必须一致，并且指定的实例名不能重复。

7） 随机分布表不支持 UNIQUE 索引。

27. 虚拟列的使用：

虚拟列的值是不存储在磁盘上的，而是在查询的时候，根据定义的表达式临时计算后得到的结果。虚拟列可以用在查询、DML、DDL 语句中。索引可以建在虚拟列上。用户可以像使用普通列一样使用虚拟列。

1） GENERATED ALWAYS 和 VIRTUAL 为可选关键字，主要用于描述虚拟列的特性，写与不写没有本质区别。

2） 虚拟列中的 VISIBLE 只是语法支持，没有实际意义。

3） 不支持在索引表、外部表、临时表上使用虚拟列。虚拟列和虚拟列中使用的列必须来自同一个表。表中至少要有一个非虚拟列。

4） 在虚拟列上建索引相当于在表上建函数索引。

5） 即使表达式中的列已经有列级安全属性，虚拟列也不会继承列的安全规则。因此，为了保护虚拟列的数据，可以复制列级安全策略或者使用函数隐式来保护虚拟列数据。例如，信用卡的卡号被列级安全策略保护，只允许员工看到卡号的后四位。在这种情况下，可以把信用卡号的后四位定义成虚拟列。

6） 虚拟列不能作为分布表的分布列。

7） 虚拟列之间不能嵌套定义。

8） 虚拟列最后的输出应该是标量性的。

9） 虚拟列不能是用户自定义类型、大字段类型。

10）不能直接插入、更新虚拟列；但是可以在更新和删除的 WHERE 子句中使用虚拟列。

11） 不能删除被虚拟列引用的实际列。表中只有一个实际列时，这个实际列不能被删除。

12）虚拟列不能被修改为实际列。实际列也不能被修改为虚拟列。

13）虚拟列不能设置 DEFAULT 值。

14）虚拟列不能设置为 IDENTITY。

15）不支持在虚拟列上建立位图连接索引和全文索引。

16）不支持在虚拟列上建立聚集索引和位图索引。

17）不支持在虚拟列上建立物化视图日志。

18）虚拟列不能在创建时作为 CLUSTER PK、UNIQUE 或者 PK。但是可以在虚拟列上创建 UNIQUE INDEX。

19）虚拟列不支持 PLUS JOIN。

20）虚拟列不支持统计信息。

21）虚拟列不支持加密。

22）虚拟列的表达式定义长度不能超过 2048 字节。

23）DM 支持系统内部函数，但是不能保证数据的确定性 。

24）虚拟列定义中不支持 ROW LIKE 表达式。

25）虚拟列定义中不支持 CONTAINS 表达式。

26）如果虚拟列定义包含函数，则必须是确定性的函数。不允许使用分组函数、子查询（集函数、分析函数都不允许）。

27）虚拟列上有约束时，使用 dmfldr 导入，将不能保证正确性。

28）虚拟列上创建有索引时，则该虚拟列不支持修改列数据类型和列表达式。

28. 局部临时表为仅作用于存储过程、函数等语句块内的临时表，不登记至系统表。仅支持在函数、过程等语句块内创建，且仅作用于当前语句块，随语句块执行结束而销毁。局部临时表除基础的表定义外仅支持表并行度子句，不支持其它定义子句。局部临时表列除基础的列定义外仅支持 NOT NULL 约束、自增列，不支持其它列定义子句。

举例说明

例 1 首先回顾一下第二章中定义的基表，它们均是用列级完整性约束定义的格式写出，

也可以将唯一性约束、引用约束和检查约束以表级完整性约束定义的格式写出的。假定用户为 SYSDBA，下面以产品的评论表为例进行说明。

```
CREATE  TABLE PRODUCTION.PRODUCT_REVIEW
(
PRODUCT_REVIEWID INT IDENTITY(1,1),
 PRODUCTID INT NOT NULL,
 NAME VARCHAR(50) NOT NULL,
 REVIEWDATE DATE NOT NULL,
 EMAIL VARCHAR(50) NOT NULL,
 RATING INT NOT NULL,
 COMMENTS TEXT,
 PRIMARY KEY(PRODUCT_REVIEWID),
 FOREIGN  KEY(PRODUCTID)  REFERENCES  PRODUCTION.PRODUCT(PRODUCTID),
 CHECK(RATING IN(1,2,3,4,5))
);
//注：该语句的执行需在“产品的信息表”已经建立的前提下
```

系统执行建表语句后，就在数据库中建立了相应的基表，并将有关基表的定义及完整性约束条件存入数据字典中。需要说明的是，由于被引用表要在引用表之前定义，本例中的产品的信息表被产品的评论表引用，所以这里应先定义产品的信息表，再定义产品的评论表，否则就会出错。

例 2 建表时指定存储信息，表 PERSON 建立在表空间 FG_PERSON 中，初始簇大小为 5，最小保留簇数目为 5，下次分配簇数目为 2，填充比例为 85。

```
CREATE TABLESPACE FG_PERSON DATAFILE 'FG_PERSON.DBF' SIZE 128;
```

```
CREATE TABLE PERSON.PERSON
( PERSONID   INT IDENTITY(1,1) CLUSTER PRIMARY KEY,
SEX CHAR(1) NOT NULL,
NAME VARCHAR(50) NOT NULL,
EMAIL VARCHAR(50),
PHONE VARCHAR(25))
STORAGE(
INITIAL          5,
MINEXTENTS     5,
NEXT     2,
ON     FG_PERSON,
FILLFACTOR     85);
```

例 3 在 MPP 集群环境下建立如下范围分布表后，表 PRODUCT_INVENTORY 将按照 QUANTITY 列值，被分布到 2 个站点上。

```
CREATE TABLE PRODUCTION.PRODUCT_INVENTORY
(PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
LOCATIONID INT NOT NULL REFERENCES PRODUCTION.LOCATION(LOCATIONID),
QUANTITY INT NOT NULL)
DISTRIBUTED BY RANGE (QUANTITY)
(
      VALUES EQU OR LESS THAN (100) ON EP01,
      VALUES EQU OR LESS THAN (MAXVALUE) ON EP02
);
```

例 4 在 MPP 集群环境下建立如下列表分布表后，表 PRODUCT_INVENTORY 将按照

LOCATIONID 列值，被分布到 2 个站点上，1，2，3，4 在 EP01 上，5，6，7，8 在 EP02，如果有插入其它值时则报错。

```
CREATE TABLE PRODUCTION.PRODUCT_INVENTORY
(PRODUCTID INT NOT NULL REFERENCES PRODUCTION.PRODUCT(PRODUCTID),
LOCATIONID INT NOT NULL REFERENCES PRODUCTION.LOCATION(LOCATIONID),
QUANTITY INT NOT NULL)
DISTRIBUTED BY LIST (LOCATIONID)
(
    VALUES (1,2,3,4) ON EP01,
    VALUES (5,6,7,8) ON EP02
);
```

例 5 在 MPP 集群环境下建立如下复制分布表后，表 LOCATION 被分布到 MPP 各个站点上，每个站点上的数据都保持一致。

```
CREATE TABLE PRODUCTION.LOCATION
(LOCATIONID INT IDENTITY(1,1) PRIMARY KEY,
PRODUCT_SUBCATEGORYID INT NOT NULL,
NAME VARCHAR(50) NOT NULL)
DISTRIBUTED  FULLY;
```

例 6 建立普通表查看 select count(*) 执行计划，再删除后重建表 test 带 without counter 属性，再查看执行计划：

```
CREATE TABLE TEST(C1 INT);
EXPLAIN SELECT COUNT(*) FROM TEST;
```

执行结果如下：

```
1    #NSET2: [1, 1, 0]
```

```
2       #PRJT2: [1, 1, 0]; exp_num(1), is_atom(FALSE)
3         #FAGR2: [1, 1, 0]; sfun_num(1)
```

删除表后再重建 test 表带 without counter 属性，再查看执行计划。

```
drop table test;
create table test(c1 int) storage(without counter);
 explain select count(*) from test;
```

执行结果如下：

```
1    #NSET2: [1, 1, 0]
2       #PRJT2: [1, 1, 0]; exp_num(1), is_atom(FALSE)
3         #AAGR2: [1, 1, 0]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4           #CSCN2: [1, 1, 0]; INDEX33555828(TEST); btr_scan(1)
```

例 7　使用 AUTO_INCREMENT 自增列。先设置 INI 参数值，然后建表，最后隐式和显式插入自增列值。

```
ALTER SESSION SET 'AUTO_INCREMENT_INCREMENT' =6;
ALTER SESSION SET 'AUTO_INCREMENT_OFFSET' =15;
ALTER SESSION SET 'NO_AUTO_VALUE_ON_ZERO' =1;
CREATE TABLE T1(id int PRIMARY KEY AUTO_INCREMENT,name varchar(100))
AUTO_INCREMENT=20;
INSERT INTO T1(NAME) VALUES ('TEST1');          //隐式插入自增列值
INSERT INTO T1(id,NAME) VALUES (34,'TEST2');   //显式插入自增列值 34
INSERT INTO T1(NAME) VALUES ('TEST1');   //隐式插入自增列值

SELECT * FROM T1;
```

查询结果如下：

```
行号          ID              NAME

---------- ----------- -----

1             21             TEST1

2              34              TEST2

3              39              TEST1
```

例 8 对表 T 的 C1 列使用<半透明加密选项>进行按列加密。

先创建用户 USER01 和 USER02。

```
CREATE USER USER01 IDENTIFIED BY DMsys_123456;

CREATE USER USER02 IDENTIFIED BY DMsys_123456;
```

创建 T 表，对 C1 列进行按列加密，加密列对用户 USER01 和 USER02 可见。

```
CREATE TABLE T(C1 INT ENCRYPT MANUAL USER (USER01,USER02));
```

3.5.1.2 定义外部表

需指定如下信息：

1. 表名、表所属的模式名；
2. 列定义；
3. 控制文件和数据文件所在目录。

语法格式

```
CREATE EXTERNAL TABLE [IF NOT EXISTS] <表名定义> <表结构定义>;

<表名定义> ::=[<模式名>.]<表名>

<表结构定义> ::= (<列定义> {,<列定义>}) <FROM 子句>

<列定义> ::= <列名> <数据类型> [COMMENT '<列注释>']

<列定义>  参见 3.5.1.1 定义数据库基表说明

<FROM 子句> = <FROM 子句 1> | <FROM 子句 2>

<FROM 子句 1> ::= FROM <控制文件选项>
```

```
<控制文件选项> ::= DEFAULT DIRECTORY <目录对象名> LOCATION ('<控制文件名>')

<FROM 子句 2>       ::= FROM DATAFILE <数据文件选项> [<数据文件参数列表>]

<数据文件选项> ::= DEFAULT DIRECTORY <目录对象名> LOCATION ('<数据文件名>')

<数据文件参数列表> ::= PARMS(<参数选项> {,<参数选项>})

<参数选项> ::=

    FIELDS DELIMITED BY <表达式> |

    RECORDS DELIMITED BY <表达式>|

    ERRORS <n>|

    BADFILE '<错误日志文件名称>'|

    LOG '<日志文件名称>'|

    NULL_STR <NULL 字符串>|

    SKIP <跳过行数>|

    CHARACTER_CODE <文件字符集>
```

参数

1. <模式名> 指明该表属于哪个模式，缺省为当前模式；

2. <表名> 指明被创建的外部基表名；

3. <列名> 指明基表中的列名；

4. <数据类型> 指明列的数据类型，暂不支持多媒体类型；其余同 3.5.1.1 定义数据库基表中的说明；

5. <参数选项>

FIELDS 表示列分隔符；

RECORDS 表示行分隔符，缺省为回车；

ERRORS 表示忽略外部表数据转换中出现错误的行数，取值范围为大于 0 的正整数，缺省为 0，表示不忽略错误；

BADFILE 表示错误日志文件（.bad 文件）名称，存放在指定目录下。缺省情况下，将在查询外部表时自动生成（若中途无错误数据，将不生成.bad 文件），缺省 BADFILE 文件前缀为“表名+表 id”；

LOG 表示日志文件（.log 文件）名称，存放在指定目录下。缺省情况下，将在查询外部表时自动生成，缺省 LOG 文件前缀为“表名+表 id”；

NULL_STR 载入时视为 NULL 值处理的字符串，仅载入有效，默认忽略此参数；

SKIP 指定跳过数据文件起始的逻辑行数，默认为 0；

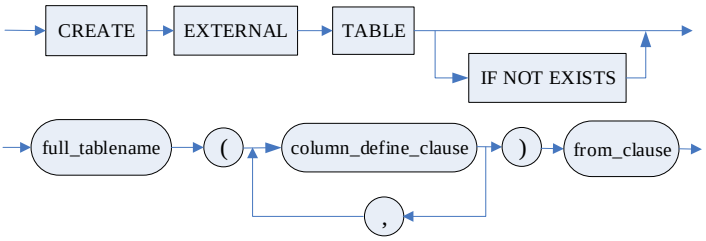
CHARACTER_CODE 指定数据文件中数据的编码格式，默认为 GBK，可选项有 GBK，UTF-8，SINGLE_BYTE 和 EUC-KR；

6. <表达式> 字符串或十六进制串类型表达式；

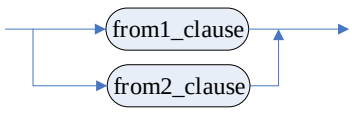
7. <目录对象名> 指控制文件或数据文件所属的目录对象的名称。

图例

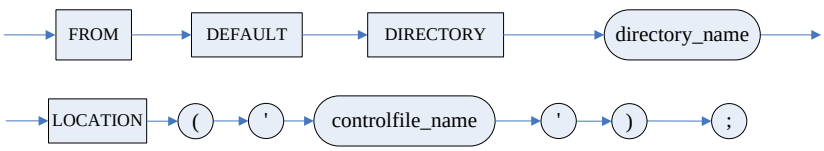
定义外部表



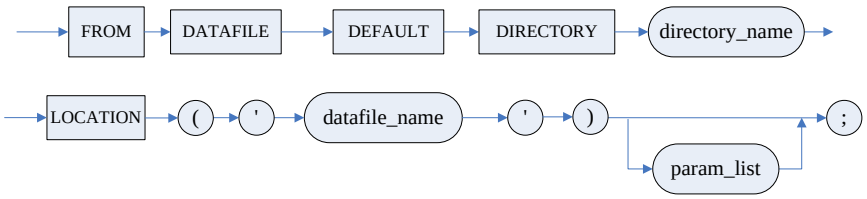
from_clause



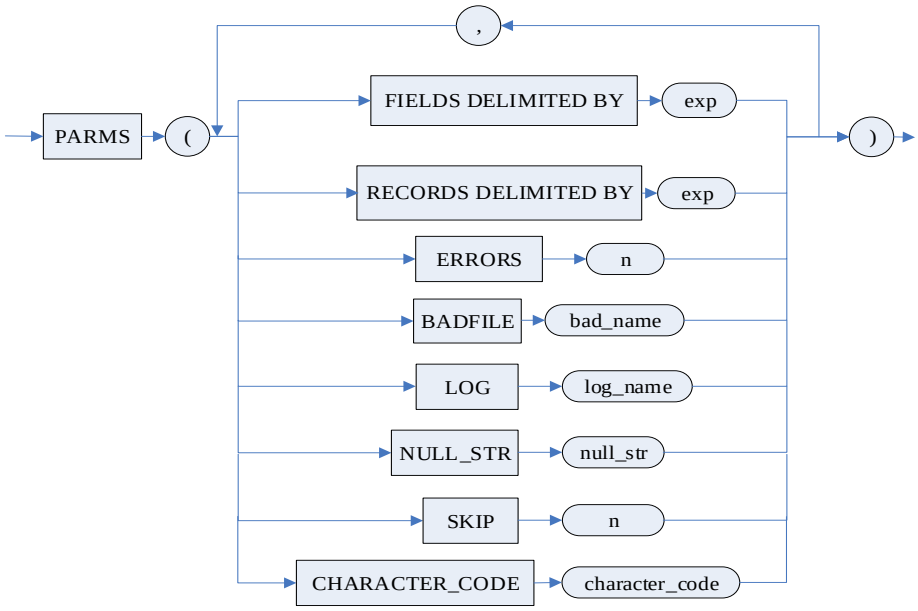
from1_clause



from2_clause



param_list



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE TABLE 或 CREATE ANY TABLE 权限的用户定义外部基表。MPP 环境、DSC 环境和 DPC 环境下不支持创建外部表。

使用说明

1. <表名>指定了所要建立的外部表名。如果<模式名>缺省，则缺省为当前模式。表名需要是合法的标识符，且满足 SQL 语法要求；

2. 表名前缀和后缀的限制规则请参考 3.5.1.1 定义数据库基表；

3. 外部表的表名最大长度为 128 个字符；

4. 所建外部基表至少要包含一个<列名>指定的列，在一个外部基表中，各<列名>不得相同。一张外部基表中至多可以包含 2048 列；

5. 外部基表不能存在大字段列；

6. 外部基表不能存在任何约束条件；

7. 外部基表不能为临时表，不能建立分区；

8. 外部基表上不能建立任何索引；

9. 外部基表是只读的，不存在表锁，不允许任何针对外部表的增删改数据操作，不允许 TRUNCATE 外部表操作；

10. 用户在创建外部表之前，须先指定文件目录；

11. 用户创建外部表时必须具有指定目录的读权限；

12. 用户查询外部表时必须具有指定目录的读权限；由于用户查询外部表时，默认会在指定目录下生成日志文件和错误日志文件，因此用户还需具有指定目录的写权限；

13. 控制文件的书写格式为：

```
[OPTIONS(
<id>=<value>
……
    )]
LOAD [DATA]
INFILE [<file_option>|<directory_option>]
[BADFILE <path_name>]
<into_table_clause>

<file_option>::= [LIST] <file_option 子句> [,<file_option 子句>]
<file_option 子句>::=<file_name> [<row_term_option>]
<file_name> ::=文件名称
<row_term_option> ::=STR [X] <delimiter>
```

```
<directory_option> ::= DIRECTORY <directory_name> [<row_term_option>]

<into_table_clause> ::= <into_table_single>{<into_table_single>}

<into_table_single> ::=INTO TABLE [<schema>.]<tablename>

                                [FIELDS [TERMINATED BY] [X] <delimiter>]

<schema> ::=模式名

<tablename> ::=表名

<delimiter> ::='<字符串常量>'
```

其中 OPTIONS 选项为可选部分，目前 OPTIONS 中支持 DATA、LOG、ERRORS、BADFILE、NULL_STR、SKIP、CHARACTER_CODE 选项。DATA 表示数据文件名称，其余选项请参考上述参数介绍部分。<file_name>和<directory_name>只能指定文件或文件夹名称，不能包含路径信息，否则报错。

14. 如果没有使用<参数选项>的 RECORDS 指定行分隔符，则在数据文件中的一行数据必须以回车结束；

15. 外部表支持查询 ROWID 和 USER 伪列，不支持查询 TRXID 伪列。

举例说明

例 1 将文本文件作为控制文件以及数据文件，指定控制文件创建外部表。

编写数据文件（d:\ext_table\data.txt）如下：

```
a|abc|varchar_data|12.34|12.34|12.34|12.34|0|1|1|1234|1234|1234|100|11|1234|1|1|14.2|12.1|12.1|1999-10-
01|9:10:21|2002-12-12|15
```

编写控制文件（d:\ext_table\ctrl.txt）如下：

```
LOAD DATA

INFILE  'data.txt'

INTO TABLE EXT

FIELDS '|'
```

创建目录对象如下：

```
CREATE OR REPLACE DIRECTORY  "EXTDIR"  AS 'd:\ext_table';
```

建表语句：

```
DROP TABLE EXT;

CREATE EXTERNAL TABLE EXT (

L_CHAR CHAR(1),

L_CHARACTER CHARACTER(3),

L_VARCHAR VARCHAR(20),

L_NUMERIC NUMERIC(6,2),

L_DECIMAL DECIMAL(6,2),

L_DEC DEC(6,2),

L_MONEY DECIMAL(19,4),

L_BIT BIT,

L_BOOL BIT,

L_BOOLEAN BIT,

L_INTEGER INTEGER,

L_INT INT,

L_BIGINT BIGINT,

L_TINYINT TINYINT,

L_BYTE BYTE,

L_SMALLINT SMALLINT,

L_BINARY BINARY,

L_VARBINARY VARBINARY,

L_FLOAT FLOAT,
```

```
L_DOUBLE DOUBLE,
L_REAL REAL,
L_DATE DATE,
L_TIME TIME,
L_TIMESTAMP TIMESTAMP,
L_INTERVAL INTERVAL YEAR
)FROM DEFAULT DIRECTORY EXTDIR LOCATION ('ctrl.txt');
```

系统执行建表语句后，就在数据库中建立了相应的外部基表。查询 EXT 表：

```
SELECT * FROM EXT;
```

查询结果如下：

```
L_CHAR L_CHARACTER L_VARCHAR      L_NUMERIC L_DECIMAL L_DEC L_MONEY L_BIT L_BOOL
L_BOOLEAN L_INTEGER     L_INT
------ ----------- ------------ --------- --------- ----- ------- ----- ------
--------- ----------- -----------
L_BIGINT                 L_TINYINT     L_BYTE        L_SMALLINT    L_BINARY L_VARBINARY L_FLOAT
-------------------- ----------- ----------- ----------- -------- -----------
-------------------------
L_DOUBLE                      L_REAL               L_DATE       L_TIME     L_TIMESTAMP
L_INTERVAL
------------------------ ------------------ ---------- --------
-------------------------- ---------------------
a         abc             varchar_data 12.34        12.34        12.34 12.3400 0        1         1
1234            1234
1234                      100             11              1234            0x01        0x01
```

```
1.420000000000000E+001
1.210000000000000E+001      1.2100000E+001      1999-10-01 09:10:21 2002-12-12
00:00:00.000000 INTERVAL '15' YEAR(2)
```

例 2 将文本文件作为数据文件，指定数据文件创建外部表。

编写数据文件（d:\ext_table_2\data.txt）如下：

```
10|9|7
4|3|2
```

创建目录对象如下：

```
CREATE OR REPLACE DIRECTORY  "EXTDIR_2"  AS 'd:\ext_table_2';
```

在 Windows 操作系统中建表语句如下：

```
DROP TABLE EXT_TABLE2;
CREATE EXTERNAL TABLE EXT_TABLE2(
C1 INT,
C2 INT,
C3 INT
) FROM DATAFILE DEFAULT DIRECTORY EXTDIR_2 LOCATION ('data.txt') PARMS(FIELDS DELIMITED BY
'|', RECORDS DELIMITED BY 0x0d0a);
```

在 Linux 操作系统中建表语句如下：

```
DROP TABLE EXT_TABLE2;
CREATE EXTERNAL TABLE EXT_TABLE2(
C1 INT,
C2 INT,
C3 INT
) FROM DATAFILE DEFAULT DIRECTORY EXTDIR_2 LOCATION ('data.txt') PARMS(FIELDS DELIMITED BY
```

```
'|', RECORDS DELIMITED BY 0x0a);
```

查询 ext_table2 表:

```
select * from ext_table2;
```

查询结果如下:

```
行号          C1              C2              C3

-------- --------- --------- -----------

1             10              9              7

2              4               3              2
```

例 3

在 Windows 操作系统中编写控制文件(d:\test_externtable\quan.ctrl), 内容如下:

```
OPTIONS(

DATA = 'quan.txt'

ERRORS   =   5

BADFILE   =   't1.bad'

LOG   =   't1.log'

NULL_STR   = ' ffff '

SKIP   =    0

CHARACTER_CODE =   'utf-8'

)

LOAD DATA

INFILE 'quan.txt' STR x '0D0A'

BADFILE 'test1.bad'

INTO TABLE fldr1

FIELDS TERMINATED BY   '||'
```

在 Linux 操作系统中编写控制文件（d:\test_externtable\quan.ctrl），内容如下：

```
OPTIONS(
DATA = 'quan.txt'
ERRORS  =  5
BADFILE  =  't1.bad'
LOG  =  't1.log'
NULL_STR  = ' ffff '
SKIP  =  0
CHARACTER_CODE =  'utf-8'
)
LOAD DATA
INFILE 'quan.txt' STR x '0A'
BADFILE 'test1.bad'
INTO TABLE fldr1
FIELDS TERMINATED BY  '||'
```

编写数据文件（d:\test_externtable\quan.txt）如下：

```
1||ab||2
1||ab||1
```

创建目录对象如下：

```
CREATE OR REPLACE DIRECTORY  "EXTDIR_3"  AS 'd:\test_externtable';
```

建表语句：

```
CREATE EXTERNAL TABLE  fldr1(
"C1" NUMBER(2,1),
"C2" VARCHAR(4),
```

```
"C3" NUMBER(2,0)
) FROM DEFAULT DIRECTORY EXTDIR_3 LOCATION ('quan.ctrl');
```

查询表 fldr1 中的数据：

```
select * from fldr1;
```

查询结果如下：

```
行号          C1   C2 C3
---------- --- -- --
1             1.0 ab 2
2             1.0 ab 1
```

3.5.1.3 定义 HUGE 表

语法格式

```
CREATE HUGE TABLE [IF NOT EXISTS] <表名定义> <表结构定义>[<PARTITION 子句>][<表空间子句>][<STORAGE 子句 1>][<压缩子句>][<日志属性>][<DISTRIBUTE 子句>];
<表名定义> ::= [<模式名>.] <表名>
<表结构定义> ::=<表结构定义 1> | <表结构定义 2>
<表结构定义 1> ::= (<列定义> {,<列定义>} [<表级约束定义>{,<表级约束定义>}])
<表结构定义 2> ::= AS <不带 INTO 的 SELECT 语句>[<DISTRIBUTE 子句>]
<列定义> ::=<列名> <数据类型>[DEFAULT [ON NULL] <列缺省值表达式>][<列级约束定义>][<STORAGE 子句 2>][<存储加密子句>][COMMENT '<列注释>']
<表级约束定义> ::= [CONSTRAINT <约束名>] <表级完整性约束>
<表级完整性约束> ::=
<唯一性约束选项> (<列名> {,<列名>}) [USING INDEX TABLESPACE {<表空间名> | DEFAULT}]|
CHECK (<检验条件>)
<列级约束定义> ::= <列级完整性约束>{ <列级完整性约束>}
```

<列级完整性约束> ::= [CONSTRAINT <约束名>]<huge_column_constraint_action>

<huge_column_constraint_action>::=

[NOT] NULL |

<唯一性约束选项> [USING INDEX TABLESPACE {<表空间名> | DEFAULT}]

<唯一性约束选项> ::=

PRIMARY KEY |

UNIQUE

<存储加密子句> ::= <透明存储加密子句>

<透明存储加密子句>参见 3.5.1.1 定义数据库基表

<PARTITION 子句> 参见 3.5.1.4 定义水平分区表

<表空间子句> ::= TABLESPACE <混合表空间名>

<STORAGE 子句 1> ::= STORAGE(<STORAGE1 子项>,{<STORAGE1 子项>,} ON <混合表空间名>)

<STORAGE1 子项>::=

[SECTION (<区大小>)] |

[INITIAL <文件初始大小>] |

[FILESIZE (<文件大小>)] |

[STAT [NONE| SYNCHRONOUS | ASYNCHRONOUS][ON | EXCEPT (col_lst)]] |

[<WITH|WITHOUT> DELTA]

<STORAGE 子句 2> ::= STORAGE(STAT NONE)

<压缩子句> ::=

COMPRESS [LEVEL <压缩级别>] [<压缩类型>]|

COMPRESS [LEVEL <压缩级别>] [<压缩类型>] (<列名> [LEVEL <压缩级别>] [<压缩类型>] {,<列名> [LEVEL <压缩级别>] [<压缩类型>] })|

COMPRESS [LEVEL <压缩级别>] [<压缩类型>] EXCEPT (<列名>{,<列名>})

```
<压缩类型>::= FOR 'QUERY [LOW | HIGH]'
<DISTRIBUTE 子句> ::=
    DISTRIBUTED [RANDOMLY | FULLY]|
    DISTRIBUTED BY [HASH](<列名> {,<列名>})|
    DISTRIBUTED BY RANGE (<列名> {,<列名>})(<范围分布项> {,<范围分布项>})|
    DISTRIBUTED BY LIST (<列名> {,<列名>})(<列表分布项> {,<列表分布项>})
<范围分布项> ::=
    VALUES LESS THAN (<表达式>{,<表达式>}) ON <实例名> |
    VALUES EQU OR LESS THAN (<表达式>{,<表达式>}) ON <实例名>
<列表分布项> ::= VALUES (<表达式>{,<表达式>}) ON <实例名>
<日志属性> ::=
    LOG NONE|
    LOG LAST|
    LOG ALL
```

参数

1. <表名> 指明被创建的 HUGE 表名。普通 HUGE 表，由于表名+辅助表名最大长度不大于 128 字节，则表名不大于 123 字节；分区 HUGE 表，由于子表名+辅助表名的长度不大于 128 字节，则子表名不大于 123 字节；

2. <区大小> 指一个区的数据行数。区的大小必须是 2 的 n 次方，如果不是则向上对齐。取值范围：1024 行~1024*1024 行。不指定则默认值为 65536 行；

3. <storage 子句 1> 中 FILESIZE (<文件大小>)指定 HUGE 表列存数据文件的大小，单位为 MB，取值范围为 16~1024*1024，不指定则默认为 64M。文件大小必须是 2 的 n 次方，如果不是则向上对齐；INITIAL (<文件初始大小>)指定 HUGE 表列存数据文件的初始大小。

1) 如果未指定 FILESIZE，则设置为 INI 参数 HUGE_DEFAULT_FILE_SIZE 指定的值；

2) 如果未指定 INITIAL，则获取 INI 参数 HUGE_DEFAULT_FILE_INIT_SIZE，如果该参数非 0，则取该 INI 参数与 FILESIZE 的较小值；如果该 INI 参数为 0，则设定为该表的 FILESIZE。

4. <storage 子句 1>中 STAT [NONE| SYNCHRONOUS | ASYNCHRONOUS] [ON | EXCEPT (col_lst)] 设置 HUGE 表的统计状态。

1) 设置表的统计状态[NONE| SYNCHRONOUS | ASYNCHRONOUS]

a) NONE：不计算统计信息，在数据修改时不做数据的统计。

b) SYNCHRONOUS：实时计算统计信息。

c) ASYNCHRONOUS：异步计算统计信息，仅支持 WITH DELTA 的 HUGE 表。

d) 省略此项：如果是 WITH DELTA 的 HUGE 表，统计状态参考 INI 参数 HUGE_STAT_MODE；如果是 WITHOUT DELTA 为实时统计。

2) 设置列[<ON | EXCEPT> (col_lst)]，用于设置表中特定某些列的统计信息状态

a) 如果列设置省略，则表示所有列上的统计开关均为打开。

b) 列设置仅对计算统计信息时有效；如果表的统计状态是 NONE 状态，此时设置列，则报错。

c) 如果是 ON，则对列表中的列计算统计信息，其余的不计算统计信息。

d) 如果是 EXCEPT，则对列表中的列不计算统计信息，其余的计算统计信息。

e) 不支持同时在表和列的定义中指定列的统计信息状态，如果在表定义<storage 子句 1>的 stat 子句中指定了列，在表中某些列定义的<storage 子句 2>中也指定了 stat 子句，则报错。

f） 允许下列情况：表设置了要计算统计信息，但所有列上都设置不计算统计信息。

5. <storage 子句 2> 的 STAT NONE 对某一列进行设置，该列不计算统计信息，在修改时不做数据的统计；此处不设置 STAT NONE，表示进行统计；

6. <混合表空间名> 指要创建的 HUGE 表所属的混合表空间。不指定则存储于默认用户表空间 MAIN 中。不支持同时使用<表空间子句>和<storage 子句 1>指定混合表空间名；

7. <压缩级别> 指定列的压缩级别，有效值范围为：0~10，分别代表不同的压缩算法和压缩级别。有两种压缩算法：SNAPPY 和 ZIP。10 采用 SNAPPY 算法轻量级方式压缩。2~9 采用 ZIP 算法压缩，2~9 代表压缩级别，值越小表示压缩比越低、压缩速率越快；值越大表示压缩比越高、压缩速度越慢。0 和 1 为快捷使用，默认值为 0。0 等价于 LEVEL 2；1 等价于 LEVEL 9；

8. <压缩类型> 指定列压缩类型。FOR 'QUERY [LOW]'表示进行规则压缩；FOR 'QUERY HIGH'表示进行规则压缩与通用压缩结合，前者的压缩比一般在 1:1 至 1:3 之间，后者一般为 1:3 至 1:5 之间。规则压缩方式一般适用于具有一定的数据规则的数据的压缩，例如重复值较多等。若某列的类型为字符串类型且定义长度超过 48，则即使指定规则压缩也无效，实际只进行通用压缩；

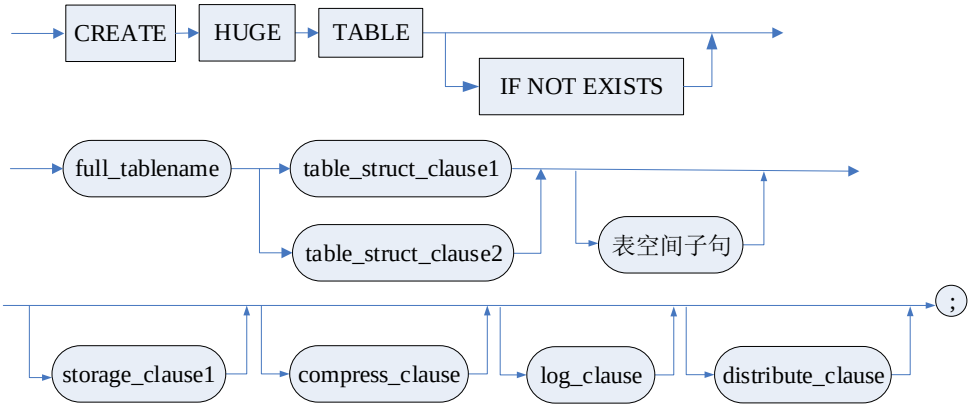
9. <日志属性> 此属性仅对非事务型 HUGE 表有效，支持通过做日志来保证数据的完整性。完整性保证策略主要是通过数据的镜像来实现的，镜像的不同程度可以实现不同程度的完整性恢复。三种选择：1）LOG NONE：不做镜像。相当于不做数据一致性的保证，如果出错只能手动通过系统函数 SF_REPAIR_HFS_TABLE(模式名,表名)来修复表数据。2）LOG LAST：做部分镜像。但是在任何时候都只对当前操作的区做镜像，如果当前区的操作完成了，这个镜像也就失效了，并且可能会被下一个被操作区覆盖，这样做的好处是镜像文件不会太大，同时也可以保证数据是完整的。但有可能遇到的问题是：一次操作很多的情况下，有可能一部分数据已经完成，另一部分数据还没有来得及做的问题。3）LOG ALL：全

部做镜像。在操作过程中，所有被修改的区都会被记录下来，当一次操作修改的数据过多时，镜像文件有可能会很大，但能够保证操作完整性。默认选择为 LOG LAST；

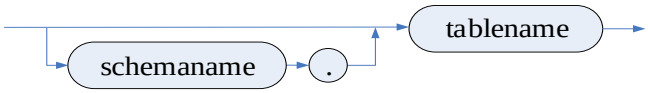
10. <WITH|WITHOUT> DELTA WITH DELTA 表示创建事务型 HUGE 表；WITHOUT DELTA 表示创建非事务型 HUGE 表，缺省为 WITH DELTA。若创建数据库时使用参数 HUGE_WITH_DELTA 的缺省值 1，则不支持创建非事务型 HUGE 表。

图例

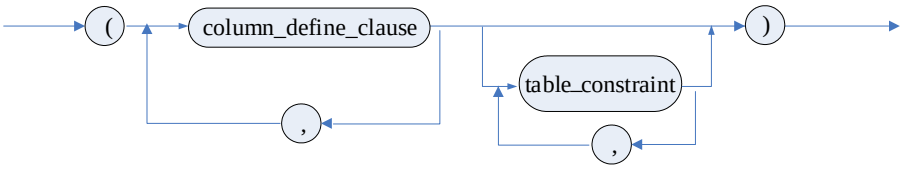
创建 HUGE 表



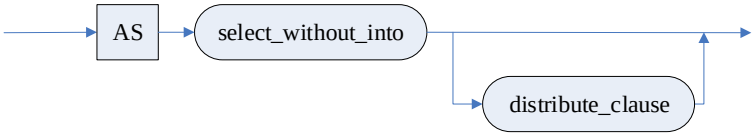
full_tablename



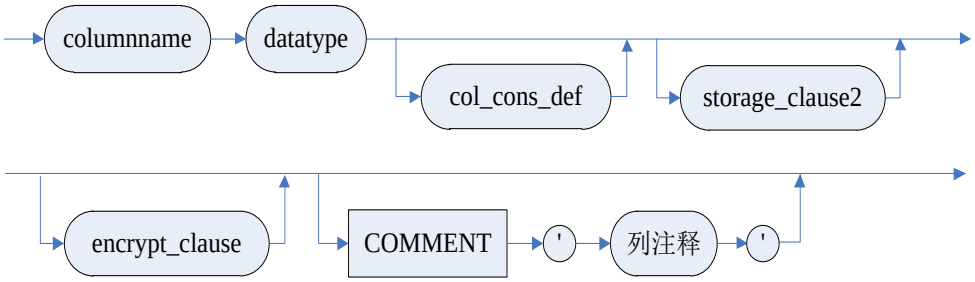
table_struct_clause1



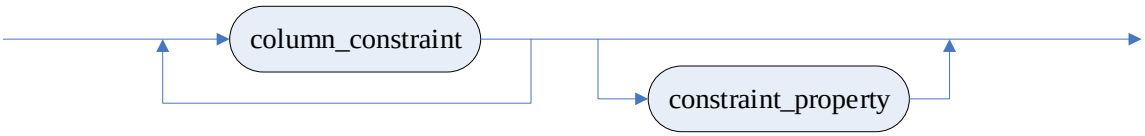
table_struct_clause2



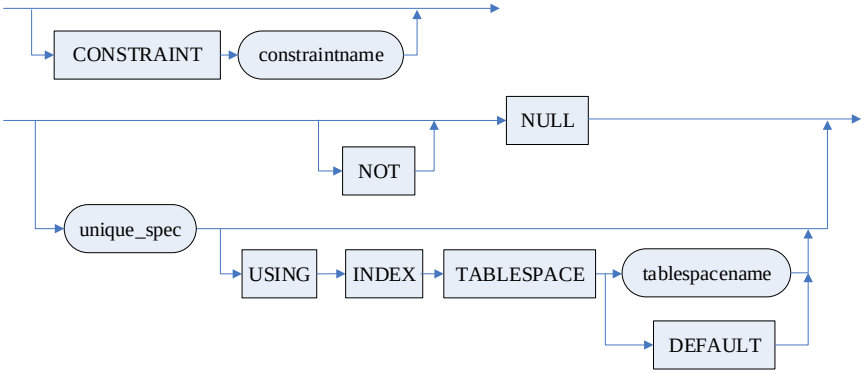
column_define_clause



col_cons_def



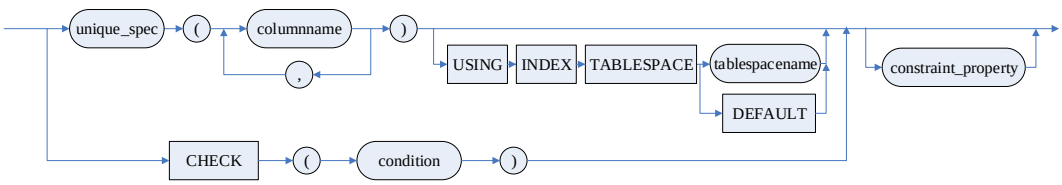
column_constraint



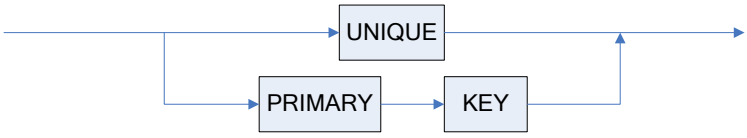
table_constraint



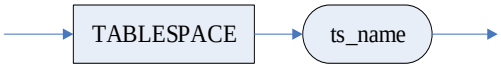
table_constraint_clause



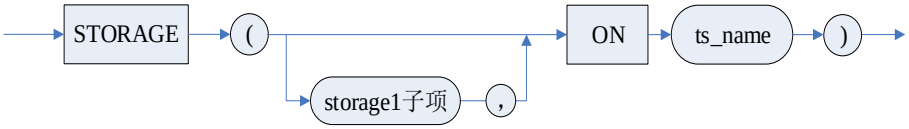
unique_spec



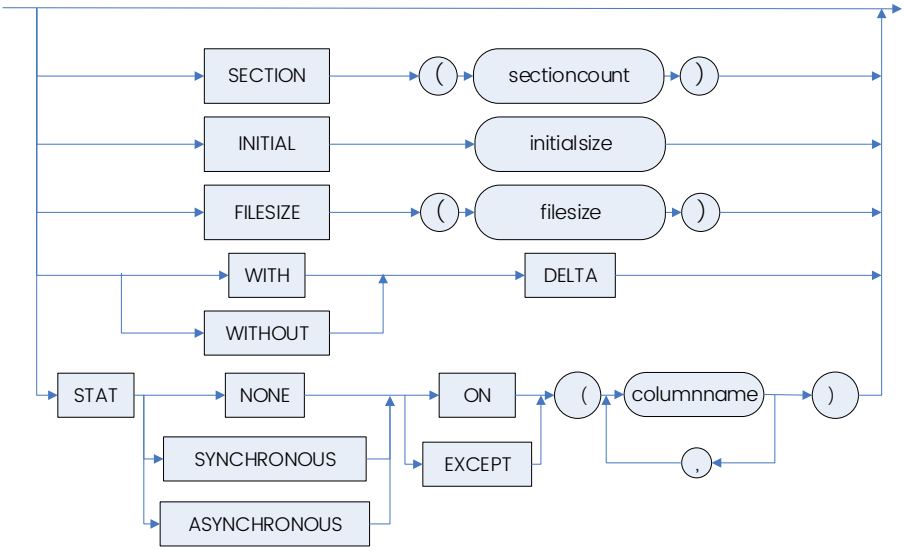
表空间子句



storage_clause1



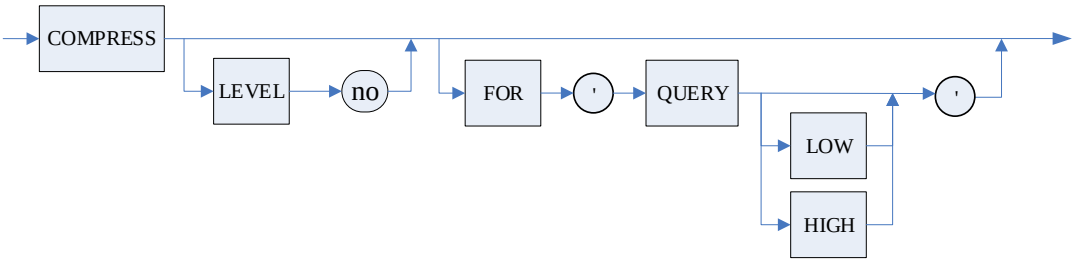
storage1 子项

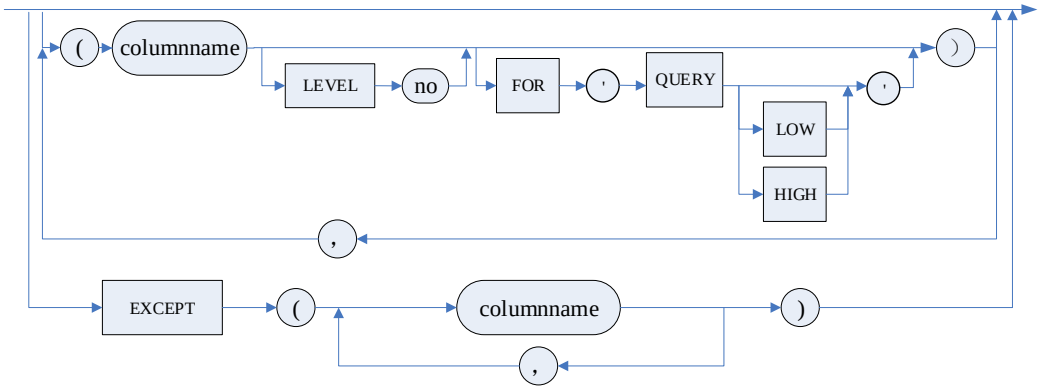


storage_clause2

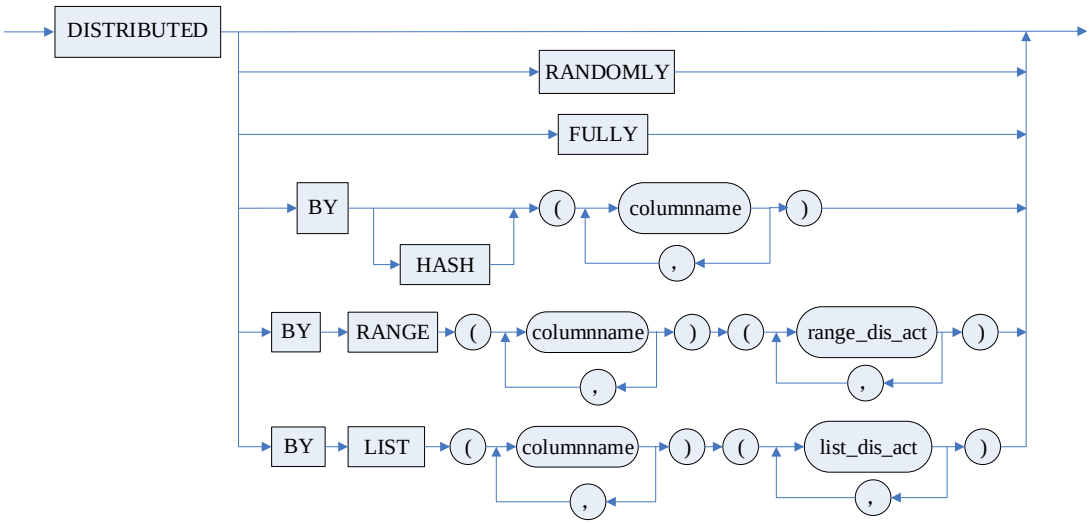


compress_clause

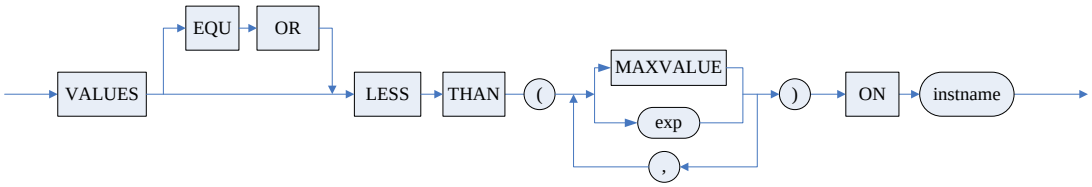




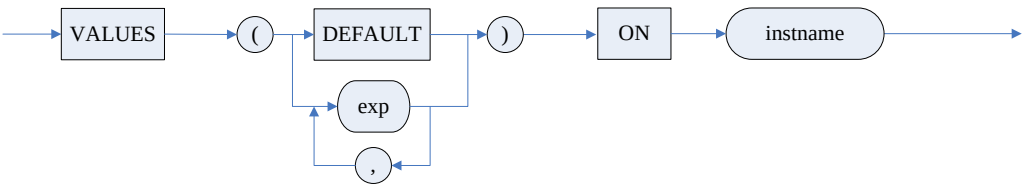
distribute_clause



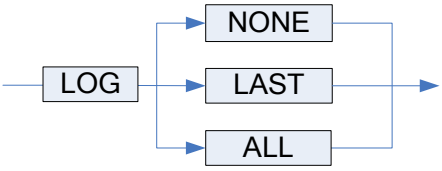
range_dis_act



list_dis_act



log_clause



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE TABLE 或 CREATE ANY TABLE 权限的用户创建 HUGE 表。

使用说明

1．表名前缀、后缀及约束名的限制规则请参考 3.5.1.1 定义数据库基表；

2．非事务型 HUGE 表的插入、删除与更新操作处理都不能进行回滚；

3．建 HUGE 表时仅支持定义 NULL、NOT NULL、UNIQUE 约束以及 PRIMARY KEY。后两种约束也可以通过 ALTER TABLE 的方式添加，但这两种约束不检查唯一性，用户需要确保实际数据符合约束，否则相关操作的结果可能不符合预期；

4．HUGE 不允许建立聚集索引，允许建立二级索引，不支持建立位图索引，DPC 环境下不支持建立位图连接索引，其中 UNIQUE 索引不检查唯一性；

5．不支持 SPACE LIMIT（空间限制）；

6．不支持建立全文索引；

7．不支持使用自定义类型；

8．不支持引用约束；

9．不支持 IDENTITY 自增列；

10．不支持大字段列；

11．不支持建触发器；

12．不支持显式游标的修改操作；

13．非事务型 HUGE 表 PRIMARY KEY 主键约束或 UNIQUE 唯一约束不检查唯一性，对应的索引都为虚索引。事务型 HUGE 表的 PRIMARY KEY 主键约束或 UNIQUE 唯一约束的唯一性检查由 DM.INI 参数 HUGE_UNIQUE_CHECK 决定，当 HUGE_UNIQUE_CHECK=1 时，PRIMARY KEY 主键约束或 UNIQUE 唯一约束对应的索引均为实索引；HUGE_UNIQUE_CHECK=0 时，创建索引规则仍然与非事务型 HUGE 表保持一致；

14．不允许对分区子表设置 SECTION 和 WITH/WITHOUT DELTA；

15．对于非事务型 HUGE 表，若指定记录区统计信息，可能因为统计信息超长造成记录插入或更新失败；

16．HUGE 表不支持查询 TRXID 和 PHYROWID 列。

举例说明

例 以 SYSDBA 身份登录数据库后，创建 HUGE 表 orders。

```
CREATE HUGE TABLE orders
(
    o_orderkey              INT,
    o_custkey               INT,
    o_orderstatus               CHAR(1),
    o_totalprice            FLOAT,
    o_orderdate             DATE,
    o_order111          CHAR(15),
    o_clerk                 CHAR(15),
    o_shippriority          INT,
    o_comment                   VARCHAR(79) STORAGE(stat none)
)STORAGE(SECTION(65536) , FILESIZE(64), WITH DELTA, ON TS1) COMPRESS LEVEL 9 FOR 'QUERY HIGH'  (o_comment);
```

这个例子创建了一个名为 ORDERS 的事务型 HUGE 表，ORDERS 表的区大小为 65536 行，文件大小为 64M，指定所在的混合表空间为 TS1，o_comment 列指定的区大小为不做统计信息，其它列（默认）都做统计信息，指定列 o_comment 列压缩类型为查询高压缩率，压缩级别为 9。

3.5.1.4 定义水平分区表

水平分区包括范围分区、哈希分区和列表分区三种。水平分区表的创建需要通过<PARTITION 子句>指定。

范围（RANGE）分区，按照分区列的数据范围，确定实际数据存放位置的划分方式。

列表（LIST）分区，通过指定表中的某一个列的离散值集，来确定应当存储在一起的数据。范围分区是按照某个列上的数据范围进行分区的，如果某个列上的数据无法通过划分范围的方法进行分区，并且该列上的数据是相对固定的一些值，可以考虑使用 LIST 分区。一般来说，对于数字型或者日期型的数据，适合采用范围分区的方法；而对于字符型数据，取值比较固定的，则适合于采用 LIST 分区的方法。

哈希（HASH）分区，对分区列值进行 HASH 运算后，确定实际数据存放位置的划分方式，主要用来确保数据在预先确定数目的分区中平均分布，允许只建立一个 HASH 分区。在很多情况下，用户无法预测某个列上的数据变化范围，因而无法实现创建固定数量的范围分区或 LIST 分区。在这种情况下，DM 哈希分区提供了一种在指定数量的分区中均等地划分数据的方法， 基于分区键的散列值（HASH 值）将行映射到分区中。当用户向表中写入数据时，数据库服务器将根据一个哈希函数对数据进行计算，把数据均匀地分布在各个分区中。在哈希分区中，用户无法预测数据将被写入哪个分区中。

在很多情况下，经过一次分区并不能精确地对数据进行分类，这时需要多级分区表。在进行多级分区的时候，三种分区类型还可以交叉使用。

语法格式

```
CREATE TABLE [IF NOT EXISTS] <表名定义> <表结构定义>;
<表名定义> ::= [<模式名>.] <表名>
<表结构定义>::=<表结构定义 1>|<表结构定义 2>
<表结构定义 1>::= (<列定义> {,<列定义>} [,<表级约束定义>{,<表级约束定义>}])<PARTITION 子句>[<属性子句>] [<延迟段分配子句>] [<压缩子句>] [<ROW MOVEMENT 子句>][<add_log 子
```

句>][<DISTRIBUTE 子句>] [<AUTO_INCREMENT 子句>]

<表结构定义 2>::= [ON COMMIT <DELETE | PRESERVE> ROWS] <PARTITION 子句><空间限制子句><STORAGE 子句><压缩子句>[<表并行度子句>] [<高级日志子句>] [<add_log 子句>] AS <不带 INTO 的 SELECT 语句>[<DISTRIBUTE 子句>] [<AUTO_INCREMENT 子句>]

其中<列定义>、<表级约束定义>、<属性子句>、<延迟段分配子句>、<压缩子句>、<add_log 子句>、<DISTRIBUTE 子句>、<AUTO_INCREMENT 子句>、<空间限制子句>、<表并行度子句>、<高级日志子句>、<不带 INTO 的 SELECT 语句>等语法，参见 3.5.1.1 定义数据库基表说明

<PARTITION 子句> ::= PARTITION BY <PARTITION 项>[<分区表封锁子句>]

<PARTITION 项> ::=

RANGE (<列名>{,<列名>}) [INTERVAL(<间隔表达式>)] [<SUBPARTITION 子句>{,<SUBPARTITION 子句>}](<RANGE 分区项> {,<RANGE 分区项>}) |

HASH (<列名>{,<列名>}) [<SUBPARTITION 子句>{,<SUBPARTITION 子句>}]PARTITIONS <分区数> [<STORAGE HASH 子句>]|

HASH(<列名>{,<列名>})[<SUBPARTITION 子句>{,< SUBPARTITION 子句>}] (<HASH 分区项>{,<HASH 分区项>})|

LIST(<列名>)[<SUBPARTITION 子句>{,< SUBPARTITION 子句>}](<LIST 分区项> {,<LIST 分区项>})

<RANGE 分区项>::= PARTITION <RANGE 分区项子句>

<RANGE 分区项子句>::= <分区名> VALUES [EQU OR] LESS THAN (<常量表达式|<日期函数表达式>|MAXVALUE>{,<常量表达式|<日期函数表达式>|MAXVALUE>})[<表空间子句>][<STORAGE 子句>][<子分区描述项>]

<日期函数表达式> ::= <to_date 函数表达式> | <to_datetime 函数表达式> | <to_timestamp 函数表达式>

<HASH 分区项>::= PARTITION <HASH 分区项子句>

<HASH 分区项子句>::= <分区名> [<表空间子句>] [<STORAGE 子句>][<子分区描述项>]

<LIST 分区项>::= PARTITION <LIST 分区项子句>

<LIST 分区项子句>::= <分区名> VALUES (DEFAULT|<表达式>,{<表达式>}>) [<表空间子句>] [<STORAGE 子句>][<子分区描述项>]

<子分区描述项> ::=

(<RANGE 子分区描述项>{,<RANGE 子分区描述项>}) |

(<HASH 子分区描述项>{,<HASH 子分区描述项>}) |

SUBPARTITIONS <分区数> [<STORAGE HASH 子句>] |

(<LIST 子分区描述项>{,<LIST 子分区描述项>})

<RANGE 子分区描述项> ::= <RANGE 子分区项>[<子分区描述项>]

<HASH 子分区描述项> ::= <HASH 子分区项>[<子分区描述项>]

<LIST 子分区描述项> ::= <LIST 子分区项>[<子分区描述项>]

<RANGE 子分区项> ::= SUBPARTITION <RANGE 子分区项子句>

<RANGE 子分区项子句>::= <分区名> VALUES [EQU OR] LESS THAN (<常量表达式|<日期函数表达式>|MAXVALUE>){,<常量表达式|<日期函数表达式>|MAXVALUE>}) [<表空间子句>] [<STORAGE 子句>]

<HASH 子分区项> ::= SUBPARTITION <HASH 子分区项子句>

<HASH 子分区项子句> ::= <分区名> [<表空间子句>] [<STORAGE 子句>]

<LIST 子分区项> ::= SUBPARTITION <LIST 子分区项子句>

<LIST 子分区项子句> ::= <分区名> VALUES (DEFAULT|<<表达式>,{<表达式>}>) [<表空间子句>] [<STORAGE 子句>]

<间隔表达式> ::= <日期间隔函数> | <数值常量>

<SUBPARTITION 子句> ::= <RANGE 子分区模板项>|<HASH 子分区模板项>|<LIST 子分区模板项>

<RANGE 子分区模板项> ::= SUBPARTITION BY RANGE (<列名>{,<列名>})[SUBPARTITION TEMPLATE

```
(<RANGE 子分区项> {,<RANGE 子分区项>})]
<HASH 子分区模板项> ::=
    SUBPARTITION BY HASH (<列名>{,<列名>})SUBPARTITION TEMPLATE SUBPARTITIONS <分区数>
[<STORAGE HASH 子句>] |
    SUBPARTITION BY HASH (<列名>{,<列名>})SUBPARTITION TEMPLATE (<HASH 子分区项>
{,<HASH 子分区项>})
<LIST 子分区模板项> ::= SUBPARTITION BY LIST (<列名>{,<列名>})[SUBPARTITION TEMPLATE (<LIST
子分区项> {,<LIST 子分区项>})]
<STORAGE HASH 子句> ::= STORE IN (<表空间名列表>)
<分区表封锁子句> ::= LOCK PARTITIONS
<表空间子句> ::= 3.5.1.1 节中<表空间子句>
<STORAGE 子句> ::= STORAGE(<STORAGE 项> {,<STORAGE 项>})
<STORAGE 项> ::=
    3.5.1.1 节中<STORAGE 项> |
    HASHPARTMAP(<哈希分区表定位方式>)
<哈希分区表定位方式> ::= 0|1|2|3
<ROW MOVEMENT 子句> ::=
        ENABLE ROW MOVEMENT |
        DISABLE ROW MOVEMENT
```

参数

1. <模式名> 指明该表属于哪个模式，缺省为当前模式；

2. <表名> 指明被创建的基表名，基表名最大长度 128 字节；如果是分区表，主表名和分区名遵循“主表名_分区名”总长度最大不超过 128 字节。按数目方式创建 HASH 分区，子分区命名规则为“DMHASHPART+分区序号”， 其他方式的分区名由用户指定；

3. <子分区描述项> 多级分区表支持自定义子分区描述项，自定义的子分区描述项分区类型与分区列必须与子分区模板一致。如果子分区模板和自定义子分区描述项均指定了分区信息则按自定义分区描述项处理；

4. <STORAGE HASH 子句> 指定哈希分区依次使用的表空间；

5. <ROW MOVEMENT 子句> 设置行迁移功能，仅对行存储的水平分区表有效，其它表类型自动忽略。ENABLE ROW MOVEMENT 打开行迁移，允许更新后数据发生跨分区的移动。DISABLE ROW MOVEMENT，关闭行迁移，不允许更新后数据发生跨分区的移动。缺省为 DISABLE ROW MOVEMENT；

6. <间隔表达式> 中日期间隔函数为 NUMTOYMINTERVAL、NUMTODSINTERVAL；数值常量为：整型、DEC 类型。使用了<间隔表达式>的分区表称为间隔分区表。当对间隔分区表中的数据进行插入或更新操作时，若新的数据无法匹配现有的分区子表，则系统将自动以用户指定的现有分区的末尾临界值为起始值，以<间隔表达式>指定的值为间隔值创建一个可以匹配新数据的间隔分区。用户可通过查看系统表 SYSHPARTTABLEINFO 来获取新建分区的分区表 ID 以及分区名等信息。该功能可方便数据库管理员对分区表的管理；

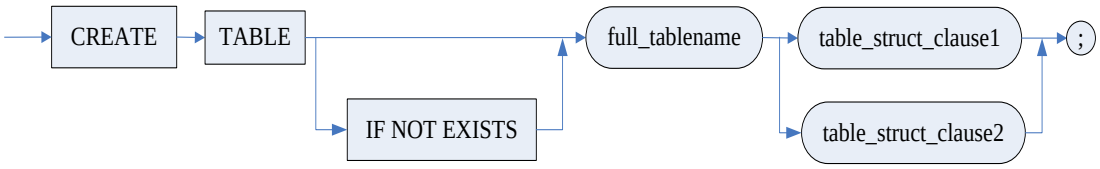
7. <分区表封锁子句>使用 LOCK PARTITIONS 关键字，指定分区表执行查询或增删子表 DDL 采用的封锁方式为细粒度封锁模式。在细粒度封锁模式下，增删子表 DDL 时从根表到目标子表路径上所有的主表都进行 IX+S 封锁，查询时同样对路径上所有的表都进行 IS 封锁，能够允许增删子表与查询的并发。指定 LOCK PARTITIONS 模式的分区表查询时，可能会增删子表，如果后续查询不涉及被增删的子表，则查询可以正常完成；否则查询可能报错。不支持将间隔分区表设置为 LOCK PARTITIONS 模式。由于指定 LOCK PARTITIONS 模式的分区表查询时可能会增删子表，所以该模式的分区表的查询计划不可重用，每次执行都需要重新分析；因此，在事务密集型环境中，LOCK PARTITIONS 配置需慎用。同样的原因，而语句块的所有语句属于同一个计划，所以语句块中对 LOCK PARTITIONS 模式的表的查询必须使用动态执行的方式。因增删 HASH 分区表子表会导致数据重组，所以即使指定了 LOCK

PARTITIONS 细粒度的封锁模式，增删 HASH 分区子表时，仍采用粗粒度封锁模式。指定 LOCK PARTITIONS 模式时，不支持 SPLIT 和 MERGE 分区。缺省为粗粒度封锁模式，即执行时只封锁根表的模式；

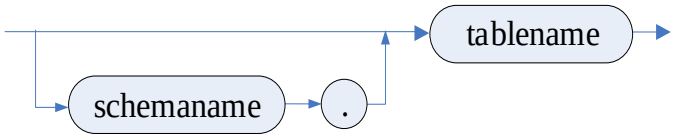
8. <表空间子句>不能和<STORAGE 子句>中的 ON <表空间名>同时使用。

图例

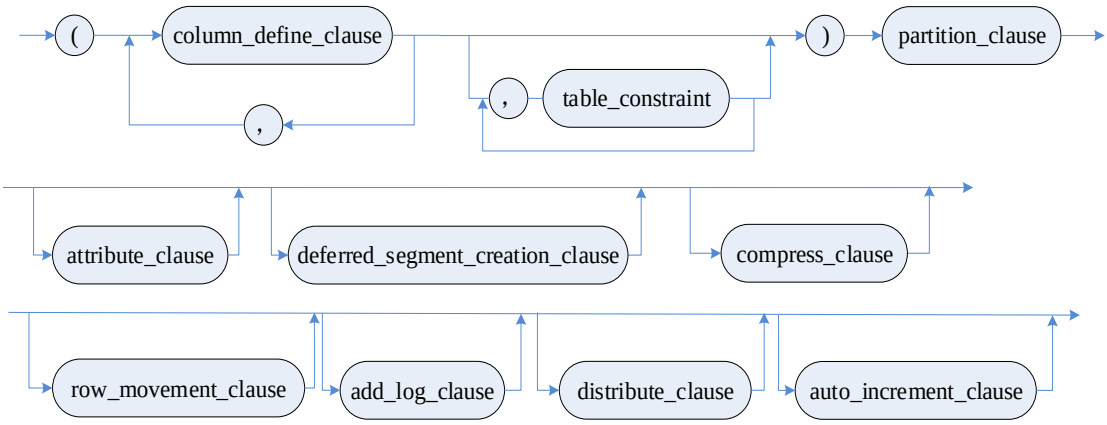
表定义语句



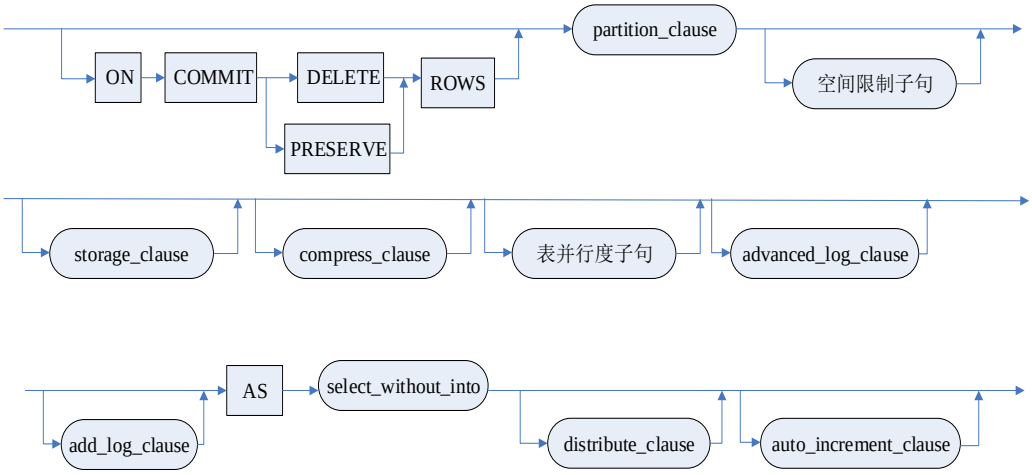
full_tablename



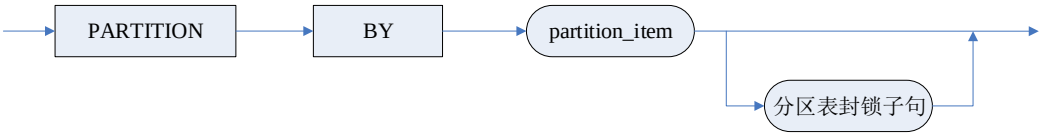
table_struct_clause1



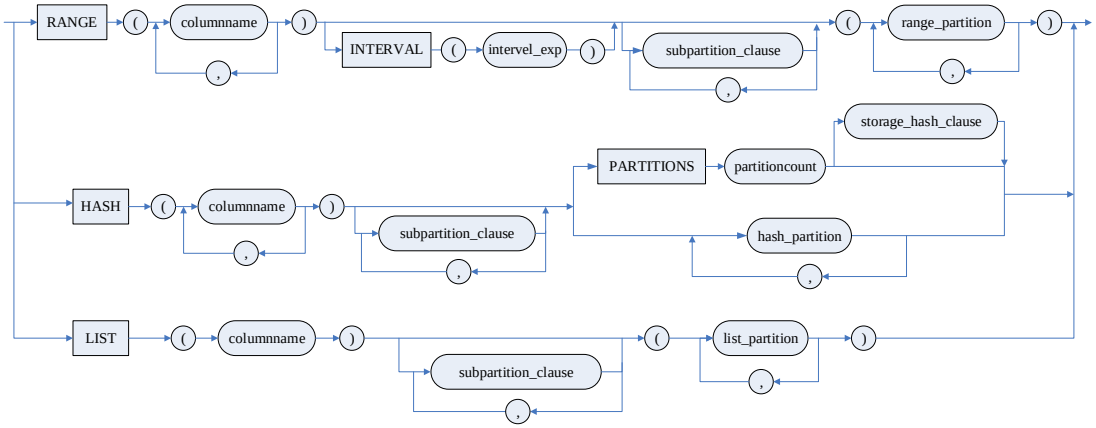
table_struct_clause2



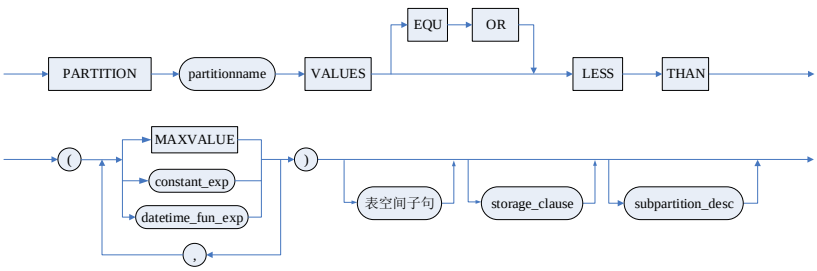
partition_clause



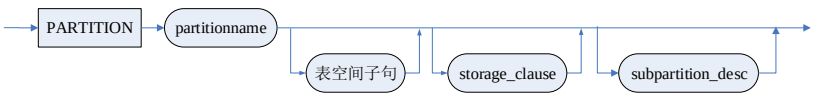
partition_item



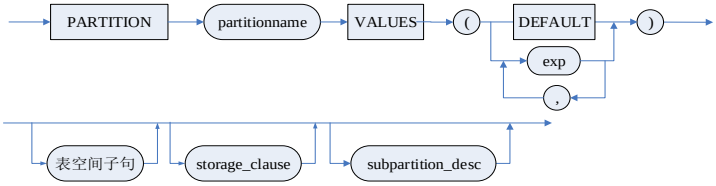
range_partition



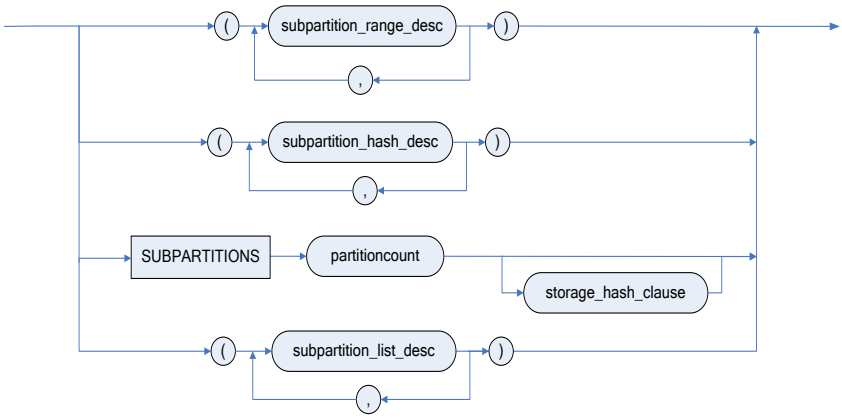
hash_partition



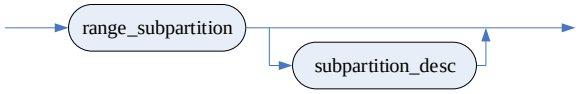
list_partition



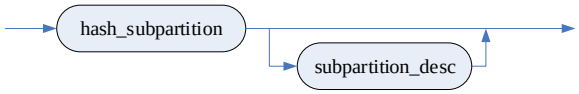
subpartition_desc



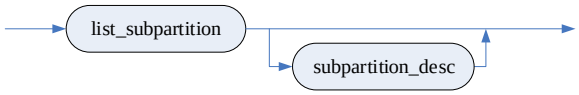
subpartition_range_desc



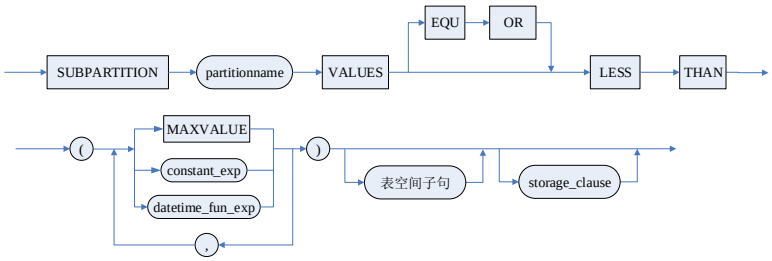
subpartition_hash_desc



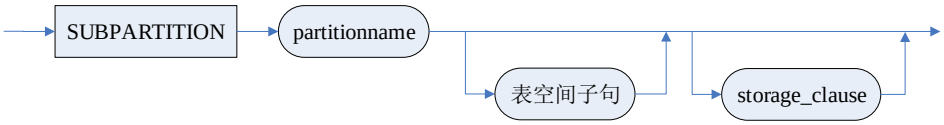
subpartition_list_desc



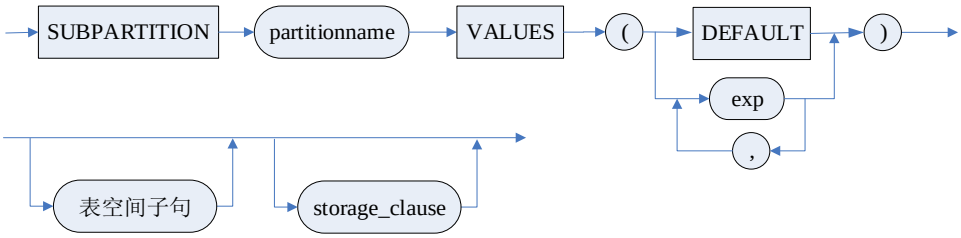
range_subpartition



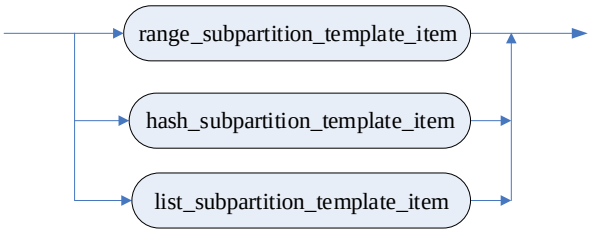
hash_subpartition



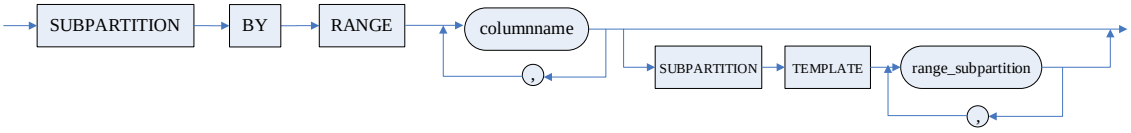
list_subpartition



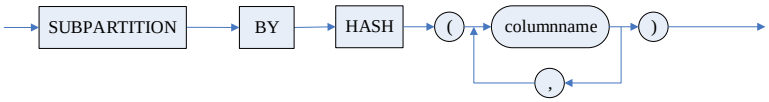
subpartition_clause

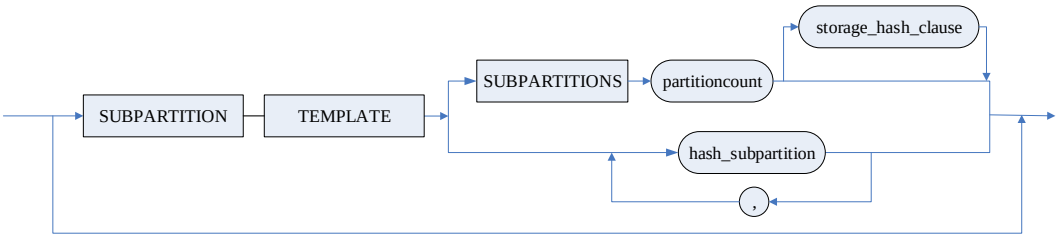


range_subpartition_template

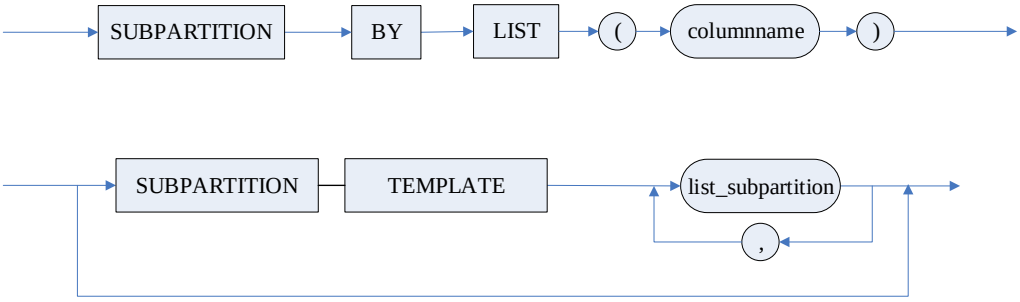


hash_subpartition_template

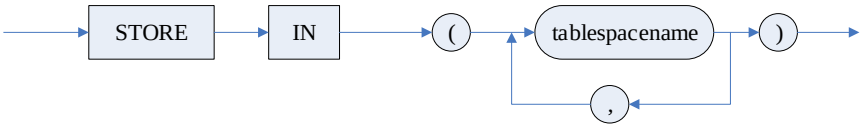




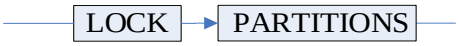
list_subpartition_template



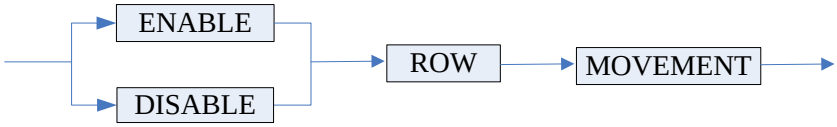
storage_hash_clause



<分区表封锁子句>



row_movement_clause



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE TABLE 或 CREATE ANY TABLE 权限的用户定义水平分区表。

使用说明

1. <表名> 指定所要建立的水平分区表名；

2. 表名前缀和后缀的限制规则请参考 3.5.1.1 定义数据库基表；

3. <PARTITION 子句> 用来指定水平分区。其中 RANGE 和 HASH 可以指定一个或多个列作为分区列，LIST 只能指定一个列作为分区列；

   1) “PARTITION BY RANGE……”子句用来指定范围分区，然后在每个分区中分区列的取值通过 VALUES 子句指定。

2）“PARTITION BY LIST……”子句用来指定 LIST 分区，然后在每个分区中分区列的取值通过 VALUES 子句指定。当用户向表插入数据时，只要分区列的数据与 VALUES 子句指定的数据之一相等，该行数据便会写入相应的分区子表中。LIST 分区的分区范围值必须唯一，不能重复。

3）“PARTITION BY HASH……”子句用来指定 HASH 分区。

4. 分区列类型必须是数值型、字符型或日期型，不支持 BLOB、CLOB、IMAGE、TEXT、LONGVARCHAR、BIT、BINARY、VARBINARY、LONGVARBINARY、BFILE、时间间隔类型、本地时区类型和用户自定义类型作为分区列；

5. 不能在水平分区表上建立自引用约束；

6. 普通环境中，水平分区表的各级分区数的总和上限是 65535；MPP 环境下，水平分区表的各级分区总数上限取决于 INI 参数 MAX_EP_SITES，上限为 $2^{(16 - \log_2 MAX_EP_SITES)}$。比如：当 MAX_EP_SITES 为默认值 64 时，分区总数上限为 1024；

7. 可以定义主表的 BRANCH 选项，但不能对水平分区子表进行 BRANCH 项设置，子表的 BRANCH 项只能通过主表继承得到；

8. 仅单机和 DMDSC 环境下，水平分区表支持自增列；

9. 不允许引用水平分区子表作为外键约束；

10. 水平分区子表删除后，会将子表上的数据一起删除；

11. 范围分区和哈希分区的分区键可以多个，每一层最多不超过 16 列；LIST 分区的分区键必须唯一；

12. 范围分区表使用说明：

1）范围分区支持 MAXVALUE 值的使用，MAXVALUE 代表一个比任何值都大的值。MAXVALUE 值需要用户指定才能使用，作为分区中的最大值。

2）范围分区的范围值表达式类型应与分区列类型一致，否则按分区列类型进行类型转换。

3） 对于范围分区，增加分区必须在最后一个分区范围值的后面添加，要想在表的开始范围或中间增加分区，应使用 SPLIT PARTITION 语句。

13. 间隔分区表使用说明：

1） 仅支持一级范围分区创建间隔分区。

2） 只能有一个分区列，且分区列类型为日期或数值。

3） 对间隔分区进行 SPLIT，只能在间隔范围内进行操作。

4） 被 SPLIT/MERGE 的分区，其左侧分区不再进行自动创建。

5） 不相邻的间隔的分区，不能 MERGE。

6） 表定义不能包含 MAXVALUE 分区。

7） 不允许新增分区。

8） 不能删除起始间隔分区。

9） 间隔分区表定义语句显示到起始间隔分区为止。

10）自动生成的间隔分区，均不包含边界值。

11） MPP 下不支持间隔分区表。

12）当间隔分区的“间隔”设置得过大时，可能导致触发扩展分区时计算次数超过上限，新增间隔分区失败。

14. LIST 分区表使用说明：

1） LIST 分区支持 DEFAULT 关键字的使用，所有不满足分区条件的数据，都划分为 DEFAULT 的分区，但 DEFAULT 关键字需要用户指定，系统不会自动创建 DEFAULT 分区。

2） LIST 分区子表范围值个数与分区列的数据类型、数据页大小和相关系统表列长度相关，存在以下限制：

a） 4K 页，单个子表最多支持 120 个范围值。

b） 8K 页，单个子表最多支持 254 个范围值。

c） 16K 或 32K 页，单个子表最多支持 270 个范围值。

15. 水平分区表为堆表时，主表及其各子表必须位于同一个表空间；

16. 组合水平分区表层次最多支持八层；

17. 普通表、堆表、列存储表均支持多级分区；

18. <哈希分区表定位方式> 用于指定哈希分区表的数据定位方式。取值 0 表示采用 DM 原有的数据定位方式；取值 1 表示采用新数据定位方式，此时可以添加分区；取值 2 为兼顾数据分布均衡和允许添加分区的数据定位方式；3 采用新的 hash 算法计算分区列哈希值，分区定位方式与 HASHPARTMAP(1) 保持一致。其默认值由 INI 参数 DEFAULT_HASHPARTMAP 指定，该参数默认为 1。当分区表根表为 RANGE 或 LIST 分区表时也可以指定<HASHPARTMAP 定位方式>，用来影响下层 HASH 分区子表；

19. 创建水平分区表时，若表的 PRIMARY KEY 未包含所有分区列，系统会自动创建全局索引，否则自动创建局部索引。

举例说明

例 1 创建一个范围分区表 callinfo，用来记录用户 2018 年的电话通讯信息，包括主叫号码、被叫号码、通话时间和时长，并且根据季度进行分区。

```
CREATE  TABLE  callinfo(
caller    CHAR(15),
callee    CHAR(15),
time  DATETIME,
duration  INT
)
PARTITION BY RANGE(time) (
PARTITION p1 VALUES LESS THAN ('2018-04-01'),
PARTITION p2 VALUES LESS THAN ('2018-07-01'),
```

```
PARTITION p3 VALUES LESS THAN ('2018-10-01'),
PARTITION p4 VALUES EQU OR LESS THAN ('2018-12-31'));
                                        //'2018-12-31'也可替换为 MAXVALUE
```

表中的每个分区都可以通过“PARTITION”子句指定一个名称。并且每一个分区都有一个范围，通过“VALUES LESS THAN”子句可以指定上界，而它的下界是前一个分区的上界。如分区 p2 的 time 字段取值范围是['2018-04-01', '2018-07-01')。如果通过“VALUES EQU OR LESS THAN”指定上界，即该分区包含上界值，如分区 p4 的 time 字段取值范围是['2018-10-01', '2018-12-31']。另外，可以对每一个分区指定 STORAGE 子句，不同分区可存储在不同表空间中。

例 2 查询分区子表，直接使用子表名称进行查询。

当在分区表中执行 DML 操作时，实际上是在各个分区子表上透明地修改数据。当执行 SELECT 命令时，可以指定查询某个分区上的数据。

例如，查询 callinfo 表中分区 p1 的数据，可以通过以下方式：

```
SELECT * FROM callinfo PARTITION (p1);
```

例 3 创建一个间隔分区表 ages，统计居民的年龄分布情况。

```
CREATE  TABLE  ages(
name  VARCHAR(30),
age   INT
)
PARTITION BY RANGE(age) INTERVAL(10)(
PARTITION p1 VALUES EQU OR LESS THAN (18),
PARTITION p2 VALUES EQU OR LESS THAN (35),
PARTITION p3 VALUES EQU OR LESS THAN (60),
PARTITION p4 VALUES EQU OR LESS THAN (100));
```

```
INSERT INTO ages VALUES('张三',121);
INSERT INTO ages VALUES('李四',120);
INSERT INTO ages VALUES('王五',103);
INSERT INTO ages VALUES('赵六',100);
```

“张三”、“李四”和“王五”的年龄无法匹配任何一个现有分区表，因此系统自动创建间隔分区以容纳新插入的值。本例中系统将以 100 为起始值，以 10 为间隔值创建 2 个间隔分区，第一个间隔分区的范围为 120~129，第二个间隔分区的范围为 100~109，由于用户定义的 p4 分区中包含值 100，因此值 100 会被划分到 p4 分区中。

1．通过查询系统表 SYSOBJECTS 获取表 ages 的 ID

```
SELECT NAME,ID FROM SYSOBJECTS WHERE NAME='AGES';
```

查询结果如下：

```
行号        NAME ID
---------- ---- -----------
1           AGES 1250
```

2．通过查询系统表 SYSHPARTTABLEINFO 获取表 ages 中的分区表信息

```
SELECT BASE_TABLE_ID,PART_TABLE_ID,PARTITION_NAME FROM SYSHPARTTABLEINFO WHERE
BASE_TABLE_ID=1250;
```

查询结果如下：

```
行号      BASE_TABLE_ID PART_TABLE_ID PARTITION_NAME
---------- ------------- ------------- --------------
1          1250          1251          P1
2          1250          1252          P2
3          1250          1253          P3
4          1250          1254          P4
```

```
5          1250          1256          SYS_P1250_1255

6          1250          1258          SYS_P1250_1257
```

3．查询 P4 分区表数据

```
SELECT * FROM ages PARTITION (p4);
```

查询结果如下:

```
行号          NAME    AGE

---------- ------ -----------

1             赵六    100
```

查询 SYS_P1250_1255 分区表数据

```
SELECT * FROM ages PARTITION (SYS_P1250_1255);
```

查询结果如下:

```
行号          NAME    AGE

---------- ------ -----------

1             张三    121

2             李四    120
```

查询 SYS_P1250_1257 分区表数据

```
SELECT * FROM ages PARTITION (SYS_P1250_1257);
```

查询结果如下:

```
行号          NAME    AGE

---------- ------ -----------

1             王五    103
```

例 4 创建多列分区。创建一个范围分区表 callinfo，以 time，duration 两列为分区列。

```
CREATE  TABLE  callinfo(

caller  CHAR(15),
```

```
callee   CHAR(15),
time   DATETIME,
duration   INT
)
PARTITION BY RANGE(time, duration)(
PARTITION p1 VALUES LESS THAN ('2018-04-01',10),
PARTITION p2 VALUES LESS THAN ('2018-07-01',20),
PARTITION p3 VALUES LESS THAN ('2018-10-01',30),
PARTITION p4 VALUES EQU OR LESS THAN ('2018-12-31', 40) );

insert into CALLINFO values('CHERRY','JACK','2018-12-31',40);
insert into CALLINFO values('CHERRY','JACK','2018-12-31',41); //报错：没有找到合适的分区
```

如果分区表包含多个分区列，采用多列比较方式定位匹配分区。首先，比较第一个分区列值，如果第一列值在范围之内，就以第一列为依据进行分区；如果第一列值处于边界值，那么需要比较第二列的值，根据第二列为依据进行分区；如果第二列的值也处于边界值，需要继续比较后续分区列值，以此类推，直到确定目标分区为止。匹配过程参看下表。

表 3.5.2 多分区列匹配

插入记录	分区范围值		
	(10,10,10)	(20,20,20)	(30,30,30)
(5,100,200)	满足		
(10,10,11)	×	满足	
(20,20,29)	×	×	满足
(31,1,1)	×	×	×

例 5 创建一个产品销售记录表 sales，记录产品的销量情况。由于产品只在几个固定的城市销售，所以可以按照销售城市对该表进行 LIST 分区。

```
CREATE   TABLE   sales(
```

```
sales_id   INT,
saleman    CHAR(20),
saledate   DATETIME,
city       CHAR(10)
)
PARTITION BY LIST(city)(
PARTITION p1 VALUES ('北京', '天津'),
PARTITION p2 VALUES ('上海', '南京', '杭州'),
PARTITION p3 VALUES ('武汉', '长沙'),
PARTITION p4 VALUES ('广州', '深圳'),
PARTITION p5 VALUES (default)
);
```

例 6 如果销售城市不是相对固定的，而是遍布全国各地，这时很难对表进行 LIST 分区。如果为该表进行哈希分区，可以很好地解决这个问题。

```
CREATE  TABLE  sales01(
sales_id   INT,
saleman    CHAR(20),
saledate   DATETIME,
city       CHAR(10)
)
PARTITION BY HASH(city)(
PARTITION p1,
PARTITION p2,
PARTITION p3,
```

```
PARTITION p4
);
```

如果不指定分区表名，还可以通过指定哈希分区个数来建立哈希分区表。

```
CREATE  TABLE  sales02(
sales_id     INT,
saleman      CHAR(20),
saledate     DATETIME,
city         CHAR(10)
)
PARTITION BY HASH(city)
PARTITIONS 4 STORE IN (ts1, ts2, ts3, ts4);
```

例 7 创建一个产品销售记录表 sales，记录产品的销量情况。由于产品需要按地点和销售时间进行统计，则可以对该表进行多级分区，一级 LIST 分区、二级 RANGE 分区。

```
DROP TABLE SALES;
CREATE  TABLE  SALES(
SALES_ID    INT,
SALEMAN     CHAR(20),
SALEDATE    DATETIME,
CITY        CHAR(10)
)
PARTITION BY LIST(CITY)
 SUBPARTITION BY RANGE(SALEDATE) SUBPARTITION TEMPLATE(
     SUBPARTITION P11 VALUES LESS THAN ('2012-04-01'),
     SUBPARTITION P12 VALUES LESS THAN ('2012-07-01'),
```

```
    SUBPARTITION P13 VALUES LESS THAN ('2012-10-01'),
    SUBPARTITION P14 VALUES EQU OR LESS THAN (MAXVALUE))
(
    PARTITION P1 VALUES ('北京', '天津')
    (
      SUBPARTITION P11_1 VALUES LESS THAN ('2012-10-01'),
      SUBPARTITION P11_2 VALUES EQU OR LESS THAN (MAXVALUE)
    ),
    PARTITION P2 VALUES ('上海', '南京', '杭州'),
    PARTITION P3 VALUES (DEFAULT)
);
```

在创建多级分区表时，指定了子分区模板，同时子分区 P1 又自定义了子分区描述项 P11_1 和 P11_2。除了 P1 使用自定义的子分区描述项，拥有两个自定义的子分区 P11_1 和 P11_2 之外，其他子分区 P2 和 P3 都使用子分区模板，各自拥有四个子分区 P11、P12、P13 和 P14。如下表所示：

表 3.5.3 子分区情况

父表	一级分区	二级分区
SALES	P1	P11_1
		P11_2
	P2	P11
		P12
		P13
		P14

<table>
<tr><td rowspan="4"></td><td rowspan="4">P3</td><td>P11</td></tr>
<tr><td>P12</td></tr>
<tr><td>P13</td></tr>
<tr><td>P14</td></tr>
</table>

例 8 创建一个三级分区，更多级别的分区表的建表语句按照语法类推。

```
CREATE TABLE STUDENT(
NAME VARCHAR(20),
AGE INT,
SEX VARCHAR(10) CHECK (SEX IN ('MALE','FEMALE')),
GRADE INT CHECK (GRADE IN (7,8,9))
)
PARTITION BY LIST(GRADE)
  SUBPARTITION BY LIST(SEX) SUBPARTITION TEMPLATE
  (
     SUBPARTITION Q1 VALUES('MALE'),
     SUBPARTITION Q2 VALUES('FEMALE')
  ),
  SUBPARTITION BY RANGE(AGE) SUBPARTITION TEMPLATE
  (
     SUBPARTITION R1 VALUES LESS THAN (12),
     SUBPARTITION R2 VALUES LESS THAN (15),
     SUBPARTITION R3 VALUES LESS THAN (MAXVALUE)
   )
(
```

```
  PARTITION P1 VALUES (7),
  PARTITION P2 VALUES (8),
  PARTITION P3 VALUES (9)
);
```

本例子中各分区表的表名详细介绍如下:

表 3.5.4 分区表详情

父表	一级分区	二级分区	三级分区
STUDENT	P1	Q1	R1
			R2
			R3
		Q2	R1
			R2
			R3
	P2	Q1	R1
			R2
			R3
		Q2	R1
			R2
			R3
	P3	Q1	R1
			R2
			R3
		Q2	R1

			R2
			R3

3.5.2 表修改语句

3.5.2.1 修改数据库表

为了满足用户在建立应用系统的过程中需要调整数据库结构的要求，DM 系统提供表修改语句。可对表的结构进行全面的修改，包括修改表名、列名、增加列、删除列、修改列类型、增加表级约束、删除表级约束、设置列缺省值、设置触发器状态等一系列修改。系统只提供外部表的文件（控制文件或数据文件）路径修改功能，如果想更改外部表的表结构，可以通过重建外部表来实现。

语法格式

```
ALTER TABLE [<模式名>.]<表名> <修改表定义子句>

<修改表定义子句> ::=

MODIFY <列定义>|

ADD [COLUMN] [IF NOT EXISTS] <列定义>|

ADD [COLUMN] [IF NOT EXISTS] (<列定义> {,<列定义>})|

REBUILD COLUMNS|

DROP [COLUMN] [IF EXISTS] <列名> [RESTRICT | CASCADE] |

ADD [CONSTRAINT [<约束名>] ] <表级约束子句> [<失效生效选项>] [<VALIDATE 选项>] |

RENAME CONSTRAINT <约束名 1> TO <约束名 2> |

RENAME COLUMN <列名 1> TO <列名 2> |

DROP CONSTRAINT <约束名> [RESTRICT | CASCADE] |

DROP CONSTRAINT <约束名> [CASCADE] {[DROP|KEEP] INDEX}|

ALTER [COLUMN] <列名> SET DEFAULT [ON NULL] <列缺省值表达式>|
```

```
ALTER [COLUMN] <列名> DROP DEFAULT |

ALTER [COLUMN] <列名> RENAME TO <列名> |

ALTER [COLUMN] <列名> SET <NULL | NOT NULL>|

ALTER [COLUMN] <列名> SET [NOT] VISIBLE|

ALTER [COLUMN] <列名> ADD USER (<用户名> {,<用户名>})|

ALTER [COLUMN] <列名> DROP USER (<用户名> {,<用户名>})|

RENAME TO <表名> |

ENABLE ALL TRIGGERS |

DISABLE ALL TRIGGERS |

MODIFY <空间限制子句>|

MODIFY CONSTRAINT <约束名> TO <表级约束子句> [<VALIDATE 选项>] [RESTRICT | CASCADE]|

MODIFY CONSTRAINT <约束名> ENABLE [<VALIDATE 选项>]|

MODIFY CONSTRAINT <约束名> DISABLE [<VALIDATE 选项>] [RESTRICT | CASCADE] |

WITH COUNTER |

WITHOUT COUNTER |

MODIFY PATH <外部表文件路径> |

DROP IDENTITY|

DROP AUTO_INCREMENT|

ADD [COLUMN] <列名> <自增列子句>|

AUTO_INCREMENT [=] <起始边界值>|

ENABLE CONSTRAINT <约束名> [<VALIDATE 选项>]|

DISABLE CONSTRAINT <约束名> [<VALIDATE 选项>] [RESTRICT | CASCADE] |

DEFAULT DIRECTORY <目录名>|

LOCATION ('<文件名>')|
```

ENABLE USING LONG ROW|

DISABLE USING LONG ROW|

ADD LOGIC LOG |

DROP LOGIC LOG |

WITHOUT ADVANCED LOG |

TRUNCATE ADVANCED LOG |

TRUNCATE PARTITION <分区名> [DROP STORAGE | REUSE STORAGE] |

TRUNCATE PARTITION (<分区名>) [DROP STORAGE | REUSE STORAGE] |

TRUNCATE SUBPARTITION <子分区名> [DROP STORAGE | REUSE STORAGE] |

TRUNCATE SUBPARTITION (<子分区名>) [DROP STORAGE | REUSE STORAGE] |

MOVE TABLESPACE <表空间名>|

PARALLEL|

PARALLEL<并行度>|

NOPARALLEL|

DROP PRIMARY KEY [RESTRICT | CASCADE]|

READ WRITE |

READ ONLY

<列定义>、<空间限制子句>、<表级约束子句>::=请参考 3.5.1.1 定义数据库基表

<VALIDATE 选项>::=[NOVALIDATE | VALIDATE]

参数

1. <模式名> 指明被操作的基表属于哪个模式，缺省为当前模式；
2. <表名> 指明被操作的基表的名称；
3. <列名> 指明修改、增加或被删除列的名称；
4. <数据类型> 指明修改或新增列的数据类型；

5. <列缺省值> 指明新增/修改列的缺省值，其数据类型与新增/修改列的数据类型一致；

6. <空间限制子句> 分区表不支持修改空间限制；

7. <VALIDATE 选项> 设置在添加、修改约束的时候，是否对表中的数据进行约束检查，若指明 NOVALIDATE 则不检查原有数据，只检查新数据；使约束生效时，默认 VALIDATE；使约束失效时，默认 NOVALIDATE；暂不支持 DISABLE VALIDATE；注意，当前仅对唯一约束和外键约束生效，其他约束仅语法支持；当有老数据不符合约束时，不允许创建聚集索引；

8. <外部表文件路径> 指明新的文件在操作系统下的路径 + 新文件名。数据文件的存放路径符合 DM 安装路径的规则，且该路径必须是已经存在的；

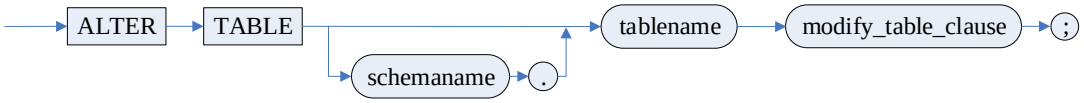
9. <表空间名> 指明被操作的基表移动的目标表空间。DPC 下仅非分区表支持单机迁移的语法，其限制与《DM9 分布计算集群》中的“基于表空间的迁移”章节所介绍内容一致。

10. <并行度> 创建数据表时，为 TABLE 指定并行度，并行度的合法值范围为 0~128；

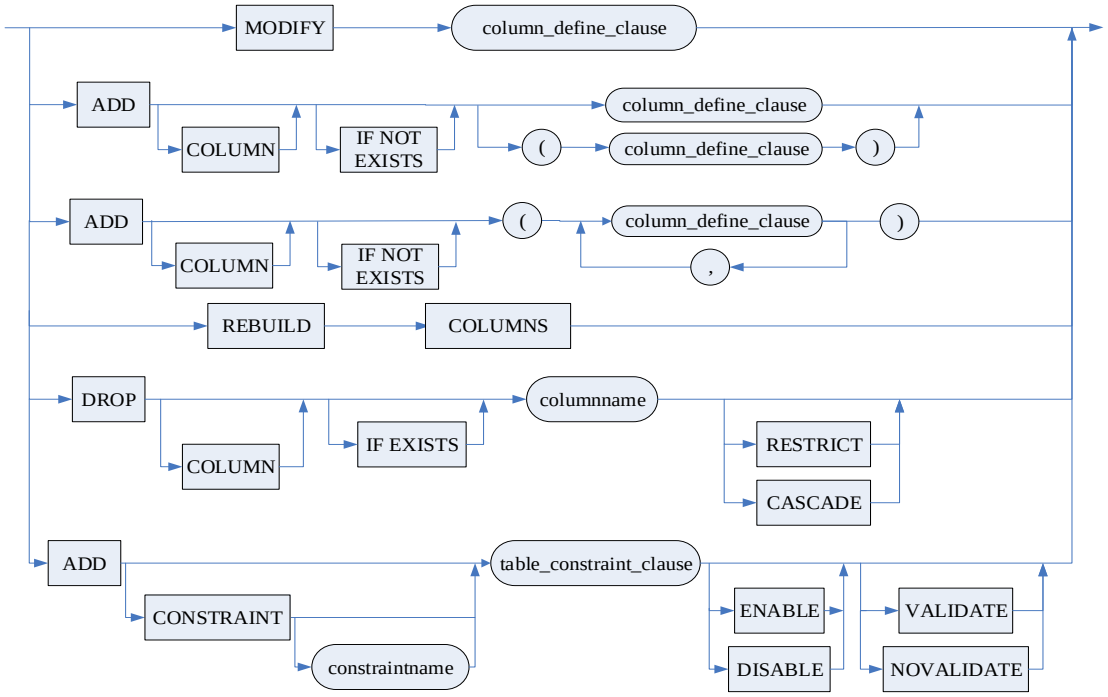
11. AUTO_INCREMENT [=] <起始边界值> 指定的<起始边界值>必须大于当前系统中的<起始边界值>。如果小于，则修改无效。

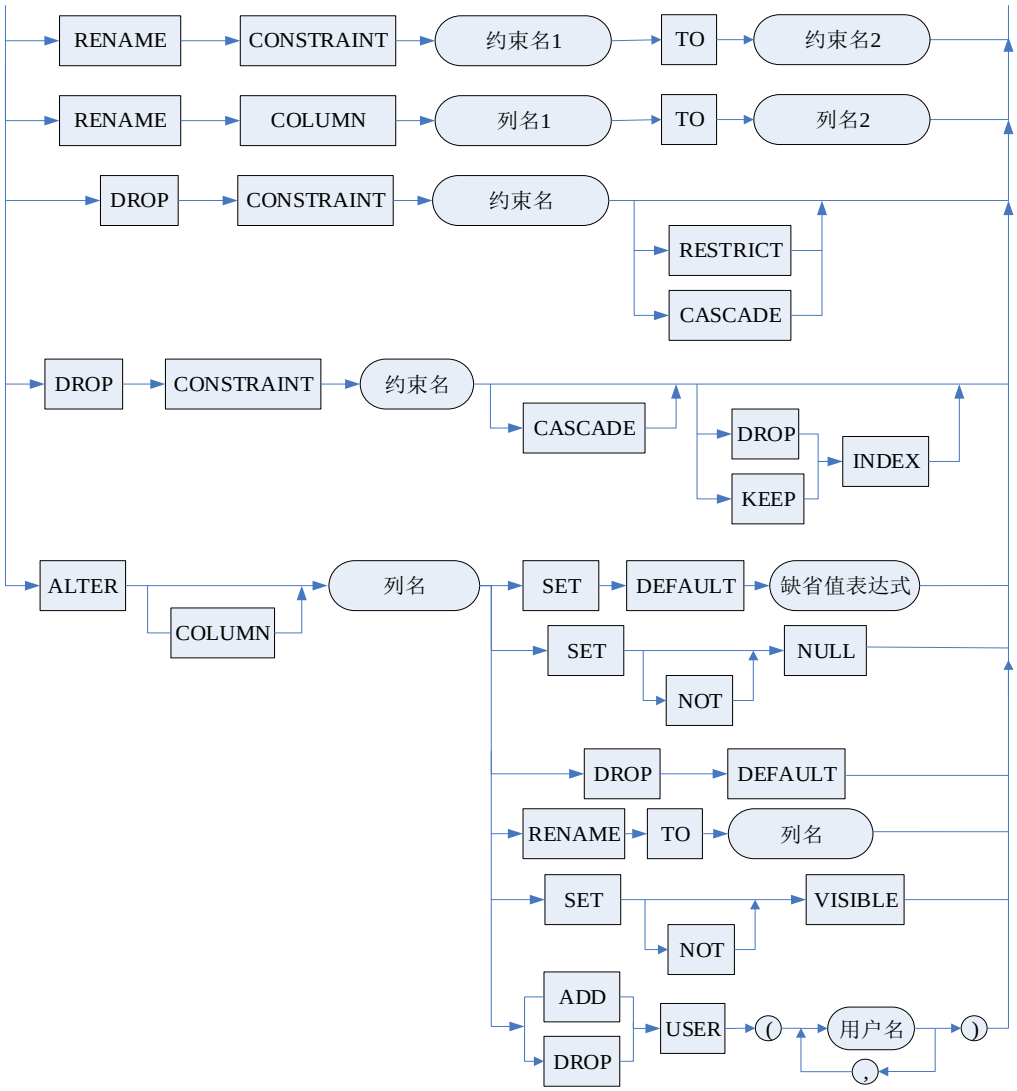
图例

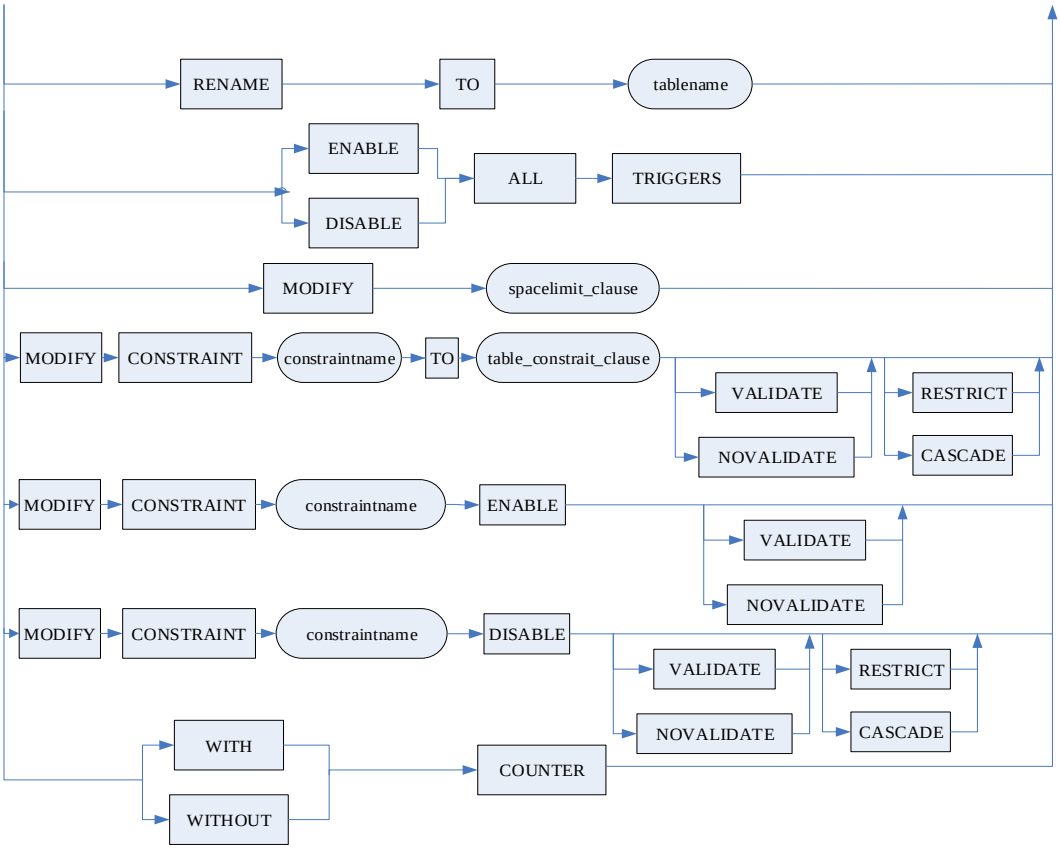
表修改语句

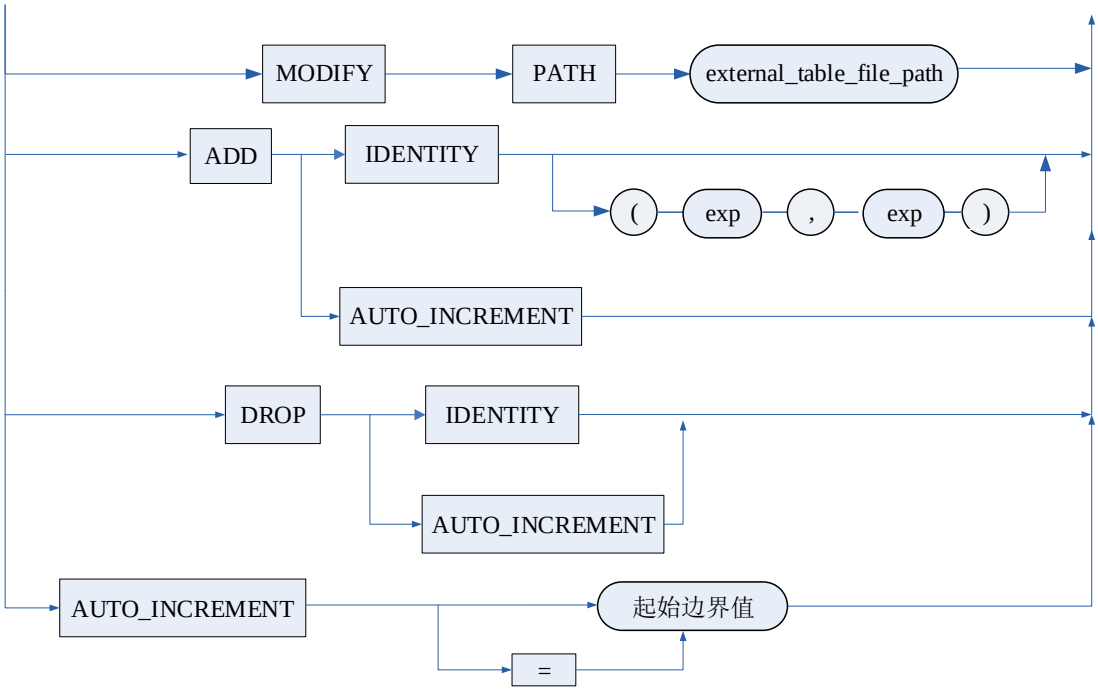


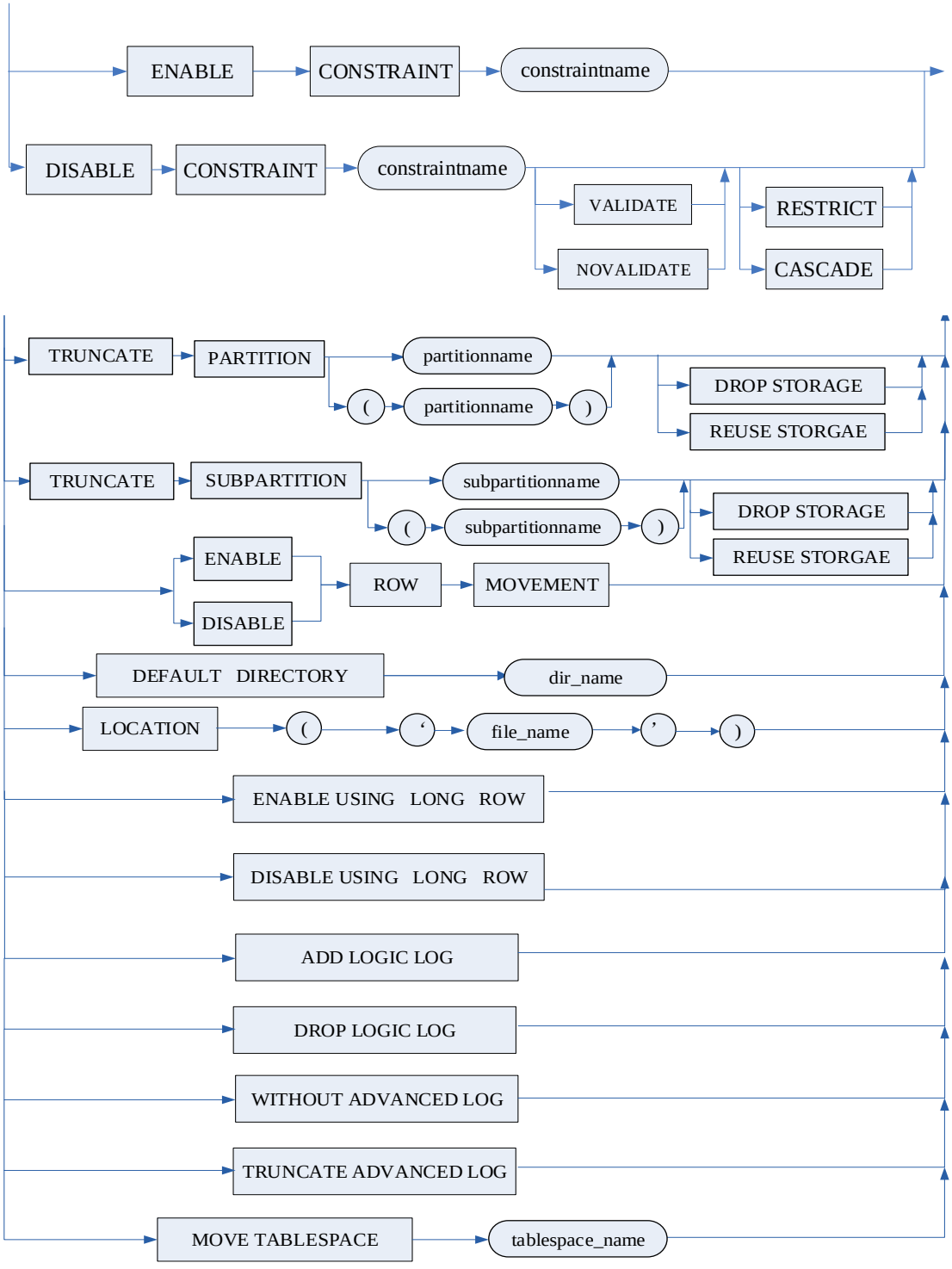
modify_table_clause

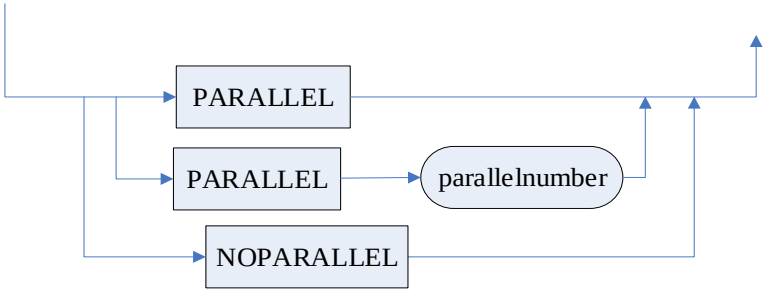

MODIFY
column_define_clause
ADD
COLUMN
IF NOT EXISTS
column_define_clause
(
column_define_clause
)
ADD
COLUMN
IF NOT EXISTS
(
column_define_clause
,
)
REBUILD
COLUMNS
DROP
COLUMN
IF EXISTS
columnname
RESTRICT
CASCADE
ADD
CONSTRAINT
constraintname
table_constraint_clause
ENABLE
DISABLE
VALIDATE
NOVALIDATE

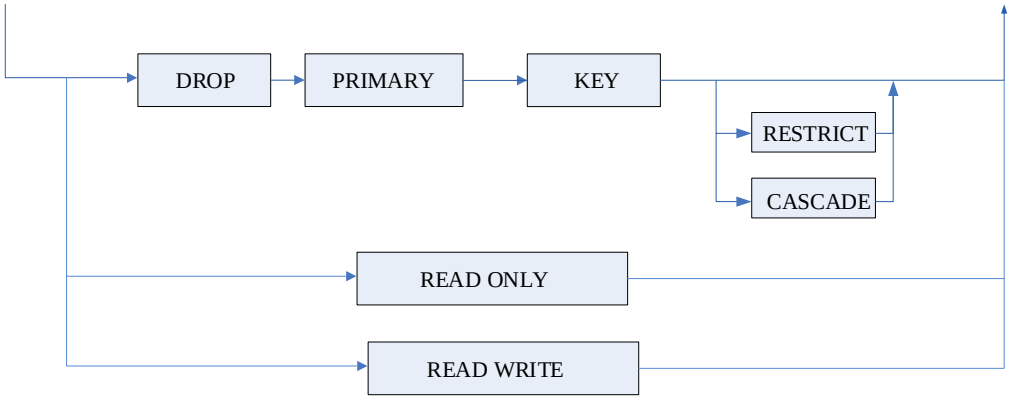

RENAME
CONSTRAINT
约束名1
TO
约束名2
RENAME
COLUMN
列名1
TO
列名2
DROP
CONSTRAINT
约束名
RESTRICT
CASCADE
DROP
CONSTRAINT
约束名
CASCADE
DROP
INDEX
KEEP
ALTER
COLUMN
列名
SET
DEFAULT
缺省值表达式
SET
NULL
NOT
DROP
DEFAULT
RENAME
TO
列名
SET
VISIBLE
NOT
ADD
DROP
USER
(
用户名
,
)


RENAME
TO
tablename
ENABLE
DISABLE
ALL
TRIGGERS
MODIFY
spacelimit_clause
MODIFY
CONSTRAINT
constraintname
TO
table_constrait_clause
VALIDATE
NOVALIDATE
RESTRICT
CASCADE
MODIFY
CONSTRAINT
constraintname
ENABLE
VALIDATE
NOVALIDATE
MODIFY
CONSTRAINT
constraintname
DISABLE
VALIDATE
NOVALIDATE
RESTRICT
CASCADE
WITH
WITHOUT
COUNTER

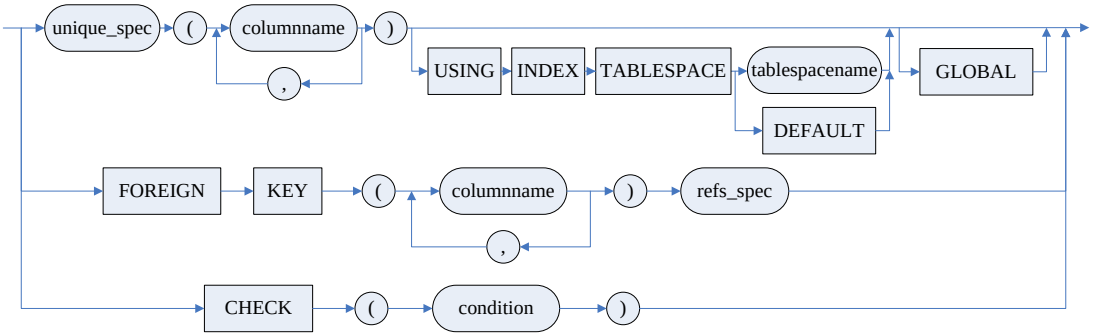

MODIFY
PATH
external_table_file_path
ADD
IDENTITY
(
exp
,
exp
)
AUTO_INCREMENT
DROP
IDENTITY
AUTO_INCREMENT
AUTO_INCREMENT
起始边界值
=


ENABLE
CONSTRAINT
constraintname
DISABLE
CONSTRAINT
constraintname
VALIDATE
NOVALIDATE
RESTRICT
CASCADE
TRUNCATE
PARTITION
partitionname
(
partitionname
)
DROP STORAGE
REUSE STORGAE
TRUNCATE
SUBPARTITION
subpartitionname
(
subpartitionname
)
DROP STORAGE
REUSE STORGAE
ENABLE
DISABLE
ROW
MOVEMENT
DEFAULT DIRECTORY
dir_name
LOCATION
(
‘
file_name
’
)
ENABLE USING LONG ROW
DISABLE USING LONG ROW
ADD LOGIC LOG
DROP LOGIC LOG
WITHOUT ADVANCED LOG
TRUNCATE ADVANCED LOG
MOVE TABLESPACE
tablespace_name

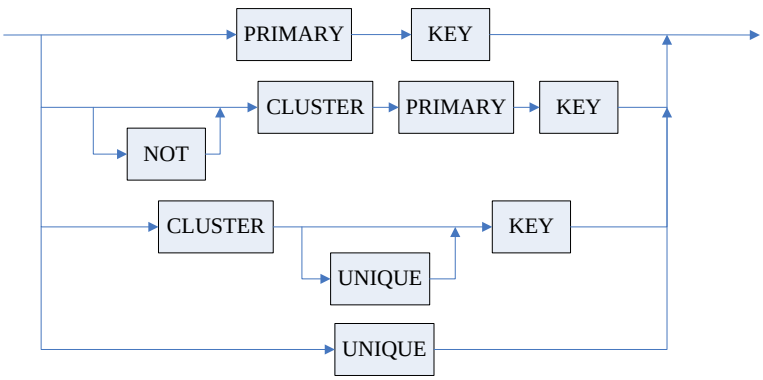




table_constraint_clause



unique_spec



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）的用户或该表的建表者或具有 ALTER ANY TABLE 权限的用户对表的定义进行修改。

修改包括：

1. 修改一列的数据类型、精度、刻度，设置列上的 DEFAULT、NOT NULL、NULL、VISIBLE、可见用户等；
2. 增加一列及该列上的列级约束；
3. 重建表上的聚集索引数据，消除附加列；
4. 删除一列；
5. 增加、删除表上的约束；
6. 启用、禁用表上的约束；
7. 表名/列名的重命名；
8. 修改触发器状态；
9. 修改表的最大存储空间限制；
10. 修改外部表的文件路径；
11. 启用、关闭超长记录（变长字符串）存储功能；
12. 增加、删除表上记录物理逻辑日志的功能；
13. 将表移动到目标表空间；
14. 删除表中的主键约束（PRIMARY KEY）；
15. 修改表的读写权限为只读或者可读可写。

使用说明

1. 各个子句中都可以含有单个或多个列定义（约束定义），单个列定义（约束定义）和多个列定义（约束定义）应该在它们的定义外加一层括号，括号要配对；
2. MODIFY <列定义>说明：
   1） 使用 MODIFY <列定义>时，支持不指定<数据类型>语法。在这种格式下，不

支持<STORAGE 子句>和<存储加密子句>属性的修改。仅修改<列定义子句>中指定的属性，其它属性保留原状。

2) 使用 MODIFY <列定义>时，不能更改聚集索引的列或者函数索引的列，位图、位图连接索引的列以及自增列不允许被修改。

3) 修改引用约束中引用和被引用的列时，列类型与引用的列类型或列类型与被引用的列类型须兼容。

4) 使用 MODIFY <列定义>子句能修改的约束有列上的 NULL/NOT NULL 约束、CHECK 约束、 唯一约束（不包括聚集唯一约束）、 主键约束（不包括聚集主键约束）；如果修改数据类型不变，则列上现有值必须满足约束条件；不允许被修改为自增列。

5) 使用 MODIFY <列定义>子句添加 NULL 约束时，不能重复添加；如果某列现有的值均非空，则允许添加 NOT NULL；主键约束和 NULL 约束不允许同时添加；表上已经有主键约束，则不可以添加主键约束。列上已有唯一约束和主键约束，则不允许添加唯一约束和主键约束。添加了唯一/主键约束就不能添加唯一/主键约束。

6) 使用 MODIFY <列定义>修改可更改列的数据类型时，若该表中无元组，则可任意修改其数据类型、长度、精度或量度和加密属性；若表中有元组，则系统会尝试修改其数据类型、长度、精度或量度，如果修改不成功，则会报错返回。

7) 修改有默认值的列的数据类型时，原数据类型与新数据类型必须是可以转换的，否则即使数据类型修改成功，但在进行插入等其他操作时，仍会出现数据类型转换错误。

3. ADD COLUMN 说明：

1) 当设置 INI 参数 ALTER_TABLE_OPT 为 1 时，添加列采用查询插入实现，可能会导致 ROWID 的改变；ALTER_TABLE_OPT 为 2 时，系统开启快速加列功能，

对于没有默认值或者默认值为 NULL 的新列，系统内部会标记为附加列，能够达到瞬间加列的效果，此时记录 ROWID 不会改变，若有默认值且默认值不为 NULL，则默认值的存储长度不能超过 4000 字节，此时仍旧采取查询插入实现；ALTER_TABLE_OPT 为 3 时，系统会开启快速加列功能，允许指定新增列的默认值，系统会为该列设置附加列标记，查询表中已存在的数据时，会自动为记录设置附加列默认值，此时记录 ROWID 不会改变。

2） 使用 ADD COLUMN 时，新增列名之间、新增列名与该基表中的其它列名之间均不能重复。若新增列有缺省值，则已存在的行的新增列值是其缺省值。添加新列对于任何涉及表的约束定义没有影响，对于涉及表的视图定义不会自动增加。例如：如果用“*”为一个表创建一个视图，那么后加入的新列不会加入到该视图。

3） 使用 ADD COLUMN 时，还有以下限制条件：

a） 列定义中如果带有列约束，只能是对该新增列的约束，但不支持引用约束、聚集唯一约束和聚集主键约束；列级约束可以带有约束名，系统中同一模式下的约束名不得重复，且不能指定为系统格式的索引名（格式为前缀关键字 INDEX 与数值组合：'INDEX< 系统索引数值>'，系统索引数值范围为 33554432~50331647）或系统格式的约束名（格式为前缀关键字 CONS 与数值组合：'CONS< 系统约束数值 >'，系统约束数值范围为 134217728~150994943）；如果不带约束名，系统自动为此约束命名；

b） 如果表上没有元组，列可以指定为 NOT NULL；如果表中有元组，对于已有列可以指定同时有 DEFAULT 和 NOT NULL，新增列不能指定 NOT NULL；

c） 该列可指定为 CHECK；

d） 该列可指定为 FOREIGN KEY；

e） 允许对空数据的表中，添加自增列；

f) ADD [COLUMN] <列名><IDENTITY 子句> 将表中的 NOT NULL 列修改为自增列。DM MPP 下不支持该功能;

g) HUGE 表加列仅允许一次增加一个列，且仅支持事务性 HUGE 表。HUGE 表加列不支持增加虚拟列、包含约束子句的列。

4. 使用 REBUILD COLUMNS 对添加过新列的表，重建表中的索引数据，消除附加列。水平分区表增加列后不允许使用 REBUILD COLUMNS;

5. 用 DROP COLUMN 子句删除一列有两种方式: RESTRICT 和 CASCADE。RESTRICT 方式为缺省选项，确保只有不被其他对象引用的列才被删除。无论哪种方式，表中的唯一列不能被删除。RESTRICT 方式下，当设置 INI 参数 DROP_CASCADE_VIEW = 1 或者 COMPATIBLE_MODE = 1时,下列类型的列不能被删除:被引用列、建有视图的列、有CHECK 约束的列。删除列的同时将删除该列上的约束。CASCADE 方式下，将删除这一列上的引用信息和被引用信息、引用该列的视图、索引和约束；系统允许直接删除 PK 列。但被删除列为 CLUSTER PRIMARY KEY 类型时除外，此时不允许删除;

6. ADD CONSTRAINT 子句用于添加表级约束。表级约束包括：PRIMARY KEY 约束、UNIQUE 约束、引用约束（REFERENCES）和检查约束（CHECK）。添加表级约束时可以带有约束名，系统中同一模式下的约束名不得重复，且不能指定为系统格式的索引名（格式为前缀关键字 INDEX 与数值组合：'INDEX<系统索引数值>'，系统索引数值范围为 33554432~50331647）或系统格式的约束名（格式为前缀关键字 CONS 与数值组合：'CONS<系统约束数值>'，系统约束数值范围为 134217728~150994943）；如果不带约束名，系统自动为此约束命名;

当 ENABLE_TMP_TAB_ROLLBACK 为 0 时,不允许对临时表创建主键约束以及唯一约束。

当添加 PRIMARY KEY 约束时，除手动指定 CLUSTER 或 NOT CLUSTER 关键字外，还可通过 INI 参数 PK_WITH_CLUSTER 控制该约束默认为 CLUSTER 或 NOT CLUSTER。其中，

若添加 CLUSTER PRIMARY KEY 约束，则将根据约束列重建聚集索引。

用 ADD CONSTRAINT 子句添加约束时，对于该基表上现有的全部元组要进行约束违规验证：

1） 添加一个 PRIMARY KEY 约束时，要求将成为关键字的字段上无重复值且值非空，并且表上没有定义主关键字；

2） 添加一个 UNIQUE 约束时，要求将成为唯一性约束的字段上不存在重复值，但允许有空值；

3） 添加一个 REFERENCES 约束时，要求将成为引用约束的字段上的值满足该引用约束。

4） 添加一个 CHECK 约束或外键时，要求该基表中全部的元组满足该约束。

7. RENAME CONSTRAINT 子句用于对表级约束进行重命名，新的名称不能为空，也不能与已有约束名重复，且不能指定为系统格式的索引名（格式为前缀关键字 INDEX 与数值组合：'INDEX<系统索引数值>'，系统索引数值范围为 33554432~50331647）或系统格式的约束名（格式为前缀关键字 CONS 与数值组合：'CONS<系统约束数值>'，系统约束数值范围为 134217728~150994943）；

8. RENAME COLUMN 子句用于对表列进行重命名，新的名称不能为空，也不能与表已有列名重复，且；

9. DROP CONSTRAINT<约束名> [RESTRICT | CASCADE]用于删除表级约束，表级约束包括：主键约束（PRIMARY KEY）、唯一性约束（UNIQUE）、引用约束（REFERENCES）和检查约束(CHECK)。用 DROP CONSTRAINT 子句删除约束时，有 RESTRICT 和 CASCADE 两种可选方式，缺省为 RESTRICT 方式。当删除主键或唯一性约束时，系统自动创建的索引也将一起删除。RESTRICT 确保只有不被其它外部约束引用的约束才被能删除。CASCADE 可删除约束和任何引用该约束的外部约束；[DROP|KEEP] INDEX 用于约束含有对应索引的情况，只能和 CASCADE 搭配使用，CASCADE 也可缺省。指定 DROP INDEX 删除约束的

同时删除对应的索引，指定 KEEP INDEX 则既不删除约束也不删除索引并报错返回；

10. 使用 SET DEFAULT [ON NULL] <列缺省值表达式>语句可以设置列默认值和 ON NULL 选项；

11. 使用 DROP DEFAULT 语句删除列缺省值时，会一并删除列上的隐式 NOT NULL 约束；

12. DISABLE CONSTRAINT 子句用于禁用约束，不支持禁用聚集唯一约束和聚集主键约束；

13. 空间限制子句说明：修改表的最大存储空间限制时，空间大小以 MB 为单位，取值范围在表的已占用空间和 1024*1024 之间，还可以利用 UNLIMITED 关键字去掉表的空间限制；

14. MODIFY PATH 说明：

    1) 只能用于修改外部表的文件路径，文件包括控制文件和数据文件；

    2) 只支持对应创建方式的文件路径修改：原表使用控制文件创建，则只支持修改控制文件路径；原表使用数据文件创建，则只支持修改数据文件路径。

15. DEFAULT DIRECTORY <目录名>和 LOCATION ('<文件名>')，专门用于修改外部表路径名和文件名；

16. ADD/DROP LOGIC LOG 开启或关闭物理逻辑日志记录；

17. WITHOUT/TRUNCATE ADVANCED LOG 用于删除日志辅助表和清除日志辅助表数据，具体可参考 19.3.1 管理日志辅助表；

18. MOVE TABLESPACE 用于将表及其索引移动到指定的表空间上。如果移动的是分区表，则连同所有分区子表一起移动。不支持移动 HUGE 表。移动后数据的 TRXID 和 ROWID 可能发生改变；

19. DROP STORAGE/REUSE STORAGE 用于删除或重用分区，默认值为 DROP STORAGE。在 DPC 环境下仅做语法兼容。

20. DROP PRIMARY KEY[RESTRICT|CASCADE]：用于删除表中的主键约束（PRIMARY KEY）。系统自动创建的与该主键相关的唯一索引也会一起被删除。有两种方式：RESTRICT 和 CASCADE。缺省为 RESTRICT 方式。RESTRICT 确保只有不被其它外部约束引用的主键约束才被能删除。CASCADE 可删除主键约束和任何引用该主键的外部约束。不允许删除聚集主键约束（CLUSTER PRIMARY KEY）。用户需要注意，当 INI 参数 PK_WITH_CLUSTER=1 时，缺省情况下创建的主键约束均为聚集主键约束。

21. USING INDEX 子句仅支持主键约束使用当前表已有的索引，且需要索引有效以及索引列定义与约束定义一致，不支持 HUGE 表。当使用非 UNIQUE 的局部索引时，需要保证索引列包含所有分区列。

22. ENABLE USING LONG ROW 用于开启超长记录存储功能，当 DM 行存储的记录长度超过页大小一半时，先尝试将过长的变长字符串转换为行外 BLOB 存储，如果转换后仍超长则报错。DISABLE USING LONG ROW 用于关闭超长记录存储功能，如果表中已有超长数据，则不能关闭超长记录存储功能。

23. 可以使用 READ WRITE 和 READ ONLY 设置表的读写权限。缺省情况下，表的读写权限是 READ WRITE 状态。

举例说明

例1 产品的评论表中COMMENTS、PRODUCT_REVIEWID、PRODUCTID、RATING列都不允许修改，分别因为：COMMENTS为多媒体数据类型；PRODUCT_REVIEWID上定义有关键字，属于用于索引的列；PRODUCTID用于引用约束（包括引用列和被引用列）；RATING用于CHECK约束。另外，关联有缺省值的列也不能修改。而其他列都允许修改。假定用户为SYSDBA，如将评论人姓名的数据类型改为VARCHAR(8)，并指定该列为NOT NULL，且缺省值为'刘青'。

```
ALTER TABLE PRODUCTION.PRODUCT_REVIEW MODIFY NAME VARCHAR(8) DEFAULT '刘青' NOT NULL;
```

此语句只有在表中无元组的情况下才能成功。

例2 具有DBA权限的用户需要对EMPLOYEE_ADDRESS表增加一列，列名为ID（序号），数据类型为INT，值小于10000。

```
ALTER TABLE RESOURCES.EMPLOYEE_ADDRESS ADD ID INT PRIMARY KEY CHECK (ID <10000);
```

如果该表上没有元组，且没有PRIMARY KEY，则可以将新增列指定为 PRIMARY KEY。表上没有元组时也可以将新增列指定为UNIQUE，但同一列上不能同时指定PRIMARY KEY和UNIQUE两种约束。

例3 具有DBA权限的用户需要对ADDRESS表增加一列，列名为PERSONID，数据类型为INT，定义该列为DEFAULT和NOT NULL。

```
ALTER TABLE PERSON.ADDRESS ADD PERSONID INT DEFAULT 10 NOT NULL;
```

如果表上没有元组，新增列可以指定为 NOT NULL；如果表上有元组且都不为空，该列可以指定同时有 DEFAULT 和 NOT NULL，不能单独指定为 NOT NULL。

例 4 具有 DBA 权限的用户需要删除 PRODUCT 表的 PRODUCT_SUBCATEGORYID 一列。

```
ALTER TABLE PRODUCTION.PRODUCT DROP PRODUCT_SUBCATEGORYID CASCADE;
```

删除PRODUCT_SUBCATEGORYID这一列必须采用CASCADE方式，因为该列引用了PRODUCT_SUBCATEGORY表的PRODUCT_SUBCATEGORYID。

例5 具有DBA权限的用户需要在PRODUCT表上增加UNIQUE约束，UNIQUE字段为NAME。

```
ALTER TABLE PRODUCTION.PRODUCT ADD CONSTRAINT CONS_PRODUCTNAME UNIQUE(NAME);
```

用ADD CONSTRAINT子句添加约束时，对于该基表上现有的全部元组要进行约束违规验证。在这里，分为三种情况：

1. 如果表商场登记里没有元组，则上述语句一定执行成功；
2. 如果表商场登记里有元组，并且欲成为唯一性约束的字段商场名上不存在重复值，

则上述语句执行成功；

3. 如果表商场登记里有元组，并且欲成为唯一性约束的字段商场名上存在重复值，则上述语句执行不成功，系统报错“违反唯一性约束”。

如果语句执行成功，用户通过

```
CALL SP_TABLEDEF('PRODUCTION', 'PRODUCT');
```

可以看到，修改后的商场登记的表结构显示为：

```
行号          COLUMN_VALUE

---------- --------------------------------------------

1             CREATE TABLE "PRODUCTION"."PRODUCT" ( "PRODUCTID" INT IDENTITY(1, 1) NOT NULL, "NAME" VARCHAR(50) NOT NULL, "AUTHOR" VARCHAR(25) NOT NULL, "PUBLISHER" VARCHAR(50) NOT NULL, "PUBLISHTIME" DATE NOT NULL, "PRODUCT_SUBCATEGORYID" INT NOT NULL, "PRODUCTNO" VARCHAR(25) NOT NULL, "SATETYSTOCKLEVEL" SMALLINT NOT NULL, "ORIGINALPRICE" DEC(19,4) NOT NULL, "NOWPRICE" DEC(19,4) NOT NULL, "DISCOUNT" DECIMAL(2,1) NOT NULL, "DESCRIPTION" TEXT, "PHOTO" IMAGE, "TYPE" VARCHAR(5), "PAPERTOTAL" INT,

2             "WORDTOTAL" INT, "SELLSTARTTIME" DATE NOT NULL, "SELLENDTIME" DATE, NOT CLUSTER PRIMARY KEY("PRODUCTID"), FOREIGN KEY("PRODUCT_SUBCATEGORYID") REFERENCES "PRODUCTION"."PRODUCT_SUBCATEGORY"("PRODUCT_SUBCATEGORYID"),

UNIQUE("PRODUCTNO"), CONSTRAINT "CONS_PRODUCTNAME" UNIQUE("NAME")) STORAGE(ON "BOOKSHOP", CLUSTERBTR) ;
```

例6 假定具有DBA权限的用户需要删除PRODUCT表上的NAME列的UNIQUE约束。当前的PRODUCT表结构请参见例5。

删除表约束，首先需要得到该约束对应的约束名，用户可以查询系统表 SYSOBJECTS，如下所示。

```
SELECT NAME FROM SYS.SYSOBJECTS WHERE SUBTYPE$='CONS' AND PID =
(SELECT ID FROM SYS.SYSOBJECTS WHERE NAME = 'PRODUCT' AND TYPE$='SCHOBJ' AND
SCHID=( SELECT ID FROM SYS.SYSOBJECTS  WHERE NAME='PRODUCTION' AND TYPE$='SCH'));
```

该系统表显示商场登记表上的所有PRIMARY KEY、UNIQUE、CHECK约束。查询得到NAME列上UNIQUE约束对应的约束名，这里为CONS_PRODUCTNAME。

然后，可采用以下的语句删除指定约束名的约束。

```
ALTER TABLE PRODUCTION.PRODUCT DROP CONSTRAINT CONS_PRODUCTNAME;
```

语句执行成功。

例 7 建立普通表查看 SELECT COUNT(*) 执行计划，再 ALTER 该表为 WITHOUT COUNTER 属性，再查看执行计划：

```
CREATE TABLE test(c1 INT);
EXPLAIN SELECT COUNT(*) FROM test;
```

执行结果如下：

```
1    #NSET2: [1, 1, 0]
2      #PRJT2: [1, 1, 0]; exp_num(1), is_atom(FALSE)
3        #FAGR2: [1, 1, 0]; sfun_num(1)
```

ALTER 该表为 WITHOUT COUNTER 属性，再查看执行计划。

```
ALTER TABLE test WITHOUT COUNTER;
EXPLAIN SELECT COUNT(*) FROM test;
```

执行结果如下：

```
1    #NSET2: [1, 1, 0]
2      #PRJT2: [1, 1, 0]; exp_num(1), is_atom(FALSE)
3        #AAGR2: [1, 1, 0]; grp_num(0), sfun_num(1), distinct_flag[0]; slave_empty(0)
4          #CSCN2: [1, 1, 0]; INDEX33555829(TEST); btr_scan(1)
```

例 8　创建、启用、禁用、删除 PERSON 表约束 unq。

```
ALTER TABLE PERSON.PERSON ADD CONSTRAINT unq UNIQUE(PHONE);

ALTER TABLE PERSON.PERSON ENABLE CONSTRAINT unq;

ALTER TABLE PERSON.PERSON DISABLE CONSTRAINT unq;

ALTER TABLE PERSON.PERSON DROP CONSTRAINT unq;
```

例 9　取消按列加密的 C1 列对用户 USER02 可见，增加对 USER03 可见。

```
ALTER TABLE T ALTER COLUMN C1 DROP USER (USER02);

ALTER TABLE T ALTER COLUMN C1 ADD USER (USER03);
```

3.5.2.2 修改水平分区表

本节专门列出水平分区表在分区方面的修改。其他方面的常规修改和普通表一样，普通表的修改方法在水平分区表上完全适用，详细请参考 3.5.2.1 修改数据库表部分。

语法格式

```
ALTER TABLE [<模式名>.]<表名> <修改表定义子句>

<修改表定义子句> ::=

MODIFY <增加多级分区子表>|

<删除一级分区子表>|

<删除多级分区子表>|

MODIFY <修改 LIST 分区子表>|

ADD PARTITION [IF NOT EXISTS] <水平分区项子句>|

EXCHANGE <PARTITION| SUBPARTITION > <分区名> WITH TABLE [<模式名.>]<表名>|

<SPLIT 子句>|

MERGE PARTITIONS <分区编号>,<分区编号>  INTO PARTITION <分区名> [<全局索引子句>] |

MERGE PARTITIONS <分区名>,<分区名>  INTO PARTITION <分区名> [<全局索引子句>] |

SET SUBPARTITION TEMPLATE <分区模板描述项> |
```

TRUNCATE PARTITION <分区名> [DROP STORAGE | REUSE STORAGE] [<全局索引子句>] |

TRUNCATE PARTITION (<分区名>)[DROP STORAGE | REUSE STORAGE] [<全局索引子句>] |

TRUNCATE PARTITION FOR (<分区列值>) [<全局索引子句>] |

TRUNCATE SUBPARTITION <子分区名> [DROP STORAGE | REUSE STORAGE] [<全局索引子句>] |

TRUNCATE SUBPARTITION (<子分区名>) [DROP STORAGE | REUSE STORAGE] [<全局索引子句>] |

TRUNCATE SUBPARTITION FOR (<分区列值>) [<全局索引子句>] |

ENABLE ROW MOVEMENT |

DISABLE ROW MOVEMENT |

RENAME <修改分区子表名> |

MOVE PARTITION <分区名> TABLESPACE <表空间名> |

MOVE SUBPARTITION <子分区名> TABLESPACE <表空间名>|

LOCK PARTITIONS

<增加多级分区子表>::= <PARTITION | SUBPARTITION> <分区名> ADD SUBPARTITION [IF NOT EXISTS]

<<RANGE 子分区项子句>|<LIST 子分区项子句>>

<删除一级分区子表>::=

    DROP PARTITION [IF EXISTS] <分区名> [<全局索引子句>] |

    DROP PARTITION FOR [IF EXISTS] (<分区列值>) [<全局索引子句>]

<删除多级分区子表>::=

    DROP SUBPARTITION [IF EXISTS] <分区名> [<全局索引子句>] |

    DROP SUBPARTITION FOR [IF EXISTS] (<分区列值>) [<全局索引子句>]

<修改 LIST 分区子表>::=<PARTITION | SUBPARTITION> <分区名> <ADD|DROP> VALUES(分区值[,分区值])

<SPLIT 子句>::=

<SPLIT 子句 1> |

<SPLIT 子句 2> |

<SPLIT 子句 3> |

<SPLIT 子句 4>

<SPLIT 子句 1>::=**SPLIT** PARTITION <分区名> AT (<表达式>{,<表达式>}) INTO ({PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]}, {PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]}) [<全局索引子句>]

<SPLIT 子句 2>::=**SPLIT** PARTITION <分区名> VALUES (<表达式>{,<表达式>}) INTO ({PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]}, {PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]}) [<全局索引子句>]

<SPLIT 子句 3>::=**SPLIT** PARTITION <分区名> INTO (<RANGE 分区项> {,<RANGE 分区项>}, PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]) [<全局索引子句>]

<SPLIT 子句 4>::=**SPLIT** PARTITION <分区名> INTO (<LIST 分区项> {,<LIST 分区项>}, PARTITION <分区名> [<表空间子句>] [<STORAGE 子句>]) [<全局索引子句>]

<修改分区子表名>::=

<修改一级分区子表名> |

<修改多级分区子表名>

<修改一级分区子表名>::=PARTITION <分区名> TO <新名称>

<修改多级分区子表名>::=SUBPARTITION <分区名> TO <新名称>

<全局索引子句>::=

UPDATE GLOBAL INDEXES |

IGNORE GLOBAL INDEXES

<分区列值>::=<常量|计算表达式{,常量|计算表达式}>

<水平分区项子句>::=<RANGE 分区项子句>|<HASH 分区项子句>|<LIST 分区项子句>

<RANGE 分区项>、<HASH 分区项>、<LIST 分区项>请参考 3.5.1.4 定义水平分区表

<RANGE 分区项子句>、<HASH 分区项子句>、<LIST 分区项子句>请参考 3.5.1.4 定义水平分区表

<RANGE 子分区项>、<HASH 子分区项>、<LIST 子分区项>请参考 3.5.1.4 定义水平分区表

<RANGE 子分区项子句>、<HASH 子分区项子句>、<LIST 子分区项子句>请参考 3.5.1.4 定义水平分区表

<表空间子句>请参考 3.5.1.4 定义水平分区表

<STORAGE 子句>请参考 3.5.1.4 定义水平分区表

<分区模板描述项>::= ([<分区模板描述项 1>])|<分区模板描述项 2>

<分区模板描述项 1>::=

<RANGE 子分区项> {,<RANGE 子分区项>} |

<HASH 子分区项> {,<HASH 子分区项>} |

<LIST 子分区项> {,<LIST 子分区项>}

<分区模板描述项 2>::= [SUBPARTITIONS] <子分区数> <STORAGE HASH 子句>

<STORAGE HASH 子句>请参考 3.5.1.4 定义水平分区表

参数

1. <模式名> 指明被操作的分区表属于哪个模式，缺省为当前模式；

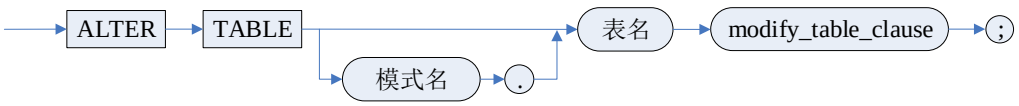
2. <表名> 指明被操作的分区表的名称；

3. <分区编号> 从 1 开始，2、3、4……以此类推，编号最大值为：实际分区数；

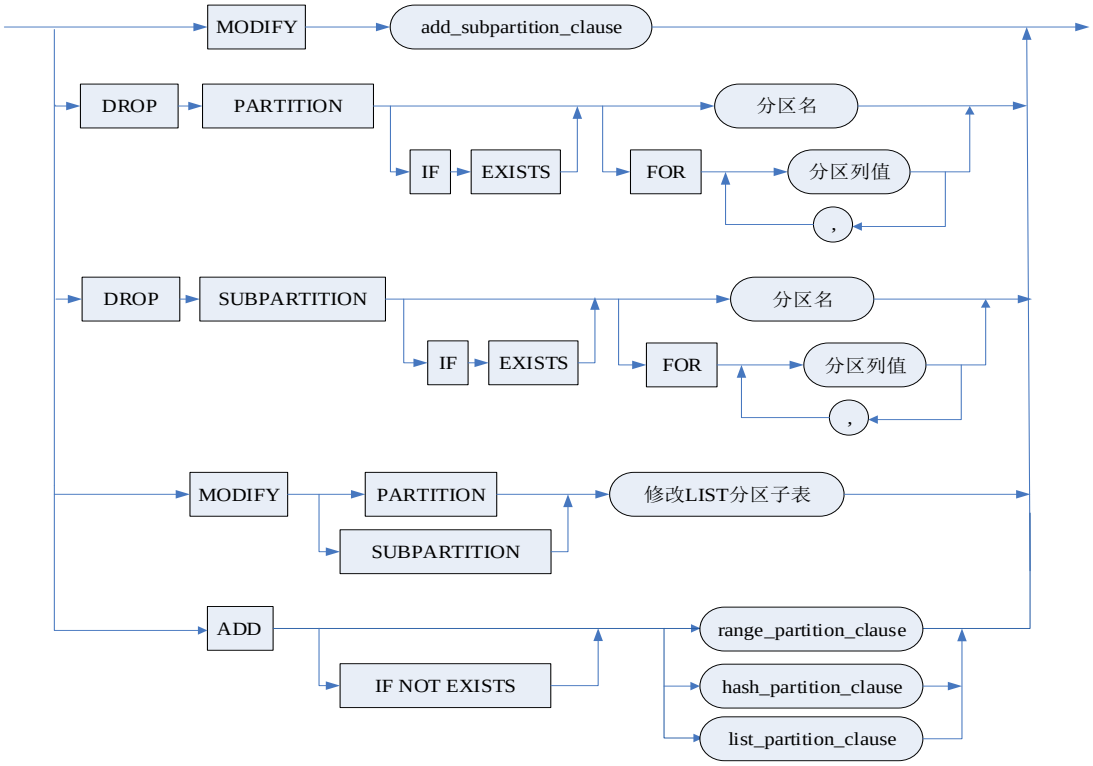
4. LOCK PARTITIONS 用于指定修改分区表的封锁模式为细粒度封锁模式，缺省为粗粒度封锁模式。不支持将间隔分区表设置为 LOCK PARTITIONS 模式。详细请参考 3.5.1.4 定义水平分区表。

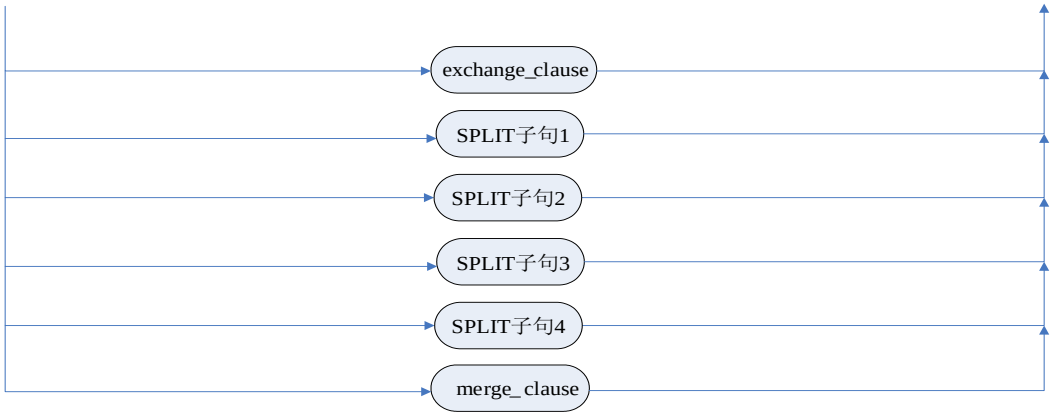
图例

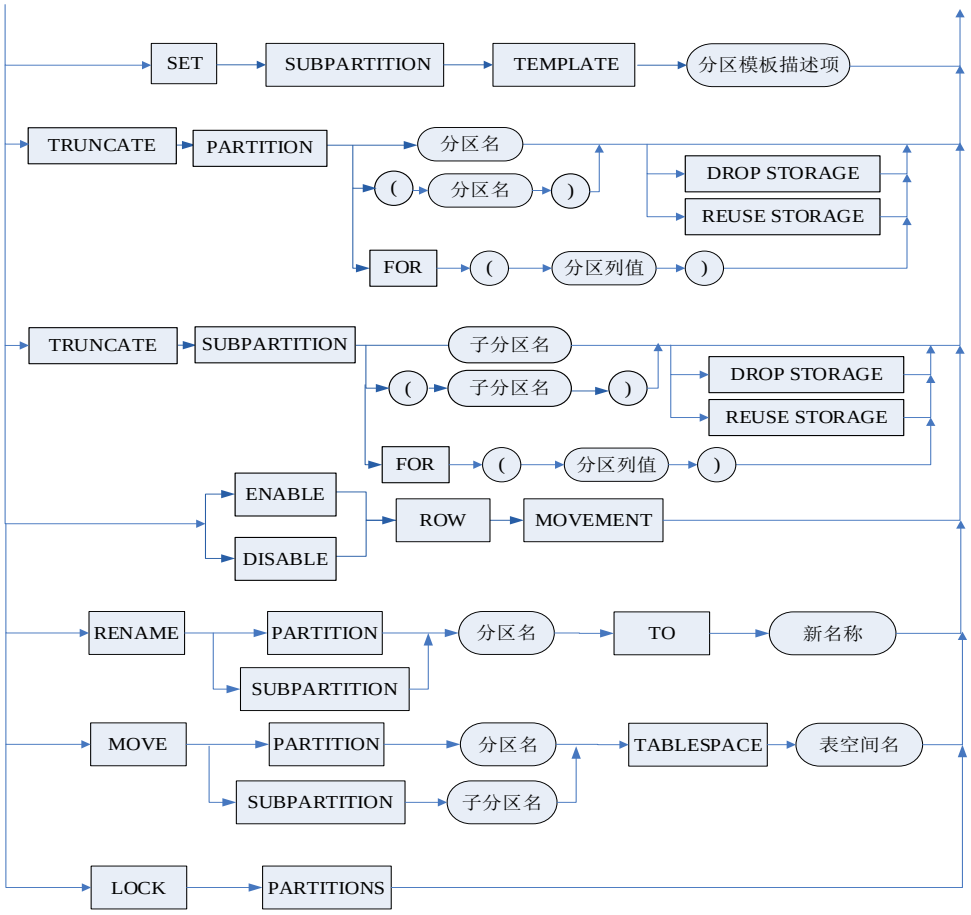
修改水平分区表



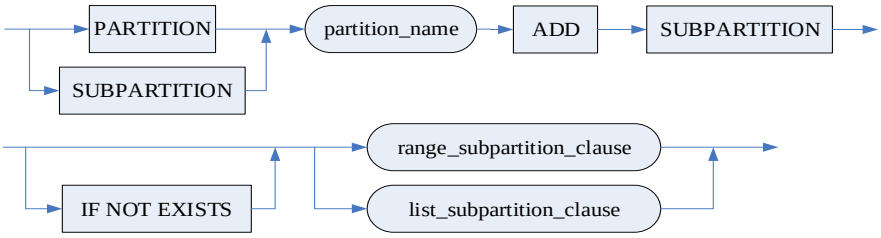
modify_table_clause



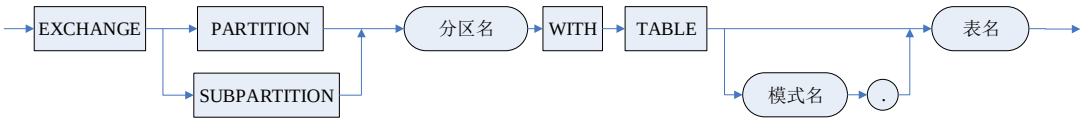




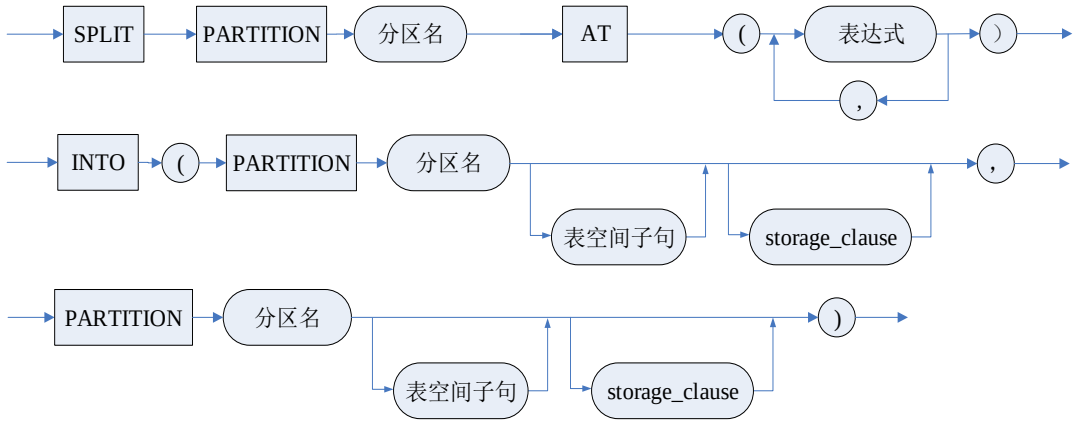
增加多级分区子句（add_subpartition_clause）



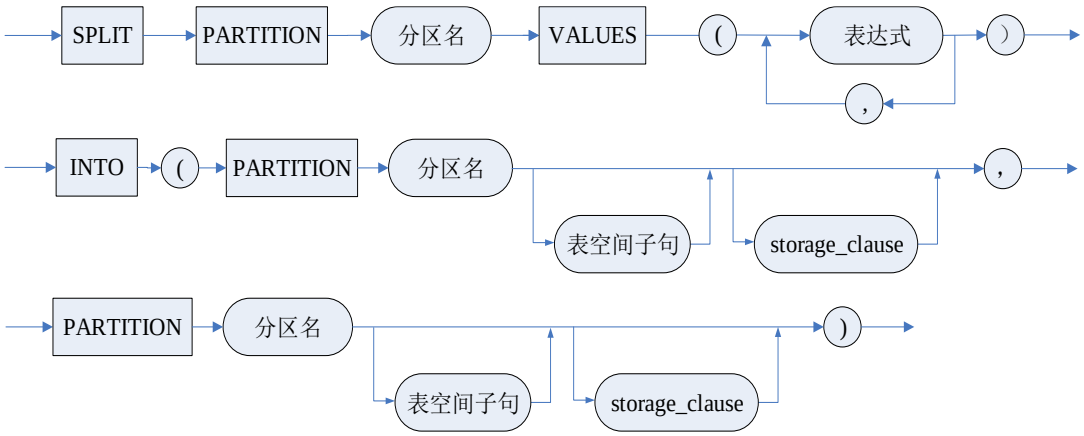
exchange_clause



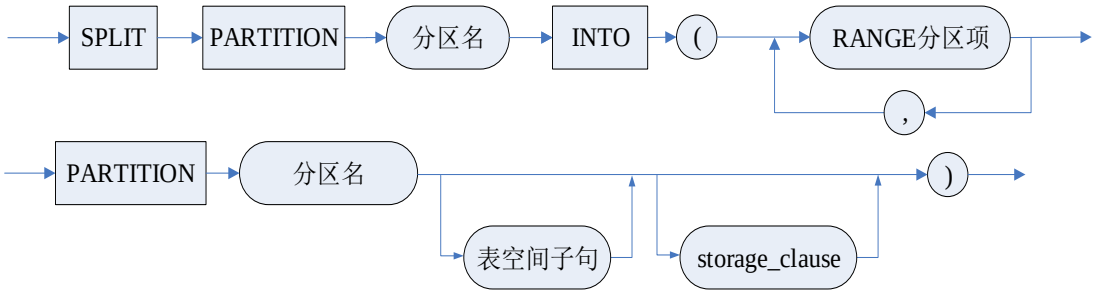
SPLIT 子句 1



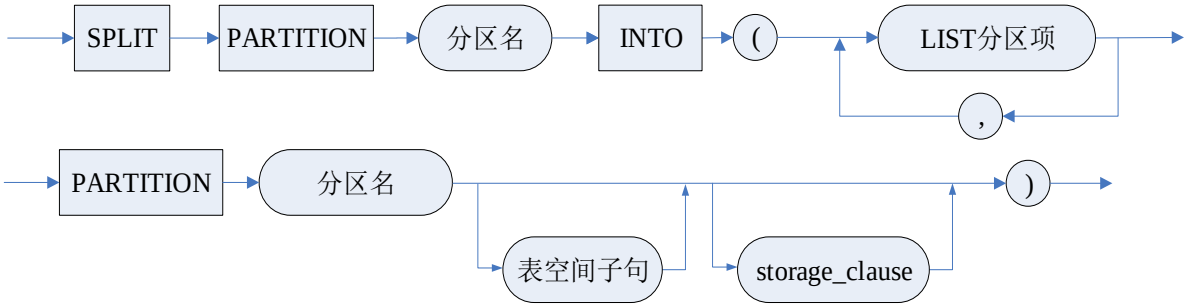
SPLIT 子句 2



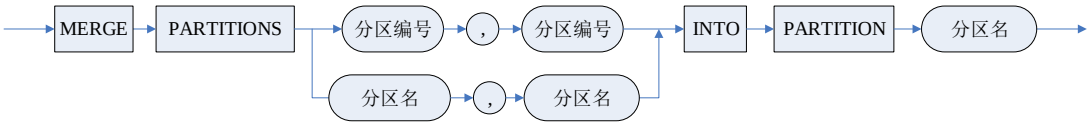
SPLIT 子句 3



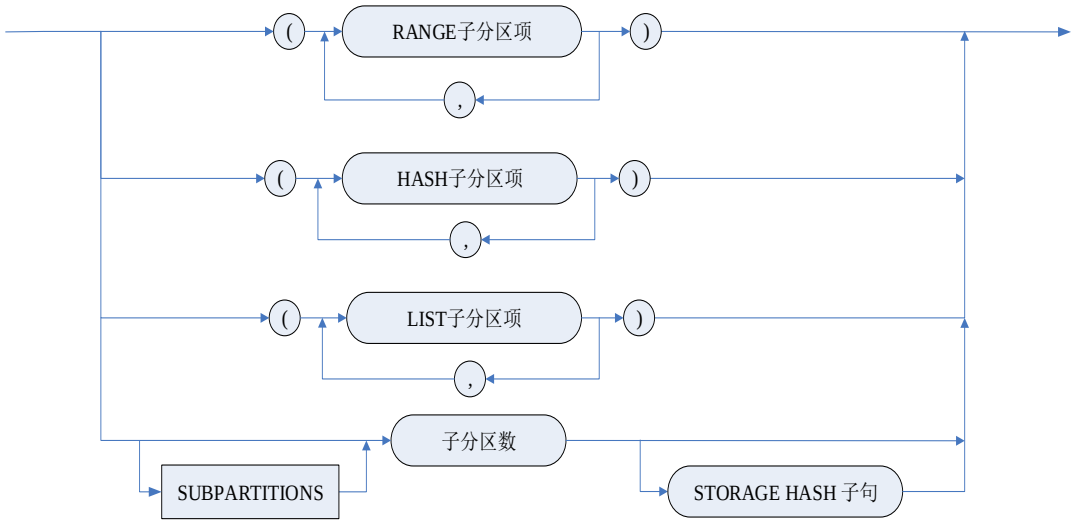
SPLIT 子句 4



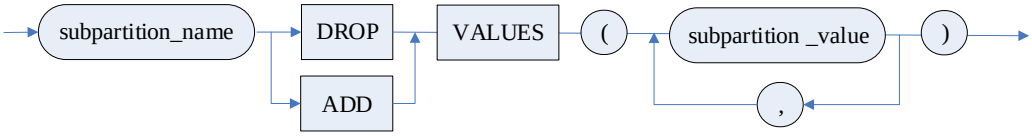
merge_clause



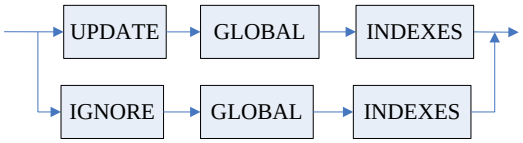
分区模板描述项



修改 LIST 分区子表



全局索引子句



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）的用户或该表的建表者或具有 ALTER ANY TABLE 权限的用户对水平分区表的定义进行修改。

修改包括:

1. 删除一级分区子表;
2. 增加、删除多级分区子表;

3. 修改一级 LIST 分区子表;
4. 拆分、合并、交换水平分区;
5. 修改水平分区子表名称;
6. 移动水平分区子表所在表空间。

使用说明

1. 具有 DBA 权限的用户或该表的建表者才能执行此操作;
2. <增加多级分区子表>说明:
   1) 对于 HASH 分区表，只有存储选项 HASHPARTMAP 为 1 或 2 的 HASH 分区表才能增加分区子表或多级分区子表;
   2) 新增的分区名不能与子表重名;
   3) 只能增加一个下层子表;
   4) 如果新增子表为中间层子表，可以为其自定义子表或根据模板进行创建;
   5) 禁止对间隔分区新增子表;
   6) 子表名称长度不能超过 128，否则报错。
3. <删除一级分区子表>和<删除多级分区子表>说明:

   <删除一级分区子表>说明:
   1) 提供两种方式删除一级子表：一是指定子表表名方式；二是指定分区列值的方式;
   2) 待删除子表为唯一分区时，禁止删除;
   3) 采用指定分区列值方式删除子表时, 分区列的数目为子表上所有分区列个数总和。例如：一级分区列(c1,c2)，那么在删除一级子表时候，分区列的数目为 2 个;
   4) 禁止删除 HASH 分区的子表。从父表到待删除子表之间也不允许存在 HASH 分区。

5） 当删除不存在的分区子表时会报错。若指定 IF EXISTS 关键字后，删除不存在的分区子表，则不会报错。

<删除多级分区子表>说明:

1） 提供两种方式删除二级或二级以上的子表：一是指定子表表名方式；二是指定分区列值的方式；

2） 待删除子表为唯一分区时，禁止删除；

3） 采用指定分区列值方式删除子表时，分区列的数目为子表上所有分区列个数总和。例如：一级分区列(c1,c2)；二级分区列(c3);三级分区列(c4)，那么在删除二级子表时候，分区列的数目为 3 个，三级分区列的数目为 4 个；

4） 禁止删除 HASH 分区的子表。从父表到待删除子表之间也不允许存在 HASH 分区。

5） 当删除不存在的分区子表时会报错。若指定 IF EXISTS 关键字后，删除不存在的分区子表，则不会报错。

4. <修改 LIST 分区子表>说明:

1） 支持修改一级、多级分区的 LIST 分区子表范围值。修改后的子分区，通过 SP_TABLEDEF()显示完整的分区定义；

2） 新增的分区值不能重复；

3） 不能删除分区所有范围值；

4） 修改的范围值必须为常量；

5） 如果表中存在某个范围值的记录，则不能删除此范围值；

6） 不能重复删除同一个范围值；

7） 不能增加、删除 DEFAULT 值；

8） 不能修改 DEFAULT 分区。

5. MERGE 合并分区说明:

将相邻的两个范围分区合并为一个分区。合并分区通过指定分区名或分区编号进行，相邻的两个分区的表空间没有特殊要求。其中，LIST 分区 MERGE 时只支持用分区名进行合并。

仅范围分区表和 LIST 分区表支持合并分区。多级分区表与一级分区表的 MERGE 语法一样。

其中，多级分区表进行 MERGE 合并的注意事项：

1） 仅支持一级子表类型为 RANGE、LIST；

2） 合并多级分区表中的一级子表时，该一级子表下的二级及以上层次子表按照级别分别判断：在创建分区表时当前层次是否指定了模板，若指定了模板，则按照模板创建当前层次子表；若未指定模板，则由系统自动合并为一个子表，子表名称为系统内部设置。RANGE 类型范围值为 MAXVALUE；LIST 类型范围值为 DEFAULT；

3） 不允许自定义二级及以上层次子表；

4） 不允许直接合并二级及以上层次子表。

6. SPLIT 拆分分区说明：

将某一个 RANGE 范围分区或 LIST 列表分区拆分为相邻的两个或多个分区。拆分范围分区时指定的常量表达式值不能是原有的分区表范围值。拆分列表分区时指定的常量表达式须包含在被拆分区中，新分区至少包含一个范围值且新分区之间范围值不能冲突。

拆分分区时，可以指定分区表空间，也可以不指定；指定的分区表空间不要求一定与原有分区相同；如果不指定表空间缺省使用拆分前分区的表空间；拆分分区会对分区中的数据从主表的角度进行重定位；拆分成多个分区的语法，一次拆分最多支持将一个分区拆分成 128 个新分区。

仅范围分区表和 LIST 分区表支持拆分分区。

多级分区表与一级分区表都只支持拆分一级子表的分区。

<SPLIT 子句 1>和<SPLIT 子句 3>用于 RANGE 分区表，<表达式>的个数和分区列的个数

一致。<SPLIT 子句 2>和<SPLIT 子句 4>用于 LIST 分区表。

<SPLIT 子句 1>和<SPLIT 子句 2>只支持 SPLIT 为 2 个分区。<SPLIT 子句 3>和<SPLIT 子句 4>支持拆分为多个分区。

SPLIT 产生的新分区二级及以上层次子表结构与被分隔子表保持一致，名称由系统内部定义。

7. 对于堆表分区，不论是拆分还是添加分区，都不能改变分区表空间，各子表必须和原表位于同一表空间；

8. 哈希分区支持重命名、增加/删除/修改约束、设置触发器是否启用的修改操作；

9. 范围分区支持分区合并、拆分、增加、删除、交换、重命名、增加/删除/修改约束、设置触发器是否生效操作；

10. LIST 分区支持分区合并、拆分、增加、删除、交换、重命名、增加/删除/修改约束、设置触发器是否生效操作；

11. 对范围分区增加分区值必须是递增的，即只能在最后一个分区后添加分区。LIST 分区增加分区值不能存在于其他已在分区；

12. 当水平分区数仅剩一个时，不允许进行删除分区；

13. 不允许删除被引用表的子分区；

14. EXCHANGE PARTITION/SUBPARTITION 用于交换一级分区/多级子分区的表。交换一级分区，仅范围分区和 LIST 分区支持交换分区，哈希分区表不支持。

1) 普通表（非 HUGE 表）交换分区要求分区表与交换表具有相同的结构（相同的表类型、相同的 BRANCH 选项、相同的列结构、相同的索引、相同的分布方式）。不支持包含全局索引的分区表与普通表进行交换分区操作。不支持分区表和含有快速加列的表进行交换分区。分区交换并不会校验数据，如交换表的数据是否符合分区范围等，即不能保证分区交换后的分区上的数据符合分区范围；

2） HUGE 表交换分区要求分区表与交换表具有相同的 STORAGE 参数（SECTION，FILESIZE，WITH/WITHOUT DELTA 和 STAT NONE）；HUGE 分区表交换表分区时，如果表空间包含多个路径，则源表与目标表必须存储在同一个表空间；

3） 若分区表或交换表上存在无效索引，则禁止交换分区。

15. TRUNCATE PARTITION/SUBPARTITION 用于清除指定一级分区子表/子分区子表的数据，对应一级分区子表/子分区子表结构仍然保留；MPP 环境下不支持对包含全局索引的分区子表进行 TRUNCATE 操作。DROP STORAGE / REUSE STORAGE 用于删除或重用分区，默认值为 DROP STORAGE。在 DPC 环境下仅做语法兼容。

16. <全局索引子句> 提供两种在对水平分区表执行 DDL 操作时维护全局索引的方式，指定为 UPDATE GLOBAL INDEXES 表示对全局索引进行重建；指定为 IGNORE GLOBAL INDEXES 表示仅将全局索引置为失效，不进行重建；缺省时指定为 UPDATE GLOBAL INDEXES。

17. 修改分区模板说明：

1） 支持模板信息从无到有、从有到无的修改；

2） 模板信息定义需要匹配子表类型，并符合对应分区类型的限制，例如范围分区值递增、列表分区不能存在重复值、范围值不能为 NULL 等；

3） 修改模板信息后，之前创建的子表不受影响，后续新增子表按照新的模板信息进行创建；

4） 仅支持修改第二层子表的模板信息。

18. 多级分区表支持下列修改表操作：新增分区、删除分区、新增列、删除列、新增自增列、删除自增列、删除表级约束、修改表名、设置与删除列的默认值、设置列 NULL 属性、设置列可见性、设置行迁移属性、启用超长记录存储功能、关闭超长记录存储功能、WITH DELTA、新增子分区、删除子分区、修改二级分区模板信息、SPLIT/MERGE 分区；

19. 水平分区表支持的列修改操作除了多级分区表支持的操作外，还支持：设置触发器

生效/失效、修改列名、修改列属性、增加表级主键约束；

20. <修改分区子表名>支持修改所有类型的分区子表名称。修改的子表名不能与现有子表重名。修改的名称需要符合对象命名规则，例如名称长度限制等；

21. MOVE PARTITION/SUBPARTITION 用于将分区移动到指定的表空间上。移动的分区若包含子表，则连同子表一起移动。不支持移动 HUGE 分区表和堆分区表。

举例说明

例 1 合并分区表，修改分区表。

```
ALTER TABLE PRODUCTION.PRODUCT_INVENTORY MERGE PARTITIONS P1,P2 INTO PARTITION P5;
```

执行后，分区结构如下：

表 3.5.5 执行合并修改后的分区结构

PARTITIONNO	P5	P3	P4
VALUES	QUANTITY<=100	100<QUANTITY<=10000	QUANTITY<99999

例 2 使用<SPLIT 子句>拆分水平分区表。

```
//使用<SPLIT 子句 1>和<SPLIT 子句 3>拆分范围分区表
DROP TABLE T1 CASCADE;
CREATE  TABLE T1(
caller  CHAR(15),
callee  CHAR(15),
time    int,
duration  INT
)
PARTITION BY RANGE(time,duration)(
PARTITION p1 VALUES LESS THAN (10,10),
PARTITION p2 VALUES LESS THAN (20,20),
```

```
PARTITION p3 VALUES LESS THAN (30,30),
PARTITION p4 VALUES EQU OR LESS THAN (100,100));

ALTER TABLE T1 SPLIT PARTITION P1 AT(5,5) INTO(PARTITION P5,PARTITION P6);
ALTER TABLE T1 SPLIT PARTITION P3 INTO(PARTITION p7 VALUES LESS THAN(25,25),PARTITION p8
VALUES LESS THAN (28,28),PARTITION p9);

//使用<SPLIT 子句 2>和<SPLIT 子句 4>拆分 LIST 分区表
DROP TABLE T1 CASCADE;
CREATE  TABLE T1(
caller   CHAR(15),
callee   CHAR(15),
time   int,
duration   INT
)
PARTITION BY list(time)(
PARTITION p1 VALUES(1,2,3),
PARTITION p2 VALUES(11,12,13),
PARTITION p3 VALUES(21,22,23,24,25,26,27,28),
PARTITION p4 VALUES(31,32,33));

ALTER TABLE T1 SPLIT PARTITION P2 values(11,12) INTO(PARTITION P6,PARTITION P7);
ALTER TABLE t1 SPLIT PARTITION P3 INTO(PARTITION p5 VALUES(21,22),PARTITION p8
VALUES(24,25),PARTITION p9);
```

例 3 增加，删除分区子表。

```
DROP TABLE T1 CASCADE;
CREATE TABLE T1 (C1 INT, C2 INT)
    PARTITION BY LIST (C1)
    SUBPARTITION BY RANGE (C2)    SUBPARTITION TEMPLATE (
        SUBPARTITION SP1 VALUES LESS THAN(10),
        SUBPARTITION SP2 VALUES LESS THAN(20))
    (
        PARTITION FP1 VALUES(1,2,3),
        PARTITION FP2 VALUES(4,5,6)
    );
// 增加一个二级分区 SP3。增加的二级分区名称为 T1_FP1_SP3。
ALTER TABLE T1 MODIFY PARTITION FP1 ADD SUBPARTITION SP3 VALUES LESS THAN(300) ;
// 为一级 LIST 分区 FP2 增加一个范围值 7。
ALTER TABLE T1 MODIFY PARTITION FP2 ADD VALUES(7);
// 删除分区子表。采用指定分区列值的方法和指定子表名的方法定位分区子表。指定第二列值为 2，第三
列值为 60，可以定位到 FP1_SP3 子表，等同于直接指定子表名 FP1_SP3。
ALTER TABLE T1 DROP SUBPARTITION FOR(2,60);
等同于
ALTER TABLE T1 DROP SUBPARTITION FP1_SP3;
```

例 4 交换分区子表。

```
CREATE HUGE TABLE PARTITION_T1(C1 INT, C2 INT)PARTITION BY RANGE(C1)(
            PARTITION PAR1 VALUES LESS THAN (2),
            PARTITION PAR2 VALUES LESS THAN (10),
```

```
        PARTITION PAR3 VALUES LESS THAN (30)
    );


    INSERT INTO PARTITION_T1 VALUES (1,1);
    INSERT INTO PARTITION_T1 VALUES (9,19);
    INSERT INTO PARTITION_T1 VALUES (21,22);
    COMMIT;
    CREATE HUGE TABLE PARTITION_T2(C1 INT, C2 INT);
INSERT INTO PARTITION_T2 VALUES (100,100);
COMMIT;
```

将分区表 PAR1 分区和表 partition_t2 进行交换。

```
ALTER TABLE PARTITION_T1 EXCHANGE PARTITION PAR1 WITH TABLE PARTITION_T2;
```

交换之后，数据发生了互换，分别对 PAR1 和 PARTITION_T2 进行查询。先对 PAR1 表进行查询：

```
SELECT * FROM PARTITION_T1_PAR1;
```

查询结果如下：

```
行号         C1             C2
------- --------- ---------
1            100            100
```

再对 PARTITION_T2 表进行查询：

```
SELECT * FROM PARTITION_T2;
```

查询结果如下：

```
行号         C1             C2
------- -------- --------
```

```
|            |            |
```

3.5.2.3 修改 HUGE 表

本节专门列出 HUGE 表的修改操作。

语法格式

```
ALTER TABLE [<模式名>.]<表名><修改表定义子句>

<修改表定义子句> ::=

……

ADD [COLUMN] <列名>[<列压缩子句>]|

ALTER [COLUMN] <列名> SET STAT NONE|

ALTER [COLUMN] (<列名>{,<列名>})SET STAT [NONE]|

WITH DELTA|

SET STAT [NONE | SYNCHRONOUS | ASYNCHRONOUS] [<ON | EXCEPT> (<列名>{,<列名>})]|

REFRESH STAT|

FORCE COLUMN STORAGE

<列压缩子句> ::= COMPRESS [LEVEL <压缩级别>] [<压缩类型>]

<压缩级别>、<压缩类型>::=请参考 3.5.1.3 定义 HUGE 表
```

省略号“……”部分包含两部分。一部分来自 3.5.2.1 修改数据库表，一部分来自 3.5.2.2 修改水平分区表。

来自 3.5.2.1 修改数据库表的部分如下：

```
MODIFY <列定义>|

ADD [CONSTRAINT [<约束名>] ] <表级约束子句> [<失效生效选项>] [<VALIDATE 选项>]|

RENAME CONSTRAINT <约束名 1> TO <约束名 2> |

RENAME COLUMN <列名 1> TO <列名 2> |

DROP CONSTRAINT <约束名> [RESTRICT | CASCADE] |
```

```
ALTER [COLUMN] <列名> SET DEFAULT <列缺省值表达式>|

ALTER [COLUMN] <列名> DROP DEFAULT |

ALTER [COLUMN] <列名> RENAME TO <列名> |

RENAME TO <表名> |

ENABLE CONSTRAINT <约束名> [<VALIDATE 选项>]|

DISABLE CONSTRAINT <约束名> [<VALIDATE 选项>] [RESTRICT | CASCADE]

<表级约束子句>::=<表级完整性约束>

<表级完整性约束> ::=<唯一性约束选项> (<列名> {,<列名>}) [USING INDEX TABLESPACE{ <表空间名> |
  DEFAULT}]

<唯一性约束选项> ::=

PRIMARY KEY |

UNIQUE
```

来自 3.5.2.2 修改水平分区表的部分如下:

```
MODIFY <增加多级分区子表>|

ADD <水平分区项>|

DROP PARTITION <分区名>|

EXCHANGE <PARTITION| SUBPARTITION > <分区名> WITH TABLE [<模式名.>]<表名>|

<SPLIT 子句>|

MERGE PARTITIONS <分区编号>,<分区编号>  INTO PARTITION <分区名>|

MERGE PARTITIONS <分区名>,<分区名>  INTO PARTITION <分区名>|

SET SUBPARTITION TEMPLATE <分区模板描述项>



<增加多级分区子表>: :  =<PARTITION> <分区名> ADD <<RANGE 子分区项>|<LIST 子分区项>>

<水平分区项>、<SPLIT 子句>请参考 3.5.2.2 修改水平分区表
```

<分区模板描述项> ::= ([<分区模板描述项 1>])| <分区模板描述项 2>

<分区模板描述项 1> ::=

<RANGE 子分区项> {,<RANGE 子分区项>} |

<HASH 子分区项> {,<HASH 子分区项>} |

<LIST 子分区项> {,<LIST 子分区项>}

<分区模板描述项 2> ::= [SUBPARTITIONS] <子分区数> <STORAGE HASH 子句>

<STORAGE HASH 子句>请参考 3.5.1.4 定义水平分区表

参数

1. <模式名> 指明被操作的基表属于哪个模式，缺省为当前模式；

2. <表名> 指明被操作的基表的名称；

3. <列名> 指明修改、增加或被删除列的名称；

4. ADD COLUMN 指明增加列。一次只能增加一个列，且仅支持事务型 HUGE 表。不支持增加虚拟列、包含约束子句的列。当 ALTER_TABLE_OPT 为 0 和 1 效果一样，均为采用查询插入实现；当 ALTER_TABLE_OPT 为 2 时，支持对于没有默认值或者默认值为 NULL 的新列使用快速加列。当 ALTER_TABLE_OPT 为 3 时，允许对指定了默认值的新增列进行快速加列；

5. <列压缩子句> 指明列的压缩级别和压缩类型；

6. ALTER [COLUMN] <列名> SET STAT NONE 修改（关闭）某一个列的统计信息属性；

7. ALTER [COLUMN] (<列名>{,<列名>})SET STAT [NONE] 修改表的某一列或多列的统计开关，STAT 为打开，STAT NONE 为关闭。只有表的统计状态是实时或异步时，支持打开或关闭列的统计开关，打开之后列的统计状态和表的统计状态一致；如果表的统计开关是关闭的，不支持打开或关闭列的统计开关。如下表所示：

表 3.5.6 打开或关闭指定列的统计开关

表统计状态	列状态	操作
实时或异步	STAT	当表的状态是实时或异步时，支持打开列统计，统计状态与表统计状态相同
	STAT NONE	当表的状态是实时或异步时，支持关闭列统计
NONE	STAT	当表的状态是 NONE 时，不支持打开或关闭列统计
	STAT NONE	

8. WITH DELTA 表示将一个非事务型 HUGE 表修改为事务型。只支持将非事务型（WITHOUT DELTA）HUGE 表修改为事务型（WITH DELTA）的。不支持将一个事务型的 HUGE 表修改为非事务型的；

9. SET STAT [NONE | SYNCHRONOUS | ASYNCHRONOUS] [<ON | EXCEPT> (col_lst)]修改表的统计状态。转换规则见下表：

表 3.5.7 设置表的统计信息

原 STAT	新 STAT	列设置	说明
实时或异步	NONE	不设置	关闭表的统计开关（即所有列都不做统计）
实时	异步	不设置	切换表统计状态
异步	实时		
NONE	实时或异步	必须设置	设置表的统计状态，并打开指定列的统计
前后状态一致		设置或不设置	报错

转换规则注意事项如下：

1) [NONE | SYNCHRONOUS | ASYNCHRONOUS] [ON | EXCEPT (col_lst)] 的详细意义请参考 3.5.1.3 定义 HUGE 表<storage 子句 1>；

2) 如果是从 NONE 到打开统计，必须要带上列设置<ON | EXCEPT> (col_lst)，否则报错；

3) 如果是从打开到 NONE，影响所有列；

4） 如果是实时和异步之间的切换，不支持列设置，不影响各列上的统计状态，只是切换计算统计信息的方式；

5） 如果修改后是实时计算统计信息，对于表中需要计算统计信息的列，要将统计信息的值设置为准确的值。

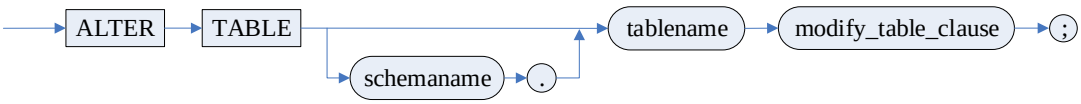
10. REFRESH STAT 刷新辅助表$AUX 的统计信息，为了更新不准确的数据区的信息；

11. FORCE COLUMN STORAGE 强制把 R 表数据写入数据文件，仅 WITH DELTA 的 HUGE 表支持；

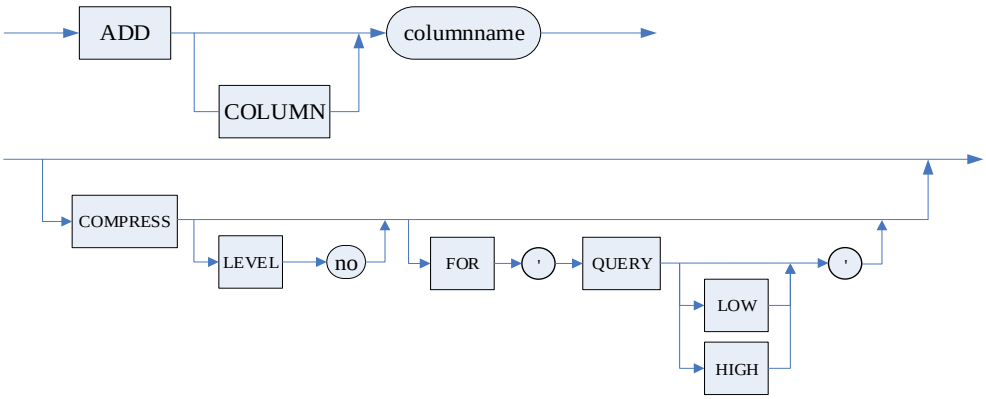
12. MODIFY <列定义> 已经插入数据的 HUGE 表仅对修改列定义进行有限度的支持：对于不需要重建数据的情况，比如变长类型扩展长度、在数据满足条件的情况下缩小数据长度或增减约束等，可以成功；而对于需要重建数据的情况，比如修改数据类型、修改加密压缩类型等，则会报错不支持。

图例

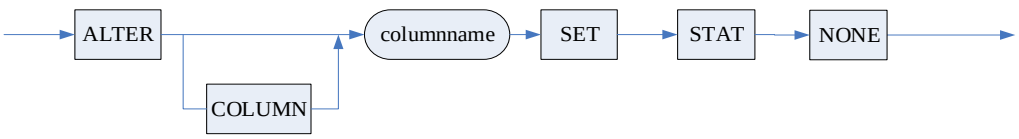
表修改语句



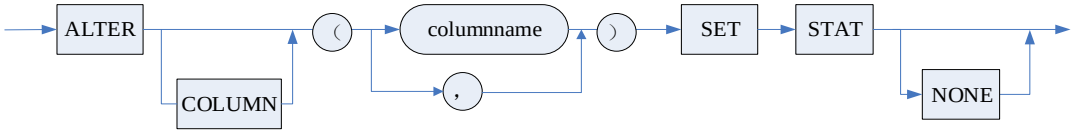
modify_table_clause



或



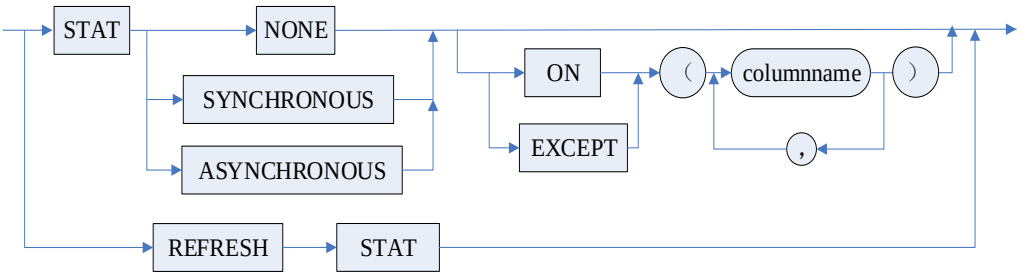
或



或



或



或



“……”部分的语法图请参考 3.5.2.1 修改数据库表和 3.5.2.2 修改水平分区表。

语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）的用户或该表的建表者或具有 ALTER ANY TABLE 权限的用户对 HUGE 表的定义进行修改。

举例说明

创建一个 HUGE 表 STUDENT，表的区大小为 65536 行，文件大小为 64M，S_COMMENT 列指定的区大小为不做统计信息，其它列（默认）都做统计信息。

```
CREATE HUGE TABLE STUDENT
    (
        S_NO                INT,
        S_CLASS             VARCHAR,
```

```
        S_COMMENT       VARCHAR(79) STORAGE(STAT NONE)
)STORAGE(SECTION(65536), WITH DELTA, FILESIZE(64));
```

重新设置列 S_COMMENT，打开 S_COMMENT 的统计开关，对该列做统计信息。

```
ALTER TABLE STUDENT ALTER COLUMN (S_COMMENT)  SET STAT;
```

修改表 STUDENT 的统计状态为关闭，即所有列都不做统计信息。

```
ALTER TABLE STUDENT SET STAT NONE;
```

修改表 STUDENT 的统计状态为 SYNCHRONOUS，并打开 S_NO、 S_CLASS、S_COMMENT 三列的统计开关，打开之后，列的统计状态和表的统计状态一致。

```
ALTER TABLE STUDENT SET STAT SYNCHRONOUS ON (S_NO, S_CLASS, S_COMMENT);
```

3.5.3 基表删除语句

DM 系统允许用户随时从数据库中删除基表。

语法格式

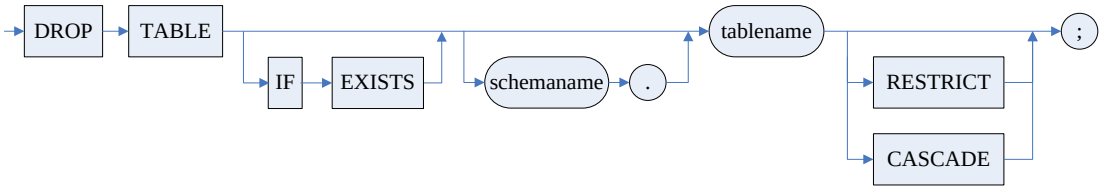
```
DROP TABLE [IF EXISTS] [<模式名>.]<表名> [RESTRICT|CASCADE];
```

参数

1. <模式名> 指明被删除基表所属的模式，缺省为当前模式；
2. <表名> 指明被删除基表的名称。

图例

表删除语句



语句功能

供具有 DBA 角色（三权分立）或 DROP ANY TABLE 权限的用户或该表的拥有者删除基

表。

使用说明

1. 删除不存在的基表会报错。若指定IF EXISTS关键字后，删除不存在的表，不会报错；

2. 表删除有两种方式：RESTRICT/CASCADE方式（外部基表除外）。其中RESTRICT为缺省值。如果以CASCADE方式删除该表，将删除表中唯一列上和主关键字上的引用完整性约束，当设置INI参数DROP_CASCADE_VIEW值为1时，还可以删除所有建立在该基表上的视图。如果以RESTRICT方式删除该表，要求该表上已不存在任何视图以及引用完整性约束，否则DM返回错误信息，而不删除该表。当INI参数DROP_CASCADE_VIEW值为0时，表与其上建立的视图的依赖关系被剥离，此时使用CASCADE和RESTRICT方式删除该表都可以成功，且只会删除该表，不会删除其上的视图；

3. 该表删除后，在该表上所建索引也同时被删除；

4. 该表删除后，所有用户在该表上的权限也自动取消，以后系统中再建同名基表是与该表毫无关系的表；

5. 删除外部基表无需指定RESTRICT和CASCADE关键字。

举例说明

例 1 用户 SYSDBA 删除 PERSON 表。

```
DROP TABLE PERSON.PERSON CASCADE;
```

例 2 假设当前用户为用户 SYSDBA。现要删除 PERSON_TYPE 表。为此，必须先删除 VENDOR_PERSON 表，因为它们之间存在着引用关系，VENDOR_PERSON 表为引用表，PERSON_TYPE 表为被引用表。

```
DROP TABLE PURCHASING.VENDOR_PERSON;

DROP TABLE PERSON.PERSON_TYPE;
```

也可以使用 CASCADE 强制删除 PERSON_TYPE 表，但是 VENDOR_PERSON 表仍然存在，只是删除了 PERSON_TYPE 表的引用约束。

```
DROP TABLE PERSON.PERSON_TYPE CASCADE;
```

3.5.4 基表数据删除语句

DM 可以从表中删除所有记录。

语法格式

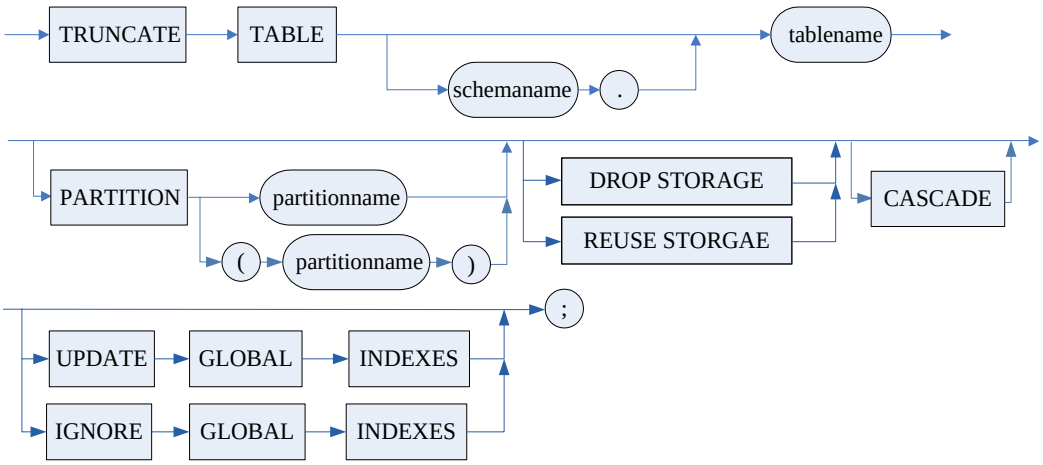
```
TRUNCATE TABLE [<模式名>.]<表名>[PARTITION <分区名>|(<分区名>)] [DROP STORAGE | REUSE
STORAGE] [CASCADE] [<全局索引子句>];
<全局索引子句>::=
    UPDATE GLOBAL INDEXES |
    IGNORE GLOBAL INDEXES
```

参数

1. <模式名> 指明表所属的模式，缺省为当前模式；
2. <表名> 指明被删除记录的表的名称。
3. <分区名> 指明被删除分区表的名称。

图例

基表数据删除语句



语句功能

供具有 DBA 角色的用户或该表的拥有者从表中删除所有记录。

使用说明

1. TRUNCATE命令一次性删除指定表的所有记录，比用DELETE命令删除记录快。但TRUNCATE不会触发表上的DELETE触发器；

2. 如果TRUNCATE删除的表上有被引用关系，且未指定CASCADE，则此语句失败；

3. 如果TRUNCATE删除的表上有被引用关系，且指定CASCADE，若所有引用表的引用约束选项指定为“ON DELETE CASCADE”，则系统将级联删除所有引用表中的数据，否则将报错；

4. TRUNCATE不同于DROP命令，因为它保留了表结构及其上的约束和索引信息；

5. TRUNCATE 命令只能用来删除表的所有的记录，而DELETE命令可以只删除表的部分记录。

举例说明

例 假定删除模式 PRODUCTION 中的 PRODUCT_REVIEW 表中所有的记录。

```
TRUNCATE TABLE PRODUCTION.PRODUCT_REVIEW;
```

3.5.5 事务型 HUGE 表数据重整

事务型 HUGE 表删除和更新的数据都存储在对应行辅助表中，查询时还需要扫描行辅助表。当随着时间推移，行辅助表中数据量较大时，需要对数据进行重整，以免影响事务型 HUGE 表的查询效率。数据重整将事务型 HUGE 表中涉及更新和删除操作的数据区进行区内数据重整，重整后 HUGE 表中每个数据区内的数据紧密排列，但数据不会跨区移动。需要注意的是，数据重整会导致 ROWID 发生变化。

仅 WITH DELTA 的 HUGE 表支持数据重整。

语法格式

```
ALTER TABLE [<模式名>.]<表名> REBUILD SECTION;
```

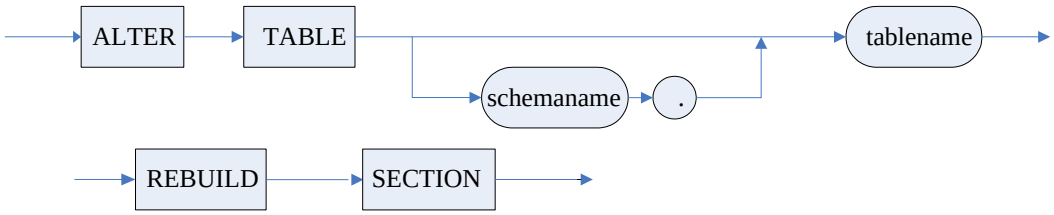
参数

1. <模式名> 指明表所属的模式，缺省为当前模式；

2. <表名> 指明被删除记录的表的名称；

图例

事务型 HUGE 表数据重整



语句功能

具有 DBA 角色的用户或该表的建表者才能执行此操作。

举例说明

例 对事务型HUGE表ORDERS进行重整。

```
ALTER TABLE ORDERS REBUILD SECTION;
```

3.6 管理索引

3.6.1 索引定义语句

为了提高系统的查询效率，DM 系统提供了索引。但也需要注意，索引会降低那些影响索引列值的命令的执行效率，如 INSERT、UPDATE、DELETE 的性能，因为 DM 不但要维护基表数据还要维护索引数据。

语法格式

```
CREATE [OR REPLACE] [CLUSTER|NOT PARTIAL][UNIQUE | BITMAP] INDEX [IF NOT EXISTS]
<索引名> ON [<模式名>.]<表名>(<索引列定义>{,<索引列定义>}) [GLOBAL] [<PARTITION 子句>]
[<表空间子句>] [<STORAGE 子句>] [NOSORT] [ONLINE] [REVERSE] [UNUSABLE] [<PARALLEL 项>];
```

<索引列定义>::= <索引列表达式>[ASC|DESC]

<表空间子句>::= TABLESPACE <表空间名>

<STORAGE 子句>::=<STORAGE 子句 1>|<STORAGE 子句 2>

<STORAGE 子句 1>::= STORAGE(<STORAGE1 项> {,<STORAGE1 项>})

<STORAGE1 项> ::=

[INITIAL <初始簇数目>] |

[NEXT <下次分配簇数目>] |

[MINEXTENTS <最小保留簇数目>] |

[ON <表空间名>] |

[FILLFACTOR <填充比例>]|

[BRANCH <BRANCH 数>]|

[BRANCH (<BRANCH 数>, <NOBRANCH 数>)]|

[NOBRANCH]|

[CLUSTERBTR]|

[SECTION (<区数>)]|

[STAT NONE]

<STORAGE 子句 2>::= STORAGE(<STORAGE2 项> {,<STORAGE2 项>})

<STORAGE2 项> ::= [ON <表空间名>]|[STAT NONE]

<PARALLEL 项> ::=

NOPARALLEL |

PARALLEL [<并行数>]

<PARTITION 子句> ::=请参考 3.5.1.4 定义水平分区表

参数

1. UNIQUE 指明该索引为唯一索引;

2. BITMAP 指明该索引为位图索引；

3. CLUSTER 指明该索引为聚集索引，不能应用到函数索引中；

4. NOT PARTIAL 指明该索引为非聚集索引，缺省即为非聚集索引；

5. <索引名> 指明被创建索引的名称，索引名称最大长度128字节；不能指定为系统格式的索引名（格式为前缀关键字INDEX与数值组合：'INDEX<系统索引数值>'，系统索引数值范围为33554432~50331647）或系统格式的约束名（格式为前缀关键字CONS与数值组合：'CONS<系统约束数值>'，系统约束数值范围为134217728~150994943）；

6. <模式名> 指明被创建索引的基表属于哪个模式，缺省为当前模式；

7. <表名> 指明被创建索引的基表的名称；

8. <索引列定义> 指明创建索引的列定义；

9. <索引列表达式> 指明被创建的索引列可以为表中列、以表中列为变量的函数或表达式；

10. GLOBAL 指明该索引为全局索引，缺省为局部索引。专门用于水平分区表，非水平分区表忽略该选项；

11. <PARTITION子句> 创建分区索引。专门用于DMDPC架构下的分区表。必须和GLOBAL关键字一起使用才有效，不支持和ONLINE关键字一起使用。<PARTITION子句>仅支持一级分区，且分区列必须是索引列的子集和前导列。<PARTITION子句>中的RANGE分区必须包含MAXVALUE分区，LIST分区必须包含DEFAULT分区。<PARTITION子句>中指定的子索引表空间、<STORAGE子句>中主索引的表空间，均需和主表的表空间保持一致；

12. ASC 递增顺序；

13. DESC 递减顺序；

14. <表空间子句> 不能和<STORAGE子句>中的ON <表空间名>同时使用；

15. <STORAGE子句> 普通表的索引参考<STORAGE子句1>，HUGE表的索引参考<STORAGE子句2>；

16. <STORAGE子句1>中，BRANCH和NOBRANCH只能用以指定聚集索引；

17. NOSORT 指明该索引相关的列已按照索引中指定的顺序有序，不需要在建索引时排序，提高建索引的效率。若数据非有序却指定了NOSORT，则在建索引时会报错；

18. ONLINE 表示支持异步索引，即创建索引过程中可以对索引依赖的表做增、删、改操作；

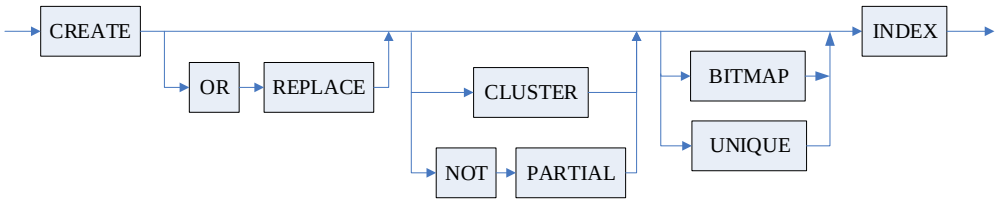
19. REVERSE 表示将当前索引创建为反向索引，即按索引数据的原始数据的反向排列顺序创建索引。(缺省为按原始数据的正向排列顺序创建索引)；

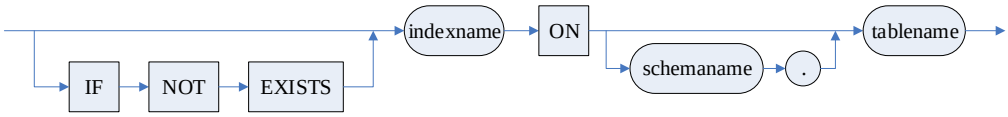
20. UNUSABLE 表示将当前索引创建为无效索引，系统不会维护无效索引，可以在SYSOBJECTS的VALID字段查看该值。对于无效索引，可以利用REBUILD来重建；

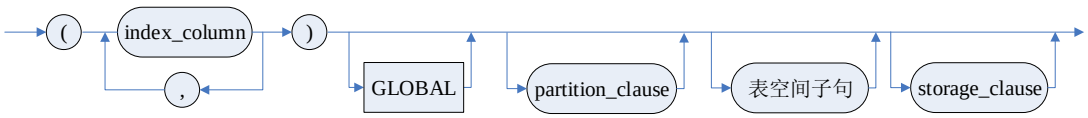
21. <PARALLEL项> 指明是否并行创建索引，NOPARALLEL表示不并行，PARALLEL表示并行，缺省为NOPARALLEL。指定PARALLEL时，需要指定并行数，并行数的合法值范围为0~128，且不能超过当前MAX_PARALLEL_DEGREE值。若不指定并行数，则默认按照INI参数MAX_PARALLEL_DEGREE的值并行创建索引；若指定的并行数为0，则按照MAX_PARALLEL_DEGREE的值并行创建索引。指定并行创建索引时，需要设置INI参数PARALLEL_POLICY开启并行模式，若PARALLEL_POLICY=0则将始终按照非并行模式创建索引。

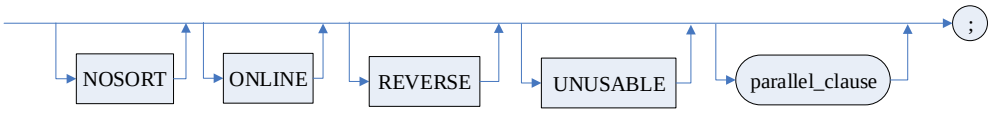
图例

索引定义语句

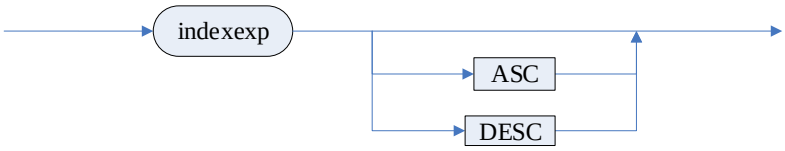




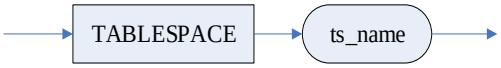




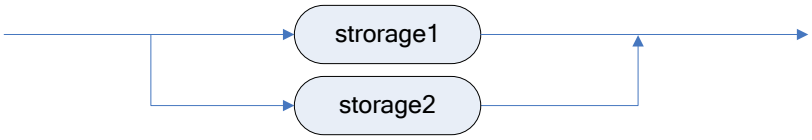
index_column



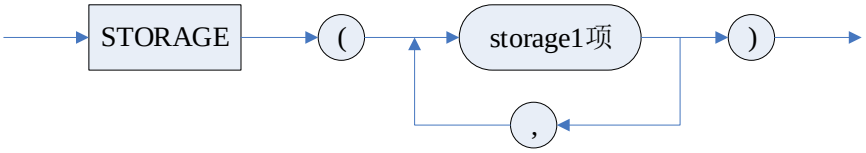
表空间子句



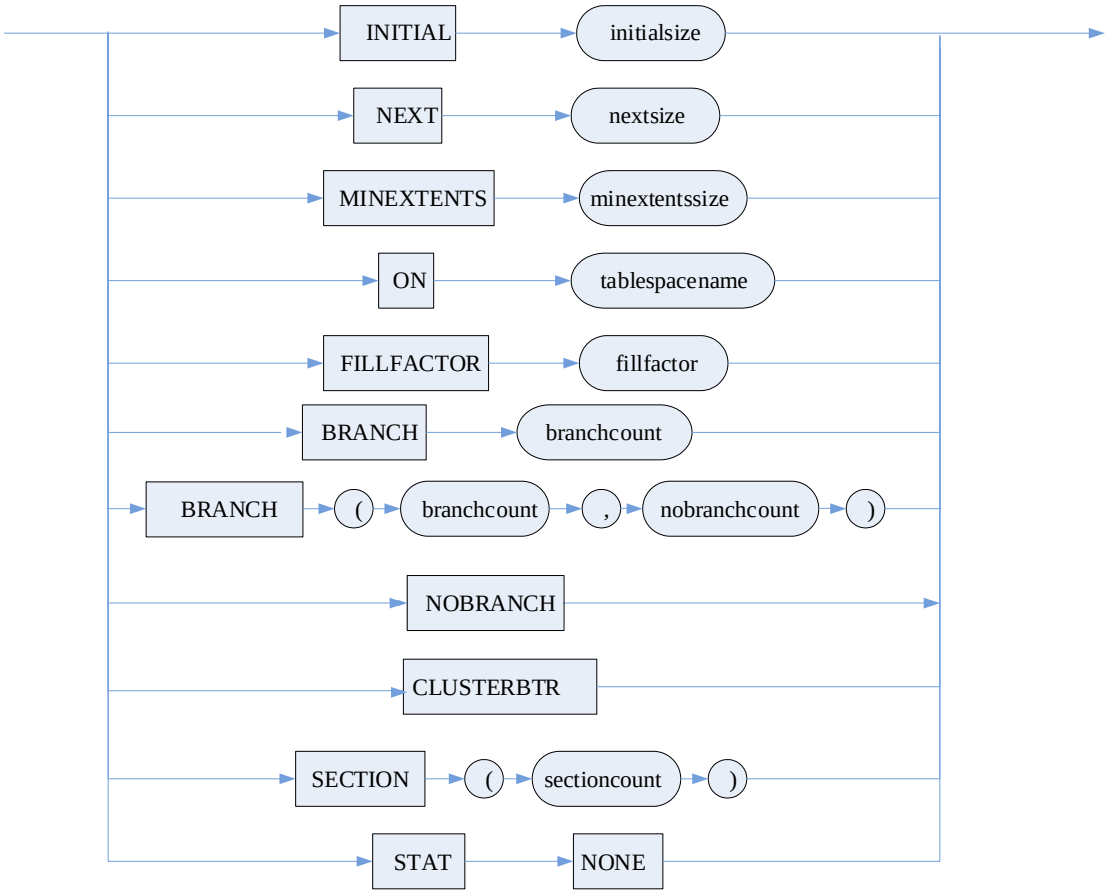
storage_clause



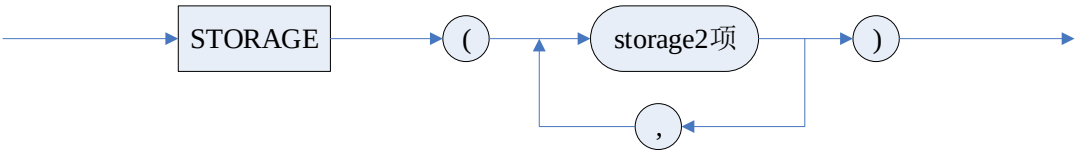
storage1



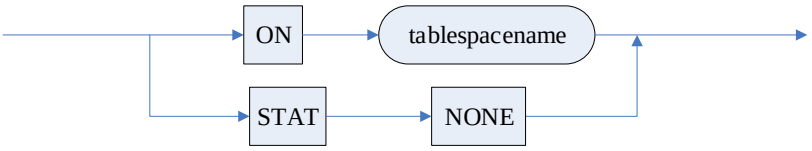
storage1 项



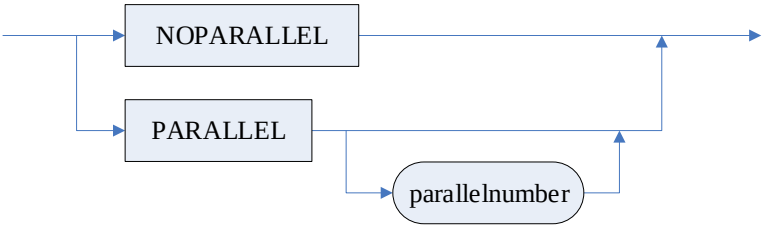
storage2



storage2 项



parallel_clause



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或该索引所属基表的拥有者且具有 CREATE INDEX 或 CREATE ANY INDEX 权限的用户定义索引。

使用说明

1. 当索引名已存在时，若指定OR REPLACE，则使用新索引替换原同名索引；若指定IF NOT EXISTS，则忽略本次索引创建操作；若同时指定OR REPLACE和IF NOT EXISTS，则按照OR REPLACE的策略执行；若均不指定，则报错；

2. 索引列不得重复出现且数据类型不得为多媒体数据类型、类类型和自定义类型，二级索引列不得为本地时区类型；

3. 索引列最多不能超过63列；

4. 可以使用STORAGE子句指定索引的存储信息，它的参数说明参见CREATE TABLE语句；

5. 索引的默认表空间与其基表的表空间一致；

6. 索引的模式名与其基表的模式名一致；

7. 索引各字段值相加得到的记录内总数据值长度不得超过页大小的1/5，二级索引各字段值相加得到的记录内总数据值长度则不能超过min(页大小1/5 , 3000)；

8. 在下列情况下，DM利用索引可以提高性能：

   1） 用指定的索引列值来搜索记录；

   2） 用索引列的顺序来存取基表。

9. 每张表中只允许有一个聚集索引，如果之前已经指定过CLUSTER INDEX或者指定了CLUSTER PK，则用户新建立CLUSTER INDEX时系统会自动删除原先的聚集索引。但如果新建聚集索引时指定的创建方式（列，顺序）和之前的聚集索引一样，则会报错；

10. 列存储表（HUGE表）和堆表不允许建立聚集索引；

11. 指定CLUSTER INDEX操作需要重建表上的所有索引，包括PK索引；若表上有唯一聚集索引且被引用，则不能创建新的聚集索引，如要创建新的聚集索引，则需先删除引用约

束;

12. 删除聚集索引时，缺省以ROWID排序，自动重建所有索引;

13. 函数索引：基于函数或表达式建立的索引称为函数索引。即<索引列表达式>是以表中列为变量的函数或表达式。函数索引创建方式与普通索引一样，并且支持UNIQUE和STORAGE设置项，对于以函数或表达式为过滤的查询，创建合适的函数索引会提升查询效率;

函数索引具有以下约束:

1) 函数索引表达式的返回结果不允许为时间间隔类型、大字段类型（CLOB/BLOB)、自定义类型、二进制类型(BINARY/VARBINARY)、BOOLEAN类型;

2) 函数索引表达式中不允许出现半透明加密列;

3) 函数索引表达式的长度理论值不能超过 816 个字符(包括函数索引表达式本身的长度加上系统生成的指令长度);

4) 函数索引不能为 CLUSTER 或 PRIMARY KEY 类型;

5) 函数索引表达式中不支持集函数和不确定函数,不确定函数为每次执行得到的结果不确定，系统中不确定函数包括：RAND、CURDATE、CURTIME、CURRENT_DATE、CURRENT_TIME、CURRENT_TIMESTAMP、GETDATE、NOW、SYSDATE、CUR_DATABASE、DBID、EXTENT、PAGE、SESSID、UID、USER、VSIZE、SET_TABLE_OPTION、SET_INDEX_OPTION、UNLOCK_LOGIN、CHECK_LOGIN、GET_AUDIT、CFALGORITHMSENCRYPT、SF_MAC_LABEL_TO_CHAR、CFALGORITHMSDECRYPT、BFALGORITHMSENCRYPT、SF_MAC_LABEL_FROM_CHAR、BFALGORITHMSDECRYPT、SF_MAC_LABEL_CMP;

6) 若函数索引中要使用用户自定义的函数,则函数必须是指定了 DETERMINISTIC

属性的确定性函数；

7） 若函数索引中使用的确定性函数发生了变更或删除，用户需手动重建函数索引；

8） 若函数索引中使用的确定性函数内有不确定因素，会导致前后计算结果不同的情况。在查询使用函数索引时，使用数据插入函数索引时的计算结果为 KEY 值；修改时可能会导致在使用函数索引过程中出现根据聚集索引无法在函数索引中找到相应记录的情况，对此进行报错处理；

14. 在水平分区表上创建索引有以下约束：

1） 指定 GLOBAL 关键字，创建全局索引，否则创建局部索引；

2） 只有当分区键都包含在索引键中时，才能创建非全局唯一索引；

3） 不能在水平分区表上创建全局聚集索引；

4） HUGE 水平分区表不支持全局索引；

5） 不能对水平分区子表单独建立索引；

6） 在水平分区表上创建唯一函数索引，需要满足以下任一条件：

(1) 创建为全局索引；

(2) 函数索引的非计算列包含所有分区列。

15. 非聚集索引和聚集索引不能使用OR REPLACE选项互相转换；

16. 位图索引：创建方式和普通索引一致，对低基数的列创建位图索引，能够有效提高基于该列的查询效率，位图索引具有以下约束：

1） 仅支持普通表、堆表和水平分区表创建位图索引；

2） 不支持对大字段创建位图索引；

3） 不支持对计算表达式列创建位图索引；

4） 不支持在 UNIQUE 列和 PRIMARY KEY 上创建位图索引；

5） 不支持对存在 CLUSTER KEY 的表创建位图索引；

6） 仅支持单列或者不超过 63 个组合列上创建位图索引；

7） MPP 环境下不支持位图索引的创建；

8） DPC 环境下不支持位图索引的创建；

9） 不支持全局位图索引；

10）包含位图索引的表不支持并发的插入、删除和更新操作；

11） 不支持在间隔分区表上创建位图索引。

17. NOSORT选项具有以下约束：

1） 不支持与[OR REPLACE]、[CLUSTER]、[BITMAP]、[ONLINE]同用；

2） 与PARALLEL同用时，指定PARALLEL数大于1时，DPC环境会忽略NOSORT。

18. ONLINE选项具有以下约束：

1） 不支持与[OR REPLACE]、[CLUSTER]、[BITMAP]同用；

2） 不支持 MPP 环境；

3） 不支持在临时表上创建索引时指定 ONLINE 选项；

4） 暂不支持列存储表创建索引时指定 ONLINE 选项；

5） 建立 PRIMARY KEY，UNIQUE 约束时，隐式创建的索引不支持 ONLINE 选项；

6） 函数索引支持 ONLINE 选项。

19. 反向索引：创建时需指定REVERSE关键字。对于访问数据呈现密集且集中的场景，普通索引的查询效率较低，使用反向索引可以提高查询效率。反向索引具有以下约束：

1） 不可与 NOSORT 关键字同时使用；

2） 位图索引不可创建为反向索引；

3） 全文索引不可创建为反向索引；

4） 聚集索引不可创建为反向索引；

5） 索引的列必须是支持反转的数据类型，例如数值、字符、日期、时间、时间间隔以及 BINARY 等类型，不支持 bit 和大字段类型；

20. 无效索引：创建时需指定UNUSABLE关键字。使用无效索引需要注意：

1） 不可与 NOSORT、ONLINE、OR REPLACE 关键字同时使用；

2） 仅支持将二级索引创建为无效索引；

3） 不支持将聚集索引、位图索引、数组索引以及虚索引创建为无效索引；

4） 不支持在水平分区子表、临时表、系统表、远程表、数组表以及位图表上创建无效索引；

5） DM 通过并发重建无效索引来达到并发创建索引的效果，具体可参考 3.6.2 索引修改语句 中的说明；

21. 在临时表上增删索引会导致临时表数据丢失。当 DM.INI 参数 ENABLE_TMP_TAB_ROLLBACK为0时，不允许对临时表创建唯一索引。

举例说明

例 1 假设具有 DBA 权限的用户在 VENDOR 表中，以 VENDORID 为索引列建立索引 S1，以 ACCOUNTNO，NAME 为索引列建立唯一索引 S2。

```
CREATE INDEX S1 ON PURCHASING.VENDOR (VENDORID);
CREATE UNIQUE INDEX S2 ON PURCHASING.VENDOR (ACCOUNTNO, NAME);
```

例 2 假设具有 DBA 权限的用户在 SALESPERSON 表中，需要查询比去年销售额超过 20 万的销售人员信息，该过滤条件无法使用到单列上的索引，每次查询都需要进行全表扫描，效率较低。如果在 SALESTHISYEAR-SALESLASTYEAR 上创建一个函数索引，则可以较大程度提升查询效率。

```
CREATE INDEX INDEX_FBI ON SALES.SALESPERSON(SALESTHISYEAR-SALESLASTYEAR);
```

例 3 创建反向索引。

```
//创建含反向索引类型的表
DROP TABLE t1;
CREATE TABLE t1(c1 int, c2 raw(100), c3 timestamp, c4 date, c5 float, c6 interval day to second,
```

```
c7 interval year to month);
//创建反向索引
CREATE INDEX i1 ON t1(c1) REVERSE;
CREATE INDEX i2 ON t1(c2) REVERSE;
CREATE INDEX i3 ON t1(c3) REVERSE;
CREATE INDEX i4 ON t1(c4) REVERSE;
CREATE INDEX i5 ON t1(c5) REVERSE;
CREATE INDEX i6 ON t1(c6) REVERSE;
CREATE INDEX i7 ON t1(c7) REVERSE;
```

例 4 创建无效索引。

```
//创建含无效索引类型的表
DROP TABLE t2;
CREATE TABLE t2(c1 int, c2 varchar);
//创建无效索引
CREATE INDEX uidx ON t2(c1) UNUSABLE;
```

例 5 在 DMDPC 环境中创建全局分区索引。

```
//创建表空间
create tablespace ts1 datafile 'd:\ts\ts01.dbf' size 128 storage (on raft_1);
create tablespace ts2 datafile 'd:\ts\ts02.dbf' size 128 storage (on raft_2);
//创建表
drop table t1;
create table t1(c1 int, c2 int, c3 int)   partition by range(c1)
(
    partition p1 values less than(100),
```

```
    partition p2 values less than(200)
);
//创建全局分区索引
create index idx1111 on t1(c2) global partition by range(c2)
(
  partition p1 values less than(10) storage(on ts2) ,
  partition p2 values less than(1000) storage(on ts1),
  partition p3 values less than(maxvalue)
);
//使用索引 idx111 查询
select * from t1 where c2 = 130;
```

3.6.2 索引修改语句

为了满足用户在建立索引后还可随时修改索引的要求，DM 系统提供索引修改语句，包括修改索引名称、设置索引的查询计划可见性、改变索引有效性、重建索引和索引监控的功能。

语法格式

```
ALTER  INDEX [<模式名>.]<索引名> <修改索引定义子句>
<修改索引定义子句> ::=
RENAME TO [<模式名>.]<索引名> |
INVISIBLE |
VISIBLE |
UNUSABLE |
REBUILD [NOSORT][ONLINE][ <重建方式>] |
```

```
<MONITORING | NOMONITORING> USAGE

<重建方式>::=

    SHARE |

    SHARE ASYNCHRONOUS [<异步任务数>] |

    EXCLUSIVE |

    <表并行度子句>

<表并行度子句>::=

    PARALLEL |

    PARALLEL <并行度> |

    NOPARALLEL
```

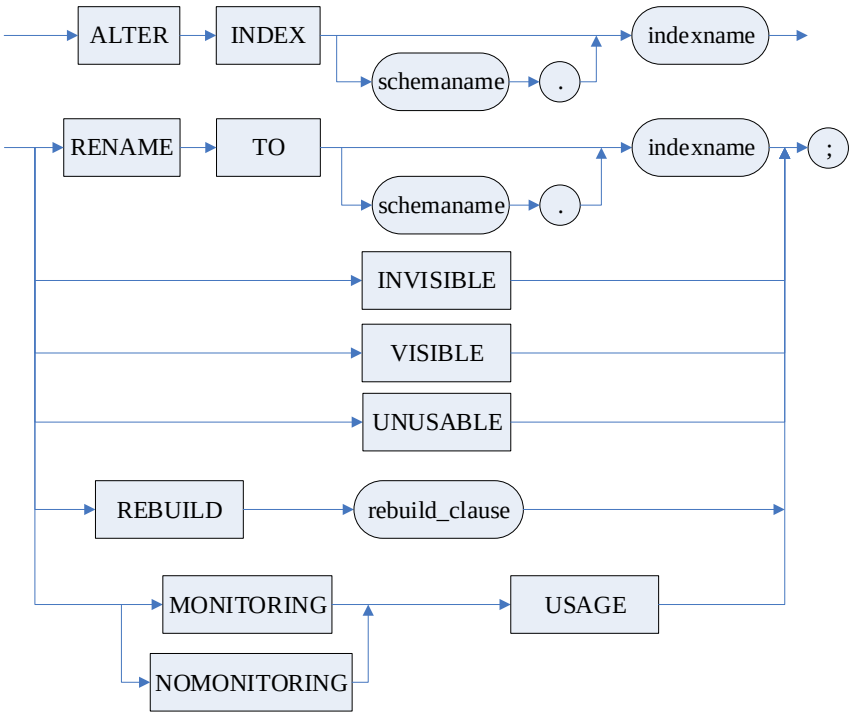
参数

1. <模式名> 索引所属的模式，缺省为当前模式；

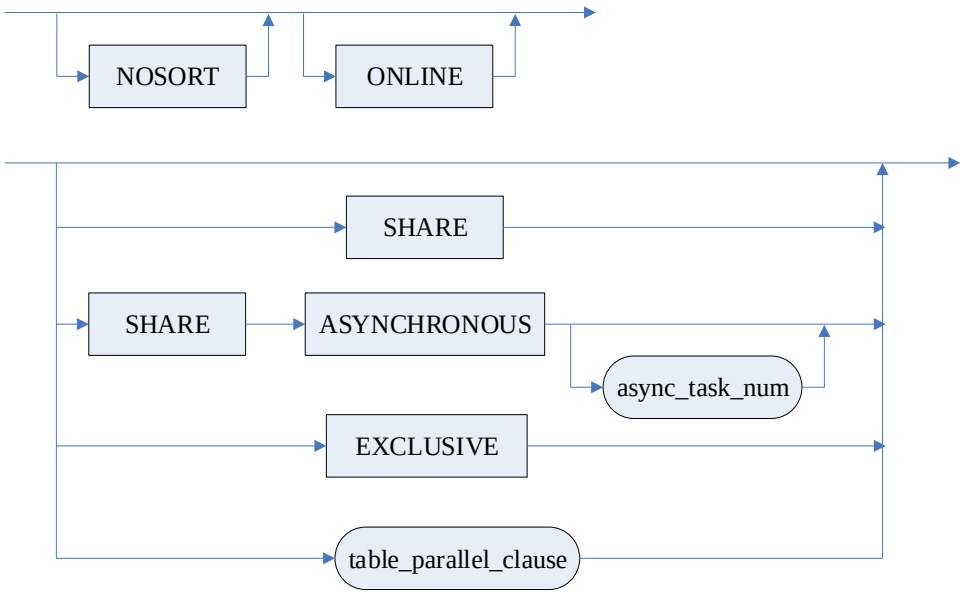
2. <索引名> 索引的名称。修改后的索引名同样需要满足命名规则，具体见3.6.1 索引定义语句。

图例

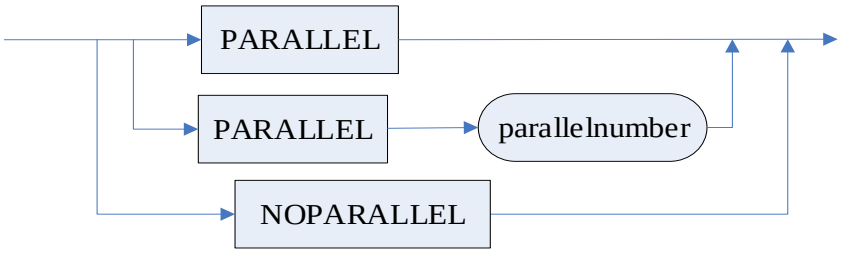
索引修改语句



重建方式（rebuild_clause）



表并行度子句（table_parallel_clause）



语句功能

1. 供具有 DBA 角色(三权分立)的用户或该索引所属基表的拥有者或具有 ALTER ANY INDEX 的用户修改索引;

2. 当索引修改成 INVISIBLE 时，查询语句的执行计划不会使用该索引，该索引相关的计划不会生成，用别的计划代替。修改成 VISIBLE，则会生成索引相关的计划。默认是 VISIBLE;

3. 当指定 UNUSABLE 时，索引将被置为无效状态，系统将不再维护此索引，可以在 SYSOBJECTS 的 VALID 字段查看该值。处于无效状态的索引，可以利用 REBUILD 来重建，或者先删除该索引再新建该索引。TRUNCATE 表会将该表所有失效索引置为生效;

4. 当指定 REBUILD 时，进行索引重建，索引将被置为生效状态。

   1) NOSORT 表示重建时不需要排序;

   2) ONLINE 表示重建时使用异步创建逻辑，在过程中可以对索引依赖的表做增、删、改操作;

   3) <重建方式> 表示重建索引的方式。缺省情况下自适应选择重建索引的方式，当待重建索引满足并发重建索引的约束时，采用并发重建索引的方式，否则采用排他重建索引的方式。

   - SHARE 并发重建索引，表示不同会话同时对同一个表重建索引;
   - SHARE ASYNCHRONOUS 并行重建索引，专门用于分区表，表示多个异步任务并行执行对同一个索引的重建任务。当表的数据量较大无法一次全部放入内存时，原索引重建方式会导致磁盘 I/O 出现明显的波动，单个索引重建性能较差。

此时若机器性能较好（CPU 核数较多、I/O 性能较好），则可通过指定并行重建的方式来提高单个索引重建的性能；

- EXCLUSIVE 排他重建索引，表示不允许并发重建索引或并行重建索引；
- PARALLEL 并行获取数据，与并发、并行、排他、重建索引皆不冲突。

5. MONITORING USAGE 对指定索引进行监控；NOMONITORING USAGE 对指定索引取消监控。索引监控（MONITORING USAGE）仅支持对用户创建的二级索引进行监控，且不支持监控虚索引、系统索引、聚集索引、数组索引，相关监控信息可查看动态视图 V$OBJECT_USAGE。

使用说明

1. 使用者应拥有 DBA 权限或是该索引所属基表的拥有者；
2. <INVISIBLE | VISIBLE> 仅支持表的二级索引修改，对聚集索引不起作用；
3. UNUSABLE 和 REBUILD EXCLUSIVE 仅支持对二级索引的修改，不支持位图索引、聚集索引、虚索引、水平分区子表和临时表上索引的修改；
4. 当把主键对应的索引设置为 UNUSABLE 时，不能再对表进行插入和更新操作；
5. REBUILD 是 UNUSABLE 的逆向操作，REBUILD 支持并发重建索引、并行重建索引以及排他重建索引。并发重建索引以及并行重建索引要求索引必须为 UNUSABLE 的，排他重建索引则无此限制；
6. 数组索引不支持以 ONLINE 或 NOSORT 选项进行重建；
7. SHARE 并发重建索引具有以下约束：

   1） 不可与 NOSORT、ONLINE 关键字同时使用；

   2） 仅支持并发重建无效索引；

   3） 仅支持并发重建二级索引；

   4） 不支持并发重建聚集索引、位图索引、数组索引以及虚索引；

   5） 不支持并发重建位于水平分区子表、临时表、系统表、远程表、数组表或者位

图表上的索引。

8. SHARE ASYNCHRONOUS 并行重建索引具有以下约束：

1） 不可与 NOSORT、ONLINE 关键字同时使用；

2） 仅支持并行重建无效索引；

3） 仅支持并行重建二级索引；

4） 不支持并行重建聚集索引、位图索引、数组索引、虚索引以及全局索引；

5） 仅支持并行重建分区表上的索引；

6） 不支持并行重建水平分区子表、临时表、系统表、远程表、数组表或者位图表上的索引；

7） 低性能机器（CPU 核数过少、I/O 性能不足）不适用于指定并行重建索引。

9. <异步任务数>并行重建时的异步任务个数。<异步任务数>的取值具有以下约束：

1） 异步任务数的取值范围为 2~100 之间的整数，若取值小于 2，则无法并行重建索引，若取值大于 100，则调整异步任务数为 100。缺省时，系统将自适应选择异步任务数，系统自适应选择的异步任务数最大为 16；

2） 建议取值不超过需要执行 INDEX ON 分区子表的个数；

3） 建议取值不超过 INI 参数 TASK_THREADS 的取值；

4） 当内存资源紧张时，建议适当调小异步任务数。

10. 附加回滚记录（包含 RPTR 字段）的二级索引不支持异步重建；

11. 当 INI 参数 MONITOR_INDEX_FLAG 为 1 或 2 时，使用 ALTER INDEX 方式设置索引监控失效。

举例说明

例 1 具有 DBA 权限的用户需要重命名 S1 索引可用以下语句实现。

```
ALTER INDEX PURCHASING.S1 RENAME TO PURCHASING.S2;
```

例 2 当索引为 VISIBLE 时，查询语句执行计划如下：

```
DROP TABLE TEST;
CREATE TABLE TEST (C1 INT, C2 INT);
CREATE  INDEX  INDEX_C1  ON TEST (C1);
explain  select c1 from test;
```

执行计划如下：

```
1   #NSET2: [1, 1, 16]
2     #PRJT2: [1, 1, 16]; exp_num(2), is_atom(FALSE)
3       #SSCN: [1, 1, 16]; INDEX_C1(TEST); btr_scan(1); is_global(0)
```

修改索引为 INVISIBLE 后，查询语句执行计划。

```
alter index  index_C1 INVISIBLE;
explain  select c1 from test;
```

执行计划如下：

```
1   #NSET2: [1, 1, 16]
2     #PRJT2: [1, 1, 16]; exp_num(2), is_atom(FALSE)
3       #CSCN2: [1, 1, 16]; INDEX33555472(TEST); btr_scan(1)
```

例 3 使用 UNUSABLE 将索引置为无效状态。

```
DROP TABLE TEST;
CREATE TABLE TEST (C1 INT, C2 INT);
CREATE  INDEX  INDEX_C1  ON TEST (C1);
ALTER  INDEX  INDEX_C1  UNUSABLE;
```

此时系统将不维护 INDEX_C1，与此相关的计划均失效。

例 4 并发重建索引。

```
//创建含无效索引类型的表
DROP TABLE TEST;
```

```
CREATE TABLE TEST(C1 INT, C2 INT);
//创建无效索引
CREATE INDEX INDEX_C1 ON TEST(C1) UNUSABLE;
CREATE INDEX INDEX_C2 ON TEST(C2) UNUSABLE;
//插入数据
INSERT INTO TEST SELECT LEVEL, LEVEL FROM DUAL CONNECT BY LEVEL <= 30000000;
//会话 1 并发重建索引 INDEX_C1
ALTER INDEX INDEX_C1 REBUILD SHARE;
//会话 2 并发重建索引 INDEX_C2
ALTER INDEX INDEX_C2 REBUILD SHARE;
//在上述两个索引重建过程中，会话 3 执行如下语句
SELECT SQL_TEXT,STATE FROM V$SESSIONS WHERE TRX_ID IN (SELECT WAIT_FOR_ID FROM
V$TRXWAIT);
//会话 3 上的查询结果如下
未选定行
```

由查询结果可以知道，并未有线程处于等待状态，会话 1 和会话 2 正在并发重建 TEST 表上的索引。

例 5 并行重建分区表上的索引。

```
//创建含无效索引类型的分区表
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT, C2 INT) PARTITION BY HASH(C1) PARTITIONS 3;
//创建无效索引
CREATE INDEX INDEX_C1 ON TEST(C1) UNUSABLE;
//并行重建分区表上的索引
```

```
ALTER INDEX INDEX_C1 REBUILD SHARE ASYNCHRONOUS 3;
```

例 6 排他重建索引。

```
DROP TABLE TEST;

CREATE TABLE TEST(C1 INT, C2 INT);

CREATE INDEX INDEX_C1 ON TEST(C1) UNUSABLE;

CREATE INDEX INDEX_C2 ON TEST(C2);

//排他重建索引

ALTER INDEX INDEX_C1 REBUILD EXCLUSIVE;

ALTER INDEX INDEX_C2 REBUILD EXCLUSIVE;
```

3.6.3 索引删除语句

DM 系统允许用户在建立索引后还可随时删除索引。

语法格式

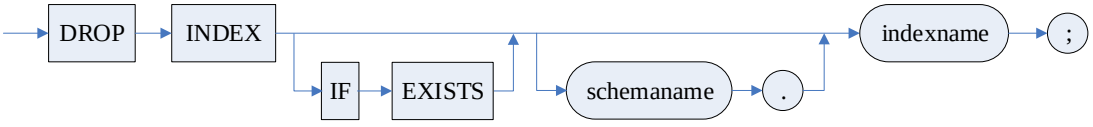
```
DROP INDEX [IF EXISTS] [<模式名>.]<索引名>;
```

参数

1. <模式名> 指明被删除索引所属的模式，缺省为当前模式；
2. <索引名> 指明被删除索引的名称。

图例

索引删除语句



语句功能

供具有 DBA 角色（三权分立）的用户或该索引所属基表的拥有者删除索引。

使用说明

1. 使用者应拥有 DBA 权限或是该索引所属基表的拥有者；

2. 删除不存在的索引会报错。若指定 IF EXISTS 关键字，删除不存在的索引，不会报错。

3. 列存储表和堆表不允许删除聚集索引

举例说明

例 具有 DBA 权限的用户需要删除 S2 索引可用以下语句实现。

```
DROP INDEX PURCHASING.S2;
```

3.7 管理位图连接索引

3.7.1 位图连接索引定义语句

位图连接索引是一种通过连接实现提高海量数据查询效率的有效方式，主要用于数据仓库环境中。它是针对两个或者多个表连接的位图索引，同时保存了连接的位图结果。对于列中的每一个值，该索引保存了索引表中对应行的 ROWID。

语法格式

```
CREATE [OR REPLACE] BITMAP INDEX [IF NOT EXISTS] <索引名>
ON <bitmap_join_index_clause> [<表空间子句>] [<STORAGE 子句>][<PARALLEL 项>];
<bitmap_join_index_clause>::=[<模式名>.]<表名>(<索引列定义>{,<索引列定义>})FROM [<模式名>.]<基表名>[别名]{,[<模式名>.]<基表名>[别名]} WHERE <条件表达式>;
<STORAGE 子句> ::= STORAGE(<STORAGE 项> {,<STORAGE 项>})
<STORAGE 项> ::=
    INITIAL <初始簇数目> |
    NEXT <下次分配簇数目> |
    MINEXTENTS <最小保留簇数目> |
```

```
    ON <表空间名> |
    LOB ON <表空间名>|
    FILLFACTOR <填充比例> |
    BRANCH  <BRANCH 数> |
    BRANCH  (<BRANCH 数>, <NOBRANCH 数>) |
    NOBRANCH |
    CLUSTERBTR |
    WITH COUNTER |
    WITHOUT COUNTER |
    USING LONG ROW|
    DISABLE USING LONG ROW|
    SECTION (<存储位图字节数>)
<表空间子句>、<STORAGE 项>中除了 SECTION 之外的部分、<PARALLEL 项>请参考本章 3.5.1.1 定义数据库基表相关内容
<索引列定义>::= [[<模式名>.]<表名|别名>]<索引列表达式>[ASC|DESC]
```

参数

1. OR REPLACE：只支持在第一次创建时使用 OR REPLACE，重建时不支持 OR REPLACE；

2. ON 子句：指定的表为事实表，括号内的列既可以是事实表的列也可以是维度表的列；

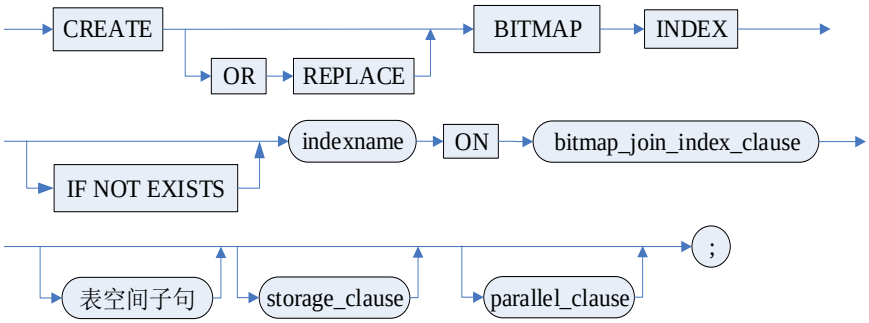
3. FROM 子句：指定参与连接的表；

4. WHERE 子句：指定连接条件；

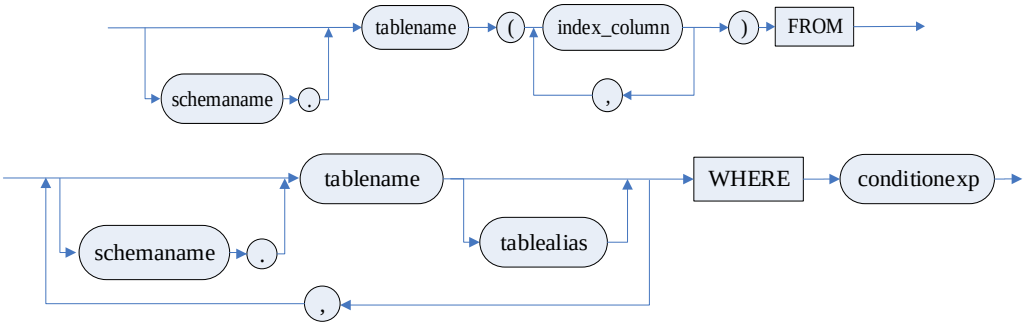
5. 其他参数说明请参考 3.6 管理索引。

图例

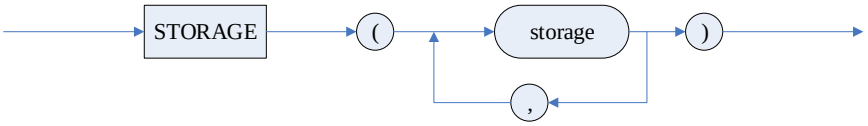
位图连接索引定义语句



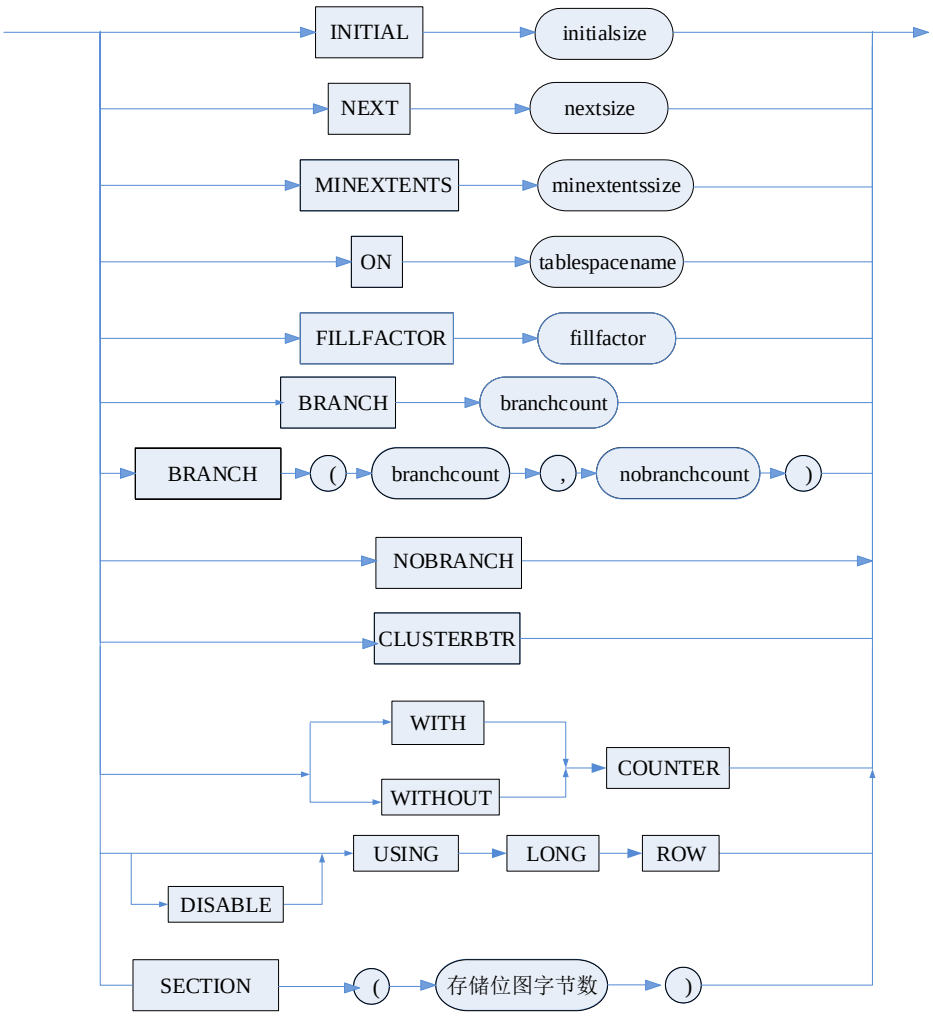
bitmap_join_index_clause



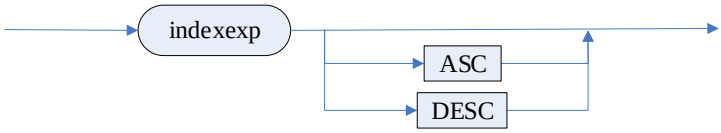
storage_clause



storage



index_column



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或该索引所属基表的拥有者且具有 CREATE INDEX 或 CREATE ANY INDEX 权限的用户定义索引。

使用说明

1. 适用于常规索引的基本限制也适用于位图连接索引；

2. 可以通过调整 INI 参数 ENABLE_CREATE_BM_INDEX_FLAG 的值控制是否允许创建位图连接索引，设置该参数为缺省值 1 时，允许创建；设置值为 0 时，不允许创建；设置

值为 2 时，将位图索引作为普通 B 树索引进行创建；

3. 用于连接的列必须是维度表中的主键或存在唯一约束；如果是复合主键，则必须使用复合主键中的所有列；

4. 当多个事务同时使用位图连接索引时，同一时间只允许更新一个表；

5. 连接索引创建时，基表只允许出现一次；

6. 不允许对存在 CLUSTER KEY 的表创建位图连接索引；

7. 位图连接索引表（内部辅助表，命名为 BMJ$ _索引名）仅支持 SELECT 操作，其他操作都不支持：如 INSERT、DELETE、UPDATE、ALTER、DROP 和建索引等；

8. 不支持对位图连接索引所在事实表和维度表的备份还原，不支持位图连接索引表的表级备份还原；

9. 不支持位图连接索引表、位图连接索引以及虚索引的导出导入；

10. 位图连接索引及其相关表不支持快速装载；

11. 位图连接索引名称的长度限制为：事实表名的长度+索引名称长度+6<128；

12. 支持普通表、堆表和 HUGE 表；

13. WHERE 条件只能是列与列之间的等值连接，并且必须含有所有表；

14. 事实表上聚集索引和位图连接索引不能同时存在；

15. 不支持对含有位图连接索引的表中的数据执行 DML，如需要执行 DML，则先删除该索引；

16. 含有位图连接索引的表不支持下列 DDL 操作：删除、修改表约束，删除、修改列，更改表名。另外，含位图连接索引的堆表不支持添加列操作；

17. 不允许对含有位图连接索引的表并发操作；

18. 创建位图连接索引时，在存储参数中可指定存储位图的字节数，有效值为：4~2048，服务器自动校正为 4 的倍数，默认值为 128。如 STORAGE(SECTION(4))，表示使用 4 个字节存储位图信息；

19. DPC 环境下不支持位图连接索引的创建。

举例说明

创建位图连接索引：

```
create bitmap index SALES_CUSTOMER_NAME_IDX
on SALES.SALESORDER_HEADER(SALES.CUSTOMER.PERSONID)
from    SALES.CUSTOMER, SALES.SALESORDER_HEADER
where   SALES.CUSTOMER.CUSTOMERID = SALES.SALESORDER_HEADER.CUSTOMERID;
```

执行查询：

```
Select TOTAL
from SALES.CUSTOMER, SALES.SALESORDER_HEADER
where SALES.CUSTOMER.CUSTOMERID = SALES.SALESORDER_HEADER.CUSTOMERID
and SALES.CUSTOMER.PERSONID = '12';
```

3.7.2 位图连接索引删除语句

如果不需要位图连接索引可以使用删除语句删除。

删除（位图连接）索引语句格式：

```
DROP INDEX [IF EXISTS] [<模式名>.]<索引名>;
```

参数

参数说明请参考本章 3.6 管理索引。

举例说明

例如：

```
DROP INDEX sales.SALES_CUSTOMER_NAME_IDX;
```

3.8 管理全文索引

3.8.1 全文索引定义语句

用户可以在指定的表的文本列上建立全文索引。

语法格式

```
CREATE CONTEXT INDEX [IF NOT EXISTS] <索引名> ON [<模式名>.] <表名> (<索引列定义>) [<表空间子句>] [<STORAGE 子句>] [LEXER <分词参数>] [<SYNC 子句>];
<索引列定义>、<表空间子句>、[<STORAGE 子句>] 请参考本章 3.6.1 索引定义语句相关内容
<SYNC 子句> ::= SYNC [TRANSACTION]
```

参数

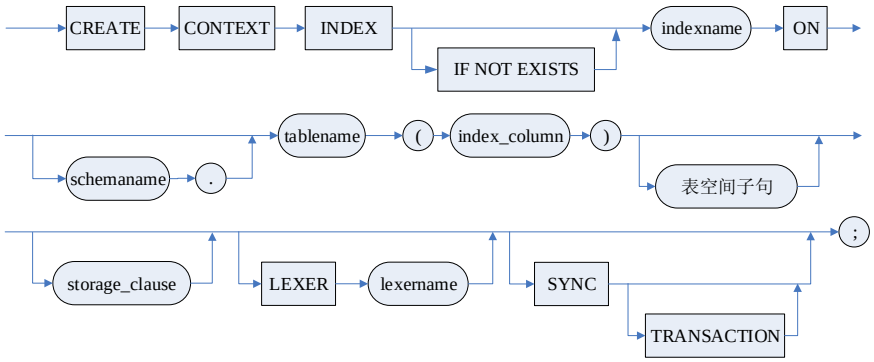
1. <索引名> 指明要创建的全文索引的名称，由于系统会为全文索引名加上前缀与后缀，因此用户指定的全文索引名长度不能超过122字节；
2. <模式名> 指明要创建全文索引的基表属于哪个模式，缺省为当前模式；
3. <表名> 指明要创建全文索引的基表的名称；
4. <列名> 指明基表中要创建全文索引的列的名称；
5. <分词参数> 指明全文索引分词器的分词参数；
6. <storage 子句> 只有指定表空间参数有效，其他参数无效（即 STORAGE ON xxx 或者 TABLESPACE xxx 有效，而诸如 INITIAL、NEXT 等无效）；
7. <SYNC子句>用于指定全文索引的更新方式，即将表中的数据更新到全文索引中。只有更新过的全文索引才能被检索，因此，在创建全文索引时或者使用全文索引之前，必须对全文索引进行更新。指定SYNC表示系统将在建立全文索引时对全文索引执行一次完全更新，如果此处未指定SYNC，创建全文索引后系统不会进行全文索引完全更新，那么必须使用使用指定了REBUILD的全文索引更新语句来执行一次完全更新，全文索引才可以被使用；指定TRANSACTION表示每次事务提交后，若基表数据发生变化，系统会自动以增量更新方

式再次更新全文索引，不需要用户手动填充。

8. 索引的模式名与其基表的模式名一致，不支持自行指定。

图例

全文索引定义语句



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或该全文索引基表的拥有者且具有 CREATE CONTEXT INDEX 或 CREATE ANY CONTEXT INDEX 权限的用户，在指定的表的文本列上建立全文索引。

使用说明

1. 全文索引必须在一般用户表上定义，而不能在系统表、视图、临时表、列存储表和外部表上定义；

2. 一个全文索引只作用于表的一个文本列，不允许为组合列和计算列；

3. 同一列只允许创建一个全文索引；

4. <列名>为文本列，类型可为CHAR、CHARACTER、VARCHAR、LONGVARCHAR、TEXT或CLOB；

5. TEXT、CLOB类型的列可存储二进制字符流数据。如果用于存储DM全文检索模块能识别的格式简单的文本文件(如.txt，html等)，则可为其建立全文索引；

6. 全文索引支持简体中文和英文；

7. 分词参数有5种：CHINESE_LEXER，中文最少分词；CHINESE_VGRAM_LEXER，

机械双字分词，CHINESE_FP_LEXER，中文最多分词；ENGLISH_LEXER，英文分词；DEFAULT_LEXER，中英文最少分词，也是默认分词；

8. 全文索引更新方式分为完全更新和增量更新。一个全文索引需要执行一次完全更新和若干次增量更新。完全更新的方式有两种：一是创建全文索引时，通过指定<SYNC子句>在建完全索引的同时对完全索引进行完全更新；二是在使用全文索引之前，执行指定了REBUILD的更新语句实现；二者选择其一即可。增量更新为每当表中数据发生增量变化后，以增量更新的方式再次更新全文索引。增量更新的方式有两种：一是通过指定<SYNC子句>的TRANSACTION实现；二是执行指定了INCREMENT关键字的更新语句实现；二者可搭配使用；

9. 不支持快速装载建有全文索引的表。

举例说明

例 用户 SYSDBA 需要在 PERSON 模式下的 ADDRESS 表的 ADDRES1 列上创建全文索引，可以用下面的语句：

```
CREATE CONTEXT INDEX INDEX0001 ON PERSON.ADDRESS(ADDRESS1) LEXER CHINESE_LEXER;
```

3.8.2 全文索引更新语句

全文索引需要根据基表的数据变化进行索引数据更新。若基表数据发生变化而没有及时更新全文索引，会引起全文检索结果不正确。

使用全文索引修改语句对全文索引进行完全更新和增量更新，使得全文索引的内容与表数据保持同步。

语法格式

```
ALTER CONTEXT INDEX <索引名> ON [<模式名>.] <表名> <REBUILD | INCREMENT | OPTIMIZE>[ONLINE]
[LEXER <分词参数>];
```

参数

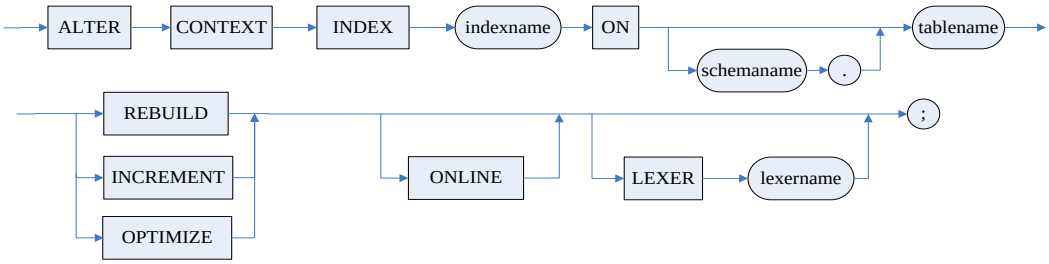
1. <索引名> 指明被操作的全文索引的名称;

2. <模式名> 指明被操作的全文索引属于哪个模式，缺省为当前模式;

3. <表名> 指明被操作的基表的名称。

图例

全文索引修改语句



语句功能

供具有 DBA 角色（三权分立）的用户，或该全文索引基表的拥有者（拥有者需同时具有 ALTER CONTEXT INDEX 权限），或具有 ALTER ANY CONTEXT INDEX 权限的用户在指定的表的文本列上修改全文索引。

REBUILD 为完全更新，此方式首先会将全文索引的辅助表清空，再将基表中所有记录逐个取出，根据分词算法获得分词结果（即字/词所在记录的 ROWID 和出现次数，出现次数又叫词频），并保存在词表中。INCREMENT 为增量更新，此方式只是将基表中发生数据变化的记录执行分词并保存分词结果。OPTIMIZE 操作仅仅对全文索引辅助表进行优化，去除冗余信息，不影响查询结果。

使用说明

1. 只有在创建全文索引时未使用 SYNC 选项，才需要指定 REBUILD 对全文索引进行一次全面更新，然后才能进行全文检索;

2. 当表中该列数据发生了改变，为了对改变后的数据进行检索，需要使用 INCREMENT 再次更新全文索引信息，才能检索到改变后的信息;

3. DM 服务器启动时，不会自动加载词库，而是在第一次执行全文索引更新时加载，

之后直到服务器停止才释放;

4. 在完全更新全文索引后，如果表数据发生少量更新，利用 INCREMENT 增量填充方式更新全文索引可以提高效率;

5. 语句中指定 ONLINE 选项时，指明对全文索引进行异步填充，允许同时对全文索引所在的表进行增删改操作;

6. LEXER 子句只能与 REBUILD 方式一起使用;

7. 在 DMDSC 环境下更新全文索引前，需要手动将执行码目录下的 SYSWORD.UTF8.LIB 文件复制到库所在的 DMASM 路径下, 否则会导致全文索引词库加载错误。

举例说明

例 用户 SYSDBA 需要在 PERSON 模式下的 ADDRESS 表的 ADDRES1 列上完全填充全文索引，可以用下面的语句:

```
ALTER CONTEXT INDEX INDEX0001 ON PERSON.ADDRESS REBUILD;
```

3.8.3 全文索引删除语句

删除全文索引。

语法格式

```
DROP CONTEXT INDEX [IF EXISTS] <索引名> ON [<模式名>.] <表名>;
```

参数

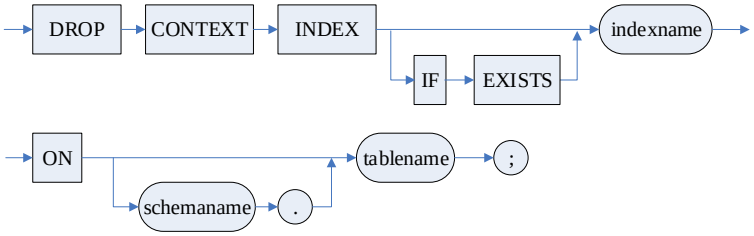
1. <索引名> 指明被操作的全文索引的名称;

2. <模式名> 指明被操作的全文索引属于哪个模式，缺省为当前模式;

3. <表名> 指明被操作的基表的名称。

图例

全文索引删除语句



语句功能

供具有 DBA 角色（三权分立）的用户或该全文索引基表的拥有者或该全文索引所属基表的拥有者删除全文索引，包括删除数据字典中的相应信息和全文索引内容。

使用说明

1. 删除不存在的全文索引会报错。若指定 IF EXISTS 关键字，删除不存在的全文索引，不会报错；

2. 除了该语句可删除全文索引外，当数据库模式发生如下改变时，系统将自动调用全文索引删除模块：

   1） 删除表时，删除表上的全文索引；

   2） 删除建立了全文索引的列时，删除列上的全文索引；

3. 不允许修改建有全文索引的列。

举例说明

例 用户 SYSDBA 需要删除在 PERSON 模式下 ADDRESS 表的全文索引，可以用下面的语句：

```
DROP CONTEXT INDEX INDEX0001 ON PERSON.ADDRESS;
```

3.8.4 用户自定义分词词库

全文索引的分词词库分为两种：一是系统分词词库（SYSWORD.UTF8.LIB），该词库是只读的，不允许用户修改；二是用户自定义分词词库。

通常情况下，系统分词词库可以满足用户的查询需求。当系统分词词库无法精确识别到

某些词语(比如人名、地名、网络新词语等)时，用户可自定义一个分词词库作为补充。

用户自定义分词词库的方法：第一步 使用自定义分词词库系统过程（详见 附录 3 系统存储过程和函数)，自定义一个分词词库；第二步 向自定义分词词库中添加分词；第三步 重启服务器；第四步 用户手动更新全文索引，系统会将用户自定义分词词库与系统分词词库组合起来，共同为分词工作提供依据；第五步 使用自定义词库查询。

用户创建、更新、删除自定义分词词库之后，均需要重启服务器才能生效。

例 当出现了类似下面这种系统词库无法满足查询语句要求，无法获取准确结果的情景，可通过自定义分词词库来解决。

```
CREATE TABLE T1(C1 VARCHAR);
INSERT INTO T1 VALUES(INSERT INTO T1 VALUES('欧阳克服了黄蓉的计谋');
CREATE CONTEXT INDEX CTI1 ON T1(C1);
ALTER CONTEXT INDEX CTI1 ON T1 REBUILD;
SELECT * FROM T1 WHERE CONTAINS(C1, '欧阳克');
//无返回结果，原因是系统分词词库将"欧阳克服了"拆分成了"欧阳"和"克服"词汇。
```

解决办法如下：

第一步，在计算机上新建文件夹'D:\USER_LIB'，并在数据库中创建目录对象'DIR1'，然后自定义分词词库 USER_LIB。

```
CREATE OR REPLACE DIRECTORY DIR1 AS 'D:\USER_LIB';
SP_WORD_LIB_CREATE('DIR1', 'LIB1.LIB');
```

第二步，将"欧阳克"分词添加到 USER_LIB 中。

```
SP_WORD_LIB_ADD_WORD('DIR1', 'LIB1.LIB', '欧阳克');
SP_WORD_LIB_ADD_WORD('DIR1', 'LIB1.LIB', '服了');
```

第三步，重启数据库服务器。

第四步，更新全文索引。

```
ALTER CONTEXT INDEX CTI1 ON T1 REBUILD;
```

第五步，使用 USER_LIB 执行查询语句。获取到准确结果。

```
SELECT * FROM T1 WHERE CONTAINS(C1, '欧阳克');

行号          C1

---------- --------------------

1             欧阳克服了黄蓉的计谋
```

3.9管理数组索引

数组索引指在一个只包含单个数组成员的对象列上创建的索引。

3.9.1 数组索引定义语句

语法格式

```
CREATE ARRAY INDEX [IF NOT EXISTS] <索引名> ON  [<模式名>.] <表名> (<索引列定义>)
```

使用说明

1. 暂不支持在水平分区表上创建数组索引；
2. 暂时不支持在有数组索引表上进行批量装载(数组索引失效的例外)；
3. 支持创建数组索引的对象只能包含数组一个成员。数组可以是 DM 静态数组、动态数组或者 ORACLE 兼容的嵌套表或 VARRAY；
4. 数组项类型只能是可比较的标量类型，不支持复合类型、对象类型或大字段类型；
5. 临时表不支持；
6. 数组索引不支持改名；
7. 数组索引列不支持改名；
8. 数组索引只能是单索引，不能为组合索引；
9. 不支持空值的检索；

10. MPP 环境不支持数组索引。

3.9.2 数组索引修改语句

数组索引修改语句与普通索引用法相同，请参考 3.6 管理索引。与普通索引不同的是，数组索引不支持 NOSORT 和 ONLINE 用法。

3.9.3 数组索引使用

使用数组索引进行查询，必须使用谓词 CONTAINS。

语法格式

```
CONTAINS(<索引列名>,val {,val})

或者

CONTAINS(<索引列名>,arr_var_exp)
```

参数

1. val：必须为与对象列数组项相同或可转换的标量类型表达式。

2. arr_var_exp：必须为数组类型（DM 静态数组、动态数组或者 ORACLE 兼容的嵌套表或 VARRAY），其数组项类型必须与对象列数组项类型相同或可转换。

举例说明

```
CREATE TYPE ARR_NUM1 IS VARRAY(1024) OF NUMBER;  //VARRAY 数组

/

CREATE TYPE ARR_NUM2 IS TABLE OF NUMBER;  //嵌套表

/

CREATE TYPE ARR_NUM3 IS ARRAY NUMBER[];  //动态

/

CREATE TYPE ARR_NUM4 IS ARRAY NUMBER[3]; //静态
```

```
/

CREATE CLASS CLS1 AS V ARR_NUM1;END;

/


CREATE TABLE TEST (C1 CLS1);

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(1,2,3)));

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(1,2)));

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(2,1)));

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(1,5)));

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(2,4)));

INSERT INTO TEST VALUES(CLS1(ARR_NUM1(4,5,6)));


CREATE ARRAY INDEX IDX ON TEST(C1);    //创建数组索引

SELECT * FROM TEST WHERE CONTAINS(C1,1,2,3);  //使用数组索引查询


//嵌套表

DECLARE

X   ARR_NUM2;

BEGIN

X := ARR_NUM2();

X.EXTEND(3);

X(1) := 1;

X(2) := 2;

X(3) := 3;
```

```
SELECT * FROM TEST WHERE CONTAINS(C1,X);

END;

/



//动态数组

DECLARE

X   ARR_NUM3;

BEGIN

X := NEW NUMBER [3];

X[1]:= 1;

X[2]:= 2;

X[3]:= 3;

SELECT * FROM TEST WHERE CONTAINS(C1,X);

END;

/



//静态数组

DECLARE

X   ARR_NUM4;

BEGIN

X[1]:= 1;

X[2]:= 2;

X[3]:= 3;

SELECT * FROM TEST WHERE CONTAINS(C1,X);
```

```
END;
/
```

3.9.4 数组索引删除语句

数组索引删除语句与普通索引用法相同，请参考 3.6 管理索引。

3.10 管理序列

3.10.1 序列定义语句

序列是一个数据库实体，通过它多个用户可以产生唯一整数值，可以用序列来自动地生成主关键字值。

语法格式

```
CREATE SEQUENCE [IF NOT EXISTS] [<模式名>.] <序列名> [<序列选项列表>];
<序列选项列表> ::=  <序列选项>{<序列选项>}
<序列选项> ::=
    INCREMENT BY <增量值>|
    START WITH <初值>|
    MAXVALUE <最大值>|
    NOMAXVALUE|
    MINVALUE <最小值>|
    NOMINVALUE|
    CYCLE|
    NOCYCLE|
    CACHE <缓存值>|
    NOCACHE|
```

```
ORDER |
NOORDER |
GLOBAL |
LOCAL
```

参数

1. <模式名> 指明被创建的序列属于哪个模式，缺省为当前模式；

2. <序列名> 指明被创建的序列的名称，序列名称最大长度128字节；

3. <增量值> 指定序列数之间的间隔，这个值可以是(-9223372036854775808，9223372036854775807]之间任意的DM正整数或负整数，但不能为0。如果此值为负，序列是下降的，如果此值为正，序列是上升的。如果忽略INCREMENT BY子句，则间隔缺省为1。增量值的绝对值必须小于等于(<最大值> - <最小值>)；

4. <初值> 指定被生成的第一个序列数，可以用这个选项来从比最小值大的一个值开始升序序列或比最大值小的一个值开始降序序列。对于升序序列，缺省值为序列的最小值，对于降序序列，缺省值为序列的最大值；

5. <最大值> 指定序列能生成的最大值，如果忽略MAXVALUE子句，则降序序列的最大值缺省为-1，升序序列的最大值缺省为9223372036854775807(0x7FFFFFFFFFFFFFFF)，若指定的最大值超出缺省最大值，则DM自动将最大值置为缺省最大值。非循环序列在到达最大值之后，将不能继续生成序列数；

6. <最小值> 指定序列能生成的最小值，如果忽略MINVALUE子句，则升序序列的最小值缺省为1，降序序列的最小值缺省为-9223372036854775808(0x8000000000000000)，若指定的最小值超出缺省最小值，则DM自动将最小值置为缺省最小值。循环序列在到达最小值之后，将不能继续生成序列数。最小值必须小于最大值；

7. CYCLE 该关键字指定序列为循环序列：当序列的值达到最大值/最小值时，序列

将从最小值/最大值计数；

8. NOCYCLE 该关键字指定序列为非循环序列：当序列的值达到最大值/最小值时，序列将不再产生新值；

9. CACHE 该关键字表示序列的值是预先分配，并保持在内存中，以便更快地访问。<缓存值>指定预先分配的值的个数，最小值为2，最大值不大于2147483647；

10. NOCACHE 该关键字表示序列的值是不预先分配；

11. ORDER 该关键字表示以保证请求顺序生成序列号；

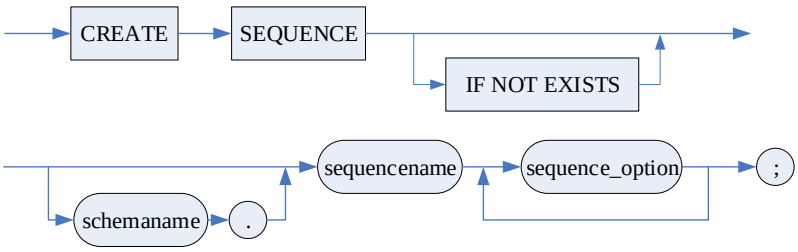
12. NOORDER 该关键字表示不保证请求顺序生成序列号；

13. GLOBAL 该关键字表示MPP环境下序列为全局序列，缺省为GLOBAL；

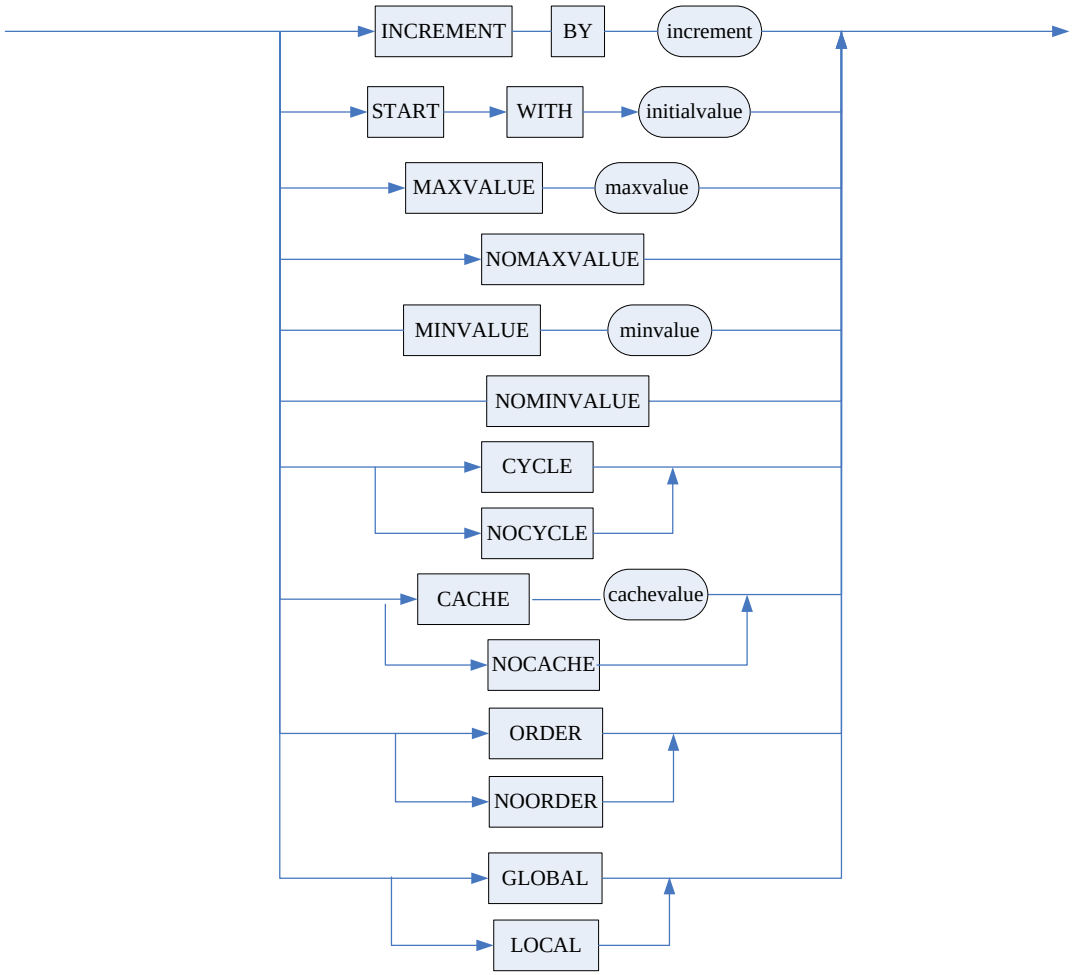
14. LOCAL 该关键字表示MPP环境下序列为本地序列。

图例

序列定义语句



sequence_option



语句功能

创建一个序列。供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE SEQUENCE 或 CREATE ANY SEQUENCE 权限的用户才能创建序列。

使用说明

1. 一旦序列生成，就可以在 SQL 语句中用以下伪列来存取序列的值；

   1） CURRVAL 返回当前的序列值；

   2） NEXTVAL 如果为升序序列，序列值增加并返回增加后的值；如果为降序序列，序列值减少并返回减少后的值。如果第一次对序列使用该函数，则返回序列当前值；

   3） 用户会话在第一次使用 CURRVAL 之前应先使用 NEXTVAL 获取序列当前值；之后除非会话使用 NEXTVAL 获取序列当前值，否则每次使用 CURRVAL 返回

的值不变。

2. 缺省序列：如果在序列中什么也没有指出则缺省生成序列，一个从 1 开始增量为 1 且无限上升(最大值为 9223372036854775807)的升序序列；仅指出 INCREMENT BY -1，将创建一个从-1 开始且无限下降(最小值为-9223372036854775808)的降序序列；

3. LOCAL 类型序列的最高 10 位用来记录标识 MPP 节点号，因此 LOCAL 类型的序列值和 GLOBAL 类型序列在范围、最大值、最小值上都有所差别。LOCAL 类型序列创建时可设置的最大值、最小值分别为 9007199254740991、-9007199254740992。需要注意的是，最高 10 位设置了 MPP 站点号以后，序列的真实值实际上可能不会在序列定义的最大最小值范围内。

举例说明

例 创建序列 SEQ_QUANTITY，将序列的前两个值插入表 PRODUCTION.PRODUCT_INVENTORY 中。

(1)创建序列 SEQ_QUANTITY

```
CREATE SEQUENCE SEQ_QUANTITY INCREMENT BY 10;
```

(2)将序列的第一个值插入表 PRODUCT_INVENTORY 中

```
INSERT INTO PRODUCTION.PRODUCT_INVENTORY VALUES(1,1, SEQ_QUANTITY.NEXTVAL);
SELECT * FROM PRODUCTION.PRODUCT_INVENTORY;
```

查询结果为：表 PRODUCT_INVENTORY 增加一行，列 QUANTITY 的值为 1。

(3)将序列的第二个值插入表 PRODUCT_INVENTORY 中

```
INSERT INTO PRODUCTION.PRODUCT_INVENTORY VALUES(1,1, SEQ_QUANTITY.NEXTVAL);
SELECT * FROM PRODUCTION.PRODUCT_INVENTORY;
```

查询结果为：表 PRODUCT_INVENTORY 增加两行，列 QUANTITY 的值分别为 1，11。

3.10.2 序列修改语句

DM 系统提供序列修改语句，包括修改序列步长值、设置序列最大值和最小值、改变序列的缓存值、循环属性、ORDER 属性、当前值、为序列重命名等。

语法格式

```
ALTER  SEQUENCE [ <模式名>.] <序列名> {<序列修改选项列表> | <序列重命名选项>};

<序列选项列表> ::=  <序列修改选项> {<序列修改选项>}

<序列修改选项> ::=

    INCREMENT BY <增量值>|

    MAXVALUE <最大值>|

    NOMAXVALUE|

    MINVALUE <最小值>|

    NOMINVALUE|

    CYCLE|

    NOCYCLE|

    CACHE <缓存值>|

    NOCACHE|

    ORDER|

    NOORDER |

    CURRENT VALUE <当前值>

<序列重命名选项> ::=

    RENAME  TO <新序列名>
```

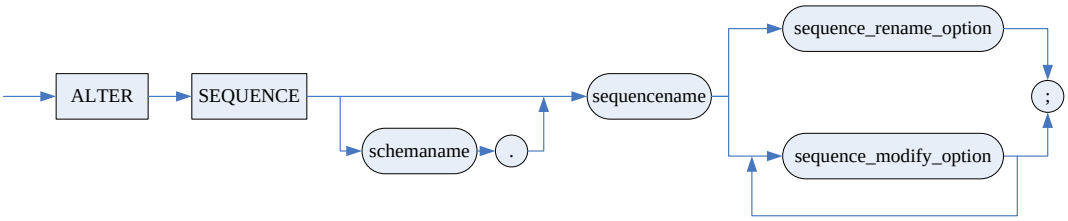
参数

<新序列名> 指修改后的序列的名字。

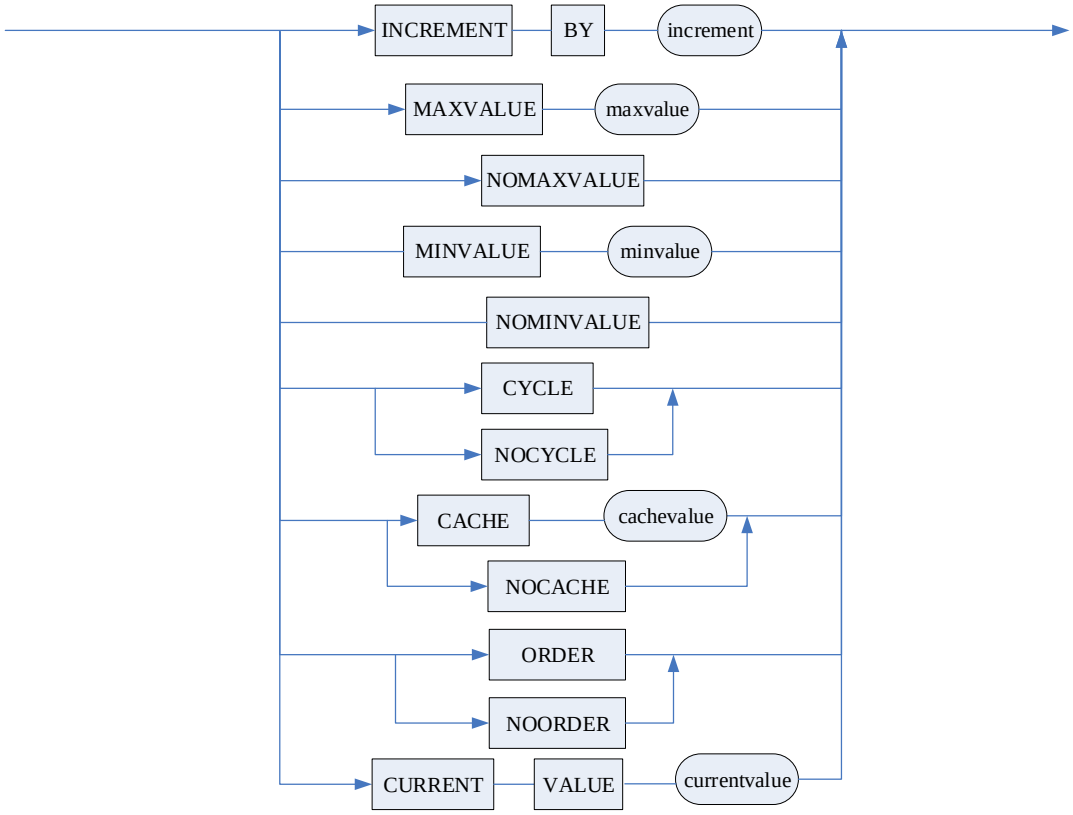
其余参数与序列定义语句的参数相同。

图例

序列修改语句



sequence_modify_option



sequence_rename_option



语句功能

修改一个序列。供具有 DBA 角色（三权分立） 或该序列的拥有者或具有 ALTER ANY SEQUENCE 权限的用户才修改序列。

使用说明

1. 关于步长的修改，分两种情况：

   a） 如果在修改前没有用 NEXTVAL 访问序列，创建完序列后直接修改序列步长值，则序列的当前值为起始值加上新步长值与旧步长值的差。此时序列的当前值可能超出最大值与最小值的限定范围，若序列当前值小于序列最小值，则将序列当前值作为新的序列最小值；若序列当前值大于序列最大值，则将序列当前值作为新的序列最大值。针对当前值超出原始限定范围的序列，导出时将报错；

   b） 如果在修改前用 NEXTVAL 访问了序列，然后修改序列步长值，则再次访问序列的当前值为序列的上一次的值加上新步长值。

2. 缺省序列选项：如果在修改序列语句中没有指出某选项则缺省是修改前的选项值。不允许未指定任何选项、禁止重复或冲突的选项说明；

3. 序列的起始值不能修改；

4. 修改序列的最小值不能大于起始值、最大值不能小于起始值；

5. 修改序列的步长的绝对值必须小于MAXVALUE与MINVALUE的差；

6. 修改序列的当前值时，指定的当前值不能大于最大值，不能小于最小值；

7. 修改序列的当前值后，需要使用NEXTVAL获取修改后的序列当前值；

8. 关于序列重命名选项：

   a） 重命名只修改序列的名称，不改变序列所属模式；

   b） 重命名选项不能与其他序列修改选项一起使用，否则进行语法报错。

举例说明

例1 创建完序列后直接修改序列的步长。

```
CREATE SEQUENCE SEQ1  INCREMENT BY 1000  START WITH 5 MAXVALUE 1000000 MINVALUE 1
CACHE 10;
ALTER SEQUENCE SEQ1 INCREMENT BY 1 ;
```

```
SELECT SEQ1.NEXTVAL FROM DUAL;
```

查询结果为：-994

由于-994小于序列最小值1，因此将-994作为新的序列最小值，序列取值范围变成-994~1000000。但是该序列不支持导出，导出时将报错。

例2 创建序列后使用NEXTVAL访问了序列，然后修改步长。

```
CREATE SEQUENCE SEQ2    INCREMENT BY 1000   START WITH 5 NOMAXVALUE NOMINVALUE
NOCACHE ;
SELECT SEQ2.NEXTVAL FROM DUAL;
ALTER SEQUENCE SEQ2 INCREMENT BY 1 ;
SELECT SEQ2.NEXTVAL FROM DUAL;
```

查询结果为：6

例3 修改序列的最小值。

```
CREATE SEQUENCE SEQ3    INCREMENT BY 1   START WITH 100      MINVALUE   3 ;
ALTER SEQUENCE SEQ3    MINVALUE   2;
```

例4 修改序列的当前值。

```
CREATE SEQUENCE SEQ4    INCREMENT BY 1   START WITH 100      MINVALUE   3 ;
ALTER SEQUENCE SEQ4 CURRENT VALUE 300;
SELECT SEQ4.NEXTVAL FROM DUAL;
```

查询结果为：300

例5 重命名序列SEQ5为SEQ6。

```
CREATE SEQUENCE SEQ5    INCREMENT BY 1   START WITH 100      MINVALUE   3 ;
ALTER SEQUENCE SEQ5 RENAME TO SEQ6;
SELECT SEQ6.NEXTVAL FROM DUAL;
```

查询结果为：100

3.10.3 序列删除语句

DM 系统允许用户在建立序列后还可随时删除序列。

语法格式

```
DROP SEQUENCE [IF EXISTS] [ <模式名>.]<序列名>;
```

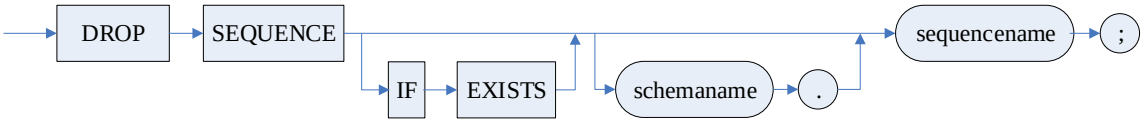
参数

1. <模式名> 指明被删除序列所属的模式，缺省为当前模式;

2. <序列名> 指明被删除序列的名称。

图例

序列删除语句



语句功能

供具有 DBA 角色（三权分立）的用户或该序列的拥有者或具有 DROP ANY SEQUENCE 权限的用户从数据库中删除序列生成器。

使用说明

1. 删除不存在的序列会报错。若指定 IF EXISTS 关键字，删除不存在的序列，不会报错;

2. 一种重新启动序列生成器的方法就是删除它然后再重新创建，例如有一序列生成器当前值为 150，而且用户想要从值 27 开始重新启动此序列生成器，他可以:

   1) 删除此序列生成器;

   2) 重新以相同的名字创建序列生成器，START WITH 选项值为 27。

举例说明

例 用户 SYSDBA 需要删除序列 SEQ_QUANTITY，可以用下面的语句:

```
DROP SEQUENCE SEQ_QUANTITY;
```

3.11 管理 SQL 域

为了支持 SQL 标准中的域对象定义与使用，DM 支持 DOMAIN 的创建、删除以及授权 DDL 语句，并支持在表定义中使用 DOMAIN。

域（DOMAIN）是一个可允许值的集合。域在模式中定义，并由<域名>标识。域是用来约束由各种操作存储于基表中某列的有效值集。域定义说明一种数据类型，它也能进一步说明约束域的有效值的<域约束>，还可说明一个<缺省子句>，该子句规定没有显式指定值时所要用的值或列的缺省值。

3.11.1 创建 DOMAIN

CREATE DOMAIN 创建一个新的数据域。定义域的用户成为其所有者。DOMAIN 为模式类型对象，其名称在模式内唯一。

语法格式

```
CREATE DOMAIN [IF NOT EXISTS] [<模式名>.] <domain_name> [AS] <数据类型>
[<default_clause> | <domain constraint>];
<domain constraint>::=[<constraint name definition>] <check constraint definition>
<constraint name definition>::= CONSTRAINT <约束名>
<check constraint definition>::= CHECK (<expression>)
```

参数

1. [<模式名>.] <domain_name> 为要创建的域名字，可以有模式前缀。如果在 CREATE SCHEMA 语句中定义了 DOMAIN，则<domain_name>的模式前缀（如果有）必须与创建的模式名一致。

2. <数据类型> 域的数据类型。仅支持定义标准的 SQL 数据类型。

3. <default_clause> DEFAULT 子句为域数据类型的字段声明一个缺省值。 该值是

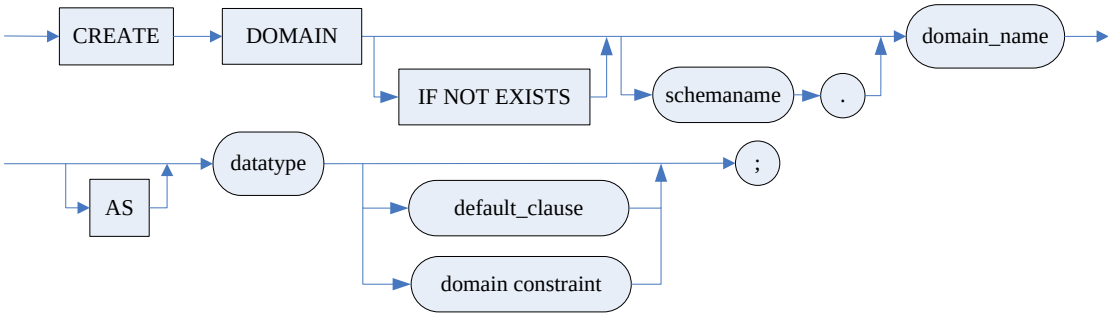
任何不含变量的表达式(但不允许子查询)。缺省表达式的数据类型必需匹配域的数据类型。如果没有声明缺省值， 那么缺省值就是空值。缺省表达式将用在任何为该字段声明数值的插入操作。 如果为特定的字段声明了缺省值，那么它覆盖任何和该域相关联的缺省值。

4. <constraint name definition> 一个约束的可选名称。如果没有名称，系统生成一个名字。

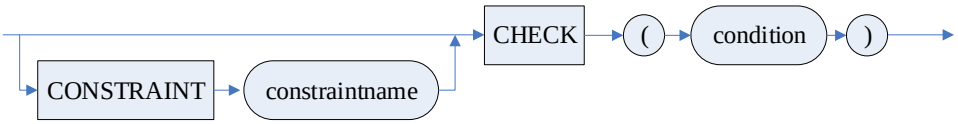
5. <check constraint definition> CHECK 子句声明完整性约束或者是测试，域的数值必须满足这些要求。 每个约束必须是一个生成一个布尔结果的表达式。它应该使用名字 VALUE 来引用被测试的数值。CHECK 表达式不能包含子查询，也不能引用除 VALUE 之外的变量。

图例

创建 DOMAIN



<domain constraint>



语句功能

供具有 DBA 角色(三权分立)、DB_OBJECT_ADMIN 角色(四权分立)或 具有 CREATE DOMAIN 或 CREATE ANY DOMAIN 系统权限的用户创建 DOMAIN。

举例说明

```
CREATE DOMAIN DA INT CHECK (VALUE < 100);
```

3.11.2 使用 DOMAIN

在表定义语句中，支持为表列声明使用域。如果列声明的类型定义使用域引用，则此列定义直接继承域中的数据类型、缺省值以及 CHECK 约束。如果列定义使用域，然后又自己定义了缺省值，则最终使用自己定义的缺省值。

用户可以使用自己的域。如果要使用其它用户的域，则必须被授予了该域的 USAGE 权限。DBA 角色默认拥有此权限。

例 在 T 表中使用第 1 节中创建的域 DA。

```
CREATE TABLE T(ID DA);
```

列定义虽然使用了域，但其 SYSCOLUMNS 系统表中类型相关字段记录域定义的数据类型，也就是说，从 SYSCOLUMNS 系统表中不会表现出对域的引用。

使用说明

使用某个域的用户必须具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或该域的 USAGE DOMAIN 或 USAGE ANY DOMAIN 权限。

3.11.3 删除 DOMAIN

语法格式

```
DROP DOMAIN [IF EXISTS] [<模式名>.]<domain_name> [<drop behavior>];

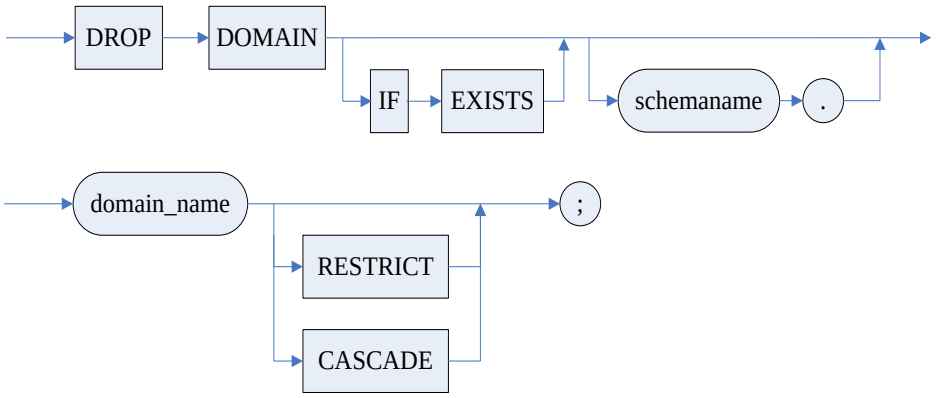
<drop behavior> ::= RESTRICT | CASCADE
```

参数

RESTRICT 表示仅当 DOMAIN 未被表列使用时才可以被删除；CASCADE 表示级联删除。

图例

删除 DOMAIN



语句功能

删除一个用户定义的域。用户可以删除自己拥有的域，具有 DBA 角色（三权分立）或 DROP ANY DOMAIN 系统权限的用户则可以删除任意模式下的域。

使用说明

删除不存在的 DOMAIN 会报错。若指定 IF EXISTS 关键字，删除不存在的 DOMAIN，不会报错。

举例说明

```
DROP DOMAIN DA CASCADE;
```

3.12 管理上下文

CONTEXT 上下文提供了一组设置以及访问服务器运行时应用数据的接口，通过这些接口可以有效控制应用数据的修改和访问。

CONTEXT 上下文有一个库级唯一的标识：名字空间（NAMESPACE），该名字空间保存了（NAME-VALUE）格式的数据，该数据通过 DBMS_SESSION 包中的过程设置其值，并通过 SYS_CONTEXT 系统函数访问其值。

应用上下文可以访问当前会话的一些属性信息，例如当前会话的 ID、模式名、用户名等。在实际应用过程中，将这些信息添加到查询的过滤条件中，起到允许或者禁止某些用户访问这些应用数据的目的。

3.12.1 创建上下文

创建上下文。

语法格式

```
CREATE [OR REPLACE] CONTEXT [IF NOT EXISTS] <namespace> USING [<模式名>.]
<packagename>;
```

参数

1. OR REPLACE 使用一个不同的 package 重新定义 namespace;

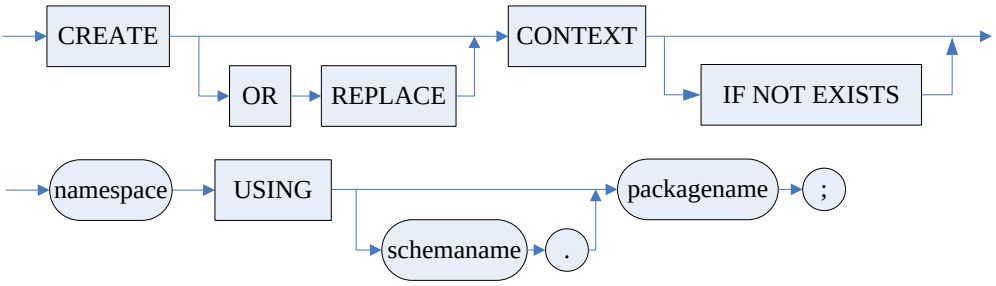
2. <namespace> 上下文的名字空间，保存到系统表 SYSOBJECTS 中，其模式为 SYS;

3. <模式名> 上下文关联的 package 的模式，默认为当前模式;

4. <packagename> 上下文关联的包，创建上下文不会影响已有的 package;

图例

创建上下文



语句功能

创建一个上下文 namespace，namespace 通过关联的 DMSQL 程序包来管理上下文的属性和值。属性和值就是用户的会话信息。

具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE ANY CONTEXT 权限的用户，才可以创建上下文。

使用说明

1. 通过 DBMS_SESSION 包的 SET_CONTEXT 过程设置上下文的属性和值，具体可

参考《DM8 系统包使用手册》第 17 章;

2. 通过系统函数 SYS_CONTEXT 访问 CONTEXT namespace 的属性值。其语法如下:

```
SYS_CONTEXT('namespace', 'parameter' [, length ])
```

使用说明如下:

1) namespace: 已创建的 namespace,不区分大小写,如果不存在,则报错;

2) parameter: namespace 中的属性,不区分大小写,如果不存在,则返回 NULL; 最大长度 30bytes;

3) 函数返回值类型为 varchar(256),如果返回值超过 256 字节,则需要设置 length。length 为 INT 类型,最大值为 4000,即返回字符串最大长度为 4000 字节;如果指定的 length 值非法,则忽略该值,而使用默认值 256。

3. 会话建立之后,只能通过关联的 package 来修改 namespace 内的属性值。namespace 中的属性值只允许会话级访问,即只有设置其属性值的会话才可以访问该值,其他会话访问该属性值都为空。namespace 保存的属性值为系统运行时的动态数据,服务器重启后该数据会丢失;

4. 成功创建上下文后,其信息保存在系统表 SYSOBJECTS 中,详见下表:

表 3.13.1 CONTEXT 与系统表 SYSOBJECTS 字段映射关系

字段	名称	类型	说明
name	NAMESPACE	VARCHAR2(30)	CONTEXT 名字空间
SCHID	SCHID	INT	CONTEXT 所在模式 ID,即 SYS 模式 ID
info5	SCHEMA	VARCHAR2(30)	关联包所在模式名
info6	PACKAGE	VARCHAR2(30)	关联包名

TYPE$	TYPE	VARCHAR2(10)	SCHOBJ
SUBTYPE$	SUBTYPE$	VARCHAR2(10)	CONTEXT

5. USERENV 为系统默认的上下文名字空间，保存了用户的上下文信息，属性如下：

表 3.13.2 USERENV 属性

属性	说明
CURRENT_SCHEMA	返回当前模式名
CURRENT_SCHEMAID	返回当前模式 ID
CURRENT_USER	返回当前的用户名
CURRENT_USERID	返回当前的用户 ID
DB_NAME	返回数据名
HOST	返回客户端的主库名
INSTANCE	返回实例 ID，单节点默认 1，MPP 或 DSC 环境下为实例序号加 1
INSTANCE_NAME	返回实例名
IP_ADDRESS	返回客户端的 IP 地址
ISDBA	如果当前会话用户拥有 DBA 权限，则返回 TRUE，否则，返回 FALSE
LANG	语言包简写，中文返回“CN”，英文返回“EN”
LANGUAGE	语言包，返回库的编码方式
NETWORK_PROTOCOL	通信协议
SERVER_HOST	实例运行的主机名
SESSION_USER	会话的用户名
SESSION_USERID	会话的用户 ID
SID	当前会话的 ID

6. 动态视图 V$CONTEXT 显示当前会话所有上下文的名字空间、属性和值。

3.12.2 删除上下文

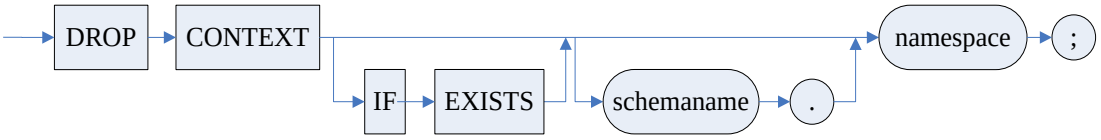
删除上下文。

语法格式

```
DROP CONTEXT [IF EXISTS] [<模式名>.] <namespace> ;
```

图例

删除上下文



语句功能

从数据库中删除上下文的 namespace，即从系统表 SYSOBJECTS 中删除。如果 namespace 不存在，则报错。如果该 namespace 成功删除，但之前已添加了属性和值，那么会话仍可以访问该属性值。即其删除的是字典对象，其实例对象不会删除。

具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 DROP ANY CONTEXT 权限的用户，才可以删除上下文。

使用说明

删除不存在的上下文会报错。若指定IF EXISTS关键字，删除不存在的上下文，不会报错。

举例说明

如何使用上下文。

第一步 创建 package:

```
CREATE or replace package test_pk as
procedure set_context(ts_name varchar, key varchar, value varchar);
procedure set_user_context(ts_name varchar, key varchar, value varchar, username varchar,
client_id varchar);
procedure get_context(ts_name varchar, key varchar);
```

```
procedure clear_context(ts_name varchar, key varchar, value varchar);
end test_pk;
/
CREATE or replace package body test_pk as
    procedure set_context(ts_name varchar, key varchar, value varchar) as
    begin
        dbms_session.set_context(ts_name, key, value);
    end;

procedure set_user_context(ts_name varchar, key varchar, value varchar, username varchar,
client_id varchar) as
begin
 dbms_session.set_context(ts_name, key, value, username, client_id);
end;

procedure get_context(ts_name varchar, key varchar) as
begin
    dbms_output.put_line('==' || sys_context(ts_name, key) || '--');
end;

procedure clear_context(ts_name varchar, key varchar, value varchar) as
begin
    dbms_session.clear_context(ts_name, key, value);
end;
```

```
end test_pk;
/
```

第二步 创建 context

```
create or replace context c_user01 using test_pk;
```

第三步 设置 namespace 的属性

```
call test_pk.set_context('c_user01', 'u_k2', 'u_v2');
```

第四步 查询该属性值

```
call test_pk.get_context('c_user01', 'u_k2');
```

打印值如下：

```
==u_v2--
```

3.13 管理目录

3.13.1 创建目录

创建一个目录对象。目录是操作系统文件在数据库中的一个映射。

语法格式

```
CREATE [OR REPLACE] DIRECTORY [IF NOT EXISTS] <目录名> AS '<dir_path>';
```

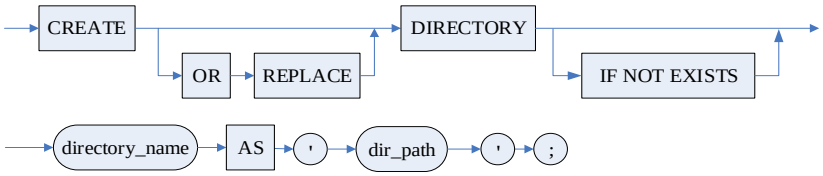
参数

1. <目录名> 创建的目录的名称；

2. <dir_path> 指明要创建的目录的路径，支持绝对路径和相对路径，使用相对路径时，与执行码路径同级。该路径需要用户保证已在操作系统中创建完成，否则使用创建的目录对象时会报错。

图例

创建目录



语句功能

供具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 CREATE ANY DIRECTORY 权限的用户创建一个目录对象。

举例说明

例 先在计算机上新建文件夹'E:\test\path'，并在数据库中创建名为'TESTDIR'的目录对象，再使用导出工具将文件导出到该目录。

```
//使用 Disql 创建目录
CREATE OR REPLACE DIRECTORY "TESTDIR" AS 'E:\test\path';
//使用 dexpdp 工具导出文件
dexpdp.exe USERID=SYSDBA/DMsys_123 FILE=dexpDP.dmp LOG=dexpDP.log FULL=Y
DIRECTORY=TESTDIR
```

可以观察到dexpDP.dmp文件会出现在'E:\test\path'文件夹下。

同样的创建过程，只是将<dir_path>替换成'../doc'，则dexpDP.dmp文件会出现在与执行码平级的doc文件夹下。

3.13.2 删除目录

删除一个目录对象。

语法格式

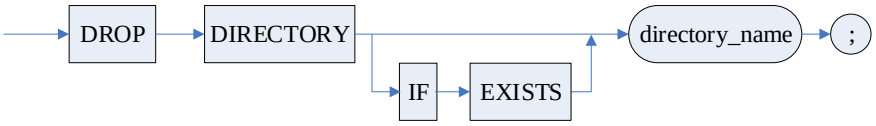
```
DROP DIRECTORY [IF EXISTS] <目录名>;
```

参数

1. <目录名> 要删除的目录的名称

图例

删除目录



语句功能

具有 DBA 角色（三权分立）、DB_OBJECT_ADMIN 角色（四权分立）或具有 DROP ANY DIRECTORY 权限的用户删除一个目录对象。

使用说明

删除不存在的目录会报错。若指定IF EXISTS关键字，删除不存在的目录，不会报错。

3.14 设置当前会话

3.14.1 时区信息

设置当前会话时区信息

语法格式

```
SET TIME ZONE <时区>;

<时区>::= LOCAL |

          '[+|-]{<整数>|<INTERVAL HOUR TO MINUTE 类型数据>}' |

          INTERVAL '[+|-]<整数>' <间隔类型>

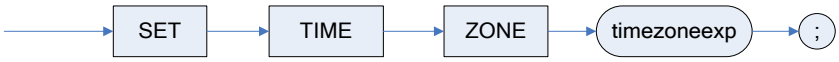
<INTERVAL HOUR TO MINUTE 类型数据>::=形式为<整数>:<整数>的数据，描述一个若干小时若干分钟的间隔
```

参数

<时区> 指明要设置的时区信息，支持的时区范围为'-12:59'与'+14:00'之间。

图例

时区信息



语句功能

设置当前会话时区信息。

使用说明

仅当前会话有效。

举例说明

例 1 设置当前会话时区为'+9:00'。

```
SET TIME ZONE   '+9:00';
```

例 2 设置当前会话时区为服务器所在地时区。

```
SET TIME ZONE LOCAL;
```

3.14.2 日期串语言

设置当前会话的日期串语言。

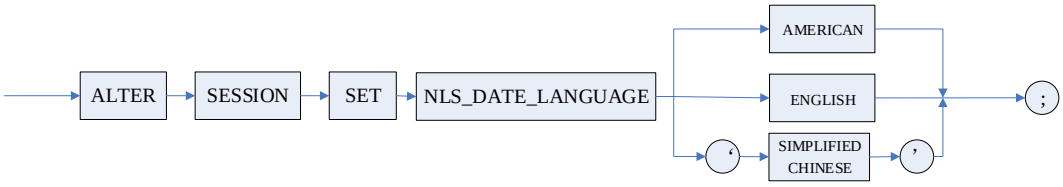
语法格式

```
ALTER SESSION SET NLS_DATE_LANGUAGE=<语言>;

<语言>:=[AMERICAN]|[ENGLISH]|['SIMPLIFIED CHINESE']
```

图例

日期串语言



语句功能

设置当前会话日期串语言。

使用说明

仅当前会话有效。设置语言为 ENGLISH 和 AMERICAN 时，添加引号也不报错。

举例说明

例 设置当前会话日期串为 ENGLISH。

```
ALTER SESSION SET NLS_DATE_LANGUAGE=ENGLISH;
```

3.14.3 日期串格式

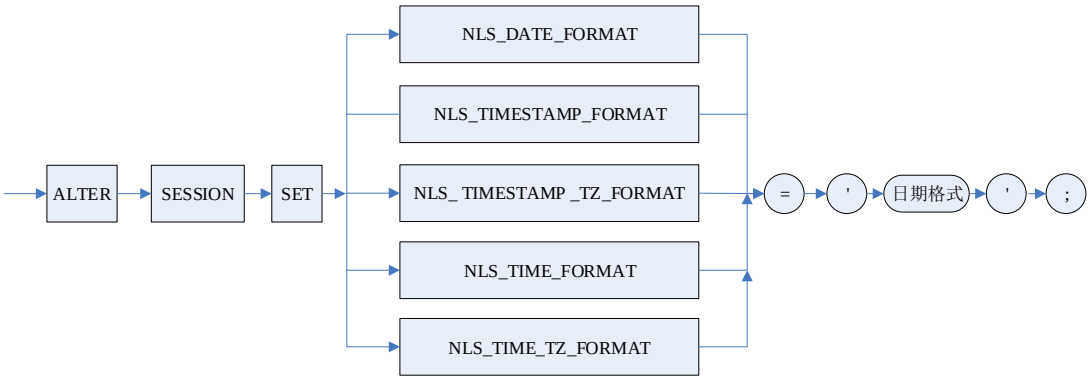
设置当前会话的日期串格式。

语法格式

```
ALTER SESSION SET <NLS_DATE_FORMAT | NLS_TIMESTAMP_FORMAT | NLS_TIMESTAMP
_TZ_FORMAT | NLS_TIME_FORMAT | NLS_TIME_TZ_FORMAT> = <日期格式>;
```

图例

日期串格式



语句功能

设置当前会话的日期串格式。

参数

1. NLS_DATE_FORMAT / NLS_TIMESTAMP_FORMAT / NLS_TIMESTAMP_TZ_FORMAT / NLS_TIME_FORMAT / NLS_TIME_TZ_FORMAT 分别用于指定 DATE / TIMESTAMP / TIMESTAMP_TZ / TIME / TIME_TZ 类型的格式;

2. <日期格式> 日期格式具体用法请参考 8.3 小节中的日期格式。

使用说明

1. 仅当前会话有效;

2. 当 COMPATIBLE_MODE=2 且 ORA_DATE_FMT=0 时，修改 NLS_DATE_FORMAT 时实际修改的是 NLS_TIMESTAMP_FORMAT。

举例说明

例 设置当前会话日期格式为不同的格式。

```
ALTER SESSION SET NLS_DATE_FORMAT='YYYY-MM-DD';

ALTER SESSION SET NLS_TIMESTAMP_FORMAT ='YYYYMMDD HH:MI:SS';

ALTER SESSION SET NLS_TIMESTAMP_TZ_FORMAT ='YYYYMMDD HH:MI:SS FF TZH:TZM';

ALTER SESSION SET NLS_TIME_FORMAT ='HH:MI:SS';

ALTER SESSION SET NLS_TIME_TZ_FORMAT='HH:MI:SS FF TZH:TZM' ;
```

3.14.4 自然语言排序方式

设置当前会话的自然语言排序方式。

语法格式

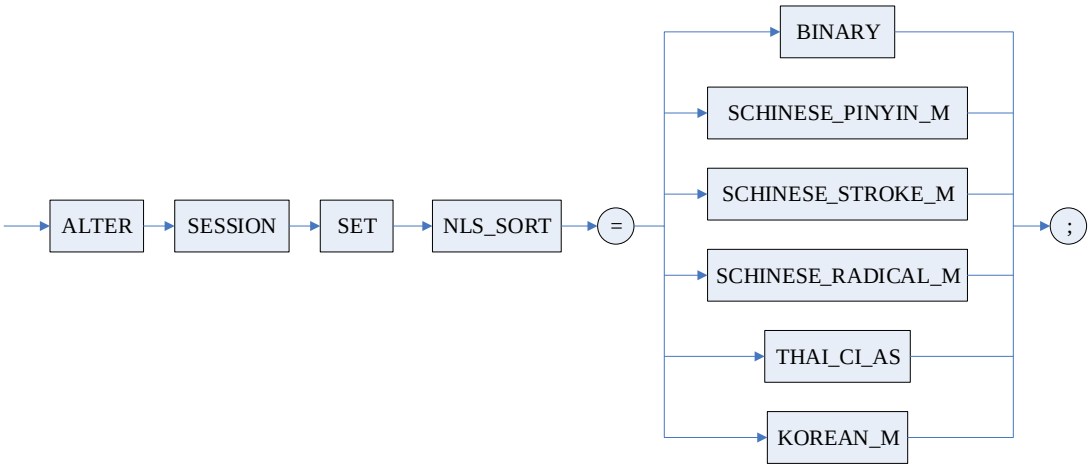
```
ALTER SESSION SET NLS_SORT=<排序方式>;

<排序方式>:= BINARY | SCHINESE_PINYIN_M | SCHINESE_STROKE_M |

                SCHINESE_RADICAL_M | THAI_CI_AS | KOREAN_M
```

图例

自然语言排序方式



语句功能

设置当前会话的自然语言排序方式。

参数

<排序方式> BINARY 表示按默认字符集二进制编码排序；SCHINESE_PINYIN_M 表示按中文拼音排序；SCHINESE_STROKE_M 表示按中文笔画排序；SCHINESE_RADICAL_M 表示按中文部首排序；THAI_CI_AS 表示按泰文排序；KOREAN_M 表示按韩文排序。

使用说明

1. 仅当前会话有效。

2. 仅字符集为 UTF-8 的数据库支持自然语言按泰文排序。

举例说明

例 设置当前会话的自然语言按照中文拼音排序。

```
ALTER SESSION SET NLS_SORT=SCHINESE_PINYIN_M;
```

3.14.5 大小写敏感

设置当前会话的大小写敏感属性。

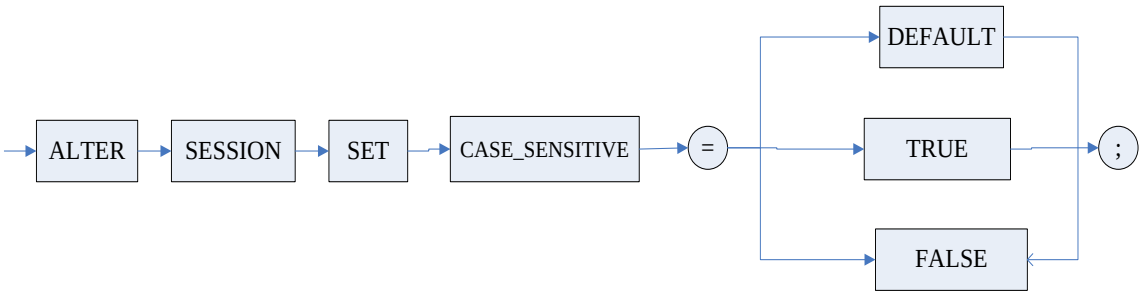
语法格式

```
ALTER SESSION SET CASE_SENSITIVE=<属性>;
```

```
<属性>:= DEFAULT | TRUE | FALSE
```

图例

大小写敏感属性



语句功能

设置当前会话的大小写敏感属性。

参数

<属性> 设置为 DEFAULT 时，代表会话与当前数据库的大小写敏感属性保持一致；设置为 TRUE 时，代表在大小写不敏感的库上可以使得会话中的字符类型数据以大小写敏感的方式进行比较，而在大小写敏感的库上则维持原始方式比较；设置为 FALSE 时，代表在大小写敏感的库上可以使得会话中的字符类型数据以大小写不敏感的方式进行比较，而在大小写不敏感的库上则维持原始方式比较。

使用说明

1. 仅当前会话有效。

2. 仅对字符类型生效，其他数据类型忽略该会话属性。

3. 创建索引时忽略该会话属性。

4. 确定性函数参数忽略该会话属性。

5. CONTAINS 表达式忽略该会话属性。

6. ALL/SOME/ANY 子查询忽略该会话属性。

7. 层次查询表达式忽略该会话属性。

8. 集函数参数包括 WITHIN GROUP 中排序表达式时忽略该会话属性。

9. 分析函数参数包括 OVER 中排序表达式、分组表达式时忽略该会话属性。

10. 该会话属性优先级低于 BINARY 前缀，即存在 BINARY 前缀时，即使会话属性被设置为 FALSE，仍会按照大小写敏感进行比较。关于 BINARY 前缀的具体信息详见 4.16 BINARY 前缀。

11. 使用该语句设置会话的大小写敏感属性后，可以通过使用系统函数 SESS_CASE_SENSITIVE 获取当前会话是否大小写敏感。

举例说明

例 设置当前会话为大小写敏感的方式。

```
ALTER SESSION SET CASE_SENSITIVE = TRUE;
```

3.14.6 本地并行开关

设置当前会话的本地并行开关。

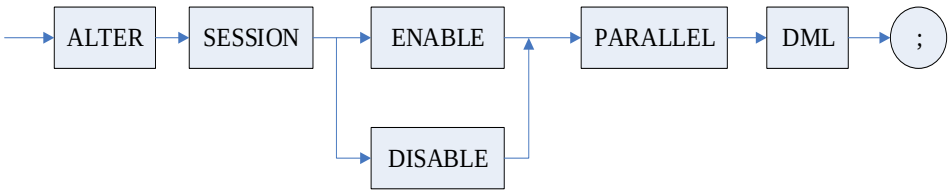
语法格式

```
ALTER SESSION <开关> PARALLEL DML;

<开关>::= ENABLE | DISABLE
```

图例

本地并行开关



语句功能

设置当前会话是否开启本地并行查询。

参数

<开关> 设置为 ENABLE 时，代表当前会话设置 PARALLEL_POLICY=1，开启本地并行查询；设置为 DISABLE 时，代表当前会话设置 PARALLEL_POLICY=0，关闭本地并行查询。

使用说明

仅当前会话有效。

举例说明

例 设置当前会话开启本地并行查询。

```
ALTER SESSION ENABLE PARALLEL DML;
```

3.15 注释语句

可以通过注释语句来创建或修改表、视图或它们的列的注释信息。表和视图上的注释信息可以通过查询字典表 SYSTABLECOMMENTS 进行查看，列的注释信息可以通过查询字典表 SYSCOLUMNCOMMENTS 进行查看。

语法格式

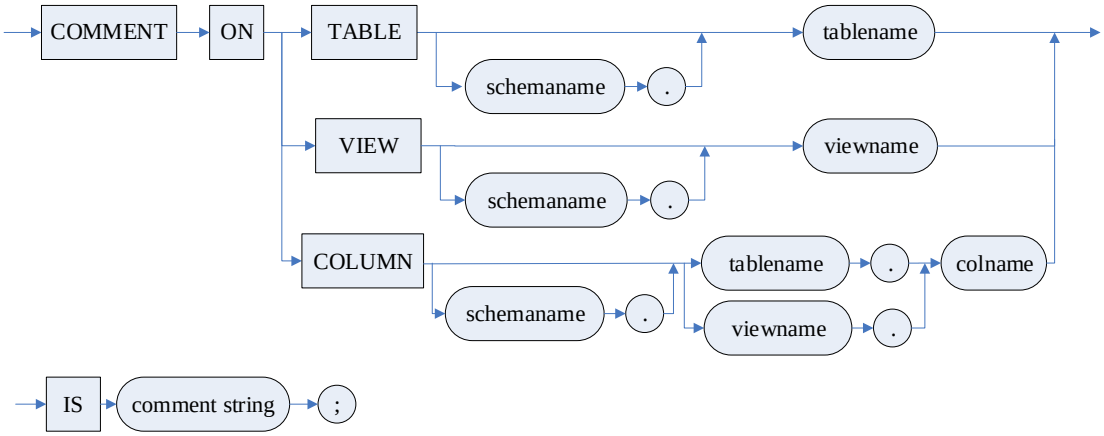
```
COMMENT ON        <对象名称> IS <注释字符串>
<对象名称> ::=
    TABLE <表名定义> |
    VIEW <视图名定义> |
    COLUMN <列名定义>
<表名定义> ::= [<模式名>.]<表名>
<视图名定义> ::= [<模式名>.]<视图名>
<列名定义> ::=
    [<模式名>.]<表名>.<列名>|
    [<模式名>.]<视图名>.<列名>
```

参数

<注释字符串> 指明要设置的注释信息，注释字符串最大长度为4000。

图例

注释语句



语句功能

供具有DBA角色（三权分立）、DB_OBJECT_ADMIN角色（四权分立），或具有ALL、ALL PRIVILEGE、COMMENT ANY TABLE权限的用户为指定模式下的表、视图或列创建注释信息；供具有ALTER权限的用户为指定模式下的表、表列创建注释信息；供用户为自己所拥有模式中的表、视图和列对象创建注释信息。

使用说明

在已有注释的对象上再次执行此语句将直接覆盖之前的注释。

举例说明

例 1 为表 PERSON 创建注释信息。

```
COMMENT ON TABLE PERSON IS 'PERSON IS A SIMPLE TABLE';
```

例 2 为表 PERSON 的列 NAME 创建注释信息。

```
COMMENT ON COLUMN PERSON.NAME IS 'SYSDBA.PERSON.NAME';
```

3.16 设置 INI 参数

INI 参数分为手动、静态和动态三种类型，分别对应 V$PARAMETER 视图中 TYPE 列的 READ ONLY、IN FILE、SYS/SESSION。服务器运行过程中，手动（READ ONLY）参数不能被修改，静态和动态参数可以修改。

静态（IN FILE）参数只能通过修改 DM.INI 文件进行修改，修改后重启服务器才能生效，为系统级参数，生效后会影响所有的会话。

动态（SYS 和 SESSION）参数可在 DM.INI 文件和内存同时修改，修改后即时生效。其中，SYS 为系统级参数，修改后会影响所有的会话；SESSION 为会话级参数，服务器运行过程中被修改时，之前创建的会话不受影响，只有新创建的会话使用新的参数值。

3.16.1 设置对全局有效

用户可以通过ALTER SYSTEM语法修改静态或动态（系统级、会话级）参数值，使修改之后的参数值能够在全局范围内起作用。对于静态参数，只有指定SPFILE情况下，才能修改。

语法格式

```
ALTER SYSTEM SET '<参数名称>' =<参数值> [DEFERRED] [MEMORY|BOTH|SPFILE];
```

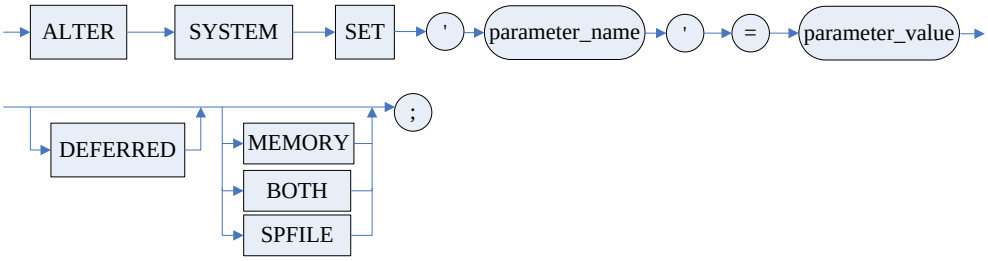
参数

1. <参数名称> 指静态、动态（系统级、会话级）INI参数名字；
2. <参数值> 指设置该INI参数的值；
3. [DEFERRED] 只适用于动态会话级参数，若指定DEFERRED，则参数值延迟生效，对当前会话不生效，只对新创建的会话生效；若不指定DEFERRED，则参数值立即生效，对当前会话和新创建的会话都生效。针对动态系统级参数，无论是否指定DEFERRED，参数值均对所有会话生效，包括所有已经创建的会话以及新创建的会话；针对静态参数，指定DEFERRED将报错；

4. [MEMORY|BOTH|SPFILE] 设置INI参数修改的位置。其中，MEMORY只对内存中的INI值做修改；SPFILE则只对INI文件中的INI值做修改；BOTH则内存和INI文件都做修改。默认情况下，为MEMORY。对于静态参数，只能指定SPFILE。

图例

设置参数值



语句功能

设置系统级的 INI 参数值。

举例说明

例 1 设置当前系统动态、会话级参数 SORT_BUF_SIZE 参数值为 200，要求延迟生效，对当前的 session 不生效，对后面创建的会话才生效。并且只修改内存。

```
ALTER SYSTEM SET 'SORT_BUF_SIZE' =200  DEFERRED  MEMORY;
```

例 2 设置静态参数 BDTA_SIZE 参数值为 1200。

```
ALTER SYSTEM SET 'BDTA_SIZE' =1200  spfile;
```

3.16.2 设置仅对当前会话有效

用户可以通过 ALTER SESSION 语法修改动态会话级参数(即 TYPE 为 SESSION 的参数)，使修改之后的 INI 参数值只对当前会话起作用，不会影响其他会话或系统的 INI 参数值。

语法格式

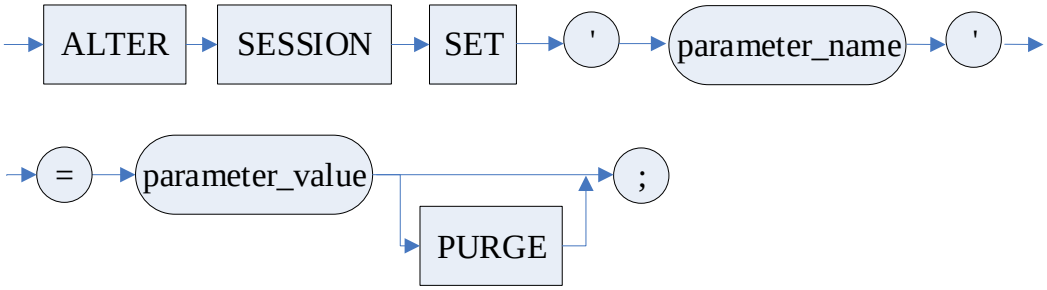
```
ALTER SESSION SET '<参数名称>' =<参数值> [PURGE];
```

参数

1. <参数名称> 指动态会话级INI参数名字;
2. <参数值> 指设置该INI参数的相应值;
3. [PURGE] 指是否清理执行计划。

图例

设置仅对当前会话起作用



语句功能

设置动态、会话级的 INI 参数值。

使用说明

设置后的值只对当前会话有效。当包含 PURGE 选项时会清除服务器保存的所有执行计划。

举例说明

例 设置当前会话的 HAGR_HASH_SIZE 参数值为 2000000。

```
ALTER SESSION SET 'HAGR_HASH_SIZE' =2000000;
```

3.17 修改系统语句

用户可以通过 ALTER SYSTEM 语句对系统进行修改，如设置系统级别参数（参见 3.17 设置 INI 参数），也可以使用系统修改语句切换归档文件和归档当前所有的 REDO 日志。

语法格式

```
ALTER SYSTEM <修改系统语句>;

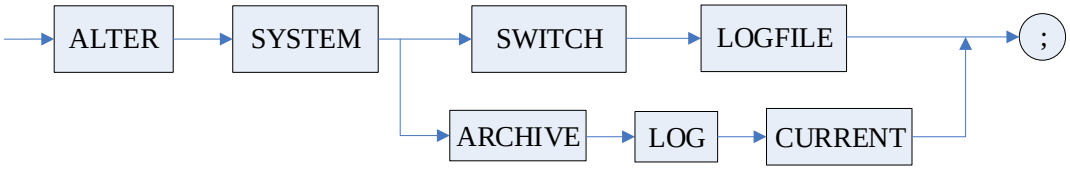
<修改系统语句>::=
```

```
SWITCH LOGFILE |

ARCHIVE LOG CURRENT
```

图例

修改系统语句



使用说明

ARCHIVE LOG CURRENT 和 SWITCH LOGFILE 功能一样，都是把新生成的，还未归档的联机日志进行归档。

3.18 设置列、索引生成统计信息

设置列、索引生成统计信息。

语法格式

```
STAT [<HINT 子句>] <统计信息采样率百分比> [SIZE <直方图桶数>] ON <统计对象> [GLOBAL]

<HINT 子句> ::=/*+ PARALLEL(<num>)*/

<统计对象>::=

[<模式名>.] <表名> (<列名>{,<列名>}) |

INDEX [<模式名>.]<索引名>
```

参数

1. <HINT子句> 用于使用优化器提示，目前仅支持指定收集统计信息的并行度PARALLEL。<HINT子句>可以放在STAT语句的任意位置，例如可以放在STAT语句的句首或句尾等;

2. <统计信息采样率百分比> 指定统计信息采样率的百分比。必须为0~100范围内的

浮点数，精确到小数点后6位；

3. <直方图桶数> 指定统计信息的直方图桶数，单列取值范围为0或1~10000范围内的整数，其中0表示不限制。不指定时系统根据数据的实际情况动态确定。多列的取值范围是1~2500；

4. <模式名> 指定生成统计信息的列所在表的模式。缺省为当前会话的模式名；

5. <表名> 指定生成统计信息的列所在的表；

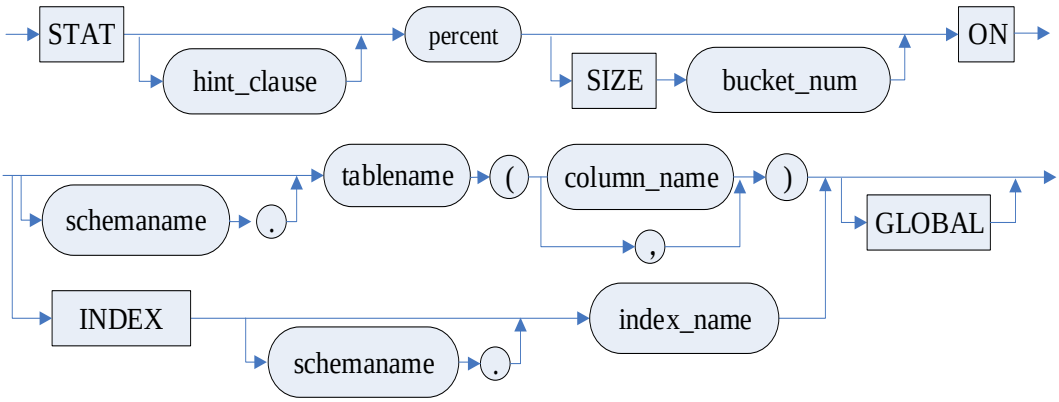
6. <列名> 指定生成统计信息的列。当前最大支持127列；

7. <索引名> 指定生成统计信息的索引；

8. GLOBAL 用于MPP或DPC环境下各节点数据收集后统一生成统计信息。

图例

设置列、索引生成统计信息



语句功能

为列或索引生成的统计信息。

使用说明

1.不支持所在表空间为OFFLINE的对象；

2.<表名> 不支持外部表、DBLINK远程表、动态视图表、记录类型数组所用的临时表；

3.<列名> 不支持 ROWID、ROWNUM等特殊列，不支持BLOB、BFILE、IMAGE、LONGVARBINARY、CLOB、TEXT、LONGVARCHAR、自定义类型列等列类型；

4.<索引名> 不支持位图索引、位图连接索引、虚索引、全文索引、数组索引、无效的索引;

5.该语句的调用将导致当前事务被提交;

6.GLOBAL 仅在MPP或DMDPC环境下使用GLOBAL登录时可用，否则报错;

7.多列统计信息只支持表，不支持索引;

8.DMDPC环境下不支持多列统计信息。

举例说明

例 1 对 SYSOBJECTS 表上 ID 列生成统计信息，采样率的百分比为 30%。

```
STAT 30 ON SYS.SYSOBJECTS (ID);
```

例 2 对 PURCHASING 模式下的索引 S1 生成统计信息，采样率为 50%。

```
STAT 50 ON INDEX PURCHASING.S1;
```

例 3 对 SYSOBJECTS 表上 PID 和 NAME 列生成统计信息，采样率的百分比为 30%，同时指定并行度为 12。

```
STAT /*+parallel(12)*/ 30 ON SYS.SYSOBJECTS (PID,NAME);

--<HINT 子句>可以放在 STAT 语句的任意位置，例如还可放在句首

/*+parallel(12)*/ STAT 30 ON SYS.SYSOBJECTS (PID,NAME);
```

3.19设置表生成统计信息

设置表生成统计信息。

语法格式

```
STAT [<HINT 子句>] ON [<模式名>.]<表名> [GLOBAL];

<HINT 子句> ::=/*+ PARALLEL(<num>)*/
```

参数

1. <HINT子句> 用于使用优化器提示，目前仅支持指定收集统计信息的并行度

PARALLEL。<HINT子句>可以放在STAT语句的任意位置，例如可以放在STAT语句的句首或句尾等；

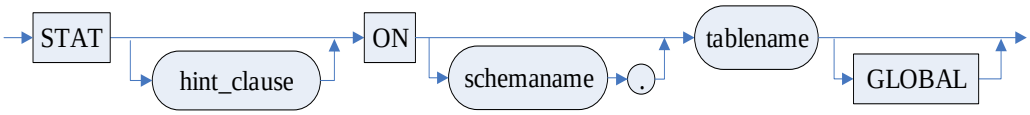
2. <模式名> 指定生成统计信息的表的模式。缺省为当前会话的模式名；

3. <表名> 指定生成统计信息的表；

4. GLOBAL 用于MPP或DPC环境下各节点数据收集后统一生成统计信息。

图例

设置表生成统计信息



语句功能

为表生成统计信息。

使用说明

1. 不支持为所在表空间为OFFLINE的表生成统计信息；

2. 不支持为外部表、DBLINK远程表、动态视图表、记录类型数组所用的临时表生成统计信息；

3. 该语句的调用将导致当前事务被提交；

4. GLOBAL 仅在MPP或DMDPC环境下使用GLOBAL登录时可用，否则报错；

5. 对分区表默认采样50个子表，可通过调整INI参数HP_STAT_SAMPLE_COUNT设置采样子表的个数。

举例说明

例 1 为 SYSOBJECTS 表生成统计信息。

```
STAT ON SYS.SYSOBJECTS;
```

例 2 为 SYSOBJECTS 表生成统计信息，同时指定并行度为 12。

```
STAT /*+parallel(12)*/ ON SYS.SYSOBJECTS;
```

```
--<HINT 子句>可以放在 STAT 语句的任意位置，例如还可放在句首

/*+parallel(12)*/ STAT ON SYS.SYSOBJECTS;
```

3.20 管理 PROFILE

为了兼容 ORACLE 的用户资源和密码限制管理方式，DM 支持 PROFILE 的创建、修改、删除以及授权 DDL 语句，并支持在用户定义中关联 PROFILE。

3.20.1 创建 PROFILE

语法格式

```
CREATE PROFILE [IF NOT EXISTS] <profile 名> [LIMIT <资源设置>];

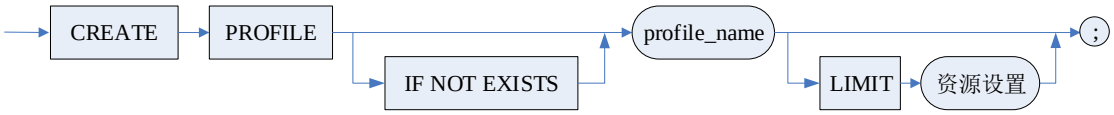
<资源设置>参考 3.2.1 用户定义语句中的<资源设置>
```

参数

1. <profile 名> 要创建的 profile 文件名字，最大长度 128 字节；
2. LIMIT <资源设置> 直接设置资源设置项。如未指定，则使用缺省值。

图例

创建 PROFILE



语句功能

供具有 DBA 角色或具有 CREATE PROFILE 系统权限的用户创建 PROFILE。

使用说明

创建 profile 文件，通过 LIMIT <资源设置>设置资源设置项。如果不指定资源设置项，则使用缺省值。

举例说明

```
CREATE PROFILE PF LIMIT SESSION_PER_USER 100 PASSWORD_REUSE_TIME 10;
```

3.20.2 修改 PROFILE

语法格式

```
ALTER PROFILE <profile 名> LIMIT <资源设置>

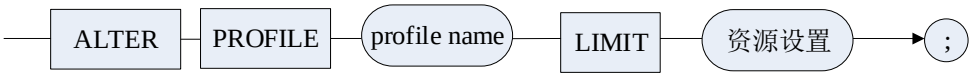
<资源设置>参考 3.2.1 用户定义语句中的<资源设置>
```

参数

同创建 PROFILE 的参数规定一样。

图例

修改 PROFILE



语句功能

供具有 DBA 角色或具有 ALTER PROFILE 系统权限的用户修改 PROFILE 的资源限制项。

举例说明

```
ALTER PROFILE DEFAULT LIMIT SESSION_PER_USER 100;
```

3.20.3 删除 PROFILE

语法格式

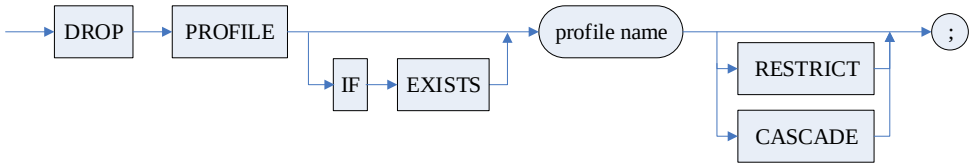
```
DROP PROFILE [IF EXISTS] <profile 名> [RESTRICT| CASCADE];
```

参数

RESTRICT 表示仅当 PROFILE 未关联任何用户时才可被删除；CASCADE 表示级联删除。

图例

删除 PROFILE



语句功能

供具有 DBA 角色或具有 DROP PROFILE 系统权限的用户删除 PROFILE。

使用说明

1. 删除不存在的PROFILE会报错。若指定IF EXISTS关键字，删除不存在的PROFILE，不会报错。

2. 不允许删除名为DEFAUTL的系统预设PROFILE。

举例说明

```
DROP PROFILE PF CASCADE;
```

第 4 章 数据查询语句

数据查询是数据库的核心操作，DM_SQL 语言提供了功能丰富的查询方式，满足实际应用需求。几乎所有的数据库操作均涉及到查询，因此熟练掌握查询语句的使用是数据库从业人员必须掌握的技能。

在 DM_SQL 语言中，有的定义语法中也包含查询语句，如视图定义语句、游标定义语句等。为了区别，我们将这类出现在其它定义语句中的查询语句称查询说明。

每种查询都有适用的场景，使用得当会大大提高查询效率。为方便用户的使用，本章对 DM_SQL 语言支持的查询方式进行讲解，测例中所用基表及各基表中预先装入的数据参见第 2 章 手册中的示例说明，各例的建表者均为用户 SYSDBA。

查询语句的语法如下：

```
<查询表达式>::=
    <simple_select>|
    <select_clause> <ORDER BY 子句> <LIMIT 限定条件> <FOR UPDATE 子句>|
    <select_clause> <ORDER BY 子句> [<FOR UPDATE 子句>] [<LIMIT 限定条件>] |
    <select_clause> <LIMIT 子句> <ORDER BY 子句> [<FOR UPDATE 子句>] |
    <select_clause> <FOR UPDATE 子句> [<LIMIT 限定条件>] |
    <select_clause> <LIMIT 限定条件>[<FOR UPDATE 子句>]
<simple_select> ::=
    <query_exp_with>|
    <select_clause> <集合运算符> [ALL | DISTINCT | UNIQUE] [CORRESPONDING [BY (<列名>
{,<列名>})]] <select_clause>
<select_clause>::=
```

```
    <simple_select>|
    (<查询表达式>)|
    (<select_clause>)
<集合运算符>::=UNION| EXCEPT | MINUS | INTERSECT
<ORDER BY 子句>::= ORDER [SIBLINGS] BY <order_by_list>
<order_by_list>::= <order_by_item >{,<order_by_item>}
<order_by_item>::=  <exp> [COLLATE <collation_name>] [ASC|DESC] [NULLS FIRST|LAST]
<exp>::=<列说明>|<无符号整数>|<值表达式>|<布尔表达式>
<FOR UPDATE 子句> ::=
    FOR READ ONLY|
    FOR UPDATE [OF <选择列表>] [NOWAIT | WAIT N | [N] SKIP LOCKED]
<LIMIT 限定条件>::= <LIMIT 子句>|<ROW_LIMIT 子句>
<LIMIT 子句>::= LIMIT <记录数> |
                LIMIT <偏移量>, <记录数> |
                LIMIT <记录数> OFFSET <偏移量> |
                OFFSET <偏移量> LIMIT <记录数>
<记录数>::= <整数>
<偏移量>::= <整数>
<ROW_LIMIT 子句>::= [OFFSET <offset> ROW[S] ] [<FETCH 说明>]
<FETCH 说明>::= FETCH <FIRST | NEXT> [<大小> | <大小> PERCENT] ROW[S] <ONLY | WITH TIES>
<query_exp_with>::= [<WITH 子句>] SELECT [<HINT 子句>] [ALL | DISTINCT | UNIQUE] [<TOP 子
句>] <选择列表>[<bulk_or_single_into_null>] <select_tail>
<选择列表> ::= [[<模式名>.]<基表名> | <视图名> .] * | <值表达式> [[AS] <列别名>]
              {,[[<模式名>.]<基表名> | <视图名>.] * | <值表达式> [[AS] <列别名>]}
```

<WITH 子句> ::= [<WITH FUNCTION 子句>] [<WITH CTE 子句>]　请参考第 4.4 节 WITH 子句

<HINT 子句> ::=/*+ hint{hint}*/

<TOP 子句>::=

TOP <n> |

<<n1>,<n2>>|

<n> PERCENT|

<n> WITH TIES|

<n> PERCENT WITH TIES

<n>::=整数（>=0）

<bulk_or_single_into_null>::=<bulk_or_single_into>　<变量名>{,<变量名>}

<bulk_or_single_into>::= <INTO>| <BULK COLLECT INTO>

<select_tail>::= <FROM 子句>[<WHERE 子句>][<层次查询子句>][<GROUP BY 子句>][<HAVING 子句>]

<FROM 子句>::= FROM <表引用>{,<表引用>}

<表引用>::=<普通表>|<连接表>

<普通表>::=<普通表 1>|<普通表 2>|<普通表 3>| ARRAY <数组> | TABLE(<collection_exp>)

<普通表 1>::=<对象名> [<SAMPLE 子句>][[[AS] <别名>] <PIVOT 子句>][[[AS] <别名>] <UNPIVOT 子句>] [<闪回查询>] [[AS] <别名>]

<普通表 2>::=(<查询表达式>)[[AS <别名>] <PIVOT 子句>][[AS <别名>] <UNPIVOT 子句>] [<闪回查询>][[AS] <表别名> [<新生列>]]

<普通表 3>::=[<模式名>.]<<基表名>|<视图名>>(<选择列>)[[AS <别名>] <PIVOT 子句>] [[AS <别名>] <UNPIVOT 子句>] [<闪回查询>] [[AS] <表别名> [<派生列表>]]

<对象名>::=<本地对象> | <索引> | <分区表>

<本地对象>::=[<模式名>.]<基表名|视图名>

<索引>::=[<模式名>.]<基表名> INDEX <索引名>

<分区表>::=

[<模式名>.]<基表名> PARTITION (<一级分区名>) |

[<模式名>.]<基表名> PARTITION FOR (<表达式>,{<表达式>})|

[<模式名>.]<基表名> SUBPARTITION (<子分区名>)|

[<模式名>.]<基表名> SUBPARTITION FOR (<表达式>,{<表达式>})

<选择列>::=<列名>[{,<列名> }]

<派生列表>::=(<列名>[{,<列名>}])

<SAMPLE 子句>::=

SAMPLE(<表达式>) |

SAMPLE(<表达式>) SEED (<表达式>) |

SAMPLE BLOCK(<表达式>) |

SAMPLE BLOCK(<表达式>) SEED (<表达式>)

<闪回查询>::= <闪回查询子句>|<闪回版本查询子句>请参考 17.2 闪回查询

<闪回查询子句>::=

WHEN <TIMESTAMP time_exp> |

AS OF <TIMESTAMP time_exp> |

AS OF <SCN|LSN lsn>

<闪回版本查询子句>::=VERSIONS BETWEEN <TIMESTAMP time_exp1 AND time_exp2> | < SCN|LSN lsn1 AND lsn2>

<连接表>::=<交叉连接>|

<限定连接>|

<join_apply_item>|

(<交叉连接>)|

```
            (<限定连接>)|

            (<join_apply_item>)

<交叉连接>::=<表引用> CROSS JOIN <普通表>|

            <表引用> CROSS JOIN (<连接表>)

<限定连接>::=<表引用> NATURAL JOIN <普通表>|

            <表引用> NATURAL JOIN (<连接表>)|

            <表引用> NATURAL <连接类型> JOIN <普通表>|

            <表引用> NATURAL <连接类型> JOIN (<连接表>)|

            <表引用> JOIN <表引用> <限定连接条件>|

            <表引用>[<PARTITION JOIN 子句>] <限定连接类型> JOIN <表引用> [<PARTITION
JOIN 子句>] <限定连接条件>

<限定连接类型>::= [<内外连接类型>] INNER|

                    <内外连接类型> [OUTER]

<内外连接类型>::=LEFT|RIGHT|FULL

<限定连接条件>::=<条件匹配>|<列匹配>

<条件匹配>::=ON<搜索条件>

<列匹配>::=USING(<连接列列名>{,<连接列列名>})

<join_apply_item>:: =

        <表引用> CROSS APPLY <普通表>|

        <表引用> CROSS APPLY (<连接表>)|

        <表引用> OUTER APPLY <普通表>|

        <表引用> OUTER APPLY (<连接表>)

<WHERE 子句> :: =

    WHERE <搜索条件>|
```

```
    < WHERE CURRENT OF 子句>

<搜索条件>::=<逻辑表达式>

< WHERE CURRENT OF 子句>: :  =WHERE CURRENT OF <游标名>

<层次查询子句>::=

    CONNECT BY [NOCYCLE] <连接条件>[START WITH <起始条件> ] |

    START WITH <起始条件> CONNECT BY [NOCYCLE] <连接条件>

<连接条件>::=<逻辑表达式>

<起始条件>::=<逻辑表达式>

<GROUP BY 子句> ::= GROUP BY <group_by 项>

<group_by 项>::=<group_by 子项 > {,<group_by 子项>}

<group_by 子项>::=<group_by 分组项> | <ROLLUP 项> | <CUBE 项> |<GROUPING SETS 项>

<group_by 分组项>=<分组项>|

                    <分组项>{,<分组项>})

<分组项>::=<列名>|<值表达式>|()

<ROLLUP 项>::=ROLLUP (<rollup 分组项>)

<rollup 分组项>::= <列名> | <值表达式>

<CUBE 项>::=CUBE (<cube 分组项>)

<cube 分组项>::= <列名> | <值表达式>

<GROUPING SETS 项>::=GROUPING SETS(<group_by 分组项>)

<HAVING 子句> ::= HAVING <搜索条件>

<PARTITION JOIN 子句> ::=PARTITION BY (<表列名>{,<表列名>})

<PIVOT 子句> ::= PIVOT [XML] (<set_func_clause> FOR <pivot_for_clause> IN

(<pivot_in_clause>))

<set_func_clause> ::= <集函数> [[AS] <别名>] {,<集函数> [[AS] <别名>]}
```

```
<pivot_for_clause> ::=
    <列名> |
    (<列名> {,<列名>})
<pivot_in_clause> ::=
    <exp_clause> [[AS] <别名>] {,<exp_clause> [[AS] <别名>]} |
    <select_clause> |
    ANY
<exp_clause> ::=
    <表达式> |
    (<表达式>  {,<表达式>})
<UNPIVOT 子句> ::= UNPIVOT [<include_null_clause>](<unpivot_val_col_lst> FOR
<unpivot_for_clause> IN (<unpivot_in_clause_low> ))
<include_null_clause> ::=
    INCLUDE NULLS |
    EXCLUDE NULLS
<unpivot_val_col_lst> ::=
    <表达式> |
    (<表达式> {,<表达式>})
<unpivot_for_clause> ::=
    <表达式> |
    (<表达式> {,<表达式>})
<unpivot_in_clause_low>  ::= <unpivot_in_clause>{,<unpivot_in_clause>}
<unpivot_in_clause>  ::=
    <列名> [AS <别名>]  |
```

(<列名> {,<列名>}) [AS (<别名> {,<别名>})]

参数

1. <集合运算符>包括并集 UNION、差集 EXCEPT、差集 MINUS 和交集 INTERSECT。具体请参考 4.5 集合查询;

2. ALL|DISTINCT|UNIQUE 用于集合查询中。ALL 表示集合查询结果中保持所有重复;DISTINCT 表示删除所有重复。DISTINCT 与 UNIQUE 等价。缺省为 DISTINCT;

3. CORRESPONDING 用于指定列名链表,通过指定列名(或列名的别名)链表来对两个查询分支的查询项进行筛选。无论分支中有多少列,最终的结果集只包含 CORRESPONDING 指定的列。查询分支和 CORRESPONDING 的关系为:<查询分支 1> CORRESPONDING [BY (<列名> {,<列名>})] <查询分支 2>。如果 CORRESPONDING 指定了列名但两个分支中没有相同列名的查询项则报错,如果 CORRESPONDING 没指定列名,则按照第一个分支的查询项列名进行筛选;例如:select c1, c2, c3 from t1 union all corresponding by (c1,c2) select d1, d2 c1, d3 c2 from t2;

4. <HINT 子句> 用于使用优化器提示,HINT 可以设置为 INI 参数、索引、连接方法等;其中具体支持使用 HINT 的 INI 参数可通过查询 V$HINT_INI_INFO 动态视图获取,视图的 PARA_NAME 列展示参数名,HINT_TYPE 列展示支持的 HINT 类型;支持 HINT 的 INI 参数分为两类:一是 HINT_TYPE 为"EXEC",表示运行阶段使用的参数,需要放在最外层的 SELECT 语句中,置于 SELECT 子句中不生效;二是 HINT_TYPE 为"OPT",表示分析阶段使用的参数,不要求放在最外层的 SELECT 语句中,但其中部分参数需要保持全局一致,即需要和上层的参数值保持一致。需要保持全局一致的参数例如:并行度"PARALLEL";

5. <模式名> 被选择的表和视图所属的模式,缺省为当前模式;

6. <基表名> 被选择数据的基表的名称;

7. <视图名> 被选择数据的视图的名称;

8. <索引名> 指定查询语句所使用的索引的名称;

9. * 指定对象的所有的列；

10. <值表达式> 可以为一个<集函数>、<函数>、<标量子查询>或<计算表达式>等等；

11. <列别名> 为列表达式提供不同的名称，使之成为列的标题，列别名不会影响实际的名称，别名在该查询中被引用；

12. <表别名> 给表、视图提供不同的名字，经常用于求子查询和相关查询的目的；

13. <列名> 指明列的名称；

14. <WHERE 子句> 限制被查询的行必须满足条件，如果忽略该子句，DM 从在 FROM 子句中的表、视图中选取所有的行；其中，<WHERE CURRENT OF 子句>专门用于游标更新、删除中，用来限定更新、删除与游标有关的数据行。

15. <HAVING 子句> 限制所选择的行组所必须满足的条件，缺省为恒真，即对所有的组都满足该条件；

16. <无符号整数> 指明了要排序的<值表达式>在 SELECT 后的序列号；

17. <列说明> 排序列的名称；

18. ORDER SIBLINGS BY 必须与 CONNECT BY 一起配合使用。可用于指定层次查询中相同层次数据返回的顺序；

19. ASC 指明为升序排列，缺省为升序；

20. DESC 指明为降序排列；

21. NULLS FIRST 指定排序列的 NULL 值放在最前面，不受 ASC 和 DESC 的影响，缺省的是 NULLS FIRST；

22. NULLS LAST 指定排序列的 NULL 值放在最后面，不受 ASC 和 DESC 的影响；

23. <PARTITION BY 子句> 指明分区外连接中的分区项，最多支持 255 个列；仅允许出现在左外连接右侧表和右外连接中的左侧表，且不允许同时出现，详见 4.2.7；

24. BULK COLLECT INTO 的作用是将检索结果批量的、一次性的赋给集合变量。与每

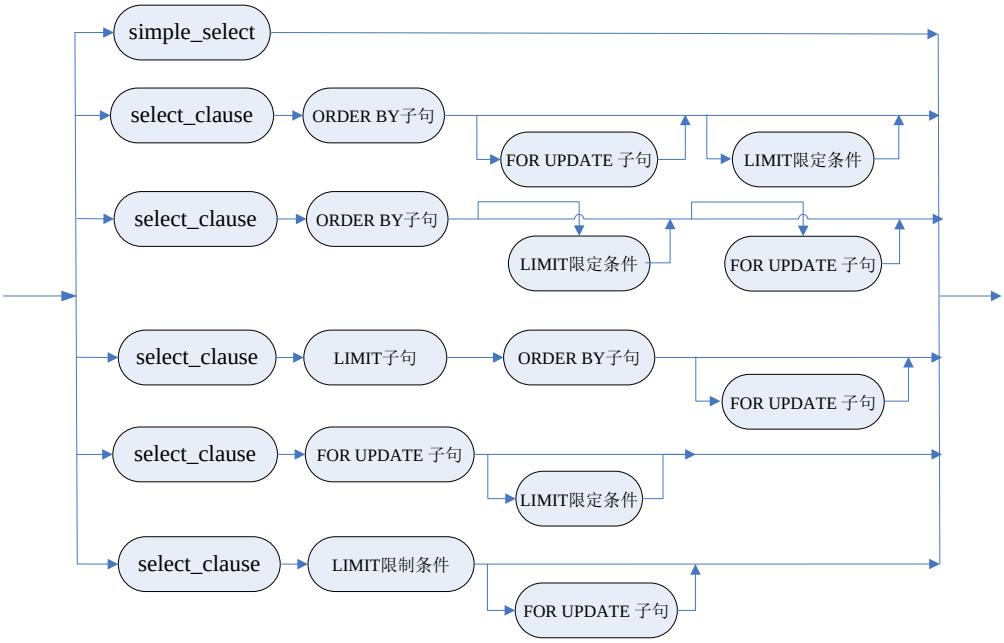
次获取一条数据，并每次都要将结果赋值给一个变量相比，可以很大程度上的节省开销。使用 BULK COLLECT 后，INTO 后的变量必须是集合类型;

25. CROSS APPLY 是在 CROSS JOIN 的基础上增加了右表可以引用左表列的功能，即同层列引用;

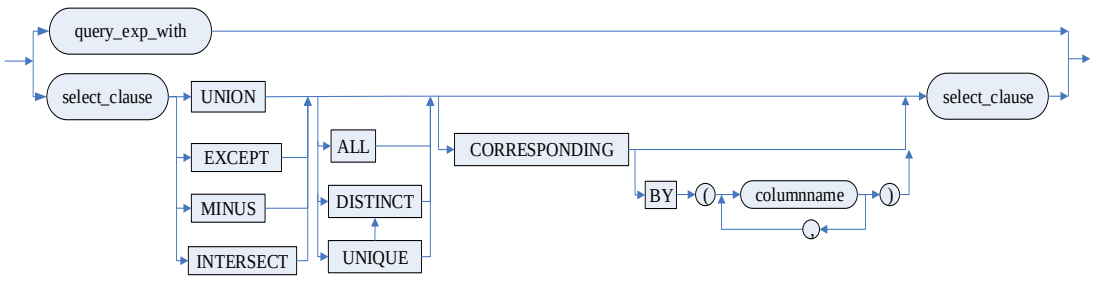
26. OUTER APPLY 是在 LEFT OUTER JOIN 的基础上增加了右表可以引用左表列的功能，即同层列引用。

图例

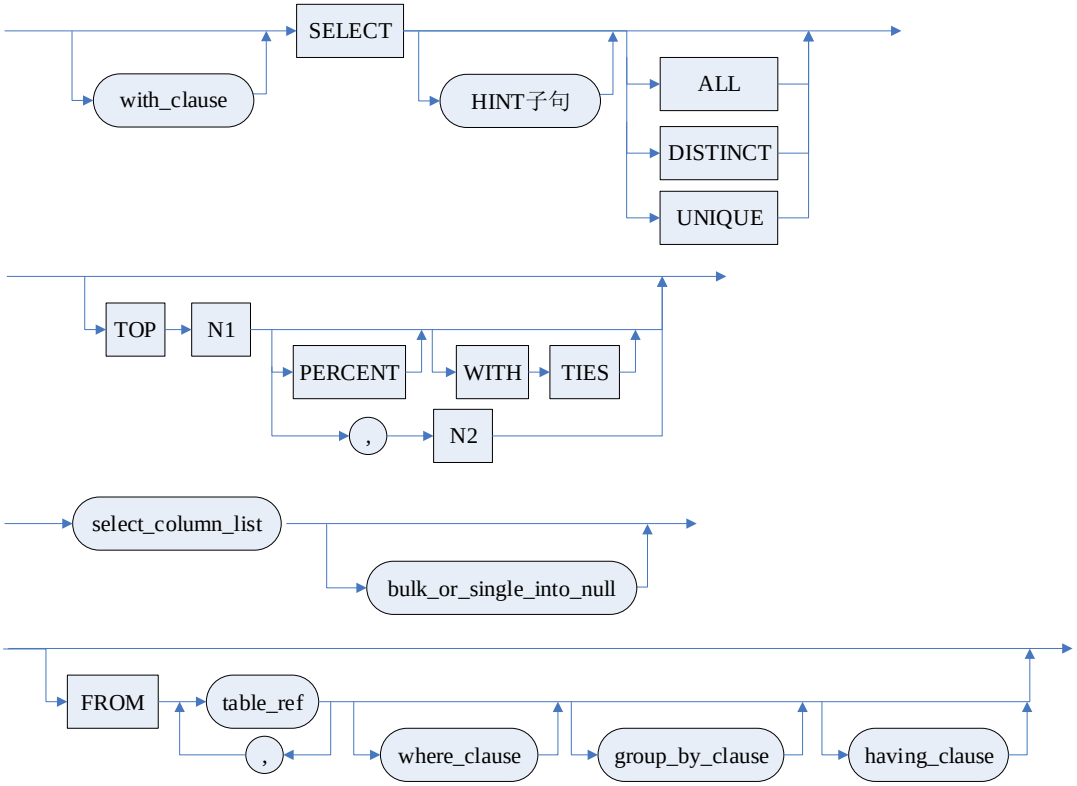
查询表达式（query_express）



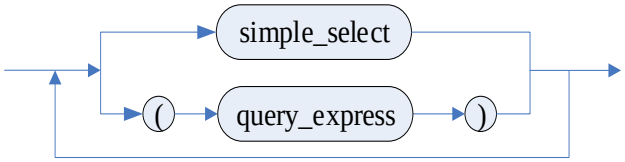
simple_select



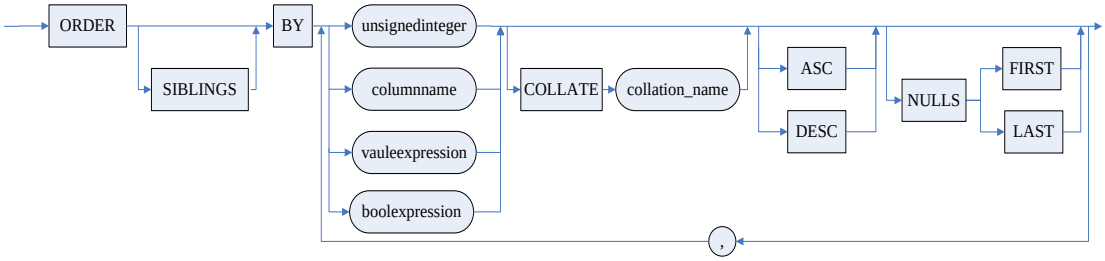
query_exp_with



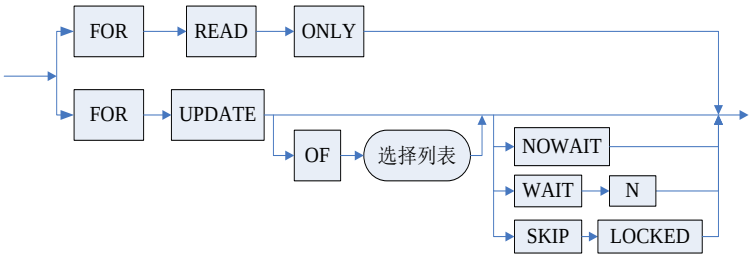
select_clause



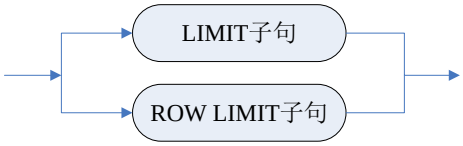
ORDER BY 子句



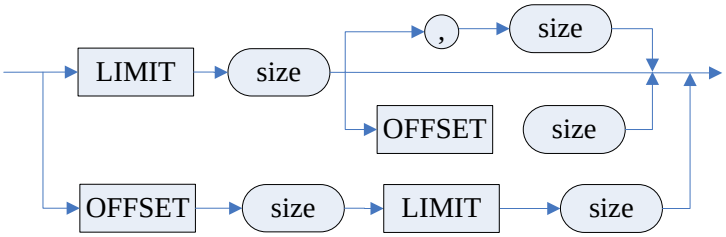
FOR UPDATE 子句



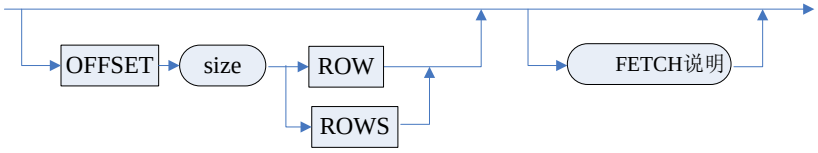
LIMIT 限定条件



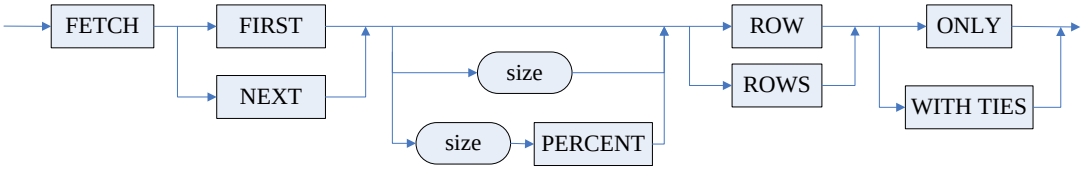
LIMIT 子句



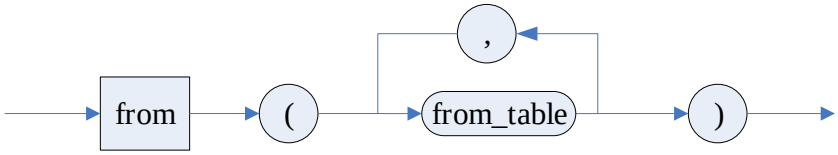
ROW LIMIT 子句



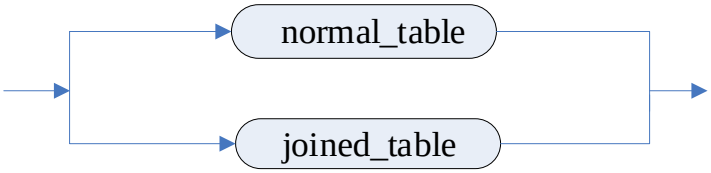
FETCH 说明



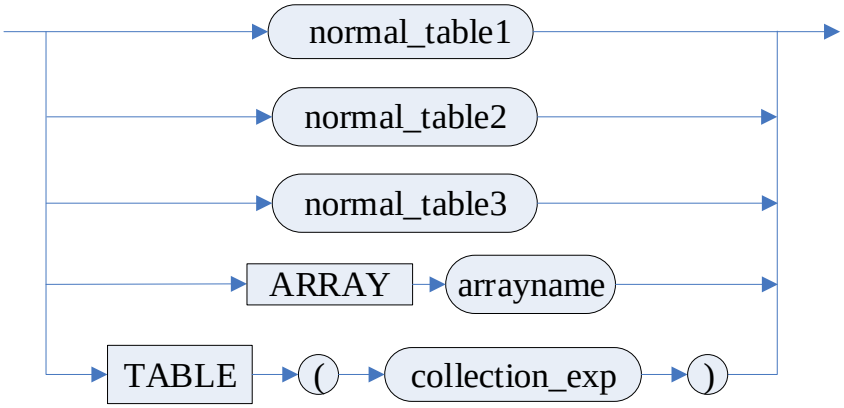
FROM 子句



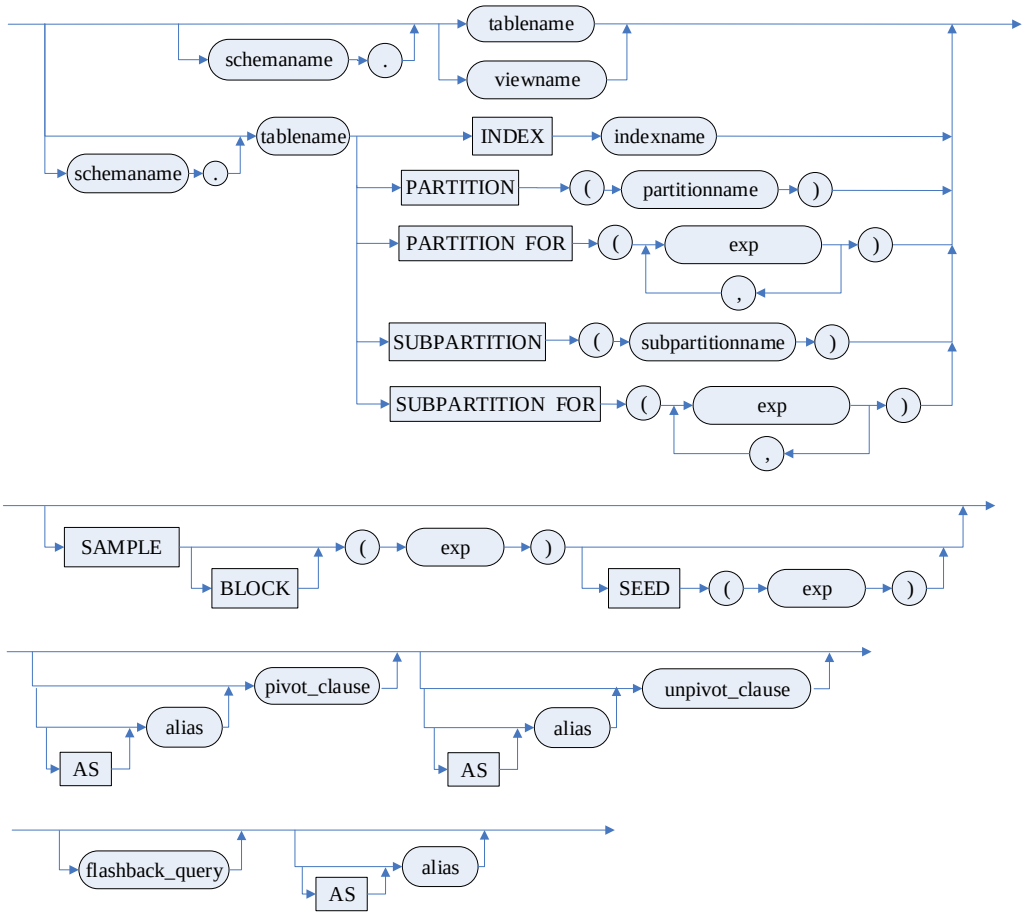
表引用（from_table）



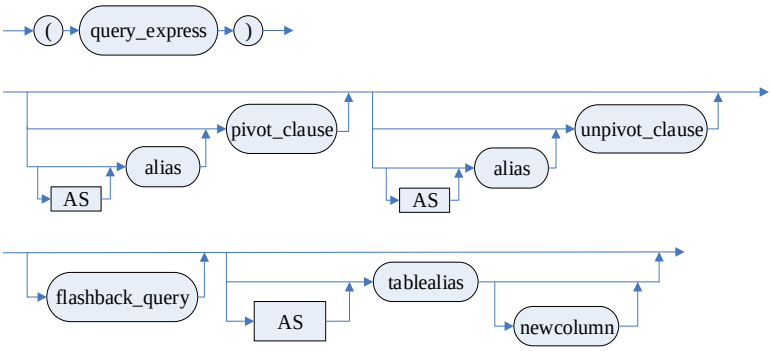
普通表（normal_table）



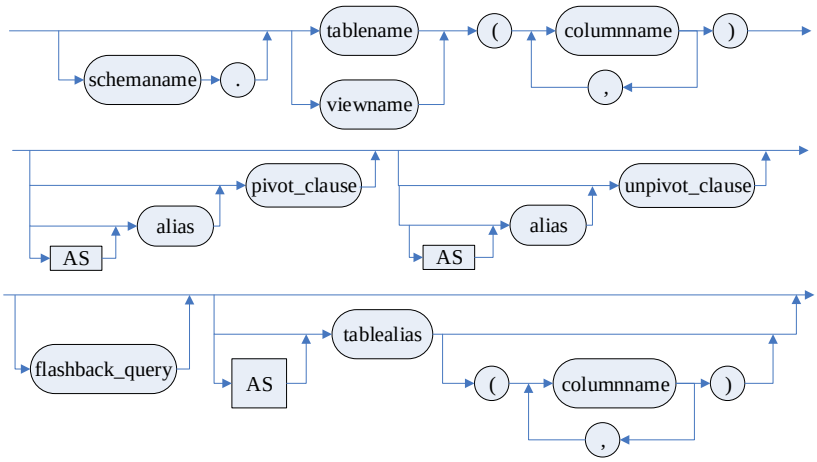
普通表 1（normal_table1）



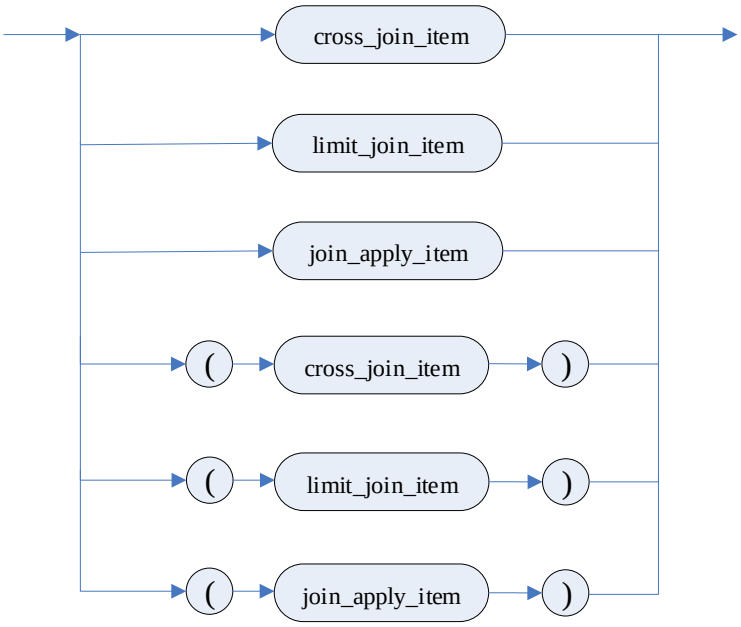
普通表 2（normal_table2）



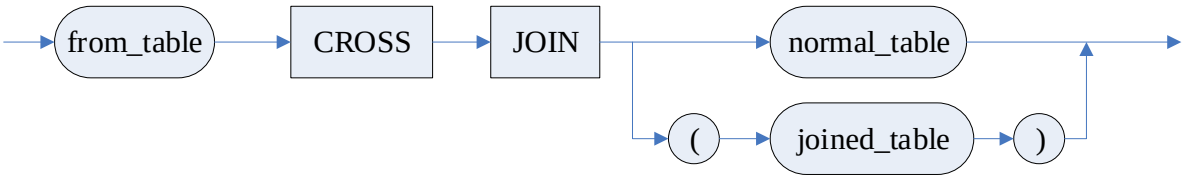
普通表 3（normal_table3）



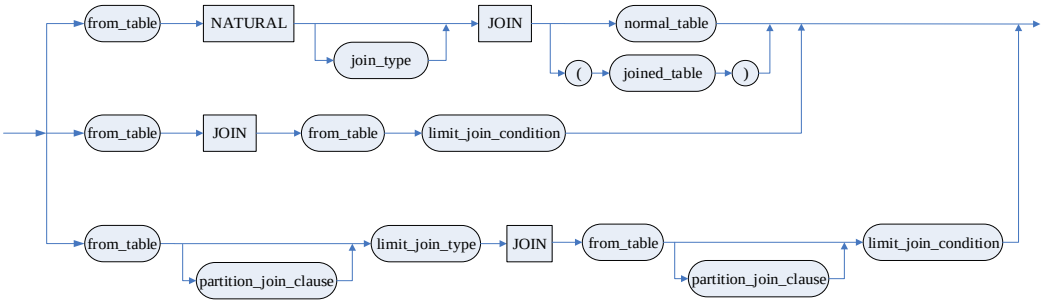
连接表（joined_table）



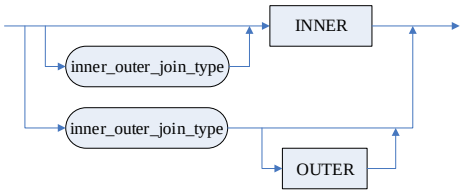
交叉连接（cross_join_item）



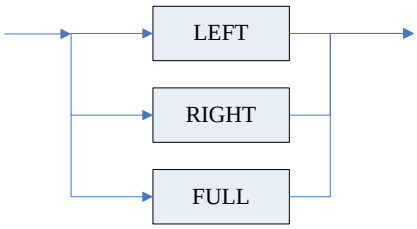
限定连接（limit_join_item）



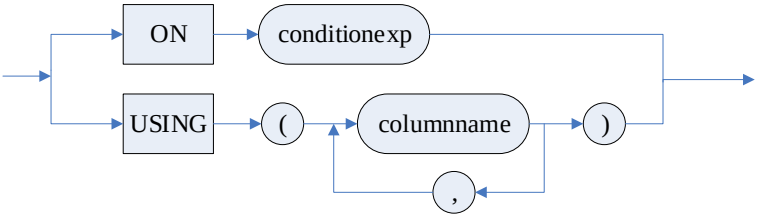
限定连接类型（limit_join_type）



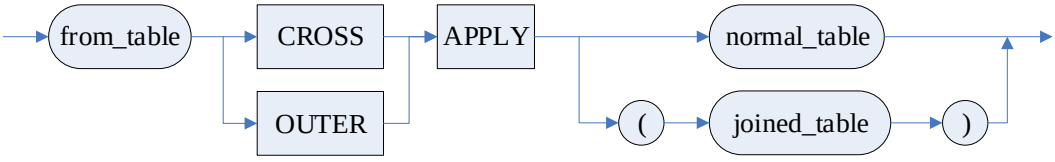
内外连接类型（inner_outer_join_type）



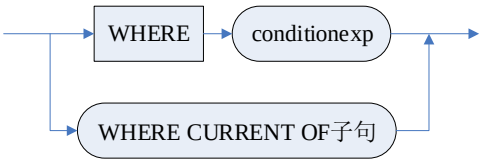
限定连接条件（limit_join_condition）



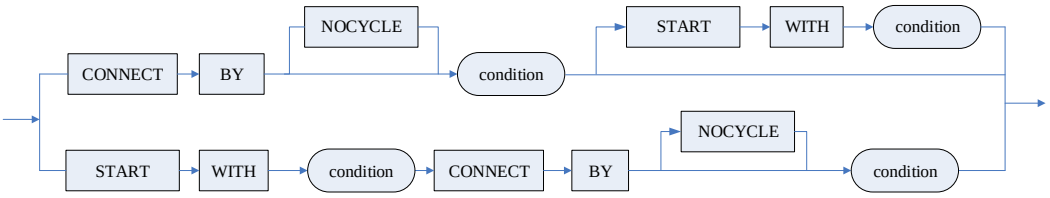
join_apply_item



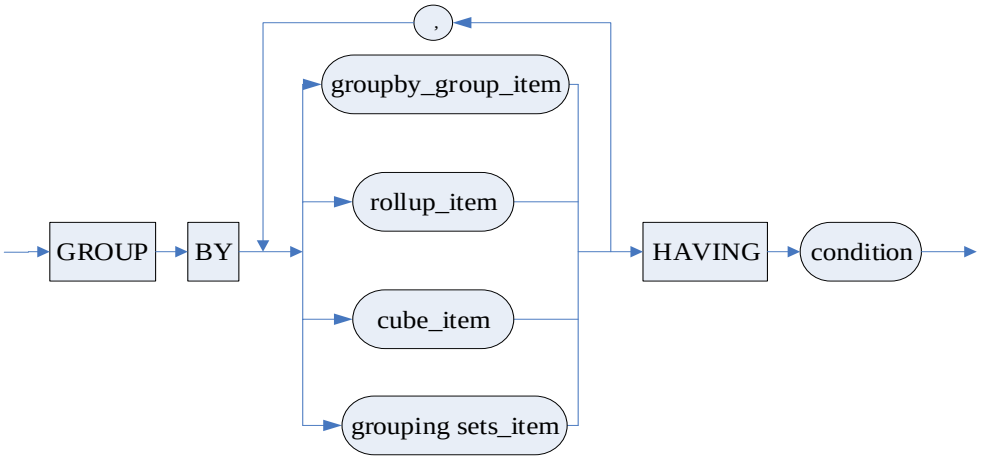
WHERE 子句



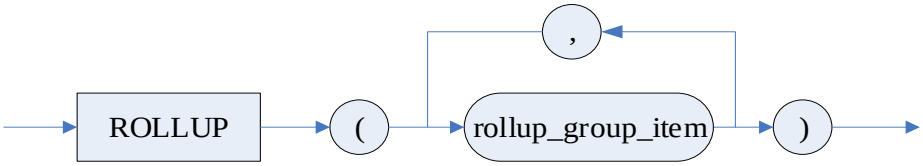
层次查询子句



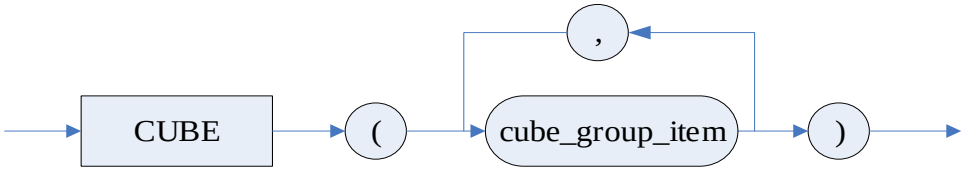
GROUP BY 子句



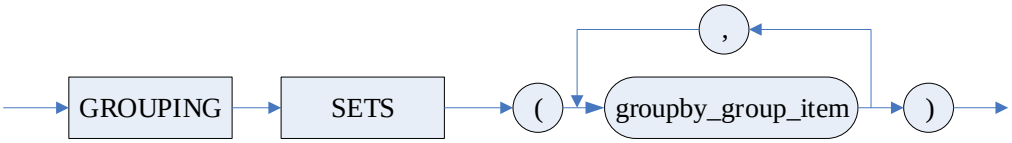
ROLLUP 项



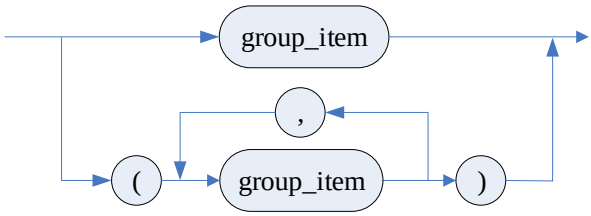
CUBE 项



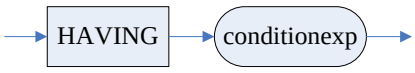
GROUPING SETS 项



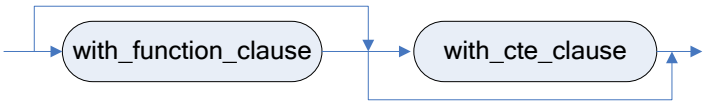
group_by 分组项（groupby_group_item）



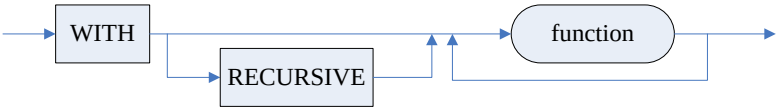
HAVING 子句（having_clause）



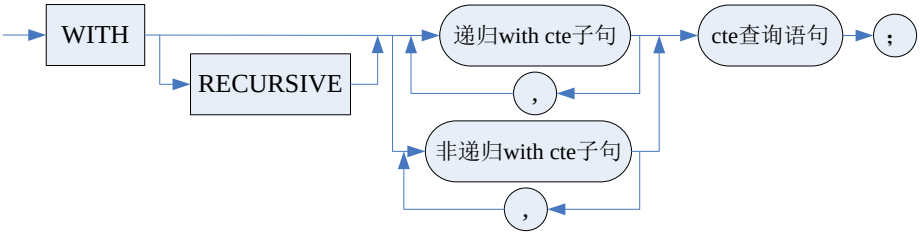
WITH 子句（with_clause）



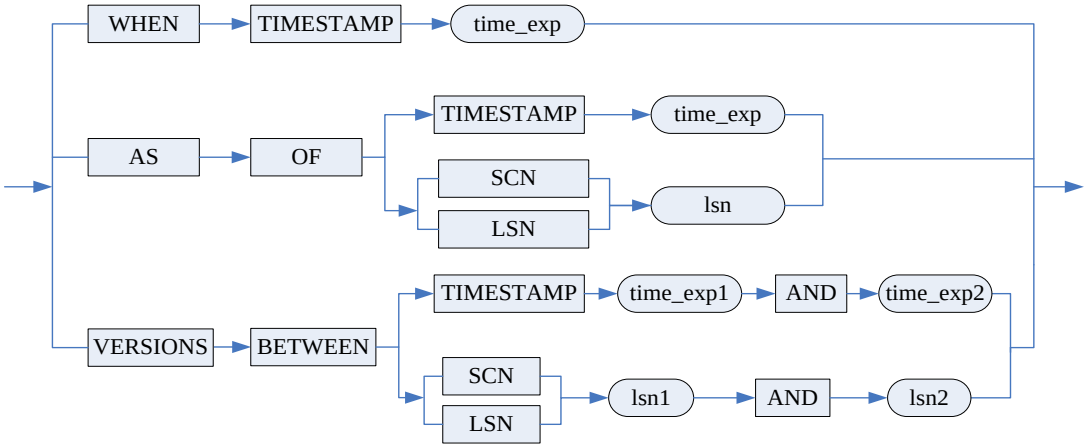
WITH FUNCTION 子句(with_function_clause)



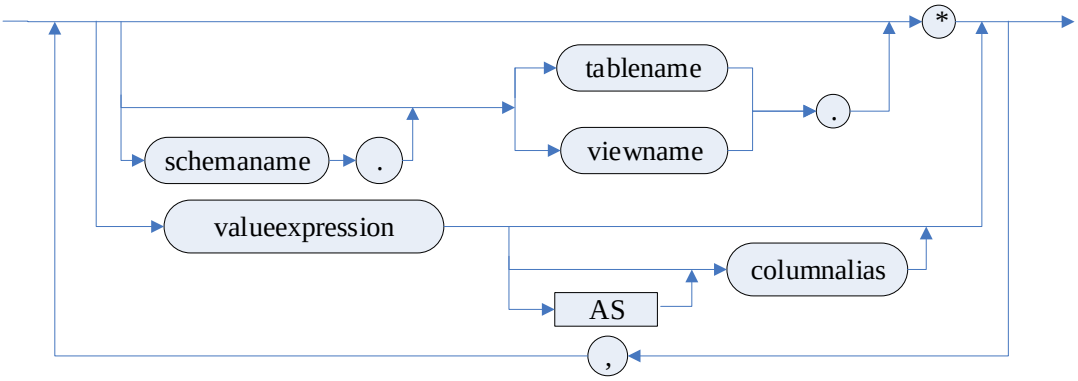
WITH CTE 子句(with_cte_clause)



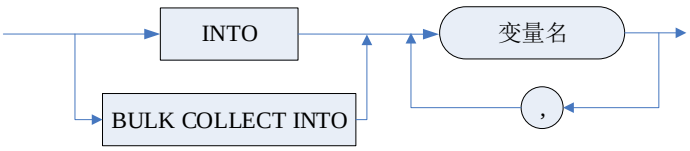
闪回查询



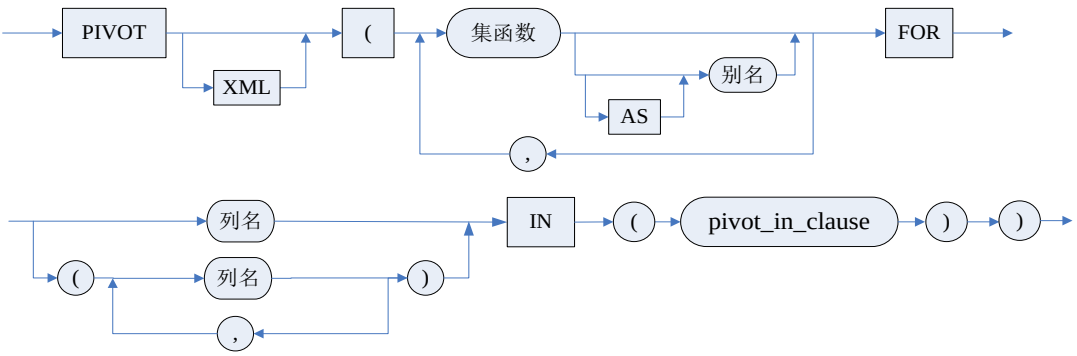
选择列表



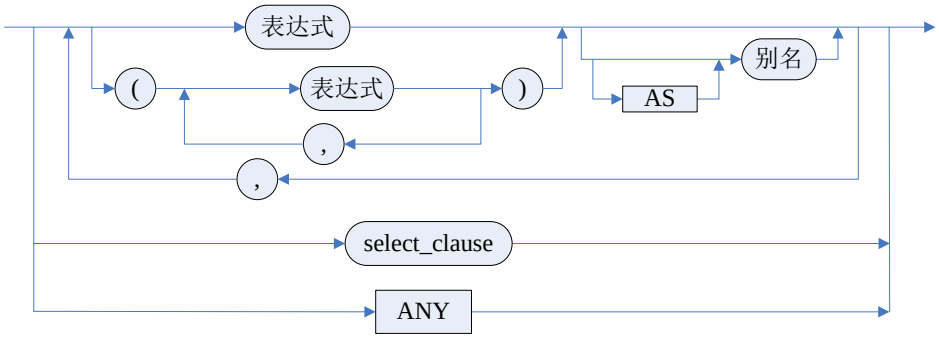
bulk_or_single_into_null



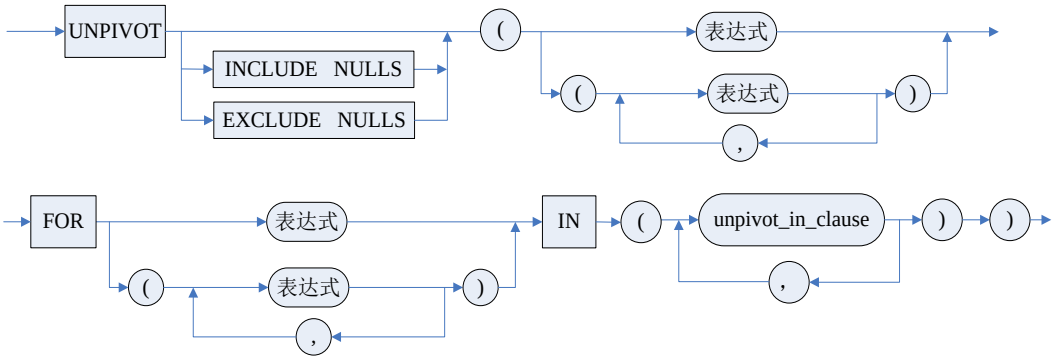
PIVOT 子句（pivot_clause）



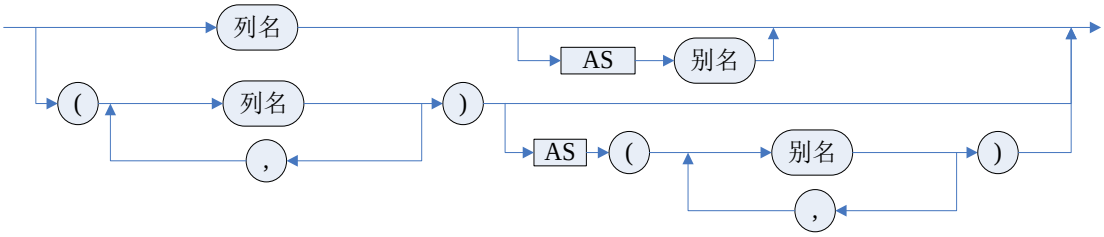
pivot_in_clause



UNPIVOT 子句（unpivot_clause）



unpivot_in_clause



使用说明

1．<选择列表>中最多可包含 2048 个查询项，且查询记录的长度限制不能超过块长的一半；

2．<FROM 子句>中最多可引用 100 张表；

3．WHERE<搜索条件> 用于设置对于行的检索条件。不在规定范围内的任何行都从结果集中去除；

4．查询语句调用的函数中，不能包含任何增删改操作（包括函数间接调用其它过程\函数产生的增删改操作）；

5．<bulk_or_single_into_null> 当前查询语句是在函数、过程等语句块内执行时，

且 INTO 的变量名在上下文中并未定义时，该条查询语句会在当前语句块创建以该变量名为表名的局部临时表，并转换为局部临时表的查询插入语句，不再输出结果集。

4.1 单表查询

SELECT 语句仅从一个表/视图中检索数据，称单表查询。即<FROM 子句>中<普通表>使用的是[<模式名>.]<基表名|视图名>。

4.1.1 简单查询

例 查询所有图书的名字、作者及当前销售价格，并消去重复。

```
SELECT  DISTINCT  NAME, AUTHOR, NOWPRICE  FROM  PRODUCTION.PRODUCT;
```

其中，DISTINCT 保证重复的行将从结果中去除。若允许有重复的元组，改用 ALL 来替换 DISTINCT，或直接去掉 DISTINCT 即可。

查询结果如下(注：除带 Order By 的查询外，本书所示查询结果中各元组的顺序与实际输出结果中的元组顺序不一定一致。)：

```
NAME                                AUTHOR              NOWPRICE
--------------------------- ------------    --------
红楼梦                              曹雪芹,高鹗         15.2000
水浒传                              施耐庵，罗贯中      14.3000
老人与海                            海明威              6.1000
射雕英雄传(全四册)                  金庸                21.7000
鲁迅文集(小说、散文、杂文)全两册    鲁迅                20.0000
长征                                王树增              37.7000
数据结构(C 语言版)(附光盘)          严蔚敏，吴伟民      25.5000
工作中无小事                        陈满麒              11.4000
```

```
突破英文基础词汇                    刘毅              11.1000
噼里啪啦丛书(全 7 册)               (日)佐佐木洋子    42.0000
```

当用户需要查出所有列的数据，且各列的显示顺序与基表中列的顺序也完全相同时，为了方便用户提高工作效率，SQL 语言允许用户将 SELECT 后的<值表达式>省略为*。

```
SELECT * FROM PERSON.PERSON;
```

等价于：

```
SELECT PERSONID, NAME, SEX, EMAIL, PHONE FROM PERSON.PERSON;
```

其查询结果是模式 PERSON 中基表 PERSON 的一份拷贝，结果从略。

4.1.2 带条件查询

带条件查询是指在指定表中查询出满足条件的元组。该功能是在查询语句中使用 WHERE 子句实现的。WHERE 子句常用的查询条件由谓词和逻辑运算符组成。谓词指明了一个条件，该条件求解后，结果为一个布尔值：真、假或未知。

逻辑运算符有：AND，OR，NOT、&&、!。其中 AND 和&&表示逻辑与运算、OR 表示逻辑或运算，NOT 和!表示逻辑非运算。

谓词包括比较谓词(=、>、<、>=、<=、<>)，BETWEEN 谓词、IN 谓词、LIKE 谓词、NULL 谓词、EXISTS 谓词。

1．使用比较谓词的查询

当使用比较谓词时，数值数据根据它们代数值的大小进行比较，字符串的比较则按序对同一顺序位置的字符逐一进行比较。若两字符串长度不同，短的一方应在其后增加空格，使两串长度相同后再作比较。

例 给出当前销售价格在 10 ~ 20 元之间的所有图书的名字、作者、出版社和当前价格。

```
SELECT NAME, AUTHOR, PUBLISHER, NOWPRICE FROM PRODUCTION.PRODUCT WHERE
```

```
NOWPRICE>=10  AND  NOWPRICE<=20;
```

查询结果如下:

```
NAME                          AUTHOR          PUBLISHER          NOWPRICE
-------------------------  -------------  --------------  --------
红楼梦                        曹雪芹,高鹗      中华书局              15.2000
水浒传                        施耐庵，罗贯中   中华书局              14.3000
鲁迅文集(小说、散文、杂文)全两册 鲁迅                                 20.0000
工作中无小事                  陈满麒           机械工业出版社        11.4000
突破英文基础词汇              刘毅             外语教学与研究出版社  11.1000
```

2．使用 BETWEEN 谓词的查询

例 给出当前销售价格在 10 ~ 20 元之间的所有图书的名字、作者、出版社和当前价格。

```
SELECT  NAME, AUTHOR, PUBLISHER, NOWPRICE  FROM  PRODUCTION.PRODUCT  WHERE
NOWPRICE  BETWEEN  10  AND  20;
```

此例查询与上例完全等价，查询结果如上所示。在 BETWEEN 谓词前面可以使用 NOT，以表示否定。

3．使用 IN 谓词的查询

谓词 IN 可用来查询某列值属于指定集合的元组。

例 查询出版社为中华书局或人民文学出版社出版的图书名称与作者信息。

```
SELECT  NAME, AUTHOR  FROM  PRODUCTION.PRODUCT WHERE  PUBLISHER  IN  ('中华书局', '人
民文学出版社');
```

查询结果如下:

```
NAME      AUTHOR
------  --------------
```

```
红楼梦    曹雪芹,高鹗
水浒传    施耐庵，罗贯中
长征      王树增
```

在 IN 谓词前面也可用 NOT 表示否定。

4．使用 LIKE 谓词的查询

LIKE 谓词一般用来进行字符串的匹配。我们先用实例来说明 LIKE 谓词的使用方法。

例 1 查询第一通讯地址中第四个字开始为“关山”且以 202 结尾的地址。

```
SELECT ADDRESSID, ADDRESS1, CITY, POSTALCODE FROM PERSON.ADDRESS WHERE
ADDRESS1 LIKE '_ _ _关山%202';
```

查询结果如下：

```
ADDRESSID   ADDRESS1                    CITY            POSTALCODE
----------- ------------------- -----------  ----------
13          洪山区关山春晓 55-1-202    武汉市洪山区    430073
14          洪山区关山春晓 10-1-202    武汉市洪山区    430073
15          洪山区关山春晓 11-1-202    武汉市洪山区    430073
```

由上例可看出，LIKE 谓词的一般使用格式为：

```
<列名> LIKE <匹配字符串常数>
```

其中，<列名>必须是可以转化为字符类型的数据类型的列。对于一个给定的目标行，如果指定列值与由<匹配字符串常数>给出的内容一致，则谓词结果为真。<匹配字符串常数>中的字符可以是一个完整的字符串，也可以是百分号“%”和下划线“_”，“%”和“_”称通配符。“%”代表任意字符串(也可以是空串)；“_”代表任何一个字符。

因此，上例中的 SELECT 语句将从 ADDRESS 表中检索出第一通讯地址中第四个字开始为“关山”且以 202 结尾的地址情况。从该例我们可以看出 LIKE 谓词是非常有用的。使用 LIKE

谓词可以找到所需要的但又记不清楚的那样一些信息。这种查询称模糊查询或匹配查询。为了加深对 LIKE 谓词的理解，下面我们再举几例：

```
ADDRESS1 LIKE '%洪山%'
```

如果 ADDRESS1 的值含有字符“洪山”，则该谓词取真值。

```
POSTALCODE LIKE '43_ _7_'
```

如果 POSTALCODE 的值由六个字符组成且前两个字符为 43，第五个字符为 7，则该谓词取真值。

```
CITY LIKE '%汉阳_'
```

如果 CITY 的值中倒数第三和第二个字为汉阳，则该谓词取真值。

```
ADDRESS1 NOT LIKE '洪山%'
```

如果 ADDRESS1 的值的前两个字不是洪山，则该谓词取真值。

阅读以上的例子，读者可能就在想这样一个问题：如果<匹配字符串常数>中所含“%”和“_”不是作通配符，而只是作一般字符使用应如何表达呢？为解决这一问题，SQL 语句对 LIKE 谓词专门提供了对通配符“%”和“_”的转义说明，这时 LIKE 谓语使用格式为：

```
<列名> LIKE '<匹配字符串常数>' [ESCAPE <转义字符>]
```

其中，<转义字符>指定了一个字符，当该字符出现在<匹配字符串常数>中时，用以指明紧跟其后的“%”或“_”不是通配符而仅作一般字符使用。

例 2 查询第一通讯地址以 C1_501 结尾的地址，则 LIKE 谓词应为：

```
SELECT ADDRESSID, ADDRESS1, CITY, POSTALCODE FROM PERSON.ADDRESS WHERE ADDRESS1 LIKE '%C1*_501' ESCAPE '*';
```

在此例中，*被定义为转义字符，因而在<匹配字符串常数>中*号后的下划线不再作通配符，而是普通字符。

查询结果如下：

```
ADDRESSID ADDRESS1 CITY POSTALCODE
```

```
---------- --------------------    ----------    ----------

16              洪山区光谷软件园 C1_501    武汉市洪山区    430073
```

为避免错误，转义字符一般不要选通配符“%”、“_”或在<匹配字符串常数>中已出现的字符。

5．使用.ROW 进行 LIKE 谓词的查询

LIKE 谓词除支持使用列的计算外，还支持通过 ROW 保留字对表或视图进行 LIKE 计算。该查询依次对表或视图中所有字符类型的列进行 LIKE 计算，只要有一列符合条件，则返回 TRUE。

其语法的一般格式为

```
<表名>.ROW  LIKE  <匹配字符串>  [ ESCAPE  <转义字符>]
```

例 查询评论中哪些与曹雪芹有关

```
SELECT * FROM PRODUCTION. PRODUCT_REVIEW WHERE  PRODUCT_REVIEW.ROW LIKE '%曹雪芹%';
```

该语句等价于

```
SELECT * FROM PRODUCTION. PRODUCT_REVIEW WHERE NAME LIKE '%曹雪芹%' OR EMAIL LIKE '%曹雪芹%' OR COMMENTS LIKE '%曹雪芹%';
```

6．使用 NULL 谓词的查询

空值是未知的值。当列的类型为数值类型时，NULL 并不表示 0；当列的类型为字符串类型时，NULL 也并不表示空串。因为 0 和空串也是确定值。NULL 只能是一种标识，表示它在当前行中的相应列值还未确定或未知，对它的查询也就不能使用比较谓词而须使用 NULL 谓词。

例 查询哪些人员的 EMAIL 地址为 NULL。

```
SELECT  NAME, SEX, PHONE  FROM  PERSON.PERSON WHERE  EMAIL  IS NULL;
```

在 NULL 谓词前，可加 NOT 表示否定。

7. 组合逻辑

可以用逻辑运算符与各种谓词相组合生成较复杂的条件查询。

例 查询当前销售价格低于15元且折扣低于7或出版社为人民文学出版社的图书名称和作者。

```
SELECT  NAME, AUTHOR  FROM  PRODUCTION.PRODUCT WHERE  NOWPRICE < 15 AND DISCOUNT
< 7 OR PUBLISHER='人民文学出版社';
```

查询结果如下:

```
NAME                AUTHOR
----------  -------
老人与海          海明威
长征                王树增
工作中无小事      陈满麒
```

4.1.3 集函数

为了进一步方便用户的使用，增强查询能力，SQL 语言提供了多种系统集函数。当根据某一限制条件从表中导出一组行集时，使用集函数可对该行集作统计操作并返回单一统计值。

集函数经常与 SELECT 语句的 GROUP BY 子句一同使用。集函数对于每个分组只返回一行数据。

4.1.3.1 函数分类

集函数可分为 14 类:

1. COUNT(*);

2. 相异集函数 AVG|MAX|MIN|SUM|COUNT(DISTINCT<列名>);

3. 完全集函数 AVG|MAX|MIN|COUNT|SUM([ALL]<值表达式>);

4．方差集函数 VAR_POP、VAR_SAMP、VARIANCE、STDDEV_POP、STDDEV_SAMP、STDDEV；

5．协方差函数 COVAR_POP、COVAR_SAMP、CORR；

6．首行函数 FIRST_VALUE；

7．任意行函数 ANY_VALUE；

8．求区间范围内最大值集函数 AREA_MAX；

9．FIRST/LAST 集函数；

10．字符串集函数 LISTAGG/LISTAGG2、WM_CONCAT、COLLECT；

11．求中位数函数 MEDIAN；

12．线性回归相关 REGR 集函数 REGR_COUNT、REGR_AVGX、REGR_AVGY、REGR_SLOPE、REGR_INTERCEPT、REGR_R2、REGR_SXX、REGR_SYY、REGR_SXY。

13．百分比集函数 PERCENTILE_CONT、PERCENTILE_DISC。

14．位运算集函数 BIT_AND、BIT_OR、BIT_XOR。

4.1.3.2 使用说明

在使用集函数时要注意以下几点：

1．相异集函数与完全集函数的区别是：相异集函数是对表中的列值消去重复后再作集函数运算，而完全集函数是对包含列名的值表达式作集函数运算且不消去重复。缺省情况下，集函数均为完全集函数；

2．集函数中的自变量可以是集函数，但最多只能嵌套 2 层。嵌套分组函数的时候，需要使用 GROUP BY；

3．**AVG、SUM** 的参数必须为数值类型；**MAX、MIN** 的结果数据类型与参数类型保持一致；对于 **SUM** 函数，如果参数类型为 BYTE、SMALLINT 或 INTEGER，那么结果类型为 BIGINT，如果参数类型为 FLOAT 或 DOUBLE，那么结果类型为 DOUBLE，如果参数类型为 NUMERIC 或 DEC，那么结果类型为 DEC；**COUNT** 的结果类型统一为 BIGINT；

对于 AVG 函数，其参数类型与结果类型对应关系如表 4.1.1 所示：

表 4.1.1 AVG 函数的参数类型与对应结果类型

参数类型	结果类型
tinyint	dec
smallint	dec
int	dec
bigint	dec
float	double
double	double
dec(x,y)	dec

4．方差集函数中参数 expr 为<列名>或<值表达式>，具体用法如下：

1） VAR_POP(expr) 返回 expr 的总体方差。其计算公式为：

$$\frac{SUM(\exp r^2)-\frac{(SUM(\exp r))^2}{COUNT(\exp r)}}{COUNT(\exp r)}$$

2） VAR_SAMP(expr) 返回 expr 的样本方差，如果 expr 的行数为 1，则返回 NULL。其计算公式为：

$$\frac{SUM(\exp r^2)-\frac{(SUM(\exp r))^2}{COUNT(\exp r)}}{COUNT(\exp r)-1}$$

3） VARIANCE(expr) 返回 expr 的方差，如果 expr 的行数为 1，则返回为 0，行数大于 1 时，与 var_samp 函数的计算公式一致；

4） STDDEV_POP(expr) 返回 expr 的标准差，返回的结果为总体方差的算术平方根，即 var_pop 函数结果的算术平方根。公式如下：

$$\sqrt{\frac{SUM(\exp r^2)-\frac{(SUM(\exp r))^2}{COUNT(\exp r)}}{COUNT(\exp r)}}$$

5） **STDDEV_SAMP(expr)** 返回 expr 的标准差，返回的结果为样本方差的算术平方根，即 var_samp 函数结果的算术平方根，所以如果 expr 的行数为 1，stddev_samp 返回 NULL；

6） **STDDEV(expr)** 与 stddev_samp 基本一致，差别在于，如果 expr 的行数为 1，stddev 返回 0，即 variance 函数结果的算术平方根。公式如下：

$$\sqrt{\frac{SUM(\exp r^2)-\frac{(SUM(\exp r))^2}{COUNT(\exp r)}}{COUNT(\exp r)-1}}$$

5．协方差集函数中参数 expr1 和 expr2 为<列名>或<值表达式>，具体用法如下：

1） **COVAR_POP(expr1, expr2)** 返回 expr1 和 expr2 的总体协方差。其计算公式为：

$$\frac{SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r2,\exp r1,NULL))*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}{COUNT(NVL2(\exp r1,\exp r2,NULL))}$$

2） **COVAR_SAMP(expr1, expr2)** 返回 expr1 和 expr2 的样本协方差，如果 expr 的行数为 1，则返回 NULL。其计算公式为：

$$\frac{SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r2,\exp r1,NULL))*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}{COUNT(NVL2(\exp r1,\exp r2,NULL))-1}$$

3） **CORR(expr1, expr2)** 返回 expr1 和 expr2 的相关系数，如果 expr 的行数为 1，则返回 NULL。其计算公式为：

$$\frac{COVAR_POP(\exp r1,\exp r2)}{STDDEV_POP(NVL2(\exp r2,\exp r1,NULL))*STDDEV_POP(NVL2(\exp r1,\exp r2,NULL))}$$

其中 NVL2(expr1, expr2, expr3)表示如果表达式 expr1 非空，NVL2 返回 expr2；如果表达 expr1 为空，NVL2 返回 expr3。

6．**FIRST_VALUE 集函数**，返回查询项的第一行记录；

7．**ANY_VALUE 集函数,** 实际返回查询项的第一行记录，等同于 FIRST_VALUE 集函数；

8．**AREA_MAX(EXPR, LOW, HIGH)** 在区间[LOW, HIGH]的范围内取 expr 的最大值。如果 expr 不在该区间内，则返回 LOW 值。如果 LOW 或 HIGH 为 NULL，则返回 NULL。expr 为<变量>、<常量>、<列名>或<值表达式>。参数 expr 类型为 TINYINT、SMALLINT、INT、BIGINT、DEC、FLOAT、DOUBLE、DATE、TIME、DATETIME、BINARY、VARBINARY、INTERVAL YEAR TO MONTH、INTERVAL DAY TO HOUR、TIME WITH TIME ZONE、DATETIME WITH TIME ZONE。LOW 和 HIGH 的数据类型和 expr 的类型一致，如果不一致，则转换为 expr 的类型，不能转换则报错。此外，LOW 和 HIGH 仅支持常量表达式。AREA_MAX 集函数返回值定义如下：

表 4.1.2 没有 GROUP BY 的情况

EXPR 集合	**是否有在[LOW, HIGH]区间内的非空值**	**结果**
空集	-	LOW
非空	否	LOW
非空	是	在[LOW,HIGH]区间的最大值

表 4.1.3 有 GROUP BY 的情况

分组前结果	**在[LOW, HIGH]区间内是否非空值**	**结果**
空集	-	整个结果为空集
非空集	是	在[LOW,HIGH]区间的最大值
非空集	否	LOW

9．**FIRST/LAST 集函数** 首先根据 SQL 语句中的 GROUP BY 分组（如果没有指定分组则所有结果集为一组），然后在组内根据 ORDER BY 子句进行排序。根据 FIRST/LAST 计算

第一名（最小值）/最后一名（最大值）的集函数值，排名按照奥林匹克排名法；

FIRST 和 LAST 集函数的用法：

```
<函数名> ([ALL] <值表达式>) KEEP ( DENSE_RANK <FIRST | LAST > <ORDER BY 子句>)

<函数名>::= AVG | MAX | MIN | COUNT | SUM

<ORDER BY 子句>::=语法参考 4.7 ORDER BY 子句
```

10．字符串集函数：

1) **LISTAGG/LISTAGG2(expr1, expr2)** 首先根据 SQL 语句中的 GROUP BY 分组（如果没有指定分组则所有结果集为一组），然后在组内按照 WITHIN GROUP 中的 ORDER BY 项进行排序（没有指定排序则按数据组织顺序），最后将表达式 expr1 用表达式 expr2 串接起来。表达式 expr1 为<常量>、<列名>或<值表达式>，支持和 DISTINCT 关键字一起使用，表示对组内的 exp1 进行去重操作后再进行串接；表达式 expr2 为指定用于分隔的分隔符，可以缺省。LISTAGG2 跟 LISTAGG 的功能是一样的，区别就是 LISTAGG 返回的是 VARCHAR 类型，LISTAGG2 返回的是 CLOB 类型。当 LISTAGG 返回串的长度超过 32767 字节时，将根据 LISTAGG_OVERFLOW 子句决定是否对返回串进行截断，若不指定 LISTAGG_OVERFLOW 子句或者指定 ON OVERFLOW ERROR，不对超长的字符串进行截断，即超长报错；当指定 ON OVERFLOW TRUNCATE 时，服务器将截断足够多的字符串，以确保返回值在不超过 32767 字节的情况下可以容纳最后一个分隔符、截断标识符和截断的字符串个数。<截断标识符>的缺省值为'…'，对于[{ WITH | WITHOUT } COUNT]选项，不指定该选项或指定 WITH COUNT 时会展示截断的字符串个数，否则不会展示截断的字符串个数。

LISTAGG 的用法：

```
<LISTAGG>([DISTINCT] <参数>[, <参数>] [<LISTAGG_OVERFLOW 子句>]) [WITHIN
```

```
GROUP(<ORDER BY 项>)]
<LISTAGG_OVERFLOW 子句>::=
ON OVERFLOW ERROR |
ON OVERFLOW TRUNCATE [ '截断标识符' ] [ { WITH | WITHOUT } COUNT ]
```

LISTAGG2 的用法:

```
<LISTAGG2>([DISTINCT] <参数>[, <参数>]) [WITHIN GROUP(<ORDER BY 项>)]
```

2) **WM_CONCAT(expr)** 首先根据 SQL 语句中的 GROUP BY 分组（如果没有指定分组则所有结果集为一组），然后将返回的组内指定参数用“,”拼接起来。expr 为<常量>、<列名>或<值表达式>，返回类型为 CLOB。WM_CONCAT 也可以写成 WMSYS.WM_CONCAT。

WM_CONCAT 的用法:

```
WM_CONCAT(expr[ || expr])
```

3) **COLLECT(expr)** 首先根据 SQL 语句中的 GROUP BY 分组（如果没有指定分组则所有结果集为一组），然后在组内按照 ORDER BY 项进行排序（没有指定排序则按数据组织顺序），最后将参数列拼接起来组合成对象类型大字段，再由外层 CAST 函数转换为嵌套表。表达式 expr 支持和 DISTINCT 或 UNIQUE 关键字一起使用，表示对组内的 expr 进行去重操作后再进行串接。COLLECT 返回的是 BLOB 类型，必须与 CAST 一起使用才能返回嵌套表类型，不支持在 CAST 之外使用。

COLLECT 的用法:

```
CAST (<COLLECT> ([DISTINCT / UNIQUE] <参数> [ORDER BY 项]) AS TYPE)
```

其中，TYPE 只能是嵌套表类型，且参数列类型可转换到嵌套表列类型。嵌套表列类型目前仅支持 DM 定义的常规数据类型（如数值类型、字符类型、多媒体类型、日期时间类型等），暂不支持非常规数据类型（如记录类型、数组

类型、集合类型、类类型等)。

11．**MEDIAN 集函数** 当组内排序后，返回组内的中位数。计算过程中忽略空值 NULL。MPP/LPQ 情况下，需要保证组内数据是全的，否则结果错误。MEDIAN()不支持和 DISTINCT 和<KEEP 子句>一起使用。<参数>：参数类型可以是数值类型（INT/DEC）、时间类型（DATETIME/DATE）、时间间隔类型（INTERVAL YEAR TO MONTH）。<参数>暂不支持带时区的时间类型。

MEDIAN 的用法：

```
MEDIAN(<参数>)
```

12．**线性回归相关 REGR 集函数** 参数 expr1 和 expr2 为<列名>或<值表达式>，当 expr1 或 expr2 为空值 NULL 时，忽略该组数值对。REGR 集函数均不支持 distinct，仅 regr_count 支持和<keep 子句>一起使用。与计算无关的参数（REGR_COUNT 的 expr1 和 expr2、REGR_AVGX 的 expr1、REGR_AVGY 的 expr2、REGR_SXX 的 expr1、REGR_SYY 的 expr2，这五个参数都与实际计算过程无关）支持包含自定义类型在内的任意类型。除与计算无关的参数外，REGR 集函数参数应为数值类型，REGR_AVGX 的 expr2 和 REGR_AVGY 的 expr1 还可以是时间间隔类型。具体用法如下：

1) REGR_COUNT(expr1, expr2) 返回所有非空(expr1,expr2)数值对的个数。等价于 COUNT(NVL2(expr1,expr2,NULL))。

2) REGR_AVGX(expr1,expr2)，去除含空值的数值对后，计算 expr2 的平均值，其计算公式为：

*SUM(NVL2(*expr1, expr2*, NULL))/COUNT(NVL2(*expr1, expr2*, NULL))*

3) REGR_AVGY(expr1,expr2)，去除含空值的数值对后，计算 expr1 的平均值，其计算公式为：

*SUM(NVL2(*expr2, expr1*, NULL))/COUNT(NVL2(*expr2, expr1*, NULL))*

4) REGR_SLOPE(expr1,expr2)，去除含空值的数值对后，计算回归曲线的斜率，

其计算公式为：

$$\frac{SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r2,\exp r1,NULL))*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}{SUM(NVL2(\exp r1,\exp r2^2,NULL))-\frac{(SUM(NVL2(\exp r1,\exp r2,NULL)))^2}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}$$

5）REGR_INTERCEPT(expr1,expr2)，去除含空值的数值对后，计算回归曲线在 y 轴(对应 expr1)上的截距，其计算公式为：

$$\frac{SUM(NVL2(\exp r2,\exp r1,NULL))-\frac{SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r2,\exp r1,NULL))*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}{SUM(NVL2(\exp r1,\exp r2^2,NULL))-\frac{(SUM(NVL2(\exp r1,\exp r2,NULL)))^2}{COUNT(NVL2(\exp r1,\exp r2,NULL))}}*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}$$

6）REGR_R2(expr1,expr2)，去除含空值的数值对后，计算回归曲线的相关系数，其计算公式为：

$$\frac{\left(SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r2,\exp r1,NULL))*SUM(NVL2(\exp r1,\exp r2,NULL))}{COUNT(NVL2(\exp r2,\exp r1,NULL))}\right)^2}{\left(SUM(NVL2(\exp r1,\exp r2^2,NULL))-\frac{(SUM(NVL2(\exp r1,\exp r2,NULL)))^2}{COUNT(NVL2(\exp r1,\exp r2,NULL))}\right)\left(SUM(NVL2(\exp r2,\exp r1^2,NULL))-\frac{(SUM(NVL2(\exp r2,\exp r1,NULL)))^2}{COUNT(NVL2(\exp r2,\exp r1,NULL))}\right)},$$

7）REGR_SXX(expr1,expr2)，计算诊断统计量 SXX，去除含空值的数值对后，相当于 COUNT(expr2) * VAR_POP(expr2)，其计算公式为：

$$SUM(NVL2(\exp r1,\exp r2^2,NULL))-\frac{(SUM(NVL2(\exp r1,\exp r2,NULL)))^2}{COUNT(NVL2(\exp r1,\exp r2,NULL))}$$

8）REGR_SYY(expr1,expr2)，计算诊断统计量 SYY，去除含空值的数值对后，相当于 COUNT(expr1) * VAR_POP(expr1)，其计算公式为：

$$SUM(NVL2(\exp r2,\exp r1^2,NULL))-\frac{(SUM(NVL2(\exp r2,\exp r1,NULL)))^2}{COUNT(NVL2(\exp r2,\exp r1,NULL))}$$

9）REGR_SXY(expr1,expr2)，计算诊断统计量 SXY，去除含空值的数值对后，相当于 REGR_COUNT(expr1,expr2) * COVAR_POP(expr1,expr2)，其计算公式为：

$$SUM(\exp r1*\exp r2)-\frac{SUM(NVL2(\exp r1,\exp r2,NULL))*SUM(NVL2(\exp r2,\exp r1,NULL))}{COUNT(NVL2(\exp r1,\exp r2,NULL))}$$

13．百分比集函数 PERCENTILE_CONT、PERCENTILE_DISC

1）PERCENTILE_CONT：计算连续分布的百分位数。当组内排序后，percentile_cont 函数返回组内某百分比位置处的数据（对应列为 order by 项的 key），计算过程中忽略空值 NULL。PERCENTILE_CONT 用法如下：

```
<PERCENTILE_CONT> (<参数>) WITHIN GROUP(<ORDER BY 项>) <GROUP BY 项>
```

说明

（1） <group by 项>为可选项；

（2） 参数的数学含义为百分比；

a）不含有 group by 语句时：合法的参数可以是[0, 1]区间内任意值，或能够计算成[0, 1]区间内值的常量表达式。

b）含有 group by 语句时：参数不仅可以是[0, 1]区间内任意值，或能够计算成[0, 1]区间内值的常量表达式，也可以是由 group by key 对应列构成的表达式（最终计算值范围依旧是[0, 1]）；

（3） <order by 项>：order by 项的 key 有且仅有 1 个，其类型可以是数值类型（INT/DEC）、时间类型（DATETIME/DATE）、时间间隔类型（INTERVAL YEAR TO MONTH），暂不支持带时区的时间类型；

（4） percentile_cont()不支持和 DISTINCT 和<KEEP 子句>一起使用，必须配合 within group 一起使用；

（5） MPP/LPQ 情况下，需要保证组内数据是全的，否则结果错误。

2）PERCENTILE_DISC：计算离散分布的百分位数。当组内排序后，percentile_disc 函数返回组内某百分比位置处的实际数据（对应列为 order by 项的 key），计算过程中忽略空值 NULL。PERCENTILE_DISC 用法如下：

```
<PERCENTILE_DISC> (<参数>) WITHIN GROUP(<ORDER BY 项>) <GROUP BY 项>
```

说明

(1) <group by 项>为可选项;

(2) 参数的数学含义为百分比;

a) 不含有 group by 语句时：合法的参数可以是[0, 1]区间内任意值，或能够计算成[0, 1]区间内值的常量表达式。

b) 含有 group by 语句时：参数不仅可以是[0, 1]区间内任意值，或能够计算成[0, 1]区间内值的常量表达式，也可以是由 group by key 对应列构成的表达式（最终计算值范围依旧是[0, 1]）;

(3) <order by 项>：order by 项的 key 有且仅有 1 个，数据类型无限制，任何支持排序操作的类型皆可;

(4) percentile_disc()不支持和 DISTINCT 和<KEEP 子句>一起使用，必须配合 within group 一起使用;

(5) MPP/LPQ 情况下，需要保证组内数据是全的，否则结果错误。

14．**位运算集函数** BIT_AND、BIT_OR、BIT_XOR 三个函数功能分别如下：BIT_AND(expr)返回所有数据按位进行与运算的结果；BIT_OR(expr)返回所有数据按位进行或运算的结果；BIT_XOR(expr) 返回所有数据按位进行异或运算的结果。参数的数据类型不同时，函数使用方式也略有不同，具体介绍如下：

1) 参数类型为 binary/varbinary 类型参数时，结果为 varbinary 类型，结果类型长度是参数类型长度加一;

2) binary/varbinary 类型参数使用 bit_and/bit_or/bit_xor，需要所有行数据长度都相等，否则报错;

3) 其余的数据类型，如果是非 bigint 类型，会先转为 bigint 类型再传入函数;

4) NULL 数据忽略，若所有数据都为 NULL，则 bit_and 结果为所有 bit 置为 1，bit_or/bit_xor 结果为所有 bit 置为 0。

4.1.3.3 举例说明

下面按集函数的功能分别举例说明。

1．求最大值集函数 MAX 和求最小值集函数 MIN

例 查询折扣小于 7 的图书中现价最低的价格。

```
SELECT MIN(NOWPRICE) FROM PRODUCTION.PRODUCT WHERE DISCOUNT < 7;
```

查询结果为：6.1000

需要说明的是：SELECT 后使用集函数 MAX 和 MIN 得到的是一个最大值和最小值，因而 SELECT 后不能再有列名出现，如果有只能出现在集函数中。如：

```
SELECT NAME,MIN(NOWPRICE) FROM PRODUCTION.PRODUCT;
```

DM 系统会报错，因为 NAME 是一个行集合，而最低价格是唯一值。

至于 MAX 的使用格式与 MIN 是完全一样的，读者可以自己举一反三。

2．求平均值集函数 AVG 和总和集函数 SUM

例 1 求折扣小于 7 的图书的平均现价。

```
SELECT AVG(NOWPRICE) FROM PRODUCTION.PRODUCT WHERE DISCOUNT < 7;
```

查询结果为：23.15

例 2 求折扣大于 8 的图书的总价格。

```
SELECT SUM(NOWPRICE) FROM PRODUCTION.PRODUCT WHERE DISCOUNT >8;
```

查询结果为：25.5

3．求总个数集函数 COUNT

例 1 查询已登记供应商的个数。

```
SELECT COUNT(*) FROM PURCHASING.VENDOR;
```

查询结果为：12

由此例可看出，COUNT(*)的结果是 VENDOR 表中的总行数，由于主关键字不允许有相同值，因此，它不需要使用保留字 DISTINCT。

例 2 查询目前销售的图书的出版商的个数。

```
SELECT  COUNT(DISTINCT  PUBLISHER)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：9

由于一个出版商可出版多种图书，因而此例中一定要用 DISTINCT 才能得到正确结果。

4．求方差集函数 VARIANCE、标准差函数 STDDEV 和样本标准差函数 STDDEV_SAMP

例 1 求图书的现价方差。

```
SELECT  VARIANCE(NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：136.648888888888888888888888888888888889

例 2 求图书的现价标准差。

```
SELECT STDDEV(NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：11.689692

例 3 求图书的现价样本标准差。

```
SELECT STDDEV_SAMP(NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：11.689692

5．求总体协方差集函数 COVAR_POP、样本协方差函数 COVAR_SAMP 和相关系数 CORR

例 1 求产品原始价格 ORIGINALPRICE 和当头销售价格 NOWPRICE 的总体协方差。

```
SELECT  COVAR_POP(ORIGINALPRICE, NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：166.226

例 2 求产品原始价格 ORIGINALPRICE 和当头销售价格 NOWPRICE 的样本协方差。

```
SELECT  COVAR_SAMP(ORIGINALPRICE, NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：184.695555555555555555555555555555555556

例 3 求产品原始价格 ORIGINALPRICE 和当头销售价格 NOWPRICE 的相关系数。

```
SELECT  CORR(ORIGINALPRICE, NOWPRICE)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：9.6276530968E-001

6．首行函数 FIRST_VALUE

例 返回查询项的首行记录。

```
SELECT FIRST_VALUE(NAME)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：红楼梦

7．任意行函数 ANY_VALUE

例 返回查询项的任意一行记录。

```
SELECT ANY_VALUE(NAME)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：红楼梦

8．求区间范围内的最大值函数 AREA_MAX

例 求图书的现价在 20~30 之间的最大值。

```
SELECT area_max(NOWPRICE,20,30)  FROM  PRODUCTION.PRODUCT;
```

查询结果为：25.5000

9.求 FIRST/LAST 集函数

例 求每个用户最早定的商品中花费最多和最少的金额。

```
SELECT CUSTOMERID,
    max(TOTAL) keep (dense_rank first order by ORDERDATE) max_val,
    min(TOTAL) keep (dense_rank first order by ORDERDATE) min_val
    from SALES.SALESORDER_HEADER group by CUSTOMERID;
```

查询结果如下:

```
CUSTOMERID      MAX_VAL      MIN_VAL

-----------    -------      -------

1                  36.9000      36.9000
```

10.求 LISTAGG/LISTAGG2 集函数、求 WM_CONCAT 集函数、求 COLLECT 集函数

例 1 求出版的所有图书，分隔符为', '，使用 LISTAGG/LISTAGG2。

```
SELECT LISTAGG(NAME, ', ') WITHIN GROUP (ORDER BY NAME) LISTAGG FROM

PRODUCTION.PRODUCT;
```

或

```
SELECT LISTAGG2(NAME, ', ')   WITHIN GROUP (ORDER BY NAME) LISTAGG FROM

PRODUCTION.PRODUCT;
```

查询结果如下:

```
LISTAGG

--------------------------------------------------------------------

长征, 工作中无小事, 红楼梦, 老人与海, 鲁迅文集(小说、散文、杂文)全两册, 射雕英雄传(全四册), 数据

结构(C 语言版)(附光盘), 水浒传, 突破英文基础词汇, 噼里啪啦丛书(全 7 册)
```

例 2 求每个出版社出版的所有图书。先根据出版社进行分组，然后将每个出版社出版的图书名用“,”拼接起来，使用 WM_CONCAT。

```
SELECT PUBLISHER, WM_CONCAT(NAME) FROM PRODUCTION.PRODUCT GROUP BY PUBLISHER;
```

查询结果如下:

```
PUBLISHER                WM_CONCAT(NAME)

------------------ ----------------------------------

中华书局                    红楼梦,水浒传
```

```
上海出版社                  老人与海
广州出版社                  射雕英雄传（全四册）
                            鲁迅文集（小说、散文、杂文）全两册
人民文学出版社              长征
清华大学出版社              数据结构（C 语言版）（附光盘）
机械工业出版社              工作中无小事
外语教学与研究出版社        突破英文基础词汇
21 世纪出版社               噼里啪啦丛书(全 7 册)
```

例 3 求出版的所有图书书名的嵌套表。

```
CREATE TYPE T_NAME AS TABLE OF VARCHAR;
/
SELECT CAST(COLLECT(NAME ORDER BY NAME) AS T_NAME) AS T_NAME FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
T_NAME
------------------------------------------------------------
RPODUCTION.T_NAME(长征,工作中无小事,红楼梦,老人与海,鲁迅文集(小说、散文、杂文)全两册,射雕英
雄传(全四册),数据结构(C 语言版)(附光盘),水浒传,突破英文基础词汇,噼里啪啦丛书(全 7 册))
```

11.求 MEDIAN 集函数

例 求按照 type 分组之后，各组内 nowprice 的中位数。

```
SELECT MEDIAN(nowprice)FROM PRODUCTION.PRODUCT   group by(type);
```

查询结果如下:

```
MEDIAN(NOWPRICE)
```

```
----------------
17.6
18.45
```

12.求线性回归相关 REGR 集函数

例 1 以 ORIGINALPRICE 为自变量，NOWPRICE 为因变量，对 ORIGINALPRICE 和 NOWPRICE 进行线性回归分析，求有效数据行数，自变量均值，因变量均值。

```
SELECT REGR_COUNT(NOWPRICE, ORIGINALPRICE) AS COUNT, REGR_AVGX(NOWPRICE,
ORIGINALPRICE) AS AVGX, REGR_AVGY(NOWPRICE, ORIGINALPRICE) AS AVGY FROM
PRODUCTION.PRODUCT;
```

查询结果如下：

```
行号        COUNT                    AVGX    AVGY
---------- -------------------- ----- ----
1           10                       29.35 20.5
```

例 2 以 ORIGINALPRICE 为自变量，NOWPRICE 为因变量，对 ORIGINALPRICE 和 NOWPRICE 进行线性回归分析，求斜率，因变量截距，相关系数。

```
SELECT REGR_SLOPE(NOWPRICE, ORIGINALPRICE) AS SLOPE, REGR_INTERCEPT(NOWPRICE,
ORIGINALPRICE) AS INTERCEPT,REGR_R2(NOWPRICE, ORIGINALPRICE) AS R2 FROM
PRODUCTION.PRODUCT;
```

查询结果如下：

```
行号        SLOPE                    INTERCEPT                R2
--------  ------------------   ---------- ---------    --------------------
1           0.685789018777860978231048346339420718563 1
0.372092298869780288918731034938001910 2
```

```
0.92691704152872502900730373397040711282 67
```

例 3 以 ORIGINALPRICE 为自变量，NOWPRICE 为因变量，对 ORIGINALPRICE 和 NOWPRICE 进行线性回归分析，求三种诊断统计量。

```
SELECT REGR_SXX(nowprice, originalprice) AS SXX, REGR_SYY(nowprice, originalprice) AS
SYY,REGR_SXY(nowprice, originalprice) AS SXY FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
行号          SXX         SYY        SXY
---------- -------- ------- -------
1             2423.865 1229.84 1662.26
```

13.百分比集函数 PERCENTILE_CONT、PERCENTILE_DISC

例 PERCENTILE_CONT、PERCENTILE_DISC 的使用

数据准备如下：

```
// dm.ini 中 USE_PLN_POOL 设为 0
drop table test_percentile;
create table test_percentile(c1 int, c2 int, c3 double);
INSERT INTO test_percentile SELECT
trunc(DBMS_RANDOM.VALUE(1,10)),trunc(DBMS_RANDOM.VALUE(1,10)),trunc(DBMS_RANDOM.
VALUE(0,10)/10.0) from dual connect by level <=1000;
commit;
```

查询 C1 排序之后排在 30%的数据。

```
select percentile_cont(0.3) within group(order by c1) from test_percentile;
select percentile_disc(0.3) within group(order by c1) from test_percentile;
```

结果如下:

```
行号        PERCENTILE_CONT(0.3)WITHINGROUP(ORDERBYC1ASC)

---------- ---------------------------------------------

1           3

行号        PERCENTILE_DISC(0.3)WITHINGROUP(ORDERBYC1ASC)

---------- ---------------------------------------------

1            3
```

14.位运算集函数 BIT_AND、BIT_OR、BIT_XOR

例 bit_and、big_or、bit_xor 的使用

数据准备:

```
create table test(c1 int, c2 int);

insert into test values(1, 1), (1, 3), (2, 5), (2, 7);

commit;
```

查询结果:

```
SQL> select c1, bit_and(c2), bit_or(c2), bit_xor(c2) from test group by c1;



行号        C1             BIT_AND(C2)             BIT_OR(C2)              BIT_XOR(C2)

---------- ----------- -------------------- --------------------

--------------------

1            1             1                       3                       2

2            2             5                       7                       2
```

4.1.4 分析函数

分析函数主要用于计算基于组的某种聚合值。

DM 分析函数为用户分析数据提供了一种更加简单高效的处理方式。如果不使用分析函数，则必须使用连接查询、子查询或者视图，甚至复杂的存储过程实现。引入分析函数后，只需要简单的 SQL 语句，并且执行效率方面也有大幅提高。

与集函数的主要区别是，分析函数对于每组返回多行数据。多行形成的组称为窗口，窗口决定了执行当前行的计算范围，窗口的大小可以由组中定义的行数或者范围值滑动。

4.1.4.1 函数分类

分析函数可按照如下方式分类：

1．COUNT(*)；

2．完全分析函数 AVG|MAX|MIN| COUNT|SUM([ALL]<值表达式>)，这 5 个分析函数的参数和作为集函数时的参数一致；

3．方差函数 VAR_POP、VAR_SAMP、VARIANCE、STDDEV_POP、STDDEV_SAMP、STDDEV；

4．协方差函数 COVAR_POP、COVAR_SAMP、CORR；

5．FIRST/LAST 分析函数；

6．相邻函数 LAG 和 LEAD；

7．首尾函数 FIRST_VALUE、LAST_VALUE；

8．分组函数 NTILE；

9．排序函数 RANK、DENSE_RANK、ROW_NUMBER；

10．百分比函数 PERCENT_RANK、CUME_DIST、RATIO_TO_REPORT、PERCENTILE_CONT、PERCENTILE_DISC；

11．字符串函数 LISTAGG、WM_CONCAT；

12．指定行函数 NTH_VALUE；

13．中位数函数 MEDIAN；

14．线性回归曲线斜率函数 REGR_SLOPE。

4.1.4.2 使用说明

1. 分析函数只能出现在选择项或者 ORDER BY 子句中；

2. 分析函数中有 DISTINCT 的时候，除 MIN、MAX 外，不允许与 ORDER BY 一起使用；

3. 分析函数参数、PARTITION BY 项和 ORDER BY 项中不允许使用分析函数，即不允许嵌套；

4. <PARTITION BY 项>为分区子句，表示对结果集中的数据按指定列进行分区。不同的区互不相干。当 PARTITION BY 项包含常量表达式时，表示以整个结果集分区；当省略 PARTITION BY 项时，将所有行视为一个分组；

5. <ORDER BY 项>为排序子句，对经<PARTITION BY 项>分区后的各分区中的数据进行排序。ORDER BY 项中包含常量表达式时，表示以该常量排序，即保持原来结果集顺序；

6. <窗口子句>为分析函数指定的窗口。窗口就是分析函数在每个分区中的计算范围；<窗口子句>必须和<ORDER BY 子句>同时使用；

7. AVG、COUNT、MAX、MIN、SUM 这 5 类分析函数的参数和返回的结果集的数据类型与对应的集函数保持一致，详细参见 4.1.3 小节 集函数部分；

8. 只有 MIN、MAX、COUNT、SUM、AVG、STDDEV、VARIANCE 的参数支持 DISTINCT，其他分析函数的参数不允许为 DISTINCT；

9. FIRST_VALUE 分析函数返回组中数据窗口的第一个值，LAST_VALUE 表示返回组中数据窗口 ORDER BY 项相同的最后一个值；

10. FIRST_VALUE/LAST_VALUE/LAG/LEAD/NTH_VALUE 函数支持 RESPECT|IGNORE NULLS 子句，该子句用来指定计算中是否跳过 NULL 值；

11. NTH_VALUE 函数支持 FROM FIRST/LAST 子句，该子句用来指定计算中是从第一行向后还是最后一行向前。

4.1.4.3 具体用法

分析函数的使用，按以下几种情况。

4.1.4.3.1 一般分析函数

分析函数的分析子句语法如下：

```
<分析函数>::=<函数名>(<参数>) OVER (<分析子句>)

<分析子句>::= [<PARTITION BY 项>] [<ORDER BY 项> [<窗口子句>]]

<PARTITION BY 项>::= PARTITION BY <<常量表达式>| <列名>>

<ORDER BY 项>::= ORDER BY <<常量表达式>| <列名>>

<窗口子句>::=<ROWS | RANGE >  < <范围子句 1>|<范围子句 2> >

<范围子句 1>::=

BETWEEN

    {<UNBOUNDED PRECEDING>|<CURRENT ROW>|<value_expr <PRECEDING|FOLLOWING> >}

    AND

    {<UNBOUNDED FOLLOWING>|<CURRENT ROW>|<value_expr <PRECEDING|FOLLOWING> >}

<范围子句 2>::=<UNBOUNDED PRECEDING>|<CURRENT ROW>| <value_expr PRECEDING>

<函数名>::=见下表
```

窗口子句：不是所有的分析函数都可以使用窗口。其对应关系如下表所示：

表 4.1.4 分析函数窗口列表

序号	函数名	是否为集函数	是否允许使用窗口子句
1	AVG	Y	Y
2	CORR	Y	Y

3	COUNT	Y	Y
4	COVAR_POP	Y	Y
5	COVAR_SAMP	Y	Y
6	CUME_DIST	Y	N
7	DENSE_RANK	N	N
8	FIRST	Y	Y
9	FIRST_VALUE	Y	Y
10	LAG	N	N
11	LAST	Y	Y
12	LAST_VALUE	N	Y
13	LEAD	N	N
14	LISTAGG	Y	N
15	NTH_VALUE	N	Y
16	MAX	Y	Y
17	MIN	Y	Y
18	NTILE	N	N
19	PERCENT_RANK	N	N
20	PERCENTILE_CONT	Y	N
21	PERCENTILE_DISC	Y	N
22	RANK	N	N
23	RATIO_TO_REPORT	N	N
24	ROW_NUMBER	N	N
25	STDDEV	Y	Y

26	STDDEV_POP	Y	Y
27	STDDEV_SAMP	Y	Y
28	SUM	Y	Y
29	VAR_POP	Y	Y
30	VAR_SAMP	Y	Y
31	VARIANCE	Y	Y
32	WM_CONCAT	Y	N
33	MEDIAN	Y	N
34	REGR_SLOPE	Y	Y

<窗口子句>通过指定滑动方式和<范围子句>两项来共同确定分析函数的计算窗口。每个分区的第一行开始往下滑动。

- 滑动方式有两种：ROWS 和 RANGE。
  - ROWS

ROWS 用来指定窗口的物理行数。ROWS 根据 ORDER BY 子句排序后，取的前 value_expr 行或后 value_expr 行的数据进行计算。与当前行的值无关，只与排序后的行号有关。

对于 ROWS 来说，value_expr 必须是一个可以计算的非负数值类型的表达式或常量。

  - RANGE

RANGE 用来指定窗口的逻辑偏移，即指定行值的取值范围。只要行值处于 RANGE 指定的取值范围内，该行就包含在窗口中。

1） 逻辑偏移值（value_expr）必须为非负的常量、表达式或者 NUMERIC 类型数值；

2） <ORDER BY 子句>中如果使用表达式，那么只能声明一个表达式；

3） value_expr 类型和 ORDER BY expr 类型应为相同的或可隐式转换计算的。

■ <范围子句>用来指定具体的窗口范围。ROW 和 RANGE 中用法不同，下面分别介绍。

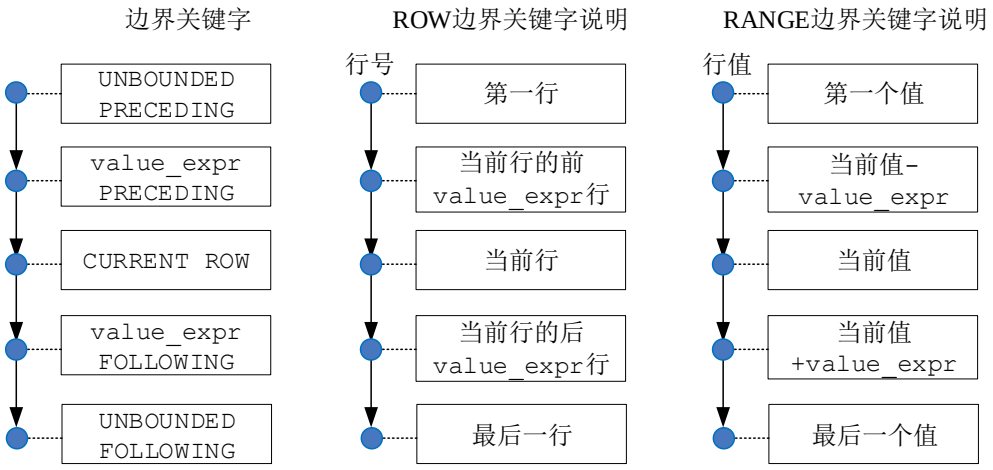


图 4.1 窗口边界关键字用法

● <范围子句>中的窗口范围边界关键字在 ROW 的用法介绍：

1） UNBOUNDED PRECEDING 窗口的边界是分区中的第一行或第一个值；

2） UNBOUNDED FOLLOWING 窗口的边界是分区中的最后一行或最后一个值；

3） CURRENT ROW 窗口的边界是当前行或者当前行的值；

4） value_expr PRECEDING 窗口的边界是当前行向前滑动 value_expr 的行或当前值-value_expr 的值；

5） value_expr FOLLOWING 窗口的边界是当前行向后滑动 value_expr 的行或当前值+value_expr 的值。

● <范围子句>中的边界关键字在 RANGE 的用法介绍

1） UNBOUNDED PRECEDING 窗口的边界是分区中的第一个值；

2） UNBOUNDED FOLLOWING 窗口的边界是分区中的最后一个值；

3） CURRENT ROW 窗口的边界是当前值；

4） value_expr PRECEDING 窗口的边界是当前值-value_expr 的值；

5） value_expr FOLLOWING 窗口的边界是当前值+value_expr 的值。

- <范围子句>中的边界关键字的使用须知

1） BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW，表示该组的第一行到当前行，或表示第一个值到当前值；

2） BETWEEN CURRENT ROW AND CURRENT ROW，表示当前行到当前行，或表示当前值到当前值；

3） BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING，表示该组的第一行到最后一行，或表示第一个值到最后一个值；

4） UNBOUNDED PRECEDING，和 1)等价；

5） CURRENT ROW，和 2)等价；

6） value_expr PRECEDING，等价于 BETWEEN value_expr PRECEDING AND CURRENT ROW；

7） 如果省略<窗口子句>，缺省为 BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW；

8） BETWEEN … AND…：窗口的范围，如果只定义一个分支，其另一个分支为当前行 CURRENT ROW；

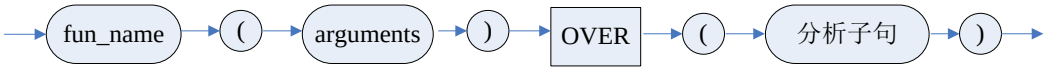
9） CURRENT ROW 用法中有两种特殊情况：一是当窗口以 CURRENT ROW 为开始位置时，窗口的结束点不能是 value_expr PRECEDING。二是当窗口以 CURRENT ROW 为结束位置时，窗口的起始点不能是 value_expr FOLLOWING。

10） value_expr PRECEDING 或 value_expr FOLLOWING 用法中有三种特殊情况：一是对于 ROWS 或 RANGE，如果 value_expr FOLLOWING 是起始位置，则结束位置也必须是 value_expr FOLLOWING；如果 value_expr PRECEDING 是结束位置，则起始位置必须是 value_expr PRECEDING。二是对于 ROWS，如果窗口函数的起始位置到结束位置没有记录，则分析函数的值返回 NULL。三是对于 RANGE，在<ORDER

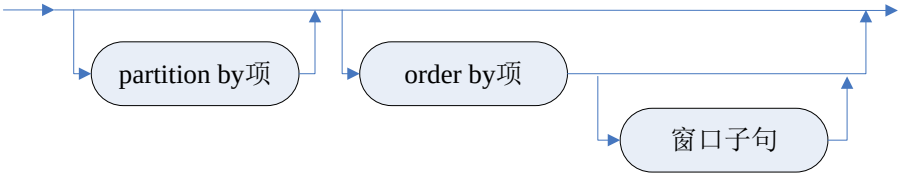
BY 子句>中，只能指定一个表达式，即排序列不能多于一个，对于 ROWS，则无此限制。

图例

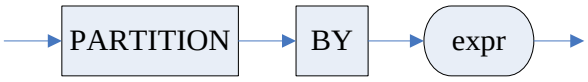
分析函数语法如下:



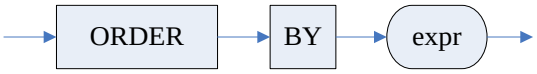
分析子句



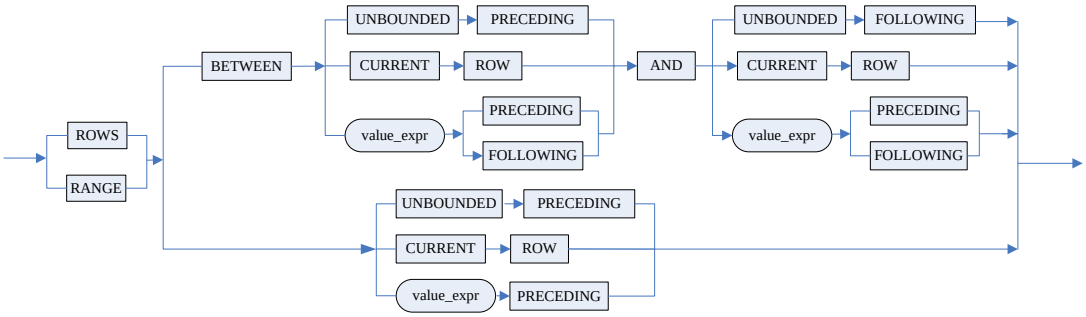
partition by 项



order by 项



窗口子句



4.1.4.3.2 FIRST/LAST 函数

FIRST 和 LAST 作为分析函数时，计算方法和对应的集函数类似，只是一组返回多行。

语法格式

```
<函数名><参数> <KEEP 子句> OVER ([<PARTITION BY 项>])

<KEEP 子句>::= KEEP (DENSE_RANK <FIRST | LAST> <ORDER BY 项>)

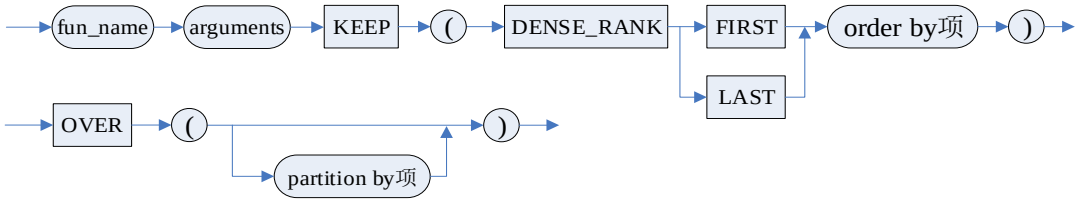
<函数名> ::= AVG | MAX | MIN | COUNT | SUM
```

参数

<KEEP 子句> 首先根据<ORDER BY 项>进行排序，然后根据 FIRST/LAST 计算出第一名（最小值）/最后一名（最大值）的函数值，排名按照奥林匹克排名法。

图例

FIRST 和 LAST 分析函数语法如下:



4.1.4.3.3 LAG 和 LEAD 函数

LAG 分析函数表示返回组中和当前行向前相对偏移 offset 行的参数的值，LEAD 方向相反，表示向后相对偏移。如果超出组的总行数范围，则返回 DEFAULT 值。

语法格式

```
<LAG|LEAD> <参数选项 1|参数选项 2> OVER ([<PARTITION BY 项>] <ORDER BY 项>)

<参数选项 1> ::= (<参数>[,<offset>[,<default>]])[<RESPECT|IGNORE> NULLS]

<参数选项 2> ::= (<参数>[<RESPECT|IGNORE> NULLS] [,<offset>[,<default>]])
```

参数

<offset> 为常量或表达式，类型为整型，默认为 1;

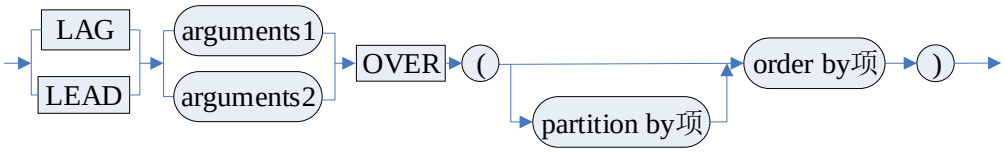
<default> 不在 offset 偏移范围内的默认值，为常量或表达式，和 LAG 和 LEAD 的参数数据类型一致;

<RESPECT|IGNORE> NULLS 计算中是否跳过 NULL 值，RESPECT NULLS 为不跳过，

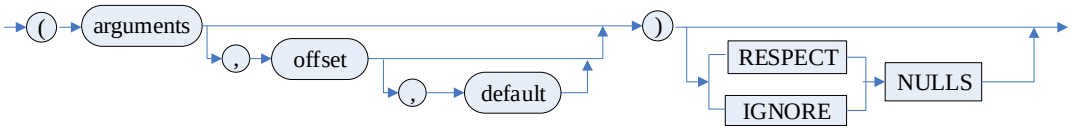
IGNORE NULLS 为跳过，默认值为 RESPECT NULLS。

图例

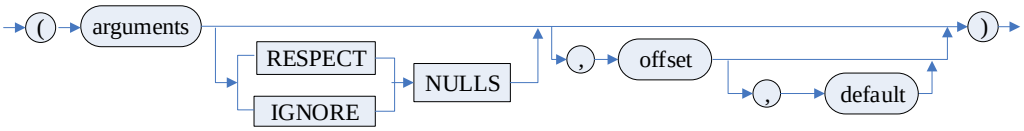
LAG 和 LEAD 函数



arguments1



arguments2



4.1.4.3.4 FIRST_VALUE 和 LAST_VALUE 函数

FIRST_VALUE 返回排序数据集合的第一行，LAST_VALUE 返回其最后一行。

语法格式

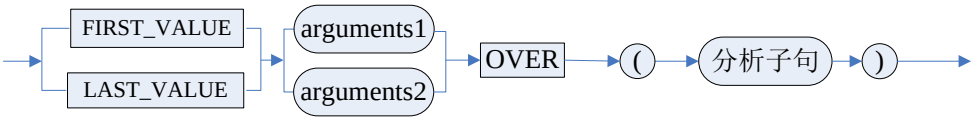
```
<FIRST_VALUE|LAST_VALUE> <参数选项 1|参数选项 2> OVER (<分析子句>)

<参数选项 1> ::= (<参数>) [<RESPECT | IGNORE> NULLS ]

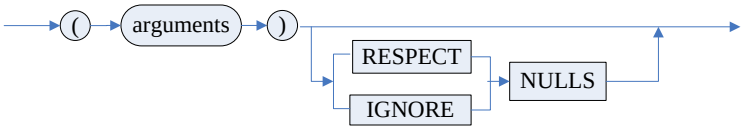
<参数选项 2> ::= (<参数> [<RESPECT | IGNORE> NULLS ])
```

图例

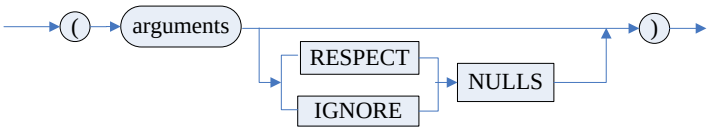
FIRST_VALUE 和 LAST_VALUE 函数



arguments1



arguments2



4.1.4.3.5 PERCENTILE_CONT 和 PERCENTILE_DISC 函数

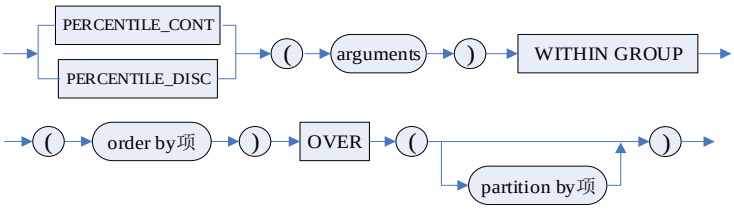
连续百分比 PERCENTILE_CONT 和分布百分比 PERCENTILE_DISC 分析函数。

语法格式

```
<PERCENTILE_CONT|PERCENTILE_DISC> (< 参 数 >) WITHIN GROUP(<ORDER BY 项 >) OVER ([<PARTITION BY 项>])
```

图例

PERCENTILE_CONT 和 PERCENTILE_DISC 函数



4.1.4.3.6 LISTAGG 函数

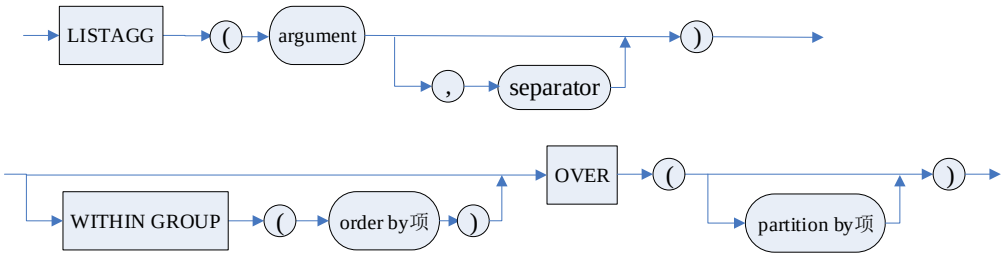
字符串分析函数 LISTAGG 按照指定的 PARTITION BY 项进行分组，组内按照 ORDER BY 项排序（没有指定排序则按数据组织顺序），将组内的参数通过分隔符拼接起来，返回的结果集行数为组数。

语法格式

```
LISTAGG (<参数> [,<分隔符>]) [WITHIN GROUP(<ORDER BY 项>)] OVER ([<PARTITION BY 项>])
```

图例

LISTAGG 函数



4.1.4.3.7 NTH_VALUE 函数

指定行分析函数 NTH_VALUE 按照指定的 PARTITION BY 项进行分组，组内按照 ORDER BY 项排序，返回组内结果集的指定行的数据。

语法格式

```
NTH_VALUE (<参数 1> ,<参数 2>) [FROM <FIRST | LAST>] [<RESPECT | IGNORE> NULLS] OVER
([<PARTITION BY 项>] [<ORDER BY 项> [<窗口子句>]])
```

<PARTITION BY 项>、<ORDER BY 项>、<窗口子句>请参考 4.1.4.3.1 一般分析函数

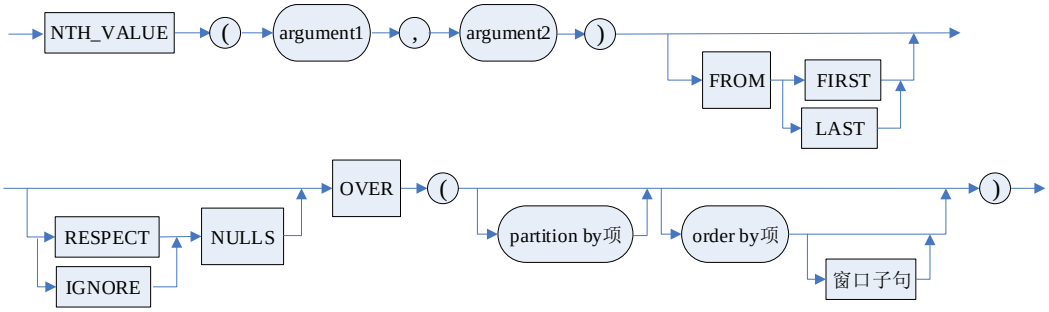
参数

1. FROM <FIRST | LAST>：指定组内数据方向，FROM FISRT 指定从第一行往后，FROM LAST 指定从最后一行往前，默认值为 FROM FIRST；

2. <RESPECT|IGNORE> NULLS：计算中是否跳过 NULL 值，RESPECT NULLS 为不跳过，IGNORE NULLS 为跳过，默认值为 RESPECT NULLS。

图例

NTH_VALUE 函数



4.1.4.3.8 WM_CONCAT 函数

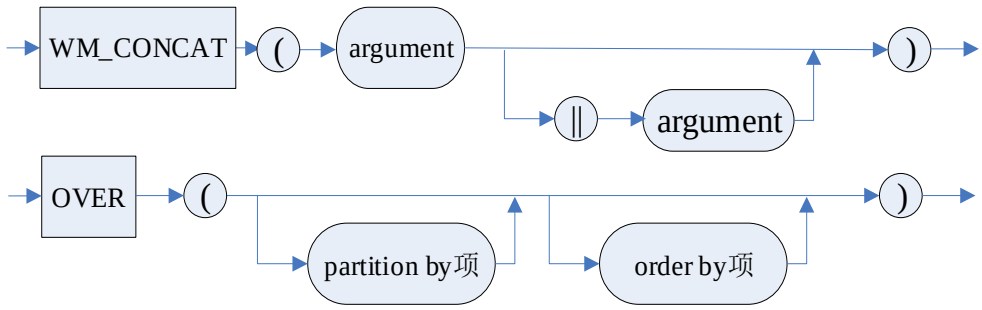
字符串分析函数 WM_CONCAT 按照指定的 PARTITION BY 项进行分组，组内按照 ORDER BY 项排序，然后将组内排序后的指定参数用“,”拼接起来，返回的结果集行数为组数。不支持 WITH IN 子句。

语法格式

```
WM_CONCAT (<参数> [|| <参数>]) OVER ([<PARTITION BY 项>][<ORDER BY 项>])
```

图例

WM_CONCAT 函数



4.1.4.3.9 MEDIAN 函数

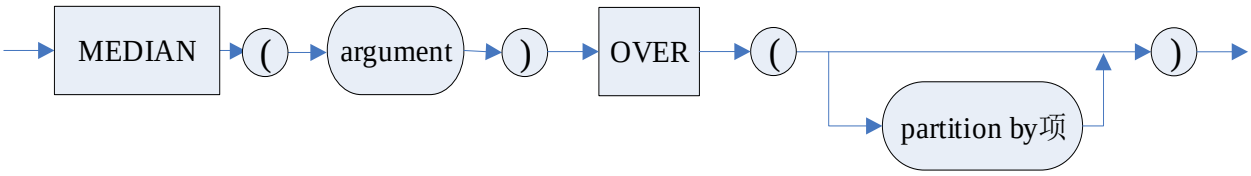
中位数计算函数 MEDIAN 按照指定的 PARTITION BY 项进行分组，不支持 WITH IN 子句，计算组内参数的中位数，返回的结果集行数为组数。

语法格式

```
MEDIAN (<参数>) OVER ([<PARTITION BY 项>])
```

图例

MEDIAN 函数



4.1.4.4 举例说明

下面按分析函数的功能分别举例说明。

1. 最大值 MAX 和最小值 MIN

例 查询折扣大于 7 的图书作者以及最大折扣。

```
SELECT  AUTHOR,  MAX(DISCOUNT) OVER (PARTITION BY AUTHOR)  AS MAX
FROM    PRODUCTION.PRODUCT
WHERE   DISCOUNT > 7;
```

查询结果如下:

```
AUTHOR              MAX
------------    ---
曹雪芹,高鹗         8.0
施耐庵，罗贯中    7.5
严蔚敏，吴伟民    8.5
```

需要说明的是：如果使用的是集函数 MAX，那么得到的是所有图书中折扣的最大值，并不能查询出作者，使用了分析函数，就可以对作者进行分区，得到每个作者所写的图书中折扣最大的值。MIN 的含义和 MAX 类似。

2. 平均值 AVG 和总和 SUM

例 1 求折扣小于 7 的图书作者和平均价格。

```
SELECT  AUTHOR,  AVG(NOWPRICE) OVER (PARTITION BY AUTHOR) as AVG
FROM     PRODUCTION.PRODUCT
WHERE    DISCOUNT < 7;
```

查询结果如下:

```
AUTHOR              AVG
```

```
------------ ----

(日)佐佐木洋子    42

陈满麒            11.4

海明威            6.1

金庸              21.7

鲁迅              20

王树增            37.7
```

例 2 求折扣大于 8 的图书作者和书的总价格。

```
SELECT    AUTHOR,   SUM(NOWPRICE) OVER (PARTITION BY AUTHOR)   as SUM

FROM      PRODUCTION.PRODUCT

WHERE     DISCOUNT >8;
```

查询结果如下:

```
AUTHOR            SUM

------------ ----

严蔚敏，吴伟民    25.5
```

3. 样本个数 COUNT

例 查询信用级别为“很好”的已登记供应商的名称和个数。

```
SELECT   NAME,   COUNT(*) OVER (PARTITION   BY   CREDIT) AS CNT

FROM     PURCHASING.VENDOR

WHERE    CREDIT = 2;
```

查询结果如下:

```
NAME              CNT

------------ -------------
```

```
长江文艺出版社     2
上海画报出版社     2
```

由此例可看出，COUNT(*)的结果是 VENDOR 表中的按 CREDIT 分组后的总行数。

4. 分析函数总体协方差 COVAR_POP

例 求产品原始价格 ORIGINALPRICE 和当前销售价格 NOWPRICE 的总体协方差。

```
SELECT    PUBLISHER,
COVAR_POP(ORIGINALPRICE, NOWPRICE) OVER(PARTITION BY PUBLISHER) AS COVAR_POP
FROM    PRODUCTION.PRODUCT;
```

查询结果如下：

```
PUBLISHER                 COVAR_POP
------------------------- ---------
                                  0
21 世纪出版社                     0
广州出版社                        0
机械工业出版社                    0
清华大学出版社                    0
人民文学出版社                    0
上海出版社                        0
外语教学与研究出版社              0
中华书局                          0
中华书局                          0
```

5. 分析函数样本协方差 COVAR_SAMP

例 求产品原始价格 ORIGINALPRICE 和当前销售价格 NOWPRICE 的样本协方差。

```
SELECT  PUBLISHER,
COVAR_SAMP(ORIGINALPRICE, NOWPRICE) OVER(PARTITION BY PUBLISHER) AS COVAR_SAMP
FROM  PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                COVAR_SAMP
------------------ ----------
                              NULL
21 世纪出版社              NULL
广州出版社                  NULL
机械工业出版社              NULL
清华大学出版社              NULL
人民文学出版社              NULL
上海出版社                  NULL
外语教学与研究出版社        NULL
中华书局                    0
中华书局                    0
```

6. 相关系数 CORR

例 求产品原始价格 ORIGINALPRICE 和当前销售价格 NOWPRICE 的相关系数。

```
SELECT  PUBLISHER,
CORR(ORIGINALPRICE, NOWPRICE) OVER(PARTITION BY PUBLISHER) AS CORR
FROM  PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                CORR
```

```
---------- ----------

                    NULL
21 世纪出版社       NULL
广州出版社          NULL
机械工业出版社      NULL
清华大学出版社      NULL
人民文学出版社      NULL
上海出版社          NULL
外语教学与研究出版社 NULL
中华书局            NULL
中华书局            NULL
```

7. 排名 RANK、DENSE_RANK 和 ROW_NUMBER

例 求按销售额排名的销售代表对应的雇员号和排名。

```
SELECT EMPLOYEEID, RANK() OVER (ORDER BY SALESLASTYEAR) AS RANK FROM
SALES.SALESPERSON;
```

查询结果如下:

```
EMPLOYEEID  RANK
----------  ----------
4           1
5           2
```

RANK()排名函数按照指定 ORDER BY 项进行排名，如果值相同，则排名相同，例如销售额相同的排名相同，该函数使用非密集排名，例如两个第 1 名后，下一个就是第 3 名；与之对应的是 DENSE_RANK()，表示密集排名，例如两个第 1 名之后，下一个就是第 2 名。

ROW_NUMBER()表示按照顺序编号，不区分相同值，即从 1 开始编号。

8. FIRST 和 LAST

例 求每个用户最早定的商品中花费最多和最少的金额以及用户当前的花费金额。

```
SELECT CUSTOMERID, TOTAL,
        MAX(TOTAL) KEEP (DENSE_RANK FIRST ORDER BY ORDERDATE) OVER (PARTITION BY
CUSTOMERID) MAX_VAL,
        MIN(TOTAL) KEEP (DENSE_RANK FIRST ORDER BY ORDERDATE) OVER (PARTITION BY
CUSTOMERID) MIN_VAL
FROM SALES.SALESORDER_HEADER;
```

查询结果如下：

```
CUSTOMERID    TOTAL      MAX_VAL    MIN_VAL
----------- -------    -------    -------
1              36.9000    36.9000    36.9000
1              36.9000    36.9000    36.9000
```

FIRST 和 LAST 分析函数计算方法和对应的集函数类似，作为分析函数时一组返回多行，而集函数只返回一行。

9. FIRST_VALUE 和 LAST_VALUE 分析函数

例 求花费最多和最少金额的用户和花费金额。

```
SELECT NAME, TOTAL,
     FIRST_VALUE(NAME) OVER (ORDER BY TOTAL) FIRST_PERSON,
     LAST_VALUE(NAME) OVER (ORDER BY TOTAL) LAST_PERSON
FROM SALES.SALESORDER_HEADER S,SALES.CUSTOMER C,PERSON.PERSON P
WHERE S.CUSTOMERID = C.CUSTOMERID AND C.PERSONID = P.PERSONID;
```

查询结果如下:

```
NAME   TOTAL      FIRST_PERSON LAST_PERSON
----   -------    ------------ -----------
刘青   36.9000    刘青         刘青
刘青   36.9000    刘青         刘青
```

FIRST_VALUE 返回一组中的第一行数据，LAST_VALUE 相反，返回组中的最后一行数据。根据 ORDER BY 项就可以返回需要的列的值。

10. LAG 和 LEAD

例 求当前订单的前一个和下一个订单的销售总额。

```
SELECT
    ORDERDATE,
    LAG(TOTAL, 1, 0) OVER (ORDER BY ORDERDATE) PRV_TOTAL,
    LEAD(TOTAL, 1, 0) OVER (ORDER BY ORDERDATE) NEXT_TOTAL
FROM   SALES.SALESORDER_HEADER;
```

查询结果如下:

```
ORDERDATE    PRV_TOTAL NEXT_TOTAL
----------   --------- ----------
2007-05-06   0         36.9
2007-05-07   36.9      0
```

LAG 返回当前组的前一个订单日期的 TOTAL 值，如果超出该组，则返回 DEFAULT 值 0。

11. 窗口的使用

例 按照作者分类，求到目前为止图书价格最贵的作者和价格。

```
SELECT AUTHOR,
```

```
    MAX(NOWPRICE) OVER(PARTITION BY AUTHOR ORDER BY NOWPRICE ROWS
    UNBOUNDED PRECEDING) AS MAX_PRICE
FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
AUTHOR              MAX_PRICE
------------  ---------
(日)佐佐木洋子    42.0000
曹雪芹,高鹗        15.2000
陈满麒              11.4000
海明威              6.1000
金庸                21.7000
刘毅                11.1000
鲁迅                20.0000
施耐庵，罗贯中    14.3000
王树增              37.7000
严蔚敏，吴伟民    25.5000
```

分析函数中的窗口限定了计算的范围，ROWS UNBOUNDED PRECEDING 表示该组的第一行开始到当前行，等价于 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW。

12. 标准差 STDDEV

例 求每个出版社图书现价的标准差。

```
SELECT PUBLISHER, STDDEV(NOWPRICE) OVER(PARTITION BY PUBLISHER) AS STDDEV FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                STDDEV
------------------------ ------------------
                                  0
21 世纪出版社                     0
广州出版社                        0
机械工业出版社                    0
清华大学出版社                    0
人民文学出版社                    0
上海出版社                        0
外语教学与研究出版社              0
中华书局                          0.636396103067893
中华书局                          0.636396103067893
```

13. 样本标准差 STDDEV_SAMP

例 求每个出版社图书现价的样本标准差。

```
SELECT PUBLISHER, STDDEV_SAMP(NOWPRICE) OVER(PARTITION BY PUBLISHER) AS
STDDEV_SAMP FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                STDDEV_SAMP
------------------------ ------------------
                                  NULL
21 世纪出版社                     NULL
广州出版社                        NULL
```

```
机械工业出版社          NULL
清华大学出版社          NULL
人民文学出版社          NULL
上海出版社              NULL
外语教学与研究出版社    NULL
中华书局                0.636396103067893
中华书局                0.636396103067893
```

14. 总体标准差 STDDEV_POP

例 求每个出版社图书现价的总体标准差。

```
SELECT PUBLISHER, STDDEV_POP (NOWPRICE) OVER(PARTITION BY PUBLISHER) AS
STDDEV_POP FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER              STDDEV_POP
------------------ ----------
                            0
21 世纪出版社               0
广州出版社                  0
机械工业出版社              0
清华大学出版社              0
人民文学出版社              0
上海出版社                  0
外语教学与研究出版社        0
中华书局                    0.45
```

```
中华书局                    0.45
```

15. 样本方差 VAR_SAMP

例 求每个出版社图书现价的样本方差。

```
SELECT PUBLISHER, VAR_SAMP(NOWPRICE) OVER(PARTITION BY PUBLISHER) AS VAR_SAMP
FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                VAR_SAMP
------------------ --------

                           NULL
21 世纪出版社              NULL
广州出版社                 NULL
机械工业出版社             NULL
清华大学出版社             NULL
人民文学出版社             NULL
上海出版社                 NULL
外语教学与研究出版社       NULL
中华书局                   0.405
中华书局                   0.405
```

16. 总体方差 VAR_POP

例 求每个出版社图书现价的总体方差。

```
SELECT PUBLISHER , VAR_POP(NOWPRICE) OVER(PARTITION BY PUBLISHER) AS VAR_POP FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                VAR_POP
------------------   -------
                            0
21 世纪出版社              0
广州出版社                 0
机械工业出版社             0
清华大学出版社             0
人民文学出版社             0
上海出版社                 0
外语教学与研究出版社       0
中华书局                   0.2025
中华书局                   0.2025
```

17. 方差 VARIANCE

例 求每个出版社图书现价的方差。

```
SELECT PUBLISHER, VARIANCE (NOWPRICE) OVER(PARTITION BY PUBLISHER) AS VARIANCE
FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                VARIANCE
------------------ --------
                            0
21 世纪出版社              0
广州出版社                 0
机械工业出版社             0
```

```
清华大学出版社          0
人民文学出版社          0
上海出版社              0
外语教学与研究出版社    0
中华书局                0.405
中华书局                0.405
```

18. 分组 NTILE

例 根据图书的现价将图书分成三个组。

```
SELECT  NAME, NTILE (3)  OVER(ORDER BY NOWPRICE)  AS NTILE FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                                NTILE
--------------------------  --------------------
老人与海                            1
突破英文基础词汇                    1
工作中无小事                        1
水浒传                              1
红楼梦                              2
鲁迅文集(小说、散文、杂文)全两册    2
射雕英雄传(全四册)                  2
数据结构(C 语言版)(附光盘)          3
长征                                3
噼里啪啦丛书(全 7 册)               3
```

19. 排列百分比 PERCENT_RANK

例 求图书的现价排列百分比。

```
SELECT NAME, PERCENT_RANK() OVER(ORDER BY NOWPRICE) AS PERCENT_RANK FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                                PERCENT_RANK
------------------------------ -------------------------
老人与海                            0.000000000000000E+000
突破英文基础词汇                        1.111111111111111E-001
工作中无小事                          2.222222222222222E-001
水浒传                             3.333333333333333E-001
红楼梦                             4.444444444444444E-001
鲁迅文集(小说、散文、杂文)全两册  5.555555555555556E-001
射雕英雄传(全四册)                    6.666666666666666E-001
数据结构(C 语言版)(附光盘)          7.777777777777778E-001
长征                               8.888888888888888E-001
噼里啪啦丛书(全 7 册)                1.000000000000000E+000
```

20. 连续百分比对应的值 PERCENTILE_CONT

例 求连续百分比占 0.5 对应的图书现价值。

```
SELECT NAME, PERCENTILE_CONT(0.5) WITHIN GROUP(ORDER BY NOWPRICE) OVER() AS
PERCENTILE_CONT FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                                PERCENTILE_CONT
```

```
--------------------------  ---------------

老人与海                    17.6

突破英文基础词汇            17.6

工作中无小事                17.6

水浒传                      17.6

红楼梦                      17.6

鲁迅文集(小说、散文、杂文)全两册  17.6

射雕英雄传(全四册)          17.6

数据结构(C 语言版)(附光盘)   17.6

长征                        17.6

噼里啪啦丛书(全 7 册)       17.6
```

21. 分布百分比对应的值 PERCENTILE_DISC

例 求分布百分比占 0.5 对应的图书现价值。

```
SELECT NAME, PERCENTILE_DISC(0.5) WITHIN GROUP(ORDER BY NOWPRICE) OVER() AS
PERCENTILE_DISC FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                        PERCENTILE_DISC

--------------------------  ---------------

老人与海                    15.2000

突破英文基础词汇            15.2000

工作中无小事                15.2000

水浒传                      15.2000

红楼梦                      15.2000
```

```
鲁迅文集(小说、散文、杂文)全两册     15.2000
射雕英雄传(全四册)                    15.2000
数据结构(C 语言版)(附光盘)            15.2000
长征                                  15.2000
噼里啪啦丛书(全 7 册)                 15.2000
```

22. 累计百分比 CUME_DIST

例 求图书现价的累计百分比。

```
SELECT  NAME, CUME_DIST()  OVER(ORDER BY NOWPRICE) AS CUME_DIST FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                          CUME_DIST
----------------------------- -------------------------
老人与海                      1.000000000000000E-001
突破英文基础词汇              2.000000000000000E-001
工作中无小事                  3.000000000000000E-001
水浒传                        4.000000000000000E-001
红楼梦                        5.000000000000000E-001
鲁迅文集(小说、散文、杂文)全两册   6.000000000000000E-001
射雕英雄传(全四册)            7.000000000000000E-001
数据结构(C 语言版)(附光盘)    8.000000000000000E-001
长征                          9.000000000000000E-001
噼里啪啦丛书(全 7 册)         1.000000000000000E+000
```

23. 某一样本值所占百分比 RATIO_TO_REPORT

例 求出版社每种图书现价所占的百分比。

```
SELECT  NAME, RATIO_TO_REPORT(NOWPRICE)  OVER(PARTITION BY PUBLISHER)  AS
RATIO_TO_REPORT FROM  PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                                RATIO_TO_REPORT
--------------------------  ----------------------------------------
鲁迅文集(小说、散文、杂文)全两册    1
噼里啪啦丛书(全 7 册)               1
射雕英雄传(全四册)                  1
工作中无小事                        1
数据结构(C 语言版)(附光盘)           1
长征                                1
老人与海                            1
突破英文基础词汇                    1
水浒传                              0.484745762711864406779661016949152542372 9
红楼梦                              0.515254237288135593220338983050847457627 1
```

24. 组内指定行 NTH_VALUE

例 1 求每个出版社第二贵的书的价格。

```
SELECT  PUBLISHER,
    NTH_VALUE(NOWPRICE, 2)
    FROM FIRST RESPECT NULLS
    OVER(PARTITION BY PUBLISHER ORDER BY NOWPRICE DESC)
    AS NTH_VALUE FROM  PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                NTH_VALUE
------------------------ ---------
                           NULL
21 世纪出版社              NULL
广州出版社                 NULL
机械工业出版社             NULL
清华大学出版社             NULL
人民文学出版社             NULL
上海出版社                 NULL
外语教学与研究出版社       NULL
中华书局                   NULL
中华书局                   14.3000
```

例 2 利用窗口子句求每个出版社第二贵的书的价格。

```
SELECT  PUBLISHER, NTH_VALUE(NOWPRICE, 2) FROM FIRST RESPECT NULLS  OVER(PARTITION BY
PUBLISHER ORDER BY NOWPRICE DESC ROWS UNBOUNDED PRECEDING)  AS NTH_VALUE FROM
PRODUCTION.PRODUCT;
```

查询结果同例 1。

25. 字符串分析函数 WM_CONCAT

例 求每个出版社出版的图书。先根据出版社进行分组，然后将每个出版社出版的图书按照销售价格进行排序，并将排好序的图书名称用“,”拼接起来。

```
SELECT PUBLISHER, WM_CONCAT(NAME) OVER (PARTITION BY PUBLISHER ORDER BY NOWPRICE
ASC) AS WM_CONCAT FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
PUBLISHER                 WM_CONCAT
------------------ --------------------------------
                           鲁迅文集(小说、散文、杂文)全两册
21 世纪出版社             噼里啪啦丛书(全 7 册)
广州出版社                射雕英雄传(全四册)
机械工业出版社            工作中无小事
清华大学出版社            数据结构(C 语言版)(附光盘)
人民文学出版社            长征
上海出版社                老人与海
外语教学与研究出版社      突破英文基础词汇
中华书局                  水浒传
中华书局                  水浒传,红楼梦
```

26. 计算中位数 MEDIAN

例 求图书作者和其所著图书价格的中位数。先根据 PARTITION BY 项进行分组，然后计算组内参数的中位数。

```
SELECT AUTHOR, MEDIAN(NOWPRICE) OVER (PARTITION BY AUTHOR) as MED FROM
PRODUCTION.PRODUCT;
```

查询结果如下:

```
AUTHOR              MED
-------------- ----
(日)佐佐木洋子     42
曹雪芹,高鹗        15.2
```

```
陈满麒              11.4
海明威              6.1
金庸                21.7
刘毅                11.1
鲁迅                20
施耐庵，罗贯中      14.3
王树增              37.7
严蔚敏，吴伟民      25.5
```

27. 线性回归曲线斜率 REGR_SLOPE

例 以原始价格 ORIGINALPRICE 为自变量，现价 NOWPRICE 为因变量，对 ORIGINALPRICE 和 NOWPRICE 进行线性回归分析，求斜率。

```
SELECT TYPE, REGR_SLOPE(NOWPRICE, ORIGINALPRICE) OVER(PARTITION BY TYPE) AS
REGR_SLOPE FROM  PRODUCTION.PRODUCT;
```

查询结果如下：

```
TYPE  REGR_SLOPE
---- ----------------------------------------
16   0.6321292775665399239543726235741444866 92
16   0.6321292775665399239543726235741444866 92
16   0.6321292775665399239543726235741444866 92
16   0.6321292775665399239543726235741444866 92
16   0.6321292775665399239543726235741444866 92
16   0.6321292775665399239543726235741444866 92
8    0.7375224463071367911991027959833265265 11
```

```
8       0.737522446307136791199102795983326526511

8       0.737522446307136791199102795983326526511

8       0.737522446307136791199102795983326526511
```

28. LISTAGG 函数

例 查询公司雇员中所有的销售代表，查询到的结果用“,”进行分隔。

```
SELECT TITLE, LISTAGG(NAME, ',')

WITHIN GROUP (ORDER BY TITLE) NAME

FROM RESOURCES.EMPLOYEE E

INNER JOIN PERSON.PERSON P on E.PERSONID = P.PERSONID

WHERE TITLE = '销售代表'

GROUP BY TITLE;
```

查询结果如下:

```
TITLE          NAME

-------    -------

销售代表     郭艳,孙丽
```

4.1.5 情况表达式

<值表达式>可以为一个<列引用>、<集函数>、<标量子查询>或<情况表达式>等等。

<情况表达式>包括<情况缩写词>和<情况说明>两大类。<情况缩写词>包括函数 NULLIF 和 COALESCE，在 DM 中被划分为空值判断函数。具体函数说明请见 8.4 节。下面详细介绍<情况说明>表达式。

<CASE 情况说明>的语法和语义如下:

语法格式

```
<情况说明> ::= <简单情况> | <搜索情况>



<简单情况> ::= CASE

<值表达式>

{<简单 WHEN 子句>}

[<ELSE 子句>]

END



<搜索情况> ::= CASE

[<搜索 WHEN 子句>]

[<ELSE 子句>]

END

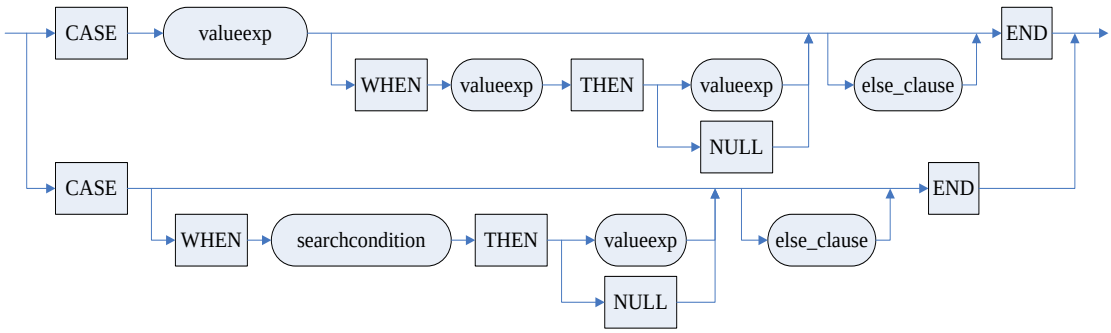
<简单 WHEN 子句> ::= WHEN <值表达式> THEN <结果>

<搜索 WHEN 子句> ::= WHEN <搜索条件> THEN <结果>

<结果> ::= <值表达式> | NULL
```

图例

情况表达式



功能

指明一个条件值。将搜索条件作为输入并返回一个标量值。

使用说明

1．在<情况说明>中至少有一个<结果>应该指明<值表达式>；

2．如果未指明<ELSE 子句>，则隐含 ELSE NULL；

3．<简单情况>中，CASE 运算数的数据类型必须与<简单 WHEN 子句>中的<值表达式>的数据类型是可比较的，且与 ELSE 子句的结果也是可比较的；

4．<情况说明>的数据类型由<结果>中的所有<值表达式>的数据类型确定；

1） 如果<结果>指明 NULL，则它的值是空值；

2） 如果<结果>指明<值表达式>，则它的值是该<值表达式>的值。

5．如果在<情况说明>中某个<搜索 WHEN 子句>的<搜索条件>为真，则<情况说明>的值是其<搜索条件>为真的第一个<搜索 WHEN 子句>的<结果>的值，并按照<情况说明>的数据类型来转换；

6．<搜索 WHEN 子句>中支持多列，如：

```
SELECT CASE WHEN (C1,C2) IN (SELECT C1,C2 FROM T2) THEN 1 ELSE 0   END FROM T1;
```

7．如果在<情况说明>中没有一个<搜索条件>为真，则<情况表达式>的值是其显式或隐式的<ELSE 子句>的<结果>的值，并按照<情况说明>的数据类型来转换。

举例说明

例 1 查询图书信息，如果当前销售价格大于 20 元，返回“昂贵”，如果当前销售价格小于等于 20 元，大于等于 10 元，返回“普通”，如果当前销售价格小于 10 元，返回“便宜”。

```
SELECT NAME,
      CASE
         WHEN NOWPRICE > 20 THEN   '昂贵'
         WHEN NOWPRICE <= 20 AND NOWPRICE >= 10 THEN '普通'
         ELSE '便宜'
      END AS  选择
```

```
FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
NAME                                选择
--------------------------- ----
红楼梦                              普通
水浒传                              普通
老人与海                            便宜
射雕英雄传(全四册)                  昂贵
鲁迅文集(小说、散文、杂文)全两册    普通
长征                                昂贵
数据结构(C 语言版)(附光盘)          昂贵
工作中无小事                        普通
突破英文基础词汇                    普通
噼里啪啦丛书(全 7 册)               昂贵
```

例 2 在 VERDOR 中如果 NAME 为中华书局或清华大学出版社，且 CREDIT 为 1 则返回“采购”，否则返回“考虑”。

```
SELECT NAME,
    CASE
        WHEN (NAME = '中华书局' OR NAME = '清华大学出版社') AND CREDIT = 1 THEN '采购'
         ELSE '考虑'
    END  AS 选择
FROM PURCHASING.VENDOR;
```

查询结果如下：

```
NAME                          选择

------------------ ----

上海画报出版社                考虑

长江文艺出版社                考虑

北京十月文艺出版社        考虑

人民邮电出版社                考虑

清华大学出版社                采购

中华书局                      采购

广州出版社                    考虑

上海出版社                    考虑

21 世纪出版社                 考虑

外语教学与研究出版社    考虑

机械工业出版社                考虑

文学出版社                    考虑
```

例 3 在上述结果中将 NAME 为中华书局，CREDIT 为 1 的元组返回。

```
SELECT NAME, CREDIT FROM PURCHASING.VENDOR

WHERE NAME IN (SELECT CASE

    WHEN CREDIT = 1 THEN '中华书局'

    ELSE 'NOT EQUAL'

    END

    FROM PURCHASING.VENDOR);
```

查询结果如下：

```
NAME          CREDIT

-------- -----------
```

```
中华书局     1
```

例 4 在上述结果中，若 CREDIT 大于 1 则修改该值为 1。

```
UPDATE PURCHASING.VENDOR SET CREDIT = CASE
                                WHEN CREDIT > 1 THEN 1
                                ELSE CREDIT
                                END;
SELECT NAME, CREDIT    FROM PURCHASING.VENDOR;
```

查询结果如下：

```
NAME                           CREDIT
------------------- -----------
上海画报出版社                 1
长江文艺出版社                 1
北京十月文艺出版社             1
人民邮电出版社                 1
清华大学出版社                 1
中华书局                       1
广州出版社                     1
上海出版社                     1
21 世纪出版社                  1
外语教学与研究出版社           1
机械工业出版社                 1
文学出版社                     1
```

4.2 连接查询

如果一个查询包含多个表(>=2)，则称这种方式的查询为连接查询。即<FROM 子句>中使用的是<连接表>。DM 的连接查询方式包括：交叉连接（cross join）、自然连接（natural join）、内连接（inner）、外连接（outer）。下面分别举例说明。

4.2.1 交叉连接

1. 无过滤条件

对连接的两张表记录做笛卡尔集，产生最终结果输出。

例 SALESPERSON 和 EMPLOYEE 通过交叉连接查询 HAIRDATE 和 SALESLASTYEAR。

```
SELECT T1.HAIRDATE, T2.SALESLASTYEAR FROM RESOURCES.EMPLOYEE T1 CROSS JOIN SALES.SALESPERSON T2;
```

查询结果如下：

```
HAIRDATE    SALESLASTYEAR

---------- -------------

2002-05-02 10.0000

2002-05-02 10.0000

2002-05-02 10.0000

2002-05-02 10.0000

2002-05-02 10.0000

2005-05-02 10.0000

2002-05-02 10.0000

2004-05-02 10.0000

2002-05-02 20.0000
```

```
2002-05-02 20.0000

2002-05-02 20.0000

2002-05-02 20.0000

2002-05-02 20.0000

2005-05-02 20.0000

2002-05-02 20.0000

2004-05-02 20.0000
```

2. 有过滤条件

对连接的两张表记录做笛卡尔集，根据 WHERE 条件进行过滤，产生最终结果输出。

例 查询性别为男性的员工的姓名与职务。

```
SELECT  T1.NAME, T2.TITLE FROM PERSON.PERSON T1, RESOURCES.EMPLOYEE T2 WHERE T1.PERSONID
= T2.PERSONID AND T1.SEX = 'M';
```

查询结果如下：

```
NAME TITLE

---- ----------

王刚  销售经理

李勇  采购经理

黄非  采购代表

张平  系统管理员
```

本例中的查询数据必须来自 PERSON 和 EMPLOYEE 两个表。因此，应在 FROM 子句中给出这两个表的表名(为了简化采用了别名)，在 WHERE 子句中给出连接条件(即要求两个表中 PERSONID 的列值相等)。当参加连接的表中出现相同列名时，为了避免混淆，可在这些列名前加表名前缀。

该例的查询结果是 PERSON 和 EMPLOYEE 在 PERSONID 列上做等值连接产生的。条件“T1.PERSONID = T2.PERSONID”称为连接条件或连接谓词。当连接运算符为“=”号时，称为等值连接，使用其它运算符则称非等值连接。

说明：

1. 连接谓词中的列类型必须是可比较的，但不一定要相同，只要可以隐式转换即可；
2. 不要求连接谓词中的列同名；
3. 连接谓词中的比较操作符可以是>、>=、<、<=、=、<>；
4. WHERE 子句中可同时包含连接条件和其它非连接条件。

4.2.2 自然连接(NATURAL JOIN)

把两张连接表中的同名列作为连接条件，进行等值连接，我们称这样的连接为自然连接。

自然连接具有以下特点：

1．连接表中存在同名列；

2．如果有多个同名列，则会产生多个等值连接条件；

3．如果连接表中的同名列类型不匹配，则报错处理。

例 查询销售人员的入职时间和去年销售总额。

```
SELECT HAIRDATE, SALESLASTYEAR FROM RESOURCES.EMPLOYEE NATURAL JOIN SALES.SALESPERSON;
```

查询结果如下：

```
HAIRDATE    SALESLASTYEAR
---------- -------------
2002-05-02  10.0000
2002-05-02  20.0000
```

4.2.3 JOIN … USING

这是自然连接的另一种写法，JOIN 关键字指定连接的两张表，USING 指明连接列。要求 USING 中的列存在于两张连接表中。

例 查询销售人员的入职时间和去年销售总额。

```
SELECT HAIRDATE,  SALESLASTYEAR FROM  RESOURCES.EMPLOYEE
JOIN  SALES.SALESPERSON  USING(EMPLOYEEID);
```

查询结果如下：

```
HAIRDATE    SALESLASTYEAR
---------- -------------
2002-05-02 10.0000
2002-05-02 20.0000
```

4.2.4 JOIN…ON

这是一种连接查询的常用写法，说明是一个连接查询。JOIN 关键字指定连接的两张表，ON 子句指定连接条件表达式，其中不允许出现 ROWNUM。具体采用何种连接方式，由数据库内部分析确定。

例 查询销售人员的入职时间和去年销售总额。

```
SELECT  T1.HAIRDATE, T2.SALESLASTYEAR
FROM  RESOURCES.EMPLOYEE  T1 JOIN  SALES.SALESPERSON  T2
ON  T1.EMPLOYEEID=T2.EMPLOYEEID;
```

查询结果如下：

```
HAIRDATE    SALESLASTYEAR
---------- -------------
```

```
2002-05-02 10.0000

2002-05-02 20.0000
```

4.2.5 自连接

数据表与自身进行连接，我们称这种连接为自连接。

自连接查询至少要对一张表起别名，否则，服务器无法识别要处理的是哪张表。

例 对 PURCHASING.VENDOR 表进行自连接查询

```
SELECT T1.NAME,  T2.NAME, T1.ACTIVEFLAG

FROM   PURCHASING.VENDOR T1, PURCHASING.VENDOR T2

WHERE   T1.NAME = T2.NAME;
```

查询结果如下：

```
NAME                      NAME                      ACTIVEFLAG

------------------ ------------------ ----------

上海画报出版社          上海画报出版社          1

文学出版社              文学出版社              1

机械工业出版社          机械工业出版社          1

外语教学与研究出版社    外语教学与研究出版社    1

21 世纪出版社           21 世纪出版社           1

上海出版社              上海出版社              1

广州出版社              广州出版社              1

中华书局                中华书局                1

清华大学出版社          清华大学出版社          1

人民邮电出版社          人民邮电出版社          1

北京十月文艺出版社      北京十月文艺出版社      1
```

```
长江文艺出版社              长江文艺出版社              1
```

4.2.6 内连接(INNER JOIN)

根据连接条件，结果集仅包含满足全部连接条件的记录，我们称这样的连接为内连接。

例 从 PRODUCT_CATEGORY、PRODUCT_SUBCATEGORY 中查询图书的目录名称和子目录名称。

```
SELECT T1.NAME, T2.NAME FROM  PRODUCTION.PRODUCT_CATEGORY T1
INNER JOIN PRODUCTION.PRODUCT_SUBCATEGORY T2 ON T1.PRODUCT_CATEGORYID =
T2.PRODUCT_CATEGORYID;
```

查询结果如下:

```
NAME NAME
---- --------------
小说  世界名著
少儿  少儿英语
少儿  励志
少儿  卡通
少儿  童话
少儿  益智游戏
少儿  幼儿启蒙
管理   财务管理
管理   经营管理
管理   商业道德
管理   质量管理与控制
管理   项目管理
```

管理　行政管理

英语　英语写作

英语　英语阅读

英语　英语口语

英语　英语听力

英语　英语语法

英语　英语词汇

计算机 多媒体

计算机 信息安全

计算机 软件工程

计算机 数据库

计算机 程序设计

计算机 操作系统

计算机 计算机体系结构

计算机 计算机理论

文学　民间文学

文学　戏剧

文学　中国现当代诗

文学　中国古诗词

文学　文学理论

文学　纪实文学

文学　文集

小说　社会

小说　军事

```
小说    四大名著
小说    科幻
小说    武侠
```

因为 PRODUCT_CATEGORY 中的 NAME 为金融的没有对应的子目录，所以结果集中没有金融类的图书信息。

4.2.7 外连接(OUTER JOIN)

外连接对结果集进行了扩展，会返回一张表的所有记录，对于另一张表无法匹配的字段用 NULL 填充返回。DM 数据库支持三种方式的外连接：左外连接、右外连接、全外连接。

外连接中常用到的术语：左表、右表。根据表所在外连接中的位置来确定，位于左侧的表，称为左表；位于右侧的表，称为右表。例如 SELECT * FROM T1 LEFT JOIN T2 ON T1.C1=T2.D1，T1 表为左表，T2 表为右表。

返回所有记录的表根据外连接的方式而定。

1. 左外连接：返回左表所有记录；
2. 右外连接：返回右表所有记录；
3. 全外连接：返回两张表所有记录。处理过程为分别对两张表进行左外连接和右外连接，然后合并结果集。

在左外连接和右外连接中，如果需要对未能匹配的缺失数据进行填充，可以使用分区外连接(PARTITION OUTER JOIN)，分区外连接通常用于处理稀疏数据以得到分析报表。

下面举例说明。

例 1 从 PRODUCT_CATEGORY、PRODUCT_SUBCATEGORY 中查询图书的所有目录名称和子目录名称，包括没有子目录的目录。

```
SELECT T1.NAME, T2.NAME FROM  PRODUCTION.PRODUCT_CATEGORY T1
LEFT OUTER JOIN PRODUCTION.PRODUCT_SUBCATEGORY T2 ON T1.PRODUCT_CATEGORYID =
```

```
T2.PRODUCT_CATEGORYID;
```

查询结果如下:

```
NAME    NAME

----    ------------

小说    世界名著

小说    武侠

小说    科幻

小说    四大名著

小说    军事

小说    社会

文学    文集

文学    纪实文学

文学    文学理论

文学    中国古诗词

文学    中国现当代诗

文学    戏剧

文学    民间文学

计算机  计算机理论

计算机  计算机体系结构

计算机  操作系统

计算机  程序设计

计算机  数据库

计算机  软件工程

计算机  信息安全
```

```
计算机  多媒体
英语    英语词汇
英语    英语语法
英语    英语听力
英语    英语口语
英语    英语阅读
英语    英语写作
管理    行政管理
管理    项目管理
管理    质量管理与控制
管理    商业道德
管理    经营管理
管理    财务管理
少儿    幼儿启蒙
少儿    益智游戏
少儿    童话
少儿    卡通
少儿    励志
少儿    少儿英语
金融    NULL
```

例 2 从 PRODUCT_CATEGORY、PRODUCT_SUBCATEGORY 中查询图书的目录名称和所有子目录名称，包括没有目录的子目录。

```
SELECT T1.NAME, T2.NAME FROM   PRODUCTION.PRODUCT_CATEGORY T1
RIGHT OUTER JOIN PRODUCTION.PRODUCT_SUBCATEGORY T2 ON T1.PRODUCT_CATEGORYID =
```

```
T2.PRODUCT_CATEGORYID;
```

查询结果如下:

```
NAME    NAME

----    ----------

小说    世界名著

小说    武侠

小说    科幻

小说    四大名著

小说    军事

小说    社会

NULL    历史

文学    文集

文学    纪实文学

文学    文学理论

文学    中国古诗词

文学    中国现当代诗

文学    戏剧

文学    民间文学

计算机  计算机理论

计算机  计算机体系结构

计算机  操作系统

计算机  程序设计

计算机  数据库

计算机  软件工程
```

```
计算机  信息安全
计算机  多媒体
英语    英语词汇
英语    英语语法
英语    英语听力
英语    英语口语
英语    英语阅读
英语    英语写作
管理    行政管理
管理    项目管理
管理    质量管理与控制
管理    商业道德
管理    经营管理
管理    财务管理
少儿    幼儿启蒙
少儿    益智游戏
少儿    童话
少儿    卡通
少儿    励志
少儿    少儿英语
```

例 3 从 PRODUCT_CATEGORY、PRODUCT_SUBCATEGORY 中查询图书的所有目录名称和所有子目录名称。

```
SELECT T1.NAME, T2.NAME FROM   PRODUCTION.PRODUCT_CATEGORY T1
FULL OUTER JOIN PRODUCTION.PRODUCT_SUBCATEGORY T2 ON T1.PRODUCT_CATEGORYID =
```

```
T2.PRODUCT_CATEGORYID;
```

查询结果如下:

```
NAME    NAME

----    ----------

小说    世界名著

小说    武侠

小说    科幻

小说    四大名著

小说    军事

小说    社会

NULL    历史

文学    文集

文学    纪实文学

文学    文学理论

文学    中国古诗词

文学    中国现当代诗

文学    戏剧

文学    民间文学

计算机  计算机理论

计算机  计算机体系结构

计算机  操作系统

计算机  程序设计

计算机  数据库

计算机  软件工程
```

```
计算机  信息安全
计算机  多媒体
英语    英语词汇
英语    英语语法
英语    英语听力
英语    英语口语
英语    英语阅读
英语    英语写作
管理    行政管理
管理    项目管理
管理    质量管理与控制
管理    商业道德
管理    经营管理
管理    财务管理
少儿    幼儿启蒙
少儿    益智游戏
少儿    童话
少儿    卡通
少儿    励志
少儿    少儿英语
金融    NULL
```

外连接还有一种写法，在连接条件或 WHERE 条件中，在列后面增加(+)指示左外连接或者右外连接。如果表 A 和表 B 连接，连接条件或者 where 条件中，A 的列带有(+)后缀，则认为是 B LEFT JOIN A。带有(+)后缀的列不能连接到子查询。如果(+)引起了外连接环，

则报错。下面举例说明。

例 4 从 PRODUCT_CATEGORY、PRODUCT_SUBCATEGORY 中查询图书的目录名称和所有子目录名称，包括没有目录的子目录。

```
SELECT T1.NAME, T2.NAME
FROM PRODUCTION.PRODUCT_CATEGORY T1, PRODUCTION.PRODUCT_SUBCATEGORY T2
WHERE T1.PRODUCT_CATEGORYID(+) = T2.PRODUCT_CATEGORYID;
```

查询结果与例 2 所示结果一致。

例 5 新建产品区域销售统计表 SALES.SALESREGION 并插入数据。

```
CREATE TABLE SALES.SALESREGION(REGION CHAR(10), PRODUCTID INT, AMOUNT INT);
INSERT INTO SALES.SALESREGION VALUES('大陆', 2, 19800);
INSERT INTO SALES.SALESREGION VALUES('大陆', 4, 20090);
INSERT INTO SALES.SALESREGION VALUES('港澳台', 6, 5698);
INSERT INTO SALES.SALESREGION VALUES('外国', 9, 3756);
COMMIT;
```

统计每个产品在各个区域的销售量，没有销售则显示 NULL，此时可使用 PARTITON OUTER JOIN 将稀疏数据转为稠密数据。

```
SELECT A.PRODUCTID, B.REGION, B.AMOUNT
FROM PRODUCTION.PRODUCT A LEFT JOIN SALES.SALESREGION B
PARTITION BY(B.REGION) ON A.PRODUCTID=B.PRODUCTID
ORDER BY A.PRODUCTID, B.REGION;
```

查询结果如下：

```
PRODUCTID   REGION     AMOUNT
---------- ------- -----------
1              大陆          NULL
```

1	港澳台	NULL
1	外国	NULL
2	大陆	19800
2	港澳台	NULL
2	外国	NULL
3	大陆	NULL
3	港澳台	NULL
3	外国	NULL
4	大陆	20090
4	港澳台	NULL
4	外国	NULL
5	大陆	NULL
5	港澳台	NULL
5	外国	NULL
6	大陆	NULL
6	港澳台	5698
6	外国	NULL
7	大陆	NULL
7	港澳台	NULL
7	外国	NULL
8	大陆	NULL
8	港澳台	NULL
8	外国	NULL
9	大陆	NULL

```
9          港澳台      NULL
9          外国        3756
10         大陆        NULL
10         港澳台      NULL
10         外国        NULL
```

4.2.8 JOIN APPLY

JOIN APPLY 是指连接中的右表可以引用左表列，即同层列引用。通过对 APPLY 右侧输入求值来获得左侧输入每一行的计算结果，生成的行最终组合起来作为最终结果。CROSS APPLY 与 CROSS JOIN 功能类似，相当于不包含连接条件的 CROSS JOIN。OUTER APPLY 与 LEFT OUTER JOIN 功能类似，相当于不包含连接条件的 LEFT OUTER JOIN。

例 1 CROSS APPLY 右侧输入没有左侧输入的相关条件时，等价于 CROSS JOIN。

```
--数据准备
CREATE TABLE T1(C1 INT,C2 INT);
CREATE TABLE T2(D1 INT,D2 INT);
INSERT INTO T1 VALUES(1,1);
INSERT INTO T1 VALUES(2,2);
INSERT INTO T2 VALUES(2,2);
INSERT INTO T2 VALUES(3,3);
COMMIT;
--查询
SELECT * FROM T1 CROSS APPLY T2;
```

查询结果如下：

```
C1         C2         D1         D2
```

```
---------- ---------- ---------- ----------
1          1          2          2
1          1          3          3
2          2          2          2
2          2          3          3
```

例 2 CROSS APPLY 右侧输入有左侧输入的相关条件时，通过对 CROSS APPLY 右侧输入求值来获得左侧输入每一行的计算结果，生成的结果行最终组合起来作为最终结果。

```
SELECT * FROM T1 CROSS APPLY(SELECT * FROM T2 WHERE C1=D1);
```

查询结果如下：

```
C1         C2         D1         D2
---------- ---------- ---------- ----------
2          2          2          2
```

例 3 OUTER APPLY 结果集中既会返回结果集的行，又会返回不生成结果集的行。不生成结果集的行左孩子原样输出，右孩子补 NULL。

```
SELECT * FROM T1 OUTER APPLY(SELECT * FROM T2 WHERE C1=D1);
```

查询结果如下：

```
C1         C2         D1         D2
---------- ---------- ---------- ----------
1          1          NULL       NULL
2          2          2          2
```

4.3 子查询

在 DM_SQL 语言中，一个 SELECT-FROM-WHERE 语句称为一个查询块，如果在一个查询块中嵌套一个或多个查询块，我们称这种查询为子查询。子查询会返回一个值（标量子

查询）或一个表（表子查询）。它通常采用（SELECT…）的形式嵌套在表达式中。子查询语法如下：

```
<子查询> ::= (<查询表达式>)
```

即子查询是嵌入括弧的<查询表达式>，而这个<查询表达式>通常是一个 SELECT 语句。它有下列限制：

1. 子查询允许 TEXT 类型与 CHAR 类型值比较。比较时，取出 TEXT 类型字段的最多 32767 字节与 CHAR 类型字段进行比较；

2. 在子查询中允许嵌套子查询。

按子查询返回结果的形式，DM 子查询可分为两大类：

1．标量子查询：只返回一行一列；

2．表子查询：可返回多行多列。

4.3.1 标量子查询

标量子查询是一个普通 SELECT 查询，它只应该返回一行一列记录。如果返回结果多于一行则会提示单行子查询返回多行，返回结果多于一列则会提示 SELECT 语句列数超长。

下面是一个标量子查询的例子(请先关闭自动提交功能，否则 COMMIT 与 ROLLBACK 会失去效果)：

```
SELECT 'VALUE IS', (SELECT ADDRESS1 FROM PERSON.ADDRESS WHERE ADDRESSID = 1)
FROM PERSON.ADDRESS_TYPE;
//子查询只有一列，结果正确


SELECT 'VALUE IS', LEFT((SELECT ADDRESS1 FROM PERSON. ADDRESS WHERE ADDRESSID = 1), 8)
FROM   PERSON.ADDRESS_TYPE;
//函数+标量子查询，结果正确
```

```
SELECT 'VALUE IS', (SELECT ADDRESS1, CITY FROM PERSON.ADDRESS WHERE ADDRESSID = 1) FROM
PERSON.ADDRESS_TYPE;
//返回列数不为 1，报错

SELECT 'VALUES IS', (SELECT ADDRESS1 FROM PERSON.ADDRESS) FROM   PERSON.ADDRESS_TYPE;
//子查询返回行值多于一个，报错

DELETE FROM SALES.SALESORDER_DETAIL;
SELECT 'VALUE IS', (SELECT ORDERQTY FROM SALES.SALESORDER_DETAIL) FROM   SALES.CUSTOMER;
//子查询有 0 行，结果返回 NULL

UPDATE PRODUCTION.PRODUCT SET PUBLISHER =
(SELECT NAME FROM PURCHASING.VENDOR WHERE VENDORID = 2)
WHERE PRODUCTID = 5;

UPDATE PRODUCTION.PRODUCT_VENDOR SET STANDARDPRICE =
(SELECT AVG(NOWPRICE) FROM PRODUCTION.PRODUCT)
WHERE PRODUCTID = 1;
//Update 语句中允许使用标量子查询

INSERT INTO PRODUCTION.PRODUCT_CATEGORY(NAME) VALUES
(( SELECT NAME FROM PRODUCTION.PRODUCT_SUBCATEGORY
WHERE PRODUCT_SUBCATEGORYID= 40));
```

```
//Insert 语句中允许使用标量子查询
```

例如，查询通常价格最小的供应商的名称和最小价格：

```
SELECT NAME, (SELECT    MIN(STANDARDPRICE)
      FROM    PRODUCTION.PRODUCT_VENDOR T1
      WHERE T1.VENDORID = T2.VENDORID)
FROM    PURCHASING.VENDOR T2;
```

4.3.2 表子查询

和标量子查询不同的是，表子查询的查询结果可以是多行多列。

一般情况下，表子查询类似标量子查询，单列构成了表子查询的选择清单，但它的查询结果允许返回多行。可以从上下文中区分出表子查询：在其前面始终有一个只对表子查询的算符：<比较算符>ALL、<比较算符>ANY(或是其同义词<比较算符> SOME)、IN 和 EXISTS。

其中，在 IN/NOT IN 表子查询的情况下，DM 支持查询结果返回多列。

例 1 查询职务为销售代表的员工的编号、今年销售总额和去年销售总额。

```
SELECT    EMPLOYEEID, SALESTHISYEAR, SALESLASTYEAR
FROM     SALES.SALESPERSON
WHERE    EMPLOYEEID IN
( SELECT    EMPLOYEEID
      FROM    RESOURCES.EMPLOYEE
      WHERE    TITLE = '销售代表'
);
```

查询结果如下：

```
EMPLOYEEID    SALESTHISYEAR SALESLASTYEAR
----------- ------------- -------------
```

```
4               8.0000          10.0000
5               8.0000          20.0000
```

该查询语句的求解方式是：首先通过子查询“SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE WHERE TITLE = '销售代表'”查到职务为销售代表的 EMPLOYEEID 的集合，然后，在 SALESPERSON 表中找到与子查询结果集中的 EMPLOYEEID 所对应员工的 SALESTHISYEAR 和 SALESLASTYEAR。

在带有子查询的查询语句中，通常也将子查询称内层查询或下层查询。由于子查询还可以嵌套子查询，相对于下一层的子查询，上层查询又称为父查询或外层查询。

由于 DM_SQL 语言所支持的嵌套查询功能可以将一系列简单查询构造成复杂的查询，从而有效地增强了 DM_SQL 语句的查询功能。以嵌套的方式构造语句是 DM_SQL 的“结构化”的特点。

需要说明的是：上例的外层查询只能用 IN 谓词而不能用比较算符“=”，因为子查询的结果包含多个元组，除非能确定子查询的结果只有一个元组时，才可用等号比较。上例语句也可以用连接查询的方式实现。

```
SELECT T1.EMPLOYEEID, T1.SALESTHISYEAR, T1.SALESLASTYEAR
FROM   SALES.SALESPERSON T1 , RESOURCES.EMPLOYEE T2
WHERE T1.EMPLOYEEID = T2.EMPLOYEEID AND T2.TITLE = '销售代表';
```

例 2 查询对目录名为小说的图书进行评论的人员名称和评论日期。

采用子查询嵌套方式写出以下查询语句：

```
SELECT   DISTINCT NAME, REVIEWDATE
FROM    PRODUCTION.PRODUCT_REVIEW
WHERE   PRODUCTID   IN
( SELECT   PRODUCTID
    FROM   PRODUCTION.PRODUCT
```

```
    WHERE   PRODUCT_SUBCATEGORYID IN
    ( SELECT   PRODUCT_SUBCATEGORYID
        FROM    PRODUCTION.PRODUCT_SUBCATEGORY
        WHERE   PRODUCT_CATEGORYID IN
        ( SELECT   PRODUCT_CATEGORYID
          FROM   PRODUCTION.PRODUCT_CATEGORY
          WHERE   NAME = '小说'
        )
      )
    );
```

查询结果如下:

```
NAME      REVIEWDATE
------ ----------
刘青       2007-05-06
桑泽恩     2007-05-06
```

该语句采用了四层嵌套查询方式，首先通过最内层子查询从 PRODUCT_CATEGORY 中查出目录名为小说的目录编号，然后从 PRODUCT_SUBCATEGORY 中查出这些目录编号对应的子目录编号，接着从 PRODUCT 表中查出这些子目录编号对应的图书的编号，最后由最外层查询查出这些图书编号对应的评论人员和评论日期。

此例也可用四个表的连接来完成。

从上例可以看出，当查询涉及到多个基表时，嵌套子查询与连接查询相比，前者由于是逐步求解，层次清晰，易于阅读和理解，具有结构化程序设计的优点。

在许多情况下，外层子查询与内层子查询常常引用同一个表，如下例所示。

例 3 查询当前价格低于红楼梦的图书的名称、作者和当前价格。

```
SELECT  NAME, AUTHOR,  NOWPRICE
FROM  PRODUCTION.PRODUCT
WHERE  NOWPRICE < ( SELECT NOWPRICE FROM PRODUCTION.PRODUCT WHERE NAME = '红楼梦');
```

查询结果如下:

```
NAME                AUTHOR          NOWPRICE
--------------- -------------- --------
水浒传              施耐庵，罗贯中    14.3000
老人与海            海明威            6.1000
工作中无小事        陈满麒            11.4000
突破英文基础词汇    刘毅              11.1000
```

此例的子查询与外层查询尽管使用了同一表名，但作用是不一样的。子查询是在该表中红楼梦的图书价格，而外查询是在 PRODUCT 表 NOWPRICE 列查找小于该值的集合，从而得到这些值所对应的名称和作者。DM_SQL 语言允许为这样的表引用定义别名:

```
SELECT  NAME, AUTHOR,  NOWPRICE
FROM  PRODUCTION.PRODUCT  T1
WHERE  T1.NOWPRICE < ( SELECT  T2.NOWPRICE
FROM  PRODUCTION.PRODUCT  T2 WHERE  T2.NAME = '红楼梦');
```

该语句也可以采用连接方式实现:

```
SELECT  T1.NAME, T1.AUTHOR, T1.NOWPRICE
FROM  PRODUCTION.PRODUCT T1 ,  PRODUCTION.PRODUCT T2
WHERE  T2.NAME = '红楼梦' AND T1.NOWPRICE < T2.NOWPRICE;
```

例 4 查询图书的出版社和产品供应商名称相同的图书编号和名称。

```
SELECT  T1.PRODUCTID, T1.NAME
FROM  PRODUCTION.PRODUCT T1, PRODUCTION.PRODUCT_VENDOR T2
```

```
WHERE  T1.PRODUCTID = T2.PRODUCTID  AND T1.PUBLISHER = ANY
    ( SELECT  NAME  FROM  PURCHASING.VENDOR T3 WHERE  T2.VENDORID = T3.VENDORID);
```

查询结果如下:

```
PRODUCTID      NAME
---------- -------------------------
1              红楼梦
2              水浒传
3              老人与海
4              射雕英雄传(全四册)
7              数据结构(C 语言版)(附光盘)
8              工作中无小事
9              突破英文基础词汇
10             噼里啪啦丛书(全 7 册)
```

此例有一点需要注意：子查询的 WHERE 子句涉及到 PRODUCT_VENDOR.VENDORID(即 T2.VENDORID)，但是其 FROM 子句中却没有提到 PRODUCT_VENDOR。在外部子查询 FROM 子句中命名了 PRODUCT_VENDOR——这就是外部引用。当一个子查询含有一个外部引用时，它就与外部语句相关联，称这种子查询为相关子查询。

例 5 查询图书的出版社和产品供应商名称不相同的图书编号和名称。

```
SELECT  T1.PRODUCTID, T1.NAME
FROM  PRODUCTION.PRODUCT T1
WHERE  T1.PUBLISHER <> ALL(SELECT  NAME  FROM  PURCHASING.VENDOR );
```

查询结果如下:

```
PRODUCTID      NAME

---------- --------------------------------

6              长征

5              鲁迅文集(小说、散文、杂文)全两册
```

4.3.3 派生表子查询

派生表子查询是一种特殊的表子查询。所谓派生表是指 FROM 子句中的查询表达式，可以以别名对其进行引用。在 SELECT 语句的 FROM 子句中可以包含一个或多个派生表。派生表嵌套层次不能超过 60 层。

例 查询每个目录的编号、名称和对应的子目录的数量，并按数量递减排列。

```
SELECT   T1.PRODUCT_CATEGORYID, T1.NAME, T2.NUM

FROM   PRODUCTION.PRODUCT_CATEGORY   T1,

( SELECT PRODUCT_CATEGORYID, COUNT(PRODUCT_SUBCATEGORYID)

    FROM   PRODUCTION.PRODUCT_SUBCATEGORY

    GROUP BY PRODUCT_CATEGORYID )

AS T2(PRODUCT_CATEGORYID,NUM) WHERE T1.PRODUCT_CATEGORYID =

T2.PRODUCT_CATEGORYID ORDER BY T2.NUM DESC;
```

查询结果如下：

```
PRODUCT_CATEGORYID NAME       NUM

------------------ ------   --------------------

3                        计算机     8

2                        文学       7

6                        少儿       6

5                        管理       6
```

```
4                    英语        6
1                    小说        6
```

4.3.4 定量比较

量化符 ALL、SOME、ANY 可以用于将一个<数据类型>的值和一个由表子查询返回的值的集合进行比较。

1．ALL

ALL 定量比较要求的语法如下：

```
<标量表达式> <比较算符> ALL <表子查询>
```

其中：

1）<标量表达式> 可以是对任意单值计算的表达式；

2）<比较算符> 包括 = 、 > 、 < 、 > = 、 < = 或 < > 。

若表子查询返回 0 行或比较算符对表子查询返回的每一行都为 TRUE，则返回 TRUE。若比较算符对于表子查询返回的至少一行是 FALSE，则 ALL 返回 FALSE。

例 1 查询没有分配部门的员工的编号、姓名和身份证号码。

```
SELECT  T1.EMPLOYEEID, T2.NAME, T1.NATIONALNO
FROM  RESOURCES.EMPLOYEE T1 ,  PERSON.PERSON T2
WHERE  T1.PERSONID = T2.PERSONID  AND  T1.EMPLOYEEID < > ALL
    ( SELECT  EMPLOYEEID  FROM  RESOURCES.EMPLOYEE_DEPARTMENT);
```

查询结果如下：

```
EMPLOYEEID  NAME  NATIONALNO
----------- ----  ------------------
7              王菲    420921197708051523
```

例 2 查询比中华书局所供应的所有图书都贵的图书的编号、名称和现在销售价格。

```
SELECT  PRODUCTID, NAME, NOWPRICE
FROM  PRODUCTION.PRODUCT
WHERE  NOWPRICE  > ALL
( SELECT  T1.NOWPRICE
    FROM  PRODUCTION.PRODUCT  T1 , PRODUCTION.PRODUCT_VENDOR T2
    WHERE T1.PRODUCTID = T2.PRODUCTID AND T2.VENDORID =
    ( SELECT  VENDORID  FROM  PURCHASING.VENDOR
    WHERE  NAME = '中华书局'
      )
    )
    AND  PRODUCTID  <>  ALL
    ( SELECT  T1.PRODUCTID
    FROM  PRODUCTION.PRODUCT_VENDOR  T1 , PURCHASING.VENDOR  T2
    WHERE  T1.VENDORID =  T2.VENDORID  AND T2.NAME = '中华书局'
);
```

查询结果如下:

```
PRODUCTID      NAME                                NOWPRICE
---------- ------------------------------ --------
10             噼里啪啦丛书(全 7 册)                42.0000
7              数据结构(C 语言版)(附光盘)            25.5000
6              长征                                37.7000
5              鲁迅文集(小说、散文、杂文)全两册      20.0000
4              射雕英雄传(全四册)                   21.7000
```

2．ANY 或 SOME

ANY 或 SOME 定量比较要求的语法如下:

```
<标量表达式> <比较算符> ANY | SOME <表子查询>
```

SOME 和 ANY 是同义词。如果它们对于表子查询返回的至少一行为 TRUE，则返回为 TRUE。若表子查询返回 0 行或比较算符对表子查询返回的每一行都为 FALSE，则返回 FALSE。

ANY 和 ALL 与集函数的对应关系如表 4.3.1 所示。

表 4.3.1　ANY 和 ALL 与集函数的对应关系

	=	< >	<	< =	>	> =
ANY	IN	不存在	< MAX	< = MAX	> MIN	> = MIN
ALL	不存在	NOT IN	< MIN	< = MIN	> MAX	> = MAX

在具体使用时，读者完全可根据自己的习惯和需要选用。

4.3.5 带 EXISTS 谓词的子查询

带 EXISTS 谓词的子查询语法如下:

```
<EXISTS 谓词> ::= [NOT] EXISTS <表子查询>
```

EXISTS 判断是对非空集合的测试并返回 TRUE 或 FALSE。若表子查询返回至少一行，则 EXISTS 返回 TRUE，否则返回 FALSE。若表子查询返回 0 行，则 NOT EXISTS 返回 TRUE，否则返回 FALSE。

例 查询职务为销售代表的员工的编号和入职时间。

```
SELECT  T1.EMPLOYEEID , T1.STARTDATE
FROM  RESOURCES.EMPLOYEE_DEPARTMENT T1
WHERE  EXISTS
( SELECT  *  FROM  RESOURCES.EMPLOYEE T2
```

```
    WHERE   T2.EMPLOYEEID = T1.EMPLOYEEID   AND T2.TITLE = '销售代表');
```

查询结果如下:

```
EMPLOYEEID   STARTDATE

---------- ----------

4              2005-02-01

5              2005-02-01
```

此例查询需要 EMPLOYEE_DEPARTMENT 表和 EMPLOYEE 表中的数据，其执行方式为:首先在 EMPLOYEE_DEPARTMENT 表的第一行取 EMPLOYEEID 的值为 2，这样对内层子查询则为:

```
(SELECT   *   FROM   RESOURCES.EMPLOYEE   T2

WHERE   T2.EMPLOYEEID='2'   AND   T2.TITLE='销售代表');
```

在 EMPLOYEE 表中，不存在满足该条件的行，子查询返回值为假，说明不能取 EMPLOYEE_DEPARTMENT 表的第一行作为结果。系统接着取 EMPLOYEE_DEPARTMENT 表的第二行，又得到 EMPLOYEEID 的值为 4，执行内层查询，此时子查询返回值为真，说明可以取该行作为结果。重复以上步骤……。只有外层子查询 WHERE 子句结果为真时，方可将 EMPLOYEE_DEPARTMENT 表中的对应行送入结果表，如此继续，直到把 EMPLOYEE_DEPARTMENT 表的各行处理完。

从以上分析得出，EXISTS 子查询的查询结果与外表相关，即连接条件中包含内表和外表列，我们称这种类型的子查询为相关子查询；反之，子查询的连接条件不包含外表列，即查询结果不受外表影响，我们称这种类型的子查询为非相关子查询。

4.3.6 多列表子查询

为了满足应用需求，DM 数据库扩展了子查询功能，目前支持多列 IN / NOT IN 子查询。

子查询可以是值列表或者查询块。

例 1 查询活动标志为 1 且信誉为 2 的供应商编号和名称。

```
SELECT    VENDORID, NAME
FROM    PURCHASING.VENDOR
WHERE   (ACTIVEFLAG, CREDIT) IN ((1, 2));
```

查询结果如下:

```
VENDORID        NAME
----------- --------------
1                   上海画报出版社
2                   长江文艺出版社
```

上例中子查询的选择清单为多列，而看到子查询算符后面跟着的形如((1，2))的表达式我们称之为多列表达式链表, 这个多列表达式链表以一个或多个多列数据集构成的集合构成。上述的例子中的多列表达式链表中的元素有两个。

例 2 查询作者为海明威且出版社为上海出版社或作者为王树增且出版社为人民文学出版社的图书名称和现在销售价格。

```
SELECT    NAME, NOWPRICE
FROM    PRODUCTION.PRODUCT
WHERE   (AUTHOR, PUBLISHER)   IN
(( '海明威', '上海出版社'), ('王树增', '人民文学出版社'));
```

查询结果如下:

```
NAME          NOWPRICE
-------- --------
老人与海     6.1000
长征          37.7000
```

子查询为值列表时，需要注意以下三点：

1．值列表需要用括号；

2．值列表之间以逗号分割；

3．值列表的个数与查询列个数相同。

子查询为查询块的情况如下例所示：

例 3 查询由采购代表下的供应商是清华大学出版社的订单的创建日期、状态和应付款总额。

```
SELECT  ORDERDATE, STATUS, TOTAL
FROM    PURCHASING.PURCHASEORDER_HEADER
WHERE  (EMPLOYEEID, VENDORID)  IN
(SELECT  T1.EMPLOYEEID,  T2.VENDORID
    FROM  RESOURCES.EMPLOYEE  T1,  PURCHASING.VENDOR  T2
    WHERE  T1.TITLE = '采购代表' AND T2.NAME = '清华大学出版社');
```

查询结果如下：

```
ORDERDATE     STATUS          TOTAL
----------    ----------- ---------
2006-07-21    1               6400.0000
```

由例子可以看到，WHERE 子句中有两个条件列，IN 子查询的查询项也由两列构成。

DM 对多列子查询的支持，满足了更多的应用场景。

4.4 WITH 子句

WITH 子句语法如下：

```
<WITH 子句> ::= [<WITH FUNCTION 子句>] [<WITH CTE 子句>]
```

4.4.1 WITH FUNCTION 子句

WITH FUNCTION 子句用于在 SQL 语句中临时声明并定义存储函数，这些存储函数可以在其作用域内被引用。相比模式对象中的存储函数，通过 WITH FUNCTION 定义的存储函数在对象名解析时拥有更高的优先级。

和公用表表达式 CTE 类似，WITH FUNCTION 定义的存储函数对象也不会存储到系统表中，且只在当前 SQL 语句内有效。

WITH FUNCTION 子句适用于偶尔需要使用存储函数的场景。和模式对象中的存储函数相比，它可以清楚地看到函数定义并避免了 DDL 操作带来的开销。

语法格式

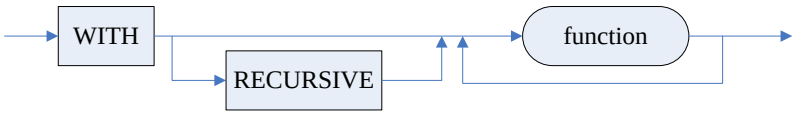
```
WITH [RECURSIVE] <函数> {<函数>}
```

参数

1．<函数> 语法遵照《DM8_SQL 程序设计》中存储函数的语法规则。

图例

with function 子句



语句功能

供用户定义同一语句内临时使用的存储函数。

使用说明

1. <WITH FUNCTION 子句>中定义的函数的作用域为<WITH 子句>所在的查询表达式内；
2. 同一<WITH FUNCTION 子句>中函数名不得重复；
3. <WITH FUNCTION 子句>中定义的函数不能是自定义集函数和外部函数；
4. 该语句的使用者并不需要 CREATE PROCEDURE 数据库权限。

举例说明

例 1 WITH FUNCTION 中定义的函数优先级高于模式对象的例子。

```
WITH FUNCTION f1(C INT) RETURN INT AS BEGIN RETURN C * 10; END;
SELECT f1(5236) FROM DUAL;
/
```

查询结果为：52360

例 2 WITH FUNCTION 和公用表表达式混合的例子。

```
WITH FUNCTION f21(C1 INT) RETURN INT AS BEGIN RETURN C1; END;
SELECT f21(1) FROM dual WHERE 100 IN
(
    WITH FUNCTION f22(C1 INT) RETURN INT AS BEGIN RETURN C1 + 2; END;
        FUNCTION f23(C1 INT) RETURN INT AS BEGIN RETURN C1 - 2; END;
            v21(C) AS (SELECT 50 FROM dual)
    SELECT f22(C) +f23(C) FROM v21
);
/
```

查询结果为：1

4.4.2 WITH CTE 子句

嵌套 SQL 语句如果层次过多，会使 SQL 语句难以阅读和维护。如果将子查询放在临时表中，会使 SQL 语句更容易维护，但同时也增加了额外的 I/O 开销，因此，临时表并不太适合数据量大且频繁查询的情况。为此，在 DM 中引入了公用表表达式（CTE，COMMON TABLE EXPRESSION），使用 CTE 可以提高 SQL 语句的可维护性，同时 CTE 要比临时表的效率高很多。

CTE 与派生表类似，具体表现在不存储为对象，并且只在查询期间有效。与派生表的

不同之处在于，CTE 可自引用，还可在同一查询中引用多次。

WITH CTE 子句会定义一个公用表达式，该公用表达式会被整个 SQL 语句所用到。它可以有效提高 SQL 语句的可读性，也可以用在 UNION ALL 中，作为提供数据的部分。

WITH CTE 子句根据 CTE 是否递归执行 CTE 自身，DM 将 WITH CTE 子句分为递归 WITH 和非递归 WITH 两种情况。

4.4.2.1 公用表表达式的作用

公用表表达式（CTE）是一个在查询中定义的临时命名结果集，将在 FROM 子句中使用它。每个 CTE 仅被定义一次（但在其作用域内可以被引用任意次），并且在该查询生存期间将一直生存，而且可以使用 CTE 来执行递归操作。

因为 UNION ALL 的每个部分可能相同，但是如果每个部分都去执行一遍的话，则成本太高，所以可以使用 WITH CTE 子句，则只要执行一遍即可。如果 WITH CTE 子句所定义的表名被调用两次以上，则优化器会自动将 WITH CTE 子句所获取的数据放入一个临时表里，如果只是被调用一次则不会，很多查询通过这种方法都可以提高速度。

4.4.2.2 非递归 WITH 的使用

语法格式

```
WITH [RECURSIVE] <非递归 with cte 子句>{,<非递归 with cte 子句>}<cte 查询语句>;
<非递归 with cte 子句>: : = <公用表表达式的名称> [<列名> ({,<列名>})] AS （ <公用表表达式子查询语句>）
```

参数

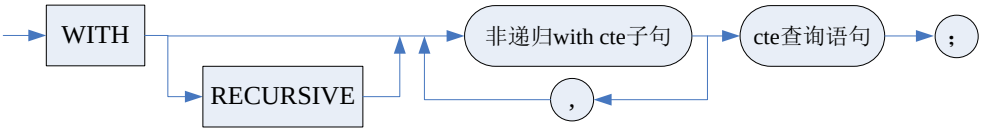
1 . RECURSIVE 一般为可选项，但在使用 RECURSIVE 本身作为公用表表达式的名称时，RECURSIVE 为必选项，即需要写成 WITH RECURSIVE RECURSIVE()的形式。

2 . <列名> 指明被创建的公用表表达式中列的名称;

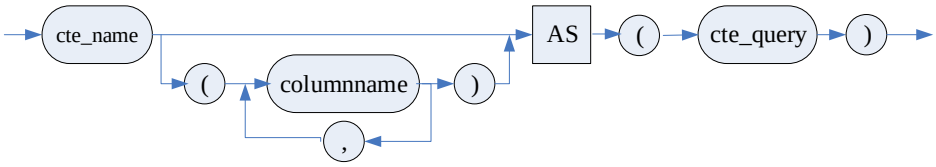
3 . <公用表表达式子查询语句> 标识公用表表达式所基于的表的行和列，其语法遵照 SELECT 语句的语法规则。

图例

非递归 with:



<非递归 with cte 子句>:



语句功能

供用户定义非递归公用表表达式，也就是非递归 WITH 语句。

使用说明

1．< 公用表表达式的名称 >必须与在同一 WITH 子句中定义的任何其他公用表表达式的名称不同，但公用表表达式名可以与基表或基视图的名称相同。在查询中对公用表表达式名的任何引用都会使用公用表表达式，而不使用基对象；

2．在一个 CTE 定义中不允许出现重复的列名。指定的列名数必须与<公用表表达式子查询语句>结果集中列数匹配。只有在查询定义中为所有结果列都提供了不同的名称时，列名称列表才是可选的；

3．<公用表表达式子查询语句>指定一个结果集填充公用表表达式的 SELECT 语句。除了 CTE 不能定义另一个 CTE 以外，<公用表表达式子查询语句> 的 SELECT 语句必须满足与创建视图时相同的要求；

4. <cte 查询语句> 支持 SELECT、UPDATE 和 DELETE 语句，用户对语句中涉及的表必须具备相应的 SELECT、UPDATE 或 DELETE 权限。此处，语法上<cte 查询语句>支持任意 SELECT、UPDATE 和 DELETE 语句，但是对于 CTE 而言，只有<cte 查询语句>中使用<公用表表达式的名称>，CTE 才有意义。

举例说明

例 1 创建一个表 TEST1 和表 TEST2，并利用公用表表达式对它们进行连接运算。

```
CREATE TABLE TEST1(I INT);

INSERT INTO TEST1 VALUES(1);

INSERT INTO TEST1 VALUES(2);



CREATE TABLE TEST2(J INT);

INSERT INTO TEST2 VALUES(5);

INSERT INTO TEST2 VALUES(6);

INSERT INTO TEST2 VALUES(7);



WITH CTE1(K) AS(SELECT I FROM TEST1 WHERE I > 1),

CTE2(G) AS(SELECT J FROM TEST2 WHERE J > 5)

SELECT K, G FROM CTE1, CTE2;
```

查询结果如下：

```
K           G

--------- -----------

2           6

2           7
```

例 2 利用公用表表达式，更新表 TEST1 中的数据。

```
WITH CTE1(K) AS(SELECT I FROM TEST1) UPDATE CTE1 SET K=K+1;
```

查询表 TEST1 中的数据。

```
SELECT * FROM TEST1;
```

查询结果如下：

```
I

-----------

2

3
```

例 3 利用公用表表达式，删除表 TEST2 中的数据。

```
WITH CTE1(K) AS(SELECT J FROM TEST2) DELETE FROM CTE1 WHERE K=5;
```

查询表 TEST2 中的数据。

```
SELECT * FROM TEST2;
```

查询结果如下:

```
J

-----------

6

7
```

4.4.2.3 递归 WITH 的使用

递归 WITH 是一个重复执行初始 CTE 以返回数据子集直到获取完整结果集的公用表表达式。递归 WITH 通常用于返回分层数据。

语法格式:

```
WITH [RECURSIVE] <递归 with cte 子句>{,<递归 with cte 子句>}<cte 查询语句>;

<递归 with cte 子句>::=<公用表表达式的名称> (<列名>{,<列名>}) AS (<定位点成员> UNION ALL <
递归成员>)
```

参数

1 . RECURSIVE 一般为可选项，但在使用 RECURSIVE 本身作为公用表表达式的名称时，RECURSIVE 为必选项，即需要写成 WITH RECURSIVE RECURSIVE()的形式。

2 . <列名> 指明被创建的递归 WITH 中列的名称；各列不能同名，列名和 AS 后的列名

没有关系，类似建视图时为视图指定的列别名。

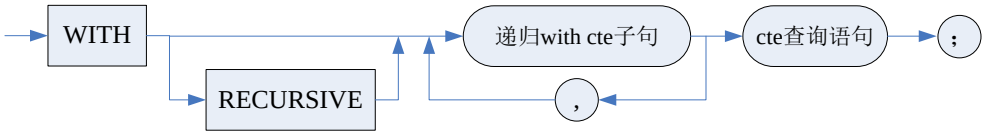
3 .<定位点成员> 任何不包含<公用表表达式的名称>的 SELECT 查询语句，可以 UNION ALL、 UNION、 INTERSECT 或 MINUS。定位点成员的查询结果集是递归成员迭代的基础。

4 . <递归成员> 引用<公用表表达式的名称>的 SELECT 查询语句。递归成员通过引用自身<公用表表达式的名称>反复迭代执行，下一次迭代的数据基于上一次迭代的查询结果，当且仅当本次迭代结果为空集时才终止迭代。

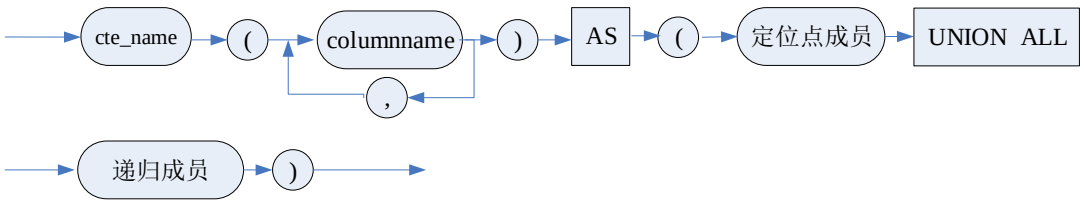
5 .<cte 查询语句> SELECT 查询语句。此处，语法上<cte 查询语句>支持任意 SELECT 语句，但是对于 CTE 而言，只有<cte 查询语句>中使用<公用表表达式的名称>，CTE 才有意义。

图例

递归 with



<递归 with cte 子句>:



语句功能

供用户定义递归公用表表达式，也就是递归 WITH 语句。

与递归 WITH 有关的两个 INI 参数为 CTE_MAXRECURSION 和 CTE_OPT_FLAG。CTE_MAXRECURSION 用来指定递归 CTE 迭代层次，取值范围 1~ULINT_MAX，缺省为 100。CTE_OPT_FLAG 用来指定递归 WITH 相关子查询是否转换为 WITH FUNCTION 优化，取值 0 或 1，缺省为 1。

递归 WITH 的执行流程如下：

1．将递归 WITH 拆分为定位点成员和递归成员；

2．运行定位点成员，创建第一个基准结果集（T0）；

3．运行递归成员，将 TI 作为输入（初始 i=0），将 TI+1 作为输出，I=I++；

4．重复步骤 3，直到返回空集；

5．返回结果集为 T0 到 TN 执行 UNION ALL 的结果。

使用说明

1．<公用表表达式的名称>在定位点成员中不能出现。<公用表表达式的名称>在递归成员中有且只能引用一次；

2．递归成员中不能包含下面元素：

- DISTINCT；
- GROUP BY；
- 集函数，但支持分析函数；
- <公用表表达式的名称>不能作为<递归成员>中外连接 OUTER JOIN 的右表。

3．<递归成员>中列的数据类型必须与定位点成员中相应列的数据类型兼容。

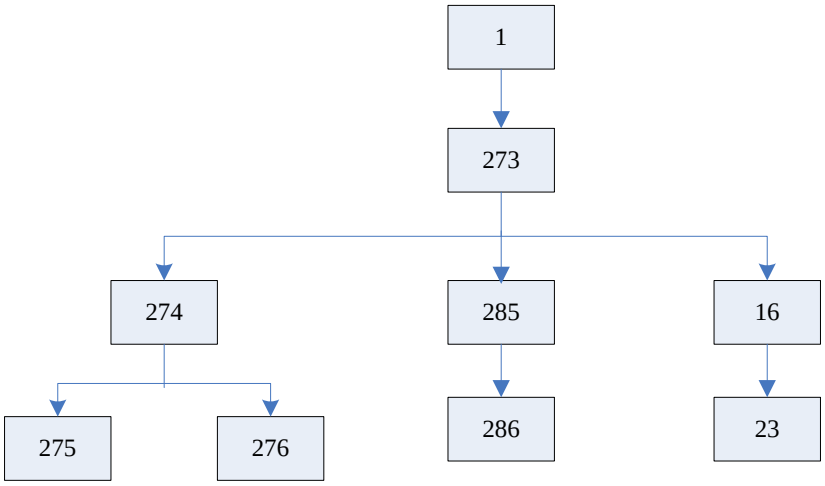
举例说明

```
DROP TABLE MYEMPLOYEES;
CREATE TABLE MYEMPLOYEES(
EMPLOYEEID   SMALLINT,
    FIRST_NAME   VARCHAR2 (30) NOT NULL,
    LAST_NAME   VARCHAR2 (40) NOT NULL,
    TITLE          VARCHAR2 (50) NOT NULL,
    DEPTID         SMALLINT NOT NULL,
    MANAGERID     INT NULL);
```

```
INSERT INTO MYEMPLOYEES VALUES (1, 'KEN', 'SANCHEZ', 'CHIEF EXECUTIVE OFFICER', 16, NULL);
INSERT INTO MYEMPLOYEES VALUES (273, 'BRIAN', 'WELCKER', 'VICE PRESIDENT OF SALES', 3, 1);
INSERT INTO MYEMPLOYEES VALUES (274, 'STEPHEN','JIANG', 'NORTH AMERICAN SALES MANAGER',3,
273);
INSERT INTO MYEMPLOYEES VALUES (275, 'MICHAEL', 'BLYTHE', 'SALES REPRESENTATIVE', 3, 274);
INSERT INTO MYEMPLOYEES VALUES (276, 'LINDA', 'MITCHELL', 'SALES REPRESENTATIVE', 3, 274);
INSERT INTO MYEMPLOYEES VALUES (285, 'SYED', 'ABBAS', 'PACIFIC SALES MANAGER', 3, 273);
INSERT INTO MYEMPLOYEES VALUES (286, 'LYNN', 'TSOFLIAS', 'SALES REPRESENTATIVE', 3, 285);
INSERT INTO MYEMPLOYEES VALUES (16, 'DAVID', 'BRADLEY', 'MARKETING MANAGER', 4, 273);
INSERT INTO MYEMPLOYEES VALUES (23, 'MARY', 'GIBSON', 'MARKETING SPECIALIST', 4, 16);

commit;
```

上下级关系如下图所示:




```
WITH DIRECTREPORTS(MANAGERID, EMPLOYEEID, TITLE, DEPTID) AS
(SELECT MANAGERID, EMPLOYEEID, TITLE, DEPTID
    FROM MYEMPLOYEES
```

```
    WHERE MANAGERID IS NULL    //定位点成员
    UNION ALL
    SELECT E.MANAGERID, E.EMPLOYEEID, E.TITLE, E.DEPTID
    FROM MYEMPLOYEES E
INNER JOIN DIRECTREPORTS D
ON E.MANAGERID = D.EMPLOYEEID   //递归成员
)
SELECT MANAGERID, EMPLOYEEID, TITLE   FROM DIRECTREPORTS;
```

递归调用执行步骤，结果如下：

(1) 产生定位点成员

```
MANAGERID   EMPLOYEEID   TITLE
---------   ----------   ------------------
NULL        1            CHIEF EXECUTIVE OFFICER
```

(2) 第一次迭代，返回一个成员

```
MANAGERID   EMPLOYEEID   TITLE
---------   ----------   -----------------
1           273          VICE    PRESIDENT OF SALES
```

(3) 第二次迭代，返回三个成员

```
MANAGERID   EMPLOYEEID   TITLE
---------   ----------   ----------------
273         16           MARKETING MANAGER
273         274          NORTH AMERICAN SALES MANAGER
273         285          PACIFIC SALES MANAGER
```

(4) 第三次迭代，返回四个成员

```
MANAGERID   EMPLOYEEID   TITLE
---------   ----------   -----------------
16          23           MARKETING SPECIALIST
274         275          SALES REPRESENTATIVE
274         276          SALES REPRESENTATIVE
285         286          SALES REPRESENTATIVE
```

(5) 第四次迭代，返回空集。递归结束。

(6) 正在运行的查询返回的最终结果集是定位点成员和递归成员生成的所有结果集的并集（UNION ALL）。

```
MANAGERID   EMPLOYEEID   TITLE
---------   ----------   ----------------------------
NULL        1            CHIEF EXECUTIVE OFFICER
1           273          VICE PRESIDENT OF SALES
273         16           MARKETING MANAGER
273         274          NORTH AMERICAN SALES MANAGER
273         285          PACIFIC SALES MANAGER
16          23           MARKETING SPECIALIST
274         275          SALES REPRESENTATIVE
274         276          SALES REPRESENTATIVE
285         286          SALES REPRESENTATIVE
```

4.5 集合查询

集合查询是指使用了集合运算符的查询。集合运算符包括 UNION、EXCEPT、MINUS 和 INTERSECT。

语法格式

```
<select_clause>  <集合运算符> [ALL | DISTINCT | UNIQUE] [CORRESPONDING [BY (<列名> {,<列名>})]] <select_clause>
<集合运算符>::=UNION| EXCEPT| MINUS| INTERSECT
<select_clause>和[CORRESPONDING [BY (<列名> {,<列名>})]]请参考<查询表达式>
```

使用说明

1．<集合运算符> UNION 运算符返回两个结果集的并集；EXCEPT 和 MINUS 运算符功能完全一样，返回两个集合的差集；INTERSECT 运算符返回两个集合的交集，INTERSECT 运算符会自动删除重复行，如果需要保留重复行，需要使用 INTERSECT ALL；

2．两个查询块<select_clause>的查询列数目必须相同，数据类型必须兼容；

3．ALL|DISTINCT|UNIQUE。ALL 表示集合查询结果中保持所有重复；DISTINCT 表示删除所有重复。DISTINCT 与 UNIQUE 等价。缺省为 DISTINCT。

举例说明

例 1　查询所有图书的出版商，查询所有图书供应商的名称，将两者连接，并去掉重复行。

```
SELECT PUBLISHER FROM PRODUCTION.PRODUCT
UNION
SELECT NAME FROM PURCHASING.VENDOR ORDER BY 1;
```

查询结果如下：

```
行号          PUBLISHER
---------- ------------------
1
2             21 世纪出版社
3             北京十月文艺出版社
```

```
4              长江文艺出版社
5              广州出版社
6              机械工业出版社
7              清华大学出版社
8              人民文学出版社
9              人民邮电出版社
10             上海出版社
11             上海画报出版社
12             外语教学与研究出版社
13             文学出版社
14             中华书局
```

例 2 UNION ALL。

```
SELECT  PUBLISHER  FROM  PRODUCTION.PRODUCT
UNION ALL
SELECT  NAME  FROM  PURCHASING.VENDOR ORDER BY 1;
```

查询结果如下:

```
行号          PUBLISHER
---------- ------------------
1
2              21 世纪出版社
3              21 世纪出版社
4              北京十月文艺出版社
5              长江文艺出版社
6              广州出版社
```

```
7              广州出版社
8              机械工业出版社
9              机械工业出版社
10             清华大学出版社
11             清华大学出版社
12             人民文学出版社
13             人民邮电出版社
14             上海出版社
15             上海出版社
16             上海画报出版社
17             外语教学与研究出版社
18             外语教学与研究出版社
19             文学出版社
20             中华书局
21             中华书局
22             中华书局
```

例 3 查询所有图书的出版商，查询所有图书供应商的名称，返回两者的差集，并去掉重复行。

```
SELECT PUBLISHER FROM PRODUCTION.PRODUCT
EXCEPT
SELECT NAME FROM PURCHASING.VENDOR ORDER BY 1;
```

查询结果如下：

```
行号          PUBLISHER
---------- ---------------
```

```
1

2              人民文学出版社
```

例 4 查询所有图书的出版商，查询所有图书供应商的名称，返回两者的交集，并去掉重复行。

```
SELECT PUBLISHER FROM PRODUCTION.PRODUCT

INTERSECT

SELECT NAME FROM PURCHASING.VENDOR ORDER BY 1;
```

查询结果如下：

```
行号          PUBLISHER

---------- --------------------

1             21 世纪出版社

2             广州出版社

3             机械工业出版社

4             清华大学出版社

5             上海出版社

6             外语教学与研究出版社

7             中华书局
```

4.6 GROUP BY 和 HAVING 子句

GROUP BY 子句主要用法是分类汇总。GROUP BY 子句一般与集函数一起使用，常见集函数包括 COUNT、SUM、AVG、MAX 和 MIN 等。

GROUP BY 子句将由 WHERE 子句返回的临时结果按一个或多个分组项进行分组，分组项相同的为一组。分组结果是行的集合。

HAVING 子句用于为组设置检索条件。

4.6.1 GROUP BY 子句的使用

GROUP BY 子句是 SELECT 语句的可选项部分。它定义了分组表。

语法如下:

```
<GROUP BY 子句> ::= GROUP BY <group_by 项>

<group_by 项>::=<group_by 子项 > {,<group_by 子项>}

<group_by 子项>::=<group_by 分组项> | <ROLLUP 项> | <CUBE 项> | <GROUPING SETS 项>

<group_by 分组项>=<分组项集>|

                    <分组项集>{,<分组项集>}

<分组项集>::=<分组项>|（<分组项>{,<分组项>}）| ()

<分组项>::=<列名>|<值表达式>

<ROLLUP 项>::=ROLLUP (<rollup 分组项>)

<rollup 分组项>::= <列名> | <值表达式>

<CUBE 项>::=CUBE (<cube 分组项>)

<cube 分组项>::= <列名> | <值表达式>

<GROUPING SETS 项>::=GROUPING SETS(<group_by 分组项>)
```

参数

<分组项>用于指定分组列。分组列可以是<列名>、<值表达式>。<值表达式>的值 N 在 COMPATIBLE_MODE=4、6、7 时表示将查询列中的第 N 列作为分组列，其他情况下表示按照常量 N 分组，无实际意义。

<分组项集>用于指定分组方式。分组项集可以是单个分组项、多个分组项的集合或()，()表示将当前所有数据作为一个整体。当分组项集用于 GROUP BY 时，多个分组项集涉及的分组列会被合并去重，并按照出现顺序确定最终分组列进行分组。当分组项集用于 GROUPING SETS 时，将按照每个分组项集独立进行分组，最后把每个分组结果进行 UNION

ALL 输出最终结果，这种情况下()不能单独使用，必须和其它分组项集组合使用。

当存在分组项时，若查询项包含列，则只能是分组项中的列，或者是作用于当前分组的集函数。当 GROUP_OPT_FLAG 参数取值包含 1 或 COMPATIBLE_MODE=4 时，也允许查询项为非分组项的列，此时该非分组列返回的是集函数 FIRST_VALUE（非分组列）的值。

例 1 统计每个部门的员工数。

```
SELECT DEPARTMENTID,COUNT(*) FROM RESOURCES.EMPLOYEE_DEPARTMENT GROUP BY DEPARTMENTID;
```

查询结果如下:

```
DEPARTMENTID COUNT(*)
------------ --------------------
2                   3
1                   2
3                   1
4                   1
```

系统执行此语句时，首先将 EMPLOYEE_DEPARTMENT 表按 DEPARTMENTID 列进行分组，相同的 DEPARTMENTID 为一组，然后对每一组使用集函数 COUNT(*)，统计该组内的记录个数，如此继续，直到处理完最后一组，返回查询结果。

如果存在 WHERE 子句，系统先根据 WHERE 条件进行过滤，然后对满足条件的记录进行分组。

此外，GROUP BY 不会对结果集排序。如果需要排序，可以使用 ORDER BY 子句。

例 2 求小说类别包含的子类别所对应的产品数量，并按子类别编号的升序排列。

```
SELECT A1.PRODUCT_SUBCATEGORYID AS 子分类编号,A3.NAME AS 子分类名,count(*)AS 数量
FROM PRODUCTION.PRODUCT A1,
PRODUCTION.PRODUCT_CATEGORY A2,
```

```
PRODUCTION.PRODUCT_SUBCATEGORY A3
WHERE A1.PRODUCT_SUBCATEGORYID=A3.PRODUCT_SUBCATEGORYID
AND   A2.PRODUCT_CATEGORYID=A3.PRODUCT_CATEGORYID
AND   A2.NAME='小说'
GROUP   BY   A1.PRODUCT_SUBCATEGORYID,A3.NAME
ORDER   BY   A1.PRODUCT_SUBCATEGORYID;
```

查询结果如下:

```
子分类编号      子分类名    数量
----------- -------- --------------------
1               世界名著    1
2               武侠        1
4               四大名著    2
```

使用 GROUP BY 要注意以下问题:

1. 分组列不能为集函数表达式或者在 SELECT 子句中定义的别名;
2. 当分组列值包含空值时，则空值作为一个独立组;
3. 当分组列包含多个列名时，则按照 GROUP BY 子句中列出现的顺序进行分组;
4. GROUP BY 子句中至多可包含 512 个分组项;
5. ROLLUP\CUBE\GROUPING SETS 组合项不能超过 8 个，且组合项产生的分支数不能超过 512 个，否则将触发报错。

4.6.2 ROLLUP 的使用

ROLLUP 主要用于统计分析，对分组列以及分组列的部分子集进行分组，输出用户需要的结果。

语法如下:

```
GROUP BY ROLLUP (<rollup 分组项>{,<rollup 分组项>} )
<rollup 分组项>::= <列名> | <值表达式>
```

假如 ROLLUP 分组列为(A, B, C)，首先对(A,B,C)进行分组，然后对(A,B)进行分组，接着对(A)进行分组，最后对全表进行查询，无分组列，其中查询项中出现在 ROLLUP 中的列设为 NULL。查询结果是把每种分组的结果集进行 UNION ALL 合并输出。如果分组列为 n 列，则共有 n+1 种组合方式。

例 按小区住址和所属行政区域统计员工居住分布情况。

```
SELECT CITY , ADDRESS1, COUNT(*) as NUMS FROM PERSON.ADDRESS GROUP BY ROLLUP(CITY, ADDRESS1);
```

查询结果如下:

```
CITY            ADDRESS1                          NUMS
------------ ------------------------------ --------------------
武汉市洪山区    洪山区 369 号金地太阳城 56-1-202     1
武汉市洪山区    洪山区 369 号金地太阳城 57-2-302     1
武汉市青山区    青山区青翠苑 1 号                    1
武汉市武昌区    武昌区武船新村 115 号                1
武汉市汉阳区    汉阳大道熊家湾 15 号                 1
武汉市洪山区    洪山区保利花园 50-1-304          1
武汉市洪山区    洪山区保利花园 51-1-702          1
武汉市洪山区    洪山区关山春晓 51-1-702          1
武汉市江汉区    江汉区发展大道 561 号                1
武汉市江汉区    江汉区发展大道 555 号                    1
武汉市武昌区    武昌区武船新村 1 号                  1
武汉市江汉区    江汉区发展大道 423 号                1
```

```
武汉市洪山区    洪山区关山春晓 55-1-202          1
武汉市洪山区    洪山区关山春晓 10-1-202          1
武汉市洪山区    洪山区关山春晓 11-1-202          1
武汉市洪山区    洪山区光谷软件园 C1_501          1
武汉市洪山区    NULL                             9
武汉市青山区    NULL                             1
武汉市武昌区    NULL                             2
武汉市汉阳区    NULL                             1
武汉市江汉区    NULL                             3
NULL            NULL                             16
```

上例中的查询等价于:

```
SELECT CITY , ADDRESS1, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY
CITY, ADDRESS1
UNION ALL
SELECT CITY , NULL, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY  CITY
UNION ALL
SELECT NULL , NULL, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS GROUP  BY  0;
```

使用 ROLLUP 要注意以下事项:

1．ROLLUP 项不能包含集函数;

2．不支持包含 WITH FUNCTION 的相关查询;

3．不支持包含存在 ROLLUP 的嵌套相关子查询;

4．不支持数组查询;

5．ROLLUP 项最多支持 511 个;

6．ROLLUP 项不能引用外层列。

4.6.3 CUBE 的使用

CUBE 的使用场景与 ROLLUP 类似，常用于统计分析，对分组列以及分区列的所有子集进行分组，输出所有分组结果。

语法如下：

```
GROUP BY CUBE (<cube 分组项>{,<cube 分组项>})
<cube 分组项>::= <列名> | <值表达式>
```

假如，CUBE 分组列为(A, B, C)，则首先对(A,B,C)进行分组，然后依次对(B,C)、(A,C)、(C)、(A,B)、(B)、(A)六种情况进行分组，最后对全表进行查询，无分组列，其中查询项存在于 CUBE 列表的列设置为 NULL。输出为每种分组的结果集进行 UNION ALL。CUBE 分组共有 2^n 种组合方式。CUBE 最多支持 9 列。

例 按小区住址、所属行政区域统计员工居住分布情况。

```
SELECT CITY , ADDRESS1, COUNT(*) AS NUMS FROM PERSON.ADDRESS GROUP BY
CUBE(CITY, ADDRESS1);
```

查询结果如下：

```
CITY              ADDRESS1                          NUMS
------------ ----------------------------- --------------------
武汉市洪山区    洪山区 369 号金地太阳城 56-1-202     1
武汉市洪山区    洪山区 369 号金地太阳城 57-2-302     1
武汉市青山区    青山区青翠苑 1 号                    1
武汉市武昌区    武昌区武船新村 115 号                1
武汉市汉阳区    汉阳大道熊家湾 15 号                 1
武汉市洪山区    洪山区保利花园 50-1-304          1
武汉市洪山区    洪山区保利花园 51-1-702          1
```

武汉市洪山区	洪山区关山春晓 51-1-702	1	
武汉市江汉区	江汉区发展大道 561 号		1
武汉市江汉区	江汉区发展大道 555 号		1
武汉市武昌区	武昌区武船新村 1 号		1
武汉市江汉区	江汉区发展大道 423 号		1
武汉市洪山区	洪山区关山春晓 55-1-202	1	
武汉市洪山区	洪山区关山春晓 10-1-202	1	
武汉市洪山区	洪山区关山春晓 11-1-202	1	
武汉市洪山区	洪山区光谷软件园 C1_501	1	
NULL	洪山区 369 号金地太阳城 56-1-202		1
NULL	洪山区 369 号金地太阳城 57-2-302		1
NULL	青山区青翠苑 1 号		1
NULL	武昌区武船新村 115 号		1
NULL	汉阳大道熊家湾 15 号		1
NULL	洪山区保利花园 50-1-304	1	
NULL	洪山区保利花园 51-1-702		1
NULL	洪山区关山春晓 51-1-702		1
NULL	江汉区发展大道 561 号		1
NULL	江汉区发展大道 555 号		1
NULL	武昌区武船新村 1 号		1
NULL	江汉区发展大道 423 号		1
NULL	洪山区关山春晓 55-1-202		1
NULL	洪山区关山春晓 10-1-202		1
NULL	洪山区关山春晓 11-1-202		1

```
NULL            洪山区光谷软件园 C1_501          1
武汉市洪山区    NULL                             9
武汉市青山区    NULL                             1
武汉市武昌区    NULL                             2
武汉市汉阳区    NULL                             1
武汉市江汉区    NULL                             3
NULL            NULL                             16
```

上例中的查询等价于：

```
SELECT CITY , ADDRESS1, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY
CITY, ADDRESS1
UNION ALL
SELECT CITY , NULL, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY  CITY
UNION ALL
SELECT NULL , ADDRESS1, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY
ADDRESS1
UNION ALL
SELECT NULL , NULL, COUNT(*) AS NUMS  FROM  PERSON.ADDRESS;
```

使用 CUBE 要注意以下事项：

1. CUBE 项不能包含集函数；
2. 不支持包含 WITH FUNCTION 的相关查询；
3. 不支持包含存在 CUBE 的嵌套相关子查询；
4. 不支持数组查询；
5. CUBE 项最多支持 9 个；
6. CUBE 项不能引用外层列。

4.6.4 GROUPING 的使用

GROUPING 可以视为集函数，一般用于含 GROUP BY 的语句中，标识某子结果集是否是按指定分组项分组的结果，如果是，GROUPING 值为 0；否则为 1。

语法如下：

```
<GROUPING 项>::=GROUPING (<grouping 分组项>)
<grouping 分组项>::= <列名> | <值表达式>
```

使用约束说明：

1. GROUPING 中只能包含一列；
2. GROUPING 只能在 GROUP BY 查询中使用；
3. GROUPING 不能在 WHERE 或连接条件中使用；
4. GROUPING 支持表达式运算。例如 GROUPING(c1) + GROUPING(c2)。

例 按小区住址和所属行政区域统计员工居住分布情况。

```
SELECT GROUPING(CITY) AS G_CITY,GROUPING(ADDRESS1) AS G_ADD, CITY , ADDRESS1,
COUNT(*) AS NUMS FROM PERSON.ADDRESS GROUP BY ROLLUP(CITY, ADDRESS1);
```

查询结果如下：

```
G_CITY    G_ADD    CITY          ADDRESS1                        NUMS
------    -------  ----------  --------------------          -----------
0         0        武汉市洪山区   洪山区 369 号金地太阳城 56-1-202   1
0         0        武汉市洪山区   洪山区 369 号金地太阳城 57-2-302   1
0         0        武汉市青山区   青山区青翠苑 1 号                  1
0         0        武汉市武昌区   武昌区武船新村 115 号              1
0         0        武汉市汉阳区   汉阳大道熊家湾 15 号               1
0         0        武汉市洪山区   洪山区保利花园 50-1-304            1
```

0	0	武汉市洪山区	洪山区保利花园 51-1-702		1
0	0	武汉市洪山区	洪山区关山春晓 51-1-702		1
0	0	武汉市江汉区	江汉区发展大道 561 号		1
0	0	武汉市江汉区	江汉区发展大道 555 号		1
0	0	武汉市武昌区	武昌区武船新村 1 号	1	
0	0	武汉市江汉区	江汉区发展大道 423 号		1
0	0	武汉市洪山区	洪山区关山春晓 55-1-202		1
0	0	武汉市洪山区	洪山区关山春晓 10-1-202		1
0	0	武汉市洪山区	洪山区关山春晓 11-1-202		1
0	0	武汉市洪山区	洪山区光谷软件园 C1_501		1
0	1	武汉市洪山区	NULL		9
0	1	武汉市青山区	NULL		1
0	1	武汉市武昌区	NULL		2
0	1	武汉市汉阳区	NULL		1
0	1	武汉市江汉区	NULL		3
1	1	NULL	NULL	16	

4.6.5 GROUPING SETS 的使用

GROUPING SETS 是对 GROUP BY 的扩展，可以指定不同的列进行分组，每个分组列集作为一个分组单元。使用 GROUPING SETS，用户可以灵活的指定分组方式，避免 ROLLUP/CUBE 过多的分组情况，满足实际应用需求。

GROUPING SETS 的分组过程为依次按照每一个分组单元进行分组，最后把每个分组结果进行 UNION ALL 输出最终结果。如果查询项不属于分组列，则用 NULL 代替。

语法如下：

```
GROUP BY GROUPING SETS(<group_by 分组项>)

<group_by 分组项>::=请参考 GROUP BY 子句
```

例 按照邮编、住址和行政区域统计员工住址分布情况。

```
SELECT CITY , ADDRESS1, POSTALCODE, COUNT(*) AS NUMS   FROM   PERSON.ADDRESS

GROUP   BY   GROUPING SETS((CITY, ADDRESS1), POSTALCODE);
```

查询结果如下:

```
      CITY              ADDRESS1                          POSTALCODE     NUMS

------------ -------------------------     ----------    -------------

武汉市洪山区     洪山区 369 号金地太阳城 56-1-202     NULL             1

武汉市洪山区     洪山区 369 号金地太阳城 57-2-302     NULL             1

武汉市青山区     青山区青翠苑 1 号                    NULL             1

武汉市武昌区     武昌区武船新村 115 号                NULL             1

武汉市汉阳区     汉阳大道熊家湾 15 号                 NULL             1

武汉市洪山区     洪山区保利花园 50-1-304              NULL             1

武汉市洪山区     洪山区保利花园 51-1-702              NULL          1

武汉市洪山区     洪山区关山春晓 51-1-702              NULL          1

武汉市江汉区     江汉区发展大道 561 号                NULL             1

武汉市江汉区     江汉区发展大道 555 号                NULL             1

武汉市武昌区     武昌区武船新村 1 号                  NULL             1

武汉市江汉区     江汉区发展大道 423 号                NULL             1

武汉市洪山区     洪山区关山春晓 55-1-202              NULL             1

武汉市洪山区     洪山区关山春晓 10-1-202              NULL             1

武汉市洪山区     洪山区关山春晓 11-1-202              NULL             1

武汉市洪山区     洪山区光谷软件园 C1_501              NULL             1
```

```
NULL        NULL                    430073      9
NULL        NULL                    430080      1
NULL        NULL                    430063      2
NULL        NULL                    430050      1
NULL        NULL                    430023      3
```

上例中的查询等价于：

```
SELECT CITY , ADDRESS1, NULL , COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP
BY  CITY, ADDRESS1
UNION ALL
SELECT NULL , NULL, POSTALCODE ,COUNT(*) AS NUMS  FROM  PERSON.ADDRESS  GROUP  BY
POSTALCODE;
```

4.6.6 GROUPING_ID 的使用

GROUPING_ID 表示参数列是否为分组列。返回值的每一个二进制位表示对应的参数列是否为分组列，如果是分组列，该位值为 0；否则为 1。

使用 GROUPING_ID 可以按照列的分组情况过滤结果集。

语法如下：

```
<GROUPING_ID 项>::=GROUPING_ID (<grouping_id 分组项>{,<grouping_id 分组项>)
<grouping_id 分组项>::= <列名> | <值表达式>
```

使用约束说明：

1. GROUPING_ID 中至少包含一列，最多包含 63 列；
2. GROUPING_ID 只能与分组项一起使用；
3. GROUPING_ID 支持表达式运算；
4. GROUPING_ID 作为分组函数，不能出现在 where 或连接条件中。

例 按小区住址和所属行政区域统计员工居住分布情况。

```
SELECT GROUPING(CITY) AS G_CITY,GROUPING(ADDRESS1) AS G_ADD,
    GROUPING_ID(CITY, ADDRESS1) AS G_CA,CITY , ADDRESS1, COUNT(*) AS NUMS
FROM  PERSON.ADDRESS  GROUP  BY  ROLLUP(CITY, ADDRESS1);
```

查询结果如下:

```
G_CITY   G_ADD    G_CA     CITY          ADDRESS1                          NUMS
------ ------  ------  ---------   --------------------------  ------
0        0         0         武汉市洪山区   洪山区 369 号金地太阳城 56-1-202   1
0        0         0         武汉市洪山区   洪山区 369 号金地太阳城 57-2-302   1
0        0         0         武汉市青山区   青山区青翠苑 1 号                  1
0        0         0         武汉市武昌区   武昌区武船新村 115 号              1
0        0         0         武汉市汉阳区   汉阳大道熊家湾 15 号               1
0        0         0         武汉市洪山区   洪山区保利花园 50-1-304            1
0        0         0         武汉市洪山区   洪山区保利花园 51-1-702            1
0        0         0         武汉市洪山区   洪山区关山春晓 51-1-702            1
0        0         0         武汉市江汉区   江汉区发展大道 561 号              1
0        0         0         武汉市江汉区   江汉区发展大道 555 号              1
0        0         0         武汉市武昌区   武昌区武船新村 1 号                1
0        0         0         武汉市江汉区   江汉区发展大道 423 号              1
0        0         0         武汉市洪山区   洪山区关山春晓 55-1-202            1
0        0         0         武汉市洪山区   洪山区关山春晓 10-1-202            1
0        0         0         武汉市洪山区   洪山区关山春晓 11-1-202            1
0        0         0         武汉市洪山区   洪山区光谷软件园 C1_501            1
0        1         1         武汉市洪山区   NULL                               9
```

```
0        1        1        武汉市青山区   NULL                          1
0        1        1        武汉市武昌区   NULL                          2
0        1        1        武汉市汉阳区   NULL                          1
0        1        1        武汉市江汉区   NULL                          3
1        1        3        NULL           NULL                          16
```

4.6.7 GROUP_ID 的使用

GROUP_ID 表示结果集来自于哪一个分组，用于区别相同分组的结果集。如果有 N 个相同分组，则 GROUP_ID 取值范围为 0~N-1。每组的初始值为 0。

当查询包含多个分组时，使用 GROUP_ID 可以方便的过滤相同分组的结果集。

语法如下：

```
<GROUP_ID 项>::=GROUP_ID()
```

使用约束说明：

1. GROUP_ID 不包含参数；
2. GROUP_ID 只能与分组项一起使用；
3. GROUP_ID 支持表达式运算；
4. GROUP_ID 作为分组函数，不能出现在 WHERE 或连接条件中。

例 按小区住址和所属行政区域统计员工居住分布情况。

```
SELECT GROUPING(CITY) AS G_CITY,GROUPING(ADDRESS1) AS G_ADD,
    GROUP_ID() AS GID,CITY , ADDRESS1, COUNT(*) AS NUMS
FROM  PERSON.ADDRESS  GROUP  BY  ROLLUP(CITY, ADDRESS1), CITY;
```

查询结果如下：

```
G_CITY G_ADD GID   CITY           ADDRESS1                          NUMS
------ ----- ---- ---------  ------------------------------      -----
```

0	0	0	武汉市洪山区 洪山区 369 号金地太阳城 56-1-202	1
0	0	0	武汉市洪山区 洪山区 369 号金地太阳城 57-2-302	1
0	0	0	武汉市青山区 青山区青翠苑 1 号	1
0	0	0	武汉市武昌区 武昌区武船新村 115 号	1
0	0	0	武汉市汉阳区 汉阳大道熊家湾 15 号	1
0	0	0	武汉市洪山区 洪山区保利花园 50-1-304	1
0	0	0	武汉市洪山区 洪山区保利花园 51-1-702	1
0	0	0	武汉市洪山区 洪山区关山春晓 51-1-702	1
0	0	0	武汉市江汉区 江汉区发展大道 561 号	1
0	0	0	武汉市江汉区 江汉区发展大道 555 号	1
0	0	0	武汉市武昌区 武昌区武船新村 1 号	1
0	0	0	武汉市江汉区 江汉区发展大道 423 号	1
0	0	0	武汉市洪山区 洪山区关山春晓 55-1-202	1
0	0	0	武汉市洪山区 洪山区关山春晓 10-1-202	1
0	0	0	武汉市洪山区 洪山区关山春晓 11-1-202	1
0	0	0	武汉市洪山区 洪山区光谷软件园 C1_501	1
0	1	1	武汉市洪山区 NULL	9
0	1	1	武汉市青山区 NULL	1
0	1	1	武汉市武昌区 NULL	2
0	1	1	武汉市汉阳区 NULL	1
0	1	1	武汉市江汉区 NULL	3
0	1	0	武汉市洪山区 NULL	9
0	1	0	武汉市青山区 NULL	1
0	1	0	武汉市武昌区 NULL	2

```
0        1        0        武汉市汉阳区  NULL                                    1

0        1        0        武汉市江汉区  NULL                                    3
```

4.6.8 HAVING 子句的使用

HAVING 子句是 SELECT 语句的可选项部分，它也定义了一个分组表。

语法如下：

```
<HAVING 子句> ::= HAVING <搜索条件>

<搜索条件>::= <表达式>
```

HAVING 子句定义了一个分组表，其中只含有搜索条件为 TRUE 的那些组，且通常跟随一个 GROUP BY 子句。HAVING 子句与组的关系正如 WHERE 子句与表中行的关系。WHERE 子句用于选择表中满足条件的行，而 HAVING 子句用于选择满足条件的组。

例 统计出同一子类别的产品数量大于 1 的子类别名称，数量，并按数量从小到大的顺序排列。

```
SELECT A2.NAME AS 子分类名, COUNT (*)AS 数量

FROM PRODUCTION.PRODUCT A1,

PRODUCTION.PRODUCT_SUBCATEGORY A2

WHERE A1.PRODUCT_SUBCATEGORYID=A2.PRODUCT_SUBCATEGORYID

GROUP  BY  A2.NAME

HAVING COUNT(*)>1

ORDER  BY  2;
```

查询结果如下：

```
子分类名     数量

-------- --------------------

四大名著     2
```

系统执行此语句时，首先将 PRODUCT 表和 PRODUCT_SUBCATEGORY 表中的各行按相同的 SUBCATEGORYID 作连接，再按子类别名的取值进行分组，相同的子类别名为一组，然后对每一组使用集函数 COUNT(*)，统计该组内产品的数量，如此继续，直到最后一组。再选择产品数量大于 1 的组作为查询结果。

4.7 ORDER BY 子句

ORDER BY 子句可以选择性地出现在<查询表达式>之后，它规定了当行由查询返回时应具有的顺序。ORDER BY 子句的语法如下：

```
<ORDER BY 子句> ::= ORDER [SIBLINGS] BY <order_by_list>
<order_by_list>::= <order_by_item >{,<order_by_item>}
<order_by_item>::=  <exp> [COLLATE <collation_name>] [ASC|DESC] [NULLS FIRST|LAST]
<exp>::=<列说明>|<无符号整数>|<值表达式>|<布尔表达式>
```

使用说明

1. ORDER BY 子句提供了要排序的项目清单和他们的排序顺序：递增顺序(ASC，默认)或是递减顺序(DESC)。它必须跟随在<查询表达式>之后，因为它是在查询计算得出的最终结果上进行操作的；

2. SIBLINGS 关键字必须与 CONNECT BY 一起配合使用，专门用于指定层次查询中相同层次数据返回的顺序。详见 4.13.5 层次查询层内排序；

3. <exp> 用于指定排序列。支持<列说明>、<无符号整数>、<值表达式>或<布尔表达式>四种指定方式；

4. COLLATE 关键字指定该排序项根据 collation_name 中指定的排序规则进行排序。目前 collation_name 仅支持指定为 Chinese_PRC_CS_AS_KS_WS，当指定为该规则时，该排序项按照中文拼音排序。COLLATE 关键字仅适用于 VARCHAR 类型的列；

5. <列说明> 用于指定排序列。排序列可以是任何在查询清单中的列名、可以是列计

算的表达式（即使这一列不在选择清单中）、可以是子查询、也可以是 NULL。

当排序列包含多个列名时，系统则按列名从左到右排列的顺序，先按左边列将查询结果排序，当左边排序列值相等时，再按右边排序列排序……如此右推，逐个检查调整，最后得到排序结果。

当排序列为列计算表达式时，则按照该列进行排序。例如，当排序列为 3*C1 时，系统则按照每一行的排序键值为 3*C1 进行排序。当排序列为 ORDER BY C1=3 ASC 时，则将符合 C1=3 的列放在整个结果集的最后面，不符合 C1=3 的列只输出在结果集中但不排序。

当排序列为子查询时，则按照子查询的结果进行排序。例如，在 SELECT * FROM T ORDER BY (SELECT C1 FROM T WHERE C2=10);中子查询 SELECT C1 FROM T WHERE C2=10 的结果为 5，那么系统按照每一行的排序键值均为 5 进行排序。

当排序列为 NULL 时，不进行排序。

对于 UNION 查询语句，排序键必须在第一个查询子句中出现；对于 GROUP BY 分组的排序，排序键可以使用集函数，但 GROUP BY 分组中必须包含查询列中所有列；

6. <无符号整数> 指定排序键的序号，对应 SELECT 后查询结果列的序号。当用<无符号整数>代替列名时，<无符号整数>不应大于 SELECT 后查询结果列的个数。例如，在 SELECT C1,C2 FROM T ORDER BY 2;中表示按照查询列中的第二列进行排序。但是如果例子中使用 ORDER BY 3，则因查询结果列只有 2 列，无法进行排序，系统将会报错。

如果排序列不是数值类型，则会直接被忽略。例如，在 SELECT C1,C2 FROM T ORDER BY '2';中，ORDER BY '2'会被直接忽略掉；

7. <值表达式> 语法支持，实际不起作用。例如，值表达式(如：-1，3×6)作为排序列，将不影响最终结果表的行输出顺序。SELECT * FROM T ORDER BY 3*6;和 SELECT * FROM T;效果一样；

8. <布尔表达式> 如果指定了布尔表达式，则按照布尔表达式的结果 1 或 0 进行排序。例如：在 SELECT * FROM T ORDER BY 1=2;语句中，因为布尔表达式 1=2 结果为 0，那么

系统按照每一行的排序键值均为 0 进行排序;

9. 当排序列值包含 NULL 时,根据指定的"NULLS FIRST|LAST"决定包含空值的行是排在最前还是最后,缺省为 NULLS FIRST;

10. 由于 ORDER BY 只能在最终结果上操作,不能将其放在查询中;

11. 如果 ORDER BY 后面使用集函数,则必须使用 GROUP BY 分组,且 GROUP BY 分组中必须包含查询列中所有列;

12. ORDER BY 子句中至多可包含 255 个排序列。

例 1 将 RESOURCES.DEPARTMENT 表中的资产总值按从大到小的顺序排列。

```
SELECT * FROM RESOURCES.DEPARTMENT ORDER BY DEPARTMENTID DESC;
```

等价于:

```
SELECT * FROM RESOURCES.DEPARTMENT ORDER BY 1 DESC;
```

查询结果如下:

```
DEPARTMENTID NAME

------------ --------

5            广告部

4            行政部门

3            人力资源

2            销售部门

1            采购部门
```

例 2 因为查询列只有两列,而 ORDER BY 指定了第 3 列为排序列,则报错。

```
SELECT DEPARTMENTID,NAME FROM RESOURCES.DEPARTMENT ORDER BY 3;
```

系统报错:无效的 ORDER BY 语句。

4.8 FOR UPDATE 子句

FOR UPDATE 子句可以选择性地出现在<查询表达式>之后。普通 SELECT 查询不会修改行数据物理记录上的 TID 事务号，FOR UPDATE 会修改行数据物理记录上的 TID 事务号并对该 TID 上锁，以保证该更新操作的待更新数据不被其他事务修改。

语法格式

```
<FOR UPDATE 子句> ::=
    FOR READ ONLY |
    <FOR UPDATE 选项>
<FOR UPDATE 选项> ::= FOR UPDATE [OF <选择列表>] [<面对上锁的处理方式>]
<面对上锁的处理方式>: : =
    NOWAIT |
    WAIT N |
    [N] SKIP LOCKED
<选择列表>: : =<选择列> {,<选择列>}
<选择列>: : =
    [<模式名>.]<基表名>.] <列名> |
    [[<模式名>.]<视图名>.] <列名>
```

参数

1. FOR READ ONLY 表示查询不可更新;

2. OF <选择列表> 指定待更新表的列。指定某张表的列，即为锁定某张表。游标更新时，仅能更新指定的列;

3. <面对上锁的处理方式>用于指定试图上锁的行数据 TID 已经被其他事务上锁后，当前事务的处理方式:

    1) NOWAIT 表示不等待，直接报错返回;

2） WAIT N 表示等待一段时间，其中的 N 值由用户指定，单位为秒。等待成功继续上锁，失败则报错返回。WAIT 的指定值必须大于 0，如果设置 0 自动转成 NOWAIT 方式；

3） [N] SKIP LOCKED 表示上锁时跳过已经被其他事务锁住的行，不返回这些行给客户端。N 是正整数。N>0 表示当取得了 N 条数据后，便不再取数据了，直接返回 N 条结果（DMDPC 环境下实际封锁的行数可能会大于 N）；N=0 表示返回除了锁定行之外的所有行；

4） 如果 FOR UPDATE 不设置以上三种子句，则会一直等待锁被其他事务释放；

5） INI 参数 LOCK_TID_MODE 用来标记 SELECT FOR UPDATE 封锁方式。0 表示结果集小于 100 行时，直接封锁 TID，超过 100 行升级为表锁。1 表示不升级表锁，一律使用 TID 锁。默认为 1。

例 查询 RESOURCES.DEPARTMENT 表中的资产。

```
SELECT * FROM RESOURCES.DEPARTMENT FOR UPDATE;
//只要 FOR UPDATE 语句不提交，其他会话就不能修改此结果集。
```

查询结果如下：

```
DEPARTMENTID NAME
------------ --------
1            采购部门
2            销售部门
3            人力资源
4            行政部门
5            广告部
```

需要说明的是：

1. SELECT FOR UPDATE 不支持的情况：

1） 带 GROUP BY 子句的查询，如 SELECT C1, COUNT(C2) FROM TEST GROUP BY C1 FOR UPDATE;

2） 带集函数的查询，如 SELECT MAX(C1)FROM TEST FOR UPDATE;

3） 带 DISTINCT 的查询，如 SELECT DISTINCT C1 FROM TEST FOR UPDATE;

4） 对以下表类型的查询：外部表、物化视图、系统表和 HUGE 表;

5） WITH 子句，如 WITH TEST(C1) AS (SELECT C1 FROM T FOR UPDATE)SELECT * FROM TEST;

6） 带层次查询子句的查询;

7） 带 PIVOT 子句或 UNPIVOT 子句的查询。

2. 涉及 DBLINK 的 SELECT FOR UPDATE 查询支持外部链接的单表、多表查询，不支持外部链接表与模式内的表的连接查询;

3. 如果结果集中包含 LOB 对象，会再封锁 LOB 对象;

4. 支持多表连接的情况，会封锁涉及到的所有表的行数据;

5. 多表连接的时候，如果用 OF <选择列表>指定具体列，只会检测和封锁对应的表。例如：SELECT C1 FROM TEST, TESTB FOR UPDATE OF TEST.C1 即使 TESTB 表类型不支持 FOR UPDATE，上述语句还是可以成功;

6. DM 数据守护环境中，备库执行 SELECT FOR UPDATE 查询时，会直接忽略其中的 FOR UPDATE 子句。

4.9 TOP 子句

在 DM 中，可以使用 TOP 子句来筛选结果。语法如下:

```
<TOP 子句>::=TOP <n>
                    | <n1>,<n2>
                    | <n> PERCENT
```

```
                    | <n> WITH TIES
                    | <n> PERCENT WITH TIES
<n>::=整数(>=0)
```

参数

1. TOP <n> 选择结果的前 n 条记录;
2. TOP <n1>,<n2> 选择第 n1 条记录之后的 n2 条记录;
3. TOP <n> PERCENT 表示选择结果的前 n%条记录;
4. TOP <n> PERCENT WITH TIES 表示选择结果的前 n%条记录，同时指定结果集可以返回额外的行。额外的行是指与最后一行以相同的排序键排序的所有行。WITH TIES 必须与 ORDER BY 子句同时出现，如果没有 ORDER BY 子句，则忽略 WITH TIES。

举例说明

例 1 查询现价最贵的两种产品的编号和名称。

```
SELECT TOP 2 PRODUCTID,NAME FROM PRODUCTION.PRODUCT ORDER BY NOWPRICE DESC;
```

查询结果如下:

```
PRODUCTID      NAME
---------- ------------------
10              噼里啪啦丛书(全 7 册)
6               长征
```

例 2 查询现价第二贵的产品的编号和名称。

```
SELECT TOP 1,1 PRODUCTID,NAME FROM PRODUCTION.PRODUCT ORDER BY NOWPRICE DESC;
```

查询结果如下:

```
PRODUCTID      NAME
---------- ----
```

```
6                长征
```

例3 查询最新出版日期的70%的产品编号、名称和出版日期。

```
SELECT TOP 70 PERCENT WITH TIES PRODUCTID,NAME,PUBLISHTIME

FROM PRODUCTION.PRODUCT ORDER BY PUBLISHTIME DESC;
```

查询结果如下:

```
PRODUCTID      NAME                                PUBLISHTIME

---------- -------------------------------- -----------

7              数据结构(C 语言版)(附光盘)          2007-03-01

5              鲁迅文集(小说、散文、杂文)全两册    2006-09-01

6              长征                                2006-09-01

3              老人与海                            2006-08-01

8              工作中无小事                        2006-01-01

4              射雕英雄传(全四册)                  2005-12-01

2              水浒传                              2005-04-01

1              红楼梦                              2005-04-01
```

4.10 LIMIT 限定条件

在 DM 中，可以使用限定条件对结果集做出筛选，支持 LIMIT 子句和 ROW_LIMIT 子句两种方式。

4.10.1 LIMIT 子句

LIMIT 子句按顺序选取结果集中某条记录开始的 N 条记录。语法如下

```
<LIMIT 子句>::=<LIMIT 子句 1> | <LIMIT 子句 2>

<LIMIT 子句 1>::= LIMIT <记录数>
```

```
                                |<偏移量>,<记录数>

                                |<记录数> OFFSET <偏移量>

<LIMIT 子句 2>::= OFFSET <偏移量> LIMIT <记录数>

<记录数>::=<整数>

<偏移量>::=<整数>
```

共支持四种方式：

1. LIMIT N：选择前 N 条记录；
2. LIMIT M , N：选择第 M 条记录之后的 N 条记录；
3. LIMIT M OFFSET N：选择第 N 条记录之后的 M 条记录；
4. OFFSET N LIMIT M：选择第 N 条记录之后的 M 条记录。

注意：

1. 一个查询表达式中不能同时包含 LIMIT 和 TOP 子句；
2. LIMIT 子句可用在 UPDATE 或 DELETE 操作之后，形式如“DELETE FROM T1 LIMIT 2;”。

例 1 查询前 2 条记录

```
SELECT PRODUCTID , NAME    FROM    PRODUCTION.PRODUCT    LIMIT 2;
```

查询结果如下：

```
PRODUCTID       NAME

----------- ------

1                   红楼梦

2                    水浒传
```

例 2 查询第 3，4 个登记的产品的编号和名称。

```
SELECT PRODUCTID, NAME    FROM    PRODUCTION.PRODUCT    LIMIT    2    OFFSET    2;
```

查询结果如下：

```
PRODUCTID       NAME
```

```
---------- ----------------

3          老人与海

4          射雕英雄传(全四册)
```

例 3 查询前第 5，6，7 个登记的姓名。

```
SELECT PERSONID,NAME FROM PERSON.PERSON LIMIT 4,3;
```

查询结果如下：

```
PERSONID   NAME

---------- ----

5          孙丽

6          黄非

7          王菲
```

4.10.2 ROW_LIMIT 子句

ROW_LIMIT 子句用于指定查询结果中偏移的行数，或从指定偏移开始返回的行数或总行数的百分比，以便更为灵活地获取查询结果。

语法如下：

```
<ROW_LIMIT 子句>::= [OFFSET <offset> ROW[S]] [<FETCH 说明>]

<FETCH 说明>::= FETCH <FIRST | NEXT> [<大小> | <大小> PERCENT] ROW[S] <ONLY | WITH TIES>
```

参数

1. OFFSET <offset> ROW[S] 指定偏移的行数，<offset>必须为数字。offset 为负数时视为 0；为 NULL 或大于等于所返回的行数时，返回 0 行；为小数时，小数部分四舍五入。ROW 也可写为 ROWS；

2. <FIRST | NEXT> 指定是否从指定偏移位置开始获取结果。FIRST 表示从指定偏移位置开始获取结果，NEXT 表示从指定偏移的下一行开始获取结果。只做注释说明的作用，没

有实际的限定作用；

3. [<大小> | <大小> PERCENT] 指定返回行的行数或者百分比。其中<大小>只能为数字。PERCENT 指定为负数时，视为 0%；为 NULL 时返回 0 行；如果没有指定 PERCENT 而指定了<大小>，返回为从偏移开始，<大小>指定的行数；如果省略返回行数和 PERCENT，默认返回偏移位置的下一行数据；

4. <ONLY | WITH TIES> 指定结果集是否返回额外的行。额外的行是指与最后一行以相同的排序键排序的所有行。ONLY 为只返回指定的行数。WITH TIES 必须与 ORDER BY 子句同时出现，如果没有 ORDER BY 子句，则忽略 WITH TIES。

使用说明

物化视图的查询定义中包含有 ROW_LIMIT 子句时，该物化视图不能增量刷新。

举例说明

例 1 查询价格最便宜的 50%的商品

```
SELECT NAME, NOWPRICE FROM PRODUCTION.PRODUCT ORDER BY NOWPRICE FETCH FIRST 50 PERCENT ROWS ONLY;
```

查询结果如下：

```
NAME                    NOWPRICE
---------------- --------
老人与海                6.1000
突破英文基础词汇        11.1000
工作中无小事            11.4000
水浒传                  14.3000
红楼梦                  15.2000
```

例 2 查询价格第 3 便宜开始的 3 条记录

```
SELECT NAME, NOWPRICE FROM PRODUCTION.PRODUCT ORDER BY NOWPRICE OFFSET 2 ROWS
```

```
FETCH FIRST 3 ROWS ONLY;
```

查询结果如下：

```
NAME                NOWPRICE

------------ --------

工作中无小事        11.4000

水浒传              14.3000

红楼梦              15.2000
```

4.11 PIVOT 和 UNPIVOT 子句

在 DM 中，可以使用 PIVOT 子句或 UNPIVOT 子句，将一组数据从行转换为列，或者从列转换为行。

4.11.1 PIVOT 子句

PIVOT 子句将一组数据从行转换为列。语法如下：

```
<PIVOT 子句> ::= PIVOT [XML] (<set_func_clause> FOR <pivot_for_clause> IN

(<pivot_in_clause>))

<set_func_clause> ::=       <集函数> [[AS] <别名>] {,<集函数> [[AS] <别名>]}

<pivot_for_clause> ::=      <列名> |

                            (<列名> {,<列名>})

<pivot_in_clause> ::= <exp_clause> [[AS] <别名>] {,<exp_clause> [[AS] <别名>]} |

                          <select_clause> |

                          ANY

<exp_clause> ::= <表达式> |

                 (<表达式> {,<表达式>})
```

<select_clause> ::= 请参考第 4 章 数据查询语句

参数

1. XML 指定使用 XML 格式输出数据，指定 XML 时必须使用<select_clause>或 ANY;

2. <set_func_clause> 指定对表或视图中的字段使用集函数;

3. <pivot_for_clause> 指定表或视图的原始列名;

4. <pivot_in_clause> 对<pivot_for_clause>指定列中的数据进行筛选，指定的数据值或别名将作为转换后新列的列名。支持以下三种筛选方式:

   1) <exp_clause> 直接指定<pivot_for_clause>列中的数据值，<表达式>即实际数据值，仅支持常量表达式;

   2) <select_clause> 使用子查询语句选取<pivot_for_clause>列中的数据值，仅在指定 XML 时生效;

   3) ANY 指定<pivot_for_clause>列中的所有数据值，仅在指定 XML 时生效。

使用说明

1. 使用多个集函数时，必须为每个集函数指定别名;

2. 不支持 COVAR_SAMP、COVAR_POP、COLLECT、CORR 和 REGR_XXX（除了 REGR_COUNT）集函数;

3. <pivot_for_clause>或<exp_clause>中，针对多个<列名>或<表达式>，必须使用小括号括起来；针对单个<列名>或<表达式>，使用或者不使用小括号均可;

4. 当<pivot_for_clause>中指定了多个列时，列个数与<exp_clause>中的表达式个数应一致，且列与表达式一一对应，例如 <pivot_for_clause> 为 (C1,C2)，<exp_clause>应当为(<C1 的数据值>,<C2 的数据值>)；可以在<pivot_in_clause>中指定多个<exp_clause>来表示多组数据值;

5. 指定的<集函数>个数与<pivot_in_clause>中表达式个数的乘积不能超过

2048;

6. 指定 XML 时，结果集返回类型为 TEXT;

7. PIVOT 子句中未涉及的所有查询项，都将作为分组项;

8. 不支持 FOR UPDATE 子句。

举例说明

创建测试表 SALES_ORDER 并插入数据。

```
CREATE TABLE SALES_ORDER (
  SALESORDERID INT,  //订单编号
  SALESMAN VARCHAR(10),  //销售员姓名
  PRODUCT_NAME VARCHAR(10),  //产品名称
  AMOUNT DEC(10,2)  //订单金额
);
//插入数据
INSERT INTO SALES_ORDER VALUES(1,'李兰','苹果',860);
INSERT INTO SALES_ORDER VALUES(2,'李兰','苹果',820);
INSERT INTO SALES_ORDER VALUES(3,'李兰','橘子',1566);
INSERT INTO SALES_ORDER VALUES(4,'李兰','草莓',3200);
INSERT INTO SALES_ORDER VALUES(5,'李兰','草莓',2750);
INSERT INTO SALES_ORDER VALUES(6,'王勇','苹果',630);
INSERT INTO SALES_ORDER VALUES(7,'王勇','苹果',750);
INSERT INTO SALES_ORDER VALUES(8,'王勇','橘子',1200);
INSERT INTO SALES_ORDER VALUES(9,'王勇','草莓',2700);
INSERT INTO SALES_ORDER VALUES(10,'王勇','草莓',3280);
INSERT INTO SALES_ORDER VALUES(11,'孙晓萌','橘子',1350);
```

```
INSERT INTO SALES_ORDER VALUES(12,'孙晓萌','橘子',1180);
INSERT INTO SALES_ORDER VALUES(13,'孙晓萌','草莓',3300);
INSERT INTO SALES_ORDER VALUES(14,'孙晓萌','草莓',3170);
```

查询 SALES_ORDER 表中数据。

```
SELECT * FROM SALES_ORDER;
```

查询结果如下:

```
SALESORDERID SALESMAN PRODUCT_NAME AMOUNT
------------ -------- ------------ ---------
1            李兰       苹果         860.00
2            李兰       苹果         820.00
3            李兰       橘子         1566.00
4            李兰       草莓         3200.00
5            李兰       草莓         2750.00
6            王勇       苹果         630.00
7            王勇       苹果         750.00
8            王勇       橘子         1200.00
9            王勇       草莓         2700.00
10           王勇       草莓         3280.00
11           孙晓萌     橘子         1350.00
12           孙晓萌     橘子         1180.00
13           孙晓萌     草莓         3300.00
14           孙晓萌     草莓         3170.00
```

例 1 使用 PIVOT 子句, 将 PRODUCT_NAME 列中的 3 种产品所对应的行数据转换为列进行展示。

```
SELECT * FROM SALES_ORDER
PIVOT (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN('苹果','橘子','草莓')
);
```

查询结果如下:

```
SALESORDERID SALESMAN '苹果' '橘子' '草莓'
------------ -------- -------- -------- --------
1            李兰       860      NULL     NULL
2            李兰       820      NULL     NULL
3            李兰       NULL     1566     NULL
4            李兰       NULL     NULL     3200
5            李兰       NULL     NULL     2750
6            王勇       630      NULL     NULL
7            王勇       750      NULL     NULL
8            王勇       NULL     1200     NULL
9            王勇       NULL     NULL     2700
10           王勇       NULL     NULL     3280
11           孙晓萌     NULL     1350     NULL
12           孙晓萌     NULL     1180     NULL
13           孙晓萌     NULL     NULL     3300
14           孙晓萌     NULL     NULL     3170
```

PIVOT 子句中未提及的所有查询项都将作为分组项，因此上述语句的分组项为

SALESORDERID 和 SALESMAN。

PIVOT 子句所在层级的 SELECT 语句的查询项应当未出现在 PIVOT 子句中，或为“*”，否则将报错。例如，执行以下语句，SELECT 语句的查询项在 PIVOT 子句中，将报错：

```
SELECT SALESMAN,PRODUCT_NAME,AMOUNT
FROM SALES_ORDER
PIVOT (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN('苹果','橘子','草莓')
);


第 7 行附近出现错误[-2111]:无效的列名[PRODUCT_NAME].
```

又例如，执行以下语句，SELECT 语句的查询项未在 PIVOT 子句中，可以成功执行：

```
SELECT SALESORDERID,SALESMAN
FROM SALES_ORDER
PIVOT (
  SUM(AMOUNT)
  FOR (PRODUCT_NAME)
  IN('苹果','橘子','草莓')
);
```

查询结果如下：

```
SALESORDERID SALESMAN
------------ --------
1            李兰
```

```
2                李兰

3                李兰

4                李兰

5                李兰

6                王勇

7                王勇

8                王勇

9                王勇

10               王勇

11              孙晓萌

12               孙晓萌

13               孙晓萌

14               孙晓萌
```

用户可以借助子查询来指定表或视图中的部分字段作为查询项。例如利用子查询选择 SALESMAN、PRODUCT_NAME 和 AMOUNT 作为查询项。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN('苹果','橘子','草莓')
);
```

查询结果如下:

```
SALESMAN   '苹果' '橘子' '草莓'
```

```
--------- -------- -------- --------

李兰          1680       1566       5950

王勇          1380       1200       5980

孙晓萌        NULL       2530       6470
```

上述语句利用子查询选择 SALESMAN、PRODUCT_NAME 和 AMOUNT 作为查询项，其中的分组项为 SALESMAN，有利于统计各销售人员针对不同产品的销售总额。

例 2 可以在 IN 子句中为每个数据值指定别名，指定的别名将作为转换后各列的列名。例如可以通过指定别名，去除列名中的单引号。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN ('苹果' AS 苹果,'橘子' AS 橘子,'草莓' AS 草莓)
);
```

查询结果如下：

```
SALESMAN   苹果  橘子  草莓
--------- ------ ------ ------
李兰          1680    1566    5950
王勇          1380    1200    5980
孙晓萌        NULL    2530    6470
```

例 3 使用多个集函数，此时需要为每个集函数指定别名。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
```

```
PIVOT (
  SUM(AMOUNT) AS 订单总额,
  COUNT(AMOUNT) AS 订单数量
  FOR PRODUCT_NAME
  IN('苹果' AS 苹果, '橘子' AS 橘子)
);
```

查询结果如下:

```
SALESMAN   苹果_订单总额 苹果_订单数量   橘子_订单总额 橘子_订单数量
--------- ------------------ ------------------- ------------------
李兰          1680          2               1566          1
王勇          1380          2               1200          1
孙晓萌        NULL          0               2530          2
```

例 4 FOR 子句中指定多列，此时需要在 IN 子句中同时指定多列的数据值。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT (
  SUM(AMOUNT)
  FOR (SALESMAN,PRODUCT_NAME)
  IN(
  ('李兰','橘子') AS 李兰_橘子总额,
  ('王勇','橘子') AS 王勇_橘子总额,
  ('孙晓萌','橘子') AS 孙晓萌_橘子总额)
);
```

查询结果如下:

```
李兰_橘子总额 王勇_橘子总额 孙晓萌_橘子总额

------------------- --------- ---------

1566           1200           2530
```

例 5 指定 XML，使用 XML 格式输出数据，同时指定 ANY 关键字，表示选择 PRODUCT_NAME 列中的全部产品。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT XML (
  SUM (AMOUNT)
  FOR PRODUCT_NAME
  IN (ANY)
);
```

查询结果如下：

```
SALESMAN PRODUCT_NAME_XML

------------ -----------------------------------

李兰

<PivotSet><item><column name = "PRODUCT_NAME">草莓</column><column name =

"SUM(AMOUNT)">5950</column></item><item><column name = "PRODUCT_NAME">苹果

</column><column name = "SUM(AMOUNT)">1680</column></item><item><column name

= "PRODUCT_NAME">橘子</column><column name =

"SUM(AMOUNT)">1566</column></item></PivotSet>



孙晓萌

<PivotSet><item><column name = "PRODUCT_NAME">草莓</column><column name =
```

```
"SUM(AMOUNT)">6470</column></item><item><column name = "PRODUCT_NAME">橘子
</column><column name = "SUM(AMOUNT)">2530</column></item></PivotSet>


王勇
<PivotSet><item><column name = "PRODUCT_NAME">草莓</column><column name =
"SUM(AMOUNT)">5980</column></item><item><column name = "PRODUCT_NAME">苹果
</column><column name = "SUM(AMOUNT)">1380</column></item><item><column name
= "PRODUCT_NAME">橘子</column><column name =
"SUM(AMOUNT)">1200</column></item></PivotSet>
```

例 6 指定 XML，使用 XML 格式输出数据，同时使用子查询语句选择 PRODUCT_NAME 列中的产品。

```
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT XML (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN(
    SELECT PRODUCT_NAME
    FROM SALES_ORDER
    WHERE PRODUCT_NAME='苹果' OR PRODUCT_NAME='橘子')
);
```

查询结果如下：

```
SALESMAN PRODUCT_NAME_XML
------------ ----------------------------------
```

```
李兰
<PivotSet><item><column name = "PRODUCT_NAME">苹果</column><column name =
"SUM(AMOUNT)">1680</column></item><item><column name = "PRODUCT_NAME">橘子
</column><column name = "SUM(AMOUNT)">1566</column></item></PivotSet>

孙晓萌
<PivotSet><item><column name = "PRODUCT_NAME">苹果</column><column name =
"SUM(AMOUNT)"></column></item><item><column name = "PRODUCT_NAME">橘子
</column><column name = "SUM(AMOUNT)">2530</column></item></PivotSet>

王勇
<PivotSet><item><column name = "PRODUCT_NAME">苹果</column><column name =
"SUM(AMOUNT)">1380</column></item><item><column name = "PRODUCT_NAME">橘子
</column><column name = "SUM(AMOUNT)">1200</column></item></PivotSet>
```

4.11.2 UNPIVOT 子句

UNPIVOT 子句将一组数据从列转换为行。语法如下:

```
<UNPIVOT 子句> ::= UNPIVOT [<include_null_clause>](<unpivot_val_col_lst> FOR
<unpivot_for_clause> IN (<unpivot_in_clause_low> ))
<include_null_clause> ::=   INCLUDE NULLS |
                                    EXCLUDE NULLS
<unpivot_val_col_lst> ::= <表达式> |
                                  (<表达式> {,<表达式>})
<unpivot_for_clause> ::= <表达式> |
```

```
                                (<表达式> {,<表达式>})
<unpivot_in_clause_low>  ::= <unpivot_in_clause>{,<unpivot_in_clause>}
<unpivot_in_clause>  ::= <列名> [AS <别名>]  |
                                (<列名> {,<列名>}) [AS (<别名> {,<别名>})]
```

参数

1. INCLUDE NULLS 转换后的结果中包含 NULL 值;

2. EXCLUDE NULLS 转换后的结果中不包含 NULL 值，缺省为 EXCLUDE NULLS;

3. <unpivot_val_col_lst> 指定转换后新列的列名，<unpivot_in_clause_low>指定列的数据将作为该列数据。<表达式>仅支持常量表达式;

4. <unpivot_for_clause> 指定转换后新列的列名，<unpivot_in_clause_low>指定列的列名或别名将作为该列数据。<表达式>仅支持常量表达式;

5. <unpivot_in_clause_low> 指定表或视图的原始列名。

使用说明

1. <unpivot_val_col_lst>、<unpivot_for_clause>或<unpivot_in_clause>中，针对多个<表达式>、<列名>或<别名>，必须使用小括号括起来；针对单个<表达式>、<列名>或<别名>，使用或者不使用小括号均可;

2. <unpivot_val_col_lst>和<unpivot_for_clause>中的表达式个数保持一致;

3. <unpivot_for_clause>中的表达式个数与<unpivot_in_clause>中的 AS 项别名个数保持一致;

4. 多个<unpivot_in_clause>之间指定的列数据类型要保持一致;

5. 仅支持对单表、视图、DBLINK 进行 UNPIVOT 转换;

6. INI 参数 UNPIVOT_OPT_FLAG 可控制输出结果的顺序, UNPIVOT_OPT_FLAG 取值包含 1 时按照不包含在 UNPIVOT 中的列进行排序;

7. UNPIVOT 中自定义的列名不能为保留字；

8. <unpivot_in_clause_low>中指定的转换列个数不能超过 1000 个；

9. <unpivot_in_clause_low>中指定的转换列不能是 ROWID、TRXID 等伪列；

10. UNPIVOT 不支持 ROLLUP、CUBE、大于等于两个查询项的 GROUPING SETS、FOR UPDATE 子句、不能同时存在 PIVOT 子句。

举例说明

首先创建测试表 SALES_ORDER 并插入数据。相关 SQL 语句可参考 4.11.1 PIVOT 子句。由于 UNPIVOT 子句功能与 PIVOT 子句功能正好相反，因此可以利用 PIVOT 子句创建测试视图，方便展示 UNPIVOT 子句使用示例。

例 1 创建测试视图 PIVOT_SALES_ORDER_1。

```
CREATE VIEW PIVOT_SALES_ORDER_1 AS
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT (
  SUM(AMOUNT)
  FOR PRODUCT_NAME
  IN('苹果' AS 苹果,'橘子' AS 橘子,'草莓' AS 草莓)
);
```

查询 PIVOT_SALES_ORDER_1 视图中数据。

```
SELECT * FROM PIVOT_SALES_ORDER_1;
```

查询结果如下：

```
SALESMAN   苹果 橘子 草莓
--------- ------ ------ ------
```

```
李兰          1680    1566    5950
王勇          1380    1200    5980
孙晓萌        NULL    2530    6470
```

使用 UNPIVOT 子句，将苹果、橘子、草莓三列数据转换为行。

```
SELECT * FROM PIVOT_SALES_ORDER_1
UNPIVOT (
  TOTAL_AMOUNT
  FOR PRODUCT_NAME
  IN(苹果,橘子,草莓)
);
```

查询结果如下：

```
SALESMAN   PRODUCT_NAME TOTAL_AMOUNT
--------- ------------ ------------
李兰          苹果         1680
王勇          苹果         1380
李兰          橘子         1566
王勇          橘子         1200
孙晓萌        橘子         2530
李兰          草莓         5950
王勇          草莓         5980
孙晓萌        草莓         6470
```

可以看到，由于缺省不显示 NULL 值，因此以上结果集中并未包含孙晓萌对苹果的销售金额信息。用户可使用 INCLUDE NULLS 来指定结果集中包含 NULL 值。

```
SELECT * FROM PIVOT_SALES_ORDER_1
```

```
UNPIVOT INCLUDE NULLS (
  TOTAL_AMOUNT
  FOR PRODUCT_NAME
  IN(苹果,橘子,草莓)
);
```

查询结果如下:

```
SALESMAN   PRODUCT_NAME TOTAL_AMOUNT
--------- ------------ ------------
李兰          苹果         1680
王勇          苹果         1380
孙晓萌        苹果         NULL
李兰          橘子         1566
王勇          橘子         1200
孙晓萌        橘子         2530
李兰          草莓         5950
王勇          草莓         5980
孙晓萌        草莓         6470
```

例 2 创建测试视图 PIVOT_SALES_ORDER_2。

```
CREATE VIEW PIVOT_SALES_ORDER_2 AS
SELECT *
FROM (SELECT SALESMAN,PRODUCT_NAME,AMOUNT FROM SALES_ORDER)
PIVOT (
  SUM(AMOUNT) AS 金额,
  COUNT(AMOUNT) AS 订单量
```

```
  FOR PRODUCT_NAME
  IN('苹果' AS 苹果,'橘子' AS 橘子,'草莓' AS 草莓)
);
```

查询 PIVOT_SALES_ORDER_2 视图中数据。

```
SELECT * FROM PIVOT_SALES_ORDER_2;
```

查询结果如下:

```
SALESMAN   苹果_金额   苹果_订单量   橘子_金额   橘子_订单量   草莓_金额   草莓_订单量
--------- ------------ ------------------- ------------ --------------------
李兰          1680        2              1566        1              5950        2
王勇          1380        2              1200        1              5980        2
孙晓萌        NULL        0              2530        2              6470        2
```

使用 UNPIVOT 子句，将所有金额列和订单量列转换为行。

```
SELECT * FROM PIVOT_SALES_ORDER_2
UNPIVOT (
  (SUM_AMOUNT,COUNT_AMOUNT)
  FOR (PRODUCT_NAME_SUM,PRODUCT_NAME_COUNT)
  IN(
    (苹果_金额,苹果_订单量) AS ('苹果销售额','苹果订单量'),
    (橘子_金额,橘子_订单量) AS ('橘子销售额','橘子订单量'),
    (草莓_金额,草莓_订单量) AS ('草莓销售额','草莓订单量')
  )
);
```

查询结果如下:

```
SALESMAN   PRODUCT_NAME_SUM PRODUCT_NAME_COUNT SUM_AMOUNT COUNT_AMOUNT
```

```
--------- ---------------- ----------------- ---------- --------------------

李兰        苹果销售额          苹果订单量          1680        2

王勇        苹果销售额          苹果订单量          1380        2

孙晓萌      苹果销售额          苹果订单量          NULL        0

李兰        橘子销售额          橘子订单量          1566        1

王勇        橘子销售额          橘子订单量          1200        1

孙晓萌      橘子销售额          橘子订单量          2530         2

李兰        草莓销售额          草莓订单量          5950         2

王勇        草莓销售额          草莓订单量          5980         2

孙晓萌      草莓销售额          草莓订单量          6470         2
```

UNPIVOT 子句缺省不显示 NULL 值，但仅当 SUM_AMOUNT 和 COUNT_AMOUNT 同时为 NULL 时才不显示，因此以上结果集中仍然包含孙晓萌对苹果的销售金额和订单量信息。

4.12 全文检索

DM 数据库提供多文本数据检索服务，包括全文索引和全文检索。全文索引为在字符串数据中进行复杂的词搜索提供了有效支持。全文索引存储关于词和词在特定列中的位置信息，全文检索利用这些信息，可以快速搜索包含某个词或某一组词的记录。

执行全文检索涉及到以下这些任务：

1. 对需要进行全文检索的表和列进行注册；

2. 对注册了的列的数据建立全文索引；

3. 对注册了的列查询填充后的全文索引。

执行全文检索步骤如下：

1．建立全文索引；

2．修改（填充）全文索引；

3．使用带 CONTAINS 谓词的查询语句进行全文检索；

4．当数据表的全文索引列数据发生变化，则需要进行增量或者完全填充全文索引，以便可以查询到更新后的数据；

5．若不再需要全文索引，可以删除该索引；

6．在全文索引定义并填充后，才可进行全文检索。

全文检索通过在查询语句的 WHERE <搜索条件>中，指定<搜索条件>为 CONTAINS 子句进行。此处对 CONTAINS 子句进行说明。

语法格式

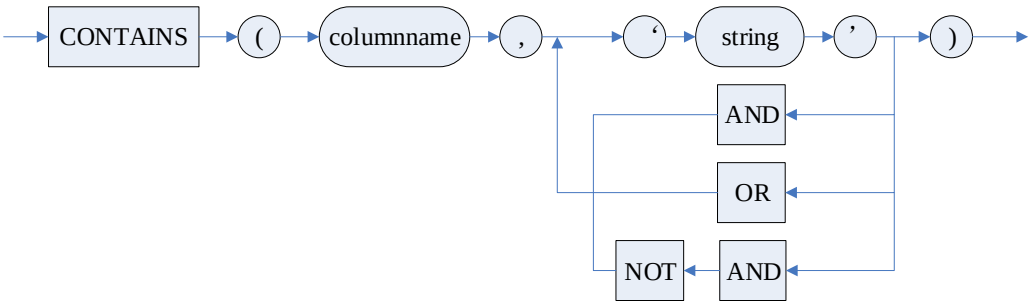
```
CONTAINS ( <列名> , <检索条件> )

<检索条件>::= <布尔项> | <检索条件> <AND | OR | AND NOT> <布尔项>

<布尔项>::= '字符串'
```

图例

全文检索 CONTAINS 子句



使用说明

1. CONTAINS 子句仅支持在对基表的查询语句中使用；使用 CONTAINS 子句查询时，<列名>必须是已经建立了全文索引并填充后的基表列，否则系统会报错；

2. 支持精确字、词、短语及一段文字的查询，CONTAINS 谓词内支持 AND | AND NOT | OR 的使用，AND 的优先级高于 OR 的优先级；

3. 支持对每个精确词（单字节语言中没有空格或标点符号的一个或多个字符）或短语

（单字节语言中由空格和可选的标点符号分隔的一个或多个连续的词）的匹配。对词或短语中字符的搜索不区分大小写；

4. 对于短语或一段文字的查询，根据词库，单个查找串被分解为若干个关键词，忽略词库中没有的词和标点符号，在索引上进行（关键词 AND 关键词）匹配查找。因而，不一定是精确查询；

5. 英文查询不区分大小写和全角半角中英文字符；

6. 不提供 Noise 文件，即不考虑忽略词或干扰词；

7. 不支持通配符“*”；

8. 不提供对模糊词或变形词的查找；

9. 不支持对结果集的相关度排名；

10. 检索条件子句可以和其他子句共同组成 WHERE 的检索条件。

举例说明

例 全文检索综合实例，以 PRODUCT 表为例。

(1)在 DESCRIPTION 列上定义全文索引。

```
CREATE CONTEXT INDEX INDEX1 ON PRODUCTION.PRODUCT(DESCRIPTION) LEXER
CHINESE_VGRAM_LEXER;
```

(2)完全填充全文索引。

```
ALTER CONTEXT INDEX INDEX1 ON PRODUCTION.PRODUCT REBUILD;
```

(3)进行全文检索，查找描述里有“语言”字样的产品的编号和名称。

```
SELECT PRODUCTID, NAME FROM PRODUCTION.PRODUCT WHERE CONTAINS(DESCRIPTION, '语言');
```

查询结果如下：

```
PRODUCTID    NAME
-----------  --------------------------
2                水浒传
```

```
7                数据结构(C 语言版)(附光盘)
```

(4) 进行全文检索，查找描述里有“语言”及“中国”字样的产品的编号和名称。

```
SELECT PRODUCTID, NAME FROM PRODUCTION.PRODUCT WHERE CONTAINS(DESCRIPTION,'语言'
AND '中国');
```

查询结果如下:

```
PRODUCTID      NAME
-----------  ------
2                水浒传
```

(5)进行全文检索，查找描述里有“语言”或“中国”字样的产品的编号和名称。

```
SELECT PRODUCTID, NAME FROM PRODUCTION.PRODUCT WHERE CONTAINS(DESCRIPTION,'语言' OR '
中国');
```

查询结果如下:

```
PRODUCTID      NAME
-----------  -------------------------
2                水浒传
7                数据结构(C 语言版)(附光盘)
1                红楼梦
```

(6)进行全文检索，查找描述里无“中国”字样的雇员的产品的编号和名称。

```
SELECT PRODUCTID, NAME FROM PRODUCTION.PRODUCT WHERE NOT CONTAINS(DESCRIPTION, '中
国');
```

查询结果如下:

```
PRODUCTID      NAME
-----------  --------------------------------
```

```
3                老人与海
4                射雕英雄传(全四册)
5                鲁迅文集(小说、散文、杂文)全两册
6                长征
7                数据结构(C 语言版)(附光盘)
8                工作中无小事
9                突破英文基础词汇
10               噼里啪啦丛书(全 7 册)
```

(7)进行全文检索，查找描述里有“C 语言”字样的产品的编号和名称。

```
SELECT PRODUCTID, NAME FROM PRODUCTION.PRODUCT WHERE CONTAINS(DESCRIPTION,'C 语言');
```

查询结果如下:

```
PRODUCTID      NAME
---------- -------------------------
7              数据结构(C 语言版)(附光盘)
```

(8)对不再需要的全文索引进行删除。

```
DROP CONTEXT INDEX INDEX1 ON PRODUCTION.PRODUCT;
```

4.13 层次查询子句

可通过层次查询子句进行层次查询，得到数据间的层次关系。在使用层次查询子句时，可以使用层次查询相关的伪列、函数或操作符来明确层次查询结果中的相应层次信息。

4.13.1 层次查询子句

语法格式

```
<层次查询子句> ::=
```

```
    CONNECT BY [NOCYCLE]  <连接条件> [ START WITH <起始条件> ] |
    START WITH <起始条件> CONNECT BY [NOCYCLE] <连接条件>
<连接条件>::= <逻辑表达式>
<起始条件>::= <逻辑表达式>
```

参数

1. <连接条件> 逻辑表达式，指明层次数据间的层次连接关系；

2. <起始条件> 逻辑表达式，指明选择层次数据根数据的条件；

3. NOCYCLE 关键字用于指定数据导致环的处理方式，如果在层次查询子句中指定 NOCYCLE 关键字，会忽略导致环元组的儿子数据。否则，返回错误。

4.13.2 层次查询相关伪列

在使用层次查询子句时，可以通过相关的伪列来明确数据的层次信息。层次查询相关的伪列有：

1．LEVEL

该伪列表示当前元组在层次数据形成的树结构中的层数。LEVEL 的初始值为 1，即层次数据的根节点数据的 LEVEL 值为 1，之后其子孙节点的 LEVEL 依次递增。

2．CONNECT_BY_ISLEAF

该伪列表示当前元组在层次数据形成的树结构中是否是叶节点（即该元组根据连接条件不存在子节点）。是叶节点时为 1，否则为 0。

3．CONNECT_BY_ISCYCLE

该伪列表示当前元组是否会将层次数据形成环，该伪列只有在层次查询子句中表明 NOCYCLE 关键字时才有意义。如果元组的存在会导致层次数据形成环，该伪列值为 1，否则为 0。

4.13.3 层次查询相关操作符

1．PRIOR

PRIOR 操作符主要使用在层次查询子句中，指明 PRIOR 之后的参数为逻辑表达式中的父节点。

PRIOR 操作符还可以出现在查询项、WHERE 条件、GROUP BY 子句、集函数参数中，表示父层记录对应的值。

例

```
SELECT HIGH_DEP, DEP_NAME FROM OTHER.DEPARTMENT CONNECT BY NOCYCLE  PRIOR
DEP_NAME = HIGH_DEP;
//DEP_NAME 为父节点。下一条记录的 HIGH_DEP 等于前一条记录的 DEP_NAME
```

或者

```
SELECT HIGH_DEP, DEP_NAME FROM OTHER.DEPARTMENT CONNECT BY NOCYCLE  DEP_NAME =
PRIOR  HIGH_DEP;
//HIGH_DEP 为父节点。下一条记录的 DEP_NAME 等于前一条记录的 HIGH_DEP
```

2．CONNECT_BY_ROOT

该操作符作为查询项，查询在层次查询结果中根节点的某列的值。

4.13.4 层次查询相关函数

语法格式

```
SYS_CONNECT_BY_PATH(col_name,char)
```

语句功能

层次查询。

使用说明

该函数得到从根节点到当前节点路径上所有节点名为 col_name 的某列的值，之间用 char 指明的字符分隔开。

4.13.5 层次查询层内排序

语法格式

```
ORDER SIBLINGS BY <order_by_list>
```

<order_by_list>请参考 4.7 ORDER BY 子句

语句功能

层次查询。

使用说明

ORDER SIBLINGS BY 用于指定层次查询中相同层次数据返回的顺序。在层次查询中使用 ORDER SIBLINGS BY，必须与 CONNECT BY 一起配合使用。但是，ORDER SIBLINGS BY 不能和 GROUP BY 一起使用。

4.13.6 层次查询的限制

1. CONNECT BY 子句中不支持集函数、嵌套集函数、TRXID 以及 GROUPING 函数；START WITH 子句中不支持集函数、嵌套集函数、TRXID、GROUPING 函数、层次查询的所有伪列以及层次查询函数、FOR UPDATE 子句；

2. ORDER SIBLINGS BY 子句中不能使用层次查询的所有伪列、层次查询函数、操作符、ROWNUM 以及子查询；

3. 层次查询子句不能使用伪列 CONNECT_BY_ISLEAF、CONNECT_BY_ISCYCLE，SYS_CONNECT_BY_PATH 伪函数和 CONNECT_BY_ROOT 操作符；

4. JOIN ON 子句中不允许出现层次查询的所有伪列、层次查询相关操作符和相关函数；

5. PRIOR、CONNECT_BY_ROOT 操作符后不能使用层次查询的所有伪列、层次查询函数、操作符以及 ROWNUM；SYS_CONNECT_BY_PATH 函数的第一个参数不能使用层次查询的 CONNECT_BY_ISLEAF 或 CONNECT_BY_ISCYCLE 伪列、层次查询函数以及操作符，SYS_CONNECT_BY_PATH 的第一个参数允许出现 LEVEL 伪列且第二个参数必须是常量字符串；

6. 函数 SYS_CONNECT_BY_PATH 的最大返回长度为 32767，超长就会报错；

7. INI 参数 CNNTB_MAX_LEVEL 表示支持层次查询的最大层次，缺省为 20000。该参数的有效取值范围为 1~100000。

8. 同一层查询中不同 SYS_CONNECT_BY_PATH 函数的个数上限为 64 个。

例如，对 OTHER.DEPARTMENT 数据进行层次查询，HIGH_DEP 表示上级部门；DEP_NAME 表示部门名称。

层次数据所建立起来的树形结构如下图：

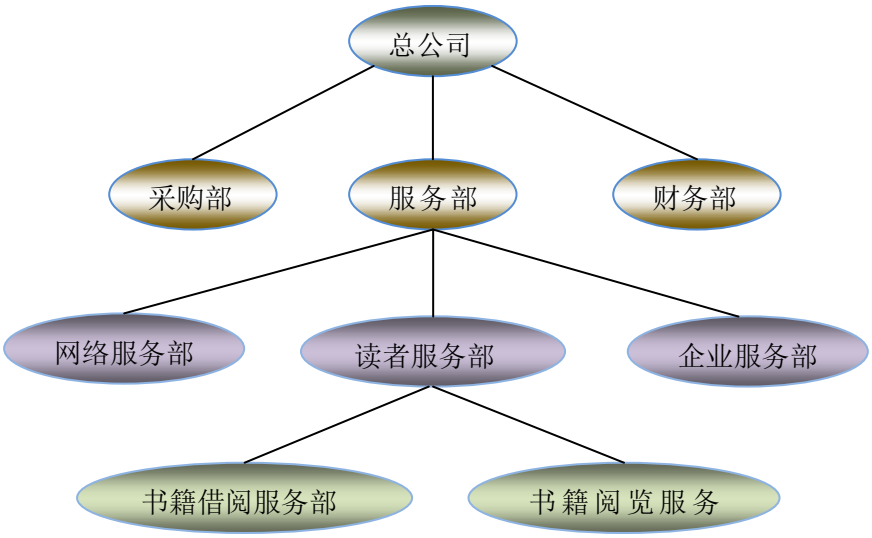


图 4.13.1 层次数据树形结构图

例 1 不带起始选择根节点起始条件的层次查询

```
SELECT HIGH_DEP, DEP_NAME FROM OTHER.DEPARTMENT CONNECT BY PRIOR DEP_NAME = HIGH_DEP;
```

查询结果如下：

```
HIGH_DEP      DEP_NAME

---------- --------------

NULL          总公司

总公司        财务部

总公司        采购部

总公司        服务部

服务部        企业服务部

服务部        读者服务部

读者服务部    书籍阅览服务部

读者服务部    书籍借阅服务部

服务部        网络服务部

总公司        服务部

服务部        企业服务部

服务部        读者服务部

读者服务部    书籍阅览服务部

读者服务部    书籍借阅服务部

服务部        网络服务部

总公司        采购部

总公司        财务部

服务部        网络服务部

服务部        读者服务部

读者服务部    书籍阅览服务部

读者服务部    书籍借阅服务部

服务部        企业服务部
```

```
读者服务部      书籍借阅服务部

读者服务部      书籍阅览服务部
```

结果是以表中所有的节点为根节点进行先根遍历进行层次查询。

例 2 带起始选择根节点起始条件的层次查询

```
SELECT HIGH_DEP, DEP_NAME FROM OTHER.DEPARTMENT CONNECT BY PRIOR

DEP_NAME=HIGH_DEP START WITH DEP_NAME='总公司';
```

查询结果如下:

```
HIGH_DEP    DEP_NAME

---------- --------------

NULL            总公司

总公司          财务部

总公司          采购部

总公司          服务部

服务部          企业服务部

服务部          读者服务部

读者服务部    书籍阅览服务部

读者服务部    书籍借阅服务部

服务部          网络服务部
```

例 3 层次查询伪列的使用

在层次查询中，伪列的使用可以更明确层次数据之间的关系。

```
SELECT LEVEL,

CONNECT_BY_ISLEAF ISLEAF,
```

```
CONNECT_BY_ISCYCLE ISCYCLE,
HIGH_DEP, DEP_NAME FROM OTHER.DEPARTMENT
CONNECT BY PRIOR DEP_NAME=HIGH_DEP
START WITH DEP_NAME='总公司';
```

查询结果如下:

```
LEVEL        ISLEAF    ISCYCLE         HIGH_DEP    DEP_NAME
---------  -------- ---------- ---------- --------------
1            0          0               NULL        总公司
2            1          0               总公司      财务部
2            1          0               总公司      采购部
2            0          0               总公司      服务部
3            1          0               服务部      企业服务部
3            0          0               服务部      读者服务部
4            1          0               读者服务部  书籍阅览服务部
4            1          0               读者服务部  书籍借阅服务部
3            1          0               服务部      网络服务部
```

通过伪列，可以清楚地看到层次数据之间的层次结构。

例 4 含有过滤条件的层次查询

在层次查询中加入过滤条件，将会先进行层次查询，然后进行过滤。

```
SELECT LEVEL,* FROM OTHER.DEPARTMENT WHERE HIGH_DEP = '总公司' CONNECT BY PRIOR
DEP_NAME=HIGH_DEP;
```

查询结果如下:

```
LEVEL          HIGH_DEP DEP_NAME
```

```
---------- -------- --------

2                总公司      财务部

2                总公司      采购部

2                总公司      服务部

1                总公司      服务部

1                总公司      采购部

1                总公司      财务部
```

例 5 含有排序子句的层次查询

在层次查询中加入排序，查询将会按照排序子句指明的要求排序，不再按照层次查询的排序顺序排序。

```
SELECT * FROM OTHER.DEPARTMENT CONNECT BY PRIOR DEP_NAME=HIGH_DEP START WITH

DEP_NAME='总公司' ORDER BY HIGH_DEP;
```

查询结果如下：

```
HIGH_DEP    DEP_NAME

---------- --------------

NULL            总公司

读者服务部    书籍阅览服务部

读者服务部    书籍借阅服务部

服务部          企业服务部

服务部          读者服务部

服务部          网络服务部

总公司          服务部

总公司          采购部
```

```
总公司          财务部
```

例 6 含层内排序子句的层次查询

在层次查询中加入 ORDER SIBLINGS BY，查询会对相同层次的数据进行排序后，深度优先探索返回数据，即 LEVEL 相同的数据进行排序。

```
SELECT HIGH_DEP, DEP_NAME, LEVEL FROM OTHER.DEPARTMENT CONNECT BY PRIOR
DEP_NAME=HIGH_DEP START WITH DEP_NAME='总公司' ORDER SIBLINGS BY DEP_NAME;
```

查询结果如下：

```
HIGH_DEP    DEP_NAME            LEVEL
---------- -------------- -----------
NULL           总公司               1
总公司         财务部               2
总公司         采购部               2
总公司         服务部               2
服务部         读者服务部           3
读者服务部     书籍借阅服务部       4
读者服务部     书籍阅览服务部       4
服务部         企业服务部           3
服务部         网络服务部           3
```

例 7 CONNECT_BY_ROOT 操作符的使用

CONNECT_BY_ROOT 操作符之后跟某列的列名，例如：

```
CONNECT_BY_ROOT DEP_NAME
```

进行如下查询：

```
SELECT CONNECT_BY_ROOT DEP_NAME,* FROM OTHER.DEPARTMENT CONNECT BY PRIOR
DEP_NAME=HIGH_DEP START WITH DEP_NAME='总公司' ;
```

查询结果如下:

```
CONNECT_BY_ROOT(DEP_NAME) HIGH_DEP      DEP_NAME
------------------------- ---------- --------------
总公司                              NULL            总公司
总公司                              总公司          财务部
总公司                              总公司          采购部
总公司                              总公司          服务部
总公司                              服务部          企业服务部
总公司                              服务部          读者服务部
总公司                              读者服务部      书籍阅览服务部
总公司                              读者服务部      书籍借阅服务部
总公司                              服务部          网络服务部
```

例 8 SYS_CONNECT_BY_PATH 函数的使用

函数的使用方式，如:

```
SYS_CONNECT_BY_PATH(DEP_NAME, '/')
```

进行如下查询:

```
SELECT SYS_CONNECT_BY_PATH(DEP_NAME, '/') PATH,* FROM OTHER.DEPARTMENT CONNECT BY
PRIOR DEP_NAME=HIGH_DEP START WITH DEP_NAME='总公司' ;
```

查询结果如下:

```
PATH                                        HIGH_DEP     DEP_NAME
----------------------------------------   ---------- --------------
```

/总公司	NULL	总公司
/总公司/财务部	总公司	财务部
/总公司/采购部	总公司	采购部
/总公司/服务部	总公司	服务部
/总公司/服务部/企业服务部	服务部	企业服务部
/总公司/服务部/读者服务部	服务部	读者服务部
/总公司/服务部/读者服务部/书籍阅览服务部	读者服务部	书籍阅览服务部
/总公司/服务部/读者服务部/书籍借阅服务部	读者服务部	书籍借阅服务部
/总公司/服务部/网络服务部	服务部	网络服务部

4.14 并行查询

达梦支持并行查询技术。首先设置好如下三个 INI 参数，之后执行 SQL 语句，即可执行并行查询。三个 INI 参数解释如下表。

表 4.14.1 并行查询相关参数

参数名	缺省值	说明
MAX_PARALLEL_DEGREE	1	用来设置最大并行任务个数。取值范围：1~128。缺省值 1，表示无并行任务。全局有效。当 PARALLEL_POLICY 值为 1 时该参数值才有效。
PARALLEL_POLICY	0	用来设置并行策略。取值范围：0、1 和 2，缺省为 0。其中，0 表示不支持并行；1 表示自动配置并行工作线程个数（与物理 CPU 核数相同）；2 表示手动设置并行工作线程数。当 PARALLEL_POLICY 值为 2 时，需手动指定当前并行任务个数。
PARALLEL_THRD_NUM	10	用来设置并行工作线程个数。取值范围：1~1024。仅当 PARALLEL_POLICY 值为 2 时才启用此参数。

注：当处于 DMSQL 程序调试状态时，并行查询的相关设置均无效。

其中，并行任务数也可以在 SQL 语句中使用“PARALLEL”关键字特别指定。如果单条查

询语句没有特别指定，则依然使用默认并行任务个数。“PARALLEL”关键字的用法为在数据查询语句的 SELECT 关键字后增加 HINT 子句。

语法格式

```
/*+ PARALLEL([<表名>] <并行任务个数>) */
```

使用说明

对于无特殊要求的并行查询用户，可以使用默认并行任务数 MAX_PARALLEL_DEGREE。只需要在 INI 参数中设置好对应参数，然后执行 SQL 查询语句，就可以启用并行查询。

举例说明

例 1 将 PARALLEL_POLICY 设置为 0，表示不支持并行查询。此时，另外两个参数不起任何作用。

```
PARALLEL_POLICY                    0
```

例 2 将 PARALLEL_POLICY 设置为 1，表示自动配置并行工作线程个数，因此，只要设置下面 2 个参数就可以。

```
MAX_PARALLEL_DEGREE                3
PARALLEL_POLICY                    1
```

然后，执行 SQL 语句。

```
SELECT * FROM SYSOBJECTS;          //本条语句使用默认并行任务数 3
```

当然，如果单条查询语句不想使用默认并行任务数 3，可以通过在 SQL 语句中增加 HINT，通过“PARALLEL”关键字特别指定。本条语句使用特别指定的并行任务数 4，例如：

```
SELECT /*+ PARALLEL(4) */ * FROM SYSOBJECTS;
```

例 3 将 PARALLEL_POLICY 设置为 2，表示手动配置并行工作线程个数，因此，指定如下 2 个参数。

```
PARALLEL_POLICY                    2
PARALLEL_THRD_NUM                  4
```

然后，在执行 SQL 语句时，需手动指定当前并行任务个数。若不指定，将不使用并行。

```
SELECT /*+ PARALLEL(2) */ * FROM SYSOBJECTS;      //本条语句使用并行任务数 2。
```

4.15 ROWNUM

ROWNUM 是一个虚假的列，表示从表中查询的行号，或者连接查询的结果集行数。它将被分配为 1，2，3，4，...N，N 是行的数量。通过使用 ROWNUM 可以限制查询返回的行数。例如，以下语句执行只会返回前 5 行数据。

```
SELECT * FROM RESOURCES.EMPLOYEE WHERE ROWNUM < 6;
```

一个 ROWNUM 值不是被永久的分配给一行。表中的某一行并没有标号，不可以查询 ROWNUM 值为 5 的行。ROWNUM 值只有当被分配之后才会增长，并且初始值为 1。即只有满足一行后，ROWNUM 值才会加 1，否则只会维持原值不变。因此，以下语句在任何时候都不能返回数据。

```
SELECT * FROM RESOURCES.EMPLOYEE WHERE ROWNUM > 11;

SELECT * FROM RESOURCES.EMPLOYEE WHERE ROWNUM = 5;
```

ROWNUM 的一个重要作用是控制返回结果集的规模，可以避免查询在磁盘中排序。

因为 ROWNUM 值的分配是在查询的谓词解析之后，任何排序和聚合之前进行的。因此，在排序和聚合使用 ROWNUM 时需要注意，可能得到并非预期的结果，例如：

```
SELECT * FROM RESOURCES.EMPLOYEE WHERE ROWNUM < 11 ORDER BY EMPLOYEEID;
```

以上语句只会对 EMPLOYEE 表前 10 行数据按 EMPLOYEEID 排序输出，并不是表的所有数据按 EMPLOYEEID 排序后输出前 10 行，要实现后者，需要使用如下语句：

```
SELECT * FROM (SELECT * FROM RESOURCES.EMPLOYEE ORDER BY EMPLOYEEID) WHERE ROWNUM <
11;

SELECT TOP 10 * FROM RESOURCES.EMPLOYEE ORDER BY EMPLOYEEID;
```

使用说明

1．在查询中，ROWNUM 可与任何数字类型表达式进行比较及运算；

2．ROWNUM 可以在非相关子查询中使用；当参数 ENABLE_RQ_TO_INV 等于 1 时，部分相关子查询支持使用；

3．在非相关子查询中，ROWNUM 只能实现与 TOP 相同的功能，因此子查询不能含 ORDER BY 和 GROUP BY；

4．ROWNUM 所处的子谓词只能为如下形式： ROWNUM op exp，exp 的类型只能是立即数、参数和变量值，op ∈ {<， <=， >， >=， =， <>}。

4.16 BINARY 前缀

数据库是否大小写敏感通过建库参数 CASE_SENSITIVE 控制，初始化后便无法修改，可以通过系统函数 SF_GET_CASE_SENSITIVE_FLAG()或 CASE_SENSITIVE()查询设置的参数值。为了便于用户在数据库初始化后依旧可以按需求进行局部大小写敏感的字符比较操作，提供 BINARY 前缀方式用于设置表达式比较时为大小写敏感。字符的局部大小写敏感还可以通过设置会话属性进行，请参考 3.14.5 大小写敏感。

BINARY 前缀支持范围：

1. SQL 项：查询项、过滤条件、连接条件、层次查询条件、having 条件、排序项、分组项。
2. 表达式类型：逻辑比较表达式、模糊查询表达式（包括 row like）、查询表达式（例如：in、逻辑比较，但不支持多列 in、多列逻辑比较）等。

BINARY 前缀在顶层查询项的含义是将查询项转换为原始值字符串 ASCII 码的十六进制形式，例如：将 123abc 转换为 0x313233616263；在除顶层查询项外的其他位置则表示该前缀修饰的表达式将按照大小写敏感进行比较，无论当前数据库为大小写敏感或不敏感。

例 在顶层查询中添加 BINARY 前缀与在子查询中添加 BINARY 前缀。

```
CREATE TABLE BT(C1 VARCHAR, C2 VARCHAR, C3 VARCHAR);
INSERT INTO BT VALUES('AaBbCc','a','A');
INSERT INTO BT VALUES('KkKkKk','B','b');
INSERT INTO BT VALUES('A','b','C');
```

```
INSERT INTO BT VALUES('avcs','A','b');
```

在顶层查询中添加 BINARY 前缀。

```
SELECT BINARY C1 FROM BT;
```

查询结果如下：

```
行号          BINARYC1

---------- --------------

1             0x416142624363

2             0x4B6B4B6B4B6B

3             0x41

4             0x61766373
```

在子查询中添加 BINARY 前缀。

```
SELECT * FROM (SELECT BINARY C1 FROM BT);
```

查询结果如下：

```
行号          C1

---------- ------

1             AaBbCc

2             KkKkKk

3             A

4             avcs
```

在条件查询的子查询中添加 BINARY 前缀。

```
SELECT C1 FROM BT WHERE C1 = (SELECT TOP 1 BINARY C1 FROM BT);

行号          C1

---------- ------

1             AaBbCc
```

BINARY 前缀位于过滤条件、连接条件、HAVING 条件或层次查询条件中的表达式之前时，该表达式在比较时按照大小写敏感比较，且 BINARY 前缀只对当前 and/or 子句生效。例如：在表达式 c1 = 'a' and binary c2 = 'b' and c3 = 'c'中只有第二个条件一定按照大小写敏感比较，其它两个条件仍按照数据库参数是否大小写敏感比较。

例 1 在数据库初始化为大小写不敏感的情况下（即参数 CASE_SENSITIVE=0），执行查询语句，其中只有一条 and 子句添加 BINARY 前缀。

```
SELECT C1,C2,C3 FROM BT WHERE C1 = 'a' AND BINARY C2 = 'b' AND C3 = 'c';
```

查询结果如下：

```
行号          C1 C2 C3

---------- -- -- --

1             A  b  C
```

例 2 在数据库初始化为大小写敏感的情况下（即参数 CASE_SENSITIVE=1），执行查询语句，其中只有一条 and 子句添加 BINARY 前缀。

```
SELECT C1,C2,C3 FROM BT WHERE C1 = 'a' AND BINARY C2 = 'b' AND C3 = 'c';
```

查询结果如下：

```
未选定行
```

BINARY 前缀位于排序项或分组项中的表达式之前时，该表达式按照大小写敏感进行排序/分组，若存在多个排序项/分组项，则仅有含有 BINARY 前缀的项生效，其余项仍按照数据库参数是否大小写敏感进行排序/分组。

例 在数据库初始化为大小写不敏感的情况下，对排序项中不添加 BINARY 前缀与添加 BINARY 前缀进行对比。

排序项中不添加 BINARY 前缀。

```
SELECT C1,C2,C3 FROM BT ORDER BY C2;
```

查询结果如下：

```
行号          C1       C2 C3

---------- ------ -- --

1             AaBbCc a    A

2              avcs     A    b

3              A          b    C

4              KkKkKk B    b
```

对排序项中添加 BINARY 前缀。

```
SELECT C1,C2,C3 FROM BT ORDER BY BINARY C2;
```

查询结果如下：

```
行号          C1       C2 C3

---------- ------ -- --

1             avcs     A    b

2              KkKkKk B    b

3              AaBbCc a    A

4              A          b    C
```

使用说明

1. 仅对字符类型生效，其他数据类型忽略 BINARY 前缀。

2. 多列比较不支持 BINARY 前缀，例如多列逻辑比较，多列 IN LIST 等。

3. 创建索引时忽略 BIANRY 前缀。

4. 确定性函数参数忽略 BINARY 前缀。

5. CONTAINS 表达式忽略 BINARY 前缀。

6. ALL/SOME/ANY 子查询忽略 BINARY 前缀。

7. 层次查询表达式忽略 BINARY 前缀。

8. 集函数参数包括 WITHIN GROUP 中排序表达式忽略 BINARY 前缀。

9. 分析函数参数包括 OVER 中的排序表达式、分组表达式忽略 BINARY 前缀。

4.17 数组查询

在 DM 中，可以通过查询语句查询数组信息。即<FROM 子句>中<普通表>使用数组。语法如下:

```
FROM ARRAY <数组>
```

目前 DM 只支持一维数组的查询。

数组类型可以是记录类型和普通数据库类型。如果为记录类型的数组，则记录的成员都必须为标量 (基本) 数据类型。记录类型数组查询出来的列名为记录类型每一个属性的名字。普通数据库类型查询出来的列名均为“COLUMN_VALUE”。

例 1　查看数组

```
SELECT * FROM ARRAY NEW INT[2]{1};
```

查询结果如下:

```
COLUMN_VALUE

---------------

1

NULL
```

例 2　数组与表的连接

```
DECLARE
    TYPE rrr IS RECORD (x INT, y INT);
    TYPE ccc IS ARRAY rrr[];
    c ccc;
BEGIN
    c = NEW rrr[2];
```

```
    FOR i IN 1..2 LOOP
        c[i].x = i;
        c[i].y = i*2;
    END LOOP;
    SELECT arr.x, o.name   FROM ARRAY c arr, SYSOBJECTS o WHERE arr.x = o.id;
END;
```

查询结果如下：

```
X                NAME
-----------   ----------
1                SYSINDEXES
2                SYSCOLUMNS
```

4.18 TABLE 函数查询

在 DM 中，可以通过 TABLE 函数查询集合信息。即在<FROM 子句>中的<普通表>选择使用 TABLE 函数。语法如下：

```
FROM TABLE(<collection_exp>)
```

<collection_exp>参数为集合类型，支持 VARRAY 数组集合、嵌套表集合或返回值为数据集合的自定义函数。使用 TABLE 函数查询将返回一张表，表中包含 COLUMN_VALUE 列，集合元素值将作为 COLUMN_VALUE 列数据，COLUMN_VALUE 列的数据类型与集合元素数据类型一致。

如果<collection_exp>为子查询，则该子查询结果集中只能包含一条记录，包含多条记录时将报错。

如果<collection_exp>为用户自定义函数（用户自定义函数的相关语法具体请参考《DM8_SQL 程序设计》），需确保该函数的返回值为集合类型，否则将会报错。

例 1　通过 TABLE 函数查看 VARRAY 数组集合中的数据。

```
--创建 VARRAY 数组集合 ARR1
CREATE OR REPLACE TYPE ARR1 AS VARRAY(3) OF NUMBER;
/
--建立表 T_ARR，表中第二列的列类型为 ARR1，并向表中插入数据。
CREATE TABLE T_ARR (C1 INT, C2 ARR1);
INSERT INTO T_ARR VALUES(1,ARR1(1,2));
INSERT INTO T_ARR VALUES(2,ARR1(3,4,5));
--查询 VARRAY 数组集合中的数据
SELECT * FROM TABLE(SELECT C2 FROM T_ARR WHERE C1=2);
```

查询结果如下：

```
行号        COLUMN_VALUE
---------- ------------
1              3
2              4
3              5
```

例 2　通过 TABLE 函数查看嵌套表中的数据。

```
--创建嵌套表类型 CHA1
CREATE OR REPLACE TYPE CHA1 AS TABLE OF CHAR;
/
--建立表 T_CHAR，表中第二列的列类型为 CHA1，并向表中插入数据。
CREATE TABLE T_CHAR (C1 INT, C2 CHA1);
INSERT INTO T_CHAR VALUES(1,CHA1('A','B'));
INSERT INTO T_CHAR VALUES(2,CHA1('C','D','E'));
```

```
--查询嵌套表中的数据
SELECT * FROM TABLE(SELECT C2 FROM T_CHAR WHERE C1=2);
```

查询结果如下:

```
行号        COLUMN_VALUE
---------- ------------
1               C
2               D
3               E
```

例 3　通过 TABLE 函数查看自定义函数返回结果集中的数据。

```
--创建嵌套表类型 NUM1
CREATE OR REPLACE TYPE NUM1 AS TABLE OF NUMBER;
/
--建立表 T_NUM，表中第二列的列类型为 NUM1，并向表中插入数据。
CREATE TABLE T_NUM (C1 INT, C2 NUM1);
INSERT INTO T_NUM VALUES(1,NUM1(1,2,3));
INSERT INTO T_NUM VALUES(2,NUM1(4,5,6,7));
--创建返回类型为 NUM1 的自定义函数 GetNum()，用于获取数据
CREATE FUNCTION GetNum()
RETURN NUM1
AS
    n NUM1;
BEGIN
     SELECT C2 INTO n FROM T_NUM WHERE C1 = 2;
     return n;
```

```
END;
/
--查询自定义函数的返回值集合
SELECT * FROM TABLE(GetNum());
```

查询结果如下:

```
行号        COLUMN_VALUE
---------- ------------
1           4
2           5
3           6
4           7
```

4.19 查看执行计划

4.19.1 EXPLAIN

EXPLAIN 语句可以查看 DML 语句的执行计划。

语法格式

```
EXPLAIN <SQL 语句>;
<SQL 语句> ::= <删除语句> | <插入语句> | <查询语句> | <更新语句> | <MERGE INTO 语句>
```

参数

1．<删除语句> 指数据删除语句;

2．<插入语句> 指数据插入语句;

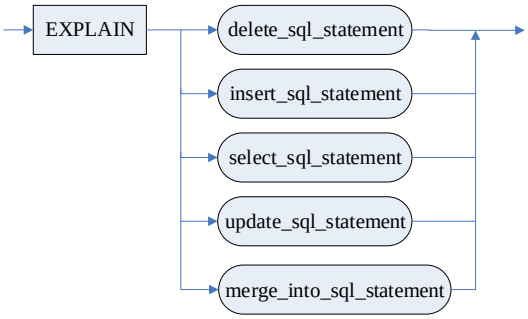
3．<查询语句> 指查询语句;

4．<更新语句> 指数据更新语句;

5．< MERGE INTO 语句> 指依据设定条件，将源表数据合并至目标表中的语句。

图例

EXPLAIN 语句



语句功能

供用户查看执行计划。用户须拥有相应 DML 语句的权限。

举例说明

例 显示如下语句的查询计划：

```
EXPLAIN SELECT NAME,schid

FROM SYSOBJECTS

WHERE SUBTYPE$ ='STAB' AND NAME

NOT IN (

SELECT NAME FROM SYSOBJECTS WHERE NAME IN (SELECT NAME FROM SYSOBJECTS WHERE

SUBTYPE$ ='STAB')

AND TYPE$ ='DSYNOM');
```

查询结果如下：

```
1    #NSET2: [1, 27, 100]

2       #PRJT2: [1, 27, 100]; exp_num(2), is_atom(FALSE)

3          #HASH LEFT SEMI JOIN2: [1, 27, 100];   (ANTI),KEY_NUM(1);

KEY(SYSOBJECTS.NAME=DMTEMPVIEW_16778306.colname) KEY_NULL_EQU(0)
```

```
4            #SLCT2: [1, 27, 100]; SYSOBJECTS.SUBTYPE$ = 'STAB'   SLCT_PUSHDOWN(TRUE)
5              #CSCN2: [1, 1092, 100]; SYSINDEXSYSOBJECTS(SYSOBJECTS as SYSOBJECTS)
NEED_SLCT(TRUE); btr_scan(1)
6            #PRJT2: [1, 1, 96]; exp_num(1), is_atom(FALSE)
7              #INDEX JOIN SEMI JOIN2: [1, 1, 96];   join condition(SYSOBJECTS.SUBTYPE$ = 'STAB')
8                #CSEK2: [1, 27, 96]; scan_type(ASC), SYSINDEXSYSOBJECTS(SYSOBJECTS as
SYSOBJECTS), scan_range[('DSYNOM',min,min),('DSYNOM',max,max))
9                #BLKUP2: [1, 2, 96]; SYSINDEXNAMESYSOBJECTS(SYSOBJECTS) ; use_clu_addr(0)
10                 #SSEK2: [1, 2, 96]; scan_type(ASC), SYSINDEXNAMESYSOBJECTS(SYSOBJECTS
as SYSOBJECTS), scan_range[SYSOBJECTS.NAME,SYSOBJECTS.NAME] ,is_global(0)
```

4.19.2 EXPLAIN FOR

EXPLAIN FOR 语句也用于查看 DML 语句的执行计划，不过执行计划以结果集的方式返回。

EXPLAIN FOR 显示的执行计划信息更加丰富，除了常规计划信息，还包括创建索引建议、分区表的起止分区信息等。重要的是，语句的计划保存在数据表中，方便用户随时查看，进行计划对比分析，可以作为性能分析的一种方法。

语法格式

```
EXPLAIN  [AS <计划名称>]  FOR <SQL 语句>;
<SQL 语句> ::= <删除语句> | <插入语句> | <查询语句> | <更新语句>
```

参数

1．<删除语句> 指数据删除语句；

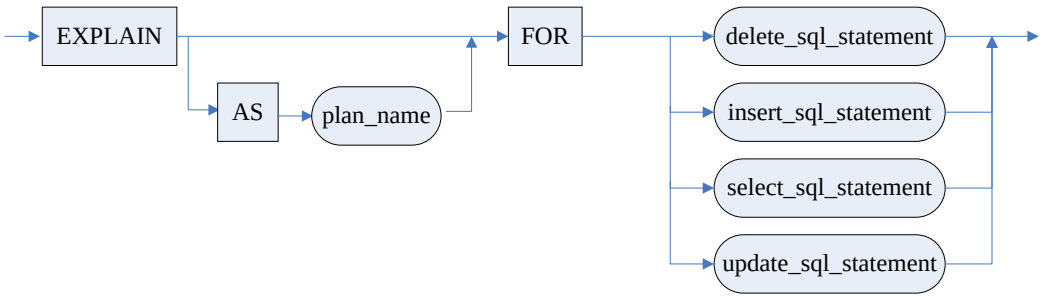
2．<插入语句> 指数据插入语句；

3．<查询语句> 指查询语句；

4．<更新语句> 指数据更新语句。

图例

EXPLAIN FOR 语句



语句功能

供用户以结果集的方式查看执行计划。用户须拥有相应 DML 语句的权限。

举例说明

例 1 以结果集的方式显示如下语句的查询计划：

```
EXPLAIN FOR SELECT NAME, SCHID FROM    SYS.SYSOBJECTS WHERE SUBTYPE$ ='STAB';
```

查询结果如下，可见未设置计划名称（即 PLAN_NAME），缺省为 NULL：

```
行号          PLAN_ID       PLAN_NAME CREATE_TIME                    LEVEL_ID      OPERATION
TAB_NAME     IDX_NAME
---------- ----------- --------- ------------------------- ----------- ---------
---------- -------------------
             SCAN_TYPE SCAN_RANGE ROW_NUMS                BYTES          COST
CPU_COST
             --------- ---------- -------------------- -----------
-------------------- --------------------
             IO_COST                  FILTER                          JOIN_COND ADVICE_INFO
PSTART         PSTOP
             -------------------- --------------------------- ---------
```

```
---------- ---------- ----------

1          4          NULL       2022-12-14 13:52:46.000000 0          NSET2      NULL

NULL

           NULL       NULL       65                    100         1                  0

           0                     NULL                          NULL       NULL       0

0


行号       PLAN_ID    PLAN_NAME CREATE_TIME                   LEVEL_ID   OPERATION

TAB_NAME   IDX_NAME

---------- ---------- --------- ------------------------- ---------- ---------

---------- ----------------

           SCAN_TYPE SCAN_RANGE ROW_NUMS              BYTES      COST

CPU_COST

           --------- ---------- ------------------- -----------

------------------- --------------------

           IO_COST              FILTER                        JOIN_COND ADVICE_INFO

PSTART     PSTOP

           ------------------- --------------------------- ---------

---------- ---------- ----------

2          4          NULL       2022-12-14 13:52:46.000000 1          PRJT2      NULL

NULL

           NULL       NULL       65                    100         1                  0

           0                     NULL                          NULL       NULL       0

0
```

```
行号          PLAN_ID       PLAN_NAME CREATE_TIME                  LEVEL_ID      OPERATION
TAB_NAME     IDX_NAME
---------- ----------- --------- -------------------------- ----------- ---------
---------- ------------------
            SCAN_TYPE SCAN_RANGE ROW_NUMS                BYTES          COST
CPU_COST
            --------- ---------- ------------------- -----------
-------------------- --------------------
            IO_COST                  FILTER                          JOIN_COND ADVICE_INFO
PSTART        PSTOP
            -------------------- ---------------------------- ---------
----------- ----------- -----------
3             4               NULL        2022-12-14 13:52:46.000000 2              SLCT2       NULL
NULL
            NULL         NULL          65                        100            1                         0
            0                        SYSOBJECTS.SUBTYPE$ = 'STAB' NULL         NULL           0
0

行号          PLAN_ID       PLAN_NAME CREATE_TIME                  LEVEL_ID      OPERATION
TAB_NAME     IDX_NAME
---------- ----------- --------- -------------------------- ----------- ---------
---------- ------------------
            SCAN_TYPE SCAN_RANGE ROW_NUMS                BYTES          COST
```

```
CPU_COST
           --------- ---------- -------------------- -----------
-------------------- --------------------
           IO_COST                FILTER                          JOIN_COND ADVICE_INFO
PSTART         PSTOP
           ------------------- ---------------------------- ---------
----------- ----------- -----------
4          4              NULL        2022-12-14 13:52:46.000000 3              CSCN2
SYSOBJECTS SYSINDEXSYSOBJECTS
           NULL        NULL        1103                100          1                      0
           0                       NULL                            NULL        NULL         0
0
```

例 2 设置计划名称为 A1，并以结果集的方式显示如下语句的查询计划：

```
EXPLAIN AS A1 FOR SELECT NAME, SCHID FROM SYS.SYSOBJECTS WHERE SUBTYPE$='STAB';
```

查询结果如下：

```
行号         PLAN_ID       PLAN_NAME CREATE_TIME                 LEVEL_ID      OPERATION
TAB_NAME     IDX_NAME
---------- ----------- --------- -------------------------- ----------- ---------
---------- ------------------
           SCAN_TYPE SCAN_RANGE ROW_NUMS               BYTES         COST
CPU_COST
           --------- ---------- -------------------- -----------
-------------------- --------------------
           IO_COST                FILTER                          JOIN_COND ADVICE_INFO
```

```
PSTART        PSTOP

            -------------------- ---------------------------- ---------

----------- ----------- -----------

1             6             A1            2022-12-14 13:55:55.000000 0             NSET2       NULL

NULL

            NULL        NULL        65                      100           1                     0

            0                       NULL                            NULL        NULL          0

0


行号          PLAN_ID       PLAN_NAME CREATE_TIME                 LEVEL_ID      OPERATION

TAB_NAME    IDX_NAME

---------- ----------- --------- -------------------------- ----------- ---------

---------- -------------------

            SCAN_TYPE SCAN_RANGE ROW_NUMS               BYTES         COST

CPU_COST

            --------- ---------- ------------------- -----------

-------------------- --------------------

            IO_COST                 FILTER                          JOIN_COND ADVICE_INFO

PSTART        PSTOP

            -------------------- ---------------------------- ---------

----------- ----------- -----------

2             6             A1            2022-12-14 13:55:55.000000 1             PRJT2       NULL

NULL

            NULL        NULL        65                      100           1                     0
```

```
           0                       NULL                            NULL        NULL          0
0


行号         PLAN_ID       PLAN_NAME CREATE_TIME                   LEVEL_ID      OPERATION
TAB_NAME    IDX_NAME
---------- ----------- --------- ------------------------- ----------- ---------
---------- ------------------
           SCAN_TYPE SCAN_RANGE ROW_NUMS                BYTES          COST
CPU_COST
           --------- ---------- -------------------- -----------
------------------- --------------------
           IO_COST                 FILTER                          JOIN_COND ADVICE_INFO
PSTART        PSTOP
           -------------------- ---------------------------- ---------
----------- ----------- -----------
3          6              A1          2022-12-14 13:55:55.000000 2             SLCT2       NULL
NULL
           NULL        NULL        65                      100          1                     0
           0                       SYSOBJECTS.SUBTYPE$ = 'STAB' NULL        NULL          0
0


行号         PLAN_ID       PLAN_NAME CREATE_TIME                   LEVEL_ID      OPERATION
TAB_NAME    IDX_NAME
---------- ----------- --------- ------------------------- ----------- ---------
```

```
---------- ------------------

            SCAN_TYPE SCAN_RANGE ROW_NUMS                BYTES          COST

CPU_COST

            --------- ---------- -------------------- -----------

-------------------- --------------------

            IO_COST                  FILTER                              JOIN_COND ADVICE_INFO

PSTART         PSTOP

            -------------------- ---------------------------- ---------

----------- ----------- -----------

4           6               A1          2022-12-14 13:55:55.000000 3              CSCN2

SYSOBJECTS SYSINDEXSYSOBJECTS

            NULL        NULL        1103                    100          1                       0

            0                       NULL                                NULL        NULL         0

0
```

4.20 SAMPLE 子句

DM 通过 SAMPLE 子句实现数据采样功能。

语法格式

```
<SAMPLE 子句>::=

    SAMPLE (<表达式>) |

    SAMPLE (<表达式>) SEED (<表达式>) |

    SAMPLE BLOCK (<表达式>) |

    SAMPLE BLOCK (<表达式>) SEED (<表达式>)
```

参数

1. <表达式> 输入整数与小数均可;

2. SAMPLE (<表达式>) 按行采样。<表达式>表示采样百分比，取值范围[0.000001,100)。重复执行相同语句，返回的结果不要求一致；

3. SAMPLE (<表达式>) SEED (<表达式>) 按行采样，并指定种子。其中 SEED (<表达式>)表示种子，取值范围 0~4294967295。重复执行相同的语句，每次返回相同的结果集；但在并行环境下，结果并不可控，即使重复执行相同的语句，也不一定返回相同的结果集；

4. SAMPLE BLOCK (<表达式>) 按块(页)采样。<表达式>表示采样百分比，取值范围[0.000001,100)。重复执行相同语句，返回的结果不要求一致，允许返回空集；

5. SAMPLE BLOCK (<表达式>) SEED (<表达式>) 按块(页)采样，并指定种子。其中，BLOCK (<表达式>)表示采样百分比，取值范围[0.000001,100)。SEED (<表达式>)表示种子，取值范围 0~4294967295。重复执行相同语句，每次返回相同的结果集。

使用说明

1. SAMPLE 只能出现在单表或仅包含单表的视图后面；

2. 包含过滤条件的 SAMPLE 查询，是对采样后的数据再进行过滤；

3. 不能对连接查询、子查询使用 SAMPLE 子句。

举例说明

例 对PERSON.ADDRESS表按行进行种子为5的10%采样。

```
SELECT * FROM PERSON.ADDRESS SAMPLE(10) SEED(5);
```

查询结果如下：

```
ADDRESSID   ADDRESS1                    ADDRESS2 CITY              POSTALCODE
----------- --------------------------- -------- ----------------- ----------
14          洪山区关山春晓 10-1-202                 武汉市洪山区      430073
```

4.21 水平分区表查询

SELECT 语句从水平分区子表中检索数据，称水平分区子表查询，即<对象名>中使用的是<分区表>。水平分区父表的查询方式和普通表完全一样。

```
<分区表>::=
    [<模式名>.]<基表名> PARTITION (<一级分区名>) |
    [<模式名>.]<基表名> PARTITION FOR (<表达式>,{<表达式>})|
    [<模式名>.]<基表名> SUBPARTITION (<子分区名>)|
    [<模式名>.]<基表名> SUBPARTITION FOR (<表达式>,{<表达式>})
```

参数

1. <基表名> 水平分区表父表名称;
2. <一级分区名> 水平分区表一级分区的名字;
3. <子分区名> 由水平分区表中多级分区名字逐级通过下划线“_”连接在一起的组合名称，例如 P1_P2_P3，其中 P1 是一级分区名、P2 是二级分区名、P3 是三级分区名;
4. <表达式> 分区中的数据值，系统将根据数据值自动定位分区。

使用说明

如果 HASH 分区不指定分区表名，而是通过指定哈希分区个数来建立哈希分区表，PARTITIONS 后的数字表示哈希分区的分区数，使用这种方式建立的哈希分区表分区名是匿名的，DM 统一使用 DMHASHPART+分区号（从 0 开始）作为分区名。

举例说明

例 1 查询一个 LIST-RANGE 三级水平分区表。

建表并插入数据:

```
DROP TABLE STUDENT;
CREATE TABLE STUDENT(
NAME VARCHAR(20),
```

```
AGE INT,
SEX VARCHAR(10) CHECK (SEX IN ('MALE','FEMALE')),
GRADE INT CHECK (GRADE IN (7,8,9))
)
PARTITION BY LIST(GRADE)
    SUBPARTITION BY LIST(SEX) SUBPARTITION TEMPLATE
    (
      SUBPARTITION Q1 VALUES('MALE'),
      SUBPARTITION Q2 VALUES('FEMALE')
    ),
    SUBPARTITION BY RANGE(AGE) SUBPARTITION TEMPLATE
    (
       SUBPARTITION R1 VALUES LESS THAN (12),
       SUBPARTITION R2 VALUES LESS THAN (15),
       SUBPARTITION R3 VALUES LESS THAN (MAXVALUE)
     )
(
    PARTITION P1 VALUES (7),
    PARTITION P2 VALUES (8),
    PARTITION P3 VALUES (9)
);
INSERT  INTO  STUDENT(NAME, AGE, SEX, GRADE) VALUES ('李丽', 12, 'FEMALE', 7), ('郭艳', 15, 'FEMALE',
8), ('王刚', 11, 'MALE', 9), ('李勇', 16, 'MALE', 7), ('王菲', 13, 'FEMALE', 8), ('刘波', 11, 'MALE', 7);
```

执行以下查询语句：

```
SELECT * FROM STUDENT;                                          //查询水平分区父表
```

执行结果如下：

```
行号        NAME     AGE            SEX      GRADE

---------- ------ ----------- ------ -----------

1             李丽  12             FEMALE 7

2             刘波  11             MALE     7

3             李勇  16              MALE     7

4             王菲  13             FEMALE 8

5             郭艳  15             FEMALE 8

6             王刚  11             MALE     9
```

执行以下查询语句：

```
//以下两个查询语句执行结果相同

SELECT * FROM STUDENT PARTITION(P1);                  //通过指定分区名查询一级分区子表

SELECT * FROM STUDENT PARTITION FOR(7);               //通过指定分区数据查询一级分区子表
```

执行结果如下：

```
行号        NAME     AGE            SEX      GRADE

---------- ------ ----------- ------ -----------

1             李丽  12             FEMALE 7

2             刘波  11             MALE     7

3             李勇  16              MALE     7
```

执行以下查询语句：

```
//以下两个查询语句执行结果相同

SELECT * FROM STUDENT SUBPARTITION(P1_Q1);             //通过指定分区名查询二级分区子表

SELECT * FROM STUDENT SUBPARTITION FOR(7, 'MALE');     //通过指定分区数据查询二级分区子表
```

执行结果如下：

```
行号        NAME     AGE             SEX    GRADE

---------- ------ ----------- ---- -----------

1              刘波  11              MALE 7

2              李勇  16              MALE 7
```

执行以下查询语句：

```
//以下两个查询语句执行结果相同

SELECT * FROM STUDENT SUBPARTITION(P1_Q1_R1);          //通过指定分区名查询三级分区子表

SELECT * FROM STUDENT SUBPARTITION FOR(7, 'MALE', 10);  //通过指定分区数据查询三级分区子表
```

执行结果如下：

```
行号        NAME     AGE             SEX    GRADE

---------- ------ ----------- ---- -----------

1              刘波  11              MALE 7
```

例2 查询一个指定HASH分区名的水平分区表。

建表并插入数据：

```
CREATE TABLESPACE TS1 DATAFILE 'TS1.DBF' SIZE 128;

CREATE TABLESPACE TS2 DATAFILE 'TS2.DBF' SIZE 128;

CREATE TABLESPACE TS3 DATAFILE 'TS3.DBF' SIZE 128;

CREATE TABLESPACE TS4 DATAFILE 'TS4.DBF' SIZE 128;



DROP TABLE CP_TABLE_HASH   CASCADE;

CREATE TABLE CP_TABLE_HASH(

                C1       INT,

                C2       VARCHAR(256),
```

```
                C3      DATETIME,
                C4      BLOB
                )
                PARTITION BY HASH (C1)
                SUBPARTITION BY HASH(C2)
                SUBPARTITION TEMPLATE
                (SUBPARTITION PAR1 STORAGE (ON MAIN),
                 SUBPARTITION PAR2 STORAGE (ON TS1),
                 SUBPARTITION PAR3 STORAGE (ON TS2),
                 SUBPARTITION PAR4)
                (PARTITION PAR1 STORAGE (ON MAIN),
                 PARTITION PAR2 STORAGE (ON TS1),
                 PARTITION PAR3 STORAGE (ON TS2),
                 PARTITION PAR4)
                STORAGE (ON TS4) ;
INSERT  INTO  CP_TABLE_HASH (C1, C2, C3, C4) VALUES (1 ,'NAME1', '2023-12-26', ''), (2 ,'NAME2',
'2022-12-26', ''), (3 ,'NAME3', '2023-1-15', ''), (4 ,'NAME4', '2022-5-1', ''), (5 ,'NAME5', '2023-3-15', ''),
(6 ,'NAME6', '2023-11-11', ''), (7 ,'NAME7', '2021-9-23', ''), (8 ,'NAME8', '2023-7-8', ''), (9 ,'NAME9',
'2023-12-26', ''), (10 ,'NAME10', '2022-12-26', ''), (11 ,'NAME11', '2023-1-15', ''), (12 ,'NAME12', '2022-5-1', ''),
(13 ,'NAME13', '2023-3-15', ''), (14 ,'NAME14', '2023-11-11', ''), (15 ,'NAME15', '2021-9-23', ''), (16 ,'NAME16',
'2023-7-8', '');
```

执行以下查询语句：

```
SELECT * FROM CP_TABLE_HASH PARTITION(PAR1);                    //查询一级分区子表
```

执行结果如下：

```
行号       C1             C2       C3                          C4

---------- ----------- ------ -------------------------- --

1             12             NAME12 2022-05-01 00:00:00.000000

2             16             NAME16 2023-07-08 00:00:00.000000

3             4              NAME4   2022-05-01 00:00:00.000000

4             8              NAME8   2023-07-08 00:00:00.000000
```

执行以下查询语句：

```
SELECT * FROM CP_TABLE_HASH SUBPARTITION(PAR1_PAR1);      //查询二级分区子表
```

执行结果如下：

```
行号       C1             C2       C3                          C4

---------- ----------- ------ -------------------------- --

1             12             NAME12 2022-05-01 00:00:00.000000

2             16             NAME16 2023-07-08 00:00:00.000000
```

例 3 查询一个指定 HASH 分区数的水平分区，查询 CP_TABLE_HASH01 第一个分区的数据。

```
DROP TABLE CP_TABLE_HASH   CASCADE;

CREATE TABLE CP_TABLE_HASH(

                C1       INT,

                C2       VARCHAR(256),

                C3       DATETIME,

                C4       BLOB

                )

                PARTITION BY HASH (C1)

                PARTITIONS 4 STORE IN (TS1, TS2, TS3, TS4);
```

```
INSERT INTO CP_TABLE_HASH (C1, C2, C3, C4) VALUES (1 ,'NAME1', '2023-12-26', ''), (2 ,'NAME2', '2022-12-26', ''), (3 ,'NAME3', '2023-1-15', ''), (4 ,'NAME4', '2022-5-1', ''), (5 ,'NAME5', '2023-3-15', ''), (6 ,'NAME6', '2023-11-11', ''), (7 ,'NAME7', '2021-9-23', ''), (8 ,'NAME8', '2023-7-8', ''), (9 ,'NAME9', '2023-12-26', ''), (10 ,'NAME10', '2022-12-26', ''), (11 ,'NAME11', '2023-1-15', ''), (12 ,'NAME12', '2022-5-1', ''), (13 ,'NAME13', '2023-3-15', ''), (14 ,'NAME14', '2023-11-11', ''), (15 ,'NAME15', '2021-9-23', ''), (16 ,'NAME16', '2023-7-8', '');
```

执行以下查询语句：

```
SELECT * FROM CP_TABLE_HASH PARTITION (DMHASHPART0); //查询一级分区子表
```

执行结果如下：

```
行号     C1          C2      C3                          C4

---------- ----------- ------ -------------------------- --

1          4           NAME4   2022-05-01 00:00:00.000000

2          8           NAME8   2023-07-08 00:00:00.000000

3          12          NAME12 2022-05-01 00:00:00.000000

4          16          NAME16 2023-07-08 00:00:00.000000
```

4.22 指定索引查询

可在 SELECT 语句中使用指定的索引进行查询。

```
<索引>::=[<模式名>.]<基表名> INDEX <索引名>
```

参数

1. <模式名> 被选择的表和视图所属的模式，缺省为当前模式；
2. <基表名> 被选择数据的基表的名称；
3. <索引名> 为查询指定的索引名称。

使用说明

如果不指定索引，系统会选择一个最优的索引进行查询；指定索引后，系统则会按照指

定的索引进行查询。为了避免指定了不合适的索引，导致查询性能变差。建议用户在熟知索引的用法之后，再为查询指定索引。

举例说明

例 1 使用指定的索引 INDEX1 进行查询。此处如果没有指定索引，系统将会为本次查询自动选择最优的索引 INDEX2。

```
CREATE INDEX INDEX1 ON PERSON.ADDRESS(ADDRESS1);
CREATE INDEX INDEX2 ON PERSON.ADDRESS(ADDRESS2);
--使用指定的 INDEX1
SQL> EXPLAIN SELECT CITY FROM PERSON.ADDRESS INDEX INDEX1 WHERE ADDRESS2='洪山区保利花园
50 号';
1    #NSET2: [1, 1, 108]
2      #PRJT2: [1, 1, 108]; exp_num(2), is_atom(FALSE)
3        #SLCT2: [1, 1, 108]; ADDRESS.ADDRESS2 = '洪山区保利花园 50 号'
4          #BLKUP2: [1, 16, 108]; INDEX1(ADDRESS) ; use_clu_addr(0)
5            #SSCN: [1, 16, 108]; INDEX1(ADDRESS); btr_scan(1); is_global(0)
--未指定的情况下，系统自动选择最优的 INDEX2
SQL> EXPLAIN SELECT CITY FROM PERSON.ADDRESS WHERE ADDRESS2='洪山区保利花园 50 号';
1    #NSET2: [1, 1, 108]
2      #PRJT2: [1, 1, 108]; EXP_NUM(2), IS_ATOM(FALSE)
3        #BLKUP2: [1, 1, 108]; INDEX2(ADDRESS) ; use_clu_addr(0)
4          #SSEK2: [1, 1, 108]; SCAN_TYPE(ASC), INDEX2(ADDRESS), SCAN_RANGE['洪山区保利花
园 50 号','洪山区保利花园 50 号'] , is_global(0)
```

第 5 章 数据的插入、删除和修改

DM_SQL 语言的数据更新语句包括：数据插入、数据修改和数据删除三种语句，其中数据插入和修改两种语句使用的格式要求比较严格。在使用时要求对相应基表的定义，如列的个数、各列的排列顺序、数据类型及关键约束、唯一性约束、引用约束、检查约束的内容均要了解得很清楚，否则就很容易出错。下面将分别对这三种语句进行讨论。在讨论中，如不特别说明，各例均使用示例库 BOOKSHOP，用户均为建表者 SYSDBA。

5.1 数据插入语句

数据插入语句用于向已定义好的表中插入单个或成批的数据。

INSERT 语句有两种形式。一种形式是值插入，即构造一行或者多行，并将它们插入到表中；另一种形式为查询插入，它通过<查询表达式>返回一个查询结果集以构造要插入表的一行或多行。

数据插入语句的语法格式如下：

语法格式

```
<插入表达式>::=
[<WITH 子句>] INSERT {<single_insert_stmt> | <multi_insert_stmt>};
<WITH 子句> ::= [<WITH FUNCTION 子句>] [WITH CTE 子句]  具体参见 4.4 WITH 子句
<single_insert_stmt>::=[INTO] <full_tv_name> [<t_alias>] <insert_tail> [<return_into_obj>]
<full_tv_name>::= <tv_name> | <子查询表达式>
<tv_name>::=
    <单表引用> [@ <dblink_name>] |
    [<模式名>.]<基表名> INDEX <索引名> |
```

```
    [<模式名>.]<基表名>  PARTITION (<分区名>)
<单表引用>::=[<模式名>.]<基表或视图名>
<基表或视图名>::=<基表名>|<视图名>
<子查询表达式>::=(<查询表达式>) [[AS] <表别名>]
<t_alias>::=[AS] <表别名>
<insert_tail>::= [(<列名>{,<列名>})]<insert_action>
<insert_action>::=
    VALUES <ins_value> |
    <查询表达式>|(<查询表达式>) |
    DEFAULT VALUES |
    TABLE <tv_name>
<return_into_obj>::=
    <RETURN|RETURNING><expr{,expr}>INTO <data_item {,data_item }> |
    <RETURN|RETURNING><expr{,expr}>BULK COLLECT INTO <data_item {,data_item}>
<查询表达式>::= 参见第 4 章 数据查询语句


<multi_insert_stmt>::=
    ALL <multi_insert_into_list> <查询表达式> |
    [ALL|FIRST]<multi_insert_into_condition_list> [<multi_insert_into_else>]<查询表达式>
<multi_insert_into_list>::= <insert_into_single>{<insert_into_single>}
<insert_into_single>::=
    INTO <tv_name> [<t_alias>] [(<列名>{,<列名>})][VALUES <ins_value>]
<ins_value>::=
 (<expr>|DEFAULT|<查询表达式>  {,<expr>|DEFAULT|<查询表达式>}){,(<expr>|DEFAULT|<查询表达
```

```
式> {,<expr>|DEFAULT|<查询表达式>})}

<multi_insert_into_condition_list> ::=

<insert_into_single_condition>{,< insert_into_single_condition>}

<insert_into_single_condition>::=

 WHEN <bool_exp> THEN <multi_insert_into_list>

<multi_insert_into_else>::= ELSE <multi_insert_into_list>
```

参数

1. <模式名> 指明该表或视图所属的模式，缺省为当前模式；

2. <基表名> 指明被插入数据的基表的名称；

3. <视图名> 指明被插入数据的视图的名称，实际上 DM 将数据插入到视图引用的基表中；

4. <列名> 表或视图的列的名称。在插入的记录中，这个列表中的每一列都被 VALUES 子句或查询说明赋一个值。如果在此列表中省略了表的一个列名，则 DM 用先前定义好的缺省值插入到这一列中。如果此列表被省略，则在 VALUES 子句和查询中必须为表中的所有列指定值；

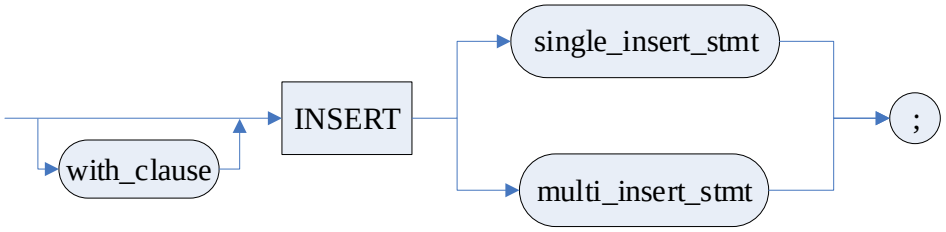
5. <ins_value> 指明在列表中对应的列的插入的列值，如果列表被省略了，插入的列值按照基表中列的定义顺序排列。所有的插入值和系统内部相关存储信息一起构成了一条记录，一条记录的长度不能大于页面大小的一半；

6. <查询表达式> 将一个 SELECT 语句所返回的记录插入表或视图的基表中，子查询中选择的列表必须和 INSERT 语句中列名清单中的列具有相同的数量；带有<查询表达式>的插入方式，称查询插入。插入中使用的<查询表达式>也称为查询说明；

7. <dblink_name> 表示创建的 dblink 名字，如果添加了该选项，则表示插入远程实例的表。

图例

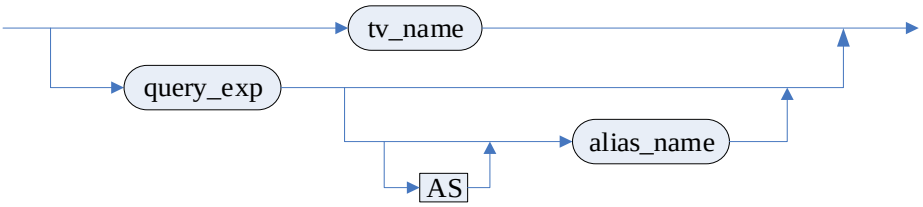
<插入表达式>



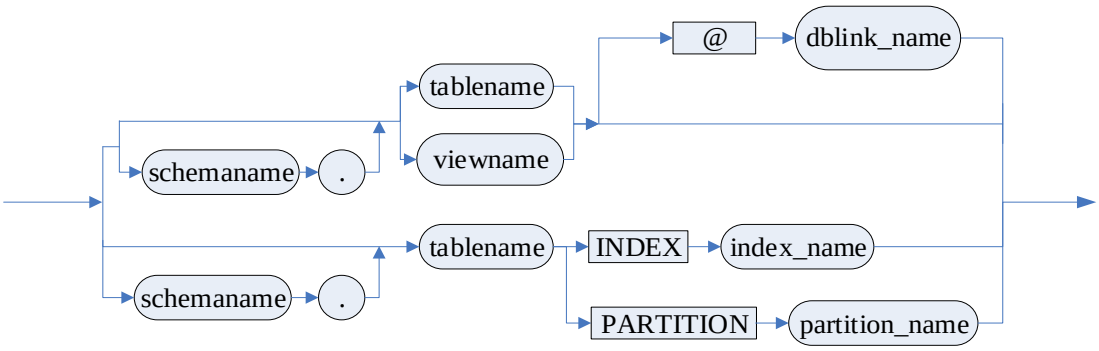
<single_insert_stmt>



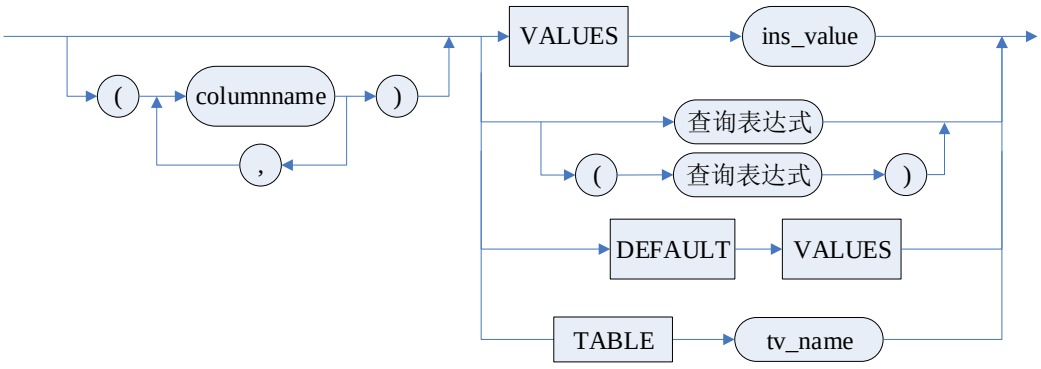
<full_tv_name>



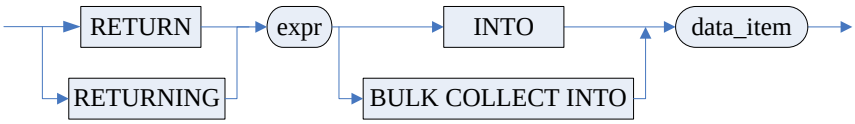
<tv_name>



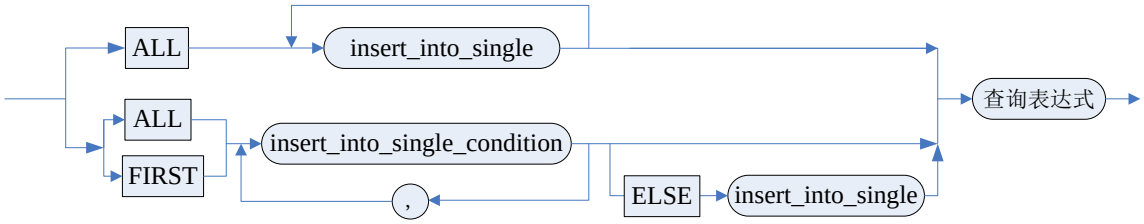
<insert_tail>



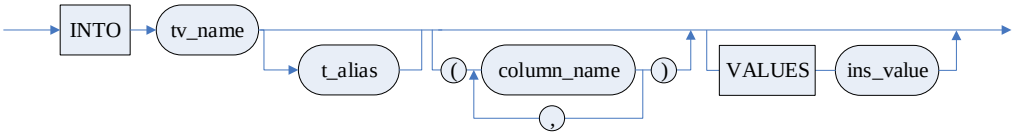
<return_into_obj>



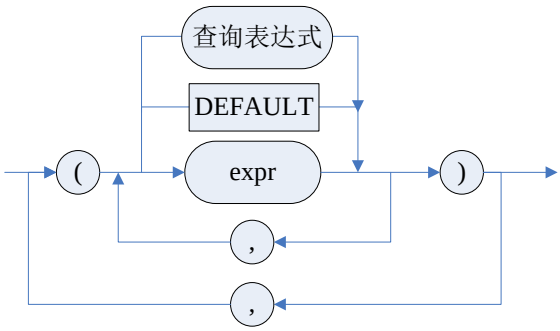
<multi_insert_stmt>



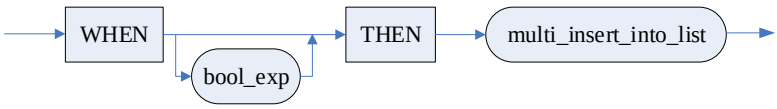
<insert_into_single>



<ins_value>



<insert_into_single_condition>



使用说明

1. <基表名>或<视图名>后所跟的<列名>必须是该表中的列，且同一列名不允许出现两次，但排列顺序可以与定义时的顺序不一致；

2. <ins_value>中插入值的个数、类型和顺序要与<列名>一一对应；

3. 插入在指定值的时候，可以同时指定多行值，这种叫做多行插入或者批量插入。多行插入不支持列存储表；

4. 如果某一<列名>未在 INTO 子句后面出现，则新插入的行在这些列上将取空值或缺省值，如该列在基表定义时说明为 NOT NULL 时将会出错；

5. 如果<基表名>或<视图名>后没指定任何<列名>，则隐含指定该表或视图的所有列，这时，新插入的行必须在每个列上均有<插入值>；

6. 当使用<子查询表达式>作为 INSERT 的目标时，实际上是对查询表达式的基表进行操作，查询表达式的查询项必须都来源于同一个基表且不能是计算列，查询项所属的基表即是查询表达式的基表，如果查询表达式是带有连接的查询，那么对于连接中视图基表以外的表，连接列上必须是主键或者带有 UNIQUE 约束。不支持 PIVOT/UNPIVOT，不支持 UNION/UNION ALL 查询；

7. 如果两表之间存在引用和被引用关系时，应先插入被引用表的数据，再插入引用表的数据；

8. <查询表达式>是指用查询语句得到的一个结果集插入到插入语句中<表名>指定的表中，因此该格式的使用可供一次插入多个行，但插入时要求结果集的列与目标表要插入的列是一一对应的，不然会报错；

9. 多行插入时，对于存在行触发器的表，每一行都会触发相关的触发器；同样如果目标表具有约束，那么每一行都会进行相应的约束检查，只要有一行不满足约束，所有的值都不能插入成功；

10. 支持多行插入时插入查询表达式，但需要注意将子查询当做表达式使用时，需要保证子查询表达式的返回值为一行一列，否则会报错“单行子查询返回多行”或者“SELECT 语句列数超长”；

11. 在嵌入方式下工作时，<ins_value>插入的值可以为主变量；

12. 如果插入对象是视图，同时在这个视图上建立了 INSTEAD OF 触发器，则会将插入操作转换为触发器所定义的操作；如果没有触发器，则需要判断这个视图是否可更新，如果不可更新则报错（不可更新视图场景参见 6.6 视图数据的更新），否则是可以插入成功的；

13. RETURN INTO 返回列支持返回 ROWID 和函数调用，但是堆表不支持 ROWID 作为 RETURN INTO 的返回列；RETURN INTO 返回列支持返回变量、方法参数；

14. RETURN INTO 语句中返回结果对象支持变量和数组。如果返回结果对象为记录数组，则只能有一个返回结果对象，且记录数组属性类型与个数须与返回列一致；如果为变量，则变量类型与个数与返回列一致；如果返回普通数组，则数组个数和数组元素类型与返回列一致；返回结果不支持变量、普通数组和记录数组混和使用；

15. 增删改语句当前修改表称为变异表（MUTATE TABLE），其调用函数中，不能对此变异表进行插入操作；

16. BULK COLLECT 的作用是将检索结果批量地、一次性地赋给集合变量。与每次获取一条数据，并每次都要将结果赋值给变量相比，可以很大程度上的节省开销。使用 BULK COLLECT 时，INTO 后的变量必须是集合类型的；

17. <multi_insert_stmt>中可以使用 WHEN 子句指定插入条件，仅当满足指定条件时，才插入相应数据。支持同时指定多个 WHEN 子句，可以使用 ALL 或 FIRST 控制插入策略，ALL 表示插入所有满足条件的数据，FIRST 表示仅插入第一条满足条件的数据。其中若 WHEN 子句所指定的插入条件里存在列名，该列名应当在后续的<查询表达式>中进行查询，否则报错；

18. 如果将<查询表达式>的结果插入到分区表中，可在<查询表达式>之前使用 HINT 提示 /*+PARALLEL_INS(< 表名 >)*/ 执行并行插入。例如 INSERT INTO t1 /*+PARALLEL_INS(T1)*/ SELECT LEVEL,LEVEL FROM DUAL CONNECT BY LEVEL<=1000;。多个线程同时执行插入，可提升插入性能。具体请参考《DM8 系统管理员手册》。

举例说明

例 1 在 VENDOR 表中插入一条供应商信息: 账户号为 00, 名称为华中科技大学出版社，活动标志为 1，URL 为空，信誉为 2。

```
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO, NAME, ACTIVEFLAG, WEBURL, CREDIT) VALUES
```

```
('00', '华中科技大学出版社', 1, '', 2);
```

如果需要同时多行插入，则可以用如下的 SQL 语句实现：

```
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO, NAME, ACTIVEFLAG, WEBURL, CREDIT) VALUES
('00', '华中科技大学出版社', 1, '', 2), ('00', '清华大学出版社', 1, '', 3);
```

在定义 VENDOR 表时，设定了检查约束：CHECK(CREDIT IN(1,2,3,4,5))，说明 CREDIT 只能是 1，2，3，4，5。在插入新数据或修改供应商的 CREDIT 时，系统按检查约束进行检查，如果不满足条件，系统将会报错，多行插入中，每一行都会做检查。

由于 DM 支持标量子查询，标量子查询允许用在标量值合法的地方，因此在数据插入语句的<插入值>位置允许出现标量子查询。

例 2 将书名为长征的图书的出版社插入到 VENDOR 表中。

```
INSERT INTO PURCHASING.VENDOR(ACCOUNTNO, NAME, ACTIVEFLAG, WEBURL,
CREDIT)VALUES('00',
(SELECT PUBLISHER FROM PRODUCTION.PRODUCT WHERE NAME ='长征'),1, '', 1);
```

若是需要插入一批数据时，可使用带<查询说明>的插入语句，如下例所示。

例 3 构造一个新的基表，表名为 PRODUCT_SELL，用来显示出售的商品名称和购买用户名称，并将查询的数据插入此表中。

```
CREATE TABLE PRODUCTION.PRODUCT_SELL
( PRODUCTNAME VARCHAR(50) NOT NULL,
  CUSTOMERNAME VARCHAR(50) NOT NULL);
INSERT INTO PRODUCTION.PRODUCT_SELL
SELECT DISTINCT T1.NAME , T5.NAME
FROM PRODUCTION.PRODUCT T1, SALES.SALESORDER_DETAIL T2,
        SALES.SALESORDER_HEADER T3, SALES.CUSTOMER T4,
        PERSON.PERSON T5
```

```
WHERE  T1.PRODUCTID = T2.PRODUCTID  AND  T2.SALESORDERID = T3.SALESORDERID
        AND  T3.CUSTOMERID = T4.CUSTOMERID  AND  T4.PERSONID = T5.PERSONID;
```

该插入语句将已销售的商品名称和购买该商品的用户名称插入到新建的 PRODUCT_SELL 表中。查询结果如下:

```
PRODUCTNAME CUSTOMERNAME
---------- ------------
红楼梦          刘青
老人与海        刘青
```

值得一提的是，BIT 数据类型值的插入与其他数据类型值的插入略有不同：其他数据类型皆为插入的是什么就是什么，而 BIT 类型取值只能为 1/0，又同时能与整数、精确数值类型、不精确数值类型和字符串类型相容(可以使用这些数据类型对 BIT 类型进行赋值和比较)，取值时有一定的规则。

数值类型常量向 BIT 类型插入的规则是：非 0 数值转换为 1，数值 0 转换为 0。例如:

```
CREATE TABLE T10 (C BIT);
INSERT INTO T10 VALUES(1);          //插入 1
INSERT INTO T10 VALUES(0);          //插入 0
INSERT INTO T10 VALUES(1.2);        //插入 1
```

字符串类型常量向 BIT 类型插入的规则是:

全部由 0 组成的字符串或表示 0 的科学计数字符串（如 0e0）转换为 0，其他全数字字符串(例如：123）或表示非 0 的科学计数字符串转换为 1。

```
INSERT INTO T10 VALUES(000);     //插入 0
INSERT INTO T10 VALUES(0);          //插入 0
INSERT INTO T10 VALUES(10); //插入 1
INSERT INTO T10 VALUES(1.0);        //插入 1
```

例 4 多行插入时插入查询表达式。

```
CREATE TABLE T (ID INT, NAME CHAR(50));
CREATE TABLE T1 (ID INT, NAME CHAR(50));
INSERT INTO T1 VALUES(2,'ALICE'),(1,'TOM');
INSERT INTO T VALUES((SELECT ID FROM T1 WHERE NAME='ALICE'),(SELECT NAME FROM T1 WHERE
ID=1)),((SELECT ID FROM T1 WHERE NAME='ALICE'),(SELECT NAME FROM T1 WHERE ID=1)),((SELECT
ID FROM T1 WHERE NAME='ALICE'),(SELECT NAME FROM T1 WHERE ID=1));
SELECT * FROM T;
```

查询结果如下：

```
行号          ID              NAME
---------- ----------- --------------------------------------------------
1             2               TOM
2             2               TOM
3             2               TOM
```

例 5 创建一个表 BOOK_20 用于记录当前价格小于 20 的书本的名称、原价、当前价格和折扣，同时创建一个表 BOOK_30 用于记录当前价格大于 30 的书本的名称、原价、当前价格和折扣。通过使用 WHEN 子句指定条件进行数据插入。

```
CREATE TABLE BOOK_20(NAME VARCHAR(50), ORIGINALPRICE DEC(19,4),NOWPRICE
DEC(19,4),DISCOUNT DECIMAL(2,1));
CREATE TABLE BOOK_30(NAME VARCHAR(50), ORIGINALPRICE DEC(19,4),NOWPRICE
DEC(19,4),DISCOUNT DECIMAL(2,1));
INSERT ALL WHEN NOWPRICE < 20 THEN INTO BOOK_20 WHEN NOWPRICE > 30 THEN INTO
BOOK_30 SELECT NAME, ORIGINALPRICE, NOWPRICE, DISCOUNT FROM PRODUCTION.PRODUCT;
SELECT * FROM BOOK_20;
```

```
SELECT * FROM BOOK_30;
```

BOOK_20 表查询结果如下:

```
行号        NAME                ORIGINALPRICE NOWPRICE DISCOUNT

---------- ---------------- ------------- -------- --------

1           红楼梦              19              15.2     8

2           水浒传              19              14.3     7.5

3           老人与海            10              6.1      6.1

4           工作中无小事        16.8            11.4     6.8

5           突破英文基础词汇    15.9            11.1     7
```

BOOK_30 表查询结果如下:

```
行号        NAME                    ORIGINALPRICE NOWPRICE DISCOUNT

---------- ------------------- ------------- -------- --------

1           长征                    53              37.7     6.4

2           噼里啪啦丛书(全 7 册)  58              42       6.1
```

5.2 数据修改语句

数据修改语句用于修改表中已存在的数据。

语法格式

```
[<WITH 子句>] UPDATE <更新列表> {<单列修改子句>|<多列修改子句>}[FROM <表引用>{,<表引用>}]

[WHERE <条件表达式>][<return_into_obj>][<LIMIT 限定条件>];

<WITH 子句>::=请参考第 4.4 节 WITH 子句

<更新列表>::=<表引用>{,<表引用>}

<单列修改子句>::= SET<列名>=<<值表达式>|DEFAULT>{,<列名>=<<值表达式>|DEFAULT>}

<多列修改子句>::= SET (<列名>{,<列名>}) = <子查询表达式>
```

```
<子查询表达式>::=(<查询表达式>) [[AS] <表别名>]

<表引用>::=请参考第 4 章 数据查询语句

<return_into_obj>::=

    <RETURN | RETURNING><列名>{,<列名>}INTO <结果对象> |

    <RETURN | RETURNING><列名>{,<列名>}BULK COLLECT INTO <结果对象>

<结果对象>::= <数组>|<变量>

<LIMIT 限定条件>::= <LIMIT 子句> | <ROW_LIMIT 子句>

<LIMIT 子句>::= LIMIT <记录数> |

                LIMIT <偏移量> , <记录数> |

                LIMIT <记录数> OFFSET <偏移量> |

                OFFSET <偏移量> LIMIT <记录数>

<记录数>::=<整数>

<偏移量>::=<整数>

<ROW_LIMIT 子句>::= [OFFSET <offset> ROW[S]] [<FETCH 说明>]

<FETCH 说明>::= FETCH <FIRST | NEXT> [<大小> | <大小> PERCENT] ROW[S] <ONLY | WITH TIES>
```

参数

1．<列名> 表或视图中被更新列的名称，如果 SET 子句中省略列的名称，列的值保持不变；

2．<值表达式> 指明赋予相应列的新值，可以为一个<集函数>、<函数>、<标量子查询>或<计算表达式>等；

3．<条件表达式> 指明限制被更新的行必须符合指定的条件，如果省略此子句，则修改表或视图中所有的行；

4．<LIMIT 限定条件> 指明更新数据时的限定条件，与查询语句中的<LIMIT 限定条件>功能类似，具体说明可以查看 4.10 LIMIT 限定条件。

1）LIMIT 子句：按顺序更新表中某条记录开始的 N 条记录。

LIMIT N：更新前 N 条记录；

LIMIT M , N：更新第 M 条记录之后的 N 条记录；

LIMIT M OFFSET N：更新第 N 条记录之后的 M 条记录；

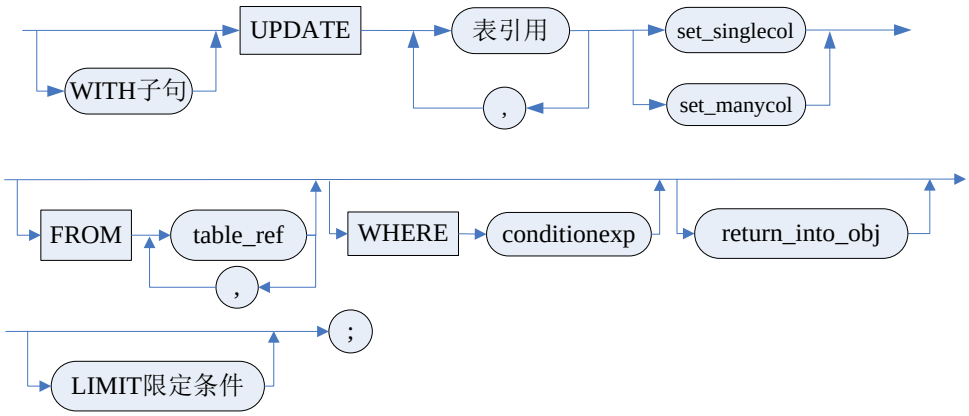
OFFSET N LIMIT M：更新第 N 条记录之后的 M 条记录。

2）ROW_LIMIT 子句：指定更新数据时的偏移的行数。

<offset>：指定待更新数据的起始偏移。

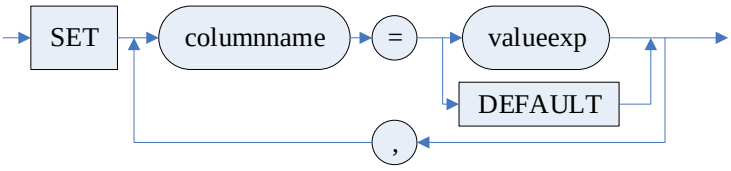
[<大小> | <大小> PERCENT] 指定要更新的行数或者所占的百分比

图例

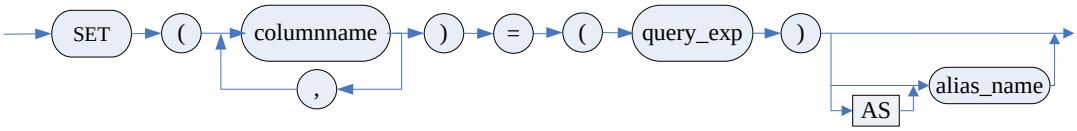


表引用：请参考第 4 章 数据查询语句

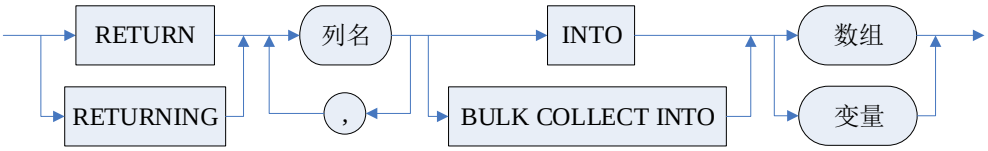
set_singlecol



set_manycol



return_into_obj



LIMIT 限定条件：请参考第 4 章 数据查询语句

使用说明

1．SET 后的<列名>不能重复出现；

2．WHERE 子句也可以包含子查询。如果省略了 WHERE 子句，则表示要修改所有的元组；

3．如果<列名>为被引用列，只有被引用列中未被引用列引用的数据才能被修改；如果<列名>为引用列，引用列的数据被修改后也必须满足引用完整性。在 DM 系统中，以上引用完整性由系统自动检查；

4. 执行基表的 UPDATE 语句触发任何与之相联系的 UPDATE 触发器；

5. 对于未指定 ENABLE ROW MOVEMENT 属性水平分区表的更新，如果更新后的值将导致记录所属分区发生修改，则不能进行更新；

6．如果更新对象是视图，同时在这个视图上建立了 INSTEAD OF 触发器，则会将更新操作转换为触发器所定义的操作；如果没有触发器，则需要判断这个视图是否可更新，如果不可更新则报错（不可更新视图场景参见 6.6 视图数据的更新），否则可以继续更新，如果上面的条件都满足，则可以更新成功；

7. RETURN INTO 支持返回变量和方法参数；

8. RETURN INTO 语句中的返回列支持 ROWID 列和函数调用，不支持 ROWNUM、TRXID 和 PHYROWID 列；

9. RETURN INTO 语句中返回列如果是更新列，则返回值为列的新值。返回结果对象支持变量和数组。如果返回变量，则变量类型与个数必须与返回列一致；如果返回数组，则数

组维度以及各维度的数据类型必须与返回列一致;

10. UPDATE 语句支持一次进行多列修改，多列修改存在限制：针对每行待更新数据，子查询中不能存在多行匹配的结果；若 INI 参数 MULTI_UPD_OPT_FLAG 为 0 并且更新普通表（非派生表），则相关子查询中允许存在多行匹配结果，此时根据第一行匹配结果进行更新，对于非相关子查询，不能存在多行匹配的结果;

11. 如果更新为子查询，则存在以下限制:

1） 更新子查询对应的最终更新对象目前仅仅必须为基表;

2） 更新的子查询的查询结果必须保证所更新基表的唯一性特性, 类似于更新视图是否可更新概念;

3） 更新子查询中不支持包含集合操作、DISTINCT 操作、集函数操作、GROUP BY、CONNECT BY、LIMIT、TOP、ROWNUM。

12. 增删改语句当前修改表称为变异表（MUTATE TABLE），其调用函数中，不能对此变异表进行删除操作;

13. 半透明加密列支持通过 UPDATE 语句进行修改，具体介绍请参考手册《DM8 安全管理》;

14. 多表联合更新说明:

1） 更新列表中有多个表时，不支持使用多列修改子句;

2） 更新列表中有多个表时，不允许指定 FROM 项;

3） 若有多个 SET 项，则各个 SET 项的左表达式必须为同一个表对象（同一个表的不同别名认为是不同对象）的列;

4） 要求用户对于更新列表的所有对象具有查询权限, 对最终修改的目标对象具有修改权限;

5） 多表联合更新最多支持 100 个表。

举例说明

例 1 将出版社为中华书局的图书的现在销售价格增加 1 元。

```
UPDATE  PRODUCTION.PRODUCT  SET  NOWPRICE = NOWPRICE - 2.0000
WHERE  PUBLISHER = '中华书局';
```

例 2 由于标量子查询允许用在标量值合法的地方，因此在数据修改语句的<值表达式>位置也允许出现标量子查询。下例将折扣高于 7.0 且出版社不是中华书局的图书的折扣设成出版社为中华书局的图书的平均折扣。

```
UPDATE  PRODUCTION.PRODUCT  SET  DISCOUNT =
( SELECT  AVG(DISCOUNT)
    FROM  PRODUCTION.PRODUCT
WHERE PUBLISHER = '中华书局')
WHERE  DISCOUNT > 7.0  AND  PUBLISHER != '中华书局';
```

注：自增列的修改例外，它一经插入，只要该列存储于数据库中，其值为该列的标识，不允许修改。关于自增列修改的具体情况，请参见 5.6 节——自增列的使用。

例 3 带 RETURN INTO 的更新语句。

```
CREATE TABLE T1(C1 INT,C2 INT,C3 INT);
DECLARE
  TYPE RRR IS RECORD(X INT, Y INT);
  TYPE CCC IS ARRAY RRR[];
  A INT;
  C CCC;
BEGIN
  C = NEW RRR[2];
  UPDATE T1 SET C2=4 WHERE C3 = 2 RETURN C1 INTO A;
  PRINT A;
```

```
  UPDATE T1 SET C2=5 WHERE C3 = 2 RETURN C1,C2 INTO C;
  SELECT * FROM ARRAY C;
END;
```

例 4 使用一次进行多列修改的更新语句。

```
UPDATE PURCHASING.PURCHASEORDER_HEADER SET(TAX,FREIGHT)=(SELECT ORIGINALPRICE,
NOWPRICE FROM PRODUCTION.PRODUCT WHERE NAME='长征');
```

5.3 数据删除语句

数据删除语句用于删除表中已存在的数据，支持单表删除和多表联合删除。

语法格式

```
[<WITH 子句>] DELETE {<TOP 删除子句> | <LIMIT 删除子句>};
<TOP 删除子句>::= <TOP 单表删除子句> | <TOP 多表联合删除子句>
<LIMIT 删除子句>::=<LIMIT 单表删除子句> | <LIMIT 多表联合删除子句>
<TOP 单表删除子句>::= <TOP 子句> [FROM] <表引用> [WHERE <条件表达式>][RETURN <列名>{,<列
名>} INTO <结果对象>,{<结果对象>}]
<TOP 多表联合删除子句>::= <TOP 子句> [FROM] <单表引用> <多表联合删除条件> [WHERE <条件表达
式>][RETURN <列名>{,<列名>} INTO <结果对象>,{<结果对象>}]
<LIMIT 单表删除子句>::= [FROM] <表引用> [WHERE <条件表达式>][RETURN <列名>{,<列名>} INTO <
结果对象>,{<结果对象>}] [<LIMIT 限定条件>]
<LIMIT 多表联合删除子句>::= [FROM] <单表引用> <多表联合删除条件> [WHERE <条件表达
式>][RETURN <列名>{,<列名>} INTO <结果对象>,{<结果对象>}][<LIMIT 限定条件>]

<WITH 子句>::=请参考 4.4 WITH 子句
<TOP 子句>::=请参考 4.9 TOP 子句
```

<表引用>::=

[<模式名>.] {<基表或视图名> | <子查询表达式>} |

<分区表>

<基表或视图名>::= <基表名> | <视图名>

<子查询表达式>::=(<查询表达式>) [[AS] <表别名> [<新生列>]]

<分区表>::=请参考 4.21 水平分区表查询

<结果对象>::=<数组>|<变量>

<单表引用>::=

[<模式名>.] {<表别名> | <基表或视图名>} |

<分区表>

<多表联合删除条件>::=

FROM <删除列表> |

USING <删除列表>

<删除列表>::= <多表联合删除表引用>{,<多表联合删除表引用>}

<多表联合删除表引用>::= <普通表>|<连接表>

<普通表>、<连接表>::=请参考第 4 章 数据查询语句

<LIMIT 限定条件>::= <LIMIT 子句> | <ROW_LIMIT 子句>

<LIMIT 子句>::= LIMIT <记录数> |

LIMIT <偏移量> , <记录数> |

LIMIT <记录数> OFFSET <偏移量> |

OFFSET <偏移量> LIMIT <记录数>

<记录数>::=<整数>

<偏移量>::=<整数>

<ROW_LIMIT 子句>::= [OFFSET <offset> ROW[S]] [<FETCH 说明>]

```
<FETCH 说明>::= FETCH <FIRST | NEXT> [<大小> | <大小> PERCENT] ROW[S] <ONLY | WITH TIES>
```

参数

1．<模式名> 指明该表或视图所属的模式，缺省为当前模式；

2．<基表名> 指明被删除数据的基表的名称；

3．<视图名> 指明被删除数据的视图的名称，实际上 DM 将从视图的基表中删除数据；

4．<条件表达式> 指明基表或视图的基表中被删除的记录须满足的条件；

5．<LIMIT 限定条件> 指明删除数据时的限定条件，与查询语句中的<LIMIT 限定条件>功能类似，具体说明可以查看 4.10 LIMIT 限定条件。

1）LIMIT 子句：按顺序删除表中某条记录开始的 N 条记录。

LIMIT N：删除前 N 条记录；

LIMIT M , N：删除第 M 条记录之后的 N 条记录；

LIMIT M OFFSET N：删除第 N 条记录之后的 M 条记录；

OFFSET N LIMIT M：删除第 N 条记录之后的 M 条记录。

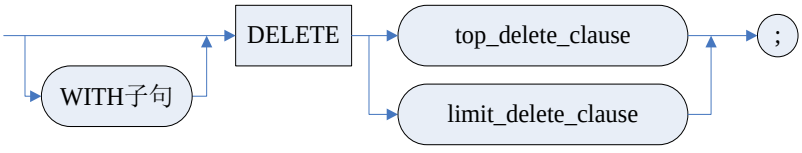
2）ROW_LIMIT 子句：指定删除数据时的偏移的行数。

<offset>：指定待删除数据的起始偏移。

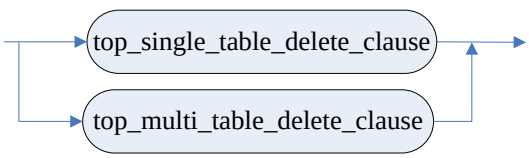
[<大小> | <大小> PERCENT] 指定要删除的行数或者所占的百分比。

图例

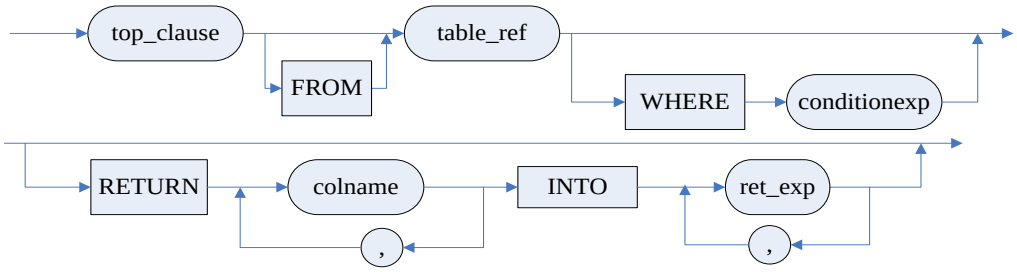
数据删除语句



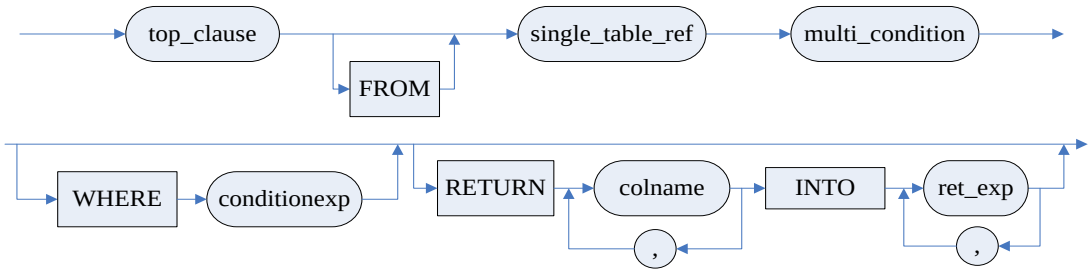
TOP 删除子句（top_delete_clause）



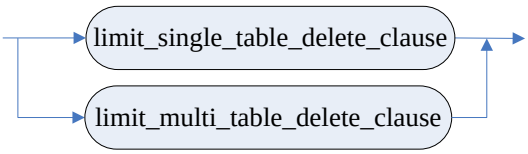
TOP 单表删除子句（top_single_table_delete_clause）



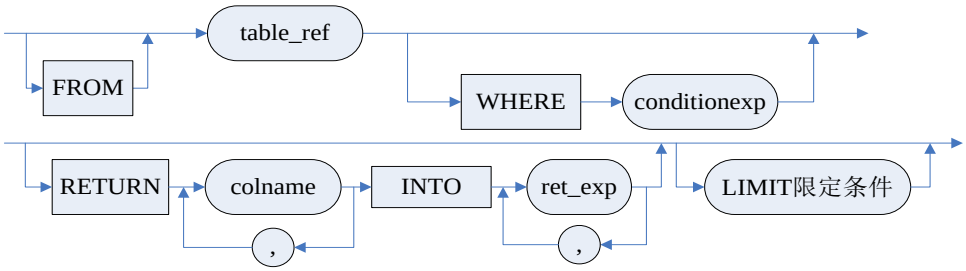
TOP 多表联合删除子句（top_multi_table_delete_clause）



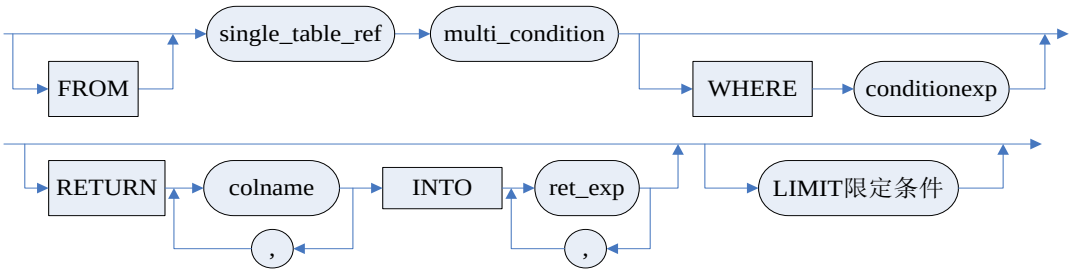
LIMIT 删除子句（limit_delete_clause）



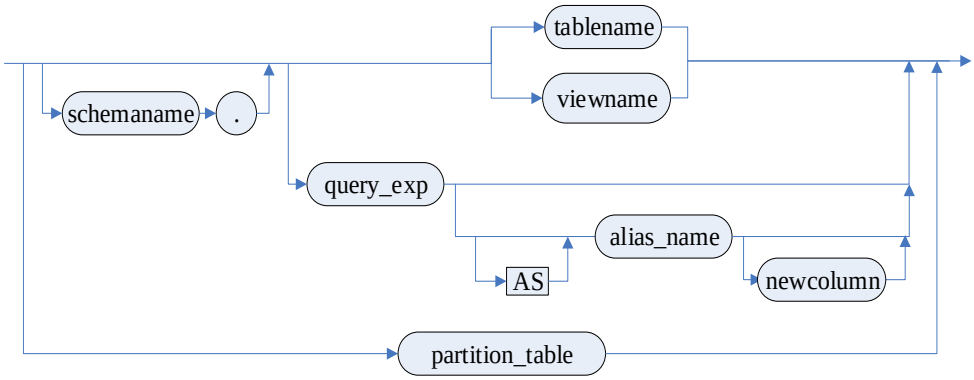
LIMIT 单表删除子句（limit_single_table_delete_clause）



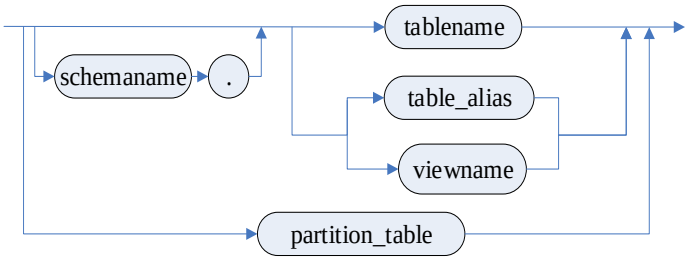
LIMIT 多表联合删除子句（limit_multi_table_delete_clause）



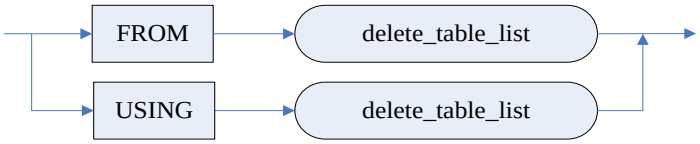
表引用（table_ref）



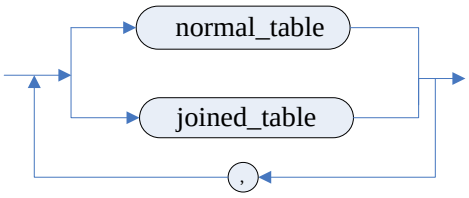
单表引用（single_table_ref）



多表联合删除条件（multi_condition）



删除列表（delete_table_list）



LIMIT 限定条件：请参考第 4 章 数据查询语句

使用说明

1. 如果不带 WHERE 子句，表示删除表中全部元组，但表的定义仍在字典中。因此，DELETE 语句删除的是表中的数据，并未删除表结构；

2. 由于 DELETE 语句一次只能对一个表进行删除，因此当两个表存在引用与被引用关系时，要先删除引用表里的记录，只有引用表中无记录时，才能删被引用表中的记录，否则系统会报错；

3. 执行与表相关的 DELETE 语句将触发所有定义在表上的 DELETE 触发器;

4. 如果视图为不可更新视图（不可更新视图场景参见 6.6 视图数据的更新），则不能从视图中删除记录;

5. 当<子查询表达式>作为 DELETE 的目标时，实际上是对查询表达式的基表进行操作，对于表子查询，查询表达式的查询项既可以来源于多个基表也可以是计算列，当查询项来自多个基表时，第一个满足条件的键值保存表即是最终删除数据的表；对于派生表子查询，查询表达式的查询项必须都来源于同一个基表且不能是计算列，查询项所属的基表即是查询表达式的基表。如果查询表达式是带有连接的查询，那么对于连接中视图基表以外的表，连接列上必须是主键或者带有 UNIQUE 约束。不支持 PIVOT/UNPIVOT，不支持 UNION/UNION ALL 查询;

6. RETURN INTO 支持返回变量和方法参数;

7. RETURN INTO 语句中的返回列支持 ROWID 列和函数调用，不支持 ROWNUM、TRXID 和 PHYROWID 列;

8. RETURN INTO 返回结果对象支持变量和数组。如果返回变量，则变量类型与个数必须与返回列一致；如果返回数组，则数组维度以及各维度的数据类型必须与返回列一致;

9. 增删改语句当前修改表称为变异表（MUTATE TABLE），其调用函数中，不能对此变异表进行删除操作;

10. 对于多表联合删除有以下限制:

1) 多表联合删除指定的删除对象，仅支持基表名、视图表、表别名，不支持子查询、连接表等，也不支持远程表;

2) 删除列表中应包含指定的删除对象，如果在删除列表中声明了别名，指定删除对象时也应该使用别名;

3) 多表联合删除时要求对删除列表中的所有对象具有查询权限，对最终删除的目标对象具有删除权限;

4） 删除列表中的表的数量存在上限，不能超过语义分析时 SELECT 语句中所支持的最大表个数 100；

5） 暂不支持使用游标进行多表联合删除。

例 将没有分配部门的员工的住址信息删除。

```
DELETE FROM RESOURCES.EMPLOYEE_ADDRESS
WHERE EMPLOYEEID IN
( SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE
WHERE EMPLOYEEID NOT IN
( SELECT EMPLOYEEID FROM RESOURCES.EMPLOYEE_DEPARTMENT));
```

5.4 MERGE INTO 语句

使用 MERGE INTO 语法可合并 UPDATE 和 INSERT 语句。通过 MERGE 语句，根据一张表（或视图）的连接条件对另外一张表（或视图）进行查询，连接条件匹配上的进行 UPDATE（可能含有 DELETE），无法匹配的执行 INSERT。其中，数据表包括：普通表、分区表、加密表、压缩表和堆表。

语法格式

```
MERGE INTO <merge_into_obj> [<表别名>] USING <表引用> ON [(] <条件判断表达式> [)]
<[<merge_update_clause>] [<merge_insert_clause>]>
<merge_into_obj> ::= <单表引用> | <子查询>
<单表引用> ::= [<模式名>.]<基表或视图名>
<子查询> ::= (<查询表达式>)
<merge_update_clause>::=WHEN MATCHED THEN UPDATE SET <set_value_list>
<where_clause_null> [DELETE <where_clause_null>]
<merge_insert_clause>::=WHEN NOT MATCHED THEN INSERT [<full_column_list>] VALUES
```

```
<ins_value_list> <where_clause_null>;

<表引用>::=<普通表> | <连接表>  请参考第 4 章 数据查询语句

<set_value_list> ::= <列名>=<值表达式| DEFAULT> {,<列名>=<值表达式| DEFAULT>}

<where_clause_null> ::= [WHERE <条件表达式>]

<full_column_list>::= (<列名>{,<列名>})

<ins_value_list>::= (<插入值>{,<插入值>})
```

参数

1．<模式名> 指明该表或视图所属的模式，缺省为当前用户的缺省模式；

2．<基表名> 指明被修改数据的基表的名称；

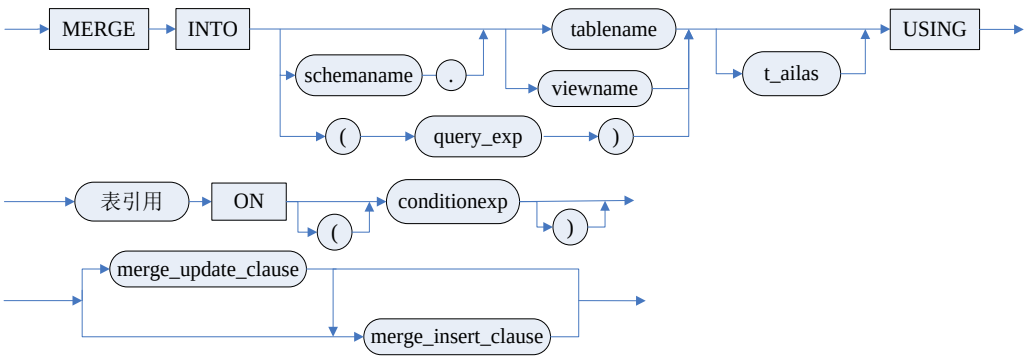
3．<视图名> 指明被修改数据的视图的名称，实际上 DM 对视图的基表更新数据；

4．<查询表达式> 指明被修改数据的子查询表达式，DM 实际是对子查询的基表进行数据更新。不支持集合运算、分组、集函数、HAVING 子句、层次查询、PIVOT/UNPIVOT 子句；不支持不可更新视图、外部表、物化视图辅助表、自定义的统计信息表、位图连接索引辅助表、系统表；不支持对计算列、集函数进行修改或插入；

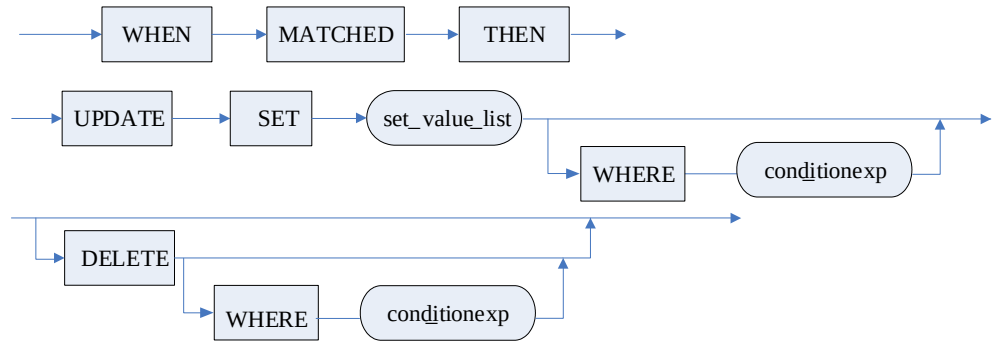
5．<条件表达式> 指明限制被操作执行的行必须符合指定的条件，如果省略此子句，则对表或视图中所有的行进行操作。

图例

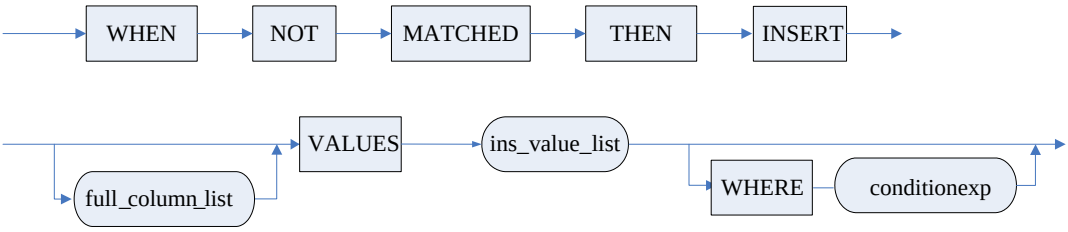
MERGE INTO 语句



merge_update_clause



merge_insert_clause



使用说明

1. INTO 后为目标表，表示待更新、插入的表、可更新视图及可更新查询表达式；

2. USING 后为源表（普通表或可更新视图），表示用于和目标表匹配、更新或插入的数据源；

3. ON（<条件判断表达式>)表示目标表和源表的连接条件，如果目标表有匹配连接条件的记录则执行更新该记录，如果没有匹配到则执行插入源表数据；

4. MERGE_UPDATE_CLAUSE：当目标表和源表的 JOIN 条件为 TRUE 时，执行该语句；

   1) 如果更新执行，更新语句会触发所有目标表上的 UPDATE 触发器，也会进行约束检查；

   2) 可以指定更新条件，如果不符合条件就不会执行更新操作。更新条件既可以和源表相关，也可以和目标表相关，或者都相关；

   3) DELETE 子句只删除目标表和源表的 JOIN 条件为 TRUE，并且是更新后的符合删除条件的记录，DELETE 子句不影响 INSERT 项插入的行。删除条件作用在更新后的记录上，既可以和源表相关，也可以和目标表相关，或者都相关。如果

JOIN 条件为 TRUE，但是不符合更新条件，并没有更新数据，那么 DELETE 将不会删除任何数据。当执行了删除操作，会触发目标表上的 DELETE 触发器，也会进行约束检查。

5. MERGE_INSERT_CLAUSE：当目标表和源表的 JOIN 条件为 FALSE 时，执行该语句。同时会触发目标表上的 INSERT 触发器，也会进行约束检查。可指定插入条件，插入条件只能在源表上设置；

6. MERGE_UPDATE_CLAUSE 和 MERGE_INSERT_CLAUSE 既可以同时指定，也可以只出现其中任何一个；

7. 需要有对源表的 SELECT 权限，对目标表的 UPDATE/INSERT 权限，如果 UPDATE 子句有 DELETE，还需要有 DELETE 权限；

8. UPDATE 子句不能更新在 ON 连接条件中出现的列；

9. 如果匹配到，源表中的匹配行必须唯一，否则报错；

10. <ins_value_list >不能包含目标表列；

11. 插入的 WHERE 条件只能包含源表列；

12. MERGE INTO 操作符方式允许在兼容 Oracle 模式下，且 MERGE_OPT_FLAG 不包含取值 8 时，将 MERGE INTO 语句中不加前缀的列名解析为目标表的列名。

举例说明

例 1 下面的例子把 T1 表中 C1 值为 2 的记录行中的 C2 列，更新为表 T2 中 C3 值为 2 的记录中 C4 列的值，同时把 T2 中 C3 列为 4 的记录行插入到 T1 中。

```
DROP TABLE T1;
DROP TABLE T2;
CREATE TABLE T1 (C1 INT, C2 VARCHAR(20));
CREATE TABLE T2 (C3 INT, C4 VARCHAR(20));
INSERT INTO T1 VALUES(1,'T1_1');
```

```
INSERT INTO T1 VALUES(2,'T1_2');
INSERT INTO T1 VALUES(3,'T1_3');
INSERT INTO T2 VALUES(2,'T2_2');
INSERT INTO T2 VALUES(4,'T2_4');
COMMIT;

MERGE INTO T1 USING T2 ON (T1.C1=T2.C3)
WHEN MATCHED THEN UPDATE SET T1.C2=T2.C4
WHEN NOT MATCHED THEN INSERT (C1,C2) VALUES(T2.C3, T2.C4);
```

例 2 下面的例子把 T1 表中 C1 值为 2, 4 的记录行中的 C2 列更新为表 T2 中 C3 值为 2, 4 的记录中 C4 列的值，同时把 T2 中 C3 列为 5 的记录行插入到了 T1 中。由于 UPDATE 带了 DELETE 子句，且 T1 中 C1 列值为 2 和 4 的记录行被更新过，而 C1 为 4 的行符合删除条件，最终该行会被删除掉。

```
DROP TABLE T1;
DROP TABLE T2;
CREATE TABLE T1 (C1 INT,  C2 VARCHAR(20));
CREATE TABLE T2 (C3 INT,  C4 VARCHAR(20));
INSERT INTO T1 VALUES(1,'T1_1');
INSERT INTO T1 VALUES(2,'T1_2');
INSERT INTO T1 VALUES(3,'T1_3');
INSERT INTO T1 VALUES(4,'T1_4');
INSERT INTO T2 VALUES(2,'T2_2');
INSERT INTO T2 VALUES(4,'T2_4');
INSERT INTO T2 VALUES(5,'T2_5');
```

```
COMMIT;



MERGE INTO T1 USING T2 ON (T1.C1=T2.C3)
WHEN MATCHED THEN UPDATE SET T1.C2=T2.C4 WHERE T1.C1 >= 2 DELETE WHERE T1.C1=4
WHEN NOT MATCHED THEN INSERT (C1,C2) VALUES(T2.C3, T2.C4);
```

5.5 伪列的使用

伪列从语法上和表中的列很相似，查询时能够返回一个值，但实际上在表中并不存在。用户可以对伪列进行查询，但不能插入、更新和删除它们的值。除了 4.13.2 层次查询相关伪列和 4.15 ROWNUM 中介绍的伪列外，DM 中还提供包括 ROWID、USER、TRXID 等伪列。

5.5.1 ROWID

DM 中行标识符 ROWID 用来标识数据库基表中每一条记录的唯一键值，标识了数据记录的确切的存储位置。ROWID 由 18 位字符组成，分别为“4 位站点号+6 位分区号+8 位物理行号”。如果是单机则 4 位站点号为 AAAA，即 0。如果是非分区表，则 6 位分区号为 AAAAAA，即 0。关于 ROWID 中站点号、分区号和物理行号的使用和转换请参考附录 3 中的 21.ROWID。

如果用户在选择数据的同时从基表中选取 ROWID，在后续的更新语句中，就可以使用 ROWID 来提高性能。如果在查询时加上 FOR UPDATE 语句，该数据行就会被锁住，以防其他用户修改数据，保证查询和更新之间的一致性。

例 查询和使用 ROWID。

```
SELECT ROWID, VENDORID, NAME, CREDIT FROM PURCHASING.VENDOR WHERE NAME = '广州出版社';
```

查询结果如下：

```
ROWID                    VENDORID      NAME          CREDIT

------------------ ----------- ---------- -----------

AAAAAAAAAAAAAAAAAH 7                广州出版社 1
```

通过指定 ROWID 来确定待执行的行。

```
UPDATE  PURCHASING.VENDOR  SET  CREDIT=2 WHERE  ROWID ='AAAAAAAAAAAAAAAAAH';
```

5.5.2 USER

伪列 USER 用来表示当前用户的用户名。

5.5.3 TRXID

伪列 TRXID 用来表示当前记录最后被修改的事务的事务标识。

5.5.4 PHYROWID

伪列 PHYROWID 用来表示当前记录的物理存储信息。

PHYROWID 值由聚集 B 树或二级 B 树中物理记录的文件号、页号、页内槽号组成，能体现聚集 B 树或二级 B 树的存储信息，聚集 B 树记录的最高位为 1。

当查询语句中实际使用 CSCN、CSEK、BLKUP 操作符时，PHYROWID 内容是聚集 B 树中记录的物理存储地址；当查询语句中实际仅使用 SSEK、SSCN 操作符时，PHYROWID 内容是二级 B 树中记录的物理存储地址。

5.6 DM 自增列的使用

DM 提供两种自增方式：IDENTITY 自增列和 AUTO_INCREMENT 自增列。本节专门介绍 IDENTITY 自增列用法。

5.6.1 DM 自增列定义

1．自增列功能定义

在表中创建一个自增列。该属性与 CREATE TABLE 语句一起使用，一个表只能有一个自增列。

语法格式

```
IDENTITY [（种子，增量）]
```

参数

1．种子 装载到表中的第一个行所使用的值；

2．增量 增量值，该值被添加到前一个已装载的行的标识值上。增量值可以为正数或负数，但不能为 0。

使用说明

1．IDENTITY 适用于 INT(-2147483648 ~ +2147483647)、BIGINT(-2^{63} ~ $+2^{63}-2$) 、DEC(n, 0)类型的列。每个表只能创建一个自增列；

2．不能对自增列使用 DEFAULT 约束；

3．必须同时指定种子和增量值，或者二者都不指定。如果二者都未指定，则取默认值(1,1)。若种子或增量为小数类型，报错；

4．最大值和最小值为该列的数据类型的边界；

5．建表种子和增量大于最大值或者种子和增量小于最小值时报错；

6．自增列一旦生成，无法更新，不允许用 UPDATE 语句进行修改；

7. 自增列的值一旦生成，无法回滚。例如，数据表 T 中包含一个自增列，该列当前值为 n，增量为 1，对表 T 执行数据插入时，如果经约束检查发现待插入数据不满足约束条件，则会回滚数据插入操作，但此时自增列的值 n+1 已经生成，无法回滚，因此自增列的当前值变为 n+1；

8. 临时表、列存储表不支持使用自增列。

2．自增列查询函数

1） IDENT_SEED(函数)

语法格式

```
IDENT_SEED ('tablename')
```

功能：返回种子值，该值是在带有自增列的表中创建自增列时指定的。

参数：tablename 是带有引号的字符串常量，也可以是变量、函数或列名。tablename 的数据类型为 CHAR 或 VARCHAR。其含义是表名，可带模式名前缀。

返回类型：返回数据类型为 INT/NULL

2） IDENT_INCR(函数)

语法格式

```
IDENT_INCR ('tablename')
```

功能：返回增量值，该值是在带有自增列的表中创建自增列时指定的。

参数：tablename 是带有引号的字符串常量，也可以是变量、函数或列名。tablename 的数据类型为 CHAR 或 VARCHAR。其含义是表名，可带模式名前缀。

返回类型：返回数据类型为 INT/NULL

例 用自增列查询函数获得表 PERSON_TYPE 的自增列的种子和增量信息。

```
SELECT  IDENT_SEED('PERSON.PERSON_TYPE');
```

查询结果为：1

```
SELECT IDENT_INCR('PERSON.PERSON_TYPE');
```

查询结果为：1

5.6.2 SET IDENTITY_INSERT 属性

设置是否允许将显式值插入表的自增列中。ON 是，OFF 否。

语法格式

```
SET IDENTITY_INSERT [<模式名>.]<表名> ON [WITH REPLACE NULL];

SET IDENTITY_INSERT [<模式名>.]<表名> OFF;
```

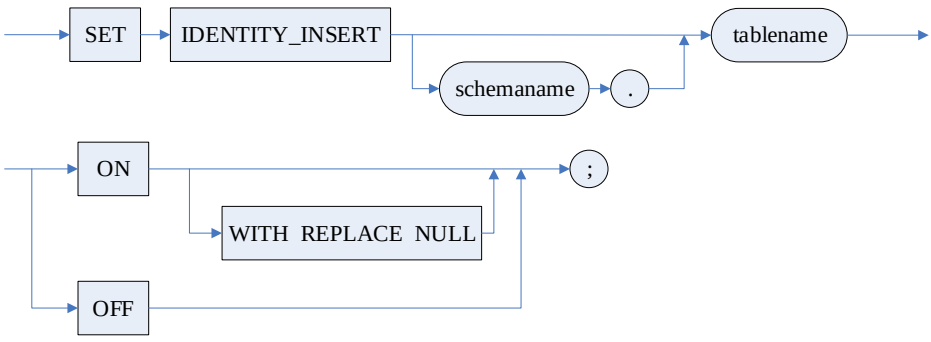
参数

1．<模式名> 指明表所属的模式，缺省为当前模式；

2．<表名> 指明含有自增列的表名。

图例

SET IDENTITY_INSERT



使用说明

1．IDENTITY_INSERT 属性的默认值为 OFF。SET IDENTITY_INSERT 的设置是在执行或运行时进行的。当一个连接结束，IDENTITY_INSERT 属性将被自动还原为 OFF；

2．DM 要求一个会话连接中只有一个表的 IDENTITY_INSERT 属性可以设置为 ON，当设置一个新的表 IDENTITY_INSERT 属性设置为 ON 时，之前已经设置为 ON 的表会自动还原为 OFF。当一个表的 IDENTITY_INSERT 属性被设置为 ON 时，该表中的自动增量列的值由用户指定。如果插入值大于表的当前标识值(自增列当前值)，则 DM 自动将新插入值作为当前标识值使用，即改变该表的自增列当前值；否则，将不影响该自增列当前值；

3．当设置一个表的 IDENTITY_INSERT 属性为 OFF 时，新插入行中自增列的当前值由系统自动生成，用户将无法指定；

4．自增列一经插入，无法修改；

5．手动插入自增列，除了将 IDENTITY_INSERT 设置为 ON，还要求在插入列表中明确指定待插入的自增列列名。插入方式与非 IDENTITY 表是完全一样的。如果插入时，既不指定自增列名也不给自增列赋值，则新插入行中自增列的当前值由系统自动生成；

6. WITH REPLACE NULL 此模式下允许显式插入 NULL 值，同时，系统自动将 NULL 值替换为自增值。

举例说明

例 SET IDENTITY_INSERT 的使用。

1） PERSON_TYPE 表中的 PERSON_TYPEID 列是自增列，目前拥有的数据如表 5.6.1 所示。

表 5.6.1

PERSON_TYPEID	NAME
1	采购经理
2	采购代表
3	销售经理
4	销售代表

2） 在该表中插入数据，自增列的值由系统自动生成。

```
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('销售总监');
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('人力资源部经理');
```

插入结果如表 5.6.2 所示：

表 5.6.2

PERSON_TYPEID	NAME
1	采购经理
2	采购代表
3	销售经理
4	销售代表
5	销售总监
6	人力资源部经理

3） 当插入数据并且要指定自增列的值时，必须要通过语句将 IDENTITY_INSERT 设置为 ON 时，插入语句中必须指定 PERSON_TYPEID 中要插入的列。例如：

```
SET IDENTITY_INSERT PERSON.PERSON_TYPE ON;
INSERT INTO PERSON.PERSON_TYPE(PERSON_TYPEID, NAME) VALUES( 8, '广告部经理');
INSERT INTO PERSON.PERSON_TYPE(PERSON_TYPEID, NAME) VALUES( 9, '财务部经理');
```

插入结果如表 5.6.3 所示：

表 5.6.3

PERSON_TYPEID	NAME
1	采购经理
2	采购代表
3	销售经理
4	销售代表
5	销售总监
6	人力资源部经理
8	广告部经理

9	财务部经理

4） 不允许用户修改自增列的值。

```
UPDATE  PERSON.PERSON_TYPE SET PERSON_TYPEID = 9 WHERE NAME = '广告部经理';
```

修改失败。对于自增列，不允许 UPDATE 操作。

5） 还原 IDENTITY_INSERT 属性。

```
SET IDENTITY_INSERT PERSON.PERSON_TYPE OFF;
```

6） 插入后再次查询。注意观察自增列当前值的变化。

```
INSERT  INTO  PERSON.PERSON_TYPE(NAME)  VALUES('市场总监');
```

表 5.6.4

PERSON_TYPEID	NAME
1	采购经理
2	采购代表
3	销售经理
4	销售代表
5	销售总监
6	人力资源部经理
8	广告部经理
9	财务部经理
10	市场总监

7）使用 WITH REPLACE NULL 模式，显式插入 NULL 值。同时，系统自动将 NULL 值替换为自增值。

```
SET IDENTITY_INSERT PERSON.PERSON_TYPE ON WITH REPLACE NULL;

INSERT INTO PERSON.PERSON_TYPE(PERSON_TYPEID, NAME) VALUES(NULL, '总经理');
```

表 5.6.5

PERSON_TYPEID	NAME
1	采购经理
2	采购代表
3	销售经理
4	销售代表
5	销售总监
6	人力资源部经理
8	广告部经理
9	财务部经理
10	市场总监
11	总经理

第 6 章 视图

视图是从一个或几个基表(或视图)导出的表，它是一个虚表，即数据字典中只存放视图的定义(由视图名和查询语句组成)，而不存放对应的数据，这些数据仍存放在原来的基表中。当需要使用视图时，则执行其对应的查询语句，所导出的结果即为视图的数据。因此当基表中的数据发生变化时，从视图中查询出的数据也随之改变了，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据和变化。由此可见，视图是关系数据库系统提供给用户以多种角度观察数据库中数据的重要机制，体现了数据库本身最重要的特色和功能，它简化了用户数据模型，提供了逻辑数据独立性，实现了数据共享和数据的安全保密。视图是数据库技术中一个十分重要的功能。

视图一经定义，就可以和基表一样被查询、修改和删除，也可以在视图之上再建新视图。由于对视图数据的更新均要落实到基表上，因而操作起来有一些限制，读者应注意如何才能在视图中正确更新数据。在本章各例中，如不特别说明，以下例子均使用 BOOKSHOP 示例库，用户均为建表者 SYSDBA。

6.1 视图的作用

视图是提供给用户以多种角度观察数据库中数据的重要机制。尽管在对视图作查询和更新时有各种限制，但只要用户对 DM_SQL 语言熟悉，合理使用视图对用户建立自己的管理信息系统会带来很多的好处和方便，归纳起来，主要有以下几点：

1．用户能通过不同的视图以多种角度观察同一数据

可针对不同需要建立相应视图，使他们从不同的需要来观察同一数据库中的数据。

2．简化了用户操作

由于视图是从用户的实际需要中抽取出来的虚表，因而从用户角度来观察这种数据库结

构必然简单清晰。另外，由于复杂的条件查询已在视图定义中一次给定，用户再对该视图查询时也简单方便得多了。

3．为需要隐蔽的数据提供了自动安全保护

所谓“隐蔽的数据”是指通过某视图不可见的数据。由于对不同用户可定义不同的视图，使需要隐蔽的数据不出现在不应该看到这些数据的用户视图上，从而由视图机制自动提供了对机密数据的安全保密功能。

4．为重构数据库提供了一定程度的逻辑独立性

在建立调试和维护管理信息系统的过程中，由于用户需求的变化、信息量的增长等原因，经常会出现数据库的结构发生变化，如增加新的基表，或在已建好的基表中增加新的列，或需要将一个基表分解成两个子表等，这称为数据库重构。数据的逻辑独立性是指当数据库重构时，对现有用户和用户程序不产生任何影响。

在管理信息系统运行过程中，重构数据库最典型的示例是将一个基表垂直分割成多个表。将经常要访问的列放在速度快的服务器上，而不经常访问的列放在较慢的服务器上。

例如将 PRODUCT 表分解成两个基表。

```
PRODUCT(PRODUCTID,NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCT_CATEGORYID,PRODUCTNO,DESCRIPTION,PHOTO,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,TYPE,PAPERTOTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME),
```

分解为两个基表：

```
PRODUCT_1(PRODUCTID,NAME,AUTHOR,PUBLISHER,NOWPRICE)
PRODUCT_2(PRODUCTID,PUBLISHERTIME,PRODUCT_CATEGORYID,PRODUCTNO,DESCRIPTION,PHOTO,SATETYSTOCKLEVEL,ORIGINALPRICE,NOWPRICE,DISCOUNT,TYPE,PAPERTOTAL,WORDTOTAL,SELLSTARTTIME,SELLENDTIME)
```

并将 PRODUCT 表中的数据分别插入这两个新建表中，再删去 PRODUCT 表。这样一来，原有用户程序中有 PRODUCT 表的操作就均无法进行了。为了减少对用户程序影响，

这时可在 PRODUCT_1 和 PRODUCT_2 两基表上建立一个名字为 PRODUCT 的视图，因为新建视图维护了用户外模式的原状，用户的应用程序不用修改仍可通过视图查询到数据，从而较好支持了数据的逻辑独立性。

6.2 视图的定义

语法格式

```
CREATE [OR REPLACE] [FORCE] VIEW [IF NOT EXISTS]
[<模式名>.]<视图名>[(<列名> {,<列名>})]
AS <查询说明>
[WITH [LOCAL|CASCADED]CHECK OPTION]|[with read only];
<查询说明>::=<表查询> | <连接表>
<表查询>::=<子查询表达式>[ORDER BY 子句]
<连接表>::= 请参考第 4 章 数据查询语句
```

参数

1．<模式名> 指明被创建的视图属于哪个模式，缺省为当前模式；

2．<视图名> 指明被创建的视图的名称；

3．<列名> 指明被创建的视图中列的名称；

4．<子查询表达式> 标识视图所基于的表的行和列。其语法遵照 SELECT 语句的语法规则；

5．FORCE 选项用于强制创建视图。指定 FORCE 时，基表可以不存在，此时创建视图不报错，但是会报警告，并且创建的视图处于无效状态；

6 .WITH CHECK OPTION 此选项用于可更新视图中。指明往该视图中 insert 或 update 数据时，插入行或更新行的数据必须满足视图定义中<查询说明>所指定的条件。如果不带该选项，则插入行或更新行的数据不必满足视图定义中<查询说明>所指定的条件；

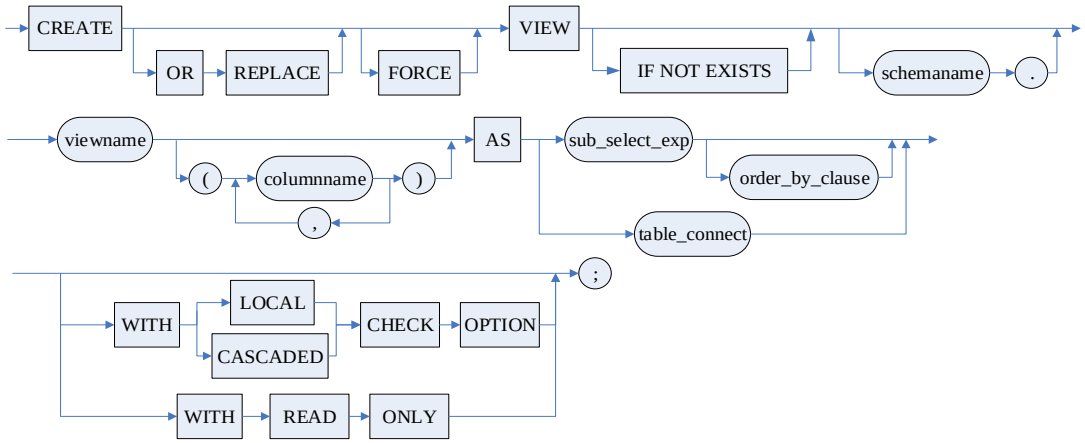
[LOCAL|CASCADED] 用于当前视图是根据另一个视图定义的情况。当通过视图向基表中 insert 或 update 数据时，LOCAL|CASCADED 决定了满足 CHECK 条件的范围。指定 LOCAL，要求数据必须满足当前视图定义中<查询说明>所指定的条件；指定 CASCADED，数据必须满足当前视图，以及所有相关视图定义中<查询说明>所指定的条件。

MPP 系统下不支持该 WITH CHECK OPTION 操作。

7．WITH READ ONLY 指明该视图是只读视图，只可以查询，但不可以做其他 DML 操作；如果不带该选项，则根据 DM 自身判断视图是否可更新的规则判断视图是否只读。

图例

视图的定义



语句功能

供 DBA 或具有 CREATE VIEW 权限的用户定义视图。

使用说明

1．当<视图名>已存在时，若指定 OR REPLACE，则使用新视图替换原同名视图；若指定 IF NOT EXISTS，则忽略本次视图创建操作；若同时指定 OR REPLACE 和 IF NOT EXISTS，则按照 OR REPLACE 的策略执行；若均不指定，则报错；

2．<视图名>后所带<列名>不得同名，个数必须与<查询说明>中 SELECT 后的<值表达式>的个数相等。如果<视图名>后不带<列名>，则隐含该视图中的列由<查询说明>中

SELECT 后的各<值表达式>组成，但这些<值表达式>必须是单纯列名。如果出现以下三种情况之一，<视图名>后的<列名>不能省：

1） <查询说明>中 SELECT 后的<值表达式>不是单纯的列名，而包含集函数或运算表达式；

2） <查询说明>包含了多表连接，使得 SELECT 后出现了几个不同表中的同名列作为视图列；

3） 需要在视图中为某列取与<查询说明>中 SELECT 后<列名>不同的名字。

最后要强调的是：<视图名>后的<列名>必须全部省略或全部写明。

3．为了防止用户通过视图更新基表数据时，无意或故意更新了不属于视图范围内的基表数据，在视图定义语句的子查询后提供了可选项 WITH CHECK OPTION。如选择，表示往该视图中插入或修改数据时，要保证插入行或更新行的数据满足视图定义中<查询说明>所指定的条件，不选则可不满足；

4．视图是一个逻辑表，它自己不包含任何数据；

5．视图上可以建立 INSTEAD OF 触发器（只允许元组级触发），但不允许创建 BEFORE/AFTER 触发器；

6．视图分为可更新视图和不可更新视图，具体规则参见 6.6 视图数据的更新。

权限

使用者必须对<查询说明>中的每个表均具有 SELECT 权限。

但是当 INI 参数 VIEW_ACCESS_MODE=1 兼容 Oracle 时，该语句的使用者必须对<查询说明>中的每个表均具有直接 SELECT 权限，而非通过角色间接拥有的 SELECT 权限。其中 PUBLIC 角色除外，但也仅限于直接授予 PUBLIC 的 SELECT 权限，而非间接授予。

举例说明

例 1 对 VENDOR 表创建一个视图，名为 VENDOR_EXCELLENT，保存信誉等级为 1 的供应商，列名有：VENDORID，ACCOUNTNO，NAME，ACTIVEFLAG。

```
CREATE  VIEW  PURCHASING.VENDOR_EXCELLENT  AS
SELECT  VENDORID, ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT
FROM  PURCHASING.VENDOR
WHERE  CREDIT = 1;
```

由于视图列名与查询说明中 SELECT 后的列名相同，所以视图名后的列名可省。

运行该语句，AS 后的查询语句并未执行，系统只是将所定义的<视图名>及<查询说明>送数据字典保存。对用户来说，就像在数据库中已经有 VENDOR_EXCELLENT 这样一个表。

如果对该视图作查询：

```
SELECT  *  FROM  PURCHASING.VENDOR_EXCELLENT;
```

查询结果如下：

```
VENDORID    ACCOUNTNO NAME                          ACTIVEFLAG CREDIT
---------- --------- -------------------- ---------- -----------
3           00        北京十月文艺出版社          1           1
4           00        人民邮电出版社              1           1
5           00        清华大学出版社              1           1
6           00        中华书局                    1           1
7           00        广州出版社                  1           1
8           00        上海出版社                  1           1
9           00        21 世纪出版社               1           1
10          00        外语教学与研究出版社        1           1
11          00        机械工业出版社              1           1
12          00        文学出版社                  1           1
```

用户可以在该表上作数据库的查询、插入、删除、修改等操作。在建好的视图之上还可以再建立视图。

由于以上定义包含可选项 WITH CHECK OPTION，以后对该视图作插入、修改和删除操作时，系统均会自动用 WHERE 后的条件作检查，不满足条件的数据，则不能通过该视图更新相应基表中的数据。

例 2 视图也可以建立在多个基表之上。构造一视图，名为 SALESPERSON_INFO，用来保存销售人员的信息，列名有：SALESPERSONID，TITLE，NAME，SALESLASTYEAR。

```
CREATE  VIEW  SALES.SALESPERSON_INFO  AS
SELECT  T1.SALESPERSONID,  T2.TITLE,  T3.NAME, T1.SALESLASTYEAR
FROM    SALES.SALESPERSON  T1,  RESOURCES.EMPLOYEE  T2,  PERSON.PERSON  T3
WHERE   T1.EMPLOYEEID = T2.EMPLOYEEID  AND  T2.PERSONID = T3.PERSONID;
```

如果对该视图作查询：

```
SELECT  *  FROM  SALES.SALESPERSON_INFO;
```

查询结果如下：

```
SALESPERSONID TITLE      NAME SALESLASTYEAR
------------- -------- ---- -------------
1                    销售代表  郭艳  10.0000
2                     销售代表  孙丽  20.0000
```

由前面的介绍可知，基表中的数据均是基本数据。为了减少数据冗余，由基本数据经各种计算统计出的数据一般是不存贮的，但这样的数据往往又要经常使用，这时可将它们定义成视图中的数据。

例 3 在 PRODUCT_VENDOR 上建立一视图，用于统计数量。

```
CREATE  VIEW  PRODUCTION.VENDOR_STATIS(VENDORID, PRODUCT_COUNT)  AS
SELECT  VENDORID, COUNT(PRODUCTID)
FROM    PRODUCTION.PRODUCT_VENDOR
GROUP  BY  VENDORID
```

```
ORDER  BY  VENDORID;
```

在该语句中，由于 SELECT 后出现了集函数 COUNT(PRODUCTID)，不属于单纯的列名，所以视图中的对应列必须重新命名，即在<视图名>后明确说明视图的各个列名。

由于该语句中使用了 GROUP BY 子句，所定义的视图也称分组视图。分组视图的<视图名>后所带<列名>不得包含集函数。

如果对该视图作查询：

```
SELECT  *  FROM  PRODUCTION.VENDOR_STATIS;
```

查询结果如下：

```
VENDORID      PRODUCT_COUNT
----------- --------------------
5                1
6                2
7                1
8                1
9                1
10               1
11               1
```

6.3 视图的删除

一个视图本质上是基于其他基表或视图上的查询，我们把这种对象间关系称为依赖。用户在创建视图成功后，系统还隐式地建立了相应对象间的依赖关系。在一般情况下，当一个视图不被其他对象依赖时可以随时删除视图。

语法格式

```
DROP VIEW [IF EXISTS] [<模式名>.]<视图名> [RESTRICT | CASCADE];
```

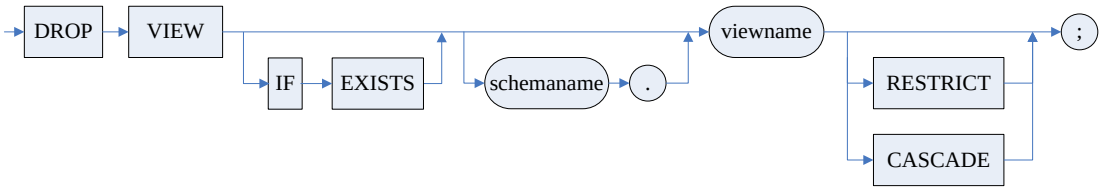
参数

1．<模式名> 指明被删除视图所属的模式，缺省为当前模式；

2．<视图名> 指明被删除视图的名称。

图例

视图的删除



使用说明

1. 删除不存在的视图会报错。若指定IF EXISTS关键字，删除不存在的视图，不会报错；

2. 视图删除有两种方式：RESTRICT/CASCADE 方式。其中 RESTRICT 为缺省值。当设置 DM.INI 中的参数 DROP_CASCADE_VIEW 值为 1 时，如果在该视图上建有其它视图，必须使用 CASCADE 参数才可以删除所有建立在该视图上的视图，否则删除视图的操作不会成功；当设置 DM.INI 中的参数 DROP_CASCADE_VIEW 值为 0 时，RESTRICT 和 CASCADE 方式都会成功，且只会删除当前视图，不会删除建立在该视图上的视图；

3. 如果没有删除参考视图的权限，那么两个视图都不会被删除；

4. 该视图删除后，用户在其上的权限也均自动取消，以后系统中再建的同名视图，是与他毫无关系的视图。

权限

使用者必须拥有 DBA 权限或是该视图的拥有者。

举例说明

例 1 删除视图 VENDOR_EXCELLENT，可使用下面的语句：

```
DROP  VIEW  PURCHASING.VENDOR_EXCELLENT;
```

当该视图对象被其他对象依赖时，用户在删除视图时必须带 CASCADE 参数，系统会

将依赖于该视图的其他数据库对象一并删除，以保证数据库的完整性。

例 2 删除视图 SALES.SALESPERSON_INFO，同时删除此视图上的其他视图，可使用下面的语句：

```
DROP  VIEW  SALES.SALESPERSON_INFO CASCADE;
```

6.4 视图的查询

视图一旦定义成功，对基表的所有查询操作都可用于视图。对于用户来说，视图和基表在进行查询操作时没有区别。

例 1 从 VENDOR_EXCELLENT 中查询 ACTIVEFLAG 为 1 的供应商的编号和名称。

```
SELECT  VENDORID, NAME
FROM  PURCHASING.VENDOR_EXCELLENT
WHERE  ACTIVEFLAG = 1;
```

系统执行该语句时，先从数据字典中取出视图 VENDOR_EXCELLENT 的定义，按定义语句查询基表，得到视图表，再根据条件:ACTIVEFLAG = 1 查询视图表，选择所需列名，查询结果如下：

```
VENDORID      NAME
----------- --------------------
3             北京十月文艺出版社
4             人民邮电出版社
5             清华大学出版社
6             中华书局
7             广州出版社
8             上海出版社
9             21 世纪出版社
```

```
10              外语教学与研究出版社

11              机械工业出版社

12              文学出版社
```

视图尽管是虚表，但它仍可与其它基表或视图作连接查询，也可以出现在子查询中。

例 2 查询信誉等级为 1 的供应商供应的图书编号、名称、通常价格和供应商名称。

```
SELECT  T1.PRODUCTID, T1.NAME, T2.STANDARDPRICE, T3.NAME

FROM  PRODUCTION.PRODUCT  T1,  PRODUCTION.PRODUCT_VENDOR  T2,

        PURCHASING.VENDOR_EXCELLENT  T3

WHERE  T1.PRODUCTID  =  T2.PRODUCTID  AND T2.VENDORID = T3.VENDORID;
```

系统执行该语句时，先从数据字典中取出视图 VENDOR_EXCELLENT 的定义，按定义语句查询基表，得到视图表，再将 PRODUCT、PRODUCT_VENDOR 和视图表按连接条件作连接，选择所需列名，得到最后结果。

查询结果如下:

```
PRODUCTID     NAME                        STANDARDPRICE NAME

----------- ---------------------- ------------- --------------------

7             数据结构(C 语言版)(附光盘)  25.0000       清华大学出版社

8             工作中无小事                25.0000       机械工业出版社

9             突破英文基础词汇            25.0000       外语教学与研究出版社

10            噼里啪啦丛书(全 7 册)       25.0000       21 世纪出版社

3             老人与海                    25.0000       上海出版社

4             射雕英雄传(全四册)          25.0000       广州出版社

1             红楼梦                      25.0000       中华书局

2             水浒传                      25.0000       中华书局
```

6.5 视图的编译

一个视图依赖于其基表或视图，如果基表定义发生改变，如修改或删除一列，或者视图的相关权限发生改变，可能导致视图无法使用。在这种情况下，可对视图重新编译，检查视图的合法性。

语法格式

```
ALTER VIEW [<模式名>.]<视图名> COMPILE;
```

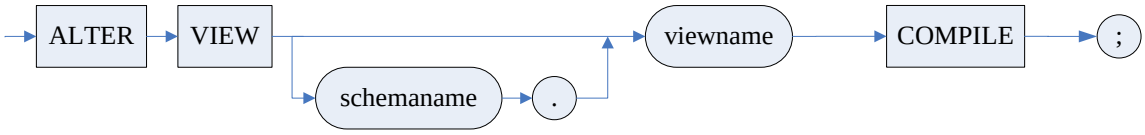
参数

1．<模式名> 指明被编译视图所属的模式，缺省为当前模式；

2．<视图名> 指明被编译视图的名称；

图例

视图的编译



使用说明

对视图的定义重新进行分析和编译，如果编译出错，则报错，可以此判断视图依赖的基表是否已被删除或修改了表定义。

权限

使用者必须拥有 DBA 权限或是该视图的拥有者。

举例说明

例 重新编译视图 PURCHASING.VENDOR_EXCELLENT。

```
ALTER VIEW PURCHASING.VENDOR_EXCELLENT COMPILE;
```

6.6 视图数据的更新

视图数据的更新包括插入(INSERT)、删除(DELETE)和修改(UPDATE)三类操作。由于视图是虚表，并没有实际存放数据，因此对视图的更新操作均要转换成对基表的操作。在 SQL 语言中，对视图数据的更新语句与对基表数据的更新语句在格式与功能方面是一致的。

例 1 从视图 VENDOR_EXCELLENT 中将名称为人民邮电出版社的 ACTIVEFLAG 改为 0。

```
UPDATE  PURCHASING.VENDOR_EXCELLENT
SET  ACTIVEFLAG = 0 WHERE NAME = '人民邮电出版社';
```

系统执行该语句时，首先从数据字典中取出视图 VENDOR_EXCELLENT 的定义，将其中的查询说明与对视图的修改语句结合起来，转换成对基表的修改语句，然后再执行这个转换后的更新语句。

```
UPDATE  PURCHASING.VENDOR
SET  ACTIVEFLAG = 0
WHERE  NAME = '人民邮电出版社'  AND  CREDIT = 1;
```

例 2 往视图 VENDOR_EXCELLENT 中插入一个新的记录，其中 ACCOUNTNO 为 00，NAME 为电子工业出版社，ACTIVEFLAG 为 1，CREDIT 为 1。则相应的插入语句为:

```
INSERT  INTO  PURCHASING.VENDOR_EXCELLENT(ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT)
VALUES('00 ', '电子工业出版社', 1, 1);
```

例 3 从视图 VENDOR_EXCELLENT 中删除名称为中华书局的供应商信息。

```
DELETE  FROM  PURCHASING.VENDOR_EXCELLENT  WHERE  NAME = '中华书局';
```

系统将该语句与 VENDOR_EXCELLENT 视图的定义相结合，转换成对基表的语句:

```
DELETE  FROM  PURCHASING.VENDOR WHERE  NAME = '中华书局'  AND  CREDIT = 1;
```

系统执行该语句，会报告违反约束错误，因为 VENDOR_EXCELLENT 尽管是视图，在做更新时一样要考虑基表间的引用关系。PRODUCT_VENDOR 表与 VENDOR 表存在着引用

关系，PRODUCT_VENDOR 表为引用表，VENDOR 表为被引用表，只有当引用表中没有相应 VENDORID 时才能删除 VENDOR 表中相应记录。

在关系数据库中，并不是所有视图都是可更新的，即并不是所有的视图更新语句均能有意义地转换成相应的基表更新语句，有些甚至是根本不能转换。例如对视图 VENDOR_STATIS：

```
UPDATE  PRODUCTION.VENDOR_STATIS
SET  PRODUCT_COUNT  = 3
WHERE VENDORID = 5;
```

由于产品数量是查询结果按供应商编号分组后各组所包含的行数，这是无法修改的。像这样的视图为不可更新视图。

目前，不同的关系数据库管理系统产品对更新视图的可操作程度均有差异。在 DM 中，包含以下情况的视图不支持更新：

1. 集合运算；
2. 分组、集函数、HAVING 子句、层次查询、TOP 子句、ROWNUM；
3. 不可更新视图、外部表、物化视图辅助表、位图连接索引辅助表、自定义的统计信息表、系统表；
4. 超过 128 个表对象；
5. INI 参数 ENABLE_DIST_VIEW_UPDATE 为 0 时，含有 DISTINCT 不支持更新视图。但若 COMPATIBLE_MODE=1，会强制设置 ENABLE_DIST_VIEW_UPDATE=1，此时含有 DISTINCT 可以更新视图；
6. INI 参数 MULTI_UPD_OPT_FLAG 不包含 2 时，含有多表连接；或多表连接条件不包含聚集主键；
7. 视图定义中包含 TOP、LIMIT、ROW_LIMIT 子句；
8. 在 TABLE 函数所得表之上建立的视图；

9. 在不允许更新视图之上建立的视图;

10. 视图查询项未包含基表所有的聚集索引列。

应该说明的是：只有当视图是可更新的时候，才可以选择 WITH CHECK OPTION 项。

第 7 章 物化视图

物化视图是从一个或几个基表导出的表，同视图相比，它存储了导出表的真实数据。当基表中的数据发生变化时，物化视图所存储的数据将变得陈旧，用户可以通过手动刷新或自动刷新来对数据进行同步。

在本章各例中，如不特别说明，以下例子用户均为建表者 SYSDBA。

7.1 物化视图的定义

语法格式

```
CREATE MATERIALIZED VIEW [<模式名>.]<物化视图名>[(<列名>{,<列名>})][<辅助表子句>|<预建表子句>][<物化视图刷新选项>][<查询改写选项>]AS<查询说明>;

<辅助表子句>::= [BUILD IMMEDIATE|BUILD DEFERRED][<表空间子句>][<STORAGE 子句>]

<表空间子句>::= 参见 3.5.1.1 定义数据库基表

<STORAGE 子句>::= 参见 3.5.1.1 定义数据库基表

<预建表子句>::=  FOR <预建表表名> ON PREBUILT TABLE <精度设置子句>

<精度设置子句> ::= [WITH REDUCED PRECISION | WITHOUT REDUCED PRECISION]

<物化视图刷新选项> ::= REFRESH <刷新选项> {<刷新选项>} | NEVER REFRESH

<刷新选项> ::= <刷新方法> | <刷新时机> | <刷新规则> | <完全刷新方式>

<刷新方法> ::= FAST | COMPLETE | FORCE

<刷新时机> ::=

    ON DEMAND | ON COMMIT |

    START WITH datetime_expr | NEXT datetime_expr |

    START WITH datetime_expr  NEXT datetime_expr
```

```
<刷新规则> ::=
    WITH PRIMARY KEY |
    WITH ROWID
<完全刷新方式> ::=
    USING DEFAULT |
    USING TRUNCATE |
    USING DELETE
<查询改写选项>::= [DISABLE | ENABLE] QUERY REWRITE
<查询说明>::= <表查询> | <连接表>
<表查询>::= <子查询表达式> [ORDER BY 子句]
<连接表>::= 请参考第 4 章 数据查询语句
<datetime_expr>::= SYSDATE [+<数值常量>]
```

参数

1. <模式名> 指明被创建的视图属于哪个模式，缺省为当前模式；
2. <物化视图名> 指明被创建的物化视图的名称；
3. <列名> 指明被创建的物化视图中列的名称；
4. [BUILD IMMEDIATE | BUILD DEFERRED] 指明 BUILD IMMEDIATE 为立即填充数据，默认为立即填充；BUILD DEFERRED 为延迟填充，使用这种方式要求第一次刷新必须为 COMPLETE 完全刷新；
5. <预建表表名> 数据库中已存在的表叫做预建表；
6. ON PREBUILT TABLE 指明创建一个预建表物化视图，即指定一个已存在的表（预建表）作为物化视图的辅助表；
7. <精度设置子句> 标识预建表的每一列的精度是否可以比物化视图定义的子查询中各对应的列的精度低，目前 DM8 仅语法支持此项，实际功能未实现；

8. 刷新方法

   - FAST

     根据相关表上的数据更改记录进行增量刷新。普通 DML 操作生成的记录存在于物化视图日志。使用 FAST 刷新之前，必须先建好物化视图日志。

   - COMPLETE

     通过执行物化视图的定义脚本进行完全刷新。

   - FORCE

     默认选项。当快速刷新可用时采用快速刷新，否则采用完全刷新。

9. 刷新时机

   - ON DEMAND

     由用户通过 REFRESH 语法进行手动刷新。如果指定了 START WITH 和 NEXT 子句就没有必要指定 ON DEMAND。

   - ON COMMIT

     在相关表上事务提交时进行快速刷新。刷新是由异步线程执行的，因此 COMMIT 执行结束后可能需要等待一段时间物化视图数据才是最新的。包含远程表的物化视图不支持 ON COMMIT 快速刷新。

   - START WITH datetime_expr | NEXT datetime_expr

     START WITH 用于指定首次刷新物化视图的时间，NEXT 指定自动刷新的间隔；

     如果省略 START WITH 则首次刷新时间为当前时间加上 NEXT 指定的间隔；

     如果指定 START WITH 省略 NEXT 则物化视图只会刷新一次；

     如果二者都未指定物化视图不会自动刷新。

10. 刷新规则

    - WITH PRIMARY KEY

      默认选项。

- 若显式指定刷新方法为 FAST，则必须含有 PRIMARY KEY 约束，此时选择列必须直接含有所有的 PRIMARY KEY（UPPER(col_name)的形式不可接受）；

- WITH ROWID
  - 如果使用 WITH ROWID 的同时使用快速刷新，则必须将 ROWID 提取出来，和其他列名一起，以别名的形式显示。

11. 完全刷新方式
    - USING DEFAULT

      默认选项。

      物化视图完全刷新时，根据 INI 参数 DEL_HP_OPT_FLAG 决定删除老数据使用的方式。
    - USING TRUNCATE

      物化视图完全刷新时，使用 TRUNCATE 删除老数据。
    - USING DELETE

      物化视图完全刷新时，使用 DELETE 删除老数据。
12. NEVER REFRESH 物化视图从不进行刷新。可以通过 ALTER MATERALIZED VIEW <物化视图名> FRESH 进行更改；
13. QUERY REWRITE 选项
    - ENABLE

    允许物化视图用于查询改写。
    - DISABLE

    禁止物化视图用于查询改写。

    目前 DM8 仅语法支持查询改写选项，实际功能未实现；
14. <子查询表达式> 标识物化视图所基于的表的行和列。其语法遵照 SELECT 语句的

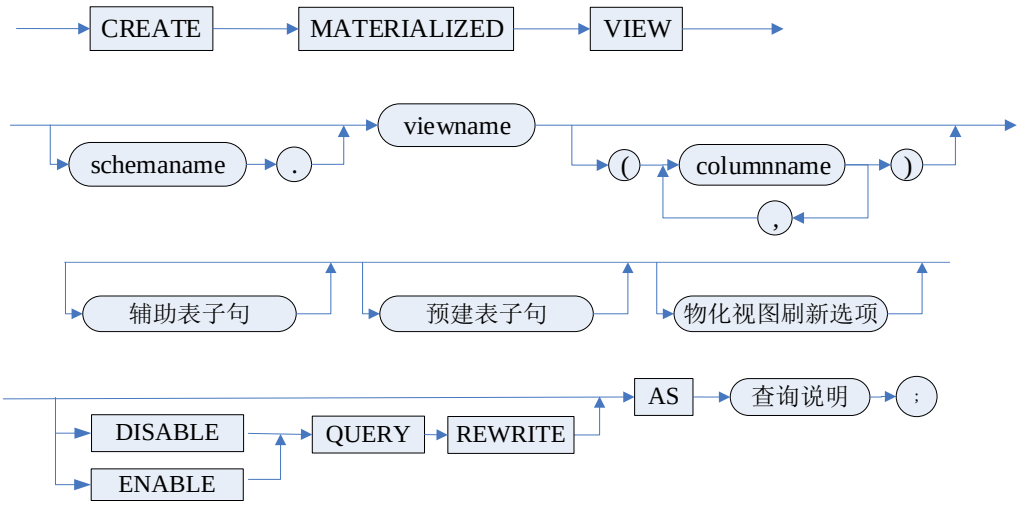
语法规则；

15. 定义查询中的 ORDER BY 子句仅在创建物化视图时使用，此后 ORDER BY 被忽略；

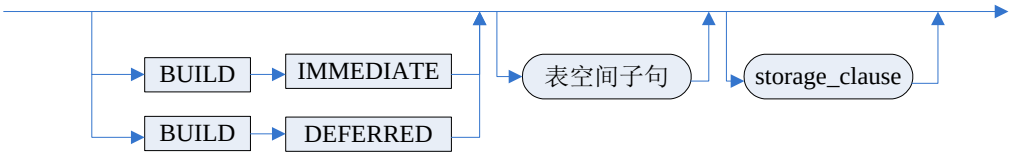
16. datetime_expr 只能是日期常量表达式，SYSDATE[+<数值常量>]或日期间隔。

图例

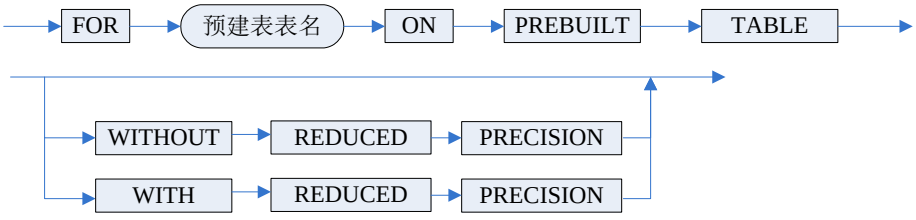
物化视图的定义



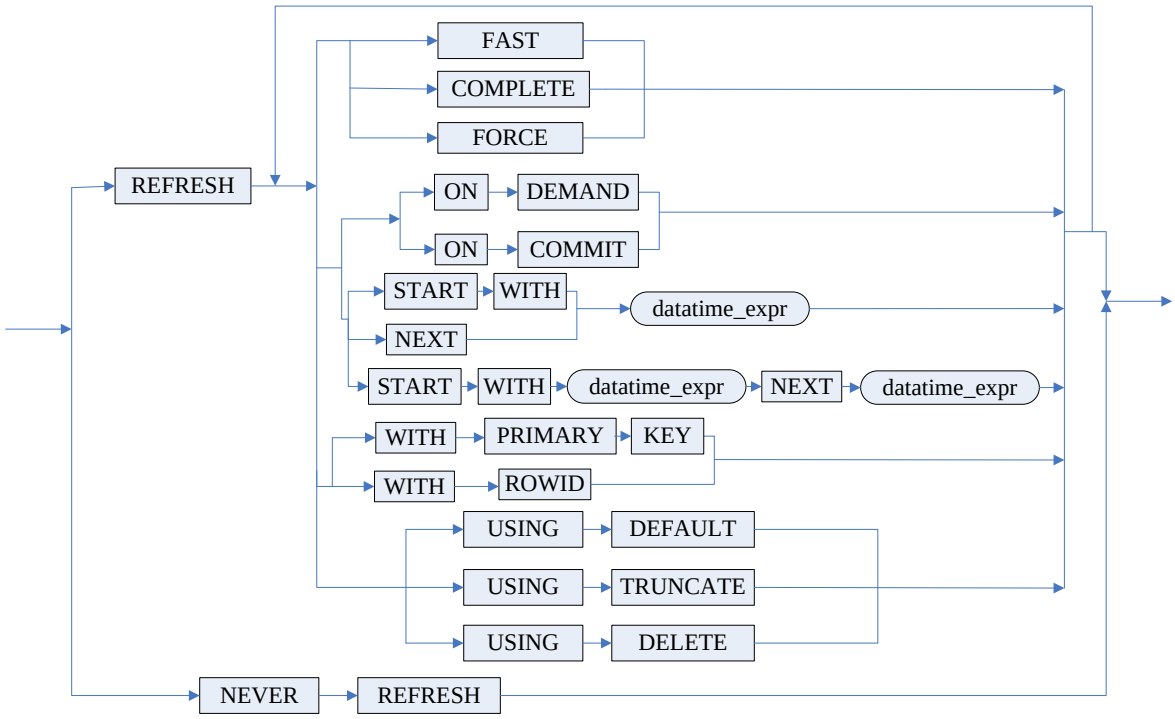
辅助表子句



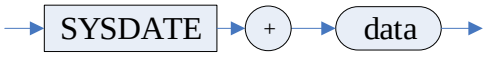
预建表子句



物化视图刷新选项



datetime_expr



语句功能

供 DBA 或该物化视图的拥有者且具有 CREATE MATERIALIZED VIEW 权限的用户定义物化视图。

使用说明

1．创建物化视图时，会产生两个字典对象：物化视图和物化视图表，后者用于存放真实的数据；

2．快速刷新物化视图的限制见本章第 8 节《物化视图的限制》；

3．由于受物化视图表的命名规则所限，物化视图名称长度必须小于 123 个字节。

权限

1．在自己模式下创建物化视图时，该语句的使用者必须被授予 CREATE MATERIALIZED VIEW 系统权限，且至少拥有 CREATE TABLE 或者 CREATE ANY TABLE 两个系统权限中的一个；

2．在其他用户模式下创建物化视图时，该语句的使用者必须具有 CREATE ANY

MATERIALIZED VIEW 系统权限，且物化视图的拥有者必须拥有 CREATE TABLE 系统权限；

3．物化视图的拥有者必须对<查询说明>中的每个表均具有 SELECT 权限或者具有 SELECT ANY TABLE 系统权限。

举例说明

例 对 VENDOR 表创建一个物化视图，名为 MV_VENDOR_EXCELLENT，保存信誉等级为 1 的供应商，列名有：VENDORID、ACCOUNTNO、NAME、ACTIVEFLAG、CREDIT。不允许查询改写，依据 ROWID 刷新且刷新间隔为一天。

```
CREATE MATERIALIZED VIEW  PURCHASING.MV_VENDOR_EXCELLENT
REFRESH WITH ROWID START WITH SYSDATE NEXT SYSDATE + 1 AS
SELECT  VENDORID, ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT
FROM  PURCHASING.VENDOR
WHERE  CREDIT = 1;
```

如果使用 WITH ROWID 的同时，后面还要使用快速刷新，则此处的语句应写为：

```
//先创建好物化视图日志，创建步骤参考 7.6 物化视图日志的定义。
CREATE MATERIALIZED VIEW  PURCHASING.MV_VENDOR_EXCELLENT
REFRESH FAST WITH ROWID START WITH SYSDATE NEXT SYSDATE + 1 AS
SELECT  VENDORID, ACCOUNTNO, NAME, ACTIVEFLAG, CREDIT,ROWID AS X
FROM  PURCHASING.VENDOR
WHERE  CREDIT = 1;
```

运行该语句后，DM 服务器将得到：1）物化视图：MV_VENDOR_EXCELLENT；2）以 MTAB$_为前缀的物化视图表：MTAB$_MV_VENDOR_EXCELLENT；3）定时刷新的物化视图触发器：MTRG_REFRESH_MVIEW_1670 （假定 MTAB$_MV_VENDOR_EXCELLENT 对象的 ID 是 1670）。

对该物化视图进行查询：

```
SELECT * FROM PURCHASING.MV_VENDOR_EXCELLENT;
```

查询结果如下:

```
VENDORID    ACCOUNTNO NAME                ACTIVEFLAG CREDIT
----------- --------- ------------------- ---------- -----------
3           00        北京十月文艺出版社  1          1
4           00        人民邮电出版社      1          1
5           00        清华大学出版社      1          1
6           00        中华书局            1          1
7           00        广州出版社          1          1
8           00        上海出版社          1          1
9           00        21 世纪出版社       1          1
10          00        外语教学与研究出版社 1         1
11          00        机械工业出版社      1          1
12          00        文学出版社          1          1
```

7.2 物化视图的修改

语法格式

```
ALTER MATERIALIZED VIEW [<模式名>.]<物化视图名>[<物化视图刷新选项>][<查询改写选项>];

<物化视图刷新选项> ::= 参见 7.1 物化视图的定义

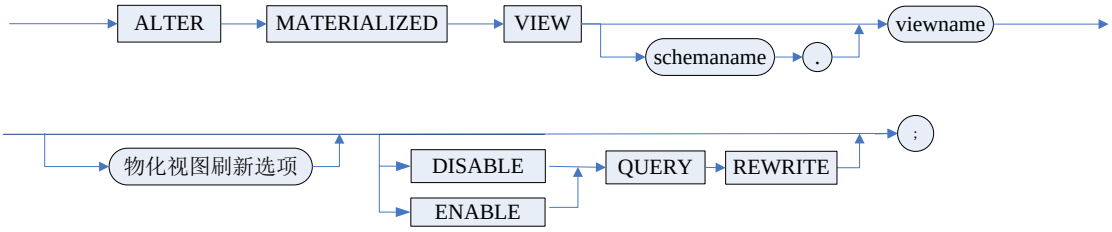
<查询改写选项> ::= 参见 7.1 物化视图的定义
```

参数

1. <模式名> 指明被创建的视图属于哪个模式，缺省为当前模式;
2. <物化视图名> 指明被创建的物化视图的名称。

图例

物化视图的修改



权限

使用者必须是该物化视图的拥有者或者拥有 ALTER ANY MATERIALIZED VIEW 系统权限。

举例说明

例 1 修改物化视图 MV_VENDOR_EXCELLENT，使之可以用于查询改写。

```
ALTER MATERIALIZED VIEW PURCHASING.MV_VENDOR_EXCELLENT ENABLE QUERY REWRITE;
```

例 2 修改物化视图 MV_VENDOR_EXCELLENT 为完全刷新。

```
ALTER MATERIALIZED VIEW PURCHASING.MV_VENDOR_EXCELLENT REFRESH COMPLETE;
```

7.3 物化视图的删除

语法格式

```
DROP MATERIALIZED VIEW [IF EXISTS] [<模式名>.]<物化视图名>;
```

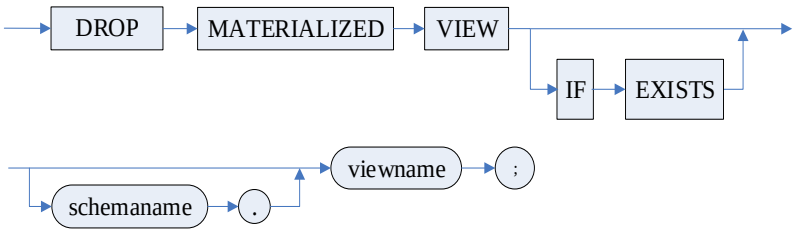
参数

1．<模式名> 指明被删除视图所属的模式，缺省为当前模式；

2．<物化视图名> 指明被删除物化视图的名称。

图例

物化视图的删除



使用说明

1. 删除不存在的视图会报错。若指定IF EXISTS关键字，删除不存在的视图，不会报错；

2. 物化视图删除时会清除物化视图和物化视图表；

3. 物化视图删除后，用户在其上的权限也均自动取消，以后系统中再建的同名物化视图，是与它毫无关系的物化视图；

4. 用户不能直接删除物化视图表对象。

权限

使用者必须是物化视图的拥有者或者拥有 DROP ANY MATERIALIZED VIEW 系统权限。

举例说明

例 删除物化视图 MV_VENDOR_EXCELLENT，可使用下面的语句：

```
DROP MATERIALIZED VIEW PURCHASING.MV_VENDOR_EXCELLENT;
```

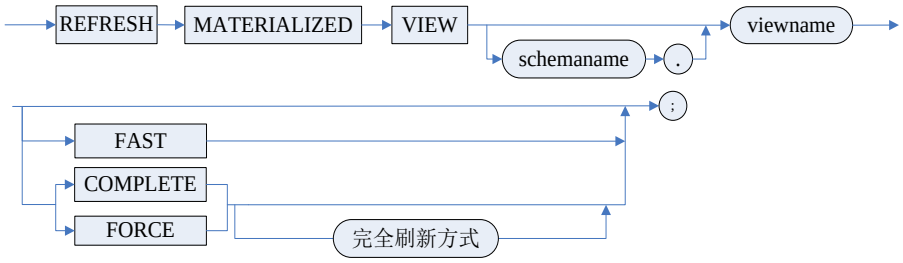
7.4 物化视图的刷新

语法格式

```
REFRESH MATERIALIZED VIEW  [<模式名>.] <物化视图名> [
  FAST |
  COMPLETE [<完全刷新方式>] |
  FORCE [<完全刷新方式>] ];
<完全刷新方式> ::= 参见 7.1 物化视图的定义
```

图例

物化视图的刷新



权限

1．如果是基于物化视图日志的刷新，则使用者必须是物化视图日志的拥有者或者具有 SELECT ANY TABLE 系统权限；

2．使用者必须是物化视图的拥有者或者具有 SELECT ANY TABLE 系统权限。

使用说明

1. 刷新方法的默认选项为FORCE；

2. 物化视图的刷新语句总是自动提交的，不能回滚。

举例说明

例 采用 FAST 方法刷新物化视图 MV_VENDOR_EXCELLENT。

```
REFRESH MATERIALIZED VIEW  PURCHASING.MV_VENDOR_EXCELLENT FAST;
//使用快速刷新前，必须先建好物化视图日志
```

7.5 物化视图允许的操作

对物化视图进行查询或建立索引时这两种操作都会转为对其物化视图表的处理。用户不能直接对物化视图及物化视图表进行插入、删除、更新和 TRUNCATE 操作，对物化视图数据的修改只能通过刷新物化视图语句进行。

7.6 物化视图日志的定义

物化视图的快速刷新依赖于基表上的物化视图日志，物化视图日志记录了基表的变化信息。

语法格式

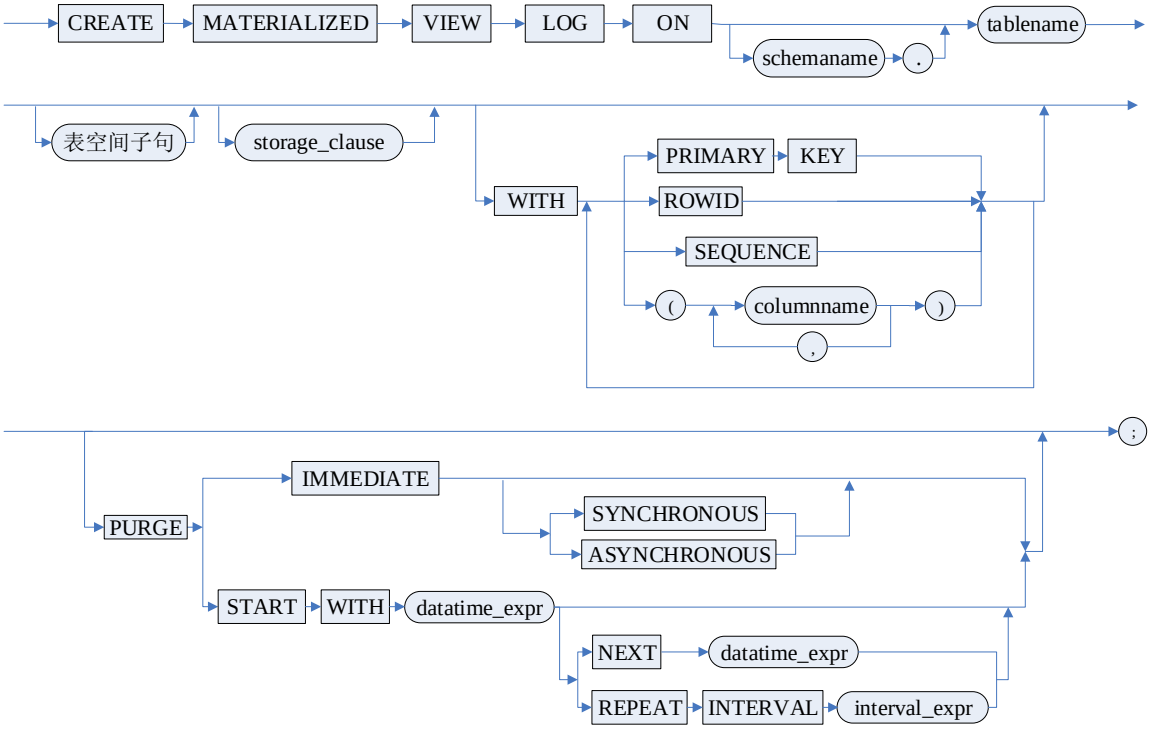
```
CREATE MATERIALIZED VIEW  LOG ON [<模式名>.]<表名>
[<表空间子句>][<STORAGE 子句>][<WITH 子句>][<PURGE 选项>];
<表空间子句>::=参见 3.5.1.1 定义数据库基表
<STORAGE 子句>::=参见 3.5.1.1 定义数据库基表
<WITH 子句>::= WITH { PRIMARY KEY| ROWID | SEQUENCE | (<列名> {, <列名>})}
<PURGE 选项>::=
    PURGE IMMEDIATE [ SYNCHRONOUS | ASYNCHRONOUS ] |
    PURGE START WITH <datetime_expr> [ NEXT <datetime_expr> | REPEAT INTERVAL <interval_expr>]
```

参数

1. <模式名> 指明物化视图日志基表所属的模式，缺省为当前模式；
2. <表名> 指明创建日志的基表；
3. <WITH 子句> 基表中的哪些列将被包含到物化视图日志中，SEQUENCE 表示物化视图日志表中有 SEQUENCE$$唯一标识列，SEQUENCE 为默认选项；
4. <PURGE 选项> 指定每隔多长时间对物化视图日志中无用的记录进行一次清除。分两种情况：一是 IMMEDIATE 立即清除；二是 START WITH 定时清除。缺省是 PURGE IMMEDIATE。SYNCHRONOUS 为同步清除；ASYNCHRONOUS 为异步清除。ASYNCHRONOUS 和 SYNCHRONOUS 的区别是前者新开启一个事务来进行日志表的清理，后者是在同一个事务里。目前，ASYNCHRONOUS 仅语法支持，功能未实现；
5. <datetime_expr> 只能是日期常量表达式，SYSDATE[+<数值常量>]或日期间隔；
6. 与物化视图可能依赖多个基表不同，物化视图日志只对应一个基表，因此物化视图日志是否使用行外大字段存储与基表保持一致。

图例

物化视图日志的定义



使用说明

1．在表 T 上创建物化视图日志后会生成：1）以 MLOG$_为前缀的名为 MLOG$_T 的日志表；2）以 MTRG$_为前缀的名为 MTRG$_T 的表级触发器；3）定时 purge 物化视图日志的触发器 MTRG_PURGE_MVLOG_1270（此处假定 MLOG$_T 对象的 ID 为 1270）。默认情况下，用户可以对 MLOG$_T 进行查询但是不能进行插入、删除和更新，触发器由系统维护，用户无法修改删除；设置 INI 参数 VIEW_OPT_FLAG=2 后，用户可以对 MLOG$_T 进行插入、删除、更新和 TRUNCATE；

2．由于物化视图日志表的命名规则所限，日志基表名称长度必须小于 123 个字符；

3．如果在物化视图 MV 上创建物化视图日志，系统会自动转为在物化视图的基表 MTAB$_MV 上创建物化视图日志，因此会生成名为 MLOG$_MTAB$_MV 的日志表和名为 MTRG$_MTAB$_MV 的表级触发器，且要求物化视图名称必须小于 116 个字符，其余限

制和普通表一致。

权限

1．如果是物化视图日志基表的拥有者，使用者必须拥有 CREATE TABLE 系统权限；

2．如果物化视图日志基表是其它模式下的表，使用者必须拥有 CREATE ANY TABLE 系统权限，且物化视图日志的拥有者必须对<查询说明>中的每个表均具有 SELECT 权限或者具有 SELECT ANY TABLE 系统权限；

3．物化视图日志表仅支持基于的表为普通表、堆表和物化视图。

举例说明

例 在 PURCHASING. VENDOR 上创建物化视图日志，每天定时 PURGE。

```
CREATE MATERIALIZED VIEW LOG ON PURCHASING.VENDOR WITH

ROWID(ACCOUNTNO,NAME,ACTIVEFLAG,WEBURL,CREDIT) PURGE      START WITH SYSDATE + 5

REPEAT INTERVAL '1' DAY;
```

注：间隔一天 PURGE 也可以写成 PURGE NEXT SYSDATE + 1。

7.7 物化视图日志的删除

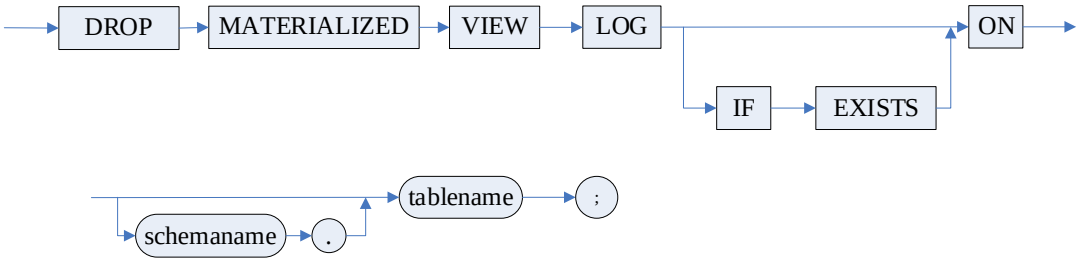
语法格式

```
DROP MATERIALIZED VIEW LOG [IF EXISTS] ON [<模式名>.]<表名>;
```

物化视图日志删除时会同时 DROP 掉日志表对象和触发器对象。另外，删除物化视图基表的同时，也会级联删除相应的物化视图日志。删除不存在的物化视图日志会报错。若指定了 IF EXISTS 关键字，删除不存在的物化视图日志，不会报错。

图例

物化视图日志的删除



权限

使用者必须拥有删除表的权限。

7.8 物化视图的限制

7.8.1 物化视图的一般限制

1．物化视图定义只能包含用户创建的表、视图和物化视图对象，且不能为外部表；

2．对物化视图日志、物化视图只能进行查询和建索引，不支持插入、删除、更新、MERGE INTO 和 TRUNCATE；当设置 INI 参数 VIEW_OPT_FLAG=2 时，物化视图日志表支持插入、删除、更新和 TRUNCATE；

3．同一表上最多允许建立 127 个物化视图；

4．物化视图需要内建辅助表，命名规则为：MTAB$_物化视图名，因此，物化视图名称的最大长度为 122；

5．包含物化视图的普通视图及游标是不能更新的；

6．如果对某表进行了 TRUNCATE 操作，那么依赖于它的物化视图必须先进行一次完全刷新后才可以使用快速刷新；

7．如果对某表进行了快速装载操作，那么依赖于它的物化视图必须先进行一次完全刷新后才可以使用快速刷新；

8．如果对某表进行了与分区子表数据交换操作，那么依赖于它的物化视图必须先进行一次完全刷新后才可以使用快速刷新；

9．对于预建表物化视图定义中子查询的每一列，在预建表中必须有唯一的列与之对

应，且列号、列名、列类型必须相同；

10．若在预建表上创建了预建表物化视图，则不能直接删除预建表，需要先删除物化视图，才能删除预建表。

7.8.2 物化视图的分类

依据物化视图定义中查询语句的不同分为以下六种。

1．SIMPLE：无 GROUP BY、无集函数、无连接操作、无远程数据库表、无视图；

2．AGGREGATE：仅包含有 GROUP BY 和集函数；

3．JOIN：仅包含有多表连接；

4．Sub-Query：仅包含有子查询；

5．SETS：包含 UNION ALL；

6．COMPLEX：除上述五种外的物化视图类型。

用户可以通过查看系统视图 SYS.USER_MVIEWS 的 MVIEW_TYPE 列来了解当前用户所拥有的物化视图的分类。

7.8.3 快速刷新通用约束

1．快速刷新物化视图要求每个基表都包含有物化视图日志，并且物化视图日志的创建时间不得晚于物化视图的最后刷新时间；

2．不能含有不确定性函数，如 SYSDATE 或 ROWNUM；

3．查询项不能含有分析函数；

4．查询不能含有 HAVING 子句；

5．不能包含 ANY、ALL 及 NOT EXISTS；

6．不能含有层次查询；

7．不能在多个站点含有相关表；

8．同一张表上最多允许建立 127 个快速刷新的物化视图；

9．不能含有除 UNION ALL 外的集合运算；

10．不能含有子查询；

11．只能基于普通表（视图，外部表，派生表等不支持）；

12．WITH PRIMARY KEY 时物化视图定义里如果是单表，则日志表里有 PK，如果是多表，则每张表的日志表里都有 PK；WITH ROWID 时物化视图里是单表，则日志表里必须有 ROWID，如果是多表，则每张日志表里都有 ROWID；

13．对于 WITH ROWID 的快速刷新需要一一选择 ROWID 并给出别名；

14．WITH PRIMARY KEY 刷新时，物化视图定义中必须包含所有其基于的表的 PK 列；

15．如果日志定义中没有 WITH PRIMARY KEY 而扩展列又包含了，那么 DM 认为这个和建立日志时指定 WITH PRIMARY KEY 效果相同。也就是说，基于这个日志建立 WITH PK 的快速刷新物化视图是允许的；

16．DM8 目前仅支持简单类型和部分连接物化视图的快速刷新。连接物化视图不支持的具体类型是外连接和自然连接；

17．连接物化视图不支持 GROUP BY 和聚集操作；

18．对于分组物化视图的快速刷新，有以下限制：1)查询项中一定要有 count(*)；2)集函数仅支持 count(*)、count、sum、avg、stddev、variance，不支持 max、min 等；3)若某集函数出现在查询项中，则其依赖的集函数也必须出现，具体见下表。

表 7.8.1 集函数依赖关系

集函数	被依赖的集函数
sum(exp)	count(exp)，当 exp 是 not null 单列时，sum 不依赖于 count
avg(exp)	count(exp)、sum(exp)
stddev(exp)	count(exp)、sum(exp)、sum(exp*exp)

variance(exp)	count(exp)、sum(exp)、sum(exp*exp)

7.8.4 物化视图信息查看

用户可以通过系统视图 SYS.USER_MVIEWS 查看当前用户所拥有的物化视图的相关信息，视图定义如下：

表 7.8.2 视图定义

列名	数据类型	备注
SCHID	INTEGER	模式 ID
MVIEW_NAME	VARCHAR(128)	物化视图名称
QUERY	TEXT	文本信息
QUERY_LEN	INTEGER	文本信息长度，单位：字节
REWRITE_ENABLED	VARCHAR(128)	是否可以被重写。Y 是，N 否
REFRESH_MODE	VARCHAR(128)	刷新模式：DEMAND,COMMIT
REFRESH_METHOD	VARCHAR(128)	刷新方法：COMPLETE,FORCE,FAST,NEVER
MVIEW_TYPE	VARCHAR(128)	快速刷新类型： SIMPLE：简单； AGGREGATE：聚合； JOIN：连接； Sub-Query：子查询； SETS：包含UNION ALL； COMPLEX：不可快速刷新
LAST_REFRESH_TYPE	VARCHAR(128)	最后一次刷新的方法
STALENESS	VARCHAR(128)	物化视图状态： UNUSEABLE：物化视图不可用，即从未刷新过；

		FRESH：物化视图数据是最新的； NEEDS_COMPILE：物化视图数据陈旧； COMPILE_ERROR：物化视图解析出错，如基表不存在； NEEDS_FULL_REFRESH：物化视图数据陈旧，需要一次全刷新； UNDEFINED：物化视图包含远程表状态不可知
LAST_REFRESH_DATE	DATETIME(6)	最后刷新开始的日期
LAST_REFRESH_END_DATE	DATETIME(6)	最后刷新结束的日期

7.9 物化视图的编译

一个物化视图依赖于其基表、如果基表定义发生改变，如修改或删除一列或多列，或者删除基表，可能导致物化视图无法刷新。另外，物化视图的快速刷新依赖于基表上的物化视图日志，如果日志表发生改变，如修改或删除一列，或者删除物化视图日志，可能导致物化视图无法刷新。对于预建表物化视图，当预建表发生改变时，也可能导致刷新失败。在这些情况下，可对物化视图重新编译，检查物化视图的合法性。

语法格式

```
ALTER MATERIALIZED VIEW [<模式名>.]<物化视图名> COMPILE;
```

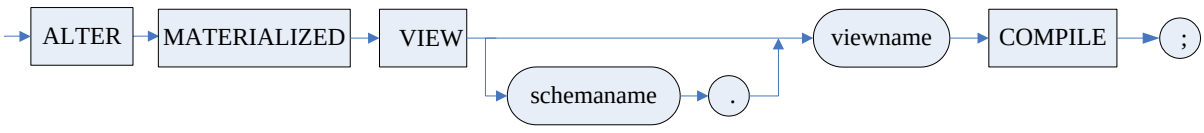
参数

1．<模式名> 指明被编译视图所属的模式，缺省为当前模式；

2．<物化视图名> 指明被编译物化视图的名称；

图例

物化视图的编译



使用说明

对物化视图视图的定义重新进行分析和编译，如果编译出错，则报错。用户可以根据报错信息判断物化视图的依赖的基表、物化视图日志表或预建表是否发生改变或已被删除。

权限

使用者必须拥有 DBA 权限或是该物化视图的拥有者。

举例说明

例 修改基表导致物化视图无法刷新，通过物化视图编译语句检查其合法性。

```
DROP TABLE IF EXISTS T1;

DROP MATERIALIZED VIEW IF EXISTS VIEW1;

CREATE TABLE T1(A INT, B VARCHAR);

INSERT INTO T1 VALUES(1,'1');

CREATE MATERIALIZED VIEW VIEW1 AS SELECT * from T1;

//修改基表列定义

ALTER TABLE T1 DROP B CASCADE;

//视图无法刷新

REFRESH MATERIALIZED VIEW VIEW1;

第 1 行附近出现错误[-2207]:无法解析的成员访问表达式[T1.B].

//重新编译物化视图结果如下

ALTER MATERIALIZED VIEW VIEW1 COMPILE;

第 1 行附近出现错误[-2207]:无法解析的成员访问表达式[T1.B].
```

第 8 章 函数

在值表达式中，除了可以使用常量、列名、集函数等之外，还可以使用函数作为组成成份。DM 中支持的函数分为数值函数、字符串函数、日期时间函数、空值判断函数、类型转换函数等。在这些函数中，对于字符串类型的参数或返回值，最大支持的长度为 32K-1。

本手册还给出了 DM 系统函数的详细介绍。下列各表列出了函数的简要说明。在本章各例中，如不特别说明，各例均使用示例库 BOOKSHOP，用户均为建表者 SYSDBA。

表 8.1 数值函数

序号	函数名	功能简要说明
01	ABS(n)	求数值 n 的绝对值
02	ACOS(n)	求数值 n 的反余弦值
03	ASIN(n)	求数值 n 的反正弦值
04	ATAN(n)	求数值 n 的反正切值
05	ATAN2(n,m)	求数值 n/m 的反正切值
06	CEIL(n)	求大于或等于数值 n 的最小整数
07	CEILING(n)	求大于或等于数值 n 的最小整数，等价于 CEIL(n)
08	COS(n)	求数值 n 的余弦值
09	COSH(n)	求数值 n 的双曲余弦值
10	COT(n)	求数值 n 的余切值
11	DEGREES(n)	求弧度 n 对应的角度值
12	EXP(n)	求数值 n 的自然指数
13	FLOOR(n)	求小于或等于数值 n 的最大整数
14	GREATEST(n {,n})	求一个或多个数中最大的一个

15	GREAT(n1,n2)	求 n1、n2 两个数中最大的一个
16	LEAST(n {,n})	求一个或多个数中最小的一个
17	LN(n)	求数值 n 的自然对数
18	LOG(n1[,n2])	求数值 n2 以 n1 为底数的对数
19	LOG10(n)	求数值 n 以 10 为底的对数
20	MOD(m,n)	求数值 m 被数值 n 除的余数
21	PI()	得到常数 π
22	POWER(n1,n2)/POWER2(n1,n2)	求数值 n2 以 n1 为基数的指数
23	RADIANS(n)	求角度 n 对应的弧度值
24	RAND([n])	求一个 0 到 1 之间的随机浮点数
25	ROUND(n[,m[,trunc_flag]])	求四舍五入值或直接进行截断后的值
26	SIGN(n)	判断数值的数学符号
27	SIN(n)	求数值 n 的正弦值
28	SINH(n)	求数值 n 的双曲正弦值
29	SQRT(n)	求数值 n 的平方根
30	TAN(n)	求数值 n 的正切值
31	TANH(n)	求数值 n 的双曲正切值
32	TO_NUMBER(str [,fmt [, 'nls']])	将 CHAR、VARCHAR、VARCHAR2、CLOB 等类型的字符串转换为 DECIMAL 类型的数值
33	TRUNC(n[,m]) 或 TRUNC(str,[,m])	截取数值函数，str 内只能为数字和'-'、'+'、'.'的组合
34	TRUNCATE(n[,m])或 TRUNCATE(str,[,m])	截取数值函数，等价于 TRUNC 函数

35	TO_CHAR(n [, fmt [, 'nls']])	将数值类型的数据转换为 VARCHAR 类型输出
36	BITAND(n1, n2)	求两个数值型数值按位进行 AND 运算的结果
37	NANVL(n1, n2)	当 n1 为 NaN 时返回 n2，否则返回 n1。此函数功能仅能处理浮点数类型，处理其他数据类型时结果为 n1
38	REMAINDER(n1, n2)	计算 n1 除 n2 的余数，余数取绝对值更小的那一个
39	TO_BINARY_FLOAT(n)	将 number、real 或 double 类型数值转换成 float 类型
40	TO_BINARY_DOUBLE(n)	将 number、real 或 float 类型数值转换成 double 类型
41	BIN_TO_NUM(n1{,n2})	将输入参数表示的二进制数值转换成十进制 number 类型
42	FOUND_ROWS()	获取查询语句的查询结果在被<LIMIT 限定条件>或<TOP 子句>筛选之前满足条件的总行数
43	CONV(n, from_base, to_base)	在不同进制之间转换数字
44	FORMAT (x,d)	将 x 四舍五入到 d 位，再将 x 按'#,###,###.##'格式返回

表 8.2 字符串函数

序号	函数名	功能简要说明
01	ASCII(char)	返回字符对应的整数
02	ASCIISTR(char)	将字符串 char 中，非 ASCII 的字符转成\XXXX(UTF-16)格式，ASCII 字符保持不变
03	BIT_LENGTH(char)	求字符串的位长度
04	CHAR(n)	返回整数 n 对应的数据库字符集所对应字符
05	CHAR_LENGTH(char)/	求字符串的串长度

	CHARACTER_LENGTH(char)	
06	CHR(n)	返回整数 n 对应的数据库字符集所对应字符。等价于 CHAR(n)
07	NCHR(n)	返回整数 n 对应的数据库字符集所对应字符。等价于 CHAR(n)
08	CONCAT(char1,char2,char3,…)	顺序联结多个字符串成为一个字符串
09	DIFFERENCE(char1,char2)	比较两个字符串的 SOUNDEX 值之差异，返回两个 SOUNDEX 值串同一位置出现相同字符的个数。
10	INITCAP(char)	将字符串中单词的首字符转换成大写的字符
11	INS(char1,begin,n,char2)	删除在字符串 char1 中以 begin 参数所指位置开始的 n 个字符，再把 char2 插入到 char1 串的 begin 所指位置。当 begin 为 0 或负数时，会将 begin 的值调整为 1；当 n 为 0 时，表示不删除 char1 中的任何字符，将 char2 插入到 begin 指定的位置，当 n 为负数时，效果与 n=0 时相同
12	INSERT(char1,n1,n2,char2) / INSSTR(char1,n1,n2,char2)	将字符串 char1 从 n1 的位置开始删除 n2 个字符，并将 char2 插入到 char1 中 n1 的位置
13	INSTR(str1,str2[,n,[m]])	从输入字符串 str1 的第 n 个字符开始查找字符串 str2 的第 m 次出现的位置，以字符计算
14	INSTRB(char1,char2[,n,[m]])	从 char1 的第 n 个字节开始查找字符串 char2 的第 m 次出现的位置，以字节计算

15	INSTRC(char1,char2[,n,[m]])	从 char1 的第 n 个字节开始查找字符串 char2 的第 m 次出现的位置，以完整字符计算
16	LCASE(char)	将大写的字符串转换为小写的字符串
17	LEFT(char,n) / LEFTSTR(char,n)	返回字符串最左边的 n 个字符组成的字符串
18	LEN(char)	返回给定字符串表达式的字符（而不是字节）个数（汉字为一个字符），其中不包含尾随空格
19	LENGTH(str)	返回给定字符串表达式的字符（而不是字节）个数（汉字为一个字符），其中包含尾随空格
20	LENGTHC(char)	返回给定字符串表达式的字符（而不是字节）个数（汉字为一个字符），其中包含尾随空格
21	LENGTH2(char)	返回给定字符串表达式的字符（而不是字节）个数（汉字为一个字符），其中包含尾随空格
22	LENGTH4(char)	返回给定字符串表达式的字符（而不是字节）个数（汉字为一个字符），其中包含尾随空格
23	OCTET_LENGTH(char)	返回输入字符串的字节数
24	LOCATE(char,str[,n])	返回 char 在 str 中首次出现的位置
25	LOWER(char)	将大写的字符串转换为小写的字符串
26	LPAD(char1,n[,char2])	在输入字符串的左边填充上 char2 指定的字符，将其拉伸至 n 个字节长度
27	LTRIM(str[,set])	删除字符串 str 左边起，出现在 set 中的任何字符，当遇到不在 set 中的第一个字符时返回结果

28	POSITION(char1 IN char2) / POSITION(char1, char2)	求串 1 在串 2 中第一次出现的位置
29	REPEAT(char,n) / REPEATSTR(char,n)	返回将字符串重复 n 次形成的字符串
30	REPLACE(str, search [,replace])	将输入字符串 str 中所有出现的字符串 search 都替换成字符串 replace ,其中 str 为 char、clob 或 text 类型
31	REPLICATE(char,times)	把字符串 char 自己复制 times 份
32	REVERSE(char)	将字符串反序
33	RIGHT / RIGHTSTR(char,n)	返回字符串最右边 n 个字符组成的字符串
34	RPAD(char1,n[,char2])	类似 LPAD 函数，只是向右拉伸该字符串使之达到 n 个字节长度
35	RTRIM(str[,set])	删除字符串 str 右边起出现的 set 中的任何字符，当遇到不在 set 中的第一个字符时返回结果
36	SOUNDEX(char)	返回一个表示字符串发音的字符串
37	SPACE(n)	返回一个包含 n 个空格的字符串
38	STRPOSDEC(char)	把字符串 char 中最后一个字节的值减一
39	STRPOSDEC(char,pos)	把字符串 char 中指定位置 pos 上的字节值减一
40	STRPOSINC(char)	把字符串 char 中最后一个字节的值加一
41	STRPOSINC(char,pos)	把字符串 char 中指定位置 pos 上的字节值加一
42	STUFF(char1,begin,n,char2)	删除在字符串 char1 中以 begin 参数所指位置

		开始的 n 个字符，再把 char2 插入到 char1 串的 begin 所指位置
43	SUBSTR(str[,m[,n]]) / SUBSTRING(char [FROM m [FOR n]]) /MID(str,m[,n])	返回 char 中从字符位置 m 开始的 n 个字符
44	SUBSTRB(char,m[,n])	SUBSTR 函数等价的单字节形式
45	TO_CHAR(str)	将 VARCHAR、CLOB、TEXT 类型的数据转化为 VARCHAR 类型输出
46	TRANSLATE(char,char_from,char_to)	将所有出现在搜索字符集中的字符转换成字符集中的相应字符
47	TRIM([<<LEADING|TRAILING|BOTH> [char] | char> FROM] str)	删去字符串 str 中由 char 指定的字符
48	UCASE(char)	将小写的字符串转换为大写的字符串
49	UPPER(char)	将小写的字符串转换为大写的字符串
50	NLS_UPPER(char1 [,nls_sort=char2])	将小写的字符串转换为大写的字符串
51	NLS_LOWER(char1 [,nls_sort=char2])	将大写的字符串转换为小写的字符串
52	REGEXP	根据符合 POSIX 标准的正则表达式进行字符串匹配
53	OVERLAY(char1 PLACING char2 FROM int [FOR int])	字符串覆盖函数，用 char2 覆盖 char1 中指定的子串，返回修改后的 char1
54	TEXT_EQUAL(n1,n2)	返回两个 LONGVARCHAR 类型的值的比较结果，相同返回 1，否则返回 0
55	BLOB_EQUAL(n1,n2)	返回两个 LONGVARBINARY 类型的值的比较

		结果，相同返回 1，否则返回 0
56	NLSSORT(char1 [,nls_sort=char2])	返回对自然语言排序的编码
57	GREATEST(char {,char})	求一个或多个字符串中最大的字符串
58	GREAT (char1, char2)	求 char 1、char 2 中最大的字符串
59	TO_SINGLE_BYTE (char)	将多字节形式的字符（串）转换为对应的单字节形式
60	TO_MULTI_BYTE (char)	将单字节形式的字符（串）转换为对应的多字节形式
61	EMPTY_CLOB ()	初始化 clob 字段
62	EMPTY_BLOB ()	初始化 blob 字段
63	UNISTR (char)	将字符串 char 中，ASCII 编码或 Unicode 编码（'\XXXX'4 个 16 进制字符格式）转成本地字符。对于其他字符保持不变
64	ISNULL(char)	判断表达式是否为 NULL
65	CONCAT_WS(delim, char1,char2,char3,...)	顺序联结多个字符串成为一个字符串，并用 delim 分割
66	SUBSTRING_INDEX(char, char_delim, count)	按关键字截取字符串，截取到指定分隔符出现指定次数位置之前
67	COMPOSE(char)	在 UTF8 库下，将 char 以本地编码的形式返回
68	FIND_IN_SET(char, charlist[,separator])	查询 charlist 中是否包含 char，返回 char 在 charlist 中第一次出现的位置或 NULL
69	TRUNC(char1, char2)	截取字符串函数

70	TO_BASE64(R)	将 VARBINARY、VARCHAR、CLOB 或 BLOB 格式的数据 R 编码成 base64 字符集格式，再以 CLOB 类型返回
71	FROM_BASE64(R)	将 VARCHAR 表示的 base64 字符集的编码 R 解码成原始的 VARBINARY 类型数据；或将 CLOB 表示的 base64 字符集的编码 R 解码成原始的 BLOB 类型数据
72	STR_TO_DATE(date_string, format_string)	将字符串按指定格式转换为日期格式
73	DATE(date)	返回日期或日期/时间表达式的日期部分

表 8.3 日期时间函数

序号	函数名	功能简要说明
01	ADD_DAYS(date,n)	返回日期加上 n 天后的新日期
02	ADD_MONTHS(date,n)	在输入日期上加上指定的几个月返回一个新日期
03	ADD_WEEKS(date,n)	返回日期加上 n 个星期后的新日期
04	CURDATE()	返回客户端当前日期
05	CURTIME(n)	返回客户端当前时间
06	CURRENT_DATE()	返回客户端当前日期
07	CURRENT_TIME(n)	返回客户端当前时间
08	CURRENT_TIMESTAMP(n)	返回客户端当前带会话时区信息的时间戳
09	DATEADD(datepart,n,date)	向指定的日期加上一段时间
10	DATEDIFF({datepart,}date1,date2)/	返回跨两个指定日期的日期和时间边

	BIGDATEDIFF(datepart,date1,date2)	界数
11	DATEPART(datepart,date)	返回代表日期的指定部分的整数
12	DAY(date)	返回日期中的天数
13	DAYNAME(date)	返回日期的星期名称
14	DAYOFMONTH(date)	返回日期为所在月份中的第几天
15	DAYOFWEEK(date)	返回日期为所在星期中的第几天
16	DAYOFYEAR(date)	返回日期为所在年中的第几天
17	DAYS_BETWEEN(date1,date2)	返回两个日期之间的天数
18	EXTRACT(时间字段 FROM date)	抽取日期时间或时间间隔类型中某一个字段的值
19	GETDATE(n)	返回系统当前时间戳
20	GREATEST(date {,date})	求一个或多个日期中的最大日期
21	GREAT (date1,date2)	求 date1、date2 中的最大日期
22	HOUR(time)	返回时间中的小时分量
23	LAST_DAY(date)	返回输入日期所在月份最后一天的日期
24	LEAST(date {,date})	求一个或多个日期中的最小日期
25	MINUTE(time)	返回时间中的分钟分量
26	MONTH(date)	返回日期中的月份分量
27	MONTHNAME(date)	返回日期中月分量的名称
28	MONTHS_BETWEEN(date1,date2)	返回两个日期之间的月份数
29	NEXT_DAY(date1,char2)	返回输入日期指定若干天后的日期
30	NOW(n)	返回系统当前时间戳

31	QUARTER(date)	返回日期在所处年中的季节数
32	SECOND(time)	返回时间中的秒分量
33	ROUND (date1[, fmt])	把日期四舍五入到最接近格式元素指定的形式
34	TIMESTAMPADD(datepart,n,timestamp)	返回时间戳 timestamp 加上 n 个 datepart 指定的时间段的结果
35	TIMESTAMPDIFF(datepart,timeStamp1,timestamp2)	返回一个表明 timestamp2 与 timestamp1 之间的指定 datepart 类型时间间隔的整数
36	SYSDATE()	返回系统当前的日期时间，为数据库服务器所在操作系统的日期时间
37	TO_DATE(CHAR[,fmt[,'nls']]) /TO_TIMESTAMP(CHAR[,fmt[,'nls']]) / TO_TIMESTAMP_TZ(CHAR[,fmt[,'nls']])	字符串转换为日期时间数据类型
38	FROM_TZ(timestamp,timezone|[tz_name])	将时间戳类型 timestamp 和时区类型 timezone（或时区名称 tz_name）转化为 timestamp with timezone 类型
39	TZ_OFFSET(timezone|[tz_name])	返回给定的时区或时区名和标准时区(UTC)的偏移量
40	TRUNC(date[,fmt])	把日期截断到最接近格式元素指定的形式
41	WEEK(date)	返回日期为所在年中的第几周
42	WEEKDAY(date)	返回当前日期的星期值

43	WEEKS_BETWEEN(date1,date2)	返回两个日期之间相差周数
44	YEAR(date)	返回日期的年分量
45	YEARS_BETWEEN(date1,date2)	返回两个日期之间相差年数
46	LOCALTIME(n)	返回系统当前时间
47	LOCALTIMESTAMP(n)	返回系统当前时间戳
48	OVERLAPS	返回两个时间段是否存在重叠
49	TO_CHAR(date[,fmt[,'nls']])	将日期数据类型 DATE 转换为一个在日期语法 fmt 中指定语法的 VARCHAR 类型字符串。
50	SYSTIMESTAMP(n)	返回系统当前的时间戳，带有数据库服务器所在操作系统的时区信息
51	NUMTODSINTERVAL(dec,interval_unit)	转换一个指定的 DEC 类型到 INTERVAL DAY TO SECOND
52	NUMTOYMINTERVAL (dec,interval_unit)	转换一个指定的 DEC 类型值到 INTERVAL YEAR TO MONTH
53	WEEK(date, mode)	根据指定的 mode 计算日期为年中的第几周
54	UNIX_TIMESTAMP (datetime)	返回自标准时区的'1970-01-01 00:00:00 +0:00'的到本地会话时区的指定时间的秒数差
55	FROM_UNIXTIME(unixtime)	返回将自'1970-01-01 00:00:00'的秒数差转成本地会话时区的时间戳类型
	FROM_UNIXTIME(unixtime, fmt)	将自'1970-01-01 00:00:00'的秒数差

		转成本地会话时区的指定 fmt 格式的时间串
56	SESSIONTIMEZONE	返回当前会话的时区
57	DBTIMEZONE	返回当前数据库的时区
58	DATE_FORMAT(d, format)	以不同的格式显示日期/时间数据
59	TIME_TO_SEC(d)	将时间换算成秒
60	SEC_TO_TIME(sec)	将秒换算成时间
61	TO_DAYS(timestamp)	转换成公元 0 年 1 月 1 日的天数差
62	DATE_ADD(datetime, interval)	返回一个日期或时间值加上一个时间间隔的时间值
63	DATE_SUB(datetime, interval)	返回一个日期或时间值减去一个时间间隔的时间值
64	SYS_EXTRACT_UTC(d timestamp)	将所给时区信息转换为 UTC 时区信息
65	TO_DSINTERVAL(d char)	将表示时间间隔的字符串转换为 INTERVAL DAY TO SECOND 类型
66	TO_YMINTERVAL(d char)	将表示时间间隔的字符串转换为 INTERVAL YEAR TO MONTH 类型
67	ADDTIME(expr1, expr2)	在指定时间或日期时间值 expr1 上增加一个时间间隔值 expr2

表 8.4 空值判断函数

序号	函数名	功能简要说明
01	COALESCE(n1,n2,...nx)	返回第一个非空的值

02	IFNULL(n1,n2)	当 n1 为非空时，返回 n1；若 n1 为空，则返回 n2
03	ISNULL(n1,n2)	当 n1 为非空时，返回 n1；若 n1 为空，则返回 n2
04	NULLIF(n1,n2)	如果 n1=n2 返回 NULL，否则返回 n1
05	NVL(n1,n2)	返回第一个非空的值
06	NULL_EQU	返回两个类型相同的值的比较

表 8.5 类型转换函数

序号	函数名	功能简要说明
01	CAST(value AS 类型说明)	将 value 转换为指定的类型
02	TREAT(<expr> AS [REF] [<schema>.]<type>)	用于在运行时检查表达式是否可以转换为另一个数据类型
03	CONVERT(类型说明,value[,style]); CONVERT(char, dest_char_set [,source_char_set])	用于当 INI 参数 ENABLE_CS_CVT=0 时，将 value 转换为指定的类型; 用于当 INI 参数 ENABLE_CS_CVT=1 时，将字符串从源串编码格式转换成目的编码格式
04	HEXTORAW(exp)	将由 string 表示的二进制字符串转换为一个 varbinary 数值类型
05	RAWTOHEX(exp)	将 exp 转换为 VARCHAR 类型
06	BINTOCHAR(exp)	将 exp 转换为 CHAR 类型
07	TO_BLOB(value)	将 value 转换为 BLOB 类型
08	UNHEX(exp)	将十六进制的 exp 转换为格式字符串或 VARBINARY
09	HEX(exp)	将字符串的 exp 转换为十六进制字符串
10	CHARTOBIN(exp)	将 exp 转换为 BINARY 类型

表 8.6 杂类函数

序号	函数名	功能简要说明
01	DECODE(exp, search1, result1, … searchn, resultn [,default])	查表译码
02	ISDATE(exp)	判断表达式是否为有效的日期
03	ISNUMERIC(exp)	判断表达式是否为有效的数值
04	DM_HASH (exp)	根据给定表达式生成 HASH 值
05	LNNVL(condition)	根据表达式计算结果返回布尔值
06	LENGTHB(value)	返回 value 的字节数
07	FIELD(value, e1, e2, e3, e4...en)	返回 value 在列表 e1, e2, e3, e4...en 中的位置序号，不在输入列表时则返回 0
08	ORA_HASH(exp [,max_bucket [,seed_value]])	为表达式 exp 生成 HASH 桶值
09	IF(expr1,expr2,expr3)	判断函数。expr1 为布尔表达式，如果其值为 TRUE，则返回 expr2 值，否则返回 expr3 值
10	WIDTH_BUCKET(expr,low_value,high_value,num_buckets)	将指定的范围[low_value,high_value)划分为 num_buckets 个等宽的直方图，每个桶的范围都是左闭右开的，返回指定表达式 expr 的值所属的桶序号
11	OBJECT_ID([schema_name.] object_name[, object_type])	返回对象的 ID

8.1 数值函数

数值函数接受数值参数并返回数值作为结果。

1．函数 ABS

```
语法：ABS(n)
```

功能：返回 n 的绝对值。n 必须是数值类型。

例 查询现价小于 10 元或大于 20 元的信息。

```
SELECT  PRODUCTID,NAME  FROM  PRODUCTION.PRODUCT WHERE  ABS(NOWPRICE-15)>5;
```

查询结果如下：

```
PRODUCTID    NAME

---------- -------------------------

3              老人与海

4              射雕英雄传(全四册)

6              长征

7              数据结构(C 语言版)(附光盘)

10             噼里啪啦丛书(全 7 册)
```

2．函数 ACOS

```
语法：ACOS(n)
```

功能：返回 n 的反余弦值。n 必须是数值类型，且取值在-1 到 1 之间，函数结果从 0 到 π。

例

```
SELECT ACOS(0);
```

查询结果为：1.570796326794897E+000

3．函数 ASIN

```
语法：ASIN(n)
```

功能:返回 n 的反正弦值。n 必须是数值类型，且取值在-1 到 1 之间，函数结果从-π/2 到 π/2。

例

```
SELECT ASIN(0);
```

查询结果为：0.000000000000000E+000

4．函数 ATAN

```
语法：ATAN(n)
```

功能：返回 n 的反正切值。n 必须是数值类型，取值可以是任意大小，函数结果从-π/2 到 π/2。

例

```
SELECT ATAN(1);
```

查询结果为：7.853981633974483E-001

5．函数 ATAN2

```
语法：ATAN2(n, m)
```

功能：返回 n/m 的 4 象限反正切值，返回值表示直角坐标轴中原点至点(m,n)的方位角，和点(m,n)所处象限有关。n 和 m 必须是数值类型，二者不能同时为 0，函数结果范围为(-π,π]。

例

```
SELECT ATAN2(0.2,0.3);
```

查询结果为：5.880026035475676E-001

6．函数 CEIL

```
语法：CEIL(n)
```

功能：返回大于等于 n 的最小整数。n 必须是数值类型。返回类型与 n 的类型相同。

例

```
SELECT    CEIL(15.6);
```

查询结果为：16

```
SELECT    CEIL(-16.23);
```

查询结果为：-16

7．函数 CEILING

```
语法：CEILING(n)
```

功能：返回大于等于 n 的最小整数。等价于函数 CEIL(n)。

8．函数 COS

```
语法：COS(n)
```

功能：返回 n 的余弦值。n 必须是数值类型，是用弧度表示的值。将角度乘以 π/180，可以转换为弧度值。

例

```
SELECT COS(14.78);
```

查询结果为：-5.994654261946543E-001

9．函数 COSH

```
语法：COSH(n)
```

功能：返回 n 的双曲余弦值。

例

```
SELECT COSH(0)"Hyperbolic cosine of 0";
```

查询结果为：1.000000000000000E+000

10．函数 COT

```
语法：COT(n)
```

功能：返回 n 的余切值。n 必须是数值类型，是用弧度表示的值。将角度乘以 π/180，可以转换为弧度值。

例

```
SELECT COT(20 * 3.1415926/180);
```

查询结果为：2.747477470356782E+000

11．函数 DEGREES

```
语法：DEGREES(n)
```

功能：返回弧度 n 对应的角度值，返回值类型与 n 的类型相同。

例

```
SELECT DEGREES(1.0);
```

查询结果为：5.729577951308238E+001

12．函数 EXP

```
语法：EXP(n)
```

功能：返回 e 的 n 次幂。

例

```
SELECT EXP(4) "e to the 4th power";
```

查询结果为：5.459815003314424E+001

13．函数 FLOOR

```
语法：FLOOR(n)
```

功能：返回小于等于 n 的最大整数值。n 必须是数值类型。返回类型与 n 的类型相同。

例

```
SELECT  FLOOR(15.6);
```

查询结果为：15

```
SELECT  FLOOR(-16.23);
```

查询结果为：-17

14．函数 GREATEST

```
语法：GREATEST(n {,n})
```

功能：求一个或多个数中最大的数。

例

```
SELECT GREATEST(1.2,3.4,2.1);
```

查询结果为：3.4

15．函数 GREAT

```
语法：GREAT(n1,n2)
```

功能：求 n1、n2 中的最大的数。

例

```
SELECT GREAT (2, 4);
```

查询结果为：4

16．函数 LEAST

```
语法：LEAST(n {,n})
```

功能：求一个或多个数中最小的一个。

例

```
SELECT LEAST(1.2,3.4,2.1);
```

查询结果为：1.2

17. 函数 LN

```
语法：LN(n)
```

功能：返回 n 的自然对数。n 为数值类型，且大于 0。

例

```
SELECT LN(95) "Natural log of 95";
```

查询结果为：4.553876891600541E+000

18. 函数 LOG

```
语法：LOG(m[,n])
```

功能：返回数值 n 以数值 m 为底的对数；若参数 m 省略，返回 n 的自然对数。m,n 为数值类型，m 大于 0 且不为 1。

例

```
SELECT LOG(10,100);
```

查询结果为：2.000000000000000E+000

```
SELECT LOG(95);
```

查询结果为：4.553876891600541E+000

19. 函数 LOG10

```
语法：LOG10(n)
```

功能：返回数值 n 以 10 为底的对数。n 为数值类型，且大于 0。

例

```
SELECT LOG10(100);
```

查询结果为：2.000000000000000E+000

20. 函数 MOD

```
语法：MOD(m,n)
```

功能：返回 m 除以 n 的余数，当 n 为 0 时直接返回 m。m，n 为数值类型。

例

```
SELECT ROUND(NOWPRICE),mod(ROUND(NOWPRICE),10) FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
ROUND(NOWPRICE) "MOD"(ROUND(NOWPRICE),10)

--------------- -------------------------

15                      5

14                      4

6                       6

22                      2

20                      0

38                      8

26                      6

11                  1

11                  1

42                      2
```

21. 函数 PI

```
语法：PI()
```

功能：返回常数 π。

例

```
SELECT PI();
```

查询结果为：3.141592653589793E+000

22. 函数 POWER/POWER2

```
语法：POWER(m,n) / POWER2(m,n)
```

功能：返回 m 的 n 次幂。m,n 为数值类型，其中 POWER 函数有如下限制：如果 m 为负数的话，n 必须为一个整数。POWER()的返回值类型为 DOUBLE，POWER2()的返回值类型为 DECIMAL。

例

```
SELECT POWER(3,2) "Raised";
```

查询结果为：9.000000000000000E+000

```
SELECT POWER(-3,3) "Raised";
```

查询结果为：-2.700000000000000E+001

23. 函数 RADIANS

```
语法：RADIANS(n)
```

功能：返回角度 n 对应的弧度值，返回值类型为 double 类型。

例

```
SELECT RADIANS(180.0);
```

查询结果为：3.141592653589790E+000

24. 函数 RAND

```
语法：RAND([n])
```

功能：返回一个[0,1]之间的随机浮点数。n 为数值类型，为生成随机数的种子，当 n 固定时，生成随机数固定，当 n 省略时，系统自动生成随机数种子。

例

```
SELECT RAND();
```

查询结果为一个随机生成的小数

```
SELECT RAND(314);
```

查询结果为：6.1265184791121 90E-02

25. 函数 ROUND

```
语法：ROUND(n [,m [,trunc_flag]])
```

功能：求一个数值类型进行四舍五入后的值，或直接进行截断后的值。

trunc_flag 为标记是否四舍五入的数值类型参数，缺省值为 0。

当 trunc_flag 取值为 0 或 NULL 时，返回四舍五入到小数点后面 m 位的 n 值。

当 trunc_flag 取值为非 0 值时，返回直接进行截断后的到小数点后 m 位的 n 值。

m 应为一个整数，缺省值为 0，m 为负整数则四舍五入到小数点的左边，m 为正整数则四舍五入到小数点的右边。若 m 为小数，系统将自动将其转换为整数。

例 1 对 PRODUCTION.PRODUCT 表中的价格使用 ROUND 函数

```
SELECT NOWPRICE,ROUND(NOWPRICE) FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
NOWPRICE ROUND(NOWPRICE)

-------- ---------------

15.2000    15

14.3000    14

6.1000     6

21.7000    22

20.0000    20

37.7000    38
```

```
25.5000   26
11.4000   11
11.1000   11
42.0000   42
```

例 2 对数字使用 ROUND 函数

```
SELECT   ROUND(15.163,-1);
```

查询结果为：20

```
SELECT   ROUND(15.163);
```

查询结果为：15

例 3 对数字使用 ROUND 函数，设置 trunc_flag 的值

```
SELECT   ROUND(15.163,1,5);
```

查询结果为：15.1

```
SELECT   ROUND(15.163,-1,-1);
```

查询结果为：10

```
SELECT   ROUND(15.163,-1,0);
```

查询结果为：20

26. 函数 SIGN

```
语法：SIGN(n)
```

功能：如果 n 为正数，SIGN(n)返回 1，如果 n 为负数，SIGN(n)返回-1，如果 n 为 0，SIGN(n)返回 0。

例

```
SELECT ROUND(NOWPRICE),SIGN(ROUND(NOWPRICE)-20) FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
ROUND(NOWPRICE) SIGN(ROUND(NOWPRICE)-20)
--------------- ------------------------
15                      -1
14                      -1
6                       -1
22                      1
20                      0
38                      1
26                      1
11                      -1
11                      -1
42                      1
```

27. 函数 SIN

```
语法：SIN(n)
```

功能：返回 n 的正弦值。n 必须是数值类型，是用弧度表示的值。将角度乘以 π/180，可以转换为弧度值。

例

```
SELECT SIN(0);
```

查询结果为：0.000000000000000E+000

28. 函数 SINH

```
语法：SINH(n)
```

功能：返回 n 的双曲正弦值。

例

```
SELECT SINH(1);
```

查询结果为：1.1752011936438014E+000

29. 函数 SQRT

```
语法：SQRT(n)
```

功能：返回 n 的平方根。n 为数值类型，且大于等于 0。

例

```
SELECT ROUND(NOWPRICE),SQRT (ROUND(NOWPRICE)) FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
ROUND(NOWPRICE) SQRT(ROUND(NOWPRICE))

--------------- -------------------------

15                    3.872983346207417E+000

14                    3.741657386773941E+000

6                     2.449489742783178E+000

22                    4.690415759823430E+000

20                    4.472135954999580E+000

38                    6.164414002968976E+000

26                    5.099019513592785E+000

11                    3.316624790355400E+000

11                    3.316624790355400E+000

42                    6.480740698407860E+000
```

30. 函数 TAN

```
语法：TAN(n)
```

功能：返回 n 的正切值。n 必须是数值类型，是用弧度表示的值。将角度乘以 π/180，

可以转换为弧度值。

例

```
SELECT TAN(45*Pi()/180);
```

查询结果为：9.999999999999999E-001

31. 函数 TANH

```
语法：TANH(n)
```

功能：返回 n 的双曲正切值。

例

```
SELECT TANH(0);
```

查询结果为：0.000000000000000E+000

32. 函数 TO_NUMBER

```
语法：TO_NUMBER(str [,fmt [,' nls ']])
```

功能：将 CHAR、VARCHAR、VARCHAR2、CLOB 等类型的字符串转换为 DECIMAL 类型的数值。str 为待转换的字符串，fmt 为目标格式串。

若指定了 fmt 格式则转换后的 str 应该遵循相应的数字格式，若没有指定则直接转换成 DECIMAL。fmt 格式串一定要包容实际字符串的数据的格式，否则报错。无格式转换中只支持小数点和正负号。合法的 fmt 格式串字符如下表：

表 8.1.1 合法的 fmt 格式串字符

元素	例子	说明
,(逗号)	9,999	指定位置处返回逗号 注意：1.逗号不能开头 2.不能在小数点右边
.(小数点)	99.99	指定位置处返回小数点
$	$9999	美元符号开头

0	0999 9990	以 0 开头，返回指定字符的数字 以 0 结尾，返回指定字符的数字
9	9999	返回指定字符的数字，如果不够正号以空格代替， 负号以-代替，0 开头也以空格代替。
D	99D99	返回小数点的指定位置，默认为'.'，格式串中最多能有一个 D
G	9G999	返回指定位置处的组分隔符，可有多个，但不能出现在小数点右边
S	S9999 9999S	负值前面返回一个-号 正值前面不返回任何值 负值后面返回一个-号 正值后面不返回任何值 只能在格式串首尾出现
X	XXXX xxxx	返回指定字符的十六进制值，如果不是整数则四舍五入到整数， 如果为负数则返回错误。
C	C9999	当前货币名称缩写
B	B9999	返回指定字符的数字

在设置兼容 Postgres（即设置 INI 参数 COMPATIBLE_MODE=7）后，fmt 格式串中指定了 G，支持待转换字符串中不加千分号（逗号,），或千分号与 G 的位置不对应。当待转换字符串中的千分号与 fmt 格式串中的 G 位置对应时，待转换字符串的数字位数不能多于格式串中 9 的位数。

nls 用来指定以下字符串中待转换的格式元素：

1） 小数点字符。和 fmt 中 D 对应的分隔符。

2） 组分隔符。和 fmt 中 G 对应的分隔符，用于分隔千、百万、十亿……之间的符号。

3） 本地货币符号。

4） 国际货币符号。

nls 书写形式如下：

```
NLS_NUMERIC_CHARACTERS = "<小数点字符><组分隔符>"
```

```
NLS_CURRENCY = "<本地化货币符号>"

NLS_ISO_CURRENCY ="<国际货币符号>""
```

nls 参数字符串如果包含空格，要用单引号括起来；如果包含单引号，也要用单引号括起来，以对单引号进行转义。

NLS_NUMERIC_CHARACTERS 参数用于指定 fmt 中字符 D 和 G 代表小数点字符和组分隔符，必须用引号引起来。NLS_NUMERIC_CHARACTERS 串的长度只能是两个，并且这两个字符不能相同。

NLS_CURRENCY 指定的字符串用来代替本地货币符号，例如￥、人民币等。当 fmt 的前缀或后缀中有 L 时有效（fmt 的具体说明请查看数值函数 to_char），不能超过 10 个字符的长度。

NLS_ISO_CURRENCY 用来指定的字符串用来代替国际货币符号，仅当 FMT 的前缀中有 C 时有效，取值只能是表 8.1.9 中国际货币符号一列的值，得到的结果是缩写的内容，国际货币符号及货币名称释义中给出了国际货币符号和货币缩写的中文释义。

存在使用注意事项：

（1）to_number 的 nls 支持由用户自定义，但暂不支持解析具有二义性的字符，即 to_number 的 NLS_NUMERIC_CHARACTERS 和 NLS_CURRENCY 暂不支持指定 0~9、<、>、+、-、E、e 等字符；

（2）由于 DM 对于 to_char 函数第三个参数 nls 不做(1)中的限制，故 to_number 无法识别由指定了(1)中所述字符的 to_char 函数转换出来的字符串；

（3）对于 fmt 格式串存在限制如下：

- 若 fmt 格式串中没有指定元素"."或"D"，则即使 fmt 格式串中指定了元素","，在源串中对于千分号的匹配仍需要以 NLS_NUMERIC_CHARACTERS 指定的组分隔符为准；
- 若 fmt 中指定使用了元素"."，则在源串中解析千分号只能使用","，不能使用"G"；即

在 fmt 中"."需要和","搭配使用，"D"需要和"G"搭配使用，两种方式不能混用；

- 若 fmt 中指定使用了元素"D"，则源串解析千分号只能使用 NLS_NUMERIC_CHARACTERS 指定的组分隔符。

例 1 使用 9、G、D 来转换字符串'2,222.22'。

```
SELECT TO_NUMBER('2,222.22', '9G999D99');
```

查询结果为：2222.22

例 2 使用 9、,(逗号)、.(小数点)来转换字符串'2,222.22'。

```
SELECT TO_NUMBER('2,222.22', '9,999.99');
```

查询结果为：2222.22

例 3 使用 9、G、.(小数点)，由于在 fmt 中混用了 G 和.(小数点)，转换时报错。

```
SELECT TO_NUMBER('2,222.22', '9G999.99');
```

查询结果报错：[-6129]:无效的数字格式掩码

例 4 使用$、9、,(逗号)、.(小数点)来转换字符串'2,222.22'。

```
SELECT TO_NUMBER('$2,222.22','$9G999D99');
```

查询结果为：2222.22

例 5 使用 S、9 和.(小数点)来转换字符串'2,222.22'。

```
SELECT TO_NUMBER('-1212.12','S9999.99');
```

查询结果为：-1212.12

例 6 使用 XXXX 来转换字符串'1,234'。

```
SELECT TO_NUMBER('1,234','XXXX');
```

查询结果为：4660

例 7 无格式转换。

```
SELECT TO_NUMBER('-123.4');
```

查询结果为：-123.4

例 8 设置兼容 Postgres，转换字符串'12454.8-'，字符串中不含千分号。

```
SELECT TO_NUMBER('12454.8-', '99G999D9S');
```

查询结果为：-12454.8

例 9 设置兼容 Postgres，转换字符串'124,54.8-'，字符串中千分号与 G 位置不对应。

```
SELECT TO_NUMBER('124,54.8-', '99G999D9S');
```

查询结果为：-12454.8

例 10 使用 9、L、G、'NLS_NUMERIC_CHARACTERS="^#"'、'NLS_CURRENCY = "￥"'来转换字符串'￥12#699^2'。

```
SELECT TO_NUMBER('￥12#699^2', 'L99G999D99', 'NLS_NUMERIC_CHARACTERS="^#"
NLS_CURRENCY = "￥"') FROM DUAL;
```

查询结果为：12699.2

33. 函数 TRUNC

```
语法：TRUNC(n [,m])
    或
    TRUNC(str [,m]) //str 内只能为数字和'-'、'+'、'.'的组合
```

功能：将数值 n 的小数点后第 m 位以后的数全部截去。当数值参数 m 为负数时表示将数值 n 小数点前的第 m 位后的所有数截去。当数值参数 m 省略时，m 缺省为 0。支持对带符号的数值或字符串类型数值进行操作，例如：+11.2、'-8.9'。对字符串类型数值使用 trunc 函数，结果的有效数字为 16 位；由于过程中会先将字符串类型数值转换为 double 类型，会发生精度丢失，截取后的第 16 位数字可能与预期有偏差。

特殊说明：当 m 为负数，其绝对值大于或等于 n 的整数位个数时，结果取 0；当 m 取正数，其值大于等于 n 的小数位个数时，结果取 n。

例 1 对 PRODUCTION.PRODUCT 表中的价格的平方根使用 TRUNC 函数

```
SELECT SQRT(NOWPRICE), TRUNC(SQRT (ROUND(NOWPRICE) ),1)FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
SQRT(NOWPRICE)                    TRUNC(SQRT(ROUND(NOWPRICE)),1)
------------------------ ------------------------------
3.898717737923585E+000        3.8
3.781534080237808E+000        3.7
2.469817807045694E+000        2.4
4.658325879540846E+000        4.6
4.472135954999580E+000        4.4
6.140032573203500E+000        6.1
5.049752469181039E+000        5
3.376388603226827E+000        3.3
3.331666249791536E+000        3.3
6.480740698407860E+000        6.4
```

例 2 对数值使用 TRUNC 函数

```
SELECT    TRUNC(15.167,-1);
```

查询结果为: 10

例 3 对带符号字符串型数值'-14.1111'使用 TRUNC 函数

```
SELECT    TRUNC('-14.1111',-1);
```

查询结果为: -1.000000000000000E+001

34. 函数 TRUNCATE

```
语法: TRUNCATE(n [,m])
```

```
或

TRUNCATE(str [,m]) //str 内只能为数字和'-'、'+'、'.'的组合
```

功能：等价于函数 TRUNC。将数值 n 的小数点后第 m 位以后的数全部截去。当数值参数 m 为负数时表示将数值 n 小数点前的第 m 位后的所有数截去。当数值参数 m 省略时，m 默认为 0。支持对带符号的数值或字符串类型数值进行操作，例如：+11.2、'-8.9'。

特殊说明：当 m 为负数，其绝对值大于或等于 n 的整数位个数时，结果取 0；当 m 取正数，其值大于等于 n 的小数位个数时，结果取 n。

例 1 对数值使用 TRUNCATE 函数

```
SELECT    TRUNCATE(123.456,1);
```

查询结果为：123.4

例 2　对带符号字符串型数值'-14.1111'使用 TRUNCATE 函数

```
SELECT    TRUNCATE ('-14.1111',-1);
```

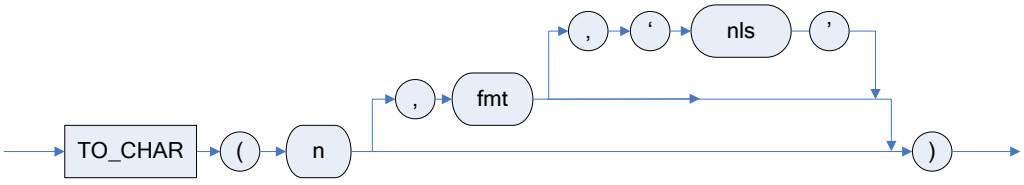
查询结果为：-1.000000000000000E+01

35. 函数 TO_CHAR

```
语法：TO_CHAR(n [, fmt [, 'nls' ] ])
```

图例

函数 TO_CHAR（数值类型）



语句功能：

将数值类型的数据转化为 VARCHAR 类型输出。其中：n 为数值类型的数据；fmt 为目标格式串。

fmt 中包含的格式控制符可分为三类：前缀标记、主体标记和后缀标记。前缀标记通常写在主体标记之前，执行产生的效果位于输入串之前；主体标记为必选项，通常产生的效果是为输入串增加格式效果，可与前缀标记和后缀标记组合使用；后缀标记通常写在主体标记之后，执行产生的效果位于输入串之后。

书写标记时，需要避免标记之间的冲突。冲突通常包括前缀之间的冲突、前缀与后缀之间的冲突、后缀与前缀之间的冲突、后缀之间的冲突四种。会产生冲突的情况将在下文中做出具体说明。建议用户按照标记的前后顺序书写：首先前缀标记，可选项；其次主体标记，必选项；最后后缀标记，可选项。例如：SELECT TO_CHAR('-1997','$999999MI');执行结果为：$1997-。

三类标记中，主体标记可以单独使用，而前缀标记或后缀标记必须要与主体标记组合才能使用。例如：SELECT TO_CHAR(-1234,'TME'),TO_CHAR(-1234,'9999');执行结果分别为：-1.234E+03 和-1234。

fmt 缺省时，fmt 格式为数字的字符串本身。例如：SELECT TO_CHAR(11.18);该语句的执行结果为：11.18。

主体标记包含的标记如表 8.1.2 所示。主体标记可单独使用，也可和其他类型标记组合使用。一个 fmt 中可包含一种或多种主体标记。

下表中，对 0 和 9 的使用需注意以下情况：

1）数据对齐：当 fmt 中没有点号（.）或 D 时，数据在舍弃小数部分后从右到左对齐，当 fmt 中存在点号（.）或 D 时，数据优先对齐小数点。

2）0/9 功能变化：对于点号（.）、D 和 V 与 0/9 合并使用时，在这些符号之前的 0/9 按照各自规则处理数据，在这些符号之后的 0/9，位数不足时统一按照 0 处理。另外，当 9 未填充且其前有 0 时，该位按照 0 处理。

表 8.1.2 主体标记

格式控制符	说明

逗号（,）	逗号可出现在整数部分（除了头部以外）的任意位置。例如：TO_CHAR(1234, '9,99,9')，结果为 1,23,4
点号（.）	作为小数点分隔符，不足的位数由后面的掩码决定
0	返回指定位数的值，位数不足时用0补齐。如果原值为正，则另带一个前导空格，如果为负，则在数字部分之前带一个前导负号
9	返回指定位数的值，位数不足时用空格补齐，如果原值为正，则另带一个前导空格，如果为负，则在数字部分之前带一个前导负号
D	表示小数点字符。用法等同于点号.
G	表示组分割符。用法等同于逗号,
X	表示16进制。X前只能使用0或FM，其它任何元素都会返回错误。 如果X前未使用0或FM，位数不足时用空格补齐，在返回值的基础上还要补充一个前导空格。
V	表示数据n乘以10的m次方（如果乘积含有小数，将四舍五入到整数），其中m是fmt中在掩码V后的9或0的个数
RN	转换为大写的罗马数字
rn	转换为小写的罗马数字
TM9	64个字符内返回原数值，超过则返回科学计数值。TM9仅可以分别与S、FM这2个前缀标记组合使用，例如：SELECT TO_CHAR(1997,'STM9')。除此以外，TM9不能与其它主体标记、后缀标记、主体和后缀标记组合使用，例如以下语句报错：SELECT TO_CHAR(1997,'TM99999')，SELECT TO_CHAR(1997,'TM9C')，SELECT TO_CHAR(1997,'TM99999C')
TME	返回科学计数值。TME仅可以分别与S、FM这2个前缀标记组合使用，例如：SELECT TO_CHAR(1997,'STME')。除此以外，TME不能与其它主体标记、后缀标记、主体和后缀

	标记组合使用，例如以下语句报错：SELECT TO_CHAR(1997,'TME9999')，SELECT TO_CHAR(1997,'TMEC')，SELECT TO_CHAR(1997,'TME9999C')

前缀标记包含的标记如表 8.1.3 所示。前缀标记均需要和其他类型标记组合使用。一个 fmt 中可包含一种或多种前缀标记。例如：SELECT TO_CHAR('-1997','S$9999');执行结果为：-$1997。

表 8.1.3 前缀标记

格式控制符	说明
FM	去掉前置空格
$	美元符号。只能有一个。在fmt书写时，$可写于主体标记的前面或后面。如TO_CHAR(1234, '$9999')， TO_CHAR(1234, '9999$')，TO_CHAR(-1234,'9999$PR')，以上语句的执行结果中$都位于数字主体之前
B	当整数部分的值为零时，返回空格
S	表示正负号， 如TO_CHAR(1234, 'S9999')结果为+1234， TO_CHAR(-1234,'S9999')结果为-1234
C	当前货币名称缩写
L	当前货币符号

后缀标记包含的标记如表 8.1.4 所示。后缀标记不可单独使用，必须和其他类型的标记组合使用（必须含有主体类型，前缀类型为可选项）。一个 fmt 中可包含一种或多种后缀标记。例如：SELECT TO_CHAR('-1997','999999EEEEMI');执行结果为：2E+03-。

表 8.1.4 后缀标记

格式控制符	说明
EEEE	科学计数符
MI	如'9999MI'，如果是负数，在尾部加上负号(-)；

	如果是正数和0，则尾部加上空格
PR	将负数放到尖括号<>中
C	当前货币名称缩写
L	当前货币符号
S	表示正负号

此外，正确书写 fmt 时，须避免标记之间冲突。三种标记之间的冲突主要包括以下几种：

1） 前缀之间的冲突；

2） 前缀与后缀的冲突；

3） 后缀与前缀的冲突；

4） 后缀之间的冲突。

- 前缀之间的冲突如表 8.1.5 所示。

表 8.1.5 前缀之间的冲突

前缀	**与指定前缀存在冲突的前缀**
$	$、C、L
B	B
S	$、B、S、C、L
FM	$、B
C	$、C、L
L	$、C、L

注：前缀之间的冲突指上表中第二列的前缀不能放在第一列的前缀之前。

如当使用前缀标记 S 时，S 之前不能再使用前缀标记$、B、S、C、L；而当使用前缀标记$时，因为未列出前缀标记$与前缀标记 S 之间的冲突，所以$之前可以使用前缀标记 S。例如，语句 SELECT TO_CHAR(1997,'$S9999');执行后报错；语句 SELECT TO_CHAR

(1997,'S$9999');执行后的结果为“+$1997”。

- 前缀与后缀之间的冲突如表 8.1.6 所示。

表 8.1.6 前缀与后缀之间的冲突

前缀	与指定前缀存在冲突的后缀
$	C、L、MI、PR、S。其中，C、L 不能和$同时使用。MI、PR 以及 S 作为后缀标记时，不能出现在$之前，例如：SELECT TO_CHAR('1977','9999S$');

- 后缀与前缀之间的冲突如表 8.1.7 所示。

表 8.1.7 后缀与前缀之间的冲突

后缀	与指定后缀存在冲突的前缀
L	L、C、$
C	L、C、$
S	S
PR	S
MI	S

如当后缀为 C 时，前缀中不能还有 L、C、$等标记，如格式'L999C'等。

- 后缀之间的冲突如表 8.1.8 所示。

表 8.1.8 后缀之间的冲突

后缀	与指定后缀存在冲突的后缀
EEEE	S、EEEE、MI、PR
S	S、MI、PR
PR	S、MI、PR
MI	S、MI、PR

C	C、L、MI、PR、S、EEEE
L	C、L、MI、PR、S、EEEE

注：后缀之间的冲突指上表中第二列的后缀不能放在第一列的后缀之前。

如当后缀为 L 时，后缀中不能还有 C、L、MI、PR、S、EEEE 等标记，即后缀 CL、LL、MIL、PRL、SL、EEEEL 不能在格式字符串中出现。

nls 用来指定以下数字格式元素返回的字符：

5） 小数点字符。和 FMT 中 D 对应的分隔符。

6） 组分隔符。和 FMT 中 G 对应的分隔符，用于分隔千、百万、十亿……之间的符号。

7） 本地货币符号。

8） 国际货币符号。

nls 书写形式如下：

```
'NLS_NUMERIC_CHARACTERS = "<小数点字符><组分隔符>"

NLS_CURRENCY = "<本地化货币符号>"

NLS_ISO_CURRENCY ="<国际货币符号>""
```

nls 参数字符串如果包含空格，要用单引号括起来；如果包含单引号，也要用单引号括起来，以对单引号进行转义。

NLS_NUMERIC_CHARACTERS 参数用于指定 fmt 中字符 D 和 G 代表小数点字符和组分隔符，必须用引号引起来。NLS_NUMERIC_CHARACTERS 串的长度只能是两个，并且这两个字符不能相同。

NLS_CURRENCY 指定的字符串用来代替本地货币符号，例如￥、人民币等。当 FMT 的前缀或后缀中有 L 时有效，不能超过 10 个字符的长度。

NLS_ISO_CURRENCY 用来指定的字符串用来代替国际货币符号，仅当 FMT 的前缀中有 C 时有效，取值只能是表 8.1.9 中国际货币符号一列的值，得到的结果是缩写的内容，国

际货币符号及货币名称释义中给出了国际货币符号和货币缩写的中文释义。

表 8.1.9 NLS_ISO_CURRENCY 的值及缩写形式

国际货币符号	结果中显示的缩写形式	国际货币符号及货币名称释义
CHINA	CNY	中国（人民币）
TAIWAN	TWD	台湾（新台币）
AMERICA	USD	美国（美元）
UNITED KINGDOM	GBP	英国（英镑）
CANADA	CAD	加拿大（加拿大元）
FRANCE	EUR	法国（欧元）
GERMANY	EUR	德国（欧元）
ITALY	EUR	意大利（欧元）
JAPAN	JPY	日本（日元）
KOREA	KRW	韩国（韩国元）
BRAZIL	BRL	巴西（巴西雷亚尔）
PORTUGAL	EUR	葡萄牙（欧元）

举例说明

例 1

```
SELECT TO_CHAR('01110' + 1);
```

查询结果如下：

```
行号        TO_CHAR('01110'+1)

---------- ------------------

1           1111
```

例 2

```
SELECT TO_CHAR(-10000,'L99G999D99MI') "Amount";
```

查询结果如下:

```
行号         Amount

---------- ------------

1            ￥10,000.00-
```

例 3

```
CREATE TABLE T_INT (C1 INT);

INSERT INTO T_INT VALUES(456),(0),(-213),(123456789);

SELECT TO_CHAR(C1,'L999G999G999D99PR','NLS_NUMERIC_CHARACTERS=": _"

NLS_CURRENCY = "￥"') AS TO_CHAR FROM T_INT;
```

查询结果如下:

```
行号         TO_CHAR

---------- ------------------

1                      ￥456:00

2                          ￥:00

3                     <￥213:00>

4             ￥123_456_789:00
```

例 4

```
SELECT TO_CHAR(C1,'C999G999G999D99PR','NLS_NUMERIC_CHARACTERS=": _"

NLS_ISO_CURRENCY = "CHINA"') AS TO_CHAR FROM T_INT;
```

查询结果如下:

```
行号         TO_CHAR

---------- -------------------

1                     CNY456:00

2                         CNY:00
```

```
3                    <CNY213:00>

4              CNY123_456_789:00
```

36. 函数 BITAND

```
语法：BITAND(n1, n2)
```

功能：返回两个数值型数值 n1 和 n2 按位进行 AND 运算后的结果。

特殊说明：当 n1 或 n2 是小数时，去掉小数点后做 AND 运算；如果 n1 或 n2 有一个是 0，则结果是 0；如果 n1 或 n2 有一个是 null，则结果是 null。

例

```
SELECT BITAND(-4, -5);
```

查询结果为：-8

37. 函数 NANVL

```
语法：NANVL(n1, n2)
```

功能： 当 n1 为 NaN 时返回 n2，否则返回 n1。此函数功能仅能处理浮点数类型，处理其他数据类型时结果为 n1。

对于 n1 和 n2 的数据类型的转换，有如下规则：

1） 任一参数为 double 类型时，若另一参数不是 double 类型，转为 double;

2） 无 double 类型参数，任一参数为 real 类型时，若另一参数不是 real 类型，转为 real;

3） 剩余情形，都转为 decimal。

例 1 n1 不是 Nan，结果为 n1。

```
SQL> select nanvl(1e-1, 2e-1);



行号          NANVL(1E-1,2E-1)
```

```
---------- -------------------------

1              1.000000000000000E-001
```

例 2　n1 是 Nan，结果为 n2，使用特殊值 Nan 需要将 ini 参数 FLOAT_MODE 的取值置为 1。

```
SQL> select nanvl(cast('Nan' as double), 2e-1);


行号          NANVL(CAST('Nan'ASDOUBLE),2E-1)

---------- -------------------------------

1              2.000000000000000E-001
```

38. 函数 REMAINDER

```
语法：REMAINDER(n1,  n2)
```

功能：计算 n1 除 n2 的余数，余数取绝对值更小的那一个。

例

```
SELECT REMAINDER(11,4)FROM DUAL;
```

查询结果为：-1.000000000000000E+000

39. 函数 TO_BINARY_FLOAT

```
语法：TO_BINARY_FLOAT(n)
```

功能：将 number、real 或 double 类型数值转换成 float 类型。

例

```
SELECT TO_BINARY_FLOAT(12) FROM DUAL;
```

查询结果为：1.2000000E+001

40. 函数 TO_BINARY_DOUBLE

```
语法：TO_BINARY_DOUBLE(n)
```

功能：将 number、real 或 float 类型数值转换成 double 类型。

例

```
SELECT TO_BINARY_DOUBLE(12) FROM DUAL;
```

查询结果为：1.200000000000000E+001

41. 函数 BIN_TO_NUM

```
语法：BIN_TO_NUM(n1{,n2})
```

功能：将输入参数表示的二进制数值转换成十进制 number 类型。参数的数量可以是任意个，取值只能为 0 或 1，参数支持隐式转换。

例

```
SELECT BIN_TO_NUM(1, 1, 0, 1) FROM DUAL;
```

查询结果为：13

42. 函数 FOUND_ROWS

```
语法：FOUND_ROWS()
```

功能：获取查询语句的查询结果在被<LIMIT 限定条件>或<TOP 子句>筛选之前满足条件的总行数。FOUND_ROWS 处理的对象是包含<LIMIT 限定条件>或<TOP 子句>的查询语句。<LIMIT 限定条件>或<TOP 子句>用于对满足条件的记录做进一步筛选，并得到最终的查询结果。

FOUND_ROWS 使用步骤：

一 必须和 SQL_CALC_FOUND_ROWS 字段搭配使用，在查询语句中嵌入 SQL_CALC_FOUND_ROWS 字段。使用 SQL_CALC_FOUND_ROWS 之后，系统会记录该查询语句被筛选之前满足条件的记录总行数。

二 获取最近一次执行的查询语句数据。

使用 FOUND_ROWS 获取最近一次执行的查询语句，被筛选之前满足条件的记录总行数。通常用于复杂的查询语句中，不需要再执行一条获取筛选之前总行数的查询语句即可获得数据，可提升查询性能。

例 使用 FOUND_ROWS 函数查询记录数。

首先，数据准备。

```
DROP TABLE T;
CREATE TABLE T(ID INT,SEC INT);
INSERT INTO T SELECT LEVEL,LEVEL*10 CONNECT BY LEVEL <=24;
```

其次，在查询语句中嵌入 SQL_CALC_FOUND_ROWS 字段。查询语句必须是包含<LIMIT 限定条件>或<TOP 子句>的查询语句。下面语句的查询结果为 10 行。

```
SELECT SQL_CALC_FOUND_ROWS  *  FROM T WHERE ID > 10 LIMIT 10;
或
SELECT SQL_CALC_FOUND_ROWS  TOP 10 * FROM T WHERE ID > 10;
```

最后，使用 FOUND_ROWS()查询。FOUND_ROWS 记录了满足条件的查询结果未筛选（不执行 LIMIT 10 和 TOP 10）之前总行数，即等价于 SELECT COUNT(*) FROM T WHERE ID > 10;。

```
SELECT FOUND_ROWS();
```

查询结果为：14

43. 函数 CONV

```
语法：CONV(N, FROM_BASE, TO_BASE)
```

功能：在不同进制之间转换数字，其中 n 为字符串类型，若为空串则返回 NULL，若全为空格则返回 0；from_base 和 to_base 为整数类型，取值范围是[2,36]和[-36,-2]，不在此范围内时结果返回 NULL，from_base 为正数则将 n 视为无符号数，反之视为有符

号数；to_base 为正数则转换结果为无符号数，反之为有符号数。函数中任意参数为 NULL 则返回 NULL。

当进制为 base 时，单个位上表示的范围应为[0,abs(base)-1]，若 base 的绝对值大于 10，则超过 9 的部分将用大小写字母'a'至'z'表示。例如当进制为 20 或-20 时，单个数值位上的有效字符为'0'到'9'，'a'到'j'，'A'到'J'。若 n 中的字符不为 from_base 进制的有效字符，则仅使用无效字符之前的部分做转换。

在实际转换中，会先将 n 转换为 2 进制格式的中间数，该中间数的精度为 64 位，若转换后的数值大小超过了 64 位精度下能表示的范围，则使用当前精度的最大值/最小值。例如在 conv(n,-10,10)的情况下，中间数的最大值为 9223372036854775807（对应 64 个'1'），最小值为-9223372036854775808（对应 1 个'1'和 63 个'0'）。

若 n 的第一个字符为'-'，则将 n 视为负数。当 n 为有符号数而 from_base 对应无符号数，则将 2 进制的中间数直接按无符号数的编码规则进行解析。例如对于 conv('-1',10,16)，-1 在 64 位的 2 进制下表示为'1111111111111111111111111111111111111111111111111111111111111111'，按无符号规则则代表十进制下的 18446744073709551615，故结果按十六进制输出返回'FFFFFFFFFFFFFFFF'。

例

```
SELECT CONV('A',16,2);
```

查询结果为：'1010'。

```
SELECT CONV('99A',10,10);
```

查询结果为：'99'。十进制下'A'不作为有效字符，仅转换'99'部分

```
SELECT CONV('20000000000000000000',10,10);
```

查询结果为：'18446744073709551615'。from_base 为正数，n 被视为无符号数，此时最大中间数的值为 18446744073709551615，再将该值转换为 to_base 的 10 进制。

```
SELECT CONV('-1',10,10);
```

查询结果为：'18446744073709551615'。

44. 函数 FORMAT

```
语法：FORMAT (X,D)
```

功能：将 x 四舍五入到 d 位，再将 x 按'#,###,###.##'格式返回。x 为 decimal 类型，d 为 int 类型。

例

```
SELECT FORMAT(1234.56789, 2);
```

查询结果为：'1,234.57'。

```
SELECT FORMAT(1234.56789, 0);
```

查询结果为：'1,235'

8.2 字符串函数

字符串函数一般接受字符类型(包括 CHAR 和 VARCHAR)和数值类型的参数，返回值一般是字符类型或是数值类型。

1. 函数 ASCII

```
语法：ASCII(char)
```

功能：返回字符 char 对应的整数（ASCII 值）。

例

```
SELECT    ASCII('B') ,ASCII('中');
```

查询结果为：66 54992

2. 函数 ASCIISTR

```
语法：ASCIISTR (char)
```

功能：将字符串 char 中，非 ASCII 的字符转成\XXXX(UTF-16)格式，ASCII 字符保持不变。

例 非 unicode 库下，执行如下操作：

```
SELECT CHr(54992),ASCIISTR('中') ,ASCIISTR(CHr(54992));
```

查询结果为：中 \4E2D \4E2D

3. 函数 BIT_LENGTH

```
语法：BIT_LENGTH(char)
```

功能：返回字符串的位(bit)长度。

例

```
SELECT BIT_LENGTH('ab');
```

查询结果为：16

4. 函数 CHAR

```
语法：CHAR(n)
```

功能：返回整数 n 对应的数据库字符集所对应字符。如果结果包含不完整的字符则会报错字符串不完整错误；如果结果包含多个字符则会报无效的参数错误。

例

```
SELECT    CHAR(66),CHAR(67),CHAR(68) , CHAR(54992);
```

查询结果为：B C D 中

5. 函数 CHAR_LENGTH / CHARACTER_LENGTH

```
语法：CHAR_LENGTH(char) 或 CHARACTER_LENGTH(char)
```

功能：返回字符串 char 的长度，以字符作为计算单位，一个汉字作为一个字符计算。

字符串尾部的空格也计数。

例 1

```
SELECT  NAME,CHAR_LENGTH(TRIM(BOTH ' ' FROM NAME)) FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                              CHAR_LENGTH(TRIM(BOTH"FROMNAME))

--------------------------  --------------------------------

红楼梦                            3

水浒传                            3

老人与海                          4

射雕英雄传(全四册)                10

鲁迅文集(小说、散文、杂文)全两册  17

长征                               2

数据结构(C 语言版)(附光盘)        15

工作中无小事                       6

突破英文基础词汇                   8

噼里啪啦丛书(全 7 册)             11
```

例 2

```
SELECT CHAR_LENGTH('我们');
```

查询结果为：2

6. 函数 CHR

```
语法：CHR(n)
```

功能：返回整数 n 对应的数据库字符集所对应字符。等价于 CHAR(n)。

7. 函数 NCHR

```
语法：NCHR(n)
```

功能：返回整数 n 对应的数据库字符集所对应字符。等价于 CHAR(n)。

8. 函数 CONCAT

```
语法：CONCAT(char1,char2,char3…)
```

功能：返回多个字符串顺序联结成的一个字符串，功能与连接符||相同。参数中有 NULL 时，将取值为 NULL 的参数当作空串，参数全为 NULL 时返回 NULL。

兼容以下规则：

1） SQL Server 规则：参数中有 NULL 时，将取值为 NULL 的参数当作空串，参数全为 NULL 时返回空串。

2） MySQL 规则：参数中有 NULL 时返回 NULL。

3） Oracle 规则：函数 CONCAT 支持两个及以上参数，参数中有 NULL 时，将取值为 NULL 的参数当作空串，参数全为 NULL 时返回 NULL。

4） PG 规则：参数中有 NULL 时，将取值为 NULL 的参数当作空串，参数全为 NULL 时返回空串。

例

```
SELECT  PRODUCTID,NAME, PUBLISHER, CONCAT(PRODUCTID,NAME,PUBLISHER)  FROM
PRODUCTION.PRODUCT;
```

查询结果如下：

```
PRODUCTID NAME                    PUBLISHER CONCAT(PRODUCTID,NAME,PUBLISHER)
--------- ------------------  --------- ----------------------------
1           红楼梦                  中华书局          1 红楼梦中华书局
2           水浒传                  中华书局          2 水浒传中华书局
3           老人与海                上海出版社        3 老人与海上海出版社
```

```
4          射雕英雄传(全四册)        广州出版社      4 射雕英雄传(全四册)广州出版社
5          鲁迅文集(小说、散文、杂文)全两册          5 鲁迅文集(小说、散文、杂文)全两册
6          长征                      人民文学出版社  6 长征人民文学出版社
7          数据结构(C 语言版)(附光盘)  清华大学出版社  7 数据结构(C 语言版)(附光盘)清华大
学出版社
8          工作中无小事              机械工业出版社  8 工作中无小事机械工业出版社
9          突破英文基础词汇          外语教学与研究出版社  9 突破英文基础词汇外语教学与研究出
版社
10         噼里啪啦丛书(全 7 册)    21 世纪出版社    10 噼里啪啦丛书(全 7 册)21 世纪出版社
```

9. 函数 DIFFERENCE()

```
语法：DIFFERENCE(char1,char2)
```

功能：比较两个字符串的 SOUNDEX 值之间的差异，返回两个 SOUNDEX 值串同一位置出现相同字符的个数。

例

```
SELECT  DIFFERENCE('she', 'he');
```

查询结果为：3

10. 函数 INITCAP

```
语法：INITCAP(char)
```

功能：返回句子字符串中，每一个单词的第一个字母改为大写，其他字母改为小写。单词用空格分隔，不是字母的字符不受影响。

例

```
SELECT   INITCAP('hello world');
```

查询结果为：Hello World

11. 函数 INS

```
语法: INS(char1,begin,n,char2)
```

功能：删除在字符串 char1 中以 begin 参数所指位置开始的 n 个字符，再把 char2 插入到 char1 串的 begin 所指位置。begin 和 n 为数值参数。当 begin 为 0 或负数时，会将 begin 的值调整为 1；当 n 为 0 时，表示不删除 char1 中的任何字符，将 char2 插入到 begin 指定的位置，当 n 为负数时，效果与 n=0 时相同。

例

```
SELECT INS ('abcdefg',1,3, 'kkk') ;
```

查询结果为：kkkdefg

12. 函数 INSERT / INSSTR

```
语法: INSERT(char1,n1,n2,char2) / INSSTR(char1,n1,n2,char2)
```

功能：将字符串 char1 从 n1 的位置开始删除 n2 个字符，并将 char2 插入到 char1 中 n1 的位置。其中，INSERT 函数在 n1<=0 或者 n1>char1 的长度时返回原串；在 n2<0 时表示截断 n1 及其之后的字符。INSSTR 函数在 n1<=0 时，从第一个字符开始操作，等价于 n1 取 0；n1>char1 的长度时返回 char1+char2；在 n2<0 时不进行字符删除，等价于 n2 取 0。

例

```
SELECT INSERT('That is a cake',2,3, 'his') ;
```

查询结果为：This is a cake

13. 函数 INSTR

```
语法: INSTR(str1,str2[,n[,m]])
```

功能：返回 str1 中包含 str2 的特定位置。INSTR 从 str1 的左边开始搜索，开始位置是 n，如果 n 为负数，则搜索从 str1 的最右边开始，当搜索到 str2 的第 m 次出现时，返回所在位

置。如果 m 为非正数，则进行报错处理。如果 n 为 0，则返回结果为 0。n 和 m 的缺省值都为 1，即返回 str1 中第一次出现 str2 的位置，这时与 POSITION 相类似。如果从 n 开始没有找到第 m 次出现的 str2，则返回 0。n 和 m 以字符作为计算单位，一个西文字符和一个汉字都作为一个字符计算。

此函数中 str1 和 str2 可以是 CHAR、VARCHAR 或 CLOB/TEXT 数据类型，n 和 m 是数值类型。

例

```
SELECT INSTR('CORPORATE FLOOR', 'OR', 3, 2) "Instring";
```

查询结果为：14

```
SELECT INSTR('我们的计算机', '计算机',1,1);
```

查询结果为：4

14. 函数 INSTRB

```
语法：INSTRB(char1,char2[,n[,m]])
```

功能：返回从 char1 的第 n 个字节开始查找字符串 char2 的第 m 次出现的位置。INSTRB 从 char1 的左边开始搜索，开始位置是 n，如果 n 为负数，则搜索从 char1 的最右边开始，当搜索到 char2 的第 m 次出现时，返回所在位置。如果 m 为非正数，则进行报错处理。如果 n 为 0，则返回结果为 0。n 和 m 的缺省值都为 1，即返回 char1 中第一次出现 char2 的位置，这时与 POSITION 相类似。如果从 n 开始没有找到第 m 次出现的 char2，则返回 0。以字节作为计算单位，一个汉字根据编码类型不同可能占据 2 个或多个字节。

此函数中 char1 和 char2 可以是 CHAR 或 VARCHAR 数据类型，n 和 m 是数值类型。

例

```
SELECT INSTRB('CORPORATE FLOOR', 'OR', 3, 2) "Instring";
```

查询结果为：14

```
SELECT INSTRB('我们的计算机', '计算机',1,1);
```

查询结果为：7

15. 函数 INSTRC

```
语法：INSTRC(char1,char2[,n[,m]])
```

功能：返回从 char1 的第 n 个字节开始查找字符串 char2 的第 m 次出现的位置。INSTRC 从 char1 的左边开始搜索，开始位置是 n，如果 n 为负数，则搜索从 char1 的最右边开始，当搜索到 char2 的第 m 次出现时，返回所在位置。如果 m 为非正数，则进行报错处理。如果 n 为 0，则返回结果为 0。n 和 m 的缺省值都为 1，即返回 char1 中第一次出现 char2 的位置，这时与 POSITION 相类似。如果从 n 开始没有找到第 m 次出现的 char2，则返回 0。以完整字符作为计算单位，一个西文字符和一个汉字都作为一个字符计算（注：INSTRC 与 INSTR 表现完全相同，仅作兼容使用）。

此函数中 char1 和 char2 可以是 CHAR 或 VARCHAR 数据类型，n 和 m 是数值类型。

例

```
SELECT INSTRC('CORPORATE FLOOR', 'OR', 3, 2) "Instring";
```

查询结果为：14

```
SELECT INSTRC('我们的计算机', '计算机',1,1);
```

查询结果为：4

16. 函数 LCASE

```
语法：LCASE(char)
```

功能：返回字符串中，所有字母改为小写，不是字母的字符不受影响。

例

```
SELECT LCASE('ABC');
```

查询结果为：abc

17. 函数 LEFT / LEFTSTR

```
语法：LEFT(char,n) / LEFTSTR(char,n)
```

功能：当 n>0 时，返回字符串 char 最左边的 n 个字符组成的字符串；当 n<0 时，返回从 char 最左边开始的 length(char) + n 个字符，若 length(char) + n < 0 则返回空串。

特别的，当 INI 参数 COMPATIBLE_MODE 取值为 4 时，n<0 时等价于 n=0 的情况。

例 1 对表 PRODUCTION.PRODUCT 的 NAME 列使用 LEFT 函数

```
SELECT  NAME,LEFT(NAME,2)  FROM  PRODUCTION.PRODUCT;
```

查询结果如下：

```
NAME                                "LEFT"(NAME,2)

------------------------------ --------------

红楼梦                              红楼

水浒传                              水浒

老人与海                            老人

射雕英雄传(全四册)                  射雕

鲁迅文集(小说、散文、杂文)全两册    鲁迅

长征                                长征

数据结构(C 语言版)(附光盘)          数据

工作中无小事                        工作

突破英文基础词汇                    突破

噼里啪啦丛书(全 7 册)               噼里
```

例 2 对字符串使用 LEFT 函数

```
SELECT  LEFT ('computer science',10);
```

查询结果为：computer s

例 3 对字符串使用 LEFT 函数，且 n<0

```
SELECT  LEFT ('HELLO WORLD', -2);
```

查询结果为：HELLO WOR

```
SELECT  LEFT ('HELLO WORLD', -100);
```

查询结果为：

18. 函数 LEN

```
语法：LEN(char)
```

功能：返回给定字符串表达式的字符（而不是字节）个数，其中不包含尾随空格。

例

```
SELECT LEN ('hi,你好□□');
```

查询结果为：5

说明：□表示空格字符

19. 函数 LENGTH

```
语法：LENGTH(str)
```

功能：返回给定字符串表达式的字符（而不是字节）个数，其中包含尾随空格。str 可以为字符串或 CLOB 类型。若输入内容为非字符串或 CLOB 类型，会隐式转换为字符串类型再进行计算。

例

```
SELECT LENGTH('hi,你好□□');
```

查询结果为：7

说明：□表示空格字符

20. 函数 LENGTHC

语法：LENGTHC(char)

功能：返回给定字符串表达式的字符（而不是字节）个数，其中包含尾随空格。char 为字符串类型。

例

```
SELECT LENGTHC('123DMDPC');
```

查询结果为：8

21. 函数 LENGTH2

语法：LENGTH2(char)

功能：与函数 LENGTHC 相同。

例

```
SELECT LENGTH2('hi,world□□');
```

查询结果为：10

说明：□表示空格字符

22. 函数 LENGTH4

语法：LENGTH4(char)

功能：与函数 LENGTHC 相同。

例

```
SELECT LENGTH4('□□hi,你好□□');
```

查询结果为：9

说明：□表示空格字符

23. 函数 OCTET_LENGTH

```
语法：OCTET_LENGTH(char)
```

功能：返回字符串 char 的长度，以字节作为计算单位，一个汉字根据编码类型不同可能占据 2 个或多个字节。

例

```
SELECT OCTET_LENGTH('大家好') "Length in bytes";
```

查询结果为：6

24. 函数 LOCATE

```
语法：LOCATE(char,str[,n])
```

功能：返回字符串 char 在 str 中从位置 n 开始首次出现的位置。当数值参数 n 省略时，则从 str 的最左边开始查找；当数值参数 n 小于或等于 0 时，则返回 0；当数值参数 n 为大于 0 的浮点数时，对其进行四舍五入取整操作后再进行查找。其中参数 str 可以为 CLOB/TEXT 数据类型，支持的最大长度为 2G-1。

对于 NULL 和空串的处理如下：

- 当任意参数为 NULL 时，返回 NULL；
- 当设置参数 n 时：若 char 为空串，且 n 的取值在[0,str 长度+1]范围内时，若 n 为整数，则返回 n，若 n 为浮点数，则对 n 进行四舍五入取整操作后返回，否则返回 0；
- 当未设置参数 n 时：若 char 为空串，则返回 1；若 char 不为空串，则返回 0。

例

```
SELECT LOCATE('man', 'The manager is a man', 10);
```

查询结果为：18

```
SELECT LOCATE('man', 'The manager is a man');
```

查询结果为：5

25. 函数 LOWER

语法：LOWER(char)

功能：将字符串中的所有大写字母改为小写，其他字符不变。等价于 LCASE(char)。

26. 函数 LPAD

语法：LPAD(char1,length[,char2])

功能：在字符串 char1 的左边，依次加入 char2 中的字符，直到总长度达到 length，返回增加后的字符串。如果未指定 char2，缺省值为空格。length 应为正整数。如果 length 的长度比 char1 大，则返回 char2 的前(length-length(char1))个字符+char1，总长度为 length。如果 length 比 char1 小，则返回 char1 的前 length 个字符。长度以字节作为计算单位，一个汉字作为二个字节计算。当任一参数为 NULL 时，结果为 NULL。

对于空串和 length 为非正数的情况，LPAD 函数的返回值与 INI 参数 COMPATIBLE_MODE 的取值有关，具体返回值如下：

1. 当 COMPATIBLE_MODE=4 时（长度以字符作为计算单位，一个汉字作为一个字符计算），length 为小于 0 的整数时返回 NULL；char2 为空串且 char1 的字符数小于 length 时返回空串；length 等于 0 时返回空串；
2. 当 COMPATIBLE_MODE=7 时（长度以字符作为计算单位，一个汉字作为一个字符计算），length 为小于或等于 0 的整数时返回空串；char2 为空串且 char1 字符数小于 length 时不做填充，直接返回 char1；
3. 当 COMPATIBLE_MODE 为其他取值时，若 length 为小于或等于 0 的整数，返回 NULL；若 char1 或 char2 为空串时返回 NULL。

例

```
SELECT LPAD(LPAD('FX',19,'Teacher'),22,'BIG') "LPAD example";
```

查询结果为：BIGTeacherTeacherTeaFX

```
SELECT LPAD('计算机',8, '我们的');
```

查询结果为：我计算机

27. 函数 LTRIM

```
语法：LTRIM(str[,set])
```

功能：str 支持字符串类型和 CLOB 类型，CLOB 的最大长度由 INI 参数 CLOB_MAX_CALC_LEN 指定。删除字符串 str 左边起，出现在 set 中的任何字符，当遇到不在 set 中的第一个字符时返回结果。返回值类型与 str 类型保持一致。set 缺省为空格。

例

```
SELECT LTRIM('xyyxxxXxyLAST WORD', 'xy') "LTRIM example";
```

查询结果为：XxyLAST WORD

```
SELECT LTRIM('我们的计算机', '我们');
```

查询结果为：的计算机

28. 函数 POSITION

```
语法：POSITION(char1 IN char2) / POSITION(char1, char2)
```

功能：返回在 char2 串中第一次出现的 char1 的位置，如果 char1 是一个零长度的字符串，POSITION 返回 1，如果 char2 中 char1 没有出现，则返回 0。以字节作为计算单位，一个汉字根据编码类型不同可能占据 2 个或多个字节。

例

```
SELECT  POSITION('数' IN '达梦数据库');
```

查询结果为：5

29. 函数 REPEAT / REPEATSTR

```
语法：REPEAT(char,n) / REPEATSTR(char,n)
```

功能：返回将字符串重复 n 次形成的字符串。

例

```
SELECT REPEAT ('Hello ',3);
```

查询结果为：Hello Hello Hello

30. 函数 REPLACE

```
语法：REPLACE(str, search [,replace])
```

功能：str 为 CHAR、CLOB 和 TEXT 类型，search 和 replace 为字符串类型。在 str 中找到字符串 search，替换成 replace。若 replace 为空，则在 str 中删除所有 search。

例

```
SELECT  NAME,REPLACE(NAME, '地址', '地点') FROM PERSON.ADDRESS_TYPE;
```

查询结果如下：

```
NAME         REPLACE(NAME,'地址','地点')
-------- ----------------------------
发货地址   发货地点
送货地址   送货地点
家庭地址   家庭地点
公司地址   公司地点
```

31. 函数 REPLICATE

```
语法：REPLICATE(char,times)
```

功能：把字符串 char 自己复制 times 份。

例

```
SELECT  REPLICATE('aaa',3);
```

查询结果为：aaaaaaaaa

32. 函数 REVERSE

```
语法：REVERSE(char)
```

功能：将输入字符串的字符顺序反转后返回。

例

```
SELECT REVERSE('abcd');
```

查询结果：dcba

33. 函数 RIGHT / RIGHTSTR

```
语法：RIGHT(char,n) / RIGHTSTR(char,n)
```

功能：当 n>0 时，返回字符串 char 最右边 n 个字符组成的字符串；当 n=0 时，返回空串；当 n<0 时，返回从 char 最右边开始的 length(char) + n 个字符，若 length(char) + n < = 0 则返回空串。

特别的，当 INI 参数 COMPATIBLE_MODE 取值为 4 时，n<0 时等价于 n=0 的情况。

例 1 对表 PERSON.ADDRESS_TYPE 的 NAME 列使用 RIGHT 函数

```
SELECT  NAME, RIGHT (NAME,2) FROM PERSON.ADDRESS_TYPE;
```

查询结果如下：

```
NAME       "RIGHT"(NAME,2)

-------- ---------------

发货地址   地址

送货地址   地址

家庭地址   地址

公司地址   地址
```

例 2 对字符串使用 RIGHT 函数

```
SELECT RIGHTSTR('computer',3);
```

查询结果为：ter

例 3 对字符串使用 RIGHT 函数，且 n<0

```
SELECT RIGHTSTR('HELLO WORLD',-3);
```

查询结果为：LO WORLD

```
SELECT RIGHTSTR('HELLO WORLD',-100);
```

查询结果为：

34. 函数 RPAD

```
语法：RPAD(char1,length[,char2])
```

功能：在字符串 char1 的右边，依次加入 char2 中的字符，直到总长度达到 length，返回增加后的字符串。如果未指定 char2，缺省值为空格。length 应为正整数。如果 length 的长度比 char1 大，则返回 char1+char2 的前(length-length(char1))个字符，总长度为 length。如果 length 比 char1 小，则返回 char1 的前 length 个字符。长度以字节作为计算单位，一个汉字作为二个字节计算。当任一参数为 NULL 时，结果为 NULL。

对于空串和 length 为非正数的情况，RPAD 函数的返回值与 INI 参数 COMPATIBLE_MODE 的取值有关，具体返回值如下：

1. 当 COMPATIBLE_MODE=4 时（长度以字符作为计算单位，一个汉字作为一个字符计算），length 为小于 0 的整数时返回 NULL；char2 为空串且 char1 的字符数小于 length 时返回空串；length 等于 0 时返回空串；
2. 当 COMPATIBLE_MODE=7 时（长度以字符作为计算单位，一个汉字作为一个字符计算），length 为小于或等于 0 的整数时返回空串；char2 为空串且 char1 字符数小于 length 时不做填充，直接返回 char1；
3. 当 COMPATIBLE_MODE 为其他取值时，若 length 为小于或等于 0 的整数，返回 NULL；若 char1 或 char2 为空串时返回 NULL。

例

```
SELECT RPAD('FUXIN',11, 'BigBig') "RPAD example";
```

查询结果为：FUXINBigBig

```
SELECT RPAD('计算机',8, '我们的');
```

查询结果为：计算机我

35. 函数 RTRIM

```
语法：RTRIM(str[,set])
```

功能：str 支持字符串类型和 CLOB 类型，CLOB 的最大长度由 INI 参数 CLOB_MAX_CALC_LEN 指定。删除字符串 str 右边起出现的 set 中的任何字符，当遇到不在 set 中的第一个字符时返回结果。返回值类型与 str 类型保持一致。set 缺省为空格。

例

```
SELECT RTRIM('TURNERyXxxxyyyxy', 'xy') "RTRIM e.g.";
```

查询结果为：TURNERyX

```
SELECT RTRIM('我们的计算机','我计算机');
```

查询结果为：我们的

36. 函数 SOUNDEX

```
语法：SOUNDEX(char)
```

功能：

1. 返回一个表示英文字符串发音的字符串，结果形式为一个英文字符后跟若干数字；
2. 当INI参数COMPATIBLE_MODE=4时，返回完整结果，其他情况下结果长度为定长4，截取完整结果的前4个字符，完整结果不足4时在末尾补0；
3. 参数为NULL时返回NULL；
4. 对于原字符串中的非英文字符，当INI参数COMPATIBLE_MODE=3时，遇到非英文

字符则不再处理后续字符，其他情况下忽略原字符串中所有非英文字符；

5. 如果原字符串为空串或者不存在英文字符，当INI参数COMPATIBLE_MODE=0或2时返回NULL，当COMPATIBLE_MODE=3时返回"0000"，当INI参数COMPATIBLE_MODE=4时返回空串；

例

```
SELECT SOUNDEX('Hello');
```

查询结果为：H400

37. 函数 SPACE

```
语法：SPACE(n)
```

功能：返回一个包含 n 个空格的字符串。

例

```
SELECT SPACE(5);
```

查询结果为：□□□□□

```
SELECT CONCAT(CONCAT('Hello',SPACE(3)), 'world');
```

查询结果为：Hello□□□world

说明：□表示空格字符

38. 函数 STRPOSDEC

```
语法：STRPOSDEC(char)
```

功能：把字符串 char 中最后一个字节的值减一。

例

```
SELECT STRPOSDEC('hello');
```

查询结果为：helln

39. 函数 STRPOSDEC

```
语法：STRPOSDEC(char,pos)
```

功能：把字符串 char 中指定位置 pos 上的字节的值减一。

例

```
SELECT STRPOSDEC('hello',3);
```

查询结果为：heklo

40. 函数 STRPOSINC

```
语法：STRPOSINC(char)
```

功能：把字符串 char 中最后一个字节的值加一。

例

```
SELECT STRPOSINC ('hello');
```

查询结果为：hellp

41. 函数 STRPOSINC

```
语法：STRPOSINC (char,pos)
```

功能：把字符串 char 中指定位置 pos 上的字节的值加一。

例

```
SELECT STRPOSINC ('hello',3);
```

查询结果为：hemlo

42. 函数 STUFF

```
语法：STUFF(char1,begin,n,char2)
```

功能：删除在字符串 char1 中以 begin 参数所指位置开始的 n 个字符，再把 char2 插

入到 char1 的 begin 所指位置。begin 与 n 为数值参数。当 begin<=0 或者 begin>char1 的长度或者 n<0 时，返回 NULL。

例

```
SELECT STUFF('ABCDEFG',1,3, 'OOO');
```

查询结果为：OOODEFG

43. 函数 SUBSTR/SUBSTRING/MID

```
语法：SUBSTR(str[,m[,n]]) / SUBSTRING(char[ from m [ for n ]])/ MID(str,m[,n])
```

功能：返回 str/char 中从字符位置 m 开始的 n 个字符。若 m 为 0，则把 m 就当作 1 对待。若 m 为正数，则返回的字符串是从左边到右边计算的；反之，返回的字符是从 str/char 的结尾向左边进行计算的。如果没有给出 n，则返回 str/char 中从字符位置 m 开始的后续子串。如果 m 大于 str/char 的长度，返回空串；如果 n 小于 0，且 m 小于等于 str/char 的长度时，返回 NULL。如果 m 和 n 都没有给出，返回 str/char。函数以字符作为计算单位，一个西文字符和一个汉字都作为一个字符计算。其中 str 支持为字符串和 CLOB 类型。

例 1 对 PRODUCTION.PRODUCT 表中的 NAME 列使用 SUBSTRING 函数

```
SELECT NAME,SUBSTRING(NAME FROM 3 FOR 2) FROM PRODUCTION.PRODUCT;
```

查询结果如下：

```
NAME                              SUBSTRING(NAMEFROM3FOR2)
--------------------------  ------------------------
红楼梦                            梦
水浒传                            传
老人与海                          与海
射雕英雄传(全四册)                英雄
鲁迅文集(小说、散文、杂文)全两册  文集
```

```
长征
数据结构(C 语言版)(附光盘)                结构
工作中无小事                               中无
突破英文基础词汇                           英文
噼里啪啦丛书(全 7 册)                      啪啦
```

例 2 对字符串使用 SUBSTR 函数

```
SELECT SUBSTR('我们的计算机',3,4) "Subs";
```

查询结果为：的计算机

44. 函数 SUBSTRB

```
语法：SUBSTRB(char,m[,n])
```

功能：返回 char 中从第 m 字节位置开始的 n 个字节长度的字符串。若 m 为 0，则 m 就当作 1 对待。若 m 为正数，则返回的字符串是从左边到右边计算的；若 m 为负数，返回的字符是从 char 的结尾向左边进行计算的。若 m 大于字符串的长度，则返回空串。如果没有 n，则缺省的长度为整个字符串的长度。如果 n 等于 0，返回空串；如果 n 小于 0，则返回 NULL。

这里假设字符串 char 的长度为 len，如果 n 的值很大，超过 len － m，则返回的子串的长度为 len － m。

如果开始位置 m 不是一个正常的字符的开始位置，那么返回的结果是 k 个空格（k 的值等于下一个有效字符的开始位置和 m 的差），空格后面是有效字符；如果字符串的 m+n-1 的位置不是一个有效的字符，那么就以空格填充。也就是不截断字符。

例

```
SELECT SUBSTRB('达梦数据库有限公司',4,15);
```

查询结果为：□数据库有限公司

说明：□表示空格字符，下同。

字符串前面是一个空格，这是因为字符串'达梦数据库有限公司'的第 4 个字节不是一个完整的字符的开始，因此用空格代替。

```
SELECT SUBSTRB('我们的计算机',3,4) "Subs", LENGTHB( SUBSTRB('我们的计算机',3,4));
```

查询结果为：们的 4

```
SELECT SUBSTRB('ABCDEFG',3,3) "Subs";
```

查询结果为：CDE

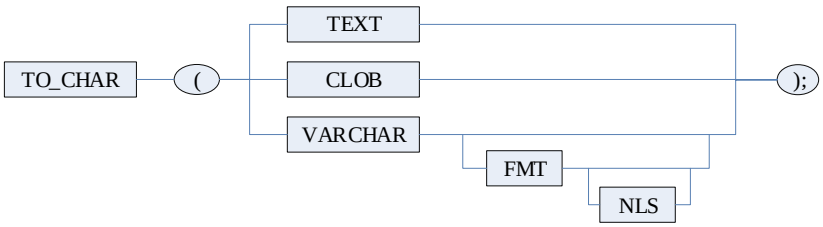
注意：函数 SUBSTRB 字节作为计算单位，一个字符在不同的编码方式下的字节长度是不同的。

45. 函数 TO_CHAR

```
语法： TO_CHAR(str)
```

图例

函数 TO_CHAR（str 可为 VARCHAR、CLOB、TEXT 类型）



功能：将 VARCHAR、CLOB、TEXT 类型的数据转化为 VARCHAR 类型输出。VARCHAR 类型的长度不能超过 32767 个字节，CLOB、TEXT 类型的长度不能超过 32766 个字节。

当参数类型为 VARCHAR 时，还可指定 FMT 与 NLS，FMT 和 NLS 的具体意义和限制可参见 8.1 数值函数的 TO_CHAR 函数介绍。

例

```
SELECT TO_CHAR('0110');
```

查询结果为：0110

```
CREATE TABLE T2(C1 VARCHAR(4000));
INSERT INTO T2 VALUES('达梦数据库有限公司成立于 2000 年，为国有控股的基础软件企业，专业从事数据库管理系统研发、销售和服务。其前身是华中科技大学数据库与多媒体研究所，是国内最早从事数据库管理系统研发的科研机构。达梦数据库为中国数据库标准委员会组长单位，得到了国家各级政府的强力支持。');
SELECT TO_CHAR(C1) FROM T2;
```

查询结果为：达梦数据库有限公司成立于 2000 年，为国有控股的基础软件企业，专业从事数据库管理系统研发、销售和服务。其前身是华中科技大学数据库与多媒体研究所，是国内最早从事数据库管理系统研发的科研机构。达梦数据库为中国数据库标准委员会组长单位，得到了国家各级政府的强力支持。

```
SELECT TO_CHAR('123','99,99','NLS_ISO_CURRENCY=CHINA');
```

查询结果为：1,23

46. 函数 TRANSLATE

```
语法：TRANSLATE(char,char_from,char_to)
```

功能：TRANSLATE 是一个字符替换函数。char、char_from和char_to分别代表一字符串。对于char字符串，首先，查找char中是否含有char_from字符串，如果找到，则将其含有的char_from与char_to中的字符一一匹配，并用char_to中相应的字符替换，直至char_from中的字符全部替换完毕。char_to中的不足或多余的字符，均视为空值。

例 1

```
SELECT TRANSLATE('我们的计算机', '我们的', '大世界');
```

查询结果为：大世界计算机　　('我'将被'大'替代,'们'将被'世'替代,'的'将被'界'替代)

```
SELECT TRANSLATE('我们的计算机', '我们的', '世界');
```

查询结果为：世界计算机　　('我'将被'世'替代,'们'将被'界'替代,'的'对应的是空值，将被

移走)

```
SELECT TRANSLATE('我们的计算机', '我们的', '大大世界');
```

查询结果为：大大世计算机　　('我'将被'大'替代,'们'将被'大'替代, '的'将被'世'替代，'界'对应的是空值，将被忽略)

例2

```
SELECT NAME,TRANSLATE (NAME,'发货','送货') FROM PERSON.ADDRESS_TYPE;
```

查询结果如下:

```
NAME              TRANSLATE(NAME,'发货','送货')

--------  ------------------------------

发货地址    送货地址

送货地址    送货地址

家庭地址    家庭地址

公司地址    公司地址
```

47. 函数 TRIM

```
语法：TRIM([<<LEADING|TRAILING|BOTH> [char] | char> FROM] str)
```

功能：str 支持字符串类型和 CLOB 类型，CLOB 的最大长度由 INI 参数 CLOB_MAX_CALC_LEN 指定。TRIM 从 str 的首端(LEADING)或末端(TRAILING)或两端(BOTH)删除 char 指定的字符，如果任何一个变量是 NULL，则返回 NULL。默认的修剪方向为 BOTH，默认的修剪字符为空格。函数返回值类型与 str 类型保持一致。

例 1 对 PERSON.ADDRESS_TYPE 表中的 NAME 列使用 TRIM 函数从末端删除'址'字符

```
SELECT   NAME,TRIM(TRAILING '址' FROM NAME) FROM PERSON.ADDRESS_TYPE;
```

查询结果如下:

```
NAME         TRIM(TRAILING'址'FROMNAME)
```

```
-------- --------------------------

发货地址    发货地

送货地址    送货地

家庭地址    家庭地

公司地址    公司地
```

例 2 对字符串使用 TRIM 函数，默认修剪方向为 BOTH（两端）

```
SELECT  TRIM( '   Hello World   ');
```

查询结果为：Hello World

例 3 对字符串使用 TRIM 函数，修剪方向为 LEADING（首端）

```
SELECT  TRIM(LEADING FROM '   Hello World   ');
```

查询结果为：Hello World□□□

说明：□表示空格字符，下同。

例 4 对字符串使用 TRIM 函数，修剪方向为 TRAILING（末端）

```
SELECT  TRIM(TRAILING FROM '   Hello World   ');
```

查询结果为：□□□Hello World

例 5 对字符串使用 TRIM 函数，修剪方向为 BOTH（两端）

```
SELECT  TRIM(BOTH FROM '   Hello World   ');
```

查询结果为：Hello World

48. 函数 UCASE

```
语法：UCASE(char)
```

功能：返回的字符串中，所有字母改为大写，不是字母的字符不受影响。

例

```
SELECT  UCASE('hello world');
```

查询结果为：HELLO WORLD

49. 函数 UPPER

```
语法：UPPER(char)
```

功能：返回的字符串中，所有字母改为大写，不是字母的字符不受影响。等价于UCASE(char)。

50. 函数 NLS_UPPER

```
语法：NLS_UPPER(char1 [,nls_sort=char2])
```

功能：将字符串 char1 中所有字母改为大写后返回，不是字母的字符不受影响。对于参数 char2，暂未支持相应功能，仅检查参数值的合法性，char2 参数的合法值与 NLSSORT 函数的 char2 参数相同，包括 BINARY、SCHINESE_PINYIN_M、SCHINESE_STROKE_M、SCHINESE_RADICAL_M、THAI_CI_AS 和 KOREAN_M。若输入两个参数且至少其中一个参数为空，则返回空值。若输入两个参数且 char2 合法，则当 char1 为空串时返回空串，当 char1 为空格时返回同样数量的空格。

例

```
SELECT NLS_UPPER('abcd123') FROM DUAL;
```

查询结果为：ABCD123

51. 函数 NLS_LOWER

```
语法：NLS_LOWER(char1 [,nls_sort=char2])
```

功能：将字符串 char1 中所有字母改为小写后返回，不是字母的字符不受影响。对于参数 char2，暂未支持相应功能，仅检查参数值的合法性，char2 参数的合法值与 NLSSORT 函数的 char2 参数相同，包括 BINARY、SCHINESE_PINYIN_M、SCHINESE_STROKE_M、SCHINESE_RADICAL_M、THAI_CI_AS 和 KOREAN_M。若输入两个参数且至少其中一个

参数为空，则返回空值。若输入两个参数且 char2 合法，则当 char1 为空串时返回空串，当 char1 为空格时返回同样数量的空格。

例

```
SELECT NLS_LOWER('AB CDe123') FROM DUAL;
```

查询结果为：ab cde123

52. 函数 REGEXP

REGEXP 函数是根据符合 POSIX 标准的正则表达式进行字符串匹配操作的系统函数，是字符串处理函数的一种扩展。使用该函数时需要保证 DM 安装目录的 bin 子目录下存在 libregex.dll（windows）或 libregex.so（linux）库文件，否则报错。

达梦支持的匹配标准如下表：

表 8.2.1 符合 POSIX 标准的正则表达式

语法	说明	示例
.	匹配任何除换行符之外的单个字符	d.m 匹配“dameng”
*	匹配前面的字符任意次	a*b 匹配“bat”中的“b”和“about”中的“ab”。
+	匹配前面的字符一次或多次	ac+ 匹配包含字母“a”和至少一个字母“c”的单词，如“race”和“ace”。
^	匹配行首	^car 仅当单词“car”显示为行中的第一组字符时匹配该单词
$	匹配行尾	end$ 仅当单词“end”显示为可能位于行尾的最后一组字符时匹配该单词
[]	字符集，匹配任何括号间的字符	be[n-t] 匹配“between”中的“bet”、“beneath”中的“ben”和“beside”中的“bes”，但不匹配“below”中的“bel”。
[^]	排除字符集。匹配任何不在括号间的字符	be[^n-t] 匹配“before”中的“bef”、“behind”中的“beh”和“below”中的“bel”，但是不匹配“beneath”中的“ben”。
(表达式)	标记正则表达式中的子正则表达式	(abc)+匹配“abcabcabc”
|	匹配 OR 符号（|）之前或之后的表达式。最常用在分组中。	(sponge|mud) bath 匹配“sponge bath”和“mud bath”。
\	按原义匹配反斜杠（\）之后的字符。这使您可以查找正则表达式表示法中使用的字符，如 { 和 ^。	\^ 搜索 ^ 字符

{n[,m]}	区间表达式，匹配在它之前的单个字符重现的次数区间。{n}指重复 n 次；{n,}为至少出现 n 次重复；{n,m}为重现 n 至 m 次	zo{2} 匹配“zoone”中的“zoo”，但不匹配“zozo”。
[[:alpha:]]	表示任意字母([a-z]+)	([A-Z]+)	
[[:digit:]]	表示任意数字\d ([0-9]+)	
[[:lower:]]	表示任意小写字母 ([a-z]+)	
[[:alnum:]]	表示任意字母和数字([a-z0-9]+)	
[[:space:]]	表示任意空格\s	
[[:upper:]]	表示任意大写字母([A-Z]+)	
[[:punct:]]	表示任意标点符号	
[[:xdigit:]]	表示任意 16 进制数([0-9a-fA-F]+)	
\w	表示一个数字、字母、下划线或中文字符	
\W	表示一个非数字、字母、下划线或中文字符	
\s	表示一个空格字符	
\S	表示一个非空格字符	
\d	表示一个数字字符	
\D	表示一个非数字字符	

值得注意的是，对于 Perl 规则的正则表达式达梦暂不支持：[==],{n}?, \A, \Z, *?, +?, ??, {n}?, {n,}?, {n,m}?。

DM8 支持的 REGEXP 函数如下表:

表 8.2.2 REGEXP 函数

序号	函数名	功能简要说明
1	REGEXP_COUNT(str, pattern[, position [, match_param]])	根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合正则表达式的子串的个数，并符合匹配参数 match_param
2	REGEXP_LIKE(str, pattern [, match_param])	根据 pattern 正则表达式，查找 str 字符串是否存在符合正则表达式的子串，并符合匹配参数 match_param
3	REGEXP_INSTR(str, pattern[, position[, occurrence [, return_opt [, match_param [, subexpr]]]]])	根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合 subexpr 正则表达式的子串，如果 return_opt 为 0，返回第 occurrence 次出现的位置，如果 return_opt 为大于 0，则返回该出现位置的下一个字符位置，并符合匹配参数。Subexpr 指定匹配的子正则表达式
4	REGEXP_SUBSTR(str, pattern [,position [, occurrence [,match_param[, subexpr]]]])	根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合 subexpr 正则表达式的子串，返回第 occurrence 次出现的子串，并符合匹配参数 match_param
5	REGEXP_REPLACE(str, pattern [, replace_str [, position [, occurrence [,match_param]]]])	根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合正则表达式的子串，并用 replace_str 进行替换第 occurrence 次出现的子串，并

		符合匹配参数 match_param

参数说明：

str：待匹配的字符串，支持字符串类型与 CLOB 类型，字符串最大长度为 32767 字节，CLOB 的最大长度由 INI 参数 CLOB_MAX_CALC_LEN 指定；

pattern：符合 POSIX 标准的正则表达式，最大长度为 512 字节；

position：匹配的源字符串的开始位置，正整数，默认为 1；

occurrence：匹配次数，正整数，默认为 1；

match_param：正则表达式的匹配参数，默认大小写敏感，如下表所示：

表 8.2.3 匹配参数

值	说明
c	表示大小写敏感。例如：REGEXP_COUNT('AbCd', 'abcd', 1, 'c')，结果为 0
i	表示大小写不敏感。例如：REGEXP_COUNT('AbCd', 'abcd', 1, 'i')，结果为 1
m	将源字符串当成多行处理，默认当成一行。 例如：REGEXP_COUNT('ab'||CHR(10)||'ac', '^a.', 1, 'm')，结果为 2
n	通配符（.）匹配换行符，默认不匹配。 例如：REGEXP_COUNT('a'||CHR(10)||'d', 'a.d', 1, 'n')，结果为 1
x	忽略空格字符。例如：REGEXP_COUNT('abcd', 'a b c d', 1, 'x')，结果为 1

return_opt：正整数，返回匹配子串的位置。值为 0：表示返回子串的开始位置；值大于 0：表示返回子串结束位置的下一个字符位置；

subexpr：正整数，取值范围为：0~9，表示匹配 pattern 中的第 subexpr 个子正则表达式，子正则表达式必须是由括号标记的表达式。如果 subexpr=0，则表示匹配整个正则表达式；如果 subexpr > 0，则匹配对应的第 subexpr 个子正则表达式；如果 subexpr 大于子正则表达式个数或者 subexpr 为 NULL，则返回 NULL。

replace_str：用于替换的字符串，最大长度为 512 字节。

DM 的 REGEXP 函数支持正则表达式的反向引用，通过“\数字”的方式进行引用，如\1 表示第一个匹配的子表达式。

如下详细介绍各函数：

1） 函数 REGEXP_COUNT

```
语法：REGEXP_COUNT(str, pattern[, position [, match_param]])
```

功能：根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合正则表达式的子串的个数，并符合匹配参数 match_param。position 默认值为 1，position 为正整数，小于 0 则报错；如果 position 为空，则返回 NULL。pattern 必须符合正则表达式的规则，否则报错。match_param 不合法，则报错。

返回值：如果 str 和 pattern 其中有一个为空串或 NULL，则返回 NULL。如果不匹配，返回 0；如果匹配，返回匹配的个数。

例

```
SELECT REGEXP_COUNT('AbCd', 'abcd', 1, 'i') FROM DUAL;
```

查询结果为：1

```
SELECT REGEXP_COUNT('AbCd', 'abcd', 1, 'c') FROM DUAL;
```

查询结果为：0

2） 函数 REGEXP_LIKE

```
语法：REGEXP_LIKE(str, pattern [, match_param])
```

功能：根据 pattern 正则表达式，查找 str 字符串是否存在符合正则表达式的子串，并符合匹配参数 match_param。

返回值：如果匹配，则返回 1；否则返回 0。如果 str 和 pattern 中任一个为空串或 NULL，则返回 NULL；

例

```
SELECT 1 FROM DUAL WHERE REGEXP_LIKE('DM database V7', 'dm', 'c');
```

查询结果为：无返回行

```
SELECT 1 FROM DUAL WHERE REGEXP_LIKE('DM database V7', 'dm', 'i');
```

查询结果为：1

3） 函数 REGEXP_INSTR

```
语法：REGEXP_INSTR(str, pattern[, position[， occurrence [, return_opt [, match_param [, subexpr]]]]])
```

功能：根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合 subexpr 正则表达式的子串，如果 return_opt 为 0，返回第 occurrence 次出现的位置，如果 return_opt 为大于 0，则返回该出现位置的下一个字符位置，并符合匹配参数。Subexpr 指定匹配的子正则表达式。

返回值：如果 str、pattern、position、occurrence、return_opt 和 subexpr 中任一个为 NULL，则返回 NULL。否则返回符合条件的子串位置，如果没有找到，则返回 0。

例

```
SELECT REGEXP_INSTR('abcaaac','aa') FROM DUAL;
```

查询结果为：4

```
SELECT REGEXP_INSTR('abcaaac','aa',5) FROM DUAL;
```

查询结果为：5

```
SELECT REGEXP_INSTR('123%4567890', '(123)%(4(56)(78))', 1, 1, 0, 'i', 2) "REGEXP_INSTR" FROM DUAL;
```

查询结果为：5

4） REGEXP_SUBSTR

```
语法：REGEXP_SUBSTR(str, pattern [,position [, occurrence [,match_param[, subexpr]]]])
```

功能：根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合 subexpr 正则表达式的子串，返回第 occurrence 次出现的子串，并符合匹配参数

match_param。occurrence 默认为 1。如果 position 或 occurrence 的输入值不为正数，则报错。

返回值：如果 str、pattern、position、occurrence 和 subexpr 中任一个为 NULL，则返回 NULL。如果找到符合正则表达式的子串，则返回匹配的子串；如果没有找到，则返回 NULL。

例

```
SELECT REGEXP_SUBSTR('a 为 aa 了 aac','(a*)',2) FROM DUAL;
```

查询结果为：NULL

```
SELECT REGEXP_SUBSTR('a 为 aa 了 aac','(a+)',2) FROM DUAL;
```

查询结果为：aa

```
SELECT REGEXP_SUBSTR('500 DM8 DATABASE, SHANG HAI, CN', ', ([^,]+),', 5, 1, 'i', 1)
"REGEXPR_SUBSTR"  FROM DUAL;
```

查询结果为：SHANG HAI

5） REGEXP_REPLACE

```
语法：REGEXP_REPLACE(str, pattern [, replace_str [, position [, occurrence [,match_param]]]])
```

功能：根据 pattern 正则表达式，从 str 字符串的第 position 个字符开始查找符合正则表达式的子串，并用 replace_str 进行替换第 occurrence 次出现的子串，并符合匹配参数 match_param。occurrence 默认为 0，替换所有出现的子串。replace_str 默认为空串，在替换过程中，则相当于删除查找到的子串； position 默认值为 1，如果 position 的值不为正整数，则报错；

返回值：返回替换后的 str。如果 str、position 和 occurrence 中任一个为 NULL，则返回 NULL；如果 pattern 为 NULL，则返回 str；如果 str 中所有的字符都被空串替换，则返回 NULL，相当于删除所有的字符。

例

```
SELECT REGEXP_REPLACE('a 为了 aaac','aa','bb') FROM DUAL;
```

查询结果为：a 为了 bbac

```
SELECT REGEXP_REPLACE('a 为了 ac','aa','bb') FROM DUAL;
```

查询结果为：a 为了 ac

```
SELECT REGEXP_REPLACE('a 为 aa 了 aac','aa','bb') FROM DUAL;
```

查询结果为：a 为 bb 了 bbc

```
SELECT REGEXP_REPLACE('500 DM8 DATABASE, SHANG HAI, CN', ',[^,]+,', ', WU HAN,', 5, 1,'i')
"REGEXPR_REPLACE" FROM DUAL;
```

查询结果为：500 DM8 DATABASE, WU HAN, CN

```
SELECT REGEXP_REPLACE('www1234xxxx3q', '([[:alpha:]]+)', 'AAA\1') FROM DUAL;
```

此处使用了正则表达式的反向引用功能，查询结果为：AAAwww1234AAAxxxx3AAAq

53. 函数 OVERLAY

```
语法：OVERLAY(char1 PLACING char2 FROM m [ FOR n ])
```

功能：用串 char2（称为“替换字符串”）覆盖源串 char1 的指定子串，该子串是通过在源串中的给定起始位置的数值（m）和长度的数值（n）而指明，来修改一个串自变量。当子串长度为 0 时，不会从源串中移去任何串；当不指定 n 或指定 n 为 NULL 时，默认 n 为 char2 的长度。当指定 m<0 或 n<0 时，进行报错处理。m=0 等价于 m=1。指定 m>char1 的长度时，函数返回 char1+char2。函数的返回串是在源串的给定起始位置插入替换字符串所得的结果。当兼容 PG 时，即 COMPATIBLE_MODE=7 时，有以下规则：当不指定 n 时，n 为 char2 的长度；当指定 n 为 NULL 时，返回 null；当指定 n<0 时，按照 n 为 char2 的长度进行拼接，拼接完成后将 char2 之后的部分进行替换，替换内容为 char1 自（m+n）位置到末尾的字段，若 m+n<=0，则取 char1 作为替换内容； 当指定 m<=0 时，进行报错处理；当指定 m>char1 的长度时，返回 char1+char2（若此时 n<0，则在末尾添加一

段内容，添加内容的计算方式和上文中替换内容一致)。

例 1 对 PRODUCTION.PRODUCT 表的 NAME 列使用 OVERLAY 函数，将 NAME 列的数据从第三个字符开始的后两个字符替换成指定的'口'字符串

```
SELECT NAME,OVERLAY(NAME PLACING '口' FROM 3 FOR 2) FROM PRODUCTION.PRODUCT;
```

查询结果如下:

```
NAME                                "OVERLAY"(NAME,'口',3,2)
---------------------------------- ------------------------------
红楼梦                              红楼口
水浒传                              水浒口
老人与海                            老人口
射雕英雄传(全四册)                  射雕口传(全四册)
鲁迅文集(小说、散文、杂文)全两册    鲁迅口(小说、散文、杂文)全两册
长征                                长征口
数据结构(C 语言版)(附光盘)          数据口(C 语言版)(附光盘)
工作中无小事                        工作口小事
突破英文基础词汇                    突破口基础词汇
噼里啪啦丛书(全 7 册)               噼里口丛书(全 7 册)
```

例 2 对字符串使用 OVERLAY 函数，将源串的从第二个字符开始的后四个字符替换成指定的'hom'字符串

```
SELECT OVERLAY('txxxxas' PLACING 'hom' FROM 2 FOR 4);
```

查询结果为：thomas

54. 函数 TEXT_EQUAL

```
语法：TEXT_EQUAL(n1,n2)
```

功能：返回 n1，n2 的比较结果，完全相等，返回 1；否则返回 0。n1，n2 的类型为 CLOB、TEXT 或 LONGVARCHAR。如果 n1 或 n2 均为空串或 NULL，结果返回为 1；否则只有一个为空串或为 NULL，结果返回 0。不忽略结果空格和英文字母大小写。

例

```
SELECT TEXT_EQUAL('a', 'b');
```

查询结果为：0

```
SELECT TEXT_EQUAL('a','a');
```

查询结果为：1

55. 函数 BLOB_EQUAL

```
语法：BLOB_EQUAL(n1,n2)
```

功能：返回 n1，n2 两个数的比较结果，完全相等，返回 1；否则返回 0。n1，n2 的类型为 BLOB、IMAGE 或 LONGVARBINARY。如果 n1 或 n2 均为空串或 NULL，结果返回为 1；否则只有一个为空串或为 NULL，结果返回 0。

例

```
SELECT BLOB_EQUAL(0xFFFEE, 0xEEEFF);
```

查询结果为：0

```
SELECT BLOB_EQUAL(0xFFFEE, 0xFFFEE);
```

查询结果为：1

56. 函数 NLSSORT

```
语法：NLSSORT(char1 [,nls_sort=char2])
```

功能：返回对自然语言排序的编码。当只有 char1 一个参数时，与 RAWTOHEX 类似，返回 16 进制字符串。char2 决定按哪种方式排序：BINARY 表示按默认字符集二进制编码排序；SCHINESE_PINYIN_M 表示按中文拼音排序；SCHINESE_STROKE_M 表示按中文笔画

排序；SCHINESE_RADICAL_M 表示按中文部首排序；THAI_CI_AS 表示按泰文排序；KOREAN_M 表示按韩文排序。当 char2 为 BINARY 时，忽略第二个参数，等价于 NLSSORT(char1)。仅字符集为 UTF-8 的数据库支持自然语言按泰文排序。

用户可以通过 ALTER SESSION 语法（具体请参考 3.14.4 自然语言排序方式）设置 nls_sort 的参数值，修改后的参数值只对当前会话起作用，当函数 NLSSORT 只有 char1 一个参数时，当前会话默认使用 ALTER SESSION 设置的 nls_sort 的参数值。

例 1 使用 NLSSORT 函数返回'abc'的 16 进制字符串

```
SELECT NLSSORT('abc') FROM DUAL;
```

查询结果为：61626300

例 2 使用 NLSSORT 函数对表中的字符串 C1 列按中文拼音排序

```
CREATE TABLE TEST(C1 VARCHAR2(200));
INSERT INTO TEST VALUES('啊');
INSERT INTO TEST VALUES('不');
INSERT INTO TEST VALUES('才');
INSERT INTO TEST VALUES('的');
INSERT INTO TEST VALUES('一');
INSERT INTO TEST VALUES('二');
INSERT INTO TEST VALUES('三');
INSERT INTO TEST VALUES('四');
INSERT INTO TEST VALUES('品');
INSERT INTO TEST VALUES('磊');
SELECT * FROM TEST ORDER BY NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M');
//拼音
```

查询结果如下：

```
行号          C1
----------  --
1             啊
2             不
3             才
4             的
5             二
6             磊
7             品
8             三
9             四
10            一
```

例 3 使用 NLSSORT 函数对表中的字符串 C1 列按中文笔画排序

```
SELECT * FROM TEST ORDER BY NLSSORT(C1, 'NLS_SORT=SCHINESE_STROKE_M');
//笔画
```

查询结果如下:

```
行号          C1
----------  --
1             一
2             二
3             三
4             才
5             不
6             四
```

```
7              的
8              品
9              啊
10             磊
```

例 4 使用 NLSSORT 函数对表中的字符串 C1 列按中文部首排序

```
SELECT * FROM TEST ORDER BY NLSSORT(C1, 'NLS_SORT=SCHINESE_RADICAL_M');
//部首
```

查询结果如下:

```
行号           C1
---------- --
1              一
2              二
3              三
4              不
5              品
6              啊
7              四
8              才
9              的
10             磊
```

例 5 使用 NLSSORT 函数对表中的字符串 C1 列按中文拼音排序, 并返回 NLSSORT(C1), NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M')

```
SELECT C1,NLSSORT(C1),NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M') FROM TEST ORDER BY
NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M');
```

分别返回 c1，返回将 c1 转化后的 16 进制字符串，返回用来为汉字排序的编码。

查询结果如下：

```
C1  NLSSORT(C1) NLSSORT(C1,'NLS_SORT=SCHINESE_PINYIN_M')
--  ----------- ----------------------------------------
啊  B0A100          3B2C
不  B2BB00          4248
才  B2C500          4291
的  B5C400          4D8D
二  B6FE00          531D
磊  C0DA00              743E
品  C6B700          8898
三  C8FD00          932C
四  CBC400              996A
一  D2BB00          B310
```

例 6 使用 NLSSORT 函数对表中的字符串 C1 列按中文拼音排序，并返回 NLSSORT(C1, 'NLS_SORT=BINARY')

```
SELECT C1,NLSSORT(C1, 'NLS_SORT=BINARY') FROM TEST ORDER BY NLSSORT(C1,
'NLS_SORT=SCHINESE_PINYIN_M');
```

NLSSORT(C1, 'NLS_SORT=BINARY')等价于 NLSSORT(C1)。

查询结果如下：

```
C1  NLSSORT(C1,'NLS_SORT=BINARY')
--  -----------------------------
啊  B0A100
不  B2BB00
```

```
才  B2C500
的  B5C400
二  B6FE00
磊  C0DA00
品  C6B700
三  C8FD00
四  CBC400
一  D2BB00
```

例 7 使用 ALTER SESSION 语法设置 nls_sort 的参数值为 schinese_pinyin_m，NLSSORT 函数使用 ALTER SESSION 设置的 nls_sort 的参数值对 C1 进行排序，并返回 NLSSORT(C1)

```
ALTER SESSION SET NLS_SORT='SCHINESE_PINYIN_M';
SELECT C1,NLSSORT(C1) FROM TEST ORDER BY NLSSORT(C1);
```

查询结果如下：

```
C1  NLSSORT(C1)
--  ----------
啊  3B2C
不  4248
才  4291
的  4D8D
二  531D
磊  743E
品  8898
三  932C
```

```
四    996A

一    B310
```

例 8 使用 NLSSORT 函数对表中的字符串 C1 列按中文拼音排序，并返回 NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M')

```
SELECT C1,NLSSORT(C1, 'NLS_SORT=SCHINESE_PINYIN_M') FROM TEST ORDER BY NLSSORT(C1,
'NLS_SORT=SCHINESE_PINYIN_M');
```

查询结果如下：

```
C1    NLSSORT(C1,'NLS_SORT=SCHINESE_PINYIN_M')
--    ----------------------------------------
啊    3B2C
不    4248
才    4291
的    4D8D
二    531D
磊    743E
品    8898
三    932C
四    996A
一    B310
```

可以看出，上述两个 SQL 语句的查询结果一致。由于使用 ALTER SESSION 语法设置 nls_sort 的参数值为 schinese_pinyin_m，因此在当前会话中，当函数 NLSSORT 只有一个参数时，默认第二个参数 nls_sort 的值为 schinese_pinyin_m。

57. 函数 GREATEST

```
语法：GREATEST(char {,char})
```

功能：求一个或多个字符串中最大的字符串。

例

```
SELECT GREATEST('abb','abd', 'abc');
```

查询结果为：abd

58. 函数 GREAT

```
语法：GREAT (char1, char2)
```

功能：求 char1、char2 中最大的字符串。

例

```
SELECT GREAT ('abb','abd');
```

查询结果为：abd

59. 函数 TO_SINGLE_BYTE

```
语法：  TO_SINGLE_BYTE(
            STR IN VARCHAR
        )
```

功能：将多字节形式的字符（串）转换为对应的单字节形式

例

```
SELECT LENGTHB(TO_SINGLE_BYTE('aa'));
```

查询结果为：2

60. 函数 TO_MULTI_BYTE

```
语法：TO_MULTI_BYTE(
            STR IN VARCHAR
        )
```

功能：将单字节形式的字符（串）转换为对应的多字节形式（不同的字符集转换结果不同）

例

```
SELECT LENGTHB(TO_MULTI_BYTE('aa') );
```

查询结果为： 4

61. 函数 EMPTY_BLOB

```
语法：EMPTY_BLOB()
```

功能：初始化blob字段

返回值：长度为0的blob数据

例

```
DROP TABLE TT;

CREATE TABLE TT(C1 BLOB, C2 INT);

INSERT INTO TT VALUES(EMPTY_BLOB(),1);

INSERT INTO TT VALUES(NULL,2);

INSERT INTO TT VALUES(0X123,3);

SELECT LENGTHB(C1) FROM TT;
```

查询结果如下：

```
LENGTHB(C1)

--------------------

0

NULL

2
```

62. 函数 EMPTY_CLOB

```
语法：EMPTY_CLOB()
```

功能：初始化clob字段

返回值：长度为0的clob数据

例

```
DROP TABLE TT;

CREATE TABLE TT(C1 CLOB, C2 INT);

INSERT INTO TT VALUES(EMPTY_CLOB(),1);

INSERT INTO TT VALUES(NULL,2);

INSERT INTO TT VALUES('0X123',3);

SELECT LENGTHB(C1) FROM TT;
```

查询结果如下：

```
LENGTHB(C1)

-----------

0

NULL

5
```

63. 函数 UNISTR

```
语法：UNISTR (char)
```

功能：将字符串char中，ASCII编码或Unicode编码（‘\XXXX’4个16进制字符格式）转成本地字符。对于其他字符保持不变。

例 在 GB18030 库下，执行如下操作：

```
SELECT UNISTR('\803F\55B5\55B5kind 又\006e\0069\0063\0065') FROM DUAL;
```

查询结果为： 耿喵喵kind又nice

64. 函数 ISNULL

```
语法：ISNULL(char)
```

功能：判断表达式是否为NULL，为NULL返回1，否则返回0。

例 查询总经理的 MANAGERID 是否为空：

```
SELECT ISNULL(MANAGERID) FROM RESOURCES.EMPLOYEE WHERE TITLE='总经理';
```

查询结果为：1

65. 函数 CONCAT_WS

```
语法：CONCAT_WS(delim, char1,char2,char3,...)
```

功能：顺序联结多个字符串成为一个字符串，并用delim分割。

如果delim取值为NULL，则返回NULL。如果其它参数中存在NULL，则在执行拼接过程中跳过取值为NULL的参数。如果其它参数全部为NULL，则返回NULL。

兼容以下规则：

1） SQL Server 规则：分隔符（delim）为 NULL 时，将分隔符当作空串，非分隔符中有取值为 NULL 的参数时，将该参数当作空串。

2） MySQL 规则：分隔符（delim）为 NULL 时，返回 NULL，非分隔符中有取值为 NULL 的参数时，将该参数当作空串。

3） PG 规则：分隔符（delim）为 NULL 时，返回 NULL，非分隔符中有取值为 NULL 的参数时，在执行拼接过程中跳过 NULL，非分隔符全部为 NULL 时，返回空串。

例

```
SELECT CONCAT_WS(',,','11','22','33');
```

查询结果为：11,,22,,33

66. 函数 SUBSTRING_INDEX

```
语法：substring_index (char, char_delim, count)
```

功能：按关键字截取字符串，截取到指定分隔符出现指定次数位置之前。

char为被截取的字符串，char_delim为关键字符串，count为关键字出现的次数，为数值参数。如果count为负，则从后往前截取，截取到指定分隔符出现指定次数位置之后。

例

```
SELECT SUBSTRING_INDEX('blog.jb51.net', '.',2);
```

查询结果为：blog.jb51

```
SELECT SUBSTRING_INDEX('blog.jb51.net', '.',-2);
```

查询结果为：jb51.net

67. 函数 compose

```
语法：COMPOSE(char)
```

功能：用于在 UTF8 库下，将 char 以本地编码的形式返回。char 可为本地编码的字符串、UNISTR()函数的输出结果、或两者的组合值。

此外，将元音字符和 UNISTR()生成的特殊符号组合之后作为 char，经 COMPOSE()转化之后，会形成一个新的特殊字符。元音字符有：a、e、i、o、u、A、E、I、O、U。

使用 UNISTR()函数表示的特殊字符如下表所示。

表 8.2.4 使用 UNISTR()函数表示的特殊字符

UNISTR()函数	UNISTR()生成的特殊符号
UNISTR('\0300')	沉音符`
UNISTR('\0301')	重音符'
UNISTR('\0302')	抑扬音符号^
UNISTR('\0303')	颚化符号~
UNISTR('\0308')	元音变音¨

只有 UTF8 库中，支持元音字符和 UNISTR()生成的特殊符号两两组合生成新的特殊字符。其它情况不能组合则两两相拼输出，则按当前库字符集输出。

可使用 MANAGER 或 DIsql 客户端工具演示下面的 COMPOSE()示例。用户需保证数据库采用的是 UTF-8 字符集和客户端工具使用的编码格式为 UTF8。数据库的 UTF-8 字符集通过 dminit 初始化库时指定 CHARSET/UNICODE_FLAG 为 1 实现。客户端工具编码格式 UTF8 可在 dm_svc.conf 文件中将 CHAR_CODE 设置为 PG_UTF8 实现。

例 1 在 UTF8 库中，将 da 和 meng 合并输出。

```
select compose('da'||'meng') from dual ;
```

查询结果为：dameng

例 2 将元音 a 和沉音符`组合生成à。

```
select compose('a'||unistr('\0300')) from dual ;
```

查询结果为：à

例 3 将元音 u、元音变音¨和沉音符`组合生成ǜ。

```
select compose('u'||unistr('\0308') || unistr('\0300')) from dual ;
```

查询结果为：ǜ

68. 函数 FIND_IN_SET

```
语法：FIND_IN_SET(char, charlist[,separator])
```

功能：查询 charlist 中是否包含 char，返回 char 在 charlist 中第一次出现的位置或 NULL。

char 为待查询的字符串，charlist 为字符串列表，separator 为分隔符，缺省为","。字符串列表由 N 个被分隔符分隔的字符串组成，字符串可为空字符串。若 char 不在 charlist 中或 charlist 为空字符串，则返回 0；若任一参数为 NULL，则返回值为 NULL；否则返回位于 1 到 N 中的数值。

例

```
SELECT FIND_IN_SET('', '');
```

查询结果为：0

```
SELECT FIND_IN_SET(' ', ' ');
```

查询结果为：1

```
SELECT FIND_IN_SET('b', 'a,b,c');
```

查询结果为：2

```
SELECT FIND_IN_SET('', 'a,b,,');
```

查询结果为：3

```
SELECT FIND_IN_SET('ab', 'q8w8es8zcd8t8ab','8');
```

查询结果为：6

```
SELECT FIND_IN_SET(NULL, '');
```

查询结果为：NULL

69. 函数 TRUNC

```
语法：TRUNC(char1, char2)
```

功能：截取字符串函数。仅在以下两种情况下可以使用：

1） 当字符串 char2 解析成日期时间分量不成功时，解析成数字格式成功时，等价于数值函数 TRUNC(n[,m])，将 char1 当作小数数字，截取规则同数值函数 TRUNC(n[,m])，对 char1 中的数值进行截取。

2） 当字符串 char2 解析成日期时间分量成功时，将 char1 当作日期时间数字，将 char1 截断到最接近格式参数 m 指定的形式。等价于日期时间函数 TRUNC(date[,fmt])。

当字符串 char2 无法解析成日期时间分量或数字格式时，将会报错“字符串转换出错”。

例 1 char2 解析成数字格式成功

```
select trunc('108011524.122','-6') from dual;
```

查询结果如下：

```
行号              TRUNC('108011524.122','-6')

---------- --------------

1                 108000000
```

例 2 char2 解析成日期时间分量成功

```
select trunc('2010-09-01 10:59:59','yyyy');
```

查询结果如下:

```
行号              TRUNC('2010-09-01','yyyy')

---------- -------------------------

1                  2010-01-01 00:00:00
```

70. 函数 TO_BASE64

```
语法：TO_BASE64(R)
```

功能：将 VARBINARY、VARCHAR、CLOB 或 BLOB 格式的数据 R 编码成 base64 字符集格式，再以 CLOB 类型返回。

例

```
SELECT TO_BASE64('ABC') FROM DUAL;
```

查询结果为：QUJD

71. 函数 FROM_BASE64

```
语法：FROM_BASE64(R)
```

功能：将 VARCHAR 表示的 base64 字符集的编码 R 解码成原始的 VARBINARY 类型数据；或将 CLOB 表示的 base64 字符集的编码 R 解码成原始的 BLOB 类型数据。

例

```
SELECT FROM_BASE64('QUJD') FROM DUAL;
```

查询结果为：0x414243

72. 函数 STR_TO_DATE

```
语法：SELECT STR_TO_DATE(date_string, format_string);
```

功能：将字符串 date_string 按指定格式 format_string 转换为日期格式，返回类型受 format_string 影响，分为 DATE\TIME\DATETIME 三种，如果只有日期格式则返回 DATE 类型，如果只有时间格式则返回 TIME 类型，若都含有则返回 DATETIME 类型。格式请参考 DATE_FORMAT 中的 format 格式（表 8.3.6），其中的分隔符均需要严格匹配。

将 VARCHAR 表示的 base64 字符集的编码 R 解码成原始的 VARBINARY 类型数据；或将 CLOB 表示的 base64 字符集的编码 R 解码成原始的 BLOB 类型数据。

例

```
SELECT STR_TO_DATE('2024-03-28', '%Y-%m-%d');
```

查询结果为：2024-03-28

```
SELECT STR_TO_DATE('Mar 28, 2024 10:30:00 PM', '%b %d, %Y %h:%i:%s %p');
```

查询结果为： 2024-03-28 22:30:00

73. 函数 DATE

```
语法：SELECT DATE(date);
```

功能：返回日期或日期/时间表达式的日期部分。

例

```
select date('2008-12-29 16:25:46.635');
```

查询结果为：2008-12-29

8.3 日期时间函数

日期时间函数的参数至少有一个是日期时间类型(TIME，DATE，TIMESTAMP)，返回值一般为日期时间类型和数值类型。对于日期时间类型数据的取值范围，请参考 1.4.2 日期时间数据类型和《DM8 系统管理员手册》2.1.1.1 中对 IFUN_DATETIME_MODE 的介绍，若日期时间类型的参数或返回值超过限制范围，则报错。

DM 支持儒略历，并考虑了历史上从儒略历转换至格里高利日期时的异常，将'1582-10-05'到'1582-10-14'之间的日期统一当做'1582-10-15'处理。

函数返回结果为日期时间类型的打印结果印格式可以通过设置当前会话的日期串格式的方式来进行调整，具体的使用说明请参考 3.14.3 日期串格式。

1. 函数 ADD_DAYS

```
语法：ADD_DAYS( date, n)
```

功能：返回日期 date 加上相应天数 n 后的日期值。n 可以是任意整数，date 是日期类型(DATE)或时间戳类型(TIMESTAMP)，返回值为日期类型(DATE)。

例

```
SELECT ADD_DAYS( DATE '2000-01-12',1);
```

查询结果为：2000-01-13

2. 函数 ADD_MONTHS

```
语法：ADD_MONTHS(date,n)
```

功能：返回日期 date 加上 n 个月的日期时间值。n 可以是任意整数，date 可以是日期类型（DATE）或时间戳类型（TIMESTAMP）。当结果日期中月份所包含的天数比原日期中月份所包含的天数少或原日期是该月的最后一天时，返回的结果日期为该月最后一天。

当设置 ini 参数 COMPATIBLE_MODE=0 时，返回类型均为日期类型（DATE）。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=0 时，返回类型为 date 对应的类型。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=1 时，返回类型均为日期类型（DATE），此时还可以通过调整参数 nls_date_format 或 DATETIME_FMT_MODE 来设置返回结果的日期格式。

对于参数 COMPATIBLE_MODE 设置为其他值的情况，返回类型为参数 date 对应的类型。

例

```
SELECT ADD_MONTHS(DATE '2000-01-31',1);
```

查询结果为：2000-02-29

```
SELECT ADD_MONTHS(TIMESTAMP '2000-01-31 20:00:00',1);
```

查询结果为：2000-02-29

3. 函数 ADD_WEEKS

```
语法：ADD_WEEKS( date, n)
```

功能：返回日期 date 加上相应星期数 n 后的日期值。n 可以是任意整数，date 是日期类型(DATE)或时间戳类型(TIMESTAMP)，返回类型固定为日期类型(DATE)。

例

```
SELECT ADD_WEEKS( DATE '2000-01-12',1);
```

查询结果为: 2000-01-19

4. 函数 CURDATE

```
语法：CURDATE()
```

功能：返回客户端当前日期值，结果类型为 DATE。

例

```
SELECT   CURDATE();
```

查询结果为：执行此查询当天日期，如 2003-02-27

5. 函数 CURTIME

```
语法：CURTIME(n)
```

功能：返回客户端当前时间值，结果类型为 TIME WITH TIME ZONE。

参数：n：指定小数秒精度。取值范围 0~6，缺省为 6。

例

```
SELECT   CURTIME();
```

查询结果为：执行此查询的当前时间，如 14:53:54.859000 +8:00

6. 函数 CURRENT_DATE

```
语法：CURRENT_DATE()
```

功能：返回客户端当前日期值。

当设置 ini 参数 COMPATIBLE_MODE=0 或 4 时，返回类型均为日期类型（DATE）。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=0 时，返回类型为参数 date 对应的类型。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=1 时，返回类型均为日期类型（DATE），此时还可以通过调整参数 nls_date_format 或 DATETIME_FMT_MODE 来设置返回结果的日期格式。

对于参数 COMPATIBLE_MODE 设置为其他值的情况，返回类型为参数 date 对应的类型。

7. 函数 CURRENT_TIME

```
语法：CURRENT_TIME(n)
```

功能：返回客户端当前时间值，结果类型为 TIME WITH TIME ZONE，等价于 CURTIME()。

参数：n：指定小数秒精度。取值范围 0~6，缺省为 6。

8. 函数 CURRENT_TIMESTAMP

```
语法：CURRENT_TIMESTAMP(n)
```

功能：返回客户端当前带会话时区的时间戳，结果类型为 TIMESTAMP WITH TIME ZONE。

参数：n：指定小数秒精度。取值范围 0~9，缺省为 6。

例

```
SELECT    CURRENT_TIMESTAMP();
```

查询结果为：执行此查询的当前日期时间，如 2011-12-27 13:03:56.000000 +8:00

9. 函数 DATEADD

```
语法：DATEADD(datepart,n,date)
```

功能：向指定的日期 date 加上 n 个 datepart 指定的时间段，返回新的 timestamp 值或 time 值。datepart 取值见下表。

表 8.3.1 datepart 取值

datepart 取值	datepart 意义
YEAR、YYYY、YY、SQL_TSI_YEAR	年
MONTH、MM、M、SQL_TSI_MONTH	月
DAY、DD、D、SQL_TSI_DAY	日
HOUR、HH、SQL_TSI_HOUR	时
MINUTE、MI、N、SQL_TSI_MINUTE	分
SECOND、S、SS、SQL_TSI_SECOND	秒
MILLISECOND、MS、SQL_TSI_FRAC_SECOND	毫秒

MICROSECOND、US	微秒
QUARTER、QQ、Q、SQL_TSI_QUARTER	所处的季度
DAYOFYEAR、DY、Y	在年份中所处的天数
WEEK、WK、WW、SQL_TSI_WEEK	在年份中所处的周数
WEEKDAY、DW	在一周中所处的天数

例

```
SELECT DATEADD(HH, 4, '2022-09-19 16:09:35');
```

查询结果为：2022-09-19 20:09:35.000000

```
SELECT DATEADD(SS, 10, '14:05:47.555');
```

查询结果为：1900-01-01 14:05:57.555000

```
SELECT DATEADD(WW, 15, '2000-06-09');
```

查询结果为：2000-09-22 00:00:00.000000

10. 函数 DATEDIFF/BIGDATEDIFF

```
语法：DATEDIFF({datepart,}date1,date2)/ BIGDATEDIFF(datepart,date1,date2)
```

功能：返回跨两个指定日期的日期和时间边界数。datepart 取值见表 8.3.1。对于 DATEDIFF 而言，当不指定 datepart 时，表示求两个指定日期的天数差异。

注：当结果超出整数值范围，DATEDIFF 会产生错误。对于微秒 MICROSECOND，最大数是 35 分 47.483647 秒；对于毫秒 MILLISECOND，最大数是 24 天 20 小时 31 分钟 23.647 秒；对于秒，最大数是 68 年。若想提高可以表示的范围，可以使用 BIGDATEDIFF，其使用方法与 DATEDIFF 函数一致，只是可以表示更广范围的微秒、毫秒和秒。

例

```
SELECT DATEDIFF(QQ, '2003-06-01', DATE '2002-01-01');
```

查询结果为：-5

```
SELECT DATEDIFF(MONTH, '2001-06-01', DATE '2002-01-01');
```

查询结果为：7

```
SELECT DATEDIFF(WK, DATE '2003-02-07',DATE '2003-02-14');
```

查询结果为：1

```
SELECT DATEDIFF(MS,'2003-02-14 12:10:10.000','2003-02-14 12:09:09.300');
```

查询结果为：-60700

```
SELECT DATEDIFF('2023-12-31', '2023-01-01');
```

查询结果为：364

11. 函数 DATEPART/DATE_PART

```
语法：DATEPART(datepart,date)
```

功能：返回代表日期 date 的指定部分的整数。datepart 取值请参考 DATEDIFF(datepart,date1,date2)的参数。

例

```
SELECT DATEPART(SECOND, DATETIME '2000-02-02 13:33:40.00');
```

查询结果为：40

```
SELECT DATEPART(DY, '2000-02-02');
```

查询结果为：33

```
SELECT DATEPART(WEEKDAY, '2002-02-02');
```

查询结果为：7

说明：日期函数：date_part，其功能与 datepart 完全一样。但是写法有点不同：select datepart(year,'2008-10-10')；如果用 date_part，则要写成：select date_part('2008-10-10','year')，即：参数顺序颠倒，同时指定要获取的日期部分的参数要带引号。

12. 函数 DAY

```
语法：DAY(date)
```

功能：返回指定日期在月份中的天数

例

```
SELECT DAY('2016-06-07');
```

查询结果为：7

13. 函数 DAYNAME

```
语法：DAYNAME(date)
```

功能：返回日期的星期名称。

例

```
SELECT DAYNAME(DATE '2012-01-01');
```

查询结果为：Sunday

14. 函数 DAYOFMONTH

```
语法：DAYOFMONTH(date)
```

功能：返回日期为所处月份中的第几天。

例

```
SELECT DAYOFMONTH('2003-01-03');
```

查询结果为：3

15. 函数 DAYOFWEEK

```
语法：DAYOFWEEK(date)
```

功能：返回日期为所处星期中的第几天。

例

```
SELECT DAYOFWEEK('2003-01-01');
```

查询结果为：4

16. 函数 DAYOFYEAR

```
语法：DAYOFYEAR(date)
```

功能：返回日期为所处年中的第几天。

例

```
SELECT DAYOFYEAR('2003-03-03');
```

查询结果为：62

17. 函数 DAYS_BETWEEN

```
语法： DAYS_BETWEEN(dt1,dt2)
```

功能：返回两个日期之间相差的天数。

例

```
SELECT DAYS_BETWEEN('2022-06-01','2021-10-01');
```

查询结果为：243

18. 函数 EXTRACT

```
语法：EXTRACT(dtfield FROM date)
```

功能：EXTRACT 从日期时间类型或时间间隔类型的参数 date 中抽取 dtfield 对应的数值，并返回一个数字值。如果 date 是 NULL，则返回 NULL。dtfield 可以是 YEAR、MONTH、DAY、HOUR、MINUTE、SECOND、TIMEZONE_HOUR、TIMEZONE_MINUTE、WEEK（仅在 COMPATIBLE_MODE=4 时支持指定为 WEEK）。对于 SECOND 之外的任何域，函数返回整数，对于 SECOND 返回小数。

例

```
SELECT EXTRACT(YEAR FROM DATE '2000-01-01');
```

查询结果为：2000

```
SELECT EXTRACT(DAY FROM DATE '2000-01-01');
```

查询结果为：1

```
SELECT EXTRACT(MINUTE FROM TIME '12:00:01.35');
```

查询结果为：0

```
SELECT EXTRACT(TIMEZONE_HOUR FROM TIME '12:00:01.35 +9:30');
```

查询结果为：9

```
SELECT EXTRACT(TIMEZONE_MINUTE FROM TIME '12:00:01.35 +9:30');
```

查询结果为：30

```
SELECT EXTRACT(SECOND FROM TIMESTAMP '2000-01-01 12:00:01.35');
```

查询结果为：1.350000000000000E+00

```
SELECT EXTRACT(SECOND FROM INTERVAL '-05:01:22.01' HOUR TO SECOND);
```

查询结果为：-2.201000000000000E+01

```
SELECT EXTRACT(WEEK FROM DATE '2024-12-11');  //在 COMPATIBLE_MODE=4 的情况下执行
```

查询结果为：49

19. 函数 GETDATE

```
语法：GETDATE(n)
```

功能：返回系统的当前时间戳。

参数：n：指定小数秒精度。取值范围 0~9，缺省为 6。

例

```
SELECT GETDATE();
```

查询结果为：返回系统的当前日期时间，如 2011-12-05 11:31:10.359000

20. 函数 GREATEST

```
语法：GREATEST(date {,date})
```

功能：求一个或多个日期中的最大日期。

例

```
SELECT GREATEST(date'1999-01-01',date'1998-01-01',date'2000-01-01');
```

查询结果为：2000-01-01

21. 函数 GREAT

```
语法：GREAT (date1,date2)
```

功能：求 date1、date2 中的最大日期。

例

```
SELECT GREAT (date'1999-01-01', date'2000-01-01');
```

查询结果为：2000-01-01

22. 函数 HOUR

```
语法：HOUR(time)
```

功能：返回时间中的小时分量。

例

```
SELECT HOUR(TIME '20:10:16');
```

查询结果为：20

23. 函数 LAST_DAY

```
语法：LAST_DAY(date)
```

功能：返回 date 所在月最后一天的日期，date 可以是日期类型（DATE）或时间戳类型（TIMESTAMP），默认配置参数下返回类型均为日期类型（DATE）。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=0 时，返回类型为 date 对应的类型；当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=1 时，返回类型均为日期类型（DATE），此时还可以通过调整参数 DATETIME_FMT_MODE 来设置返回结果的日期时间格式。

当设置 INI 参数 COMPATIBLE_MODE=4 时，返回日期类型（DATE）。

当设置 COMPATIBLE_MODE 取其他值的情况下，返回类型为输入参数 date 对应的数据类型。

若需要返回结果展示为 TIMESTAMP 类型，可以通过 ALTER SESSION SET NLS_DATE_FORMAT='YYYY-MM-DD HH:MI:SS';设置返回结果的日期时间格式。

例

```
SELECT LAST_DAY(SYSDATE) "Days Left";
```

查询结果为：如：若当前日期为 2003 年 2 月的某一天，则结果为 2003-02-28

```
SELECT LAST_DAY(TIMESTAMP '2000-01-11 12:00:00');
```

查询结果为：2000-01-31

24. 函数 LEAST

```
语法：LEAST(date {,date})
```

功能：求一个或多个日期中的最小日期。

例

```
SELECT LEAST(date'1999-01-01',date'1998-01-01',date'2000-01-01');
```

查询结果为：1998-01-01

25. 函数 MINUTE

```
语法：MINUTE(time)
```

功能：返回时间中的分钟分量。

例

```
SELECT MINUTE('20:10:16');
```

查询结果为：10

26. 函数 MONTH

```
语法：MONTH(date)
```

功能：返回日期中的月份分量。

例

```
SELECT MONTH('2002-11-12');
```

查询结果为：11

27. 函数 MONTHNAME

```
语法：MONTHNAME(date)
```

功能：返回日期中月份分量的名称。

例

```
SELECT MONTHNAME('2002-11-12');
```

查询结果为：November

28. 函数 MONTHS_BETWEEN

```
语法：MONTHS_BETWEEN(date1,date2)
```

功能：返回 date1 和 date2 之间的月份值。如果 date1 比 date2 晚，返回正值，否则返回负值。如果 date1 和 date2 这两个日期为同一天，或者都是所在月的最后一天，则返回整数，否则返回值带有小数。date1 和 date2 是日期类型(DATE)或时间戳类型(TIMESTAMP)。

例

```
SELECT MONTHS_BETWEEN(DATE '1995-02-28', DATE '1995-01-31') "Months";
```

查询结果为：1

```
SELECT MONTHS_BETWEEN(TIMESTAMP '1995-03-28 12:00:00',TIMESTAMP '1995-01-31 12:00:00')
"Months";
```

查询结果为：1.90322580645161（具体返回值可能因为小数点后面保留位数的不同而有细微差别）

29. 函数 NEXT_DAY

```
语法：NEXT_DAY(date,char)
```

功能：返回在日期 date 之后满足由 char 给出的条件的第一天。date 可以是日期类型（DATE）或时间戳类型（TIMESTAMP），char 指定了一周中的某一个天(星期几)，char 是大小写无关的。

当设置 ini 参数 COMPATIBLE_MODE=0 时，返回类型均为日期类型（DATE）。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=0 时，返回类型为 date 对应的类型。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=1 时，返回类型均为日期类型（DATE），此时还可以通过调整参数 nls_date_format 或 DATETIME_FMT_MODE 来设置返回结果的日期格式。

对于参数 COMPATIBLE_MODE 设置为其他值的情况，返回类型为参数 date 对应的类型。

char 取值如表 8.3.2 所示。

表 8.3.2 星期描述说明

输入值	含义
SUN	星期日

SUNDAY	
MON	星期一
MONDAY	
TUES	星期二
TUESDAY	
WED	星期三
WEDNESDAY	
THURS	星期四
THURSDAY	
FRI	星期五
FRIDAY	
SAT	星期六
SATURDAY	

例

```
SELECT NEXT_DAY(DATE '2001-08-02', 'MONDAY');
```

查询结果为：2001-08-06

```
SELECT NEXT_DAY('2001-08-02 12:00:00', 'FRI');
```

查询结果为：2001-08-03

30. 函数 NOW

```
语法：NOW(n)
```

功能：返回系统的当前时间戳。等价于 GETDATE()。

参数：n：指定小数秒精度。取值范围 0~9，缺省为 6。

31. 函数 QUARTER

```
语法：QUARTER(date)
```

功能：返回日期在所处年中的季度数。

例

```
SELECT QUARTER('2002-08-01');
```

查询结果为：3

32. 函数 SECOND

```
语法：SECOND(time)
```

功能：返回时间中的秒分量。

例

```
SELECT SECOND('08:10:25.300');
```

查询结果为：25

33. 函数 ROUND

```
语法：ROUND(date[, fmt])
```

功能：将日期时间 date 四舍五入到最接近格式参数 fmt 指定的形式。如果没有指定语法的话，到今天正午 12P.M.为止的时间舍取为今天的日期，之后的时间舍取为第二天 12A.M.。日期时间 12A.M.，为一天的初始时刻。参数 date 的类型可以是 DATE、TIME 或 TIMESTAMP，但应与 fmt 相匹配。函数的返回结果的类型与参数 date 相同。fmt 具体如下表所示。

表 8.3.3 日期时间说明

fmt 的格式	含义	date 数据类型
CC, SCC	世纪，从 1950、2050 等年份的一月一号午夜凌晨起的日期，舍取至下个世纪的一月一号	DATE TIMESTAMP

syear，syyy，y，yy，yyy，yyyy, year	年，从七月一号午夜凌晨起的日期，舍取至下个年度的一月一号	DATE TIMESTAMP
Q	季度，从十六号午夜凌晨舍取到季度的第二个月，忽略月中的天数	DATE TIMESTAMP
month,mon, mm, m, rm	月，从十六号午夜凌晨舍取	DATE TIMESTAMP
Ww	舍取为与本年第一天星期数相同的最近的那一天	DATE TIMESTAMP
W	舍取为与本月第一天星期数相同的最近的一天	DATE TIMESTAMP
iw	舍取为最近的周一	DATE TIMESTAMP
ddd, dd, j	从正午起，舍取为下一天，默认值	DATE TIMESTAMP
day, dy, d	星期三正午起，舍取为下个星期天	DATE TIMESTAMP
hh, hh12, hh24	在一个小时的 30 分 30 秒之后的时间舍取为下一小时	DATE TIME TIMESTAMP
Mi	在一个分钟 30 秒之后的时间舍取为下一分	DATE TIME TIMESTAMP

有关 ww 和 w 的计算进一步解释如下(下面的时间仅当 date 参数为时间戳时才有效):

ww 产生与本年第一天星期数相同的最近的日期。因为每两个星期数相同日期之间相隔

六天，这意味着舍取结果在给定日期之后三天以内。例如，如果本年第一天为星期二，若给定日期在星期五午夜 23:59:59 之前(包含星期五 23:59:59)，则舍取为本星期的星期二的日期；否则舍取为下星期的星期二的日期。

w 计算的方式类似，不是产生最近的星期一 00:00:00，而是产生与本月第一天相同的星期数的日期。

例

```
SELECT ROUND(DATE '1992-10-27', 'scc');
```

查询结果为：2001-01-01

```
SELECT ROUND(DATE '1992-10-27', 'YEAR') "FIRST OF THE YEAR";
```

查询结果为：1993-01-01

```
SELECT ROUND(DATE '1992-10-27', 'q');
```

查询结果为：1992-10-01

```
SELECT ROUND(DATE '1992-10-27', 'month');
```

查询结果为：1992-11-01

```
SELECT ROUND(TIMESTAMP '1992-10-27 11:00:00', 'ww');
```

查询结果为：1992-10-28 00:00:00.000000

```
SELECT ROUND(TIMESTAMP '1992-10-27 11:00:00', 'w');
```

查询结果为：1992-10-29 00:00:00.000000

```
SELECT ROUND(TIMESTAMP '1992-10-27 12:00:01', 'ddd');
```

查询结果为：1992-10-28 00:00:00.000000

```
SELECT ROUND(DATE '1992-10-27', 'day');
```

查询结果为：1992-10-25

```
SELECT ROUND(TIMESTAMP '1992-10-27 12:00:31', 'hh');
```

查询结果为：1992-10-27 12:00:00.000000

```
SELECT ROUND(TIMESTAMP '1992-10-27 12:00:31', 'mi');
```

查询结果为：1992-10-27 12:01:00.000000

34. 函数 TIMESTAMPADD

```
语法：TIMESTAMPADD(datepart,n,timestamp)
```

功能：返回时间戳 timestamp 加上 n 个 datepart 指定的时间段的结果，datepart 取值见表 8.3.1。

例

```
SELECT TIMESTAMPADD(SQL_TSI_FRAC_SECOND, 5, '2003-02-10 08:12:20.300' );
```

查询结果为：2003-02-10 08:12:20.305000

```
SELECT TIMESTAMPADD(SQL_TSI_YEAR, 30, DATE '2002-01-01');
```

查询结果为：2032-01-01 00:00:00.000000

```
SELECT TIMESTAMPADD(SQL_TSI_QUARTER, 2, TIMESTAMP '2002-01-01 12:00:00');
```

查询结果为：2002-07-01 12:00:00.000000

```
SELECT TIMESTAMPADD(SQL_TSI_DAY, 40, '2002-12-01 12:00:00');
```

查询结果为：2003-01-10 12:00:00.000000

```
SELECT TIMESTAMPADD(SQL_TSI_WEEK, 1, '2002-01-30');
```

查询结果为：2002-02-06 00:00:00.000000

35. 函数 TIMESTAMPDIFF

```
语法：TIMESTAMPDIFF(datepart,timestamp1,timestamp2)
```

功能：返回一个表明 timestamp2 与 timestamp1 之间的指定 datepart 类型的时间间隔的整数，datepart 取值见表 8.3.1。

注: 当结果超出整数值范围, TIMESTAMPDIFF 产生错误。对于秒级 SQL_TSI_SECOND，最大数是 68 年。

例

```
SELECT TIMESTAMPDIFF(SQL_TSI_FRAC_SECOND,'2003-02-14 12:10:10.000', '2003-02-14
12:09:09.300');
```

查询结果为：-60700

```
SELECT TIMESTAMPDIFF(SQL_TSI_QUARTER, '2003-06-01', DATE '2002-01-01');
```

查询结果为：-5

```
SELECT TIMESTAMPDIFF(SQL_TSI_MONTH, '2001-06-01', DATE '2002-01-01');
```

查询结果为：7

```
SELECT TIMESTAMPDIFF(SQL_TSI_WEEK, DATE '2003-02-07',DATE '2003-02-14');
```

查询结果为：1

36. 函数 SYSDATE

```
语法： SYSDATE()
```

功能：获取系统当前的日期时间，为数据库服务器所在操作系统的日期时间（时区为当前会话时区）。需要注意的是，当机器时区调整时，需要管理员重启服务器，或先执行一次涉及 SYSTIMESTAMP 的调用，时区调整才会生效，SYSDATE 获取的日期时间才为调整后的日期时间。

例

```
SELECT  SYSDATE();
```

查询结果为：当前系统时间，如 2024-10-17 15:58:54。

37. 函数 TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ

```
语法：TO_DATE(char [,fmt[,'nls']])
      或
      TO_TIMESTAMP(char [,fmt[,'nls']])
```

```
或
TO_TIMESTAMP_TZ(char [,fmt[,'nls']])
```

功能：将 CHAR 或者 VARCHAR 类型的值转换为 TIMESTAMP 数据类型。TO_DATE 的结果不带小数秒精度；TO_TIMESTAMP 的结果带小数秒精度；TO_TIMESTAMP_TZ 的结果带服务器的时区。

当设置 ini 参数 COMPATIBLE_MODE=2 与 ORA_DATE_FMT=1 时，TO_DATE 函数返回值是 DATE 类型，可以通过调整参数 NLS_DATE_FORMAT 或 DATETIME_FMT_MODE 来设置返回结果的日期格式。对于参数 COMPATIBLE_MODE 设置为其他值的情况，TO_DATE 函数返回类型为 TIMESTAMP 数据类型。

当输入缺少时分秒信息时，结果补充为 00。当输入缺少年月日信息时，对于 COMPATIBLE_MODE=3 或 7 的情况，年缺省值为 1900，月缺省值为 01，日缺省值为 01；其余 COMPATIBLE_MODE 取值则补充为当前年月日。

参数：

NLS：指定日期时间串的语言类型，取值：AMERICAN、ENGLISH 或 SIMPLIFIED CHINESE，分别表示美式英语、英语和简体中文，其中 AMERICAN 和 ENGLISH 的效果相同。例如，当将日期时间串指定为 2022-DECEMBER-12 时，应将 NLS 设置为 AMERICAN 或 ENGLISH。缺省为 SIMPLIFIED CHINESE。这个参数的使用形式是：'NLS_DATE_LANGUAGE="语言类型"'。

FMT：日期格式。具体用法请参考日期格式。

例

```
SELECT TO_DATE('20200215 14.47.38','YYYY-MM-DD HH24:MI:SS');
```

查询结果为：2020-02-15 14:47:38

```
SELECT TO_TIMESTAMP('DECEMBER 15 2020 14.47.38','MONTH DD YYYY
HH24:MI:SS','NLS_DATE_LANGUAGE="AMERICAN"');
```

查询结果为：2020-12-15 14:47:38.000000

日期格式

日期格式在很多地方都会用到。例如，置当前会话的日期串格式和 TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ 函数中，此处统一介绍用法。

日期格式有三种写法：DATE 格式、TIME 格式或 TIMESTAMP 格式。其中，DATE 格式为年月日、月日年或日月年。TIME 格式为时分或时分秒。TIMESTAMP 格式为 DATE 格式+TIME 格式。日期格式各部分之间可以有分隔符或者没有分隔符。例如'YYYY/MM/DD'、'HH24:MI'、'YYYYMMDD HH24:MI:SS'，其中 YYYY、MM、DD、HH24、MI 和 SS 为格式符；/、:和空格为分隔符。DM 缺省的 TIMESTAMP 格式 FMT 为：'YYYY-MM-DD HH24:MI:SS.FF'。

日期格式书写需遵循特定的书写规则。日期格式由格式符、分隔符和 FX 固定格式器组成。其中 FX 固定格式器为可选项。下面分别介绍。

- **格式符**

日期格式 FMT 中的格式符由年、月、日、时、分、秒等元素组成，格式符中的字母不区分大小写。详细的元素介绍，参见下表。

表 8.3.4 格式符

元素	说明	备注
D	周中的某一天，星期天算起	
DD	月中的某一天	
DDD	年中的某一天	
HH/HH12	天中的时（01 - 12），12 小时制	
HH24	天中的时（00 - 23），24 小时制	
MI	分（00 - 59）	
MM	月（数字。1-12）	当COMPATIBLE_MODE=2时，MM会匹配数字或英文（例如12，DEC，DECEMBER）；若需要匹配英文月必须设置英文环境NLS_DATE_LANGUAGE=AMERICAN或ENGLISH
SS	秒（00 - 59）	
SSSSS	一天从午夜开始的累积秒数（00000 - 86399），当累积秒数位数不足 5 位时，用前置 0 补齐	

	5 位	
TZH	时区中的小时，例如'HH:MI:SS FF TZH:TZM'	TZH 支持“+”或“-”;当使用 FX 时只能指定“-”表示负时区，或不指定表示正时区，不能指定“+”。当 COMPATIBLE_MODE=2 时，只有目标数类型包含时区时才能指定此格式，如：TO_TIMESTAMP_TZ，否则报错
TZM	时区中的分钟	TZM 虽然支持“+”或“-”，但被忽略，没有实际意义;当使用 FX 时指定“+”或“-”则报错。当 COMPATIBLE_MODE=2 时，只有目标数类型包含时区时才能指定此格式，如：TO_TIMESTAMP_TZ，否则报错
FF[1…9]	小数秒精度。[1…9]指定小数秒精度，不指定时若 INI 参数 IFUN_DATETIME_MODE=0，则小数秒精度默认值为 6，否则为 9	当 COMPATIBLE_MODE=2，ORA_DATE_FMT=1 时，TO_DATE 结果不包含小数秒，不支持 FF 相关格式
X	X 表示秒和小数秒的间隔，如 SSXFF，默认情况下等价于“.”	X 也受 NLS_NUMERIC_CHARACTERS=DG 中 D 指定的字符的影响，X 等价于 D 指定的字符。不适用于 TO_DATE 中
SYYYY	四位的年份，S 前缀表示公元前 BC	SYYYY 支持指定“+”或“-”; 当 COMPATIBLE_MODE=2，TO_DATE 函数使用 FX 时，需要指定“+”或“-”，否则报错; TO_TIMESTAMP/TO_TIMESTAMP_TZ 函数使用 FX 时，只能指定“-”表示负年，或不指定表示正年，不能指定“+”
YYYY	四位的年份	当 COMPATIBLE_MODE=2 时， 若源串中数字位数不够四位: 1）不指定 FX 时，YYYY 可能尝试当作 2 位年来解析: 在 TO_DATE 中会报错; 在 TO_TIMESTAMP/TO_TIMESTAMP_TZ 中会当 2 位年解析; 由于 TO_TIMESTAMP/TO_TIMESTAMP_TZ 处理规则不一样，返回结果不同; 2）指定 FX 时: 在 TO_DATE 中不会将 YYYY 调整为 2 位年; 在 TO_TIMESTAMP 和 TO_TIMESTAMP_TZ 中会将 YYYY 调整为 2 位年处理,但 TO_TIMESTAMP_TZ 当 2 位年处理时等价于 YY，系统会自动使用'20'补全结果中年的高 2 位；而 TO_TIMESTAMP 虽将 YYYY 当作 2 位年处理，但还是不会补全结果中年的高 2 位
YYY	年份的最后 3 位数字	
YY	年份的最后 2 位数字	当 COMPATIBLE_MODE=2，不指定 FX 时，若源串中数字位数过多，YY 可能尝试当作 4 位年解析，此规则对 TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ

		都有效；但存在差别： 在 TO_TIMESTAMP/TO_TIMESTAMP_TZ 中，若 YY 后面的分隔符为限定分隔符且以数字开头，则不调整 YY 当作 4 位年解析；否则，虽可调整，但对应的年的位数必须是 2 位或 4 位； 在 TO_DATE 中，若 YY 后面的分隔符为限定分隔符且以数字开头，可调整 YY 当作 4 位年解析，但要求匹配分隔符后对应的年的位数必须是 2 位或 4 位；否则，1 到 4 位年都可以匹配
Y	年份的最后 1 位数字	
AD/A.D.	公元,不能为 0	
AM/A.M.	上午	
BC/B.C.	公元前	
CC/SCC	世纪	不适用于TO_DATE中
DAY	星期（如星期五或FRIDAY）	
DL	返回长日期格式,包括年月日和星期几	
DS	返回短日期格式，包括年月日	
DY	星期的缩写形式(如星期五或 FRI)	
IW	星期数(当前日期所在星期是这一年的第几个星期，基于 ISO 标准)	不适用于 TO_DATE 中
MON	月份名称的缩写形式（如 12 月或 DEC）	
MONTH	月份名称(如 12 月或 DECEMBER)	
PM/P.M.	下午	
Q	季度号（1、2、3、4）	不适用于 TO_DATE 中
FM	去除结果中的前导 0	供 TO_CHAR 使用
RR/RRRR	RR：输入参数年份的 2 位数字和数据库服务器上当前年的后 2 位数字（当年）共同确定 当指定的两位年份数字在 00~49 之间时:若当前年的后两位数字在 00~49 之间，则返回年份的前两位数字和当前年的前两位数字相同；若当前年的后两位数字在 50~99 之间，则返回年份的前两位数字为当前年的前两位数字加 1 当指定的两位年份数字在 50~99 之间时:若当前年份的后两位数字在 00~49 之间，则返回年份的前两位数字为当前年的前两位数字减 1；若当前年的后两位数字在 50~99 之间，则返回年份的前两	

	位数字和当前年的前两位数字相同。 若不指定 FX 固定格式器，仅后面无其他分隔符且有其它格式符的情况才固定处理两位数字的年份，如：rrmm；当在 RR 前指定使用了 FX 固定格式器时，固定处理两位数字的年份 RRRR：如果输入参数只有两位，则同 RR，否则同 YYYY 作用	
WW	星期数(当前日期所在星期是这一年的第几个星期，第一个星期从 1 月 1 日开始，到 1 月 7 日结束)	不适用于 TO_DATE 中
W	星期数(当前日期所在星期是这个月的第几个星期)	不适用于 TO_DATE 中
Y,YYY	带逗号的四位年份	Y,YYY 中的逗号表示分组符，但不受 NLS_NUMERIC_CHARACTERS 中分组符的影响，只能是逗号。 存在限制： 源串中不能连续使用 2 个分组符，否则报错； 源串中连续数字的个数不能大于 4，否则报错； 使用 FX 固定格式时，源串有且仅需要 1 个分组符
IYYY,IYY,IY,I	最后倒数 4 位，3 位，2 位，1 位 ISO 标准年份。 ISO 标准认为日期是从周一到周日，按周计算。普通的标准则指定任何一年的一月一号都是周一	不适用于 TO_DATE 中
YEAR/SYEAR	拼写出的年份（比如 TWENTY FIFTEEN）S 前缀表示负年	不适用于 TO_DATE 中
J	儒略日，从公元前 4712 年 1 月 1 日到指定日期的天数	
JSP	儒略日，值与元素 J 相同，在 to_char 中以英文单词的方式拼写。其中 JsP/Jsp 对应首字母大写，JSP/JSp 对应全字母大写，jsp/jsP/jSp/jSP 对应全字母小写	在 TO_DATE 中和 J 的使用相同

- **DATE 分隔符**

下面介绍 DATE 格式中用到的分隔符。分隔符分为两种：一是非限定分隔符，通常指除大小写字母、数字以及双引号之外的所有单字节字符且可打印的。例如：空格、回车键、tab 键、-、/、,、.、:、*等标点符号。单个双引号"可以作为源串的分隔符，但是不能在 FMT 中

作分割符；二是限定分隔符，指由双引号括起来的任意长度串，比如中文。例如 “年”“月”“日”里的年、月、日。

TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ 函数目前支持的分隔符的规则如下：

一 分隔符中首空字符的处理方法

空字符包括空格、TAB 键、回车符和换行符。首空字符是指位于分隔符（限定或非限定）最前端的空字符。在限定符和非限定符组合中，出现在组合最前端的空字符为首空字符。

当 FMT 中包含首空字符，且为非 FX 固定格式时，TO_DATE 第一个限定分隔符内和之前的首空字符将被自动去除，第一个限定分隔符后的首空字符将被保留，包括其它限定或非限定分隔符；源串处理与分隔符处理机制一致。

当 FMT 中包含首空字符，且为非 FX 固定格式时，TO_TIMESTAMP 和 TO_TIMESTAMP_TZ 与 TO_DATE 不同，不论限定或非限定分隔符，首空字符均将被自动去除；源串处理与分隔符处理机制一致。同时 TO_TIMESTAMP/TO_TIMESTAMP_TZ 在不指定 FX 时，会忽略两个限定分隔符之间的非限定分隔符。

例 1 在 TO_DATE 的限定分隔符"□兔年"和非限制分隔符□:中，首位的空格均为首空字符，将被直接去除。本节示例中□代表空格。

```
SELECT TO_DATE('2023 兔年 10:10','YYYY" 兔年"MM :DD');
```

查询结果如下：

```
TO_DATE('2023 兔年 10:10','YYYY"兔年"MM:DD')

----------------------------------------

2023-10-10 00:00:00
```

例 2 在 TO_DATE 的第一个限定分隔符"□兔年"中，TO_DATE 第一个限定分隔符内的首空字符将自动去除；"□测试"为第一个限定分隔符后的限定分隔符，其首空字符将被保留。

```
SELECT TO_DATE('2023 兔年  测试 10:10','YYYY" 兔年" " 测试"MM:DD');
```

查询结果如下：

```
TO_DATE('2023 兔年测试 10:10','YYYY"兔年""测试"MM:DD')

-------------------------------------------------

2023-10-10 00:00:00
```

例 3 在 TO_TIMESTAMP 的分隔符"□兔年"、"□测试"中，不管限定或非限定分隔符首空字符均将被自动去除。

```
SELECT TO_TIMESTAMP('2023 兔年测试 10:10','YYYY" 兔年" " 测试"MM:DD');
```

查询结果如下：

```
TO_TIMESTAMP('2023 兔年测试 10:10','YYYY"兔年""测试"MM:DD')

----------------------------------------------------

2023-10-10 00:00:00.000000
```

例 4 在"兔年"//"--"中，TO_TIMESTAMP 中两个限定分隔符之间的非限定分隔符//会被忽略。

```
SELECT TO_TIMESTAMP('2023 兔年--10:10','YYYY"兔年"//"--"MM:DD');
```

查询结果如下：

```
TO_TIMESTAMP('2023 兔年--10:10','YYYY"兔年"//"--"MM:DD')

----------------------------------------------------

2023-10-10 00:00:00.000000
```

二 实际分隔符的处理方法

实际分隔符是指根据 TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ 的首空字符处理规则去除首空字符后的分隔符。

为非 FX 固定格式时，FMT 与源串对应位置的实际分隔符将按以下规则进行匹配：

1） 源串中对应位置非限定实际分隔符的个数必须小于或等于 FMT 中对应位置非限定实际分隔符的个数。如果 FMT 非限定实际分隔符数量为 m 个，则源串对应位置的非限定实

际分隔符要小于等于 m 个，如果源串对应位置第 m 个之后的多余字符全是空字符，则忽略源串分隔符末尾空字符。

例 1 FMT 中只包含非限定分隔符-，源串对应位置的分隔符为:，匹配成功。

```
SELECT TO_DATE('2001:1010','YYYY-MMDD');
```

查询结果如下:

```
TO_DATE('2001:1010','YYYY-MMDD')

---------------------------------------------

2001-10-10 00:00:00
```

例 2　FMT 中格式符 MM 与 DD 之间的分隔符为 3 个连续的:，那么源串对应位置的分隔符个数要小于等于 3 个。

```
SELECT TO_DATE('2001-10--10','YYYY:MM:::DD');
```

查询结果如下:

```
TO_DATE('2001-10--10','YYYY:MM:::DD')

---------------------------------------------

2001-10-10 00:00:00
```

例 3　FMT 中格式符 MM 与 DD 之间的分隔符为 3 个连续的:，源串对应位置的分隔符为 2 个连续的-和 3 个连续的空格，忽略末尾 3 个空格，匹配成功。

```
SELECT TO_DATE('2001-10--    10','YYYY:MM:::DD');
```

查询结果如下:

```
TO_DATE('2001-10--10','YYYY:MM:::DD')

---------------------------------------------

2001-10-10 00:00:00
```

2） 如果 FMT 的分隔符中存在限定分隔符，则源串对应位置的实际分隔符必须与 FMT

中的实际分隔符完全一致。若源串分隔符末尾存在空字符，则可以适当忽略末尾空字符使得分隔符能够匹配。

例 1 FMT 中指定了“□猪年”作为限定分隔符，去除掉首空字符后实际限定分隔符为“猪年”，那么源串中也要指定个数相同的“猪年”。

```
SELECT TO_DATE('2019 猪年 10 月 10 日','YYYY" 猪年"MM"月"DD"日"');
```

查询结果如下：

```
TO_DATE('2019 猪年 10 月 10 日','YYYY"猪年"MM"月"DD"日"')

--------------------------------------------------

2019-10-10 00:00:00
```

例 2 FMT 中指定了“猪年□□”作为限定分隔符，源串中对应位置的分隔符为“猪年□□□”，忽略源串分隔符末尾的一个空格，可以成功匹配。

```
SELECT TO_DATE('2019 猪年   10 月 10 日','YYYY"猪年  "MM"月"DD"日"');
```

查询结果如下：

```
TO_DATE('2019 猪年 10 月 10 日','YYYY"猪年"MM"月"DD"日"')

---------- ----------------------------------------------------

2019-10-10 00:00:00
```

3） 当 FMT 分隔符中仅包含空字符时，如果 FMT 分隔符中包含限定分隔符，则去除所有空字符，最终的分隔符为空串；如果 FMT 分隔符中只包含非限定分隔符，则保留所有空字符。

例 1 FMT 中 YYYY 和 MM 之间的分隔符为"□□"□□，其中包含限定分隔符，因此去除所有空字符，最终的分隔符为空串""，源串对应位置可以没有分隔符。

```
SELECT TO_DATE('20011210','YYYY"  "  MMDD');
```

查询结果如下：

```
TO_DATE('20011210','YYYY""MMDD')

--------------------------------------------

2001-12-10 00:00:00
```

例 2 FMT 中 YYYY 和 MM 之间的分隔符为□□□□，只包含非限定分隔符，因此保留所有空字符，源串对应位置的分隔符要小于等于 4 个非限定分隔符。

```
SELECT TO_DATE('2001@--!1210','YYYY      MMDD');
```

查询结果如下：

```
TO_DATE('2001@--!1210','YYYYMMDD')

-----------------------------------------

2001-12-10 00:00:00
```

4） 如果 FMT 未设置分隔符，则源串对应位置不能有除空字符外的分隔符。

例 1 以下为 FMT 未设置分隔符的情况。

```
SELECT TO_DATE('200112       10','YYYYMMDD');
```

查询结果如下：

```
TO_DATE('20011210','YYYYMMDD')

-------------------------------------------

2001-12-10 00:00:00
```

5） X 规则：若分隔符 X 不在限定分隔符中，X 表示秒和小数秒的间隔，如 SSXFF，默认情况下等价于“.”；X 会受 NLS_NUMERIC_CHARACTERS=DG 中 D 指定字符的影响，等价于 D 指定的字符。如果分隔符中存在限定分隔符，则当 X 前一个分隔符为非限定分隔符.时，忽略 X 前所有连续的.分隔符；如果分隔符中不存在限定分隔符，则忽略 X 前所有符号，最终分隔符为 X。

例 1 X 不在限定分隔符中，分隔符中存在限定分隔符"秒"，并且因此忽略 X 前所有连续

的.符号，最终分隔符为"秒"X，X 等价于.，因此最终分隔符等价于"秒".。

```
SELECT TO_TIMESTAMP ('10 秒.123000', 'SS"秒"...XFF');
```

查询结果如下：

```
TO_TIMESTAMP('10 秒.123000','SS"秒"...XFF')

--------------------------------------------------

2024-05-01 00:00:10.123000
```

例 2 以下 FMT 分隔符为"秒..."...X，忽略 X 前所有连续的.非限定分隔符号，由于 X 等价于.，最终分隔符为"秒...".。

```
SELECT TO_TIMESTAMP ('10 秒.123000', 'SS"秒..."...XFF');
```

由于限定分隔符中的.无法被忽略，因此查询报错：

```
[-6130]:文字与格式字符串不匹配.
```

例 3 以下 FMT 分隔符为"秒..."X，由于 X 前不存在非限定分隔符.，因此不会忽略"秒..."中的.，最终分隔符为"秒...".。

```
SELECT TO_TIMESTAMP ('10 秒....123000', 'SS"秒..."XFF');
```

查询结果如下：

```
TO_TIMESTAMP('10 秒....123000','SS"秒..."XFF')

--------------------------------------------------

2024-05-01 00:00:10.123000
```

例 4 以下 FMT 分隔符为--..-@-XFF，由于分隔符中不存在限定分隔符，因此忽略 X 前所有符号，最终分隔符为 X，等价于.。

```
SELECT TO_TIMESTAMP ('10.123000', 'SS--..-@-XFF');
```

查询结果如下：

```
TO_TIMESTAMP('10.123000','SS--..-@-XFF')

--------------------------------------------------
```

```
2024-05-01 00:00:10.123000
```

6） 若分隔符后面的格式符可匹配正负号，且源串对应位置分隔符的末尾符号为正负号，则将该正负号匹配到后面的格式符上。

例 1 以下 YYYY 前面的分隔符为@，因此源串中对应位置的分隔符不能超过一个，但是源串中对应位置的分隔符为@@，因此匹配失败报错。

```
SELECT TO_DATE('@@2023-5-17','@YYYY-MM-DD');
```

查询结果报错：

```
[-6132]:年份值必须介于 -4713 和 +9999 之间, 且不为 0.
```

而在下面的查询中，源串中对应位置的分隔符为@+，末尾符号是正号，由于后面的格式符 YYYY 可以匹配正负号，即+2023 与格式符 YYYY 匹配，因此源串中对应位置的分隔符变为@，匹配成功。

```
SELECT TO_DATE('@+2023-5-17','@YYYY-MM-DD');
```

查询结果如下：

```
TO_DATE('@+2023-5-17','@YYYY-MM-DD')

-----------------------------------

2023-05-17 00:00:00
```

7） 源串还支持两个特殊的符号 T/t 和 Z/z。其使用方式分别如下：

（1）T/t：相当于空格分隔符，使用时需注意第一个字符不能是 T 之外的字母，或当分隔符全是空格时，T 可以在空格符的中间。针对两种格式 YYYY-MM-DD HH:MI:SS 和 YYYY-MM-DD HH:MI:SS.FF +tzh:tzm 原串中年和月之间可以允许出现字符 T 作为年月日与时分秒的分隔符。

例 使用 T 作为源串中的分隔符进行日期转换

```
SELECT TO_DATE('2021-10-12T12:20:15') FROM DUAL;
```

结果如下：

```
TO_DATE('2021-10-12T12:20:15')

-------------------------

2021-10-12 12:20:15
```

（2）Z/z：是一个标记，代表使用 UTC 时区，可与 T 一起使用，也可放在空格中间。

例 使用 Z 标记该时间为 UTC 时区的时间

```
SET TIME ZONE  '1:00';

SELECT CAST('2021-10-12 12:20:1.8z' AS  timestamp with time zone);
```

结果如下：

```
CAST('2021-10-1212:20:1.8z'ASTIMESTAMPWITHTIMEZONE)

-------------------------------------------

2021-10-12 12:20:01.800000 +00:00
```

8） YY 与分隔符搭配用法

“YY+分隔符”进行搭配之后，与源串对应位置的“年份数字+分隔符”将按以下规则进行匹配：

- 常规用法

“YY+分隔符”搭配使用，可识别出源串分隔符之前的两位年份数字，然后系统根据当前所处的年份，自动将其补充成一个完整的年份数字。

TO_TIMESTAMP、TO_TIMESTAMP_TZ 用法与 TO_DATE 基本相同，不同之处下文会特别指出，如不特别指出，均表示相同。本节以 TO_DATE 为例进行介绍。

例 根据 YY-，可识别出源串中的两位年份数字 49。

```
SELECT TO_DATE('49-10-01','YY-MM-DD');
```

查询结果分别如下：

```
2049-10-01 00:00:00
```

- 特殊用法

为了防止源串或 FMT 写法不规范造成无法识别出年份数字的情况，DM 特支持了多种解决方法。下面分别一一介绍。

（1） 当源串的格式和 FMT 不同。当源串总长度未超过 FMT，且源串包含 1、2、3、4 位年份数字时，可分别识别出 1、2、2、2 位有效年份数字，然后系统根据当前所处的年份，自动将其补充成一个完整的年份数字。当源串总长度超过 FMT 则报错。

例 源串中年份分别包含 1、2、3、4 位年份数字，根据 YY-，分别匹配出 4、43、43、43 位年份数字。缺少的 dd 数据，用 01 补齐。

```
SELECT TO_DATE('4-12','YY-MM-DD');

SELECT TO_DATE('43-12','YY-MM-DD');

SELECT TO_DATE('432-12','YY-MM-DD');

SELECT TO_DATE('4312-12','YY-MM-DD');
```

查询结果分别如下：

```
2004-12-01 00:00:00

2043-12-01 00:00:00

2043-02-12 00:00:00

2043-12-12 00:00:00
```

（2） 当 YY 后面无分隔符或分隔符首位是数字时，YY 可识别出两位年份数字，然后系统根据当前所处的年份，自动补齐成一个完整的年份。

例 1 YY 后面无分隔符，可识别出源串中的两位数字 50。

```
SELECT TO_DATE('500112','YYMMDD');
```

查询结果如下：

```
2050-01-12 00:00:00
```

例 2 YY 后面是数字 8 年，可识别出源串中的两位数字 50。

```
SELECT TO_DATE('508 年 0112','YY"8 年"MM-DD');
```

查询结果如下：

```
2050-01-12 00:00:00
```

（3） 当 YY 后面是非限定分隔符，源串中的年份位数不同时：TO_TIMESTAMP 的 YY 可识别 1 位或 2 位有效年份数字；一般情况下，TO_DATE 的 YY 可识别 1 位、2 位有效年份数字，然后系统根据当前所处的年份，自动补齐成一个完整的年份；特别的是，仅当 COMPATIBLE=2 时，TO_DATE 可识别 3 位有效年份数字，直接读出。

例 1 YY 后面非限定分隔符*，TO_TIMESTAMP 可识别出源串中的一位数字 9。

```
SELECT TO_TIMESTAMP ('9*0112','YY*MM-DD');
```

查询结果如下：

```
2009-01-12 00:00:00.000000
```

例 2 YY 后面非限定分隔符*，TO_TIMESTAMP 可识别出源串中的两位数字 89。

```
SELECT TO_TIMESTAMP ('89*0112','YY*MM-DD');
```

查询结果如下：

```
2089-01-12 00:00:00.000000
```

例 3 YY 后面非限定分隔符*，TO_DATE 可识别出源串中的三位数字 189。需设置 COMPATIBLE_MODE=2，否则报错。

```
SELECT TO_DATE('189*0112','YY*MM-DD');
```

查询结果如下：

```
189-01-12 00:00:00
```

（4） 当 YY 后面是限定分隔符且首位不是数字，源串中的年份位数不同时：TO_TIMESTAMP 的 YY 可识别 1 位、2 位有效年份数字；一般情况下，TO_DATE 可识别 1

位、2 位有效数字，然后系统根据当前所处的年份，自动补齐成一个完整的年份；特别的是，仅当 COMPATIBLE=2 时，TO_DATE 可识别 3 位有效年份数字，直接读出。

例 1 YY 后面限定分隔符位年份 YEAR-，TO_TIMESTAMP 可识别出源串中的两位数字 29。

```
SELECT TO_TIMESTAMP('29 位年份 YEAR-10-10 12:13:14','YY"位年份 YEAR-"MM-DD HH24:MI:SS');
```

查询结果如下：

```
2029-10-10 12:13:14.000000
```

例 2 YY 后面限定分隔符位年份 YEAR-，TO_DATE 可识别出源串中的三位数字 199。需设置 COMPATIBLE_MODE=2，否则报错。

```
SELECT TO_DATE('199 位年份 YEAR-10-10 12:13:14','YY"位年份 YEAR-"MM-DD HH24:MI:SS');
```

查询结果如下：

```
199-10-10 12:13:14
```

三 数据的处理方法

源串中所有数据的位数必须大于 0，且不能多于 FMT 中分隔符的位数。其中，源串结尾处的数据位数比较特殊，某些情况下，支持源串比格式串短，即当源串中没有数据与格式匹配时，该格式位使用默认的年月日时分秒值。例如：COMPATIBLE_MODE=2，为非 FX 固定格式时，源串中没有数据对应 TO_DATE/TO_TIMESAMP/TO_TIMESTAMP_TZ 的格式 YYYY/YYY/YY/YMM/DD 时会报错；源串中没有数据对应 HH/MI/SS/FF 格式时，则会使用默认值。

- **FX 固定格式器**

FX 是 FMT 固定格式全局修改器。使用了 FX 之后，要求源串对应位置的内容必须和 FMT 中 FX 之后的格式严格匹配。FX 可以出现在任何分隔符可以出现的位置。

例 1　无 FX 情况下，源串格式和 FMT 不需要完全匹配（非限定分隔符个数少于等于源串、非限定分隔符内容不同、固定格式位数不一样等模糊匹配），也能执行成功。

```
SELECT TO_DATE('19 年 08 月 01','YYYY"年"MM"月"DD');
```

查询结果如下：

```
TO_DATE('19 年 08 月 01','YYYY"年"MM"月"DD')

------------------------------------------------------------

 19-08-01 00:00:00
```

例 2 有 FX 情况下，源串对应位置的内容必须和 FMT 中 FX 之后的格式严格匹配，才能执行成功。

以下语句中，FMT 中 FX 之后的格式为 YYYY"年"MM"月"DD，源串对应位置的内容必须严格匹配上述格式。

```
SELECT TO_DATE('2019 年 08 月 01','FXYYYY"年"MM"月"DD');
```

查询结果如下：

```
TO_DATE('2019 年 08 月 01','FXYYYY"年"MM"月"DD')

------------------------------------------------------------

2019-08-01 00:00:00
```

以下语句中，FMT 中 FX 之后的格式为 MM"月"DD，源串对应位置的内容必须严格匹配上述格式。

```
SELECT TO_DATE('19 年 08 月 01','YYYY"年"FXMM"月"DD');
```

查询结果如下：

```
TO_DATE('19 年 08 月 01','YYYY"年"FXMM"月"DD')

------------------------------------------------------------

19-08-01 00:00:00
```

38. 函数 FROM_TZ

```
语法：FROM_TZ(timestamp,timezone|[tz_name])
```

功能：将时间戳类型 timestamp 和时区类型 timezone（或时区名称 tz_name）转化为 timestamp with timezone 类型 。

timestamp 缺省的日期语法为："YYYYMMDD HH:MI:SS"或者"YYYYMMDD "。

时区设置范围为：-12:59~+14:00。

时区名：ASIA/HONG_KONG（即+08:00）。

例 1 使用时区

```
SELECT FROM_TZ(TO_TIMESTAMP('20091101 09:10:21','YYYYMMDD HH:MI:SS '),'+09:00') ;
```

查询结果为：2009-11-01 09:10:21.000000 +09:00

例 2 使用时区名

```
SELECT FROM_TZ(TO_TIMESTAMP('20091101','YYYYMMDD'),'ASIA/HONG_KONG') ;
```

查询结果为：2009-11-01 00:00:00.000000 +08:00

例 3 不指定格式：

```
select from_tz('20091101', 'ASIA/HONG_KONG');
```

查询结果为：2009-11-01 00:00:00.000000 +08:00

39. 函数 TZ_OFFSET

```
语法：TZ_OFFSET(timezone|[tz_name])
```

功能：返回给定的时区和标准时区(UTC)的偏移量。

TZ_OFFSET 的参数可以是：

1) 一个合法的时区名，支持下列时区：

{"Asia/Hong_kong", "+8:00"}：香港时间

{"US/Eastern", "-4:00"}：美国东部时间

{"Asia/Chongqing", "+08:00"}：重庆时间

{"Etc/GMT-8", "+08:00"}：东八区

{"Asia/Urumqi", "+08:00"}：乌鲁木齐时间

{"Asia/Taipei", "+08:00"}：台北时间

{"Asia/Macao", "+08:00"}：澳门时间

{"Asia/Kashgar", "+08:00"}：喀什时间

{"Asia/Harbin", "+08:00"}：哈尔滨时间

{"Singapore", "+08:00"}：新加坡时间

{"PRC", "+08:00"}：中国标准时间

2） 一个与 UTC 标准时区的时间间隔

3） SESSIONTIMEZONE 或 DBTIMEZONE

例 1 TZ_OFFSET 的参数为 DBTIMEZONE

```
SELECT TZ_OFFSET(DBTIMEZONE);
```

查询结果为：+08:00

例 2 TZ_OFFSET 的参数为 US/Eastern

```
SELECT TZ_OFFSET('US/Eastern');
```

查询结果为：-04:00

40. 函数 TRUNC

```
语法：TRUNC(date[, fmt])
```

功能：将日期时间 date 截断到最接近格式参数 fmt 指定的形式。若 fmt 缺省，则返回 date 当天日期。语法与 ROUND 类似，但结果是直接截断，而不是四舍五入。参数及函数的返回类型与 ROUND 相同。参见 ROUND。

例

```
SELECT TRUNC(DATE '1992-10-27', 'scc');
```

查询结果为：1901-01-01

```
SELECT TRUNC(DATE '1992-10-27', 'YEAR') "FIRST OF THE YEAR";
```

查询结果为：1992-01-01

```
SELECT TRUNC(DATE '1992-10-27', 'q');
```

查询结果为：1992-10-01

```
SELECT TRUNC(DATE '1992-10-27', 'month');
```

查询结果为：1992-10-01

```
SELECT TRUNC(TIMESTAMP '1992-10-27 11:00:00', 'ww');
```

查询结果为：1992-10-21 00:00:00.000000

```
SELECT TRUNC(TIMESTAMP '1992-10-27 11:00:00', 'w');
```

查询结果为：1992-10-22 00:00:00.000000

```
SELECT TRUNC(TIMESTAMP '1992-10-27 12:00:01', 'ddd');
```

查询结果为：1992-10-27 00:00:00.000000

```
SELECT TRUNC(DATE '1992-10-27', 'day');
```

查询结果为：1992-10-25

```
SELECT TRUNC(TIMESTAMP '1992-10-27 12:00:31', 'hh');
```

查询结果为：1992-10-27 12:00:00.000000

```
SELECT TRUNC(TIMESTAMP '1992-10-27 12:00:31', 'mi');
```

查询结果为：1992-10-27 12:00:00.000000

41. 函数 WEEK

```
语法：WEEK(date)
```

功能：返回指定日期属于所在年中的第几周。

例

```
SELECT WEEK(DATE '2003-02-10');
```

查询结果为：6

42. 函数 WEEKDAY

```
语法：WEEKDAY(date)
```

功能：返回指定日期的星期值，如果是星期日则返回 0。

例

```
SELECT WEEKDAY(DATE '1998-10-26');
```

查询结果为：1

43. 函数 WEEKS_BETWEEN

```
语法：WEEKS_BETWEEN(date1,date2)
```

功能：返回两个日期之间相差周数。

例

```
SELECT WEEKS_BETWEEN(DATE '1998-2-28', DATE '1998-10-31');
```

查询结果为：-35

44. 函数 YEAR

```
语法：YEAR(date)
```

功能：返回日期中的年分量。

例

```
SELECT YEAR(DATE '2001-05-12');
```

查询结果为：2001

45. 函数 YEARS_BETWEEN

```
语法：YEARS_BETWEEN(date1,date2)
```

功能：返回两个日期之间相差年数。

例

```
SELECT  YEARS_BETWEEN(DATE '1998-2-28', DATE '1999-10-31');
```

查询结果为： -1

46. 函数 LOCALTIME

```
语法：LOCALTIME (n)
```

功能：返回当前时间值，结果类型为 TIME。

参数：n：指定小数秒精度。取值范围 0~6，缺省为 6。

例

```
SELECT LOCALTIME();
```

查询结果为：当前时间值，结果类型为 TIME，如 16:02:45.794391。

47. 函数 LOCALTIMESTAMP

```
语法：LOCALTIMESTAMP (n)
```

功能：返回当前日期时间值，结果类型为 TIMESTAMP。

参数：n：指定小数秒精度。取值范围 0~9，缺省为 6。

例

```
SELECT LOCALTIMESTAMP();
```

查询结果为：当前日期时间值，结果类型为 TIMESTAMP，如 2024-10-17 16:03:30.886407。

48. 函数 OVERLAPS

语法：OVERLAPS (date1,date2,date3,date4)

功能：返回两个时间段是否存在重叠，date1 为 datetime 类型、date2 可以为 datetime 类型也可以为 interval 类型，date3 为 datetime 类型，date4 可为 datetime 类型，也可以 interval 类型，判断(date1,date2),(date3,date4)有无重叠。其中 date2 与 date4 类型必须一致，如果 date2 为 interval year to month，date4 也必须是此类型。结果类型为 BIT，若两个时间段存在重叠返回 1，不重叠返回 0。

例　以下为 date1、date2、date3、date4 分别取 datetime 或 interval 类型的情况。

```
SELECT OVERLAPS('2011-10-3','2011-10-9','2011-10-6','2011-10-13');
```

查询结果为：1

```
SELECT OVERLAPS('2011-10-3','2011-10-9','2011-10-10','2011-10-11');
```

查询结果为：0

```
SELECT OVERLAPS('2011-10-3',INTERVAL '09 23' DAY TO HOUR,'2011-10-10',INTERVAL '09 23' DAY TO HOUR);
```

查询结果为：1

```
SELECT OVERLAPS('2011-10-3',INTERVAL '01 23' DAY TO HOUR,'2011-10-10',INTERVAL '09 23' DAY TO HOUR);
```

查询结果为：0

```
SELECT OVERLAPS('2011-10-3',INTERVAL '1' YEAR TO MONTH,'2012-10-10',INTERVAL '1-1' YEAR TO MONTH);
```

查询结果为：0

```
SELECT OVERLAPS('2011-10-3',INTERVAL '2' YEAR TO MONTH,'2012-10-10',INTERVAL '1-1' YEAR TO MONTH);
```

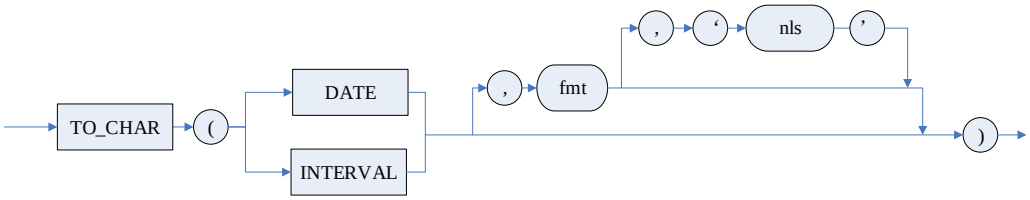
查询结果为：1

49. 函数 TO_CHAR

语法：TO_CHAR(date[,fmt[,'nls']])

图例

函数 TO_CHAR（日期数据类型）



功能：将日期数据类型 DATE 转换为一个在日期格式（FMT）中指定语法的 VARCHAR 类型字符串。FMT、NLS 的用法请参考函数 TO_DATE/TO_TIMESTAMP/TO_TIMESTAMP_TZ 用法。

转换规则如下：

1） 参数为单独字符串的日期时间数据类型

（1） 若日期格式（FMT）中 FF 未指明列精度时，默认精度为 9；

（2） 若日期格式（FMT）缺省，返回原字符串；

（3） 若在日期格式（FMT）中指定'RR'，仅返回原字符串中年份的后两位。

2） 参数为带有 DATE、TIME 等关键字的日期时间数据类型

（1） 若日期格式（FMT）中 FF 未指明列精度时，TIME 默认精度为 6，TIME_TZ，TIMESTAMP_TZ 默认精度为 9，TIMESTAMP 在 ifun_datetime_mode=0 时，默认精度为 6，在 ifun_datetime_mode=1 时，默认精度为 9；

（2） 若日期格式（FMT）缺省：

DATE 的 FMT 默认为'YYYY-MM-DD';

TIME 的 FMT 默认为'HH24:MI:SS.FF6';

TIMESTAMP_TZ 的 FMT 默认为'YYYY-MM-DD HH24:MI:SS.FF TZH:TZM';

TIME_TZ 的 FMT 默认为'HH24:MI:SS.FF TZH:TZM';

TIMESTAMP 在 ifun_datetime_mode=0 时，FMT 默认为'YYYY-MM-DD HH24:MI:SS.FF6'，在 ifun_datetime_mode=1 时，FMT 默认为'YYYY-MM-DD HH24:MI:SS.FF9'。

例 以下是将 DATE 类型数据分别转换为指定 FMT 格式的字符串的情况。

```
SELECT TO_CHAR(SYSDATE,'YYYYMMDD'); //SYSDATE 为系统当前时间
```

查询结果为：20110321

```
SELECT TO_CHAR(SYSDATE,'YYYY/MM/DD');
```

查询结果为：2011/03/21

```
SELECT TO_CHAR(SYSDATE,'HH24:MI');
```

查询结果为：16:56

```
SELECT TO_CHAR(SYSDATE,'YYYYMMDD HH24:MI:SS');
```

查询结果为：20110321 16:56:19

```
SELECT TO_CHAR(SYSDATE,'YYYY-MM-DD HH24:MI:SS');
```

查询结果为：2011-03-21 16:56:28

```
SELECT TO_CHAR(INTERVAL '123-2' YEAR(3) TO MONTH) FROM DUAL;
```

查询结果为：INTERVAL '123-2' YEAR(3) TO MONTH

```
select to_char(sysdate(), 'mon', 'NLS_DATE_LANGUAGE = ENGLISH');
```

查询结果为：DEC

```
select to_char(sysdate(),'mon','NLS_DATE_LANGUAGE="SIMPLIFIED CHINESE" ');
```

查询结果为：12 月

50. 函数 SYSTIMESTAMP

```
语法：SYSTIMESTAMP (n)
```

功能：返回系统当前的时间戳，带有数据库服务器所在操作系统的时区信息。结果类型为 TIMESTAMP WITH TIME ZONE。当机器时区调整时，需要管理员重启服务器，或先执行一次涉及 systimestamp 的调用使其生效。

参数：n：指定小数秒精度。取值范围 0~9，缺省为 6

例

```
SELECT SYSTIMESTAMP();
```

查询结果为：2012-10-10 11:06:12.171000 +08:00

51. 函数 NUMTODSINTERVAL

```
语法：NUMTODSINTERVAL (number, interval_unit)
```

功能：转换一个指定的 DEC 类型到 INTERVAL DAY TO SECOND

参数：

number：任何 dec 类型的值或者可以转换到 dec 类型的表达式

interval_unit：字符串限定 number 的类型： DAY、HOUR、MINUTE、或 SECOND

例

```
SELECT NUMTODSINTERVAL (2.5,'DAY');
```

查询结果为：INTERVAL '2 12:0:0.000000' DAY(9) TO SECOND(6)

52. 函数 NUMTOYMINTERVAL

```
语法：NUMTOYMINTERVAL (number, interval_unit)
```

功能：转换一个指定的 DEC 类型值到 INTERVAL YEAR TO MONTH

参数：

number：任何 dec 类型的值或者可以转换到 dec 类型的表达式

interval_unit：字符串限定 number 的类型：YEAR 或 MONTH

例

```
SELECT NUMTOYMINTERVAL (2.5,'YEAR');
```

查询结果为：INTERVAL '2-6' YEAR(9) TO MONTH

53. 函数 WEEK

语法：WEEK(date, mode)

功能：根据指定的 mode 返回日期为所在年的第几周

其中 mode 可取值及其含义见下表。

表 8.3.5 mode 取值及其含义

mode 值	周起始	返回值范围	说明
0	周日	0~53	本年第一个周日开始为第 1 周，之前算本年第 0 周
1	周一	0~53	本年第一个周一之前如果超过 3 天则算第 1 周，否则算第 0 周
2	周日	1~53	本年第一个周日开始为第 1 周，之前算去年第 5x 周
3	周一	1~53	本年第一个周一之前如果超过 3 天则算第 1 周，否则算去年第 5x 周；年末不足 4 天算明年第 1 周
4	周日	0~53	本年第一个周日之前如果超过 3 天则算第 1 周，否则算第 0 周
5	周一	0~53	本年第一个周一开始为第 1 周，之前算本年第 0 周
6	周日	1~53	本年第一个周日之前如果超过 3 天则算第 1 周，否则算去年第 5x 周；年末不足 4 天算明年第 1 周
7	周一	1~53	本年第一个周一开始为第 1 周，之前算去年第 5x 周

mode 的取值范围为-2147483648~2147483647，但在系统处理时会取 mode=

mode%8。

由于 DM 支持儒略历，并考虑了历史上从儒略历转换至格里高利日期时的异常，将'1582-10-05'到'1582-10-14'之间的日期统一当做'1582-10-15'处理，因此 WEEK 函数对于 1582-10-15 之前日期的计算结果不能保证正确性。

例　以下是根据指定的 mode 返回日期为所在年的第几周的情况

```
SELECT WEEK('2013-12-31',0);
```

查询结果为：52

```
SELECT WEEK('2013-12-31',1);
```

查询结果为：53

```
SELECT WEEK('2013-12-31',2);
```

查询结果为：52

```
SELECT WEEK('2013-12-31',3);
```

查询结果为：1

54. 函数 UNIX_TIMESTAMP

```
语法：UNIX_TIMESTAMP (d datetime)
```

功能：自标准时区的'1970-01-01 00:00:00 +0:00'到本地会话时区的指定时间的秒数差。如果为空，表示到当前时间。

参数：d 可以是一个 DATETIME、TIME、DATE、timestamp with time zone、timestamp with LOCAL time zone 类型（时区忽略，使用当前时区），也可以是一个字符串。或一个 YYYYMMDD、YYMMDD、YMMDD、YYYMMDD 格式的整型 BIGINT。

例　以下是查询自标准时区的'1970-01-01 00:00:00 +0:00'到本地会话时区的指定时间的秒数差的结果。

当前会话时区是+8:00

```
SELECT UNIX_TIMESTAMP(timestamp '1970-01-01 08:00:00');
```

查询结果为：0

```
SELECT UNIX_TIMESTAMP('1970-01-01 17:00:00');
```

查询结果为：32400

```
SELECT UNIX_TIMESTAMP( 20120608 );
```

查询结果为：1339084800

55. 函数 FROM_UNIXTIME

```
语法 1：FROM_UNIXTIME（unixtime number）
```

功能：将自'1970-01-01 00:00:00'的秒数差转成本地会话时区的时间戳类型。

unixtime 为需要处理的参数(该参数是 Unix 时间戳)，可以直接是 Unix 时间戳字符串。

例 1　以下是将自'1970-01-01 00:00:00'的秒数差转成本地会话时区的时间戳类型的查询结果。

```
select FROM_UNIXTIME（'539712061'）;
```

查询结果为：1987-02-08 00:01:01.000000

```
SELECT FROM_UNIXTIME(1249488000);
```

查询结果为：2009-08-06 00:00:00.000000

```
语法 2：FROM_UNIXTIME（unixtime number,fmt varchar）
```

功能：将自'1970-01-01 00:00:00'的秒数差转成本地会话时区的指定 fmt 格式的时间串。

unixtime 为需要处理的参数(该参数是 Unix 时间戳)，可以直接是 Unix 时间戳字符串。

fmt 见 DATE_FORMAT 中的 format 格式（表 8.3.6)。

例 2 以下是将自'1970-01-01 00:00:00'的秒数差转成本地会话时区的指定 fmt 格式的

时间串的查询结果。

```
SELECT FROM_UNIXTIME(1249488000 ,'%D');
```

查询结果为：6th

56. 函数 SESSIONTIMEZONE

```
语法：SESSIONTIMEZONE
```

功能：查看当前会话的时区

例

```
SELECT SESSIONTIMEZONE FROM DUAL;
```

查询结果如下：

```
SESSIONTIMEZONE

---------------

+08:00
```

57. 函数 DBTIMEZONE

```
语法：DBTIMEZONE
```

功能：查看当前数据库时区，即安装数据库时操作系统的时区。

例

```
SELECT DBTIMEZONE FROM DUAL;
```

查询结果如下：

```
DBTIMEZONE

---------------

+08:00
```

58. 函数 DATE_FORMAT

语法：DATE_FORMAT（d datetime, format varchar）

功能：以不同的格式显示日期/时间数据。对于非数值结果（例如：月份，星期，上/下午），受 NLS_DATE_LANGUAGE 影响，可能会有中文或英文两种输出。

参数：

d：可以是一个 DATETIME、TIME、DATE、timestamp with time zone、timestamp with LOCAL time zone 类型（时区忽略，使用当前时区）。

format： 规定日期/时间的输出格式，具体见下表。

表 8.3.6 format 释义

format	释义
%a	缩写星期名
%b	缩写月名
%c	月，数值(0-12)
%D	带有英文后缀的月中的天
%d	月的天，数值(00-31)
%e	月的天，数值(0-31)
%f	有效的小数秒
%H	小时，数值(00-23)
%h	小时，数值(01-12)
%I	小时，数值(01-12)
%i	分钟，数值(00-59)
%j	年的天，数值(001-366)
%k	小时，数值(0-23)
%l	小时，数值(1-12)

%M	月名
%m	月，数值(00-12)
%p	AM 或 PM
%r	时间，12-小时(hh:mm:ss AM 或 PM)
%S	秒，数值(00-59)
%s	秒，数值(00-59)
%T	时间，24-小时(hh:mm:ss)
%U	周，数值(00-53)，星期日是一周的第一天
%u	星期，数值(00-52)，这里星期一是星期的第一天
%W	星期名字(SUNDAY、MONDAY、TUESDAY、WEDNESDAY、THURSDAY、FRIDAY、SATURDAY)
%w	表示星期几的数字(0= SUNDAY、1= MONDAY、2= TUESDAY、3= WEDNESDAY、4= THURSDAY、5= FRIDAY、6= SATURDAY)
%Y	年，数字，4 位
%y	年，数字，2 位
%V	周，数值(01-53)，星期日是一周的第一天。该年的第一周从第一个周日开始计算，第一个周日前的日期作为上一年的最后一周
%X	年，4 位数字，星期日是一周的第一天。该年从第一个周日开始计算，第一个周日前的日期作为上一年。
%v	周，数值(01-53)，星期一是一周的第一天；如果第一周超过三天则算第一周，否则算上年最后一周，年末不足 4 天的算明年第一周
%x	年，4 位数字，星期一是一周的第一天；如果第一周超过三天则算第一周，否则算上年最后一周，年末不足 4 天的算明年的第一周

%%	一个文字“%”

格式符均是一个“%“和一个英文字母的格式，用以控制显示日期或时间数据的不同内容，其余 FORMAT 串中的内容均视为分隔符，直接显示。

例

```
select date_format(timestamp '1980-1-1 1:1:1.123456789','%Y-%m-%d %H:%i:%s');
```

查询结果为：1980-01-01 01:01:01

59. 函数 TIME_TO_SEC

```
语法：TIME_TO_SEC（d datetime）
```

功能：将时间换算成秒

d 可以是一个 DATETIME、TIME、DATE、timestamp with time zone、timestamp with LOCAL time zone 类型（时区忽略，使用当前时区）。

例

```
select time_to_sec(timestamp '1900-1-1 23:59:59 +8:00');
```

查询结果为：86399

```
select time_to_sec(time '23:59:59');
```

查询结果为：86399

60. 函数 SEC_TO_TIME

```
语法：SEC_TO_TIME（sec numeric）
```

功能：将秒换算成时间，返回值范围为-838:59:59~838:59:59。

例

```
select sec_to_time(104399);
```

查询结果为： 28:59:59

61. 函数 TO_DAYS

```
语法：TO_DAYS (d timestamp)
```

功能：转换成公元 0 年 1 月 1 日的天数差。

d 是要转换的日期时间类型。或一个 YYYYMMDD YYMMDD YMMDD YYYMMDD 格式的整型 BIGINT。

例

```
select  to_days('2021-11-11');
```

查询结果为：738470

62. 函数 DATE_ADD

```
语法：DATE_ADD(d datetime, expr interval)
```

功能：返回一个日期或时间值加上一个时间间隔的时间值。d 可以是 DATETIME、TIME、DATE 类型。

对于 DATE_ADD 计算结果类型的规则说明如下：

- 如果 d 是 NULL 则返回 NULL。
- 如果 d 是 DATETIME 类型，则返回 DATETIME 类型的结果。
- 如果 d 是 DATE 类型，并且计算仅涉及 YEAR 和 MONTH 日期部分（即不包含 DAY 和时间部分），则返回 DATE 类型的结果。
- 如果 d 是 DATE 类型，并且计算仅涉及 DAY 及 HOUR、MINUTE、SECOND 等时间部分（即不包含 YEAR、MONTH 部分），则返回 DATETIME 类型的结果。
- 如果 d 是 TIME 类型，并且计算仅涉及 DAY 及 HOUR、MINUTE、SECOND 等时间部分（即不包含 YEAR、MONTH 部分），则返回 TIME 类型的结果。
- 如果 d 是 TIME 类型，并且计算仅涉及 YEAR、MONTH 部分（即不包含 DAY 和时间部分），则返回 NULL。

例

```
SELECT DATE_ADD('2020-07-12 12:20:30', INTERVAL '2 1 ' DAY TO SECOND);
```

查询结果为：2020-07-14 13:20:30

```
SELECT DATE_ADD(DATE '2020-07-12', INTERVAL '2' YEAR);
```

查询结果为：2022-07-12

```
SELECT DATE_ADD(DATE '2020-07-12', INTERVAL '2 1 ' DAY TO HOUR);
```

查询结果为：2020-07-14 01:00:00.000000

```
SELECT DATE_ADD(TIME '12:20:30', INTERVAL '2 1 ' HOUR TO SECOND);
```

查询结果为：14:21:30.000000

```
SELECT DATE_ADD(TIME '12:20:30', INTERVAL '2' MONTH);
```

查询结果为：NULL

63. 函数 DATE_SUB

```
语法：DATE_SUB(d datetime, expr interval)
```

功能：返回一个日期或时间值减去一个时间间隔的时间值。d 可以是 DATETIME、TIME、DATE 类型。

DATE_SUB 计算结果类型的规则与 DATE_ADD 一致。

例

```
SELECT DATE_SUB ('2020-07-12 12:20:30', INTERVAL '2 1 ' DAY TO SECOND);
```

查询结果为：2020-07-10 11:20:30

```
SELECT DATE_SUB (DATE '2020-07-12', INTERVAL '2' YEAR);
```

查询结果为：2018-07-12

```
SELECT DATE_SUB (DATE '2020-07-12', INTERVAL '2 1 ' DAY TO HOUR);
```

查询结果为：2020-07-09 23:00:00.000000

```
SELECT DATE_SUB (TIME '12:20:30', INTERVAL '2 1 ' HOUR TO SECOND);
```

查询结果为：10:19:30.000000

```
SELECT DATE_SUB (TIME '12:20:30', INTERVAL '2' MONTH);
```

查询结果为：NULL

64. 函数 SYS_EXTRACT_UTC

```
语法：SYS_EXTRACT_UTC(d timestamp)
```

功能：提取 UTC 时区信息，将所给时区信息转换为 UTC 时区信息。

例

```
SELECT SYS_EXTRACT_UTC('2000-03-28 11:30:00.00 -08:00') FROM DUAL;
```

查询结果为： 2000-03-28 19:30:00.000000

65. 函数 TO_DSINTERVAL

```
语法：TO_DSINTERVAL(d char)
```

功能：将表示时间间隔的字符串转换为 INTERVAL DAY TO SECOND 类型。

例

```
SELECT TO_DSINTERVAL('100 00:00:00') FROM DUAL;
```

查询结果为：INTERVAL '100 0:0:0.000000' DAY(9) TO SECOND(6)

66. 函数 TO_YMINTERVAL

```
语法：TO_YMINTERVAL(d char)
```

功能：将表示时间间隔的字符串转换为 INTERVAL YEAR TO MONTH 类型。

例

```
SELECT TO_YMINTERVAL('01-02') FROM DUAL;
```

查询结果为：INTERVAL '1-2' YEAR(9) TO MONTH

67. 函数 ADDTIME

```
语法：ADDTIME(expr1, expr2)
```

功能：在指定时间或日期时间值 expr1 上增加一个时间间隔值 expr2，返回新的时间或新的日期时间值，返回类型与 expr1 一致。expr2 可以为有效的日-时间隔类数据（具体可见 1.4.2 日期时间数据类型中的介绍），当指定为数值类型时会将该值转换为间隔秒。

当 expr1 为字符串类型时，DM 会优先将其转换为 DATETIME 类型进行运算；若想返回结果为 TIME 类型，需要先对 expr1 进行类型转换。

当 expr2 指定为数值类型或字符串形式的数值类型时，须为小于等于 99.999999 的值，否则返回 NULL。

当 expr2 指定为形如'1 11'的表达式时，识别为间隔秒，'1 11'会作为 1.11 秒进行处理；当 expr2 指定为形如'1 1:11'的表达式时，识别为 DAY TO MINUTE 间隔，'1 1:11'会作为 1 天 1 小时 11 分钟进行处理。

例

```
SELECT ADDTIME('00:00:00.999999', '99 23:59:59.999999');
```

查询结果为：1900-04-11 00:00:00.999998

```
SELECT ADDTIME(TIME '23:00:00.999999', '23:01:01.000099');
```

查询结果为：22:01:02.000098

```
SELECT ADDTIME('2007-12-31 05:05:05.01', INTERVAL '09 23:56' DAY TO MINUTE);
```

查询结果为：2008-01-10 05:01:05.010000

```
SELECT ADDTIME(cast('12:0:0' as time),'5 50');
```

查询结果为：12:00:05.500000

8.4 空值判断函数

空值判断函数用于判断参数是否为 NULL，或根据参数返回 NULL。

1．函数 COALESCE

```
语法：COALESCE(n1,n2,...,nx)
```

功能：返回其参数中第一个非空的值，如果所有参数均为 NULL，则返回 NULL。参数数据类型不相同时，DM 会进行隐式数据类型转换。

例　以下是分别返回不同传入值的情况。

```
SELECT COALESCE(1,NULL);
```

查询结果为：1

```
SELECT COALESCE(NULL,TIME '12:00:00',TIME '11:00:00');
```

查询结果为：12:00:00.000000

```
SELECT COALESCE(NULL,NULL,NULL,NULL);
```

查询结果为：NULL

2．函数 IFNULL

```
语法：IFNULL(n1,n2)
```

功能：当表达式 n1 为非 NULL 时，返回 n1；若 n1 为 NULL，则返回表达式 n2 的值。若 n1 与 n2 为不同数据类型时，DM 会进行隐式数据类型转换，若数据类型转换出错，则会报错。

例　以下是分别返回不同传入值的情况。

```
SELECT IFNULL(1,3);
```

查询结果为：1

```
SELECT IFNULL(NULL,3);
```

查询结果为：3

```
SELECT IFNULL(' ',2);
```

查询结果为：□。其中，□表示空格

3．函数 ISNULL

```
语法：ISNULL(n1,n2)
```

功能：当表达式 n1 为非空时，返回 n1；若 n1 为空，则返回表达式 n2 的值。n2 的数据类型应能转为 n1 的数据类型，否则会报错。

例

```
SELECT ISNULL(1,3);
```

查询结果为：1

4．函数 NULLIF

```
语法：NULLIF(n1,n2)
```

功能：如果 n1=n2，返回 NULL，否则返回 n1。

例

```
SELECT NULLIF(1,2);
```

查询结果为：1

```
SELECT NULLIF(1,1);
```

查询结果为：NULL

5．函数 NVL

```
语法：NVL(n1,n2)
```

功能：返回第一个非空的值。若 n1 与 n2 为不同数据类型时，DM 会进行隐式数据类型转换，若数据类型转换出错，则会报错。当 n1/n2 都为 null 时，返回 null。

转换规则说明如下：

1） 当 n1 为确定性数据类型时，以 n1 为准；当 n1 的数据类型不确定时，以找到的第一个确定性数据类型为准；如果都为不确定数据类型时则定为 varchar 数据类型；

2） 若参数一个为精确浮点数另一个为不精确浮点数，结果为不精确浮点数，可能导致数据精度丢失；

3） 两个参数为不同数据类型时，结果为精度大的数据类型；

4） 参数若为字符串类型，不论是 char 还是 varchar，结果类型均为 varchar，精度以大的为准；

5） 参数类型都为时间日期类型时，结果类型为 n1 的类型；但若参数有 DATE 或 TIME 类型时，结果类型为 n1 和 n2 中精度较大的类型。

例

```
select nvl(1,1.123);
```

查询结果为：1（结果数据类型为 INT，精度为默认值 10）

```
create table t1(a int,b double);
insert into t1 values(1,2.111);
select nvl(a,b) from t1;
```

查询结果为：1.0（结果数据类型为 DOUBLE，精度为默认值 53）

```
create table t2(a double,b varchar);
insert into t2 values (1.111,'avcc');
select nvl(a,b) from t2;
```

查询结果报错：

```
[-6128]:无效的数字
```

6．函数 NULL_EQU

```
语法：NULL_EQU(n1,n2)
```

功能：返回两个类型相同的值的比较，当 n1=n2 或 n1、n2 两个值中出现 null 时，返回 1。类型可以是 INT、BIT、BIGINT、FLOAT、DOUBLE、DEC、VARCHAR、DATE、TIME、

TIME ZONE、DATETIME、DATETIME ZONE、INTERVAL 等。

例

```
select null_equ(1,1);
```

查询结果为：1

```
select null_equ(1,3);
```

查询结果为：0

```
select null_equ(1,null);
```

查询结果为：1

8.5 类型转换函数

1．函数 CAST

```
语法：CAST(value AS type)
```

功能：将参数 value 转换为 type 类型返回。类型之间转换的相容性如下表所示：表中，“允许”表示这种语法有效且不受限制，“—”表示语法无效，“受限”表示转换还受到具体参数值的影响。

数值类型为：精确数值类型和近似数值类型。

精确数值类型为：NUMERIC、NUMBER、DECIMAL、DEC、INTEGER、INT、PLS_INTEGER、BIGINT、TINYINT、BYTE、SMALLINT。

近似数值类型为：FLOAT、REAL、DOUBLE PRECISION。

字符串为：变长字符串、固定字符串和 ROWID 类型。其中 ROWID 类型只能和字符串中的 VARCHAR（或 VARCHAR2）相互转换。

变长字符串为：VARCHAR（或 VARCHAR2）。

固定字符串为：CHAR、CHARACTER。

ROWID 类型：ROWID。

字符串大对象为：CLOB、TEXT。

二进制为：BINARY、VARBINARY、RAW。

二进制大对象为：BLOB、IMAGE。

判断类型为：BIT、BOOLEAN。

日期为：DATE。时间为：TIME。时间戳为：TIMESTAMP。

时间时区为：TIME WITH TIME ZONE。

时间戳时区为：TIMESTAMP WITH TIME ZONE。

年月时间间隔为：INTERVAL YEAR TO MONTH、INTERVAL YEAR、INTERVAL MONTH。

日时时间间隔为：INTERVAL DAY、INTERVAL DAY TO HOUR、INTERVAL DAY TO MINUTE、INTERVAL DAY TO SECOND、INTERVAL HOUR、INTERVAL HOUR TO MINUTE、INTERVAL HOUR TO SECOND、INTERVAL MINUTE、INTERVAL MINUTE TO SECOND、INTERVAL SECOND。

其中，NUMBER 向 BINARY 的转换受参数 DEC_BIN_CVT_MODE 影响。

表 8.5.1 CAST 类型转换相容矩阵

Value	type 数据类型													
数据类型	数值类型	字符串	ROWID 类型	字符串大对象	二进制	二进制大对象	判断类型	日期	时间	时间戳	时间时区	时间戳时区	年月时间间隔	日时时间间隔
数值类型	受限	受限	—	—	允许	—	允许	受限	受限	受限	—	—	受限	受限
字符串	允	允	允许	允许	允	允许	受	受	受	受	受	受	允许	允许

	许	许			许		限	限	限	限	限	限		
ROWID 类型	受限	允许	允许	—	—	—	—	—	—	—	—	—	—	—
字符串大对象	—	允许	—	—	—	允许	-	—	—	—	—	—	—	—
二进制	允许	允许	—	—	允许	允许	受限	—	—	—	—	—	—	—
二进制大对象	—	受限	—	允许	—	允许	受限	—	—	—	—	—	—	—
判断类型值（不支持判断类型表达式）	受限	允许	允许	允许	允许	-	允许	允许	允许	允许	-	-	受限	受限
日期	—	允许	—	允许	—	—	受限	允许	—	允许	—	允许	—	—
时间	—	允许	—	允许	—	—	受限	—	允许	允许	允许	允许	—	—
时间戳	—	允许	—	—	—	—	受限	允许	允许	允许	允许	允许	—	—
时间时区	—	允许	—	—	—	—	受限	—	允许	允许	允许	—	—	—
时间戳	—	允	—	—	—	—	受	允	允	允	—	允	—	—

时区		许					限	许	许	许		许		
年月时间间隔	—	允许	—	—	—	—	-	—	—	—	—	—	受限	—
日时时间间隔	—	允许	—	—	—	—	-	—	—	—	—	—	—	受限

例　以下分别将参数 value 转换为指定 type 类型返回的情况。

```
SELECT CAST(100.5678 AS NUMERIC(10,2));
```

查询结果为：100.57

```
SELECT CAST(100.5678 AS VARCHAR(8));
```

查询结果为：100.5678

```
SELECT CAST('100.5678' AS INTEGER);
```

查询结果为：101

```
SELECT CAST('1900-01-01 00:00:00.000000' AS BOOLEAN);
```

查询结果为：1

```
SELECT CAST(12345 AS char(5));
```

查询结果为：12345

2．函数 TREAT

TREAT 函数用于在运行时检查表达式是否可以转换为另一个数据类型，通常用于检查自定义的父子类型之间的转换。TREAT 函数可以将一个父类对象指针指向一个子类对象并保留子类对象的精度信息，类似于一个指针引用。具体作用为：

1）改变变量类型，将子类型对象值赋值给父类型变量，即一个父类对象指针指向一个子类型对象实例。

2）获取行或列对应对象数据类型的子类型拥有的属性或者方法。

语法格式

```
TREAT(<expr> AS [ REF ] [<schema>.]<type>)
```

参数

<exp>：变量表达式。

<schema>：模式名。

<type>：数据类型。

说明

1）目前 DM 还没有支持引用数据类型，所以 REF 部分仅做语法支持，暂无实际意义；

2）对象表合并后，TREAT 可以将对象表中保存的子类对象打包信息转换成子类对象，但是对于常规表的对象列无法获取子类对象的精度信息。

举例说明

例

```
//创建基类型，即父类型
create or replace type tp1 as object(
    a int
) not final
//第一个子类
create or replace type tp1_1 under tp1(
    b int
)
//第二个子类
create or replace type tp1_2 under tp1(
    c varchar(120)
)
```

```
declare
    tp1_v     tp1; //父类型变量 1
    tp1_v1    tp1; //父类型变量 2
    tp11_v    tp1_1; //第一子类型变量 1
    tp11_v1   tp1_1; //第一子类型变量 2
    tp12_v    tp1_2; //第二子类型变量
begin
    tp1_v  := tp1(1);
   tp11_v := tp1_1(11, 2);
    tp12_v := tp1_2(12, 3);
    //父类型变量转子类型，结果为空
    select treat(tp1_v as tp1_1) into tp11_v1 from dual;
    //子类型变量转父类型，成功
    select treat(tp11_v as tp1) into tp1_v1 from dual;
    //子类型变量转子类型，报错无法转换的数据类型
   select treat(tp11_v as tp1_2) into tp12_v from dual;
    //父类型指针存放子类型对象，再转子类型，成功
    tp1_v1 := tp1_1(113, 4);
    select treat(tp1_v1 as tp1_1) into tp11_v1 from dual;
    dbms_output.put_line('a is : ' || tp11_v1.a);
    dbms_output.put_line('b is : ' || tp11_v1.b);
end;
/
```

3．函数 CONVERT

语法 1：CONVERT(type,value [,style])

功能：用于当 INI 参数 ENABLE_CS_CVT=0 时，将参数 value 转换为 type 类型返回，该情况下 CONVERT 函数转换为 CAST 函数执行，其类型转换相容矩阵与函数 CAST()的相同。

在 style 合法的前提下，若同时满足 type 为字符类型且 value 为日期时间类型数据，则日期时间数据可以通过 style 的值进行其对应的日期掩码转化。对于不是日期表达式转字符类型的情况，忽略 style 表达式。

style 为指定 CONVERT 函数如何转换日期时间表达式的整数表达式，默认为空，值对应的日期掩码如下表 8.5.2。在 COMPATIBLE_MODE 取值为 0 的情况下，style=126 时，value 值不带时区会进行报错，带时区的输出结果是带时间格式 T 的；style=127 时，VALUE 值无论是否带时区都可以成功输出且都带时间格式 T，但是结果都不带时区。COMPATIBLE_MODE 取 3 时（兼容 SQL SERVER），style=126 时所有时间类型的输出结果都带时间格式 T，对于带时区的数据类型输出时区；style=127 时都带 T，对于带时区的数据类型转化为标准时区后输出，并且带 Z，其余正常输出。DM 目前未支持回历，style 为 130 或 131 时不转换为回历日期。若 style 参数值取 NULL，则结果返回 NULL；除表中的其他值均作为非法值进行处理。

表 8.5.2 style 取值与对应日期掩码

style 取值	标准	日期掩码格式
0 或 100	datetime 和 smalldatetime 的默认值	mon dd yyyy hh:mi 上午（或下午）tzh
1 或 101	美国	1 = mm/dd/yy 101 = mm/dd/yyyy

2 或 102	ANSI	2 = yy.mm.dd 102 = yyyy.mm.dd
3 或 103	英国/法国	3 = dd/mm/yy 103 = dd/mm/yyyy
4 或 104	德语	4 = dd.mm.yy 104 = dd.mm.yyyy
5 或 105	意大利语	5 = dd-mm-yy 105 = dd-mm-yyyy
6 或 106	-	6 = dd mon yy 106 = dd mon yyyy
7 或 107	-	7 = Mon dd, yy 107 = Mon dd, yyyy
8、24 或 108	-	hh:mi:ss tzh（24 小时制）
9 或 109	默认格式 + 毫秒	mon dd yyyy hh:mi:ss.mmm 上午（或 下午）
10 或 110	USA	10 = mm-dd-yy 110 = mm-dd-yyyy
11 或 111	日本	11 = yy/mm/dd 111 = yyyy/mm/dd
12 或 112	ISO	12 = yymmdd 112 = yyyymmdd
13 或 113	欧洲默认格式 + 毫秒	dd mon yyyy hh:mi:ss.mmm（24 小时制）

14 或 114	-	hh:mi:ss.mmm tzh（24 小时制）
20 或 120	ODBC 规范	yyyy-mm-dd hh:mi:ss（24 小时制）
21、25 或 121	time、date、datetime2 和 datetimeoffset 的 ODBC 规范（用毫秒表示）默认值	yyyy-mm-dd hh:mi:ss.mmm（24 小时制）
22	美国	mm/dd/yy hh:mi:ss 上午（或下午）
23	ISO8601	yyyy-mm-dd
126	ISO8601	yyyy-mm-ddThh24:mi:ss.mstzh:tzm
127	包括时区 Z 的 ISO8601	yyyy-mm-ddThh24:mi:ss.msz
130	回历 5	dd mon yyyy hh:mi:ss.mmm 上午（或 下午）
131	回历 5	dd/mm/yyyy hh:mi:ss.mmm 上午（或 下午）

例

```
SELECT CONVERT(VARCHAR(8),100.5678);
```

查询结果为：100.5678

```
SELECT CONVERT(INTEGER, '100.5678');
```

查询结果为：101

```
SELECT CONVERT(CHAR(5),12345);
```

查询结果为：12345

```
SELECT CONVERT(CHAR(50), NOW, 120);
```

查询结果为：2023-05-08 11:01:18

```
SELECT CONVERT(CHAR(50),GETDATE(),14);
```

查询结果为：11:02:51.526

```
语法 2：CONVERT(char, dest_char_set [,source_char_set ] )
```

功能：用于当 INI 参数 ENABLE_CS_CVT=1 时，将字符串 char 从源串字符集（source_char_set）转换成目的字符集（dest_char_set）。一般要求源串字符集与数据库服务器所在操作系统的字符集一致，否则转换结果会跟源串字符集与数据库服务器所在操作系统的字符集一致时不一样。如果没有指定源串字符集，默认为数据库服务器字符集。目前只支持 ZHS16GBK、AL32UTF8、UTF8 和 ZHS32GB18030 四种字符集。

例

```
Select convert('席 μ', 'UTF8', 'ZHS16GBK') from dual;
```

查询结果为：甯?

4．函数 HEXTORAW

```
语法：HEXTORAW (string)
```

功能：将由 string 表示的二进制字符串转换为一个 varbinary 数值类型。

例

```
SELECT HEXTORAW ('abcdef');
```

查询结果为：0xABCDEF

```
SELECT HEXTORAW ('B4EFC3CECAFDBEDDBFE2D3D0CFDEB9ABCBBE');
```

查询结果为：0xB4EFC3CECAFDBEDDBFE2D3D0CFDEB9ABCBBE

5．函数 RAWTOHEX

```
语法：RAWTOHEX (binary)
```

功能：将 RAW 类数值 binary 转换为一个相应的十六进制表示的字符串。binary 中的每个字节都被转换为一个双字节的字符串。RAWTOHEX 和 HEXTORAW 是两个相反的函数。

例

```
SELECT RAWTOHEX('达梦数据库有限公司');
```

查询结果为：B4EFC3CECAFDBEDDBFE2D3D0CFDEB9ABCBBE

```
SELECT RAWTOHEX('13');
```

查询结果为：3133

6．函数 BINTOCHAR

```
语法：BINTOCHAR (binary)
```

功能：将数值 binary 转换为字符串。

例

```
SELECT BINTOCHAR ('0x61626364');
```

查询结果为：abcd

7.函数 TO_BLOB

```
语法：TO_BLOB (varbinary)
```

功能：将数值 varbinary 转换为 blob。

例

```
SELECT TO_BLOB(utl_raw.cast_to_raw('abcd'));
```

查询结果为：0x61626364

8.函数 UNHEX

```
语法：UNHEX(char)
```

功能：将十六进制格式的字符串转化为原来的格式字符串或 VARBINARY。当参数为空串时，返回空串；当参数为 NULL 时，返回 NULL。

返回值类型通过静态 INI 参数 BINARY_AS_HEX 控制，修改参数值并重启服务器后生效，缺省值为 1。当 BINARY_AS_HEX=0，返回值类型为 VARCHAR；当 BINARY_AS_HEX=1，返回值类型为 VARBINARY。

例 1 当 BINARY_AS_HEX=0 时。

```
SELECT UNHEX('616263');
```

查询结果为：abc

例 2 当 BINARY_AS_HEX=1 时。

```
SELECT UNHEX('616263');
```

查询结果为：0x616263

9.函数 HEX

```
语法：HEX(char)
```

功能：当参数为 NULL 时，返回 NULL；当 value 数据类型为字符串时，将字符串转换为一个相应的十六进制表示的字符串，value 为空串时，返回空串；当 value 数据类型为 BIGINT 时，将其转为 16 进制形式，BIGINT 之外的各种数值类型的数据，会先转为 BIGINT，再传入 HEX 函数。

例 1 value 为字符串

```
SELECT HEX('abc');
```

查询结果为：616263

例 2 value 为整数

```
SELECT HEX(255);
```

查询结果为：FF

10.函数 CHARTOBIN

```
语法：CHARTOBIN(char)
```

功能：将字符串转换为一个 binary 数值类型。当参数为 NULL 或空串时，返回 NULL。

例

```
SELECT CHARTOBIN ('abcdef');
```

查询结果为：0x616263646566

8.6 杂类函数

1. 函数 DECODE

```
语法：DECODE(exp, search1, result1, … searchn, resultn[,default])
```

功能：查表译码，DECODE 函数将 exp 与 search1,search2, … searchn 相比较，如果等于 searchx, 则返回 resultx, 如果没有找到匹配项, 则返回 default, 如果未定义 default, 返回 NULL。

例

```
SELECT DECODE(1, 1, 'A', 2, 'B');
```

查询结果为：'A'

```
SELECT DECODE(3, 1, 'A', 2, 'B');
```

查询结果为：NULL

```
SELECT DECODE(3, 1, 'A', 2, 'B', 'C');
```

查询结果为：'C'

2. 函数 ISDATE

```
语法：ISDATE(exp)
```

功能：判断给定表达式是否为有效的日期，是返回 1，否则返回 0。

例

```
SELECT ISDATE('2012-10-9');
```

查询结果为：1

```
SELECT ISDATE('2012-10-9 13:23:37');
```

查询结果为：1

```
SELECT ISDATE(100);
```

查询结果为：0

3. 函数 ISNUMERIC

```
语法：ISNUMERIC(exp)
```

功能：判断给定表达式是否为有效的数值，是返回 1，否则返回 0。

例

```
SELECT ISNUMERIC(1.323E+100);
```

查询结果为：1

```
SELECT ISNUMERIC('2a');
```

查询结果为：0

4. 函数 DM_HASH

```
语法：DM_HASH (exp)
```

功能：根据给定表达式生成 HASH 值，返回结果为整型。

例

```
SELECT DM_HASH('DM HASH VALUE');
```

查询结果为：3086393668

```
SELECT DM_HASH(101);
```

查询结果为：1653893674

5. 函数 LNNVL

```
语法：LNNVL(condition)
```

参数：condition为布尔表达式。

功能：当condition表达式计算结果值为FALSE或者UNKNOWN时，返回1；当计算结果

值为TRUE时，返回0。

例

```
SELECT LNNVL(1=0);
```

查询结果为：1

```
SELECT T1.NAME, T2.NAME

FROM  PRODUCTION.PRODUCT_CATEGORY T1  RIGHT OUTER JOIN

PRODUCTION.PRODUCT_SUBCATEGORY T2

ON T1.PRODUCT_CATEGORYID = T2.PRODUCT_CATEGORYID WHERE LNNVL(T1.NAME<>'计算机');
```

查询结果如下：

```
NAME     NAME

------  ---------------

NULL     历史

计算机    计算机理论

计算机    计算机体系结构

计算机    操作系统

计算机    程序设计

计算机    数据库

计算机    软件工程

计算机    信息安全

计算机    多媒体
```

6. 函数 LENGTHB

```
语法：LENGTHB(value)
```

参数：value可为除BFILE外DM_SQL支持的所有数据类型，具体数据类型介绍可见1.4

DM_SQL所支持的数据类型。

功能：返回value的字节数。当value为定长类型时，返回定义的字节长度；当value为NULL时，返回NULL。

例

```
SELECT LENGTHB(0x1234567) "Length in bytes";
```

查询结果为：4

7. 函数 FIELD

```
语法：函数 FIELD(value, e1, e2, e3, e4...en)
```

功能：根据指定元素value在输入列表“e1、e2、e3、e4...en”中的位置返回相应的位置序号，不在输入列表时则返回0。

FIELD()一般用在ORDER BY子句之后，将获取到的结果集按照输入列表的顺序进行排序。value不在输入列表的结果，排在结果集的前面。

例 1 查询 50 在后面列表 10、50、100 中的位置序号。

```
SELECT field(50,10,50,100);
```

查询结果为：2

例 2 按照列表中指定的顺序输出结果集。不符合条件的结果放在结果集前面。

```
select * from PERSON.ADDRESS order by field(city,'武汉市洪山区','武汉市汉阳区','武汉市武昌区','武汉市江汉区');
```

查询结果如下：

```
ADDRESSID   ADDRESS1                          ADDRESS2 CITY          POSTALCODE
----------- ------------------------------ -------- ------------ ----------
3           青山区青翠苑 1 号                          武汉市青山区   430080
15          洪山区关山春晓 11-1-202                     武汉市洪山区   430073
```

8	洪山区关山春晓 51-1-702	武汉市洪山区	430073
13	洪山区关山春晓 55-1-202	武汉市洪山区	430073
6	洪山区保利花园 50-1-304	武汉市洪山区	430073
7	洪山区保利花园 51-1-702	武汉市洪山区	430073
16	洪山区光谷软件园 C1_501	武汉市洪山区	430073
2	洪山区 369 号金地太阳城 57-2-302	武汉市洪山区	430073
14	洪山区关山春晓 10-1-202	武汉市洪山区	430073
1	洪山区 369 号金地太阳城 56-1-202	武汉市洪山区	430073
5	汉阳大道熊家湾 15 号	武汉市汉阳区	430050
11	武昌区武船新村 1 号	武汉市武昌区	430063
4	武昌区武船新村 115 号	武汉市武昌区	430063
12	江汉区发展大道 423 号	武汉市江汉区	430023
10	江汉区发展大道 555 号	武汉市江汉区	430023
9	江汉区发展大道 561 号	武汉市江汉区	430023

8. 函数 ORA_HASH

语法：ORA_HASH(exp [,max_bucket [,seed_value]])

功能：为表达式exp生成HASH桶值。根据exp和随机数seed_value生成位于0到max_bucket（包括0和max_bucket）之间的HASH桶值，返回结果为整型。

参数：

exp：输入值。

max_bucket：返回的HASH桶值的最大值。取值范围为0~4294967295，缺省为4294967295。

seed_value：随机数。同一个exp搭配不同的seed_value会返回不同的结果（偶尔

也会有巧合，得到相同值)。取值范围为0~4294967295。

使用说明:

只要参数存在NULL就返回NULL。

例

```
SELECT ORA_HASH('ORA HASH VALUE');
```

查询结果为：1038192070

```
SELECT ORA_HASH('ORA HASH VALUE',5);
```

查询结果为：4

```
SELECT ORA_HASH('ORA HASH VALUE',5,100);
```

查询结果为：1

```
SELECT ORA_HASH('ORA HASH VALUE',5,200);
```

查询结果为：2

```
SELECT ORA_HASH('ORA HASH VALUE',88,100);
```

查询结果为：14

9. 函数 IF

```
语法：IF(expr1,expr2,expr3)
```

功能：expr1为布尔表达式，如果其值为TRUE，则返回expr2值，否则返回expr3值。若expr2与expr3为不同数据类型，则需要两者的数据类型可进行转换比较，否则报错。

参数:

expr1：布尔表达式，其值为1则为TRUE，为0则为FALSE。

expr2：输入值1。

expr3：输入值2。

例

```
SELECT IF(1, 2, 3);
```

查询结果为：2

```
SELECT IF(1, 'apple', 'banana');
```

查询结果为：apple

```
SELECT IF(0, 'apple', 'banana');
```

查询结果为：banana

10. 函数 WIDTH_BUCKET

```
语法：WIDTH_BUCKET(expr,low_value,high_value,num_buckets)
```

功能：将指定的范围[low_value,high_value)划分为num_buckets个等宽的直方图，每个桶的范围都是左闭右开的，返回指定表达式expr的值所属的桶序号。整个坐标轴被划分为num_buckets+2个区域，从左向右区域的序号从0依次到num_buckets+1；指定范围的左边界值也可以大于右边界值。

参数：

expr：指定表达式，此表达式的计算结果必须为数值类型、日期时间类型、INTERVAL YEAR TO MONTH类型或INTERVAL DAY TO SECOND类型，或可以隐式转换为上述类型；当表达式计算结果为NULL时，函数返回NULL。

low_value：指定范围的左边界值，必须和expr的计算结果类型一致，且不能为NULL。

high_value：指定范围的右边界值，必须和expr的计算结果类型一致，且不能为NULL。

num_buckets：指定范围划分后的桶的个数，必须是正数，若不是整数，则向下取整，且不能为NULL。

例

```
SELECT WIDTH_BUCKET(NULL,1,5,4);
```

查询结果为：NULL

```
SELECT WIDTH_BUCKET(0,1,5,4);
```

查询结果为：0

```
SELECT WIDTH_BUCKET(1,1,5,4);
```

查询结果为：1

```
SELECT WIDTH_BUCKET(5,1,5,4);
```

查询结果为：5

```
SELECT WIDTH_BUCKET(6,5,1,4);
```

查询结果为：0

```
SELECT WIDTH_BUCKET(5,5,1,4);
```

查询结果为：1

```
SELECT WIDTH_BUCKET(1,5,1,4);
```

查询结果为：5

```
SELECT WIDTH_BUCKET(DATE '2022-12-12 09:10:21',DATE '2025-2-5 10:5:6',DATE '2021-5-6
2:5:1',24);
```

查询结果为：14

11. 函数 OBJECT_ID

```
语法：OBJECT_ID([schema_name.]object_name[, object_type])
```

功能：返回对象的ID。

参数：

object_name：表示对象名称的字符串，支持模式名schema_name前缀；不指定前缀的情况下，默认从当前用户的当前模式查找。

object_type：表示对象类型的字符串，取值如下：

1）对象类型全称：表TABLE、视图VIEW、约束CONSTRAINT、过程PROCEDURE/函数

FUNCTION、同义词SYNONYM、包PACKAGE、序列SEQUENCE、触发器TRIGGER；其中TABLE包含UTAB/STAB两种；PACKAGE包含PKG/TYPE/CLASS/JCLASS四种；

2）用于兼容的缩写U/S、V、C/UQ/F/PK、P/FN、SN、SO、TR；其中U表示用户自定义表，S表示系统表，V表示视图，C表示CHECK约束，UQ表示UNIQUE约束，F表示外键约束，PK表示主键约束，P表示过程，FN表示函数，SN表示同义词，SO表示序列，TR表示触发器；

3）DM的SYSOBJECTS表的SUBTYPE$列对应的缩写：UTAB/STAB、VIEW、CONS、PROC、SYNOM、PKG/TYPE/CLASS/JCLASS、SEQ、TRIG；其中PROC包含过程PROCEDURE和函数FUNCTION两种。

使用说明：

1）任意参数为NULL时，返回NULL；

2）支持的对象类型包括：表、视图、约束、过程/函数、同义词、包、序列、触发器；输入不支持的对象类型时，返回NULL，不支持的对象类型包括：列、模式、索引/全文索引、上下文、全局同义词、目录、外部链接等；

3）对象名与对象类型不匹配时，返回NULL；

4）只要有权限就能查看所指定模式下的对象的ID，其中需要注意的有：

a. 约束：当有表权限时可以查看约束；

b. 表：若表是分区子表，检查的是分区根表的权限，且不限定于查询权限，可以是增删改查和引用中的任意权限；

c. 视图：不限定于查询权限，可以是增删改查任意权限；

d. 同义词：只允许查看用户当前模式下的同义词，或者有所指定的同义词对象的权限；

e. 触发器：可以查看用户当前模式下的触发器，或者对触发器所在的表或视图有权限；

5）多模式时，查找非用户当前模式时，需要指定schema_name，如：模式名.约束名；

6）若存在同名对象，当查到第一个对象后，停止查找并返回对应ID；且当同名时，表比约束的优先级高。

例

```
DROP TABLE T;
CREATE TABLE T(C1 INT);
SELECT OBJECT_ID('T'); //查询表对象 ID
```

查询结果为：1017（查询结果以实际为准）

```
SELECT OBJECT_ID('T','VIEW');//object_type 错误
```

查询结果为：NULL

```
SELECT OBJECT_ID('T.C1');//不支持的对象类型（列）
```

查询结果为：NULL

第 9 章 一致性和并发性

数据一致性是指表示客观世界同一事务状态的数据，不管出现在何时何处都是一致的、正确的和完整的。数据库是一个共享资源，可为多个应用程序所共享，它们同时存取数据库中的数据，这就是数据库的并发操作。此时，如果不对并发操作进行控制，则会存取不正确的数据，或破坏数据库数据的一致性。

DM 利用事务和封锁机制提供数据并发存取和数据完整性。在一事务内由语句获取的全部封锁在事务期间被保持，防止其它并行事务的破坏性干扰。一个事务的 SQL 语句所做的修改在它提交后才可能为其它事务所见。

DM 自动维护数据库的一致性和完整性，并允许选择实施事务级读一致性，它保证同一事务内的可重复读，为此 DM 提供用户各种手动上锁语句和设置事务隔离级别语句。

本章介绍 DM 中和事务管理相关的 SQL 语句和手动上锁语句。在本章各例中，如不特别说明，各例的当前用户均为建表者 SYSDBA。

9.1 DM 事务相关语句

DM 中事务是一个逻辑工作单元，由一系列 SQL 语句组成。DM 把一个事务的所有 SQL 语句作为一个整体，即事务中的操作，要么全部执行，要么一个也不执行。

9.1.1 事务的开始

DM 没有提供显式定义事务开始的语句，第一个可执行的 SQL 语句(除登录语句外)隐含事务的开始。

9.1.2 事务的结束

用户可以使用显式的提交或回滚语句来结束一个事务，也可以隐式地提交一个事务。

1. 提交语句

语法格式

```
COMMIT [WORK] [IMMEDIATE|BATCH] [WAIT|NOWAIT];
```

参数

1） WORK 支持与标准 SQL 的兼容性，COMMIT 和 COMMIT WORK 等价；

2） IMMEDIATE 目前仅语法支持，无实际作用；

3） BATCH 目前仅语法支持，无实际作用。指定 BATCH 时必须指定 WAIT 或 NOWAIT，否则报语法错误；

4） WAIT 事务提交等待事务刷盘；

5） NOWAIT 事务提交不等待事务刷盘。

功能

该语句使当前事务工作单元中的所有操作“永久化”，并结束该事务。

举例说明

例 1 插入数据到表 DEPARTMENT 并提交。

```
INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('采购部门');
COMMIT WORK;
```

例 2 插入数据到表 DEPARTMENT 并提交，提交不等待事务刷盘。

```
INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('采购部门');
COMMIT WORK IMMEDIATE NOWAIT;
```

2．回滚语句

语法格式

```
ROLLBACK [WORK];
```

功能

该语句回滚(废除)当前事务工作单元中的所有操作，并结束该事务。

使用说明

建议用户退出时，用 COMMIT 或 ROLLBACK 命令来显式地结束应用程序。如果没有显式地提交事务，而应用程序又非正常终止，则最后一个未提交的工作单元被回滚。特别说明：ALTER TABLESPACE 和 ALTER USER 两种 DDL 语句是不能回滚的。

举例说明

例 插入数据到表 DEPARTMENT 后回滚。

```
(1)往表 DEPARTMENT 中插入一个数据

INSERT INTO RESOURCES.DEPARTMENT(NAME) VALUES('销售部门');

(2)查询表 DEPARTMENT

SELECT * FROM RESOURCES.DEPARTMENT;

//部门名为'销售部门'的记录可查询到

(3)回滚插入操作

ROLLBACK WORK;

(4)查询表 DEPARTMENT

SELECT * FROM RESOURCES.DEPARTMENT;

//插入操作被回滚，表 DEPARTMENT 中不存在部门名为'销售部门'的记录
```

3．隐式提交

当遇到 DDL 语句时，DM 数据库会自动提交前面的事务，然后开始一个新的事务执行 DDL 语句。这种事务提交被称为隐式提交。DM 数据库在遇到以下 SQL 语句时自动提交前面

的事务：

1） CREATE；

2） ALTER；

3） TRUNCATE；

4） DROP；

5） GRANT；

6） REVOKE；

7） 审计设置语句。

9.1.3 保存点相关语句

SAVEPOINT 语句用于在事务中设置保存点。保存点提供了一种灵活的回滚，事务在执行中可以回滚到某个保存点，在该保存点以前的操作有效，而以后的操作被回滚掉。一个事务中可以设置多个保存点。用户可通过 V$TRX_SAVEPOINT 动态视图查询所有活动事务的保存点信息。

1．设置保存点

语法格式

```
SAVEPOINT <保存点名>;
```

参数

<保存点名> 指明保存点的名字。

使用说明

一个事务中可以设置多个保存点，且一个事务中可创建的最大保存点个数由 INI 参数 SAVEPOINT_LIMIT 决定，可根据需要动态调整；该参数缺省值为 512，即单个事务中默认最大可创建 512 个保存点。重复创建名字相同的保存点，先创建的保存点会失效，回滚时以

后创建的保存点为准。

2．回滚到保存点

语法格式

```
ROLLBACK [WORK] TO SAVEPOINT <保存点名>;
```

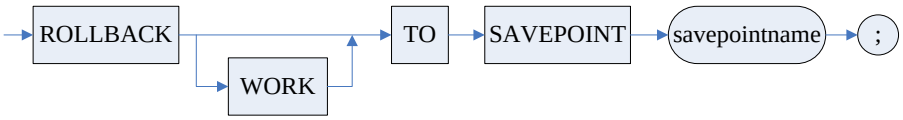
参数

1） WORK 支持与标准 SQL 的兼容性，ROLLBACK 和 ROLLBACK WORK 等价；

2） <保存点名> 指明部分回滚时要回滚到的保存点的名字。

图例

回滚保存点



使用说明

回滚到保存点后事务状态和设置保存点时事务的状态一致，在保存点以后对数据库的操作被回滚。

举例说明

例 插入数据到表 ADDRESS_TYPE 后设置保存点，然后再插入另一数据，回滚到保存点。

```
(1)往表 ADDRESS_TYPE 中插入一个数据

INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('发货地址');

(2)查询表 ADDRESS_TYPE

SELECT * FROM PERSON.ADDRESS_TYPE;

//地址类型名为'发货地址'的记录已经被插入到表中

(3)设置保存点

SAVEPOINT A;
```

```
(4)往表 ADDRESS_TYPE 中插入另一个数据

INSERT INTO PERSON.ADDRESS_TYPE(NAME) VALUES('家庭地址');

(5)回滚到保存点

ROLLBACK TO SAVEPOINT A;

(6)查询表 ADDRESS_TYPE

SELECT * FROM PERSON.ADDRESS_TYPE;

//插入操作被回滚，ADDRESS_TYPE 中不存在地址类型名为'家庭地址'的记录
```

3．销毁保存点

语法格式

```
RELEASE SAVEPOINT <保存点名>;
```

参数

<保存点名> 指明要销毁的保存点的名字。

图例

销毁保存点



使用说明

销毁指定的保存点，同时销毁指定保存点之后的保存点。

举例说明

例 设置多个保存点后销毁其中的保存点。

```
(1)设置保存点 A、B、C、D 和 E

SAVEPOINT A;

SAVEPOINT B;

SAVEPOINT C;
```

```
SAVEPOINT D;

SAVEPOINT E;

(2)查询保存点

SELECT * FROM V$TRX_SAVEPOINT;

//可以查询到保存点 A、B、C、D 和 E

(3)销毁保存点 C

RELEASE SAVEPOINT C;

(4)查询保存点

SELECT * FROM V$TRX_SAVEPOINT;

//只能查询到保存点 A 和 B，保存点 C、D 和 E 被销毁
```

上述三个保存点相关语句均支持用户自定义变量以及存储过程的参数。

当设置 dm.ini 中参数 MS_PARSE_PERMIT=2 时（该参数为静态参数，修改后需要重启服务器），保存点名支持用户自定义变量以及存储过程的参数，需要在变量前增加@标识符，最终保存点名全部会转换为大写。

例 设置 ini 参数 MS_PARSE_PERMIT=2。

```
SP_SET_PARA_VALUE (2,'MS_PARSE_PERMIT',2);

DECLARE

NAME VARCHAR(20)='abc';

BEGIN

    savepoint @name;          //生成保存点 ABC

    Release savepoint @name;  //销毁保存点 ABC

    savepoint @name;          //生成保存点 ABC

    Rollback to savepoint @name;  //回滚到保存点 ABC
```

```
END;
/

CREATE OR REPLACE PROCEDURE p1(name VARCHAR(128))
AS
BEGIN
    savepoint @name;
    SELECT * FROM V$TRX_SAVEPOINT;
END;
/

CALL p1('abc');
```

9.1.4 设置事务隔离级及读写特性

事务的隔离级描述了给定事务的行为对其它并发执行事务的暴露程度。通过选择三个隔离级中的一个，用户能增加对其它未提交事务的暴露程度，获得更高的并发度。DM 允许用户改变未启动的事务的隔离级和读写特性，即下列语句必须在事务开始时执行，否则无效。

1．设置事务隔离级语句

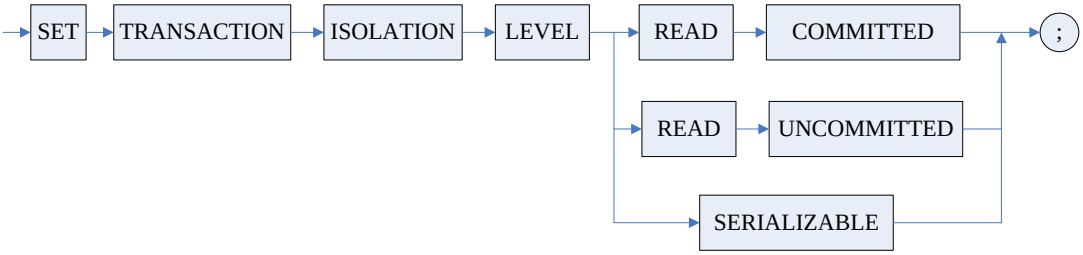
事务的隔离级描述了给定事务的行为对其它并发执行事务的暴露程度。通过选择三个隔离级中的一个，用户能增加对其它未提交事务的暴露程度，获得更高的并发度。

语法格式

```
SET TRANSACTION ISOLATION LEVEL <事务隔离级>;
<事务隔离级> ::= READ COMMITTED | READ UNCOMMITTED | SERIALIZABLE
```

图例

设置事务隔离级



使用说明

1） 该语句设置事务的隔离级别：

✓ 读提交(READ COMMITTED)：DM 默认级别，保证不读脏数据；

✓ 读未提交(READ UNCOMMITTED)：可能读到脏数据；

✓ 可串行化(SERIALIZABLE)：事务隔离的最高级别，事务之间完全隔离。

一般情况下，使用读提交隔离级别可以满足大多数应用，如果应用要求可重复读以保证基于查询结果的更新的正确性就必须使用可重复读或可串行读隔离级别。在访问只读表和视图的事务，以及某些执行 SELECT 语句的事务(只要其他事务的未提交数据对这些语句没有负面效果)时，可以使用读未提交隔离级。

2） 只能在事务未开始执行前设置隔离级，事务执行期间不能更改隔离级。

2．设置事务读写属性的语句

语法格式

```
SET TRANSACTION <事务读写属性>;

<事务读写属性> ::= READ ONLY | READ WRITE
```

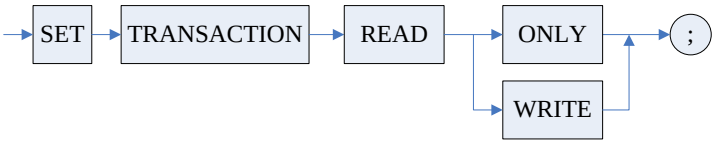
参数

1） READ ONLY 只读事务，该事务只能做查询操作，不能更新数据库；

2） READ WRITE 读写事务，该事务可以查询并更新数据库，是 DM 的默认设置。

图例

设置事务读写属性



语句功能

该语句设置事务的读写属性。

3．设置某条查询语句为脏读

DM 允许用户在 SELECT 语句的末尾加上 WITH UR 以指定当前查询语句的隔离级为读未提交，即允许脏读，并在该语句结束时自动恢复为原来的隔离级。

举例说明

例 会话 1 创建表 T，插入一行数据且不提交，会话 2 查询表 T，因为缺省的事务隔离级为读提交，此时查询不到数据，但是当会话 2 在 SELECT 语句末尾加上 WITH UR 时，可以查询到会话 1 插入的还未提交的数据。

```
//会话 1 执行

SQL> CREATE TABLE T(C1 INT, C2 INT);

操作已执行

已用时间: 4.320(毫秒). 执行号:53700.

SQL> INSERT INTO T VALUES(1,1);

影响行数 1

已用时间: 0.573(毫秒). 执行号:53701.



//会话 2 执行

SQL> SELECT * FROM T;

未选定行
```

```
已用时间: 1.691(毫秒). 执行号:53800.

SQL> SELECT * FROM T WITH UR;



行号          C1              C2

---------- ----------- -----------

1             1               1

已用时间: 0.532(毫秒). 执行号:53801.
```

9.2 DM 手动上锁语句

DM 的隐式封锁足以保证数据的一致性，但用户可以根据自己的需要手动显式锁定表，允许或禁止在当前用户操作期间其它用户对此表的存取。DM 提供给用户四种表锁的封锁：意向共享锁(IS)、共享锁(S)、意向排他锁(IX)和排他锁(X)，并且支持同时执行共享锁(S)和意向排他锁(IX)的封锁，即共享意向排他锁(S+IX)。

语法格式

```
LOCK TABLE [<模式名>.]<表名> IN <封锁方式> MODE [NOWAIT];

<封锁方式>::=

INTENT SHARE |

ROW SHARE |

SHARE UPDATE |

INTENT EXCLUSIVE |

ROW EXCLUSIVE |

SHARE |

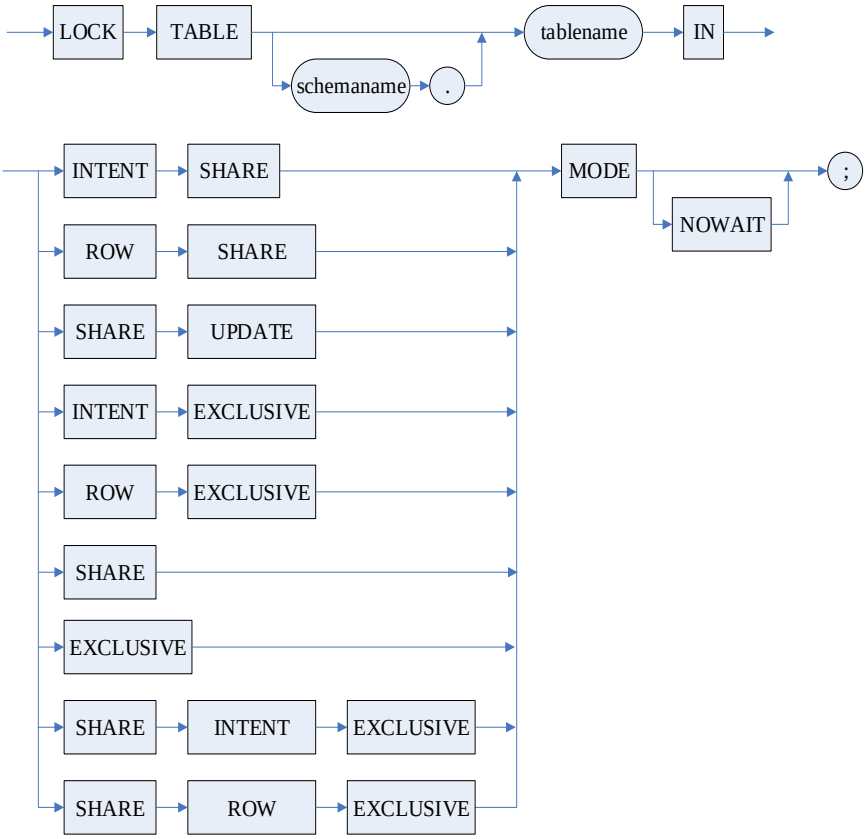
EXCLUSIVE |

SHARE INTENT EXCLUSIVE |
```

SHARE ROW EXCLUSIVE

图例

DM 手动上锁语句



使用说明

1. 意向共享表封锁：INTENT SHARE TABLE LOCKS（IS）

该封锁表明该事务封锁了表上的一些元组并试图修改它们（但是还未做修改，其它事务可读这些元组，但是不能修改这些元组）。意向共享表封锁是限制最少的锁，提供了表上最大的并发度。

1） 等价关键字：INTENT SHARE、ROW SHARE、SHARE UPDATE。

2） 允许操作：

其他事务对该表的并发查询、插入、更新、删除或在该表上进行封锁，其他事务可以同时上意向共享锁(IS)、意向排他锁(IX)和共享锁(S)。

3） 禁止操作：

其它事务以排他锁方式(X)存取该表。

```
LOCK TABLE tablename IN EXCLUSIVE MODE;
```

2. 意向排他表封锁：INTENT EXCLUSIVE TABLE LOCKS (IX)

该锁表明该事务对表的元组进行一次或多次修改（其它事务不能访问这些元组），行排他表封锁较行共享表封锁稍严格。

1） 等价关键字：INTENT EXCLUSIVE、ROW EXCLUSIVE。

2） 允许操作：

其它事务并行查询、插入、更新、删除或封锁该表上行，允许多个事务在同一表上获得意向排他锁(IX)和意向共享锁(IS)。

3） 禁止操作：

其它事务对表执行共享锁(S)、排他锁(X)或共享意向排他锁(S+IX)封锁。

```
LOCK TABLE tablename IN SHARE MODE;

LOCK TABLE tablename IN EXCLUSIVE MODE;

LOCK TABLE tablename IN SHARE INTENT EXCLUSIVE MODE;
```

3. 共享表封锁：SHARE TABLE LOCKS (S)

该锁表明该事务访问表中所有元组，其他事务不能对该表做任何更新操作。

1） 关键字：SHARE。

2） 允许操作：

其它事务在该表上作查询，但是不允许作修改，且允许多个事务在同一表上并发地持有共享表封锁(S)。

3） 禁止操作：

其它事务对表执行意向排他锁(IX)、排他锁(X)或共享意向排他锁(S+IX)封锁。

```
LOCK TABLE tablename IN INTENT EXCLUSIVE MODE;
```

```
LOCK TABLE tablename IN EXCLUSIVE MODE;

LOCK TABLE tablename IN SHARE INTENT EXCLUSIVE MODE;
```

4. 排他表封锁：EXCLUSIVE TABLE LOCKS（X）

该封锁是表封锁中最严格的方式，只允许持有封锁的事务可对该表进行修改。

1） 关键字：EXCLUSIVE。

2） 允许操作：

不允许任何操作。

3） 禁止操作：

其它事务对表执行任何 DML 语句，即不能插入、修改和删除该表中的行，封锁该表中的行或以任何方式封锁表。

用户上锁成功后锁将一直有效，直到当前事务结束时，该锁被系统自动解除。

5. 共享意向排他表封锁：SHARE INTENT EXCLUSIVE TABLE LOCKS（S+IX）

该锁是共享锁和意向排他锁的组合，表明该事务访问表中所有元组，允许其他事务在该表上做查询，但不允许对该表做任何更新操作。

1） 等价关键字：SHARE INTENT EXCLUSIVE、SHARE ROW EXCLUSIVE。

2） 允许操作：

其它事务在该表上执行查询操作，或者在该表上执行意向共享锁（IS）封锁。

3） 禁止操作：

其它事务对表执行任何 DML 语句，即不能插入、修改和删除该表中的行，或者对表执行意向排他锁（IX）、共享锁（S）或排他锁（X）封锁。

6. 当使用 NOWAIT 时，若不能立即上锁成功则立刻返回报错信息，不再等待。

举例说明

例 当用户 SYSDBA 希望独占某表，他可以对该表显式地上排他锁。

```
LOCK TABLE PERSON.ADDRESS IN EXCLUSIVE MODE;
```

第 10 章 外部函数

为了能够在 DMSQL 程序中使用其他开发语言实现的方法，DM8 提供了 C、JAVA 外部函数功能。使用外部函数的好处是即使外部函数在执行中出现了任何问题，都不会影响到服务器的正常执行。

无论是 C 外部函数还是 JAVA 外部函数，都需要提前将编译好的动态库或 jar 包上传到服务器端，或在服务器端编译生成动态库或 jar 包。系统管理员须对动态库和 jar 包进行严格审查，以防止外部函数中包含病毒或恶意代码，引发安全问题。为了保证数据库的安全性和灵活性，DM 提供了 INI 参数 ENABLE_EXTERNAL_CALL 来控制外部函数是否开启，缺省情况下，ENABLE_EXTERNAL_CALL=0，表示数据库关闭外部函数功能。

需要注意的是，DM 不支持 C 或 JAVA 外部函数存放在 ASM 文件系统上的调用。

10.1 C 外部函数

C 外部函数是使用 C、C++语言编写，在数据库外编译并保存在.dll、.so 共享库文件中，被用户通过 DMSQL 程序调用的函数。

C 外部函数的执行一般通过代理 dmap 工具进行，此时为了执行 C 外部函数，需要先启动 dmap 服务。dmap 执行程序在 DM8 安装目录的 bin 子目录下，直接执行即可启动 dmap 服务。

同时，DM 结构化的 C 外部函数支持结合 INI 参数 EFC_USE_AP 进行性能优化，当指定参数值为 0 时，结构化的 C 外部函数会在 dmserver 内部调用，不再和 dmap 进行通信，可以提高函数的执行效率。

当用户调用 C 外部函数时，服务器操作步骤如下：首先，确定调用的（外部函数使用的）共享库及函数；然后，通知代理进程工作。代理进程装载指定的共享库，并在函数执行

后将结果返回给服务器。

10.1.1 生成动态库

DM8 提供两种方案编写 C 外部函数：

- **DM 结构化参数**

该方案中，C 用户必须使用 DM8 提供的编写 C 外部函数动态库的接口，严格按照如下格式书写外部函数的 C 代码。

C 外部函数格式

```
de_data 函数名(de_args *args)
{
        C 语言函数实现体;
}
```

参数

1. <de_data> 返回值类型。de_data 结构体类型如下：

```
struct de_data{
    int  null_flag;   //参数是否为空，1 表示非空，0 表示空
    union             //只支持 int、double、float、binary_float、char 类型。其中 float 类型在系统内部被转化为 double 类型执行，相关接口请使用 double 类型的接口
    {
        int     v_int;
        double  v_double;
        char    v_str[];
    }data;
};
```

2. <de_args> 参数类型。de_args 结构体类型如下:

```
struct de_args
{
    int          n_args;       //参数个数
    de_data*     args;         //参数列表
};
```

3. < C 语言函数实现体> C 语言函数对应的函数实现体。

使用说明

1. C 语言函数的参数可通过调用 DM8 提供的一系列 get 函数得到，同时可调用 set 函数重新设置这些参数的值;
2. 根据返回值类型，调用不同的 return 函数接口;
3. 必须根据参数类型、返回值类型，调用相同类型的 get、set 和 return 函数。当调用 de_get_str 和 de_get_str_with_len 得到字符串后，必须调用 de_str_free 释放空间;
4. DM8 提供的编写 C 外部函数动态库的接口如表 10.1.1 所示。

表 10.1.1 DM8 支持的编写 C 外部函数动态库的接口

函数类型	函数名	功能说明
get	int de_get_int(de_args *args, int arg_id);	第 arg_id 参数的数据类型为整型，从参数列表 args 中取出第 arg_id 参数的值
	double de_get_double(de_args *args, int arg_id);	第 arg_id 参数的数据类型为 double 类型，从参数列表 args 中取出第 arg_id 参数的值
	char* de_get_str(de_args* args, int arg_id);	第 arg_id 参数的数据类型为字符串类型，从参数列表 args 中取出第 arg_id 参数的值。 二者区别: de_get_str 使用服务器自动申请的空
	char* de_get_str_with_space (	

	de_args* args, int arg_id, byte* v_str_in, int v_str_in_len);	间，使用完毕后需要 de_str_free 释放空间。 de_get_str_with_space 使用第三方申请的空间（通过 v_str_in 和 v_str_in_len 传入），使用完毕后不需要 de_str_free 释放空间。 v_str_in 为已申请内存的字符串空间，由调用者分配，其大小不能少于 v_str_in_len。若已分配长度等于 0，v_str_in 和返回值均返回 NULL。 若分配空间不够，则只返回部分字符串。 v_str_in_len 为空间长度，包含结尾 0。如果 v_str_in_len 小于实际参数加结尾 0 长度，则截断输出
	char* de_get_str_with_len(de_args* args, int arg_id, int* len);	第 arg_id 参数的数据类型为字符串类型，从参数列表 args 中取出第 arg_id 参数的值以及字符串长度。 二者区别： de_get_str_with_len 使用服务器自动申请的空间，需要使用 de_str_free 释放空间。 de_get_str_with_len_and_space 使用第三方申请的空间（通过 v_str_in 和 v_str_in_len 传入），不需要使用 de_str_free 释放空间。 v_str_in 为已申请内存的字符串空间，由调用者分配，其大小不能少于 v_str_in_len。若已分配长度等于 0，v_str_in 和返回值均返回 NULL。 若分配空间不够，则只返回部分字符串。 v_str_in_len 为空间长度，包含结尾 0。如果 v_str_in_len 小于实际参数加结尾 0 长度，则截断输出。 len 为实际返回长度，不包含结尾 0
	char* de_get_str_with_len_and_space (de_args* args, int arg_id, byte* v_str_in, int v_str_in_len, int * len);	
set	void de_set_int(de_args* args, int arg_id, int ret);	第 arg_id 参数的数据类型为整型，设置参数列表 args 的第 arg_id 参数的值为 ret
	void de_set_double(de_args *args, int arg_id, double ret);	第 arg_id 参数的数据类型为 double 类型，设置参数列表 args 的第 arg_id 参数的值为 ret
	void de_set_str(	第 arg_id 参数的数据类型为字符串类

	de_args *args, int arg_id, char* ret);	型，设置第 arg_id 参数的值为 ret
	void de_set_str_with_len(de_args *args, int arg_id, char* ret, int len);	第 arg_id 参数的数据类型为字符串类型，将字符串 ret 的前 len 个字符赋值给参数列表 args 的第 arg_id 参数
	void de_set_null(de_args* args, int arg_id);	设置参数列表args的第arg_id个参数为空
return	de_data de_return_int(int ret);	返回值类型为整型
	de_data de_return_double(double ret);	返回值类型为 double 型
	de_data de_return_str(char* ret);	返回值为字符串类型
	de_data de_return_str_with_len(char* ret, int len);	返回字符串 ret 的前 len 个字符
	de_data de_return_null();	返回空值
de_str_free	void de_str_free(char* str);	调用 de_get_str 函数后，需要调用此函数释放字符串空间
de_is_null	int de_is_null(de_args *args, int arg_id);	判断参数列表args的第arg_id个参数是否为空

注：参数个数 arg_id 的起始值为 0。

- **标量类型参数**

该方案中，用户不必引用 DM 提供的外部函数接口，可以按照标准的 C 风格编码，使用 C 标量类型作为参数类型。使用该方案编写的 C 函数，只能在使用 X86 CPU 的 64 位非 Windows 系统中，被数据库引用作为外部函数。

C 外部函数格式

```
返回类型 函数名(参数列表)
{
    C 语言函数实现体;
}
```

使用说明

1. 返回类型及参数列表中参数的数据类型只支持 int、double、float、binary_float、char*类型。其中 float 的用法和 double 完全一样;
2. 参数类型为 binary_float 时，对应的 C 函数数据类型应为 double;
3. 参数列表中不支持 out 型参数;
4. 如果参数列表中有 char*的参数，不必在函数中对其进行释放；为了安全考虑，最好只对其进行只读操作;
5. 如果返回 char*类型，返回值必须使用 malloc 申请空间，且必须有结尾 0，不允许直接返回常量或返回参数列表中传入的字符类型参数。

10.1.2 C 外部函数创建

语法格式

```
CREATE [OR REPLACE] FUNCTION [IF NOT EXISTS] [<模式名>.]<函数名>[(<参数列表>)]
RETURN <返回值类型>
EXTERNAL '<动态库路径>' [AND][<引用的函数名>] USING < C | CS >;
```

参数

1. <函数名> 指明被创建的 C 外部函数的名字;
2. <模式名> 指明被创建的 C 外部函数所属模式的名字，缺省为当前模式名;
3. <参数列表> 指明 C 外部函数参数信息，如果是使用 DM 结构化参数编写的 C 函数，参数模式可设置为 IN、OUT 或 IN OUT（OUT IN），缺省为 IN 类型；而使用标量类型参

数编写的 C 函数则参数模式只能是 IN 类型。参数类型、个数都应和动态库里定义的一致；

4. <返回值类型> 必须和动态库里定义的一致；

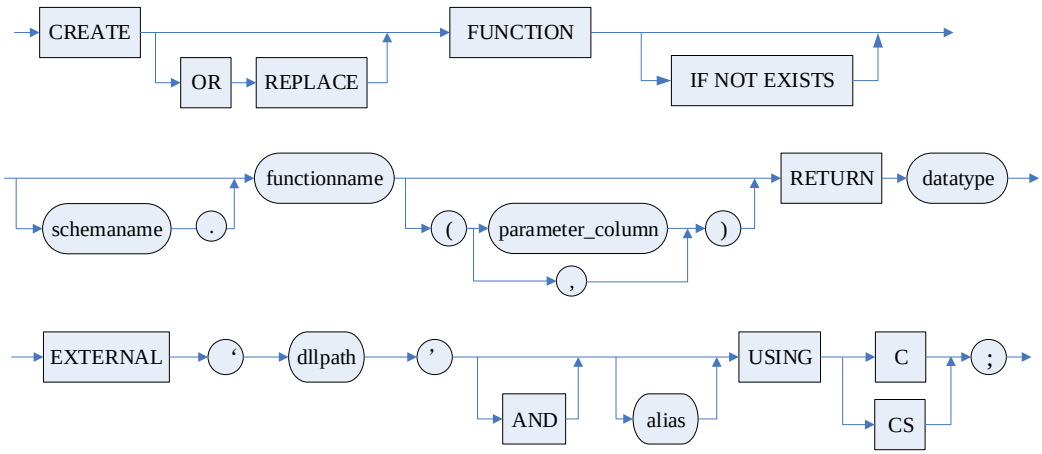
5. <动态库路径> 用户按照 DM 规定的 C 语言函数格式编写的 DLL 文件生成的动态库所在的路径；动态库分为 64 位和 32 位两种，使用的时候要和操作系统一一对应。例如，64 位的操作系统要用 64 位的动态库；

6. <引用的函数名> 指明<函数名>在<动态库路径>中对应的函数名。如果不使用双引号，则系统会自动将函数名转换成大写字母，如果使用双引号则不会进行大小写转换。建议使用双引号将函数名括起来，避免由于大小写问题而导致函数名不匹配。示例如下：……EXTERNAL '/mnt/mylibtest.so' **"myy_add"** USING CS；

7. USING 子句指明函数的类型，如果是 DM 结构化参数的 C 函数，类型为 C；标量类型参数的 C 函数，类型为 CS。

图例

C 外部函数创建



语句功能

创建自定义 C 外部函数。

使用说明

1. 三权分立下，仅允许拥有 CREATE PROCEDURE 权限的用户创建外部函数；四权分立下，仅允许拥有 CREATE PROCEDURE 权限的用户创建外部函数；

2. 创建或执行外部函数需要设置 INI 参数 ENABLE_EXTERNAL_CALL 为 1;

3. <引用函数名>如果为空,则默认与<函数名>相同。当<引用函数名>的长度超过 128 个字节并且不超过 32767 个字节时，不能使用双引号，需要改用单引号括起来，并且需要指定 AND 关键字。

4. <动态库路径>分为.dll 文件（windows）和.so 文件（linux）两种;

5. 外部函数定义的返回值类型与对应 C 语言函数的返回值类型必须一致，否则会导致返回值不可预测甚至系统异常。

10.1.3 举例说明

例如，编写（C 语言）外部函数 C_CONCAT，用于将两个字符串连接。下面针对两种编写 C 外部函数的方案，分别进行描述。

一 使用 DM 结构化参数方案来完成

首先 生成动态库。

第一步，使用 Microsoft Visual Studio 2010 新建空项目 newp，位于 d:\xx\tt 文件夹中。新建完毕后，若本机操作系统为 x64，因为 VS 默认的解决方案平台不是 x64，需要在解决方案平台下拉框选择“配置管理器-活动方案解决平台-新建-键入或选择新平台-x64”。在 d:\xx\tt\newp\newp 文件夹中，直接拷入 dmde.lib 动态库和 de_pub.h 头文件。dmde.lib 和 de_pub.h 位于安装文件 x:\...\dmdbms\include 中。

第二步，在 newp 项目中，添加头文件。将已有 de_pub.h 头文件添加进来，同时，添加新的 tt.h 头文件。tt.h 文件内容如下:

```
#include "de_pub.h"
#include "string.h"
#include "stdlib.h"
```

第三步，在 newp 项目中，添加源文件。名为 tt.c。tt.c 内容如下:

```
#include "tt.h"

de_data C_CONCAT(de_args *args)

{

    de_data   de_ret;

    char* str1 = NULL;

    char* str2 = NULL;

    char* str3 = NULL;

    int len1 = 0;

    int len2 = 0;


    if(!de_is_null(args, 0))

    {

        str1 = (char*)de_get_str(args, 0);      //从参数列表中取第 0 个参数

        len1 = strlen(str1);

    }

    if(!de_is_null(args, 1))

    {

        //从参数列表中取第 1 个参数的值以及长度

        str2 = (char*)de_get_str_with_len(args, 1, (udint4*)&len2);

    }


    str3 = (char*)malloc(len1 + len2);

    memcpy(str3, str1, len1);

    memcpy(str3 + len1, str2, len2);
```

```
    //调用 get 函数得到字符串之后，需要调用此函数释放字符串空间
    if(!de_is_null(args, 0))
    {
        de_str_free((sdbyte*)str1);
    }
    if(!de_is_null(args, 1))
    {
        de_str_free((sdbyte*)str2);
    }
    de_ret = de_return_str_with_len((udbyte*)str3, len1 + len2); //返回字符串
    free(str3);
    return de_ret;
}
```

第四步，在 newp 项目的源文件中，添加模块定义文件 tt.def，内容如下：

```
LIBRARY   "tt.dll"
EXPORTS
    C_CONCAT
```

第五步，配置项目属性。

在 Microsoft Visual Studio 2010 界面上，右击 newp 项目-属性，点击打开。

在“配置属性—链接器—输入”中添加附加依赖项 dmde.lib。

在“配置属性—常规”中：输出目录设为 D:\xx\tt\；配置类型选择动态库（.dll）；字符集选择使用多字节字符集。

第六步，生成 newp 项目。右击 newp 项目-生成。得到 D:\xx\tt\newp.dll 文件。

至此，外部函数的使用环境准备完毕。

其次，创建并使用外部函数。

第一步，启动数据库服务器 dmserver、启动 DMAP、启动 DIsql。dmserver、DIsql 等工具位于安装目录 x:\...\dmdbms\bin 中。

需要先开启系统允许创建外部函数的开关。通过设置 DM.INI 参数 ENABLE_EXTERNAL_CALL=1 开启。设置语句如下:

```
SF_SET_SYSTEM_PARA_VALUE('ENABLE_EXTERNAL_CALL',1,0,2);
```

设置完毕后需重启数据库服务器，参数才能生效。

第二步，在 DIsql 中，创建外部函数 MY_CONCAT。语句如下:

```
CREATE OR REPLACE FUNCTION MY_CONCAT(A VARCHAR, B VARCHAR)
RETURN VARCHAR
EXTERNAL 'd:\xx\tt\newp.dll' "C_CONCAT" USING C;
```

第三步，调用 C 外部函数。语句如下:

```
select MY_CONCAT ('hello ', 'world!');
```

第四步，查看结果:

```
hello world!
```

二 使用 C 标量类型参数方案完成，不过需要注意该方案仅支持使用 X86 CPU 的 64 位非 Windows 系统。

生成动态库，这里改用 LINUX 操作系统作为示例。

第一步，创建 C 文件 test.c，编写 C 函数。

```
#include <string.h>
#include "stdlib.h"
char*   C_CONCAT (char*   str1, char* str2)
{
    char* str3 = NULL;
```

```
    int len1 = 0;
    int len2 = 0;
    len1 = strlen(str1);
    len2 = strlen(str2);
    str3 = (char*)malloc(len1 + len2 + 1); //要多一个字节作为结尾 0
    memcpy(str3, str1, len1);
    memcpy(str3 + len1, str2, len2);
    str3[len1 + len2] = 0;  //必须有结尾 0
    return str3;
}
```

第二步，生成动态库。

```
gcc -o /mnt/libtest.so -fPIC -shared test.c
```

获得 libtest.so 文件。

至此，外部函数的使用环境准备完毕。

其次，创建并使用外部函数。

第一步，启动数据库服务器 dmserver、启动 DMAP、启动 DIsql。

第二步，在 DIsql 中，创建外部函数 MY_CONCAT。语句如下：

```
CREATE OR REPLACE FUNCTION MY_CONCAT(A VARCHAR, B VARCHAR)
RETURN VARCHAR
EXTERNAL '/mnt/libtest.so' "C_CONCAT" USING CS;
```

第三步，调用 C 外部函数。语句如下：

```
select MY_CONCAT ('hello ', 'world!');
```

第四步，查看结果：

```
hello world!
```

10.2 JAVA 外部函数

JAVA 外部函数是使用 JAVA 语言编写，在数据库外编译生成的 jar 包，被用户通过 DMSQL 程序调用的函数。

JAVA 外部函数的执行都通过代理 dmagent 工具进行，为了执行 JAVA 外部函数，需要先启动 dmagent 服务。dmagent 执行程序在 DM8 安装目录的 tool/dmagent 子目录下，其使用说明文档可参看该目录下的《readme》文档。

当用户调用 JAVA 外部函数时，服务器操作步骤如下：首先，确定调用（外部函数使用的）jar 包及函数；然后，通知代理进程工作。代理进程装载指定的 jar 包，并在函数执行后将结果返回给服务器。

需要注意的是，进行 JAVA 外部函数调用应保证当前用户可以运行 JAVA 命令，否则会导致调用失败。

10.2.1 生成 jar 包

用户必须严格按照 JAVA 语言的格式书写代码，书写完成后生成 jar 包。

10.2.2 JAVA 外部函数创建

语法格式

```
CREATE [OR REPLACE] FUNCTION [IF NOT EXISTS] [<模式名>.]<函数名>[(<外部函数参数列表>)]
RETURN <返回值类型>
EXTERNAL '<jar 包路径>' [AND] <引用的 JAVA 函数名>[(<JAVA 函数参数列表>)] USING JAVA;
<外部函数参数列表>: : =<外部函数参数声明> {,<外部函数参数声明>}
<外部函数参数声明>: : =<外部函数参数名> [<外部函数参数模式>]<外部函数参数类型>
<JAVA 函数参数列表>: : =<JAVA 函数参数声明> {,<JAVA 函数参数声明>}
<JAVA 函数参数声明>: : =<JAVA 函数参数类型>
```

参数

1. <函数名> 被创建的 JAVA 外部函数的名字;

2. <模式名> 被创建的 JAVA 外部函数所属模式的名字，缺省为当前模式名;

3. <外部函数参数列表> JAVA 外部函数参数信息;

4. <外部函数参数模式> 可设置为 IN、OUT 或 IN OUT（OUT IN），缺省为 IN 类型。参数类型、个数都应和 jar 包里的一致;

5. <外部函数参数类型>目前支持的函数参数类型：int、字符串(char、varchar、varchar2)、bigint、double/float/binary_float、binary/varbinary、clob、blob，分别对应 java 类型：int、string、 long、double、byte[]、string、byte[];

6. <返回值类型> 必须 jar 包里定义的一致;

7. <jar 包路径>用户按照 JAVA 语言格式编写的源码生成的 jar 包，及其依赖的 jar 包所在的相对路径或绝对路径;

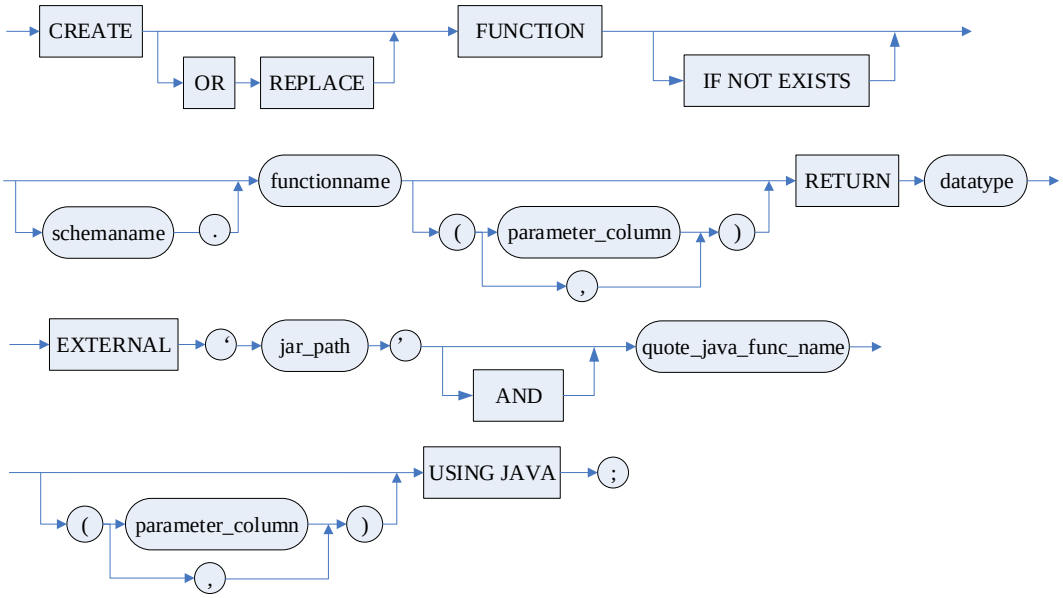
8. <引用的 JAVA 函数名> 指明<函数名>在<jar 包路径>中对应的 JAVA 函数名，函数调用通过 JAVA 反射实现，建议调用函数访问限定符为 public，另外，所属类须不含构造函数或包含无参的构造函数。若<函数名>包含包名，则包名（或.包名）与类名的分隔符要用.或/。而类与方法的分隔符要用.。若不指定<引用函数名>，则后续使用 JAVA 外部函数时会报错。一个正确的 JAVA 函数名示例如下：……EXTERNAL '/data/sda/test.jar' "**com.test.package1.test.pass**(java.lang.String,java.lang.String[])" USING JAVA;

9. <JAVA 函数参数列表> 用于指定 JAVA 函数的参数列表。缺省情况下，系统自动定位参数。只有未指定<外部函数参数模式>或<外部函数参数模式>中所有参数模式均为 IN 时，<JAVA 函数参数列表>才可缺省;

10. <JAVA 函数参数声明>由一个或多个<JAVA 函数参数类型>组成。<JAVA 函数参数类型>书写时必须包括 JAVA 类的完整名称。例如：……EXTERNAL '/data/sda/test.jar' "com.test.package1.test.pass(**java.lang.String,java.lang.String[]**)" USING JAVA;。

图例

JAVA 外部函数创建



语句功能

创建自定义 JAVA 外部函数。

使用说明

1. 当<引用的 JAVA 函数名>[(<JAVA 函数参数列表>)]的长度超过 128 个字节并且不超过 32767 个字节时，不能使用双引号，需要改用单引号括起来，并且需要指定 AND 关键字。例如：EXTERNAL '/data/sda/test.jar' **AND** 'com.test.package1.test.pass(java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.lang.String, java.lang.String)' USING JAVA;

2. 三权分立下，仅允许拥有 CREATE PROCEDURE 权限的用户创建外部函数；四权分立下，仅允许拥有 CREATE PROCEDURE 权限的用户创建外部函数；

3. 创建或执行外部函数需要设置 INI 参数 ENABLE_EXTERNAL_CALL 为 1；

4. 只能执行 agent_home/resources/ap 目录下的 jar 包。

10.2.3 举例说明

例 编写（JAVA 语言）外部函数：testAdd 用于求两个数之和，testStr 用于在一个字符串后面加上 hello。

首先，生成 jar 包。

第一步，使用 eclipse 创建新项目 newp，位于 F:\workspace 文件夹中。

第二步，在 newp 项目中，添加包。右击 newp，新建（new）一个 package，命名（name）为 com.test.package1。

第三步，在 package1 包中，添加类文件。右击 src，新建（new）一个 class，命名（name）为 test。Modifiers 选择 public。class 文件内容如下：

```
package com.test.package1;

public class test {

    public static int testAdd(int a, int b) {

        return a + b;

    }

    public static String testStr(String str) {

        return str + " hello";

    }

}
```

第四步，生成 jar 包。在 newp 项目中，右击，选中 EXPORT，选择 Java\JAR file，取消.classpath 和.project 的勾选。目标路径 JAR file 设置为：E:\test.jar，然后 finish。

第五步，查看 E 盘中 test.jar。已经存在。

第六步，在外部函数对应的 dmagent 的 resources 目录下，创建一个 ap 文件夹（若 resources 目录下已有 ap 文件夹则不用创建），将 test.jar 拷入其中。

至此，外部函数的使用环境准备完毕。

其次，创建并使用外部函数。

第一步，启动数据库服务器 dmserver，启动 DM 管理工具。

第二步，在 DM 管理工具中，创建外部函数 MY_INT 和 MY_chr，语句如下：

```
CREATE OR REPLACE FUNCTION MY_INT(a int, b int)
RETURN int
EXTERNAL 'E:\develop\tool\agent\resources\ap\test.jar' "com.test.package1.test.testAdd"
USING java;


CREATE OR REPLACE FUNCTION MY_chr(s varchar)
RETURN varchar
EXTERNAL 'E:\develop\tool\agent\resources\ap\test.jar' "com.test.package1.test.testStr" USING
java;
```

第三步，调用 JAVA 外部函数，语句如下：

```
select MY_INT(1,2);
select MY_chr('abc');
```

第四步，查看结果，分别为：

```
3
abc hello
```

10.3 DMAP 使用说明

DMAP（DM Assit progress）作为数据库管理系统的辅助进程，提供 C 外部函数、备份还原等功能的执行。

10.3.1 启动 DMAP

安装 DM 数据库以后，DMAP 服务会自动启动。如果需要手动启动，有两种途径：一是启动 DM 服务查看器中的 DmAPService；二是通过手动启动 DMAP 执行码实现，DMAP 执行码位于 DM 安装目录的 bin 子目录下。除此之外，LINUX 下，还可以调用 bin 目录下的 DmAPService 脚本启动 DMAP 服务。

默认不带参数启动 DMAP 时，DMAP 监听端口号为 4236，与 DM 服务器 INI 参数 EXTERNAL_AP_PORT 的默认值一致。

也可以使用参数 dmap_ini 指定 DMAP.INI 配置文件启动 DMAP，如下例所示：

```
dmap   dmap_ini=d:\dmap.ini
```

DMAP.INI 配置项如下表所示：

参数	说明
AP_PORT	dmap 端口号
AP_LOG_FLAG	备份还原相关日志切换模式，包含 dmrman、BAKRES、dmap、dmap_br 日志，下同。 0：老的切换模式，日志名称包含时间戳 1：新的切换模式，日志生成时名称中不包含时间戳，切换后重命名增加时间戳
AP_LOG_SIZE	备份还原相关日志大小限制，AP_LOG_FLAG 为 1 时生效。有效值为 0∪[64,2048]，0 表示没有大小限制。
AP_LOG_SWITCH_MODE	备份还原相关日志切换频率，AP_LOG_FLAG 为 1 时生效。 0：按月切换 1：按天切换 2：按小时切换

需要注意的是，当使用 DMAP.INI 对 AP_PORT 进行配置时，DM 服务器 INI 参数 EXTERNAL_AP_PORT 的配置值应与 AP_PORT 一致。

10.3.2 使用 DMAP 执行 C 外部函数

例 创建并使用 C 外部函数 MY_CONCAT。

```
CREATE OR REPLACE FUNCTION MY_CONCAT(A VARCHAR, B VARCHAR)
RETURN VARCHAR
EXTERNAL 'D:\testroot\for_dmserver\smoketest_data\dameng\detest64.dll' C_CAT USING C;
/
select MY_CONCAT ('hello ', 'a');
```

查询结果如下:

```
行号          MY_CONCAT('hello','a')
---------- ----------------------
1             hello a
```

10.3.3 DMAP 日志

DMAP 日志文件记录了 DMAP 工具产生的相关日志。DMAP 日志文件命名规则为:dm_dmap_年月.log。

DMAP 日志记录的信息格式为:

时间+日志类型（INFO/WARN/ERROR/FATAL）+进程(dmap)+进程 ID+线程名（线程名为空则使用 T 开头的线程 ID）+日志内容。

例 展示一段 dm_dmap_202211.log 日志信息。

```
2022-11-30 09:00:58.357 [INFO] dmap P0000015192 T0000000000000032076   dmap V8
2022-11-30 09:00:58.372 [INFO] dmap P0000015192 T0000000000000032076   dmap is ready
2022-11-30 09:00:58.372 [INFO] dmap P0000015192 T0000000000000032076   [for
dem]SYSTEM IS READY.
```

第 11 章 包

DM 支持 DMSQL 程序包来扩展数据库功能，用户可以通过包来创建应用程序或者使用包来管理过程和函数。

11.1 创建包

包的创建包括包规范和包主体的创建。

11.1.1 创建包规范

包规范中包含了有关包中的内容信息，但是它不包含任何过程的代码。定义一个包规范的详细语法如下。

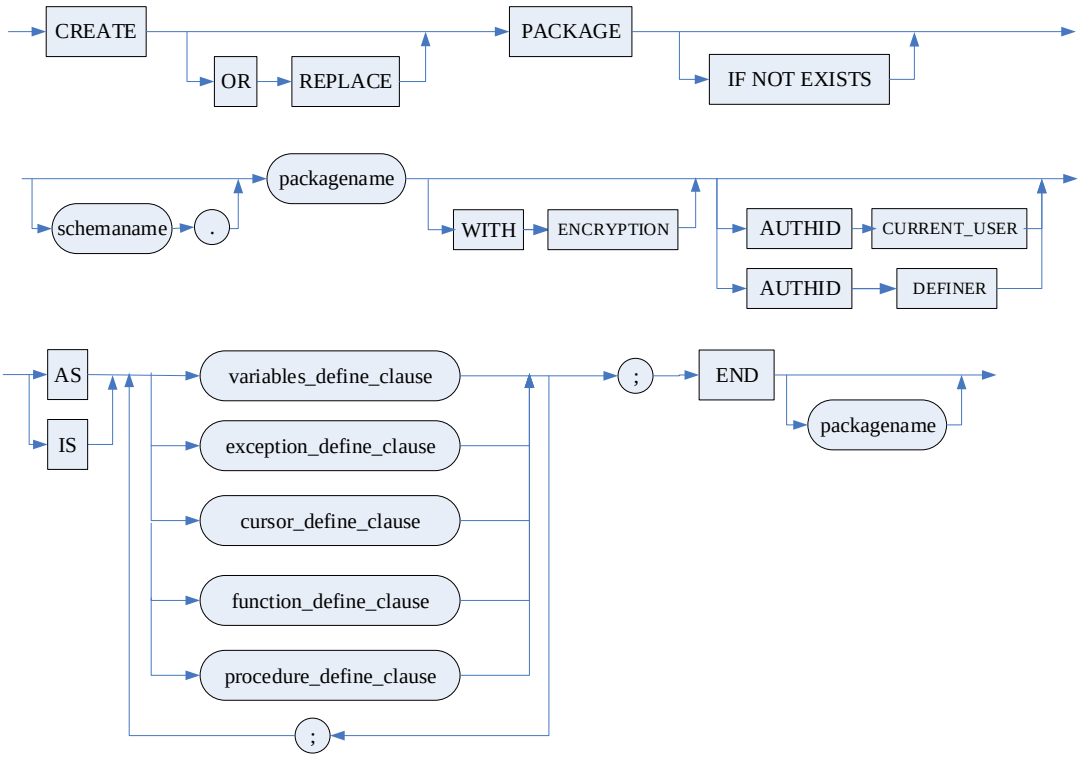
语法格式

```
CREATE [OR REPLACE] PACKAGE [IF NOT EXISTS] [<模式名>.]<包名> [WITH ENCRYPTION] [<调用者权限>] AS|IS <包内声明列表>  END [包名]
<调用者权限>::=
    AUTHID DEFINER |
    AUTHID CURRENT_USER
<包内声明列表> ::= <包内声明>;{<包内声明>;}
<包内声明> ::= <变量声明>|<游标定义>|<异常定义>|<过程定义>|<函数定义>
<游标定义> ::= CURSOR <游标名> [FOR <查询语句>]
<异常定义> ::= <异常名> EXCEPTION [FOR <错误号> [,<错误描述>]]
<过程定义> ::= PROCEDURE <过程名> <参数列表>
<函数定义> ::= FUNCTION <函数名><参数列表> RETURN <返回值数据类型>[RESULT_CACHE]
```

[DETERMINISTIC] [PIPELINED]

图例

创建包规范



使用说明

1. 创建包的名称不能与系统创建的模式名称相同;

2. 包部件可以以任意顺序出现，其中的对象必须在引用之前被声明;

3. 过程和函数的声明都是前向声明，包规范中不包括任何实现代码;

4. <赋值标识>中 DEFAULT、ASSIGN 和:=均用于为变量赋值，三种赋值标识功能和用法完全一样;

5. <变量声明>与 DMSQL 程序中的变量声明方式一致，具体请参考《DM8_SQL 程序设计》手册“3.5 变量”章节。

权限

1. 使用该语句的用户必须是 DBA 或该包对象的拥有者且具有 CREATE PACKAGE 数

据库权限的用户；

2. <调用者权限>中用关键字 AUTHID DEFINER 或 AUTHID CURRENT_USER 指定包的调用者权限。DEFINER 为采用包定义者权限；CURRENT_USER 为当前用户权限。缺省为 DEFINER。

11.1.2 创建包主体

包主体中包含了在包规范中的前向子程序声明相应的代码。它的创建语法如下。

语法格式

```
CREATE [OR REPLACE] PACKAGE BODY [<模式名>.]<包名> [WITH ENCRYPTION] AS|IS <包体部分>
END [包名]
<包体部分> ::= <包体声明列表> [<初始化代码>]
<包体声明列表> ::=<包体声明>;[{<包体声明>;}]
<包体声明>::=<变量声明>|<游标定义>|<异常定义>|<过程定义>|<函数定义>|<存储过程实现>|<函数实现>
<游标定义> ::= CURSOR <游标名> [FOR <查询语句>]
<异常定义> ::= <异常名> EXCEPTION [FOR <错误号> [, <错误描述>]]
<过程定义> ::= PROCEDURE <过程名> <参数列表>
<函数定义> ::= FUNCTION <函数名> <参数列表> RETURN <返回值数据类型>
<存储过程实现> ::= PROCEDURE <过程名> <参数列表>  AS|IS  BEGIN <实现体> END [<过程名>];
<函数实现> ::= FUNCTION <函数名><参数列表> RETURN <返回值数据类型>[DETERMINISTIC]
[PIPELINED]<AS|IS> BEGIN <实现体> END [<函数名>];
<初始化代码> ::= [[<说明部分>]BEGIN<执行部分>[<异常处理部分>]]
<说明部分> ::=[DECLARE]<说明定义>{<说明定义>}
<说明定义>::=<变量列表说明>|<异常变量说明>|<子游标定义>|<子过程定义>|<子函数定义>;
```

<变量列表说明>::= <变量初始化>{<变量初始化>}

<异常变量说明>::=<异常变量名>EXCEPTION[FOR <错误号> [, <错误描述>]]

<子游标定义>::=CURSOR <游标名> [FOR<查询表达式>|<连接表>]

<子过程定义>::=PROCEDURE<过程名>[(<参数列>)]<IS|AS><模块体>

<子函数定义>::=FUNCTION<函数名>[(<参数列>)]RETURN<返回数据类型><IS|AS><模块体>

<执行部分>::=<SQL 过程语句序列>{< SQL 过程语句序列>}

<SQL 过程语句序列>::=[<标号说明>]<SQL 过程语句>;

<标号说明>::=<<<标号名>>>

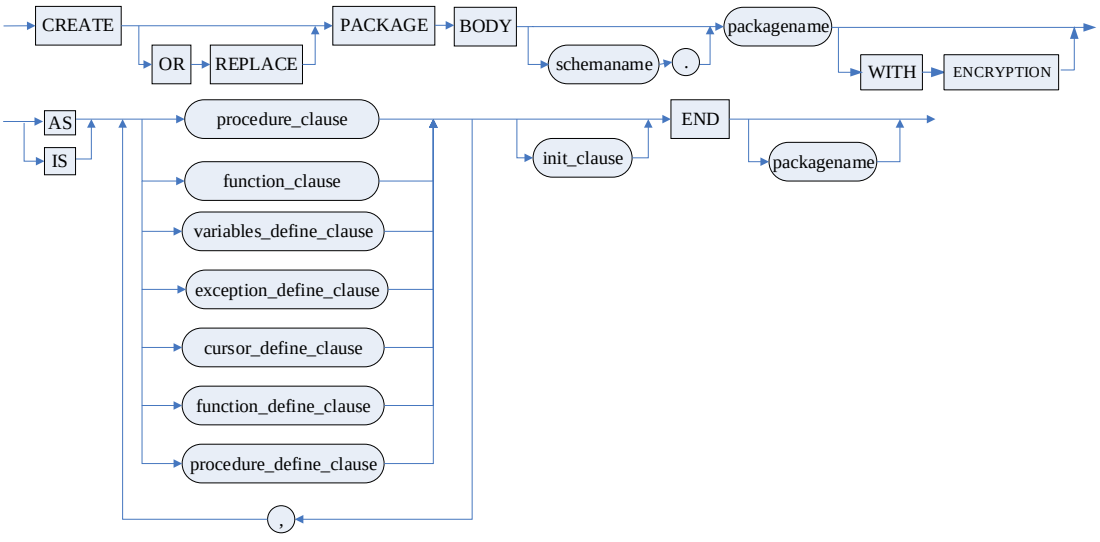
<SQL 过程语句>::=<SQL 语句>|<SQL 控制语句>

<异常处理部分>::=EXCEPTION<异常处理语句>{<异常处理语句>}

<异常处理语句>::= WHEN <异常名> THEN <执行部分>;

图例

创建包主体



使用说明

1．创建包规范时<变量声明>、<游标定义>和<异常定义>中定义的对象对于包主体而言都是可见的，不需要声明就可以直接引用；

2．包主体中不能使用未在包规范或者包主体中声明的对象；

3．包主体中的过程、函数定义必须和包规范中的前向声明完全相同。包括过程的名字、参数定义列表的参数名和数据类型定义；

4．若包规范中的过程、函数声明中的参数包含默认值，则允许包主体中的过程、函数定义中的参数忽略该默认值，若未忽略则默认值必须与包规范中保持一致；若包规范中的过程、函数声明中的参数不包含默认值，则包主体中的过程、函数定义中的参数也不能包含默认值；

5．包中可以有重名的过程和函数，只要它们的参数定义列表不相同。系统会根据用户的调用情况进行重载(OVERLOAD)；

6．用户在第一次访问包（如调用包内过程、函数，访问包内变量）时，系统会自动将包对象实例化。每个会话根据数据字典内的信息在本地复制包内变量的副本。如果用户定义了 PACKAGE 的初始化代码，还必须执行这些代码(类似于一个没有参数的构造函数执行)；

7．对于一个会话，包头中声明的对象都是可见的，只要指定包名，用户就可以访问这些对象。可以将包头内的变量理解为一个 SESSION 内的全局变量；

8．关于包内过程、函数的调用：DM 支持按位置调用和按名调用参数两种模式。除了需要在过程、函数名前加入包名作为前缀，调用包内的过程、函数的方法和普通的过程、函数并无区别；

9．包主体<变量声明>、<游标定义>和<异常定义>中定义的对象以及实现的未在包头内声明的方法被称作本地对象、方法。本地对象、方法只能在包体内使用，用户无法直接使用；

10．在包体声明列表中，本地对象必须在所有的方法实现之前进行声明；本地方法必须在使用之前进行声明或实现；

11．如果创建包时，在函数定义中使用 DETERMINISTIC 指定该函数为确定性函数，在函数实现中可以省略指定该函数的确定性；如果函数定义中没有指定该函数为确定性函数，则

函数实现时不能指定该函数为确定性函数。

12．结果集缓存 RESULT_CACHE，只是语法支持，没有实际意义。

权限

使用该语句的用户必须是 DBA 或该包对象的拥有者且具有 CREATE PACKAGE 数据库权限的用户。

11.2 重编译包

重新对包进行编译，如果重新编译失败，则将包置为禁止状态。

重编功能主要用于检验包的正确性。

语法格式

```
ALTER PACKAGE [<模式名>.]<包名> COMPILE [CASCADE] [DEBUG];
```

参数

1．<模式名> 指明被重编译的包所属的模式；

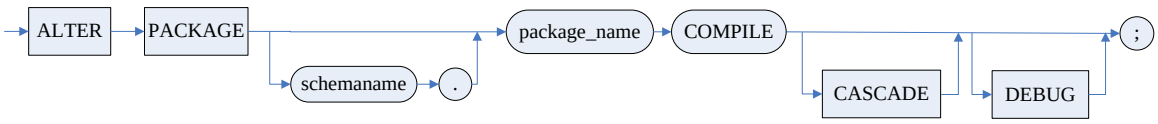
2．<包名> 指明被重编译的包的名字；

3．[CASCADE] 当指定 CASCADE 后，将级联重编译所有直接或间接引用该包的对象，需要考量影响范围，建议谨慎使用；

4．[DEBUG] 可忽略。

图例

重编译包



权限

执行该操作的用户必须是包的创建者，或者具有 DBA 权限。

11.3 删除包

和创建方式类似，包对象的删除分为包规范的删除和包主体的删除。

11.3.1 删除包规范

从数据库中删除一个包对象。

语法格式

```
DROP PACKAGE [IF EXISTS] [<模式名>.]<包名>;
```

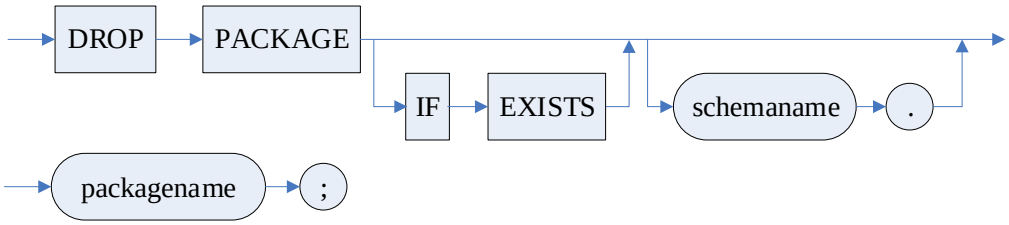
参数

1．<模式名> 指明被删除的包所属的模式，缺省为当前模式；

2．<包名> 指明被删除的包的名字。

图例

删除包规范



使用说明

1．删除不存在的包规范会报错。若指定 IF EXISTS 关键字，删除不存在的包规范，不会报错；

2．如果被删除的包不属于当前模式，必须在语句中指明模式名；

3．如果一个包规范被删除，那么对应的包主体被自动删除。

权限

执行该操作的用户必须是 DBA，或者是该包的拥有者且具有 DROP PACKAGE 权限。

11.3.2 删除包主体

从数据库中删除一个包的主体对象。

语法格式

```
DROP PACKAGE BODY [IF EXISTS] [<模式名>.]<包名>;
```

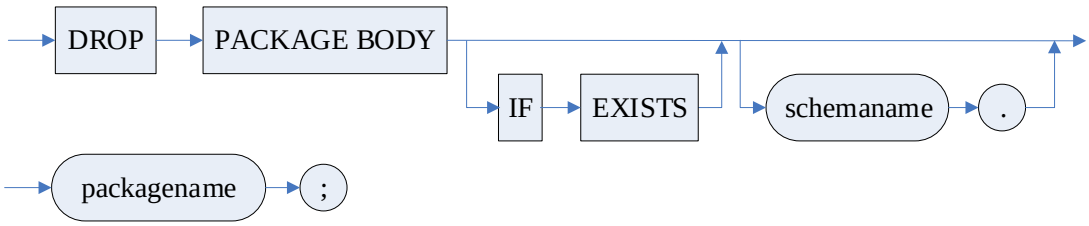
参数

1．<模式名> 指明被删除的包所属的模式，缺省为当前模式；

2．<包名> 指明被删除的包的名字。

图例

删除包主体



使用说明

1．删除不存在的包主体会报错。若指定 IF EXISTS 关键字，删除不存在的包主体，不会报错；

2．如果被删除的包不属于当前模式，必须在语句中指明模式名。

权限

执行该操作的用户必须是 DBA，或者是该包的拥有者且具有 DROP PACKAGE 权限。

11.4 应用实例

以下是一个包规范的使用实例：

创建表并插入数据，这些数据将在之后的例子中用到。

```
CREATE TABLE Person(Id INT IDENTITY, Name VARCHAR(100), City VARCHAR(100));
```

```
INSERT INTO Person(Name, City) VALUES('TOM','武汉');
INSERT INTO Person(Name, City) VALUES('JACK','北京');
INSERT INTO Person(Name, City) VALUES('MARY','上海');
COMMIT;
```

表中数据如下表所示。

表 11.4.1

ID	NAME	CITY
1	TOM	武汉
2	JACK	北京
3	MARY	上海

创建包规范：

```
CREATE OR REPLACE PACKAGE PersonPackage AS
        E_NoPerson EXCEPTION;
        PersonCount INT;
        Pcur CURSOR;
        PROCEDURE AddPerson(Pname VARCHAR(100), Pcity varchar(100));
        PROCEDURE RemovePerson(Pname VARCHAR(100), Pcity varchar(100));
        PROCEDURE RemovePerson(Pid INT);
        FUNCTION GetPersonCount RETURN INT;
        PROCEDURE PersonList;
END PersonPackage;
```

这个包规范的部件中包括 1 个变量定义，1 个异常定义，1 个游标定义，4 个过程定义和 1 个函数定义。

以下是一个包主体的实例，它对应于前面的包规范定义，包括 4 个子过程和 1 个子函数

的代码实现。在包主体的末尾，是这个包对象的初始化代码。当一个会话第一次引用包时，变量 PersonCount 被初始化为 Person 表中的记录数。

创建包主体：

```
CREATE OR REPLACE PACKAGE BODY PersonPackage AS


  PROCEDURE AddPerson(Pname VARCHAR(100), Pcity varchar(100) )AS
    BEGIN
      INSERT INTO Person(Name, City) VALUES(Pname, Pcity);
      PersonCount = PersonCount + SQL%ROWCOUNT;
    END AddPerson;


  PROCEDURE RemovePerson(Pname VARCHAR(100), Pcity varchar(100)) AS
    BEGIN
      DELETE FROM Person WHERE NAME LIKE Pname AND City like Pcity;
        IF SQL%ROWCOUNT=0 THEN
        RAISE E_NoPerson;
        END IF;
      PersonCount = PersonCount - SQL%ROWCOUNT;
    END RemovePerson;


  PROCEDURE RemovePerson(Pid INT) AS
    BEGIN
      DELETE FROM Person WHERE Id = Pid;
        IF SQL%ROWCOUNT=0 THEN
```

```
        RAISE E_NoPerson;
        END IF;
      PersonCount = PersonCount - SQL%ROWCOUNT;
  END RemovePerson;

FUNCTION GetPersonCount RETURN INT AS
BEGIN
  RETURN PersonCount;
END GetPersonCount;

PROCEDURE PersonList AS
DECLARE
  V_id INT;
  V_name VARCHAR(100);
  V_city VARCHAR(100);
BEGIN
  IF PersonCount = 0 THEN
      RAISE E_NoPerson;
  END IF;
  OPEN Pcur FOR SELECT Id, Name, City FROM Person;
  LOOP
    FETCH Pcur INTO V_id,V_name,V_city;
    EXIT WHEN Pcur%NOTFOUND;
    PRINT ('No.' || (cast (V_id as varchar(100))) || '  ' || V_name || '来自' || V_city );
```

```
    END LOOP;
    CLOSE Pcur;
  END PersonList;

BEGIN
    SELECT COUNT(*) INTO PersonCount FROM Person;

END PersonPackage;
```

重新编译包：

```
ALTER PACKAGE PersonPackage COMPILE;
```

调用包中的 AddPerson 过程，往数据表中增加一条记录：

```
CALL  PersonPackage. AddPerson ('BLACK', '南京') ;
```

当前记录变化如下表所示。

表 11.4.2

ID	NAME	CITY
1	TOM	武汉
2	JACK	北京
3	MARY	上海
4	BLACK	南京

调用包中的 RemovePerson 过程，删除第二条记录：

```
CALL  PersonPackage. RemovePerson ('JACK', '北京') ;
```

或者

```
CALL  PersonPackage. RemovePerson (2) ;
```

在此例中，以上两种写法可以得到相同的结果，系统对同名过程根据实际参数进行了重

载。如果过程执行结果没有删除任何一条表中的记录，那么会抛出一个包内预定义的异常：E_NoPerson。

此时表中的数据如下表所示。

表 11.4.3

ID	NAME	CITY
1	TOM	武汉
3	MARY	上海
4	BLACK	南京

引用包中的变量。

```
SELECT PersonPackage. PersonCount;
```

或者

```
SELECT PersonPackage. GetPersonCount;
```

以上两句语句的作用是等价的。前一句是直接引用了包内变量，后一句是通过调用包内的子函数来得到想要的结果。

调用包中的过程 PersonList 查看表中的所有记录：

```
CALL   PersonPackage.PersonList;
```

可以得到以下输出：

```
No.1   TOM 来自武汉

No.3   MARY 来自上海

No.4   BLACK 来自南京
```

第 12 章 类类型

DM 通过类类型在 DMSQL 程序中实现面向对象编程的支持。类将结构化的数据及对其进行操作的过程或函数封装在一起。允许用户根据现实世界的对象建模，而不必再将其抽象成关系数据。

DM 的类类型分为普通类类型和 JAVA CLASS 类型。DM 文档中的示例除了特别声明使用的是 JAVA CLASS 类型，否则使用的都是普通类类型。

12.1 普通 CLASS 类型

DM 的类的定义分为类头和类体两部分，类头完成类的声明；类体完成类的实现。

类中可以包括以下内容:

1. 成员变量

类中的成员变量包括变量、游标和异常。成员变量必须先进行声明，声明后方可使用。变量的类型可以是 INT、CHAR 等常规数据类型，也可以是记录类型、数组类型等 DMSQL 程序数据类型，关于 DMSQL 程序数据类型的详细介绍请参考《DM8_SQL 程序设计》。

2. 成员方法

类中的函数或过程，在类头中进行声明；其实现在类体中完成；

成员方法及后文的构造函数包含一个隐含参数，即自身对象，在方法实现中可以通过 this 或 self 来访问自身对象，self 等价于 this。如果不存在重名问题，也可以直接使用对象的属性和方法。this 和 self 只能在包或对象脚本中调用。

3. 构造函数

构造函数是类内定义及实现的一种特殊的函数，这类函数用于实例化类的对象，构造函数满足以下条件:

1） 函数名和类名相同；

2） 函数返回值类型为自身类。

构造函数存在以下的约束：

1） 系统为每个类提供两个默认的构造函数，分别为 0 参的构造函数和全参的构造函数；

2） 0 参构造函数的参数个数为 0，实例的对象内所有的属性初始化值为 NULL；

3） 全参构造函数的参数个数及类型和类内属性的个数及属性相同，按照属性的顺序依次读取参数的值并给属性赋值；

4） 用户可以自定义构造函数，一个类可以有多个构造函数，但每个构造函数的参数个数必须不同；

5） 如果用户自定义了 0 个参数、或参数个数同属性个数相同的构造函数，则会覆盖相应的默认构造函数。

下面从类的声明、类的实现、类的删除、类体的删除和类的使用几部分来详细介绍类类型的实现过程。

12.1.1 声明类

类的声明在类头中完成。类头定义通过 CREATE CLASS 语句来完成，其语法为：

语法格式

```
CREATE [OR REPLACE] CLASS [IF NOT EXISTS] [<模式名>.]<类名> [WITH ENCRYPTION] [UNDER [<模式名>.]<父类名>] [<final 属性>] [<instantiable 属性>] [AUTHID DEFINER | AUTHID CURRENT_USER] AS|IS <类内声明列表>  END [<类名>];
<类内声明列表> ::= <类内声明>;{<类内声明>;}
<类内声明> ::= <变量声明>|<过程定义>|<函数定义>
<过程定义> ::= <方法继承属性>[STATIC|MEMBER] PROCEDURE <过程名> <参数列表>
```

<函数定义> ::= <方法继承属性> [MAP] [STATIC|MEMBER] FUNCTION <函数名><参数列表> RETURN <返回值数据类型>[DETERMINISTIC][PIPELINED]

<方法继承属性> ::= [<重载属性>] [<final 属性>] [<instantiable 属性>]

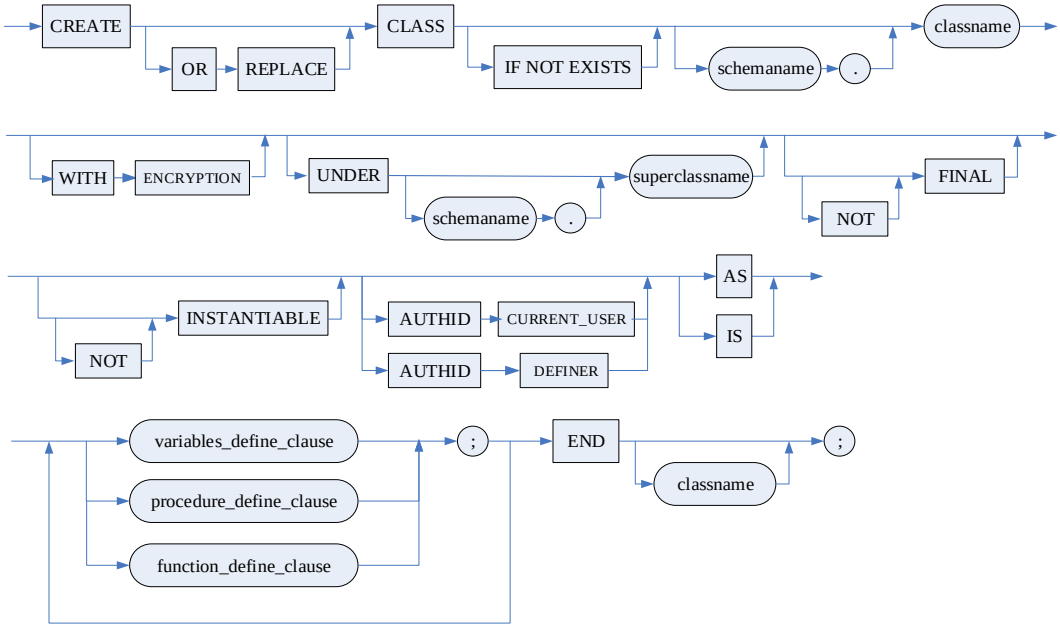
<重载属性> ::= [NOT] OVERRIDING

<final 属性> ::= [NOT] FINAL

<instantiable 属性> ::= [NOT] INSTANTIABLE

图例

声明类



使用说明

1．类的名称不能与系统创建的模式名称相同；

2．<变量声明>与 DMSQL 程序中的变量声明方式一致，具体请参考《DM8_SQL 程序设计》手册“3.5 变量”章节；

3．类中元素可以以任意顺序出现，其中的对象必须在引用之前被声明；

4．过程和函数的声明都是前向声明，类声明中不包括任何实现代码；

5．支持对象静态方法声明与调用。可以在 PROCEDURE/FUNCTION 关键字前添加

static 保留字，以此指明方法为静态方法。静态方法只能以对象名为前缀调用，而不能在对象实例中调用；

6．支持对象成员方法声明与调用。可以在 PROCEDURE/FUNCTION 关键字前添加 MEMBER 以指明方法为成员方法。MEMBER 与 STATIC 不能同时使用，非 STATIC 类型的非构造函数方法默认为成员方法。MAP 表示将对象类型的实例映射为标量数值，只能用于成员类型的 FUNCTION；

7．关于类继承，有以下使用限制：

1） 类定义默认为 FINAL，表示该对象类型不能被继承，定义父类时必须指定 NOT FINAL 选项；

2） 定义子类时必须指定 UNDER 选项；

3） NOT INSTANTIABLE 对象不能为 FINAL；

4） NOT INSTANTIABLE 对象不能实例化，但是可以用其子类赋值；

5） 对象实例化时，必须对父类和子类的成员变量都赋值，且从父类到子类逐个赋值；

6） 不支持对象的循环继承；

7） 不支持对象的多继承，即一个类有多个父类；

8） 不支持父类和子类包含同名变量；

9） 父类和子类可以同名同参，此时子类必须指定 OVERRIDING；

10）方法默认为 NOT OVERRIDING，OVERRIDING 不能与 static 一起使用；

11） 父类和子类支持同名不同参（参数个数不同、参数个数相同但类型不同）的方法；

12）同名且参数个数相同但类型不同时，根据参数类型选择使用的方法；

13）方法默认为 INSTANTIABLE，如果声明为 NOT INSTANTIABLE，则不能与 FINAL、STATIC 一起使用；

14）如果父类有多个 NOT INSTANTIABLE 方法，子类可以只部分重写，但此时子类必须定义为 NOT FINAL NOT INSTANTIABLE；

15）NOT INSTANTIABLE 方法不能具有主体；

16）方法默认为 NOT FINAL，如果声明为 FINAL，则不能被子类重写；

17）子类可以赋值给父类；

18）如果父类对应的实例是子类或者子类的孩子，则该父类可以赋值给子类；

19）可以用 INSTANTIABLE 子类对 NOT INSTANTIABLE 父类进行赋值；

20）子类实例赋值给父类后，调用时使用的是父类方法而不是子类方法；

21）支持使用 as 语句转换为父类。

权限

1、使用该语句的用户必须是 DBA 或具有 CREATE PACKAGE 数据库权限的用户；

2、可以用关键字 AUTHID DEFINER |AUTHID CURRENT_USER 指定类的调用者权限，若为 DEFINER，则采用类定义者权限，若为 CURRENT_USER 则为当前用户权限，默认为类定义者权限。

12.1.2 实现类

类的实现通过类体完成。类体的定义通过 CREATE CLASS BODY 语句来完成，其语法为：

语法格式

```
CREATE [OR REPLACE] CLASS BODY [<模式名>.]<类名> [WITH ENCRYPTION] AS|IS <类体部分> END [类名];
<类体部分> ::= <过程/函数列表> [<初始化代码>]
<过程/函数列表> ::= <过程实现|函数实现>{,<过程实现|函数实现> }
<过程实现> ::= <方法继承属性>[STATIC|MEMBER]PROCEDURE <过程名> <参数列表> AS|IS  BEGIN <
```

```
实现体> END [过程名]

<函数实现> ::= <方法继承属性>[MAP] [STATIC|MEMBER]FUNCTION <函数名><参数列表> RETURN <

返回值数据类型>[DETERMINISTIC] [PIPELINED] AS|IS BEGIN <实现体> END [函数名]

<方法继承属性> ::= [<重载属性>] [<final 属性>] [<instantiable 属性>]

<重载属性> ::= [NOT] OVERRIDING

<final 属性> ::= [NOT] FINAL

<instantiable 属性> ::= [NOT] INSTANTIABLE

<初始化代码> ::= [[<说明部分>]BEGIN<执行部分>[<异常处理部分>]]

<说明部分> ::=[DECLARE]<说明定义>{<说明定义>}

<说明定义>::=<变量声明>|<异常变量说明>|<子游标定义>|<子过程定义>|<子函数定义>

<异常变量说明>::=<异常变量名> EXCEPTION [FOR <错误号> [, <错误描述>]]

<子游标定义>::=CURSOR <游标名> [FOR<查询表达式>|<连接表>]

<子过程定义>::=PROCEDURE<过程名>[(<参数列>)] IS|AS <模块体>

<子函数定义>::=FUNCTION<函数名>[(<参数列>)]RETURN<返回数据类型> [PIPELINED] IS|AS <模块

体>

<执行部分>::=<SQL 过程语句序列>{;< SQL 过程语句序列>}

< SQL 过程语句序列>::=[<标号说明>]<SQL 过程语句>;

<标号说明>::=<<<标号名>>>

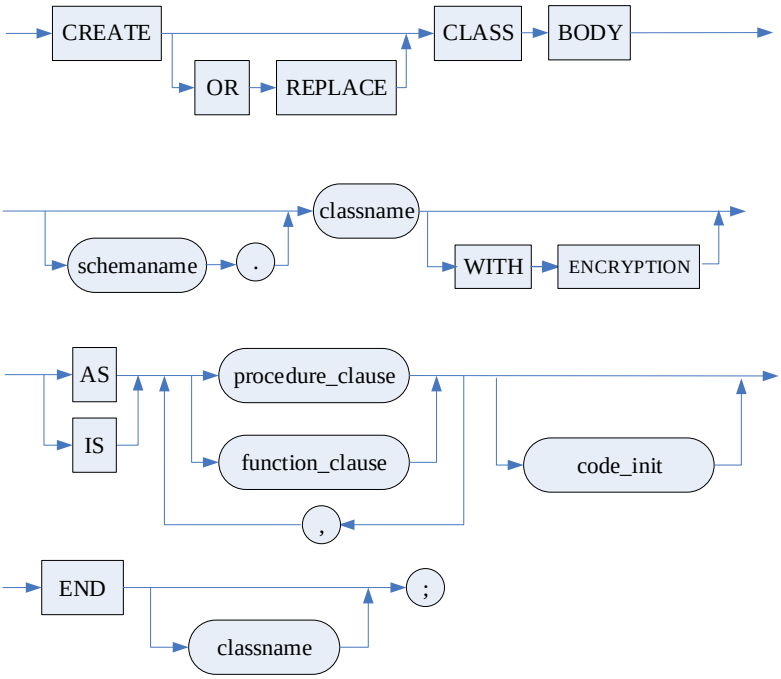
<SQL 过程语句>::=<SQL 语句>|<SQL 控制语句>

<异常处理部分>::=EXCEPTION<异常处理语句>{;<异常处理语句>};

<异常处理语句>::= WHEN <异常名> THEN <执行部分>
```

图例

实现类



使用说明

1．声明类时<变量声明>中定义的对象在实现类时可以直接引用；

2．类体中的过程、函数定义必须和类声明中的声明完全相同。包括过程的名字、参数定义列表的参数名和数据类型定义；

3．若类声明中的过程、函数声明中的参数包含默认值，则允许类体中的过程、函数定义中的参数忽略该默认值，若未忽略则默认值必须与类声明中保持一致；若类声明中的过程、函数声明中的参数不包含默认值，则类体中的过程、函数定义中的参数也不能包含默认值；

4．类中可以有重名的成员方法，要求其参数定义列表各不相同。系统会根据用户的调用情况进行重载(OVERLOAD)；

5．声明类与实现类时，对于确定性函数的指定逻辑与包内函数相同。目前不支持类的确定性函数在函数索引中使用。

权限

使用该语句的用户必须是 DBA 或该类对象的拥有者且具有 CREATE PACKAGE 数据库权限的用户。

完整的类头、类体的创建如下所示：

```
//类头创建
create class mycls
    as
    type rec_type is  record (c1 int, c2  int);   //类型声明
    id    int;    //成员变量
    r     rec_type;    //成员变量
    function f1(a int, b int) return rec_type;    //成员函数
    function mycls(id int , r_c1 int, r_c2 int) return mycls;
//用户自定义构造函数
end;
/
//类体创建
create or replace class body mycls
    as
    function f1(a int, b int) return rec_type
    as
    begin
    r.c1 = a;
    r.c2 = b;
    return r;
    end;
    function mycls(id int, r_c1 int, r_c2 int) return mycls
    as
```

```
    begin
    this.id = id;          //可以使用 this.来访问自身的成员
    r.c1 = r_c1;           //this 也可以省略
    r.c2 = r_c2;
    return this;           //使用 return this  返回本对象
    end;
end;
/
```

12.1.3 重编译类

重新对类进行编译，如果重新编译失败，则将类置为禁止状态。

重编功能主要用于检验类的正确性。

语法格式

```
ALTER CLASS [<模式名>.]<类名> COMPILE [DEBUG];
```

参数

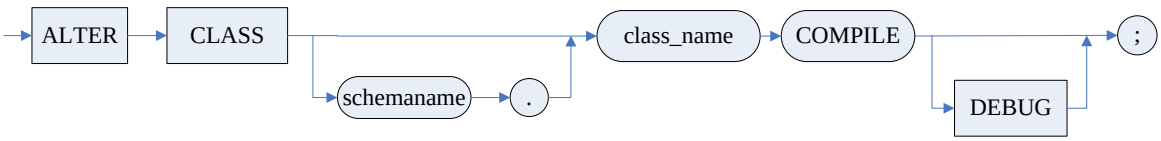
1．<模式名> 指明被重编译的类所属的模式；

2．<类名> 指明被重编译的类的名字；

3．[DEBUG] 可忽略。

图例

重编译类



权限

执行该操作的用户必须是类的创建者，或者具有 DBA 权限。

举例说明

例 重新编译类

```
ALTER CLASS mycls COMPILE;
```

12.1.4 删除类

类的删除分为两种方式：一是类头的删除，删除类头则会顺带将类体一起删除；另外一种是类体的删除，这种方式只能删除类体，类头依然存在。

12.1.4.1 删除类头

类的删除通过 DROP CLASS 完成，即类头的删除。删除类头的同时会一并删除类体。

语法格式

```
DROP CLASS [IF EXISTS] [<模式名>.]<类名>[RESTRICT | CASCADE];
```

使用说明

1 . 删除不存在的类头会报错。若指定 IF EXISTS 关键字，删除不存在的类头，不会报错；

2 . 如果被删除的类不属于当前模式，必须在语句中指明模式名；

3 . 如果一个类的声明被删除，那么对应的类体被自动删除。

权限

执行该操作的用户必须是 DBA，或者是该类的拥有者且具有 DROP PACKAGE 权限。

12.1.4.2 删除类体

从数据库中删除一个类的实现主体对象。

语法格式

```
DROP CLASS BODY [IF EXISTS] [<模式名>.]<类名>[RESTRICT | CASCADE];
```

使用说明

1 . 删除不存在的类体会报错。若指定 IF EXISTS 关键字，删除不存在的类体，不会报错；

2. 如果被删除的类不属于当前模式，必须在语句中指明模式名。

权限

执行该操作的用户必须是 DBA，或者是该类的拥有者且具有 DROP PACKAGE 权限。

12.1.5 应用实例

下面列举一个简单的应用实例。在列对象上如何使用普通 CLASS。

1. 变量对象的应用实例

```
declare
    type ex_rec_t is  record (a int, b  int);  //使用一个同结构的类型代替类定义的类型
        rec   ex_rec_t;
        o1   mycls;
        o2   mycls;
begin
        o1 = new mycls(1,2,3);
        o2 = o1;              //对象引用
        rec = o2.r;                  //变量对象的成员变量访问
        print rec.a; print rec.b;
        rec = o1.f1(4,5);    //成员函数调用
        print rec.a; print rec.b;
        print  o1.id;   //成员变量访问
end;
```

2. 列对象的应用实例

表的创建。

```
Create table tt1(c1 int, c2 mycls);
```

列对象的创建：插入数据。

```
Insert into tt1 values(1, mycls(1,2,3));
```

列对象的复制及访问。

```
Declare
    o mycls;
    id int;
begin
select top 1 c2 into o from tt1;              //列对象的复制
select top 1 c2.id into id from tt1;          //列对象成员的访问
end;
```

3. 类继承的应用实例

```
CREATE OR REPLACE CLASS cls01 NOT FINAL IS
    name VARCHAR2(10);
    MEMBER FUNCTION get_info RETURN VARCHAR2;
 END;


CREATE OR REPLACE CLASS cls02 UNDER cls01 IS
    ID INT;
    OVERRIDING MEMBER FUNCTION get_info RETURN VARCHAR2;
END;
```

12.2 JAVA CLASS 类型

JAVA 类的定义类似 JAVA 语言语法，类中可定义。

JAVA 类中可以包括以下内容:

1. 成员变量

类中的成员变量包括变量、游标和异常。成员变量必须先进行声明，声明后方可使用。变量的类型可以是 INT、CHAR 等常规数据类型，也可以是记录类型、数组类型等 DMSQL 程序数据类型，关于 DMSQL 程序数据类型的详细介绍请参考《DM8_SQL 程序设计》。

2. 成员方法

JAVA 类中的成员方法及后文的构造函数包含一个隐含参数，即自身对象，在方法实现中可以通过 this 或 self 来访问自身对象，self 等价于 this。如果不存在重名问题，也可以直接使用对象的属性和方法。

3. 构造函数

构造函数是类内定义及实现的一种特殊的函数，这类函数用于实例化类的对象，构造函数满足以下条件:

1） 函数名和类名相同;

2） 函数没有返回值类型。

构造函数存在以下的约束:

1） 系统为每个类提供两个默认的构造函数，分别为 0 参的构造函数和全参的构造函数;

2） 0 参构造函数的参数个数为 0，实例的对象内所有的属性初始化值为 NULL;

3） 全参构造函数的参数个数及类型和类内属性的个数及属性相同，按照属性的顺序依次读取参数的值并给属性赋值;

4） 用户可以自定义构造函数，一个类可以有多个构造函数，但每个构造函数的参数个

数必须不同；

5） 如果用户自定义了 0 个参数、或参数个数同属性个数相同的构造函数，则会覆盖相应的默认构造函数。

12.2.1 定义 JAVA 类

定义通过 CREATE JAVA CLASS 语句来完成，其语法为：

语法格式

```
CREATE [OR REPLACE] JAVA [PUBLIC] [ABSTRACT] [FINAL] CLASS <类名>

[EXTENDS [<模式名>.]<父类名>] {<类内定义部分> }

<类内定义部分> ::  = <类内定义>;{<类内定义>;}

<类内定义> ::= PUBLIC|PRIVATE <定义子句>

<定义子句> ::= <变量声明>|<方法定义>

<变量声明> ::= <变量属性> <数据类型><变量名列表> [<默认值定义>]

<变量属性> ::  = [STATIC] <final 属性>

<方法定义> ::= [PUBLIC|PRIVATE] [<方法继承属性>] [STATIC]  <返回类型> <函数名><参数列表> { <

实现体> }

<方法继承属性> ::= <重载属性> | <FINAL 属性> | <ABSTRACT 属性>

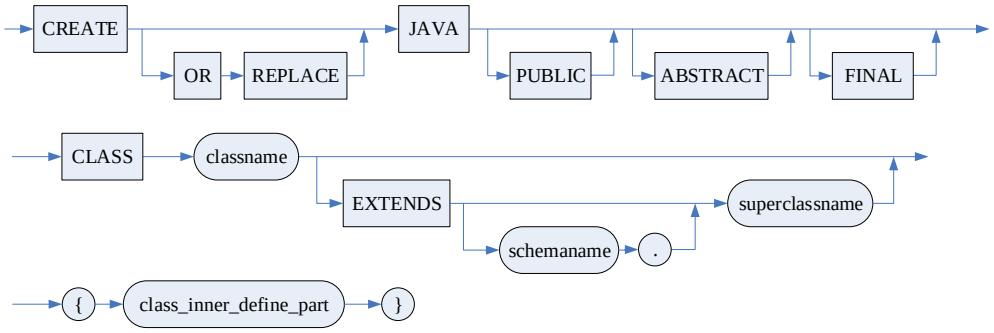
<ABSTRACT 属性> ::= ABSTRACT

<FINAL 属性> ::= FINAL

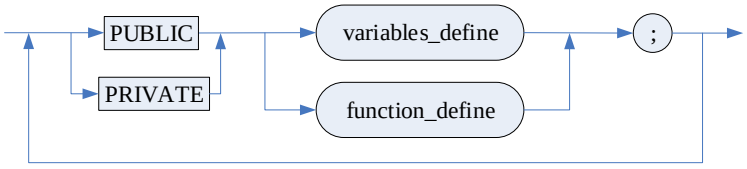
<重载属性> ::= OVERRIDE
```

图例

定义 JAVA 类



<类内定义部分>（class_inner_define_part）



使用说明

1．类的名称不能与系统创建的模式名称相同；

2．类中元素可以以任意顺序出现，其中的对象必须在引用之前被声明；

3．支持对象静态方法声明与调用。可以在方法前添加 static 保留字，以此指明方法为静态方法。静态方法只能以对象名为前缀调用，而不能在对象实例中调用；

4．支持对象成员方法声明与调用。非 STATIC 类型的非构造函数方法默认为成员方法。成员方法调用时，需要先实例化，实例化参数值缺省为 null；

5．变量声明还包括游标、异常声明。使用游标时需要在 OPEN、FETCH 及 CLOSE 后使用“CURSOR”关键字；

6．变量的数据类型可以是 INT、CHAR 等常规数据类型，也可以是记录类型、数组类型等 DMSQL 程序数据类型，若是后者则需要提前定义该类型，详细介绍请参考《DM8_SQL 程序设计》；

7．方法属性是 PUBLIC，则访问类时可以访问，如果是 PRIVATE 属性，则访问类时不可以访问该方法；

8．关于 JAVA 类继承，有以下使用限制：

1） JAVA CLASS 定义默认可继承，FINAL 表示该类不能被继承；

2） 定义子类时必须指定 EXTENDS 选项；

3） ABSTRACT 对象不能为 FINAL；

4） ABSTRACT 对象不能实例化，但是可以用其子类赋值；

5） 子类对象实例化时，必须对父类和子类的成员变量都赋值，且从父类到子类逐个赋值；

6） 不支持对象的循环继承；

7） 不支持对象的多继承，即一个类只能有一个父类；

8） 不支持父类和子类包含同名变量；

9） 父类和子类可以同名同参，此时子类必须指定 OVERRIDE；

10）方法默认为 NOT OVERRIDING，OVERRIDING 不能与 static 一起使用；

11） 父类和子类支持同名不同参（参数个数不同、参数个数相同但类型不同）的方法；

12）同名且参数个数相同但类型不同时，根据参数类型选择使用的方法；

13）方法如果声明为 ABSTRACT，则不能与 FINAL、STATIC 一起使用；

14）如果父类有多个 ABSTRACT 方法，子类可以只部分重写，但此时子类必须定义为 ABSTRACT；

15）ABSTRACT 方法不能具有主体；

16）方法默认为可继承，如果声明为 FINAL，则不能被子类重写；

17）子类可以赋值给父类；

18）如果父类对应的实例是子类或者子类的孩子，则该父类可以赋值给子类；

19）可以用 ABSTRACT 子类对非 ABSTRACT 父类进行赋值；

20）子类实例赋值给父类后，调用时使用的是父类方法而不是子类方法；

21）支持使用 super 无参方法转换为父类引用；

22) 支持使用 this()调用该类构造函数，super()调用父类构造函数;

23) 子类必须有新增成员或方法，不能完全为空。

权限

使用该语句的用户必须是 DBA 或具有 CREATE PACKAGE 数据库权限的用户。

12.2.2 重编译 JAVA 类

重新对 JAVA 类进行编译，如果重新编译失败，则将 JAVA 类置为禁止状态。

重编功能主要用于检验 JAVA 类的正确性。

语法格式

```
ALTER CLASS [<模式名>.]<JAVA 类名> COMPILE [DEBUG];
```

参数

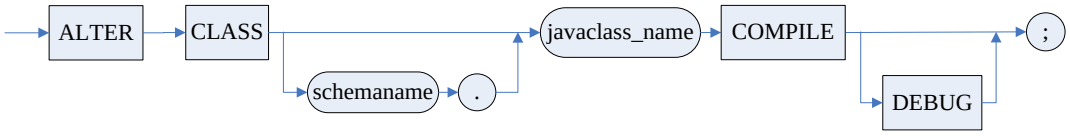
1．<模式名> 指明被重编译的 JAVA 类所属的模式;

2．<JAVA 类名> 指明被重编译的 JAVA 类的名字;

3．[DEBUG] 可忽略。

图例

重编译 JAVA 类



权限

执行该操作的用户必须是 JAVA 类的创建者，或者具有 DBA 权限。

12.2.3 删除 JAVA 类

JAVA 类的删除通过 DROP CLASS 语句完成。

语法格式

```
DROP CLASS [IF EXISTS] [<模式名>.] <JAVA 类名> [RESTRICT | CASCADE];
```

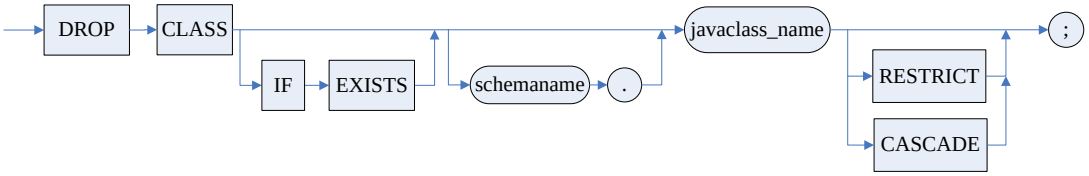
参数

1．<模式名> 指明待删除的 JAVA 类所属的模式，可忽略；

2．<JAVA 类名> 指明待删除的 JAVA 类的名字。

图例

删除 JAVA 类



使用说明

删除不存在的 JAVA 类会报错。若指定 IF EXISTS 关键字，删除不存在的 JAVA 类，不会报错。

权限

执行该操作的用户必须是 DBA，或者是该类的拥有者且具有 DROP PACKAGE 权限。

12.2.4 应用实例

下面列举一个简单的应用实例。在列对象上如何使用 JAVA CLASS。

1.创建 JAVA CLASS。

```
create or replace java class jcls
{
 int a;
  public static int testAdd2(int a, int b) {        //此处创建的是静态 STATIC 方法
            return a + b;
```

```
    }
  public int testAdd3(int a, int b, int c) {        //此处创建的是成员方法
          return a + b +c;
    }
}
```

2. 在列对象中使用 JAVA CLASS。

```
create table tt2(c1 int, c2 jcls);
insert into tt2 values(jcls.testadd2(1,2),jcls(1));          //静态方法调用
insert into tt2 values(jcls().testadd3(1,2,3),jcls(2));  //成员方法调用之前必须实例化
```

12.3 使用规则

类类型同普通的数据类型一样，可以作为表中列的数据类型，DMSQL 程序语句块中变量的数据类型或过程及函数参数的数据类型。

1．作为表中列类型或其他类成员变量属性的类不能被修改，删除时需要指定 CASCADE 级联删除

类中定义的数据类型，其名称只在类的声明及实现中有效。如果类内的函数的参数或返回值是类内的数据类型，或是进行类内成员变量的复制，需要在 DMSQL 程序中定义一个结构与之相同的类型。

根据类使用方式的不同，对象可分为变量对象及列对象。变量对象指的是在 DMSQL 程序语句块中声明的类类型的变量；列对象指的是在表中类类型的列。变量对象可以修改其属性的值而列对象不能。

2．变量对象的实例化

类的实例化通过 NEW 表达式调用构造函数完成。

3．变量对象的引用

通过‘=’进行的类类型变量之间的赋值所进行的是对象的引用，并没有复制一个新的对象。

4．变量对象属性访问

可以通过如下方式进行属性的访问。

```
<对象名>.<属性名>
```

5．变量对象成员方法调用

成员方法的调用通过以下方式调用：

```
<对象名>.<成员方法名>(<参数>{,<参数>})
```

如果函数内修改了对象内属性的值，则该修改生效。

6．列对象的插入

列对象的创建是通过 INSERT 语句向表中插入数据完成，插入语句中的值是变量对象，插入后存储在表中的数据即为列对象。

7．列对象的复制

存储在表中的对象不允许对对象中成员变量的修改，通过 into 查询或’=’进行的列到变量的赋值所进行的是对象的赋值，生成了一个与列对象数据一样的副本，在该副本上进行的修改不会影响表中列对象的值。

8．列对象的属性访问

通过如下方式进行属性的访问：

```
<列名>.<属性名>
```

9．列对象的方法调用

```
<列名>.<成员方法名>(<参数>{,<参数>})
```

列对象方法调用过程中对类型内属性的修改，都是在列对象的副本上进行的，不会影响列对象的值。

第 13 章 自定义类型

DM 支持使用 CREATE TYPE 语句创建自定义类型，具体为记录类型、对象类型、数组类型和集合类型。

其中对象类型的创建方法和另外三个略有不同，包含创建类型和创建类型体两部分。如果对象类型中声明了过程或方法，那么需要使用 CREATE TYPE BODY 定义这些过程和方法。

13.1 创建类型

可以使用 CREATE TYPE 语句创建记录类型、对象类型、数组和集合类型。

语法格式

```
CREATE [OR REPLACE] TYPE [<模式名>.]<类型名>[WITH ENCRYPTION] [<调用权限子句>]
<自定义类型定义子句>;
[<调用权限子句>]::=
    AUTHID DEFINER |
    AUTHID CURRENT_USER
<自定义类型定义子句>::=
    <对象类型定义子句> |
    <记录类型定义子句> |
    <数组类型定义子句> |
    <集合类型定义子句>
<对象类型定义子句> ::= <object 子句>|<under 子句> (<对象定义>,{<对象定义>}) [<final 属性>]
[<instantiable 属性>]
<object 子句> ::= AS|IS OBJECT
```

<under 子句> ::= UNDER [<模式名>.]<父类型名>

<对象定义> ::= <变量列表定义>|<过程声明>|<函数声明>|<构造函数声明>

<过程声明> ::= <方法继承属性>[STATIC|MEMBER] PROCEDURE <过程名> <参数列表>

<函数声明> ::= <方法继承属性>[MAP] [STATIC|MEMBER] FUNCTION <函数名> <参数列表> RETURN <返回值数据类型>[DETERMINISTIC][PIPELINED]

<方法继承属性> ::= [<重载属性>] [<final 属性>] [<instantiable 属性>]

<重载属性> ::= [NOT] OVERRIDING

<final 属性> ::= [NOT] FINAL

<instantiable 属性> ::= [NOT] INSTANTIABLE

<构造函数声明> ::= CONSTRUCTOR FUNCTION <函数名> <参数列表> RETURN SELF AS RESULT

<记录类型定义子句> ::= AS|IS RECORD(变量列表定义)

<数组类型定义子句> ::= AS|IS ARRAY <数据类型>' [' [<常量表达式>]{,[<常量表达式>]}']'

<集合类型定义子句> ::= <数组集合定义子句>|<嵌套表定义子句>|<索引表定义子句>

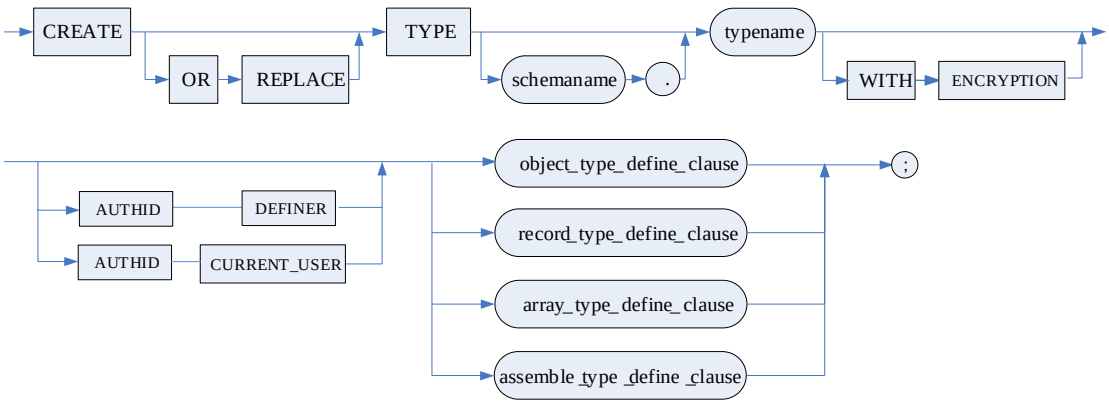
<数组集合定义子句> ::= AS|IS VARRAY(<数组长度>) OF <数据类型> [NOT NULL]

<嵌套表定义子句> ::= AS|IS TABLE OF <数据类型> [NOT NULL]

<索引表定义子句> ::= AS|IS TABLE OF <数据类型> [INDEX BY <数据类型>]

图例

创建类型



使用说明

1．创建的自定义类型的名称不能与系统创建的模式名称相同；

2．对象类型中过程和函数的声明都是前向声明，类型定义中不包括任何实现代码；达梦系统中对象类型与类是等价的，关于类的说明详见第 12 章 类类型；

3．对象类型中过程和函数可以声明为 STATIC 类型，表明为静态过程或函数；也可以声明为 MEMBER，表明为成员过程或函数，非 STATIC 且非构造函数的方法缺省为成员方法。MAP 表示将对象类型的实例映射为标量数值，只能用于成员函数；

4．关于对象类型的继承，参考 12.1 普通 CLASS 类型中类继承的相关说明；

5．WITH ENCRYPTION 选项，指定是否对自定义类型定义进行加密；

6．记录类型的定义格式与对象类型类似，但记录类型中不能有过程和函数声明；

7．在<数组类型定义子句>的数组长度定义的[]内添加','可以定义多维数组。<数组长度>为正整数，若指定了数组长度，则定义的是静态数组，数组长度是固定的。若没有指定数组长度，则定义的是动态数组，其数组长度是在使用时指定。理论上 DM 支持静态数组的每一个维度的最大长度为 65534，动态数组的每一个维度的最大长度为 10485760，但是数组最大长度同时受系统内部空间大小的限制，如果超出堆栈/堆的空间限制，系统会报错。

8．数组集合类型中的常量表达式定义了其最大容量，其数组元素数据类型可以是基础类型，也可以是自定义数据类型。

9．嵌套表类型和索引表类型没有元素个数限制，元素数据类型可以是基础数据类型也可以是其它自定义类型或是对象、记录、静态数组，但是不能是动态数组；第二个则是索引表的下标类型，目前仅支持 INTEGER/INT 和 VARCHAR 两种类型，分别代表整数下标和字符串下标。对于 VARCHAR 类型，长度不能超过 1024。

权限

1. 使用该语句的用户必须是 DBA 或具有 CREATE PACKAGE 数据库权限的用户。

2.可以用关键字 AUTHID DEFINER |AUTHID CURRENT_USER 指定自定义类型的调用者权限，若为 DEFINER，则采用自定义类型定义者权限，若为 CURRENT_USER 则为当前用户权限，默认为定义者权限。

13.2 创建类型体

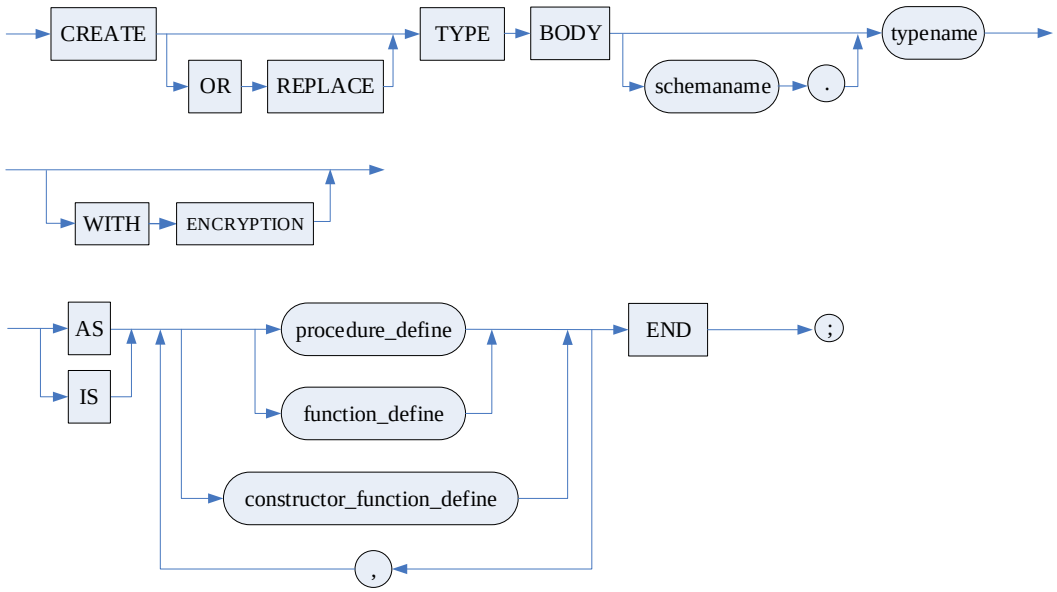
专用于对象类型。为可选项。当在创建类型中声明了过程或函数时使用，用于对声明的过程或者函数进行实现。如果没有声明过程或函数，则创建类型体部分可以省略。

语法格式

```
CREATE [OR REPLACE] TYPE BODY [<模式名>.]<类型名>[WITH ENCRYPTION] AS|IS <对象类型体定义子句> END;
<对象类型体定义子句>::= <对象类型体定义>,{<对象类型体定义>}
<对象类型体定义>::= <过程实现>|<函数实现>|<构造函数实现>
<过程实现> ::= <方法继承属性>[STATIC|MEMBER] PROCEDURE <过程名> <参数列表> AS|IS  BEGIN <实现体> END [过程名]
<函数实现> ::= <方法继承属性>[MAP] [STATIC|MEMBER] FUNCTION <函数名><参数列表> RETURN <返回值数据类型>[DETERMINISTIC] [PIPELINED] AS|IS BEGIN <实现体> END [函数名]
<方法继承属性> ::= [<重载属性>] [<final 属性>] [<instantiable 属性>]
<重载属性> ::= [NOT] OVERRIDING
<final 属性> ::= [NOT] FINAL
<instantiable 属性> ::= [NOT] INSTANTIABLE
<构造函数实现> ::= CONSTRUCTOR FUNCTION <函数名> <参数列表>  RETURN SELF AS RESULT AS|IS BEGIN <实现体> END [函数名]
```

图例

创建类型体



使用说明

1．对象类型体中的过程、函数定义必须和类型定义中的前向声明完全相同。包括过程的名字、参数定义列表的参数名和数据类型定义；

2．支持<实现体>中使用形如"self:= new obj"的自赋值语法，可以在构造函数内部手动生成新的对象并赋值给 self；

3．在<参数列表>中，支持 self 参数作为第一个参数使用，在调用该构造函数时会自动忽略该 self 参数；开启 oracle 兼容模式后，函数调用时指定第一个参数为 self 时自动忽略。

权限

使用该语句的用户必须是 DBA 或该类型对象的拥有者且具有 CREATE PACKAGE 数据库权限的用户。

13.3 重编译类型

重新对类型进行编译，如果重新编译失败，则将类型置为禁止状态。

重编功能主要用于检验类型的正确性。

语法格式

```
ALTER TYPE [<模式名>.]<类型名> COMPILE [DEBUG];
```

参数

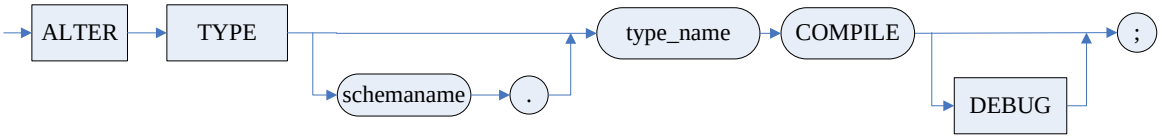
1．<模式名> 指明被重编译的类型所属的模式；

2．<类型名> 指明被重编译的类型的名字；

3．[DEBUG] 可忽略。

图例

重编译类型



权限

执行该操作的用户必须是类型的创建者，或者具有 DBA 权限。

13.4 删除类型

类型的删除分为类型删除和类型体的删除。对于拥有类型体的对象类型，删除类型会将类型体一起删除；删除类型体的话，类型本身依然存在。

13.4.1 删除类型

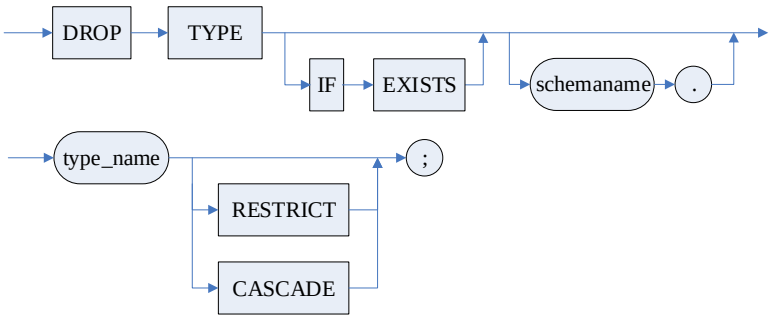
使用 DROP TYPE 完成类型的删除。对于拥有类型体的对象类型，删除类型会将类型体一起删除。

语法格式

```
DROP TYPE [IF EXISTS] [<模式名>.]<类型名>[RESTRICT | CASCADE];
```

图例

删除类型



使用说明

1 . 删除不存在的类型会报错。若指定 IF EXISTS 关键字，删除不存在的类型，不会报错；

2 . 如果被删除的类型不属于当前模式，必须在语句中指明模式名；

3 . 如果一个拥有类型体的对象类型被删除，那么对应的类型体被自动删除。

权限

执行该操作的用户必须是 DBA，或者是该类型的拥有者且具有 DROP PACKAGE 权限。

13.4.2 删除类型体

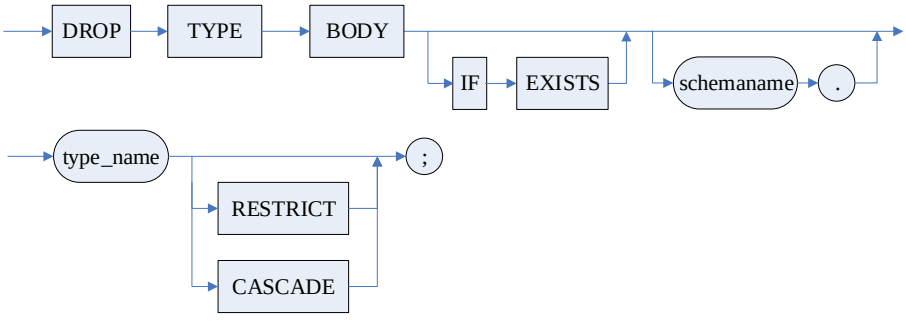
使用 DROP TYPE BODY 删除一个对象类型的类型体。

语法格式

```
DROP TYPE BODY [IF EXISTS] [<模式名>.]<类型名>[RESTRICT | CASCADE];
```

图例

删除体类型



使用说明

1．删除不存在的类型体会报错。若指定 IF EXISTS 关键字，删除不存在的类型体，不会报错；

2. 如果被删除的类型体不属于当前模式，必须在语句中指明模式名。

权限

执行该操作的用户必须是 DBA，或者是该类型的拥有者且具有 DROP PACKAGE 权限。

13.5 自定义类型的使用

13.5.1 使用规则

1. 对象类型与类等价，类的使用规则可详见第 12 章《类类型》 ；

2. 创建的记录类型、数组类型和集合类型，可以直接在 DMSQL 程序语句块中使用，不必在语句块中声明类型，使用方式请参考《DM8_SQL 程序设计》;

3. 用户自定义数据类型可以作为其他用户自定义数据类型的元素类型或成员变量类型；

4. 只有对象类型、VARRAY 类型以及嵌套表类型可以直接作为表中列的数据类型；其他类型只能作为上述类型中成员变量的类型或类型中嵌套使用的数据类型。但含有 BOOL、VOID、游标类型、异常变量、ANYTYPE, RESTOBJ, AOT 的自定义类型也不能作为表中列的数据类型。

13.5.2 应用实例

例 1 创建一个用来表示复数的对象类型，有实数部分和虚数部分，并实现了复数的加与减的操作。

```
CREATE TYPE COMPLEX AS OBJECT(
RPART   REAL,
IPART   REAL,
```

```
FUNCTION PLUS(X COMPLEX) RETURN COMPLEX,
FUNCTION LES(X COMPLEX) RETURN COMPLEX
);
/

CREATE TYPE BODY COMPLEX AS
FUNCTION PLUS(X COMPLEX) RETURN COMPLEX IS
BEGIN
RETURN COMPLEX(RPART+X.RPART, IPART+X.IPART);
END;

FUNCTION LES(X COMPLEX) RETURN COMPLEX IS
BEGIN
    RETURN COMPLEX(RPART-X.RPART, IPART-X.IPART);
END;
END;
```

建立表 c_tab，表中的第二列的列类型为 complex 对象类型。

```
CREATE TABLE C_TAB(C1 INT, C2 COMPLEX);
```

向表 c_tab 中插入数据。

```
INSERT INTO C_TAB VALUES(1, COMPLEX(2,3));
INSERT INTO C_TAB VALUES(2, COMPLEX(4,2).PLUS(COMPLEX(2,3)));
```

例 2　创建一个数组集合类型。

```
CREATE OR REPLACE TYPE ARR_NUM AS VARRAY(3) OF NUMBER;
```

```
/
```

建立表 T1，表中的第二列的列类型为 ARR_NUM 数组集合类型，并向表中插入数据。

```
CREATE TABLE T1 (C1 INT, C2 ARR_NUM);

INSERT INTO T1 VALUES(1,ARR_NUM(1,2));

INSERT INTO T1 VALUES(2,ARR_NUM(3,4,5));
```

查询表中数据。

```
SELECT * FROM T1;
```

查询结果如下：

```
行号        C1              C2

---------- ----------- ---------------------

1              1               SYSDBA.ARR_NUM(1,2)

2              2               SYSDBA.ARR_NUM(3,4,5)
```

查询表的数组集合中的数据。

```
SELECT * FROM TABLE(SELECT C2 FROM T1 WHERE C1=2);
```

查询结果如下：

```
行号        COLUMN_VALUE

---------- ------------

1              3

2              4

3              5
```

例 3　创建一个嵌套表类型。

```
CREATE OR REPLACE TYPE T_CHA AS TABLE OF CHAR;

/
```

建立表 T2，表中的第二列的列类型为 T_CHA 嵌套表类型，并向表中插入数据。

```
CREATE TABLE T2 (C1 INT, C2 T_CHA);

INSERT INTO T2 VALUES(1,T_CHA('A','B'));

INSERT INTO T2 VALUES(2,T_CHA('C','D','E'));
```

查询表中数据。

```
SELECT * FROM T2;
```

查询结果如下：

```
行号        C1              C2

---------- ----------- --------------------

1              1               SYSDBA.T_CHA(A,B)

2              2               SYSDBA.T_CHA(C,D,E)
```

查询表的嵌套表中的数据。

```
SELECT * FROM TABLE(SELECT C2 FROM T2 WHERE C1=1);
```

查询结果如下：

```
行号        COLUMN_VALUE

---------- ------------

1              A

2              B
```

13.5.3 IS OF TYPE 的使用

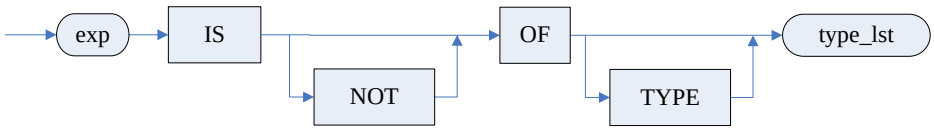
IS OF TYPE 谓词用于判断一个表达式对应实例是否是指定 TYPE 列表的子类或者是完全相同的类型。

语法格式

```
<表达式> IS [NOT] OF [TYPE] (<TYPE 列表>)
```

图例

IS OF TYPE



使用说明

1. 用户需要具有表达式对应 TYPE 和 TYPE 列表中涉及所有 TYPE 的执行权限。

2. 如果表达式对应实例不是指定 TYPE 列表的子类或者是完全相同的类型，则直接报错表达式类型错误。

举例说明

例 1

```
CREATE OR REPLACE TYPE TYPE01 AS OBJECT(

NAME VARCHAR2(10))

NOT FINAL;

/

CREATE TABLE T1(C1 INT , C2 TYPE01);

INSERT INTO T1 VALUES(1, TYPE01('WSY'));

COMMIT;



SELECT * FROM T1 WHERE C2 IS OF (TYPE01);
```

查询结果如下：

```
行号        C1              C2

---------- ----------- ------------------

1             1               SYSDBA.TYPE01(WSY)
```

例 2

```
SELECT * FROM T1 WHERE C1 IS OF (TYPE01);
```

查询结果报错:

```
第 1 行附近出现错误[-2059]:表达式类型错误.
```

例 3

```
SELECT * FROM T1 WHERE C1 IS NOT OF (TYPE01);
```

查询结果报错:

```
第 1 行附近出现错误[-2059]:表达式类型错误.
```

第 14 章 触发器

DM 是一个具有主动特征的数据库管理系统，其主动特征包括约束机制和触发器机制。通过触发器机制，用户可以定义、删除和修改触发器。DM 自动管理和运行这些触发器，从而体现系统的主动性，方便用户使用。

触发器(Trigger)定义为当某些与数据库有关的事件发生时，数据库应该采取的操作。这些事件包括全局对象、数据库下某个模式、模式下某个基表上的 INSERT、DELETE 和 UPDATE 操作。触发器与存储模块类似，都是在服务器上保存并执行的一段 DMSQL 程序语句。不同的是：存储模块必须被显式地调用执行，而触发器是在相关的事件发生时由服务器自动地隐式地触发。触发器是语句的一个组成部分，即直到一个语句触发的所有触发器执行完成之后该语句才结束，而其中任何一个触发器执行的失败都将导致该语句的失败，触发器所做的任何工作都属于使该触发器启动执行的语句。

触发器为用户提供了一种自己扩展数据库功能的方法。关于触发器应用的例子有：

1. 利用触发器实现表约束机制(如：PRIMARY KEY、FOREIGN KEY、CHECK 等)无法实现的复杂的引用完整性；
2. 利用触发器实现复杂的事务规则(如：想确保薪水增加量不超过 25%)；
3. 利用触发器维护复杂的缺省值(如：条件缺省)；
4. 利用触发器实现复杂的审计功能；
5. 利用触发器防止非法的操作。

触发器是应用程序分割技术的一个基本组成部分，它将事务规则从应用程序的代码中移到数据库中，从而可确保加强这些事务规则并提高它们的性能。触发器中可以定义变量，但是必须以 DECLARE 开头。

需要说明的是，在 DM 的数据守护环境下，备库上定义的触发器是不会被触发的。

在本章各例中，如不特别说明，各例均使用示例库 BOOKSHOP，用户均为建表者 SYSDBA。

14.1 触发器的定义

触发器分为表触发器、事件触发器和时间触发器。表触发器是对表里数据操作引发的数据库的触发；事件触发器是对数据库对象操作引起的数据库的触发；时间触发器是一种特殊的事件触发器。

14.1.1 表触发器

14.1.1.1 表触发器语法

用户可使用触发器定义语句(CREATE TRIGGER)在一张基表上创建触发器。下面是表触发器定义语句的语法。

语法格式

```
CREATE [OR REPLACE] TRIGGER [<模式名>.]<触发器名> [WITH  ENCRYPTION]
<触发限制描述> [REFERENCING <trig_referencing_list>][FOR EACH {ROW | STATEMENT}]
[<trigger_ordering_clause>] [WHEN (<条件表达式>)]<触发器体>
<trig_referencing_list>::= <referencing_1>|<referencing_2>
<referencing_1>::=OLD [ROW] [AS] <引用变量名> [ NEW [ROW] [AS] <引用变量名>]
<referencing_2>::=NEW [ROW] [AS] <引用变量名>
<触发限制描述>::=<触发限制描述 1> | <触发限制描述 2>
<触发限制描述 1>::= <BEFORE|AFTER> <触发事件列表> [LOCAL] ON <触发表名>
<触发限制描述 2>::= INSTEAD OF <触发事件列表> [LOCAL] ON <触发视图名>
<触发表名>::=[<模式名>.]<基表名>
<触发事件>::=INSERT|DELETE|{UPDATE|{UPDATE OF<触发列清单>}}
```

```
<触发事件列表>::=<触发事件> | {<触发事件列表> OR <触发事件>}
<trigger_ordering_clause>::= FOLLOWS | PRECEDES [<模式名>.]<触发器名> {, [<模式名>.]<触发器名>}
```

参数

1．<触发器名> 指明被创建的触发器的名称；

2．BEFORE 指明触发器在执行触发语句之前启动；

3．AFTER 指明触发器在执行触发语句之后启动；

4．INSTEAD OF 指明触发器执行时替换原始操作；

5．<触发事件> 指明使触发器启动的事件。INSTEAD OF 中不支持｛UPDATE OF <触发列清单>｝；

6．<基表名> 指明被创建触发器的基表的名称；

7．WITH ENCRYPTION 选项，指定是否对触发器定义进行加密；

8 .REFERENCING 子句 指明相关名称可以在元组级触发器的触发器体和 WHEN 子句中利用相关名称来访问当前行的新值或旧值，缺省的相关名称为 OLD 和 NEW；

9．<引用变量名> 标识符，指明行的新值或旧值的相关名称；

10．FOR EACH 子句 指明触发器为元组级或语句级触发器。FOR EACH ROW 表示为元组级触发器，它受被触发命令影响、且 WHEN 子句的表达式计算为真的每条记录执行一次。FOR EACH STATEMENT 为语句级触发器，它对每个触发命令执行一次。FOR EACH 子句缺省则为语句级触发器；

11. <trigger_ordering_clause> 指定触发器的触发顺序。FOLLOWS 表示当前触发器需要在指定触发器之后触发；PRECEDES 表示当前触发器需要在指定触发器之前触发。指定的触发器必须和当前创建的触发器类型完全一致，且不存在循环触发结构。最多支持指定 254 个触发器；

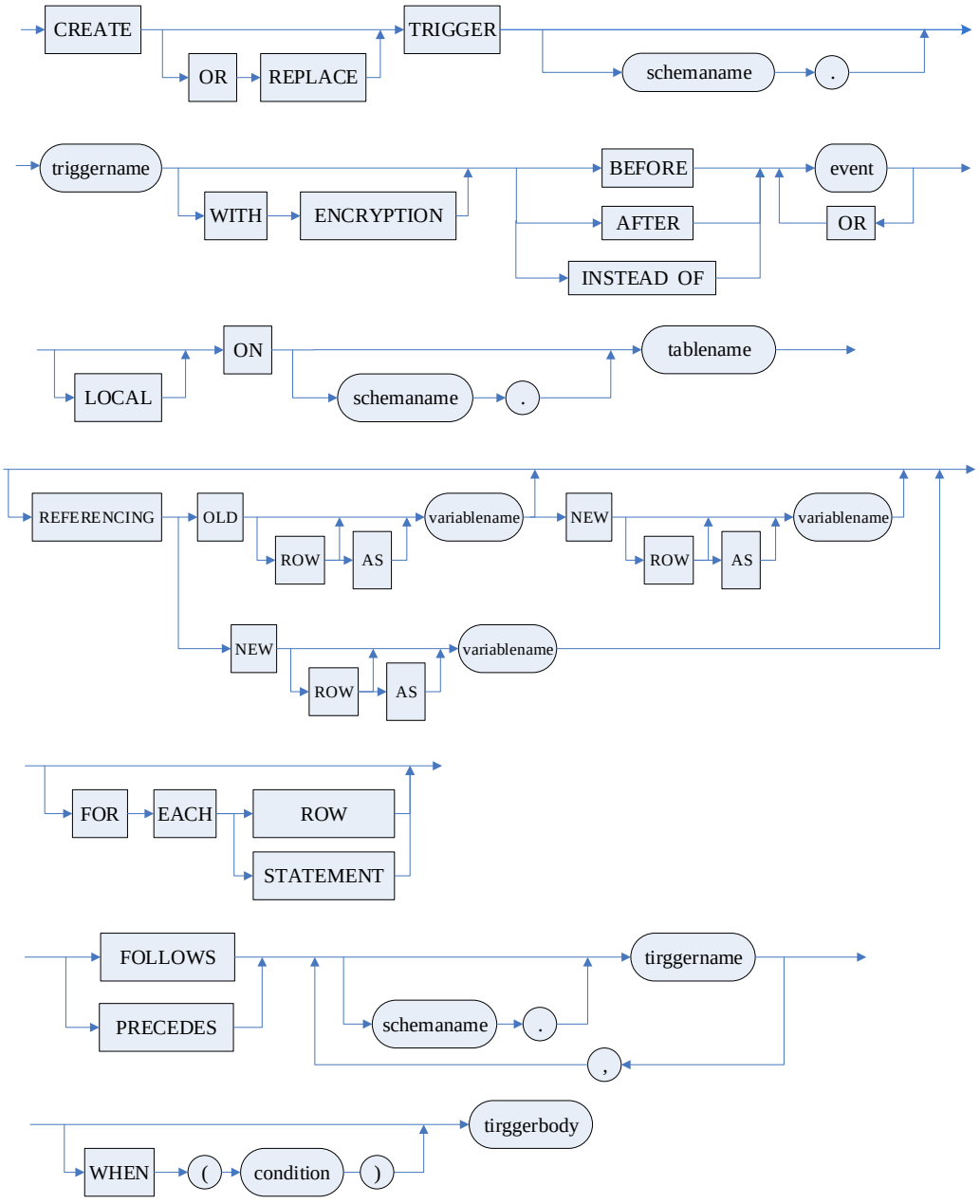
12．WHEN 子句 表触发器中只允许为元组级触发器指定 WHEN 子句，它包含一个布尔

表达式，当表达式的值为 TRUE 时，执行触发器；否则，跳过该触发器；

13．<触发器体> 触发器被触发时执行的 SQL 过程语句块。

图例

表触发器



功能

创建触发器，并使其处于允许状态。

使用说明

1．<触发器名>是触发器的名称，它不能与模式内的其他模式级对象同名，并且不能使用"MTRG$_"/"TRIG_ARR$"/"BM$_TRG_"/"REF_CON_TRIG"/"SREF_CON_TRIG"/"TRIG_MDRT$TMP"/"TRIG_HNSW$TMP"/"TRIG_IVFFLAT$TMP" 作为前缀；

2．可以使用 OR REPLACE 选项来替换一个触发器，但是要注意被替换的触发器的触发表不能改变。如果要在同一模式内不同的表上重新创建一个同名的触发器，则必须先删除该触发器，然后再创建；

3．<触发事件子句>说明使触发器启动的事件；<触发器体>是触发器的执行代码；<引用子句>用来引用正处于修改状态下的行中的数据。如果指定了<触发条件>子句，则首先对该条件表达式求值，<触发器体>只有在该条件为真值时才运行。<触发器体>是一个 DMSQL 程序语句块，它与存储模块定义语句中<模块体>的语法基本相同；

4．在一张基表上允许创建的表触发器的个数没有限制，一共允许有 12 种类型。它们分别是：BEFORE INSERT 元组级、BEFORE INSERT 语句级、AFTER INSERT 元组级、AFTER INSERT 语句级、BEFORE UPDATE 元组级、BEFORE UPDATE 语句级、AFTER UPDATE 元组级、AFTER UPDATE 语句级、BEFORE DELETE 元组级、BEFORE DELETE 语句级、AFTER DELETE 元组级和 AFTER DELETE 语句级；

5．触发器是在 DML 语句运行时触发的。执行 DML 语句的算法步骤如下：

1） 如果有语句级前触发器的话，先运行该触发器；

2） 对于受语句影响每一行：

a） 如果有元组级前触发器的话，运行该触发器；

b） 执行该语句本身；

c） 如果有元组级后触发器的话，运行该触发器。

3） 如果有语句级后触发器的话，运行该触发器。

6．INSTEAD OF 触发器仅允许建立在视图上，并且只支持元组级触发；

7．表触发器支持跨模式，创建跨模式触发器时创建者需要拥有 CREATE ANY TRIGGER 权限；触发器体中的操作根据触发器所在的用户检测权限。若创建表触发器时不指定模式名，则模式名默认为创建者所在的模式；

8．水平分区子表、HUGE 表不支持表触发器；

9．在 MPP 环境下，执行 LOCAL 类型触发器时，会话会被临时变为 LOCAL 类型，因此触发器体只会在本节点执行，不会产生节点间的数据交互，触发器体中只能包含表的值插入操作，如果插入数据的目标节点不是于本节点，则会报错，随机分布表没有此限制。

14.1.1.2 表触发器详解

下面对表触发器的触发动作、级别和时机进行详细介绍。

14.1.1.2.1 触发动作

使表触发器启动的触发动作是三种数据操作命令，即 INSERT、DELETE 和 UPDATE 操作。在触发器定义语句中用关键字 INSERT、DELETE 和 UPDATE 指明构成一个触发器事件的数据操作的类型，其中 UPDATE 触发器会依赖于所修改的列，在定义中可通过 UPDATE OF <触发列清单>的形式来指定所修改的列，<触发列清单>指定的字段数不能超过 128 个。

在 PERSON.PERSON 上建立触发器。如下例所示：

```
SET SCHEMA PERSON;



CREATE OR REPLACE TRIGGER TRG_UPD

AFTER UPDATE OF NAME,PHONE ON PERSON.PERSON

BEGIN

  PRINT 'UPDATE OPERATION ON COLUMNS NAME OR PHONE OF PERSON';

END;
```

```
SET SCHEMA SYSDBA;
```

当对表 PERSON 进行更新操作，并且更新的列中包括 NAME 或 PHONE 时，此例中定义的触发器 TRG_UPD 将被启动。

如果一个触发器的触发事件为 INSERT，则该触发器被称为 INSERT 触发器，同样也可以这样来定义 DELETE 触发器和 UPDATE 触发器。一个触发器的触发事件也可以是多个数据操作命令的组合，这时这个触发器可由多种数据操作命令触发。如下例所示：

```
SET SCHEMA PERSON;



CREATE OR REPLACE TRIGGER TRG_INS_DEL

AFTER INSERT OR DELETE ON PERSON.PERSON

BEGIN

   PRINT 'INSERT OR DELETE OPERATION ON PERSON';

END;



SET SCHEMA SYSDBA;
```

此例中的触发器 TRG_INS_DEL 既是 INSERT 触发器又是 DELETE 触发器，对表 PERSON 的 INSERT 和 DELETE 操作都会使该触发器启动。

14.1.1.2.2 触发级别

根据触发器的级别可分为元组级（也称行级）和语句级。

元组级触发器，对触发命令所影响的每一条记录都触发一次。假如一个 DELETE 命令从表中删除了 1000 行记录，那么这个表上的元组级 DELETE 触发器将被执行 1000 次。元组级触发器常用于数据审计、完整性检查等应用中。元组级触发器是在触发器定义语句中通过 FOR EACH ROW 子句创建的。对于元组级触发器，可以用一个 WHEN 子句来限制针对当前

记录是否执行该触发器。WHEN 子句包含一条布尔表达式，当它的值为 TRUE 时，执行触发器；否则，跳过该触发器。

语句级触发器，对每个触发命令执行一次。例如，对于一条将 500 行记录插入表 TABLE_1 中的 INSERT 语句，这个表上的语句级 INSERT 触发器只执行一次。语句级触发器一般用于对表上执行的操作类型引入附加的安全措施。语句级触发器是在触发器定义语句中通过 FOR EACH STATEMENT 子句创建的，该子句可缺省。

以下分别是元组级触发器和语句级触发器的例子。

```
SET SCHEMA PERSON;



CREATE OR REPLACE TRIGGER TRG_DEL_ROW
BEFORE DELETE ON PERSON.PERSON
FOR EACH ROW                    //元组级：此子句一定不能省略
BEGIN
   PRINT 'DELETE' || :OLD.NAME|| ' ON PERSON';
END;



CREATE OR REPLACE TRIGGER TRG_INS_ST
AFTER INSERT ON PERSON.PERSON
FOR EACH STATEMENT        //语句级：此子句可省略
BEGIN
   PRINT 'AFTER INSERT ON PERSON';
END;



SET SCHEMA SYSDBA;
```

14.1.1.2.3 触发时机

触发时机通过两种方式指定。一是通过指定 BEFORE 或 AFTER 关键字，选择在触发动作之前或之后运行触发器；二是通过指定 INSTEAD OF 关键字，选择在动作触发的时候，替换原始操作，INSTEAD OF 允许建立在视图上，并且只支持元组级触发。

在元组级触发器中可以引用当前修改的记录在修改前后的值，修改前的值称为旧值，修改后的值称为新值。对于插入操作不存在旧值，而对于删除操作则不存在新值。

对于新、旧值的访问请求常常决定一个触发器是 BEFORE 类型还是 AFTER 类型。如果需要通过触发器对插入的行设置列值，那么为了能设置新值，需要使用一个 BEFORE 触发器，因为在 AFTER 触发器中不允许用户设置已插入的值。在审计应用中则经常使用 AFTER 触发器，因为元组修改成功后才有必要运行触发器，而成功地完成修改意味着成功地通过了该表的引用完整性约束。

例 1 BEFORE 触发器和 AFTER 触发器的举例。

BEFORE 触发器示例

```
SET SCHEMA OTHER;



CREATE OR REPLACE TRIGGER TRG_INS_BEFORE
BEFORE INSERT ON OTHER.READER
FOR EACH ROW
BEGIN
   :NEW.READER_ID:=:NEW.READER_ID+1;
END;



SET SCHEMA SYSDBA;
```

该触发器在插入一条记录前，将记录中 READER_ID 列的值加 1。

```
CREATE TABLE T_TEMP(C1 INT,C2 CHAR(20));
```

新建表 T_TEMP。

```
SET SCHEMA OTHER;



CREATE OR REPLACE TRIGGER TRG_INS_AFTER

AFTER INSERT ON OTHER.READER

FOR EACH ROW

BEGIN

  INSERT INTO SYSDBA.T_TEMP VALUES(:NEW.READER_ID, 'INSERT ON READER');

END;



SET SCHEMA SYSDBA;
```

该触发器在插入一条记录后，将插入的值以及操作类型记录到用于审计的表 T_TEMP 中。

例 2 INSTEAD OF 触发器举例。

```
create table t1(a int,b int);

insert into t1 values(10,10);

insert into t1 values(11,11);

create view v1 as select * from t1;
```

在视图 v1 上创建 INSTEAD OF 触发器。

```
CREATE OR REPLACE TRIGGER tri1

INSTEAD OF UPDATE ON v1

BEGIN

  insert into t1 values(111,111);    //替换动作
```

```
END;
```

当执行 UPDATE 动作时候，就会触发。将下面的动作替换成触发器里的动作。

```
update v1 set a=100 where a=10;
```

查询结果如下：

```
A                B

---------- ----------

10               10

11               11

111              111
```

由上面的查询结果可以看出。更新操作并没有成功，而是被触发器中的替换动作替换了。这就是 INSTEAD OF 的妙用之处。

14.1.1.2.4 总结

综上所述，在一张基表上所允许的可能的合法表级触发器类型共有 12 种，如表 14.1.1 所示。

表 14.1.1 表触发器类型

名　　称	功　　能
BEFORE INSERT	在一个 INSERT 处理前触发一次
AFTER INSERT	在一个 INSERT 处理后触发一次
BEFORE DELETE	在一个 DELETE 处理前触发一次
AFTER DELETE	在一个 DELETE 处理后触发一次
BEFORE UPDATE	在一个 UPDATE 处理前触发一次
AFTER UPDATE	在一个 UPDATE 处理后触发一次
BEFORE INSERT FOR EACH ROW	每条新记录插入前触发

AFTER INSERT FOR EACH ROW	每条新记录插入后触发
BEFORE DELETE FOR EACH ROW	每条记录被删除前触发
AFTER DELETE FOR EACH ROW	每条记录被删除后触发
BEFORE UPDATE FOR EACH ROW	每条记录被修改前触发
AFTER UPDATE FOR EACH ROW	每条记录被修改后触发

14.1.1.3 触发器启动顺序

下面是执行 DML 语句的算法步骤:

1．如果有语句级前触发器的话，先运行该触发器;

2．对于受语句影响每一行:

1) 如果有元组级前触发器的话，运行该触发器;

2) 执行该语句本身;

3) 如果有元组级后触发器的话，运行该触发器。

3．如果有语句级后触发器的话，运行该触发器。

为了说明上面的算法，假设我们用 OTHER.READER 表为例，并在其上创建了所有四种 UPDATE 触发器，即之前、之后、元组级前和元组级后。其代码如下:

```
SET SCHEMA OTHER;



CREATE OR REPLACE TRIGGER Reader_Before_St

BEFORE UPDATE ON OTHER.READER

BEGIN

   PRINT 'BEFORE UPDATE TRIGGER FIRED';

END;



CREATE OR REPLACE TRIGGER Reader_After_St
```

```
AFTER UPDATE ON OTHER.READER

BEGIN

   PRINT 'AFTER UPDATE TRIGGER FIRED';

END;



CREATE OR REPLACE TRIGGER   Reader_Before_Row

BEFORE UPDATE ON OTHER.READER

FOR EACH ROW

BEGIN

   PRINT 'BEFORE UPDATE EACH ROW TRIGGER FIRED';

END;



CREATE OR REPLACE TRIGGER   Reader_After_Row

AFTER UPDATE ON OTHER.READER

FOR EACH ROW

BEGIN

   PRINT 'AFTER UPDATE EACH ROW TRIGGER FIRED';

END;



SET SCHEMA SYSDBA;
```

现在，执行更新语句：

```
UPDATE OTHER.READER SET AGE=AGE+1;
```

该语句对三行有影响。语句级前触发器和语句级后触发器将各自运行一次，而元组级前触发器和元组级后触发器则各运行三次。因此，服务器返回的打印消息应为：

```
BEFORE UPDATE TRIGGER FIRED

BEFORE UPDATE EACH ROW TRIGGER FIRED

AFTER UPDATE EACH ROW TRIGGER FIRED

BEFORE UPDATE EACH ROW TRIGGER FIRED

AFTER UPDATE EACH ROW TRIGGER FIRED

BEFORE UPDATE EACH ROW TRIGGER FIRED

AFTER UPDATE EACH ROW TRIGGER FIRED

AFTER UPDATE TRIGGER FIRED
```

同类型触发器的启动顺序可以通过 FOLLOWS 或 PRECEDES 关键字来指定，具体请参考 14.1.1.1 表触发器语法。

14.1.1.4 新、旧行值的引用

前面曾经提到，在元组级触发器内部，可以访问正在处理中的记录的数据，这种访问是通过两个引用变量:OLD 和:NEW 实现的。:OLD 表示记录被处理前的值，:NEW 表示记录被处理后的值，标识符前面的冒号说明它们是宿主变量意义上的连接变量，而不是一般的 DMSQL 程序变量。我们还可以通过引用子句为这两个行值重新命名。

引用变量与其它变量不在同一个命名空间，所以变量可以与引用变量同名。在触发器体中使用引用变量时，必须采用下列形式：

```
:引用变量名.列名
```

其中，列名必须是触发表中存在的列，否则编译器将报错。

下表总结了标识符:OLD 和:NEW 的含义。

表 14.1.2 标识符:OLD 和:NEW 的含义

触发语句	标识符:OLD	标识符:NEW
INSERT	无定义，所有字段都为 NULL	该语句结束时将插入的值

UPDATE	更新前行的旧值	该语句结束时将更新的值
DELETE	行删除前的旧值	无定义，所有字段都为 NULL

:OLD 引用变量只能读取，不能赋值(因为设置这个值是没有任何意义的)；而:NEW 引用变量则既可读取，又可赋值(当然必须在 BEFORE 类型的触发器中，因为数据操作完成后再设置这个值也是没有意义的)。通过修改:NEW 引用变量的值，我们可以影响插入或修改的数据。

注意：对于 INSERT 操作，引用变量:OLD 无意义；而对于 DELETE 操作，引用变量:NEW 无意义。如果在 INSERT 触发器体中引用:OLD，或者在 DELETE 触发器体中引用:NEW，不会产生编译错误。但是在执行时，对于 INSERT 操作，:OLD 引用变量的值为空值；对于 DELETE 操作，:NEW 引用变量的值为空值，且不允许被赋值。

例1 下例中触发器GenerateValue使用了:OLD引用变量。该触发器是一个UPDATE 前触发器，其目的是做更新操作时不论是否更新表中某列的值，该列的值保持原值不变。这里使用 PRODUCTION.PRODUCT 表为例。

```
SET SCHEMA PRODUCTION;



CREATE OR REPLACE TRIGGER GenerateValue

BEFORE UPDATE ON PRODUCTION.PRODUCT

FOR EACH ROW

BEGIN

:new.NOWPRICE:=:old.NOWPRICE;

END;



SET SCHEMA SYSDBA;
```

当执行一个 UPDATE 语句时，无论用户是否给定 NOWPRICE 字段的值，触发器都将自

动保持原来的NOWPRICE值不变。例如,用户查询PRODUCTID=1的一行数据的NOWPRICE值。

```
SELECT NOWPRICE FROM PRODUCTION.PRODUCT WHERE PRODUCTID=1;
```

可得到结果为 NOWPRICE=15.2000;

用户可以执行如下所示的 UPDATE 语句。

```
UPDATE PRODUCTION.PRODUCT SET NOWPRICE =1.0000 WHERE PRODUCTID=1;
```

再次执行查询语句。

```
SELECT NOWPRICE FROM PRODUCTION.PRODUCT WHERE PRODUCTID=1;
```

可发现结果仍然为 NOWPRICE=15.2000，而非修改的 1.0000。

例2 下例中触发器GenerateValue使用了:NEW引用变量。该触发器是一个INSERT 前触发器，其目的是自动生成一些字段值。这里使用 PRODUCTION.PRODUCT 表为例。

```
SET SCHEMA PRODUCTION;



CREATE OR REPLACE TRIGGER GenerateValue

BEFORE INSERT ON PRODUCTION.PRODUCT

FOR EACH ROW

BEGIN

:new.NOWPRICE:=:new.ORIGINALPRICE*:new.DISCOUNT;

END;



SET SCHEMA SYSDBA;
```

触发器 GenerateValue 实际上是修改引用变量:NEW 的值，这就是:NEW 引用变量的用途之一。当执行一个 INSERT 语句时，无论用户是否给定 NOWPRICE 字段的值，触发器都将自动利用 ORIGINALPRICE 字段和 DISCOUNT 字段来计算 NOWPRICE。例如，用户可以执行如

下所示的 INSERT 语句。

```
INSERT INTO PRODUCTION.PRODUCT
(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATEGORYID,
SATETYSTOCKLEVEL,ORIGINALPRICE,DISCOUNT,SELLSTARTTIME)
VALUES('老人与海','海明威','上海出版社','2006-8-1','9787532740088',1,'10','100','7.5','2008-1-10');
```

即使为 NOWPRICE 字段指定了值，该值也会被忽略，因为触发器将改变该值。

```
INSERT INTO PRODUCTION.PRODUCT
(NAME,AUTHOR,PUBLISHER,PUBLISHTIME,PRODUCTNO,PRODUCT_SUBCATEGORYID,
SATETYSTOCKLEVEL,ORIGINALPRICE,DISCOUNT,NOWPRICE,SELLSTARTTIME)
VALUES('老人与海','海明威','上海出版社','2006-8-1','9787532740089',1,'10','100','7.5','88','2008-1-10');
```

新插入元组的 NOWPRICE 字段将被触发器修改为 750.0000，而不是语句中的 88。

14.1.1.5 触发器谓词

如前面介绍的，触发事件可以是多个数据操作的组合，即一个触发器可能既是 INSERT 触发器，又是 DELETE 或 UPDATE 触发器。当一个触发器可以为多个 DML 语句触发时，在这种触发器体内部可以使用三个谓词：INSERTING、DELETING 和 UPDATING 来确定当前执行的是何种操作。这三个谓词的含义如下表所示。

表 14.1.3 触发器谓词

谓 词	状 态
INSERTING	当触发语句为 INSERT 时为真，否则为假
DELETING	当触发语句为 DELETE 时为真，否则为假
UPDATING[(<列名>)]	未指定列名时，当触发语句为 UPDATE 时为真，否则为假；指定某一列名时，当触发语句为对该列的 UPDATE 时为真，否则为假

虽然在其他 DMSQL 程序语句块中也可以使用这三个谓词，但这时它们的值都为假。

下例中的触发器 LogChanges 使用这三个谓词来记录表 OTHER.READER 发生的所有变

化。除了记录这些信息外，它还记录对表进行变更的用户名。该触发器的记录存放在表 OTHER.READERAUDIT 中。

触发器 LogChanges 的创建语句如下：

```
SET SCHEMA OTHER;



CREATE OR REPLACE TRIGGER LogChanges

AFTER INSERT OR DELETE OR UPDATE ON OTHER.READER

FOR EACH ROW

DECLARE

v_ChangeType CHAR(1);

BEGIN

//'I'表示 INSERT 操作，'D'表示 DELETE 操作，'U'表示 UPDATE 操作

IF INSERTING THEN

v_ChangeType := 'I';

ELSIF UPDATING THEN

v_ChangeType := 'U';

ELSE

v_ChangeType := 'D';

END IF;

//记录对 Reader 做的所有修改到表 ReaderAudit 中，包括修改人和修改时间

INSERT INTO OTHER.READERAUDIT

VALUES

(v_ChangeType, USER, SYSDATE,

:old.reader_id, :old.name, :old.age, :old.gender, :old.major,
```

```
:new.reader_id, :new.name, :new.age, :new.gender, :new.major);
END;

SET SCHEMA SYSDBA;
```

14.1.2 事件触发器

14.1.2.1 事件触发器语法

用户可使用触发器定义语句(CREATE TRIGGER)在数据库全局对象上创建触发器。下面是触发器定义语句的语法:

语法格式

```
CREATE [OR REPLACE] TRIGGER [<模式名>.]<触发器名> [WITH  ENCRYPTION]
<BEFORE| AFTER> <触发事件子句> ON <触发对象名> [EXECUTE AT <触发的 RAFT 组名><DMDSC 节点号>][WHEN <条件表达式>]<触发器体>
<触发事件子句>:=<DDL 事件子句>| <系统事件子句>
<DDL 事件子句>:=<DDL 事件>{OR <DDL 事件>}
<DDL 事件>:= DDL | <CREATE | ALTER | DROP | GRANT | REVOKE | TRUNCATE | COMMENT>
<系统事件子句>:=<系统事件>{OR <系统事件>}
<系统事件>:= LOGIN | LOGOUT | SERERR | <BACKUP DATABASE> | <RESTORE DATABASE> | AUDIT | NOAUDIT | TIMER | STARTUP | SHUTDOWN | CHECKPOINT
<触发对象名>:=
    [<模式名>.]SCHEMA |
    DATABASE
```

参数

1 .<模式名> 指明被创建的触发器的所在的模式名称或触发事件发生的对象所在的模式

名，缺省为当前模式；

2．<触发器名> 指明被创建的触发器的名称；

3．BEFORE 指明触发器在执行触发语句之前启动；

4. AFTER 指明触发器在执行触发语句之后启动；

5. <触发的 RAFT 组名> 专门用于 DMDPC，用于指定在 RAFT 组中的节点触发，不涉及的节点上不触发。其中，多副本的 MP RAFT 只在主库触发。缺省<触发的 RAFT 组名>时，由于 TIMER 触发器数量与复杂程度均不可控，默认选择 ID 最小的 SP 执行，以减小对 MP 自身事务管理任务的干扰。对确定在 MP 执行的触发器（如 DDL）忽略指定的 RAFT。BP 模式的节点上不触发任何触发器；

6．<DDL 触发事件子句> 指明使触发器启动的 DDL 事件，可以是 DDL 或 CREATE、ALTER、DROP、GRANT、REVOKE、TRUNCATE、COMMENT 等；

7．<系统事件子句> LOGIN/LOGON、LOGOUT/LOGOFF、SERERR、BACKUP DATABASE、RESTORE DATABASE、AUDIT、NOAUDIT、TIMER、STARTUP、SHUTDOWN、CHECKPOINT；

8．WITH ENCRYPTION 选项，指定是否对触发器定义进行加密；

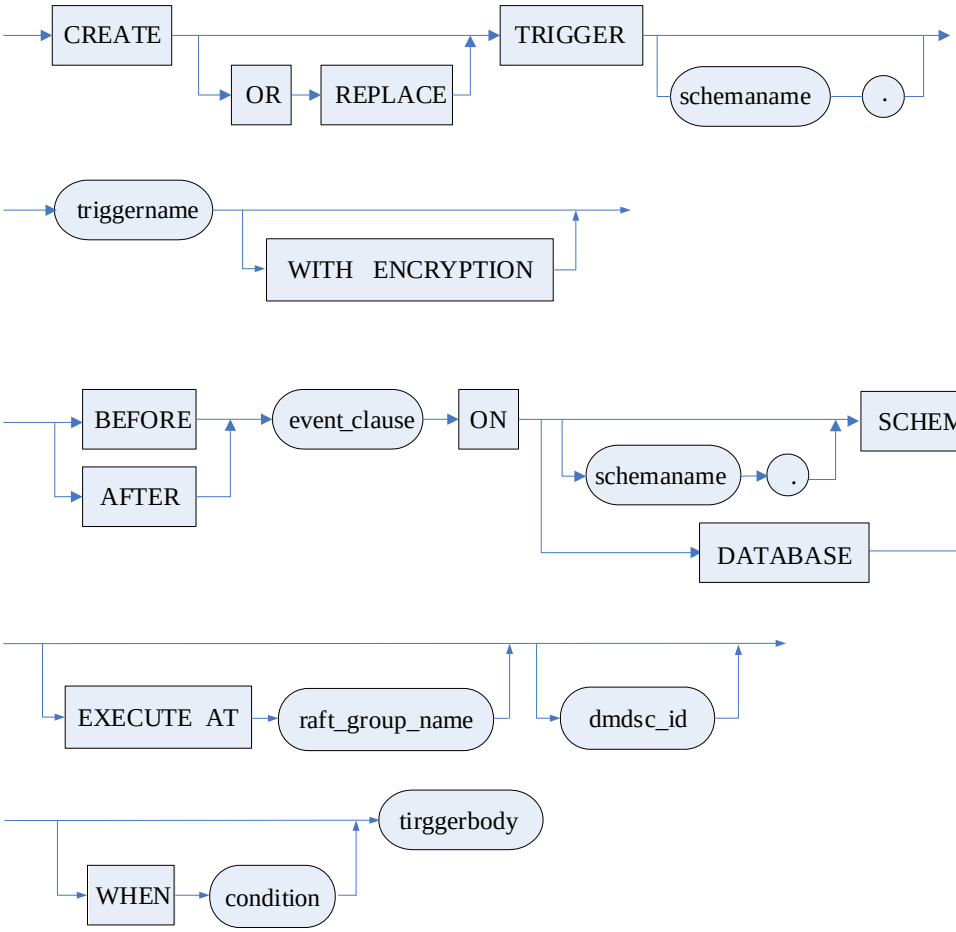
9．WHEN 子句 WHEN 子句包含一个布尔表达式，当表达式的值为 TRUE 时，执行触发器；否则，跳过该触发器；

10．<触发器体> 触发器被触发时执行的 SQL 过程语句块；

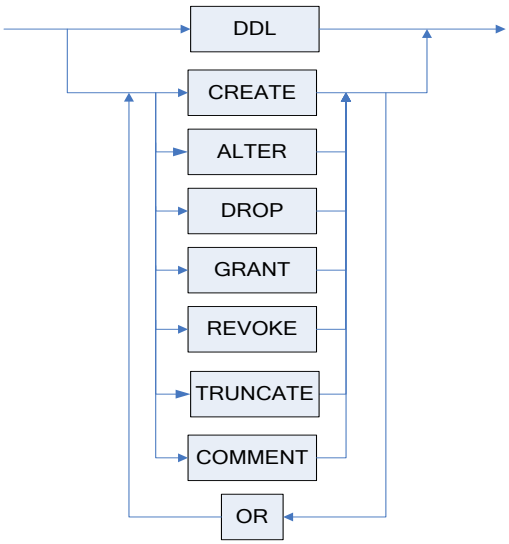
11.<DMDSC 节点号> 指定 DMDSC 环境下触发器执行的节点号，仅 CHECKPOINT 事件触发器支持指定<DMDSC 节点号>。

图例

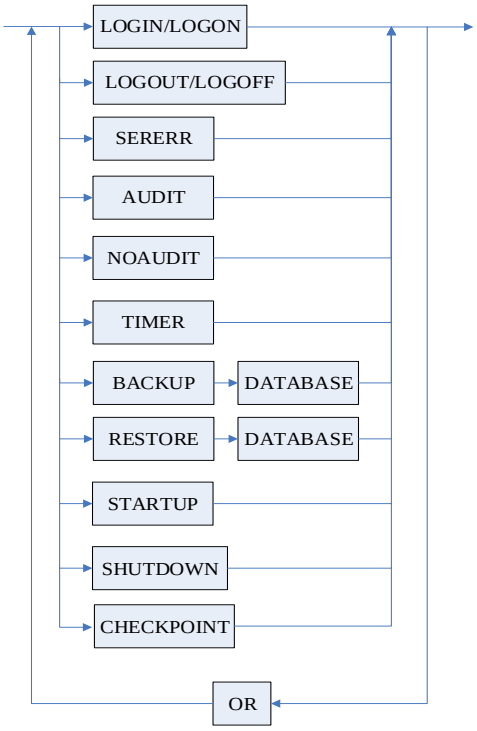
事件触发器


CREATE
OR
REPLACE
TRIGGER
schemaname
.
triggername
WITH ENCRYPTION
BEFORE
AFTER
event_clause
ON
schemaname
.
SCHEMA
DATABASE
EXECUTE AT
raft_group_name
dmdsc_id
tirggerbody
WHEN
condition

DDL 事件


DDL
CREATE
ALTER
DROP
GRANT
REVOKE
TRUNCATE
COMMENT
OR

系统事件



使用说明

1．<触发器名>是触发器的名称，它不能与模式内的其他模式级对象同名，并且不能使用"MTRG$_"/"TRIG_ARR$"/"BM$_TRG_"/"REF_CON_TRIG"/"SREF_CON_TRIG"/"TRIG_MDRT$TMP"/"TRIG_HNSW$TMP"/"TRIG_IVFFLAT$TMP"作为前缀；

2．可以使用 OR REPLACE 选项来替换一个触发器，但是要注意被替换的触发器的触发对象名不能改变。如果要在模式中不同的对象上重新创建一个同名的触发器，则必须先删除该触发器，然后再创建；

3．<触发事件子句>说明使触发器启动的事件，DDL 事件以及系统事件。DDL 事件包括数据库和模式上的 DDL 操作；系统事件包括数据库上的除 DDL 操作以外系统事件；以上事件可以有多个，用 OR 列出。触发事件按照兼容性可以分为以下几个集合：

{CREATE, ALTER, DROP, TRUNCATE,COMMENT }、{ GRANT, REVOKE }、{ LOGIN/LOGON, LOGOUT/LOGOFF }、{ SERERR }、{ BACKUP DATABASE, RESTORE DATABASE }、{AUDIT, NOAUDIT}、{TIMER}、{ STARTUP, SHUTDOWN }、{CHECKPOINT}。

只有同一个集合中，不同名的事件，才能在创建语句中并列出现。

DDL 事件中，DDL 关键字的作用相当于 CREATE OR DROP OR ALTER OR TRUNCATE OR COMMENT。

4 .<触发对象名>是触发事件发生的对象，DATABASE 和<模式名>只对 DDL 事件有效，<模式名>可以缺省;

5 .在一个数据库或模式上创建的事件触发器个数没有限制，可以有以下类型: CREATE、ALTER 、 DROP 、 GRANT 、 REVOKE 、 TRUNCATE 、 COMMENT 、 LOGIN/LOGON 、LOGOUT/LOGOFF、SERERR、BACKUP DATABASE、STARTUP、SHUTDOWN，且仅表示指该类操作，不涉及到具体数据库对象如 CREATE/ALTER/DROP TABLE，只要能引起任何数据字典表中的数据对象变化，都可以启动相应触发器，触发时间分为 BEFORE 和 AFTER;所有 DDL 事件触发器都可以设置 BEFORE 或 AFTER 的触发时机，但系统事件中 LOGOUT 和 SHUTDOWN 仅能设置为 BEFORE，而其它则只能设置为 AFTER。模式级触发器不能是 LOGIN/LOGON、LOGOUT/LOGOFF、SERERR、BACKUP DATABASE、RESTORE DATABASE、CHECKPOINT、STARTUP 和 SHUTDOWN 事件触发器。

6. 通过系统存储过程 SP_ENABLE_EVT_TRIGGER 和 SP_ENABLE_ALL_EVT_TRIGGER 可以禁用/启用指定的事件触发器或所有的事件触发器。

7 . 事件操作说明如下:

对于事件触发器，所有的事件信息都通过伪变量 :EVENTINFO 来取得。

下面对每种事件可以获得的信息进行详细说明:

1) CREATE: 添加新的数据库对象(包括用户、基表、视图等)到数据字典时触发;

```
对象类型描述: :eventinfo.objecttype
指明事件对象的类型，类型为 VARCHAR(128)，对于不同的类型其值如下:
用户:  'USER'
表:    'TABLE'
```

视图：'VIEW'

索引：'INDEX'

过程：'PROCEDURE'

函数：'FUNCTION'

角色：'ROLE'

模式：'SCHEMA'

序列：'SEQUENCE'

触发器：'TRIGGER'

同义词：'SYNONYM'

包：'PACKAGE'

类：'CLASS'

类型：'TYPE'

包体：'PACKAGEBODY'

类体：'CLASSBODY'

类型体：'TYPEBODY'

表空间：'TABLESPACE'

域：'DOMAIN'

目录：'DIRECTORY'

外部链接：'LINK'

对象名称：　　:eventinfo.objectname

　　指明事件对象的名称，类型为 VARCHAR(128)

所属模式：　　:eventinfo.schemaname

　　指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

所属数据库：　:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

操作类型： :eventinfo.optype

指明事件的操作类型，类型为 VARCHAR(20)，其值为“CREATE”

操作用户名： :eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(128)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

2) ALTER：只要ALTER修改了数据字典中的数据对象(包括用户、基表、视图等)，就激活触发器;

对象类型描述： :eventinfo.objecttype

指明事件对象的类型，类型为 CHAR(1)，对于不同的类型其值如下:

用户： 'USER'

表： 'TABLE'

视图： 'VIEW'

索引： 'INDEX'

过程： 'PROCEDURE'

函数： 'FUNCTION'

序列： 'SEQUENCE'

触发器：'TRIGGER'

表空间：'TABLESPACE'

对象名称： :eventinfo.objectname

指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

操作类型：:eventinfo.optype

指明事件的操作类型，类型为 VARCHAR(20)，其值为“ALTER”

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(128)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

3） DROP：从数据字典删除数据库对象(包括用户、登录、基表、视图等)时触发；

对象类型描述：:eventinfo.objecttype

指明事件对象的类型，类型为 VARCHAR(128)，对于不同的类型其值如下：

用户： 'USER'

表： 'TABLE'

视图： 'VIEW'

索引： 'INDEX'

过程： 'PROCEDURE'

函数： 'FUNCTION'

角色： 'ROLE'

模式： 'SCHEMA'

序列： 'SEQUENCE'

触发器：'TRIGGER'

同义词：'SYNONYM'

包：'PACKAGE'

类：'CLASS'

类型：'TYPE'

表空间：'TABLESPACE'

域：'DOMAIN'

目录：'DIRECTORY'

外部链接：'LINK'

对象名称： :eventinfo.objectname

指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

操作类型：:eventinfo.optype

指明事件的操作类型，类型为 VARCHAR(20)，其值为“DROP”

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(128)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

4） GRANT：执行GRANT命令时触发；

权限类型描述：:eventinfo.granttype，对于不同的类型其值如下：

对象权限：'OBJECT_PRIV'

系统权限：'SYSTEM_PRIV'

角色权限：'ROLE_PRIV'

指明授予权限的类型，类型为 varchar(256)

授予权限对象的用户名：:eventinfo.grantee

指明授予权限的对象用户，类型为 varchar(256)

对象名称：:eventinfo.objectname

对象权限有效，指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

对象权限有效，指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

对象名称：:eventinfo.objectname

指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

5） REVOKE：执行REVOKE命令时触发；

权限类型描述： :eventinfo.granttype，对于不同的类型其值如下：

对象权限：'OBJECT_PRIV'

系统权限：'SYSTEM_PRIV'

角色权限：'ROLE_PRIV'

指明回收权限的类型，类型为 varchar(256)

授予权限对象的用户名： :eventinfo.grantee

指明回收权限的对象用户，类型为 varchar(256)

对象名称：:eventinfo.objectname

对象权限有效，指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

对象权限有效，指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(256)

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

对象名称：:eventinfo.objectname

指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

6） TRUNCATE：执行TRUNCATE命令时触发；

对象名称： :eventinfo.objectname，对于不同的类型其值如下：

表： 'TABLE'

指明事件对象的名称，类型为 VARCHAR(256)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空

操作类型：:eventinfo.optype

指明事件的操作类型，类型为 VARCHAR(20)，其值为“TRUNCATE”

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

7） LOGIN/LOGON：登录时触发；

登录名： :eventinfo.loginname

指明登录时的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

8） LOGOUT/LOGOFF：退出时触发；

登录名： :eventinfo.loginname

指明退出时的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

9） BACKUP DATABASE:备份数据库时触发；

备份的数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空。

备份名： :eventinfo.backuname

指明的备份名，类型为 VARCHAR(256)

操作用户名： :eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

10）RESTORE DATABASE:还原数据库时触发；

还原的数据库： :eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空。

还原的备份名：:eventinfo.backuname

指明的备份名，类型为 VARCHAR(256)

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

11） SERERR：只要服务器记录了错误消息就触发；

错误号：:eventinfo. ERRCODE

指明错误的错误号，类型为 INT

错误信息：:eventinfo.errmsg

指明错误的错误信息，类型为 VARCHAR(256)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

12）COMMENT ON DATABASE/SCHEMA：执行COMMENT命令时触发；

操作类型： :eventinfo.objecttype

指明事件对象类型，类型为 VARCHAR(20)

对象名称：:eventinfo.objectname

指明事件对象的名称，类型为 VARCHAR(128)

所属模式：:eventinfo.schemaname

指明事件对象所属的模式名，类型为 VARCHAR(128)，针对不同类型的对象有可能为空

所属数据库：:eventinfo.databasename

指明事件对象所属的数据库名，类型为 VARCHAR(256)，针对不同类型的对象有可能为空

操作类型：:eventinfo.optype

指明事件的操作类型，类型为 VARCHAR(20)，其值为“COMMENT”

操作用户名：:eventinfo.opuser

指明事件操作者的用户名，类型为 VARCHAR(128)

事件发生时间：:eventinfo.optime

指明事件发生的时间，类型为 DATETIME

13）AUDIT：进行审计时触发（用于收集，处理审计信息）；

14）NOAUDIT：不审计时触发；

15）TIMER：定时触发。见下文时间触发器；

16）STARTUP：服务器启动后触发，只能 AFTER STARTUP。

SHUTDOWN：服务器关闭前触发，只能BEFORE SHUTDOWN。SHUTDOWN触发，不要执行花费时间多于5秒的操作。

8．<触发器体>是触发器的执行代码，是一个 DMSQL 程序语句块，语句块与存储模块定义语句中<模块体>的语法基本相同。有关详细语法，可参考《DM8_SQL 程序设计》的相关部分。<引用子句>用来引用正处于修改状态下表中行的数据。如果指定了<触发条件>子句，则首先对该条件表达式求值，<触发器体>只有在该条件为真值时才运行；

9．创建模式触发器时，触发对象名直接用 SCHEMA；

10．创建的触发器可以分为以下几类：

1） 在自己拥有的模式中创建自己模式的对象上的触发器或创建自己模式上的触发器；

2） 在任意模式中创建任意模式的对象上的触发器或创建其他用户模式上(. SCHEMA)的触发器，即支持跨模式的触发器，表现为<触发器名>和<触发对象名>的<模式名>不同；

3） 创建数据库上(DATABASE)的触发器。

11 .触发器的创建者必须拥有 CREATE TRIGGER 数据库权限并具有触发器定义中引用对象的权限;

12 . DDL 触发事件的用户必须拥有对模式或数据库上相应对象的 DDL 权限; 系统触发事件的用户必须有 DBA 权限;

13 . 如果触发器执行 SQL 语句或调用过程或函数，那么触发器的拥有者必须拥有执行这些操作所必需的权限。这些权限必须直接授予触发器拥有者，而不是通过角色授予，这与存储模块或函数的限制一致;

14 . 如果触发器同时也是触发事件对象，则该触发器不会被触发，例如: 当删除触发器本身被删除时不会触发 DROP 触发器。

权限

用户必须是基表的拥有者，或者具有 DBA 权限。

需要强调的是，由于触发器是使其启动执行的语句的一个组成部分，为保证语句的原子性，在<触发器体>以及<触发器体>调用的存储模块中不允许使用可能导致事务提交或回滚的 SQL 语句，如: COMMIT、ROLLBACK。

具体地说，在触发器中允许的 SQL 语句有: SELECT、INSERT、DELETE、UPDATE、DECLARE CURSOR、OPEN、FETCH、CLOSE 语句等。

每张基表上的可创建的触发器的个数没有限制，但是触发器的个数越多，处理 DML 语句所需的时间就越长，这是显而易见的。注意，不存在触发器的执行权限，因为用户不能主动"调用"某个触发器，是否启动一个触发器是由系统来决定的。

事件触发器（DDL 触发事件）使用示例如下:

```
CREATE TABLE T01_TRI_10000(OBJECTTYPE VARCHAR(500),OBJECTNAME VARCHAR(500),
SCHEMANAME VARCHAR(500),DATABASENAME VARCHAR(500),OPUSER VARCHAR(500), OPTIME
VARCHAR(500));
```

```
CREATE TABLE T02_TRI_10000 (C1 INT,C2 VARCHAR(10));
INSERT INTO T02_TRI_10000 VALUES (1,'ABCD');
CREATE TRIGGER TRI01_TRI_10000 BEFORE CREATE ON DATABASE BEGIN INSERT INTO
T01_TRI_10000
VALUES(:EVENTINFO.OBJECTTYPE,:EVENTINFO.OBJECTNAME,:EVENTINFO.SCHEMANAME,:EVENTINFO.
DATABASENAME,:EVENTINFO.OPUSER, :EVENTINFO.OPTIME); END;

CREATE USER L01_TRI_10000 IDENTIFIED BY DMsys_123;
CREATE TABLE T03_TRI_10000(C1 INT);
CREATE VIEW V01_TRI_10000 AS SELECT * FROM T01_TRI_10000;
CREATE INDEX I01_TRI_10000 ON T01_TRI_10000(OBJECTTYPE);
CREATE OR REPLACE PROCEDURE P01_TRI_10000 AS BEGIN SELECT * FROM T02_TRI_10000; END;

CREATE FUNCTION F01_TRI_10000 RETURN VARCHAR(30) AS A1 VARCHAR(30); BEGIN SELECT C2
INTO A1 FROM T02_TRI_10000 WHERE C1=1; PRINT A1; RETURN A1; END;
CREATE ROLE R01_TRI_10000;
CREATE SEQUENCE S01_TRI_10000 INCREMENT BY 10;
CREATE TRIGGER TRI02_TRI_10000 AFTER CREATE ON DATABASE BEGIN PRINT 'SUCCESS';END;

SELECT OBJECTTYPE, OBJECTNAME, SCHEMANAME, DATABASENAME, OPUSER FROM T01_TRI_10000;
```

查询结果如下：

```
行号     OBJECTTYPE    OBJECTNAME            SCHEMANAME DATABASENAME OPUSER
----     ------------ ----------------- ---------- ----------- --------
1        USER          L01_TRI_10000      NULL       DAMENG      SYSDBA
```

```
2       TABLE          T03_TRI_10000       SYSDBA       DAMENG       SYSDBA
3       VIEW           V01_TRI_10000       SYSDBA       DAMENG       SYSDBA
4       INDEX          I01_TRI_10000       SYSDBA       DAMENG       SYSDBA
5       PROCEDURE      P01_TRI_10000       SYSDBA       DAMENG       SYSDBA
6       FUNCTION       F01_TRI_10000       SYSDBA       DAMENG       SYSDBA
7       ROLE           R01_TRI_10000       NULL         DAMENG       SYSDBA
8       SEQUENCE       S01_TRI_10000       SYSDBA       DAMENG       SYSDBA
9       TRIGGER        TRI02_TRI_10000     SYSDBA       DAMENG       SYSDBA
```

事件触发器（系统触发事件）使用示例如下：

```
create or replace trigger test_trigger after LOGIN on database begin print'SUCCESS'; end;
//只要一登录，服务器就会打印出 SUCCESS
```

14.1.2.2 事件属性函数用法

当事件触发器被触发时，可以通过这些事件属性函数获取当前事件的属性。

针对用户设置的数据库事件(DDL 语句执行)，获取事件触发时的相关属性。事件属性函数如下：

1、DM_DICT_OBJ_NAME，无参数，返回事件对象名；

2、DM_DICT_OBJ_TYPE，无参数，返回事件对象类型；

3、DM_DICT_OBJ_OWNER，无参数，返回事件对象所在模式；

4、DM_SQL_TXT，有 1 个输出参数，参数类型为 DM_NAME_LIST_T，返回值为 DDL 语句占用的嵌套表单元个数。DM_SQL_TXT 帮助用户获取事件被触发时正在执行的 DDL 语句，用于存储获取到的 DDL 语句。DM_NAME_LIST_T 为元素类型为 varchar(64)的嵌套表。因此如果 DDL 语句过长会导致分片存储，用户在获取 DDL 语句的时候，尤其要注意根据返回值来循环读取嵌套表以获取完整的语句。

使用说明

1、系统内部 DDL 将不触发事件触发器

2、MPP 从节点的 DDL 将不触发事件触发器

下面用一个具体的例子来说明事件属性函数如何使用。

```
CREATE TABLE T_EAF(
    N           INT,
    SQLTEXT VARCHAR,
    OBJECTNAME VARCHAR(128),
    OBJECTTYPE VARCHAR(128),
    OBJECTOWNER    VARCHAR(128)
);
CREATE OR REPLACE TRIGGER TRIG_EAF_01 BEFORE DDL ON DATABASE
   DECLARE
       N           NUMBER;
       STR_STMT VARCHAR;
       SQL_TEXT DM_NAME_LIST_T;
   BEGIN
       N := DM_SQL_TXT(SQL_TEXT);  //N 为占用嵌套表单元个数
         FOR I IN 1..N
       LOOP
          STR_STMT := STR_STMT || SQL_TEXT(I);  //STR_STMT 为获取的 DDL 语句
       END LOOP;
      INSERT  INTO  T_EAF  VALUES(N,STR_STMT,DM_DICT_OBJ_NAME,  DM_DICT_OBJ_TYPE,
DM_DICT_OBJ_OWNER);
END;
```

```
/
```

执行建模式建表语句。

```
create schema systest;

create table T_systest(c1 int);
```

然后，可以在 T_EAF 中查询到相关的建模式、建表语句。

```
SELECT * FROM T_EAF;
```

查询结果如下：

```
N      SQLTEXT                                    OBJECTNAME OBJECTTYPE OBJECTOWNER

--- ------------------------------ ---------- ---------- -----------

1      create schema systest;                  SYSTEST      SCHEMA         SYSTEST

1      create table T_systest(c1 int); T_SYSTEST    TABLE          SYSTEST
```

14.1.3 时间触发器

时间触发器属于一种特殊的事件触发器，它使得用户可以定义一些有规律性执行的、定点执行的任务，比如在晚上服务器负荷轻的时候通过时间触发器做一些更新统计信息的操作、自动备份操作等等，因此时间触发器是非常有用的。

语法格式

```
CREATE [OR REPLACE] TRIGGER [<模式名>.]<触发器名>[WITH   ENCRYPTION]

AFTER TIMER ON DATABASE[EXECUTE AT <触发的 RAFT 组名>]

<{FOR ONCE AT DATETIME [<时间表达式>] <exec_ep_seqno>} | {{<month_rate> |

<week_rate> | <day_rate>} {<once_in_day> |

<times_in_day>}{<during_date>}<exec_ep_seqno>}|

{ [<时间表达式>]{<during_date>} <exec_ep_seqno>}>

[WHEN <条件表达式>]
```

```
<触发器体>
<month_rate>:= {FOR EACH <整型变量> MONTH {<day_in_month>}}| {FOR EACH <整型变量>
MONTH {< day_in_month_week>}}
<day_in_month>:= DAY <整型变量>
<day_in_month_week>:= {DAY <整型变量> OF WEEK<整型变量>} | {DAY <整型变量> OF WEEK
LAST}
<week_rate>: =FOR EACH <整型变量> WEEK {<day_of_week_list>}
<day_of_week_list >:= {<整型变量>} | {, <整型变量>}
<day_rate>: =FOR EACH <整型变量> DAY
<once_in_day >:= AT TIME <时间表达式>
<times_in_day >:={ <duaring_time> } FOR EACH <整型变量> <freq_sub_type>
<freq_sub_type>:= MINUTE | SECOND
<duaring_time>: ={NULL}|{FROM TIME <时间表达式>}|{FROM TIME <时间表达式> TO TIME <时间表
达式>}
<duaring_date>:={NULL}|{FROM DATETIME <日期时间表达式>}|{FROM DATETIME <日期时间表达
式> TO DATETIME <日期时间表达式>}
<exec_ep_seqno>:=EXECUTE AT <DMDSC 节点号>
```

参数

1 .<模式名> 指明被创建的触发器的所在的模式名称或触发事件发生的对象所在的模式名，缺省为当前模式；

2 . <触发器名> 指明被创建的触发器的名称；

3 . <触发的 RAFT 组名> 专门用于 DMDPC，用于指定在 RAFT 组中的节点触发，不涉及的节点上不触发。和事件触发器的<触发的 RAFT 组名>用法相同，详细介绍请参考事件触发器<触发的 RAFT 组名>；

4．WHEN 子句 包含一个布尔表达式，当表达式的值为 TRUE 时，执行触发器；否则，跳过该触发器；

5．<触发器体> 触发器被触发时执行的 SQL 过程语句块；

6．<exec_ep_seqno> 指定 DMDSC 环境下触发器执行所在的节点号；

7．<freq_sub_type> 指定触发器按分钟间隔或者秒间隔触发。其中，按秒间隔触发需设置 INI 参数 TIMER_TRIG_CHECK_INTERVAL 为 1。

使用说明

1．<触发器名>是触发器的名称，它不能与模式内的其他模式级对象同名，并且不能使用

"MTRG$_"/"TRIG_ARR$"/"BM$_TRG_"/"REF_CON_TRIG"/"SREF_CON_TRIG"/"TRIG_MDRT$TMP"/"TRIG_HNSW$TMP"/"TRIG_IVFFLAT$TMP"作为前缀；

2．时间触发器的最低时间频率精确到秒级，定义很灵活，完全可以实现数据库中的代理功能，只要通过定义一个相应的时间触发器即可。在触发器体中定义要做的工作，可以定义操作的包括执行一段 SQL 语句、执行数据库备份、执行重组 B 树、执行更新统计信息、执行数据迁移（DTS）。

下面的简单例子在屏幕上每隔一分钟输出一行“HELLO WORLD”。

```
CREATE OR REPLACE TRIGGER timer2
AFTER TIMER on database
for each 1 day for each 1 minute
BEGIN
    print 'HELLO WORLD';
END;
/
```

关闭时间触发器和普通触发器是一样的，这里不再叙述。

14.2 触发器替换

在定义触发器的语法中，“OR REPLACE”选项用于替换一个已存在的同名触发器。当触发器替换是以下情况之一时，DM 会报错“替换触发器属性不一致”。

1. 表触发器和事件触发器之间的替换；
2. 表触发器所基于的表或视图发生变化时；
3. 事件触发器的触发对象名（SCHEMA 或 DATABASE）发生变化时；
4. 事件触发器的可触发的模式发生变化时；
5. 事件触发器对应的事件类型发生变化时，事件类型分为以下几类：

   DDL：CREATE、ALTER、DROP、GRANT、REVOKE、TRUNCATE 等

   AUDIT：AUDIT、NOAUDIT

   PRIV：GRANT、REVOKE

   LOGIN： LOGIN/LOGON、LOGOUT/LOGOFF

   SERVER：SERERR

   BACK：BACKUP DATABASE、RESTORE DATABASE

   TIMER：TIMER

   STARTUP：STARTUP、SHUTDOWN

14.3 设计触发器的原则

在应用中使用触发器功能时，应遵循以下设计原则，以确保程序的正确和高效：

1．如果希望保证一个操作能引起一系列相关动作的执行，请使用触发器；

2．不要用触发器来重复实现 DM 中已有的功能。例如，如果用约束机制能完成希望的完整性检查，就不要使用触发器；

3．避免递归触发。所谓递归触发，就是触发器体内的语句又会使该触发器启动，导致

语句的执行无法终止。例如，在表 T1 上创建 BEFORE UPDATE 触发器，而该触发器中又有对表 T1 的 UPDATE 语句；

4．合理地控制触发器的大小和数目。要知道，一旦触发器被创建，任何用户在任何时间执行的相应操作都会导致触发器的执行，这将是一笔不小的开销。

14.4 触发器的删除

当用户需要从数据库中删除一个触发器时，可以使用触发器删除语句。其语法如下:

语法格式

```
DROP TRIGGER [IF EXISTS] [<模式名>.]<触发器名>;
```

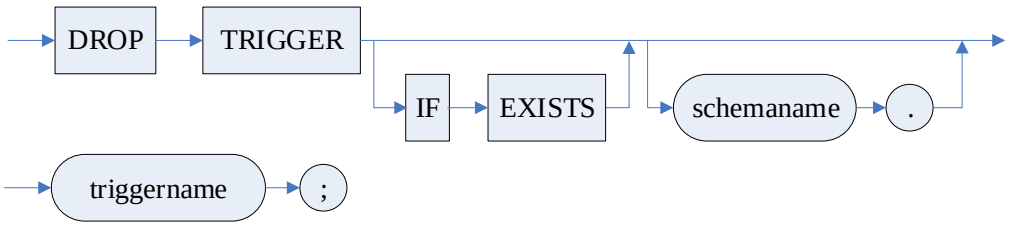
参数

1．<模式名> 指明被删除触发器所属的模式；

2．<触发器名> 指明被删除的触发器的名字。

图例

触发器的删除



使用说明

1．删除不存在的触发器会报错。若指定 IF EXISTS 关键字，删除不存在的触发器，不会报错；

2．当触发器的触发表被删除时，表上的触发器将被自动地删除；

3. 除了 DBA 用户外，其他用户必须是该触发器所属基表的拥有者才能删除触发器。

权限

执行该操作的用户必须是具有 DBA 角色（三权分立）的用户，或该触发器的拥有者（且具有 DROP TRIGGER 权限）。

举例说明

例 1 删除触发器 TRG1。

```
DROP TRIGGER TRG1;
```

例 2 删除模式 SYSDBA 下的触发器 TRG2。

```
DROP TRIGGER SYSDBA.TRG2;
```

14.5 禁止和允许触发器

每个触发器创建成功后都自动处于允许状态(ENABLE)，只要基表被修改，触发器就会被触发。但是在某些情况下，例如：

1．触发器体内引用的某个对象暂时不可用；

2．载入大量数据时，希望屏蔽触发器以提高执行速度；

3．重新载入数据。

用户可能希望触发器暂时不被触发，但是又不想删除这个触发器。这时，可将其设置为禁止状态(DISABLE)。

当触发器处于允许状态时，只要执行相应的 DML 语句，且触发条件计算为真，触发器体的代码就会被执行；当触发器处于禁止状态时，则在任何情况下触发器都不会被触发。根据不同的应用需要，用户可以使用触发器修改语句将触发器的状态设置为允许或禁止状态。其语法如下：

语法格式

```
ALTER TRIGGER [<模式名>.]<触发器名> <DISABLE | ENABLE>;
```

参数

1．<模式名> 指明被修改的触发器所属的模式；

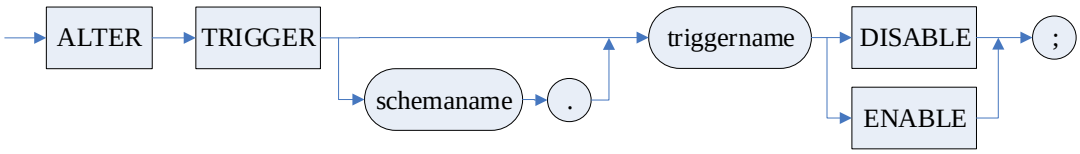
2．<触发器名> 指明被修改的触发器的名字；

3．DISABLE 指明将触发器设置为禁止状态。当触发器处于禁止状态时，在任何情况下触发器都不会被触发；

4．ENABLE 指明将触发器设置为允许状态。当触发器处于允许状态时，只要执行相应的 DML 语句，且触发条件计算为真，触发器就会被触发。

图例

禁止和允许触发器



权限

执行该操作的用户必须是具有 DBA 角色（三权分立）的用户，或该触发器的拥有者（且具有 ALTER TRIGGER 权限），或具有 ALTER ANY TRIGGER 权限的用户。

14.6 触发器的重编

重新对触发器进行编译，如果重新编译失败，则将触发器置为禁止状态。

重编功能主要用于检验触发器的正确性。

语法格式

```
ALTER TRIGGER [<模式名>.]<触发器名> COMPILE [DEBUG];
```

参数

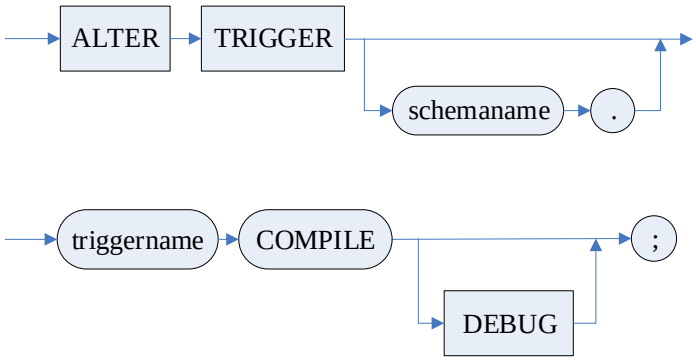
1．<模式名> 指明被重编译的触发器所属的模式；

2．<触发器名> 指明被重编译的触发器的名字；

3．[DEBUG] 可忽略。

图例

触发器重编



权限

执行该操作的用户必须是具有 DBA 角色（三权分立）的用户，或该触发器的拥有者（且具有 ALTER TRIGGER 权限），或具有 ALTER ANY TRIGGER 权限的用户。

举例说明

例 重新编译触发器

```
ALTER TRIGGER OTHER.TRG_AI_ACCOUNT COMPILE;
```

14.7 触发器应用举例

正如我们在本章所介绍的，触发器是 DM 系统提供的重要机制。我们可以使用该机制来加强比正常的审计机制、完整性约束机制、安全机制等所能提供的功能更复杂的事务规则。为帮助用户更好地使用该机制，我们提供了一些触发器应用的例子供用户参考。

14.7.1 使用触发器实现审计功能

尽管 DM 系统本身已经提供了审计机制，但是在许多情况下我们还是可以利用触发器完成条件更加复杂的审计。与内置的审计机制相比，采用触发器实现的审计有如下优点：

1．使用触发器可针对更复杂的条件进行审计；

2．使用触发器不仅可以记录操作语句本身的信息，还可以记录被该语句修改的数据的具体值；

3．内置的审计机制将所有审计信息集中存放，而触发器实现的审计可针对不同的操作

对象分别存放审计信息，便于分析。

虽然如此，触发器并不能取代内置的审计机制。因为内置审计机制的某些功能触发器是无法做到的。例如：

1. 内置审计机制可审计的类型更多。触发器只能审计表上的 DML 操作，而内置审计机制可以针对各种操作、对象和用户进行审计；
2. 触发器只能审计成功的操作，而内置审计机制能审计失败的操作；
3. 内置审计机制使用起来更简单，并且其正确性更有保障。

用于审计的触发器通常都是 AFTER 类型。关于审计的实例，请参考《DM8 安全管理》。

14.7.2 使用触发器维护数据完整性

触发器与完整性约束机制都可以用于维护数据的完整性，但是二者之间存在着显著的区别。一般情况下，如果使用完整性约束机制可以完成约束检查，我们不建议用户使用触发器。这是因为：

1．完整性约束机制能保证表上所有数据符合约束，即使是约束创建前存在的数据也必须如此；而触发器只保证其创建后的数据满足约束，但之前存在数据的完整性则得不到保证；

2．完整性约束机制使用起来更简单，并且其正确性更有保障。

触发器通常用来实现完整性约束机制无法完成的约束检查和维护，例如：

1．引用完整性维护

删除被引用表中的数据时，级联删除引用表中引用该数据的记录；更新被引用表中的数据时，更新引用表中引用该数据的记录的相应字段。下例中，表 OTHER.DEPTTAB 为被引用表，其主关键字为 Deptno；表 OTHER.EMPTAB 为引用表。其结构如下：

```
SET SCHEMA OTHER;
```

```
CREATE OR REPLACE TRIGGER Dept_del_upd_cascade
AFTER DELETE OR UPDATE ON OTHER.DEPTTAB FOR EACH ROW
BEGIN
    IF DELETING THEN
        DELETE FROM OTHER.EMPTAB
            WHERE Deptno = :old.Deptno;
    ELSE
        UPDATE OTHER.EMPTAB SET Deptno = :new.Deptno
            WHERE Deptno = :old.Deptno;
    END IF;
END;


SET SCHEMA SYSDBA;
```

2．CHECK 规则检查

增加新员工或者调整员工工资时，保证其工资不超过规定的范围，并且涨幅不超过 25%。该例中，表 OTHER.EMPTAB 记录员工信息；表 OTHER.SALGRADE 记录各个工种的工资范围，其结构如下：

```
SET SCHEMA OTHER;


CREATE OR REPLACE TRIGGER Salary_check
BEFORE INSERT OR UPDATE ON OTHER.EMPTAB
FOR EACH ROW
DECLARE
```

```
    Minsal    FLOAT;
    Maxsal    FLOAT;
    Salary_out_of_range EXCEPTION FOR -20002;
BEGIN
//取该员工所属工种的工资范围
SELECT Losal, Hisal INTO Minsal, Maxsal FROM OTHER.SALGRADE
WHERE Job_classification = :new.Job;
//如果工资超出工资范围，报告异常
IF (:new.Sal < Minsal OR :new.Sal > Maxsal) THEN
        RAISE Salary_out_of_range;
END IF;
//如果工资涨幅超出 25%，报告异常
IF UPDATING AND (:new.Sal - :old.Sal) / :old.Sal > 0.25 THEN
        RAISE Salary_out_of_range;
END IF;
END;


SET SCHEMA SYSDBA;
```

14.7.3 使用触发器保障数据安全性

在复杂的条件下，可以使用触发器来保障数据的安全性。同样，要注意不要用触发器来实现 DM 安全机制已提供的功能。使用触发器进行安全控制时，应使用语句级 BEFORE 类型的触发器，其优点如下：

1．在执行触发事件之前进行安全检查，可以避免系统在触发语句不能通过安全检查的

情况下做不必要的工作；

2．使用语句级触发器，安全检查只需要对每个触发语句进行一次，而不必对语句影响的每一行都执行一次。

下面这个例子显示如何用触发器禁止在非工作时间内修改表 OTHER.EMPTAB 中的工资(Sal)栏。非工作时间包括周末、公司规定的节假日以及下班后的时间。为此，我们用表 OTHER.C OMPANYHOLIDAYS 来记录公司规定的节假日。其结构如下：

```
SET SCHEMA OTHER;



CREATE OR REPLACE TRIGGER Emp_permit_changes
BEFORE INSERT OR DELETE OR UPDATE
ON OTHER.EMPTAB
DECLARE
    Dummy        INTEGER;
    Invalid_Operate_time    EXCEPTION FOR -20002;
BEGIN
    //检查是否周末
    IF (DAYNAME(Sysdate) = 'Saturday' OR
        DAYNAME(Sysdate) = 'Sunday') THEN
        RAISE Invalid_Operate_time;
    END IF;
    //检查是否节假日
    SELECT COUNT(*) INTO Dummy FROM OTHER.COMPANYHOLIDAYS
        WHERE Holiday= Current_date;
    IF dummy > 0 THEN
```

```
        RAISE Invalid_Operate_time;
    END IF;
    //检查是否上班时间
    IF (EXTRACT(HOUR FROM Current_time) < 8 OR
        EXTRACT(HOUR FROM Current_time) >= 18) THEN
        RAISE Invalid_Operate_time;
    END IF;
END;

SET SCHEMA SYSDBA;
```

14.7.4 使用触发器生成字段默认值

触发器还经常用来自动生成某些字段的值，这些字段的值有时依赖于本记录中的其他字段的值，有时是为了避免用户直接对这些字段进行修改。这类触发器应该是元组级 BEFORE INSERT 或 UPDATE 触发器。因为：

1．必须在 INSERT 或 UPDATE 操作执行之前生成字段的值；

2．必须为每条元组自动生成一次字段的值。

```
SET SCHEMA OTHER;

CREATE OR REPLACE TRIGGER Emp_auto_value
BEFORE INSERT
ON OTHER.EMPTAB
FOR EACH ROW
BEGIN
```

```
  :new.Sal = 999.99;

END;



SET SCHEMA SYSDBA;
```

第 15 章 同义词

同义词(Synonym)让用户能够为数据库的一个模式下的对象提供别名。同义词通过掩盖一个对象真实的名字和拥有者，并且对远程分布式的数据库对象给予了位置透明特性以此来提供了一定的安全性。同时使用同义词可以简化复杂的 SQL 语句。同义词可以替换模式下的表、视图、序列、函数、存储过程等对象。

15.1 创建同义词

语法格式

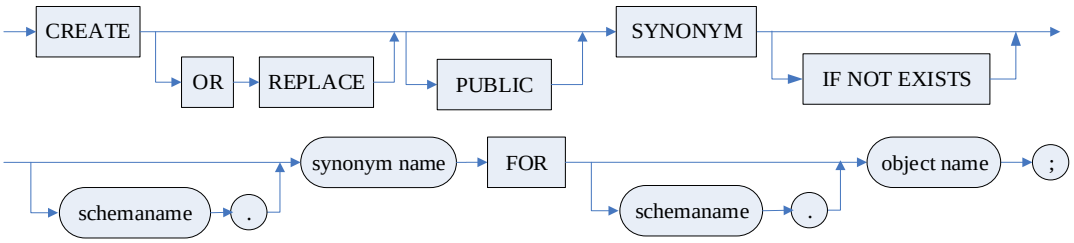
```
CREATE [OR REPLACE] [PUBLIC] SYNONYM [IF NOT EXISTS] [<模式名>.]<同义词名>  FOR [<模式名>.]<对象名>
```

参数

1. [PUBLIC]指定同义词为全局同义词。
2. <同义词名> 指待创建同义词的名字。不能和系统对象同名。
3. <对象名> 指示同义词替换的对象。该对象可以为远程服务器的对象，远程服务器的对象可以通过创建外部链接来获取，详细介绍请参考第 16 章 外部链接。

图例

创建同义词



语句功能

创建一个同义词。

权限要求

用户在自己的模式下创建同义词，必须有 CREATE SYNONYM 权限。用户要创建全局同义词(PUBLIC SYNONYM)，必须有 CREATE PUBLIC SYNONYM 权限。

使用说明

1．同义词分为全局同义词(PUBLIC SYNONYM，又名公有同义词)和非全局同义词（又名私有同义词)。

2．全局同义词创建时不能指定同义词的模式名限定词，它能够被所有用户使用，使用时不需要加任何模式限定名。非全局同义词被非同义词所属模式拥有者引用需要在前面加上模式名；全局同义词和非全局同义词，可以具有相同的名字；非全局同义词不能与目标对象同名。

3．用户使用 SQL 语句对某个模式下的同义词进行操作，那么解析这个对象的顺序，首先是查看模式内是否存在该同义词对象（若未指定模式名，则指定当前默认模式)，然后再查看是否存在同名全局同义词。例如，用户 OE 和 SH 在他们的模式下都有一个表叫 customer，SYSDBA 为 OE 模式下的 customer 表创建了一个全局同义词 customer_syn，SYSDBA 为 SH 模式下的 customer 表创建了一个非全局同义词 customer_syn，如果用户 SH 查询：SELECT COUNT(*) FROM customer_syn，则此时返回的结果全局同义词指向对象 OE.CUSTOMER 表的行数。而对于同义词指向的目标对象，解析逻辑为优先查找指定模式下的目标同义词对象，若查找失败则查找同模式下的同名对象，最后查找同名全局同义词对象。特别的是，若目标对象未显式指定模式名，则在创建该同义词的模式下进行对象查找。

4. 若模式对象的非全局同义词与系统对象同名，在此模式下不加模式名前缀调用时，系统函数的优先级比非全局同义词高，系统包和系统过程的优先级比非全局同义词低。

5．如果创建时候没有指定 REPLACE 语法要素，则不允许创建同名同类型同义词(不同类型则可以，这里的不同类型指的是 PUBLIC 和非 PUBLIC)。

6．同义词创建时，并不会检查他所指代的同义词对象是否存在，用户使用该同义词时候，如果不存在指代对象或者对该指代对象不拥有权限，则会报错。

7. 当 INI 参数 ENABLE_PL_SYNONYM=0 时，禁止通过同义词执行非系统用户创建的包或者 DMSQL 程序。

举例说明

例 1 用户 B 对 A 模式下的表 T1 创建同义词。

在 A 模式下建立表 T1。

```
CREATE TABLE "A"."T1" ("ID" INTEGER, "NAME" VARCHAR(50), PRIMARY KEY("ID"));
INSERT INTO "A"."T1" ("ID", "NAME") VALUES (1, '张三');
INSERT INTO "A"."T1" ("ID", "NAME") VALUES (2, '李四');
```

对 A 模式下的表 T1 创建同义词。

```
CREATE SYNONYM A.S1 FOR A.T1;
```

如果用户 B 想查询 T1 表的行数，可以通过如下语句来获得结果：

```
SELECT COUNT(*) FROM A.S1;
```

例 2 当 INI 参数 ENABLE_PL_SYNONYM=0 时，禁止通过同义词执行非系统用户创建的包或者 DMSQL 程序。

```
1. dba 登录创建新用户并赋予创建同义词权限
//SYSDBA 登录
create user DBSEC   identified by DMsys_123456;
GRANT CREATE PUBLIC SYNONYM TO DBSEC;
grant resource to DBSEC;
2. 创建同义词。通过同义词调用 DMSQL 程序。
//DBSEC 登录
CREATE OR REPLACE FUNCTION GET_DBA2(a INT, b INT) RETURN INT AS
```

```
    s    INT;
    BEGIN
    s:=a+b;
    RETURN s;
    EXCEPTION
    WHEN OTHERS THEN NULL;
    END;
    /
CREATE OR REPLACE PUBLIC SYNONYM SP_CLOSE_SESSION FOR GET_DBA2; //创建同义词
call SP_CLOSE_SESSION (1,2);    //报错无法解析的成员访问表达式[SP_CLOSE_SESSION]
```

15.2 删除同义词

语法格式

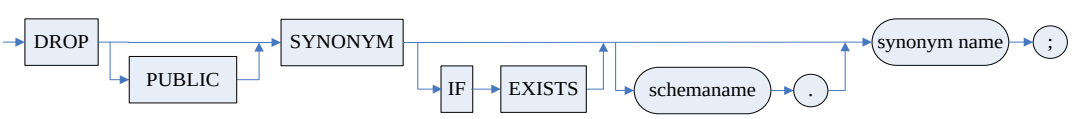
```
DROP [PUBLIC] SYNONYM [IF EXISTS] [<模式名>.]<同义词名>;
```

参数

1. [PUBLIC]在删除全局同义词时指定，在删除非全局同义词时不能指定。
2. <同义词名> 待删除同义词的名字。

图例

删除同义词



语句功能

删除一个同义词。

权限要求

用户删除自己模式下的非全局同义词，必须拥有 DROP SYNONYM 权限。用户要删除全局同义词(PUBLIC SYNONYM)，必须有 DROP PUBLIC SYNONYM 权限。

使用说明

1． 删除不存在的同义词会报错。若指定 IF EXISTS 关键字，删除不存在的同义词，不会报错。

2． 必须是 DBA 或者拥有此同义词的用户才能删除该同义词。

3． 如果要删除全局同义词，则必须要指定 PUBLIC，因为在达梦数据库中，全局同义词和非全局同义词可以同名，所以如果不指定，则删除的是当前模式下的同名同义词。

4． 删除全局同义词，需要指定 PUBLIC，而删除非全局同义词，则不能指定，否则报错；如果删除当前模式下的同义词，可以不指定模式名，如果删除其它模式下的同义词，需要指定相应的模式名，否则报错。

举例说明:

例 1 删除模式 A 下的同义词 S1。

```
DROP SYNONYM A.S1;
```

例 2 删除全局同义词 S2。

```
DROP PUBLIC SYNONYM S2;
```

第 16 章 外部链接

外部链接对象（LINK）是 DM 中的一种特殊的数据库实体对象，它记录了远程数据库的连接和路径信息，用于建立与远程数据的联系。通过多台数据库主库间的相互通讯，用户可以透明地操作远程数据库的数据，使应用程序看起来只有一个大型数据库。用户远程数据库中的数据请求，都被自动转换为网络请求，并在相应节点上实现相应的操作。用户可以建立一个数据库链接，以说明一个对象在远程数据库中的访问路径。这个链接可以是公用的（数据库中所有用户使用），也可以是私有的（只能被某个用户使用）。

用户可以通过外部链接对远程数据库的表进行查询和增删改操作，以及本地调用远程的存储过程。

16.1 创建外部链接

创建一个外部链接。

语法格式

```
CREATE [OR REPLACE] [PUBLIC] LINK [IF NOT EXISTS] <外部链接名> CONNECT ['<连接库类型>'] WITH
<登录名> IDENTIFIED BY <登录口令> USING  '<外部连接串>' [<OPTION 子句>];
<连接库类型>::=
        DPI |
        ODBC |
        DAMENG |
        ORACLE |
        GATEWAY
<外部链接串>::=
```

<DPI 外部链接串> |

<ODBC 外部链接串> |

<DAMENG 外部链接串> |

<ORACLE 外部链接串> |

<GATEWAY 外部链接串>

<DPI 外部链接串>::=

<IP 地址>:<端口号> |

<服务名>

<ODBC 外部链接串>::= <ODBC 数据源 DSN>

<GATEWAY 外部链接串>::= <ODBC 数据源 DSN>(<达梦 odbc 驱动>)

<DAMENG 外部链接串>::= [<连接类型>;]<服务器列表>

<连接类型>::=

PRIMARY FIRST |

STANDBY FIRST |

PRIMARY ONLY |

STANDBY ONLY

<服务器列表>::=

<服务器地址> |

<服务器地址>{,<服务器地址>}

<服务器地址>::=

<实例 IP 地址>/<实例端口号> |

<MAL IP 地址>/<MAL 端口号> |

<实例名>

<ORACLE 外部链接串>::=

```
        <tsn_name> |
        <description> |
        <IP 地址>/<服务名>
<description>::=(DESCRIPTION=(ADDRESS_LIST=(ADDRESS=(PROTOCOL=TCP)(HOST=<IP 地址>)(PORT=<端口号>)))(CONNECT_DATA=(SERVICE_NAME=<服务名>)))
<OPTION 子句>:: =(<option 项>{, <option 项>})
<option 项>:: =
        LOCAL_CODE=<选项值> |
        CONVERT_MODE=<选项值> |
        BYTES_IN_CHAR=<选项值> |
        DB_TYPE=<选项值> |
        DATA_CHARSET=<选项值> |
        CASE_OPT=<选项值>|
        ROLLBACK_OPTION=<选项值>
```

参数

1. PUBLIC 此链接对象是否能够被创建者之外的用户引用；
2. <外部链接名> 数据库链接的名称；
3. <连接库类型> 目前只支持 DPI、ODBC、GATEWAY、DAMENG 或 ORACLE，默认为 DPI。DPI 和 DAMENG 用于连接达梦数据库；ODBC 可以连接所有支持 ODBC 协议的数据库；ORACLE 用于连接 ORACLE 数据库；GATEWAY 可以使用达梦的 ODBC 驱动，需配置扩展选项 GWDSN，指定远程 GATEWAY 的 DSN，通过 GATEWAY 中转，连接到远程数据库；
4. <登录名> 登录用户名称；
5. <登录口令> 登录用户口令；

6. <DPI 外部链接串> 通过 DPI 接口访问远程达梦数据库，不需要进行额外配置；

7. <ODBC 外部链接串> ODBC 数据源 DSN 需要用户手动配置；

8. <DAMENG 外部链接串>通过 MAL 系统访问远程达梦数据库，需要配置 DMMAL.INI；

9. <连接类型>用来指定 DBLINK 将采用何种优先级别连接到<服务器列表>中的机器，共四种连接类型：PRIMARY FIRST 为主机优先连接；STANDBY FIRST 为备机优先连接；PRIMARY ONLY 为只连主机；STANDBY ONLY 为只连备机。缺省为 PRIMARY FIRST。<连接类型>为可选项，一旦指定了<连接类型>，则必须指定<服务器地址>数大于等于 2；

10. <服务器地址>支持三种格式，分别对应目标节点在 DMMAL.INI 中的配置项，具体如下：

- <实例 IP 地址>/<实例端口号> 对应 mal_inst_host/mal_inst_port。
- <MAL IP 地址>/<MAL 端口号> 对应 mal_host/mal_port。
- 实例名 对应 mal_inst_name 。

11. <ORACLE 外部链接串> 可以使用配置的网络服务名 tsn_name(网络服务名需要配置)，或者连接描述符 description（连接描述符是网络连接目标特殊格式的描述，它包括网络协议、主库 IP 地址、端口号和服务名），或者<IP 地址>/<服务名>；

12. <option 项> 目前所支持的选项与取值如下：

- LOCAL_CODE：接口（DPI、OCI、JDBC 等接口）的字符集。若与服务器的字符集不一致，则需要进行转换。可取值为 UTF-8，GBK 或 GB18030。
- CONVERT_MODE：表示字符转换过程中对不完整字符的处理规则：0 表示截断不完整字符，不报错；1 表示报错处理。
- BYTES_IN_CHAR：整型数字，表示一个字符由多少个字节组成。
- DB_TYPE：设置目标数据库类型，目前支持 SQLSERVER、MYSQL、PI、ORACLE、DAMENG、OCEANBASE_ORACLE，仅对 ODBC 类型的 DBLINK 有效。

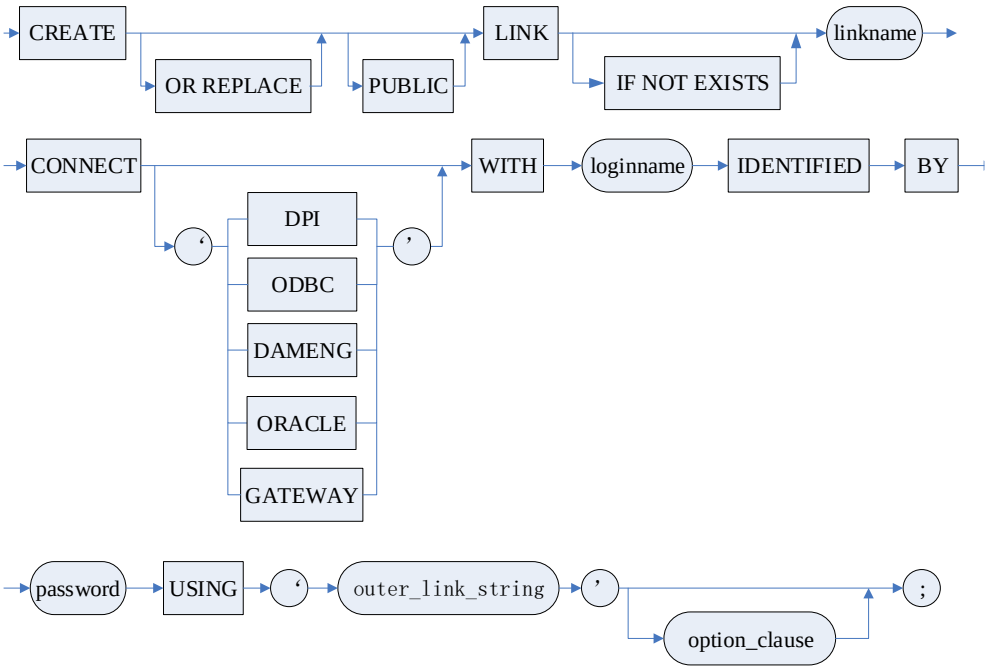
- DATA_CHARSET：设置接口返回给达梦服务器数据的实际字符集。取值为 UTF-8，GBK 或 GB18030。

- CASE_OPT：字符串类型，可取值'UPPER'、'LOWER'、'MIX'、'SENSITIVE'，用于指定本地字符串与远程字符串比较时采取的规则'；UPPER 将本地字符串转成大写再按本地大小写规则比较，LOWER 将本地字符串转成小写后再按本地大小写规则比较，MIX 不改变本地字符串大小写，但忽略大小写进行比较，SENSITIVE 是默认选项，不改变本地字符串大小写，且按本地大小写规则进行比较。本地大小写比较规则由数据库初始化参数 CASE_SENSITIVE 控制，建库后可以通过在视图 V$DM_INI 中查询 INI 参数 GLOBAL_STR_CASE_SENSITIVE 的值获取当前本地大小写比较规则情况，该 INI 参数的取值含义与 CASE_SENSITIVE 相同。

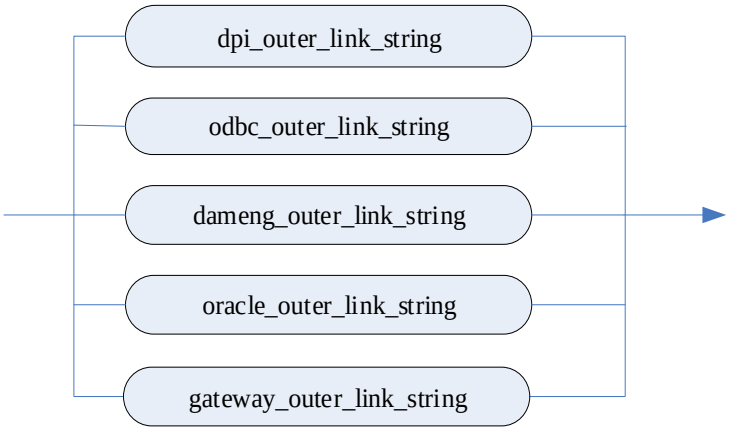
- ROLLBACK_OPTION：当语句执行失败进行回滚时选择的回滚级别，仅 oci 和 dpi 类型 dblink 有效。设置为 0 表示进行语句级回滚，即只回滚本次语句的操作；设置为 1 时表示进行事务级回滚，一旦事务中某个语句执行失败，整个事务操作都将被回滚。默认情况下进行语句级回滚。

图例

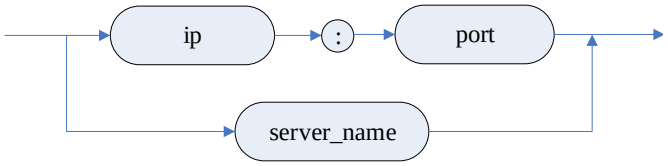
创建外部链接：



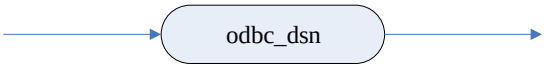
外部链接串（outer_link_string）：



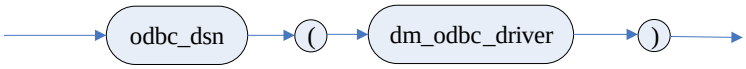
DPI 外部链接串（dpi_outer_link_string）：



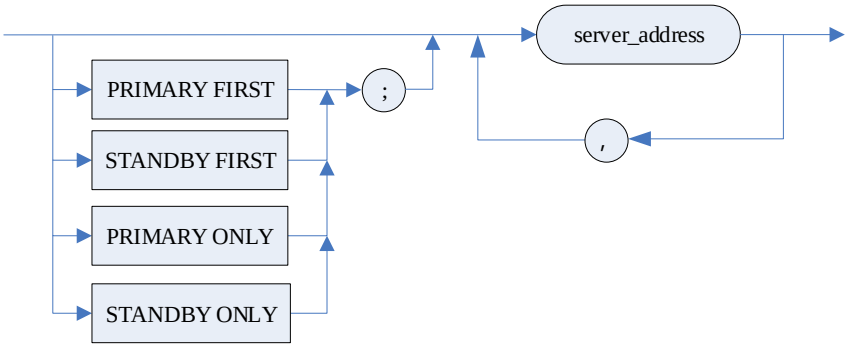
ODBC 外部链接串（odbc_outer_link_string）：



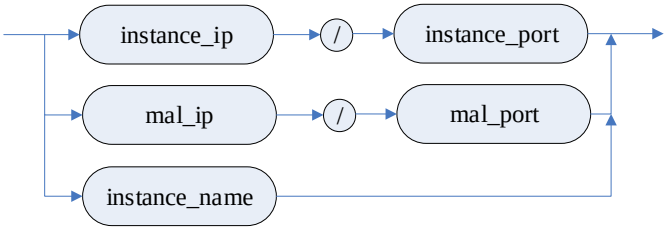
GATEWAY 外部链接串（odbc_outer_link_string）：



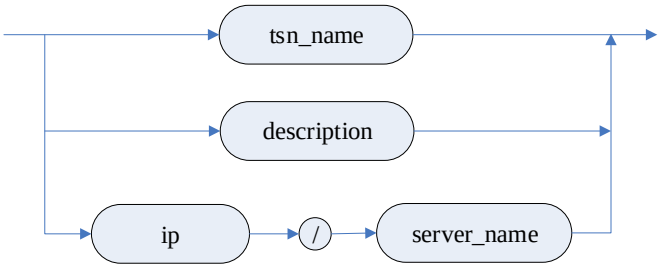
DAMENG 外部链接串（dameng_outer_link_string）:



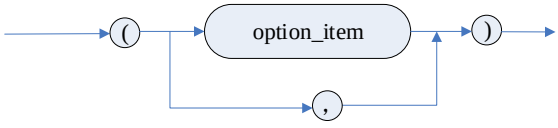
服务器地址（server_address）:



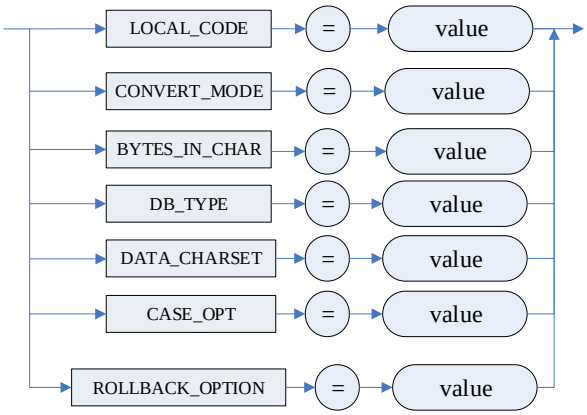
ORACLE 外部链接串（oracle_outer_link_string）:



OPTION 子句（option_clause）:



option 项（option_item）:



语句功能

创建一个外部链接。

使用说明

1．当系统中已经有同名的数据库链接名时，若指定 OR REPLACE，则服务器会自动用新的代码覆盖原来的代码；若指定 IF NOT EXISTS，则服务器会忽略本次外部链接创建操作；若同时指定 OR REPLACE 和 IF NOT EXISTS，则按照 OR REPLACE 的策略执行；若均不指定，则报错；

2. 对于 DM-DM 的同构外部链接，推荐使用 DPI 外部链接串；

3. DPI 外部链接串可以使用 DPI 接口能够识别的 IP 地址和端口号，也可以使用 dm_svc.conf 文件中配置的服务名。达梦服务器安装成功后自带 DPI 库，因此用户无需进行其他配置操作，即可使用 DPI 外部链接串链接到远程达梦服务器；

4. DAMENG 外部链接串必须首先配置 DMMAL.INI，才能使用 LINK。DM 的连接串有两种格式：

INSTANCE_NAME:直接使用远程库的实例名（该实例名必须配置到 DM.INI 中）；

<IP 地址>/<端口号>:其中端口号为 DM 外部链接服务器的 DMMAL.INI 配置中的 MAL_PORT 端口号。

DMMAL.INI 各配置项的值、前后顺序，必须保持完全一致。DMMAL.INI 的详细配置可参考《DM8 系统管理员手册》，需要注意同时将 DM.INI 中的 MAL_INI 参数置为 1 以开启 MAL

系统。

5. 当 DBLINK 链接的是多机系统且指定了多台服务器时，若所连接的服务器发生宕机等意外情况导致无法对外提供服务时，DBLINK 将根据设定的<连接类型>，在剩余的服务器中挑选合适的服务器进行连接。

6. ODBC 外部链接用到第三方库文件 libodbc.so（Linux 环境），请用户自行从 UNIXODBC 官网下载，放置到本地目录，通过“环境变量-系统变量”配置后方可使用；Windows 环境下，DBLINK 用到的第三方库文件 odbc32.dll 为操作系统自带，无需用户配置；

7. 要创建 ORACLE 的外部链接，首先需要在当前机器安装 ORACLE 的 OCI 接口，且需要保证 OCI 接口与 DM 版本在 32 位/64 位保持一致。其次本功能将用到第三方库文件 libclntsh.so（Linux 环境）或 oci.dll(Windows 环境)，请用户自行从 ORACLE 官网下载，放置到本地目录，通过“环境变量-系统变量”进行配置；

8. 通过 LINK 对远程服务器所作的修改，由用户在本地服务器通过 commit 或 rollback 进行提交或回滚；

9. 只支持普通用户，不支持 SSL 和 Kerberos 认证；

10. 不支持连接自身实例的 LINK；

11. 支持在 CREATE SCHEMA 中 CREATE LINK，但是不支持 CREATE PUBLIC LINK；

12. 只有 DBA 和具有 CREATE LINK 权限的用户可以创建外部链接。

举例说明

例 1 使用 DPI 外部链接串连接 DM 数据库，IP 地址为 192.168.0.31，端口号为 5369，登录到此站点使用的用户名为 USER01，密码为 DMsys_123，dm_svc.conf 文件中配置的服务名为 DMSERVER01。

```
CREATE PUBLIC LINK LINK1 CONNECT 'DPI' WITH USER01 IDENTIFIED BY DMsys_123 USING
'192.168.0.31:5369';
```

或

```
CREATE PUBLIC LINK LINK1 CONNECT WITH USER01 IDENTIFIED BY DMsys_123 using 'DMSERVER01';
//DMSERVER01 为服务名
```

例 2 使用 DM 数据库，创建一个连接到 IP 地址为 192.168.0.225 机器上的 oracle 数据库的外部链接。可以通过三种方式创建：一网络服务名 tsn_name；二连接描述符 description；三 <IP 地址>/<服务名>。

（一） 通过网络服务名创建

首先介绍 Oracle 网络服务名的配置方法。网络服务名配置成功才能创建 DBLINK

配置本地的 Oracle 网络服务名 ORCL，连接 225 机器上的 orcl 实例。详细步骤如下：



图 16.1.1 步骤一



图 16.1.2 步骤二



图 16.1.3 步骤三



图 16.1.4 步骤四

其次，创建 dblink。

网络服务名配置成功后，就可以使用网络服务名 ORCL 或网络连接描述符创建 DBLINK。

```
CREATE LINK LINK1 CONNECT 'ORACLE' WITH USER01 IDENTIFIED BY Oracle_123 USING 'ORCL';
```

（二） 通过连接描述符创建

```
CREATE OR REPLACE LINK LINK1 CONNECT 'ORACLE' WITH USER01 IDENTIFIED BY Oracle_123 USING
'(DESCRIPTION =
    (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = 192.168.0.225)(PORT =
1521)))
    (CONNECT_DATA = (SERVICE_NAME = orcl) )
)';
```

（三） 通过<IP 地址>/<服务名>创建

```
CREATE LINK LINK1 CONNECT 'ORACLE' WITH USER01 IDENTIFIED BY Oracle_123 USING '192.168.0.225/orcl';
```

例 3 使用 DM 数据库，创建一个连接到实例名为 ORCL 的外部链接，登录到此站点使用的用户名为 USER01，设置登录口令为 Oracle_123，设置使用本地字符集为 UTF-8，字符集转换模式为 1。

```
CREATE LINK LINK1 CONNECT 'ORACLE' WITH USER01 IDENTIFIED BY Oracle_123 USING 'ORCL' OPTION (LOCAL_CODE='UTF-8', CONVERT_MODE=1);
```

16.2 删除外部链接

删除一个外部链接。

语法格式

```
DROP [PUBLIC] LINK [IF EXISTS] [<模式名>.]<外部链接名>;
```

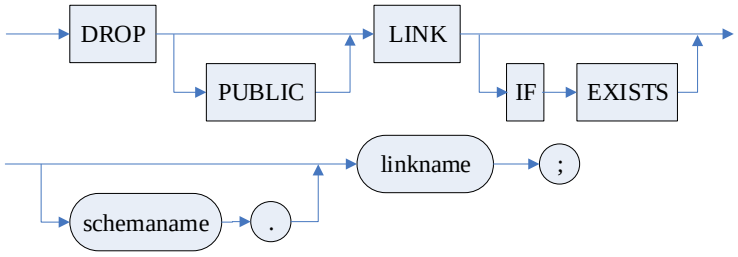
参数

1．<模式名> 指明被操作的外部链接属于哪个模式，缺省为当前模式；

2．<外部链接名> 指明被操作的外部链接的名称。

图例

删除外部链接



语句功能

删除一个外部链接。

使用说明

1. 删除不存在的外部链接会报错。若指定 IF EXISTS 关键字，删除不存在的外部链接，不会报错；

2. 只有拥有 DROP DBLINK 和 DROP ANY DBLINK 权限的用户可以删除自己模式下的非 PUBLIC DBLINK 对象。具有 DROP ANY LINK 权限的用户才可删除任何模式下的 PUBLIC DBLINK 对象。

举例说明

删除外部链接 LINK1。

```
DROP LINK LINK1;
```

16.3 使用外部链接

通过外部链接，可以对远程服务器的对象进行查询或进行增删改操作，可以调用远程的过程或函数。

使用外部链接进行查询或增删改的语法格式与普通格式基本一致，唯一的区别在于指定外部链接对象时需要使用如下格式：

```
<对象名> @ <外部链接名>
```

<对象名>可以为远程服务器的表、视图、序列、存储过程或函数的名称。

举例说明

使用外部链接查询 LINK1 上的远程表进行查询

```
SELECT * FROM SYSOBJECTS@LINK1;
```

或对远程表进行插入数据：

```
INSERT INTO T1@LINK1 VALUES(1,2,3);
```

也可以查询本地表或其他链接的表对远程表进行操作，如

```
UPDATE T1@LINK1 SET C1 = C1+1 WHERE C2 NOT IN (SELECT ID FROM LOCAL_TABLE);

DELETE FROM T1@LINK1 WHERE C1 IN (SELECT ID FROM T2@LINK2);
```

使用外部链接调用远程的存储过程或函数时，存在以下约束：

1. 参数数据类型为 SQL 类型，不允许为 DMSQL 程序类型；

2. 参数数据类型不允许为复合类型。

若远程的存储过程或函数存在参数列表，则写法如下：

```
[<模式名>.][<包名>.]<过程/函数名> @ <外部链接名>(<参数列表>);
```

使用限制

外部链接的使用有以下限制：

1. DM-DM 的同构外部链接不支持 MPP 环境，DM 与异构数据库的外部链接支持 MPP 环境；

2. 增删改不支持 INTO 语句；

3. 不支持使用游标进行增删改操作；

4. 不支持操作远程表的复合类型列；

5. DBLINK 理论上不支持 LOB 类型列的操作，但支持简单的增删改语句中使用常量来对 LOB 类型列进行操作；

6. DBLINK 的本地库和远程库的大小写敏感参数 CASE_SENSITIVE 应保持一致，若不一致，应用需要保证 SQL 语句书写符合远程库的规范；

7. DBLINK 的本地库和远程库的字符集编码应一致，否则可能导致字符串操作出错；

另外，DM 连接异构数据库的外部链接还有如下使用限制：

1. 数据类型以 DM 为基础，不支持 DM 没有的数据类型；

2. 语法以 DM 的语法为标准，不支持 DM 不兼容的语法；

3. 主键更新，如果是涉及到多个服务器的语句，不能保证更新操作一定成功；

4. 使用 CREATE VIEW view_name(view_col_name) AS SELECT ITEM FROM

T@LINK 方式创建的查询远程对象的本地视图，对于异构库，不能保证操作一定成功。对于查询异构库远程对象的本地视图，最好采用 CREATE VIEW AS SELECT ITEM AS alias_name FROM T@LINK 方式创建。

第 17 章 闪回

当系统 INI 参数 ENABLE_FLASHBACK 置为 1 时，闪回功能开启，可以使用闪回表或进行闪回查询。DM MPP 环境暂不支持闪回功能。

17.1 闪回表

闪回表是在数据库联机时，通过只回退对指定表及其相关对象所做的更改，将表里的数据回退到历史的某个时间点，而不需要执行传统的时间点恢复操作。比如回退到用户误删除数据之前的时间点，从而将误删除的数据恢复回来，在这个操作过程中，数据库仍然可用而且不需要额外的空间。

闪回表利用的是 UNDO 表空间里所记录的数据被改变前的值。因此，如果因保留时间超过了初始化参数 UNDO_RETENTION 所指定的值，从而导致闪回表时所需要的 UNDO 数据被其他事务覆盖的话，那么就不能将表中数据恢复到指定的时间了。

与介质恢复相比，闪回表在易用性、可用性和还原时间方面有明显的优势。

17.1.1 闪回表定义

语法格式:

```
FLASHBACK TABLE  [<模式名>.]<表名> {,[<模式名>.]<表名>} TO <SCN | LSN | TIMESTAMP> <expr>
[ <ENABLE|DISABLE> TRIGGERS ];
```

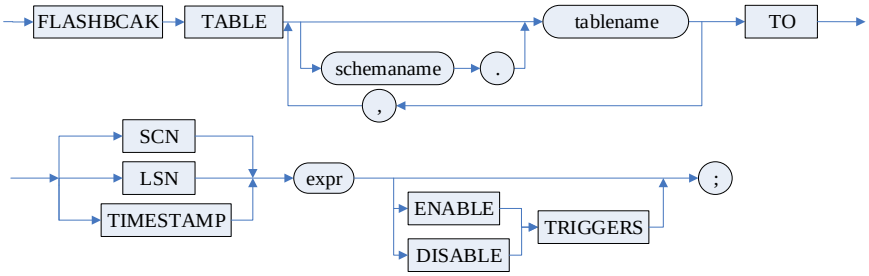
参数

1. <模式名> 指明该表属于哪个模式，缺省为当前模式；
2. <表名> 指明被创建的基表名，基表名最大长度 128 字节；
3. <expr> 指明闪回到的 LSN 值或 TIMESTAMP 值；

4. <ENABLE|DISABLE> TRIGGERS 指定是否开启触发器，ENABLE 为开启触发器，DISABLE 为关闭触发器，不指定则默认为关闭。

图例

闪回表语句



17.1.2 使用说明

1. 使用闪回功能需要打开 dm.ini 中的 ENABLE_FLASHBACK 参数；

2. 当前闪回表功能支持：批量闪回多个表、触发器的禁用与启用、DMDPC 环境（除使用 LOCAL 登录外）、DMDSC 环境；

3. 当前闪回表功能不支持在 DM MPP 环境下使用；

4. 闪回表利用的是 UNDO 表空间里记录的数据被改变前的值，只能闪回到 UNDO_RETENTION 指定值范围内的时间点；

5. 用户必须具有 FLASHBACK ANY TABLE 系统权限或 FLASHBACK 对象权限；

6. 闪回表语句是作为单个事务处理来执行。同时闪回多个表时，必须成功闪回所有表，否则会回退整个事务。闪回作为 DDL 语句，开启自动提交时，闪回成功后会自动提交；

7. 必须对要执行闪回操作的表启动行移动（分区表具有 MOVEMENT 功能的不能关闭，否则闪回可能会报错）；

8. 不会闪回受影响对象的统计信息；

9. 闪回会保留所有现有的索引；

10. 闪回中会正常检查约束条件，如果在闪回执行期间违反了任何约束条件，则会回滚

闪回操作；

11. 闪回不能跨越修改了表结构的 DDL。比如，在闪回数据之前，做过删除一个字段的操作，那么是无法闪回的；

12. 不能对系统表、临时表、HUGE 表、内部辅助表、动态表等执行闪回表操作；

13. DMDPC 环境下由于各节点 SCN/LSN 不同，只支持闪回到时间点 TIMESTAMP；

14. 限制重复闪回。闪回作为 DDL，对于同一个表，不允许再次闪回到上一次闪回之前的 LSN/TIMESTAMP。

17.1.3 使用示例

使用闪回功能需要先开启闪回参数 ENABLE_FLASHBACK。

```
ALTER SYSTEM SET 'ENABLE_FLASHBACK'=1;
```

例 1 闪回到指定 LSN，例子中的 LSN 需根据实际替换。

```
DROP TABLE T;

CREATE TABLE T(C1 INT);

INSERT INTO T SELECT LEVEL CONNECT BY LEVEL < 4;

COMMIT;

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             3
```

```
SELECT CUR_LSN FROM V$RLOG;
```

查询结果如下：

```
行号          CUR_LSN

---------- --------------------

1             366234
```

```
DELETE FROM T WHERE C1=3;

INSERT INTO T VALUES(4);

COMMIT;

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             4
```

```
FLASHBACK TABLE T TO LSN 366234; //366234 为前面查询出的 CUR_LSN 值

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2
```

```
3              3
```

例 2 闪回到指定 TIMESTAMP，例子中的 TIMESTAMP 需根据实际替换。

```
DROP TABLE T;

CREATE TABLE T(C1 INT);

INSERT INTO T SELECT LEVEL CONNECT BY LEVEL < 4;

COMMIT;

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             3



//由于 V$LSN_TIME 有 3S 延迟，对 TO TIMESTAMP 的闪回，这里延迟 3S 再进行操作

//查询当前时间（也可借助 V$LSN_TIME）

SELECT SYSDATE;
```

查询结果如下：

```
行号          SYSDATE

---------- -------------------

1             2023-04-11 15:17:28



DELETE FROM T WHERE C1=3;

INSERT INTO T VALUES(4);
```

```
COMMIT;

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             4



//执行闪回操作，'2023-04-11 15:17:28'为前面查询出的 SYSDATE，前面查询已考虑了 3 秒误差

FLASHBACK TABLE T TO TIMESTAMP '2023-04-11 15:17:28';

SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             3
```

例 3 同时闪回多个表，指定 LSN，例子中的 LSN 需根据实际替换。

```
DROP TABLE TT;

DROP TABLE T CASCADE;



CREATE TABLE T(C1 INT PRIMARY KEY);

CREATE TABLE TT(C2 INT, FOREIGN KEY (C2) REFERENCES T(C1));
```

```
INSERT INTO T SELECT LEVEL CONNECT BY LEVEL < 4;
INSERT INTO TT VALUES(2);
COMMIT;

SELECT * FROM T;
```

查询结果如下：

```
行号         C1
---------- -----------
1            1
2            2
3            3

SELECT * FROM TT;
```

查询结果如下：

```
行号         C2
---------- -----------
1            2

SELECT CUR_LSN FROM V$RLOG;
```

查询结果如下：

```
行号         CUR_LSN
---------- --------------------
1            367479
```

```
DELETE FROM TT;

DELETE FROM T WHERE C1=2;

COMMIT;



SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             3



SELECT * FROM TT;
```

查询结果为：未选定行

```
//367479 为前面查出的 CUR_LSN

FLASHBACK TABLE T, TT TO LSN 367479;



SELECT * FROM T;
```

查询结果如下：

```
行号          C1

---------- -----------

1             1

2             2

3             3
```

```
SELECT * FROM TT;
```

查询结果如下：

```
行号          C2

---------- -----------

1             2
```

例 4 带触发器的表闪回时禁用触发器的表现，例子中的 LSN 需根据实际替换。

```
DROP TABLE T;

CREATE TABLE T(A INT,B INT);

INSERT INTO T VALUES(10,10);

INSERT INTO T VALUES(11,11);

COMMIT;



//在表上创建触发器

CREATE OR REPLACE TRIGGER TR

BEFORE DELETE ON T

BEGIN

  INSERT INTO T VALUES(111,111);    //替换动作

END;

/



SELECT * FROM T;
```

查询结果如下：

```
行号          A               B
```

```
---------- ----------- -----------

1          10          10

2          11          11



SELECT CUR_LSN FROM V$RLOG;
```

查询结果如下：

```
行号       CUR_LSN

---------- --------------------

1          368114



INSERT INTO T VALUES(1,4);

COMMIT;

FLASHBACK TABLE T TO LSN 368114 DISABLE TRIGGERS;

SELECT * FROM T;
```

查询结果如下：

```
行号       A           B

---------- ----------- -----------

1          10          10

2          11          11
```

17.2 闪回查询

DM MPP 环境不支持闪回查询。数据守护环境下，备库不支持闪回查询。

DMDPC 环境下仅支持基于时间进行闪回查询，不支持基于 LSN 进行闪回查询。

17.2.1 闪回查询子句

闪回查询子句的语法，是在数据查询语句（参考第 4 章 数据查询语句）的基础上，为 FROM 子句增加了闪回查询子句。

语法格式

```
<闪回查询子句>::=
    WHEN <TIMESTAMP time_exp> |
    AS OF <TIMESTAMP time_exp> |
    AS OF <SCN|LSN lsn>
```

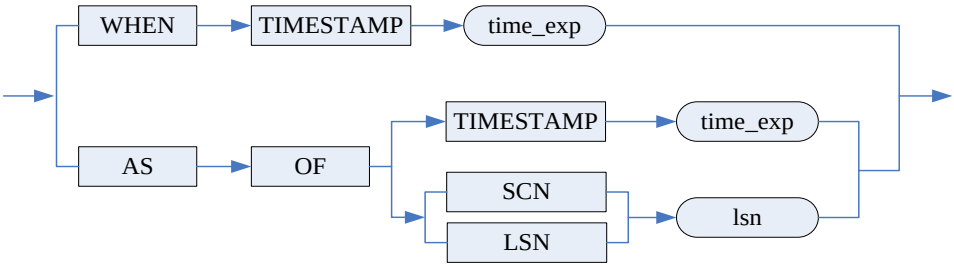
参数

1．time_exp 一个日期表达式，一般用字符串方式表示

2．lsn 指定 LSN 值

图例

闪回查询子句



语句功能

用户通过闪回查询子句，可以得到指定表过去某时刻的结果集。指定条件可以为时刻或 LSN。

使用说明

1． 闪回查询只支持普通表（包括加密表与压缩表）、水平分区表和堆表，不支持临时表、列存储表、外部表与视图；

2．闪回查询中 lsn 的值，可以通过查询动态视图 V$RLOG 或 V$LSN_TIME 来确定，也可以通过闪回版本查询（见下节）的伪列来确定；

3．由于视图 V$LSN_TIME 每三秒收集一次 LSN/TIME 映射关系，因此基于时间进行闪回查询时可能存在三秒的误差，如果需要进行精确度更高的闪回查询，建议基于 LSN 进行闪回查询。

4．当闪回表作为 DDL 时，对于同一个表，不允许基于对上一次闪回表的 LSN 或 TIMESTAMP 进行闪回查询。

举例说明

例 1 闪回查询特定时刻的 PERSON_TYPE 表。

查询 PERSON_TYPE 表。

```
SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1                    采购经理

2                     采购代表

3                     销售经理

4                     销售代表
```

在 2012-01-01 12:22:49 时刻插入数据，并提交。

```
INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('防损员');

INSERT INTO PERSON.PERSON_TYPE(NAME) VALUES('保洁员');

COMMIT;

SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1                    采购经理

2                     采购代表

3                     销售经理

4                     销售代表

5                     防损员

6                     保洁员
```

使用闪回查询取得 2012-01-01 12:22:45 时刻的数据。此时刻在插入数据的操作之前，可见此时的结果集不应该有 2012-01-01 12:22:49 时刻插入的数据。

```
SELECT * FROM PERSON.PERSON_TYPE WHEN TIMESTAMP '2012-01-01 12:22:45';
```

或

```
SELECT * FROM PERSON.PERSON_TYPE AS OF TIMESTAMP '2012-01-01 12:22:45';
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1                    采购经理

2                     采购代表

3                     销售经理

4                     销售代表
```

在 2012-01-01 12:23:29 时刻删除数据，并提交。

```
DELETE FROM PERSON.PERSON_TYPE WHERE PERSON_TYPEID > 5;

COMMIT;

SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下:

```
PERSON_TYPEID NAME

------------- --------

1                    采购经理

2                     采购代表

3                     销售经理

4                     销售代表

5                     防损员
```

使用闪回查询得到删除前的数据。

```
SELECT * FROM PERSON.PERSON_TYPE WHEN TIMESTAMP '2012-01-01 12:23:00';
```

或

```
SELECT * FROM PERSON.PERSON_TYPE AS OF TIMESTAMP '2012-01-01 12:23:00';
```

查询结果如下:

```
PERSON_TYPEID NAME

------------- --------

1                    采购经理

2                     采购代表

3                     销售经理

4                     销售代表

5                     防损员

6                     保洁员
```

例 2 闪回查询指定 LSN 的 PERSON_TYPE 表。

查询 PERSON_TYPE 表。

```
SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1             采购经理

2             采购代表

3             销售经理

4             销售代表
```

通过查询 V$RLOG 视图的 CUR_LSN 字段来获取当前 LSN 值。

```
SELECT CUR_LSN FROM V$RLOG;
```

查询结果如下：

```
行号          CUR_LSN

------- --------------------

1             85233
```

删除 PERSON_TYPE 表中的一行数据，并提交。

```
DELETE FROM PERSON.PERSON_TYPE WHERE PERSON_TYPEID=4;

COMMIT;

SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1             采购经理

2             采购代表

3             销售经理
```

通过删除操作之前的 LSN 值进行闪回查询得到删除前的数据。

```
SELECT * FROM PERSON.PERSON_TYPE AS OF SCN 85233;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------- --------

1                采购经理

2                 采购代表

3                 销售经理

4                 销售代表
```

17.2.2 闪回版本查询

INI 参数 UNDO_RETENTION 设置了事务提交后回滚页保持时间，缺省为 90 秒。因此，超出 UNDO_RETENTION 时间之外的过期闪回版本，无法被闪回查询到。

语法格式

```
<闪回版本查询子句>::=VERSIONS BETWEEN <闪回版本查询条件>

<闪回版本查询条件>::=

    TIMESTAMP <time_exp1> AND <time_exp2> |

    SCN|LSN <lsn1> AND <lsn2>
```

参数

1．time_exp 日期表达式，一般用字符串方式表示。<time_exp1>表示起始时间，<time_exp2>表示结束时间

2. lsn 指定 LSN 值。<lsn1>表示起始 LSN，<lsn2>表示结束 LSN

使用说明

1． 闪回版本查询支持普通表（包括加密表与压缩表）、水平分区表和堆表，不支持临时表、列存储表、外部表与视图；

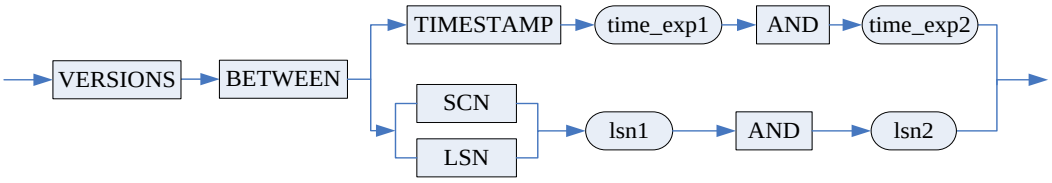
2．支持将表 17.2.1 中的伪列作为闪回版本查询的查询项，辅助获取精准的闪回查询信息。

表 17.2.1 闪回版本查询支持的伪列

伪列	说明
VERSIONS_STARTTIME、VERSIONS_STARTSCN、VERSIONS_STARTTRXID	起始时间戳、起始 LSN、起始 TRXID
VERSIONS_ENDTIME、VERSIONS_ENDSCN、VERSIONS_ENDTRXID	提交时间戳、提交 LSN、提交 TRXID。如果该值为 NULL，表示行版本仍然是当前版本
VERSIONS_OPERATION	在行上的操作。I 表示 Insert、D 表示 Delete、U 表示 Update

图例

闪回版本查询



语句功能

用户通过闪回版本查询子句，可以得到指定表过去某个时间段内，事务导致记录变化的全部记录。指定条件可以为时刻或 LSN。

举例说明

例 1 闪回版本查询指定时间段内，PERSON_TYPE 表的记录变化。

查询 PERSON_TYPE 表。

```
SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME
```

```
------------ --------

1            采购经理

2            采购代表

3            销售经理

4            销售代表

5            防损员
```

在 2021-12-28 10:05:05 时刻修改数据，并提交。

```
UPDATE PERSON.PERSON_TYPE SET NAME='保安员' WHERE PERSON_TYPEID=5;

COMMIT;

UPDATE PERSON.PERSON_TYPE SET NAME='收银员' WHERE PERSON_TYPEID=5;

COMMIT;

SELECT * FROM PERSON.PERSON_TYPE;
```

查询结果如下：

```
PERSON_TYPEID NAME

------------ --------

1            采购经理

2            采购代表

3            销售经理

4            销售代表

5            收银员
```

例 2 进行闪回版本查询，获得指定时间段内变化的记录。

```
SELECT VERSIONS_STARTSCN, VERSIONS_ENDSCN, NAME FROM PERSON.PERSON_TYPE VERSIONS

BETWEEN TIMESTAMP '2021-12-28 10:05:05' AND SYSDATE;
```

查询结果如下：

```
VERSION_STARTSCN    PERSON_TYPEID     NAME
---------------- --------------   --------
41061                    NULL                    采购经理
41061                    NULL                    采购代表
41061                    NULL                    销售经理
41061                    NULL                    销售代表
41074                     NULL                    收银员
41071                     41074                   保安员
41061                     41071                   防损员
```

例 3 可以利用伪列 VERSION_STARTSCN 进行闪回查询得到修改前的数据。

```
SELECT * FROM PERSON.PERSON_TYPE AS OF SCN 41071;
```

查询结果如下:

```
PERSON_TYPEID NAME
------------- --------
1                    采购经理
2                     采购代表
3                     销售经理
4                     销售代表
5                     保安员
```

17.2.3 闪回事务查询

闪回事务查询提供系统视图 V$FLASHBACK_TRX_INFO 供用户查看在事务级对数据库所做的更改。根据视图信息，可以确定如何还原指定事务或指定时间段内的修改。

使用说明

系统视图名为 V$FLASHBACK_TRX_INFO，定义如下表所示。

表 17.2.2 系统视图 V$FLASHBACK_TRX_INFO 定义

列名	数据类型	说明
START_TRXID	BIGINT	事务中第一个 DML 的 TRXID
START_TIMESTAMP	DATETIME(6)	事务中第一个 DML 的时间戳
COMMIT_TRXID	BIGINT	提交事务的 TRXID
COMMIT_TIMESTAMP	DATETIME(6)	提交事务时的时间戳
LOGON_USER	VARCHAR(256)	拥有事务的用户
UNDO_CHANGE#	INTEGER	记录修改顺序序号
OPERATION	CHAR(1)	DML 操作类型。 D: 删除; U: 修改; I: 插入; N: 更新插入 (专门针对 CLUSTER PRIMARY KEY 的插入); C：事务提交；P：预提交记录；O：default
TABLE_NAME	VARCHAR(256)	DML 修改的表
TABLE_OWNER	VARCHAR(256)	DML 修改表的拥有者
ROW_ID	BIGINT	DML 修改行的 ROWID
UNDO_SQL	VARCHAR(3900)	撤销 DML 操作的 SQL 语句
COMMIT_LSN	BIGINT	事务提交时的 LSN

举例说明

例 查询指定时间之后的事务信息，可为闪回查询操作提供参考。

```
SELECT * FROM V$FLASHBACK_TRX_INFO WHERE COMMIT_TIMESTAMP > '2012-01-01 12:00:00';
```

第 18 章 JSON

DM 支持 JSON（JavaScript Object Notation）和 JSONB（JavaScript Object Notation Binary）数据，JSON 和 JSONB 基本相似，二者的主要区别在于：其一，JSON 类型数据直接存储文本，最大长度为 2G-1 字节；JSONB 类型数据还需要存储 JSON 结构信息，以提高 JSONB 类型数据的处理速度，最大长度为 256M-1 字节。其二，JSON object 和 JSONB object 规则不同，具体规则由 DM.INI 参数 JSON_MODE 决定，详细介绍请参考 18.1.5 object。除 object 数据类型外，JSON 和 JSONB 其余数据类型规则均相同。

本章中的例子，除了特殊说明之外，建表语句都请参考 18.8 应用实例。

18.1 数据类型

JSON 支持的数据类型包括：字符串 string、数字 number、布尔值 true 和 false、null、对象 object 和数组 array。其中 string、number、true、false 和 null 为标量类型，object 和 array 为非标量类型。

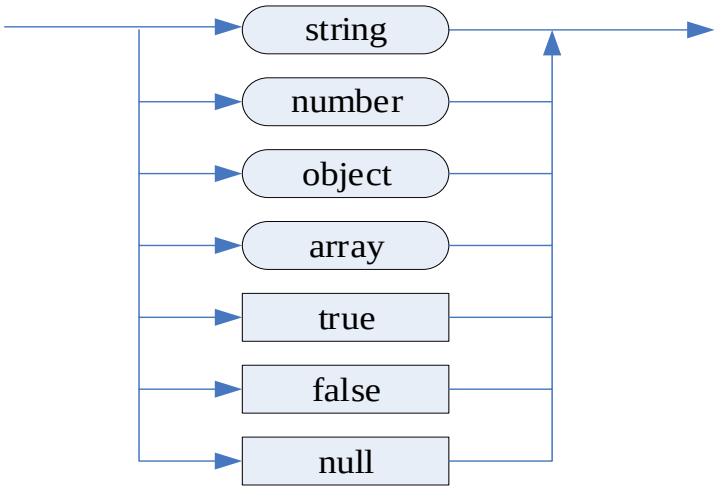
JSON 数据支持 LAX 和 STRICT 两种数据格式，STRICT 数据格式比 LAX 要求更严格，两种数据格式对不同数据类型的具体要求参见各数据类型的详细介绍。JSON_MODE 为 0 时采用 LAX 数据格式，JSON_MODE 为 1 或 2 时采用 STRICT 数据格式。由于 JSON_MODE 参数默认为 0，因此默认采用 LAX 数据格式。

DM 支持使用 VARCHAR 或 CLOB 作为载体处理 JSON 和 JSONB 数据，通过对字符串添加 CHECK 约束来保证数据符合 JSON 或 JSONB 标准，具体说明可以查看 18.5 CHECK 约束。

JSON 的各种数据类型可以嵌套使用。

图例

JSON 数据类型



18.1.1 string

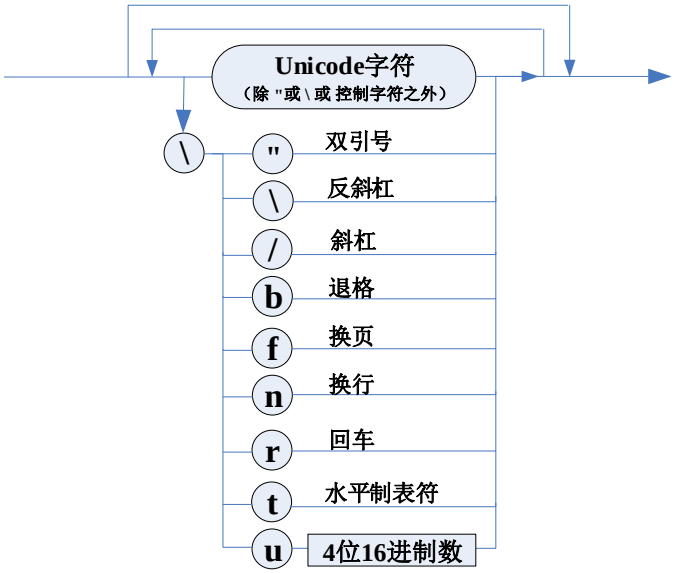
字符串(string)长度需要大于等于 0，且以 a-z 或 A-Z 开头，后续字符可以包含 0-9，如果不符合该书写规则或者包含其他字符，则必须使用引号括起来，DM 支持双引号和单引号。可以为空串。

STRICT 和 LAX 格式对字符串的不同要求如下：

- STRICT 格式下，字符串必须使用引号括起来。
- LAX 格式下，字符串作为 object 中的 key 并且符合上述书写规则时，可以不用引号，但作为 value 时必须使用引号。

图例

JSON string



图例中的第一个“\”表示转义字符。

举例说明

创建测试表。

```
drop table t_json_string CASCADE;

create table t_json_string (c1 json);
```

向表中插入数据。

```
insert into t_json_string values('{"a":"bc123"}');

insert into t_json_string values('{"":""}');
```

查询测试表。

```
select * from t_json_string;
```

查询结果如下：

```
行号        C1

---------- -------------

1              {"a":"bc123"}

2              {"":""}
```

18.1.2 number

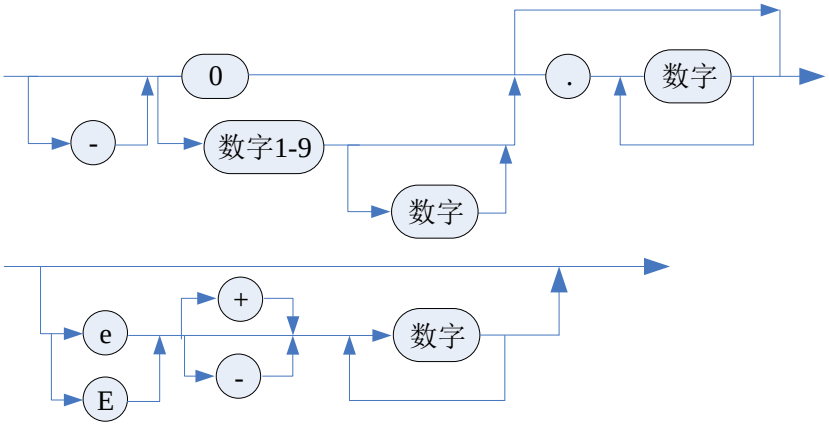
数字（number）不支持八进制和十六进制。支持数字作为 object 中的 value。

STRICT 和 LAX 格式对数字的不同要求如下：

- STRICT 格式下，正数不能以“+”或“.”开头，不能有前导 0（如 001），不能以“.”结尾。
- LAX 格式下没有上述限制。

图例

JSON number



举例说明

创建测试表。

```
drop table t_json_number CASCADE;

create table t_json_number (c1 json);
```

向表中插入数据。

```
insert into t_json_number values('{"a":123}');
```

查询测试表。

```
select * from t_json_number;
```

查询结果如下：

```
行号      C1

---------- ---------

1           {"a":123}
```

18.1.3 true、false

true 和 false 代表布尔值，使用时不需要加引号。一般作为 object 中的 value。

STRICT 和 LAX 格式对布尔值的不同要求如下：

- STRICT 格式下，true 和 false 必须是小写，否则报错；
- LAX 格式下，在 JSON 列中插入数据时，true 和 false 不区分大小写，但在查询语句中必须是小写形式才能有返回值。
- 仅 LAX 格式下支持 true 和 false 作为 key，该用法较为特殊，正常情况下不建议将 true 和 false 作为 key。

举例说明

创建测试表。

```
drop table t_json_boolean CASCADE;

create table t_json_boolean(c1 json);
```

例 1 向表中插入数据。

```
insert into t_json_boolean values('{"a":true}');

insert into t_json_boolean values('{"b":FALSE}');
```

查询测试表。

```
select * from t_json_boolean;
```

查询结果如下：

```
行号      C1

---------- -----------
```

```
1              {"a":true}

2              {"b":false}
```

例 2 LAX 格式下支持将 true 或 false 作为 key。

```
insert into t_json_boolean values('{TRUE:1}');
```

查询测试表。

```
select * from t_json_boolean;
```

查询结果如下:

```
行号       C1

---------- -----------

1              {"a":true}

2              {"b":false}

3              {"true":1}
```

18.1.4 null

null代表JSON数据为空，它与SQL语句中的NULL值不同。null使用时不需要加引号，一般作为object中的value。

STRICT 和 LAX 格式对 null 的不同要求如下:

- STRICT 格式下，null 必须是小写，否则报错;
- LAX 格式下， 在 JSON 列中插入数据时，null 不区分大小写，但在查询语句中必须是小写形式才能有返回值。
- 仅 LAX 格式下支持 null 作为 key，该用法较为特殊，正常情况下不建议将 null 作为 key。

json_value 时，null 以 SQL 语句中的值 NULL 的形式返回，此时无法区分是 SQL 的 NULL 还是 json 数据的 null；json_query 时，null 必须以指定 WITH WRAPPER 的形式返

回，如[null]，查询语句中必须以小写形式才能有返回值；

举例说明

创建测试表。

```
drop table t_json_null;

create table t_json_null(c1 json);
```

在表中插入数据。

```
insert into t_json_null values(null);              //SQL 语句的 null

insert into t_json_null values('{"a":null}');  //json 数据的 null
```

例 1 使用 json_value 函数查询数据。

```
select json_value(c1, '$.a') from t_json_null;
```

查询结果如下：

```
行号        JSON_VALUE(C1,'$.a')

---------- --------------------

1               NULL

2               NULL
```

可以看到两行数据并无不同。

例 2 使用 json_query 函数时，以指定 WITH WRAPPER 形式进行查询。

```
select json_query(c1, '$.a' WITH WRAPPER) from t_json_null;
```

查询结果如下：

```
行号        JSON_QUERY(C1,'$.a'WITHWRAPPER)

---------- --------------------------------

1               NULL

2               [null]
```

可以看出两行数据的不同，第 2 行为 json 数据。

18.1.5 object

对象（object）由 0 对或多对 key/value 组成，在{}中书写。key/value 的书写格式：<key>:<value>。

JSON object 和 JSONB object 规则由 DM.INI 参数 JSON_MODE 决定，JSON_MODE 缺省为 0：

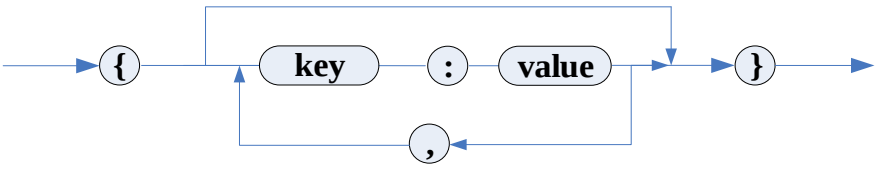
- JSON_MODE=0 时，JSON 和 JSONB 规则相同，二者均要求 object 中的 key 值不能重复，若重复则报错。当 VARCHAR 或 CLOB 表示的 JSON object 向 JSONB object 转换时，会检查 key 是否重复，若重复则报错。
- JSON_MODE=1 时，JSON 和 JSONB 规则不同，JSON 对 object 中的 key 值没有限制，也不进行额外处理。而 JSONB 会自动对 object 中的 key 值进行去重排序，去重时仅保留输入的最后一对 key/value。当 VARCHAR 或 CLOB 表示的 JSON object 向 JSONB object 转换时，会自动对 key 进行去重与排序，去重时仅保留输入的最后一对 key/value。
- JSON_MODE=2 时，JSON 和 JSONB 规则相同，二者均会自动对 object 中的 key 值进行去重排序，去重时仅保留输入的最后一对 key/value。当 VARCHAR 或 CLOB 表示的 JSON object 向 JSONB object 转换时，会自动对 key 进行去重与排序，去重时仅保留输入的最后一对 key/value。

STRICT 和 LAX 格式对 object 的不同要求如下：

- STRICT 格式下，object 的 key/value 链表后不能追加多余的逗号“,”。
- LAX 格式下没有上述限制。

语法格式

JSON object



<key>：对象名称。支持的数据类型包括 string、true、false、null。

<value>：对象值。支持本章节介绍的所有数据类型。

举例说明

例 1 JSON_MODE=0，创建测试表。

```
drop table t_json_object_0 CASCADE;

create table t_json_object_0 (c1 json);
```

向测试表中插入 object 对象。

```
insert into t_json_object_0 values( '{"b":1, "a":2}');
```

查询测试表。

```
SELECT * FROM t_json_object_0 ;
```

结果如下。

```
行号        C1

---------- -------------

1              {"b":1,"a":2}
```

如果插入的数据中 key 值重复，则报错

```
insert into t_json_object_0 values( '{"b":1, "b":2}');

[-3123]:JSON 对象中的键名重复.
```

例 2 JSON_MODE=1，创建测试表。

```
drop table t_json_object_1 CASCADE;

create table t_json_object_1 (c1 json, c2 jsonb);
```

向测试表中插入 object 对象。

```
insert into t_json_object_1 values( '{"b":1, "b":2, "a":2}', '{"b":1, "b":2, "a":2}');
```

查询测试表。

```
SELECT * FROM t_json_object_1 ;
```

结果如下。

```
行号       C1                       C2

---------- ------------------ -------------

1              {"b":1,"b":2,"a":2} {"a":2,"b":2}
```

可以看到 JSON_MODE=1 时，JSON 列对插入的 object 不进行额外处理，而 JSONB 列对 object 中的 key 进行了去重排序。

18.1.6 array

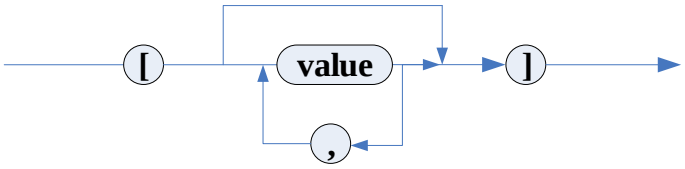
数组（array）是 value 的有序集合，数组在[]中书写。

STRICT 和 LAX 格式对 array 的不同要求如下：

- STRICT 格式下，array 的 value 链表后不能追加多余的逗号“,”。
- LAX 格式下没有上述限制。

语法格式

JSON array



<value>：支持本章节介绍的所有数据类型。

举例说明

创建测试表。

```
drop table t_json_array CASCADE;
```

```
create table t_json_array (c1 json);
```

数组中的值支持本章节介绍的所有数据类型。

```
insert into t_json_array values('{a:["a",1,true,false,null,{b:2},[3,4]]}');
```

查询测试表。

```
SELECT * FROM t_json_array;
```

查询结果如下:

```
行号        C1

---------- ----------------------------------------

1               {"a":["a",1,true,false,null,{"b":2},[3,4]]}
```

18.2 函数

为方便用户查阅，将 JSON 相关函数依据函数名称分为 JSON 函数、JSONB 函数以及其他函数三个章节。

部分 JSON 函数会自动对 object 对象 key/value 中的 key 进行去重排序，去重时保留输入的第一对或者最后一对 key/value，具体请参考各 JSON 函数的详细介绍。

所有 JSONB 函数均自动对 object 对象 key/value 中的 key 进行去重排序，去重时保留输入的最后一对 key/value。

18.2.1 JSON 函数

18.2.1.1 to_json

to_json 将输入参数转换为 JSON 类型的数据。

语法格式

```
<to_json 函数> ::= to_json(<exp>)
```

参数

<exp>：待转换数据，对于不支持的数据类型将在执行后返回提示信息。

返回值

JSON 类型

使用说明

1. 当参数<exp>为 NULL 时，返回 NULL;

2. 当参数<exp>为字符串类型时，返回 String 类型的 JSON，其中特殊字符会被转义;

3. 参数<exp>为数值类型时，返回 Number 类型的 JSON;

4. 参数<exp>为布尔类型时，返回 JSON 类型中对应的 True 或 False。

举例说明

例 1 参数<exp>为 NULL 时，返回 NULL。

```
select to_json(null);
```

查询结果如下:

```
行号       TO_JSON(NULL)

---------- --------------

1            NULL
```

例 2 参数<exp>为 TRUE 时，返回 true。

```
select to_json(TRUE);
```

查询结果如下:

```
行号       TO_JSON(TRUE)

---------- --------------

1            true
```

例 3 参数<exp>为 FALSE 时，返回 false。

```
select to_json(FALSE);
```

查询结果如下：

```
行号        TO_JSON(FALSE)

---------- ---------------

1             false
```

例 4 参数<exp>中包含转义字符时，将转义字符改写为“\”表示的转义字符。

```
select to_json('{"b":1, "a":1, "a":3, "a":2}');
```

查询结果如下：

```
行号        TO_JSON('{"b":1,"a":1,"a":3,"a":2}')

---------- ----------------------------------------

1              "{\"b\":1, \"a\":1, \"a\":3, \"a\":2}"
```

18.2.1.2 json_value

json_value 函数获取 JSON 数据中给定路径下的值，本函数的策略在 INI 参数 JSON_MODE 的不同取值下有差异。当 JSON_MODE 值为 1 时，不支持 JSON_VALUE 函数，执行 JSON_VALUE 函数时报错，当 JSON_MODE 值为 0 或 2 时都支持 JSON_VALUE 函数，策略差异见下方使用说明。

语法格式

```
<json_value 函数> ::= json_value(<json_exp>, <path_exp> [<RETURNING 项>] [ASCII]

[<EMPTY 项>][<ERROR 项>])

<RETURNING 项> :: = RETURNING VARCHAR | VARCHAR2 | NUMBER | DATE | DATETIME | VARBINARY

| CLOB | JSON

<EMPTY 项> :: =  <NULL | ERROR | <DEFAULT 项>> ON EMPTY

<ERROR 项> :: =  <NULL | ERROR | <DEFAULT 项>> ON ERROR

<DEFAULT 项> :: = DEFAULT  '<value>'
```

参数

<json_exp>：待查找的 JSON 数据，数据类型可以是 JSON，JSONB，VARCHAR 或 CLOB。源数据不是 JSONB 时，其结构中的 object 可能有重复 key 且未经过排序，当 JSON_MODE 值为 2 时，会先对 key/value 对进行排序和去重，再查找给定路径对应的数据，去重时，key 相同的 key/value 对仅保留最后一对。

<path_exp>：路径表达式，请参考 18.3.1 路径表达式。

<RETURNING 项>：返回值类型。当 JSON_MODE 值为 0 时，默认为 VARCHAR(4000)；当 JSON_MODE 值为 2 时，默认为 VARCHAR(512)。仅当 JSON_MODE 值为 2 时可指定 RETURNING JSON。

ASCII：以\uXXXX 十六进制的形式显示非 Unicode 字符，请参考 18.3.2 PRETTY 和 ASCII。该选项仅在 JSON_MODE=0 时支持。

<EMPTY 项>：指定当<path_exp>指定的项不存在时的返回值，返回值策略与<ERROR 项>一致，请参考 18.3.4 ERROR 项。当 JSON_MODE 值为 0 时，若未指定<EMPTY 项>，则默认按照<ERROR 项>决定指定项不存在时的返回值；当 JSON_MODE 值为 2 时，若未指定<EMPTY 项>，则默认为 NULL ON EMPTY。

<ERROR 项>：指定出错时的返回值，请参考 18.3.4 ERROR 项。

<DEFAULT 项>：返回值需要与<RETURNING 项>中定义的类型匹配。当 RETURNING 字符串时，默认值可以是数字或字符串。返回值类型和<RETURNING 项>中定义的类型不匹配时，处理策略受 JSON_MODE 取值影响。JSON_MODE=0 时，兼容 ORACLE，直接进行报错处理；JSON_MODE=2 时，兼容 MYSQL，尝试将返回值类型转化为<RETURNING 项>中定义的类型，转化失败时才进行报错。

使用说明

1. <json_exp>为 NULL 时，返回空结果集；
2. 数据类型中的 NUMBER 对应 DM 的 dec 类型，可以有多种写法，如：DECIMAL，

也可以指写精度，例如:DEC(10,5)；

3. 当 JSON_MODE 值为 0 时，路径表达式指定的项必须为标量类型数据，否则按照<ERROR 项>策略返回；当 JSON_MODE 值为 2 时，无上述限制。

举例说明:

例 创建测试表并插入数据。

```
create table t_json_value (c1 json);
insert into t_json_value(c1) values('{"b":2}');
insert into t_json_value(c1) values('{"c":2}');
insert into t_json_value(c1) values('{"c":"3"}');
insert into t_json_value(c1) values('{"c":null}');
insert into t_json_value(c1) values('{"c":true}');
insert into t_json_value(c1) values('{"c":10,"d":20}');
insert into t_json_value(c1) values('{"c":"a\tb"}');
insert into t_json_value(c1) values('{"c":[1, 2]}');
insert into t_json_value(c1) values('[1,"a",true]');
insert into t_json_value(c1) values('[null,[1, 2],{"a":1}]');
```

例 1 JSON_MODE=0，使用 json_value 查询对象的值。

```
SELECT json_value(c1, '$.c') FROM t_json_value;
```

查询结果如下:

```
行号        JSON_VALUE(C1,'$.c')
---------- --------------------
1              NULL
2               2
3               3
```

```
4              NULL

5              true

6              10

7              a b

8              NULL

9              NULL

10             NULL
```

例 2 JSON_MODE=2，使用 json_value 查询对象的值。

```
SELECT json_value(c1, '$.c') FROM t_json_value;
```

查询结果如下：

```
行号        JSON_VALUE(C1,'$.c')

---------- --------------------

1              NULL

2              2

3              3

4              NULL

5              true

6              10

7              a b

8              [1,2]

9              NULL

10             NULL
```

例 3 JSON_MODE=0，使用 json_value 查询数组的值。

```
SELECT json_value(c1, '$[1]') FROM t_json_value;
```

查询结果如下：

```
行号        JSON_VALUE(C1,'$[1]')

---------- --------------------

1              NULL

2              NULL

3              NULL

4              NULL

5              NULL

6              NULL

7              NULL

8              NULL

9              a

10             NULL
```

例 4 JSON_MODE=2，使用 json_value 查询数组的值。

```
SELECT json_value(c1, '$[1]') FROM t_json_value;
```

查询结果如下：

```
行号        JSON_VALUE(C1,'$[1]')

---------- --------------------

1              NULL

2              NULL

3              NULL

4              NULL

5              NULL

6              NULL
```

```
7       NULL
8       NULL
9       a
10      [1,2]
```

18.2.1.3 json_query

json_query 函数获取 JSON 数据中给定路径下的值，获取结果可以为 1 个或多个 JSON 数据。

语法格式

```
<json_query 函数> ::= json_query(<json_exp>, <path_exp> [<RETURNING 项>] [PRETTY] [ASCII] [<WRAPPER 项>] [<ERROR 项>])
<RETURNING 项> :: =
    RETURNING VARCHAR |
    VARCHAR2 |
    VARBINARY |
    BLOB |
    JSON
<WRAPPER 项> :: =
    WITH[CONDITIONAL|UNCONDITIONAL][ARRAY] WRAPPER|
    WITHOUT[ARRAY]WRAPPER
<ERROR 项> :: =  < NULL | ERROR | EMPTY>  ON ERROR
```

参数

<json_exp>：待查找的 JSON 数据，数据类型可以是 JSON，JSONB，VARCHAR 或 CLOB。当<json_exp>为 NULL 时，返回空结果集。

<path_exp>：路径表达式，请参考 18.3.1 路径表达式。

<RETURNING 项>：返回值类型，当<json_exp>是 JSONB 类型数据时，默认为 RETURNING JSON，其他情况默认为 RETURNING VARCHAR(4000)。

PRETTY：以缩进的形式显示字符，请参考 18.3.2 PRETTY 和 ASCII。

ASCII：以\uXXXX 十六进制的形式显示非 Unicode 字符，请参考 18.3.2 PRETTY 和 ASCII。

<WRAPPER 项>：指定查询结果的返回形式，请参考 18.3.3 WRAPPER 项。单值返回时，标量类型必须指定 WITH WRAPPER；多值返回时必须指定 WITH WRAPPER。

<ERROR 项>：指定出错时的返回值，请参考 18.3.4 ERROR 项。

举例说明

例 创建测试表并插入数据。

```
create table t_json_query (c1 json);

insert into t_json_query(c1) values('{"b":2}');

insert into t_json_query(c1) values('{"c":2}');

insert into t_json_query(c1) values('{"c":"3"}');

insert into t_json_query(c1) values('{"c":{"a":1}}');

insert into t_json_query(c1) values('{"c":[1,true]}');

insert into t_json_query(c1) values('{"c":[1,2],"d":{"a":3}}');
```

使用 json_query，以数组的形式返回 JSON 数据。

```
SELECT json_query(c1, '$.c' returning varchar2 with wrapper) FROM t_json_query;
```

查询结果如下：

```
行号       JSON_QUERY(C1,'$.c'WITHWRAPPER)

---------- --------------------------------

1             NULL
```

```
2           [2]
3           ["3"]
4           [{"a":1}]
5           [[1,true]]
6           [[1,2]]
```

18.2.1.4 json_table

json_table 函数获取 array 数组或 object 对象中指定项的值，返回结果为表记录。JSON_MODE=1 时不支持 json_table，JSON_MODE=2 时，兼容 MySQL 的表现。

语法格式

```
<json_table 函数> ::= json_table(<json_exp>, <path_exp1> [<EMPTY 项_1>] [<ERROR 项_1>]
<json_columns_clause>
<json_columns_clause>::= COLUMNS (<column_clause>{,<column_clause>})
<column_clause>::=
    <json_value_col> |
    <json_exists_col>|
    <json_query_col>|
    <json_nested_col>|
    <ordinality_col>
<json_value_col>::=<column_name> <datatype> PATH <path_exp2> [<EMPTY 项_1>]
[<ERROR 项_1>]
<json_exists_col>::=<column_name> <datatype> EXISTS PATH <path_exp2> [<EMPTY 项_2>]
[<ERROR 项_2>]
<json_query_col>::=<column_name> <datatype> FORMAT JSON PATH <path_exp2>
```

```
[<EMPTY 项_3>] [<ERROR 项_3>]

<json_nested_col>::= NESTED PATH <path_exp2> <json_columns_clause>

<ordinality_col>::=<column_name> FOR ORDINALITY

<EMPTY 项_1> :: = <NULL | ERROR | <DEFAULT 项>> ON EMPTY

<ERROR 项_1> :: = <NULL | ERROR | <DEFAULT 项>> ON ERROR

<DEFAULT 项> :: = DEFAULT '<value>'

<EMPTY 项_2>:: = <ERROR | TRUE | FALSE > ON EMPTY

<ERROR 项_2>:: = <ERROR | TRUE | FALSE> ON ERROR

<EMPTY 项_3>:: = <ERROR | NULL | EMPTY | EMPTY ARRAY | EMPTY OBJECT> ON EMPTY

<ERROR 项_3>:: = <ERROR | NULL | EMPTY | EMPTY ARRAY | EMPTY OBJECT> ON ERROR
```

参数

<json_exp>：待查找的 JSON 数据，数据类型可以是 JSON，JSONB，VARCHAR 或 CLOB。当<json_exp>为 NULL 时，返回空结果集。<json_exp>可以是表列，当列来自于与 json_table 同层的表时，则需要开启 ENABLE_TABLE_EXP_REF_FLAG 参数，DMDPC 场景下 json_table 暂不支持对同层表列的引用。

<path_exp1>：路径表达式，表示<json_exp>中的数据查询路径。请参考 18.3.1 路径表达式。

<column_clause>：列定义语句，支持<json_value_col>、<json_exists_col>、<json_query_col>、<json_nested_col>和<ordinality_col>五种格式的列定义语句。

<json_value_col>：将<path_exp2>路径下的数据作为该列的值。其中，<path_exp2>路径下的数据为 JSON 标量。该语句中的参数含义如下：

- <column_name>：列名。
- <datatype>：列的数据类型。支持 VARCHAR、CLOB、TINYINT、SMALLINT、INT、BIGINT、DEC、DATE、TIMESTAMP、TIMESTAMP WITH TIME ZONE 等。

- <path_exp2>：路径表达式，表示列独有路径，其结果基于<path_exp1>路径下的数据。请参考 18.3.1 路径表达式。
- <EMPTY 项_1>：指定当<path_exp2>指定的项不存在时的返回值，返回值策略与<ERROR 项>一致，请参考 18.3.4 ERROR 项。对于表级的<EMPTY 项_1>仅支持 NULL/ERROR 两种取值。
- <ERROR 项_1>：指定出错时的返回值，请参考 18.3.4 ERROR 项。对于表级的<ERROR 项_1>仅支持 NULL/ERROR 两种取值。

<json_exists_col>：检查<path_exp2>路径下存在数据，若存在则该列返回 1/true，否则返回 0/false(字符串类型时返回 true/false)。若查询过程返回空值或者出错，则按照<EMPTY 项_2>或<ERROR 项_2>处理。该语句中的其余参数含义与<json_value_col>相同。

<json_query_col>：将<path_exp2>路径下的数据作为该列的值。其中，<path_exp2>路径下的数据为 JSON ARRAY 或 JSON OBJECT。该语句中的参数含义如下:

- <datatype>：列的数据类型。支持 VARCHAR 和 CLOB。
- 其他参数含义与<json_value_col>相同。

<json_nested_col>：嵌套查询。<path_exp2>路径下的数据为 JSON ARRAY 或 JSON OBJECT，后续指定的<json_columns_clause>可以针对<path_exp2>路径下的数据进行嵌套查询。

<ordinality_col>：当前遍历的 JSON 数组中的元素序号（从 1 开始递增），或者当前遍历的 JSON 对象中的 key/value 序号（从 1 开始递增）。

返回值

表记录。

使用说明

1．<json_exp>为 NULL 时，返回空结果集；

2．各个列值需要先进行<path_exp1>查询，随后进行<path_exp2>查询。

举例说明

例 1 按照<json_value_col>语句查询指定路径下的值。

```
SELECT * FROM JSON_TABLE('{ "a":100, "b":200, "c":{"d":300, "e":400}}', '$.c' COLUMNS(C1 DEC
PATH '$.d', C2 DEC PATH '$.e'));
```

查询结果如下：

```
行号            C1   C2
---------- --- ---
1               300 400
```

其中，“$.c”是所有列共有的路径，而“$.d”是 C1 列的路径，“$.e”是 C2 列的路径，两列的数据类型均为 DEC 类型。

例 2 按照<json_exists_col>语句查询指定路径下是否存在数据，若存在则返回 1，若不存在则根据指定的 EMPTY 项返回。

```
SELECT * FROM JSON_TABLE('{a:100, b:200, c:{d:300, e:400}}', '$.c' COLUMNS(C1 INT EXISTS
PATH '$.d' FALSE ON EMPTY, C2 INT EXISTS PATH '$.f' FALSE ON EMPTY));
```

查询结果如下：

```
行号            C1   C2
---------- --- ---
1               1    0
```

例 3 按照<json_query_col>语句查询指定路径下的 JSON OBJECT。

```
SELECT * FROM JSON_TABLE('[{a:100, b:200, c:{d:300, e:400}},{a:500, b:600, c:{d:700,
e:800}}]', '$[*]' COLUMNS(C1 VARCHAR2(20) FORMAT JSON PATH '$.c'));
```

查询结果如下：

```
行号              C1

---------- -------

1                 {"d":300,"e":400}

2                 {"d":700,"e":800}
```

例 4 按照<json_nested_col>语句进行嵌套查询。

```
SELECT * FROM JSON_TABLE('{ "a":100, "b":200, "c":{"d":300, "e":400}}', '$' COLUMNS(NESTED PATH

'$.c' COLUMNS(C1 INT PATH '$.d', C2 INT PATH '$.e')));
```

查询结果如下:

```
行号       C1             C2

---------- ----------- -----------

1             300             400
```

例 5 按照<ordinality_col>语句输出当前遍历的 JSON 数组中的元素序号（从 1 开始递增）。

```
SELECT * FROM JSON_TABLE('[{a:100, b:200, c:{d:300, e:400}},{a:500, b:600, c:{d:700,

e:800}}]', '$[*]' COLUMNS(ID FOR ORDINALITY, C1 VARCHAR2(20) FORMAT JSON PATH '$.c'));
```

查询结果如下:

```
行号       ID       C1

---------- -- -----------------

1             1       {"d":300,"e":400}

2             2       {"d":700,"e":800}
```

18.2.1.5 json_overlaps

json_overlaps 检查两个 JSON 数据是否存在交集。

如果 JSON 数据中包含 object 对象，则对 key/value 中的 key 进行去重后再检查两

个 JSON 数据是否存在交集，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_overlaps 函数> ::= json_overlaps(<json_exp1>, <json_exp2>)
```

参数

<json_exp1>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<json_exp2>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

返回值

0：不存在交集；

1：存在交集。

使用说明

1. 若<json_exp1>或<json_exp2>为 NULL，则返回 NULL；

2. 若 JSON 数据存在嵌套情况，例如 array 中包含 object 或 array 等各种类型的数据，仅检查最外层 JSON 数据是否存在交集。

举例说明

例 1 <json_exp1>和<json_exp2>均为 NULL，返回 NULL。

```
SELECT JSON_OVERLAPS(NULL,NULL);
```

查询结果如下：

```
行号       JSON_OVERLAPS(NULL,NULL)

---------- ------------------------

1             NULL
```

例 2 <json_exp1>的 JSON 数据类型为 object，首先对 object 对象 key/value 中的 key 进行去重后检查是否存在交集。

在语句中查询{"a":1}。

```
SELECT JSON_OVERLAPS('{"a":1,"a":2,"a":3,"b":4}', '{"a":1}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('{"a":1,"a":2,"a":3,"b":4}','{"a":1}')

---------- ----------------------------------------------------

1             0
```

在语句中查询{"a":3}。

```
SELECT JSON_OVERLAPS('{"a":1,"a":2,"a":3,"b":4}', '{"a":3}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('{"a":1,"a":2,"a":3,"b":4}','{"a":3}')

---------- ----------------------------------------------------

1             1
```

本例中，首先对<json_exp1>中的 object 进行去重，去重时仅保留输入的最后一对 key/value，<json_exp1>去重后为“{"a":3,"b":4}”。

例 3 <json_exp1>的 JSON 数据类型为 object，并且 key/value 中的 value 也为 object，则对该 value 也将进行去重操作。

在语句中查询{"a":{"b":2}}。

```
SELECT JSON_OVERLAPS('{"a":{"b":2,"b":3}}', '{"a":{"b":2}}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('{"a":{"b":2,"b":3}}','{"a":{"b":2}}')

---------- ----------------------------------------------------

1             0
```

在语句中查询{"a":{"b":3}}。

```
SELECT JSON_OVERLAPS('{"a":{"b":2,"b":3}}', '{"a":{"b":3}}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('{"a":{"b":2,"b":3}}','{"a":{"b":3}}')
```

```
---------- ------------------------------------------------------

1            1
```

例 4 <json_expl>的 JSON 数据类型为 array，并且 array 中包含 object，则对该 object 对象 key/value 中的 key 也将进行去重操作。

在语句中查找{"a":2}。

```
SELECT JSON_OVERLAPS('[1,{"a":2,"a":3}]','{"a":2}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,{"a":2,"a":3}]','{"a":2}')

---------- ---------------------------------------------

1            0
```

在语句中查找{"a":3}。

```
SELECT JSON_OVERLAPS('[1,{"a":2,"a":3}]','{"a":3}');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,{"a":2,"a":3}]','{"a":3}')

---------- ---------------------------------------------

1            1
```

例 5 <json_expl>的 JSON 数据类型为 array，仅检查最外层 JSON 数据是否存在交集。

在语句中查找 1。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','1');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','1')

---------- ---------------------------------------------

1            1
```

在语句中查找 2。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','2');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','2')

---------- ------------------------------------------

1             0
```

在语句中查询[2,3]。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[2,3]');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[2,3]')

---------- ------------------------------------------------

1             0
```

在语句中查询[[2,3]]。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[[2,3]]');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[[2,3]]')

---------- --------------------------------------------------

1             1
```

在语句中查找[[3,2]]。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[[3,2]]');
```

查询结果如下：

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','[[3,2]]')

---------- --------------------------------------------------

1             0
```

在语句中查找{"a":2}。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','{"a":2}');
```

查询结果如下:

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','{"a":2}')

---------- ------------------------------------------------

1             0
```

在语句中查找{"a":2,"b":4}。

```
SELECT JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','{"a":2,"b":4}');
```

查询结果如下:

```
行号       JSON_OVERLAPS('[1,[2,3],{"a":2,"b":4}]','{"a":2,"b":4}')

---------- ------------------------------------------------------

1             1
```

本例中，<json_exp1>中共包含三个数组元素，即 number 类型数据“1”，array 类型数据“[2,3]”以及 object 类型数据“{"a":2,"b":4}”，因此仅当<json_exp2>中至少包含上述三个数据中的任意一个时，<json_exp1>和<json_exp2>才会存在交集。

18.2.1.6 json_set

json_set 替换或新增 JSON 数据中用户指定的项。

如果 JSON 数据中包含 object 对象，则自动对输入的 object 对象 key/value 中的 key 进行去重排序，去重时仅保留输入的最后一对 key/value。向 JSON 数据中新增 object 对象时，自动查找合适的位置插入 key/value 以保证 key 有序。

语法格式

```
<json_set 函数> ::= json_set(<json_exp>, <path_exp>, <value_exp>{, <path_exp>,
<value_exp>})
```

参数

<json_exp>：待查找的 JSON 数据，数据类型可以是 JSON，JSONB，VARCHAR 或 CLOB。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

1） 若<path_exp>最后一级为 ARRAY 下标，下标不为 0，但定位到的 JSON 不是 ARRAY 类型时，会在该位置创建新的 ARRAY，元素为原有 JSON 和待加入的 JSON。

2） <path_exp>不支持使用通配符或指定多个元素。

3） 当<path_exp>指定的项存在时，替换指定项的值；当<path_exp>指定的项不存在时，新增<path_exp>指定的项，并将值设为<value_exp>。

<value_exp>：指定替换后的值或新增项的值，数据类型为 NUMBER、VARCHAR、TRUE 或 FALSE。

返回值

JSON 数据，为 JSONB 类型。

使用说明

1. 当<json_exp>为 NULL 时，返回 NULL；
2. 支持同时替换或新增 JSON 数据中的多项。

举例说明

例 1 参数<json_exp>为 NULL。

```
select json_set(NULL,'$.f1.a',15);
```

查询结果如下：

```
行号        JSON_SET(NULL,'$.f1.a',15)

---------- --------------------------

1             NULL
```

例 2 替换 JSON 数据中的一项，参数<value_exp>的数据类型为 NUMBER。

```
select json_set('{"f1":{"a":1}}','$.f1.a',15);
```

查询结果如下：

```
行号       JSON_SET('{"f1":{"a":1}}','$.f1.a',15)

---------- ---------------------------------------

1              {"f1":{"a":15}}
```

例 3 替换 JSON 数据中的一项，参数<value_exp>的数据类型为 VARCHAR。

```
select json_set('{"f1":{"a":1}}','$.f1.a','str');
```

查询结果如下：

```
行号       JSON_SET('{"f1":{"a":1}}','$.f1.a','str')

---------- ----------------------------------------

1              {"f1":{"a":"str"}}
```

例 4 替换 JSON 数据中的一项，参数<value_exp>为 TRUE。

```
select json_set('[{"f1":{"a":1}}]','$[0]',TRUE);
```

查询结果如下：

```
行号          JSON_SET('[{"f1":{"a":1}}]','$[0]',TRUE)

---------- ----------------------------------------

1              [true]
```

例 5 同时替换 JSON 数据中的多项。

```
select json_set('[{"f1":{"a":1}},2,"b"]','$[0].f1.a',15,'$[1]','c');
```

查询结果如下：

```
行号       JSON_SET('[{"f1":{"a":1}},2,"b"]','$[0].f1.a',15,'$[1]','c')

---------- ------------------------------------------------------------

1              [{"f1":{"a":15}},"c","b"]
```

例 6 <path_exp>指定的项不存在时，新增<path_exp>指定的项，并将值设为

<value_exp>。

```
select json_set('{"f1":{"a":1}}','$.f1.b',15);
```

查询结果如下:

```
行号       JSON_SET('{"f1":{"a":1}}','$.f1.b',15)

---------- ----------------------------------------

1              {"f1":{"a":1,"b":15}}
```

例 7 json_set 函数创建新的 ARRAY，并在指定位置增加指定值 2。

```
select json_set('{"a":1}','$.a[1]',2);
```

查询结果如下:

```
行号          JSON_SET('{"a":1}','$.a[1]',2)

---------- ------------------------------

1              {"a":[1,2]}
```

例 8 使用通配符或指定多个元素时会报错。

```
select json_set('["abc"]','$.*','de');
```

查询结果报错:

```
[-3113]:JSON 处理错误.
```

18.2.1.7 json_replace

json_replace 替换 JSON 数据中用户指定的项。

如果 JSON 数据中包含 object 对象，则自动对输入的 object 对象 key/value 中的 key 进行去重排序，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_replace 函数> ::= json_replace(<json_exp>, <path_exp>, <value_exp>{, <path_exp>,
<value_exp>})
```

参数

<json_exp>：表示目标 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

<value_exp>：指定替换后的值，数据类型为 NUMBER、VARCHAR、TRUE 或 FALSE。

返回值

JSON 数据，为 JSONB 类型。

使用说明

1. 当<json_exp>为 NULL 时，返回 NULL；
2. 当<path_exp>指定的项不存在时，则不进行替换；
3. 支持同时替换 JSON 数据中的多项。

举例说明

例 替换 JSON 数据中的多项。

```
select json_replace('[{"f1":{"a":1}},2,"b"]','$[0].f1.a',15,'$[1]','c');
```

查询结果如下：

```
行号        JSON_REPLACE('[{"f1":{"a":1}},2,"b"]','$[0].f1.a',15,'$[1]','c')

---------- ----------------------------------------------------------------

1              [{"f1":{"a":15}},"c","b"]
```

18.2.1.8 json_insert

json_insert 新增 JSON 数据中用户指定的项。

如果 JSON 数据中包含 object 对象，则自动对输入的 object 对象 key/value 中的 key 进行去重排序，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_insert 函数> ::= json_insert(<json_exp>, <path_exp>, <value_exp>{, <path_exp>,
```

```
<value_exp>})
```

参数

<json_exp>:表示目标 JSON 数据,数据类型为 JSON, JSONB, VARCHAR 或 CLOB。

<path_exp>: 路径表达式，具体书写规则请参考 18.3.1 路径表达式。

<value_exp>: 指定新增项的值，数据类型为 NUMBER、VARCHAR、TRUE 或 FALSE。

返回值

JSON 数据，为 JSONB 类型。

使用说明

1. 当<json_exp>为 NULL 时，返回 NULL;
2. 当<path_exp>指定的项不存在时，则不进行替换;
3. 支持同时替换 JSON 数据中的多项。

举例说明

例 新增 JSON 数据中的多项。

```
select json_insert('[{"f1":{"a":1}},2,"b"]','$[0].f1.a[1]',2,'$[3]','c');
```

查询结果如下:

```
行号        JSON_INSERT('[{"f1":{"a":1}},2,"b"]','$[0].f1.a[1]',2,'$[3]','c')

---------- ----------------------------------------------------------------------

1              [{"f1":{"a":[1,2]}},2,"b","c"]
```

18.2.1.9 json_extract

json_extract 获取 JSON 数据中若干个给定路径下的值。

语法格式

```
<json_extract 函数> ::= json_extract(<json_exp>, <path_exp>{, <path_exp>})
```

参数

<json_exp>：目标 JSON 数据，数据类型可以是 JSON，JSONB，VARCHAR 或 CLOB，源数据不是 JSONB 时，其结构中的 object 可能有重复 key 且未经过排序，执行前会先对 key/value 对进行排序和去重，去重时，key 相同的 key/value 对仅保留最后一对。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

返回值

JSONB 类型数据。

使用说明

1. 当根据给定路径查找不到数据时，返回 NULL；

2. 路径中有多值过滤路径（通配符和数组下标范围）时，或查找到的结果不止一个时，需将结果包装为 JSONB ARRAY 形式。

举例说明

例 1 多值过滤路径。

```
SQL> SELECT JSON_EXTRACT('{"c": [{"a":1, "b":"2"}]}', '$.c[*].b' );

行号        JSON_EXTRACT('{"c":[{"a":1,"b":"2"}]}','$.c[*].b')

---------- --------------------------------------------------

1           ["2"]
```

例 2 返回多个结果。

```
SQL> SELECT JSON_EXTRACT('{"c": [{"a":1, "b":"2"}]}', '$.c[0].a', '$.c[0].b');

行号        JSON_EXTRACT('{"c":[{"a":1,"b":"2"}]}','$.c[0].a','$.c[0].b')

---------- ------------------------------------------------------------

1           [1,"2"]
```

18.2.1.10 json_unquote

json_unquote 取消指定的 string 类型的 JSON 数据外层的双引号，并将结果作为字

符串返回。

语法格式

```
<json_unquote 函数> ::= json_unquote(<json_value>)
```

参数

<json_value>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

返回值

TEXT 类型

使用说明

1. 当<json_value>为 NULL 时，返回 NULL；
2. 当<json_value>是 STRING 类型的 JSON 数据，去掉其双引号然后作为结果返回，其中包含转义字符时，对转义字符进行转义后输出；
3. 当<json_value>不是 STRING 类型的 JSON 数据，或是一个不表示合法 JSON 数据的字符串时，直接将输入参数作为结果返回。

举例说明

例 创建测试表并插入数据。

```
create table t_json_unquote (c1 json);

insert into t_json_unquote(c1) values('{"b":2}');

insert into t_json_unquote(c1) values('{"c":2}');

insert into t_json_unquote(c1) values('{"c":"3"}');

insert into t_json_unquote(c1) values('{"c":null}');

insert into t_json_unquote(c1) values('{"c":true}');

insert into t_json_unquote(c1) values('{"c":10,"d":20}');

insert into t_json_unquote(c1) values('{"c":"a\tb"}');

insert into t_json_unquote(c1) values('{"c":"abc"}');
```

```
insert into t_json_unquote(c1) values('{"c":[1,2]}');

insert into t_json_unquote(c1) values('{"c":{"a":2,"b":3}}');
```

json_unquote 可以搭配 json_extract 一起使用，首先利用 json_extract 函数获取 key/value 中的 value，然后利用 json_unquote 函数去掉 value 外层的双引号。

```
select json_unquote(json_extract(c1,'$.c')) from t_json_unquote;
```

查询结果如下：

```
行号       JSON_UNQUOTE(JSON_EXTRACT(C1,'$.c'))

---------- ------------------------------------

1              NULL

2               2

3               3

4               null

5               true

6               10

7               a      b

8               abc

9               [1,2]

10              {"a":2,"b":3}
```

18.2.1.11 json_array

json_array 从可变的参数列表（可以为空）中创建 array 数组并返回。

语法格式

```
<json_array 函数> ::= json_array([<value_exp>[, <value_exp>]...])
```

参数

<value_exp>：传入的参数列表。可以传入任意类型的任意数量的参数。

返回值

JSON 数据，为 JSONB 类型。

使用说明

若参数列表中的任一参数不能转化成数字、字符串、布尔值、数组、对象、NULL 等 JSON 类型则报错。

举例说明

例 使用 json_array 函数创建 array 数组。

```
SELECT JSON_ARRAY(1,'1',FALSE,NULL,'[1,2]','{SS:1600}');
```

查询结果如下：

```
行号          JSON_ARRAY(1,'1',FALSE,NULL,'[1,2]','{SS:1600}')

---------- ------------------------------------------------

1             [1,"1",false,null,"[1,2]","{SS:1600}"]
```

18.2.1.12 json_object

json_object 返回一个用给定 key/value 创建的 object 对象。

返回的 object 对象自动对 key/value 中的 key 进行去重排序，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_object 函数> ::= json_object([<key_exp>, <value_exp>[, <key_exp>,

<value_exp>]...])
```

参数

<key_exp>：字符串、数字或布尔值，函数会自动将其转化成字符串。

<value_exp>：任意类型。

返回值

JSON 数据，为 JSONB 类型。

使用说明

1．传入参数需为偶数个，若传入奇数个参数则报错；

2．如果作为 key 的参数不是字符串、数字、布尔值；或者作为 value 的参数不能转化成数字、字符串、布尔值、数组、对象、null 等 JSON 类型则报错。作为 key 的参数的布尔值将被转换为"0"/"1"。

举例说明

例 1 参数的 key 为字符串、数字、布尔值时，使用 json_object 函数。

```
SELECT JSON_OBJECT('A',2,3,4,TRUE,5);
```

查询结果如下：

```
行号        JSON_OBJECT('A',2,3,4,TRUE,5)

---------- ------------------------------

1           {"1":5,"3":4,"A":2}
```

例 2 参数的 key 为其他类型时，使用 json_object 函数。

```
SELECT JSON_OBJECT([1,2],5);
```

查询结果报错：

```
[-2007]:语法分析出错.
```

例 3 传入参数为奇数个。

```
SELECT JSON_OBJECT('1',false,5);
```

查询结果报错：

```
[-5402]:参数个数不匹配.
```

18.2.1.13 json_keys

json_keys 将 object 对象的最外层 key 以 array 数组形式返回，若指定了 path 参数，则将给定 path 查找到的 object 对象的最外层 key 以 array 数组形式返回。

该函数自动对输入的 object 对象 key/value 中的 key 进行去重排序，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_key 函数> ::= json_keys(<json_exp>[, <path_exp>])
```

参数

<json_exp>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

返回值

JSON 数据，为 JSONB 类型。

使用说明

1．任一参数为 NULL，或 json_doc 不是 object 对象，或给定 path 查找到的不是 object 对象时返回 NULL；

2．若 json_doc 不是合法 JSON；或 path 参数不是合法 path 表达式或含有*和**通配符时报错。

举例说明

例 1 不指定 path 参数。

```
SELECT JSON_KEYS('{"a":1, "b":{"c":2, "d":3}}');
```

查询结果如下：

```
行号        JSON_KEYS('{"a":1,"b":{"c":2,"d":3}}')

---------- ---------------------------------------

1           ["a","b"]
```

例 2 指定 path 参数。

```
SELECT JSON_KEYS('{"a":1, "b":{"c":2, "d":3}}', '$.b');
```

查询结果如下：

```
行号          JSON_KEYS('{"a":1,"b":{"c":2,"d":3}}','$.b')

---------- ----------------------------------------------

1             ["c","d"]
```

例 3 path 不合法。

```
SELECT JSON_KEYS('{"a":1, "b":{"c":2, "d":3}}', 'b');
```

查询结果报错：

```
[-3102]:JSON 路径表达式语法错误.
```

18.2.1.14 json_object_keys

json_object_keys 返回顶层 object 对象 key 的集合。

该函数不会对 object 对象 key/value 中的 key 进行去重排序。

语法格式

```
<json_object_keys 函数> ::= json_object_keys(<json_exp>)
```

参数

<json_exp>：JSON 类型的数据或表示合法 JSON 的字符串。

返回值

字符串集合。

使用说明

1．此函数在查询中须位于 from 项。

2．参数表示的不是 object 对象时报错。

举例说明

例 1 使用 json_object_keys 返回 object 对象 key 的集合。

```
SELECT * FROM JSON_OBJECT_KEYS('{"F1":"ABC","F2":{"F3":"A","F4":"B"}}');
```

查询结果如下:

```
行号          JSON_OBJECT_KEYS

---------- ----------------

1             F1

2             F2
```

例 2 json_object_keys 的参数不是 object 对象。

```
SELECT * FROM JSON_OBJECT_KEYS('[1,2,3]');
```

查询结果报错:

```
[-3117]:只能在对象上调用此函数.
```

例 3 在查询项中使用 json_object_keys。

```
SELECT JSON_OBJECT_KEYS('{"f1":"abc","f2":{"f3":"a","f4":"b"}}');
```

查询结果报错:

```
[-2207]:无法解析的成员访问表达式[JSON_OBJECT_KEYS].
```

18.2.1.15 json_contains

json_contains 函数判断 json 间的包含关系。判断根据路径<path_exp>在<json_exp1>中查找到的 JSON 是否包含<json_exp2>，未给定<path_exp>参数时，判断<json_exp1>是否包含<json_exp2>。

若输入的 JSON 数据中包含 object 对象，则首先对 object 对象 key/value 中的 key 进行去重后再判断包含关系，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_contains 函数> ::= json_contains(<json_exp1>, <json_exp2>[, <path_exp>])
```

参数

<json_exp1>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<json_exp2>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

返回值

返回值为数值类型，1 表示包含，0 表示不包含。

使用说明

1. 参数<json_exp1>或<json_exp2>不是表示合法 JSON 的字符串时，报错；

2. 任一参数为 NULL，或<path_exp>在<json_exp1>中无法查找到数据时，返回 NULL；

3. path 表达式含有多重路径，例如：通配符、数组下标范围时报错；

4. 两标量在当且仅当它们可比较且相等时可视为包含；可比较的情况：JSON 类型相同可比较，json_int64 和 json_decimal 可比较；

5. 下述为视为包含关系的情况，除下述情况外均不是包含关系：

   1) 一个 array 数组包含于另一个 array 数组，当且仅当前者每个元素都被后者的某个元素包含；

   2) 一个非数组 JSON 包含于一个 array 数组，当且仅当前者被后者的某个元素包含；

   3) 一个 object 对象包含于另一个 object 对象，当且仅当前者的所有 key 都在后者中有同名的 key，且前者 key 对应的 value，均包含于后者相应 key 对应的 value。

举例说明

例 1 任一参数为 NULL 时，返回 NULL。

```
SELECT JSON_CONTAINS('1','1',NULL);
```

查询结果如下：

```
行号         JSON_CONTAINS('1','1',NULL)

---------- ----------------------------

1            NULL
```

例 2 标量间的包含关系。

```
SELECT JSON_CONTAINS('1.0', '1');
```

查询结果如下：

```
行号         JSON_CONTAINS('1.0','1')

---------- ------------------------

1            1
```

例 3 非标量间的包含关系。

```
SELECT JSON_CONTAINS('{"a":[{"b":[1]}, ["c"]]}', '[{"b":1}, "c"]', '$.a');
```

查询结果如下：

```
行号         JSON_CONTAINS('{"a":[{"b":[1]},["c"]]}','[{"b":1},"c"]','$.a')

---------- ------------------------------------------------------------

1            1
```

18.2.1.16 json_contains_path

json_contains_path 返回是否能根据给定的路径在 JSON 中查找到数据。

若输入的 JSON 数据中包含 object 对象，则首先对 object 对象 key/value 中的 key 进行去重后再进行查找，去重时仅保留输入的最后一对 key/value。

语法格式

```
<json_contains_path 函数> ::= json_contains_path(<json_exp>, <one_or_all 项>,

<path_exp>[, <path_exp>] ...)
```

```
<one_or_all 项> :: = 'ONE' | 'ALL'
```

参数

<json_exp>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

<one_or_all 项>：指定'ONE'表示给定的 path 中存在一个能查找到即可返回 1(true)；指定'ALL'表示给定的 path 需要全部查找到才可返回 1(true)。

<path_exp>：路径表达式，具体书写规则请参考 18.3.1 路径表达式。

返回值

返回 1 或 0。

使用说明

任一参数为 NULL 时返回 NULL。

举例说明

例 1 <one_or_all 项>取'ONE'。

```
SELECT JSON_CONTAINS_PATH('{"a":1,"b":2,"c":{"d":4}}', 'one', '$.a', '$.e');
```

查询结果如下：

```
行号         JSON_CONTAINS_PATH('{"a":1,"b":2,"c":{"d":4}}','one','$.a','$.e')

---------- ----------------------------------------------------------------

1            1
```

例 2 <one_or_all 项>取'ALL'。

```
SELECT JSON_CONTAINS_PATH('{"a":1,"b":2,"c":{"d":4}}', 'all', '$.a', '$.e');
```

查询结果如下：

```
行号         JSON_CONTAINS_PATH('{"a":1,"b":2,"c":{"d":4}}','all','$.a','$.e')

---------- ----------------------------------------------------------------

1            0
```

例 3 <one_or_all 项>取其他值。

```
SELECT JSON_CONTAINS_PATH('{"a":1,"b":2,"c":{"d":4}}', 'any', '$.a', '$.e');
```

查询结果报错:

```
[-6803]:非法的参数数据.
```

18.2.1.17 json_build_array

json_build_array 从可变的参数列表（可以为空）中创建 array 数组并返回。

语法格式

```
<json_build_array 函数> ::= json_build_array(<exp>)
```

参数

<exp>：传入的参数列表。可以传入任意类型的任意数量的参数。

返回值

array 类型的 JSON

使用说明

任一参数不能转化成数字、字符串、布尔值、数组、对象、null 等 JSON 类型则报错。

举例说明

例 使用 json_build_array 函数创建 array 数组。

```
SELECT JSON_BUILD_ARRAY(1,'1',FALSE,NULL,'[1,2]','{ss:1600}');
```

查询结果如下:

```
行号        JSON_BUILD_ARRAY(1,'1',FALSE,NULL,'[1,2]','{ss:1600}')

---------- ----------------------------------------------------

1           [1,"1",false,null,"[1,2]","{ss:1600}"]
```

18.2.1.18 json_build_object

json_build_object 从可变的参数列表（可以为空）中创建 object 对象并返回。

该函数不会对 object 对象 key/value 中的 key 进行去重排序。

语法格式

```
<json_build_object 函数> ::= json_build_object([<key_exp>, <value_exp>[, <key_exp>,
<value_exp>]...])
```

参数

<key_exp>：字符串，还可以是数字或布尔值，函数会将其转化成字符串。

<value_exp>：任意类型。

返回值

object 类型的 JSON

使用说明

1．传入参数需为偶数个，若传入奇数个参数则报错。

2．取作 key 的参数不是字符串、数字、布尔值；或取作 value 的参数不能转化成数字、字符串、布尔值、数组、对象、null 等 JSON 类型则报错。作为 key 的参数的布尔值将被转换为"1"/"0"。

举例说明

例 1 参数的 key 为字符串、数字、布尔值时，使用 json_build_object 函数。

```
SELECT JSON_BUILD_OBJECT('a',2,3,4,true,5);
```

查询结果如下：

```
行号        JSON_BUILD_OBJECT('a',2,3,4,TRUE,5)
---------- ------------------------------------
1           {"a":2,"3":4,"1":5}
```

例 2 参数的 key 为其他类型时，使用 json_build_object 函数。

```
SELECT json_build_object([1,2],5);
```

查询结果报错：

```
[-2007]:语法分析出错.
```

例 3 传入参数为奇数个。

```
SELECT JSON_BUILD_OBJECT('1',false,5);
```

查询结果报错：

```
[-5402]:参数个数不匹配.
```

18.2.1.19 json_type

json_type 返回指定 JSON 数据的 JSON 类型。

语法格式

```
<json_type 函数> ::= json_type(<json_exp>)
```

参数

<json_exp>：表示 JSON 数据，数据类型为 JSON，JSONB，VARCHAR 或 CLOB。

返回值

返回<json_exp>对应的 JSON 类型。

使用说明

1．当<json_exp>为 NULL 时，返回 NULL；

2．当<json_exp>为'null'时，返回 NULL；

3．当<json_exp>为'NULL'时，报错返回；

4．当<json_exp>表示整数时，返回 INTEGER；

5．当<json_exp>表示小数时，返回 DOUBLE；

6．当<json_exp>为非法的 JSON 数据时，报错返回。

举例说明

例 1 <json_exp>为 NULL 或者'NULL'。

```
SELECT JSON_TYPE(NULL);
```

```
--查询结果如下:

行号        JSON_TYPE(NULL)

---------- ---------------

1               NULL



SELECT JSON_TYPE('null');

--查询结果如下

行号        JSON_TYPE('null')

---------- -----------------

1               NULL

--查询结果如下

SELECT JSON_TYPE('NULL');

[-3105]:JSON 值语法错误.
```

例 2 <json_exp>表示数值类型。

```
SELECT JSON_TYPE('1');

--查询结果如下

行号        JSON_TYPE('1')

---------- --------------

1               INTEGER



SELECT JSON_TYPE('1.23');

--查询结果如下

行号        JSON_TYPE('1.23')

---------- -----------------
```

```
1              DOUBLE
```

例 3 <json_exp>表示字符串、布尔类型、object、array。

```
SELECT JSON_TYPE('"abc"');

--查询结果如下

行号        JSON_TYPE('"abc"')

---------- ------------------

1              STRING



SELECT JSON_TYPE('true');

--查询结果如下

行号        JSON_TYPE('true')

---------- -----------------

1              BOOLEAN



SELECT JSON_TYPE('{"a":1}');

--查询结果如下

行号        JSON_TYPE('{"a":1}')

---------- --------------------

1              OBJECT



SELECT JSON_TYPE('[1,2,3]');

--查询结果如下

行号        JSON_TYPE('[1,2,3]')

---------- --------------------
```

```
1              ARRAY
```

例 4 <json_exp>为非法的 JSON 数据时，报错返回。

```
SELECT JSON_TYPE('abc');

[-3105]:JSON 值语法错误.
```

18.2.1.20 json_typeof

json_typeof 返回指定 JSON 数据的 JSON 类型。

语法格式

```
<json_typeof 函数> ::= json_typeof(<json_exp>)
```

参数

<json_exp>：JSON 类型的数据或表示合法 JSON 的字符串。

返回值

返回<json_exp>对应的 JSON 类型。

使用说明

1．当<json_exp>为 NULL 时，返回 NULL；

2．当<json_exp>为'null'时，返回 null；

3．当<json_exp>为'NULL'时，报错返回；

4．当<json_exp>表示整数或者小数时，均返回 number；

5．当<json_exp>为非法的 JSON 数据时，报错返回。

举例说明

例 1 <json_exp>为 NULL 或者'NULL'。

```
SELECT JSON_TYPEOF(NULL);

//查询结果如下

行号       JSON_TYPEOF(NULL)
```

```
---------- -----------------

1              NULL



SELECT JSON_TYPEOF('null');

//查询结果如下

行号       JSON_TYPEOF('null')

---------- -------------------

1              null



SELECT JSON_TYPEOF('NULL');

[-3105]:JSON 值语法错误.
```

例 2 <json_exp>表示数值类型。

```
SELECT JSON_TYPEOF('1');

//查询结果如下

行号       JSON_TYPEOF('1')

---------- ----------------

1              number



SELECT JSON_TYPEOF('1.23');

//查询结果如下

行号       JSON_TYPEOF('1.23')

---------- -------------------

1              number
```

例 3 <json_exp>表示字符串、布尔类型、object、array。

```
SELECT JSON_TYPEOF('"abc"');

//查询结果如下

行号        JSON_TYPEOF('"abc"')

---------- --------------------

1               string


SELECT JSON_TYPEOF('true');

//查询结果如下

行号        JSON_TYPEOF('true')

---------- -------------------

1               boolean


SELECT JSON_TYPEOF('{"a":1}');

//查询结果如下

行号        JSON_TYPEOF('{"a":1}')

---------- ----------------------

1               object


SELECT JSON_TYPEOF('[1,2,3]');

//查询结果如下

行号        JSON_TYPEOF('[1,2,3]')

---------- ----------------------

1               array
```

例 4 <json_exp>为非法的 JSON 数据时，报错返回。

```
SELECT JSON_TYPEOF('abc');

[-3105]:JSON 值语法错误.
```

18.2.1.21 json_concat

json_concat 将两个 JSON 数据合并成一个 JSON 数据。

语法格式

```
<json_concat 函数> ::= json_concat(<json_exp1>,<json_exp2>)
```

参数

<json_exp1>：JSON 类型的数据或表示合法 JSON 的字符串。

<json_exp2>：JSON 类型的数据或表示合法 JSON 的字符串。

返回值

JSON 类型

使用说明

1. 当参数<json_exp1>或<json_exp2>为 NULL 时，则返回 NULL；
2. 当参数<json_exp1>或<json_exp2>不是 JSON 数据时，则报错；
3. 合并两个 OBJECT 时，返回 OBJECT，结果包含两个 OBJECT 的所有 key:value 对，不递归合并，只处理最外层 key:value；
4. 合并两个 ARRAY 时，返回 ARRAY，结果包含两个 ARRAY 的所有元素；
5. ARRAY 与非 ARRAY 合并以及两个非 ARRAY 合并，将非 ARRAY 包装为单元素 ARRAY，然后当成两个 ARRAY 合并，返回 ARRAY。

举例说明

例 对两个 JSON 数据进行合并。本例结果和 jsonb_concat 不同，没有排序和去重。

```
select JSON_CONCAT('{"A":1,"A":2,"a":3}','{"A":4,"A":5}');

//结果如下
```

```
{"A":1,"A":2,"a":3,"A":4,"A":5}
```

18.2.1.22 json_valid

json_valid 函数用于检查输入参数是否为合法的 JSON 数据。

语法格式

```
<json_valid 函数> ::= json_valid(<exp>)
```

参数

<exp>：支持任意数据类型。

返回值

0：<exp>为非法的 JSON 数据；

1：<exp>为合法的 JSON 数据。

使用说明

1. 当<exp>为 NULL 时，返回 NULL；
2. 当<exp>为 JSON 或 JSONB 类型数据时，返回 1；
3. 当<exp>为表示合法 JSON 的字符串数据时，返回 1。

举例说明

例 创建测试表并插入数据。

```
create table t_json_valid (c1 varchar(50));

insert into t_json_valid values(null);

insert into t_json_valid values('abc');

insert into t_json_valid values('[1');

insert into t_json_valid values('[1,true]');

insert into t_json_valid values('{"a":1,"b":2}');
```

使用 json_valid 函数判断输入参数是否为合法的 JSON 数据。

```
select json_valid(c1) from t_json_valid;
```

查询结果如下：

```
行号       JSON_VALID(C1)

---------- --------------

1             NULL

2              0

3              0

4              1

5              1
```

18.2.1.23 row_to_json

row_to_json 将传入的每一条数据转换成 JSON。可以传入表名，将整个表中的数据都转换为 JSON；或仅对指定的一列或多列中的数据进行转换。

语法格式

```
<row_to_json 函数> ::= row_to_json (<exp>)
```

参数

<exp>：可为表名或为 ROW 构造函数对象。

返回值

JSON 数据类型

使用说明

需要与对应表的 from 项一起使用。转换指定的列时，需要配合 ROW()构造函数进行查询，ROW()中的参数可为一个或多个列的列名；或使用别名方式对表列进行查询。

举例说明

例 1 使用 row_to_json 转换整个表 T 中的数据。

```
CREATE TABLE T(
id INT NOT NULL,
name VARCHAR(100),
age INT,
NOT CLUSTER PRIMARY KEY(id)
) ;

INSERT INTO T(id, name, age) VALUES (1, 'john doe', 25);
INSERT INTO T(id, name, age) VALUES (2, 'jane smith', 30);
INSERT INTO T(id, name, age) VALUES (3, 'bob johnson', 22);
COMMIT;

SELECT row_to_json(T) FROM T;
//查询结果如下
行号          ROW_TO_JSON(T)
---------- ----------------------------------------
1             {"ID":1,"NAME":"john doe","AGE":25}
2              {"ID":2,"NAME":"jane smith","AGE":30}
3              {"ID":3,"NAME":"bob johnson","AGE":22}
```

例 2 使用 row_to_json 转换表 T 中 ID 列、ID 列和 NAME 列里的数据，且支持通过别名方式查询。

```
//转换 ID 列

SELECT row_to_json(row(id)) FROM T;

//查询结果如下

行号          ROW_TO_JSON("ROW"(ID))

---------- ----------------------

1             {"f1":1}

2             {"f1":2}

3             {"f1":3}



//转换 ID 和 NAME 列

SELECT row_to_json(row(id, name)) FROM T;

//查询结果如下

行号          ROW_TO_JSON("ROW"(ID,NAME))

---------- --------------------------

1             {"f1":1,"f2":"john doe"}

2             {"f1":2,"f2":"jane smith"}

3             {"f1":3,"f2":"bob johnson"}



//通过别名进行查询

SELECT row_to_json(b) AS id FROM (SELECT id FROM T) b;

//查询结果如下

行号          ID

---------- --------

1             {"ID":1}
```

```
2              {"ID":2}

3              {"ID":3}
```

18.2.2 JSONB 函数

18.2.2.1 to_jsonb

to_jsonb 将输入参数转换为 JSONB 数据。

语法格式

```
<to_jsonb 函数> ::= to_jsonb(<exp>)
```

参数

<exp>：待转换数据，对于不支持的数据类型将在执行后返回提示信息。

返回值

JSONB 类型

使用说明

1. 当参数<exp>为 NULL 时，返回 NULL;
2. 当参数<exp>为字符串类型时，返回 String 类型的 JSONB，其中特殊字符会被转义;
3. 参数<exp>为数值类型时，返回 Number 类型的 JSONB;
4. 参数<exp>为布尔类型时，返回 JSONB 类型中对应的 True 或 False。

举例说明

例 1 参数<exp>为 NULL 时，返回 NULL。

```
select to_jsonb(null);
```

查询结果如下:

```
行号       TO_JSONB(NULL)

---------- --------------
```

```
1              NULL
```

例 2 参数<exp>为 TRUE 时，返回 true。

```
select to_jsonb(TRUE);
```

查询结果如下：

```
行号       TO_JSONB(TRUE)

---------- --------------

1              true
```

例 3 参数<exp>为 FALSE 时，返回 false。

```
select to_jsonb(FALSE);
```

查询结果如下：

```
行号       TO_JSONB(FALSE)

---------- ---------------

1              false
```

例 4 参数<exp>中包含转义字符时，将转义字符改写为“\”表示的转义字符。

```
select to_jsonb('{"b":1, "a":1, "a":3, "a":2}');
```

查询结果如下：

```
行号       TO_JSONB('{"b":1,"a":1,"a":3,"a":2}')

---------- --------------------------------------

1              "{\"b\":1, \"a\":1, \"a\":3, \"a\":2}"
```

18.2.2.2 jsonb_each

jsonb_each 将最外层 object 对象扩展为一组 key/value。

语法格式

```
<jsonb_each 函数> ::= jsonb_each(<json_exp>)
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

返回值表的结构为（key，value）。

key：object 对象 key/value 键值对中的键 key，数据类型为 CLOB。

value：object 对象 key/value 键值对中的值 value，数据类型为 JSONB。

使用说明

1. 当参数<json_exp>为 NULL 时，返回的结果集为空集；

2. 参数<json_exp>对应的 JSON 数据类型必须为 OBJECT，否则报错。

举例说明

例 1 使用 jsonb_each，将最外层 object 对象扩展为 key/value。

```
select * from
jsonb_each('{"a":1,"b":true,"c":null,"d":"str\ttest","e":[1,2,3],"f":{"name1":"aaa\ttest","name2":"bbb\tt
est"}}');
```

查询结果如下：

```
行号        KEY VALUE
---------- --- ----------------------------------------
1             a    1
2             b    true
3             c    null
4             d    "str\ttest"
5             e    [1,2,3]
6             f    {"name1":"aaa\ttest","name2":"bbb\ttest"}
```

例 2 当参数<json_exp>为 NULL 时，返回的结果集为空集。

```
select * from jsonb_each(null);
```

查询结果如下：

```
未选定行
```

例 3 当参数<json_exp>对应的 JSON 数据类型不是 OBJECT 时，报错。

```
select * from jsonb_each('[1,2,3]');
```

查询结果报错：

```
[-3117]:只能在对象上调用此函数.
```

18.2.2.3 jsonb_each_text

jsonb_each_text 将最外层 object 对象扩展为一组 key/value。

语法格式

```
<jsonb_each_text 函数> ::= jsonb_each_text(<json_exp>)
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

返回值表的结构为（key，value）。

key：object 对象 key/value 键值对中的键 key，数据类型为 CLOB。

value：object 对象 key/value 键值对中的值 value，数据类型为 CLOB。

使用说明

jsonb_each_text 与 jsonb_each 的区别仅在于：jsonb_each_text 当 value 的返回结果为 JSON STRING 类型时，将其转换为相应值的字符串，例如将其中的转义字符“\t”转换为 tab 键。

举例说明

例 使用 jsonb_each_text，将最外层 object 对象扩展为 key/value。

```
select * from
jsonb_each_text('{"a":1,"b":true,"c":null,"d":"str\ttest","e":[1,2,3],"f":{"name1":"aaa\ttest","name2":"b
bb\ttest"}}');
```

查询结果如下：

```
行号        KEY VALUE
---------- --- ----------------------------------------
1             a    1
2             b    true
3             c    null
4             d    str test
5             e    [1,2,3]
6             f    {"name1":"aaa\ttest","name2":"bbb\ttest"}
```

其中，对于字符串“str\ttest”，字符串中的转义字符“\t”被改写成了对应的 tab 键。

18.2.2.4 jsonb_array_elements

jsonb_array_elements 将 array 数组扩展为值的集合。

语法格式

```
<jsonb_array_elements 函数> ::= jsonb_array_elements(<json_exp>)
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

返回值表的结构为（value）。

value：array 中的各元素，数据类型为 JSONB。

使用说明

1. 当参数<json_exp>为 NULL 时，返回的结果集为空集；

2. 参数<json_exp>对应的 JSON 数据类型必须为 ARRAY，否则报错。

举例说明

例 1 使用 jsonb_array_elements，将 array 数组扩展为值的集合。

```
select * from jsonb_array_elements('[1, true, null, "str\ttest", [1,2,3],
{"name1":"aaa\ttest","name2":"bbb\ttest"}]');
```

查询结果如下：

```
行号        VALUE
---------- ----------------------------------------
1              1
2              true
3              null
4              "str\ttest"
5              [1,2,3]
6              {"name1":"aaa\ttest","name2":"bbb\ttest"}
```

例 2 当参数<json_exp>为 NULL 时，返回的结果集为空集。

```
select * from jsonb_array_elements(null);
```

查询结果如下：

```
未选定行
```

例 3 当参数<json_exp>对应的 JSON 数据类型不是 ARRAY 时，报错。

```
select * from jsonb_array_elements('1');
```

查询结果报错：

```
[-3118]:只能从数组中提取元素.
```

18.2.2.5 jsonb_array_elements_text

jsonb_array_elements_text 将 array 数组扩展为值的集合。

语法格式

```
<jsonb_array_elements_text 函数> ::= jsonb_array_elements_text(<json_exp>)
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

返回值表的结构为（value）。

value：array 中的各元素，数据类型为 CLOB。

使用说明

jsonb_array_elements_text 与 jsonb_array_elements 的区别仅在于：jsonb_array_elements_text 当 value 的返回结果为 JSON STRING 类型时，将其转换为相应值的字符串，例如将其中的转义字符“\t”转换为 tab 键。

举例说明

例 使用 jsonb_array_elements_text，将 array 数组扩展为值的集合。

```
select * from jsonb_array_elements_text('[1, true, null, "str\ttest", [1,2,3],
{"name1":"aaa\ttest","name2":"bbb\ttest"}]');
```

查询结果如下：

```
行号        VALUE
---------- ----------------------------------------
1           1
2           true
3           null
4           str test
```

```
5              [1,2,3]

6              {"name1":"aaa\ttest","name2":"bbb\ttest"}
```

其中，对于字符串“str\ttest”，字符串中的转义字符“\t”被改写成了对应的 tab 键。

18.2.2.6 jsonb_strip_nulls

当 object 对象 key/value 中的 value 为 NULL 时，jsonb_strip_nulls 忽略该 key/value，并返回处理后的 JSON 字符串。

语法格式

```
<jsonb_strip_nulls 函数> ::= jsonb_strip_nulls(<json_exp>)
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

JSONB 类型

使用说明

1. 当参数<json_exp>为 NULL 时，返回 NULL；
2. 该函数不会忽略除 OBJECT 类型外其他 JSON 数据类型中的 NULL 值。

举例说明

例 1 使用 jsonb_strip_nulls，忽略 value 为 NULL 的 key/value。

```
select jsonb_strip_nulls('[1, null, "str", [1,2,null], {"a":null}, {"b":2, "c":null}]') from dual;
```

查询结果如下：

```
行号       JSONB_STRIP_NULLS('[1,null,"str",[1,2,null],{"a":null},{"b":2,"c":null}]')

---------- ------------------------------------------------------------

1              [1,null,"str",[1,2,null],{},{"b":2}]
```

例 2 当参数<json_exp>为 NULL 时，返回 NULL。

```
select jsonb_strip_nulls(null) from dual;
```

查询结果如下:

```
行号              JSONB_STRIP_NULLS(NULL)

---------- ------------------------

1                  NULL
```

18.2.2.7 jsonb_set

jsonb_set 函数设置 JSONB 中给定路径下的值。

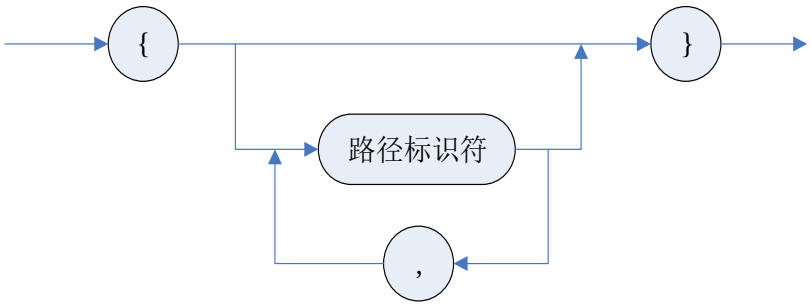
语法格式

```
<jsonb_set 函数> ::= jsonb_set(<jsonb_exp1>, <path_exp>, <jsonb_exp2>[, <exp>])
```

参数

<json_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

<path_exp>：路径表达式，以“{”开始，并以“}”结束。函数通过<path_exp>指定的路径在<jsonb_exp1>中查找相应的项并进行替换。该路径表达式的语法图如下:



“路径标识符”表示 array 数组的下标，或者 object 对象的 key/value 的 key。当“路径标识符”表示 array 数组的下标且为负整数时，表示从 array 数组的末尾开始计数，-1 表示数组末尾的第一位。路径表达式中可以包含多个“路径标识符”，以“,”分隔。

<jsonb_exp2>:表示新替换 JSONB 的字符串,数据类型为 JSON, JSONB, VARCHAR 或 CLOB。

<exp>：表示当<path_exp>指定的项不存在时，是否新增<path_exp>指定的项，并将值设为<jsonb_exp2>，取值为 TRUE（是）、FALSE（否），缺省为 TRUE。

返回值

设置完成后的数据，JSONB 类型

使用说明

1. 当参数<jsonb_exp1>为 NULL 时，返回 NULL；

2. 当参数<path_exp>的“路径标识符”对应项的 JSONB 类型是 ARRAY 时，该“路径标识符”必须为整数，否则报错；

3. 当参数<path_exp>的“路径标识符”对应项的 JSONB 类型是 OBJECT 时，该“路径标识符”无论是数值还是字符串都将作为 key/value 中的 key 处理；

4. 当 key/value 中 key 包含转义字符时，“路径标识符”描述该 key 时需要将转义字符改写为相应字符。例如 key 为“a\tb”，“路径标识符”需要写成“a　b”（中间为 tab 键），才能成功查找到指定项。

举例说明

例 1 使用 jsonb_set，替换 JSONB 数据中的一项。

1.替换为 NUMBER 类型

```
select jsonb_set('[{"f1":{"a":1}}]','{0,f1,a}','15');
```

查询结果如下：

```
行号        JSONB_SET('[{"f1":{"a":1}}]','{0,f1,a}','15')

---------- ----------------------------------------------

1           [{"f1":{"a":15}}]
```

2.替换为 STRING 类型

```
select jsonb_set('[{"f1":{"a":1}}]','{0,f1,a}','"str"');
```

查询结果如下：

```
行号            JSONB_SET('[{"f1":{"a":1}}]','{0,f1,a}','"str"')
---------- ------------------------------------------------
1               [{"f1":{"a":"str"}}]
```

3．<path_exp>指定项不存在，且<exp>为 TRUE，则<jsonb_exp1>新增一项

```
select jsonb_set('[{"f1":{"a":1}}]','{0,f1,b}','15',true);
```

查询结果如下：

```
行号            JSONB_SET('[{"f1":{"a":1}}]','{0,f1,b}','15',TRUE)
---------- ------------------------------------------------
1               [{"f1":{"a":1,"b":15}}]
```

4. <path_exp>指定项不存在，且<exp>为 FALSE，则<jsonb_exp1>保持不变

```
select jsonb_set('[{"f1":{"a":1}}]','{0,f1,b}','15',false);
```

查询结果如下：

```
行号            JSONB_SET('[{"f1":{"a":1}}]','{0,f1,b}','15',FALSE)
---------- -------------------------------------------------
1               [{"f1":{"a":1}}]
```

例 2 当参数<jsonb_exp1>为 NULL 时，返回 NULL。

```
select jsonb_set(null,'{0,f1,a}','15');
```

查询结果如下：

```
行号            JSONB_SET(NULL,'{0,f1,a}','15')
---------- -------------------------------
1               NULL
```

例 3 当参数<jsonb_exp1>对应的 JSONB 数据类型不是 OBJECT 或 ARRAY 时，报错。

```
select jsonb_set('1','{}','15');
```

查询结果报错:

```
[-3114]:无法在标量中设置路径.
```

18.2.2.8 jsonb_object_agg

jsonb_object_agg 将两个参数聚合成一个 object 对象。

语法格式

```
<jsonb_object_agg 函数> ::= jsonb_object_agg(<name>, <value>)
```

参数

<name>：作为 object 对象 key/value 键值对中的键 key，数据类型为 VARCHAR。

<value>：作为 object 对象 key/value 键值对中的值 value，支持的数据类型与 TO_JSONB 函数支持的数据类型相同。

返回值

聚合成的 object，数据类型为 JSONB。

使用说明

1. 参数<name>不能为 NULL，否则报错;

2. 如果参数<name>或<value>中包含转义字符，则函数将转义字符改写为“\”表示的转义字符。例如<value>为“a　b”（中间为 tab 键），则聚合时改写为“a\tb”。

举例说明

例 1 使用 jsonb_object_agg，将表中的两列聚合成 object 对象。

```
drop table test;

create table test(c1 varchar(20),c2 varchar(20));

insert into test values('a',1);

insert into test values('b','true');

insert into test values('c',null);
```

```
insert into test values('d','[1,2,3]');

insert into test values('e','str\ttest');

insert into test values('f','"str   test"');

insert into test values('str      test','{"name":"str     test"}');

select jsonb_object_agg(c1,c2) from test;
```

查询结果如下：

```
行号            JSONB_OBJECT_AGG(C1,C2)

---------- ------------------------------

1

{"a":"1","b":"true","c":null,"d":"[1,2,3]","e":"str\\ttest","f":"\"str\ttest\"","str\ttest":"{\"name\":\"str\ttest

\"}"}
```

可以看出，c1 和 c2 列中的“\”、“"”以及 tab 键聚合时分别被改写成了“\\”、“\"”以及“\t”。

例 2 参数<name>为 NULL，报错。

```
drop table test;

create table test(c1 varchar(20),c2 varchar(20));

insert into test values(null,1);

select jsonb_object_agg(c1,c2) from test;
```

查询结果报错：

```
[-3116]:字段名不能为空.
```

18.2.2.9 jsonb_concat

jsonb_concat 将两个 JSONB 数据合并成一个 JSONB 数据。

语法格式

```
<jsonb_concat 函数> ::= jsonb_concat(<jsonb_exp1>,<jsonb_exp2>)
```

参数

<json_exp1>：JSONB 类型的数据或表示合法 JSON 的字符串。

<json_exp2>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

JSONB 类型

使用说明

1. 当参数<jsonb_exp1>或<jsonb_exp2>为 NULL 时，则返回 NULL；
2. 当参数<jsonb_exp1>或<jsonb_exp2>不是 JSON 数据时，则报错；
3. 合并两个 OBJECT 时，返回 OBJECT，结果包含两个 OBJECT 的所有 key:value 对，有相同 key 时取第二个 OBJECT 中的 value，不递归合并，只处理最外层 key:value；
4. 合并两个 ARRAY 时，返回 ARRAY，结果包含两个 ARRAY 的所有元素；
5. ARRAY 与非 ARRAY 合并以及两个非 ARRAY 合并，将非 ARRAY 包装为单元素 ARRAY，然后当成两个 ARRAY 合并，返回 ARRAY。

举例说明

例 1 参数<jsonb_exp1>为 NULL 时，返回 NULL。

```
select jsonb_concat(null,'1');
```

查询结果如下：

```
行号       JSONB_CONCAT(NULL,'1')

---------- ----------------------

1             NULL
```

例 2 标量类型数据与标量类型数据的合并结果类型为 ARRAY。

```
select jsonb_concat('1','"abc"');
```

查询结果如下：

```
行号        JSONB_CONCAT('1','"abc"')

---------- --------------------------

1           [1,"abc"]
```

例 3 标量类型数据与 OBJECT 类型数据的合并结果类型为 ARRAY。

```
select jsonb_concat('1','{"b":2}');
```

查询结果如下：

```
行号          JSONB_CONCAT('1','{"b":2}')

---------- ----------------------------

1           [1,{"b":2}]
```

例 4 jsonb_concat 函数与 jsonb_object_agg 函数配合使用。

1. jsonb_concat 函数的结果作为 jsonb_object_agg 函数的参数

```
//jsonb_concat 函数的结果作为 jsonb_object_agg 函数的参数

select jsonb_object_agg('a',jsonb_concat('1', '2'));
```

查询结果如下：

```
行号          JSONB_OBJECT_AGG('a',JSONB_CONCAT('1','2'))

---------- ------------------------------------------

1           {"a":[1,2]}
```

2. 普通字符串作为 jsonb_object_agg 函数的参数

```
//普通字符串作为 jsonb_object_agg 函数的参数

select jsonb_object_agg('a','[1,2]');
```

查询结果如下：

```
行号          JSONB_OBJECT_AGG('a','[1,2]')

---------- ----------------------------
```

```
1               {"a":"[1,2]"}
```

可以看到，将 jsonb_concat 函数的结果作为 jsonb_object_agg 函数的<value>参数时，返回结果 object 对象 key/value 中的 value 没有双引号。

18.2.2.10 jsonb_build_object

jsonb_build_object 根据指定的 key 和 value 创建 object 对象。

语法格式

```
<jsonb_build_object 函数> ::= jsonb_build_object(<exp1>, <exp2> {, <exp3>, <exp4>})
```

参数

<exp1>：指定 key，数据类型为 VARCHAR。

<exp2>：指定 value，支持的数据类型与 TO_JSONB 函数支持的数据类型相同。

返回值

生成的 object，数据类型为 JSONB。

使用说明

支持指定多个 key 和 value，jsonb_build_object 参数的个数必须为偶数。

举例说明

例 根据表 TEST_JSONB_BUILD_OBJECT 中的各列数据创建 object 对象。

```
DROP TABLE TEST_JSONB_BUILD_OBJECT CASCADE;

CREATE TABLE TEST_JSONB_BUILD_OBJECT(NAME VARCHAR, AGE INT, CLASS VARCHAR);

INSERT INTO TEST_JSONB_BUILD_OBJECT VALUES('张三', 10, '一班');

INSERT INTO TEST_JSONB_BUILD_OBJECT VALUES('李四', 11, '一班');

INSERT INTO TEST_JSONB_BUILD_OBJECT VALUES('王五', 11, '二班');

INSERT INTO TEST_JSONB_BUILD_OBJECT VALUES('赵六', 9, '三班');

SELECT JSONB_BUILD_OBJECT('NAME', NAME, 'AGE', AGE, 'CLASS', CLASS) FROM
```

```
TEST_JSONB_BUILD_OBJECT;
```

查询结果如下：

```
行号        JSONB_BUILD_OBJECT('NAME',NAME,'AGE',AGE,'CLASS',"CLASS")

---------- ------------------------------------------------------------

1              {"AGE":10,"NAME":"张三","CLASS":"一班"}

2              {"AGE":11,"NAME":"李四","CLASS":"一班"}

3              {"AGE":11,"NAME":"王五","CLASS":"二班"}

4              {"AGE":9,"NAME":"赵六","CLASS":"三班"}
```

18.2.2.11 jsonb_agg

jsonb_agg 为集函数，将指定数据聚合成一个 array 数组。

语法格式

```
<jsonb_agg 函数> ::= jsonb_agg(<exp>)
```

参数

<exp>：指定数据，支持的数据类型与 TO_JSONB 函数支持的数据类型相同。

返回值

聚合成的 array，数据类型为 JSONB。

使用说明

1. 支持在<exp>参数前指定 DISTINCT 关键字，即 jsonb_agg(DISTINCT <exp>)，表示对<exp>进行去重操作。
2. 支持通过结合 GROUP BY 子句实现分组聚合，但不支持通过结合 ORDER BY 子句进行排序聚合。

举例说明

例 1 将表 TEST_JSONB_AGG 中的 NAME 列数据聚合成一个 array 数组。

```
DROP TABLE TEST_JSONB_AGG CASCADE;

CREATE TABLE TEST_JSONB_AGG(NAME VARCHAR, AGE INT, CLASS VARCHAR);

INSERT INTO TEST_JSONB_AGG VALUES('张三', 10, '一班');

INSERT INTO TEST_JSONB_AGG VALUES('李四', 11, '一班');

INSERT INTO TEST_JSONB_AGG VALUES('王五', 11, '二班');

INSERT INTO TEST_JSONB_AGG VALUES('赵六', 9, '三班');

SELECT JSONB_AGG(NAME) FROM TEST_JSONB_AGG;
```

查询结果如下：

```
行号        JSONB_AGG(NAME)

---------- ------------------------------------

1              ["张三","李四","王五","赵六"]
```

例 2 通过结合 GROUP BY 子句，将表 TEST_JSONB_AGG 中的 NAME 列数据按照 AGE 列分组聚合成三个 array 数组。

```
SELECT JSONB_AGG(NAME) FROM TEST_JSONB_AGG GROUP BY AGE;
```

查询结果如下：

```
行号          JSONB_AGG(NAME)

---------- ---------------

1             ["赵六"]

2              ["张三"]

3             ["王五","李四"]
```

例 3 将表 TEST_JSONB_AGG 中的 CLASS 列数据聚合成一个 array 数组，并通过指定 DISTINCT 关键字对 CLASS 列数据进行去重。

```
SELECT JSONB_AGG(DISTINCT CLASS) FROM TEST_JSONB_AGG;
```

查询结果如下：

```
行号        JSONB_AGG(DISTINCT"CLASS")

---------- ----------------------------

1              ["一班","二班","三班"]
```

例 4 jsonb_agg 和 jsonb_build_object 函数配合使用，将 jsonb_build_object 返回的 object 对象聚合成一个 array 数组。

```
SELECT JSONB_AGG(JSONB_BUILD_OBJECT('NAME', NAME, 'AGE', AGE, 'CLASS', CLASS)) FROM

TEST_JSONB_AGG;
```

查询结果如下：

```
行号        JSONB_AGG(JSONB_BUILD_OBJECT('NAME',NAME,'AGE',AGE,'CLASS',"CLASS"))

--------------------------------------------------------------------------------

1        [{"AGE":10,"NAME":"张三","CLASS":"一班"},{"AGE":11,"NAME":"李四","CLASS":"一班

"},{"AGE":11,"NAME":"王五","CLASS":"二班"},{"AGE":9,"NAME":"赵六","CLASS":"三班"}]
```

18.2.2.12 jsonb_build_array

json_build_array 从可变的参数列表（可以为空）中创建 array 数组并返回。

语法格式

```
< json_build_array 函数> ::= json_build_array (<exp>)
```

参数

<exp>：传入的参数列表。可以传入任意数量的参数，支持的数据类型与 TO_JSONB 函数支持的数据类型相同。

返回值

生成的 array，数据类型为 JSONB。

使用说明

jsonb_build_array 的用法以及示例与 json_build_array 一致，请参考 18.2.1.17

json_build_array。

18.2.2.13 jsonb_object_keys

jsonb_object_keys 返回顶层 object 对象 key 的集合。

语法格式

```
< jsonb_object_keys 函数> ::= JSONB_OBJECT_KEYS(<jsonb_exp>)
```

参数

<jsonb_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

字符串集合。

使用说明

1．此函数在查询中须位于 from 项。

2．参数表示的不是 object 对象时报错。

3．jsonb_object_keys 相比于 json_object_keys 会对 key 进行去重和排序。

举例说明

例 使用 jsonb_object_keys 返回 object 对象 key 的集合。会对结果的 key 进行去重和排序。

```
SELECT * FROM JSONB_OBJECT_KEYS('{"F2":"ABC","F1":{"F3":"A","F4":"B"},"F2":"D"}');
```

查询结果如下：

```
行号       JSONB_OBJECT_KEYS

---------- -----------------

1            F1

2             F2
```

其余示例与 json_object_keys 函数类似，可以参考 18.2.1.14 json_object_keys 小

节。

18.2.2.14 jsonb_typeof

jsonb_typeof 返回指定 JSONB 数据的类型。

语法格式

```
<jsonb_typeof 函数> ::= JSONB_TYPEOF(<json_exp>)
```

参数

<jsonb_exp>：JSONB 类型的数据或表示合法 JSON 的字符串。

返回值

返回<json_exp>对应的 JSON 类型。

使用说明

jsonb_typeof 的用法以及示例与 json_typeof 一致，请参考 18.2.1.20 json_typeof。

18.2.3 CAST

用户可使用 CAST 函数来实现 JSON/JSONB 数据和其他类型数据间的类型转换。

语法格式

```
CAST(value AS type)
```

其中，参数 type 可以是 JSON、JSONB 或其他数据类型，CAST 也可以写成等效的::运算符。语法如下：

```
value :: type
```

参数

value：指定参数。

type：待转换的类型。

返回值

返回<json_exp>对应的 JSON 类型。

使用说明

可以和 JSON/JSONB 类型数据进行类型转换的数据类型有：字符串类型(如 CHAR、VARCHAR)，CLOB 类型，数值类型(如 INTEGER、DECIMAL)，日期时间类型(如 DATE、TIME)。其中与字符串类型的互相转换不受限，与其他类型的转换受限，值得注意的是，INI 参数 JSON_MODE 的不同取值对应了不同策略。

举例说明

例 1 将其他类型数据转为 JSON/JSONB 数据类型时，数据需要是合法 JSON 格式，否则报错。

执行如下语句：

```
SQL> select cast('{"a":1}' as jsonb);
```

运行结果如下：

```
行号          CAST('{"a":1}'ASJSONB)
---------- ----------------------
1             {"a":1}
```

执行如下语句：

```
SQL> select cast('{"a":1' as jsonb);
```

运行结果如下：

```
select cast('{"a":1' as jsonb);
[-3105]:JSON 值语法错误.
```

例 2 JSON_MODE = 0 时，JSON 关键字对应服务器内部的 JSONB 类型。此时语句 CAST(value AS JSON)和 CAST(value AS JSONB)没有区别，将其他类型数据转为 JSONB 数据类型时，object 中不能有重复 key，否则报错。

执行如下语句：

```
SQL> select cast('{"a":1, "a":2}' as json);
```

运行结果如下：

```
select cast('{"a":1, "a":2}' as json);
[-3123]:JSON 对象中的键名重复.
```

执行如下语句：

```
SQL> select cast('{"a":1, "b":2}' as json);
```

运行结果如下：

```
行号        CAST('{"a":1,"b":2}'ASJSON)
---------- ----------------------------
1           {"a":1,"b":2}
```

例 3 JSON_MODE = 1 时，JSON 关键字对应服务器内部的 JSON 类型。此时执行将其他类型数据转为 JSON/JSONB 数据类型的操作，转为 JSON 数据类型时，对 object 中的 key 没有要求；转为 JSONB 数据类型时，要求 key 有序无重复，会自动对其进行排序去重，去重时保留相同 key 的最后一个 value。

执行如下语句：

```
SQL> select cast('{"a":1, "a":2}' as json);
```

运行结果如下：

```
行号        CAST('{"a":1,"a":2}'ASJSON)
---------- ----------------------------
1           {"a":1,"a":2}
```

执行如下语句：

```
SQL> select cast('{"a":1, "a":2}' as jsonb);
```

运行结果如下：

```
行号        CAST('{"a":1,"a":2}'ASJSONB)
---------- ----------------------------
1           {"a":2}
```

例 4 JSON_MODE = 2 时，JSON 关键字对应服务器内部的 JSONB 类型。此时语句 CAST(value AS JSON)和 CAST(value AS JSONB)没有区别，将其他类型数据转为 JSONB 数据类型时，要求 key 有序无重复，会自动对其进行排序去重，去重时保留相同 key 的最后一个 value。

执行如下语句：

```
SQL> select cast('{"a":1, "a":2}' as json);
```

运行结果如下：

```
行号        CAST('{"a":1,"a":2}'ASJSON)
---------- ----------------------------
1           {"a":2}
```

18.3 函数参数详解

18.3.1 路径表达式

JSON 数据的查询需要使用路径表达式。路径表达式为 object 和 array 范围。使用规则如下：

1. object 和 array 必须以“$”开始；
2. object 紧跟“.”，则表明对象是 object，且需要指定<key>；
3. object 的“.”后紧跟通配符“*”，则表示 object 的所有<key>；
4. <key>为字符串类型，需要遵循 18.1.1 string 中的使用约束；
5. array 紧跟“[”，以“]”结束，可以使用通配符“[*]”查找数组所有<value>；
6. array 索引可以是 0, 1, 2,...，起始值为 0；
7. array 中的范围必须非递减，例如：[3, 8 to 10, 12]、[1,2,3,3 to 3,3]；
8. 对于不是 array 的数据，$[0]表示本身。

例 1 使用通配符“[*]”，定位 ARRAY 所有元素。

```
select json_query('[0,1,2,3,4]', '$[*]' with wrapper);
```

查询结果如下：

```
行号        JSON_QUERY('[0,1,2,3,4]','$[*]'WITHWRAPPER)
---------- ------------------------------------------
1           [0,1,2,3,4]
```

例 2 ARRAY 下标范围定位多个元素。

```
select json_query('[0,1,2,3,4]', '$[0, 2 to 3]' with wrapper);
```

查询结果如下：

```
行号          JSON_QUERY('[0,1,2,3,4]','$[0,2to3]'WITHWRAPPER)

---------- --------------------------------------------------

1             [0,2,3]
```

例 3 通配符“*”，则表示 object 的所有<key>。

```
select json_query('{"a":1, "b":2}', '$.*[0]' with wrapper);
```

查询结果如下：

```
行号          JSON_QUERY('{"a":1,"b":2}','$.*[0]'WITHWRAPPER)

---------- --------------------------------------------------

1             [1,2]
```

18.3.2 PRETTY 和 ASCII

PRETTY 以缩进的形式显示字符，ASCII 以\uXXXX 十六进制的形式显示非 Unicode 字符，使用原则如下：

1. 默认为非 PRETTY；
2. 两者一起使用时，PRETTY 必须在 ASCII 之前；
3. json_value 和 json_query 都可以使用 ASCII；
4. 只有 json_query 可以使用 PRETTY。

举例说明

```
SET KEEPDATA ON

//返回结果中的换行符不用空格替代



//j_purchaseorder 表的创建语句以及数据插入语句请参考 18.8 应用实例
```

```
SELECT json_query(po_document,'$.ShippingInstructions'      RETURNING VARCHAR PRETTY
WITH WRAPPER ERROR ON ERROR) from j_purchaseorder;
```

查询结果如下:

```
json_query(PO_DOCUMENT,'$.ShippingInstructions'PRETTYWITHWRAPPERERROR
ONERROR)
---------- -----------------------------------------------------------
[
  {
    "name" : "Alexis Bull",
    "Address" :
    {
      "street" : "200 Sporting Green",
      "city" : "South San Francisco",
      "state" : "CA",
      "zipCode" : 99236,
      "country" : "United States of America"
    },
    "Phone" :
    [
      {
        "type" : "Office",
        "number" : "909-555-7307"
      },
      {
```

```
          "type" : "Mobile",
          "number" : "415-555-1234"
        }
      ]
    }
]
```

18.3.3 WRAPPER 项

只有 json_query 可以使用<WRAPPER 项>。缺省为 WITHOUT WRAPPER。

- WITH WRAPPER：以 array 的形式返回字符串，显示匹配路径表达式下的所有 JSON 数据，array 元素的顺序不是固定的；
- WITHOUT WRAPPER：只返回匹配路径表达式的单个 object 对象或 array 数组。如果是标量类型或多于 1 条数据则报错返回；
- WITH CONDITIONAL WRAPPER：单个 object 对象或 array 数组时，等价于 WITHOUT WRAPPER；其他情况等价于 WITH WRAPPER；
- WITH UNCONDITIONAL WRAPPER 和 WITH WRAPPER 是等价的；
- ARRAY 关键字可以省略，省略和不省略意义一样。

表 18.3.1 对比<WRAPPER 项>中不同组合情况

路径表达式	WITH WRAPPER	WITHOUT WRAPPER	WITH CONDITIONALWRAPPER
{"id": 38327}	[{"id": 38327}]	{"id": 38327}	{"id": 38327}
[42, "a", true]	[[42, "a", true]]	[42, "a", true]	[42, "a", true]
42	[42]	Error	[42]
42, "a", true	[42, "a", true]	Error	[42, "a", true]
none	[]	Error	[]

举例说明

使用 WITH WRAPPER 关键字，以数组的形式返回查询结果。

```
//j_purchaseorder 表的创建语句以及数据插入语句请参考 18.8 应用实例
```

```
SELECT json_query(po_document, '$.ShippingInstructions.Phone[*].type' WITH WRAPPER)
FROM j_purchaseorder;
```

查询结果如下：

```
JSON_QUERY(PO_DOCUMENT,'$.ShippingInstructions.Phone[*].type'WITHWRAPPER)
--------------------------------------------------------------------------
["Office","Mobile"]
```

18.3.4 ERROR 项

JSON 函数和条件表达式的错误处理。只有路径表达式语法正确时，ERROR 语句才有效。缺省为 NULL ON ERROR。

- ERROR ON ERROR：出错时则返回该错误；
- NULL ON ERROR：出错时返回 NULL；
- EMPTY ON ERROR：出错时返回[]；
- EMPTY ARRAY ON ERROR：出错时返回[]；
- EMPTY OBJECT ON ERROR：出错时返回{}；
- TRUE ON ERROR：出错时返回 TRUE 或 1；
- FALSE ON ERROR：出错时返回 FALSE 或 0；
- DEFAULT '<value>' ON ERROR：出错时返回指定的值，且<value>必须是字符串或者数值类型常量。布尔类型有两种表示方法：一是返回'true'或'false'（VARCHAR 类型），二是返回 1 或 0（NUMBER 类型）。

举例说明

展示<ERROR 项>的使用效果。

例 1 返回错误的信息

```
SELECT JSON_VALUE('[1,2]','$[0,1]' ERROR ON ERROR) FROM DUAL;
```

查询结果报错:

```
[-3107]:JSON_VALUE 求值为多个值
```

例 2 返回错误的信息

```
SELECT JSON_VALUE('[[1]]','$[0]' ERROR ON ERROR) FROM DUAL;
```

查询结果报错:

```
[-3106]:JSON_VALUE 的计算结果为非标量值
```

例 3 用 DEFAULT '<value>' ON ERROR 返回错误时指定的值，此例返回值类型与值不对应

```
SELECT JSON_VALUE('[1]','$[1]' RETURNING VARCHAR DEFAULT 1 ON ERROR) FROM DUAL;
```

json_mode 取值为 0 时，查询结果报错:

```
[-3109]:默认值不匹配在 RETURNING 子句中定义的类型
```

json_mode 取值为 2 时，默认<EMPTY 项>的 DEFAULT NULL ON EMPTY，且 DEFAULT 1 ON ERROR 中的默认值 1 成功转换为 RETURNING 中的 VARCHAR 类型，不会报错。查询结果输出如下:

```
JSON_VALUE('[1]','$[1]'RETURNINGVARCHAR2(32767)DEFAULT1ONERROR)

----------------------------------------------------------------

NULL
```

例 4 用 DEFAULT '<value>' ON ERROR 返回错误时指定的值

```
SELECT JSON_VALUE('[aa]','$[0]' RETURNING number default '1' on error) FROM DUAL;
```

查询结果如下:

```
JSON_VALUE('[aa]','$[0]'RETURNINGNUMBERDEFAULT'1'ONERROR)

-----------------------------------------------------------

1
```

18.4 运算符

JSON 运算符->、->>和@>为系统自定义运算符，新建库第一次启动服务器时会自动创建系统自定义运算符，用户也可调用系统过程 SP_CREATE_SYSTEM_OPERATORS()来创建或删除系统自定义运算符。

18.4.1 <json_exp> -> <exp>

<json_exp> -> <exp>获取 ARRAY 数组元素或者 OBJECT 对象指定 key 的 value。

语法格式

```
<json_exp> -> <exp>
```

参数

<json_exp>：待查找的 JSON 数据，数据类型为 JSON 或 JSONB。

<exp>：指定数组元素或对象 key。该参数的书写规则与 INI 参数 JSON_MODE 的取值有关，具体规则如下：

当 JSON_MODE=1 时，<exp>为 array 数组的索引号或 object 对象的 key。array 数组元素索引从 0 开始，数据类型为数值类型（如：INT、BIGINT、NUMBER）；object 对象的 key 为字符串，对应数据类型为 VARCHAR。

当 JSON_MODE=2 时，<exp>为路径表达式，请参考 18.3.1 路径表达式。

返回值

返回 array 数组元素或者 object 对象指定 key 的 value。返回值数据类型与参数<json_exp>的数据类型一致。

使用说明

1. 当参数<json_exp>为 NULL 时，返回 NULL；
2. 当前运算符仅支持 JSON_MODE 取值 1 或 2，当 JSON_MODE 取值为 0 时，将

报错。

举例说明

例 1 JSON_MODE=1，查询 array 数组元素。

```
SELECT '[1,true,"abc"]'::JSON->0;

--查询结果如下

行号         CAST('[1,true,"abc"]'ASJSON)OPERATOR("->")0

---------- -------------------------

1              1



SELECT '[1,true,"abc"]'::JSON->1;

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->")1

---------- -------------------------

1              true



SELECT '[1,true,"abc"]'::JSON->2;

--查询结果如下

行号       '[1,true,"abc"]'::JSON->2

---------- -------------------------

1              "abc"
```

例 2 JSON_MODE=1，查询 object 对象指定 key 的 value。

```
SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->'a';

--查询结果如下

行号         CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->")'a'
```

```
---------- ------------------------------------------------------

1            "bcd"



SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->'e';

--查询结果如下

行号       CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->")'e'

---------- ------------------------------------

1            [1,2,3]
```

例 3 JSON_MODE=2，查询 array 数组元素。

```
SELECT '[1,true,"abc"]'::JSON->'$[0]';

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->")'$[0]'

---------- -------------------------------------------------

1            1



SELECT '[1,true,"abc"]'::JSON->'$[1]';

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->")'$[1]'

---------- -------------------------------------------------

1            true



SELECT '[1,true,"abc"]'::JSON->'$[2]';

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->")'$[2]'
```

```
---------- ------------------------------------------------

1            "abc"
```

例 4 JSON_MODE=2，查询 object 对象指定 key 的 value。

```
SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->'$.a';

--查询结果如下

行号       CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->")'$.a'

---------- ---------------------------------------

1            "bcd"



SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->'$.e';

--查询结果如下

行　号                          CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->")'$.e'----------

-------------------------------------

1            [1,2,3]
```

18.4.2 <json_exp> ->> <exp>

<json_exp> ->> <exp>获取 ARRAY 数组元素或者 OBJECT 对象指定 key 的 value，并取消数组元素或 VALUE 外层的双引号。

语法格式

```
<json_exp> ->> <exp>
```

参数

<json_exp>：待查找的 JSON 数据，数据类型为 JSON 或 JSONB。

<exp>：指定数组元素或对象 key。该参数的书写规则与 INI 参数 JSON_MODE 的取值有关，具体规则如下：

当 JSON_MODE=1 时，<exp>为 array 数组的索引号或 object 对象的 key。array 数组元素索引从 0 开始，数据类型为数值类型（如：INT、BIGINT、NUMBER）；object 对象的 key 为字符串，对应数据类型为 VARCHAR。

当 JSON_MODE=2 时，<exp>为路径表达式，请参考 18.3.1 路径表达式。

返回值

返回 array 数组元素或者 object 对象指定 key 的 value。数据类型为 TEXT。

使用说明

1. 当参数<json_exp>为 NULL 时，返回 NULL；
2. 当前运算符仅支持 JSON_MODE 取值 1 或 2，当 JSON_MODE 取值为 0 时，将报错。

举例说明

例 1 JSON_MODE=1，查询 array 数组元素，取消数组元素外层的双引号。

```
SELECT '[1,true,"abc"]'::JSON->>0;

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->>")0

---------- --------------------------

1             1



SELECT '[1,true,"abc"]'::JSON->>1;

--查询结果如下

行号       CAST('[1,true,"abc"]'ASJSON)OPERATOR("->>")1

---------- --------------------------

1             true
```

```
SELECT '[1,true,"abc"]'::JSON->>2;

--查询结果如下

行号        CAST('[1,true,"abc"]'ASJSON)OPERATOR("->>")2

---------- --------------------------

1              abc
```

例 2 JSON_MODE=1，查询 object 对象指定 key 的 value，取消 value 外层的双引号。

```
SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->>'a';

--查询结果如下

行号        CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->>")'a'

---------- ---------------------------------------

1              bcd



SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->>'e';

--查询结果如下

行号        CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->>")'e'

---------- ---------------------------------------

1              [1,2,3]
```

例 3 JSON_MODE=2，查询 array 数组元素，取消数组元素外层的双引号。

```
SELECT '[1,true,"abc"]'::JSON->>'$[0]';

--查询结果如下

行号        CAST('[1,true,"abc"]'ASJSON)OPERATOR("->>")'$[0]'

---------- --------------------------------

1              1
```

```
SELECT '[1,true,"abc"]'::JSON->>'$[1]';
--查询结果如下
行号        '[1,true,"abc"]'::JSON->>'$[1]'
---------- --------------------------------
1                true


SELECT '[1,true,"abc"]'::JSON->>'$[2]';
--查询结果如下
行号         CAST('[1,true,"abc"]'ASJSON)OPERATOR("->>")'$[2]'
---------- --------------------------------
1                abc
```

例 4 JSON_MODE=2，查询 object 对象指定 key 的 value，取消 value 外层的双引号。

```
SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->>'$.a';
--查询结果如下
行号        CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->>")'$.a'
---------- ---------------------------------------
1                bcd


SELECT '{"a":"bcd","e":[1,2,3]}'::JSON->>'$.e';
--查询结果如下
行号        CAST('{"a":"bcd","e":[1,2,3]}'ASJSON)OPERATOR("->>")'$.e'
---------- ---------------------------------------
```

```
1               [1,2,3]
```

18.4.3 <json_exp> - <exp>

<json_exp> - <exp>删除 JSONB 数据中指定 key 的 key/value，或指定下标的数组元素。

语法格式

```
<json_exp> - <exp>
```

参数

<json_exp>：待处理的 JSONB 数据，数据类型为 JSONB

<exp>：当<json_exp>为 OBJECT 类型数据时，<exp>必须为字符串，用于指定待删除 key/value 中的 key；当<json_exp>为 ARRAY 类型数据时，<exp>必须为整数，用于指定待删除数组元素的下标。

返回值

删除指定 key/value 或数组元素后的 JSONB 数据。

使用说明

1. 若<json_exp>或<exp>为 NULL，则返回 NULL；
2. 若<json_exp>为 OBJECT 类型数据，且<exp>为整数，则报错；
3. 若 <json_exp> 为 ARRAY 类型数据，且 <exp> 为字符串，检查 TO_JSONB(<exp>)是否是 ARRAY 中的元素，如果是，删除该元素后返回，如果不是，返回<json_exp>。
4. 当前运算符仅支持 JSON_MODE 取值 1，当 JSON_MODE 取值为 0 或 2 时，将报错。

举例说明

例 1 参数<exp>为 NULL 时，返回 NULL。

```
select '{"a":1, "b":2}'::jsonb-null;
```

查询结果如下：

```
行号        CAST('{"a":1,"b":2}'ASJSONB)-NULL

---------- ----------------------------

1              NULL
```

例 2 删除指定 key 的 key/value。

```
select '{"a":1, "b":2}'::jsonb-'a';
```

查询结果如下：

```
行号        CAST('{"a":1,"b":2}'ASJSONB)-'a'

---------- --------------------------

1              {"b":2}
```

例 3 当<json_exp>为 OBJECT 类型数据，且<exp>为整数时，报错。

```
select '{"a":1, "b":2}'::jsonb-1;
```

查询结果报错：

```
[-3118]:只能从数组中提取元素.
```

例 4 删除指定下标的数组元素。

```
select '[1,{"a":1, "b":2},"c"]'::jsonb-1;
```

查询结果如下：

```
行号        '[1,{"a":1,"b":2},"c"]'::JSONB-1

---------- --------------------------------

1              [1,"c"]
```

例 5 当<json_exp>为 ARRAY 类型数据，且<exp>为字符串时，返回<json_exp> :: JSONB。

```
select '[1,{"a":1, "b":2},"c"]'::jsonb-'a';
```

查询结果如下：

```
行号       CAST('[1,{"a":1,"b":2},"c"]'ASJSONB)-'a'

---------- ----------------------------------

1              [1,{"a":1,"b":2},"c"]
```

18.4.4 <jsonb_exp1> @> <jsonb_exp2>

<jsonb_exp1> @> <jsonb_exp2>判断左边的 JSONB 值是否包含右边的 JSONB 值。

语法格式

```
<jsonb_exp1> @> <jsonb_exp2>
```

参数

<jsonb_exp1>：JSONB 或表示 JSONB 的字符串，字符串数据类型可为 JSON，JSONB，VARCHAR 或 CLOB。

<jsonb_exp2>：JSONB 或表示 JSONB 的字符串，字符串数据类型可为 JSON，JSONB，VARCHAR 或 CLOB。

返回值

返回值为数值类型，1 表示包含，0 表示不包含。

使用说明

1. 该运算符仅支持 JSONB 类型数据使用，不支持 JSON 类型数据。

2. 两个参数中如果有一个是 JSONB 类型，另一个参数也可以是字符串类型，它会被转化为 JSONB 类型。但不允许两个参数均为字符串类型。

举例说明

例 1 两参数均为 JSONB 类型。

```
SELECT '{"a":1, "b":2}'::JSONB @> '{"b":2}'::JSONB;
```

查询结果如下:

```
行号       CAST('{"a":1,"b":2}'ASJSONB)OPERATOR("@>")CAST('{"b":2}'ASJSONB)

---------- ----------------------------------------

1          1
```

例 2 一参数为 JSONB 类型，一参数为字符串，字符串被转化为 JSONB 类型。

```
SELECT '{"a":1, "b":2}'::JSONB @> '{"b":2, "b":2}';
```

查询结果如下（转化为 JSONB 去除了重复 key/value）:

```
行号          CAST('{"a":1,"b":2}'ASJSONB)OPERATOR("@>")'{"b":2,"b":2}'

---------- ----------------------------------------

1          1
```

例 3 不支持 JSON 类型。仅 json_mode 取值为 1 时，JSON 关键字对应 JSON 类型，所以本例中 json_mode=1。

```
SELECT '{"a":1, "b":2}'::JSONB @> '{"b":2}'::JSON;
```

查询结果报错:

```
SELECT '{"a":1, "b":2}'::JSONB @> '{"b":2}'::JSON;

第 1 行附近出现错误[-5403]:参数不兼容.
```

例 4 不允许两个参数均为字符串。

```
SELECT '{"a":1, "b":2}' @> '{"b":2}';
```

查询结果报错:

```
SELECT '{"a":1, "b":2}' @> '{"b":2}';

第 1 行附近出现错误[-5403]:参数不兼容.
```

18.4.5 <jsonb_exp1> || <jsonb_exp2>

<jsonb_exp1> || <jsonb_exp2>连接两个 JSONB。

语法格式

```
<jsonb_exp1> || <jsonb_exp2>
```

等价于

```
jsonb_concat(<jsonb_exp1>, <jsonb_exp2>)
```

参数

<jsonb_exp1>：待连接的 JSONB。

<jsonb_exp2>：待连接的 JSONB。

返回值

返回值为 JSONB 类型

使用说明

该运算符仅在 ini 参数 json_mode=1 时可用，连接规则与 jsonb_concat 函数相同，json_mode 取其他值时，对 JSONB 数据使用||运算符，会先将其转为字符串类型数据，再进行连接。

举例说明

```
SQL> select '[1,2]'::jsonb || '3'::jsonb;


行号          CAST('[1,2]'ASJSONB)||CAST('3'ASJSONB)
---------- ---------------------------------------
1             [1,2,3]
```

18.5 CHECK 约束

DM 支持使用 VARCHAR 或 CLOB 作为载体处理 JSON 和 JSONB 数据，通过对字符串添加 CHECK 约束来保证数据符合 JSON 或 JSONB 标准。

JSON 和 JSONB 的 CHECK 约束用法完全一致，本章节将以 JSON 为例进行说明。

CHECK 约束语法如下：

```
CHECK (<IS_JSON_clause>)
<IS_JSON_clause>::=
    IS [NOT] JSON [(STRICT | LAX)] [<unique_clause>]
<unique_clause>::=
    WITH UNIQUE KEYS |
    WITHOUT UNIQUE KEYS
```

参数

1. IS JSON/IS NOT JSON：用于保证字符串符合/不符合 JSON 标准。当对字符串使用该约束时，插入速度较慢。当能够保证 JSON 数据的合法性时，可以 DISABLE 该约束，建议不要 DROP 该约束。

2. STRICT/LAX 用于指定 JSON 数据格式。STRICT 数据格式比 LAX 要求更严格，二者对不同数据类型的具体要求请参考 18.1 数据类型。缺省为 LAX。

3. WITH UNIQUE KEYS/ WITHOUT UNIQUE KEYS：与 IS JSON 一起使用。具体使用规则如下：

1） 使用 WITH UNIQUE KEYS 时，对象中不可以有同名的 KEY，即 KEY 必须唯一。

2） 使用 WITHOUT UNIQUE KEYS 时，对象中可以有同名的 KEY，但是查询时只会随机选择其中一个，DM 默认选择第一个。

3） 缺省为使用 WITHOUT UNIQUE KEYS。

举例说明

例 1 在 CHECK 中使用 IS JSON，保证插入的数据符合 JSON 标准。

```
drop table json_is_json cascade;
CREATE TABLE json_is_json
(id int NOT NULL,
```

```
po_document CLOB
CONSTRAINT is_json_con CHECK (po_document IS JSON));
//创建成功
INSERT INTO json_is_json VALUES (1,'{"PONumber" : 1600, "PONumber" : 1800}');
//插入成功
INSERT INTO json_is_json VALUES (2,'OK');  //不是 JSON 数据，报错违反 CHECK 约束
INSERT INTO json_is_json VALUES (3,NULL);  //IS JSON 可以成功插入 NULL，IS NOT JSON 时也可以成功插入 NULL。
```

例 2 在插入语句中使用 IS JSON，保证从其他表中拷入的数据是符合 JSON 标准的。

```
drop table j_purchaseorder_insert;
//第一步：创建表
CREATE TABLE j_purchaseorder_insert
(c1 int NOT NULL,
c2 TIMESTAMP (6) WITH TIME ZONE,
c3 VARCHAR);
//第二步：插入数据
//j_purchaseorder 表的创建语句以及数据插入语句请参考 18.8 应用实例
INSERT INTO j_purchaseorder_insert select id, date_loaded, po_document from j_purchaseorder where po_document IS JSON;
```

例 3 在 CHECK 约束中使用 IS JSON WITH UNIQUE KEYS，保证插入的数据没有重复。

```
DROP TABLE json_unique CASCADE;
CREATE TABLE json_unique
(id int NOT NULL,
date_loaded TIMESTAMP (6) WITH TIME ZONE,
```

```
po_document CLOB
CONSTRAINT ensure_json_unique CHECK (po_document IS JSON WITH UNIQUE KEYS));
//创建成功
INSERT INTO json_unique VALUES (
111,SYSTIMESTAMP,'{"PONumber" : 1600, "PONumber" : 1800}');
```

执行结果报错:

```
[-6604]:违反 CHECK 约束条件（ENSURE_JSON_UNIQUE）
```

18.6 比较规则

当前支持根据 JSON_MODE 的取值来确定 JSON 和 JSONB 类型之间的比较规则。

18.6.1 JSON_MODE=1 时

当 JSON_MODE=1 时，支持 JSONB 类型之间使用比较符进行比较。要求比较符两边都是 JSONB 类型数据，才会使用该小节所述的比较规则，一边是 JSONB 数据另一边不是则会报错。

该比较规则的适用对象需要为 JSONB 类型，可以是 JSONB 类型列中的数据，可以是显式转为 JSONB 类型的数据，如 CAST(EXP AS JSONB);也可以是返回值为 JSONB 数据的函数的结果，如 JSONB_SET 函数的结果。

需要注意的是，有 CHECK IS JSON 约束的列数据不满足这一条件，IS JSON 的检查规则与 JSONB 数据类型要求并不完全相同。

比较规则:

1. 不同的 JSON 类型的大小比较顺序为: JSON Object > JSON Array > JSON Boolean（true>false） > JSON Number > JSON String > JSON Null;
2. JSON Object 之间，key/value 对数量决定大小关系，数量相等时，依次比较第

一个 key、第一个 value、第二个 key、第二个 value，以此类推；

3. JSON Array 之间，元素数量决定大小关系，当数量相等时，依次比较各个元素；

4. 同类型 JSON 标量间的比较：两个 JSON Null 相等；JSON Boolean 间 true > false；JSON Number 间比较数值大小；JSON String 间与字符串比较方法一致，且大小写敏感；

5. 特殊情况有：没有父对象的空 JSON Array '[]' 视作比较中的最小对象，但当其作为 JSON Object 中的 value 和 JSON Array 中的元素时按常规方式进行比较，例如：

CAST('NULL' AS JSONB) > CAST('[]' AS JSONB)

CAST('{"A": []}' AS JSONB) > CAST('{"A": NULL}' AS JSONB)

举例说明：

例

```
create table t_json_1(c1 jsonb);

insert into t_json_1 values('1');

insert into t_json_1 values('1.000');

insert into t_json_1 values('"a"');

insert into t_json_1 values('{"a":1}');

insert into t_json_1 values('[]');

insert into t_json_1 values('[1,2,3]');

insert into t_json_1 values('false');

insert into t_json_1 values('true');

insert into t_json_1 values('null');



select c1, json_type(c1) from t_json_1 where c1 > cast('null' as jsonb);

--查询结果如下
```

```
行号          C1         JSON_TYPE(C1)

---------- ------- -------------

1             1          INTEGER

2             1.000      DOUBLE

3             "a"        STRING

4             {"a":1}    OBJECT

5             [1,2,3]    ARRAY

6             false      BOOLEAN

7             true       BOOLEAN


select c1, json_type(c1) from t_json_1 where c1 = cast('null' as jsonb);

--查询结果如下

行号          C1     JSON_TYPE(C1)

---------- ---- -------------

1             null NULL


select c1, json_type(c1) from t_json_1 where c1 < cast('"b"' as jsonb);

--查询结果如下

行号          C1     JSON_TYPE(C1)

---------- ---- -------------

1             "a"    STRING

2             []     ARRAY

3             null NULL
```

18.6.2 JSON_MODE=2 时

JSON_MODE=2 时，支持 JSON 之间，JSON 与其他数据类型使用比较符进行比较。只要比较符的两边有一边是 JSON 数据就会使用该小节的比较规则。

该比较规则的适用对象需要为 JSON 类型，可以是 JSON 类型列中的数据，可以是显式转为 JSON 类型的数据，如 CAST(EXP AS JSON)；也可以是返回值为 JSON 数据的函数的结果，如 JSON_EXTRACT 函数的结果。

需要注意的是，有 CHECK IS JSON 约束的列数据不满足这一条件，IS JSON 的检查规则与 JSON_MODE=2 时的 JSON 数据类型要求并不完全相同。

JSON 之间的比较规则：

1. 不同的 JSON 类型的大小比较顺序为：JSON Boolean（true>false） > JSON Array > JSON Object > JSON String > JSON Number > JSON null

2. JSON Object 之间，key/value 对数量决定大小关系，数量相等时，依次比较第一个 key、第一个 value、第二个 key、第二个 value，以此类推；

3. JSON Array 之间，从前往后依次比较各个元素，若不相等即为结果，若相等则继续比较；元素都相等时由元素数量决定大小关系，仅元素数量和每个元素都相等时结果为相等；

4. 同类型 JSON 标量间比较：两个 JSON Null 相等，JSON Boolean 间 true > false，JSON Number 间比较数值大小，JSON String 间与字符串比较方法一致，且大小写敏感。

JSON 与其他数据类型的比较规则：

对于 JSON 与其他数据类型，会先将其他数据类型转化为对应的 JSON 类型，再使用上述规则比较。布尔类型数据转化为 JSON Boolean，字符串类型数据转化为 JSON String，数值类型转化为 JSON Number。需要注意的是 NULL 并不会转化为 JSON Null，当比较符任意一边是 NULL 时，处理策略与普通的数据比较时相同，比较结果为

UNKNOWN。

举例说明：

例

```
create table t_json_2(c1 varchar(50));

insert into t_json_2 values('1');

insert into t_json_2 values('1.000');

insert into t_json_2 values('"a"');

insert into t_json_2 values('{"a":1}');

insert into t_json_2 values('[]');

insert into t_json_2 values('[1,2,3]');

insert into t_json_2 values('false');

insert into t_json_2 values('true');

insert into t_json_2 values('null');



select c1, json_type(c1) from t_json_2 where cast(c1 as json) = 1.0;

--查询结果如下

行号          C1      JSON_TYPE(C1)

---------- ----- -------------

1               1       INTEGER

2               1.000 DOUBLE



select c1, json_type(c1) from t_json_2 where cast(c1 as json) < 'b';

--查询结果如下

行号          C1      JSON_TYPE(C1)
```

```
---------- ----- -------------

1              1       INTEGER

2              1.000 DOUBLE

3              "a"     STRING

4              null    NULL


select c1, json_type(c1) from t_json_2 where cast(c1 as json) > cast('{}' as json);

--查询结果如下

行号           C1         JSON_TYPE(C1)

---------- ------- -------------

1              {"a":1} OBJECT

2              []         ARRAY

3              [1,2,3] ARRAY

4              false     BOOLEAN

5              true      BOOLEAN
```

18.7 视图

JSON 数据信息都存储在 DBA_JSON_COLUMNS、USER_JSON_COLUMNS 和 ALL_JSON_COLUMNS 视图中，下面进行详细介绍。

18.7.1 视图使用说明

1. 当 JSON 列的 IS JSON 约束被失效后，该列仍然在视图中显示；

2. 当 IS JSON 涉及多列时，则所有涉及的列均在视图中显示。例如：c1||c2 is json，则 c1 和 c2 列均在视图中显示；

3. 如果 IS JSON 与其他约束进行"与"（AND）运算时，则所有涉及的列均在视图中显示。例如：c1 = '1' and c2 is json;

4. 如果 IS JSON 与其他约束进行"或"（OR）运算时，则所有列均不在视图中显示。例如：c1 is json OR c2 < 1000，即使是 c1 is json OR c2 is json 也不行;

5. 如果 IS NOT JSON 作为 CHECK 约束时，则该列不在视图中显示。例如：c1 is not json。同理：c1||c2 is not json，则 c1 和 c2 均不能在下列视图中显示;

6. 如果 NOT IS JSON 作为 CHECK 约束时，则该列也不能在下列视图中显示。例如：not (c2 is json)。

18.7.2 DBA_JSON_COLUMNS

显示数据库中所有的 JSON 数据信息。

表 18.7.1 数据库中所有的 JSON 数据信息

列名	数据类型	说明
OWNER	VARCHAR(128)	模式名
TABLE_NAME	VARCHAR(128)	表名
COLUMN_NAME	VARCHAR(128)	列名
FORMAT	VARCHAR(6)	格式化。取值由 DATA_TYPE 字段确定
DATA_TYPE	VARCHAR(11)	列的数据类型。可能的取值：VARCHAR2、CLOB、LONGVARCHAR、TEXT、JSON、JSONB、UNDEFINED(对于 CHAR 类型)

18.7.3 USER_JSON_COLUMNS

显示当前用户所拥有的 JSON 数据信息。该视图比 DBA_JSON_COLUMNS 视图少了一列 OWNER。

18.7.4 ALL_JSON_COLUMNS

显示当前用户有权访问的 JSON 数据信息。该视图列与 DBA_JSON_COLUMNS 完全相同。

18.8 应用实例

在数据库表中插入 JSON 数据，并查询它。

第一步，创建表并插入数据。

```
drop table j_purchaseorder;

//创建表

CREATE TABLE j_purchaseorder

(id int NOT NULL,

date_loaded TIMESTAMP (6) WITH TIME ZONE,

po_document JSON);



//插入数据。插入 po_document 列的就是 JSON 数据，也可以使用 dmfldr 工具导入数据。

INSERT INTO j_purchaseorder VALUES (1,SYSTIMESTAMP,'{

"PONumber" : 1600,

"Reference" : "ABULL-20140421",

"Requestor" : "Alexis Bull",

"User" : "ABULL",

"CostCenter" : "A50",

"ShippingInstructions" : {"name" : "Alexis Bull",

            "Address": {"street" : "200 Sporting Green",

            "city" : "South San Francisco",

            "state" : "CA",

            "zipCode" : 99236,

            "country" : "United States of America"},
```

```
            "Phone" : [{"type" : "Office", "number" : "909-555-7307"},
                                {"type" : "Mobile", "number" : "415-555-1234"}]},
"Special Instructions" : null,
"AllowPartialShipment" : true,
"LineItems" : [{"ItemNumber" : 1,
                        "Part" : {"Description" : "One Magic Christmas",
                                        "UnitPrice" : 19.95,
                                        "UPCCode" : 13131092899},
                        "Quantity" : 9.0},
                {"ItemNumber" : 2,
                "Part" : {"Description" : "Lethal Weapon",
                                "UnitPrice" : 19.95,
                                "UPCCode" : 85391628927},
                "Quantity" : 5.0}]}');
```

第二步，查询表中的数据。

```
SELECT json_query(po_document,'$.*'      RETURNING VARCHAR PRETTY WITH WRAPPER ERROR
ON ERROR) from j_purchaseorder;
```

查询结果如下：

```
行号   json_query(PO_DOCUMENT,'$.*'PRETTYWITHWRAPPERERRORONERROR)
--------------------------------------------------------------------------
1       [
  1600,
  "ABULL-20140421",
  "Alexis Bull",
```

```
"ABULL",
"A50",
{
   "name" : "Alexis Bull",
   "Address" :
   {
      "street" : "200 Sporting Green",
      "city" : "South San Francisco",
      "state" : "CA",
      "zipCode" : 99236,
      "country" : "United States of America"
   },
   "Phone" :
   [
      {
         "type" : "Office",
         "number" : "909-555-7307"
      },
      {
         "type" : "Mobile",
         "number" : "415-555-1234"
      }
   ]
},
```

```
null,

true,

  [

    {

      "ItemNumber" : 1,

      "Part" :

      {

        "Description" : "One Magic Christmas",

        "UnitPrice" : 19.95,

        "UPCCode" : 13131092899

      },

      "Quantity" : 9.0

    },

    {

      "ItemNumber" : 2,

      "Part" :

      {

        "Description" : "Lethal Weapon",

        "UnitPrice" : 19.95,

        "UPCCode" : 85391628927

      },

      "Quantity" : 5.0

    }

  ]
```

```
]
```

第 19 章 高级日志

19.1简介

行表和 HUGE 表在增删改查性能上存在差异，因此在实际的生产环境中，用户可能会同时使用一个行表来管理数据和一个 HUGE 表来分析数据。具体做法是对行表进行增删改操作，然后把行表中的数据复制到 HUGE 表中用于查询或分析。如果每次分析数据时都对行表进行全表查询插入 HUGE 表，性能较低。

为此提出一种解决方案：给行表添加日志辅助表用于记录行表的增删改和 TRUNCATE 操作，可以根据日志表实现对 HUGE 表的增量更新，以此来提高从行表复制数据到 HUGE 表的性能。

19.2使用须知

增量更新过程，我们只提供日志的记录以及日志记录规则的制定，真正执行增量更新是由用户根据日志记录自行操作。辅助表中登记信息，为某一时间点后源表数据的增量变化信息登记。

19.3语法

19.3.1管理日志辅助表

19.3.1.1 创建日志辅助表

创建日志辅助表，有两种方式：一是建表时创建；二是修改表时创建。

1. 建表时候使用<高级日志子句>创建日志辅助表

语法格式

```
CREATE  TABLE <表名定义> <表结构定义>;
```

<表名定义> ::= [<模式名>.] <表名>

<表结构定义>::=<表结构定义 1> | <表结构定义 2>

<表结构定义 1>::= (<列定义> {,<列定义>} [,<表级约束定义>{,<表级约束定义>}]) [ON COMMIT <DELETE | PRESERVE> ROWS] [<PARTITION 子句>][<空间限制子句>][<表空间子句>] [<STORAGE 子句>][<压缩子句>] [<ROW MOVEMENT 子句>][**<高级日志子句>**] [<add_log 子句>] [<DISTRIBUTE 子句>]

……

<高级日志子句>::= WITH ADVANCED LOG

省略号（……）部分请参考《DM8 Sql 语言使用手册》3.5.1.1 定义数据库基表

2. 修改表时使用<高级日志子句>添加日志表

语法格式

```
ALTER TABLE <TABLE_NAME> <高级日志子句>;
```

19.3.1.2 删除日志辅助表

语法格式

```
ALTER TABLE <TABLE_NAME> WITHOUT ADVANCED LOG;
```

19.3.1.3 删除日志辅助表的数据

语法格式

```
ALTER TABLE <TABLE_NAME> TRUNCATE ADVANCED LOG;
```

数据清除后可能导致源表和 HUGE 无法同步，需慎重操作。

19.3.2使用日志辅助表的规则与约束

日志辅助表命名为“表名$ALOG”，用于记录源表的操作但不涉及具体数据。

规则与约束：

1. 每个源表仅支持设置一个日志辅助表。

2. 表删除的同时删除其日志辅助表。

3. 表更名时，日志表同步更名。

4. 由于其日志表名长度不得超过 128，因此表名长度不得超过 123。

5. 辅助表仅登记源表相关增删改及 TRUNCATE 等涉及数据变化的操作，却不涉及具体数据。

6. 源表执行 ADD [COLUMN]…/DROP [COLUMN]…/MODIFY <列定义>的 DDL 操作时，也必须保证日志辅助表为空。

7. 如果表设置了高级日志功能，禁止或者不建议以下操作：

   1） 禁止对源表创建聚集索引

   2） 禁止删除源表上本存在的聚集索引

   3） 禁止直接对分区表的子表执行 DELETE、UPDATE、INSERT 以及 TRUNCATE

   4） 禁止在 ALTER TABLE 时，新建、删除或者修改主键，使主键失效或者生效，或者删除主键列

   5） 禁止对临时表、HUGE 表和间隔分区表设置高级日志表，禁止查询插入建表方式设置高级日志表。

   6） 禁止直接删除高级日志表以及创建后缀为"$ALOG"的表

   7） 禁止合并分区

   8） 禁止对表加列、删除列和修改列，禁止添加、分裂、交换和删除分区。交换分区时的普通表也禁止带有高级日志

   9） 表备份还原后无法控制数据跟踪，无法保证同步数据的正确性。因此不建议对该表进行备份还原操作，或操作后需要人工干预处理

19.3.3日志辅助表结构

高级日志辅助表"表名$ALOG"的结构如下：

表 19.3.1 高级日志辅助表结构

序号	列	数据类型	说明
1	ORG_ROWID	ROWID	源表 ROWID。当 OP_TYPE=0 时，ORG_ROWID='AAAAAAAAAAAAAAAAAA'
2	OP_TYPE	SMALLINT	登记记录日志动作。 0:TRUNCATE 1:行插入 2:批量插入起始 3:批量插入结束 4:更新 5:删除 6:删除后再插入(仅用于堆表)
3	COLMAP	VARBINARY(2048)	当 OP_TYPE=3 时，记录的是批量插入结束的 ROWID； 当 OP_TYPE=4 时，是记录的更新列的列号。例如 0xA3，即二进制的 10100011，表示更新的列为第 1、2、6、8 列，与 DM_BIT_TEST()配合使用； 其他情况为 null
4	COL_0	与源表的第一个主键列类型相同	源表的第一个主键列
5	COL_1	与源表的第二个主键列类型相同	源表的第二个主键列
6	Col_n	...	...

19.3.4系统过程

高级日志辅助表中的 COLMAP 列记录的数据，用&操作只能获取前 64 列的更新情况，因为会数据溢出。

增加系统过程 DM_BIT_TEST()用于获取一个 VARBINARY 数据的第 N 位的数值。

语法格式

```
DM_BIT_TEST(DATA   varbinary, nth   int);
```

功能：返回二进制数据 varbinary 第 nth 位是 0 还是 1(最低位序号为 1)。如果超过了位数则返回 0。

例 0xF1 转为二进制后为 11110001，从低位开始第 5 位为 1。二进制 1011 从低位开始第三位为 0。

```
SELECT DM_BIT_TEST(0xF1,5),DM_BIT_TEST(1011,3);
```

查询结果如下：

```
行号          DM_BIT_TEST(0xF1,5) DM_BIT_TEST(1011,3)

---------- ------------------- ------------------

  1                1                           0
```

19.4使用高级日志同步数据的原则

用户根据表定义创建数据同步的目标表，自己编写同步 DMSQL 脚本来进行同步。对于同步，建议遵守如下的原则：

1. 如果源表有主键，如果用户没有特殊的限制或要求，目标表最好也设置同样的主键。
2. 如果源表没有主键，为了准确同步，最好在目标表上添加一个辅助同步的主键列，同步时将 org_rowid 列的值插入该列中。
3. 用户同步数据的脚本基本逻辑如下：

```
declare
    //遍历日志表的游标
    cursor c IS select * from t01$alog for update;
    //同步用的变量
    r t01$alog %rowtype;
    //同步批量插入用的变量
    bi_start t01$alog %rowtype;
    org_rec t01%rowtype;
begin
  //遍历日志表，根据各记录的 op_type 进行同步
  open c;
  loop
      fetch c into r;
      exit when c%notfound;
        if (r.op_type = 0) then
            print 'truncate' ;
            execute immediate 'truncate table t01';
        elseif (r.op_type = 1 or r.op_type = 6) then
            print 'insert ' || r.org_rowid;
            execute immediate 'insert ....'
        elseif (r.op_type = 2) then
            bi_start = r;
            print 'batch insert start';
        elseif (r.op_type = 3) then
```

```
            print 'batch insert last ' || bi_start.org_rowid || ' ' || cast( r.colmap as bigint);
            execute immediate 'insert ....'
        elseif (r.op_type = 4) then
            print 'update ' || r.org_rowid;
    select * into org_rec from t01 where ……;
            execute immediate 'update ....' using bi_start… r…;
        elseif (r.op_type = 5) then
            print 'delete '   || r.org_rowid;
            execute immediate 'delete ....'
        end if;
    end loop;
    close c;
    //清理日志表
    execute immediate 'alter table t01 truncate advanced log';
end;
/
```

4. 如果在数据同步时源表仍有并发的 DML，脚本中查询日志时要使用 for update 子句。

5. 同步脚本根据源表的结构有所不同：

1）如果源表有聚集主键

在同步时可使用日志辅助表中的 org_rowid 和主键列辅助源表定位。使用主键列定位目标表。

2） 如果源表有主键，但不是聚集主键

直接根据 org_rowid 定位数据，最好不要使用主键列来定位源表。主键列仅用来定位

目标表。

如果该情况下更新了主键列，对于聚集主键，将是删除后更新，如果不是聚集主键，仍是记录更新，日志辅助表中的主键列仍是原值，所以非聚集主键时主键列不要用来定位源表。

3） 如果没有主键

使用 org_rowid 来进行源表的定位；目标表的定义根据用户自己的方式使用 org_rowid 定位。

6. 如果源表中没有聚集索引，批量插入时可以根据 OP_TYPE=3 时的 org_rowid(批量插入起始 ROWID)和 COLMAP 中的数据(批量插入结束 ROWID)范围查询源表插入目标表；如果有聚集索引，考虑到组合索引无法进行范围查询，只能使用第一个主键和 rowid 进行范围查询。

7. MPP 环境下，因为高级日志表是本地表，所以同步数据的时候，只能各个节点单独做同步。

19.5应用实例

19.5.1创建不带主键的源表

1. 创建源表

```
CREATE TABLE T01(A INT, B INT, C VARCHAR);
INSERT INTO T01 VALUES(88,88, '原始数据 1');
INSERT INTO T01 VALUES(99,99, '原始数据 2');
```

2. 在源表上创建日志辅助表

```
ALTER TABLE T01  WITH ADVANCED LOG;
```

3. 查看日志辅助表结构

```
CALL SP_TABLEDEF('SYSDBA','T01$ALOG');
```

调用结果如下：

```
COLUMN_VALUE

------------------------------------------------------------------------

CREATE TABLE "SYSDBA"."T01$ALOG"

(

"ORG_ROWID" ROWID NOT NULL,

"OP_TYPE" SMALLINT NOT NULL,

"COLMAP" VARBINARY(2048),

CLUSTER PRIMARY KEY("ORG_ROWID", "OP_TYPE")) STORAGE(ON "MAIN", CLUSTERBTR);
```

4. 在源表中删除 1 行数据。

```
DELETE FROM T01 WHERE A=88;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE      COLMAP

------------------ ----------- ----------

AAAAAAAAAAAAAAAAAB 5               NULL
```

5. 在源表中更新 1 行数据。

```
UPDATE T01 SET C='HELLO WORLD' WHERE A=99;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE      COLMAP

------------------ ----------- ----------

AAAAAAAAAAAAAAAAAB 5               NULL

AAAAAAAAAAAAAAAAAC 4               0x04
```

6. 在源表中再次更新同 1 行数据。这一操作在日志表中没有记录。因为将源表上一条（99,99,'原始数据 2'）的数据更新为（99,99,'hello world'）之后，又再次更新为（99,99,'hello world!'）。这两步更新操作的最终结果就和直接更新为（99,99,'hello world!'）一样，所以两步操作只有一条记录。

```
UPDATE T01 SET C='HELLO WORLD!' WHERE A=99;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE       COLMAP

------------------ ----------- ----------

AAAAAAAAAAAAAAAAAB 5               NULL

AAAAAAAAAAAAAAAAAC 4               0x04
```

7. 先清空源表数据，再查看日志辅助表的变化。发现日志辅助表中也清空了之前的记录，只记录下了清空源表的操作。

```
TRUNCATE TABLE T01;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE       COLMAP

------------------ ----------- ----------

AAAAAAAAAAAAAAAAAA 0               NULL
```

8. 在源表中批量插入 100 行数据。单机情况下，大于 100 条才叫批量插入。

```
INSERT INTO T01 SELECT LEVEL A,LEVEL+1 B,LEVEL C CONNECT BY LEVEL<=100 ORDER BY A,B;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE       COLMAP
```

```
------------------ ----------- --------------------------

AAAAAAAAAAAAAAAAAA 0                NULL

AAAAAAAAAAAAAAAAAB 2                NULL

AAAAAAAAAAAAAAAAAB 3                0x000000000000000000000064
```

9. 在源表中插入 1 行数据。

```
INSERT INTO T01 VALUES(1001,1002,1003);

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID               OP_TYPE       COLMAP

------------------ ----------- --------------------------

AAAAAAAAAAAAAAAAAA 0                NULL

AAAAAAAAAAAAAAAAAB 2                NULL

AAAAAAAAAAAAAAAAAB 3                0x000000000000000000000064

AAAAAAAAAAAAAAAAABl 1               NULL
```

10. 同步数据

创建 huge 表。因为不带主键，为了准确同步，在目标表 huge_t01 上添加一个辅助同步的主键列 c_rowid，同步时将 org_rowid 列的值插入该列中

```
CREATE HUGE TABLE HUGE_T01 (C_ROWID ROWID, A INT, B INT, C VARCHAR(1024));
```

运行同步脚本。同步脚本由用户根据实际情况自行编写。本例中脚本如下：

```
declare

    //遍历日志表的游标

    cursor c IS select * from t01$alog for update;

    //同步用的变量

    r t01$alog %rowtype;
```

```
    //同步批量插入用的变量
    bi_start t01$alog %rowtype;
    set_sql      varchar;
    upd_sql      varchar;
    i   int;
begin
  //遍历日志表，根据各记录的 op_type 进行同步
  open c;
  loop
      fetch c into r;
      exit when c%notfound;
        if (r.op_type = 0) then
           print 'truncate' ;
           execute immediate 'truncate table huge_t01;';
        elseif (r.op_type = 1 or r.op_type = 6) then
           print 'insert ' || r.org_rowid;
           execute immediate 'insert into huge_t01 select rowid,* from t01 where rowid=?;' using
r.org_rowid;
        elseif (r.op_type = 2) then
           bi_start = r;
           print 'batch insert start';
        elseif (r.op_type = 3) then
           print 'batch insert last ' || bi_start.org_rowid || ' ' || cast( r.colmap as bigint);
           execute immediate 'insert into huge_t01 select rowid,* from t01 where rowid>= ? and
```

```
rowid<= ?;' using r.org_rowid, sf_build_rowid(1,1,cast(r.colmap as bigint));
        elseif (r.op_type = 4) then
          print 'update ' || r.org_rowid;
          set_sql = '';
          i       = 0;
          if (dm_bit_test(r.colmap,1)) = 1 then set_sql = set_sql || 'a = org.a'; i = i+1; end if;
          if (dm_bit_test(r.colmap,2)) = 1 then  if i > 0 then set_sql = set_sql ||','; end if;
set_sql = set_sql || 'b = org.b'; i = i+1; end if;
          if (dm_bit_test(r.colmap,3)) = 1 then  if i > 0 then set_sql = set_sql ||','; end if;
set_sql = set_sql || 'c = org.c'; i = i+1; end if;
          upd_sql = 'declare org t01%rowtype; begin select * into org from t01 where rowid=?;
update huge_t01 set ' || set_sql || ' where c_rowid=?; end;';
          execute immediate upd_sql using r.org_rowid, r.org_rowid;
        elseif (r.op_type = 5) then
          print 'delete '  || r.org_rowid;
          execute immediate 'delete from huge_t01 where c_rowid=?;' using r.org_rowid;
        end if;
  end loop;
  close c;
  //清理日志表
  execute immediate 'alter table t01 truncate advanced log';
end;
/
```

11. 查询 huge 表中的数据。可以看出，huge_t01 上的数据都是源表创建了日志辅助

表之后的增量数据。

```
SELECT COUNT(*) FROM HUGE_T01;
```

查询结果如下：

```
COUNT(*)

----------

101
```

19.5.2创建带主键的源表

1. 创建带有日志辅助表的源表

```
CREATE TABLE T01(A INT, B INT, C VARCHAR, PRIMARY KEY(A,B)) WITH ADVANCED LOG;
```

2. 查看日志辅助表结构

```
CALL SP_TABLEDEF('SYSDBA','T01$ALOG');
```

调用结果如下：

```
行号          COLUMN_VALUE

-------- ----------------------------------------------------------------

1             CREATE TABLE "SYSDBA"."T01$ALOG"  (  "ORG_ROWID" BIGINT NOT NULL,  "OP_TYPE"

SMALLINT NOT NULL,  "COLMAP" VARBINARY(2048),  "COL_0" INTEGER,  "COL_1" INTEGER,

CLUSTER PRIMARY KEY("ORG_ROWID", "OP_TYPE")) STORAGE(ON "MAIN", CLUSTERBTR) ;
```

3. 清空源表

```
TRUNCATE TABLE T01;

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID            OP_TYPE      COLMAP       COL_0        COL_1

------------------ ----------- ---------- ----------- -----------
```

```
AAAAAAAAAAAAAAAAA 0              NULL          NULL          NULL
```

4. 在源表中插入一条记录

```
INSERT INTO T01 VALUES(1001,1002,1003);

SELECT * FROM T01$ALOG;
```

查询结果如下：

```
ORG_ROWID              OP_TYPE       COLMAP       COL_0          COL_1

------------------ ----------- ---------- ----------- -----------

AAAAAAAAAAAAAAAAA 0              NULL          NULL          NULL

AAAAAAAAAAAAAAAAB 1              NULL          1001          1002
```

5. 同步数据

创建 huge 表。

```
CREATE HUGE TABLE HUGE_T01 (A INT, B INT, C VARCHAR(1024), PRIMARY KEY(A,B));
```

运行同步脚本。同步脚本由用户根据实际情况自行编写。本例中脚本如下：

```
declare
    //遍历日志表的游标
    cursor c IS select * from t01$alog for update;
    //同步用的变量
    r t01$alog %rowtype;
    //同步批量插入用的变量
    bi_start t01$alog %rowtype;
    set_sql      varchar;
    upd_sql      varchar;
    i   int;
begin
```

```
  //遍历日志表，根据各记录的 op_type 进行同步
  open c;
  loop
      fetch c into r;
      exit when c%notfound;
        if (r.op_type = 0) then
           print 'truncate' ;
           execute immediate 'truncate table huge_t01;';
        elseif (r.op_type = 1 or r.op_type = 6) then
           print 'insert ' || r.org_rowid;
           execute immediate 'insert into huge_t01 select * from t01 where rowid=?;' using
r.org_rowid;
        elseif (r.op_type = 2) then
           bi_start = r;
           print 'batch insert start';
        elseif (r.op_type = 3) then
           print 'batch insert last ' || bi_start.org_rowid || ' ' || cast( r.colmap as bigint);
           execute immediate 'insert into huge_t01 select * from t01 where rowid>= ? and
rowid<= ?;' using r.org_rowid, sf_build_rowid(1,1,cast(r.colmap as bigint));    elseif
(r.op_type = 4) then
           print 'update ' || r.org_rowid;
           set_sql = '';
           i = 0;
           if (dm_bit_test(r.colmap,1)) = 1 then set_sql = set_sql || 'a = org.a'; i = i+1; end if;
```

```
            if (dm_bit_test(r.colmap,2)) = 1 then  if i > 0 then set_sql = set_sql ||','; end if;
set_sql = set_sql || 'b = org.b'; i = i+1; end if;
            if (dm_bit_test(r.colmap,3)) = 1 then  if i > 0 then set_sql = set_sql ||','; end if;
set_sql = set_sql || 'c = org.c'; i = i+1; end if;
            upd_sql = 'declare org t01%rowtype; begin select * into org from t01 where rowid=?;
update huge_t01 set ' || set_sql || ' where a = ? and b = ?; end;';
            execute immediate upd_sql using r.org_rowid, r.col_0, r.col_1;
         elseif (r.op_type = 5) then
            print 'delete '  || r.org_rowid;
            execute immediate 'delete from huge_t01 where a= ? and b = ?;' using r.col_0, r.col_1;
         end if;
   end loop;
   close c;
   //清理日志表
   execute immediate 'alter table t01 truncate advanced log';
end;
/
```

6. 查询 huge 表中的数据。可以看出，huge_t01 上的数据都是源表创建了日志辅助表之后的增量数据。

```
SELECT * FROM HUGE_T01;
```

查询结果如下：

```
A           B           C
----------- ----------- ----
1001        1002        1003
```

第 20 章 自定义运算符

DM 支持用户自定义运算符。创建自定义运算符前需要首先创建一个存储函数，存储函数中需要包含输入参数、运算过程以及返回值，然后便可为该存储函数定义一个运算符。使用自定义运算符时，运算对象将作为存储函数的输入参数，参与存储函数中的运算过程，存储函数的返回值即运算结果。

20.1 创建自定义运算符

语法格式

```
CREATE OPERATOR [<模式名>.]<运算符名> (FUNCTION [<模式名>.]<函数名>, <运算对象类型>);

<运算对象类型>::=

    LEFTARG <左侧对象类型> [, RIGHTARG <右侧对象类型>] |

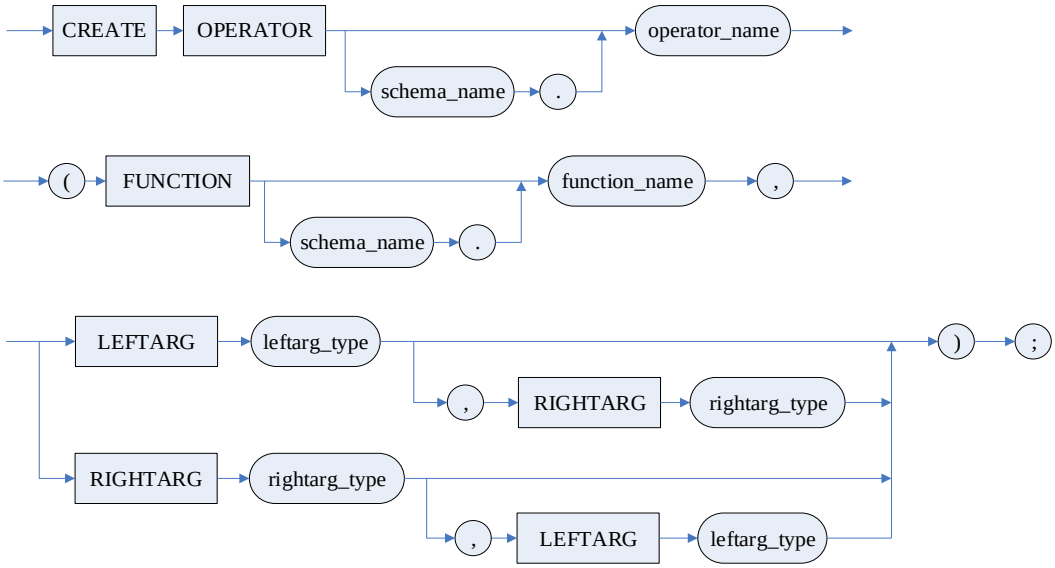
    RIGHTARG <右侧对象类型> [, LEFTARG <左侧对象类型>]
```

参数

1. <运算符名> 运算符的名称，长度为 2~63 个字符；
2. <函数名> 存储函数的名称；
3. <左侧对象类型> 运算符左侧运算对象的数据类型，为 NULL 表示不存在左侧运算对象；
4. <右侧对象类型> 运算符右侧运算对象的数据类型，为 NULL 表示不存在右侧运算对象。

图例

创建自定义运算符



语句功能

创建自定义运算符。

使用说明

1. 只有拥有 CREATE OPERATOR 或 CREATE ANY OPERATOR 权限的用户可以创建自定义运算符。只有拥有 CREATE ANY OPERATOR 的用户才能在其他模式下创建自定义运算符；

2. 自定义运算符至少存在一侧运算对象；

3. 运算符名称必须由以下字符组成：“+”、“-”、“*”、“/”、“<”、“>”、“=”、“~”、“!”、“@”、“%”、“^”、“&”、“|”和“`”。其中，“!”不能作为起始字符，“+”、“-”、“~”和“@”不能作为结束字符；

4. 运算符名称中不允许使用“--”、“/*”和“//”，因为会被识别为注释；

5. 开启 MYSQL 兼容，"`"反引号不能作为运算符名称的字符；

6. 运算对象的数据类型支持基本数据类型和自定义类型。不支持%TYPE 和%ROWTYPE 类型、不支持 INTERVAL 类型、不支持指定精度；

7. 部分不同名称的数据类型会被视为同一类型，例如 NUMBER 与 DECIMAL 被视为同一类型，CHAR 和 VARCHAR 不会被视为同一类型；

8. 支持重载自定义运算符，即支持创建<函数名>相同，但<运算对象类型>不同的多个同名的自定义运算符。其中，<运算对象类型>不同指实际数据类型不同，若仅类型名称不同但实际数据类型相同，则报错；

9. 只有拥有 CREATE ANY OPERATOR 权限的用户可以为其他用户创建的运算符创建重载对象；

10. 用户可以查询系统表 SYSOPARGS 获取自定义运算符的重载信息；

20.2 删除自定义运算符

语法格式

```
DROP OPERATOR [IF EXISTS] [<模式名>.]<运算符名> (<左侧对象类型>, <右侧对象类型>);
```

参数

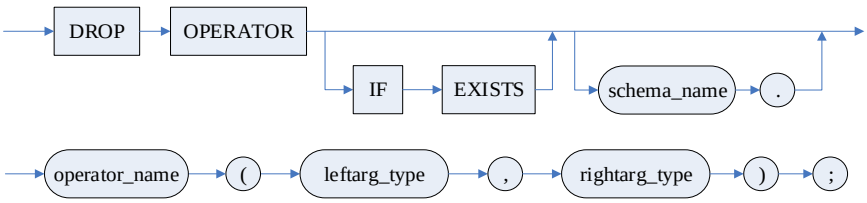
1. <运算符名> 运算符的名称；

2. <左侧对象类型> 运算符左侧运算对象的数据类型，为 NULL 表示不存在左侧运算对象；

3. <右侧对象类型> 运算符右侧运算对象的数据类型，为 NULL 表示不存在右侧运算对象。

图例

删除自定义运算符



语句功能

删除自定义运算符。

使用说明

只有拥有 DROP OPERATOR 和 DROP ANY OPERATOR 权限的用户可删除自己模式下的自定义运算符。只有拥有 DROP ANY OPERATOR 权限的用户才可以删除其他用户模式下的自定义运算符。

20.3 使用自定义运算符

语法格式

```
左一元运算符:  (<left_exp> <运算符名称子句>)

右一元运算符:  <运算符名称子句> <right_exp>

二元运算符:  <left_exp> <运算符名称子句> <right_exp>

<运算符名称子句>::=

        <运算符名> |

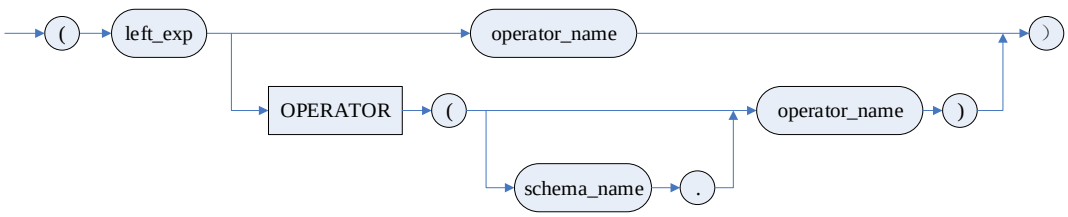
        OPERATOR ([<模式名>.]<运算符名>)
```

参数

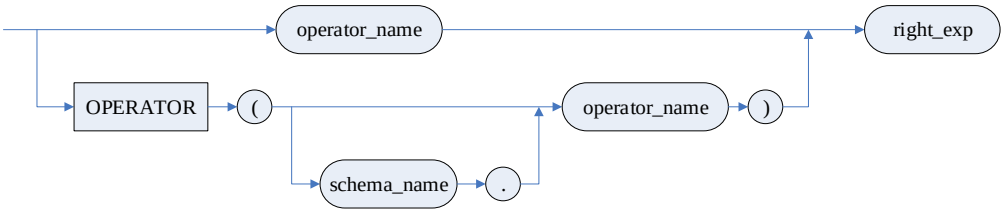
1. <left_exp> 左侧运算对象表达式;
2. <right_exp> 右侧运算对象表达式;
3. <运算符名称子句> 可以直接使用运算符名称，也可使用 OPERATOR 子句。若使用 OPERATOR 子句，则指定运算符名称时可以带有模式名或双引号，否则不能带有模式名或双引号。

图例

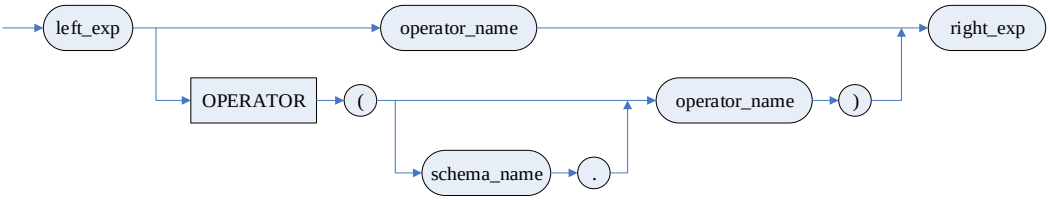
左一元运算符



右一元运算符



二元运算符



语句功能

使用自定义运算符。

使用说明

自定义运算符的运算优先级低于系统原有的基础运算符。例如：“@^”为自定义二元运算符，则表达式“1+2@^3*4”等价于“(1+2)@^(3*4)”。

20.4 应用实例

例 1 创建一个左一元运算符，并使用该运算符进行计算。

首先在 SYSDBA 模式下创建一个存储函数 FUNC_OPL_TEST。

```
CREATE FUNCTION FUNC_OPL_TEST(
    C IN INT
) RETURN INTEGER
AS
BEGIN
    RETURN C*2;
```

```
END;
/
```

在 SYSDBA 模式下，为存储函数 FUNC_OPL_TEST 定义一个左一元运算符“<%”。

```
CREATE OPERATOR <% (FUNCTION FUNC_OPL_TEST, LEFTARG INT);
```

使用自定义运算符“<%”进行计算。

```
SELECT (2<%);
```

查询结果如下：

```
行号        2OPERATOR("<%")
---------- ---------------
1                4
```

以上查询结果为 2*2 的计算结果，结果为 4。

例 2 创建一个右一元运算符，并使用该运算符进行计算。

首先在 SYSDBA 模式下创建一个存储函数 FUNC_OPR_TEST。

```
CREATE FUNCTION FUNC_OPR_TEST(
    C IN VARCHAR
) RETURN INTEGER
AS
BEGIN
    RETURN LENGTH(C);
END;
/
```

在 SYSDBA 模式下，为存储函数 FUNC_OPR_TEST 定义一个右一元运算符“/>”。

```
CREATE OPERATOR /> (FUNCTION FUNC_OPR_TEST, RIGHTARG VARCHAR);
```

使用自定义运算符“/>”进行计算。

```
SELECT />'abc';
```

查询结果如下：

```
行号        OPERATOR("/>")'abc'

---------- -------------------

1                3
```

以上查询结果为运算对象“abc”中的字符个数，结果为 3。

例 3 创建一个二元运算符，并使用该运算符进行计算。

首先在 SYSDBA 模式下创建一个存储函数 FUNC_OP_TEST。

```
CREATE FUNCTION FUNC_OP_TEST(
    C1 IN INT,
    C2 IN INT
) RETURN INTEGER
AS
BEGIN
    RETURN (C1 + C2) * 2;
END;
/
```

在 SYSDBA 模式下，为存储函数 FUNC_OP_TEST 定义一个二元运算符“@^”。

```
CREATE OPERATOR @^ (FUNCTION FUNC_OP_TEST, LEFTARG INT, RIGHTARG INT);
```

使用自定义运算符“@^”进行计算。

```
SELECT 2@^3;
```

查询结果如下：

```
行号        2OPERATOR("@^")3

---------- ----------------

1               10
```

以上查询结果为(2+3)*2 的计算结果，结果为 10。

例 4 删除自定义运算符。

删除左一元运算符“<%”。

```
DROP OPERATOR IF EXISTS <%(INT,NULL);
```

删除右一元运算符“/>”。

```
DROP OPERATOR IF EXISTS />(NULL,VARCHAR);
```

删除二元运算符“@^”。

```
DROP OPERATOR IF EXISTS @^(INT,INT);
```

第 21 章 自定义集函数

DM 支持自定义集函数，它不仅可以对复杂数据类型进行处理，还支持复杂的聚集计算。自定义集函数的使用和 count（）等系统提供的集函数类似，可以在 DQL(数据查询语言)和 DML 中正常使用。

21.1 创建自定义集函数

自定义集函数的创建包括两个步骤:

一 创建对象类型，实现 ODCIAggregate 接口。ODCIAggregate 接口是在对象类型中实现的。具体为使用 CREATE TYPE 语句创建相应的自定义类型。

二 创建集函数。创建自定义集函数的过程是通过指定对象类型实现的。下面对这两个步骤进行详细介绍。

21.1.1 创建对象类型

DM 须创建一个对象类型以实现 ODCIAggregate 接口。创建对象类型需要使用 CREATE TYPE 语句，有关具体语法的详细介绍，请参考本手册 13.1 创建类型。本章节将主要介绍对象类型中的方法。

创建用户定义集函数须用到三个接口来实现集函数的所有内部操作：即初始化 ODCIAggrecateInitialize()，计算 ODCIAggregateIterate()，终止 ODCIAggregateTerminate()。

1. ODCIAggregateInitialize()

用于初始化实现对象类型的聚合上下文。DM 调用该程序以初始化用户定义的聚集计算，初始化的聚合上下文作为对象类型实例传递回 DM。只有完成聚合上下文的初始化才能

进行后续聚合操作。

语法格式

```
STATIC FUNCTION ODCIAggregateInitialize(
    actx IN OUT <impltype>)
RETURN NUMBER
```

参数

actx(输入/输出参数)：由接口初始化的聚合上下文。对于常规聚合情况，此值为 NULL。在窗口聚合中，actx 是前一个窗口的上下文。此对象实例作为参数传递给下一个聚合接口。<impltype>为自定义对象类型。

返回值

0 成功；1 失败。

2. ODCIAggregateIterate()

通过处理输入值、计算并返回聚合上下文来迭代输入行，为每个值调用。在整个计算过程中，DM 会反复调用改程序。每次调用都会传入一个或一组新值，以及当前的聚合上下文。程序处理新值后返回更新后的聚合上下文。DM 会对本组每个非空值调用此进程，NULL 则会被忽略，不会传递给该程序。

语法格式

```
MEMBER FUNCTION ODCIAggregateIterate(
    self IN OUT <impltype>,
    val <inputdatatype>)
RETURN NUMBER
```

参数

self（输入/输出参数）：作为输入，是当前聚合上下文的值；作为输出，是更新后的值。<impltype>为自定义对象类型。

val（输入参数）：待聚合的值。<inputdatatype>为输入参数类型，该参数类型可以为 DM 支持 ROWID 除外的任何数据类型，也可以为任何不包含 ROWID 的自定义数据类型。

返回值

0 成功；1 失败。

3. ODCIAggregateTerminate()

计算聚合的结果后输出并执行所有必要的清理，例如释放内存。该方法作为聚合计算的最后一步，由 DM 调用。该程序将聚合上下文作为输入，返回生成的聚合值。

语法格式

```
MEMBER FUNCTION ODCIAggregateTerminate(
    self IN <impltype>,
    returnvalue OUT <return_type>,
    flags IN number)
RETURN NUMBER
```

参数

self（输入参数）：聚合上下文的值，<impltype>为自定义对象类型。

returnvalue（输出参数）：结果聚合值。<return_type>为输入参数类型，该参数类型可以为 DM 支持 ROWID 除外的任何数据类型，也可以为任何不包含 ROWID 的自定义数据类型。

flags（输入/输出参数）： 表示各种选项的位向量。ODCI_AGGREGATE_REUSE_CTX 的一个设置位表示上下文被重用，不应释放外部上下文。

返回值

0 成功；1 失败。

21.1.2 创建集函数

语法格式:

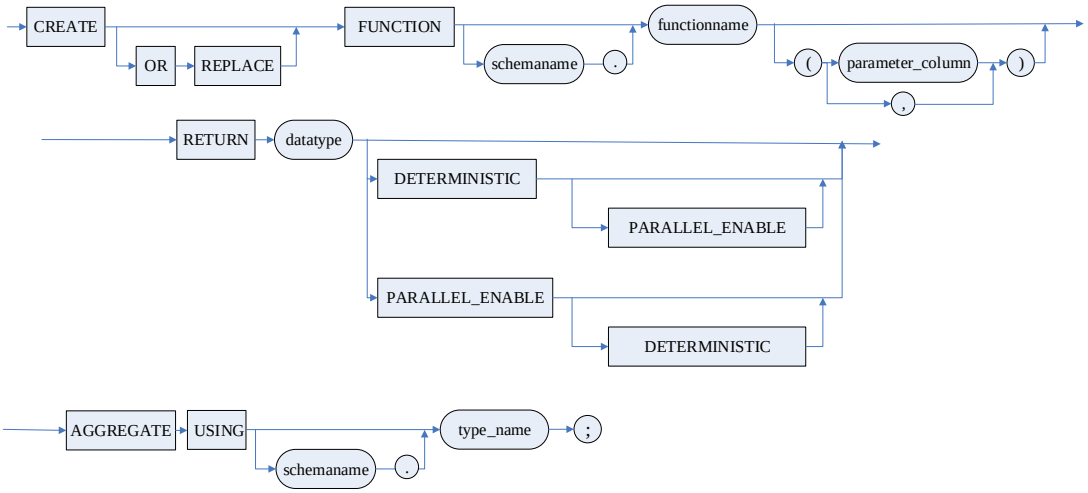
```
CREATE [OR REPLACE] FUNCTION [<模式名>.]<函数名>[(<参数列表>)]
RETURN <返回值类型>
[<usedf_agg_keywords>]AGGREGATE USING [<模式名>.]<类对象名>
<userdf_agg_keywords>::=PARALLEL_ENABLE[DETERMINISTIC]|
                        DETERMINISTIC[ PARALLEL_ENABLE]
```

参数:

1. <函数名>指被创建的自定义集函数的名称;
2. <模式名>指明被创建的自定义集函数所属模式的名称，缺省为当前模式名;
3. <参数列表> 指明自定义集函数信息, 参数模式可设置为 IN、OUT 或 IN OUT (OUT IN), 缺省为 IN 类型，参数类型和参数个数需和自定义对象类型中定义保持一致;
4. <返回值类型> 必须和自定义对象类型中定义的一致;
5. DETERMINISTIC：当使用用户自定义函数进行物化视图创建时，为保证物化视图能够进行查询改写（QUERY REWRITE），需对用户自定义函数添加该关键字;
6. PARALLEL_ENABLE：指定该用户自定义函数可以进行并行聚集，仅语法支持;
7. <自定义对象类型名>实现 ODCIAggergate 接口的对象类型名称。

图例

自定义集函数创建:



语句功能

创建自定义集函数

使用说明

创建自定义集函数在语法上和创建存储函数类似，但需要注意的是，创建自定义集函数不支持“IF NOT EXISTS”。

21.2 使用场景

自定义集函数的使用和系统集函数的使用几乎完全相同。其主要用于以下几种情景中：

一 用于 DQL 语句中，可以用于 SELECT 子句；也可用于 ORDER BY 子句或者作为 HAVING 子句中的部分谓词出现；还可以与 DISTINCT（忽略重复值）、ALL（缺省值）、ROLLUP 或 CUBE 等关键字联合使用。需注意的是，在与 ROLLUP，CUBE 联用时，情况会和系统集函数略有不同，系统在产生结果的基础上进行了排序。

二 用于 DML 语句中，一般借助查询语句使用。

三 用于 DDL 语句中，可以用于表的创建和视图的创建，一般借助查询语句使用。需要注意的是，对于用于物化视图定义的自定义集函数，需要用到关键字 DETERMIINISTIC，以保证物化视图能够进行查询改写。

四 用于分析函数中。自定义集函数可以和 OVER（<分析子句>）组合之后作为分析函数

使用，使用方式和系统提供的分析函数完全一致，OVER(<分析子句>)具体使用方式可参考本手册 4.1.4 分析函数。

21.3 删除自定义集函数

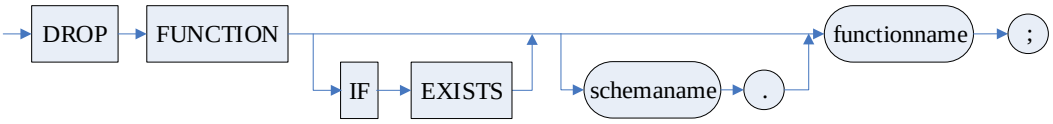
删除自定义集函数使用 DROP FUNCTION 完成。删除之前，建议用户先删除相关的自定义对象类型。

语法格式

```
DROP FUNCTION [IF EXISTS] [<模式名>.]<自定义集函数名>;
```

图例

删除自定义集函数



使用说明

1. 删除不存在的函数会报错。若指定 IF EXISTS 关键字，删除不存在的集函数不会报错；

2. 如果被删除的集函数不属于当前模式，必须在语句中指明模式名；

权限

执行该操作的用户必须是 DBA，或者是函数的拥有者且具有 DROP PACKAGE 权限。

21.4 应用实例

编写一个自定义集函数 MyMax，用于查找最大值，功能和系统集函数完全一致。下面将针对其创建、使用和删除过程，进行具体描述。

21.4.1 创建

首先，删除可能存在的同名对象类型和同名函数。

```
DROP TYPE IF EXISTS MyMaxImpl;

DROP FUNCTION IF EXISTS MyMax;
```

其次，创建对象类型 MyMaxImpl，在对象类型中实现声明 ODCIAggregate 接口。

```
CREATE TYPE MyMaxImpl AS OBJECT
(
  max NUMBER, -- 目前最大值
  //初始化
  STATIC FUNCTION ODCIAggregateInitialize(sctx IN OUT MyMaxImpl)
    RETURN NUMBER,
  //迭代计算
  MEMBER FUNCTION ODCIAggregateIterate(self IN OUT MyMaxImpl,
     value IN NUMBER) RETURN NUMBER,
  //终止
  MEMBER FUNCTION ODCIAggregateTerminate(self IN MyMaxImpl,
    returnValue OUT NUMBER, flags IN NUMBER) RETURN number
);
/
```

接着，创建类型体 MyMaxImpl，对 ODCIAggregate 接口进行实现。

对于初始化，直接设置初始值为 0，由于初始值设为 0，故此自定义集函数不可用于列数据全部为负数的情况；对于迭代部分，比较聚合的新值和目前聚合值的最大值进行比较，取较大值赋给 max；对于合并部分，将两个并行聚合产生的两个聚合上下文中的 max 进行

对比，并取最大值作为结果输出；对于终止部分，聚合上下文中的 max 即为结果值，所以直接输出即可。

```
CREATE OR REPLACE TYPE BODY MyMaxImpl IS
//初始化
STATIC FUNCTION ODCIAggregateInitialize(sctx IN OUT MyMaxImpl)
RETURN NUMBER IS
BEGIN
      sctx := MyMaxImpl(0);
      return ODCIConst.Success;
END;
//迭代
MEMBER FUNCTION ODCIAggregateIterate(self IN OUT MyMaxImpl, value IN NUMBER) RETURN
NUMBER IS
BEGIN
  IF value > self.max THEN
     self.max := value;
  END IF;
  RETURN ODCIConst.Success;
END;
//终止
MEMBER FUNCTION ODCIAggregateTerminate(self IN MyMaxImpl,
     returnValue OUT NUMBER, flags IN NUMBER) RETURN NUMBER IS
BEGIN
  returnValue := self.max;
```

```
  RETURN ODCIConst.Success;
END;
END;
/
```

然后，创建自定义集函数 MyMax。

```
CREATE  FUNCTION  MyMax (INPUT NUMBER) RETURN NUMBER
AGGREGATE USING MyMaxImpl;
```

21.4.2 使用

1. 用于 DQL 语句中。

例 1 自定义集函数 MyMax 用于 SELECT 子句查找最大薪资项。

```
SELECT MyMax(SALARY)
FROM RESOURCES.EMPLOYEE;
```

结果如下

```
行号         MYMAX(SALARY)
---------- -------------
1               40000
```

例 2 自定义集函数 MyMax 用于 ORDER BY 和 HAVING 子句查找每个职位的最高薪资。

```
SELECT TITLE, MyMax(SALARY)
FROM RESOURCES.EMPLOYEE
GROUP BY TITLE
HAVING MyMax(SALARY)>10000
ORDER BY MyMax(SALARY);
```

结果如下：

```
行号        TITLE            MYMAX(SALARY)
---------- -------------- -------------
1           采购代表         12000
2           销售代表         16000
3           系统管理员       20000
4           采购经理         23000
5           人力资源部经理   25000
6           销售经理         26000
7           总经理           40000
```

例 3 自定义集函数 MyMax 与关键字 ROLLUP 联用查询每个出版社同一批出版的图书的最大 ID 号。

```
SELECT MYMAX(PRODUCTID),PUBLISHER, PUBLISHTIME
FROM PRODUCTION.PRODUCT
GROUP BY ROLLUP(PUBLISHER,PUBLISHTIME);
```

结果如下:

```
行号        MYMAX(PRODUCTID) PUBLISHER              PUBLISHTIME
---------- ---------------- -------------------- -----------
1           5                                     2006-09-01
2           10                21 世纪出版社       1901-01-01
3           4                 广州出版社          2005-12-01
4           8                 机械工业出版社      2006-01-01
5           7                 清华大学出版社      2007-03-01
6           6                 人民文学出版社      2006-09-01
7           3                 上海出版社          2006-08-01
```

```
8          9          外语教学与研究出版社  2003-08-01
9          2          中华书局              2005-04-01
10         5                                NULL
11         10         21 世纪出版社         NULL
12         4          广州出版社            NULL
13         8          机械工业出版社        NULL
14         7          清华大学出版社        NULL
15         6          人民文学出版社        NULL
16         3          上海出版社            NULL
17         9          外语教学与研究出版社  NULL
18         2          中华书局              NULL
19         10         NULL                  NULL
```

2. 用于 DML 语句中。

例 薪资最高者加薪 5000 元。

为进行对比，首先查看最高薪资

```
SELECT MyMax(SALARY) FROM RESOURCES.EMPLOYEE;
```

结果如下所示

```
行号        MYMAX(SALARY)
---------- -------------
1           40000
```

使用 DML 语句对薪资最高者数据进行更新

```
UPDATE RESOURCES.EMPLOYEE SET SALARY=SALARY+5000 WHERE SALARY IN (SELECT
MYMAX(SALARY) FROM RESOURCES.EMPLOYEE);
```

更新后再次查看

```
SELECT MyMax(SALARY) FROM RESOURCES.EMPLOYEE;
```

结果如下所示：

```
行号          MYMAX(SALARY)

---------- -------------

1             45000
```

3. 用于 DDL 语句。

例 1 使用自定义集函数创建表用于查看每个岗位的最高薪资

```
DROP TABLE IF EXISTS T1;

CREATE TABLE T1 AS SELECT TITLE, MyMax(SALARY)AS MAXSAL

FROM RESOURCES.EMPLOYEE

GROUP BY TITLE;

SELECT * FROM T1;
```

结果如下所示：

```
行号          TITLE              MAXSAL

---------- -------------- ------

1             采购代表          12000

2             采购经理          23000

3             人力资源部经理  25000

4             系统管理员        20000

5             销售代表          16000

6             销售经理          26000

7               总经理            40000
```

例 2 使用自定义集函数创建允许查询改写的物化视图。

```
//创建带关键字 DETERMINISTIC 的自定义集函数
```

```
DROP FUNCTION IF EXISTS MyMax;

CREATE  FUNCTION MyMax (INPUT NUMBER) RETURN NUMBER

DETERMINISTIC

AGGREGATE USING MyMaxImpl;

//创建允许查询改写的物化视图

DROP MATERIALIZED VIEW IF EXISTS MYVIEW;

CREATE MATERIALIZED VIEW MYVIEW

ENABLE QUERY REWRITE AS

SELECT MyMax(SALARY)

FROM RESOURCES.EMPLOYEE;

//查询该物化视图

SELECT * FROM MYVIEW;
```

结果如下所示：

```
行号          MYMAX(SALARY)

---------- -------------

1     40000
```

4. 用于分析函数。

例 查询折扣大于 7 的图书作者以及最大折扣。此时的 MyMax 作为一个分析函数。

```
SELECT  AUTHOR,  MyMax(DISCOUNT) OVER (PARTITION BY AUTHOR)  AS MYMAX

FROM    PRODUCTION.PRODUCT

WHERE   DISCOUNT > 7;
```

结果如下所示：

```
行号          AUTHOR             MYMAX

---------- -------------- -----
```

```
1               曹雪芹,高鹗      8
2               施耐庵，罗贯中 7.5
3               严蔚敏，吴伟民 8.5
```

21.4.3 删除

删除自定义集函数 MyMax 之前，建议先删除自定义对象类型 MyMaxImpl。

```
DROP TYPE IF EXISTS MyMaxImpl;

DROP FUNCTION IF EXISTS MyMax;
```

第 22 章 XML 数据解析

XML（Xtended Markup Language，可扩展标记语言）是一种基于文本、用于结构化可标记文档的格式语言，是一种轻便的、可扩展的、标准且简学易懂的保存数据的语言，起源于 SGML，并可模拟为树型。DM 提供了一些过程与函数用来解析和查询 XML 数据，目前 DM 支持解析的 XML 数据大小不得超过 500MB。

后文基于功能的不同，将 XML 相关的方法进行了分类：数据构造章节提供了基础的 XML 构造方法，用于存储和初始化 XML 数据；数据查询章节的方法能够查询 XML 数据指定路径下的结点或者值；数据修改章节提供的方法支持用户更新、插入或删除 XML 文档中的结点或属性；数据转换章节提供的方法将 XML 转换为其他格式，或对 XML 数据进行格式化处理。

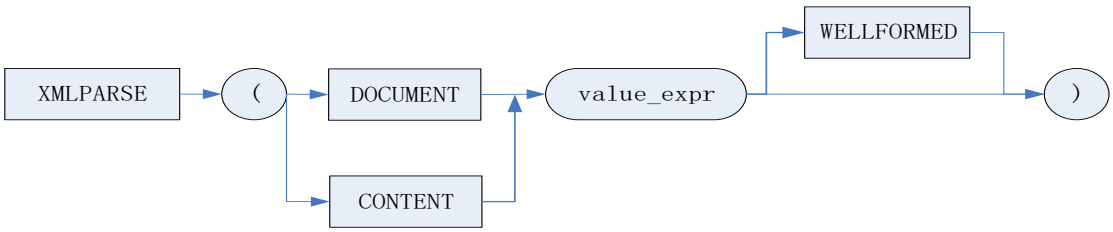
22.1 数据构造

22.1.1 XMLPARSE

语法格式：

```
XMLPARSE ({DOCUMENT|CONTENT}<value_expr>[WELLFORMED])
```

图例：



功能说明：

XMLPARSE 用于解析 XMLTYPE 类型数据，亦即 value_expr 的值。当指定为 DOCUMENT 时，value_expr 被解析为只有一个根结点的标准 XML 文档；当指定为 CONTENT 时，value_expr 被解析为有效的 XML 值，value_expr 可以没有 XML 申明部分。若指定了

WELLFORMED 参数，则不对 XML 内容进行检查，否则会对内容的合法性进行检查，内容不合法则报错。

举例说明：

```
SELECT XMLPARSE(DOCUMENT '<a>good</a>' WELLFORMED);
```

查询结果如下：

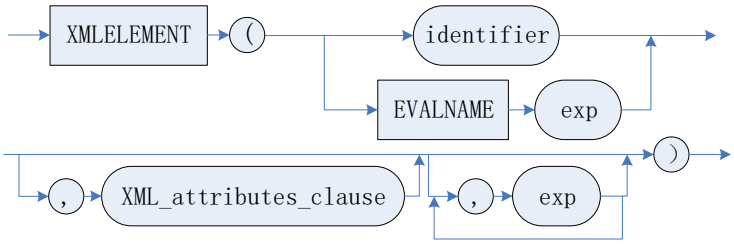
```
<a>good</a>
```

22.1.2 XMLELEMENT

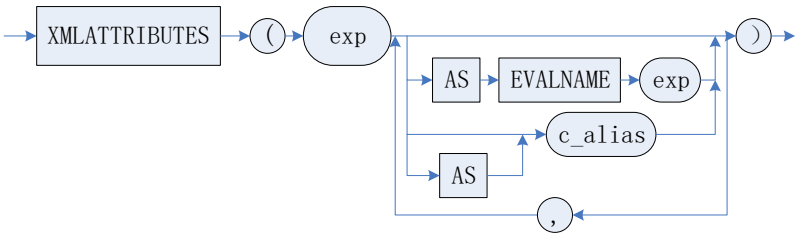
语法格式：

```
XMLELEMENT(<identifier> | EVALNAME <exp> [, XMLATTRIBUTES (<exp> [AS EVALNAME <exp>] |
[[AS] c_alias] {, AS EVALNAME <exp> | [AS] c_alias})][, <exp> {, <exp>}])
```

图例：



XML_attributes_clause



功能说明：

XMLELEMENT 用于将 exp 构建成 xml 数据类型元素，返回 xmltype 类型数据。当前将 XMLELEMENT 的功能拆分为三个子函数进行调用：XMLELEMENT：一个输入参数，用于创建 XML 元素结点；XMLATTRIBUTE：用于向 XML 元素结点添加属性；XMLCHILD：用于向 XML 元素结点添加孩子。

参数说明:

identifier：标识符。

exp：字符串类型的数据。

c_alias：列别名。

举例说明:

```
SELECT XMLELEMENT(DM, 'TXT1', 'TXT2') FROM DUAL;
```

查询结果如下:

```
行号        XMLCHILD(XMLCHILD(XMLELEMENT(DM),'TXT1',0),'TXT2',0)
---------- ----------------------------------------------------
1           <DM>TXT1TXT2</DM>
```

使用 XMLATTRIBUTES 子句。

```
CREATE TABLE TABLE1 (ID NUMBER(10), COL1 VARCHAR2(5));
INSERT INTO TABLE1 VALUES (1,'HELLO');
INSERT INTO TABLE1 VALUES (2,NULL);
COMMIT;

SELECT
    XMLELEMENT("OUTER",
        XMLFOREST(ID AS "ID"),
        (CASE NVL(COL1,'NULL')
            WHEN 'NULL' THEN XMLELEMENT(COL1, XMLATTRIBUTES('TRUE' AS "XSI:NIL"), NULL)
            ELSE XMLFOREST(COL1 AS "COL1")
            END)
    )AS XML_TESTTEMP
FROM TABLE1
/
```

查询结果如下:

```
行号        XML_TESTTEMP
---------- ------------------------------------------------------
```

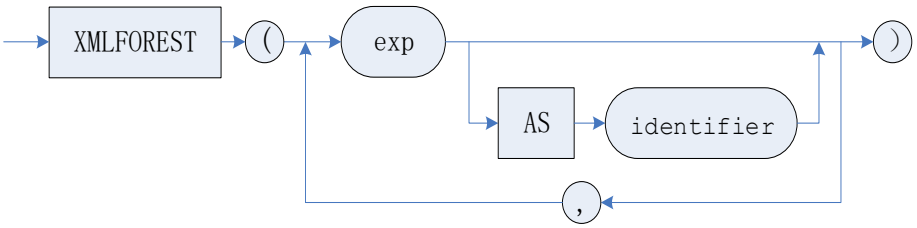
```
1           <OUTER>
                <ID>1</ID>
                <COL1>HELLO</COL1>
            </OUTER>
2           <OUTER>
                <ID>2</ID>
                <COL1 XSI:NIL="TRUE"/>
            </OUTER>
```

22.1.3 XMLFOREST

语法格式:

```
XMLFOREST(<exp> [AS <identifier>] {,<exp> [AS <identifier>]})
```

图例:



功能说明:

XMLFOREST 用于将 exp 构建成 xml 数据类型元素，返回 xmltype 类型数据。当 exp 为列名时，可以省略标识符，此时将列名作为标识符，并将该列的查询结果作为字符串类型数据，其他情况不能省略标识符。

参数说明:

exp：字符串类型的数据。

identifier：标识符。

举例说明:

exp 不为列名，不能省略标识符:

```
SELECT XMLFOREST('TXT1' AS A, 'TXT2' AS B) FROM DUAL;
```

查询结果如下：

```
行号        XMLFOREST('TXT1'ASA,'TXT2'ASB)
---------- --------------------
1           <A>TXT1</A><B>TXT2</B>
```

exp 为列名，可以省略标识符，省略后将列名作为标识符：

```
CREATE TABLE TEST(C1 INT,C2 VARCHAR);
INSERT INTO TEST VALUES(1, 'TXT');
SELECT XMLFOREST(C1,C2) FROM TEST;
```

查询结果如下：

```
行号        XMLFOREST(C1,C2)
---------- --------------------
1           <C1>1</C1><C2>TXT</C2>
```

22.1.4 XMLAGG

语法格式：

```
XMLAGG(<XMLdata>[<ORDER BY 子句>])
```

图例：



功能说明：

XMLAGG 函数，拼接 xml 数据。拼接 xml 数据时采用换行符作为分隔符。

参数说明：

XMLdata：待拼接的数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，XMLdata 参数仅允许为 xmltype 类型。

ORDER BY 子句：请参考第 4 章 数据查询语句。

举例说明：

例 1 拼接非 xml 类型数据（即普通字符串）。

```
create table X (A varchar(10),B int);
insert into X values('aaa',3);
insert into X values('bb',2);
insert into X values('c',1);
select XMLAGG(XMLPARSE(content A WELLFORMED) order by B) from X;
```

查询结果如下:

```
c
bb
aaa
```

例 2 拼接 xml 数据。

```
CREATE TABLE X_1 (C_INT1 INT,C_XMLTYPE XMLTYPE);
INSERT INTO X_1 VALUES(20,
'<LineItems><Reference>DUSTIN12345</Reference><SpecialItem>Counter</SpecialItem></Lin
eItems>');
INSERT INTO X_1 VALUES(21,'<Deptment>产品测试</Deptment>');
SELECT XMLAGG(XMLPARSE(DOCUMENT C_XMLTYPE WELLFORMED)) FROM X_1;
```

查询结果如下:

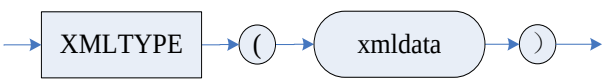
```
<LineItems>
  <Reference>DUSTIN12345</Reference>
  <SpecialItem>Counter</SpecialItem>
</LineItems>
<Deptment>产品测试</Deptment>
```

22.1.5 XMLTYPE

语法格式:

```
XMLTYPE(<xmldata>)
```

图例:



功能说明：

XMLTYPE 函数，将数据转换为 xmltype 类型数据，如果参数 xmldata 的数据不符合 XML 格式要求将报错。

参数说明：

xmldata：待转换的数据。

举例说明：

```
SELECT xmltype('<Envelope>abc</Envelope>') FROM DUAL;
```

查询结果如下：

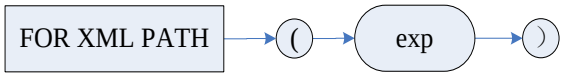
```
<Envelope>abc</Envelope>
```

22.1.6 FOR XML PATH

语法格式：

```
FOR XML PATH (<exp>)
```

图例：



功能说明：

FOR XML PATH 用于将 SQL 查询结果转换为 XML 格式，用户可指定行结点名称，并通过列别名定义元素结点。

参数说明：

<exp>：字符串类型的数据。

举例说明：

例 1 通过 PATH('row')指定行节点名称，元素节点默认使用列名：

```
CREATE TABLE test_forxmlpath(id INT ,name VARCHAR);
INSERT INTO test_forxmlpath VALUES(1 ,'aaa');
INSERT INTO test_forxmlpath VALUES(2 ,'bbb');
SELECT * FROM test_forxmlpath FOR XML PATH('row');
```

查询结果如下:

```
行号        ##TMP_COL_FOR_XML_PATH
---- ------------------------------------------------------------------------
1           <row>
  <ID>1</ID>
  <NAME>aaa</NAME>
</row><row>
  <ID>2</ID>
  <NAME>bbb</NAME>
</row>
```

例 2 通过列别名定义元素节点:

```
CREATE TABLE test_forxmlpath_1(id int ,name varchar);
INSERT INTO test_forxmlpath_1 VALUES(1 ,'aaa');
INSERT INTO test_forxmlpath_1 VALUES(2 ,'bbb');
SELECT id AS "id_num", name AS "name_str" FROM test_forxmlpath_1 FOR XML PATH('row');
```

查询结果如下:

```
行号        ##TMP_COL_FOR_XML_PATH
------ ------------------------------------------------------------
1           <row>
  <id_num>1</id_num>
  <name_str>aaa</name_str>
</row><row>
  <id_num>2</id_num>
  <name_str>bbb</name_str>
</row>
```

22.2 数据查询

22.2.1 SF_XMLQUERY

定义:

CLOB

SF_XMLQUERY(

xmldata clob,

xpath clob

)

功能说明:

解析 xmldata 并查询指定路径 xpath 下的数据。本方法的功能可由 XMLQUERY 代替，不再推荐使用。

参数说明:

xmldata：待解析的 XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

返回值:

解析得到的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。

举例说明:

```
SELECT SF_XMLQUERY(XMLTYPE('<a><b>b</b></a>'), '/a');
```

查询结果如下:

```
<a>
  <b>b</b>
</a>
```

22.2.2 XMLQUERY

定义:

```
CLOB
XMLQUERY(
xmldata     clob,
xpath       varchar
)
```

功能说明:

解析 xmldata 并查询指定路径 xpath 下的数据。

参数说明:

xmldata：待解析的 XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

返回值:

解析得到的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。

举例说明:

```
SELECT XMLQUERY(xmltype('<a><b>b</b></a>'),'/a');
```

查询结果如下:

```
<a>
  <b>b</b>
</a>
```

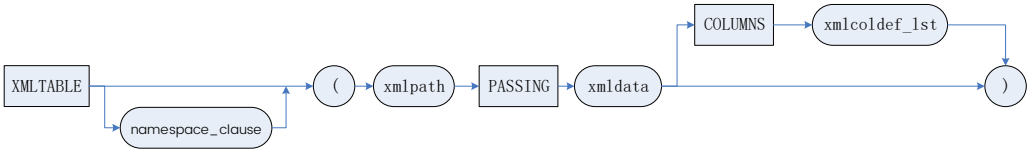
22.2.3 XMLTABLE

语法格式:

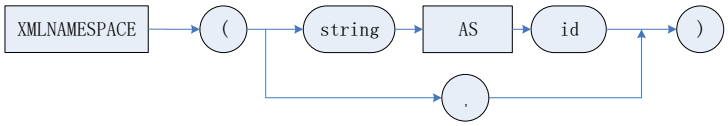
```
XMLTABLE([<namespace_clause>,]<xmlpath> PASSING <xmldata> [COLUMNS
<xmlcoldef_lst>])
```

```
<namespace_clause>::=XMLNAMESPACE( <string> AS "<id>"{, <string> AS "<id>"})
```

图例：



<namespace_clause>



功能说明：

查询 XML 数据在指定路径下的结点，并将结果按照关系表的方式展示。

需要注意的是，当 INI 参数 COMPATIBLE_MODE 不为 2 时，输出的 XML 数据结果将会自动换行；当 INI 参数 COMPATIBLE_MODE=2 时，输出的 XML 数据结果将不会自动换行。

参数说明：

namespaces：表示命名空间。

string：表示字符串类型数据。

id：表示标识符。

xmlpath：XML 数据的路径。

xmldata：XMLTYPE 类型数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xmlcoldef_lst：列定义列表。

举例说明：

例 1 获取 XML 数据在指定路径下的结点。

```
CREATE TABLE T3(C1 VARCHAR);
SF_SET_SESSION_PARA_VALUE('ENABLE_TABLE_EXP_REF_FLAG', 1);  //开启同层列引用参数。
SELECT 语句中 T3.C 引用了同层表 T3 的列。
INSERT INTO T3 VALUES('<A><标签 属性="属性名">HELLO</标签></A>');
```

```
SELECT X1,X2 FROM T3, XMLTABLE('/A/标签' PASSING T3.C1 COLUMNS X1 VARCHAR(12) PATH '@属性
', X2 VARCHAR(12) PATH 'text()');
```

查询结果如下：

```
行号          X1       X2
------- ------ -----
1             属性名  HELLO
```

支持多个命名空间。

```
SELECT * FROM XMLTABLE(XMLNAMESPACES('hxtp://www.a.com' AS "d",'hxtp://www.a.com' AS
"f"), '/a' PASSING XMLTYPE('<a ibm:id="123"
xmlns:ibm="hxtp://www.a.com" ><ibm:b>textaaa</ibm:b></a>') columns id varchar(20)
path '/a/f:b');
```

查询结果如下：

```
行号          ID
---------- -------
1             textaaa
```

例 2 对比在 INI 参数 COMPATIBLE_MODE 为 2 与非 2 情况下的返回结果。

```
SELECT * FROM XMLTABLE('/wor/ins'
PASSING XMLtype('<wor>
 <ins>
   <joi id="15">
     <result id="39" s="38">
     </result>
   </joi>
   <joi id="55">
     <result id="58" s="22">
     </result>
   </joi>
 </ins>
</wor>
') COLUMNS result XMLtype PATH 'joi');
```

当 INI 参数 COMPATIBLE_MODE=0 时，查询结果如下：

```
RESULT
------------------
<joi id="15">
  <result id="39" s="38">
    </result>
</joi>
<joi id="55">
  <result id="58" s="22">
    </result>
</joi>
```

当 INI 参数 COMPATIBLE_MODE=2 时，查询结果如下：

```
RESULT
-----------------
<joi id="15"><result id="39" s="38">
    </result></joi><joi id="55"><result id="58" s="22">
    </result></joi>
```

22.2.4 EXISTSNODE

定义：

INT

EXISTSNODE(

xmldata clob,

xpath varchar

)

或

INT

EXISTSNODE(

xmldata clob,

xpath varchar,

xmlns varchar

)

功能说明:

xmldata 中是否存在指定的路径结点，若存在则返回 1，不存在返回 0。

参数说明:

xmldata：待解析的 XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

xmlns: XML 命名空间。

返回值:

1 代表结点存在，0 代表结点不存在。xmldata 为 NULL 的情况下一律返回 0，xpath 为 NULL 的情况下一律返回 NULL。

举例说明:

例 1 指定 xmldata 和 xpath 参数执行 EXISTSNODE。

```
SELECT EXISTSNODE(XMLTYPE('<a><b>b</b></a>'), '/a');
```

查询结果如下:

```
1
```

例 2 指定 xmldata、xpath 和 xmlns 参数执行 EXISTSNODE。

```
SELECT EXISTSNODE(XMLTYPE('<a xmlns:ns1="http://localhost:8080/source">
<ns1:b>First-202503172548153</ns1:b></a>'),
'/a/ns1:b',
'xmlns:ns1="http://localhost:8080/source"');
```

查询结果如下:

```
1
```

22.2.5 EXTRACTVALUE

定义:

```
VARCHAR
EXTRACTVALUE(
xmldata     clob,
xpath       varchar
)
或
VARCHAR
EXTRACTVALUE(
xmldata     clob,
xpath       varchar,
xmlns       varchar
)
```

功能说明:

获取 xmldata 中指定路径下的结点值，若路径下不止一个结点，或者结点不是叶子结点，将报错处理。

参数说明:

xmldata：待解析的 XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

xmlns：XML 命名空间。

返回值:

获取的结点值。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。

举例说明:

例 1 指定 xmldata 和 xpath 参数执行 EXTRACTVALUE 函数获取指定结点路径下的值。

```
SELECT EXTRACTVALUE(XMLTYPE('<a><b>b</b></a>'), '/a/b');
```

查询结果如下：

```
b
```

例 2 指定 xmldata、xpath 和 xmlns 参数执行 EXTRACTVALUE 函数获取指定结点路径下的值。

```
SELECT EXTRACTVALUE(XMLTYPE('<a xmlns:ns1="http://localhost:8080/source">
<ns1:b>First-202503172548153</ns1:b></a>'),
'/a/ns1:b',
'xmlns:ns1="http://localhost:8080/source"');
```

查询结果如下：

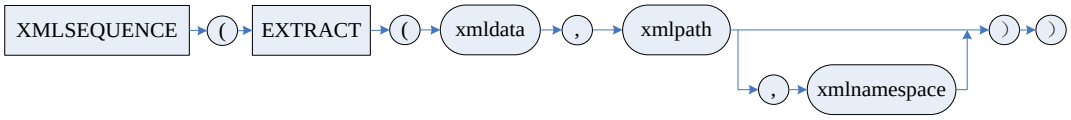
```
First-202503172548153
```

22.2.6 XMLSEQUENCE

语法格式：

```
XMLSEQUENCE (EXTRACT ( <xmldata>, <xmlpath> [, <xmlnamespace>]) )
```

图例：



功能说明：

XMLSEQUENCE 用于获取指定路径 xmlpath 下不同结点的 XML 数据，并将结点数据集合转换为行集合，通常与 TABLE 函数和 EXTRACT 函数结合使用。

参数说明：

xmlpath：XML 数据的路径。

xmldata：XMLTYPE 类型数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xmlnamespace：表示命名空间，通常为字符串类型数据。

举例说明：

例 1 指定 XMLDATA 和 XPATH 参数执行 XMLSEQUENCE。

```
SELECT TAB.* FROM
TABLE(XMLSEQUENCE(EXTRACT(XMLTYPE('<a><b>First-202503172548153</b></a>'), '/a/b')))
TAB;
```

查询结果如下：

```
行号        COLUMN_VALUE
---------- ----------------------------
1           <b>First-202503172548153</b>
```

例 2 指定 xmldata、xpath 参数执行 XMLSEQUENCE，如果 xpath 存在命名空间且未指定命名空间参数将报错。

```
SELECT TAB.* FROM TABLE(XMLSEQUENCE(EXTRACT(XMLTYPE('<a
xmlns:ns1="http://localhost:8080/source"><ns1:b>First-202503172548153</ns1:b></a>'),
'/a/ns1:b'))) TAB;
```

查询结果报错：

```
-9903: XML 执行异常 [Undefined namespace prefix]
```

例 3 指定 xmldata、xpath、xmlnamespace 参数执行 XMLSEQUENCE。

```
SELECT TAB.* FROM TABLE(XMLSEQUENCE(EXTRACT(XMLTYPE('<a
xmlns:ns1="http://localhost:8080/source"><ns1:b>First-202503172548153</ns1:b></a>'),
'/a/ns1:b', 'xmlns:ns1="http://localhost:8080/source"'))) TAB;
```

查询结果如下：

```
行号        COLUMN_VALUE
-----  -------------------------------------------------------------
1           <ns1:b xmlns:ns1="http://localhost:8080/source">First-202503172548153</ns1:b>
```

22.2.7 EXTRACT

语法格式：

语法 1:

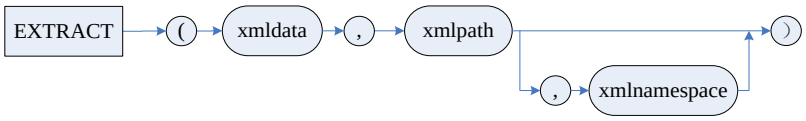
```
EXTRACT (<xmldata>, <xmlpath>[, <xmlnamespace>] )
```

语法 2:

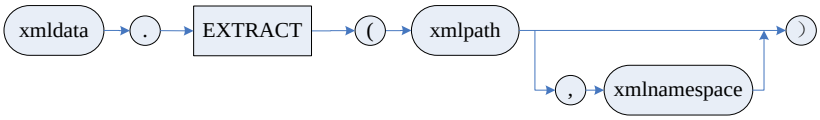
```
<xmldata>.EXTRACT(<xmlpath>[, <xmlnamespace>] )
```

图例:

语法 1:



语法 2:



功能说明:

EXTRACT 用于从 XML 文档中提取符合 xpath 表达式的结点或片段，如果 xpath 中存在命名空间，则需要指定命名空间。

参数说明:

xmlpath：XML 数据的路径。

xmldata：XMLTYPE 类型数据。

xmlnamespace：表示命名空间，通常为字符串类型数据。

举例说明:

例 1 指定 xmldata 和 xpath 参数执行 EXTRACT，以下两种调用方式获得的结果一致。

```
//使用语法 1
SELECT EXTRACT(XMLTYPE('<?xml version="1.0"
encoding="utf-8"?><Envelope><Body><root><result></result></root></Body></Envelope>'),'
/Envelope/Body') AS RESULT FROM DUAL;
//使用语法 2
SELECT XMLTYPE('<?xml version="1.0"
encoding="utf-8"?><Envelope><Body><root><result></result></root></Body></Envelope>').E
```

```
XTRACT('/Envelope/Body') AS RESULT FROM DUAL;
```

查询结果如下：

```
行号        RESULT
---------- ------------------------------------------------
1           <Body>
              <root>
                <result/>
              </root>
            </Body>
```

例 2 指定 xmldata、xpath、xmlnamespace 参数执行 EXTRACT，以下两种方式均可获得结果。

```
//使用语法 1
SELECT EXTRACT(XMLTYPE('<?xml version="1.0" encoding="utf-8"?><soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"><Body><ns1:root
xmlns:ns1="http://imageCenter.web.com"><result></result></ns1:root></Body></soapenv:En
velope>'),'/soapenv:Envelope/Body',
'xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"') AS RESULT FROM DUAL;
//使用语法 2
SELECT XMLTYPE('<?xml version="1.0" encoding="utf-8"?><soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"><Body><ns1:root
xmlns:ns1="http://imageCenter.web.com"><result></result></ns1:root></Body></soapenv:En
velope>').EXTRACT('/soapenv:Envelope/Body',
'xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"') AS RESULT FROM DUAL;
```

查询结果如下：

```
行号        RESULT
-------------------------------------------------------------------------
1           <Body>
              <ns1:root xmlns:ns1="http://imageCenter.web.com">
                <result/>
              </ns1:root>
```

```
</Body>
```

22.3 数据修改

22.3.1 DELETEXML

定义:

```
CLOB
DELETEXML(
xmldata    clob,
xpath    varchar
)
或
CLOB
DELETEXML(
xmldata    clob,
xpath    varchar,
xmlns    varchar
)
```

功能说明:

从 xmldata 中删除 xpath 路径下的结点。

参数说明:

xmldata: XML 数据。需要注意的是, 当 INI 参数 COMPATIBLE_MODE=2 时, xmldata 参数仅允许为 xmltype 类型。

xpath: XML 路径。

xmlns: XML 命名空间。

返回值:

删除结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。

举例说明:

例 1 指定 xmldata 和 xpath 参数执行 DELETEXML，删除指定路径下的内容。

```
SELECT  deleteXML(XMLTYPE('<a><b2>地方<c>d</c></b2><b2>dd</b2></a>'), 'a/b2/c');
```

查询结果如下:

```
<a>
  <b2>地方</b2>
  <b2>dd</b2>
</a>
```

例 2 指定 xmldata、xpath 和 xmlns 参数执行 DELETEXML，删除指定路径下的内容。

```
SELECT  deleteXML(XMLTYPE('<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>
地方<c>d</c></b2><b2>dd</b2></xsl:a>'), 'xsl:a/b2/c',
'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下:

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <b2>地方</b2>
  <b2>dd</b2>
</xsl:a>
```

22.3.2 APPENDCHILDXML

定义:

```
CLOB
APPENDCHILDXML(
xmldata     clob,
xpath       varchar,
child       clob
)
```

或

CLOB

APPENDCHILDXML(

xmldata clob,

xpath varchar,

child clob,

xmlns varchar

)

功能说明:

将 child 结点插入到 xmldata 的 xpath 下的结点中，并将插入结点后的新 xmldata 返回。

参数说明:

xmldata: 源 XML 数据。需要注意的是, 当 INI 参数 COMPATIBLE_MODE=2 时, xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

child：指定的待插入结点。

xmlns：XML 命名空间。

返回值:

插入结点后的新 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 child 为 NULL 则不会进行添加子结点操作。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 child 参数为 xmltype 类型时返回运算后的 xmldata，否则将返回原始 xmldata。

举例说明:

例 1 指定 xmldata、xpath 和 child 参数执行 APPENDCHILDXML，在指定路径下添加 child 结点；如果路径下存在多个孩子节点，则添加到最后一个。

```
SELECT APPENDCHILDXML(XMLTYPE('<a><b>b</b></a>'),'/a', XMLTYPE('<d>xxx</d>'));
```

查询结果如下:

```
<a>
  <b>b</b>
  <d>xxx</d>
</a>
```

例 2 指定 xmldata、xpath、child 和 xmlns 参数执行 APPENDCHILDXML，在指定路径下添加 child 结点；如果路径下存在多个孩子节点，则添加到最后一个。

```
SELECT APPENDCHILDXML(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b>b</b></xsl:a>'), '/xsl:a',
xmltype('<d>xxx</d>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下：

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <b>b</b>
  <d>xxx</d>
</xsl:a>
```

22.3.3 INSERTCHILDXML

定义：

CLOB

INSERTCHILDXML(

xmldata clob,

xpath varchar,

exp varchar,

insnode clob

)

或

CLOB

INSERTCHILDXML(

xmldata clob,

xpath varchar,

exp varchar,

insnode clob,

xmlns varchar

)

功能说明:

将 insnode 结点插入到 xmldata 的 xpath 路径下，其中 insnode 结点名必须与 exp 保持一致。

参数说明:

xmldata: XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

exp：结点名。

insnode：待插入的 XML 结点。

xmlns：XML 命名空间。

返回值:

已插入结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 exp 或 insnode 为 NULL 将报错参数非法。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 insnode 参数为 xmltype 类型时返回运算后的 xmldata，否则将报错。

举例说明:

例 1 指定 xmldata、xpath、exp 和 insnode 参数执行 INSERTCHILDXML，在指定路径下添加 insnode 的内容。

```
SELECT  INSERTchildXML(XMLTYPE('<a><b2>地方</b2><b2>dd</b2></a>'),'a/b2','c',
XMLTYPE('<c>china</c>'));
```

查询结果如下:

```
<a>
  <b2>地方<c>china</c></b2>
  <b2>dd<c>china</c></b2>
</a>
```

例 2 指定 xmldata、xpath、exp、insnode 和 xmlns 参数执行 INSERTCHILDXML，在指定路径下添加 insnode 的内容。

```
SELECT  INSERTchildXML(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>地方</b2><b2>dd</b2></xsl:a>'),
'xsl:a/b2','c', xmltype('<c>china</c>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下：

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <b2>地方<c>china</c></b2>
  <b2>dd<c>china</c></b2>
</xsl:a>
```

22.3.4 INSERTCHILDXMLAFTER

定义：

```
CLOB
INSERTCHILDXMLAFTER(
xmldata clob,
xpath varchar,
exp varchar,
insnode clob
)
```

或

```
CLOB
INSERTCHILDXMLAFTER(
xmldata clob,
```

xpath varchar,

exp varchar,

insnode clob,

xmlns varchar

)

功能说明:

将 insnode 结点插入到 xmldata 的 xpath 路径下 exp 子结点后面。

参数说明:

xmldata: XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

exp：xpath 下的子结点名，insonde 在此结点之后插入。

insnode：待插入的 XML 结点。

xmlns：XML 命名空间。

返回值:

已插入结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 exp 为 NULL 将报错参数非法。如果 insnode 为 NULL 则不会进行插入子结点操作。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 insnode 为 xmltype 类型时返回运算后的 xmldata，否则将报错。

举例说明:

例 1 指定 xmldata、xpath、exp 和 insnode 参数执行 INSERTCHILDXMLAFTER，在 xpath 指定路径下添加 insnode 结点到 exp 指定结点名后。

```
SELECT  INSERTchildXMLafter(XMLTYPE('<a><b2>地方</b2><b2>dd</b2></a>'),'a/','b2',
XMLTYPE('<c>china</c>'));
```

查询结果如下:

```
<a>
```

```
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
  <c>china</c>
</a>
```

例 2 指定 xmldata、xpath、exp、insnode 和 xmlns 参数执行 INSERTCHILDXMLAFTER，在 xpath 指定路径下添加 insnode 结点到 exp 指定结点名后。

```
SELECT  INSERTchildXMLafter(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>地方</b2><b2>dd</b2></xsl:a>'),
'xsl:a/','b2', xmltype('<c>china</c>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下：

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
  <c>china</c>
</xsl:a>
```

22.3.5 INSERTCHILDXMLBEFORE

定义：

```
CLOB
INSERTCHILDXMLBEFORE (
xmldata clob,
xpath varchar,
exp varchar,
insnode clob
)
```

或

CLOB

INSERTCHILDXMLBEFORE（

xmldata clob,

xpath varchar,

exp varchar,

insnode clob,

xmlns varchar

）

功能说明：

将 insnode 结点插入到 xmldata 的 xpath 路径下 exp 子结点前面。

参数说明：

xmldata：XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

exp：xpath 下的子结点名，insonde 在此结点之前插入。

insnode：待插入的 XML 结点。

xmlns：XML 命名空间。

返回值：

已插入结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 exp 为 NULL 将报错参数非法。如果 insnode 为 NULL 则不会进行插入子结点操作。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 insnode 为 xmltype 类型时返回运算后的 xmldata，否则将报错。

举例说明：

例 1 指定 xmldata、xpath、exp 和 insnode 参数执行 INSERTCHILDXMLBEFORE，在 xpath 指定路径下添加 insnode 结点到 exp 指定结点名前。

```
SELECT  INSERTchildXMLbefore(XMLTYPE('<a><b2>地方</b2><b2>dd</b2></a>'), 'a/','b2',
```

```
XMLTYPE('<c>china</c>'));
```

查询结果如下:

```
<a>
  <c>china</c>
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
</a>
```

例 2 指定 xmldata、xpath、exp、insnode 和 xmlns 参数执行 INSERTCHILDXMLBEFORE，在 xpath 指定路径下添加 insnode 结点到 exp 指定结点名前。

```
SELECT  INSERTchildXMLbefore(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>地方</b2><b2>dd</b2></xsl:a>'),
'xsl:a/','b2', xmltype('<c>china</c>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下:

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <c>china</c>
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
</xsl:a>
```

22.3.6 INSERTXMLBEFORE

定义:

```
CLOB
INSERTXMLBEFORE(
xmldata clob,
xpath varchar,
insnode clob
```

)

或

CLOB

INSERTXMLBEFORE(

xmldata clob,

xpath varchar,

insnode clob,

xmlns varchar

)

功能说明:

将 insnode 结点插入到 xmldata 的 xpath 路径下 exp 结点前面。

参数说明:

xmldata: XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

insnode：待插入的 XML 结点。

xmlns：XML 命名空间。

返回值:

已插入结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 insnode 为 NULL 则不会进行插入子结点操作。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 insnode 为 xmltype 类型时返回运算后的 xmldata，否则将返回原始 xmldata。

举例说明:

例 1 指定 xmldata、xpath 和 insnode 参数执行 INSERTXMLBEFORE，在 xpath 指定路径下添加 insnode 结点到指定路径结点前。

```
SELECT  INSERTXMLbefore(XMLTYPE('<a><b2>hello</b2><b2>dd</b2></a>'), '/a/b2[1]',
```

```
XMLTYPE('<c>china</c>'));
```

查询结果如下:

```
<a>
  <c>china</c>
  <b2>hello</b2>
  <b2>dd</b2>
</a>
```

例 2 指定 xmldata、xpath、insnode 和 xmlns 参数执行 INSERTXMLBEFORE, 在 xpath 指定路径下添加 insnode 结点到指定路径结点前。

```
SELECT  INSERTXMLbefore(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>hello</b2><b2>dd</b2></xsl:a>'),
'/xsl:a/b2[1]', xmltype('<c>china</c>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下:

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <c>china</c>
  <b2>hello</b2>
  <b2>dd</b2>
</xsl:a>
```

22.3.7 INSERTXMLAFTER

定义:

```
CLOB
INSERTXMLAFTER(
xmldata clob,
xpath varchar,
insnode clob
)
```

或

CLOB

INSERTXMLAFTER(

xmldata clob,

xpath varchar,

insnode clob,

xmlns varchar

)

功能说明:

将 insnode 结点插入到 xmldata 的 xpath 路径下 exp 结点后面。

参数说明:

xmldata: XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

insnode：待插入的 XML 结点。

xmlns：XML 命名空间。

返回值:

已插入结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 insnode 为 NULL 则不会进行插入子结点操作。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 insnode 为 xmltype 类型时返回运算后的 xmldata，否则将返回原始 xmldata。

举例说明:

例 1 指定 xmldata、xpath 和 insnode 参数执行 INSERTXMLAFTER，在 xpath 指定路径下添加 insnode 结点到指定路径结点后。

```
SELECT  INSERTXMLAFTER(XMLTYPE('<a><b2>地方</b2><b2>dd</b2></a>'), 'a/b2',
XMLTYPE('<c>china</c>'));
```

查询结果如下:

```
<a>
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
  <c>china</c>
</a>
```

例 2 指定 xmldata、xpath、insnode 和 xmlns 参数执行 INSERTXMLAFTER，在 xpath 指定路径下添加 insnode 结点到指定路径结点后。

```
SELECT  INSERTXMLAFTER(xmltype('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><b2>地方</b2><b2>dd</b2></xsl:a>'),
'xsl:a/b2', xmltype('<c>china</c>'),  'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下：

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <b2>地方</b2>
  <c>china</c>
  <b2>dd</b2>
  <c>china</c>
</xsl:a>
```

22.3.8 UPDATEXML

定义：

CLOB

UPDATEXML(

xmldata clob,

xpath varchar,

updnode clob

)

或

CLOB

UPDATEXML(

xmldata clob,

xpath varchar,

updnode clob,

xmlns varchar

)

功能说明:

用 updnode 替换 xmldata 的 xpath 路径下结点。

参数说明:

xmldata: XML 数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，xmldata 参数仅允许为 xmltype 类型。

xpath：XML 路径。

updnode：用于替换的 XML 结点。

xmlns：XML 命名空间。

返回值:

更新结点后的 XML 数据。xmldata 或 xpath 为 NULL 的情况下一律返回 NULL。如果 insnode 为 NULL 会将指定路径下结点的子结点替换为空结点。

当 INI 参数 COMPATIBLE_MODE=2 时，当且仅当 updnode 为 xmltype 类型时返回运算后的 xmldata，否则将返回原始 xmldata。

举例说明:

例 1 指定 xmldata、xpath 和 updnode 参数执行 UPDATEXML，替换 xpath 指定路径下节点为 updnode 结点。

```
SELECT UPDATEXML(XMLTYPE('<a><a1>d1</a1><a2>d2</a2></a>'), '/a/a2',
XMLTYPE('<x>f</x>'));
```

查询结果如下:

```
<a>
  <a1>d1</a1>
  <x>f</x>
</a>
```

例 2 指定 xmldata、xpath、updnode 和 xmlns 参数执行 UPDATEXML，替换 xpath 指定路径下节点为 updnode 结点。

```
SELECT UPDATEXML(XMLTYPE('<xsl:a
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><a1>d1</a1><a2>d2</a2></xsl:a>'),
'/xsl:a/a2', XMLTYPE('<x>f</x>'), 'xmlns:xsl="http://www.w3.org/1999/XSL/Transform"');
```

查询结果如下：

```
<xsl:a xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <a1>d1</a1>
  <x>f</x>
</xsl:a>
```

22.4 数据转换

22.4.1 XMLTRANSFORM

定义：

```
CLOB
XMLTRANSFORM(
xmldata     clob,
xslt        clob
)
```

功能说明：

将 xmldata 中的内容按照 xslt 中指定的格式生成一个新的 xml 数据。使用 xmltransform 须遵循 xslt 标准，DM 通过引入第三方库 libxslt 来支持 xslt 标准，使用时需

要保证有 libxslt 库。

参数说明:

xmldata: XML 数据。需要注意的是, 当 INI 参数 COMPATIBLE_MODE=2 时, xmldata 参数仅允许为 xmltype 类型。

xslt: 表单格式。

返回值:

变换后的新 XML 数据。

举例说明:

```
DECLARE
xmldata   clob;
xslt   clob;
BEGIN
xmldata :='
<s1 title="s1 foo">
  <s2 title="Foo">
    <p>Hello</p>
  </s2>
</s1>';

xslt := '<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
  <xsl:output method="html" indent="yes"/>

  <xsl:template match="/">
    <xsl:apply-templates/>
  </xsl:template>

<xsl:template match="s1">
    <html>
      <head><title><xsl:value-of select="@title"/></title></head>
```

```
        <body>
          <xsl:apply-templates select="s2"/>
        </body>
      </html>
   </xsl:template>
</xsl:stylesheet>';

select xmltransform(xmldata, xslt);
END;
```

查询结果如下:

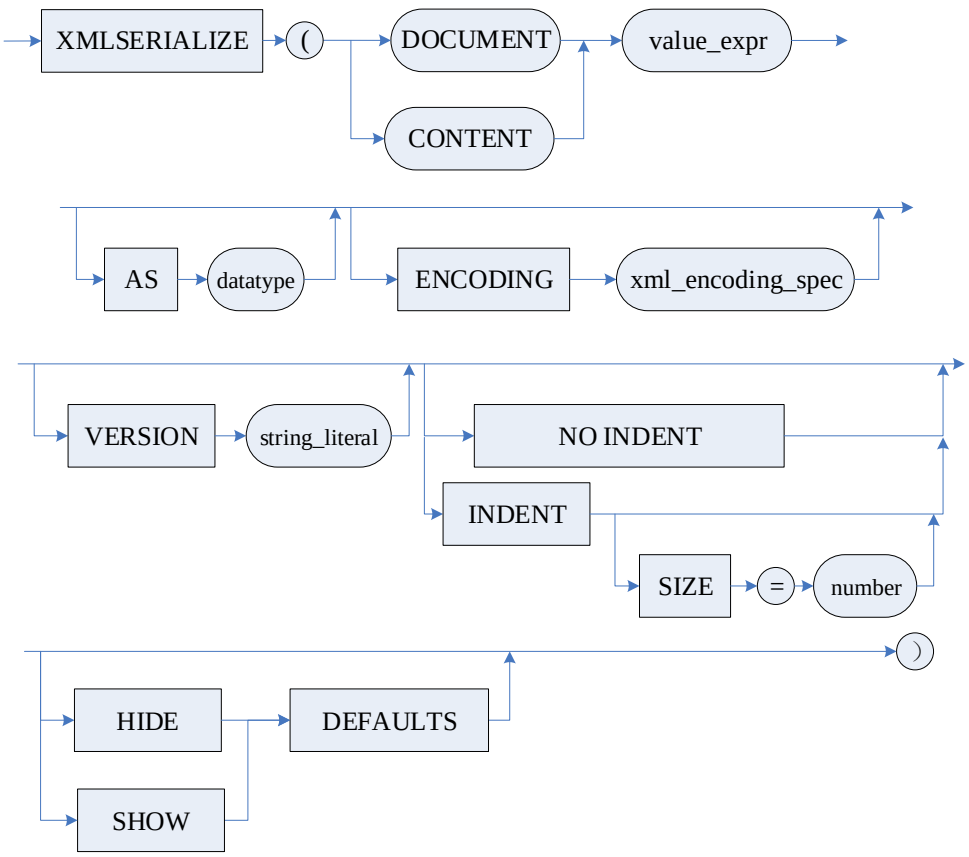
```
XMLTRANSFORM(XMLDATA,XSLT)
--------------------------------------------------------
<html>
<head>
<title>s1 foo</title>
</head>
<body>Hello</body>
</html>
```

22.4.2 XMLSERIALIZE

语法格式:

```
XMLSERIALIZE(<DOCUMENT | CONTENT> <value_expr> [AS <datatype>] [ENCODING
<xml_encoding_spec>] [VERSION <string_literal>] [<NO INDENT> | <INDENT [SIZE =
<number>]>] [<HIDE | SHOW> DEFAULTS])
```

图例:



功能说明:

XMLSERIALIZE 用于将 xml 实例按照指定的选项进行格式化输出。

参数说明:

<value_expr>：xml 实例。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，<value_expr>参数仅允许为 xmltype 类型。

<datatype>：仅支持 VARCHAR、BLOB、CLOB 三种数据类型。

<xml_encoding_spec>：字符串类型，用于指定输出串的编码，目前支持指定为 GBK 、 GB18030 、 UTF-8 。 仅 在 datatype 为 BLOB 时 可 指 定 ENCODING <xml_encoding_spec>语句。

<string_literal>：字符串类型，设置 xml 版本类型，不作合法性校验。

<number>：整型数字，表示结点缩进格式。默认没有格式缩进，该参数取值不超过 20，即缩进不能超过 20。

<HIDE|SHOW> DEFAULTS：为语法兼容项，暂无实际意义。

举例说明:

例 1 指定为 CLOB 类型进行输出。

```
SELECT XMLSERIALIZE(content XMLTYPE('<?xml version="1.1" encoding="GB18030"?>
<a><c><x>d</x></c></a>') as Clob version '1.2' indent  size = 10) AS xmlserialize_doc  from
dual;
```

查询结果如下:

```
行号        XMLSERIALIZE_DOC
----------
------------------------------------------------------------------------------
----------------------------
1           <?xml version="1.2" encoding="GB18030"?>
<a>
          <c>
                    <x>d</x>
          </c>
</a>
```

例 2 指定为 BLOB 类型进行输出。

```
SELECT XMLSERIALIZE(content XMLTYPE('<?xml version="1.1" encoding="UTF-8"?>
<a><c><x>d</x></c></a>') as Blob  ENCODING 'UTF8' version '1.2' indent  size = 10) AS
xmlserialize_doc  from dual;
```

查询结果如下:

```
行号        XMLSERIALIZE_DOC
----------
------------------------------------------------------------------------------
----------------------------
1
0x3C3F786D6C2076657273696F6E3D22312E322220656E636F64696E673D225554462D38223F3
E0A3C613E0A2020202020202020202020203C633E0A20202020202020202020202020202020202
0203C783E643C2F783E0A2020202020202020202020203C2F633E0A3C2F613E0A
```

22.4.3 XMLCAST

语法格式:

```
XMLCAST(exp AS type)
```

图例:



功能说明:

XMLCAST 用于将 exp 构建的 xml 数据中的 content 内容转换为指定类型输出。

参数说明:

exp：XMLTYPE 类型的数据。需要注意的是，当 INI 参数 COMPATIBLE_MODE=2 时，exp 参数仅允许为 xmltype 类型。

type：指定类型。

返回值:

转换为指定类型的 xml 数据。exp 为 NULL 的情况下一律返回 NULL。

举例说明:

```
SELECT XMLCAST(XMLTYPE( '<test><a>Hello</a><b> World</b>!</test>' ) AS varchar2(100))
AS XML_TO_VARCHAR2 FROM DUAL;
```

查询结果如下:

```
行号        XML_TO_VARCHAR2
---------- --------------------
1           Hello World!
```

附录 1 关键字和保留字

以下不带*号的为关键字，带*号的为系统保留字。

DM 还将保留字进一步划分为 SQL 保留字、DMSQL 程序保留字、模式保留字、变量保留字和别名保留字。对于关键字和保留字的详细信息请查询系统视图 V$RESERVED_WORDS。

注意：关键字 ROWID、TRXID、VERSIONS_STARTTIME、VERSIONS_ENDTIME、VERSIONS_STARTTRXID、VERSIONS_ENDTRXID 和 VERSIONS_OPERATION 不能作为表的列名，即使加上双引号也不行。

A

ABORT、ABSOLUTE*、ABSTRACT*、ACCESSED、ACCOUNT、ACROSS、ACTION、ADD*、ADMIN*、ADVANCED、AFTER、AGGREGATE、ALL*、ALLOW_DATETIME、ALLOW_IP、ALTER*、ANALYZE、AND*、ANY*、APPLY、APR、ARCHIVE、ARCHIVEDIR、ARCHIVELOG、ARCHIVESTYLE、ARRAY*、ARRAYLEN*、AS*、ASC*、ASCII、ASENSITIVE、ASSIGN*、ASYNCHRONOUS、AT、ATTACH、AUDIT*、AUG、AUTHID、AUTHORIZATION*、AUTO、AUTO_INCREMENT*、AUTOEXTEND、AUTONOMOUS_TRANSACTION、AVG

B

BACKED、BACKUP、BACKUPDIR、BACKUPINFO、BACKUPSET、BADFILE、BAKFILE、BASE、BATCH、BEFORE、BEGIN*、BETWEEN*、BIGDATEDIFF*、BIGINT、BINARY*、BIT、BITMAP、BLOB、BLOCK、BOOL*、BOOLEAN、BOTH*、BRANCH、BREADTH、BREAK*、BSTRING*、BTREE、BUFFER、BUILD、BULK、BY*、BYDAY、BYHOUR、BYMINUTE、BYMONTH、BYMONTHDAY、BYSECOND、BYTE*、BYWEEKNO、BYYEARDAY

C

CACHE、CALCULATE、CALL*、CASCADE、CASCADED、CASE*、CASE_SENSITIVE、CAST*、CATALOG、CATCH*、CHAIN、CHANGE、CHAR*、CHARACTER、CHARACTERISTICS、CHECK*、CHECKPOINT、CIPHER、CLASS*、CLOB、CLOSE、CLUSTER*、CLUSTERBTR*、COLLATE、COLLATION*、COLLECT、COLUMN*、COLUMNS、COMMENT*、COMMIT*、COMMITTED、COMMITWORK*、COMPILE、COMPLETE、COMPRESS、COMPRESSED、CONDITIONAL、CONNECT*、CONNECT_BY_ISCYCLE、CONNECT_BY_ISLEAF、CONNECT_BY_ROOT*、CONNECT_IDLE_TIME、CONNECT_TIME、CONST*、CONSTANT、CONSTRAINT*、CONSTRAINTS、CONSTRUCTOR、CONTAINS*、CONTEXT*、CONTINUE*、CONVERT*、COPY、CORRESPONDING*、CORRUPT、COUNT、COUNTER、CPU_PER_CALL、CPU_PER_SESSION、CREATE*、CROSS*、CRYPTO*、CTLFILE、CUBE*、CUMULATIVE、CURRENT*、CURRENT_SCHEMA、CURRENT_USER、CURSOR*、CYCLE

D

DAILY、DANGLING、DATA、DATABASE、DATAFILE、DATE、DATEADD*、DATEDIFF*、DATEPART*、DATETIME、DAY、DBFILE、DDL、DDL_CLONE、DEBUG、DEC、DECIMAL*、DECLARE*、DECODE*、DEFAULT*、DEFERRABLE、DEFERRED、DEFINER、DELETE*、DELETING、DELIMITED、DELTA、DEMAND、DENSE_RANK、DEPTH、DEREF、DESC*、DETACH、DETERMINISTIC、DEVICE、DIAGNOSTICS、DICTIONARY、DIRECTORY、DISABLE*、DISCONNECT、DISKGROUP、DISKSPACE*、DISTINCT*、DISTRIBUTED*、DML、DO*、DOMAIN*、DOUBLE*、DOWN、DROP*、DUMP

E

EACH、EDITIONABLE、ELSE*、ELSEIF*、ELSIF*、EMPTY、ENABLE*、ENCRYPT、ENCRYPTION、END*、EQU*、ERROR、ERRORS、ESCAPE、EVALNAME、EVENTINFO、EVENTS、EXCEPT、EXCEPTION、EXCEPTION_INIT、EXCEPTIONS、EXCHANGE*、EXCLUDE、

EXCLUDING、EXCLUSIVE、EXEC*、EXECUTE*、EXISTS*、EXIT*、EXPIRE、EXPLAIN*、EXTENDS、EXTERN*、EXTERNAL、EXTERNALLY、EXTRACT*

F

FAILED_LOGIN_ATTEMPS、FAILED_LOGIN_ATTEMPTS、FAST、FEB、FETCH*、FIELDS、FILE、FILEGROUP、FILESIZE、FILLFACTOR、FINAL*、FINALLY*、FIRST*、FLASHBACK*、FLOAT*、FOLLOWS、FOLLOWING、FOR*、FORALL、FORCE、FOREIGN*、FORMAT、FREQ、FREQUENCE、FRI、FROM*、FULL*、FULLY*、FUNCTION*

G

GENERATED、GET*、GLOBAL、GLOBALLY、GLOBAL_SESSION_PER_USER、GOTO*、GRANT*、GREAT、GROUP*、GROUPING*

H

HASH、HASHPARTMAP、HAVING*、HEXTORAW、HIGH、HOLD、HOUR、HOURLY、HUGE

I

IDENTIFIED、IDENTIFIER、IDENTITY*、IDENTITY_INSERT、IF*、IFNULL*、IGNORE_ROW_ON_DUPKEY_INDEX、IMAGE、IMMEDIATE*、IN*、INCLUDE、INCLUDING、INCREASE、INCREMENT、INDEX*、INDEXES、INDICES、INITIAL、INITIALIZED、INITIALLY、INLINE*、INNER*、INNERID、INPUT、INSENSITIVE、INSERT*、INSERTING、INSTANCE、INSTANTIABLE、INSTEAD、INT*、INTEGER、INTENT、INTERSECT*、INTERVAL*、INTO*、INVISIBLE、IS*、ISOLATION、INACTIVE_ACCOUNT_TIME

J

JAN、JAVA、JOB、JOIN*、JSON、JSON_TABLE*、JUL、JUN

K

KEEP*、KEY、KEYS

L

LABEL、LARGE*、LAST、LAX、LEADING*、LEFT*、LEFTARG、LESS*、LEVEL、LEVELS、LEXER*、LIKE*、LIMIT、LINK、LIST*、LNNVL*、LOB、LOCAL、LOCAL_OBJECT、LOCALLY、LOCATION、LOCK、LOCKED、LOG、LOGFILE、LOGGING、LOGIN*、LOGOFF、LOGON、LOGOUT、LONG、LONGVARBINARY、LONGVARCHAR、LOOP*、LSN

M

MANUAL、MAP、MAPPED、MAR、MATCH、MATCHED、MATERIALIZED、MAX、MAX_RUN_DURATION、MAXPIECESIZE、MAXSIZE、MAXVALUE、MAY、MEM_SPACE、MEMBER*、MEMORY、MERGE、MICRO、MIN、MINEXTENTS、MINUS*、MINUTE、MINUTELY、MINVALUE、MIRROR、MOD、MODE、MODIFY、MON、MONEY、MONITORING、MONTH、MONTHLY、MOUNT、MOVE、MOVEMENT、MULTISET*

N

NATIONAL、NATURAL*、NCHAR、NCHARACTER、NEVER、NEW*、NEXT*、NO、NOARCHIVELOG、NOAUDIT、NOBRANCH、NOCACHE、NOCOPY*、NOCYCLE*、NODE、NOLOGGING、NOMAXVALUE、NOMINVALUE、NOMONITORING、NONE、NONEDITIONABLE、NOORDER、NOPARALLEL、NORMAL、NOROWDEPENDENCIES、NOSORT*、NOT*、NOT_ALLOW_DATETIME、NOT_ALLOW_IP、NOV、NOVALIDATE、NOWAIT、NULL*、NULLS、NUMBER、NUMERIC

O

OBJECT*、OCT、OF*、OFF、OFFLINE、OFFSET、OIDINDEX、OLD、ON*、ONCE、ONLINE、ONLY、OPEN、OPERATOR、OPTIMIZE*、OPTION、OR*、ORDER*、ORDINALITY、OUT*、OUTER、OVER*、OVERLAPS、OVERLAY*、OVERRIDE*、OVERRIDING

P

PACKAGE、PAD、PAGE、PARALLEL、PARALLEL_ENABLE、PARMS、PARTIAL、PARTITION*、PARTITIONS、PASSING、PASSWORD、PASSWORD_GRACE_TIME、PASSWORD_LIFE_TIME、PASSWORD_LOCK_TIME、PASSWORD_POLICY、PASSWORD_REUSE_MAX、PASSWORD_REUSE_TIME、PATH、PENDANT*、PERCENT*、PIPE*、PIPELINED、PIVOT、PLACING、PLS_INTEGER、PRAGMA、PREBUILT、PRECEDES、PRECEDING、PRECISION、PRESERVE、PRETTY、PRIMARY*、PRINT*、PRIOR*、PRIVATE*、PRIVILEGE、PRIVILEGES*、PROCEDURE*、PROFILE、PROTECTED*、PUBLIC*、PURGE

Q

QUERY_REWRITE_INTEGRITY、QUOTA

R

RAISE*、RANDOMLY、RANGE、RAWTOHEX、READ、READ_PER_CALL、READ_PER_SESSION、READONLY、REAL、REBUILD、RECORD*、RECORDS、REDUCED、REF*、REFERENCE*、REFERENCES*、REFERENCING*、REFRESH、REJECT、RELATED、RELATIVE*、RELEASE、RENAME、REPEAT*、REPEATABLE、REPLACE、REPLAY、REPLICATE*、RESIZE、RESTORE、RESTRICT、RESTRICT_REFERENCES、RESULT、RESULT_CACHE、RETURN*、RETURNING*、REVERSE*、REVOKE*、RIGHT*、RIGHTARG、ROLE、ROLLBACK*、ROLLFILE、ROLLUP*、ROOT、ROW*、ROWCOUNT、ROWDEPENDENCIES、ROWID、ROWNUM*、ROWS*、RULE

S

SALT、SAMPLE、SAT、SAVE、SAVEPOINT*、SBYTE*、SCHEMA*、SCHEMABINDING、SCN、SCOPE、SCROLL、SEALED*、SEARCH、SECOND、SECONDLY、SECTION*、SEED、SELECT*、SELF、SENSITIVE、SEP、SEQUENCE、SERERR、SERIALIZABLE、SERVER、SESSION、SESSION_PER_USER、SET*、SETS*、SHADOW、SHARE、SHORT*、SHUTDOWN、SIBLINGS、SIMPLE、SINCE、SIZE、SIZEOF*、SKIP、SMALLINT、SNAPSHOT、SOME*、SOUND、SPACE、

SPAN、SPATIAL、SPEED、SPFILE、SPLIT、SQL、STANDBY、STARTUP、STAT、STATEMENT、STATIC*、STDDEV、STOP、STORAGE、STORE、STRICT、STRING、STRIPING、STRUCT*、STYLE、SUBPARTITION*、SUBPARTITIONS、SUBSCRIBE、SUBSTITUTABLE、SUBSTRING、SUBTYPE、SUCCESSFUL、SUM、SUN、SUSPEND、SWITCH*、SYNC、SYNCHRONOUS、SYNONYM*、SYS_CONNECT_BY_PATH、SYSTEM

T

TABLE*、TABLESPACE、TASK、TEMPLATE、TEMPORARY、TEXT、THAN*、THEN、THREAD、THROUGH、THROW*、THU、TIES、TIME、TIME_ZONE、TIMER、TIMES、TIMESTAMP、TIMESTAMPADD*、TIMESTAMPDIFF*、TINYINT、TO*、TOP*、TRACE、TRACKING、TRAILING*、TRANSACTION、TRANSACTIONAL、TRIGGER*、TRIGGERS、TRIM*、TRUNCATE*、TRUNCSIZE、TRXID、TRY*、TUE、TYPE、TYPEDEF*、TYPEOF*

U

UINT*、ULONG*、UNBOUNDED、UNCOMMITTED、UNCONDITIONAL、UNDER、UNION*、UNIQUE*、UNLIMITED、UNLOCK、UNPIVOT、UNTIL*、UNUSABLE、UP、UPDATE*、UPDATING、USAGE、USE_HASH、USE_MERGE、USE_NL、USE_NL_WITH_INDEX、USER*、USHORT*、USING*

V

VALUE、VALUES*、VARBINARY、VARCHAR、VARCHAR2、VARIANCE、VARRAY*、VARYING、VERIFY*、VERSIONS*、VERSIONS_ENDTIME、VERSIONS_ENDTRXID、VERSIONS_OPERATION、VERSIONS_STARTTIME、VERSIONS_STARTTRXID、VERTICAL、VIEW*、VIRTUAL*、VISIBLE、VOID*、VSIZE

W

WAIT、WED、WEEK、WEEKLY、WHEN*、WHENEVER*、WHERE*、WHILE*、WITH*、WITHIN*、WITHOUT*、WORK、WRAPPED、WRAPPER、WRITE

X

XML*、XMLAGG*、XMLATTRIBUTES*、XMLELEMENT*、XMLPARSE*、XMLTABLE*、XMLNAMESPACES*、XMLSERIALIZE*

Y

YEAR、YEARLY

Z

ZONE

附录 2 SQL 语法书写规则

一 SQL 语法符号

下面分别介绍各 SQL 语法符号的含义：

< >　　表示一个语法对象，但是小括号本身不能出现在语句中。

::=　　定义符，用来定义一个语法对象。定义符左边为语法对象，右边为相应的语法描述。

|　　或者符，或者符限定的语法选项在实际的语句中只能出现一个。

{ }　　大括号指明大括号内的语法选项在实际的语句中可以出现 0...N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。

[ ]　　中括号指明中括号内的语法选项在实际的语句中可以出现 0...1 次，但是中括号本身不能出现在语句中。

关键字　关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字可以为大写也可以为小写。

二 SQL 语法图的说明

SQL 语法图是用来帮助用户正确地理解和使用 DM SQL 语法的图形。阅读语法图时，请按照从上到下，从左到右的方式，依箭头所指方向进行阅读。

SQL 命令、语法关键字等终结符以全大写方式在长方形框内显示，使用时直接输入这些内容；语法参数或语法子句等非终结符的名称以全小写方式在圆角框内显示；各类标点符号显示在圆圈之中。注：如果小写参数中不带下划线_，则表示是由用户输入的参数，带下划线则表示是还需要进一步解释的子句或语法对象，如果在前面已解释过，则未重复列出。

1.必须关键字和参数

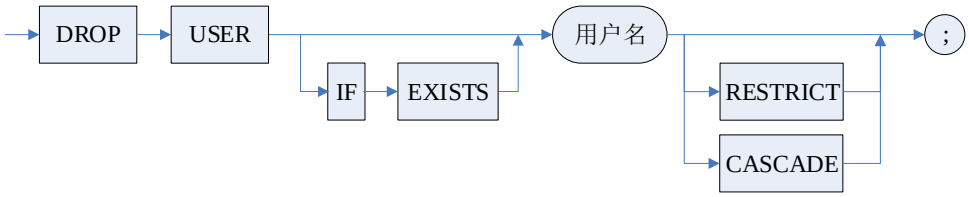
1） 必须关键字和参数出现在语法参考图的主干路径上，也就是说，出现在当前阅读的水平线上。

例 以用户删除语句为例。DROP、USER、<用户名>和;为必选内容。

用户删除语句语法：

```
DROP USER [IF EXISTS] <用户名> [RESTRICT | CASCADE];
```

用户删除语句语法图：



2） 如果多个关键字或参数并行地出现在从主路径延伸出的多条可选路径中，则只选择其中的一个即可。

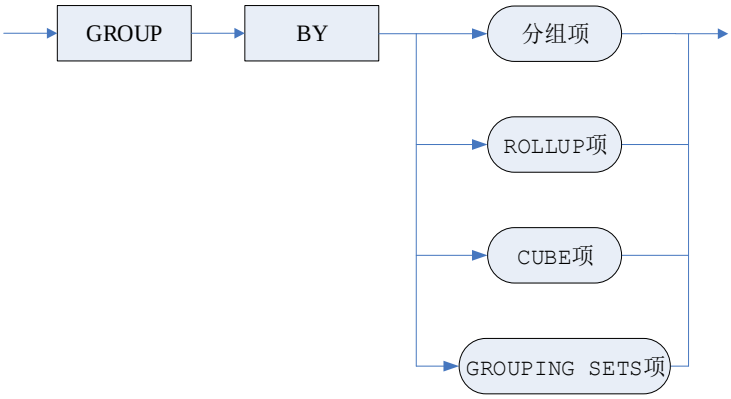
例 以<GROUP BY 子句>为例。<group_by 项>有四种形式，可以任选一种。如果存在 GROUP BY 子句，则必须从<group_by 项>中选择一种。

<GROUP BY 子句>语法：

```
<GROUP BY 子句> ::= GROUP BY <group_by 项>{,<group_by 项>}

<group_by 项>::=<分组项> | <ROLLUP 项> | <CUBE 项> | <GROUPING SETS 项>
```

<GROUP BY 子句>语法图：



2.可选关键字和参数

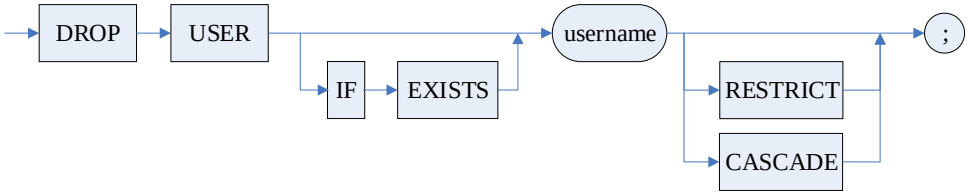
如果关键字或参数并行地出现在主路径下方，而主路径是一条直线，则这些关键字和参数是可选的。

例 1 以用户删除语句为例。IF EXISTS、RESTRICT、CASCADE 均是可选项。IF EXISTS 和主干道是二选一的关系；RESTRICT、CASCADE 和主干道是三选一的关系。

用户删除语句语法：

```
DROP USER [IF EXISTS] <用户名> [RESTRICT | CASCADE];
```

用户删除语句语法图：



例 2 以<分析子句>为例。<PARTITION BY 项>、<ORDER BY 项> 和<窗口子句>都是可选的，但出现的前后顺序不能颠倒。

分析函数的<分析子句>语法：

```
<分析子句>::= [<PARTITION BY 项>] [<ORDER BY 项> [<窗口子句>]]
```

用语法图表示<分析子句>：



3.多条路径

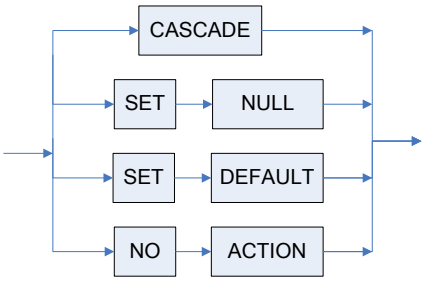
如果一张语法参考图有一条以上的路径，可以从任意一条路径进行阅读。如果可以选择多个关键字、操作符、参数或者语法子句，这些选项将被并行地列出。

例 以<引用动作>为例。引用动作可以选择这四条路径的任一种。

<引用动作>语法：

```
<引用动作> ::= CASCADE |
               SET NULL |
               SET DEFAULT |
               NO ACTION
```

<引用动作>语法图:



4.循环语法

循环语法表示可以按照需要，使用循环内的语法一次或者多次。

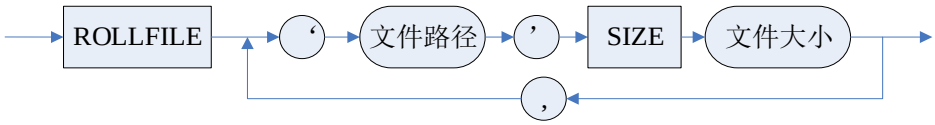
例 以<回滚文件子句>为例。通过逗号隔开，可以重复语法对象 'filepath' SIZE filesize 多个。

<回滚文件子句>语法:

```
<回滚文件子句> ::= ROLLFILE <文件说明子句>{,<文件说明子句>}

<文件说明子句> ::= <文件路径> SIZE <文件大小>
```

<回滚文件子句>语法图:



5.多行语法图

由于有些 SQL 语句的语法十分复杂，生成的语法参考图无法完整地显示在一行之内，于是将其分行显示。阅读此类图形时，请从上至下，从左至右地进行。

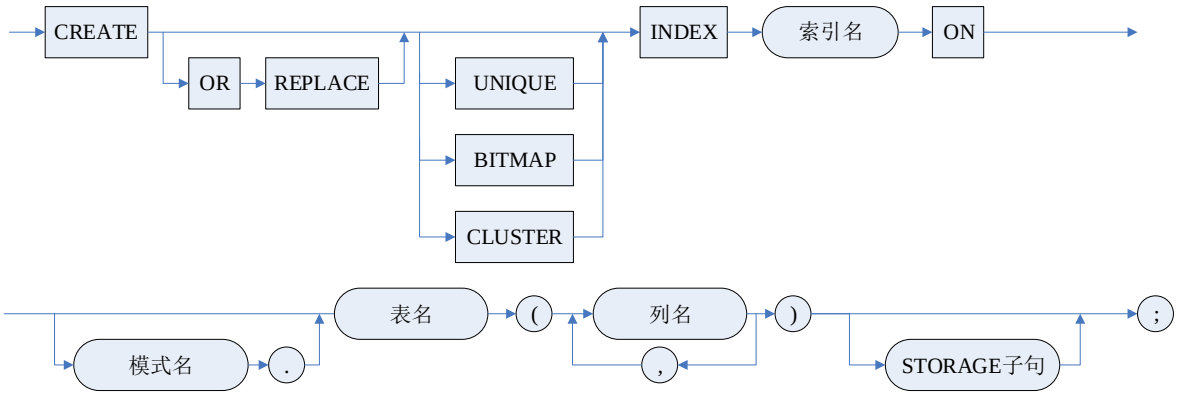
例 以索引定义语句为例。索引定义的语法被分成了两行显示。

索引定义语句语法:

```
CREATE [OR REPLACE] [UNIQUE | BITMAP | CLUSTER] INDEX <索引名>

ON [<模式名>.]<表名>(<列名>{,<列名>}) [<STORAGE 子句>];
```

索引定义语句语法图:



附录 3 系统存储过程和函数

以下为达梦数据库所用到的系统存储过程和函数。

注：函数名右上角有“*”标记的，表示此过程/函数的使用有下面两点限制：

1. 此过程/函数不能在 MPP 全局模式下的存储过程中直接调用，在 MPP LOCAL 模式下可在存储过程中直接调用；
2. 此过程/函数不能在存储过程中带参数进行动态调用。

1. INI 参数管理

1) SF_GET_PARA_VALUE

定义：

```
BIGINT
SF_GET_PARA_VALUE (
scope              int,
ini_param_name   varchar(256)
)
```

功能说明：

返回 DM.INI 文件中整型的参数值。

参数说明：

scope： 取值为 1、2 。 1 表示从 DM.INI 文件中读取；2 表示从内存中读取。当取值为 1，且 DM.INI 文件中该参数值设置为非法值时，若设置值与参数类型不兼容，则返回默认值；若设置值小于参数取值范围的最小值，则返回最小值；若设置值大于参数取值范围的最

大值，则返回最大值。

ini_param_name：DM.INI 文件中的参数名。

返回值：

当前 INI 文件中对应的参数值。

举例说明：

获得 DM.INI 文件中 BUFFER 参数值：

```
SELECT SF_GET_PARA_VALUE (1, 'BUFFER');
```

2） SP_SET_PARA_VALUE*

定义：

```
SP_SET_PARA_VALUE (
scope              int,
ini_param_name   varchar(256)
value              bigint
)
```

或

```
SP_SET_PARA_VALUE (
bro_flag           int,
scope              int,
ini_param_name   varchar(256)
value              bigint
)
```

或

```
SP_SET_PARA_VALUE(
```

```
inst_name    varchar(128),
scope        int,
paraname     varchar(256),
value        bigint
)
```

功能说明:

供具有 DBA 权限的用户设置 DM.INI 文件中整型的参数值。DSC 环境下，对于需要在各节点保持一致的参数，设置的参数值会在 OK 节点上同步。

参数说明:

scope：取值为 0、1、2 。0 表示修改内存中的动态配置参数值；1 表示 DM.INI 文件和内存参数都修改，不需要重启服务器；2 表示修改 DM.INI 文件的参数值或不在 DM.INI 中的 INI 配置项（具体可参看《DM8 系统管理员手册》2.1.1.1.26 节），修改后需重启服务器生效。

ini_param_name：DM.INI 文件中的参数名。

value：设置的值。

bro_flag：广播标记，取值为 0、1、2、3。0 表示不广播；1 表示广播至 MP 节点；2 表示广播至 BP 节点；3 表示广播至 SP 节点。该参数仅在 DMDPC 集群环境下指定有效。

inst_name：用于指定实例名，该函数重载用于 DPC 环境下修改指定实例的静态或动态配置参数，被修改的配置参数不能是必须同步的参数。

返回值:

无

举例说明:

将 DM.INI 文件中 HFS_CACHE_SIZE 参数值设置为 320：

```
SP_SET_PARA_VALUE (1,'HFS_CACHE_SIZE',320);
```

3） SF_GET_PARA_DOUBLE_VALUE

定义:

DOUBLE

SF_GET_PARA_DOUBLE_VALUE (

scope int,

ini_param_name varchar(256)

)

功能说明:

返回DM.INI文件中参数中浮点型的参数值。

参数说明:

scope：取值为1、2 。1表示从DM.INI文件中读取，不在DM.INI中的参数不能读取；2表示从内存中读取。

ini_param_name：DM.INI文件中的参数名。

返回值:

当前INI文件中对应的参数值。

举例说明:

获得DM.INI中CKPT_FLUSH_RATE的参数值:

```
SELECT SF_GET_PARA_DOUBLE_VALUE (1, 'CKPT_FLUSH_RATE');
SELECT SF_GET_PARA_DOUBLE_VALUE (2, 'CKPT_FLUSH_RATE');
```

4） SP_SET_PARA_DOUBLE_VALUE*

定义:

SP_SET_PARA_DOUBLE_VALUE (

```
scope              int,
ini_param_name   varchar(256),
value              double
)
```

或

```
SP_SET_PARA_DOUBLE_VALUE (
bro_flag           int,
scope              int,
ini_param_name   varchar(256),
value              double
)
```

或

```
SP_SET_PARA_DOUBLE_VALUE (
inst_name          varchar(128),
scope              int,
ini_param_name   varchar(256),
value              double
)
```

功能说明:

供具有DBA权限的用户设置DM.INI参数中浮点型的参数值。DSC环境下，除了特殊参数以外，其他参数值会在OK节点上同步。关于特殊参数的介绍请参考函数SP_SET_PARA_VALUE的功能说明。

参数说明:

scope：取值为 0、1、2。0 表示修改内存中的动态配置参数值；1 表示 DM.INI 文件和

内存参数都修改，不需要重启服务器；2 表示只可修改 DM.INI 文件，服务器重启后生效。

ini_param_name：DM.INI文件中的参数名。

value：设置的值。

bro_flag：广播标记，取值为0、1、2、3。0表示不广播；1表示广播至MP节点；2表示广播至BP节点；3表示广播至SP节点。该参数仅在DMDPC集群环境下指定有效。

inst_name：用于指定实例名，该函数重载用于 DPC 环境下修改指定实例的静态或动态配置参数，被修改的配置参数不能是必须同步的参数。

返回值：

无

举例说明：

将DM.INI文件中INDEX_SKIP_SCAN_RATE参数值设置为0.2：

```
SP_SET_PARA_DOUBLE_VALUE(1, 'INDEX_SKIP_SCAN_RATE', 0.2);
```

5） SF_GET_PARA_STRING_VALUE

定义：

VARCHAR

SF_GET_PARA_STRING_VALUE (

scope int,

ini_param_name varchar(256)

)

功能说明：

返回 DM.INI 文件中字符串类型的参数值。

参数说明：

scope：取值为1、2。1表示从DM.INI文件中读取；2表示从内存中读取。

ini_param_name：DM.INI文件中的参数名。

返回值：

当前 INI 文件中对应的参数值。

举例说明：

获得DM.INI文件中TEMP_PATH参数值：

```
SELECT SF_GET_PARA_STRING_VALUE (1, 'TEMP_PATH');
```

6) SP_SET_PARA_STRING_VALUE*

定义：

```
SP_SET_PARA_STRING_VALUE (
scope                int,
ini_param_name    varchar(256) ,
value                varchar(32767)
)
或
SP_SET_PARA_STRING_VALUE (
bro_flag             int,
scope                int,
ini_param_name    varchar(256) ,
value                varchar(32767)
)
或
SP_SET_PARA_STRING_VALUE (
inst_name            varchar(128),
```

scope int,

ini_param_name varchar(256),

value varchar(32767)

)

功能说明:

供具有DBA权限的用户设置DM.INI文件中的字符串型参数值。DSC环境下，除了特殊参数以外，其他参数值会在OK节点上同步。关于特殊参数的介绍请参考函数SP_SET_PARA_VALUE的功能说明。

参数说明:

scope：取值为 0、1、2 。0 表示修改内存中的动态配置参数值；1 表示 DM.INI 文件和内存参数都修改，不需要重启服务器；2 表示只修改 DM.INI 文件，服务器重启后生效。

ini_param_name：DM.INI 文件中的参数名。

value：设置的字符串的值。

bro_flag：广播标记，取值为 0、1、2、3。0 表示不广播；1 表示广播至 MP 节点；2 表示广播至 BP 节点；3 表示广播至 SP 节点。该参数仅在 DMDPC 集群环境下指定有效。

inst_name：用于指定实例名，该函数重载用于 DPC 环境下修改指定实例的静态或动态配置参数，被修改的配置参数不能是必须同步的参数。

返回值:

无

举例说明:

将 DM.INI 文件中 COMMIT_WRITE 参数值设置为 WAIT：

```
SP_SET_PARA_STRING_VALUE(1,'COMMIT_WRITE','WAIT');
```

7) SF_SET_SESSION_PARA_VALUE*

定义:

SF_SET_SESSION_PARA_VALUE (

paraname varchar(256),

value bigint

)

功能说明:

设置会话级 INI 参数在当前会话上的值。

参数说明:

paraname: 会话级 INI 参数的参数名。

value: 要设置的新值。

返回值:

无

举例说明:

将会话级 INI 参数 JOIN_HASH_SIZE 在当前会话上的值设置为 2000:

```
SF_SET_SESSION_PARA_VALUE ('JOIN_HASH_SIZE', 2000);
```

8) SP_RESET_SESSION_PARA_VALUE*

定义:

SP_RESET_SESSION_PARA_VALUE (

paraname varchar(256)

)

功能说明:

重置会话级 INI 参数在当前会话上的值，使得当前会话的参数值和全局值一致。

参数说明:

paraname：会话级 INI 参数的参数名。

返回值：

无

举例说明：

重置会话级 INI 参数 JOIN_HASH_SIZE 在当前会话上的值：

```
SP_RESET_SESSION_PARA_VALUE ('JOIN_HASH_SIZE');
```

9）SF_GET_SESSION_PARA_VALUE

定义：

INT

SF_GET_SESSION_PARA_VALUE (

paraname　varchar(256)

)

功能说明：

获得整型的会话级 INI 参数的值。

参数说明：

paraname：会话级 INI 参数的参数名。

返回值：

整型的会话级 INI 参数的值。

举例说明：

获取会话级 INI 参数 JOIN_HASH_SIZE 的值：

```
SELECT SF_GET_SESSION_PARA_VALUE ('JOIN_HASH_SIZE');
```

10）SF_GET_SESSION_PARA_DOUBLE_VALUE

定义:

DOUBLE

SF_GET_SESSION_PARA_DOUBLE_VALUE (

paraname varchar(256)

)

功能说明:

获得浮点型的会话级 INI 参数的值。

参数说明:

paraname: 会话级 INI 参数的参数名。

返回值:

浮点型会话级 INI 参数的值。

举例说明:

获取会话级 INI 参数 SEL_RATE_SINGLE 的值:

```
SELECT SF_GET_SESSION_PARA_DOUBLE_VALUE ('SEL_RATE_SINGLE');
```

11) SF_GET_SESSION_PARA_STRING_VALUE

定义:

VARCHAR

SF_GET_SESSION_PARA_STRING_VALUE (

paraname varchar(256)

)

功能说明:

获得字符串类型的会话级 INI 参数的值。

参数说明:

paraname：会话级 INI 参数的参数名。

返回值：

字符串类型的会话级 INI 参数的值。

举例说明：

获取会话级 INI 参数 SQLTUNE_CATEGORY 的值：

```
SELECT SF_GET_SESSION_PARA_STRING_VALUE ('SQLTUNE_CATEGORY');
```

12）SF_SET_SYSTEM_PARA_VALUE*

定义：

SF_SET_SYSTEM_PARA_VALUE (

paraname varchar(256),

value bigint/double/varchar(256),

deferred int,

scope int

)

或

SF_SET_SYSTEM_PARA_VALUE (

bro_flag int,

paraname varchar(256),

value bigint/double/varchar(256),

deferred int,

scope int

)

或

SF_SET_SYSTEM_PARA_VALUE (

inst_name varchar(128),

paraname varchar(256),

value bigint/double/varchar(256),

deferred int,

scope int

)

功能说明:

修改整型、double、varchar 的静态配置参数或动态配置参数。DSC 环境下，除了特殊参数以外，其他参数值会在 OK 节点上同步。关于特殊参数的介绍请参考函数 SP_SET_PARA_VALUE 的功能说明。

参数说明:

paraname：INI 参数的参数名。

value：要设置的新值。

deferred：是否立即生效。为 0 表示当前 session 修改的参数立即生效；1 表示当前 session 不生效，后续再生效。默认为 0。

scope：取值为 0、1、2 。0 表示修改内存中的动态的配置参数值；1 表示修改内存和 INI 文件中动态的配置参数值；2 表示修改 DM.INI 文件或不在 DM.INI 总中的 INI 配置项中的静态或动态参数之后，需重启服务器生效。

bro_flag：广播标记，取值为 0、1、2、3。0 表示不广播；1 表示广播至 MP 节点；2 表示广播至 BP 节点；3 表示广播至 SP 节点。该参数仅在 DMDPC 集群环境下指定有效。

inst_name：用于指定实例名，该函数重载用于 DPC 环境下修改指定实例的静态或动

态配置参数，被修改的配置参数不能是必须同步的参数。

返回值:

无

举例说明:

修改 INI 参数 JOIN_HASH_SIZE 的值:

```
SF_SET_SYSTEM_PARA_VALUE ('JOIN_HASH_SIZE',50,1,1);
```

13) SF_SET_SQL_LOG

定义:

INT

SF_SET_SQL_LOG (

svrlog int,

svrmsk varchar(1000)

)

功能说明:

设置服务器日志相关 INI 参数 SVR_LOG 和 SQL_TRACE_MASK。

参数说明:

svrlog: INI 参数 SVR_LOG 的设置值。

svrmsk: INI 参数 SQL_TRACE_MASK 的设置值。

返回值:

是否成功。

举例说明:

设置服务器日志相关 INI 参数:

```
SELECT SF_SET_SQL_LOG(1, '3:5:7');
```

14）SF_SYNC_INI

定义:

INT

SF_SYNC_INI(

level int

)

功能说明:

用于备库从主库同步 DM.INI 参数。直连备库执行，只对当前的备库有效，不同的备库需要分别单独执行。

对于不同类型的 INI 参数，同步情况说明如下:

1） 静态 INI 参数：仅同步 dm.ini 文件值，不同步内存值，备库需要重启才能使用新的参数值;

2） 系统级动态 INI 参数：同时同步 dm.ini 文件值与内存值;

3） 会话级动态 INI 参数：同步全局内存值，不同步会话上的 INI 参数值;

4） 同步的两个节点须同为 V8.1.3.31 之前的版本，或同为大于等于 V8.1.3.31 的版本。若在小于 V8.1.3.31 和大于等于 V8.1.3.31 的两个版本间执行，不会真正同步，而是直接返回成功。

参数说明:

level：同步级别，取值 0 和 1。

0：表示同步所属环境下必须同步的参数 。专用于 DMDSC 和 DMDPC 架构的主备中。在普通环境的主备中执行 SF_SYNC_INI(0)不会同步任何参数。当前环境下必须同步的参数，请参考 V$DM_INI 中 SYNC_FLAG 为 MUST_SYNC 的参数。

1：表示同步所有可以同步的参数。可用于 DMDPC 架构的主备、DMDSC 架构的主备

和普通环境的主备中。除了不可被同步的 INI 参数外，其余 INI 参数均可被同步。当前环境下可以被同步的参数，请参考 V$DM_INI 中 SYNC_FLAG 为 CAN_SYNC 的参数。

返回值:

0：执行成功。

<0：执行失败。

100：未执行同步，对方 MAL 消息版本过低或 INI 版本一致无需同步。

举例说明:

在备库上执行，同步主库上所有可以同步的 INI 参数：

```
SELECT SF_SYNC_INI (1);
```

15）SF_DSC_SYNC_INI

定义:

INT

SF_DSC_SYNC_INI(

level int

)

功能说明:

用于普通节点从控制节点同步 DM.INI 参数。仅 DMDSC 集群节点生效。函数直连 DMDSC 普通节点执行。只对当前普通节点有效。不同的普通节点需要分别单独执行。在控制节点执行会报错。

仅同步 INI 参数的内存值及 DM.INI 文件值。对于某个会话上的 INI 参数修改，不进行同步。

同步的两个节点须同为 V8.1.3.31 之前的版本，或同为大于等于 V8.1.3.31 的版本。若在小于 V8.1.3.31 和大于等于 V8.1.3.31 的两个版本间执行，不会真正同步，而是直接返回成功。

参数说明:

level：同步级别。取值 0 和 1。0：表示同步所属环境下必须同步的部分参数。1：表示同步所有可以同步的参数。必须同步的部分参数和所有可以同步的参数与 SF_SYNC_INI()中的 DMDSC 参数一样，具体的参数请参考 SF_SYNC_INI()。

返回值:

0：执行成功。

<0：执行失败。

100：对方 MAL 消息版本过低或 INI 版本一致无需同步。

举例说明:

在普通节点上执行，同步控制节点上所有可以同步的 INI 参数:

```
SELECT SF_DSC_SYNC_INI (1);
```

16) SP_SET_SQLLOG_INI

定义:

SP_SET_SQLLOG_INI(

raft_id int,

sqllog_config varchar(24575)

)

或

SP_SET_SQLLOG_INI(

sqllog_config varchar(24575)

)

功能说明:

修改 SQLLOG.INI 文件的内容。

参数说明:

raft_id: 要修改的 SQLLOG.INI 文件所在的服务器站点号。无此参数表示修改本地的 SQLLOG.INI 文件。

sqllog_config: 要修改的内容，模式名用'[]'标出，配置项之间用';'分隔；若模式名已存在，则修改其配置，对于未设置的配置项，继承原值；对于不存在的模式名，新增模式。当不指定模式名时，认为是全局配置区，全局配置区的模式内配置项只有写在配置串开头时才认为其合法。

返回值:

无。

举例说明:

新增模式 SLOG_CONFIG1 和 SLOG_CONFIG2:

```
SP_SET_SQLLOG_INI('[SLOG_CONFIG1]FILE_PATH=../log;PART_STOR=0;[SLOG_CONFIG2]SWIT
CH_MODE=2;SWITCH_LIMIT=128');
```

17) SP_DELETE_SQLLOG_INI_MODE

定义:

```
SP_DELETE_SQLLOG_INI_MODE(
raft_id int,
mode_name        varchar(128)
)
或
SP_DELETE_SQLLOG_INI_MODE(
mode_name        varchar(128)
)
```

功能说明:

删除 SQLLOG.INI 文件中的模式。

参数说明:

raft_id: 要修改的 SQLLOG.INI 文件所在的服务器站点号。无此参数表示修改本地的 SQLLOG.INI 文件。

mode_name: 要删除的模式名。

返回值:

无。

举例说明:

删除模式名为 SLOG_ALL 的 SQL 日志模式:

```
SP_DELETE_SQLLOG_INI_MODE('SLOG_ALL');
```

18) SP_SET_PARAM_IN_SESSION

定义:

```
SP_SET_PARAM_IN_SESSION (
SESS_IDBIGINT,
SESS_SEQ    INT,
PARANAME   VARCHAR(256),
VALUE          BIGINT
)
```

功能说明:

用于设置指定会话的会话级 INI 参数的值。调用此系统过程需要管理员权限。

参数说明:

SESS_ID: 会话 ID。

SESS_SEQ：会话序列号。

PARANAME：会话级 INI 参数的参数名。

VALUE：要设置的新值。

返回值：

无

举例说明：

查看当前已存在的会话 ID 和序列号等信息：

```
SELECT SESS_ID, SESS_SEQ FROM V$SESSIONS;

行号          SESS_ID                  SESS_SEQ

---------- -------------------- -----------

1             12429400                 6

2             13518936                 7
```

设置会话 id 为 13518936 的会话级 INI 参数 JOIN_HASH_SIZE 的值为 2000：

```
SP_SET_PARAM_IN_SESSION (13518936, 7, 'JOIN_HASH_SIZE', 2000);
```

19）SP_SET_INI_PARA_VALUE*

定义：

```
SP_SET_INI_PARA_VALUE (
scope                int,
ini_param_name    varchar(256) ,
value                varchar(32767)
)
```

功能说明：

设置DM.INI文件中的整型，浮点型，字符串型参数值。DSC环境下，除了特殊参数以外，

其他参数值会在OK节点上同步。关于特殊参数的介绍请参考函数SP_SET_PARA_VALUE的功能说明。

参数说明:

scope：取值为 0、1、2。0 表示修改内存中的动态配置参数值；1 表示 DM.INI 文件和内存参数都修改，不需要重启服务器；2 表示只修改 DM.INI 文件，服务器重启后生效。

ini_param_name：DM.INI 文件中的参数名。

value：设置的字符串的值。

返回值:

无

举例说明:

将 DM.INI 文件中 HFS_CACHE_SIZE 参数值设置为 320：

```
SP_SET_INI_PARA_VALUE (1,'HFS_CACHE_SIZE',320);
```

将DM.INI文件中SEL_RATE_EQU参数值设置为0.3：

```
SP_SET_INI_PARA_VALUE(1, 'SEL_RATE_EQU', 0.3);
```

将 DM.INI 文件中 COMMIT_WRITE 参数值设置为 WAIT：

```
SP_SET_INI_PARA_VALUE(1,'COMMIT_WRITE','WAIT');
```

2. 系统信息管理

1) SP_SET_SESSION_READONLY

定义:

SP_SET_SESSION_READONLY (

readonly int

)

功能说明:

设置当前会话的只读模式。当 INI 参数 USER_READ_ONLY_MODE 的取值为 1（禁止只读用户设置会话为非只读）时，只读用户不能通过 SP_SET_SESSION_READONLY(0)命令取消会话的只读限制。

参数说明:

取值 1 或 0。1 表示对数据库只读；0 表示对数据库为读写。

返回值:

无

举例说明:

设置当前会话为只读模式:

```
SP_SET_SESSION_READONLY (1);
```

2） SP_CLOSE_SESSION

定义:

SP_CLOSE_SESSION (

session_id bigint

)

或

SP_CLOSE_SESSION (

ep_seqno int,

session_id bigint

)

或

SP_CLOSE_SESSION (

session_id bigint,

force_commit boolean

)

功能说明:

停止一个活动的会话，会话 ID 可通过 V$SESSIONS 或 GV$SESSIONS 查询，DM 系统创建的内部 SESSION（PORT_TYPE = 12，但 CONNECTED = 1 的 SESSION）也可通过 V$SESSIONS 或 GV$SESSIONS 查询到。

SP_CLOSE_SESSION() 会回滚会话上的当前操作并释放会话，但 SP_CLOSE_SESSION()并不会等待会话真正释放后才返回。

当设置 force_commit 为 TRUE 时，会进行强制会话提交。仅单机和 DSC 环境支持指定 force_commit=true，DPC/MPP 环境执行会报错。如果会话在执行 DDL 类型语句、快速装载或者 XA 分布式事务，指定 force_commit=true 不会强制事务提交，依然进行回滚事务。

中断会话，强制事务提交可能导致事务不一致，用户要能接受可能的不一致，或者说有其他手段恢复事务不一致。

需要注意的是，对于任意会话 sess，假设其会话 ID 为 sess_id，如果先执行 SP_CLOSE_SESSION (sess_id)或者 SP_CLOSE_SESSION (sess_id, false)，或者 sess 客户端主动断开，sess 事务已经开始回滚，再执行 SP_CLOSE_SESSION (sess1_id, true)是不能强制事务提交的。

参数说明:

session_id：指定会话 ID。

ep_seqno：指定会话所在的 DMDSC 集群节点的节点号，或者 DMDPC 集群的 RAFT_ID。

force_commit：指定是否强制提交事务，true 为强制提交，false 为回滚。

举例说明:

例 1 停止 ID 为 510180488 的会话:

```
SP_CLOSE_SESSION (510180488);
```

例 2 强制提交事务

```
//假设会话 SESS1 的 ID 为 510180489，且其中对表 test1，test2 有如下操作
update test1 set count=count+100 where id=100;
update test2 set count=count-100 where id=100;
//对 SESS1 执行强制事务提交
SP_CLOSE_SESSION (510180489, true);
//查看两表数据，则有可能发现 test1 和 test2 的数据无法满足事务一致性，如 test1 的数据部分更新了，
但是 test2 的数据都还没有更新。
select * from test1;
select * from test2;
```

3) SF_GET_CASE_SENSITIVE_FLAG/ CASE_SENSITIVE

定义:

INT

SF_GET_CASE_SENSITIVE_FLAG()

或者

INT

CASE_SENSITIVE ()

功能说明:

返回大小写敏感信息。

参数说明:

无

返回值:

1：敏感。

0：不敏感。

举例说明:

获得大小写敏感信息:

```
SELECT SF_GET_CASE_SENSITIVE_FLAG();
```

4） SESS_CASE_SENSITIVE

定义:

INT

SESS_CASE_SENSITIVE ()

功能说明:

返回会话级大小写敏感信息。

参数说明:

无

返回值:

1：敏感。

0：不敏感。

举例说明:

获得会话级大小写敏感信息:

```
SELECT   SESS_CASE_SENSITIVE();
```

5） SF_GET_EXTENT_SIZE

定义:

INT

SF_GET_EXTENT_SIZE()

功能说明:

返回簇大小。

参数说明:

无

返回值:

系统建库时指定的簇大小。

举例说明:

获得系统建库时指定的簇大小:

```
SELECT SF_GET_EXTENT_SIZE ();
```

6) SF_GET_PAGE_SIZE/PAGE

定义:

INT

SF_GET_PAGE_SIZE()

或者

INT

PAGE()

功能说明:

返回页大小。

参数说明:

无

返回值:

系统建库时指定的页大小。

举例说明:

获得系统建库时指定的页大小:

```
SELECT SF_GET_PAGE_SIZE ();
```

补充说明:

获得系统建库时指定的页大小也可使用:

```
SELECT PAGE();
```

7) SF_GET_FILE_BYTES_SIZE

定义:

BIGINT

SF_GET_FILE_BYTES_SIZE (

groupid int,

fileid int

)

功能说明:

获取文件字节长度。

参数说明:

groupid: 所属的表空间 ID。

fileid: 数据库文件 ID。

返回值:

文件字节长度。

举例说明:

获取 0 号文件组中 0 号文件的字节长度：

```
SELECT SF_GET_FILE_BYTES_SIZE (0,0);
```

8） SF_GET_UNICODE_FLAG/UNICODE

定义：

INT

SF_GET_UNICODE_FLAG()

或者

INT

UNICODE()

功能说明：

返回建库时指定的字符集。

参数说明：

无

返回值：

0 表示 GB18030，1 表示 UTF-8，2 表示 EUC-KR。

举例说明：

获得系统建库时指定字符集：

```
SELECT SF_GET_UNICODE_FLAG ();
```

9） SF_GET_SGUID

定义：

INT

SF_GET_SGUID()

功能说明:

返回数据库唯一标志 sguid。

参数说明:

无

返回值:

返回数据库唯一标志 sguid。

举例说明:

获取数据库唯一标志 sguid:

```
SELECT SF_GET_SGUID();
```

10）GUID()

定义:

VARCHAR

GUID()

功能说明:

生成一个唯一编码串（32 个字符）。

返回值:

返回一个唯一编码串。

举例说明:

获取一个唯一编码串:

```
SELECT GUID();
```

11） NEWID()

定义:

VARCHAR

NEWID()

功能说明:

生成一个 SQLSERVER 格式的 guid 字符串 SQLSERVER 的 guid 格式 xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx。

返回值:

返回一个 SQLSERVER 格式的唯一编码串。

举例说明:

获取一个唯一编码串:

```
SELECT NEWID();
```

12) UUID()

定义:

VARCHAR

UUID()

功能说明:

生成一个 MySQL 格式的 guid 字符串,格式为 xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx,字符串中字母为小写。

返回值:

返回一个 aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee 格式的唯一编码串。

举例说明:

获取一个唯一编码串:

```
SELECT UUID();
```

13）SESSID()

定义:

BIGINT

SESSID()

功能说明:

获取当前连接的 id。

返回值:

返回当前连接 id。

举例说明:

返回当前连接 id:

```
SELECT  SESSID ();
```

14）CHECK_INDEX

定义:

INT

CHECK_INDEX (

schname varchar,

indexid int

)

功能说明:

检查一个索引的合法性（正确性和有效性）。检查过程中，会使用 S+IX 锁来封锁索引对应的表对象，如果封锁失败，会忽略索引检查，并记录相关日志到 dmserver 的 log 文件中。

参数说明:

schname：模式名。

indexid：索引 id。

返回值：

0：表示不合法；1：表示合法；2：表示存在未校验的索引。

举例说明：

```
CREATE INDEX PRODUCT_IND ON PRODUCTION.PRODUCT(PRODUCTID);
//查询系统表得到索引 ID
select name, id from sysobjects where name='PRODUCT_IND' and subtype$='INDEX';
select CHECK_INDEX ('PRODUCTION',33555531);
```

15）CHECK_DB_INDEX

定义：

INT

CHECK_DB_INDEX ()

功能说明：

检查数据库中所有索引的合法性（正确性和有效性）。检查过程中，会使用 S+IX 锁来封锁索引对应的表对象，如果封锁失败，会忽略索引检查，并记录相关日志到 dmserver 的 log 文件中。

返回值：

0：表示不合法；1：表示合法；2：表示除未能检查的索引外，其它索引都合法。

举例说明：

```
SELECT CHECK_DB_INDEX ();
```

16）UID

定义:

INT

UID ()

功能说明:

返回当前用户 ID。

参数说明:

无

返回值:

返回当前用户 ID。

举例说明:

返回当前用户 ID:

```
SELECT UID();
```

17) USER

定义:

VARCHAR

USER ()

功能说明:

返回当前用户名。

参数说明:

无

返回值:

返回当前用户名。

举例说明:

返回当前用户名：

```
SELECT USER();
```

18）CUR_DATABASE

定义：

VARCHAR

CUR_DATABASE ()

功能说明：

返回数据库名。

参数说明：

无

返回值：

返回数据库名。

举例说明：

获取数据库名：

```
SELECT CUR_DATABASE();
```

19）VSIZE

定义：

INT

VSIZE(n)

功能说明：

返回 n 的核心内部表示的字节数。如果 n 为 NULL，则返回 NULL。

参数说明：

n：待求字节数的参数，可以为任意数据类型。

返回值：

n 占用的字节数。

举例说明：

```
SELECT VSIZE(256); //整数类型
```

查询结果为：4

```
SELECT VSIZE('数据库'); //中文字符
```

查询结果为：6

20) SP_RECLAIM_TS_FREE_EXTENTS

定义：

SP_RECLAIM_TS_FREE_EXTENTS (

tsname varchar(128)

)

功能说明：

重组表空间空闲簇。

参数说明：

tsname：表空间名。

返回值：

无

举例说明：

重组表空间空闲簇：

```
SP_RECLAIM_TS_FREE_EXTENTS('SYSTEM');
```

21) SP_CLEAR_PLAN_CACHE

定义:

SP_CLEAR_PLAN_CACHE()

功能说明:

清空当前站点的执行缓存信息。

参数说明:

无

返回值:

无

举例说明:

清空当前站点的执行缓存信息:

```
SP_CLEAR_PLAN_CACHE();
```

22) SP_CLEAR_PLAN_CACHE

定义:

SP_CLEAR_PLAN_CACHE(

plan_id bigint

)

功能说明:

清空当前站点指定的执行缓存信息。

参数说明:

plan_id: 指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

返回值:

无

举例说明:

清空当前站点 ID 为 139688237135984 的执行缓存信息:

```
SP_CLEAR_PLAN_CACHE(139688237135984);
```

23) SP_CLEAR_PLAN_CACHE

定义:

SP_CLEAR_PLAN_CACHE(

plan_id bigint,

hash_value int

)

功能说明:

清空当前站点指定的执行缓存信息。

参数说明:

plan_id: 指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

hash_value: 指定缓存项的 HASH 值，其值可以从动态视图 V$CACHEPLN 中的 HASH_VALUE 列获得。

返回值:

无

举例说明:

清空当前站点 ID 为 139688237144176，HASH 值为 1719382609 的执行缓存信息:

```
SP_CLEAR_PLAN_CACHE(139688237144176,1719382609);
```

24) SP_SET_PLN_RS_CACHE

定义:

SP_SET_PLN_RS_CACHE(

plan_id bigint,

to_cache int

)

功能说明:

强制设置指定计划结果集缓存的生效及失效。

参数说明:

plan_id：指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

to_cache：指定缓存与否，0：不缓存；1：缓存。

返回值:

无

举例说明:

设置计划 ID 为 139688237234288 的计划结果集缓存生效:

```
SP_SET_PLN_RS_CACHE(139688237234288,1);
```

25) SP_SET_PLN_RS_CACHE

定义:

SP_SET_PLN_RS_CACHE(

plan_id bigint,

hash_value int,

to_cache int

)

功能说明:

强制设置指定计划结果集缓存的生效及失效。

参数说明:

plan_id：指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

hash_value：指定缓存项的 HASH 值，其值可以从动态视图 V$CACHEPLN 中的 HASH_VALUE 列获得。

to_cache：指定缓存与否，0：不缓存；1：缓存。

返回值:

无

举例说明:

设置计划 ID 为 139688237234288，HASH 值为 1140234264 的计划结果集缓存失效:

```
SP_SET_PLN_RS_CACHE(139688237234288, 1140234264,0);
```

26) CUR_TICK_TIME

定义:

VARCHAR

CUR_TICK_TIME ()

功能说明:

获取系统当前时钟记数。

参数说明:

无

返回值:

时钟记数的字符串。

举例说明:

获取系统当前时钟记数:

```
SELECT CUR_TICK_TIME();
```

27) SP_SET_LONG_TIME

定义:

SP_SET_LONG_TIME (

long_exec_time int

)

功能说明:

设置 V$LONG_EXEC_SQLS 和 V$SYSTEM_LONG_EXEC_SQLS 动态视图中监控 SQL 语句的最短执行时间预定值，以毫秒为单位，取值范围 50~3600000。仅 INI 参数 ENABLE_MONITOR=1 时设置有效。仅 DBA 有权限操作该过程。

参数说明:

无

返回值:

无

举例说明:

监控执行时间超过 5 秒的 SQL 语句:

```
SP_SET_LONG_TIME(5000);
```

28) SF_GET_LONG_TIME

定义:

INT

SF_GET_LONG_TIME ()

功能说明:

返回 V$LONG_EXEC_SQLS 和 V$SYSTEM_LONG_EXEC_SQLS 动态视图中监控的最短执行时间预定值，以毫秒为单位。仅 DBA 有权限操作该过程。

参数说明:

无

返回值:

V$LONG_EXEC_SQLS和V$SYSTEM_LONG_EXEC_SQLS所监控的最短执行时间预定值。

举例说明:

查看 V$LONG_EXEC_SQLS 监控的最短执行时间预定值:

```
SELECT SF_GET_LONG_TIME();
```

29) PERMANENT_MAGIC

定义:

INT

PERMANENT_MAGIC ()

功能说明:

返回数据库永久魔数。

参数说明:

无

返回值:

返回整型值：永久魔数。

举例说明:

获取数据库永久魔数:

```
SELECT PERMANENT_MAGIC();
```

30) SP_CANCEL_SESSION_OPERATION

定义:

SP_CANCEL_SESSION_OPERATION (

session_id bigint

)

或

SP_CANCEL_SESSION_OPERATION (

ep_seqno int,

session_id bigint

)

功能说明:

终止指定会话的操作。

参数说明:

session_id: 指定会话 ID，可通过 V$SESSIONS 或 GV$SESSIONS 查询。

ep_seqno: 指定会话所在的 DMDSC 集群节点的节点号，或者 DMDPC 集群的 RAFT_ID。

返回值:

无

举例说明:

终止 ID 为 310509680 的会话的操作:

```
SP_CANCEL_SESSION_OPERATION (310509680);
```

31) SP_PURGE_TS

定义:

SP_PURGE_TS(

timeout int

)

功能说明:

清理已提交事务回滚记录。

参数说明:

timeout: 超时等待时间，单位秒，指的是当存在其他 purge 清理操作并发时，本操作需要等待其他 purge 操作完成的超时等待时间。当等待超过 timeout 设定时间，其他 purge 操作还未完成时，报错处理。

返回值:

无

举例说明:

清理已提交事务回滚记录。如果等待并发 purge 操作超过 5 秒还未完成时，报错。

```
SP_PURGE_TS(5);
```

32) SF_GET_SESSION_SQL

定义:

CLOB

SF_GET_SESSION_SQL (

sess_id bigint

)

或

CLOB

SF_GET_SESSION_SQL (

ep_seqno int,

sess_id bigint

)

功能说明:

返回指定会话上最近处理的完整的 SQL 语句。

参数说明:

sess_id：指定会话 ID，可通过 V$SESSIONS 或 GV$SESSIONS 查询。

ep_seqno：指定会话所在的 DMDSC 集群节点的节点号。

返回值:

指定会话上最近处理的完整的 SQL 语句。

举例说明:

在 ID 为 96710784 的会话上执行如下语句:

```
CREATE OR REPLACE PROCEDURE xx  AS
BEGIN
SELECT SF_GET_SESSION_SQL(96710784);
EXCEPTION
  WHEN OTHERS THEN NULL;
END;
/
```

重新打开一个会话，调用 xx 过程。可以查看到 ID 为 96710784 的会话上的最后一次执

行的 SQL 语句。

```
SQL> call xx;
```

执行结果如下:

```
行号          SF_GET_SESSION_SQL(96710784)

---------- ----------------------------------------

1             CREATE OR REPLACE PROCEDURE xx    AS

BEGIN

SELECT SF_GET_SESSION_SQL(96710784);

EXCEPTION

  WHEN OTHERS THEN NULL;

END;
```

33) SF_CLOB_LEN_IS_VALID

定义:

INT

SF_CLOB_LEN_IS_VALID (

clob

)

功能说明:

检查系统存储的 clob 字符长度是否正常。

参数说明:

clob: clob 对象。

返回值:

0: 不正常，1: 正常。

举例说明:

```
select SF_CLOB_LEN_IS_VALID ('PRODUCTION.PRODUCT.DESCRIPTION');
```

34) SP_VALIDATE_CLOB_LEN

定义:

SP_VALIDATE_CLOB_LEN(

clob

)

功能说明:

修复系统存储的 clob 字符长度。

参数说明:

clob：clob 对象。

举例说明:

```
SP_VALIDATE_CLOB_LEN ('PRODUCTION.PRODUCT.DESCRIPTION');
```

35) CHECK_INDEX_PAGE_USED

定义:

INT

CHECK_INDEX_PAGE_USED (

indexid int

)

功能说明:

检查 ID 为 indexid 的索引数据页（包含 BLOB 字段）分配是否与对应的簇分配情况一致。

参数说明:

indexid：索引 ID，如果不是数据库中的索引 ID 或者为空，则报错。

返回值:

1：一致。

0：不一致。

举例说明:

```
DROP TABLE T1_CHECK;

CREATE TABLE T1_CHECK(c1 INT);



SELECT CHECK_INDEX_PAGE_USED(a.id) FROM sysobjects a WHERE a.subtype$='INDEX' AND

a.pid IN(SELECT id FROM sysobjects WHERE name = 'T1_CHECK');
```

36) SF_FILE_SYS_CHECK_REPORT

定义:

INT

SF_FILE_SYS_CHECK_REPORT(

ts_id int

)

功能说明:

校验检查指定表空间的簇是否正常。

参数说明:

ts_id：指定检测的表空间，如果表空间不存在则返回 0。

返回值:

1：表示表空间的簇都是正常的。

0：表空间中存在检验不通过的簇，问题的详细描述输出到服务器的运行日志中。

举例说明:

```
SELECT SF_FILE_SYS_CHECK_REPORT(4);
```

37) SP_LOAD_LIC_INFO()

定义:

SP_LOAD_LIC_INFO()

功能说明:

进行 DM 服务器的 LICENSE 校验，检查 LICENSE 与当前 DM 版本及系统运行环境是否一致。一致则执行成功；不一致则报错。

参数说明:

无

返回值:

无

举例说明:

```
SP_LOAD_LIC_INFO();
```

38) SF_PROXY_USER

定义:

VARCHAR

SF_PROXY_USER()

功能说明:

返回当前代理用户名。

参数说明:

无

返回值:

返回当前代理用户名。

举例说明:

返回当前代理用户名:

```
SELECT SF_PROXY_USER();
```

39) SP_CLEAR_PLAN_CACHE_BY_DICT

定义:

SP_CLEAR_PLAN_CACHE_BY_DICT(

dict_id int

)

功能说明:

清除涉及指定字典对象的缓存计划。

参数说明:

dict_id: 指定字典对象的 ID 值。

返回值:

无

举例说明:

清除涉及 ID 值为 1250 的字典对象的缓存计划:

```
SP_CLEAR_PLAN_CACHE_BY_DICT(1250);
```

40) SF_GET_LOGIN_ID

定义:

INT

SF_GET_LOGIN_ID()

功能说明:

获取当前登录用户 ID，功能同系统函数 UID()。

参数说明:

无

返回值:

当前登录用户 ID。

举例说明:

获取当前登录用户 ID:

```
SELECT SF_GET_LOGIN_ID();
```

41) SF_GET_LOGIN_IP

定义:

VARCHAR

SF_GET_LOGIN_IP()

功能说明:

获取当前执行登录操作的 IP 地址。

参数说明:

无

返回值:

当前执行登录操作的 IP 地址。

举例说明:

获取当前执行登录操作的 IP 地址:

```
SELECT SF_GET_LOGIN_IP();
```

42) SF_GET_LOGIN_APP

定义:

VARCHAR

SF_GET_LOGIN_APP()

功能说明:

获取当前执行登录操作的应用名。

参数说明:

无

返回值:

当前执行登录操作的应用名。

举例说明:

获取当前执行登录操作的应用名:

```
SELECT SF_GET_LOGIN_APP();
```

43) SF_LOGIN_SUCCESS

定义:

DMBOOL

SF_LOGIN_SUCCESS()

功能说明:

判断当前会话是否登录成功，一般在 LOGIN/LOGON 触发器中使用。

参数说明:

无

返回值:

0：登录失败;

1：登录成功。

举例说明:

判断当前会话是否登录成功:

```
SELECT SF_LOGIN_SUCCESS();
```

44) SF_GET_TABLE_GROUP_INFO

定义:

CLOB

SF_GET_TABLE_GROUP_INFO(

schname varchar(128)

tabname varchar(128)

group_count int

)

功能说明:

按照数据页分布对表数据进行分组。不支持堆表、HUGE 表和分区主表。

参数说明:

schname：模式名。

tabname：表名。

group_count：指定分组数，取值范围 1~1000。实际分组时由于数据页分布不同，返回结果分组数可能小于指定分组数。

返回值:

返回表数据分组信息，各字段含义如下:

COLUMN：分组列。如果表中存在 CLUSTER KEY，则根据 CLUSTER KEY 分组；否则根据 ROWID 分组。

GROUP_NO：组别编号，从 1 开始依次递增。

START：当前组的起始分组列值，“-”表示当前组无起始值。

END：当前组的终止分组列值，“-”表示当前组无终止值。

COUNT：当前组的表记录行数。

举例说明：

准备数据：

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT CLUSTER KEY, C2 INT);
INSERT INTO TEST SELECT LEVEL,LEVEL FROM DUAL CONNECT BY LEVEL <=20000;
COMMIT;
```

将表 TEST 数据按照数据页分布分组，指定分组数为 4：

```
SELECT SF_GET_TABLE_GROUP_INFO('SYSDBA','TEST',4);
```

执行结果如下：

```
COLUMN: C1
GROUP_NO    START    END      COUNT
1           1        5040     5040
2           5041     10080    5040
3           10081    15120    5040
4           15121    20000    4880
```

45) SF_GET_TABLE_GROUP_INFO_BY_ROWS/

SP_GET_TABLE_GROUP_INFO_BY_ROWS

定义:

CLOB

SF_GET_TABLE_GROUP_INFO_BY_ROWS(

schname varchar(128)

tabname varchar(128)

row_count int

)

或

SP_GET_TABLE_GROUP_INFO_BY_ROWS(

schname varchar(128)

tabname varchar(128)

row_count int

)

功能说明:

按照数据页分布对表数据进行分组。不支持堆表、HUGE 表和分区主表。

参数说明:

schname：模式名。

tabname：表名。

row_count：指定行数，除最后一组外每组需要达到的最小行数。

返回值:

返回表数据分组信息，各字段含义如下:

COLUMN：分组列。如果表中存在 CLUSTER KEY，则根据 CLUSTER KEY 分组；否则根据 ROWID 分组。

GROUP_NO：组别编号，从 1 开始依次递增。

START：当前组的起始分组列值，“-”表示当前组无起始值。

END：当前组的终止分组列值，“-”表示当前组无终止值。

COUNT：当前组的表记录行数。

举例说明：

准备数据：

```
DROP TABLE TEST;
CREATE TABLE TEST(C1 INT CLUSTER KEY, C2 INT);
INSERT INTO TEST SELECT LEVEL,LEVEL FROM DUAL CONNECT BY LEVEL <=20000;
COMMIT;
```

使用系统函数 SF_GET_TABLE_GROUP_INFO_BY_ROWS 将表 TEST 数据按照数据页分布分组，指定除最后一组外，其余每组需要至少包含 4000 行记录：

```
SELECT SF_GET_TABLE_GROUP_INFO_BY_ROWS('SYSDBA','TEST',4000);
```

执行结果如下：

```
COLUMN: C1
GROUP_NO    START    END      COUNT
1           1        4032     4032
2           4033     8064     4032
3           8065     12096    4032
4           12097    16128    4032
5           16129    20000    3872
```

使用系统过程 SP_GET_TABLE_GROUP_INFO_BY_ROWS 将表 TEST 数据按照数据页分布分组，指定除最后一组外，其余每组需要至少包含 4000 行记录：

```
SP_GET_TABLE_GROUP_INFO_BY_ROWS('SYSDBA','TEST',4000);
```

执行结果如下：

```
GROUP_NO    COLUMN  START     END       COUNT
----------- -------- ------- -------- -----
1           C1      1         4032      4032
2           C1      4033      8064      4032
3           C1      8065      12096     4032
4           C1      12097     16128     4032
5           C1      16129     20000     3872
```

46) SP_CLOSE_LSNR

定义:

SP_CLOSE_LSNR()

功能说明:

关闭 dmserver 监听端口。

参数说明:

无。

返回值:

无。

举例说明:

关闭当前连接 dmserver 的监听端口:

```
SP_CLOSE_LSNR();
```

执行结果如下:

```
DMSQL 过程已成功完成
```

执行成功后，客户端再次连接服务器:

```
disql V8
```

```
用户名:

密码:

[-70028]:创建 SOCKET 连接失败.
```

47) SP_OPEN_LSNR

定义:

SP_OPEN_LSNR()

功能说明:

打开 dmserver 监听端口。

参数说明:

无。

返回值:

无。

举例说明:

打开当前连接 dmserver 的监听端口:

```
SP_OPEN_LSNR();
```

执行结果如下:

```
DMSQL 过程已成功完成
```

执行成功后，客户端再次连接服务器:

```
服务器[LOCALHOST:5236]:处于普通打开状态
```

Linux 系统可以通过给 dmserver 进程发送 SIGUSR2 信号来打开监听端口。当用户关闭监听端口并退出客户端，无法重新连接时，可以通过该方式打开监听端口并连接服务器。操作如下:

```
kill -SIGUSR2 [dmserver 进程号]
```

48) SP_CLEAR_RAFT_PLAN_CACHE

定义:

SP_CLEAR_RAFT_PLAN_CACHE(

raft_id INT

)

功能说明:

清空指定站点的执行缓存信息。

参数说明:

raft_id: 站点号。如果站点号为 NULL，就清除所有站点的执行缓存信息；否则清理指定站点的执行缓存信息。非 DPC 环境忽略站点号。

返回值:

无。

举例说明:

清空所有站点的执行缓存信息:

```
SP_CLEAR_RAFT_PLAN_CACHE(NULL);
```

49) SP_CLEAR_RAFT_PLAN_CACHE

定义:

SP_CLEAR_RAFT_PLAN_CACHE(

plan_id BIGINT,

raft_id INT

)

功能说明:

清空指定站点的指定执行缓存信息。

参数说明:

plan_id：指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

raft_id：站点号。如果站点号为 NULL，就清除所有站点指定 ID 的执行缓存信息；否则清理指定站点指定 ID 的执行缓存信息。非 DPC 环境忽略站点号。

返回值:

无。

举例说明:

在所有站点中查找并清空 ID 为 139688237135984 的执行缓存信息:

```
SP_CLEAR_RAFT_PLAN_CACHE(139688237135984，NULL);
```

在站点 3 中清空 ID 为 139688237135984 的执行缓存信息:

```
SP_CLEAR_RAFT_PLAN_CACHE(139688237135984，3);
```

50) SP_CLEAR_RAFT_PLAN_CACHE

定义:

```
SP_CLEAR_RAFT_PLAN_CACHE(
plan_id       BIGINT,
hash_value       INT,
raft_id       INT
)
```

功能说明:

清空指定站点的指定执行缓存信息。

参数说明:

plan_id：指定计划 ID，其值可以从动态视图 V$CACHEPLN 中的 CACHE_ITEM 列获得。

hash_value：指定缓存项的 HASH 值，其值可以从动态视图 V$CACHEPLN 中的 HASH_VALUE 列获得。

raft_id：站点号。如果站点号为 NULL，就在所有站点上查找并清除指定的执行缓存信息；否则清理指定站点的指定执行缓存信息。非 DPC 环境忽略站点号。

返回值：

无。

举例说明：

清空 ID 为 139688237144176，HASH 值为 1719382609 的执行缓存信息：

```
SP_CLEAR_RAFT_PLAN_CACHE(139688237144176,1719382609, NULL);
```

DPC 环境下某 SP 站点号为 3，清空此站点 ID 为 139688237135984，HASH 值为 1719382609 的执行缓存信息：

```
SP_CLEAR_RAFT_PLAN_CACHE(139688237135984，1719382609, 3);
```

51）SP_CLEAR_OS_MEM_INFO

定义：

```
SP_CLEAR_OS_MEM_INFO (
raft_id   int
)
```

功能说明：

清除所有或指定节点上的操作系统内存分配释放信息。

参数说明：

raft_id：站点号。如果站点号为 NULL，就清除所有站点上的内存信息；否则清理指定

站点的内存信息。

返回值:

无

举例说明:

清理所有站点上的内存信息。

```
SP_CLEAR_OS_MEM_INFO(NULL);
```

52) SF_CLEAR_DICT_CACHE

定义:

INT

SF_CLEAR_DICT_CACHE()

或

INT

SF_CLEAR_DICT_CACHE(

OBJID INT,

CASCADE INT,

TYPE VARCHAR(128),

SUBTYPE VARCHAR(128)

)

功能说明:

不指定任何参数时，清理所有字典缓存，指定对象时，清理指定对象 ID 的字典缓存。

参数说明:

OBJID：对象 ID。

CASCADE：是否级联删除。

TYPE：类型，对应 SYSOBJECTS 的 TYPE$。

SUBTYPE：子类型，对应 SYSOBJECTS 的 SUBTYPE$。

返回值:

清理的字典个数。

举例说明:

例 1 清理所有字典信息。

```
SELECT SF_CLEAR_DICT_CACHE();
```

例 2 清理对象 ID 为 1024 的字典缓存

```
SELECT SF_CLEAR_DICT_CACHE(1024,1,'SCHOBJ','UTAB');
```

定义 2:

INT

SF_CLEAR_DICT_CACHE(

RAFT_ID INT

)

或

INT

SF_CLEAR_DICT_CACHE(

RAFT_ID INT,

OBJID INT,

CASCADE INT,

TYPE VARCHAR(128),

SUBTYPE VARCHAR(128)

)

功能说明:

DPC 下使用，其他环境清理当前节点。广播到指定节点，如果不指定对象 ID，清理所有字典缓存，否则清理指定 ID 的字典缓存。

参数说明:

RAFT_ID：广播的目标节点号。

OBJID：对象 ID。

CASCADE：是否级联删除。

TYPE：类型，对应 SYSOBJECTS 的 TYPE$。

SUBTYPE：子类型，对应 SYSOBJECTS 的 SUBTYPE$。

返回值:

清理的字典个数。

举例说明:

例 1 清理目标节点号为 1 的所有字典信息。

```
SELECT SF_CLEAR_DICT_CACHE(1);
```

例 2 清理目标节点号为 1，对象 ID 为 1024 的字典缓存

```
SELECT SF_CLEAR_DICT_CACHE(1,1024,1,'SCHOBJ','UTAB');
```

53) SP_CREATE_SYSTEM_VIEWS

定义:

VOID

SP_CREATE_SYSTEM_VIEWS(

CREATE_FLAG　INT

)

或

VOID

SP_CREATE_SYSTEM_VIEWS(

CREATE_FLAG INT,

VIEWNAME VARCHAR(128)

)

功能说明:

用于删除或创建所有系统视图，也支持删除指定的系统视图。

参数说明:

CREATE_FLAG：为 1 时表示创建所有的系统视图，为 0 时表示删除所有的系统视图。

VIEWNAME：系统视图名称。当 CREATE_FLAG 为 0 时，可以对该参数赋值，表示删除指定的系统视图。

返回值:

无。

举例说明:

```
SP_CREATE_SYSTEM_VIEWS(1);
```

54) SP_DROP_OLD_DYNAMIC_TAB

定义:

SP_DROP_OLD_DYNAMIC_TAB();

功能说明:

用于升级后删除老版本库中系统表残留的动态视图定义。DM8 的 8.1.1.189 版本之前的库中，动态性能视图相关定义保存在系统表中，升级后残留的定义会在部分特殊场景导致崩溃，故提供此系统过程。

该系统过程需要 DBA 权限执行，执行后影响执行码降级。

参数说明:

无。

返回值:

无。

举例说明:

清理老版本中动态性能视图在系统表中的字典信息。

```
SP_DROP_OLD_DYNAMIC_TAB();
```

55) SP_CLEAR_INVALID_QUOTA

定义:

VOID

SP_CLEAR_INVALID_QUOTA(

TS_ID SMALLINT

);

功能说明:

用于删除用户后清理用户在表空间设置的配额信息。部分场景下删除用户时，用户在表空间设置的配额信息不会被及时清理（如表空间 OFFLINE 或是 DPC 环境中 BP 节点下线），通常情况下服务器会在后续运行中自动清理此类已经无效的表空间配额信息，用户不必为此费心，但也允许用户主动调用此系统过程清理。

参数说明:

TS_ID：表空间 ID，smallint 类型。

返回值:

无。

举例说明:

清理 MAIN 表空间无效的配额信息。

```
--通过 V$TABLESPACE 查询 MAIN 表空间对应 ID

SELECT ID FROM V$TABLESPACE WHERE NAME ='MAIN';

行号        ID

---------- -----------

1               4

--使用 SP_CLEAR_INVALID_QUOTA 清理 MAIN 表空间

SP_CLEAR_INVALID_QUOTA(4);
```

56) SP_FIX_SOI_PRIVS

定义:

VOID SP_FIX_SOI_PRIVS();

功能说明:

DM8 的 8.1.1.21 版本之前的库，升级后可能缺失 SOI 相关角色，可手动调用此过程修复问题。

调用该过程需要 DBA 权限。

参数说明:

无。

返回值:

无。

举例说明:

DM8 8.1.1.21 版本之前的库，升级后执行此过程，可以再次使用 SOI 角色。

```
SP_FIX_SOI_PRIVS();
```

57) SP_CLEAR_ALL_CACHEITEMS

定义:

SP_CLEAR_ALL_CACHEITEMS();

功能说明:

清空当前站点的所有缓存项信息，即令所有缓存项失效。注意，不能失效未构造完成的包信息缓存和仍被引用的 SQL 语句缓存。

参数说明:

无。

返回值:

无。

举例说明:

清空当前站点的所有缓存项信息。

```
SP_CLEAR_ALL_CACHEITEMS ();
```

58) SP_CLEAR_RAFT_ALL_CACHEITEMS

定义:

SP_CLEAR_RAFT_ALL_CACHEITEMS(

raft_id INT

);

功能说明:

清空指定站点的所有缓存项信息，即令所有缓存项失效。注意，不能失效未构造完成的包信息缓存和仍被引用的 SQL 语句缓存。

参数说明:

raft_id：站点号，非 DMDPC 环境下忽略站点号。如果站点号为 NULL，则清除所有站点的执行缓存信息；否则清理指定站点的执行缓存信息。

返回值:

无。

举例说明:

清空所有站点的执行缓存信息。

```
SP_CLEAR_RAFT_ALL_CACHEITEMS (NULL);
```

59) SP_RESET_MPOOL_PEAK_SIZE();

定义:

SP_RESET_MPOOL_PEAK_SIZE();

功能说明:

重置各内存池的峰值为当前 TOTAL_SIZE。

参数说明:

无。

返回值:

无。

举例说明:

重置各内存池的峰值为当前 TOTAL_SIZE。TOTAL_SIZE 可通过 V$MEM_POOL 查看。

```
SP_RESET_MPOOL_PEAK_SIZE();
```

3. 备份恢复管理

1) SF_BAKSET_BACKUP_DIR_ADD

定义:

INT

SF_BAKSET_BACKUP_DIR_ADD(

device_type varchar,

backup_dir varchar(256)

)

功能说明:

添加备份目录。若添加目录已经存在或者为库默认备份路径，则认为已经存在，系统不添加但也不报错。

参数说明:

device_type：待添加的备份目录对应存储介质类型，DISK 或者 TAPE。目前，无论指定介质类型为 DISK 或者 TAPE，都会同时搜索两种类型的备份集。

backup_dir：待添加的备份目录。

返回值:

1：目录添加成功；其它情况下报错。

举例说明:

```
SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
```

2) SF_BAKSET_BACKUP_DIR_REMOVE

定义:

INT

SF_BAKSET_BACKUP_DIR_REMOVE (

device_type varchar,

backup_dir varchar(256)

)

功能说明:

删除备份目录。若删除目录为库默认备份路径，不进行删除，认为删除失败。若指定目

录存在于记录的合法目录中，则删除；不存在或者为空则跳过，正常返回。

参数说明:

device_type：待删除的备份目录对应存储介质类型。待删除的备份目录对应存储介质类型，DISK 或者 TAPE。

backup_dir：待删除的备份目录。

返回值:

1：目录删除成功、目录不存在或者目录为空；

0：目录为库默认备份路径；其他情况报错。

举例说明:

```
SELECT SF_BAKSET_BACKUP_DIR_REMOVE('DISK','/home/dm_bak');
```

3） SF_BAKSET_BACKUP_DIR_REMOVE_ALL

定义:

INT

SF_BAKSET_BACKUP_DIR_REMOVE_ALL()

功能说明:

清理全部备份目录，默认备份目录除外。

返回值:

1：目录全部清理成功；其它情况下报错。

举例说明:

```
SELECT SF_BAKSET_BACKUP_DIR_REMOVE_ALL();
```

4） SF_BAKSET_CHECK

定义:

```
INT
SF_BAKSET_CHECK(
device_type varchar,
bakset_path varchar(256)
)
```

功能说明:

对备份集进行校验。

参数说明:

device_type：设备类型，disk 或 tape。

bakset_path：待校验的备份集目录。

返回值:

1：备份集目录存在且合法；否则报错。

举例说明:

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_check';
SELECT SF_BAKSET_CHECK('DISK','/home/dm_bak/db_bak_for_check');
```

5) SF_BAKSET_REMOVE

定义:

```
INT
SF_BAKSET_REMOVE (
device_type varchar,
backsetpath varchar(256),
option integer
)
```

功能说明:

删除指定设备类型和指定备份集目录的备份集。一次只检查一个合法.meta 文件，然后删除对应备份集；若存在非法或非正常备份的.meta 文件，则报错或直接返回，不会接着检查下一个.meta 文件;若同一个备份集下还存在其它备份文件或备份集,则只删除备份文件，不会删除整个备份集。

参数说明:

device_type：设备类型，disk 或 tape。

backsetpath：待删除的备份集目录。

option：删除备份集选项，0 默认删除，1 级联删除。可选参数。并行备份集中子备份集不允许单独删除；目标备份集被其他备份集引用为基备份的，默认删除，报错；级联删除情况下，会递归将相关的增量备份也删除。

返回值:

1：备份集目录删除成功；其它情况下报错。

举例说明:

例 1

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove';
BACKUP DATABASE INCREMENT BACKUPSET '/home/dm_bak/db_bak_for_remove_incr';
SELECT SF_BAKSET_REMOVE('DISK','/home/dm_bak/db_bak_for_remove');
```

执行结果如下:

```
[-8202]:[/home/dm_bak/db_bak_for_remove_incr]的基备份，不能删除.
```

例 2

```
SELECT SF_BAKSET_REMOVE('DISK','/home/dm_bak/db_bak_for_remove',1);
```

执行结果如下:

```
1
```

6） SF_BAKSET_REMOVE_BATCH

定义:

```
INT
SF_BAKSET_REMOVE_BATCH (
device_type varchar,
end_time      datetime,
range         int,
obj_name      varchar(257)
)
```

功能说明:

批量删除满足指定条件的所有备份集。

参数说明:

device_type：设备类型，disk 或 tape。为 NULL 时表示 disk 设备类型。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

range：指定删除备份的级别。1 代表库级；2 代表表空间级；3 代表表级；4 代表归档备份。若指定 NULL，则忽略备份集备份级别的区分。

obj_name：待删除备份集中备份对象的名称，仅表空间级和表级有效。若为表级备份删除，则需指定完整的表名（模式.表名）。否则，将认为删除会话当前模式下的表备份。若指定为 NULL，则忽略备份集中备份对象名称区分。

返回值:

1：备份集目录删除成功；其它情况下报错。

举例说明:

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove';
BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove';
SELECT SF_BAKSET_REMOVE_BATCH ('DISK', now(), NULL, NULL);
```

7) SF_BAKSET_REMOVE_BATCH_S

定义:

```
INT
SF_BAKSET_REMOVE_BATCH_S (
device_type varchar,
end_time      datetime,
range         int,
obj_name      varchar(257)
)
```

功能说明:

批量安全删除满足指定条件的所有库级备份集，保留备份时间最新的库级完全备份集。

参数说明:

device_type：设备类型，disk 或 tape。为 NULL 时表示 disk 设备类型。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

range：指定删除备份的级别。1 代表库级，2 代表表空间级，3 代表表级，4 代表归档备份。若指定为 NULL，则忽略备份集备份级别的区分。SF_BAKSET_REMOVE_BATCH_S 目前仅支持批量安全删除库级备份集，若参数指定为 2、3、4，则 SF_BAKSET_REMOVE_BATCH_S 的功能与 SF_BAKSET_REMOVE_BATCH 相同。

obj_name：待删除备份集中备份对象的名称，仅表空间级和表级有效。若为表级备

份删除，则需指定完整的表名（模式.表名）。否则，将认为删除会话当前模式下的表备份。若指定为 NULL，则忽略备份集中备份对象名称区分。

返回值:

1：备份集目录删除成功；其它情况下报错。

举例说明:

SF_BAKSET_REMOVE_BATCH_S 将保留库级完全备份 db_bak_for_remove_s_2，删除其余三个备份集。

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove_s_1';
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove_s_2';
BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove_s_1';
BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove_s_2';
SELECT SF_BAKSET_REMOVE_BATCH_S ('DISK', now(), NULL, NULL);
```

8) SF_BAKSET_REMOVE_BATCH_N

定义:

```
INT
SF_BAKSET_REMOVE_BATCH_N (
device_type   varchar,
end_time      datetime,
range         int,
obj_name      varchar(257),
count         int
)
```

功能说明:

批量删除满足指定条件的所有备份集，保留备份时间最新的 count 个库级完全备份集。

参数说明:

device_type：设备类型，disk 或 tape。为 NULL 时表示 disk 设备类型。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

range：指定删除备份的级别。1 代表库级，2 代表表空间级，3 代表表级，4 代表归档备份。若指定为 NULL，则忽略备份集备份级别的区分。SF_BAKSET_REMOVE_BATCH_N 目前仅支持批量安全删除库级备份集，若参数指定为 2、3、4，则 SF_BAKSET_REMOVE_BATCH_N 的功能与 SF_BAKSET_REMOVE_BATCH 相同。

obj_name：待删除备份集中备份对象的名称，仅表空间级和表级有效。若为表级备份删除，则需指定完整的表名（模式.表名）。否则，将认为删除会话当前模式下的表备份。若指定为 NULL，则忽略备份集中备份对象名称区分。

count：指定保留备份集的数量。

返回值:

1：备份集目录删除成功；其它情况下报错。

举例说明:

SF_BAKSET_REMOVE_BATCH_N 将保留库级完全备份 db_bak_for_remove_s_1 和 db_bak_for_remove_s_2，删除其余两个备份集。

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove_s_1';
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_remove_s_2';
BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove_s_1';
BACKUP TABLESPACE MAIN FULL BACKUPSET '/home/dm_bak/ts_bak_for_remove_s_2';
SELECT SF_BAKSET_REMOVE_BATCH_S ('DISK', now(), NULL, NULL, 2);
```

9） SP_DB_BAKSET_REMOVE_BATCH

定义:

SP_DB_BAKSET_REMOVE_BATCH (

device_type varchar,

end_time datetime

)

功能说明:

批量删除指定时间之前的数据库备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

参数说明:

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

举例说明:

```
BACKUP DATABASE FULL BACKUPSET '/home/dm_bak/db_bak_for_batch_del';
SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SP_DB_BAKSET_REMOVE_BATCH('DISK', NOW());
```

10) SP_TS_BAKSET_REMOVE_BATCH

定义:

SP_TS_BAKSET_REMOVE_BATCH (

device_type varchar,

end_time datetime,

ts_name varchar(128)

)

功能说明:

批量删除指定表空间对象及指定时间之前的表空间备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

参数说明:

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

ts_name：表空间名，若未指定，则认为删除所有满足条件的表空间备份集。

举例说明:

```
BACKUP TABLESPACE MAIN BACKUPSET '/home/dm_bak/ts_bak_for_batch_del';

SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');

SP_TS_BAKSET_REMOVE_BATCH('DISK',NOW(),'MAIN');
```

11) SP_TAB_BAKSET_REMOVE_BATCH

定义:

```
SP_TAB_BAKSET_REMOVE_BATCH (
device_type varchar,
end_time      datetime,
sch_name varchar(128),
tab_name varchar(128)
)
```

功能说明:

批量删除指定表对象及指定时间之前的表备份集。使用该方法前，需要先使用

SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径下的备份集。

参数说明:

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

sch_name：表所属的模式名。

tab_name：表名，只要模式名和表名有一个指定，就认为需要匹配目标；若均指定为 NULL，则认为删除满足条件的所有表备份。

举例说明:

```
CREATE TABLE TAB_FOR_BATCH_DEL(C1 INT);
BACKUP TABLE TAB_FOR_BATCH_DEL BACKUPSET'/home/dm_bak/tab_bak_for_batch_del';
SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');
SP_TAB_BAKSET_REMOVE_BATCH('DISK',NOW(),'SYSDBA','TAB_FOR_BATCH_DEL');
```

12）SP_ARCH_BAKSET_REMOVE_BATCH

定义:

SP_ARCH_BAKSET_REMOVE_BATCH (

device_type varchar,

end_time datetime

)

功能说明:

批量删除指定时间之前的归档备份集。使用该方法前，需要先使用 SF_BAKSET_BACKUP_DIR_ADD 添加将要删除的备份集目录，否则只删除默认备份路径

下的备份集。

参数说明:

device_type：设备类型，disk 或 tape。指定 NULL，则忽略存储设备的区分。

end_time：删除备份集生成的结束时间，仅删除 end_time 之前的备份集，必须指定。

举例说明:

```
BACKUP ARCHIVELOG BACKUPSET '/home/dm_bak/arch_bak_for_batch_del';

SELECT SF_BAKSET_BACKUP_DIR_ADD('DISK','/home/dm_bak');

SP_ARCH_BAKSET_REMOVE_BATCH('DISK', NOW());
```

4. 定时器管理

1) SP_ADD_TIMER*

定义一:

```
SP_ADD_TIMER(
timer_name                  varchar(128),
type                        int,
freq_month_week_interval    int,
freq_sub_interval           int,
freq_minute_interval        int,
start_time                  time,
end_time                    time,
during_start_date           datetime(0),
during_end_time             datetime(0),
no_end_date_flag            bool,
```

describe varchar(128),

is_valid bool

)

功能说明:

创建一个定时器。

参数说明:

timer_name: 定时器名称，应使用普通标识符，包含特殊符号可能导致无法正常使用。

type: 定时器调度类型，取值范围为 1~9;: 1: 执行一次；2: 按日执行；3: 按周执行；4: 按月执行的第几天；5: 按月执行的第一周；6: 按月执行的第二周；7: 按月执行的第三周；8: 按月执行的第四周； 9: 按月执行的最后一周。

freq_month_week_interval: 间隔的月/周/天（调度类型决定）数。

freq_sub_interval: 第几天/星期几/一个星期中的某些天。

freq_minute_interval: 间隔的分钟数。

start_time: 开始时间。

end_time: 结束时间。

during_start_date: 有效日期时间段的开始日期时间。只有当前时间大于该字段时，该定时器才有效。

during_end_date: 有效日期时间段的结束日期时间。

no_end_date_flag: 是否有结束日期（0: 有结束日期；1: 没有结束日期）。

describe: 描述。

is_valid: 定时器是否有效。

返回值:

无

说明:

1. type = 1 时,只有 start_time、during_start_date 有意义;freq_sub_interval、freq_month_week_interval、freq_minute_interval、end_time、during_end_date 无效，no_end_date_flag 会被自动调整为 FALSE，during_end_date 会被自动调整为 during_start_date 的下一分钟。

2. type = 2 时，freq_month_week_interval 有效，表示相隔几天，取值范围为 1~100；freq_sub_interval 无效；freq_minute_interval <=24* 60 有效。

a) 当 freq_minute_interval = 0 时，当天只执行一次。end_time 无效;

b) 当 0<freq_minute_interval <=24*60 时，表示当天从 start_time 时间开始，每隔 freq_minute_interval 分钟执行一次;

c) 当 freq_minute_interval > 24* 60 时，非法。

3. type = 3 时，意思是每隔多少周开始工作（从开始日期算起)。计算方法为：(当前日期和开始日期的天数之差/7)% freq_month_week_interval =0 且当前是星期 freq_sub_interval，其中 freq_sub_interval 的八位如下表所示:

8	7	6	5	4	3	2	1

1-7 位分别代表星期天，星期一、星期二...、星期六，第 8 位无意义。这几位为 1 表示满足条件，为 0 表示不满足条件。

a) 1<=freq_month_week_interval<=100，代表每隔多少周;

b) 1<=freq_sub_interval<=127 代表星期中的某些天;

c) freq_minute_interval 代表分钟数。

当 freq_minute_interval = 0 时，当天只执行一次。end_time 无效；当 freq_minute_interval > 0 且,表示当天可执行多次;当 freq_minute_interval > 24* 60 时，非法。

4. type = 4 时，每 freq_month_week_interval 个月的第 freq_sub_interval

日开始工作。其中，是否满足 freq_month_week_interval 个月的判断条件是：（月份的差 +（日期的差>=15||日期的差<= -15)?1:0 ）% freq_month_week_interval = 0 且当前是当月的 freq_sub_interval 日，表示满足条件；否则不满足。其中，“月份的差”和“日期的差”分别指的是系统当前时间和字段“during_start_date”中的月份差值和日期差值。

a） 1<=freq_sub_interval<31，代表第几日；

b） 1<=freq_month_week_interval<=100，代表每隔多少个月；

c） freq_minute_interval 代表分钟数。

当 freq_minute_interval = 0 时，当天只执行一次。end_time 无效；当 freq_minute_interval > 0 且，表示当天可执行多次；当 freq_minute_interval >24* 60 时，非法。

5. type = 5，6，7，8，9 时，每 freq_month_week_interval 个月的第 type-4 周的周 freq_sub_interval 开始工作。其中，是否满足 freq_month_week_interval 个月的判断条件是：（月份的差 + 日期的差/15）% freq_month_week_interval = 0 且当前是当月的第 type-4 周的周 freq_sub_interval，表示满足条件；否则不满足。

a） 1<=freq_sub_interval<=7，代表星期天到星期六（星期天是一个星期的第一天）；

b） 1<=freq_month_week_interval<=100，代表每隔多少个月；

c） freq_minute_interval 代表分钟数。

当 freq_minute_interval = 0 时，当天只执行一次。end_time 无效；当 freq_minute_interval > 0 且，表示当天可执行多次；当 freq_minute_interval > 24* 60 时，非法。

6. 如果 no_end_date_flag = TRUE：表示永远不结束，一直存在下去。

7. 如果 is_valid= TRUE：表示定时器创建时就有效。

8. 总结 type 取值 1~9 时，freq_sub_interval、freq_month_week_interval 和

freq_minute_interval 各自对应的有效值范围如下表所示:

type	1		2		3		4		5,6,7,8,9	
	max	min	max	min	max	min	max	min	max	min
freq_sub_interval	0	0	0	0	127	1	31	1	7	1
freq_month_week_interval	0	0	100	1	100	1	100	1	100	1
freq_minute_interval	0	0	1440	0	1440	0	1440	0	1440	0

举例说明:

创建一个定时器, 每天 02: 00 进行调度, 开始日期: 2011-02-01, 结束日期: 2011-09-01, 间隔天数 1 天，每隔一分钟循环执行:

```
SP_ADD_TIMER('TIMER1', 2, 1, 0, 1, '02:00:00', '20:00:00', '2011-02-01 14:30:34', '2011-09-01', 0, '每天凌晨两点进行调度', 1);
```

2) SP_ADD_TIMER*

定义二:

```
SP_ADD_TIMER(
timer_name              varchar(128),
repeat_interval         varchar(128),
start_time              time,
end_time                time,
during_start_date       datetime(0),
during_end_time         datetime(0),
no_end_date_flag        bool,
describe                varchar(128),
is_valid                bool
)
```

功能说明

创建一个定时器，按 REPEAT_INTERVAL 指定的日历表达式的频率进行。

参数说明

timer_name：定时器名称，应使用普通标识符，包含特殊符号可能导致无法正常使用。

repeat_interval: 指定日历表达式，语法详见《DM9 系统包使用手册》DBMS_SCHEDULER 包中关于 REPEAT_INTERVAL 的介绍

start_time：开始时间。

end_time：结束时间。

during_start_date：有效日期时间段的开始日期时间。只有当前时间大于该字段时，该定时器才有效。

during_end_date：有效日期时间段的结束日期时间。

no_end_date_flag：是否有结束日期（0：有结束日期；1：没有结束日期）。

describe：描述。

is_valid：定时器是否有效。

返回值

无

举例说明

```
SP_ADD_TIMER('TIMER1', ' FREQ=SECONDLY;INTERVAL=1', '02:00:00', '20:00:00', '2011-02-01 14:30:34',
'2011-09-01', 0, '每天凌晨两点进行调度', 1);
```

3） SP_DROP_TIMER*

定义：

SP_DROP_TIMER（

timer_name varchar(128)

)

功能说明:

删除一个定时器。

参数说明:

timer_name：定时器名。

返回值:

无

举例说明:

删除定时器 TIMER1:

```
SP_DROP_TIMER('TIMER1');
```

4) SP_OPEN_TIMER*

定义:

SP_OPEN_TIMER (

timer_name varchar(128)

)

功能说明:

打开一个定时器。仅在内存中生效。

参数说明:

timer_name：定时器名。

返回值:

无

举例说明:

打开定时器 TIMER1:

```
SP_OPEN_TIMER('TIMER1');
```

5) SP_CLOSE_TIMER*

定义:

SP_CLOSE_TIMER (

timer_name varchar(128)

)

功能说明:

关闭一个定时器。仅在内存中生效。

参数说明:

timer_name: 定时器名。

返回值:

无

举例说明:

关闭定时器 TIMER1:

```
SP_CLOSE_TIMER('TIMER1');
```

5. 模式对象相关信息管理

用户必须拥有相应对象的权限才可以获取到相关信息。

1) SP_TABLEDEF

定义:

SP_TABLEDEF (

schname varchar(128),

tablename varchar(128)

)

功能说明:

以结果集的形式返回表的定义，当表定义过长时，会以多行返回。当水平分区表的 hash 分区超过 30 个时，则使用 PARTITIONS n_subs 代替完整定义，避免子表过多展示不全。

参数说明:

schname：模式名。

tablename：表名。

返回值:

无

举例说明:

```
SP_TABLEDEF('PRODUCTION','PRODUCT');
```

2) TABLEDEF

定义:

TEXT

TABLEDEF (

username varchar(128),

tablename varchar(128)

)

功能说明:

以结果集的形式返回表的定义，当表定义过长时，会以多行返回。当水平分区表的 hash 分区超过 30 个时，则使用 PARTITIONS n_subs 代替完整定义，避免子表过多展示不全。

参数说明:

username：模式名。

tablename：表名。

返回值:

表的定义

举例说明:

```
TABLEDEF('PERSON','PERSON');
```

3) SF_VIEW_EXPIRED

定义:

INT

SF_VIEW_EXPIRED(

schname varchar(128),

viewname varchar(128)

)

功能说明:

检查当前系统表中视图列定义是否有效。

返回值:

返回 0 或 1。0 表示有效；1 表示无效。

举例说明:

```
CREATE TABLE T_01_VIEW_DEFINE_00(C1 INT,C2 INT);

CREATE VIEW TEST_T_01_VIEW_DEFINE_00 AS SELECT* FROM T_01_VIEW_DEFINE_00;

SELECT SF_VIEW_EXPIRED('SYSDBA','TEST_T_01_VIEW_DEFINE_00');

//查询结果为 0



ALTER TABLE T_01_VIEW_DEFINE_00 DROP COLUMN C1 CASCADE ;

SELECT SF_VIEW_EXPIRED('SYSDBA','TEST_T_01_VIEW_DEFINE_00');

//查询结果为 1
```

4） CHECKDEF

定义:

VARCHAR

CHECKDEF（

consid int,

preflag int

）

功能说明:

获得 check 约束的定义。

参数说明:

consid：check 约束 id 号。

preflag：对象前缀个数。1 表示导出模式名；0 表示只导出对象名。

返回值:

check 约束的定义。

举例说明:

```
CREATE TABLE TEST_CHECKDEF(C1 INT CHECK(C1>10));
//通过查询系统表
SELECT A.NAME, A.ID FROM SYSOBJECTS A, SYSOBJECTS B WHERE B.NAME='TEST_CHECKDEF' AND
A.PID=B.ID AND A.SUBTYPE$='CONS';
//得到约束 ID 为 134217770
SELECT  CHECKDEF(134217770,1);
```

5） CONSDEF

定义:

VARCHAR

CONSDEF (

indexid int,

preflag int

)

功能说明:

获取 unique 约束的定义。

参数说明:

indexid：索引号数字字符串。

preflag：对象前缀个数。1 表示导出模式名；0 表示只导出对象名。

返回值:

unique 约束的定义。

举例说明:

```
CREATE TABLE TEST_CONSDEF(C1 INT PRIMARY KEY,C2 INT, CONSTRAINT CONS1 UNIQUE(C2));
//通过查询系统表
SELECT C.INDEXID FROM SYSOBJECTS O,SYSCONS C WHERE O.NAME='CONS1' AND O.ID=C.ID;
//系统生成 C2 上的 INDEX 为 33555481
SELECT CONSDEF(33555481,1);
```

6) INDEXDEF

定义:

VARCHAR

INDEXDEF (

indexid int,

preflag int

)

功能说明:

获取 INDEX 的创建定义。

参数说明:

indexid: 索引 ID。

preflag: 对象前缀个数。1 表示导出模式名; 0 表示只导出对象名。

返回值:

索引的创建定义。

举例说明:

```
CREATE INDEX PRODUCT_IND ON PRODUCTION.PRODUCT(PRODUCTID);
//查询系统表得到索引 ID
SELECT NAME, ID FROM SYSOBJECTS WHERE NAME='PRODUCT_IND' AND SUBTYPE$='INDEX';
SELECT INDEXDEF(33555530,1);
```

7) SP_REORGANIZE_INDEX

定义:

SP_REORGANIZE_INDEX (

schname varchar(128),

indexname varchar(128)

)

功能说明:

对指定索引进行空间整理。拥有 DBA/DB_OBJECT_ADMIN 角色权限的用户才能对其

他模式下的索引进行空间整理，普通用户仅能对自己模式下的索引进行空间整理。

参数说明:

schname：模式名。

indexname：索引名。

返回值:

无

举例说明:

```
CREATE INDEX PRODUCT_IND ON PRODUCTION.PRODUCT(PRODUCTID);

SP_REORGANIZE_INDEX('PRODUCTION','PRODUCT_IND');
```

8） SP_REBUILD_INDEX

定义:

SP_REBUILD_INDEX (

schname varchar(128),

indexid int

)

功能说明:

重建索引。约束：1. 水平分区子表、临时表和系统表上建的索引不支持重建；2. 虚索引和聚集索引不支持重建。

只有拥有 DBA/DB_OBJECT_ADMIN 角色权限的用户才能重建其他模式下的索引，普通用户仅能重建自己模式下的索引。

参数说明:

schname：模式名。

indexid：索引 ID。

返回值:

无

举例说明:

```
CREATE INDEX PRODUCT_IND ON PRODUCTION.PRODUCT(PRODUCTID);

select ID from SYS.SYSOBJECTS where name='PRODUCT_IND';

//查询系统表 SYSOBJECTS 得到索引 ID（33555595)

SP_REBUILD_INDEX('PRODUCTION', 33555595);
```

9） CONTEXT_INDEX_DEF

定义:

VARCHAR

CONTEXT_INDEX_DEF (

indexid int,

preflag int

)

功能说明:

获取全文索引的创建定义。

参数说明:

indexid：全文索引 ID。

preflag：对象前缀个数。1 表示导出模式名；0 表示只导出对象名。

返回值:

全文索引的创建定义。

举例说明:

```
CREATE CONTEXT INDEX PRODUCT_CIND ON PRODUCTION.PRODUCT(NAME) LEXER
```

```
DEFAULT_LEXER;

--查询系统表得到全文索引 ID

SELECT NAME, ID FROM SYSOBJECTS WHERE NAME='PRODUCT_CIND';

SELECT CONTEXT_INDEX_DEF(33555531, 1);
```

10）SYNONYMDEF

定义:

VARCHAR

SYNONYMDEF (

username varchar(128),

synname varchar(128),

type int,

preflag int

)

功能说明:

获取同义词的创建定义。

参数说明:

username：用户名。

synname：同义词名。

type：同义词类型。 0，public 1，user。

preflag：对象前缀个数。1 表示导出模式名；0 表示只导出对象名。

返回值:

同义词的创建定义。

举例说明:

```
SELECT SYNONYMDEF('SYSDBA', 'SYSOBJECTS',0,1);
```

11） SEQDEF

定义:

VARCHAR

SEQDEF (

seqid int,

preflag int

)

功能说明:

获取序列的创建定义。

参数说明:

seqid：序列 id 号。

preflag：对象前缀个数。1 表示导出模式名；0 表示只导出对象名。

返回值:

序列的创建定义。

举例说明:

```
CREATE SEQUENCE SEQ1;

SELECT ID FROM SYSOBJECTS WHERE NAME='SEQ1'; /查出 id 为 167772160

SELECT SEQDEF(167772160, 1);
```

12）IDENT_CURRENT

定义:

INT

IDENT_CURRENT（

fulltablename varchar(261)

）

功能说明:

获取自增列当前值，不能用于 AUTO_INCREMENT 自增列。

参数说明:

fulltablename：表全名；格式为“模式名.表名”。

返回值:

自增列当前值。

举例说明:

```
SELECT IDENT_CURRENT('PRODUCTION.PRODUCT');
```

13）IDENT_SEED

定义:

INT

IDENT_SEED（

fulltablename varchar(261)

）

功能说明:

获取自增列种子，不能用于 AUTO_INCREMENT 自增列。

参数说明:

fulltablename：表全名；格式为“模式名.表名”。

返回值:

自增列种子。

举例说明:

```
SELECT IDENT_SEED('PRODUCTION.PRODUCT');
```

14）IDENT_INCR

定义:

INT

IDENT_INCR (

fulltablename varchar(261)

)

功能说明:

获取自增列增量值 increment，不能用于 AUTO_INCREMENT 自增列。

参数说明:

fulltablename：表全名；格式为“模式名.表名”。

返回值:

自增列增量值 increment。

举例说明:

```
SELECT IDENT_INCR('PRODUCTION.PRODUCT');
```

15）SCOPE_IDENTITY

定义:

INT

SCOPE_IDENTITY ();

功能说明:

返回插入到同一作用域中的 identity 列内的最后一个 identity 值。

返回值:

RVAL：函数返回值，长度为 8。

举例说明:

详见GLOBAL_IDENTITY例子。

16）GLOBAL_IDENTITY

定义:

INT

GLOBAL_IDENTITY();

功能说明:

返回在当前会话中的任何表内所生成的最后一个标识值，不受限于特定的作用域。一个作用域就是一个模块：存储过程、触发器、函数或批处理，若两个语句处于同一个存储过程、函数或批处理中，则它们位于相同的作用域中。

返回值:

RVAL：函数返回值，长度为 8。

举例说明:

```
DROP TABLE T1;

DROP TABLE T2;

CREATE TABLE T1(C1 INT IDENTITY(1,1), C2 CHAR);

CREATE TABLE T2(C1 INT IDENTITY(1,1), C2 CHAR);

INSERT INTO T1(C2) VALUES('a');

INSERT INTO T1(C2) VALUES('b');

INSERT INTO T1(C2) VALUES('c');

COMMIT;
```

```
SELECT SCOPE_IDENTITY();

//返回值：3

SELECT GLOBAL_IDENTITY();

//返回值：3

CREATE OR REPLACE TRIGGER TRI1 AFTER INSERT ON T1

BEGIN

    INSERT INTO T2(C2) VALUES('a');

END;

/



INSERT INTO T1(C2) VALUES('d');

COMMIT;



SELECT SCOPE_IDENTITY();

//返回值：4

SELECT GLOBAL_IDENTITY();

//返回值：1
```

17) SF_CUR_SQL_STR

定义：

```
    CLOB

    SF_CUR_SQL_STR (

    is_top        int

    )
```

功能说明:

用于并行环境中，获取当前执行的 SQL 语句。

参数说明:

is_top：取 0 时返回当前层计划执行的语句；取 1 时返回顶层计划语句。

返回值:

sql 语句。

举例说明:

```
SELECT SF_CUR_SQL_STR(0);
```

18）SF_COL_IS_CHECK_KEY

定义:

INT

SF_COL_IS_CHECK_KEY (

key_num　　int,

key_info　　varchar(32767),

col_id　　int

)

功能说明:

判断一个列是否为CHECK约束列。

参数说明:

key_num：约束列总数。

key_info：约束列信息。

col_id：列id。

返回值:

返回1表示该列是check约束列，否则返回0。

举例说明:

```
CREATE TABLE TC (C1 INT, C2 DOUBLE, C3 DATE, C4 VARCHAR, CHECK(C1 < 100 AND C4 IS NOT
NULL));
SELECT TBLS.NAME, COLS.NAME, COLS.COLID, COLS.TYPE$, COLS.LENGTH$, COLS.SCALE,
COLS.NULLABLE$, COLS.DEFVAL
FROM (SELECT ID, NAME FROM SYS.SYSOBJECTS WHERE NAME = 'TC' AND TYPE$ = 'SCHOBJ' AND
SUBTYPE$ = 'UTAB'  AND SCHID = (SELECT ID FROM SYS.SYSOBJECTS WHERE NAME = 'SYSDBA' AND
TYPE$ = 'SCH')) AS TBLS,
(SELECT ID, PID, INFO1, INFO6 FROM SYS.SYSOBJECTS WHERE TYPE$ = 'TABOBJ' AND SUBTYPE$ =
'CONS') AS CONS_OBJ,SYS.SYSCOLUMNS AS COLS,SYS.SYSCONS AS CONS WHERE  TBLS.ID =
CONS_OBJ.PID AND TBLS.ID = COLS.ID AND SF_COL_IS_CHECK_KEY(CONS_OBJ.INFO1,
CONS_OBJ.INFO6, COLS.COLID) = 1 AND  CONS.TABLEID = TBLS.ID AND CONS.TYPE$ = 'C';
```

19) SF_REPAIR_HFS_TABLE

定义:

```
INT
SF_REPAIR_HFS_TABLE (
schname   varchar(128),
tabname   varchar(128)
)
```

功能说明:

HUGE表日志属性为LOG NONE时，如果系统出现故障，导致该表数据不一致，则通过该函数修复表数据，保证数据的一致性。

参数说明:

schname：模式名。

tabname：表名。

返回值:

成功返回0，否则报错。

举例说明:

```
CREATE HUGE TABLE T_DM(C1 INT, C2 VARCHAR(20)) LOG NONE;

INSERT INTO T_DM VALUES(99, 'DM8');

COMMIT;

UPDATE T_DM SET C1 = 100;//系统故障

SF_REPAIR_HFS_TABLE('SYSDBA','T_DM');
```

20) SP_ENABLE_EVT_TRIGGER

定义:

SP_ENABLE_EVT_TRIGGER (

schname varchar(128),

triname varchar(128),

enable bool

)

功能说明:

禁用/启用指定的事件触发器。

参数说明:

schname：模式名。

triname：触发器名。

enable：1 表示启用，0 表示禁用。

返回值：

成功返回0，否则报错。

举例说明：

```
SP_ENABLE_EVT_TRIGGER('SYSDBA', 'TRI_1', 1);

SP_ENABLE_EVT_TRIGGER('SYSDBA', 'TRI_1', 0);
```

21）SP_ENABLE_ALL_EVT_TRIGGER

定义：

SP_ENABLE_ALL_EVT_TRIGGER (

enable bool

)

功能说明：

禁用/启用数据库上的所有事件触发器。

参数说明：

ENABLE：1 表示启用，0 表示禁用。

返回值：

成功返回0，否则报错。

举例说明：

```
SP_ENABLE_ALL_EVT_TRIGGER(1);

SP_ENABLE_ALL_EVT_TRIGGER(0);
```

22）SF_GET_TRIG_DEPENDS

定义：

BINARY

SF_GET_TRIG_DEPENDS(

trigid int

)

功能说明:

查看触发器的依赖项。

参数说明:

trigid：触发器id。

返回值:

触发器的依赖项。

举例说明:

```
CREATE TABLE T1_TRI_10000(OBJECTTYPE VARCHAR);

CREATE TRIGGER TRI1_TRI_10000 BEFORE CREATE ON DATABASE BEGIN INSERT INTO T1_TRI_10000

VALUES(:EVENTINFO.OBJECTTYPE);

END;

/


select SF_GET_TRIG_DEPENDS((Select id from sysobjects where name like 'TRI1_TRI_10000'));
```

23) SF_GET_PROC_DEPENDS

定义:

BINARY

SF_GET_PROC_DEPENDS(

procid int

)

功能说明:

查看过程/函数的依赖项。

参数说明:

procid：过程/函数id。

返回值:

过程/函数的依赖项。

举例说明:

```
CREATE OR REPLACE FUNCTION FUN1 (A INT) RETURN INT AS
        S   INT;
BEGIN
  S:=A*A;
  RETURN S;
EXCEPTION
  WHEN OTHERS THEN NULL;
END;
/


CREATE OR REPLACE FUNCTION FUN2 (A INT, B INT) RETURN INT AS
        S   INT;
BEGIN
  S:=A+B+FUN1(A);
  RETURN S;
EXCEPTION
```

```
  WHEN OTHERS THEN NULL;
END;
/

SELECT SF_GET_PROC_DEPENDS((SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'FUN2'));
```

24) SP_TMP_TABLE_CLEAR

定义:

SP_TMP_TABLE_CLEAR (

schname varchar(128),

tablename varchar(128)

)

功能说明:

清除指定的临时表中的数据, 本操作不会改变事务的状态, 但无论事务后续提交或回滚, 临时表清除操作都被永久化。

参数说明:

schname: 模式名。

tablename: 临时表名。

返回值:

无

举例说明:

SYSDBA 用户创建临时表 T1，之后清除临时表 T1 的数据。

```
CREATE TEMPORARY TABLE T1(C1 INT);
INSERT INTO T1 VALUES(1);
```

```
SP_TMP_TABLE_CLEAR('SYSDBA','T1');
```

25) SF_GET_VIEW_DEPEND_OBJINFO

定义:

CLOB

SF_GET_VIEW_DEPEND_OBJINFO (

schname varchar(128),

viewname varchar(128),

col_seq int

depend_flag int

)

功能说明:

获取 viewname 视图所依赖的对象信息。只支持视图查询项表达式为列类型情况，经过表达式计算得来的视图列不支持。

参数说明:

schname：模式名。

viewname：视图名。

col_seq： 视图的列序号。0表示返回视图中所有列依赖的基表列信息。N表示返回视图中第N列所依赖的基表列信息。

depend_flag:需要返回对象类型标记。目前只支持 1 返回基表对象信息。

返回值:

视图依赖的对象信息。

举例说明:

返回V1中第二列CITY所依赖的基表列信息。

```
CREATE VIEW V1 AS SELECT ADDRESS1,CITY FROM PERSON.ADDRESS;

SELECT SF_GET_VIEW_DEPEND_OBJINFO ('SYSDBA','V1',2,1);
```

返回V2中第一列ADDRESS1+10所依赖的基表列信息，表达式不支持，显示为UNKOWN。

```
CREATE VIEW V2 AS SELECT ADDRESS1,CITY FROM PERSON.ADDRESS;

SELECT SF_GET_VIEW_DEPEND_OBJINFO ('SYSDBA','V2',1,1);
```

26) SP_GET_MV_DEPEND_COLS_INFO

定义：

```
SP_GET_MV_DEPEND_COLS_INFO (
schname          varchar(128),
mv_name   varchar(128)
)
```

功能说明：

获取 mv_name 物化视图的列所直接依赖的对象列信息。当前用户必须对物化视图具有查询权限，否则不支持。

参数说明：

schname：模式名。

mv_name：物化视图名。

返回值：

物化视图依赖的对象列信息。

举例说明：

返回MV1所依赖的基表T1的列信息。

```
CREATE TABLE T1(C1 INT, C2 INT);

CREATE MATERIALIZED VIEW MV1 REFRESH ON COMMIT AS SELECT C1, C2 FROM T1;
```

```
SP_GET_MV_DEPEND_COLS_INFO('SYSDBA','MV1');
```

返回MV2所依赖的基视图V1的列信息。

```
CREATE VIEW V1 AS SELECT C1, C2 FROM T1;

CREATE MATERIALIZED VIEW MV2 REFRESH ON COMMIT AS SELECT C1, C2 FROM V1;

SP_GET_MV_DEPEND_COLS_INFO('SYSDBA','MV2');
```

27) SP_INIT_LIMITED_USER_QUOTA

定义:

SP_INIT_LIMITED_USER_QUOTA()

功能说明:

重新加载所有用户的表空间配额限制。

参数说明:

无

返回值:

无

举例说明:

重新加载所有用户的表空间配额限制。

```
SP_INIT_LIMITED_USER_QUOTA();
```

28) SP_SWITCH_SYSVIEW

定义:

SP_SWITCH_SYSVIEW (

flag int

)

功能说明:

对 DPC 环境下涉及分区相关的系统视图查询进行优化，使查询优化后的视图耗时降低，在数据量大时推荐切换为优化后的视图进行查询。

通过使用该存储过程可以在视图的原始定义和针对 DPC 优化后的视图定义之间切换，支持切换的系统视图包括：分区子表信息相关的 DBA_TAB_SUBPARTITIONS、ALL_TAB_SUBPARTITIONS、USER_TAB_SUBPARTITIONS；分区列信息相关的 USER_SUBPART_KEY_COLUMNS、ALL_SUBPART_KEY_COLUMNS。

DPC 环境默认使用优化后的视图。且本存储过程为 DPC 环境专用，在非 DPC 环境下使用时报错。

参数说明:

flag：取值 0 表示同义词指向原始视图定义，取值 1 表示使用优化后的视图定义；缺省为 1。

返回值:

无

举例说明:

DPC 环境下，设置使用原始系统视图定义。

```
SP_SWITCH_SYSVIEW(0);
```

29) SF_GET_EXTFUN_REF_NAME

定义:

VARCHAR

SF_GET_EXTFUN_REF_NAME (

proc_id int

)

功能说明:

获取外部函数的引用函数名。

参数说明:

proc_id：外部函数 id。

返回值:

外部函数的引用函数名。

举例说明:

查询 SYSDBA 模式下 JAVA 外部函数 MY_INT 的引用函数名。

```
SELECT SF_GET_EXTFUN_REF_NAME(ID) FROM SYSEXTS WHERE ID = (SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'MY_INT' AND SCHID = (SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'SYSDBA' AND TYPE$='SCH'));
```

30) SF_XBOX_SYS_DUMP_LEN

定义:

BIGINT

SF_XBOX_SYS_DUMP_LEN()

功能说明:

获取实例上所有 XBOX 临时文件保存的邮件总量，单位 BYTE。在 DMDPC 环境下需要直连实例所在的节点执行。

参数说明:

无

返回值:

实例上所有XBOX临时文件保存的邮件总量。

举例说明:

```
SELECT SF_XBOX_SYS_DUMP_LEN();
```

31) RESET_ATTRINFO_BY_ID

定义:

RESET_ATTRINFO_BY_ID (

objid int

)

功能说明:

根据传入的对象 ID，将其属性信息在系统表 SYSTYPEATTRINFO 中重置。

参数说明:

objid：需要重置的对象 ID。

返回值:

无

举例说明:

```
CREATE OR REPLACE TYPE TP1 AS OBJECT (
    A INT,
    B VARCHAR2(100)
);
SELECT ID FROM SYSOBJECTS WHERE NAME = 'TP1';
--根据查询到的对象 ID，调用系统函数
RESET_ATTRINFO_BY_ID(234882104);
```

32) RESET_ATTRINFO_BY_NAME

定义:

RESET_ATTRINFO_BY_NAME (

schname varchar(128),

objname varchar(128)

)

功能说明:

根据传入的schname和objname定位对象，并将系统表SYSTYPEATTRINFO中的对象属性信息重置。

参数说明:

schname：对象所属模式名。

objname：对象名。

返回值:

无

举例说明:

```
CREATE OR REPLACE TYPE SYSDBA.TP1 AS OBJECT (
    A INT,
    B VARCHAR2(100)
);
RESET_ATTRINFO_BY_NAME('SYSDBA', 'TP1');
```

33) SF_HUGE_CHECK_SECIND

定义:

SF_HUGE_CHECK_SECIND(

schname varchar(128),

tabname varchar(128)

)

功能说明:

检查huge表所有二级索引中数据是否与列存数据一致。

参数说明:

schname：对象所属模式名。

tabname：表名。

返回值:

二级索引数据一致时返回1，不一致时返回0，出错返回错误码。

举例说明:

```
CREATE HUGE TABLE HTAB_SECIND(C1 INT, C2 VARCHAR);

CREATE INDEX HTAB_SECIND_1 ON HTAB_SECIND(C1);

SELECT SF_HUGE_CHECK_SECIND('SYSDBA','HTAB_SECIND');
```

34) SF_GET_USER_DEFAULT_TS_GROUP

定义:

VARCHAR(128)

SF_GET_USER_DEFAULT_TS_GROUP

(

USER_NAME VARCHAR(128)

)

功能说明:

获取指定用户的默认表空间组名。如果此用户未指定默认表空间组，返回 NULL。

参数说明:

USER_NAME：用户名。

返回值:

指定用户的默认表空间组名。

举例说明:

获取用户 USER1 的默认表空间组名。

```
SELECT SF_GET_USER_DEFAULT_TS_GROUP('USER1');
```

6. 数据守护管理

本小节的存储过程都与 DM 的数据守护功能相关, 关于数据守护的概念和相关环境配置与操作可以参考《DM8 数据守护与读写分离集群 V4.0》相关章节。

1) SP_SET_OGUID

定义:

SP_SET_OGUID (

oguid int

)

功能说明:

设置主备库监控组的 ID 号。

参数说明:

oguid: oguid。

返回值:

无

举例说明:

```
SP_SET_OGUID (451245);
```

2) SF_GET_BROADCAST_ADDRESS

定义:

VARCHAR

SF_GET_BROADCAST_ADDRESS (

ip_addr varchar,

subnet_mask varchar

)

功能说明:

根据 IPv4 的 IP 地址以及子网掩码计算广播地址。

参数说明:

ip_addr：输入的 IP 地址。

subnet_mask：子网掩码地址。

返回值:

广播地址。

举例说明:

```
SELECT SF_GET_BROADCAST_ADDRESS('15.16.193.6','255.255.248.0');
```

3) SP_SET_RT_ARCH_VALID

定义:

SP_SET_RT_ARCH_VALID ()

功能说明:

设置实时归档有效。

参数说明:

无

返回值:

无

举例说明:

```
SP_SET_RT_ARCH_VALID ();
```

4) SP_SET_RT_ARCH_INVALID

定义:

SP_SET_RT_ARCH_INVALID ()

功能说明:

设置实时归档无效。

参数说明:

无

返回值:

无

举例说明:

```
SP_SET_RT_ARCH_INVALID ();
```

5) SF_GET_RT_ARCH_STATUS

定义:

VARCHAR

SF_GET_RT_ARCH_STATUS ()

功能说明:

获取实时归档状态。

参数说明:

无

返回值:

有效：VALID。

无效：INVALID

举例说明：

```
SELECT SF_GET_RT_ARCH_STATUS ();
```

6） SP_SET_ARCH_STATUS

定义：

SP_SET_ARCH_STATUS (

arch_dest varchar,

arch_status int

)

功能说明：

设置指定归档目标的归档状态。

如果主库是 DSC 集群，则将备库的归档状态置为无效后，需要连接对应备库执行 SP_CLEAR_PKG_DEPENDS 清理正在等待依赖包的待重演日志包，避免备库上的重演操作卡住。

参数说明：

arch_dest：归档目标名称。

arch_status：要设置的归档状态，取值 0 或 1，0 表示有效状态，1 表示无效状态。

返回值：

无

举例说明：

```
SP_SET_ARCH_STATUS('DM1', 1);
```

7） SP_SET_ALL_ARCH_STATUS

定义:

SP_SET_ALL_ARCH_STATUS (

arch_status int

)

功能说明:

设置所有归档目标的归档状态。

如果主库是 DSC 集群，且要将备库的归档状态置为无效时，要求主库的各节点都处于 SUSPEND 状态。

参数说明:

arch_status：要设置的归档状态，取值 0 或 1，0 表示有效状态，1 表示无效状态。

返回值:

无

举例说明:

```
SP_SET_ALL_ARCH_STATUS(1);
```

8） SP_APPLY_KEEP_PKG

定义:

SP_APPLY_KEEP_PKG()

功能说明:

APPLY 备库的 KEEP_PKG 数据。

参数说明:

无

返回值:

无

举例说明:

```
SP_APPLY_KEEP_PKG();
```

9) SP_DISCARD_KEEP_PKG

定义:

SP_DISCARD_KEEP_PKG()

功能说明:

丢弃备库的 KEEP_PKG 数据。

参数说明:

无

返回值:

无

举例说明:

```
SP_DISCARD_KEEP_PKG();
```

10) SP_CLEAR_ARCH_SEND_INFO

定义一:

SP_CLEAR_ARCH_SEND_INFO()

或

SP_CLEAR_ARCH_SEND_INFO(inst_name varchar)

功能说明:

此系统函数在主库上执行有效，用于清理到备库最近 N 次的归档发送信息。保留总的归档发送信息和最大发送信息。

如果不指定 inst_name，则清理所有备库最近 N 次的归档发送信息。

N 值为 V$ARCH_SEND_INFO 中的 RECNT_SEND_CNT 值。

参数说明:

inst_name：备库实例名。

返回值:

无

举例说明:

```
SP_CLEAR_ARCH_SEND_INFO();

SP_CLEAR_ARCH_SEND_INFO('GRP1_RWW_02');
```

定义二:

SP_CLEAR_ARCH_SEND_INFO(

clear_all　　int

)

功能说明:

此系统函数在主库上执行有效，用于清理到所有备库最近 N 次的归档发送信息。

可以通过指定 clear_all 判断是否需要清理总的归档发送信息和最大发送信息。

N 值为 V$ARCH_SEND_INFO 中的 RECNT_SEND_CNT 值。

参数说明:

clear_all：用于设置是否清理总的归档发送信息和最大发送信息。取值 0 或 1，若 clear_all 指定为 0，则不清理总的发送信息和最大发送信息；若 clear_all 指定为 1，则清理总的发送信息和最大发送信息。

返回值:

无

举例说明:

```
SP_CLEAR_ARCH_SEND_INFO(0);

SP_CLEAR_ARCH_SEND_INFO(1);
```

定义三:

SP_CLEAR_ARCH_SEND_INFO(

inst_name varchar,

clear_all int

)

功能说明:

此系统函数在主库上执行有效, 用于清理到指定备库 inst_name 最近 N 次的归档发送信息。

可以通过指定 clear_all 判断是否需要清理总的归档发送信息和最大发送信息。N 值为 V$ARCH_SEND_INFO 中的 RECNT_SEND_CNT 值。

参数说明:

inst_name：备库实例名。该参数可不指定，不指定则和定义二一致。

clear_all：用于设置是否清理总的归档发送信息和最大发送信息。取值 0 或 1，若 clear_all 指定为 0，则不清理总的归档发送信息和最大发送信息；若 clear_all 指定为 1，则清理总的归档发送信息和最大发送信息。

返回值:

无

举例说明:

```
SP_CLEAR_ARCH_SEND_INFO('RT_02',0);

SP_CLEAR_ARCH_SEND_INFO('RT_03', 1);
```

11) SP_CLEAR_RAPPLY_STAT

定义:

SP_CLEAR_RAPPLY_STAT()

功能说明:

此系统函数在备库上执行有效，用于清理此备库最近 N 次的日志重演信息。

N 值为 V$RAPPLY_STAT 中的 RECNT_APPLY_NUM 值。

参数说明:

无

返回值:

无

举例说明:

```
SP_CLEAR_RAPPLY_STAT();
```

12) SP_ADD_TIMELY_ARCH(

定义:

SP_ADD_TIMELY_ARCH(

inst_name varchar

)

功能说明:

OPEN 状态下动态扩展 TIMELY 归档。

参数说明:

inst_name: TIMELY 归档的目标实例名。

返回值:

无

举例说明:

```
SP_ADD_TIMELY_ARCH('DB3');
```

13) SF_MAL_CONFIG

定义:

SF_MAL_CONFIG(

cfg_flag int,

bro_flag int

)

功能说明:

设置 MAL 配置状态。使用前需开启 MAL_INI。

需要进入配置状态后才可以执行 SF_MAL_INST_ADD 和 SF_MAL_CONFIG_APPLY。

参数说明:

cfg_flag: 1，设置配置状态；0 取消配置状态。

bro_flag: 1，多节点广播设置；0 本地设置。若为 1，会广播通知 MAL 配置中的所有实例设置 cfg_flag。若广播失败，返回报错"MAL 配置修改广播给远程节点失败"，但本节点配置状态已修改成功且不会重置。若为 0，仅会修改本地的 MAL 配置状态。

返回值:

无

举例说明:

```
SF_MAL_CONFIG(1,0); --设置本地 MAL 配置状态

SF_MAL_CONFIG(0,0); --取消本地 MAL 配置状态

SF_MAL_CONFIG(1,1); --设置本地 MAL 配置状态并广播
```

```
SF_MAL_CONFIG(0,1); --取消本地 MAL 配置状态并广播
```

14) SF_MAL_INST_ADD

定义:

```
SF_MAL_INST_ADD(
item_name       varchar,
inst_name       varchar,
mal_ip          varchar,
mal_port        int,
mal_inst_ip     varchar,
mal_inst_port   int,
dw_port         int,
link_magic  int,
inst_dw_port    int
)
```

功能说明:

增加 MAL 配置项。需要在配置状态下才可以执行该函数，使用前需开启 MAL_INI。不支持使用该函数增加 dblink 下的 mal 配置。仅适用于数据守护集群的场景下。

参数说明:

item_name：配置项名称。必须配置。

inst_name：实例名。必须配置。

mal_ip：MAL IP 地址。必须配置。

mal_port：MAL 端口。必须配置。

mal_inst_ip：实例 IP 地址。

mal_inst_port：实例端口。

dw_port：守护进程端口，用于和远程守护进程/或者监视器通信。

link_magic：链路魔数。

inst_dw_port：实例和本地守护进程通信的端口。

举例说明：

```
SF_MAL_CONFIG(1,0); --进入 MAL 配置状态
SF_MAL_INST_ADD('MAL_INST3','GRP1_RT_DSC01','192.168.0.143',8338,'192.168.1.133',31431, 3567,0,
4567); --增加一个 MAL 配置
SF_MAL_CONFIG_APPLY(); --将增加的 MAL 配置生效
SF_MAL_CONFIG(0,0); --退出 MAL 配置状态
```

15）SF_MAL_CONFIG_APPLY

定义：

SF_MAL_CONFIG_APPLY()

功能说明：

将未生效的 MAL 配置生效。需要在配置状态下才可以执行该函数，使用前需开启 MAL_INI。

参数说明：

无

举例说明：

```
SF_MAL_INST_ADD('MAL_INST3','GRP1_RT_DSC01','192.168.0.143',8338,'192.168.1.133',31431, 3567,0,
4567); --增加一个 MAL 配置
SF_MAL_CONFIG_APPLY(); --将增加的 MAL 配置生效
SF_MAL_CONFIG(0,0); --退出 MAL 配置状态
```

16）SP_SET_ARCH_SEND_UNTIL_TIME

定义：

```
SP_SET_ARCH_SEND_UNTIL_TIME(
dest                varchar,
until_time   varchar
)
```

功能说明：

设置异步备库重演到指定时间。

参数说明：

dest：异步归档目标库实例名。

until_time：重演到指定时间点。若为空串，则表示取消重演到指定时间点。

举例说明：

```
SP_SET_ARCH_SEND_UNTIL_TIME('EP01', '2020-09-14 10:00:00');
```

取消重演到指定时间点：

```
SP_SET_ARCH_SEND_UNTIL_TIME('EP01', '');
```

17）SF_GET_ARCH_SEND_UNTIL_TIME

定义：

```
VARCHAR
SF_GET_ARCH_SEND_UNTIL_TIME(
dest          varchar
)
```

功能说明：

获取异步备库的重演指定时间。

参数说明:

dest: 异步归档目标库实例名。

返回值:

返回异步备库的重演指定时间。

举例说明:

```
SF_GET_ARCH_SEND_UNTIL_TIME('EP01');
```

18) SP_NOTIFY_ARCH_SEND

定义:

```
SP_NOTIFY_ARCH_SEND(
dest          varchar
)
```

功能说明:

通知源库立即发送归档到指定异步备库。

参数说明:

dest: 异步归档目标库实例名。

举例说明:

```
SP_NOTIFY_ARCH_SEND('EP01');
```

19) SF_GET_ARCHIVE_SIZE

定义:

```
BIGINT
SF_GET_ARCHIVE_SIZE(
```

dsc_seqno int,

type int,

start bigint

)

功能说明:

根据指定的节点号、比较方式和起始位置，计算出该节点上从起始位置开始的剩余归档日志量。

此系统函数主要用于备库归档无效场景下计算主备相差的日志量。在备库归档有效（和主库保持实时同步）的情况下，主库很可能会因为日志包还未归档导致无法计算日志量。

参数说明:

dsc_seqno：指定 DMDSC 节点号。若调用该函数的数据库为单节点库，则该字段需要传入 0。

type：比较方式。取值范围 0 或 1。0：使用全局包序号 G_SEQNO 进行比较；1：使用 LSN 进行比较。

start：起始位置。若 type 输入值为 0，则此字段需要传入起始的 G_SEQNO；若 type 为 1，则该字段需要传入起始 LSN。

返回值:

返回从 start 指定位置开始的剩余归档日志量，单位为字节（Byte）。

举例说明:

查询 0 号节点上从 G_SEQNO=8000 开始的剩余归档日志量:

```
SELECT SF_GET_ARCHIVE_SIZE(0, 0, 8000);
```

查询 1 号节点上从 LSN=40000 开始的剩余归档日志量:

```
SELECT SF_GET_ARCHIVE_SIZE(1, 1, 40000);
```

20) SF_SET_STANDBY_DETACH

定义:

```
SP_SET_STANDBY_DETACH(
dest            varchar,
clear_flag      int
)
```

功能说明:

向数据库中插入或删除一条 DETACH 记录，用于将指定备库 DETACH 出集群或者 ATTACH 回集群。

此过程只允许在主库上执行。若主库是 DSC，则只允许在控制节点上执行。

此过程只允许具有 DBA 权限的用户执行，并且要求系统状态为 OPEN。

参数说明:

dest：需要 DETACH 或 ATTACH 的备库实例名。若备库是 DSC，则指定 DSC 集群内任一节点的实例名即可。在 clear_flag 为 1 的情况下，此参数允许传空。

clear_flag：是插入还是删除指定备库的 DETACH 记录，取值 0 或 1。

0：插入一条指向 dest 指定库的 detach 记录，要求 dest 指定库的归档状态为 INVALID。最多能够插入 8 条 DETACH 记录。

1：删除指向 dest 指定库的 DETACH 记录。若 dest 为空，则删除所有已登记的 DETACH 记录。

返回值:

无

举例说明:

插入一条指向 DM1 备库的 DETACH 记录：

```
SP_SET_STANDBY_DETACH('DM1', 0);
```

删除指向 DM2 备库的 DETACH 记录:

```
SP_SET_STANDBY_DETACH('DM2', 1);
```

删除已登记的所有 DETACH 记录:

```
SP_SET_STANDBY_DETACH('',1);
```

21) SP_CLEAR_PKG_DEPENDS

定义:

SP_CLEAR_PKG_DEPENDS()

功能说明:

适用于 DMDSC 主库的数据守护环境，在备库上执行，用于清理备库上正在等待依赖包的待重演日志包，并截断日志。

该系统过程建议和 SP_SET_ARCH_STATUS 搭配使用，先在主库上执行 SP_SET_ARCH_STATUS 将备库的归档状态置为无效，之后再在备库上执行 SP_CLEAR_PKG_DEPENDS 清理日志包并截断日志，否则可能导致备库后续无法进行故障恢复。

执行该系统过程要求备库处于 OPEN 状态。如果备库是 DMDSC 集群，则只需在备库控制节点上执行该系统过程即可。

参数说明:

无

返回值:

无

举例说明:

```
SP_CLEAR_PKG_DEPENDS();
```

22) SF_MAL_INST_MODIFY

定义一:

```
SF_MAL_INST_MODIFY(
inst_name       varchar(128),
para_name           varchar(256),
para_value      int
)
```

功能说明:

修改 MAL 配置项，目前仅用于将 normal 模式的 DSC 动态扩展为 DSC 主备集群，但 DSC 集群已有的 dmmal.ini 中没有配置 MAL_INST_PORT、MAL_DW_PORT、MAL_INST_DW_PORT 时使用。其他场景暂时不支持，避免用户误操作导致通信异常。

该函数使用时需要注意:

1. 在执行 DSC 动态拓展为 DSC 主备集群的流程中，DSC 各节点的 dmmal.ini 修改完成之前，不允许启动守护进程，避免守护进程使用到配置过程中的信息导致异常。

2. 此系统函数仅支持将 MAL_INST_PORT、MAL_DW_PORT、MAL_INST_DW_PORT 从 0 值修改为非 0 值，若配置错误，只能手动修改 dmmal.ini 并重启集群，因此操作前需要确认参数值正确、并且端口号没有被占用。

3. 修改前需要保证 MAL_INI 已开启。

4. 在修改 MAL_INST_PORT 时，指定的 MAL_INST_PORT 需要和每个节点各自 dm.ini 中的 PORT_NUM 保持一致，若指定的 inst_name 为当前连接的节点实例，则在不一致时会进行检查报错。

参数说明:

inst_name：实例名。必须配置。

para_name：待修改的参数名。必须配置。

para_value：待修改的参数值。必须配置。

举例说明:

```
--为 DSC01 配置 MAL_INST_PORT
SP_MAL_INST_MODIFY('DSC01', 'MAL_INST_PORT, 9001);
```

定义二:

SF_MAL_INST_MODIFY(
inst_name varchar(128),
para_name varchar(256),
para_value varchar(128)
)

功能说明:

修改 MAL 配置项，目前仅用于将 normal 模式的 DSC 动态扩展为 DSC 主备集群，但 DSC 集群已有的 dmmal.ini 中没有配置 MAL_INST_HOST 时使用。其他场景暂时不支持，避免用户误操作导致通信异常。

该函数使用时需要注意:

1. 在执行 DSC 动态扩展为 DSC 主备集群的流程中，DSC 各节点的 dmmal.ini 修改完成之前，不允许启动守护进程，避免守护进程使用到配置过程中的信息导致异常。

2. 此系统函数仅支持将 MAL_INST_HOST 从空值修改为非空值，若配置错误，只能手动修改 dmmal.ini 并重启集群，因此操作前需要确认参数值正确。

3. 修改前需要保证 MAL_INI 已开启。

参数说明:

inst_name: 实例名。必须配置。

para_name: 待修改的参数名。必须配置。

para_value: 待修改的参数值。必须配置。

举例说明:

```
--为 DSC01 配置 MAL_INST_HOST
```

```
SP_MAL_INST_MODIFY('DSC01', 'MAL_INST_HOST', '192.168.1.68');
```

7. DMMPP 管理

1) SP_SET_SESSION_MPP_SELECT_LOCAL

定义:

SP_SET_SESSION_MPP_SELECT_LOCAL (

local_flag int

)

功能说明:

MPP 系统下设置当前会话是否只查询本节点数据。如果不设置，表示可以查询全部节点数据。

参数说明:

local_flag: 设置标记。1 代表只查询本节点数据；0 代表查询全部节点数据。

返回值:

无

举例说明:

```
SP_SET_SESSION_MPP_SELECT_LOCAL(1);
```

2) SF_GET_SESSION_MPP_SELECT_LOCAL

定义:

INT

SF_GET_SESSION_MPP_SELECT_LOCAL ()

功能说明:

查询 MPP 系统下当前会话是否只查询本节点数据。

参数说明:

无

返回值:

1 代表只查询本节点数据，0 查询全部节点数据。

举例说明:

```
SELECT SF_GET_SESSION_MPP_SELECT_LOCAL();
```

3） SP_SET_SESSION_LOCAL_TYPE

定义:

SP_SET_SESSION_LOCAL_TYPE (

ddl_flag int

)

功能说明:

MPP 下本地登录时，设置本会话上是否允许 DDL 操作。本地登录默认不允许 DDL 操作。

参数说明:

ddl_flag：为 1 时表示允许当前本地会话执行 DDL 操作，为 0 时则不允许。

返回值:

无

举例说明:

MPP 下本地登录会话:

```
SP_SET_SESSION_LOCAL_TYPE (1);
CREATE TABLE TEST(C1 INT);
SP_SET_SESSION_LOCAL_TYPE (0);
```

4） SF_GET_SELF_EP_SEQNO

定义:

INT

SF_GET_SELF_EP_SEQNO ()

功能说明:

获取本会话连接的 EP 站点序号。

参数说明:

无

返回值:

获取本会话连接的 EP 站点序号。

举例说明:

```
SELECT SF_GET_SELF_EP_SEQNO();
```

5） SP_GET_EP_COUNT

定义:

INT

SP_GET_EP_COUNT (

sch_name varchar(128),

tab_name varchar(128)

);

功能说明:

统计 MPP 环境下表在各个站点的数据行数。

参数说明:

sch_name：表所在模式名。

tab_name：表名。

举例说明：

```
SP_GET_EP_COUNT('SYSDBA','T');
```

6） SF_MPP_INST_ADD

定义：

SF_MPP_INST_ADD(

item_name　　varchar,

inst_name　　varchar

)

功能说明：

增加 MPP 实例配置。

参数说明：

item_name：配置项名称。

inst_name：实例名。

举例说明：

```
SF_MPP_INST_ADD(' SERVICE_NAME3', 'EP03');
```

7） SF_MPP_INST_REMOVE

定义：

SF_MPP_INST_REMOVE(

inst_name　　varchar

)

功能说明：

删除 MPP 实例。

参数说明:

inst_name：实例名。

举例说明:

```
SF_MPP_INST_REMOVE('EP03');
```

8. 日志与检查点管理

1) CHECKPOINT

定义:

INT

CHECKPOINT (

rate int

)

功能说明:

设置检查点。需要注意的是：在 DSC 环境下，OK 节点个数大于 1 时，MOUNT 状态下调用该函数不会生效。

参数说明:

rate：刷脏页百分比，取值范围：1~100。

返回值:

检查点是否成功，0 表示成功，非 0 表示失败。

举例说明:

设置刷脏页百分比为 30%的检查点:

```
SELECT CHECKPOINT(30);
```

2) SF_ARCHIVELOG_DELETE_BEFORE_TIME

定义:

```
INT
SF_ARCHIVELOG_DELETE_BEFORE_TIME (
time    datetime
)
或
INT
SF_ARCHIVELOG_DELETE_BEFORE_TIME (
time    datetime,
dest_name   varchar(128)
)
```

功能说明:

数据库以归档模式打开的情况下，删除指定时间之前的本地归档日志文件。待删除的文件必须处于未被使用状态。在 DSC 集群中，本地归档会共享为其他节点的远程节点，连接任一节点执行 SF_ARCHIVELOG_DELETE_BEFORE_TIME，会删除该节点本地归档及远程归档中指定时间之前的归档文件, 即删除 DSC 集群中所有节点在指定时间之前的本地归档。在 DPC 环境下，可删除全部节点的本地归档日志文件，也可通过指定 dest_name 参数删除某个 RAFT 组的归档日志。

参数说明:

time：指定删除的最大关闭时间，若大于当前使用归档日志文件的创建时间，则从当前使用归档文件之前的归档日志文件开始删除。

dest_name：DPC 环境下指定某个 RAFT 组。

返回值:

删除归档日志文件数，-1 表示出错。在 DPC 环境下的返回值为某一个节点的删除个数，并不会返回所有节点删除归档日志文件数的总和。

举例说明:

删除三天之前的归档日志:

```
SELECT SF_ARCHIVELOG_DELETE_BEFORE_TIME(SYSDATE - 3);
```

3) SF_ARCHIVELOG_DELETE_BEFORE_LSN

定义:

INT

SF_ARCHIVELOG_DELETE_BEFORE_LSN (

lsn bigint

)

或

INT

SF_ARCHIVELOG_DELETE_BEFORE_LSN (

lsn bigint ,

dest_name varchar(128)

)

功能说明:

数据库以归档模式打开的情况下，删除小于指定 LSN 值的本地归档日志文件。待删除的文件必须处于未被使用状态。DPC 环境下，既可以删除全部节点中符合条件的归档日志文件，也可以通过设置 dest_name 删除指定 RAFT 组的归档日志文件。

参数说明:

lsn：指定删除的最大 LSN 值文件，若指定 lsn 值大于当前正在使用归档日志的起始 LSN（arch_lsn），则从当前使用归档文件之前的文件开始删除。

dest_name：RAFT 组名，DPC 环境下可指定。

返回值：

删除归档日志文件数，-1 表示出错。在 DPC 环境下的返回值为某一个节点的删除个数，并不会返回所有节点删除归档日志文件数的总和。

举例说明：

删除 LSN 值小于 95560 的归档日志文件：

```
SELECT SF_ARCHIVELOG_DELETE_BEFORE_LSN(95560);
```

4）SP_ELOG_FILE_DELETE

定义：

```
SP_ELOG_FILE_DELETE (
    [SITE_ID INT,]
    FILE_NAME VARCHAR(128)
)
```

功能说明：

删除当前实例生成的 ELOG 日志文件。将删除日志文件的功能给角色 SYSDBA。

参数说明：

SITE_ID：要删除的 ELOG 日志文件所在的服务器站点号，为空时删除本地的 ELOG 日志文件，并且为空时支持直连多副本系统中的备库或影子库执行。

FILE_NAME：要删除的 ELOG 日志文件名，包含文件后缀，不包含文件路径。

返回值：

无。

举例说明:

删除文件名为"dm_DMSEVER_202209.log"的本地日志文件:

```
SP_ELOG_FILE_DELETE('DM_DMSEVER_202209.LOG');
```

5) SP_REFRESH_SVR_LOG_CONFIG

定义:

SP_REFRESH_SVR_LOG_CONFIG(

)

功能说明:

重新加载 SQL 日志模块，使 SQLLOG.INI 中新修改的值生效。

参数说明:

无。

返回值:

无。

举例说明:

```
SP_REFRESH_SVR_LOG_CONFIG();
```

9. 统计信息

达梦数据库的统计数据对象分三种：表统计信息、列统计信息和索引统计信息。使用表、索引统计信息相关的过程需要具有表查询权限；使用列统计信息相关的过程需要具有列查询权限。

以下对象不支持统计信息：1. 外部表、DBLINK 远程表、动态视图、记录类型数组所用的临时表；2. 所在表空间为 OFFLINE 的对象；3. 位图索引，位图连接索引、虚索引、无效的索引、全文索引；4. BLOB、IMAGE、LONGVARBINARY、CLOB、TEXT、LONGVARCHAR

等列类型；5.ROWID，ROWNUM 等特殊列。

1） SP_TAB_INDEX_STAT_INIT*

定义：

SP_TAB_INDEX_STAT_INIT (

schname varchar(128),

tablename varchar(128)

)

功能说明：

对表上所有的索引生成统计信息。

参数说明：

schname：模式名。

tablename：表名。

举例说明：

对 SYSOBJECTS 表上所有的索引生成统计信息：

```
SP_TAB_INDEX_STAT_INIT ('SYS', 'SYSOBJECTS');
```

2） SP_TAB_INDEX_STAT_INIT*

定义：

SP_TAB_INDEX_STAT_INIT (

schname varchar(128),

tablename varchar(128),

estimate_percent double

)

功能说明:

对表上所有的索引生成统计信息。

参数说明:

schname：模式名。

tablename：表名。

estimate_percent: 收集的百分比，范围为 0~100，低于 0.000001 时按 0.000001 计算。默认系统自定。

举例说明:

对 SYSOBJECTS 表上所有的索引按照 50%的采样比例生成统计信息:

```
SP_TAB_INDEX_STAT_INIT ('SYS', 'SYSOBJECTS', 50);
```

3) SP_DB_STAT_INIT*

定义:

SP_DB_STAT_INIT (

)

功能说明:

对库上所有模式下的所有用户表以及表上的所有索引生成统计信息。

举例说明:

对库上所有模式下的所有用户表以及表上的所有索引生成统计信息:

```
SP_DB_STAT_INIT ();
```

4) SP_DB_STAT_INIT*

定义:

SP_DB_STAT_INIT (

estimate_percent double,

flag int default 1

)

功能说明:

对库上所有模式下的所有用户表以及表上的所有索引生成统计信息。

参数说明:

estimate_percent: 收集的百分比，范围为 0~100，低于 0.000001 时按 0.000001 计算。默认系统自定。

flag: 是否收集列统计信息的标记，flag=0 表示不收集列的统计信息，flag= 1 表示收集列的统计信息。默认收集列的统计信息。

举例说明:

例 1 对库上所有模式下的所有用户表、表上的所有索引、表上所有列按照 50%的采样百分比生成统计信息:

```
SP_DB_STAT_INIT (50, 1);
```

例 2 对库上所有模式下的所有用户表、表上的所有索引、表上所有列按照系统自定义的采样百分比生成统计信息:

```
SP_DB_STAT_INIT (NULL, 1);
```

例 3 对库上所有模式下的所有用户表、表上的所有索引按照系统 50%的采样百分比生成统计信息:

```
SP_DB_STAT_INIT (50, 0);
```

5) SP_INDEX_STAT_INIT*

定义:

SP_INDEX_STAT_INIT (

schname varchar(128),

indexname varchar(128)

)

功能说明:

对指定的索引生成统计信息。

参数说明:

schname: 模式名。

indexname: 索引名。

举例说明:

对指定的索引 IND 生成统计信息:

```
SP_INDEX_STAT_INIT ('SYSDBA', 'IND');
```

6) SP_COL_STAT_INIT*

定义:

SP_COL_STAT_INIT (

schname varchar(128),

tablename varchar(128),

colname varchar(128)

)

功能说明:

对指定的列生成统计信息，不支持大字段列和虚拟列。

参数说明:

schname: 模式名。

tablename: 表名。

colname：列名。

举例说明：

对表 SYSOBJECTS 的 ID 列生成统计信息：

```
SP_COL_STAT_INIT ('SYS', 'SYSOBJECTS', 'ID');
```

7） SP_TAB_COL_STAT_INIT*

定义：

SP_TAB_COL_STAT_INIT (

schname varchar(128),

tablename varchar(128)

)

功能说明：

对某个表上所有的列生成统计信息。

参数说明：

schname：模式名。

tablename：表名。

举例说明：

对'SYSOBJECTS'表上所有的列生成统计信息：

```
SP_TAB_COL_STAT_INIT ('SYS', 'SYSOBJECTS');
```

8） SP_STAT_ON_TABLE_COLS

定义：

SP_STAT_ON_TABLE_COLS(

schname varchar(128),

tablename varchar(128),

percent int

)

功能说明:

对某个表上所有的列，按照指定的采样率生成统计信息。

参数说明:

schname：模式名。

tablename：表名。

percent：采样率（0-100]。

举例说明:

对'SYSOBJECTS'表上所有的列生成统计信息，采样率 90:

```
SP_STAT_ON_TABLE_COLS ('SYS','SYSOBJECTS',90);
```

9) SP_TAB_STAT_INIT*

定义:

SP_TAB_STAT_INIT (

schname varchar(128),

tablename varchar(128)

)

功能说明:

对某张表生成统计信息。

参数说明:

schname：模式名。

tablename：表名。

举例说明:

对表 SYSOBJECTS 生成统计信息:

```
SP_TAB_STAT_INIT ('SYS', 'SYSOBJECTS');
```

10) SP_SQL_STAT_INIT*

定义:

SP_SQL_STAT_INIT (

sql varchar(32767)

)

功能说明:

对某个 SQL 查询语句中涉及的所有表和过滤条件中的列(不包括大字段、ROWID)生成统计信息。

可能返回的错误提示:

1) 语法分析出错，sql 语句语法错误;

2) 对象不支持统计信息，统计的表或者列不存在，或者不允许被统计。

参数说明:

sql: sql 语句。

举例说明:

对'SELECT * FROM SYSOBJECTS'语句涉及的所有表生成统计信息:

```
SP_SQL_STAT_INIT ('SELECT * FROM SYSOBJECTS');
```

11) SP_INDEX_STAT_DEINIT*

定义:

SP_INDEX_STAT_DEINIT (

schname varchar(128),

indexname varchar(128)

)

功能说明:

清空指定索引的统计信息。

参数说明:

schname: 模式名。

indexname: 索引名。

举例说明:

清空索引 IND 的统计信息:

```
SP_INDEX_STAT_DEINIT ('SYSDBA', 'IND');
```

12) SP_COL_STAT_DEINIT*

定义:

SP_COL_STAT_DEINIT (

schname varchar(128),

tabname varchar(128),

colname varchar(128)

)

功能说明:

删除指定列的统计信息。

参数说明:

schname: 模式名。

tabname: 表名。

colname：列名。

举例说明：

删除 SYSOBJECTS 的 ID 列的统计信息：

```
SP_COL_STAT_DEINIT ('SYS', 'SYSOBJECTS', 'ID');
```

13）SP_TAB_COL_STAT_DEINIT*

定义：

```
SP_TAB_COL_STAT_DEINIT (
schname      varchar(128),
tablename    varchar(128)
)
```

功能说明：

删除表和表上所有列的统计信息。

参数说明：

schname：模式名。

tablename：表名。

举例说明：

删除 SYSOBJECTS 表和表上所有列的统计信息：

```
SP_TAB_COL_STAT_DEINIT ('SYS', 'SYSOBJECTS');
```

14）SP_TAB_STAT_DEINIT*

定义：

```
SP_TAB_STAT_DEINIT (
schname      varchar(128),
```

tablename varchar(128)

)

功能说明:

删除某张表的统计信息。

参数说明:

schname: 模式名。

tablename: 表名。

举例说明:

删除表 SYSOBECTS 的统计信息:

```
SP_TAB_STAT_DEINIT ('SYS', 'SYSOBJECTS');
```

15) ET

定义:

ET(

id_in bigint

)

功能说明:

统计执行 ID 为 ID_IN 的所有操作符的执行时间。需设置 INI 参数 ENABLE_MONITOR=1 和 MONITOR_SQL_EXEC=1。

参数说明:

id_in: SQL 语句的执行 ID。

举例说明:

```
select count(*) from sysobjects where name='SYSDBA';
行号        COUNT(*)
```

```
---------- --------------------

1              2



已用时间: 14.641(毫秒). 执行号:26.
```

可以得到执行号为 26。

```
et(26);
```

执行结果如下:

```
OP      TIME(US)                PERCENT RANK                    SEQ           N_ENTER

MEM_USED(KB)

----- ----------------- ------- ---------------- ---------- -----------

--------------------

DISK_USED(KB)        HASH_USED_CELLS        HASH_CONFLICT        DHASH3_USED_CELLS

DHASH3_CONFLICT HASH_SAME_VALUE

----------------- ------------------ ----------------- -----------------

--------------- --------------------

PRJT2 5                      2.30%    4                    2             4             0

0                       0                       0                    NULL

NULL                    0



AAGR2 33                     15.21%   3                    3             4             0

0                       0                       0                    NULL

NULL                    0



NSET2 59                     27.19%   2                    1             3             0
```

```
0                    0                    0                    NULL
NULL                 0

SSEK2 120            55.30%  1            4            2            0
0                    0                    0                    NULL
NULL                 0
```

结果中每一行对应一个操作符，每一列对应的解释如下：

列名	含义
OP	操作符名称
TIME(US)	执行耗时，单位：微秒
PERCENT	在整个计划中用时占比
RANK	执行耗时排名，由高到低正序排序
SEQ	计划中的序号
N_ENTER	操作符进入的次数
MEM_USED(KB)	操作符使用的内存空间，单位：KB
DISK_USED(KB)	操作符使用的磁盘空间，单位：KB
HASH_USED_CELLS	哈希表使用的槽数
HASH_CONFLICT	哈希表存在冲突的记录数
DHASH3_USED_CELLS	动态哈希表中使用的槽数
DHASH3_CONFLICT	动态哈希表中的冲突情况
HASH_SAME_VALUE	开启哈希相同值挂链优化时，记录哈希表中相同值的总数，唯一值不纳入计数。 例如：哈希表中存在一个 0 和两个相同值链 1—1—1、3—3，此时 HASH_SAME_VALUE=5

如果某些操作符并没有真正执行或者耗时很短，则不会在 ET 中显示。另外，结果中出现了 EXPLAIN 计划中没有的 DLCK 操作符，它是用来对字典对象的上锁处理，通常情况下可以忽略。

16）SP_UPDATE_SYSSTATS

定义:

SP_UPDATE_SYSSTATS（

flag　　int

）

功能说明:

用以升级及迁移统计信息。7.1.5.173 及之后的版本支持该功能。

参数说明:

flag：标记，取值 0、1、2、3、4、99。 0：回退版本；1：创建统计信息升级辅助表；2：升级 SYS.SYSSTATS 表结构； 3：导入 SYS.SYSSTATS 表数据； 4：删除升级辅助表；参数 99：理想状态下使用，集合了参数 1、2、3、4 的功能。

举例说明:

如果想要将老库升级，需要手动升级，升级步骤如下:

```
SP_UPDATE_SYSSTATS(99);
或者
SP_UPDATE_SYSSTATS(1);
SP_UPDATE_SYSSTATS(2);
SP_UPDATE_SYSSTATS(3);
SP_UPDATE_SYSSTATS(4);
```

如果想使用老库，且使用了之前的加列版本服务器且升级失败，那么需要先回退版本，

再升级：

```
SP_UPDATE_SYSSTATS(0);

SP_UPDATE_SYSSTATS(99);
```

17) SP_TAB_MSTAT_DEINIT

定义：

```
SP_TAB_MSTAT_DEINIT(
schname      varchar(128),
tablename    varchar(128)
)
```

功能说明：

删除一个表的多维统计信息。

参数说明：

schname：模式名。

tablename：表名。

举例说明：

删除表 SYSDBA.L1 上所有的多维统计信息：

```
SP_TAB_MSTAT_DEINIT('SYSDBA','L1');
```

18) SF_GET_MD_COL_STR

定义：

```
VARCHAR
SF_GET_MD_COL_STR(
tabid        int,
```

col_info varbinary(4096),

col_num int

)

功能说明:

获取表上建多维统计信息的列信息。

参数说明:

tabid: 表 id。

col_info: 多维的维度信息。

col_num: 维度。

返回值:

多维统计信息的列信息。

举例说明:

获取表 L1 上建的多维统计信息的列信息:

```
SELECT SF_GET_MD_COL_STR(ID, MCOLID, N_DIMENSION) FROM SYS.SYSMSTATS WHERE ID IN (SELECT ID FROM SYSOBJECTS WHERE NAME='L1');
```

19) SP_CREATE_AUTO_STAT_TRIGGER

定义:

SP_CREATE_AUTO_STAT_TRIGGER(

type int,

freq_interval int,

freq_sub_interval int,

freq_minute_interval int,

starttime varchar(128),

during_start_date varchar(128),

max_run_duration int,

enable int

)

功能说明:

INI 参数 AUTO_STAT_OBJ 开启时，启用自动收集统计信息功能。

参数说明:

type：指定调度类型。可取值 1、2、3、4、5、6、7、8，缺省为 1，不同取值意义分别介绍如下:

1：按天的频率来执行;

2：按周的频率来执行;

3：在一个月的某一天执行;

4：在一个月的第一周第几天执行;

5：在一个月的第二周的第几天执行;

6：在一个月的第三周的第几天执行;

7：在一个月的第四周的第几天执行;

8：在一个月的最后一周的第几天执行。

freq_interval：与 type 有关，表示不同调度类型下的发生频率，默认为 1。具体说明如下:

当 type=1 时，表示每几天执行，取值范围为 1~100;

当 type=2 时，表示每几个星期执行，取值范围为 1~100;

当 type=3 时，表示每几个月中的某一天执行，取值范围为 1~100;

当 type=4 时，表示每几个月的第一周执行，取值范围为 1~100;

当 type=5 时，表示每几个月的第二周执行，取值范围为 1~100;

当 type=6 时，表示每几个月的第三周执行，取值范围为 1~100；

当 type=7 时，表示每几个月的第四周执行，取值范围为 1~100；

当 type=8 时，表示每几个月的最后一周执行，取值范围为 1~100。

freq_sub_interval：与 type 和 freq_interval 有关，表示不同 type 的执行频率，在 freq_interval 基础上，继续指定更为精准的频率，缺省为 1。具体说明如下：

当 type=1 时，这个值无效，系统不做检查；

当 type=2 时，表示某一个星期的星期几执行，可以同时选中七天中的任意几天，取值范围 1~127。具体可参考如下规则：因为每周有七天，所以 DM 数据库系统内部用七位二进制来表示选中的日子，从最低位开始算起，依次表示周日、周一到周五、周六。选中周几，就将该位置 1，否则置 0。例如，选中周二和周六，7 位二进制就是 1000100，转化成十进制就是 68，所以 FREQ_SUB_INTERVAL 取值 68；

当 type=3 时，表示将在一个月的第几天执行，取值范围 1~31；

当 type 为 4、5、6、7 或 8 时，都表示将在某一周内第几天执行，取值范围 1~7，分别表示从周一到周日。

freq_minute_interval：开始时间后，当天每隔几分钟再次执行，取值范围为 1~1439，缺省为 1439。

starttime：开始时间，缺省为 22:00。

during_start_date：有效日期时间段的开始日期时间，只有当前时间大于该参数值时，该定时器才有效，缺省为 1900/1/1。

max_run_duration：收集统计信息触发器最大执行时间，单位秒，0 表示不限制，缺省为 0。

enable：定时器的操作。0：使触发器无效；1：使触发器有效；2：删除触发器。缺省为 1。

举例说明：

示例请参考《DM8 系统管理员手册》的 22.5 统计信息章节。

20) SYSDBA.GET_AUTO_STAT_INFO_FUNC

定义：

```
cREATE OR REPLACE PROCEDURE  SYSDBA.GET_AUTO_STAT_INFO_FUNC(
task_id INT,
total_stat INT,
table_id INT,
sch_name varchar(24),
table_name varchar(24),
curr_gath_tab_id INT,
curr_gath_sch_name varchar(24),
curr_gath_tab_name varchar(24),
success_stat INT,
fail_stat INT,
task_start_time DATETIME,
task_end_time DATETIME,
gather_tbl_start_time DATETIME,
gather_tbl_end_time DATETIME
) as
BEGIN
/*用户自定义如何使用统计信息的代码*/
END;
/
```

功能说明:

支持对自动收集统计信息的过程进行监控。

通过创建确定的过程 SYSDBA.GET_AUTO_STAT_INFO_FUNC，用以接收服务器在自动收集统计信息时的相关信息。该过程的模块体为开放式，用户可以根据自己的需求来编写如何使用统计信息的代码。

参数说明:

task_id: 任务 id，同一个任务的 task_id 相同。

total_stat: 一次任务需要收集的表的总个数。

table_id: 收集完成的一个表 table 的 id，每收集完一个表的统计信息，就会调用一次 SYSDBA.GET_AUTO_STAT_INFO_FUNC，传出一次数据，用户可自定义该过程，自定义处理接收到的数据。

sch_name: 对应 table_id 的模式名。

table_name: 对应 table_id 的表名。

curr_gath_tab_id: 当前正在收集的表 id。收集完一个表 table 后，传出数据时，该字段为接下来要收集的表 id，也是服务器当前正要或正在收集的表。

curr_gath_sch_name: 对应 curr_gath_tab_id 的模式名。

curr_gath_tab_name: 对应 curr_gath_tab_id 的表名。

success_stat: 截止到目前这次收集任务一共成功收集了多少张表。

fail_stat: 截止到目前这次收集任务一共失败收集了多少张表。

task_start_time: 这次任务的开始时间。

task_end_time: 这次任务的结束时间，未结束为 NULL。

gather_tbl_start_time: 收集完一个表 table 时，该表收集的开始时间。

gather_tbl_end_time: 收集完一个表 table 时，该表收集的结束时间，收集失败，则结束时间为 NULL。

举例说明:

示例请参考《DM8 系统管理员手册》的 22.5 统计信息章节。

21) SP_FLUSH_MODIFICATIONS_INFO

定义:

SP_FLUSH_MODIFICATIONS_INFO ()

功能说明:

更新系统表 SYSMODIFICATIONS 中的数据，将内存中表对象的监控信息记录到系统表 SYSMODIFICATIONS 中。

举例说明:

将内存中表对象的监控信息记录到系统表 SYSMODIFICATIONS 中。

```
SP_FLUSH_MODIFICATIONS_INFO();
```

22) SP_CLEAN_MODIFICATIONS

定义:

SP_CLEAN_MODIFICATIONS ()

功能说明:

清理系统表 SYSMODIFICATIONS 中的冗余数据，将已经不存在的对象从该系统表中清除。

举例说明:

清除系统表 SYSMODIFICATIONS 中已经不存在的对象。

```
SP_CLEAN_MODIFICATIONS();
```

10. 资源监测

1） SP_CHECK_IDLE_MEM

定义:

SP_CHECK_IDLE_MEM ()

功能说明:

对可用内存空间进行检测，并在低于阈值（对应 INI 参数 IDLE_MEM_THRESHOLD）的情况下打印报警记录到日志，同时报内存不足的异常。

举例说明:

监测当前系统的内存空间是否低于阈值:

```
SP_CHECK_IDLE_MEM ();
```

2） SP_CHECK_IDLE_DISK

定义:

SP_CHECK_IDLE_DISK (

path varchar(256)

)

功能说明:

对指定位置的磁盘空间进行检测，并在低于阈值（对应 INI 参数 IDLE_DISK_THRESHOLD）的情况下打印报警记录到日志，同时报磁盘空间不足的异常。

参数说明:

path：监测的路径。

举例说明:

监测 d:\data 路径下的磁盘空间是否低于阈值:

```
SP_CHECK_IDLE_DISK ('d:\data');
```

3) SF_GET_CMD_RESPONSE_TIME

定义:

SF_GET_CMD_RESPONSE_TIME()

功能说明:

查看 DM 服务器对用户命令的平均响应时间。

返回值:

命令的平均响应时间，单位秒。

举例说明:

在 DM.INI 中 ENABLE_MONITOR=1 的前提下执行:

```
SELECT SYS.SF_GET_CMD_RESPONSE_TIME();
```

4) SF_GET_TRX_RESPONSE_TIME

定义:

SF_GET_TRX_RESPONSE_TIME()

功能说明:

查看事务的平均响应时间。

返回值:

事务的平均响应时间，单位秒。

举例说明:

在 DM.INI 中 ENABLE_MONITOR=1 的前提下执行:

```
SELECT SYS.SF_GET_TRX_RESPONSE_TIME();
```

5) SF_GET_DATABASE_TIME_PER_SEC

定义:

SF_GET_DATABASE_TIME_PER_SEC()

功能说明:

查看数据库中用户态时间占总处理时间的比值。

返回值:

用户态时间占总处理时间的比值，该比值越大表明用于 IO、事务等待等耗费的时间越少。

举例说明:

在 DM.INI 中 ENABLE_MONITOR=1 的前提下执行:

```
SELECT SYS.SF_GET_DATABASE_TIME_PER_SEC();
```

6) TABLE_USED_SPACE

定义:

BIGINT

TABLE_USED_SPACE (

schname varchar(256);

tabname varchar(256)

)

功能说明:

获取指定表所占用的页数。

参数说明:

schname：模式名，必须大写。

tabname：表名，必须大写。

返回值:

表所占用的页数。

举例说明:

查看 SYSOJBECTS 的所占用的页数:

```
SELECT  TABLE_USED_SPACE ('SYS','SYSOBJECTS');
```

查询结果如下:

```
32
```

7) HUGE_TABLE_USED_SPACE

定义:

BIGINT

HUGE_TABLE_USED_SPACE (

schname varchar(256);

tabname varchar(256)

)

功能说明:

获取指定 HUGE 表所占用的大小。

参数说明:

schname: 模式名，必须大写。

tabname: huge 表名，必须大写。

返回值:

HUGE 表所占用的大小，单位 MB。

举例说明:

查看 huge 表 test 所占用的大小:

```
CREATE  HUGE TABLE TEST(A INT);

INSERT INTO TEST SELECT LEVEL FROM DUAL CONNECT BY LEVEL <=5000000;

SELECT  HUGE_TABLE_USED_SPACE ('SYSDBA','TEST');
```

查询结果如下：

```
20
```

8） USER_USED_SPACE

定义：

BIGINT

USER_USED_SPACE (

username varchar(256)

)

功能说明：

获取指定用户所占用的页数，不包括用户占用的 HUGE 表页数。

参数说明：

username：用户名。

返回值：

用户所占用的页数。

举例说明：

查看 SYSDBA 的所占用的页数：

```
SELECT  USER_USED_SPACE ('SYSDBA');
```

查询结果如下：

```
64
```

9） TS_USED_SPACE

定义:

BIGINT

TS_USED_SPACE (

tsname varchar(256)

)

功能说明:

获取指定表空间所有文件所占用的页数之和。RLOG 和 ROLL 表空间不支持。

参数说明:

tsname: 表空间名。

返回值:

表空间占用的页数。

举例说明:

查看 MAIN 表空间所有文件占用的页数之和:

```
SELECT TS_USED_SPACE ('MAIN');
```

查询结果如下:

```
16384
```

10) DB_USED_SPACE

定义:

BIGINT

DB_USED_SPACE ()

功能说明:

获取整个数据库占用的页数。

返回值:

整个数据库占用的页数。

举例说明:

查看数据库所占用的页数:

```
SELECT  DB_USED_SPACE ();
```

查询结果如下:

```
19712
```

11) INDEX_USED_SPACE

定义 1:

BIGINT

INDEX_USED_SPACE (

indexid int

)

功能说明:

根据索引 ID，获取指定索引所占用的页数。

参数说明:

indexid: 索引 ID。

返回值:

索引占用的页数。

举例说明:

查看索引号为 33554540 的索引所占用的页数:

```
SELECT  INDEX_USED_SPACE (33554540);
```

查询结果如下:

```
32
```

定义 2:

BIGINT

INDEX_USED_SPACE (

schname varchar(256),

indexname varchar(256)

)

功能说明:

根据索引名获取指定索引占用的页数。

参数说明:

schname: 模式名。

indexname: 索引名。

返回值:

索引占用的页数。

举例说明:

查看 SYSDBA 模式下索引名为 INDEX_TEST 的索引占用的页数:

```
SELECT INDEX_USED_SPACE ('SYSDBA', 'INDEX_TEST');
```

查询结果如下:

```
14
```

定义 3:

BIGINT

INDEX_USED_SPACE (

schname varchar(256),

tablename varchar(256),

nth_index int

)

功能说明:

获取表指定序号的索引占用的页数。

参数说明:

schname：模式名。

tablename：表名。

nth_index：要获取表的第几个索引。

返回值:

索引占用的页数。

举例说明:

查看 SYSDBA 模式下，TEST 表的第二个索引占用的页数:

```
SELECT  INDEX_USED_SPACE ('SYSDBA', 'TEST', 2);
```

查询结果如下:

```
14
```

12）INDEX_USED_PAGES

定义 1:

BIGINT

INDEX_USED_PAGES (

indexid int

)

功能说明:

根据索引 id，获取指定索引已使用的页数，为估算值。

参数说明:

indexid：索引 ID。

返回值：

索引已使用的页数。

举例说明：

查看索引号为 33554540 的索引已使用的页数：

```
SELECT  INDEX_USED_PAGES (33554540);
```

查询结果如下：

```
14
```

定义 2：

BIGINT

INDEX_USED_PAGES (

schema_name varchar(128),

index_name varchar(128)

)

功能说明：

根据模式下的索引名获取指定索引已使用的页数，为估算值。

参数说明：

schema_name：模式名。

index_name：索引名。

返回值：

指定索引已使用的页数。

举例说明：

查看模式 SYSDBA 下索引名为 IDX1 的索引已使用的页数：

```
SELECT INDEX_USED_PAGES('SYSDBA','IDX1');
```

查询结果如下:

```
2
```

定义 3:

BIGINT

INDEX_USED_PAGES (

schema_name varchar(128),

table_name varchar(128),

index_no int

)

功能说明:

根据模式下的指定表的第 index_no 个索引获取该索引使用的页数，为估算值。

参数说明:

schema_name: 模式名。

table_name: 表名。

index_no: 索引序号(从 1 开始)。

返回值:

指定索引已使用的页数。

举例说明:

查看模式 SYSDBA 下表名为 TAB1 的第一个索引已使用的页数:

```
SELECT INDEX_USED_PAGES('SYSDBA','TAB1',1);
```

13) TABLE_USED_PAGES

定义:

BIGINT

TABLE_USED_PAGES (

schname varchar(256);

tabname varchar(256)

)

功能说明:

获取指定表已使用的页数，为估算值。

参数说明:

schname：模式名。

tabname：表名。

返回值:

表已使用的页数。

举例说明:

查看 SYSOJBECTS 已使用的页数:

```
SELECT TABLE_USED_PAGES('SYS','SYSOBJECTS');
```

查询结果如下:

```
14
```

14) TS_FREE_SPACE

定义:

INT

TS_FREE_SPACE (

tsname varchar(256)

)

功能说明:

获取指定表空间可分配的空闲页数。RLOG 和 ROLL 表空间不支持。

参数说明:

tsname：表空间名。

返回值:

表空间可分配的空闲页数，包含 TS_RESERVED_SPACE()的预留页数。

举例说明:

查看 MAIN 表空间可分配的空闲页数:

```
SELECT  TS_FREE_SPACE ('MAIN');
```

查询结果如下:

```
8192
```

15) TS_RESERVED_SPACE

定义:

INT

TS_RESERVED_SPACE (

tsname varchar(256)

)

功能说明:

获取指定表空间系统预留的页数。RLOG 和 ROLL 表空间不支持。系统启动时根据 DM.INI 参数 TS_RESERVED_EXTENTS 预留部分空间，避免 ROLLBACK/PURGE 等操作过程中分配数据页失败，这些预留空间用户无法使用。

参数说明:

tsname：表空间名。

返回值:

表空间系统预留的页数。

举例说明:

查看 MAIN 表空间系统预留的页数:

```
SELECT  TS_RESERVED_SPACE ('MAIN');
```

查询结果如下:

```
512
```

16) TS_FREE_SPACE_CALC

定义:

INT

TS_FREE_SPACE_CALC (

tsname varchar(256)

)

功能说明:

重新计算指定表空间可分配的空闲页数。RLOG 和 ROLL 表空间不支持。

参数说明:

tsname: 表空间名。

返回值:

表空间可分配的空闲页数，包含 TS_RESERVED_SPACE 的预留页数。

举例说明:

重新计算 MAIN 表空间可分配的空闲页数:

```
SELECT  TS_FREE_SPACE_CALC('MAIN');
```

查询结果如下:

```
8192
```

17）TABLE_USED_LOB_PAGES

定义:

BIGINT

TABLE_USED_LOB_PAGES (

schname varchar(256);

tabname varchar(256)

)

功能说明:

获取指定表已使用的 lob 页数，包括大字段列使用的行外数据 lob 页和指定了 USING LONG ROW 存储选项时变长字符串列使用的行外数据 lob 页。返回值为估算值。

参数说明:

schname：模式名。

tabname：表名。

返回值:

表已使用的 lob 页数。

举例说明:

查看 SYS.SYSTEXTS 表已使用的 lob 页数:

```
SELECT TABLE_USED_LOB_PAGES('SYS','SYSTEXTS');
```

查询结果如下:

```
139
```

18）TABLE_FREE_LOB_PAGES

定义:

BIGINT

TABLE_FREE_LOB_PAGES(

schema_name in varchar,

table_name in varchar

)

功能说明:

获取指定表的大字段的段首页登记的空闲页个数。

参数说明:

schema_name: 表所在模式名。

table_name: 表名。

返回值:

失败则报错，成功返回空闲页个数。

举例说明:

查询 T_CLOB 表的大字段的段首页登记的空闲页个数:

```
SELECT TABLE_FREE_LOB_PAGES('SYSDBA','T_CLOB');
```

19) SP_TABLE_LOB_RECLAIM

定义:

SP_TABLE_LOB_RECLAIM(

schema_name in varchar,

table_name in varchar

)

功能说明:

清理指定表的大字段的段首页登记的空闲页。

参数说明:

schema_name：表所在模式名。

table_name：表名。

返回值:

无。失败则报错。

举例说明:

清理表 T_CLOB 的大字段的段首页登记的空闲页:

```
SP_TABLE_LOB_RECLAIM('SYSDBA','T_CLOB');
```

20) SP_SET_SQL_STAT_THRESHOLD

定义:

SP_SET_SQL_STAT_THRESHOLD(

name varchar,

val int64)

功能说明:

设置语句级资源监控，SQL 监控项生成的条件阈值，当资源大于设置的阈值时才生成统计项。可以设置的监控项为 VSQL_STAT/VSQL_STAT_HISTORY 视图中的 5~62、64 和 66 列。该过程只有 DBA 权限才能执行。

参数说明:

name：允许为 NULL。需要设置阈值的监控项名字，可选名字为 VSQL_STAT/VSQL_STAT_HISTORY 视图中第 5~62、64 和 66 列的列名。当 name 为 NULL 时，val 只能设置为 0 或者-1。0 表示无条件生成历史监控项，-1 表示不生成历史监控项。

val：不允许为 NULL 且必须大于等于 0。表示需要设置的阈值。

返回值:

无

举例说明:

```
SP_SET_SQL_STAT_THRESHOLD('INS_IN_PL_CNT', 60000);
```

21) TABLE_ROWCOUNT

定义:

BIGINT

TABLE_ROWCOUNT (

schema_name varchar(128),

table_name varchar(128)

)

功能说明:

获取指定模式下指定表的总行数。

参数说明:

schema_name：模式名。

table_name：表名。

返回值:

指定表的总行数。

举例说明:

查看模式 SYSDBA 下表 TAB1 的总行数:

```
SELECT TABLE_ROWCOUNT('SYSDBA','TAB1');
```

22) TABLE_FREE_PAGE_STACK_USED_SPACE

定义:

BIGINT

TABLE_FREE_PAGE_STACK_USED_SPACE (

schname varchar(256),

tabname varchar(256)

)

功能说明:

获取指定模式下指定表的 B 树空闲页堆栈所占用的页数。

参数说明:

schname：模式名，必须大写。

tabname：表名，必须大写。

返回值:

指定表 B 树空闲页堆栈所占用的页数。

举例说明:

将 dm.ini 中 BTR_FREE_PAGE_STACK 设为 1，开启空闲页堆栈，查看表 test 空闲页堆栈所占用的页数。

```
CREATE TABLE TEST(A INT);

INSERT INTO TEST SELECT LEVEL FROM DUAL CONNECT BY LEVEL <= 10000;

SELECT TABLE_FREE_PAGE_STACK_USED_SPACE('SYSDBA', 'TEST');
```

11. 类型别名

DM 支持用户对各种基础数据类型定义类型别名，定义的别名可以在 SQL 语句和 DMSQL 程序中使用。

1) SP_INIT_DTYPE_SYS*

定义:

SP_INIT_DTYPE_SYS (

create_flag int

)

功能说明:

初始化或清除类型别名运行环境。

参数说明:

create_flag: 1 表示创建，0 表示删除。

返回值:

无

举例说明:

初始化类型别名运行环境:

```
SP_INIT_DTYPE_SYS(1);
```

2) SF_CHECK_DTYPE_SYS

定义:

INT

SF_CHECK_DTYPE_SYS ()

功能说明:

系统类型别名系统的启用状态检测。

返回值:

0: 未启用; 1: 已启用。

举例说明:

获得系统类型别名系统的启用状态:

```
SELECT SF_CHECK_DTYPE_SYS;
```

3) SP_DTYPE_CREATE*

定义:

SP_DTYPE_CREATE (

name varchar(32),

base_name varchar(32),

len int,

scale int

)

功能说明:

创建一个类型别名。最多只能创建 100 个类型别名。仅 DBA 有权限调用此过程。

参数说明:

name: 类型别名的名称。

base_name: 基础数据类型名。

len: 基础数据类型长度，当为固定精度类型时，应该设置为 NULL。

scale: 基础数据类型刻度。当基础数据类型为 CHAR/VARCHAR 时，若指定为 1，则表示字符长度；否则忽略，表示字节长度。

返回值:

无

举例说明:

创建 VARCHAR(100)的类型别名'STR':

```
SP_DTYPE_CREATE('STR', 'VARCHAR', 100, NULL);
```

4) SP_DTYPE_DELETE*

定义:

SP_DTYPE_DELETE (

name varchar(32)

)

功能说明:

删除一个类型别名。仅 DBA 有权限调用此过程。

参数说明:

name：类型别名的名称。

返回值:

无

举例说明:

删除类型别名'STR':

```
SP_DTYPE_DELETE('STR');
```

12. 杂类函数/过程

1) TO_DATETIME

定义:

DATETIME

TO_DATETIME(

year int,

month int,

day int,

hour int,

minute int

)

功能说明:

将 int 类型值组合并转换成日期时间类型。

参数说明:

year：年份，必须介于-4713 和+9999 之间，且不为 0。

month：月份。

day：日。

hour：小时。

minute：分钟。

返回值:

日期时间值。

举例说明:

将整型数 2010，2，2，5，5 转换成日期时间类型:

```
SELECT  TO_DATETIME (2010,2,2,5,5);
```

查询结果如下:

```
2010-02-02 05:05:00.000000
```

2) SP_SET_ROLE*

定义:

SP_SET_ROLE (

role_name varchar(128),

enable int

)

功能说明:

设置角色启用禁用。

参数说明:

role_name: 角色名。

enable: 0/1 (0: 禁用 1: 启用)。

返回值:

无

举例说明:

禁用角色 ROLE1:

```
SP_SET_ROLE ( 'ROLE1', 0);
```

3) SF_CHECK_SYSTEM_VIEWS

定义:

INT

SF_CHECK_SYSTEM_VIEWS()

功能说明:

系统视图的启用状态检测。

返回值:

0: 未启用; 1: 已启用。

举例说明:

获得系统视图的启用状态:

```
SELECT SF_CHECK_SYSTEM_VIEWS;
```

4) SP_DYNAMIC_VIEW_DATA_CLEAR

定义:

SP_DYNAMIC_VIEW_DATA_CLEAR (

view_name varchar(128)

)

功能说明:

清空动态性能视图的历史数据，仅对存放历史记录的动态视图起作用。

参数说明:

view_name：动态性能视图名。

返回值:

无

举例说明:

清空动态性能视图 V$SQL_HISTORY 的历史数据:

```
SP_DYNAMIC_VIEW_DATA_CLEAR('V$SQL_HISTORY');
```

5) SP_INIT_INFOSCH*

定义:

SP_INIT_INFOSCH (

create_flag int

)

功能说明:

创建或删除信息模式。

参数说明:

create_flag：为 1 时表示创建信息模式；为 0 表示删除信息模式。

举例说明:

创建信息模式：

```
SP_INIT_INFOSCH (1);
```

6） SF_CHECK_INFOSCH

定义：

INT

SF_CHECK_INFOSCH ()

功能说明：

系统的信息模式启用状态检测。

返回值：

0：未启用；1：已启用。

举例说明：

获得系统信息模式的启用状态：

```
SELECT SF_CHECK_INFOSCH;
```

7） SP_INIT_CPT_SYS

定义：

SP_INIT_CPT_SYS (

flag int

)

功能说明：

初始化数据捕获环境。

参数说明：

flag： 1 表示初始化环境；0 表示删除环境。

举例说明:

初始化数据捕获环境:

```
SP_INIT_CPT_SYS(1);
```

8) SF_CHECK_CPT_SYS

定义:

INT

SF_CHECK_CPT_SYS ()

功能说明:

系统的数据捕获环境启用状态检测。

返回值:

0: 未启用; 1: 已启用。

举例说明:

获得系统数据捕获环境的启用状态:

```
SELECT SF_CHECK_CPT_SYS;
```

9) SF_SI

定义:

VARCHAR

SF_SI(

INDEX_SQL varchar

)

功能说明:

输入索引创建语句或索引重建语句，查看预计的执行信息。

参数说明:

INDEX_SQL: 指定的索引创建语句或索引重建语句。

返回值:

索引创建或重建的统计信息，排序区大小的建议值。

举例说明:

```
--输入索引创建语句
SELECT SF_SI('CREATE INDEX idx_t1 ON t1(c1,c2);');
--输入索引重建语句
SELECT SF_SI('ALTER INDEX idx_t1 REBUILD ONLINE;');
--索引重建语句支持系统过程
SELECT SF_SI('SP_REBUILD_INDEX("SYSDBA", 33555473);');
```

10) SP_UNLOCK_USER

定义:

SP_UNLOCK_USER(

user_name varchar(128) not null

)

功能说明:

为指定的用户解锁。

参数说明:

user_name: 需要解锁的用户名。

举例说明:

```
CREATE USER USER123 IDENTIFIED BY DMsys_123456;
//错误登录 3 次，导致用户被锁
```

```
SQL>LOGIN

服务名:

用户名:USER123

密码:

SSL 路径:

SSL 密码:

UKEY 名称:

UKEY PIN 码:

MPP 类型:

是否读写分离(y/n):

协议类型:

[-2501]:用户名或密码错误.



//解锁

//SYSDBA 用户登录:

SP_UNLOCK_USER('USER123');
```

11) DUMP 函数

定义:

VARCHAR

DUMP(

exp int/bigint/dec/double/float/varbinary/

varchar/date/time/datetime/time with time zone/timestamp with time

zone/interval year/interval day/blob/clob/rowid,

fmt int,

start int,

len int

)

功能说明:

获得表达式的内部存储字节。

参数说明:

exp：输入参数，必选。可以是任何基本数据类型。

fmt：输出格式进制，可选，缺省为 10。8：以八进制的形式返回结果集；10：以十进制的形式返回结果集；16：以十六进制的形式返回结果集；17：以字符的形式返回结果集；1000+N：以上四种加上 1000，表示在返回值中加上当前字符集名称，N 为 8、10、16 或 17。

start：开始进行返回的字节的位置。可选。

len：需要返回的字节长度。可选。

返回值:

依次是数据类型代码、字节长度、表达式在系统内部存储字节。其中，数据类型代码有：2 代表 VARCHAR、7 代表 INT、8 代表 BIGINT、9 代表 DEC、10 代表 FLOAT、11 代表 DOUBLE、12 代表 BLOB、14 代表 DATE、15 代表 TIME、16 代表 DATETIME、18 代表 VARBINARY、19 代表 CLOB、20 代表 INTERVAL YEAR MONTH、21 代表 INTERVAL DAY TIME、22 代表 TIME WITH ZONE、23 代表 DATE TIME WITH ZONE、28 代表 ROWID。

举例说明:

例 1 查询字符串'an'的存储字节，显示成带字符集名称的字符：

```
select dump('an',1017);
```

查询结果如下：

```
Typ=2 Len=2 CharacterSet=GBK: a,n
```

例 2 查询日期类型'2005-01-01'的默认字节，默认进制显示：

```
select dump(date '2005-01-01' ) ;
```

查询结果如下：

```
Typ=14 Len= 13: 213,7,1,1,0,0,0,0,0,0,232,3,0
```

例 3 查询数字 1234567890 的从第 2 个字节开始的 4 个字节，以 16 进制显示：

```
select dump(1234567890,16,2,4);
```

查询结果如下：

```
Typ=7 Len=4: 2,96,49
```

例 4 查询 clob 类型的默认字节，默认进制显示。

```
CREATE TABLE testdump(c1 clob);
INSERT INTO testdump VALUES('adbca');
INSERT INTO testdump VALUES('1.曹雪芹，是中国文学史上最伟大也是最复杂的作家，《红楼梦》也是中国文学史上最伟大而又最复杂的作品。《红楼梦》写的是封建贵族的青年贾宝玉、林黛玉、薛宝钗之间的恋爱和婚姻悲剧，而且以此为中心，写出了当时具有代表性的贾、王、史、薛四大家族的兴衰，其中又以贾府为中心，揭露了封建社会后期的种种黑暗和罪恶，及其不可克服的内在矛盾，对腐朽的封建统治阶级和行将崩溃的封建制度作了有力的批判，使读者预感到它必然要走向覆灭的命运。本书是一部具有高度思想性和高度艺术性的伟大作品，从本书反映的思想倾向来看，作者具有初步的民主主义思想，他对现实社会包括宫廷及官场的黑暗，封建贵族阶级及其家庭的腐朽，封建的科举制度、婚姻制度、奴婢制度、等级制度，以及与此相适应的社会统治思想即孔孟之道和程朱理学、社会道德观念等等，都进行了深刻的批判并且提出了朦胧的带有初步民主主义性质的理想和主张。这些理想和主张正是当时正在滋长的资本主义经济萌芽因素的曲折反映。2.曹雪芹，是中国文学史上最伟大也是最复杂的作家，《红楼梦》也是中国文学史上最伟大而又最复杂的作品。《红楼梦》写的是封建贵族的青年贾宝玉、林黛玉、薛宝钗之间的恋爱和婚姻悲剧，而且以此为中心，写出了当时具有代表性的贾、王、史、薛四大家族的兴衰，其中又以贾府为中心，揭露了封建社会后期的种种黑暗和罪恶，及其不可克服的内在矛盾，对腐朽的封建统治阶级和行将崩溃的
```

```
封建制度作了有力的批判，使读者预感到它必然要走向覆灭的命运。本书是一部具有高度思想性和高度艺术性的伟大作品，从本书反映的思想倾向来看，作者具有初步的民主主义思想，他对现实社会包括宫廷及官场的黑暗，封建贵族阶级及其家庭的腐朽，封建的科举制度、婚姻制度、奴婢制度、等级制度，以及与此相适应的社会统治思想即孔孟之道和程朱理学、社会道德观念等等，都进行了深刻的批判并且提出了朦胧的带有初步民主主义性质的理想和主张。这些理想和主张正是当时正在滋长的资本主义经济萌芽因素的曲折反映。');
SELECT dump(c1) FROM testdump;
```

查询结果如下：

```
DUMP(C1)
--------------------------------------------------------------------------------
--------------------------------------------------
Typ=19                                                          Len=52:
1,104,2,0,0,0,0,0,5,0,0,0,0,0,0,0,0,0,0,146,4,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,97,100,98,99,97
Typ=19                                                          Len=47:
2,105,2,0,0,0,0,0,0,108,6,0,0,4,0,0,86,0,0,0,146,4,0,0,0,0,0,0,0,0,0,0,0,0,0,2,0,0,0,0,0,0,0
```

例 5 查询 ROWID 类型数据内部存储字节。

```
select DUMP(cast('AAACAAAAAAAAAAAAAC'AS ROWID));
```

查询结果如下：

```
DUMP(CAST('AAACAAAAAAAAAAAAAC'ASROWID))
---------------------------------------
Typ=28 Len=12: 0,2,0,0,0,0,0,0,0,0,2
```

12）SP_CLOSE_DBLINK

定义：

SP_CLOSE_DBLINK(

dblink varchar(128)

)

功能说明:

关闭系统缓冲区中指定的空闲的外部链接，若指定的外部链接不处于空闲状态（可通过 V$DBLINK 查询），则不关闭。

参数说明:

dblink：指定的待关闭的外部链接名。

返回值:

无

举例说明:

关闭之前创建的外部链接 LINK1:

```
SP_CLOSE_DBLINK('LINK1');
```

13）SP_XA_TRX_PROCESS

定义:

SP_XA_TRX_PROCESS(

trxid bigint,

flag boolean

)

功能说明:

指定提交或回滚 TRXID 对应的 XA TRX。指定的 TRXID 对应的 TRX 必须是 XA TRX，且已经通过接口执行过 xa_end 或 xa_prepare 操作，否则报错。

TRXID 可通过 V$TRX 中的 ID 查询。

参数说明:

trxid：事务的 ID 号。

flag：false 表示回滚，true 表示提交。

返回值:

无

14）GETUTCDATE

定义:

DATETIME

GETUTCDATE ()

功能说明:

获取当前 UTC 时间。

返回值:

当前 UTC 时间。

举例说明:

获取当前 UTC 时间:

```
SELECT GETUTCDATE();
```

15）SLEEP

定义:

SLEEP(time dec)

功能说明:

表示让一个线程进入睡眠状态，等待一段时间 time 之后，该线程自动醒来进入到可运行状态。

参数说明:

time：睡眠时间，单位秒。

返回值:

无

举例说明:

让一个线程睡眠 1 秒钟之后，再醒过来继续运行:

```
sleep(1);
```

16）SF_GET_TRIG_EP_SEQ

定义:

INT

SF_GET_TRIG_EP_SEQ(id int)

功能说明:

获取 DMDSC 环境下时间触发器执行节点号。

参数说明:

id：时间触发器 id。

返回值:

执行节点号。

举例说明:

```
SELECT SF_GET_TRIG_EP_SEQ(117440523);
```

17）SF_EXTRACT_BIND_DATA

定义:

VARCHAR

SF_EXTRACT_BIND_DATA(

binddata varbinary,

nth integer,

attr integer

)

功能说明:

获取绑定参数的信息。

参数说明:

binddata：绑定的参数数据。

nth：指定参数序号，从 1 开始以 1 为单位递增。

attr：为 1 时表示获取参数类型，对于小数类型，不显示精度和标度；为 2 时表示获取参数内容。

返回值:

指定参数的参数类型或参数内容。若 binddata 为 NULL 或空值，则返回 NULL；若 nth 大于实际绑定的参数个数且小于等于绑定语句中的参数个数，则返回字符串“NO DATA”；若 nth<=0 或大于绑定语句中的参数个数，则报错；若 attr 不为 1 或 2，则报错。

举例说明:

获取执行号为 1701 的绑定语句中绑定的第 3 个参数的参数类型和参数内容:

```
SELECT SF_EXTRACT_BIND_DATA(BINDDATA, 3, 1), SF_EXTRACT_BIND_DATA(BINDDATA, 3, 2)
FROM V$SQL_BINDDATA_HISTORY WHERE EXEC_ID=1701;
```

查询结果如下:

```
DEC 123456789.123
```

18) SF_EXTRACT_BIND_DATA_NUM

定义:

INT

SF_EXTRACT_BIND_DATA_NUM(

binddata varbinary,

num int

)

功能说明:

获取绑定参数的个数。

参数说明:

binddata：绑定的参数数据。

num：为 1 时表示获取实际绑定的参数个数；为 2 时表示获取绑定语句中的参数个数。

返回值:

参数个数。

举例说明:

获取执行号为 1701 的绑定语句实际绑定的参数个数:

```
SELECT SF_EXTRACT_BIND_DATA_NUM(BINDDATA, 1) FROM V$SQL_BINDDATA_HISTORY WHERE
EXEC_ID=1701;
```

查询结果如下:

```
17
```

19) SP_CREATE_SYS_OBJTYPE

定义:

SP_CREATE_SYS_OBJTYPE(

create_flag int

)

功能说明:

创建或删除内置对象(ODCI 相关)。

需要注意的是，由于新旧版本系统初始化时对某些 ODCI 相关系统类型有不同的处理，可能对该函数有不同的使用需求。

V8.1.3.97 到 V8.1.4.65 之间的版本系统初始化的时候默认存在 odcinumberlist 系统类型，表现为动态数组，最大容量为 20。此时执行 SP_CREATE_SYS_OBJTYPE(1)重建 ODCI 相关的全局系统类型，重建后 odcinumberlist 表现为嵌套表。v8.1.4.65 及其之后的版本系统初始化不存在 odci 相关系统类型，如果用到了 ODCI 相关数据类型需要先手动执行 SP_CREATE_SYS_OBJTYPE(1)，创建后 ODCINUMBERLIST 表现为嵌套表。

重建 ODCI 系统类型时，如果已经基于动态数组的 odcinumberlist 创建了表，需要先执行 SP_CREATE_SYS_OBJTYPE(0)级联删除 ODCI 相关系统类型以及关联的表，再执行 SP_CREATE_SYS_OBJTYPE(1)进行重建。

参数说明:

create_flag：1 表示创建，0 表示删除。

返回值:

无

举例说明:

创建内置对象:

```
SP_CREATE_SYS_OBJTYPE(1);
```

20) SP_SET_PLN_BINDED

定义:

SP_SET_PLN_BINDED(

```
sql_text      varchar(32767),
schname       varchar(128),
type          varchar(12),
binded        int
)
```

或者

```
SP_SET_PLN_BINDED(
hash_value    int,
schid         int,
type          varchar(12),
binded        int
)
```

或者

```
SP_SET_PLN_BINDED(
sql_text      varchar(32767),
schid         int,
type          varchar(12),
binded        int
)
```

或者

```
SP_SET_PLN_BINDED(
hash_value    int,
schname       varchar(128),
type          varchar(12),
```

binded int

)

功能说明:

绑定或解绑指定的执行计划。

参数说明:

sql_text: 执行计划对应的 SQL 语句，该语句可以从动态视图 V$SQL_PLAN 中的 SQLSTR 列获得。

schname: 执行计划的模式名。

type: 执行计划的类型，可取值: SQL: 查询语句类型; PL/OBJ: 存储过程或触发器类型。

binded: 是否绑定执行计划，可取值: 0、1、2。0: 解除内存中绑定; 1: 内存中绑定; 2: 持久化绑定。持久化绑定的计划当且仅当参数 LOAD_BINDED_PLN 开启且计划缓存池中没有该 SQL 对应的执行计划时才会被启用。解除持久化绑定可以通过系统过程函数 SP_REMOVE_STORE_PLN 完成。对于长度超过 1000 字节的 SQL 语句建议使用内存中绑定。由于持久化绑定只保存 SQL 语句的前 1000 字节，通过执行计划哈希值以及前 1000 字节字符共同校验以查找计划，故可能存在 SQL 语句不同但哈希值相同的情况，导致查找到错误的计划。

hash_value: 执行计划的哈希值，其值可以从动态视图 V$SQL_PLAN 中的 HASH_VALUE 列获得。

schid: 执行计划的模式 ID。

返回值:

无

举例说明:

绑定 SQL 语句“SELECT * FROM T1”对应的执行计划:

```
SP_SET_PLN_BINDED('SELECT * FROM T1;', 'SYSDBA', 'SQL', 1);
```

21) SP_SET_PLN_DISABLED

定义:

SP_SET_PLN_DISABLED(

pln_id bigint,

disabled int

)

功能说明:

设置绑定执行计划为禁用或启用。

参数说明:

pln_id：绑定执行计划 ID，对应于系统表 SYSPLNINFO 的 PLN_ID 列。系统表 SYSPLNINFO 请参考《DM8 系统管理员手册》中的附录 1。

disabled：是否禁用执行计划。取值为 0、1。0 表示启用；1 表示禁用。

返回值:

无

举例说明:

禁用系统表 SYSPLNINFO 中 PLN_ID 为 1 的执行计划：

```
SP_SET_PLN_DISABLED(1, 1);
```

22) SF_GET_PLN_VERSION

定义:

SF_GET_PLN_VERSION(

type int

)

功能说明:

获取执行计划指定类型的版本号。

参数说明:

type：版本号类型。取值为 0、1。0 表示指令版本号；1 表示格式版本号。

返回值:

执行计划指定类型的版本号。

举例说明:

获取执行计划的指令版本号:

```
SELECT SF_GET_PLN_VERSION(0);
```

23) SP_REMOVE_STORE_PLN

定义:

SP_REMOVE_STORE_PLN(

pln_id bigint

)

功能说明:

移除持久化绑定的执行计划。

参数说明:

pln_id：绑定执行计划 ID，对应于系统表 SYSPLNINFO 的 PLN_ID 列。系统表 SYSPLNINFO 请参考《DM8 系统管理员手册》中的附录 1。

返回值:

无

举例说明:

移除系统表 SYSPLNINFO 中 PLN_ID 为 1 的执行计划：

```
SP_REMOVE_STORE_PLN(1);
```

24) $$PLSQL_UNIT

定义：

$$PLSQL_UNIT

功能说明：

获取当前方法所在 DMSQL 程序名称。其中，语句块因为无名称所以不显示。

返回值：

DMSQL 程序名称。

举例说明：

使用$$PLSQL_UNIT 获取当前方法所在的 DMSQL 程序名称。

```
create or replace package test_dm
    is
    procedure test_plsql_prc;
    function test_plsql_fnc return varchar;
end test_dm;


create or replace package body test_dm is
   procedure test_plsql_prc IS
     i pls_integer;
     begin
       DBMS_OUTPUT.PUT_LINE('inside test_plsql_prc');
       i := $$PLSQL_LINE;
```

```
        DBMS_OUTPUT.PUT_LINE('i = ' || i);
        DBMS_OUTPUT.PUT_LINE('$$PLSQL_LINE = ' || $$PLSQL_LINE);
        DBMS_OUTPUT.PUT_LINE('$$PLSQL_UNIT = ' || $$PLSQL_UNIT);
    end test_plsql_prc;
  function test_plsql_fnc return varchar
  IS
    j       pls_integer;
    res     varchar(100);
    begin
      DBMS_OUTPUT.PUT_LINE('Inside test_plsql_fnc');
      j := $$PLSQL_LINE;
      res := $$PLSQL_UNIT;
      DBMS_OUTPUT.PUT_LINE('j = ' || j);
      DBMS_OUTPUT.PUT_LINE('$$PLSQL_LINE = ' || $$PLSQL_LINE);
      DBMS_OUTPUT.PUT_LINE('$$PLSQL_UNIT = ' || $$PLSQL_UNIT);
      return (res);
    end test_plsql_fnc;
end   test_dm;


declare
  res varchar(100);
  begin
      res := test_dm.test_plsql_fnc;
      DBMS_OUTPUT.PUT_LINE('$$PLSQL_UNIT2 = ' || res);
```

```
    end;
```

打印结果如下：

```
Inside test_plsql_fnc

j = 17

$$PLSQL_LINE = 20

$$PLSQL_UNIT = TEST_DM

$$PLSQL_UNIT2 = TEST_DM
```

25) $$PLSQL_LINE

定义：

$$PLSQL_LINE

功能说明：

获取当前方法在 DMSQL 程序中的行号。

返回值：

行号。

举例说明：

使用$$PLSQL_LINE 获取当前方法在 DMSQL 程序中的行号。

```
CREATE OR REPLACE PROCEDURE p

IS

    i pls_integer;

    c varchar(100);

BEGIN

    DBMS_OUTPUT.PUT_LINE('Inside p');

    i := $$PLSQL_LINE;
```

```
    DBMS_OUTPUT.PUT_LINE('i = ' || i);
    DBMS_OUTPUT.PUT_LINE('$$PLSQL_LINE = ' || $$PLSQL_LINE);
    DBMS_OUTPUT.PUT_LINE('$$PLSQL_UNIT = ' || $$PLSQL_UNIT);
END;

BEGIN
    DBMS_OUTPUT.PUT_LINE('当前块中的信息：');
    DBMS_OUTPUT.PUT_LINE($$PLSQL_LINE);
    DBMS_OUTPUT.PUT_LINE('$$PLSQL_LINE = ' || $$PLSQL_LINE);
    DBMS_OUTPUT.PUT_LINE('P 中的信息：');
    p;
END;
```

打印结果如下：

```
当前块中的信息：
3
$$PLSQL_LINE = 4
P 中的信息：
Inside p
i = 7
$$PLSQL_LINE = 9
$$PLSQL_UNIT = P
```

26) SP_SESSION_BT

定义：

SP_SESSION_BT (

session_id bigint

)

功能说明:

打开 INI 监控参数 ENABLE_MONITOR=1 、 MONITOR_SQL_EXEC=1 ，SP_SESSION_BT()才有效。

通过指定会话 ID，查询该会话虚拟机当前执行的堆栈信息。如果输入的会话处于空闲状态，则本过程不输出信息。例如：当会话 X 执行了一个复杂存储过程，长时间没有结束，此时 DBA 可在另一个活动的会话 Y 中执行 SP_SESSION_BT(X 会话 ID)，来查询会话 X 中的长耗过程的信息（过程名、代码行及对应的 SQL 文本片段等），便于诊断性能问题。

参数说明:

session_id：指定的会话 ID。

返回值:

无

举例说明:

准备数据:

```
CREATE TABLE TEST1(ID INT);

CREATE TABLE TEST2(ID INT);

INSERT INTO TEST1 SELECT LEVEL FROM DUAL CONNECT BY LEVEL <=20000;

INSERT INTO TEST2  SELECT LEVEL FROM DUAL CONNECT BY LEVEL <=20000;

COMMIT;

--创建系统过程 PROC_1

CREATE OR REPLACE PROCEDURE PROC_1() AS

V_SQL VARCHAR2(2000);
```

```
BEGIN
V_SQL := 'SELECT /*+USE_NL(A,B)*/ COUNT(*) FROM TEST1 A, TEST2 B WHERE A.ID=B.ID';
EXECUTE IMMEDIATE V_SQL;
END;
/
--开启 ENABLE_MONITOR 和 MONITOR_SQL_EXEC 参数
SP_SET_PARA_VALUE(1,'ENABLE_MONITOR',1);
SP_SET_PARA_VALUE(1,'MONITOR_SQL_EXEC',1);
```

开启两个会话，会话 1 和会话 2。

在会话 1 中获取会话 ID，然后执行系统过程 PROC_1。

```
SQL> SELECT SESSID();
行号        SESSID()
---------- --------------------
1           139918747748408

SQL> CALL PROC_1();
```

在会话 1 执行 PROC_1 期间，在会话 2 中调用 SP_SESSION_BT，查询会话 1 当前堆栈信息，包括当前执行的过程名、代码行、SQL 文本和 sql_id 等信息。

```
SQL> SP_SESSION_BT(139918747748408);
行号  METHOD_NAME  LINE_NO  CMD
---------- -------------- ---------- ------------------------------------
1  0x7f40f9260688  1  select /*+use_nl(A,B)*/ count(*) from test1 A, test2 B Where A.ID=B.ID
2  SYSDBA.PROC_1  5  EXEC IMMEDIATE, pln:cd40afa8 sql_id:1376
3  $$TOP_METHOD  1
```

对于具体的执行计划，可以通过 sql_id 查询 V$PLN_HISTORY 视图来获取。

```
SQL>  SELECT * FROM V$PLN_HISTORY WHERE SQL_ID = 1376;

行号     SQL_ID     TOP_SQL_TEXT     SQL_PLAN

---------- ---------- ---------- ----------------------------------

1     1376     select /*+use_nl(A,B)*/ count(*) from test1 A, test2 B Where A.ID=B.ID



1    #NSET2: [667883600, 1, 8]

2       #PRJT2: [667883600, 1, 8]; exp_num(1), is_atom(FALSE)

3                #AAGR2:  [667883600,  1,  8];  grp_num(0),  sfun_num(1),  distinct_flag[0];
slave_empty(0)

4             #SLCT2: [667883600, 3920399, 8]; A.ID = B.ID

5               #NEST LOOP INNER JOIN2: [667883600, 3920399, 8];

6                 #CSCN2: [2, 20000, 4]; INDEX33555464(TEST1 as A)

7                 #CSCN2: [2, 20000, 4]; INDEX33555465(TEST2 as B)
```

若会话 1 中 PROC_1 过程执行结束，则在会话 2 上调用 SP_SESSION_BT 查询会话 1 的堆栈信息时不会输出任何信息。

```
SQL> SP_SESSION_BT(139918747748408);

DMSQL 过程已成功完成
```

27) SF_WHO_CALLED_ME

定义:

SF_WHO_CALLED_ME(

owner_name varchar2,

caller_name varchar2,

line_num number,

caller_type varchar2

)

功能说明:

获取上层语句块对当前语句块的调用信息。

参数说明:

owner_name：输出参数，上层语句块所属模式名。

caller_name：输出参数，上层语句块的过程名或函数名。

line_num：输出参数，上层语句块调用当前语句块的行号。

caller_type：输出参数，上层语句块类型，取值包括：ANONYMOUS BLOCK：匿名块；PROCEDURE：过程；FUNCTION：函数。当上层语句块非过程或函数时，语句块类型为 ANONYMOUS BLOCK，此时参数 owner_name 和 caller_name 均为空串。

返回值:

无

举例说明:

创建过程 CHILD_PROC 和 PARENT_PROC，PARENT_PROC 调用 CHILD_PROC，使用系统函数 SF_WHO_CALLED_ME 获取 PARENT_PROC 对 CHILD_PROC 的调用信息。

```
//创建过程 CHILD_PROC
CREATE OR REPLACE PROCEDURE CHILD_PROC(ID NUMBER)
AS
    OWNER_NAME   VARCHAR2(100);
    CALLER_NAME VARCHAR2(100);
    LINE_NUMBER NUMBER;
    CALLER_TYPE VARCHAR2(100);
```

```
BEGIN
    SF_WHO_CALLED_ME(OWNER_NAME, CALLER_NAME, LINE_NUMBER, CALLER_TYPE);
    DBMS_OUTPUT.PUT_LINE('[ID:] ' || ID || ' [CALLER_TYPE:] ' || CALLER_TYPE || ' [OWNER_NAME:]
' || OWNER_NAME || ' [CALLER_NAME:] ' || CALLER_NAME || ' [LINE_NUMBER:] ' || LINE_NUMBER);
END;
/

//创建过程 PARENT_PROC
CREATE OR REPLACE PROCEDURE PARENT_PROC
AS
    V_CHILD_PROC VARCHAR2(100) := 'BEGIN SYSDBA.CHILD_PROC (1); END;';
BEGIN
    EXECUTE IMMEDIATE V_CHILD_PROC;
    SYSDBA.CHILD_PROC(2);
END;
/

//调用过程 PARENT_PROC
CALL PARENT_PROC();
```

执行结果如下：

```
[ID:] 1 [CALLER_TYPE:] ANONYMOUS BLOCK [OWNER_NAME:]  [CALLER_NAME:]  [LINE_NUMBER:]
1
[ID:] 2 [CALLER_TYPE:] PROCEDURE [OWNER_NAME:] SYSDBA [CALLER_NAME:] PARENT_PROC
[LINE_NUMBER:] 6
```

28) SP_CREATE_SYSTEM_OPERATORS

定义:

SP_CREATE_SYSTEM_OPERATORS (

create_flag int

)

功能说明:

创建或删除所有系统自定义运算符。

参数说明:

create_flag: 为 1 时表示创建所有系统自定义运算符; 为 0 表示删除所有系统自定义运算符。

返回值:

无

举例说明:

创建所有系统自定义运算符:

```
SP_CREATE_SYSTEM_OPERATORS(1);
```

29) LSN_TO_TIMESTAMP

定义:

DATETIME

LSN_TO_TIMESTAMP(

lsn bigint

)

功能说明:

将 LSN 转换为对应时间戳，返回的时间戳为通过 V$LSN_TIME 估算的结果，可能存在一定误差。

参数说明:

lsn：需要转换为时间戳的 LSN 值，可通过 V$LSN_TIME 查询 LSN 和时间的映射关系，对于在[V$LSN_TIME 查询结果中最小的 LSN,系统 CUR_LSN]范围之间的 LSN 值，都能返回其对应时间戳。

返回值:

指定 LSN 值对应的时间戳

举例说明:

```
SELECT LSN_TO_TIMESTAMP(52317);
```

30) TIMESTAMP_TO_LSN

定义:

BIGINT

TIMESTAMP_TO_LSN(

d timestamp

)

功能说明:

将时间戳转换为对应 LSN，返回的 LSN 为通过 V$LSN_TIME 估算的结果，可能存在一定误差。

参数说明:

d：需要转换为 LSN 值的时间戳，可通过 V$LSN_TIME 查询 LSN 和时间的映射关系。

返回值:

指定时间戳对应的 LSN 值

举例说明:

```
SELECT TIMESTAMP_TO_LSN('2023-03-02 11:01:50');
```

31）MD5

定义:

MD5(

src varchar

)

功能说明:

计算并返回源串的 MD5 散列值。

参数说明:

src：源串，字符串类型，长度小于 32767。

返回值:

源串的 MD5 值。

返回值类型通过静态 INI 参数 MD5_TYPE 控制，修改参数值并重启服务器后生效，缺省值为 0。当 MD5_TYPE=0，返回值类型为 VARBINARY；当 MD5_TYPE=1，返回值类型为 VARCHAR。

举例说明:

例 1 当 MD5_TYPE=0 时。

```
SELECT MD5('ABC');
```

查询结果如下:

```
行号          MD5('ABC')
---------- ----------------------------------
1             0X902FBDD2B1DF0C4F70B4A5D23525E932
```

例 2 当 MD5_TYPE=1 时。

```
SELECT MD5('ABC');
```

查询结果如下:

```
行号          MD5('ABC')

---------- --------------------------------

1             902FBDD2B1DF0C4F70B4A5D23525E932
```

32) SP_FLUSH_HIST_PLAN

定义:

SP_FLUSH_HIST_PLAN()

功能说明:

将内存中暂存的执行计划历史信息主动记录到系统表 SYS.SYSPLANHIST 中。不支持 DMDPC 环境。

举例说明:

首先，启用执行计划历史信息功能，随后重启数据库使修改的静态参数生效。

```
SP_SET_PARA_VALUE(2, 'ENABLE_MONITOR_PLNHIST', 1);

SP_SET_PARA_VALUE(2, 'USE_PLN_POOL', 1);
```

其次，执行查询语句。

```
SELECT 1 FROM DUAL;
```

接着，运行 SP_FLUSH_HIST_PLAN()将暂存执行计划历史信息主动记录到系统表 SYS.SYSPLANHIST 中。

```
SP_FLUSH_HIST_PLAN();
```

然后，使用 DBMS_XPLAN 系统包相关包方法查看对应的执行计划历史信息。

```
DBMS_XPLAN.DISPLAY_PLANHIST('SELECT 1 FROM DUAL;');
```

查询结果如下:

```
行号          SHOW_STR

--------- ----------------------------------------------------------------------

1             SELECT 1 FROM DUAL;

2             HASH VALUE: 476121253

3

1    #NSET2: [1, 1, 1]

2       #PRJT2: [1, 1, 1]; EXP_NUM(1), IS_ATOM(FALSE)

3          #CSCN2: [1, 1, 1]; SYSINDEXSYSDUAL2(SYSDUAL2 AS DUAL); BTR_SCAN(1)

4             PLAN_ADDR: 0XB5C116D8

5             TIME: 2024-08-15 15:27:38
```

33) AES_ENCRYPT

定义:

```
VARBINARY
AES_ENCRYPT(
str             varchar,
key             varchar,
iv              varchar,
kdf_name        varchar,
salt            varchar,
info            varchar
)
```

参数说明:

str：待加密的字符串。

key：密钥。

iv：初始化向量。当 ini 参数 BLOCK_ENCRYPTION_MODE 取值中包含 CBC、CFB、OFB、CFB1 或 CFB8 时，必须指定 iv，如果 iv 为空或者长度不足则会报错。

kdf_name：保留参数，无实际作用，可指定为 NULL。

salt：保留参数，无实际作用，可指定为 NULL。

info：保留参数，无实际作用，可指定为 NULL。

功能说明：

使用指定的密钥对字符串进行 AES 加密。加密算法由 ini 参数 BLOCK_ENCRYPTION_MODE 指定。

返回值：

加密后的二进制数据。

举例说明：

首先，通过 ini 参数 BLOCK_ENCRYPTION_MODE 设置加密算法。

```
SP_SET_PARA_STRING_VALUE(0, 'BLOCK_ENCRYPTION_MODE', 'AES-256-OFB');
```

然后，使用 AES_ENCRYPT 函数对字符串“aaa”进行 AES 加密。

```
SELECT AES_ENCRYPT('aaa','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB', '1234567890123456', NULL, NULL, NULL);
```

返回结果如下：

```
行                                                                              号
AES_ENCRYPT('aaa','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB','1234567890123456',NULL,NULL,NULL)
--------------------------------------------------------------------------------
1               0xE860F8
```

34) AES_DECRYPT

定义:

```
VARCHAR
AES_DECRYPT(
crypt_str       varbinary,
key             varchar,
iv              varchar,
kdf_name        varchar,
salt            varchar,
info            varchar
)
```

参数说明:

crypt_str：待解密的二进制数据。

key：密钥。和 AES_ENCRYPT 加密时使用的密钥相同。

iv：初始化向量。和 AES_ENCRYPT 加密时使用的初始化向量相同。

kdf_name：保留参数，无实际作用，可指定为 NULL。

salt：保留参数，无实际作用，可指定为 NULL。

info：保留参数，无实际作用，可指定为 NULL。

功能说明:

使用加密时相同的密钥对二进制数据进行 AES 解密。加密算法由 ini 参数 BLOCK_ENCRYPTION_MODE 指定。

返回值:

解密后的明文数据。

举例说明:

首先，通过 ini 参数 BLOCK_ENCRYPTION_MODE 设置加密算法。

```
SP_SET_PARA_STRING_VALUE(0, 'BLOCK_ENCRYPTION_MODE', 'AES-256-OFB');
```

然后，使用 AES_DECRYPT 函数对二进制数据 0xE860F8 进行 AES 解密。

```
SELECT AES_DECRYPT('0xE860F8','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB',
'1234567890123456', NULL, NULL, NULL);
```

返回结果如下:

```
行 号
AES_DECRYPT('0xE860F8','AAAAAAAAAAAAAAAABBBBBBBBBBBBBBBB','1234567890123456',NULL,N
ULL,NULL)
----------------------------------------------------------------------------
1 aaa
```

13. 编目函数调用的系统函数

1) SF_GET_BUFFER_LEN

定义:

```
INT
SF_GET_BUFFER_LEN(
name varchar,
length int,
scale int
)
```

参数说明:

name：某列数据类型名，数据类型为 VARCHAR。

length：列的精度，INT 类型。

scale：列的刻度，INT 类型。

功能说明：

根据某列数据类型名 name 和列的精度 length 刻度 scale 获取该列存贮在硬盘上的长度。

返回值：

列存贮长度，数据类型为 INT。

举例说明：

```
SELECT SF_GET_BUFFER_LEN('VARCHAR',3,2);
```

查询结果如下：

```
3
```

2） SF_GET_DATA_TYPE

定义：

INT

SF_GET_DATA_TYPE(

name varchar,

scale int,

version int

)

参数说明：

name：某列数据类型名，数据类型为 VARCHAR。

scale：时间间隔类型刻度，INT 类型。

version：版本号，INT 类型。

功能说明:

根据数据类型关键字获取对应的 SQL 数据类型。

返回值:

数据类型值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_DATA_TYPE('VARCHAR',3,2);
```

查询结果如下:

```
12
```

3) SF_GET_DATE_TIME_SUB

定义:

INT

SF_GET_DATE_TIME_SUB(

name varchar,

scale int

)

参数说明:

name：某列数据类型名，数据类型为 VARCHAR。

scale：类型刻度，INT 类型。

功能说明:

获得时间类型的子类型。

返回值:

时间类型子类型值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_DATE_TIME_SUB('datetime',2);
```

查询结果如下：

```
3
```

4） SF_GET_DECIMAL_DIGITS

定义：

INT

SF_GET_DECIMAL_DIGITS(

name varchar,

scale int

)

参数说明：

name：某列数据类型名，数据类型为 VARCHAR。

scale：预期类型刻度，INT 类型。

功能说明：

根据某数据类型名和预期的刻度获取该数据类型的实际刻度。

返回值：

类型实际刻度值，数据类型为 INT。

举例说明：

```
SELECT SF_GET_DECIMAL_DIGITS('INT',2);
```

查询结果如下：

```
0
```

5） SF_GET_SQL_DATA_TYPE

定义:

INT

SF_GET_SQL_DATA_TYPE(

name varchar

)

参数说明:

name：某列数据类型名，数据类型为 VARCHAR。

功能说明:

根据某数据类型名返回该数据类型的 SQL 数据类型值。

返回值:

SQL 数据类型值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_SQL_DATA_TYPE('INT');
```

查询结果如下:

```
4
```

6) SF_GET_SYS_PRIV

定义:

VARCHAR

SF_GET_SYS_PRIV(

privid int

)

参数说明:

privid：数据类型为 INT。取值范围：数据库权限 4096~4293，对象权限 8192~8204；

其中除去 4120、4121、4123、4126、4127、4204、4205、4208、4209 和 4253，这些值没有代表的权限操作。

功能说明:

获得 privid 所代表的权限操作。

返回值:

前导字符串，数据类型为 VARCHAR。

举例说明:

```
SELECT SF_GET_SYS_PRIV(4096);

SELECT SF_GET_SYS_PRIV(4099);
```

查询结果如下:

```
CREATE DATABASE

CREATE LOGIN
```

7) SF_GET_OCT_LENGTH

定义:

INT

SF_GET_OCT_LENGTH(

name varchar,

length int

)

参数说明:

name：数据类型名，数据类型为 VARCHAR。

length：类型长度，INT 类型。

功能说明:

返回变长数据类型的长度。

返回值:

类型长度，数据类型为 INT。

举例说明:

```
SELECT SF_GET_OCT_LENGTH('VARCHAR',3);
```

查询结果如下:

```
3
```

8) SF_GET_TABLES_TYPE

定义:

VARCHAR

SF_GET_TABLES_TYPE(

type varchar

)

参数说明:

type：表的类型名，数据类型为 VARCHAR。

功能说明:

返回表类型名。

返回值:

表类型名，数据类型为 VARCHAR。

举例说明:

```
SELECT SF_GET_TABLES_TYPE('UTAB');
```

查询结果如下:

```
TABLE
```

9） CURRENT_SCHID

定义:

INT

CURRENT_SCHID()

参数说明:

无

功能说明:

返回当前会话的当前模式 ID。

返回值:

当前会话的当前模式 ID。

举例说明:

```
SELECT CURRENT_SCHID();
```

查询结果如下:

```
150994945
```

10）SF_GET_SCHEMA_NAME_BY_ID

定义:

VARCHAR

SF_GET_SCHEMA_NAME_BY_ID(

schid int

)

参数说明:

schid：模式 ID，INT 类型。

功能说明:

根据模式 ID 返回模式名。

返回值:

模式名，数据类型为 VARCHAR。

举例说明:

```
SELECT SF_GET_SCHEMA_NAME_BY_ID(150994945);
```

查询结果如下:

```
SYSDBA
```

11) SF_COL_IS_IDX_KEY

定义:

INT

SF_COL_IS_IDX_KEY(

key_num int,

key_info varbinary,

col_id int

)

参数说明:

key_num：键值个数，数据类型为 INT。

key_info：键值信息，数据类型为 VARBINARY。

col_id：列 ID，INT 类型。

功能说明:

判断所给的 colid 是否是 index key。

返回值:

是否索引键，数据类型为 INT。

举例说明：

```
CREATE TABLE TT1(C1 INT);
CREATE INDEX ID1 ON TT1(C1);
SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'TT1';//1334
SELECT INDS.KEYNUM, INDS.KEYINFO, SF_COL_IS_IDX_KEY(INDS.KEYNUM, INDS.KEYINFO, COLS.COLID)
FROM (SELECT ID,PID,NAME FROM SYSOBJECTS WHERE SUBTYPE$='INDEX') AS OBJ_INDS,
SYSCOLUMNS AS COLS, SYSINDEXES AS INDS WHERE COLS.ID = 1334 AND OBJ_INDS.PID = 1334 AND
INDS.ID = OBJ_INDS.ID;
```

查询结果如下：

```
1   000041   1        //ID1
0              0        //聚集索引
```

12）SF_COL_IS_IDX_KEY

定义：

INT

SF_COL_IS_IDX_KEY(

key_num int,

key_info varbinary,

col_id int,

idx_type int

)

参数说明：

key_num：键值个数，数据类型为 INT。

key_info：键值信息，数据类型为 VARBINARY。

col_id：列 ID，INT 类型。

idx_type：索引类型，0/1/2 分别表示聚集索引/函数索引/位图索引。

功能说明：

判断所给的 colid 是否是指定索引类型的 index key。

返回值：

是否是指定索引类型的索引键，数据类型为 INT。

举例说明：

```
DROP INDEX IDX1;

DROP TABLE TAB2;

DROP TABLE TAB1;



CREATE TABLE TAB1(

      C1 INT IDENTITY(1,1) PRIMARY KEY ,

      C2 INT NOT NULL);



CREATE TABLE TAB2(

      C3 INT IDENTITY(1,1) PRIMARY KEY,

      C4 INT NOT NULL REFERENCES TAB1(C1));



//创建位图索引:

CREATE BITMAP INDEX IDX1 ON TAB2(TAB1.C2)

     FROM     TAB1, TAB2

     WHERE    TAB1.C1 = TAB2.C4;
```

```
SELECT
    distinct OBJ_INDS.NAME, COLS.NAME, INDS.TYPE$, COLS.COLID
FROM
    (SELECT * FROM SYSINDEXES) INDS,
    SYSCOLUMNS AS COLS,
    (SELECT * FROM SYSOBJECTS WHERE SYSOBJECTS.NAME='TAB1') OBJ_TAB1,
    (SELECT * FROM SYSOBJECTS WHERE SYSOBJECTS.SUBTYPE$='INDEX') OBJ_INDS
WHERE
    COLS.ID=OBJ_TAB1.ID
    AND OBJ_TAB1.ID=OBJ_INDS.PID
    AND SF_COL_IS_IDX_KEY(INDS.KEYNUM, INDS.KEYINFO, COLS.COLID,2)=1
    AND INDS.TYPE$='BM';
```

查询结果如下:

```
行号        NAME              NAME TYPE$ COLID
---------- -------------- ---- ----- -----------
1           INDEX33556047   C2    BM     1
2           INDEX33556048   C2    BM     1
3           IDX1_1371_1370   C2    BM     1
```

13) SF_GET_INDEX_KEY_ORDER

定义:

```
VARCHAR
SF_GET_INDEX_KEY_ORDER(
```

key_num int,

key_info varbinary,

col_id int

)

参数说明:

key_num：索引键个数，数据类型为 INT。

key_info：键值信息，数据类型为 VARBINARY。

col_id：列 ID，INT 类型。

功能说明:

获得当前 column 的 key 的排序。

返回值:

类型长度，数据类型为 VARCHAR。

举例说明:

```
CREATE TABLE TT1(C1 INT);

CREATE INDEX ID1 ON TT1(C1);

SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'TT1';//1135

SELECT SF_GET_INDEX_KEY_ORDER(INDS.KEYNUM, INDS.KEYINFO, COLS.COLID) FROM (SELECT ID,PID,NAME FROM SYSOBJECTS WHERE SUBTYPE$='INDEX') AS OBJ_INDS, SYSCOLUMNS AS COLS, SYSINDEXES AS INDS WHERE COLS.ID = 1135 AND OBJ_INDS.PID = 1135 AND INDS.ID = OBJ_INDS.ID;
```

查询结果如下:

```
A

NULL
```

14) SF_GET_INDEX_KEY_SEQ

定义:

INT

SF_GET_INDEX_KEY_SEQ(

key_num int,

key_info varbinary,

col_id int

)

参数说明:

key_num：索引键个数，数据类型为 INT。

key_info：键值信息，数据类型为 VARBINARY。

col_id：列 ID，INT 类型。

功能说明:

获得当前 column 所在的 key 序号。

返回值:

当前列 KEY 序号，数据类型为 INT。

举例说明:

```
CREATE TABLE TT1(C1 INT);

CREATE INDEX ID1 ON TT1(C1);

SELECT ID FROM SYSOBJECTS WHERE NAME LIKE 'TT1';//1135

SELECT SF_GET_INDEX_KEY_SEQ(INDS.KEYNUM, INDS.KEYINFO, COLS.COLID) FROM (SELECT

ID,PID,NAME FROM SYSOBJECTS WHERE SUBTYPE$='INDEX') AS OBJ_INDS, SYSCOLUMNS AS COLS,

SYSINDEXES AS INDS WHERE COLS.ID = 1135 AND OBJ_INDS.PID = 1135 AND INDS.ID = OBJ_INDS.ID;
```

查询结果如下:

```
1
```

```
-1
```

15）SF_GET_UPD_RULE

定义：

INT

SF_GET_UPD_RULE(

rule varchar(2)

)

参数说明：

rule：规则名，数据类型为 VARCHAR。rule 参数只会用到 rule[0]，取值为"/'C'/'N'/'D'，分别表示 no act/cascade/set null/set default。

功能说明：

解析在 SYSCONS 中 faction 中保存的外键更新规则。

返回值：

外键更新规则值，数据类型为 INT。

举例说明：

```
SELECT SF_GET_UPD_RULE('C');
```

查询结果如下：

```
0
```

16）SF_GET_DEL_RULE

定义：

INT

SF_GET_DEL_RULE(

rule varchar(2)

)

参数说明:

rule: 规则名，数据类型为 VARCHAR，rule 参数只会用到 rule[1]，取值为"/'C'/'N'/'D'，分别表示 no act/cascade/set null/set default。

功能说明:

解析在 SYSCONS 中 faction 中保存的外键删除规则。

返回值:

外键删除规则值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_DEL_RULE('AC');
```

查询结果如下:

```
0
```

17) SF_GET_OLEDB_TYPE

定义:

INT

SF_GET_OLEDB_TYPE(

name varchar

)

参数说明:

name: 类型名，数据类型为 VARCHAR。

功能说明:

获得 OLEDB 的数据类型长度。

返回值:

数据类型值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_OLEDB_TYPE('INT');
```

查询结果如下:

```
3
```

18) SF_GET_OLEDB_TYPE_PREC

定义:

INT

SF_GET_OLEDB_TYPE_PREC(

name varchar,

length int

)

参数说明:

name：类型名，数据类型为 VARCHAR。

length：类型长度，数据类型为 INT。

功能说明:

获得 OLEDB 的数据类型的精度。

返回值:

数据类型的精度值，数据类型为 INT。

举例说明:

```
SELECT SF_GET_OLEDB_TYPE_PREC('INT',2);
```

查询结果如下:

```
10
```

19）SP_GET_TABLE_COUNT

定义：

BIGINT

SP_GET_TABLE_COUNT(

table_id int

)

参数说明：

table_id：表 id，数据类型为 INT。

功能说明：

获得表行数。

返回值：

表的行数，数据类型为 BIGINT。

举例说明：

```
SELECT SP_GET_TABLE_COUNT(1097);
```

查询结果如下：

```
69
```

20）SF_OLEDB_TYPE_IS_LONG

定义：

INT

SF_OLEDB_TYPE_IS_LONG(

typename varchar

)

参数说明:

typename：类型名，数据类型为 VARCHAR。

功能说明:

判断类型是否较长。

返回值:

0 或者 1，数据类型为 INT。

举例说明:

```
SELECT SF_OLEDB_TYPE_IS_LONG('IMAGE');

SELECT SF_OLEDB_TYPE_IS_LONG('INT');
```

查询结果如下:

```
1

0
```

21) SF_OLEDB_TYPE_IS_BESTMATCH

定义:

INT

SF_OLEDB_TYPE_IS_BESTMATCH(

typename varchar

)

参数说明:

typename：类型名，数据类型为 VARCHAR。

功能说明:

判断类型是否精确匹配类型。

返回值:

0 或者 1，数据类型为 INT。

举例说明:

```
SELECT SF_OLEDB_TYPE_IS_BESTMATCH('INT');

SELECT SF_OLEDB_TYPE_IS_BESTMATCH('IMAGE');
```

查询结果如下:

```
1

0
```

22) SF_OLEDB_TYPE_IS_FIXEDLEN

定义:

INT

SF_OLEDB_TYPE_IS_FIXEDLEN(

typename varchar

)

参数说明:

typename：类型名，数据类型为 VARCHAR。

功能说明:

判断类型是否为定长类型。

返回值:

0 或者 1，数据类型为 INT。

举例说明:

```
SELECT SF_OLEDB_TYPE_IS_FIXEDLEN('INT');

SELECT SF_OLEDB_TYPE_IS_FIXEDLEN('IMAGE');
```

查询结果如下：

```
1

0
```

23) SF_GET_TABLE_COUNT

定义：

BIGINT

SF_GET_TABLE_COUNT(

schema_name varchar(128),

table_name varchar(128)

)

参数说明：

schema_name：模式名，数据类型为 varchar。

table_name：表名，数据类型为 varchar。

功能说明：

获得表行数。功能和 SP_GET_TABLE_COUNT 一样。

返回值：

表的行数，数据类型为 BIGINT。

举例说明：

```
SELECT SF_GET_TABLE_COUNT('SYSDBA','T');
```

查询结果如下：

```
69
```

14. BFILE

1) BFILENAME

定义:

BFILE

BFILENAME(

dir varchar,

filename varchar

)

参数说明:

dir：数据库的目录对象名，字符串中不能包含":"。

filename：操作系统文件名，字符串中不能包含":"。

功能说明:

生成一个 BFILE 类型数据。

返回值:

BFILE 数据类型对象。

15. 定制会话级 INI 参数

提供用户定制会话级 INI 参数默认值的功能。定制以后，当用户再次登录时无需复杂的设置就可以在当前会话中使用定制的会话级 INI 参数，并且使无法设置参数的 B/S 或者 C/S 应用也能使用特定的会话级 INI 参数。

定制后的 INI 参数值可以通过数据字典 SYSUSERINI 查看。

使用时有以下限制:

1) 用户对会话级参数的定制和取消对于当前会话不会立刻生效，也不会影响当前已经

连接的该用户的其他会话。该用户新登录的会话会使用定制的值作为默认值。

2) MPP 环境下，参数的定制在整个 MPP 环境所有的服务器节点生效。

3) 对于一个会话级 INI 参数，其取值优先级顺序为： 会话中设置的值 > 用户定制的值 > INI 文件中匹配的值。

4) DBA 用户拥有定制、查询和删除所有用户 INI 的权限；普通用户拥有定制、查询和删除自身 INI 的权限。

对于不同数据类型的参数要使用不同的系统过程。INI 参数的 VALUE 有三种类型：INT、VARCHAR 和 DOUBLE，分别对应三个系统过程：

1) SP_SET_USER_INI

定义：

SP_SET_USER_INI(

user varchar,

para_name varchar,

value int

)

功能说明：

定制 INT 类型的 INI 参数。

参数说明：

user：用户名。

para_name：参数名。

value：INT 类型参数值。

举例说明：

```
SP_SET_USER_INI('USER1','SORT_BUF_SIZE','1234');
```

2) SP_SET_USER_STRING_INI

定义:

SP_SET_USER_STRING_INI(

user varchar,

para_name varchar,

value varchar

)

功能说明:

定制 VARCHAR 类型的 INI 参数。

参数说明:

user: 用户名。

para_name: 参数名。

value: VARCHAR 类型参数值。

3) SP_SET_USER_DOUBLE_INI

定义:

SP_SET_USER_DOUBLE_INI(

user varchar,

para_name varchar,

value varchar

)

功能说明:

定制 DOUBLE 类型的 INI 参数。

参数说明:

user：用户名。

para_name：参数名。

value：DOUBLE 类型参数值。

举例说明：

```
SP_SET_USER_DOUBLE_INI('USER2','INDEX_SKIP_SCAN_RATE','0.0035');
```

4） SP_CLEAR_USER_INI

定义：

SP_CLEAR_USER_INI(

user varchar,

para_name varchar

)

功能说明：

清除用户 USER 的 PARA_NAME 的参数定制。当 USER 和 PARA_NAME 都等于 NULL 时，表示清除所有用户的所有定制；当 PARA_NAME 为 NULL 时，表示清除用户 USER 的所有定制。不支持 USER 为 NULL，PARA_NAME 不为 NULL 的情况。

参数说明：

user：用户名。

para_name：参数名。

举例说明：

```
SP_CLEAR_USER_INI('USER2',NULL);
```

16. 为 SQL 注入 HINT 规则

为 SQL 语句注入 HINT 规则，是指无需修改 SQL 语句，通过 SF_INJECT_HINT 为该 SQL

注入 HINT 规则，随后该 SQL 按照注入的 HINT 规则运行的功能。通过 SF_INJECT_HINT 注入的 HINT 称为 INJECT HINT。

使用方法:

第一步，DM.INI 参数 ENABLE_INJECT_HINT 设置为 1。

第二步，通过 SF_INJECT_HINT 为 SQL 注入 HINT 规则。

HINT 规则一经注入，就全局存在。HINT 规则名称必须全局唯一。HINT 规则的名称是指 SF_INJECT_HINT 中 name 参数指定的名称或 name 参数为 NULL 时系统自动创建的名称。

SF_INJECT_HINT 方法创建的 HINT 规则默认无生效类别限制。如果需要为该 HINT 规则设置生效类别，可通过 SF_ALTER_HINT 过程修改。

第三步，使用 SF_ALTER_HINT 修改指定 HINT 规则的属性。

本步骤为可选项，需要修改 HINT 规则时使用。

对于指定 SQL，在一个指定生效类别下，精确匹配 HINT 规则至多指定一个，模糊匹配 HINT 规则可以一个或多个。当指定 SQL 已经存在无生效类别限制的精确匹配 HINT 规则时，无法再为该 SQL 指定新的精确匹配 HINT 规则。在符合 SQL 匹配条件的 HINT 规则同时存在多个时，精确匹配总是优先于模糊匹配，且 SQL 匹配方式（精确/模糊匹配）相同时，有生效类别限制的优先。

第四步，改变当前会话的 HINT 规则。

本步骤为可选项，需要改变当前会话的 HINT 规则时使用。

当某 HINT 规则存在生效类别限制，且其和 DM.INI 参数 SQLTUNE_CATEGORY 的会话值不一致时，该 HINT 规则在本会话范围内将不生效。因此，可以通过设置 SQLTUNE_CATEGORY 的会话值，让 HINT 规则在指定会话中达到不生效的目的。

第五步，验证是否指定成功。

通过 SYSINJECTHINT 视图查看 SQL 语句及其注入的 HINT 规则信息。

第六步，使用 SF_DEINJECT_HINT 删除 SQL 的 HINT 规则。

本步骤为可选项，需要删除 SQL 的 HINT 规则时使用。

下面详细介绍为 SQL 注入 HINT 规则的相关系统函数：

1) SF_INJECT_HINT

定义：

定义 1:

```
VARCHAR
SF_INJECT_HINT (
sql_text      text,
hint_text     text,
name          varchar(128),
description   varchar(256),
validate      boolean
)
```

或者

定义 2:

```
VARCHAR
SF_INJECT_HINT (
sql_text      text,
hint_text     text,
name          varchar(128),
description   varchar(256),
validate      boolean,
fuzzy         boolean
```

)

或者

定义 3:

VARCHAR

SF_INJECT_HINT (

sql_text text,

hint_text text,

name varchar(128),

description varchar(256),

validate boolean,

fuzzy boolean,

need_clear boolean

)

或者

定义 4:

VARCHAR

SF_INJECT_HINT (

sql_text text,

hint_text text,

name varchar(128),

description varchar(256),

validate boolean,

fuzzy int

)

或者

定义 5:

```
VARCHAR
SF_INJECT_HINT (
sql_text        text,
hint_text       text,
name            varchar(128),
description     varchar(256),
validate    boolean,
fuzzy           int,
need_clear      boolean
)
```

或者

定义 6:

```
VARCHAR
SF_INJECT_HINT (
sql_text        text,
hint_text       text,
name            varchar(128),
description     varchar(256),
validate    boolean,
fuzzy           boolean,
need_clear      int
)
```

或者

定义 7:

```
VARCHAR
SF_INJECT_HINT (
sql_text          text,
hint_text         text,
name              varchar(128),
description       varchar(256),
validate      boolean,
fuzzy             int,
need_clear        int
)
```

功能说明:

为 SQL 注入 HINT 规则。SF_INJECT_HINT 方法创建的 HINT 规则无生效类别限制，如果需要为该 HINT 规则设置生效类别，可通过 SF_ALTER_HINT 过程修改实现。

参数说明:

sql_text：待注入 HINT 规则的 SQL 语句。fuzzy 参数类型为 INT 时，值为 2 表示支持通过 sql_text_id 指定待注入的 SQL 语句，sql_text_id 字段可通过 V$SQLTEXT 视图进行查询。

hint_text：待注入的 HINT 规则，必须指定为非 NULL 值。

name：HINT 规则的名称，指定为 NULL 值时系统为其命名。

description：对 HINT 规则的详细描述。

validate：HINT 规则是否生效。TRUE 是；FALSE 否。

fuzzy：指定 SQL 的匹配规则为精准匹配或模糊匹配，该参数值不能为 NULL。当该参数

缺省时，表示使用定义 1 的语法，仅支持精准匹配。指定该参数时：

- 该参数类型为 BOOLEAN 时，使用定义 2 或定义 3 的语法，值为 TRUE 时，为模糊匹配；值为 FALSE 时，为精准匹配。精准匹配时，待注入 HINT 规则的 SQL 语句必须为语法正确的 INSERT/DELETE/UPDATE/SELECT/MERGE INTO 语句（语句以 EXPLAIN/EXPLAIN FOR 开头时，去掉 EXPLAIN/EXPLAIN FOR 后的语句必须完全正确），精准匹配要求 SQL 语句完全匹配，不支持 SQL 语句中的子查询匹配；模糊匹配时，待注入 HINT 规则的 SQL 语句应为非 NULL 值。
- 该参数类型为 INT 时，使用定义 4 或定义 5 的语法，值为 0 表示精确匹配，值为 1 表示模糊匹配，值为 2 表示通过 sql_text_id 精确匹配。fuzzy 取值为 2 时，参数 sql_text 应该输入长度为 13 的 sql_text_id 字段，如果 fuzzy 取值为 2 但参数 sql_text 依旧输入 SQL 语句，预期会报错。

need_clear：是否同步清空所有缓存的计划，该参数值不能为 NULL。当该参数缺省时，表示使用定义 1、定义 2 或定义 4 的语法，此时默认不清空缓存计划。指定该参数时：

- 该参数类型为 BOOLEAN 时，使用定义 3 或定义 5 的语法，值为 TRUE 时，清空缓存的计划；值为 FALSE 时，模糊匹配和通过 sql_text_id 匹配时，不清空缓存的计划，需要手动清除对应 SQL 的计划后，指定的 HINT 才能生效，精确匹配时，默认会删除受影响 SQL 的计划缓存。
- 该参数类型为 INT 时，使用定义 6 或定义 7 的语法，值为 0 时，不清空缓存的计划；值为 1 时，清空缓存的计划；值为 2 时，只清空相关 SQL 的计划缓存，具体规则为：（1）对于精确匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况均清空对应 SQL 的计划缓存；（2）对于模糊匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况，对于计划缓存池中的所有 SQL，只要其包含了 HINT 指定的子串，就删除其计划缓存；（3）对于通过 SQL_TEXT_ID 进行匹配的情况，不删相关 SQL 的计划缓存，与

need_clear 为 0 时表现一样。

返回值:

执行成功返回名称，执行失败报错误信息。

举例说明:

进行建表操作。

```
DROP TABLE A;

CREATE TABLE A(C1 INT,C2 CHAR(20));
```

在指定 HINT 前，LIKE_OPT_FLAG 默认为 127，将对 LIKE 表达式进行优化处理。执行下列语句:

```
EXPLAIN SELECT * FROM A WHERE C2 LIKE '%DM%';
```

对 LIKE 表达式进行优化处理，执行结果如下:

```
1    #NSET2: [1, 1, 64]

2       #PRJT2: [1, 1, 64]; exp_num(3), is_atom(FALSE)

3          #SLCT2: [1, 1, 64]; exp11 > 0

4             #CSCN2: [1, 6, 64]; INDEX33555472(A); btr_scan(1)
```

通过以下语句指定 HINT 为 LIKE_OPT_FLAG=0 的精准匹配规则，此时将不对 LIKE 表达式进行优化处理。

```
SF_INJECT_HINT('SELECT * FROM A WHERE C2 LIKE "%DM%";', 'LIKE_OPT_FLAG(0)', 'TEST_INJECT', 'to test function of injecting hint', TRUE);
```

或者:

```
SF_INJECT_HINT('SELECT * FROM A WHERE C2 LIKE "%DM%";', 'LIKE_OPT_FLAG(0)', 'TEST_INJECT', 'to test function of injecting hint', TRUE, FALSE);
```

在指定 HINT 为 LIKE_OPT_FLAG=0 的精准匹配规则后，执行下列语句:

```
EXPLAIN SELECT * FROM A WHERE C2 LIKE '%DM%';
```

此时不对 LIKE 表达式进行优化处理，规则生效，执行结果如下：

```
1    #NSET2: [1, 1, 64]
2      #PRJT2: [1, 1, 64]; exp_num(3), is_atom(FALSE)
3        #SLCT2: [1, 1, 64]; A.C2 LIKE '%DM%' SLCT_PUSHDOWN(TRUE)
4          #CSCN2: [1, 6, 64]; INDEX33555472(A) NEED_SLCT(TRUE); btr_scan(1)
```

通过以下语句指定 HINT 为 LIKE_OPT_FLAG=0 的模糊匹配规则：

```
SF_INJECT_HINT('SELECT * FROM A WHERE C2 LIKE "%DM%";', 'LIKE_OPT_FLAG(0)', 'TEST_INJECT', 'to
test function of injecting hint', TRUE, TRUE);
```

通过以下语句指定 HINT 为 LIKE_OPT_FLAG=0 的模糊匹配规则，且不清空缓存的计划：

```
SF_INJECT_HINT('SELECT * FROM A WHERE C2 LIKE "%DM%";', 'LIKE_OPT_FLAG(0)', 'TEST_INJECT', 'to
test function of injecting hint', TRUE, TRUE, FALSE);
```

2) SF_ALTER_HINT

定义:

定义 1:

```
INT
SF_ALTER_HINT (
name                    varchar(128),
attribute_name varchar(12),
attribute_value varchar(256)
)
```

或者

定义 2:

```
INT
```

SF_ALTER_HINT (

name varchar(128),

attribute_name varchar(12),

attribute_value varchar(256),

need_clear boolean

)

或者

定义 3:

INT

SF_ALTER_HINT (

name varchar(128),

attribute_name varchar(12),

attribute_value varchar(256),

need_clear int

)

功能说明:

修改 HINT 规则的属性。

参数说明:

name: 要修改的规则名称，必须指定为非 NULL 值。

attribute_name: 要修改的属性名，必须指定为非 NULL 值。支持的可修改的属性名和属性值包括: 属性名 NAME，属性值为修改后的规则名; 属性名 DESCRIPTION，属性值为修改后的规则描述; 属性名 STATUS, 属性值为 ENABLED/DISABLED; 属性名 CATEGORY，属性值为修改后的 HINT 的生效类别（由于 SF_INJECT_HINT 方法仅能注入无类别限制的 HINT，若需要注入带生效类别的 HINT，则还需要再调用一次 SF_ALTER_HINT 更改其生效

类别)。

attribute_value：设置的属性值，必须指定为非 NULL 值。

need_clear：是否同步清空所有缓存的计划。当该参数缺省时，表示使用定义 1 的语法，此时默认不清空缓存的计划。指定该参数时：

- 该参数类型为 BOOLEAN 时，使用定义 2 的语法，值为 TRUE 时，清空缓存的计划；值为 FALSE 时，模糊匹配时，不清空缓存的计划，需要手动清除对应 SQL 的计划后，指定的 HINT 才能生效，精确匹配时，默认会删除受影响 SQL 的计划缓存。需要特别注意的是，若 need_clear 为 FALSE，调用 SF_ALTER_HINT 仅修改 HINT 规则的名字，并不会删除相应 SQL 的计划缓存。
- 该参数类型为 INT 时，使用定义 3 的语法，值为 0 时，不清空缓存的计划；值为 1 时，清空缓存的计划；值为 2 时，只清空相关 SQL 的计划缓存，具体规则为：(1) 对于精确匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况均清空对应 SQL 的计划缓存；(2) 对于模糊匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况，对于计划缓存池中的所有 SQL，只要其包含了 HINT 指定的子串，就删除其计划缓存；(3) 对于通过 SQL_TEXT_ID 进行匹配的情况，不删相关 SQL 的计划缓存，与 need_clear 为 0 时表现一样。

返回值：

执行成功返回 0，执行失败返回错误码。

举例说明：

让指定的 HINT 规则失效：

```
SF_ALTER_HINT('TEST_INJECT', 'STATUS', 'DISABLED');
```

修改规则名，且不清空缓存的计划：

```
SF_ALTER_HINT('TEST_INJECT', 'NAME', 'TEST_INJECT1', FALSE);
```

3) SF_DEINJECT_HINT

定义:

定义 1:

```
INT
SF_DEINJECT_HINT (
name            varchar(128)
)
```

或者

定义 2:

```
INT
SF_DEINJECT_HINT (
name            varchar(128),
need_clear      boolean
)
```

或者

定义 3:

```
INT
SF_DEINJECT_HINT (
name            varchar(128),
need_clear      int
)
```

功能说明:

删除 SQL 中已注入的 HINT 规则。

参数说明:

name: 要删除的规则名称，必须指定为非 NULL 值。

need_clear: 是否同步清空所有缓存的计划。当该参数缺省时，表示使用定义 1 的语法，此时默认不清空缓存计划。指定该参数时:

- 该参数类型为 BOOLEAN 时，使用定义 2 的语法，值为 TRUE 时，清空缓存的计划；值为 FALSE 时，模糊匹配时，不清空缓存的计划，需要手动清除对应 SQL 的计划后，指定的 HINT 才能生效，精确匹配时，默认会删除受影响 SQL 的计划缓存。
- 该参数类型为 INT 时，使用定义 3 的语法，值为 0 时，不清空缓存的计划；值为 1 时，清空缓存的计划；值为 2 时，只清空相关 SQL 的计划缓存，具体规则为:

(1) 对于精确匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况均清空对应 SQL 的计划缓存；(2) 对于模糊匹配，除了调用 SF_ALTER_HINT 函数仅修改 HINT 规则描述的情况不清空计划缓存，其余情况，对于计划缓存池中的所有 SQL，只要其包含了 HINT 指定的子串，就删除其计划缓存；(3) 对于通过 SQL_TEXT_ID 进行匹配的情况，不删相关 SQL 的计划缓存，与 need_clear 为 0 时表现一样。

返回值:

执行成功返回 0，执行失败返回错误码。

举例说明:

为 SQL 删除已注入的 HINT 规则:

```
SF_DEINJECT_HINT('TEST_INJECT');
```

为 SQL 删除已注入的 HINT 规则，且不清空缓存的计划:

```
SF_DEINJECT_HINT('TEST_INJECT', FALSE);
```

17. 时区设置

本小节的过程与函数都是用来设置时区相关信息。

1) SP_SET_TIME_ZONE

定义:

SP_SET_TIME_ZONE(interval day)

SP_SET_TIME_ZONE(interval day to hour)

SP_SET_TIME_ZONE(interval day to minute)

SP_SET_TIME_ZONE(interval day to second)

SP_SET_TIME_ZONE(interval hour)

SP_SET_TIME_ZONE(interval hour to minute)

SP_SET_TIME_ZONE(interval hour to second)

功能说明:

设置时区为标准时区加上 interval day 中的小时和分钟。

返回值:

无

举例说明:

```
SP_SET_TIME_ZONE(INTERVAL '12 3:9' hour to second);

drop table ttime;

create table ttime (c2 timestamp with time zone);

insert into ttime values('1977-01-10 8:0:0');

select * from ttime;
```

查询结果如下:

```
1977-01-10 08:00:00.000000 +12:03
```

2） SP_SET_TIME_ZONE_STRING

定义:

SP_SET_TIME_ZONE_STRING(

varchar

)

功能说明:

设置时区为标准时区加上 varchar 的 interval hour。varchar 只能是 interval hour 或 hour to minute。

返回值:

无。

举例说明:

```
SP_SET_TIME_ZONE_string('7  3');
drop table ttime;
create table ttime (c2 timestamp with local time zone);
insert into ttime values('1977-01-10 8:0:0 +4:00');
select * from ttime;
```

查询结果如下:

```
1977-01-10 11:03:00.000000
```

3） SP_SET_TIME_ZONE_LOCAL

定义:

SP_SET_TIME_ZONE_LOCAL()

功能说明:

设置时区为当前服务器的时区。

返回值:

无。

举例说明:

```
SP_SET_TIME_ZONE_LOCAL();

drop table ttime;

create table ttime (c2 timestamp with local time zone);

insert into ttime values('1977-01-10 8:0:0 +3:00');

select * from ttime;
```

查询结果如下:

```
1977-01-10 13:00:00.000000
```

4) SF_TIME_ADD_TIME_ZONE_INTERVAL/

SF_DATETIME_ADD_TIME_ZONE_INTERVAL

定义:

DATETIME SF_TIME_ADD_TIME_ZONE_INTERVAL(time)

或

DATETIME SF_DATETIME_ADD_TIME_ZONE_INTERVAL(datetime)

功能说明:

将指定时间加上 LOCAL 时区时间。调用 SF_TIME_ADD_TIME_ZONE_INTERVAL 时，日期将被强制设置为 1900-01-01。

返回值:

返回将指定时间加上 LOCAL 时区时间的值。

举例说明:

```
select SF_TIME_ADD_TIME_ZONE_INTERVAL('2:00:36');

或

select SF_DATETIME_ADD_TIME_ZONE_INTERVAL('1988-01-01 2:00:36');
```

查询结果如下:

```
1900-01-01 10:00:36 +00:00

或

1988-01-01 10:00:36 +00:00
```

18. IP

本小节的过程与函数都是用来对 IP 地址进行相关处理。

1) SF_INET_EQUAL

定义:

```
INT
SF_INET_EQUAL(
ip1        varchar(256),
ip2        varchar(256)
)
```

功能说明:

比较两个 IP 地址是否相等。

参数说明:

ip1: 进行比较的第一个 IP 地址。

ip2: 进行比较的第二个 IP 地址。

返回值:

是否相等，1 表示相等，0 表示不相等。

举例说明:

```
SELECT SF_INET_EQUAL('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下:

```
0
```

2) SF_INET_NEQUAL

定义:

```
INT
SF_INET_NEQUAL(
ip1        varchar(256),
ip2        varchar(256)
)
```

功能说明:

比较两个 IP 地址是否不等。

参数说明:

ip1: 进行比较的第一个 IP 地址。

ip2: 进行比较的第二个 IP 地址。

返回值:

是否不等，1 表示不等，0 表示相等。

举例说明:

```
SELECT SF_INET_NEQUAL('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下:

```
1
```

3) SF_INET_LESS

定义:

```
INT
SF_INET_LESS(
ip1        varchar(256),
ip2        varchar(256)
)
```

功能说明:

比较第一个 IP 地址是否小于第二个 IP 地址。

参数说明:

ip1: 进行比较的第一个 IP 地址。

ip2: 进行比较的第二个 IP 地址。

返回值:

比较结果。

举例说明:

```
SELECT SF_INET_LESS('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下:

```
1
```

4) SF_INET_LESS_EQUAL

定义:

```
INT
SF_INET_LESS_EQUAL(
ip1        varchar(256),
```

ip2 varchar(256)

)

功能说明:

比较第一个 IP 地址是否小于等于第二个 IP 地址。

参数说明:

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值:

比较结果。

举例说明:

```
SELECT SF_INET_LESS_EQUAL('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下:

```
1
```

5) SF_INET_GREAT

定义:

INT

SF_INET_GREAT(

ip1 varchar(256),

ip2 varchar(256)

)

功能说明:

比较第一个 IP 地址是否大于第二个 IP 地址。

参数说明:

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值：

比较结果。

举例说明：

```
SELECT SF_INET_GREAT('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下：

```
0
```

6） SF_INET_GREAT_EQUAL

定义：

```
INT
SF_INET_GREAT_EQUAL(
ip1        varchar(256),
ip2        varchar(256)
)
```

功能说明：

比较第一个 IP 地址是否大于等于第二个 IP 地址。

参数说明：

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值：

比较结果。

举例说明：

```
SELECT SF_INET_GREAT_EQUAL('192.168.1.1', '192.168.2.1') FROM DUAL;
```

查询结果如下：

```
0
```

7） SF_INET_CONTAIN

定义：

INT

SF_INET_CONTAIN(

ip1 varchar(256),

ip2 varchar(256)

)

功能说明：

比较第一个 IP 地址是否包含第二个 IP 地址。

参数说明：

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值：

比较结果，1 表示包含，0 表示不包含。

举例说明：

```
SELECT SF_INET_CONTAIN('145.13.255.1/16', '145.13.176.1/16') FROM DUAL;
```

查询结果如下：

```
0
```

8） SF_INET_CONTAIN_EQUAL

定义:

```
INT
SF_INET_CONTAIN_EQUAL(
ip1     varchar(256),
ip2     varchar(256)
)
```

功能说明:

比较第一个 IP 地址是否包含或等于第二个 IP 地址。

参数说明:

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值:

比较结果，1 表示包含或等于，0 表示不包含且不等于。

举例说明:

```
SELECT SF_INET_CONTAIN_EQUAL('145.13.255.1/16', '145.13.176.1/16') FROM DUAL;
```

查询结果如下:

```
1
```

9) SF_INET_BECONTAINED

定义:

```
INT
SF_INET_BECONTAINED(
ip1     varchar(256),
ip2     varchar(256)
```

)

功能说明:

比较第一个 IP 地址是否被第二个 IP 地址包含。

参数说明:

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值:

比较结果，1 表示包含，0 表示不包含。

举例说明:

```
SELECT SF_INET_BECONTAINED('145.13.255.1/16', '145.13.176.1/16') FROM DUAL;
```

查询结果如下:

```
0
```

10）SF_INET_BECONTAINED_EQUAL

定义:

INT

SF_INET_BECONTAINED_EQUAL(

ip1 varchar(256),

ip2 varchar(256)

)

功能说明:

比较第一个 IP 地址是否被包含或等于第二个 IP 地址。

参数说明:

ip1：进行比较的第一个 IP 地址。

ip2：进行比较的第二个 IP 地址。

返回值：

比较结果，1 表示包含或等于，0 表示不包含且不等于。

举例说明：

```
SELECT SF_INET_BECONTAINED_EQUAL('145.13.255.1/16', '145.13.176.1/16') FROM DUAL;
```

查询结果如下：

```
1
```

11） SF_INET_SORT

定义：

VARCHAR

SF_INET_SORT(

ip varchar2,

)

功能说明：

将 IP 转换成一个可用来比较的字符串值。

参数说明：

ip：待转换的 IP 串，IP 串中的子网掩码为可选项。

返回值：

转换后的字符串。

举例说明：

将 IP 地址 192.168.100.2/12 转换成可比较的字符串值，其中/12 为子网掩码。

```
SELECT SF_INET_SORT ('192.168.100.2/12') FROM DUAL;
```

查询结果如下：

```
A192160000000012192168100002
```

19. ROWID

1) ROWIDTOCHAR

定义:

VARCHAR(18)

ROWIDTOCHAR (

rowid rowid

)

功能说明:

将 ROWID 类型的 ROWID 值转换成 BASE64 编码的定长为 18 字节的字符串。

参数说明:

rowid: ROWID 类型的数据。

返回值:

以 BASE64 编码的定长为 18 字节的字符串。

举例说明:

查询 TEST 表中的 ROWIDTOCHAR(ROWID)和 ROWID:

```
CREATE TABLE TEST(C1 INT);

INSERT INTO TEST VALUES(10);

INSERT INTO TEST VALUES(11);

SELECT ROWIDTOCHAR(ROWID),ROWID FROM TEST;
```

查询结果如下:

```
ROWIDTOCHAR(ROWID) ROWID

------------------ ------------------
```

```
AAAAAAAAAAAAAAAAB AAAAAAAAAAAAAAAAB

AAAAAAAAAAAAAAAAC AAAAAAAAAAAAAAAAC
```

2) CHARTOROWID

定义:

ROWID

CHARTOROWID (

c1 varchar(18)

)

功能说明:

将 BASE64 编码的定长 18 字节的字符串 STR 转换成 ROWID 数据类型。

参数说明:

c1: BASE64 编码的定长 18 字节的字符串，可为 NULL 或空串。

返回值:

ROWID 类型的值，参数为 NULL 或空串时返回 NULL。

举例说明:

将 BASE64 编码的定长 18 字节的字符串转换成 ROWID 类型的值:

```
CREATE TABLE T1(C1 INT);

CREATE TABLE T2(D1 ROWID,D2 VARCHAR(18));

INSERT INTO T1 SELECT LEVEL FROM DUAL CONNECT BY LEVEL< =5;

INSERT INTO T2(D1,D2) SELECT ROWID,ROWIDTOCHAR(ROWID) FROM T1;

SELECT D1,D2,CHARTOROWID(D2) FROM T2;
```

查询结果如下:

```
D1                  D2                  CHARTOROWID(D2)
```

```
----------------- ----------------- -----------------

AAAAAAAAAAAAAAAAB AAAAAAAAAAAAAAAAB AAAAAAAAAAAAAAAAB

AAAAAAAAAAAAAAAAC AAAAAAAAAAAAAAAAC AAAAAAAAAAAAAAAAC

AAAAAAAAAAAAAAAAD AAAAAAAAAAAAAAAAD AAAAAAAAAAAAAAAAD

AAAAAAAAAAAAAAAAE AAAAAAAAAAAAAAAAE AAAAAAAAAAAAAAAAE

AAAAAAAAAAAAAAAAF AAAAAAAAAAAAAAAAF AAAAAAAAAAAAAAAAF
```

3） SF_BUILD_ROWID

定义:

ROWID

SF_BUILD_ROWID(

epno int,

partno bigint,

real_rowid bigint

)

功能说明:

根据站点号、分区号和数据在表中的物理行号构造一个 ROWID 类型的数据。

参数说明:

epno：站点号。取值范围 0~65535。

partno：分区号。取值范围 0~65534。

real_rowid：数据在表中的物理行号。取值范围 1~281474976710655。

返回值:

一个 ROWID 类型的数据。

举例说明:

假定站点号为 1，分区号为 2，数据的物理行号为 50。使用 SF_BUILD_ROWID 函数构造出一个 ROWID 类型数据。

```
SELECT SF_BUILD_ROWID(1,2,50);
```

查询结果如下：

```
AAABAAAAACAAAAAAAy
```

其中，AAAB 为站点号、AAAAAC 为分区号、AAAAAAAy 为 ROWID 值。

4） SF_GET_EP_SEQNO

定义：

INT

SF_GET_EP_SEQNO(

rowid rowid

)

功能说明：

在 DSC 环境下返回配置的站点号；MPP 或者 DPC 环境下返回 ROWID 上的站点号；否则返回 0。

参数说明：

rowid：ROWID 类型的数据。

返回值：

EP 站点号。

举例说明：

```
SELECT  SF_GET_EP_SEQNO('AAABAAAAACAAAAAAAy');
```

查询结果如下：

```
0
```

5） SF_ROWID_GET_EP_SEQNO

定义:

INT

SF_ROWID_GET_EP_SEQNO(

rowid rowid

)

功能说明:

根据 ROWID 数据类型获取本条数据上的 EP 站点。本函数不适用于堆表的 ROWID。

参数说明:

rowid：ROWID 类型的数据。

返回值:

EP 站点号。

举例说明:

```
SELECT  SF_ROWID_GET_EP_SEQNO('AAABAAAAACAAAAAAAy');
```

查询结果如下:

```
1
```

6） SF_ROWID_GET_PARTNO

定义:

INT

SF_ROWID_GET_PARTNO(

rowid rowid

)

功能说明:

根据 ROWID 数据类型获取本条数据的分区号。本函数不适用于堆表的 ROWID。

参数说明:

rowid：ROWID 类型的数据。

返回值:

分区号。

举例说明:

```
SELECT  SF_ROWID_GET_PARTNO('AAABAAAAACAAAAAAAy');
```

查询结果如下:

```
2
```

7) SF_GET_REAL_ROWID

定义:

BIGINT

SF_GET_REAL_ROWID(

rowid rowid

)

功能说明:

根据 ROWID 数据类型获取本条数据的物理行号。本函数不适用于堆表的 ROWID。

参数说明:

rowid：ROWID 类型的数据。

返回值:

物理行号。

举例说明:

```
SELECT  SF_GET_REAL_ROWID('AAABAAAAACAAAAAAAy');
```

查询结果如下：

```
50
```

8） SF_GET_ROWID_BY_PAGE

定义：

CBLOB

SF_GET_ROWID_BY_PAGE（

ts_id int,

file_id int,

page_no int

）

功能说明：

获取目标页页内记录的所有 ROWID，仅限于包含数据的叶子节点页。

参数说明：

ts_id：指定目标页的表空间，如果表空间不存在则报错。

file_id：指定目标页的文件，如果文件不存在则报错。

page_no：指定目标页的编号，如果编号超出文件范围则报错。

返回值：

rowid：数据在表中的一系列物理行号，用空格隔开，范围 1~281474976710655。

举例说明：

本例中的参数值需根据实际情况进行指定。

```
SELECT  SF_GET_ROWID_BY_PAGE(5, 0, 52);
```

查询结果如下：

```
5762 5763 5764 5765 5766 5767 5768 5769 5770 5771 5772 5773 5774 5775 5776 5777 ……
6035 6036 6037 6038 6039 6040 6041 6042 6043 6044 6045 6046 6047 6048
```

20. 系统包

本小节的过程与函数都与系统包相关。

1) SP_INIT_DBG_SYS*

定义:

```
SP_INIT_DBG_SYS(
create_flag      int
)
```

功能说明:

创建或删除 DBMS_DBG 系统包。DBMS_DBG 属于系统内部系统包，并未在《DM8 系统包使用手册》中进行介绍，若用户需要使用 dmdbg 调试工具，则必须首先创建 DBMS_DBG 系统包。关于 dmdbg 调试工具的介绍可以参考《DM8_SQL 程序设计》。

参数说明:

create_flag：为 1 时表示创建 DBMS_DBG 包；为 0 表示删除该系统包。

返回值:

无

举例说明:

创建 DBMS_DBG 系统包:

```
SP_INIT_DBG_SYS(1);
```

2) SF_CHECK_DBG_SYS

定义:

INT

SF_CHECK_DBG_SYS ()

功能说明:

DBMS_DBG 系统包启用状态检测。

返回值:

0：未启用；1：已启用。

举例说明:

获得 DBMS_DBG 系统包的启用状态:

```
SELECT SF_CHECK_DBG_SYS;
```

3) SP_CREATE_SYSTEM_PACKAGES*

定义:

SP_CREATE_SYSTEM_PACKAGES (

create_flag int

)

功能说明:

创建或删除所有系统包，其中不包含 DBMS_JOB、DBMS_WORKLOAD_REPOSITORY 和 DBMS_SCHEDULER 包。若在创建过程中某个系统包由于特定原因未能创建成功，会跳过继续创建后续的系统包。关于系统包的详细介绍可参考《DM8 系统包使用手册》。

参数说明:

create_flag：为 1 时表示创建所有系统包；为 0 表示删除这些系统包。

返回值:

无

举例说明:

创建所有系统包：

```
SP_CREATE_SYSTEM_PACKAGES (1);
```

4) SP_CREATE_SYSTEM_PACKAGES*

定义：

SP_CREATE_SYSTEM_PACKAGES (

create_flag int,

pkgname varchar(128)

)

功能说明：

创建或删除指定的系统包。不支持 DBMS_JOB、DBMS_WORKLOAD_REPOSITORY 和 DBMS_SCHEDULER 系统包。关于系统包的详细介绍可参考《DM8 系统包使用手册》。

参数说明：

create_flag：为 1 时表示创建指定的系统包；为 0 表示删除这个系统包。

pkgname：指定要创建的包名。

返回值：

无

举例说明：

创建 DBMS_LOB 系统包：

```
SP_CREATE_SYSTEM_PACKAGES(1, 'DBMS_LOB');
```

5) SF_CHECK_SYSTEM_PACKAGES

定义：

INT

SF_CHECK_SYSTEM_PACKAGES（）

功能说明:

系统包的启用状态检测。

返回值:

0：未启用；1：已启用。

举例说明:

获得系统包的启用状态:

```
SELECT SF_CHECK_SYSTEM_PACKAGES;
```

6） SF_CHECK_SYSTEM_PACKAGE

定义:

INT

SF_CHECK_SYSTEM_PACKAGE（

package_name varchar(128)

）

功能说明:

检测指定系统包的启用状态。

参数说明:

package_name: 指定的系统包名。

返回值:

0：未启用；1：已启用。

举例说明:

获得系统包 DBMS_SCHEDULER 的启用状态:

```
SELECT SF_CHECK_SYSTEM_PACKAGE('DBMS_SCHEDULER');
```

7) SP_INIT_JOB_SYS*

定义:

SP_INIT_JOB_SYS(

create_flag int

)

功能说明:

创建或删除 DBMS_JOB 系统包。

参数说明:

create_flag: 为 0 时表示删除 DBMS_JOB 系统包；为 1 时表示创建 DBMS_JOB 系统包；为 2 时表示重建 DBMS_JOB 包中的相关方法，一般用于版本升级后，包中方法发生了改变的情况下使用。

返回值:

无

举例说明:

创建 DBMS_JOB 系统包:

```
SP_INIT_JOB_SYS(1);
```

8) SP_INIT_AWR_SYS*

定义:

SP_INIT_AWR_SYS (

create_flag int

)

功能说明:

创建或删除 DBMS_WORKLOAD_REPOSITORY 系统包。

参数说明:

create_flag：为 1 时表示创建 DBMS_WORKLOAD_REPOSITORY 系统包；为 0 表示删除该系统包。

返回值:

无

举例说明:

创建 DBMS_WORKLOAD_REPOSITORY 系统包:

```
SP_INIT_AWR_SYS(1);
```

9） SF_CHECK_AWR_SYS

定义:

INT

SF_CHECK_AWR_SYS()

功能说明:

系统的 DBMS_WORKLOAD_REPOSITORY 系统包启用状态检测。

返回值:

0：未启用；1：已启用。

举例说明:

获得 DBMS_WORKLOAD_REPOSITORY 系统包的启用状态:

```
SELECT SF_CHECK_AWR_SYS;
```

10）SP_INIT_DBMS_SCHEDULER_SYS*

定义:

SP_INIT_DBMS_SCHEDULER_SYS(

create_flag int

)

功能说明:

创建或删除 DBMS_SCHEDULER 系统包。

参数说明:

create_flag：为 1 时表示创建 DBMS_SCHEDULER 系统包；为 0 表示删除该系统包。

返回值:

无

举例说明:

创建 DBMS_SCHEDULER 系统包:

```
SP_INIT_DBMS_SCHEDULER_SYS(1);
```

11) SP_INIT_RLS_SYS*

定义:

SP_INIT_RLS_SYS(

create_flag int

)

功能说明:

创建或删除 DBMS_RLS 系统包。

为兼容 V8.4.1.113 之前的版本，依旧支持通过 SP_CREATE_SYSTEM_PACKAGES(create_flag int, pkgname varchar(128))函数创建或删除 DBMS_RLS 系统包，但建议使用本函数进行创建或删除 DBMS_RLS 系统包。

参数说明:

create_flag：为 1 时表示创建 DBMS_RLS 系统包；为 0 表示删除该系统包。

返回值：

无

举例说明：

创建 DBMS_RLS 系统包：

```
SP_INIT_RLS_SYS(1);
```

21. 用户自定义分词词库

本小节的过程均与全文索引相关。

1） SP_WORD_LIB_CREATE

定义：

SP_WORD_LIB_CREATE(

lib_dir varchar(256),

lib_name varchar(256)

)

功能说明：

创建用户自定义的分词词库。

参数说明：

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

返回值：

无

2） SP_WORD_LIB_ADD_WORD

定义:

SP_WORD_LIB_ADD_WORD(

lib_dir varchar(256),

lib_name varchar(256),

word varchar(48)

)

功能说明:

向指定的自定义分词词库添加新的分词。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

word：表示新增的分词词语，最长不超过 48 字节。

返回值:

无

3) SP_WORD_LIB_DELETE_WORD

定义:

SF_WORD_LIB_DELETE_WORD(

lib_dir varchar(256),

lib_name varchar(256),

word varchar(48)

)

功能说明:

向指定的自定义分词词库删除一个分词。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

word：表示待删除的分词词语。

返回值:

无

4) SP_WORD_LIB_MODIFY_WORD

定义:

```
SP_WORD_LIB_MODIFY_WORD(
lib_dir varchar(256),
lib_name varchar(256),
old_word varchar(48),
new_word varchar(48)
)
```

功能说明:

将用户自定义的分词词库中的旧分词用新分词替换。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

old_word：旧分词。

new_word：新分词。

返回值:

无

5) SP_WORD_LIB_DELETE

定义:

SP_WORD_LIB_DELETE(

lib_dir varchar(256),

lib_name varchar(256)

)

功能说明:

删除用户自定义的分词词库。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

返回值:

无

6) SP_WORD_LIB_CLEAR

定义:

SP_WORD_LIB_CLEAR(

lib_dir varchar(256),

lib_name varchar(256)

)

功能说明:

清空用户自定义的分词词库中的全部分词词语。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

返回值：

无

7） SP_WORD_LIB_BATCH_ADD_WORDS

定义：

```
SP_WORD_LIB_BATCH_ADD_WORDS(
lib_dir varchar(256),
lib_name varchar(256),
words_path varchar(256)
)
```

功能说明：

向用户自定义的分词词库批量添加分词词语。

参数说明：

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

word_path：为用户自定义词语所在文件路径，文件内词语按换行分隔。

返回值：

无

8） SP_WORD_LIB_BATCH_DELETE_WORDS

定义：

```
SP_WORD_LIB_BATCH_DELETE_WORDS(
lib_dir varchar(256),
lib_name varchar(256),
```

words_path varchar(256)

)

功能说明:

向用户自定义的分词词库批量删除分词词语。

参数说明:

lib_dir：表示目录对象名。

lib_name：表示词库文件名。

word_path：为用户自定义词语所在文件路径，文件内词语按换行分隔。

返回值:

无

附录 4 DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖高新技术开发区高新大道 999 号未来科技大厦 C3 栋 16-19 层

邮编：430073

电话：(+86)027-87588000

传真：(+86)027-87588000-8039

达梦数据库（北京）有限公司

地址：北京市海淀区北三环西路 48 号数码大厦 A 座 905

邮编：100086

电话：(+86)010-51727900

传真：(+86)010-51727983

达梦数据库（上海）有限公司

地址：上海市闸北区江场三路 28 号 301 室

邮编：200436

电话：(+86)021-33932716

传真：(+86)021-33932718

地址：上海市浦东张江高科技园区博霞路 50 号 201 室

邮编：201203

电话：（+86）021-33932717

传真：（+86）021-33932717-801

达梦数据库（广州）有限公司

地址：广州市荔湾区中山七路 330 号荔湾留学生科技园 703 房

邮编：510145

电话：(+86)020-38371832

传真：(+86)020-38371832

达梦数据库（海南）有限公司

地址：海南省海口市玉沙路富豪花园 B 座 1602 室

邮编：570125

电话：(+86)0898-68533029

传真：(+86)0898-68531910

达梦数据库（南宁）办事处

地址：广西省南宁市科园东五路四号南宁软件园五楼

邮编：530003

电话：（+86）0771-2184078

传真：（+86）0771-2184080

达梦数据库（合肥）办事处

地址：合肥市包河区马鞍山路金帝国际城 7 栋 3 单元 706 室

邮编：230022

电话：（+86）0551-3711086

达梦数据库（深圳）办事处

地址：深圳市福田区皇岗路高科利大厦 A 栋 24E

邮编：518033

电话：0755-83658909

传真：0755-83658909

技术服务：

电话：400-991-6599

邮箱：dmtech@dameng.com

手册版本

版本号：V8

发版日期：2025 年 7 月

达梦数据

达梦就在您身边

武汉达梦数据库股份有限公司

总部：武汉市东湖高新技术开发区高新大道999号

未来科技大厦C3栋16—19层

电话：(+86) 027-87588000